

The Zcash Arboretum

The complete series in one volume

m@rek.onl

Contents

I	Math Guide: Mathematical foundations for Halo 2 and Zcash Orchard	7
1	Notation: sets, functions, and relations	7
2	The integers and modular arithmetic	8
3	Groups	18
4	Rings and fields	27
5	Polynomials over a field	33
6	Finite fields	45
7	Number theory for cryptography	50
8	Linear algebra over a field	57
9	Roots of unity and evaluation domains	64
10	Elliptic curves	75
11	Probability, asymptotics, and computation	86
II	Crypto Guide: Cryptographic primitives from scratch	100
1	The provable-security model	100
2	Hardness assumptions	111
3	Hash functions and the random oracle model	122
4	Pseudorandom functions, block ciphers, and format-preserving encryption	132
5	Symmetric encryption, AEAD, and key derivation	140
6	Key agreement	149
7	Commitment schemes	158
8	Digital signatures	171
9	Interactive proofs, zero knowledge, and SNARKs	183
10	Merkle trees and commitments to sets	196
III	Halo 2 Guide: The Halo 2 proof system, from arithmetisation to recursion	207
1	Introduction	207
2	Polynomial IOPs and the SNARK design pattern	207

3	A rigorous treatment of knowledge soundness	209
4	Recursive proof composition and accumulation schemes	211
5	The Halo 2 proof system	212
IV	How Halo 2 Proves: One computation, carried from claim to conviction	227
1	What we are going to prove, and what “prove” means	227
2	From computation to a grid of constraints	228
3	From the grid to polynomials	229
4	From vanishing to a single identity	230
5	Why checking one random point is a proof	231
6	Why the prover cannot wriggle: binding commitments	232
7	Enforcing the wiring: the permutation argument	233
8	Proving a value sits in a table: lookups	233
9	Binding the public statement, and hiding the witness	234
10	Removing the verifier: the Fiat–Shamir transform	235
11	The “Halo” in Halo 2: deferring the expensive check	235
12	Why the verifier is convinced	236
V	Consensus Guide: Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule	238
1	Scope, sources, and the two double-spends	238
2	The block tree and the most-work rule	239
3	A probability toolkit	241
4	Playing catch-up	243
5	Waiting for confirmations	244
6	What the numbers say	247
7	The economics of the attack	248
8	The deployed rule and its safety rails	250
9	Relation to the rest of the series	253
VI	Ironwood Guide: The Zcash Orchard protocol and its Ironwood pool	254

1	Introduction	254
2	The Orchard application: the relation proved by the SNARK	254
3	End-to-end security: derivation of Orchard's properties from the SNARK	274
4	The Zcash proof-system landscape	278
5	The Ironwood pool	279
6	Post-quantum security of the Orchard protocol	285
VII	Wallet Guide: The Zcash wallet layer: keys, addresses, fees, and transaction construction	291
1	Hierarchical key derivation (ZIP-32)	291
2	Diversified and unified addresses (ZIP-316)	299
3	Fees (ZIP-317)	309
4	Transaction construction	317
5	Partially created transactions (PCZT)	326
6	Broadcast, expiry, and the transaction lifecycle	340
7	Payment requests (ZIP-321)	349
VIII	Sync Guide: Light-client synchronisation: compact blocks, the service, and the scanning pipeline	358
1	Compact blocks (ZIP-307)	358
2	The light-client service	366
3	The scanning pipeline	378
4	Commitment-tree synchronisation	387
5	What the server learns	394
IX	FlyClient Guide: Succinct chain verification: the sampling protocol, the deployed ZIP-221 commitment, and the unbuilt bridge	401
1	Scope, sources, and the deployment boundary	401
2	The problem: verifying a chain without downloading it	402
3	Merkle mountain ranges	404
4	The FlyClient protocol	407
5	ZIP-221: the deployed chain-history tree	411

6	NU5: the commitment becomes one of two	415
7	Deployed implementations	416
8	The deployment gap	418
X	Voting Guide: The May–June 2026 Zally shielded-voting deployment snapshot	422
1	The protocol and its provenance	422
2	Ballots and eligibility	425
3	Vote encryption, nullifiers, and tally	430
4	Trust model and verdict	436
XI	ZSA Guide: Zcash Shielded Assets: issuance, transfer, and the counterfeiting boundary	441
1	Scope, status, and sources	441
2	Asset identity	445
3	Issuance and finalization (ZIP-227)	451
4	Transfer and burn (ZIP-226)	460
5	Split notes and the counterfeiting boundary	466
6	Transaction integration and outlook	473
XII	Tachyon Guide: The Tachyon scaling architecture, at design stage: what is specified, what is designed, what is open	482
1	Scope, sources, and method	482
2	The proof-carrying wallet	485
3	The design as published	489
4	What is specified today	498
5	Designed but unspecified, and open problems	501
6	Security posture and outlook	508
XIII	Crosslink Guide: Trailing finality for Zcash: the construction, its formal models, and the wallet-visible contract	512
1	Scope, status, and sources	512

2	The ebb-and-flow model	517
3	The Crosslink construction	528
4	Stake, delegation, and slashing	539
5	The wallet-visible contract	548
6	Security posture and open problems	556
XIV	FROST Guide: Threshold RedPallas: FROST, its re-randomised form, and where Zcash uses it	565
1	Scope, sources, and deployment surfaces	565
2	FROST	568
3	Re-Randomized FROST for RedPallas	575
4	Deployment and open questions	582
XV	PQ Guide: Post-quantum Zcash: the cross-layer assumption inventory, the reversibility divide, and the migration surface	588
1	Scope, method, and the two clocks	588
2	The inventory: every deployed surface, sorted	589
3	Quantum mining	592
4	The frontier surfaces	593
5	ZIP-2005 in the cross-layer frame	594
6	The migration surface	594
7	Relation to the rest of the series	596

Part I

Math Guide: Mathematical foundations for Halo 2 and Zcash Orchard

Abstract

This is the foundational volume of *The Zcash Arboretum*, developing the cryptography of Zcash’s shielded protocols from the ground up. Assuming only elementary algebra and calculus, it builds the mathematics the later volumes rely on: sets and notation; the integers and modular arithmetic; groups; rings and fields; polynomials; finite fields and the number theory of the discrete logarithm; linear algebra and inner products; roots of unity and evaluation domains; elliptic curves and their group law; and the probability, asymptotics, and model of computation needed to state cryptographic claims precisely.

1 Notation: sets, functions, and relations

This short section fixes the small amount of set-theoretic language we use throughout the series. A reader comfortable with elementary mathematics may skim it and refer back as needed.

1.1 Sets

A *set* is a collection of distinct objects, its *elements*; one writes $x \in A$ for membership and $x \notin A$ for its negation. A set may be given by listing, $\{a, b, c\}$, or by *set-builder* notation $\{x \in A : P(x)\}$ (“the x in A such that $P(x)$ ”). One writes $A \subseteq B$ when every element of A lies in B , and $A = B$ when $A \subseteq B$ and $B \subseteq A$. The usual operations are union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, and the empty set $\emptyset$. The *Cartesian product* is $A \times B = \{(a, b) : a \in A, b \in B\}$, and A^n is the set of n -tuples over A . For a finite set A , $|A|$ denotes its number of elements. The standard number systems are the naturals $\mathbb{N}$, the integers $\mathbb{Z}$, the rationals $\mathbb{Q}$, the reals $\mathbb{R}$, and the complex numbers $\mathbb{C}$, with $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$; we construct the prime field $\mathbb{F}_p$ in §2 and treat general finite fields $\mathbb{F}_q$ in §6.

1.2 Functions

A *function* $f : A \rightarrow B$ assigns to each $a \in A$ a unique value $f(a) \in B$; A is the *domain* and B the *codomain*. It is *injective* (one-to-one) if $f(a) = f(a')$ implies $a = a'$; *surjective* (onto) if every $b \in B$ equals $f(a)$ for some a ; and *bijective* if both. The *composition* of $f : A \rightarrow B$ and $g : B \rightarrow C$ is $(g \circ f)(a) = g(f(a))$. A function has a two-sided *inverse* $f^{-1} : B \rightarrow A$ (with $f^{-1} \circ f = \text{id}_A$ and $f \circ f^{-1} = \text{id}_B$) if and only if it is bijective. For $S \subseteq A$ the *image* is $f(S) = \{f(a) : a \in S\}$, and for $T \subseteq B$ the *preimage* is $f^{-1}(T) = \{a \in A : f(a) \in T\}$ (defined whether or not f is invertible).

1.3 Relations and equivalences

A *binary relation* on A is a subset $R \subseteq A \times A$; one writes $a \sim b$ for $(a, b) \in R$. It is an *equivalence relation* if it is *reflexive* ($a \sim a$), *symmetric* ($a \sim b \Rightarrow b \sim a$), and *transitive* ($a \sim b$ and $b \sim c$ imply $a \sim c$). The *equivalence class* of a is $[a] = \{x \in A : x \sim a\}$; the distinct classes *partition* A into disjoint nonempty pieces whose union is A , and the set of classes is the *quotient* $A/\sim$. We use this construction twice: to form the integers modulo n (§2) and the cosets of a subgroup (§3).

1.4 Logic and proof conventions

We employ the symbols $\forall$ (“for all”), $\exists$ (“there exists”), $\exists!$ (“there exists a unique”), and “:” or “s.t.” for “such that”. Negation swaps the quantifiers: $\neg(\forall x P(x))$ is $\exists x \neg P(x)$, and $\neg(\exists x P(x))$ is $\forall x \neg P(x)$. The

order of unlike quantifiers matters: $\forall x \exists y P(x, y)$ (the y may depend on x) is weaker than $\exists y \forall x P(x, y)$ (one y works for all). We prove an implication $P \Rightarrow Q$ directly (assume P , derive Q), by contraposition (assume $\neg Q$, derive $\neg P$), or by contradiction; we prove a statement $\forall x P(x)$ by taking an arbitrary x , and $\exists x P(x)$ by exhibiting a witness. We prove statements about all natural numbers by induction.

1.5 Symbols and how to say them

The series draws on three alphabets: blackboard-bold letters for the standard number systems and structures, Greek letters, and a handful of relational and operator symbols. This subsection collects the ones the series actually uses and gives a spoken form for each — useful when reading aloud, and because several Greek letters are routinely mispronounced. Pronunciations are given in the received-English form (so β is “BEE-ta”, not “BAY-ta”); a capitalised syllable carries the stress. Each symbol is defined where it is first used; the notes here are reminders, not definitions.

Symbol	Name	Said	Typical role in the series
$\mathbb{N}$	blackboard N	“natural numbers”	the naturals
$\mathbb{Z}$	blackboard Z	“the integers”	the integers (German <i>Zahlen</i>)
$\mathbb{Q}$	blackboard Q	“the rationals”	the rationals (<i>quotients</i>)
$\mathbb{R}$	blackboard R	“the reals”	the real numbers
$\mathbb{C}$	blackboard C	“the complexes”	the complex numbers
$\mathbb{F}_q$	blackboard F	“eff-cue”	the finite field of q elements
$\mathbb{G}$	blackboard G	“group G”	an abstract group
$\mathbb{E}$	blackboard E	“expectation”	expectation of a random variable
$\mathbb{P}$	blackboard P	“the Pallas curve”	the Pallas group (protocol-spec notation)

Table 1: Blackboard-bold letters. “Blackboard bold” names the double-struck style itself; in speech one usually says only the letter or the system it denotes. Elliptic curves are written italic E or calligraphic $\mathcal{E}$, never blackboard bold.

Sym.	Name	Said	Sym.	Name	Said
α	alpha	“AL-fa”	μ	mu	“mew”
β	beta	“BEE-ta”	ν	nu	“new”
γ	gamma	“GAM-a”	ξ	xi	“ksigh” / “zigh”
δ	delta	“DEL-ta”	π	pi	“pie”
ϵ, ε	epsilon	“EP-si-lon”	ρ	rho	“roe”
ζ	zeta	“ZEE-ta”	σ	sigma	“SIG-ma”
η	eta	“EE-ta”	τ	tau	“taw” (rhymes “paw”)
θ	theta	“THEE-ta”	ϕ, φ	phi	“fie” or “fee”
ι	iota	“eye-OH-ta”	χ	chi	“kigh” (hard k)
κ	kappa	“KAP-a”	ψ	psi	“sigh” or “psigh”
λ	lambda	“LAM-da”	ω	omega	“OH-mig-a”

Table 2: Lower-case Greek letters used in the series. The commonly muddled ones: ξ , χ , and ψ rhyme with “eye” (“ksigh”, “kigh”, “sigh” — never “ksee”), χ with a hard k , ψ with a silent or lightly sounded p , and ζ is “ZEE-ta”, not “ZAY-ta”. The letter τ is also heard rhyming with “cow”. Capitals reuse the same names: Δ (“delta”), Σ (“sigma”; the summation sign, and the name of the Σ -protocol family), Π (“pi”; the product sign, and a conventional name for a protocol or problem), Φ , Ψ , Ω , Θ , Λ .

2 The integers and modular arithmetic

The integers $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ are the arithmetical bedrock on which every algebraic and cryptographic structure in this monograph ultimately rests. The finite fields underlying elliptic curves,

Symbol	Said	Meaning
$\in, \notin$	“in”, “not in”	set membership
$\subseteq, \subset$	“subset of”	(proper) inclusion
$\cup, \cap$	“union”, “intersect”	set union and intersection
$A \setminus B$	“A minus B”	set difference
$\times$	“times” / “cross”	Cartesian product
$, \nmid$	“divides”, “does not divide”	divisibility
$\equiv$	“congruent to”	congruence, qualified (mod n)
$a \bmod n$	“a mod n”	remainder of division by n
$:=, =:$	“is defined as”	definitional equality
$\mapsto$	“maps to”	the action of a function
$g \circ f$	“g after f”	function composition
$\lfloor x \rfloor, \lceil x \rceil$	“floor”, “ceiling”	round down, round up
$\langle a, b \rangle$	“inner product”	inner product; $\langle g \rangle$ the subgroup generated by g
$\oplus$	“x-or”	bitwise exclusive or
$x \parallel y$	“x concat y”	concatenation of bit or byte strings
$x \leftarrow D, x \stackrel{\$}{\leftarrow} S$	“drawn from”	sampling ($\$$ marks uniform); also assignment
$[k]P$	“k times P”	scalar multiplication of a point
$P^\star$	“P star”	canonical bit-encoding of a point or element
$\cdot$	“dot”	product; argument placeholder, as in $F(\cdot)$
$ A $	“size of A” / “order of A”	cardinality

Table 3: Relational and operator symbols. The symbol $\oplus$ is read “x-or”; every use in the series is the bitwise exclusive or.

the scalar arithmetic of Halo 2, and the polynomial commitments of Orchard (short values that bind a prover to entire polynomials, which a verifier then tests by evaluation) all reduce, in the end, to computations with congruence classes of integers. This section develops the theory of the integers from a small set of assumptions and arrives at the ring $\mathbb{Z}/n\mathbb{Z}$ of integers modulo n , its group of units, and the algorithms that make arithmetic in it effective. We keep the treatment constructive throughout: every existence statement comes, wherever possible, with an algorithm that produces the object in question, since cryptography requires not merely that inverses and representatives exist but that one can compute them efficiently.

We take the basic algebraic properties of $\mathbb{Z}$ for granted: that it is a commutative ring with the usual addition and multiplication, that it has no zero divisors (if $ab = 0$ then $a = 0$ or $b = 0$), and that it carries a total order $\leq$ compatible with the arithmetic. We assume axiomatically the single deep property that follows.

Theorem 2.1 (Well-ordering principle). *Every nonempty subset $S \subseteq \mathbb{N} = \{0, 1, 2, \dots\}$ of the natural numbers has a least element: there exists $m \in S$ such that $m \leq s$ for all $s \in S$.*

The well-ordering principle is logically equivalent to the principle of mathematical induction; we use both freely. Nearly every existence result below is, at bottom, an application of well-ordering: one exhibits a nonempty set of natural numbers and extracts its minimum.

2.1 Divisibility

Definition 2.2. Let $a, b \in \mathbb{Z}$. One says that a divides b , written $a \mid b$, if there exists an integer $c \in \mathbb{Z}$ with $b = ac$. In this case a is called a *divisor* or *factor* of b , and b is a *multiple* of a . If no such c exists one writes $a \nmid b$.

Several immediate consequences of the definition deserve to be recorded; we shall use them constantly and usually without explicit citation.

Proposition 2.3. *Let $a, b, c, d \in \mathbb{Z}$. Then:*

1. $a \mid a$ (reflexivity), $1 \mid a$, and $a \mid 0$.
2. If $a \mid b$ and $b \mid c$ then $a \mid c$ (transitivity).
3. If $a \mid b$ and $a \mid c$ then $a \mid (bx + cy)$ for all $x, y \in \mathbb{Z}$; in particular $a \mid (b \pm c)$.
4. If $a \mid b$ and $b \neq 0$ then $|a| \leq |b|$.
5. If $a \mid b$ and $b \mid a$ then $a = \pm b$.
6. $0 \mid b$ if and only if $b = 0$.

Proof. (1) is immediate: $a = a \cdot 1$, $a = 1 \cdot a$, $0 = a \cdot 0$. For (2), write $b = ae$ and $c = bf$; then $c = a(e f)$, so $a \mid c$. For (3), write $b = ae$ and $c = af$; then $bx + cy = a(ex + fy)$. For (4), write $b = ac$ with $b \neq 0$, so $c \neq 0$ and hence $|c| \geq 1$; then $|b| = |a| |c| \geq |a|$. For (5), if either is 0 then by (6) so is the other and the claim holds; otherwise (4) gives $|a| \leq |b|$ and $|b| \leq |a|$, so $|a| = |b|$ and $a = \pm b$. For (6), $0 \mid b$ means $b = 0 \cdot c = 0$; conversely $0 \mid 0$. $\square$

The relation $\mid$ is thus reflexive and transitive; it is a partial order when restricted to the natural numbers (where (5) becomes $a = b$), but on all of $\mathbb{Z}$ it fails to be antisymmetric precisely because of signs. The *units* of $\mathbb{Z}$ —the integers dividing 1—are exactly ± 1 , and two integers that divide each other are called *associates*; in $\mathbb{Z}$ the associates of a are $\pm a$.

2.2 The division algorithm

Divisibility is the special case of exact division. The fundamental fact that makes the arithmetic of $\mathbb{Z}$ tractable is that one can always divide *with remainder*, and that the quotient and remainder are uniquely determined once a convention for the remainder's range is fixed.

Theorem 2.4 (Division algorithm). *Let $a \in \mathbb{Z}$ and $b \in \mathbb{Z}$ with $b > 0$. Then there exist unique integers q (the quotient) and r (the remainder) such that*

$$a = bq + r, \quad 0 \leq r < b.$$

Proof. Existence. Consider the set

$$S = \{a - bk : k \in \mathbb{Z}, a - bk \geq 0\} \subseteq \mathbb{N}.$$

First, $S \neq \emptyset$. If $a \geq 0$ then $a = a - b \cdot 0 \in S$. If $a < 0$ then, since $b \geq 1$, taking $k = a$ gives $a - ba = a(1 - b) \geq 0$ (a product of a nonpositive and a nonpositive factor), so $a - ba \in S$. Thus S is a nonempty subset of $\mathbb{N}$, and by the well-ordering principle (Theorem 2.1) it has a least element $r = a - bq$ for some $q \in \mathbb{Z}$. By construction $r \geq 0$. If $r \geq b$, then $r - b = a - b(q + 1) \geq 0$ would lie in S and be strictly smaller than r , contradicting minimality. Hence $0 \leq r < b$.

Uniqueness. Suppose $a = bq + r = bq' + r'$ with $0 \leq r, r' < b$. Then $b(q - q') = r' - r$, so $b \mid (r' - r)$. But $-b < r' - r < b$, and the only multiple of b strictly between $-b$ and b is 0. Hence $r' = r$, and then $b(q - q') = 0$ with $b \neq 0$ forces $q = q'$. $\square$

Remark 2.5. The hypothesis $b > 0$ is only for definiteness. For general $b \neq 0$ one obtains unique q, r with $a = bq + r$ and $0 \leq r < |b|$, by applying the theorem to $|b|$ and adjusting the sign of q . Note that the remainder convention $0 \leq r < |b|$ differs from the truncation behaviour of many programming languages, where the remainder of a negative dividend may be negative; for the number-theoretic results below the convention above is the appropriate one.

We denote the remainder r produced by the division algorithm by $a \bmod b$, and the quotient q by $\lfloor a/b \rfloor$ when $b > 0$ (the floor of the rational number a/b). With this notation, $b \mid a$ holds precisely when $a \bmod b = 0$.

2.3 The greatest common divisor and the Euclidean algorithm

Definition 2.6. A *common divisor* of integers a and b is an integer d with $d \mid a$ and $d \mid b$. The *greatest common divisor* of a and b , not both zero, is the largest such d ; it is denoted $\gcd(a, b)$. By convention $\gcd(0, 0) = 0$. Integers a and b are *coprime* (or *relatively prime*) if $\gcd(a, b) = 1$.

The greatest common divisor is well defined when $(a, b) \neq (0, 0)$: the set of common divisors is nonempty (it contains 1) and bounded above (every common divisor of, say, the nonzero a has absolute value at most $|a|$ by Proposition 2.3(4)), so it has a largest element, which is positive since 1 is a common divisor. We establish a much stronger characterisation shortly: the gcd is not merely the numerically largest common divisor but is divisible by every common divisor. The key computational engine is the following observation.

Lemma 2.7. Let $a, b \in \mathbb{Z}$. For any $q \in \mathbb{Z}$,

$$\gcd(a, b) = \gcd(b, a - qb).$$

In particular, if $a = bq + r$ then $\gcd(a, b) = \gcd(b, r)$, and $\gcd(a, 0) = |a|$.

Proof. It suffices to show the two pairs (a, b) and $(b, a - qb)$ have exactly the same common divisors; equal sets of common divisors have equal greatest elements (and the same coprimality status). If $d \mid a$ and $d \mid b$, then $d \mid (a - qb)$ by Proposition 2.3(3), so d is a common divisor of b and $a - qb$. Conversely, if $d \mid b$ and $d \mid (a - qb)$, then $d \mid ((a - qb) + qb) = a$, so d is a common divisor of a and b . The two sets of common divisors coincide. Finally $\gcd(a, 0) = |a|$ because the common divisors of a and 0 are exactly the divisors of a , the largest of which in absolute value is $|a|$ (and $|a| > 0$ when $a \neq 0$; the case $a = 0$ is the convention $\gcd(0, 0) = 0$). $\square$

Iterating the lemma with the division algorithm yields the oldest nontrivial algorithm in mathematics.

Theorem 2.8 (Euclidean algorithm). Let a, b be integers with $b \neq 0$. Define a sequence of remainders by $r_0 = |a|$, $r_1 = |b|$, and, as long as $r_i \neq 0$, let $r_{i+1} = r_{i-1} \bmod r_i$, so that

$$r_{i-1} = r_i q_i + r_{i+1}, \quad 0 \leq r_{i+1} < r_i.$$

The sequence $r_1 > r_2 > \dots$ of positive remainders is strictly decreasing, so some $r_{n+1} = 0$; the last nonzero remainder satisfies $r_n = \gcd(a, b)$.

Proof. The remainders $r_1, r_2, \dots$ are nonnegative integers and, while nonzero, strictly decreasing because $r_{i+1} = r_{i-1} \bmod r_i < r_i$. A strictly decreasing sequence of nonnegative integers cannot be infinite (again by well-ordering: otherwise the set of values would have no least element), so the algorithm terminates with some $r_{n+1} = 0$ and $r_n > 0$. By Lemma 2.7, each step preserves the gcd:

$$\gcd(a, b) = \gcd(r_0, r_1) = \gcd(r_1, r_2) = \dots = \gcd(r_n, r_{n+1}) = \gcd(r_n, 0) = r_n,$$

using $\gcd(|a|, |b|) = \gcd(a, b)$ (signs do not affect common divisors). $\square$

Example 2.9. Compute $\gcd(1071, 462)$:

$$\begin{aligned} 1071 &= 2 \cdot 462 + 147, \\ 462 &= 3 \cdot 147 + 21, \\ 147 &= 7 \cdot 21 + 0. \end{aligned}$$

The last nonzero remainder is 21, so $\gcd(1071, 462) = 21$. Indeed $1071 = 21 \cdot 51$ and $462 = 21 \cdot 22$, with $\gcd(51, 22) = 1$.

Remark 2.10. The Euclidean algorithm is efficient. The number of division steps for inputs $a > b > 0$ is $O(\log b)$; the worst case occurs at consecutive Fibonacci numbers (Lamé’s theorem), where every quotient q_i equals 1 except the final one (which equals 2) and the remainders shrink as slowly as possible. With schoolbook arithmetic on k -bit inputs the whole computation costs $O(k^2)$ bit operations. This efficiency is what renders gcd-based cryptographic operations—above all modular inversion, below—practical at the field sizes used in Halo 2 and Orchard.

2.4 The Bézout identity and the extended Euclidean algorithm

The Euclidean algorithm computes the gcd, but the back-substitution of its equations yields something more powerful: the gcd is an *integer linear combination* of the inputs. This is among the most useful facts in elementary number theory.

Theorem 2.11 (Bézout’s identity). *Let $a, b \in \mathbb{Z}$, not both zero, and let $d = \gcd(a, b)$. Then there exist integers $u, v \in \mathbb{Z}$ with*

$$au + bv = d.$$

Moreover d is the smallest positive integer expressible in the form $ax + by$ with $x, y \in \mathbb{Z}$, and the set of all such combinations is exactly the set of multiples of d :

$$\{ax + by : x, y \in \mathbb{Z}\} = d\mathbb{Z} = \{kd : k \in \mathbb{Z}\}.$$

Proof. Let $I = \{ax + by : x, y \in \mathbb{Z}\}$. This set contains a and b and is closed under addition and under multiplication by any integer; concretely, if $s = ax_1 + by_1$ and $t = ax_2 + by_2$ lie in I , then for any $m, n \in \mathbb{Z}$ one has

$$ms + nt = a(mx_1 + nx_2) + b(my_1 + ny_2) \in I.$$

Since a, b are not both zero, I contains a positive element (one of $\pm a, \pm b$), so the set $I \cap \mathbb{N}_{>0}$ of positive elements of I is nonempty. By well-ordering it has a least element $g = au + bv > 0$.

We claim that g divides every element of I . Indeed, let $c \in I$ and write $c = gq + r$ with $0 \leq r < g$ by the division algorithm. Then $r = c - gq \in I$ because I is closed under the operations just described. By minimality of g among positive elements of I , and $0 \leq r < g$, necessarily $r = 0$. Hence $g \mid c$ for every $c \in I$; in particular $g \mid a$ and $g \mid b$, so g is a common divisor of a and b . Conversely, any common divisor e of a and b divides $au + bv = g$ by Proposition 2.3(3). Therefore g is a common divisor that is divisible by every common divisor; being positive, it is in particular the *greatest* common divisor, so $g = d$. The displayed equation $au + bv = d$ follows, d is the least positive element of I , and $I = g\mathbb{Z} = d\mathbb{Z}$ because every element of I is a multiple of g , as shown, and conversely every multiple $kg = a(ku) + b(kv)$ lies in I . $\square$

Remark 2.12. The proof yields a characterisation of the gcd that is independent of the order $\leq$: $d = \gcd(a, b)$ is, up to sign, the unique common divisor of a and b that is divisible by every common divisor. This is the definition that generalises to arbitrary commutative rings, where “largest” may make no sense but “divisible by every common divisor” still does. We call the subsets $I \subseteq \mathbb{Z}$ closed under addition and under multiplication by arbitrary integers *ideals*; the content of Bézout’s identity is that every ideal of $\mathbb{Z}$ has the form $d\mathbb{Z}$ for a single generator d , i.e. $\mathbb{Z}$ is a *principal ideal domain*. Ideals recur in the construction of quotient rings.

The Bézout coefficients u, v are not unique. If $au + bv = d$, then for every $k \in \mathbb{Z}$,

$$a\left(u + k\frac{b}{d}\right) + b\left(v - k\frac{a}{d}\right) = d,$$

and these are all the solutions. To compute a particular pair (u, v) efficiently, we augment the Euclidean algorithm so that it carries each remainder as an explicit combination of a and b .

Theorem 2.13 (Extended Euclidean algorithm). *There is an algorithm that, on input $a, b \in \mathbb{Z}$ not both zero, outputs $d = \gcd(a, b)$ together with integers u, v satisfying $au + bv = d$, using the same number of division steps as the Euclidean algorithm.*

Proof. Alongside the remainder sequence of Theorem 2.8, maintain two auxiliary sequences (s_i) and (t_i) with the invariant

$$r_i = as_i + bt_i \quad \text{for all } i. \quad (*)$$

Initialise (taking $a, b \geq 0$ for clarity; we restore signs at the end)

$$r_0 = a, s_0 = 1, t_0 = 0; \quad r_1 = b, s_1 = 0, t_1 = 1,$$

so $(*)$ holds for $i = 0, 1$. At step $i \geq 1$, having $r_{i-1} = r_i q_i + r_{i+1}$ from the division algorithm, set

$$s_{i+1} = s_{i-1} - q_i s_i, \quad t_{i+1} = t_{i-1} - q_i t_i.$$

Then

$$as_{i+1} + bt_{i+1} = (as_{i-1} + bt_{i-1}) - q_i(as_i + bt_i) = r_{i-1} - q_i r_i = r_{i+1},$$

so induction preserves $(*)$. When the algorithm terminates with $r_{n+1} = 0$ and $r_n = d = \gcd(a, b)$, the invariant for $i = n$ reads $d = as_n + bt_n$; output $u = s_n, v = t_n$. Termination and the value of d are exactly as in Theorem 2.8, and each step does only a constant amount of extra work. For general inputs, run the algorithm on $(|a|, |b|)$ to obtain $|a|u' + |b|v' = d$ (recall $\gcd(|a|, |b|) = \gcd(a, b)$) and output $u = \text{sgn}(a)u', v = \text{sgn}(b)v'$, so that $au + bv = d$. $\square$

In practice it is convenient to lay the computation out as a table with columns (r_i, q_i, s_i, t_i) , computing each new row from the previous two.

Example 2.14. Run the extended algorithm on $a = 1071, b = 462$ (continuing Example 2.9). The quotients are $q_1 = 2, q_2 = 3, q_3 = 7$.

i	r_i	s_i	t_i
0	1071	1	0
1	462	0	1
2	147	1	-2
3	21	-3	7
4	0	22	-51

For instance, row 2 is row 0 minus $q_1 = 2$ times row 1: $s_2 = 1 - 2 \cdot 0 = 1, t_2 = 0 - 2 \cdot 1 = -2$, and indeed $1071 \cdot 1 + 462 \cdot (-2) = 1071 - 924 = 147$. The last nonzero remainder is $r_3 = 21 = \gcd(1071, 462)$, with

$$1071 \cdot (-3) + 462 \cdot 7 = -3213 + 3234 = 21.$$

Thus $(u, v) = (-3, 7)$. The final row $(s_4, t_4) = (22, -51) = (b/d, -a/d)$ up to sign records the homogeneous solution: $1071 \cdot 22 + 462 \cdot (-51) = 0$.

A first, decisive application of Bézout's identity is Euclid's lemma, the property of primes that ultimately forces unique factorisation.

Proposition 2.15 (Euclid's lemma). *Let $a, b, c \in \mathbb{Z}$.*

1. *If $c \mid ab$ and $\gcd(c, a) = 1$, then $c \mid b$.*
2. *If $\gcd(a, b) = 1$, $a \mid n$, and $b \mid n$, then $ab \mid n$.*
3. *If $\gcd(a, b) = 1$ and $\gcd(a, c) = 1$, then $\gcd(a, bc) = 1$.*

Proof. (1) By Bézout there are u, v with $cu + av = 1$. Multiply by b : $b = cub + avb = c(ub) + (ab)v$. Since $c \mid ab$, c divides both terms on the right, hence $c \mid b$.

(2) Write $n = ak$. Since $b \mid n = ak$ and $\gcd(b, a) = 1$, part (1) gives $b \mid k$, say $k = bm$. Then $n = a(bm) = (ab)m$, so $ab \mid n$.

(3) By Bézout, $ax_1 + by_1 = 1$ and $ax_2 + cy_2 = 1$ for suitable integers. Multiplying,

$$1 = (ax_1 + by_1)(ax_2 + cy_2) = a \underbrace{(ax_1x_2 + x_1cy_2 + by_1x_2)}_{\in \mathbb{Z}} + bc(y_1y_2),$$

which exhibits 1 as an integer combination of a and bc . Hence $\gcd(a, bc)$ divides 1, so it equals 1. $\square$

2.5 Primes and the fundamental theorem of arithmetic

Definition 2.16. An integer $p > 1$ is *prime* if its only positive divisors are 1 and p . An integer $n > 1$ that is not prime is *composite*; thus n is composite iff $n = ab$ for some integers $1 < a, b < n$. The integer 1 is a *unit* and is by convention neither prime nor composite.

The exclusion of 1 from the primes is not arbitrary: it is exactly what makes factorisation into primes unique, since otherwise one could insert arbitrarily many factors of 1. The decisive property of primes is the prime case of Euclid's lemma.

Lemma 2.17 (Euclid's lemma for primes). *Let p be prime and $a, b \in \mathbb{Z}$. If $p \mid ab$ then $p \mid a$ or $p \mid b$. More generally, if $p \mid a_1a_2 \cdots a_k$ then $p \mid a_i$ for some i .*

Proof. The only positive divisors of p are 1 and p , so $\gcd(p, a) \in \{1, p\}$. If $\gcd(p, a) = p$ then $p \mid a$ and the claim holds; otherwise $\gcd(p, a) = 1$, and Proposition 2.15(1) gives $p \mid b$. The general statement follows by induction on k : from $p \mid a_1(a_2 \cdots a_k)$ it follows that $p \mid a_1$ or $p \mid a_2 \cdots a_k$, and the inductive hypothesis handles the latter. $\square$

Theorem 2.18 (Fundamental theorem of arithmetic). *Every integer $n > 1$ can be written as a product of primes,*

$$n = p_1 p_2 \cdots p_k,$$

and this factorisation is unique up to the order of the factors. Equivalently, there is a unique expression

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}, \quad p_1 < p_2 < \cdots < p_r \text{ prime, } e_i \geq 1.$$

Proof. Existence. Suppose, for contradiction, that some integer > 1 is not a product of primes, and let n be the least such (well-ordering). Then n is not prime (a prime is trivially a product of one prime), so $n = ab$ with $1 < a, b < n$. By minimality, each of a, b is a product of primes; concatenating these factorisations expresses n as a product of primes, a contradiction. Hence every $n > 1$ factors into primes.

Uniqueness. Suppose

$$p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_\ell$$

are two factorisations of the same integer into primes. The argument proceeds by induction on k . If $k = 0$ the product is the empty product 1, forcing $\ell = 0$ as well (a nonempty product of primes is > 1). For the inductive step, p_1 divides the right-hand product, so by Lemma 2.17 $p_1 \mid q_j$ for some j . Since q_j is prime, its only divisor > 1 is itself, so $p_1 = q_j$. Cancelling this common factor (legitimate as $\mathbb{Z}$ has no zero divisors) yields

$$p_2 \cdots p_k = q_1 \cdots \hat{q}_j \cdots q_\ell,$$

where the hat denotes omission. By the inductive hypothesis these two factorisations agree up to order, with $k - 1 = \ell - 1$; restoring $p_1 = q_j$ shows the original factorisations agree up to order. Collecting equal primes into prime powers gives the canonical form, whose uniqueness is immediate from the uniqueness of the multiset of primes. $\square$

Remark 2.19. Both halves of the theorem are needed and have different characters. Existence is a statement about the order structure (well-ordering) and would hold in any reasonable factorisation; uniqueness rests squarely on Euclid's lemma, hence on Bézout, hence on the division algorithm. There are number systems with a division-like structure failing Euclid's lemma in which factorisation into irreducibles is *not* unique; the integers avoid this pathology precisely because they form a Euclidean domain.

2.6 Congruences and the ring $\mathbb{Z}/n\mathbb{Z}$

The development now passes from divisibility to its quotient, the arithmetic of remainders. Fix an integer $n \geq 1$, the *modulus*.

Definition 2.20. Let $n \geq 1$ and $a, b \in \mathbb{Z}$. The integer a is *congruent to b modulo n* , written

$$a \equiv b \pmod{n},$$

if $n \mid (a - b)$, equivalently if a and b leave the same remainder upon division by n .

The two formulations agree: if $a = nq_1 + r_1$ and $b = nq_2 + r_2$ with $0 \leq r_1, r_2 < n$, then $a - b = n(q_1 - q_2) + (r_1 - r_2)$, and $n \mid (a - b)$ iff $n \mid (r_1 - r_2)$ iff $r_1 = r_2$ (since $|r_1 - r_2| < n$).

Proposition 2.21. Fix $n \geq 1$. Congruence modulo n is an equivalence relation on $\mathbb{Z}$ that is moreover compatible with addition and multiplication: if $a \equiv a' \pmod{n}$ and $b \equiv b' \pmod{n}$, then

$$a + b \equiv a' + b' \pmod{n}, \quad ab \equiv a'b' \pmod{n}.$$

Consequently $a^k \equiv (a')^k \pmod{n}$ for all $k \geq 0$, and one may substitute congruent values freely in any polynomial expression with integer coefficients.

Proof. Reflexivity ($n \mid 0$), symmetry ($n \mid (a - b) \Rightarrow n \mid (b - a)$), and transitivity ($n \mid (a - b)$ and $n \mid (b - c)$ give $n \mid (a - c)$ by Proposition 2.3(3)) are immediate. For compatibility, write $a - a' = ns$ and $b - b' = nt$. Then

$$(a + b) - (a' + b') = n(s + t),$$

and

$$ab - a'b' = ab - a'b + a'b - a'b' = (a - a')b + a'(b - b') = n(sb + a't),$$

so both $n \mid ((a + b) - (a' + b'))$ and $n \mid (ab - a'b')$. The statement about powers and polynomials follows by induction from the multiplicative case. $\square$

Because congruence is an equivalence relation, it partitions $\mathbb{Z}$ into disjoint *congruence classes* (or *residue classes*).

Definition 2.22. The *congruence class* of a modulo n is

$$[a]_n = \{b \in \mathbb{Z} : b \equiv a \pmod{n}\} = \{a + kn : k \in \mathbb{Z}\} = a + n\mathbb{Z}.$$

We write the set of all congruence classes as

$$\mathbb{Z}/n\mathbb{Z} = \{[a]_n : a \in \mathbb{Z}\}.$$

By the division algorithm every integer is congruent modulo n to exactly one of $0, 1, \dots, n-1$, namely its remainder; hence the n classes $[0]_n, [1]_n, \dots, [n-1]_n$ are distinct and exhaust $\mathbb{Z}/n\mathbb{Z}$. The set $\{0, 1, \dots, n-1\}$ is the standard *system of representatives*, called the *least nonnegative residues*.

Theorem 2.23. For each $n \geq 1$, the operations

$$[a]_n + [b]_n := [a + b]_n, \quad [a]_n \cdot [b]_n := [ab]_n$$

are well defined and make $\mathbb{Z}/n\mathbb{Z}$ a commutative ring with additive identity $[0]_n$ and multiplicative identity $[1]_n$. It has exactly n elements.

Proof. Well-definedness is precisely the compatibility of Proposition 2.21: the sum and product of classes do not depend on the chosen representatives. Each ring axiom for $\mathbb{Z}/n\mathbb{Z}$ (associativity and commutativity of $+$ and $\cdot$, distributivity, existence of 0 and 1, additive inverses $-[a]_n = [-a]_n$) is inherited directly from the corresponding axiom in $\mathbb{Z}$ by passing to classes; for instance associativity of multiplication reads $([a][b])[c] = [abc] = [a]([b][c])$. The count of exactly n classes appeared above. $\square$

Remark 2.24. Structurally, $\mathbb{Z}/n\mathbb{Z}$ is the quotient of the ring $\mathbb{Z}$ by the ideal $n\mathbb{Z} = \{kn : k \in \mathbb{Z}\}$ (Remark 2.12). The map $\pi : \mathbb{Z} \rightarrow \mathbb{Z}/n\mathbb{Z}, \pi(a) = [a]_n$, is a surjective ring homomorphism with kernel $n\mathbb{Z}$; this is the prototype of the quotient constructions used throughout the algebra to come. One often drops the brackets and the subscript, writing a for $[a]_n$ when the modulus is clear, and uses “ $\mathbb{Z}/n\mathbb{Z}$ ” or “ $\mathbb{Z}_n$ ” interchangeably; $\mathbb{Z}_n$ denotes this ring (not the n -adic integers, which do not appear here).

A basic structural question is when $\mathbb{Z}/n\mathbb{Z}$ has no zero divisors—equivalently, when it is a field.

Proposition 2.25. For $n \geq 2$, the ring $\mathbb{Z}/n\mathbb{Z}$ has no zero divisors if and only if n is prime; in that case, as we show below, it is a field, written $\mathbb{F}_p$ when $n = p$.

Proof. If $n = ab$ with $1 < a, b < n$ is composite, then $[a]_n \neq [0]_n \neq [b]_n$ but $[a]_n[b]_n = [n]_n = [0]_n$, so there are zero divisors. Conversely, if $n = p$ is prime and $[a]_p[b]_p = [0]_p$, then $p \mid ab$, so by Lemma 2.17 $p \mid a$ or $p \mid b$, i.e. $[a]_p = [0]_p$ or $[b]_p = [0]_p$. That $\mathbb{Z}/p\mathbb{Z}$ is a field follows from the discussion of units below (Corollary 2.28). $\square$

2.7 Units modulo n and modular inverses

In any ring, the elements with multiplicative inverses are the most important. In $\mathbb{Z}/n\mathbb{Z}$ the Euclidean machinery characterises them and computes them.

Definition 2.26. An element $[a]_n \in \mathbb{Z}/n\mathbb{Z}$ is a *unit* (or is *invertible*) if there exists $[x]_n$ with $[a]_n[x]_n = [1]_n$, i.e. $ax \equiv 1 \pmod{n}$. Such an $[x]_n$ is the (*modular*) *inverse* of $[a]_n$, written $[a]_n^{-1}$. We write the set of units as

$$(\mathbb{Z}/n\mathbb{Z})^\times = \{[a]_n : [a]_n \text{ is a unit}\}.$$

Theorem 2.27. *Let $n \geq 1$ and $a \in \mathbb{Z}$. The class $[a]_n$ is a unit in $\mathbb{Z}/n\mathbb{Z}$ if and only if $\gcd(a, n) = 1$. When $\gcd(a, n) = 1$, the inverse is unique, and the extended Euclidean algorithm computes a representative: if $au + nv = 1$, then $[a]_n^{-1} = [u]_n$.*

Proof. Suppose $\gcd(a, n) = 1$. By Bézout (Theorem 2.11) there are u, v with $au + nv = 1$. Reducing modulo n , $au \equiv 1 \pmod{n}$, so $[u]_n$ is an inverse of $[a]_n$. The extended Euclidean algorithm (Theorem 2.13) produces such a u explicitly.

Conversely, if $[a]_n$ is a unit, say $ax \equiv 1 \pmod{n}$, then $ax - 1 = nk$ for some k , i.e. $ax - nk = 1$. Any common divisor of a and n divides the left side, hence divides 1; thus $\gcd(a, n) = 1$.

Uniqueness: if $ax \equiv 1$ and $ax' \equiv 1 \pmod{n}$, then $x \equiv x(ax') \equiv (xa)x' \equiv x' \pmod{n}$, so $[x]_n = [x']_n$. $\square$

Corollary 2.28. *If p is prime then every nonzero class in $\mathbb{Z}/p\mathbb{Z}$ is a unit, so $\mathbb{Z}/p\mathbb{Z} = \mathbb{F}_p$ is a field. More generally, $\mathbb{Z}/n\mathbb{Z}$ is a field if and only if n is prime.*

Proof. If p is prime and $[a]_p \neq [0]_p$ then $p \nmid a$, so $\gcd(a, p) = 1$ (the only positive divisors of p are 1, p), and Theorem 2.27 makes $[a]_p$ a unit. Thus every nonzero element is invertible and $\mathbb{F}_p$ is a field. If n is composite, $n = ab$ with $1 < a, b < n$, then $[a]_n \neq [0]_n$ yet $\gcd(a, n) = a > 1$, so $[a]_n$ is not a unit; a ring with a nonzero non-unit is not a field. The case $n = 1$ gives the zero ring, not a field by convention. $\square$

Example 2.29. Consider the computation of $[17]_{43}^{-1}$. The extended Euclidean algorithm on $(43, 17)$:

$$\begin{aligned} 43 &= 2 \cdot 17 + 9, \\ 17 &= 1 \cdot 9 + 8, \\ 9 &= 1 \cdot 8 + 1, \\ 8 &= 8 \cdot 1 + 0, \end{aligned}$$

so $\gcd(43, 17) = 1$. Back-substituting:

$$1 = 9 - 8 = 9 - (17 - 9) = 2 \cdot 9 - 17 = 2(43 - 2 \cdot 17) - 17 = 2 \cdot 43 - 5 \cdot 17.$$

Hence $-5 \cdot 17 \equiv 1 \pmod{43}$, and $[17]_{43}^{-1} = [-5]_{43} = [38]_{43}$. Check: $17 \cdot 38 = 646 = 15 \cdot 43 + 1$, so $17 \cdot 38 \equiv 1 \pmod{43}$.

The set of units forms a group under multiplication, a fact we use heavily in what follows.

Proposition 2.30. *Under multiplication of classes, $(\mathbb{Z}/n\mathbb{Z})^\times$ is an abelian group. Its order is denoted $\varphi(n)$ (Euler's totient): the number of integers in $\{1, \dots, n\}$ coprime to n .*

Proof. Multiplication of classes is associative and commutative (Theorem 2.23) with identity $[1]_n$. It is closed on units: if $[a]_n, [b]_n$ are units with inverses $[a]_n^{-1}, [b]_n^{-1}$, then $[ab]_n$ has inverse $[b]_n^{-1}[a]_n^{-1}$, since $(ab)(b^{-1}a^{-1}) \equiv 1$. Every unit has an inverse by definition, and that inverse is itself a unit (it is invertible, with inverse the original element). Hence $(\mathbb{Z}/n\mathbb{Z})^\times$ is an abelian group. By Theorem 2.27 its elements are exactly the classes $[a]_n$ with $\gcd(a, n) = 1$, $1 \leq a \leq n$, of which there are $\varphi(n)$ by definition. $\square$

Remark 2.31. The totient $\varphi(n)$ governs the multiplicative order structure modulo n . For prime p , $\varphi(p) = p - 1$, so $\mathbb{F}_p^\times$ has order $p - 1$; for a prime power, $\varphi(p^k) = p^{k-1}(p - 1)$. Lagrange's theorem (developed in the group-theory section) then gives Euler's theorem $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $\gcd(a, n) = 1$, and in particular Fermat's little theorem $a^{p-1} \equiv 1 \pmod{p}$ for $p \nmid a$. These facts underlie modular

exponentiation, the cyclic structure of $\mathbb{F}_p^\times$, and the choice of scalar field sizes in the elliptic-curve constructions of Halo 2 and Orchard. The Chinese Remainder Theorem (proved in the number-theory section: for coprime m_1, m_2 , a residue modulo $m_1 m_2$ corresponds exactly to a pair of residues modulo m_1 and m_2) yields the multiplicativity of φ , which then computes $\varphi(n)$ for general n .

3 Groups

The notion of a *group* is the foundational abstraction of modern algebra. A group axiomatises the bare minimum needed to discuss *symmetry* and *invertible composition*: a set together with one associative way of combining its elements, possessing an identity and inverses. This short list of axioms is rich enough to support a deep structure theory, and almost every object encountered later—the integers modulo a prime under addition, the nonzero residues under multiplication, the points of an elliptic curve, the symmetries of a finite field—is a group. This section develops group theory from first principles, proving the central structural results in full: Lagrange’s theorem, the classification of cyclic groups, the quotient of an abelian group by a subgroup, and the basic theory of homomorphisms and isomorphisms.

3.1 Definition and axioms

Before we can discuss groups, we must make precise the meaning of “combining” two elements of a set.

Definition 3.1. Let S be a set. A *binary operation* on S is a function

$$* : S \times S \rightarrow S, \quad (a, b) \mapsto a * b.$$

The element $a * b$ denotes the image of the pair (a, b) . The defining feature is *closure*: for all $a, b \in S$, the element $a * b$ again lies in S .

Closure is built into the very statement that $*$ is a function *into* S ; we name it explicitly nonetheless because, when we later examine a subset $H \subseteq S$, the question of whether $*$ restricts to a binary operation on H is exactly the question of whether H is closed under $*$.

Definition 3.2. A *group* is a pair $(G, *)$ consisting of a set G and a binary operation $*$ on G satisfying the following three axioms.

(G1) *Associativity.* For all $a, b, c \in G$,

$$(a * b) * c = a * (b * c).$$

(G2) *Identity element.* There exists an element $e \in G$ such that for all $a \in G$,

$$e * a = a * e = a.$$

(G3) *Inverses.* For each $a \in G$ there exists an element $b \in G$ such that

$$a * b = b * a = e,$$

where e is an identity element as in (G2).

A set with a binary operation satisfying only (G1) is called a *semigroup*; if it also satisfies (G2) it is a *monoid*. Thus a group is a monoid in which every element is invertible.

We frequently use the phrase “the group G ”, with the operation understood. The number of elements of G , written $|G|$, is called the *order* of the group; if $|G|$ is finite G is a *finite group*, otherwise an *infinite group*.

The axioms assert the *existence* of an identity and of inverses, but a priori not their uniqueness. Uniqueness is a theorem, and a useful one, since it licenses the notation e and a^{-1} .

Proposition 3.3. *Let $(G, *)$ be a group.*

1. *The identity element is unique.*
2. *Each $a \in G$ has a unique inverse, which is denoted a^{-1} .*
3. *(Cancellation.) For all $a, x, y \in G$, if $a * x = a * y$ then $x = y$, and if $x * a = y * a$ then $x = y$.*
4. *For all $a, b \in G$, $(a^{-1})^{-1} = a$ and $(a * b)^{-1} = b^{-1} * a^{-1}$.*

Proof. (1) Suppose e and e' both satisfy (G2). Then, using that e' is an identity, $e = e * e'$; and using that e is an identity, $e * e' = e'$. Hence $e = e'$.

(2) Suppose b and b' are both inverses of a . Then

$$b = b * e = b * (a * b') = (b * a) * b' = e * b' = b',$$

using (G2), the inverse property of b' , associativity (G1), and the inverse property of b . Thus the inverse is unique.

(3) Suppose $a * x = a * y$. Applying a^{-1} on the left,

$$x = e * x = (a^{-1} * a) * x = a^{-1} * (a * x) = a^{-1} * (a * y) = (a^{-1} * a) * y = e * y = y.$$

The right-cancellation law is symmetric.

(4) Since $a * a^{-1} = a^{-1} * a = e$, the element a is an inverse of a^{-1} ; by uniqueness $(a^{-1})^{-1} = a$. For the second identity, compute

$$(a * b) * (b^{-1} * a^{-1}) = a * (b * b^{-1}) * a^{-1} = a * e * a^{-1} = a * a^{-1} = e,$$

and symmetrically $(b^{-1} * a^{-1}) * (a * b) = e$. By uniqueness of inverses, $(a * b)^{-1} = b^{-1} * a^{-1}$. $\square$

Notation. Two notational conventions dominate. In *multiplicative* notation one writes the operation as juxtaposition ab (or $a \cdot b$), the identity as 1, and the inverse as a^{-1} ; powers are $a^n = \underbrace{a \cdots a}_n$ for $n \geq 1$, $a^0 = 1$, and $a^{-n} = (a^{-1})^n$. In *additive* notation, reserved almost exclusively for commutative groups, one writes $a + b$, the identity as 0, the inverse as $-a$, and na for the n -fold sum. The usual exponent laws $a^m a^n = a^{m+n}$ and $(a^m)^n = a^{mn}$ hold for all integers m, n (proved by induction together with Proposition 3.3). Henceforth ab usually denotes $a * b$ and we drop the symbol $*$ unless clarity demands it.

3.2 Abelian groups

Definition 3.4. A group $(G, *)$ is *abelian* (or *commutative*) if in addition

$$a * b = b * a \quad \text{for all } a, b \in G.$$

A group that is not abelian we call *non-abelian*.

The term honours Niels Henrik Abel. Commutativity is a strong simplifying assumption: in an abelian group one may freely reorder products, the formula $(ab)^{-1} = b^{-1}a^{-1} = a^{-1}b^{-1}$ loses its order-sensitivity, and—as we show below—the structure theory becomes especially clean. Convention reserves additive notation for abelian groups precisely because $+$ connotes commutativity.

3.3 Examples of groups

We now exhibit a generous supply of groups. The reader should check each example against the three axioms; we comment only on the points that are not immediate.

Example 3.5 (Integers under addition). The set $\mathbb{Z}$ of integers with the operation $+$ is an abelian group. Addition is associative and commutative, 0 is the identity, and the inverse of n is $-n$. The same holds for $(\mathbb{Q}, +)$, $(\mathbb{R}, +)$, and $(\mathbb{C}, +)$. All of these are infinite. Note $(\mathbb{Z}, \cdot)$ is *not* a group: 2 has no multiplicative inverse in $\mathbb{Z}$.

Example 3.6 (Nonzero elements of a field under multiplication). For $\mathbb{Q}$, $\mathbb{R}$, or $\mathbb{C}$, the set of *nonzero* elements, denoted $\mathbb{Q}^\times$, $\mathbb{R}^\times$, $\mathbb{C}^\times$, forms an abelian group under multiplication, with identity 1 and inverse $a^{-1} = 1/a$. We must exclude the element 0 because 0 has no multiplicative inverse. More generally, for any field $\mathbb{F}$ the nonzero elements $\mathbb{F}^\times = \mathbb{F} \setminus \{0\}$ form an abelian group under multiplication; this fact is part of the very definition of a field.

Example 3.7 (Integers and units modulo n). $(\mathbb{Z}/n\mathbb{Z}, +)$ is an abelian group of order n (Theorem 2.23), and the invertible classes form the abelian group $(\mathbb{Z}/n\mathbb{Z})^\times$ of order $\varphi(n)$ under multiplication (Theorems 2.27 and 2.30). When $n = p$ is prime, every nonzero class is a unit, so $(\mathbb{Z}/p\mathbb{Z})^\times = \mathbb{F}_p^\times$ has order $p - 1$; this is Example 3.6 for the finite field $\mathbb{F}_p$.

Example 3.8 (Trivial group and small groups). The one-element set $\{e\}$ with the only possible operation is a group, the *trivial group*. The group $\mathbb{Z}/2\mathbb{Z} = \{0, 1\}$ is the smallest nontrivial group. The *Klein four-group* $V = \{e, a, b, c\}$, in which every nonidentity element squares to e and the product of any two distinct nonidentity elements is the third, is abelian of order 4 and is *not* isomorphic to $\mathbb{Z}/4\mathbb{Z}$ (it has no element of order 4).

3.4 Order of an element

Definition 3.9. Let G be a group with identity e and let $a \in G$. The *order* of a , written $\text{ord}(a)$, is the least positive integer n such that $a^n = e$, if such an n exists; in that case a has *finite order*. If no positive power of a equals e , then a has *infinite order* and we write $\text{ord}(a) = \infty$.

The theory of cyclic groups below reconciles the two uses of the word “order”—for an element and for a group: $\text{ord}(a)$ equals the order of the smallest subgroup containing a .

Proposition 3.10. Let $a \in G$ have finite order $d = \text{ord}(a)$. Then for any integer m ,

$$a^m = e \iff d \mid m.$$

Consequently $a^m = a^k$ iff $m \equiv k \pmod{d}$, and the distinct powers of a are exactly $e = a^0, a^1, \dots, a^{d-1}$.

Proof. If $d \mid m$, write $m = dq$; then $a^m = (a^d)^q = e^q = e$. Conversely, suppose $a^m = e$. By the division algorithm write $m = dq + r$ with $0 \leq r < d$. Then

$$e = a^m = (a^d)^q a^r = a^r.$$

Since $0 \leq r < d$ and d is the *least* positive integer with $a^d = e$, it follows that $r = 0$, i.e. $d \mid m$. The remaining claims follow: $a^m = a^k \iff a^{m-k} = e \iff d \mid (m-k)$, and the d elements $a^0, \dots, a^{d-1}$ are pairwise distinct because their exponents are incongruent modulo d . $\square$

Example 3.11. In $(\mathbb{Z}/n\mathbb{Z}, +)$ the order of the element 1 is n , since $\underbrace{1 + \dots + 1}_k = k$ is 0 iff $n \mid k$. The order of an element a is $n/\gcd(a, n)$ (Corollary 3.21). In $\mathbb{C}^\times$, the element i has order 4: $i^1 = i, i^2 = -1, i^3 = -i, i^4 = 1$. In $\mathbb{Z}$ under addition, every nonzero element has infinite order.

3.5 Subgroups and the subgroup test

Definition 3.12. A subset H of a group $(G, *)$ is a *subgroup*, written $H \leq G$, if H is itself a group under the restriction of $*$ to $H \times H$. Explicitly: H is nonempty, closed under $*$ (so $*$ restricts to a binary operation on H), contains the identity e of G , and contains a^{-1} whenever it contains a .

Every group G has at least two subgroups: the *trivial subgroup* $\{e\}$ and G itself. We call a subgroup other than G *proper*. Note that the identity of a subgroup necessarily coincides with the identity of G (apply cancellation in G to $e_H * e_H = e_H = e_H * e$), and inverses computed in H agree with those in G ; thus the definition is not as redundant as it may appear.

Verifying all the group axioms for a candidate subgroup is wasteful, since associativity is inherited for free. The following test reduces the verification to a single condition.

Proposition 3.13 (One-step subgroup test). *Let G be a group and $H \subseteq G$. Then $H \leq G$ if and only if*

1. $H \neq \emptyset$, and
2. for all $a, b \in H$, the element $ab^{-1} \in H$.

Proof. ($\Rightarrow$) If H is a subgroup it is nonempty and, given $a, b \in H$, it contains b^{-1} and hence ab^{-1} .

($\Leftarrow$) Assume (1) and (2). Pick any $a \in H$ (possible by (1)). Taking $b = a$ in (2) gives $aa^{-1} = e \in H$. Taking $a = e$ and b arbitrary in H gives $eb^{-1} = b^{-1} \in H$, so H is closed under inverses. Finally, for $a, b \in H$ we now know $b^{-1} \in H$, so applying (2) to the pair (a, b^{-1}) yields $a(b^{-1})^{-1} = ab \in H$; thus H is closed under the operation. Associativity is inherited from G . Hence H is a group, i.e. $H \leq G$. $\square$

Remark 3.14. For a *finite* nonempty subset H closed under the operation, closure alone forces $H \leq G$: given $a \in H$, the powers $a, a^2, a^3, \dots$ all lie in H and cannot all be distinct (as H is finite), so $a^i = a^j$ for some $i < j$, giving $a^{j-i} = e \in H$ and $a^{-1} = a^{j-i-1} \in H$. Thus for finite subsets, closure under multiplication is the only condition one must verify.

Example 3.15. The even integers $2\mathbb{Z} = \{2k : k \in \mathbb{Z}\}$ form a subgroup of $(\mathbb{Z}, +)$; more generally $n\mathbb{Z} \leq \mathbb{Z}$ for every $n \geq 0$, and these are *all* the subgroups of $\mathbb{Z}$ (Proposition 3.24).

Proposition 3.16. *The intersection of any family $\{H_i\}_{i \in I}$ of subgroups of G is a subgroup of G .*

Proof. Let $H = \bigcap_i H_i$. Each H_i contains e , so $e \in H$ and $H \neq \emptyset$. If $a, b \in H$ then $a, b \in H_i$ for every i , so $ab^{-1} \in H_i$ for every i by the subgroup test, whence $ab^{-1} \in H$. By Proposition 3.13, $H \leq G$. $\square$

This lets us define the subgroup “generated” by an arbitrary subset as the smallest subgroup containing it.

Definition 3.17. Let $S \subseteq G$. The *subgroup generated by S* , written $\langle S \rangle$, is the intersection of all subgroups of G that contain S . By Proposition 3.16 it is a subgroup, and it is the smallest subgroup containing S . Concretely, $\langle S \rangle$ consists of all finite products $s_1^{\epsilon_1} \cdots s_k^{\epsilon_k}$ with $s_i \in S$ and $\epsilon_i \in \{+1, -1\}$ (the empty product being e).

3.6 Cyclic groups, generators, and their classification

The simplest groups are those generated by a single element.

Definition 3.18. A group G is *cyclic* if there exists $g \in G$ with $G = \langle g \rangle$, where

$$\langle g \rangle = \{g^k : k \in \mathbb{Z}\}$$

is the subgroup generated by g . We call such a g a *generator* of G . Every cyclic group is abelian, since $g^m g^n = g^{m+n} = g^n g^m$.

The description $\langle g \rangle = \{g^k : k \in \mathbb{Z}\}$ matches Definition 3.17: the right-hand side is closed under products and inverses and contains g , and any subgroup containing g must contain all its powers.

Proposition 3.19. Let $G = \langle g \rangle$ be cyclic.

1. If g has infinite order, then the powers g^k ($k \in \mathbb{Z}$) are pairwise distinct, so G is infinite and $|G| = \infty$.
2. If g has finite order d , then $G = \{e, g, g^2, \dots, g^{d-1}\}$ has exactly d elements, so $|G| = \text{ord}(g) = d$.

In both cases $|\langle g \rangle| = \text{ord}(g)$.

Proof. (1) If g has infinite order and $g^i = g^j$ with $i \neq j$, then $g^{i-j} = e$ with $i - j \neq 0$, contradicting infinite order; so all powers are distinct. (2) is Proposition 3.10: the distinct powers are exactly $g^0, \dots, g^{d-1}$. $\square$

Example 3.20. $(\mathbb{Z}, +)$ is infinite cyclic, generated by 1 (and also by -1). $(\mathbb{Z}/n\mathbb{Z}, +)$ is cyclic of order n , generated by 1. The group $(\mathbb{Z}/8\mathbb{Z})^\times = \{1, 3, 5, 7\}$ is *not* cyclic: every element squares to 1, so no element has order 4. By contrast $(\mathbb{Z}/5\mathbb{Z})^\times = \{1, 2, 3, 4\}$ is cyclic, generated by 2: $2^1 = 2$, $2^2 = 4$, $2^3 = 3$, $2^4 = 1$.

We now determine *which* powers of a generator are themselves generators, and classify the subgroups of a cyclic group.

Corollary 3.21. Let g have finite order n , and let $k \in \mathbb{Z}$. Then

$$\text{ord}(g^k) = \frac{n}{\gcd(n, k)}.$$

In particular g^k generates $\langle g \rangle$ if and only if $\gcd(n, k) = 1$, so a cyclic group of order n has exactly $\varphi(n)$ generators.

Proof. Let $d = \gcd(n, k)$ and write $k = dk'$, $n = dn'$ with $\gcd(n', k') = 1$. Then $(g^k)^m = g^{km} = e$ iff $n \mid km$, i.e. iff $n' \mid k'm$; since $\gcd(n', k') = 1$, this holds iff $n' \mid m$. The least positive such m is $n' = n/d$, so $\text{ord}(g^k) = n/\gcd(n, k)$. Now g^k generates $\langle g \rangle$ iff $\text{ord}(g^k) = n$ iff $\gcd(n, k) = 1$. The number of k with $1 \leq k \leq n$ and $\gcd(n, k) = 1$ is $\varphi(n)$. $\square$

Theorem 3.22 (Classification of subgroups of cyclic groups). *Let $G = \langle g \rangle$ be cyclic.*

1. *Every subgroup of G is cyclic.*
2. *If G is infinite, its subgroups are exactly $\langle g^m \rangle$ for $m \geq 0$; distinct $m \geq 0$ give distinct subgroups, and $\langle g^m \rangle \cong (\mathbb{Z}, +)$ for $m \geq 1$.*
3. *If $|G| = n$, then for each positive divisor $d \mid n$ there is exactly one subgroup of order d , namely $\langle g^{n/d} \rangle$, and these are all the subgroups. Hence the subgroups of G are in bijection with the positive divisors of n .*

Proof. (1) Let $H \leq G$. If $H = \{e\}$ it is cyclic. Otherwise H contains some g^k with $k \neq 0$; since H is closed under inverses it contains g^{-k} too, so H contains a positive power of g . Let m be the least positive integer with $g^m \in H$. We claim that $H = \langle g^m \rangle$. Clearly $\langle g^m \rangle \subseteq H$. Conversely take any $g^t \in H$; by the division algorithm $t = mq + r$ with $0 \leq r < m$, so

$$g^r = g^{t-mq} = g^t(g^m)^{-q} \in H.$$

By minimality of m and $0 \leq r < m$, it follows that $r = 0$, so $g^t = (g^m)^q \in \langle g^m \rangle$. Hence $H = \langle g^m \rangle$ is cyclic.

(2) If G is infinite, then g has infinite order, so the subgroups $\langle g^m \rangle$ ($m \geq 1$) are all infinite cyclic, hence isomorphic to $(\mathbb{Z}, +)$ by Theorem 3.40(1) below (whose proof uses only Propositions 3.10 and 3.19, so no circularity arises), and $\langle g^0 \rangle = \{e\}$. They are distinct because $\langle g^m \rangle$ has g^m as its least positive power but does not contain g^j for $0 < j < m$. Part (1) shows these are all subgroups.

(3) Suppose $|G| = n$, so $\text{ord}(g) = n$. Let $d \mid n$. The element $g^{n/d}$ has order $\text{ord}(g^{n/d}) = n/\gcd(n, n/d) = n/(n/d) = d$ by Corollary 3.21, so $\langle g^{n/d} \rangle$ is a subgroup of order d . For uniqueness, let $H \leq G$ with $|H| = d$. By (1), $H = \langle g^m \rangle$ for the least positive m with $g^m \in H$; and by Proposition 3.19 and Corollary 3.21, $|H| = \text{ord}(g^m) = n/\gcd(n, m) = d$, so $\gcd(n, m) = n/d$. Writing $m = (n/d)m'$, every element of H is a power of $g^{n/d}$ raised to a power, hence $H \subseteq \langle g^{n/d} \rangle$; comparing orders gives $H = \langle g^{n/d} \rangle$. Thus there is exactly one subgroup of each order $d \mid n$, and by (1) every subgroup arises this way. $\square$

Remark 3.23. Specialised to $G = \mathbb{Z}/n\mathbb{Z}$, part (3) states: for each $d \mid n$ there is a unique subgroup of order d , namely $\langle n/d \rangle = \{0, n/d, 2n/d, \dots\}$, which is itself isomorphic to $\mathbb{Z}/d\mathbb{Z}$. For $n = 12$, the subgroup lattice mirrors the divisor lattice $1 \mid 2 \mid 4, 1 \mid 2 \mid 6, 3 \mid 6, \dots$ of 12.

Proposition 3.24. *Every subgroup of $(\mathbb{Z}, +)$ has the form $n\mathbb{Z}$ for a unique integer $n \geq 0$.*

Proof. This is Theorem 3.22(2) read additively; we give the direct argument. Let $H \leq \mathbb{Z}$. If $H = \{0\}$ then $H = 0\mathbb{Z}$. Otherwise H contains a nonzero element and hence (being closed under negation) a positive one; let n be the least positive element of H . Then $n\mathbb{Z} \subseteq H$. For any $h \in H$, division gives $h = nq + r$ with $0 \leq r < n$, and $r = h - nq \in H$; minimality forces $r = 0$, so $h \in n\mathbb{Z}$. Thus $H = n\mathbb{Z}$. Uniqueness is clear since one recovers n as the least positive element of H . $\square$

3.7 Cosets and Lagrange's theorem

Definition 3.25. Let $H \leq G$ and $a \in G$. The *left coset* of H by a is

$$aH := \{ah : h \in H\},$$

and the *right coset* is $Ha := \{ha : h \in H\}$. We denote the set of left cosets G/H , and the number of left cosets, $[G : H] := |G/H|$, is the *index* of H in G .

Cosets are the blocks of a partition, and they all have the same size. This is the mechanism underlying Lagrange's theorem.

Lemma 3.26. *Let $H \leq G$. Then:*

1. $a \in aH$ for every a ; the cosets cover G .
2. Two left cosets are either equal or disjoint: $aH = bH$ if and only if $a^{-1}b \in H$, and otherwise $aH \cap bH = \emptyset$.
3. Every left coset has the same cardinality as H ; the map $h \mapsto ah$ is a bijection $H \rightarrow aH$.

The same statements hold for right cosets, with $ba^{-1} \in H$ in (2).

Proof. (1) $a = ae \in aH$ since $e \in H$. Hence the union of all cosets is G .

(2) Suppose $aH \cap bH \neq \emptyset$, say $ah_1 = bh_2$ with $h_1, h_2 \in H$. Then $a^{-1}b = h_1h_2^{-1} \in H$. Conversely if $a^{-1}b = h \in H$, then $b = ah$, so for any $h' \in H$, $bh' = a(hh') \in aH$, giving $bH \subseteq aH$; by symmetry ($b^{-1}a = h^{-1} \in H$) we obtain $aH \subseteq bH$, so $aH = bH$. Thus two cosets intersect only if they coincide; equivalently, distinct cosets are disjoint.

(3) The map $\lambda_a : H \rightarrow aH$, $h \mapsto ah$, is surjective by definition of aH and injective by left-cancellation ($ah = ah' \Rightarrow h = h'$). Hence $|aH| = |H|$. $\square$

Theorem 3.27 (Lagrange). *Let G be a finite group and $H \leq G$. Then $|H|$ divides $|G|$, and*

$$|G| = [G : H] |H|.$$

Proof. By Lemma 3.26, the distinct left cosets of H partition G into $[G : H]$ blocks, each of cardinality $|H|$. Summing the sizes of the blocks,

$$|G| = \sum_{\text{distinct cosets } aH} |aH| = [G : H] \cdot |H|.$$

In particular $|H|$ divides $|G|$. $\square$

The corollaries are immediate but far-reaching.

Corollary 3.28. *In a finite group G , the order of every element divides $|G|$.*

Proof. For $a \in G$, the cyclic subgroup $\langle a \rangle$ has order $\text{ord}(a)$ by Proposition 3.19. By Lagrange, $\text{ord}(a) = |\langle a \rangle|$ divides $|G|$. $\square$

Corollary 3.29. *If G is finite of order N , then $a^N = e$ for every $a \in G$.*

Proof. Let $d = \text{ord}(a)$. By Corollary 3.28, $d \mid N$, say $N = dm$. Then $a^N = (a^d)^m = e^m = e$. $\square$

Corollary 3.30 (Euler, Fermat). *For $\gcd(a, n) = 1$ one has $a^{\varphi(n)} \equiv 1 \pmod{n}$. In particular, for a*

prime p and any a not divisible by p , $a^{p-1} \equiv 1 \pmod{p}$, and $a^p \equiv a \pmod{p}$ for all a .

Proof. Apply Corollary 3.29 to the group $(\mathbb{Z}/n\mathbb{Z})^\times$ of order $\varphi(n)$: the class $[a]$ satisfies $[a]^{\varphi(n)} = [1]$. For $n = p$ prime, $\varphi(p) = p - 1$, yielding Fermat's little theorem; multiplying $a^{p-1} \equiv 1$ by a gives $a^p \equiv a$, which holds even when $p \mid a$ since both sides are then 0. $\square$

Corollary 3.31. *Every group of prime order p is cyclic, hence isomorphic to $\mathbb{Z}/p\mathbb{Z}$, and has no proper nontrivial subgroups.*

Proof. Let $|G| = p$ and pick $a \neq e$. Then $\langle a \rangle$ has order > 1 dividing p , so $|\langle a \rangle| = p = |G|$, forcing $\langle a \rangle = G$. Thus G is cyclic. Any subgroup has order dividing p , hence order 1 or p . The isomorphism with $\mathbb{Z}/p\mathbb{Z}$ follows from the classification of cyclic groups (Theorem 3.40). $\square$

We close the subsection with the quotient construction in the abelian case, which is all the later development requires: it underlies the quotient rings of section 4 and, through them, the construction of $\mathbb{Z}/n\mathbb{Z}$ and of polynomial quotients.

Proposition 3.32 (Quotient of an abelian group). *Let A be an abelian group, written additively, and let $H \leq A$. On the set A/H of cosets, the operation*

$$(a + H) + (b + H) := (a + b) + H$$

is well defined and makes A/H an abelian group, the quotient group of A by H , with identity $H = 0 + H$ and inverse $-(a + H) = (-a) + H$. If A is finite, $|A/H| = [A : H] = |A|/|H|$.

Proof. Well-definedness is the only point of substance. Suppose $a + H = a' + H$ and $b + H = b' + H$; by Lemma 3.26(2) (written additively) this means $a' - a \in H$ and $b' - b \in H$. Then $(a' + b') - (a + b) = (a' - a) + (b' - b) \in H$, since H is closed under addition, so $(a' + b') + H = (a + b) + H$: the sum of cosets does not depend on the chosen representatives. The group axioms now pass from A to A/H coset-wise: associativity and commutativity read $((a + H) + (b + H)) + (c + H) = (a + b + c) + H$ in either grouping, the coset H is the identity, and $(-a) + H$ inverts $a + H$. The cardinality statement is Lagrange's theorem (Theorem 3.27). $\square$

Remark 3.33. For a non-abelian group G the same construction succeeds exactly for the subgroups $N \leq G$ whose left and right cosets coincide ($gN = Ng$ for all g), the *normal* subgroups; commutativity makes every subgroup of an abelian group normal, which is why no extra hypothesis appeared above. We need only the abelian case.

3.8 Homomorphisms, kernels, and images

We compare groups by means of maps that respect their operations.

Definition 3.34. Let $(G, *)$ and $(G', \circ)$ be groups. A *group homomorphism* is a function $\varphi : G \rightarrow G'$ such that

$$\varphi(a * b) = \varphi(a) \circ \varphi(b) \quad \text{for all } a, b \in G.$$

We denote the set of all homomorphisms $G \rightarrow G'$ by $\text{Hom}(G, G')$.

Proposition 3.35. *Let $\varphi : G \rightarrow G'$ be a homomorphism, with identities $e \in G$, $e' \in G'$. Then*

1. $\varphi(e) = e'$;

2. $\varphi(a^{-1}) = \varphi(a)^{-1}$ for all a ;
3. $\varphi(a^n) = \varphi(a)^n$ for all $n \in \mathbb{Z}$;
4. the image $\varphi(G)$ is a subgroup of G' .

Proof. (1) From $\varphi(e) = \varphi(e * e) = \varphi(e) \circ \varphi(e)$; cancelling $\varphi(e)$ in G' gives $e' = \varphi(e)$.

(2) $\varphi(a) \circ \varphi(a^{-1}) = \varphi(a * a^{-1}) = \varphi(e) = e'$, and likewise on the other side, so $\varphi(a^{-1})$ is the inverse of $\varphi(a)$.

(3) For $n \geq 0$ by induction using the homomorphism property; for $n < 0$ combine with (2).

(4) Nonempty since $e' = \varphi(e) \in \varphi(G)$. If $x = \varphi(a), y = \varphi(b)$ lie in $\varphi(G)$, then $xy^{-1} = \varphi(a)\varphi(b)^{-1} = \varphi(ab^{-1}) \in \varphi(G)$. By the subgroup test, $\varphi(G) \leq G'$. $\square$

Definition 3.36. The *kernel* of a homomorphism $\varphi : G \rightarrow G'$ is

$$\ker \varphi := \{a \in G : \varphi(a) = e'\} = \varphi^{-1}(\{e'\}),$$

and its *image* is $\text{im } \varphi := \varphi(G) = \{\varphi(a) : a \in G\}$.

Proposition 3.37. Let $\varphi : G \rightarrow G'$ be a homomorphism. Then $\ker \varphi \leq G$, and φ is injective if and only if $\ker \varphi = \{e\}$.

Proof. $\ker \varphi$ is nonempty ($e \in \ker \varphi$ by Proposition 3.35(1)). If $a, b \in \ker \varphi$, then $\varphi(ab^{-1}) = \varphi(a)\varphi(b)^{-1} = e'(e')^{-1} = e'$, so $ab^{-1} \in \ker \varphi$; thus $\ker \varphi \leq G$.

If φ is injective, then $\varphi(a) = e' = \varphi(e)$ forces $a = e$, so $\ker \varphi = \{e\}$. Conversely, suppose $\ker \varphi = \{e\}$ and $\varphi(a) = \varphi(b)$. Then $\varphi(ab^{-1}) = \varphi(a)\varphi(b)^{-1} = e'$, so $ab^{-1} \in \ker \varphi = \{e\}$, giving $ab^{-1} = e$, i.e. $a = b$. Hence φ is injective. $\square$

Kernels are exactly the subgroups by which one may quotient: for an abelian G , Proposition 3.32 forms $G/\ker \varphi$, and in general the kernel satisfies the normality condition of Remark 3.33.

3.9 Isomorphisms

Definition 3.38. A homomorphism $\varphi : G \rightarrow G'$ that is bijective is an *isomorphism*; we then write $G \cong G'$ and call G, G' *isomorphic*. An isomorphism from G to itself is an *automorphism*. A homomorphism is a *monomorphism* if injective and an *epimorphism* if surjective.

Proposition 3.39. If $\varphi : G \rightarrow G'$ is an isomorphism, so is its set-theoretic inverse $\varphi^{-1} : G' \rightarrow G$. The relation $\cong$ is an equivalence relation on any collection of groups.

Proof. Let $x, y \in G'$, say $x = \varphi(a), y = \varphi(b)$. Then $\varphi^{-1}(xy) = \varphi^{-1}(\varphi(a)\varphi(b)) = \varphi^{-1}(\varphi(ab)) = ab = \varphi^{-1}(x)\varphi^{-1}(y)$, so φ^{-1} is a homomorphism, and it is bijective. Reflexivity of $\cong$ uses the identity automorphism, symmetry uses φ^{-1} , and transitivity uses that a composite of isomorphisms is an isomorphism (the composite of homomorphisms is a homomorphism, and the composite of bijections is a bijection). $\square$

Isomorphic groups are “the same” for all group-theoretic purposes: every property expressible in terms of the operation transfers across an isomorphism. For this reason we speak of classifying groups *up to isomorphism*.

Theorem 3.40 (Classification of cyclic groups). *Every cyclic group is isomorphic to exactly one of the following:*

1. *the infinite cyclic group $(\mathbb{Z}, +)$, if it is infinite;*
2. *the finite cyclic group $(\mathbb{Z}/n\mathbb{Z}, +)$, if it has order n .*

In particular two cyclic groups are isomorphic if and only if they have the same order.

Proof. Let $G = \langle g \rangle$. Define $\varphi : \mathbb{Z} \rightarrow G$ by $\varphi(k) = g^k$. This is a homomorphism from $(\mathbb{Z}, +)$: $\varphi(k + \ell) = g^{k+\ell} = g^k g^\ell = \varphi(k)\varphi(\ell)$. It is surjective since G consists exactly of the powers of g .

If g has infinite order, φ is injective by Proposition 3.19(1) (distinct exponents give distinct powers), so φ is an isomorphism $\mathbb{Z} \cong G$.

If g has order n , then $\ker \varphi = \{k : g^k = e\} = n\mathbb{Z}$ by Proposition 3.10. The induced map $\bar{\varphi} : \mathbb{Z}/n\mathbb{Z} \rightarrow G$, $[k] \mapsto g^k$, is well defined (if $[k] = [\ell]$ then $n \mid k - \ell$, so $g^k = g^\ell$), a homomorphism, surjective, and injective (if $g^k = e$ then $n \mid k$, so $[k] = [0]$). Hence $\mathbb{Z}/n\mathbb{Z} \cong G$. Finally, $\mathbb{Z}$ and $\mathbb{Z}/n\mathbb{Z}$ are pairwise non-isomorphic for distinct n since isomorphic groups have equal order. $\square$

4 Rings and fields

The preceding development treated groups: sets equipped with a single associative binary operation possessing an identity and inverses. Most of the structures of genuine interest in cryptography, however, support *two* interacting operations, an addition and a multiplication, that together obey the familiar arithmetic laws of $\mathbb{Z}$. The integers, the rationals, the residues modulo n , polynomials, and matrices are all of this kind. The abstract structure that captures these examples is the *ring*, and the most important special case — the one in which division by every nonzero element is possible — is the *field*. Fields are the natural arena for linear algebra and for the theory of polynomials, and the finite fields $\mathbb{F}_p$ and $\mathbb{F}_q$ are the computational substrate of essentially all of the elliptic-curve and proof-system machinery developed later in this monograph. This section builds the theory from the ground up.

Throughout, we assume only the theory of abelian groups developed earlier. We write the additive structure of a ring additively, with identity element 0, and the multiplicative structure multiplicatively, with (when it exists) identity element 1.

4.1 Rings and commutative rings

Definition 4.1. A *ring* is a set R equipped with two binary operations, *addition* $+: R \times R \rightarrow R$ and *multiplication* $\cdot : R \times R \rightarrow R$, satisfying the following axioms.

1. **Additive abelian group.** $(R, +)$ is an abelian group; that is, addition is associative and commutative, there is an element $0 \in R$ with $a + 0 = a$ for all a , and every $a \in R$ has an additive inverse $-a$ with $a + (-a) = 0$.
2. **Multiplicative associativity.** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in R$.
3. **Multiplicative identity.** There is an element $1 \in R$ with $1 \cdot a = a \cdot 1 = a$ for all $a \in R$.
4. **Distributivity.** For all $a, b, c \in R$,

$$a \cdot (b + c) = a \cdot b + a \cdot c \quad \text{and} \quad (a + b) \cdot c = a \cdot c + b \cdot c.$$

A ring R is *commutative* if in addition $a \cdot b = b \cdot a$ for all $a, b \in R$.

We customarily write ab for $a \cdot b$, adopting the convention that multiplication binds more tightly than addition, so that $ab + ac$ means $(ab) + (ac)$. The element 0 is the *additive identity* or *zero*, and 1 is the *multiplicative identity* or *unity*.

Remark 4.2. Conventions in the literature vary. Some authors do not require a multiplicative identity 1, reserving the term *ring with unity* for the structure of Definition 4.1 and using *rng* (“ring without the *i* of identity”) for the weaker notion. In this monograph *every ring has a 1*, every ring homomorphism preserves it, and “ring” will frequently mean “commutative ring” in the examples that matter to us; we always state commutativity explicitly when a result depends on it.

Before we produce examples, we record the elementary arithmetic consequences of the axioms. These are the rules we use constantly and without comment; it is worth establishing once that they all follow from distributivity.

Proposition 4.3. *Let R be a ring and let $a, b \in R$. Then:*

1. $0 \cdot a = a \cdot 0 = 0$;
2. $(-a)b = a(-b) = -(ab)$;
3. $(-a)(-b) = ab$;
4. *more generally, for integers m, n (interpreting na as the n -fold sum in the additive group), $(ma)(nb) = (mn)(ab)$.*

Proof. (1) Since $0 + 0 = 0$, distributivity gives $0 \cdot a = (0 + 0) \cdot a = 0 \cdot a + 0 \cdot a$. Adding $-(0 \cdot a)$ to both sides yields $0 = 0 \cdot a$. The argument for $a \cdot 0$ is symmetric.

(2) One computes $ab + (-a)b = (a + (-a))b = 0 \cdot b = 0$ by distributivity and part (1). Hence $(-a)b$ is the additive inverse of ab , i.e. $(-a)b = -(ab)$. The identity $a(-b) = -(ab)$ is symmetric.

(3) Applying (2) twice, $(-a)(-b) = -(a(-b)) = -(-(ab)) = ab$, where the last step uses that negation is an involution in the abelian group $(R, +)$.

(4) This follows from (2) and repeated distributivity (a routine induction on $|m|$ and $|n|$); we omit the bookkeeping. $\square$

Remark 4.4. If $1 = 0$ in a ring R , then for every $a \in R$ one has $a = 1 \cdot a = 0 \cdot a = 0$ by Proposition 4.3(1), so $R = \{0\}$. This single-element structure is the *zero ring* (or *trivial ring*); it is a perfectly legitimate ring. Whenever we wish to exclude it we impose the assumption $1 \neq 0$, and indeed the definition of a field below builds in the standing assumption $1 \neq 0$.

Example 4.5. We list the basic examples; the reader should verify the axioms in each case.

1. **The integers** $\mathbb{Z}$ with ordinary addition and multiplication form a commutative ring. This is the prototype, and much of ring theory is an attempt to isolate which features of $\mathbb{Z}$ survive in greater generality.
2. **The rationals** $\mathbb{Q}$, **the reals** $\mathbb{R}$, **and the complexes** $\mathbb{C}$ are commutative rings (indeed fields, see §4.4).
3. **The integers modulo n** , $\mathbb{Z}/n\mathbb{Z}$. For an integer $n \geq 1$, the set of residue classes $\{\overline{0}, \overline{1}, \dots, \overline{n-1}\}$ with addition and multiplication induced from $\mathbb{Z}$ is a commutative ring, the additive group being the cyclic group of order n . One writes $\mathbb{Z}/n\mathbb{Z}$ (or $\mathbb{Z}_n$ when no confusion with n -adic integers can arise). The case $n = 1$ gives the zero ring.
4. **Polynomial rings.** If R is a commutative ring, the set $R[x]$ of polynomials $\sum_{i=0}^d a_i x^i$ with coefficients $a_i \in R$, under the usual addition and multiplication, is a commutative ring. The next section treats polynomials over a field systematically; here we note only that $R[x]$ is again a commutative ring with the same 0 and 1 as R (viewed as constant polynomials).

5. **Matrix rings.** For $n \geq 1$ the set $M_n(R)$ of $n \times n$ matrices over a ring R , with matrix addition and matrix multiplication, is a ring. For $n \geq 2$ it is *not* commutative even when R is: for instance $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ while $\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$. This is the first important example of a noncommutative ring.
6. **Product rings.** If R and S are rings, the Cartesian product $R \times S$ with componentwise operations $(r, s) + (r', s') = (r + r', s + s')$ and $(r, s)(r', s') = (rr', ss')$ is a ring, with zero $(0, 0)$ and identity $(1, 1)$. More generally any finite product $R_1 \times \cdots \times R_k$ is a ring.
7. **The zero ring** $\{0\}$ of Remark 4.4.

4.2 Units, zero divisors, and integral domains

The integers $\mathbb{Z}$ enjoy two properties beyond the bare ring axioms that will guide the rest of this section: a product of nonzero integers is nonzero (*cancellation*), and the only integers by which one may divide within $\mathbb{Z}$ are ± 1 . We now abstract both.

Definition 4.6. Let R be a ring. An element $u \in R$ is a *unit* (or is *invertible*) if there exists $v \in R$ with $uv = vu = 1$. The element v is then unique (if $uv = vu = 1$ and $uv' = v'u = 1$ then $v = v(uv') = (vu)v' = v'$), is called the *inverse* of u , and is written u^{-1} . We denote the set of units of R by $R^\times$.

Proposition 4.7. For any ring R , the set $R^\times$ of units is a group under multiplication, the group of units (or multiplicative group) of R .

Proof. The identity 1 is a unit (it is its own inverse), so $1 \in R^\times$. If $u, w \in R^\times$ with inverses u^{-1}, w^{-1} , then $(uw)(w^{-1}u^{-1}) = u(ww^{-1})u^{-1} = uu^{-1} = 1$ and symmetrically $(w^{-1}u^{-1})(uw) = 1$, so $uw \in R^\times$ with $(uw)^{-1} = w^{-1}u^{-1}$; thus $R^\times$ is closed under multiplication. Associativity is inherited from R . Finally every $u \in R^\times$ has the inverse $u^{-1} \in R^\times$ (whose inverse is u). Hence $R^\times$ is a group. $\square$

Example 4.8. 1. $\mathbb{Z}^\times = \{1, -1\}$.

2. $\mathbb{Q}^\times = \mathbb{Q} \setminus \{0\}$, and likewise $\mathbb{R}^\times = \mathbb{R} \setminus \{0\}$, $\mathbb{C}^\times = \mathbb{C} \setminus \{0\}$. In a field, every nonzero element is a unit; this is the defining feature, taken up in §4.4.
3. In $\mathbb{Z}/n\mathbb{Z}$, the class $\bar{a}$ is a unit if and only if $\gcd(a, n) = 1$ (Theorem 2.27); thus $(\mathbb{Z}/n\mathbb{Z})^\times$ consists of the residues coprime to n , and its order is $\varphi(n)$, Euler's totient.
4. $M_n(R)^\times$ is the group of invertible matrices. When R is a field this is the *general linear group* GL_n , consisting of the matrices of nonzero determinant.

We now isolate the failure of cancellation.

Definition 4.9. Let R be a ring. A nonzero element $a \in R$ is a *left zero divisor* if there exists a nonzero $b \in R$ with $ab = 0$; *right zero divisor* is defined symmetrically with $ba = 0$. In a commutative ring the two notions coincide and one simply says *zero divisor*. An element that is neither a left nor a right zero divisor (and is nonzero) is called *regular* or *cancellable*.

The following observation justifies the terminology, making precise the sense in which zero divisors obstruct “dividing out”.

Proposition 4.10. *Let R be a ring and $a \in R$ a nonzero element. Then a is not a left zero divisor if and only if it is left-cancellable: $ab = ac$ implies $b = c$ for all $b, c \in R$.*

Proof. Suppose a is not a left zero divisor and $ab = ac$. Then $a(b - c) = ab - ac = 0$ by distributivity and Proposition 4.3(2). Since a is not a left zero divisor and $a \neq 0$, the factor $b - c$ must be 0, i.e. $b = c$. Conversely, if a is a left zero divisor, say $ab = 0$ with $b \neq 0$, then $ab = a \cdot 0$ but $b \neq 0$, so cancellation fails. $\square$

Remark 4.11. A unit is never a zero divisor: if u is a unit and $ub = 0$, then $b = u^{-1}(ub) = u^{-1} \cdot 0 = 0$. The converse fails in general — in $\mathbb{Z}$ every nonzero element is a non-zero-divisor, but only ± 1 are units — though it does hold in finite commutative rings, by the injectivity-implies-surjectivity argument that proves Theorem 4.23.

Example 4.12. 1. In $\mathbb{Z}/6\mathbb{Z}$, $\bar{2} \cdot \bar{3} = \bar{6} = \bar{0}$, so $\bar{2}$ and $\bar{3}$ are zero divisors. More generally $\bar{a} \neq \bar{0}$ is a zero divisor in $\mathbb{Z}/n\mathbb{Z}$ precisely when $\gcd(a, n) > 1$; thus in $\mathbb{Z}/n\mathbb{Z}$ every nonzero element is either a unit or a zero divisor (Theorem 4.23 revisits this dichotomy).

2. In the product ring $R \times S$ with both factors nonzero, $(1, 0)(0, 1) = (0, 0)$, so product rings always have zero divisors.

3. In $M_2(\mathbb{R})$, $\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$, so singular nonzero matrices are zero divisors.

The rings with no zero divisors are precisely those in which the cancellation law of $\mathbb{Z}$ holds; in the commutative case they receive a name.

Definition 4.13. An *integral domain* (or simply a *domain*) is a commutative ring R with $1 \neq 0$ and no zero divisors: for all $a, b \in R$, $ab = 0$ implies $a = 0$ or $b = 0$.

Equivalently, by Proposition 4.10, an integral domain is a nonzero commutative ring in which every nonzero element is cancellable.

Example 4.14. $\mathbb{Z}$ is an integral domain — this is the example that names the concept. Every field is an integral domain (Proposition 4.22). The ring $\mathbb{Z}/n\mathbb{Z}$ is an integral domain if and only if n is prime, by Example 4.12(1) together with the fact that for $n = p$ prime, $ab \equiv 0 \pmod{p}$ forces $p \mid a$ or $p \mid b$ (Euclid's lemma). If R is an integral domain then so is the polynomial ring $R[x]$: the leading coefficient of a product of nonzero polynomials is the product of the leading coefficients, hence nonzero. By contrast $\mathbb{Z}/6\mathbb{Z}$, product rings, and $M_n(R)$ for $n \geq 2$ are not domains.

4.3 Ideals and quotient rings

The formation of quotients of rings — the ring-theoretic analogue of the quotient of an abelian group (Proposition 3.32) — requires the appropriate notion of substructure by which to quotient. For rings this is the *ideal*. We treat this material briefly, as we need it chiefly to construct $\mathbb{Z}/n\mathbb{Z}$ conceptually and to indicate how the finite fields $\mathbb{F}_q$ would arise; a fuller treatment belongs to a course in commutative algebra.

Definition 4.15. Let R be a commutative ring. A subset $I \subseteq R$ is an *ideal* if

1. I is an additive subgroup of $(R, +)$: $0 \in I$, and $a - b \in I$ whenever $a, b \in I$; and
2. I is closed under multiplication by arbitrary ring elements: $ra \in I$ whenever $r \in R$ and $a \in I$ (the *absorption* property).

In a noncommutative ring one distinguishes left, right, and two-sided ideals; for the present purposes commutative rings suffice, where all three coincide. Note that an ideal is almost never a subring: if an ideal I contains 1, then by absorption $r = r \cdot 1 \in I$ for every $r \in R$, so $I = R$. Thus the only ideal containing a unit is R itself.

Example 4.16. 1. In any commutative ring R , the subsets $\{0\}$ and R are ideals, the *trivial* and *improper* ideals respectively.

2. For a fixed $a \in R$, the set $(a) := aR = \{ar : r \in R\}$ is an ideal, the *principal ideal generated by a* . In $\mathbb{Z}$ the principal ideal $(n) = n\mathbb{Z}$ consists of all multiples of n . In fact every ideal of $\mathbb{Z}$ is principal: an ideal $I \neq \{0\}$ contains a least positive element n , and the division algorithm shows $I = n\mathbb{Z}$ (any $a \in I$ gives $a = qn + r$ with $0 \leq r < n$, whence $r = a - qn \in I$ forces $r = 0$). A domain in which every ideal is principal is a *principal ideal domain*.

3. More generally, for $a_1, \dots, a_k \in R$, the set $(a_1, \dots, a_k) = \{\sum_i r_i a_i : r_i \in R\}$ is the ideal *generated by the a_i* .

The significance of ideals is that they are exactly the subsets by which one may quotient.

Definition 4.17. Let I be an ideal of a commutative ring R . Since $(I, +)$ is a subgroup of the abelian group $(R, +)$, we may form the quotient group R/I of additive cosets $a + I$ (Proposition 3.32). We define multiplication of cosets by

$$(a + I)(b + I) := ab + I.$$

Proposition 4.18. With the operations of Definition 4.17, R/I is a commutative ring, the quotient ring of R by I , with zero $0 + I = I$ and identity $1 + I$. In particular the multiplication is well defined.

Proof. We must check that the product of cosets does not depend on the chosen representatives. Suppose $a + I = a' + I$ and $b + I = b' + I$, so that $a' = a + s$ and $b' = b + t$ with $s, t \in I$. Then

$$a'b' = (a + s)(b + t) = ab + at + sb + st.$$

The three terms at , sb , and st all lie in I : $at \in I$ and $st \in I$ by absorption ($t \in I$), and $sb \in I$ by absorption ($s \in I$). Hence $a'b' - ab \in I$, i.e. $a'b' + I = ab + I$, and multiplication is well defined. The ring axioms for R/I are then inherited coset-wise from those of R : for instance distributivity reads $(a + I)((b + I) + (c + I)) = (a(b + c)) + I = (ab + ac) + I = (a + I)(b + I) + (a + I)(c + I)$. Commutativity, associativity, and the identity laws follow identically. $\square$

Example 4.19. Taking $R = \mathbb{Z}$ and $I = n\mathbb{Z}$, we recover the quotient ring $\mathbb{Z}/n\mathbb{Z}$ of Example 4.5(3) as a *quotient ring*: its elements are the cosets $a + n\mathbb{Z}$, i.e. the residue classes modulo n , and the coset operations are exactly residue arithmetic. This is the conceptual origin of modular arithmetic, and it generalises: quotients of polynomial rings $\mathbb{F}_p[x]/(f)$ by an irreducible polynomial f furnish the finite fields $\mathbb{F}_q$ with $q = p^k$. That construction lies off our path — the curves of Halo 2 and Orchard work over prime fields — so we record its existence and do not develop it.

4.4 Fields

We now introduce the central object of this section.

Definition 4.20. A *field* is a commutative ring F in which $1 \neq 0$ and every nonzero element is a unit. Equivalently, F is a set with two operations $+$ and $\cdot$ such that $(F, +)$ is an abelian group with identity 0, $(F \setminus \{0\}, \cdot)$ is an abelian group with identity 1, and multiplication distributes over

addition.

Unwinding the definition: in a field one may add, subtract, and multiply as in any commutative ring, and additionally *divide by any nonzero element*, with $a/b := ab^{-1}$. The group $(F \setminus \{0\}, \cdot) = F^\times$ is the *multiplicative group* of the field.

Example 4.21. 1. $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$ are fields; $\mathbb{Z}$ is not, since 2 has no inverse in $\mathbb{Z}$.

2. For p prime, $\mathbb{F}_p := \mathbb{Z}/p\mathbb{Z}$ is a field, the *prime field* of characteristic p (Corollary 2.28; Theorem 4.23 below recovers this abstractly). These fields are fundamental to the cryptography developed later; their finite extensions $\mathbb{F}_q$ with $q = p^k$ exist for every prime power but lie off our path (Example 4.19).
3. The field $\mathbb{Q}(i) = \{a + bi : a, b \in \mathbb{Q}\} \subseteq \mathbb{C}$ and, more generally, number fields, are fields obtained by adjoining algebraic elements to $\mathbb{Q}$.
4. The field of rational functions $F(x) = \{f/g : f, g \in F[x], g \neq 0\}$ over a field F is a field; it is the field of fractions of $F[x]$.

Proposition 4.22. *Every field is an integral domain.*

Proof. Let F be a field; by definition $1 \neq 0$ and F is commutative. Suppose $ab = 0$ with $a \neq 0$. Since a is a unit, multiply by a^{-1} on the left: $b = 1 \cdot b = (a^{-1}a)b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$. Hence $a = 0$ or $b = 0$, so F has no zero divisors. $\square$

The converse is false ($\mathbb{Z}$ is a domain but not a field), but it becomes true under finiteness — a fact of central importance for finite fields.

Theorem 4.23. *Every finite integral domain is a field.*

Proof. Let R be a finite integral domain and let $a \in R$ with $a \neq 0$; we must produce a multiplicative inverse for a . Consider the “multiplication by a ” map

$$\mu_a : R \rightarrow R, \quad \mu_a(x) = ax.$$

This map is injective: if $ax = ay$ then $a(x - y) = 0$, and since $a \neq 0$ and R is a domain, $x - y = 0$, i.e. $x = y$ (this is cancellation, Proposition 4.10). An injective map from a finite set to itself is necessarily surjective. Hence μ_a is onto, so there exists $b \in R$ with $ab = \mu_a(b) = 1$. Because R is commutative, $ba = 1$ as well, so $b = a^{-1}$. As a was an arbitrary nonzero element, every nonzero element is a unit, and R is a field. $\square$

Applied to $\mathbb{Z}/n\mathbb{Z}$, the theorem recovers Corollary 2.28: for prime p the finite domain $\mathbb{Z}/p\mathbb{Z}$ (Example 4.14) is a field, and there the extended Euclidean algorithm computed each inverse explicitly.

4.5 The characteristic of a ring

There is a canonical way in which a copy of $\mathbb{Z}$ maps into any ring, and the size of its image is a fundamental invariant.

Definition 4.24. Let R be a ring. For an integer n and the identity $1 \in R$, write $n \cdot 1$ for the n -fold sum $1 + 1 + \cdots + 1$ (n terms) if $n > 0$, the element 0 if $n = 0$, and $-((-n) \cdot 1)$ if $n < 0$. The *characteristic* of R , written $\text{char}(R)$, is the least positive integer n such that $n \cdot 1 = 0$, if such an n exists; otherwise $\text{char}(R) = 0$.

Equivalently, the map $\iota: \mathbb{Z} \rightarrow R$ given by $\iota(n) = n \cdot 1$ is a ring homomorphism, and $\text{char}(R)$ is the nonnegative generator of its kernel, an ideal of $\mathbb{Z}$. Thus $\text{char}(R) = 0$ when ι is injective (a copy of $\mathbb{Z}$ sits inside R), and $\text{char}(R) = n$ when the kernel is $n\mathbb{Z}$.

Example 4.25. $\text{char}(\mathbb{Z}) = \text{char}(\mathbb{Q}) = \text{char}(\mathbb{R}) = \text{char}(\mathbb{C}) = 0$. For the residue rings, $\text{char}(\mathbb{Z}/n\mathbb{Z}) = n$, since $n \cdot \bar{1} = \bar{n} = \bar{0}$ and no smaller positive multiple of $\bar{1}$ vanishes. In particular $\text{char}(\mathbb{F}_p) = p$.

Theorem 4.26. *The characteristic of an integral domain (in particular of a field) is either 0 or a prime number.*

Proof. Let R be an integral domain and suppose $\text{char}(R) = n > 0$. First, $n \neq 1$: if $n = 1$ then $1 = 1 \cdot 1 = 0$, forcing R to be the zero ring, contrary to $1 \neq 0$ in a domain. Suppose, for contradiction, that n is composite, $n = ab$ with $1 < a, b < n$. Then in R ,

$$0 = n \cdot 1 = (ab) \cdot 1 = (a \cdot 1)(b \cdot 1),$$

using the distributive bookkeeping of Proposition 4.3(4). Since R is a domain, either $a \cdot 1 = 0$ or $b \cdot 1 = 0$. But $0 < a < n$ and $0 < b < n$, contradicting the minimality of $n = \text{char}(R)$. Hence n has no nontrivial factorisation, i.e. n is prime. $\square$

Remark 4.27. A field F of characteristic 0 contains a copy of $\mathbb{Z}$ (via ι) and hence, dividing, a copy of $\mathbb{Q}$; a field of characteristic p contains a copy of $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$. In either case this smallest subfield is the *prime subfield* of F . Every field is therefore an extension either of $\mathbb{Q}$ or of some $\mathbb{F}_p$. This dichotomy — characteristic zero versus characteristic p — pervades the subject; the fields $\mathbb{F}_q$ underlying the cryptography treated here all have prime characteristic p , where the “freshman’s dream” identity $(a + b)^p = a^p + b^p$ holds and gives rise to the Frobenius endomorphism, developed in the finite-fields section (theorem 6.4).

5 Polynomials over a field

Throughout this section, $\mathbb{F}$ denotes a field. Polynomials over a field are among the central objects of the entire development: they are the engine behind finite fields, error-correcting codes, and — most importantly for the present purposes — the polynomial commitment schemes and *arithmetization* (the encoding of a computation as identities between polynomials) that underlie Halo 2 and Zcash Orchard. The single most exploited fact in that entire edifice is elementary: *a nonzero polynomial of degree d over a field has at most d roots*. From it we extract Lagrange interpolation and the Schwartz–Zippel principle, the probabilistic heart of every interactive proof we eventually construct. This section develops all of this from the definition of a polynomial, assuming only the theory of fields of the preceding section.

5.1 The polynomial ring $\mathbb{F}[X]$

We begin with the formal definition. The crucial pedagogical point — to which the development returns at length in Subsection 5.9 — is that a polynomial is *not* a function. It is a formal expression, a finite list of coefficients, that one may *evaluate* to obtain a function, but which carries more information than that function in general.

Definition 5.1 (Polynomial). A *polynomial* over $\mathbb{F}$ in the indeterminate X is a formal expression

$$f = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n = \sum_{i=0}^n a_iX^i,$$

where $n \in \mathbb{N}$ (taking $\mathbb{N} = \{0, 1, 2, \dots\}$) and the *coefficients* $a_0, a_1, \dots, a_n$ lie in $\mathbb{F}$. Formally, we may identify f with the sequence $(a_0, a_1, a_2, \dots)$ of its coefficients, in which only finitely many entries are nonzero. Two polynomials are *equal* precisely when their coefficient sequences agree term by term, i.e. $\sum_i a_i X^i = \sum_i b_i X^i$ if and only if $a_i = b_i$ for every $i \in \mathbb{N}$. We call the symbol X an *indeterminate* or *formal variable*; it is not an element of $\mathbb{F}$ and stands for nothing in particular — it is merely a placeholder that organizes the coefficients. We denote the set of all polynomials over $\mathbb{F}$ in X by $\mathbb{F}[X]$.

Remark 5.2. The sequence model is the cleanest way to render Definition 5.1 fully rigorous: $\mathbb{F}[X]$ is the set of all functions $a : \mathbb{N} \rightarrow \mathbb{F}$ that are *eventually zero* (there is some N with $a(i) = 0$ for all $i > N$). The expression $\sum_i a_i X^i$ is then merely suggestive notation for the sequence $(a_0, a_1, \dots)$. We use the two points of view interchangeably. The notation is convenient because, as we show below, the arithmetic of these sequences is exactly the arithmetic the notation suggests.

We now equip the ring $\mathbb{F}[X]$ with addition and multiplication. Let $f = \sum_i a_i X^i$ and $g = \sum_i b_i X^i$, where we pad the shorter coefficient sequence with zeros so that both run over the same index set.

Definition 5.3 (Ring operations on $\mathbb{F}[X]$). The *sum* and *product* of $f = \sum_i a_i X^i$ and $g = \sum_i b_i X^i$ are

$$f + g = \sum_{i=0}^{\infty} (a_i + b_i) X^i, \quad f \cdot g = \sum_{k=0}^{\infty} c_k X^k, \quad \text{where} \quad c_k = \sum_{i+j=k} a_i b_j = \sum_{i=0}^k a_i b_{k-i}.$$

The coefficient c_k is the *convolution* of the two coefficient sequences. Both sums are finite: only finitely many a_i and b_j are nonzero, so only finitely many c_k are nonzero, and the result is again a polynomial.

The product formula is exactly what we obtain by multiplying out $(\sum_i a_i X^i)(\sum_j b_j X^j)$ and collecting like powers of X , using the rule $X^i \cdot X^j = X^{i+j}$. Stating it as a convolution makes no appeal to “substituting a value for X ”; it is a purely formal manipulation of coefficient sequences.

Theorem 5.4 ($\mathbb{F}[X]$ is a commutative ring). *With the operations of Definition 5.3, the set $\mathbb{F}[X]$ is a commutative ring with the following distinguished elements:*

- the zero polynomial 0, the sequence $(0, 0, 0, \dots)$, which is the additive identity;
- the constant polynomial 1, the sequence $(1, 0, 0, \dots)$, which is the multiplicative identity.

Moreover, the map $c \mapsto (c, 0, 0, \dots)$ embeds $\mathbb{F}$ into $\mathbb{F}[X]$ as a subring (the constant polynomials), so that $\mathbb{F} \subseteq \mathbb{F}[X]$.

Proof. We verify the ring axioms. *Additive structure.* Addition is defined coordinatewise, so it inherits commutativity, associativity, the identity $(0, 0, \dots)$, and inverses $(-a_0, -a_1, \dots)$ directly from the additive group of $\mathbb{F}$. Thus $(\mathbb{F}[X], +)$ is an abelian group.

Commutativity of multiplication. With $f \cdot g = \sum_k (\sum_{i+j=k} a_i b_j) X^k$, swapping the roles of f and g replaces $a_i b_j$ by $b_i a_j$; reindexing the inner sum $(i, j) \mapsto (j, i)$ and using commutativity of $\mathbb{F}$ shows $\sum_{i+j=k} a_i b_j = \sum_{i+j=k} b_i a_j$. Hence $f \cdot g = g \cdot f$.

Associativity of multiplication. For f, g, h with coefficients a_i, b_j, c_ℓ , the coefficient of X^m in $(fg)h$ is

$$\sum_{k+\ell=m} \left(\sum_{i+j=k} a_i b_j \right) c_\ell = \sum_{i+j+\ell=m} a_i b_j c_\ell,$$

and the same triple sum arises symmetrically as the coefficient of X^m in $f(gh)$. The two agree, so $(fg)h = f(gh)$.

Distributivity. The coefficient of X^k in $f(g+h)$ is $\sum_{i+j=k} a_i(b_j + c_j) = \sum_{i+j=k} a_i b_j + \sum_{i+j=k} a_i c_j$, which is the coefficient of X^k in $fg + fh$; this uses distributivity in $\mathbb{F}$. Hence $f(g+h) = fg + fh$, and the other distributive law follows by commutativity.

Identities. The sequence $(0, 0, \dots)$ is plainly additively neutral. For $1 = (1, 0, 0, \dots)$, the convolution gives $(1 \cdot f)_k = \sum_{i+j=k} 1_{[i=0]} a_j = a_k$, so $1 \cdot f = f$.

The embedding. The map $\iota : c \mapsto (c, 0, 0, \dots)$ sends sums to sums and, since $(c, 0, \dots) \cdot (d, 0, \dots)$ has k -th coefficient $\sum_{i+j=k} \iota(c)_i \iota(d)_j = cd 1_{[k=0]}$, products to products; it is injective because distinct constants give distinct sequences. Thus ι is an injective ring homomorphism, and $\mathbb{F}[X]$ is a commutative ring containing a copy of $\mathbb{F}$. $\square$

Remark 5.5. Multiplication of a polynomial $f = \sum_i a_i X^i$ by a constant $c \in \mathbb{F}$ (viewed as the constant polynomial) is the *scalar multiple* $cf = \sum_i (c a_i) X^i$. Together with addition this makes $\mathbb{F}[X]$ a vector space over $\mathbb{F}$ — indeed an infinite-dimensional one, with basis $\{1, X, X^2, X^3, \dots\}$. We rely on this linear structure constantly, for example when counting polynomials of bounded degree.

5.2 Degree and leading coefficients

Definition 5.6 (Degree, leading coefficient, monic). Let $f = \sum_i a_i X^i \in \mathbb{F}[X]$ be nonzero. Its *degree* $\deg f$ is the largest index n for which $a_n \neq 0$. The corresponding coefficient a_n is the *leading coefficient* of f , written $\text{lc}(f)$, and $a_n X^n$ is the *leading term*. The coefficient a_0 is the *constant term*. A nonzero polynomial is *monic* if its leading coefficient is 1. The *constant polynomials* are the elements of $\mathbb{F}$ (embedded as in Theorem 5.4); the nonzero ones have degree 0, while the zero polynomial receives degree $-\infty$ by the following convention. We set $\deg 0 = -\infty$, with the arithmetic conventions $-\infty < n$, $-\infty + n = -\infty$, and $-\infty + (-\infty) = -\infty$ for every $n \in \mathbb{Z}$. We call polynomials of degree 1, 2, 3 *linear*, *quadratic*, *cubic* respectively.

The $-\infty$ convention for the zero polynomial is not pedantry: it makes the degree formulas below hold without exceptions, which streamlines every later induction on degree.

Proposition 5.7 (Degree of sums and products). For all $f, g \in \mathbb{F}[X]$,

$$\deg(f + g) \leq \max\{\deg f, \deg g\}, \quad (1)$$

$$\deg(f \cdot g) = \deg f + \deg g. \quad (2)$$

Equality holds in (1) whenever $\deg f \neq \deg g$.

Proof. If either f or g is zero, both claims reduce to the $-\infty$ conventions, so assume $f, g \neq 0$ with $\deg f = m$, $\deg g = n$, and leading coefficients $a_m, b_n \neq 0$.

For the sum, the coefficient of X^k in $f + g$ is $a_k + b_k$, which is 0 for every $k > \max\{m, n\}$; hence $\deg(f+g) \leq \max\{m, n\}$. If $m \neq n$, say $m > n$, then the coefficient of X^m in $f+g$ equals $a_m + 0 = a_m \neq 0$, giving $\deg(f+g) = m = \max\{m, n\}$. (When $m = n$, cancellation $a_m + b_m = 0$ can occur, lowering the degree; this is why (1) is only an inequality in general.)

For the product, the coefficient of X^{m+n} is $c_{m+n} = \sum_{i+j=m+n} a_i b_j$. Every term with $i > m$ has $a_i = 0$, and every term with $i < m$ forces $j = m+n-i > n$, so $b_j = 0$; thus the only surviving term is $i = m, j = n$, giving $c_{m+n} = a_m b_n$. For $k > m+n$ the same argument shows every term vanishes, so $c_k = 0$. Since $\mathbb{F}$ is a field it has no zero divisors, so $a_m b_n \neq 0$; hence $\deg(fg) = m+n$ and the leading coefficient of fg is $a_m b_n$. $\square$

The product formula (2) is the first place the field hypothesis does real work: it required $a_m b_n \neq 0$, i.e. the absence of zero divisors. This single fact has sweeping consequences.

Corollary 5.8 ($\mathbb{F}[X]$ is an integral domain). *The ring $\mathbb{F}[X]$ has no zero divisors: if $fg = 0$ then $f = 0$ or $g = 0$. Equivalently, $\mathbb{F}[X]$ is an integral domain.*

Proof. If both f and g were nonzero, then $\deg(fg) = \deg f + \deg g \geq 0$ by (2), so $fg \neq 0$. Contrapositively, $fg = 0$ forces $f = 0$ or $g = 0$. $\square$

Corollary 5.9 (Cancellation). *If $f, g, h \in \mathbb{F}[X]$ with $h \neq 0$ and $fh = gh$, then $f = g$.*

Proof. From $fh = gh$ it follows that $(f - g)h = 0$; since $h \neq 0$, Corollary 5.8 forces $f - g = 0$. $\square$

Proposition 5.10 (Units of $\mathbb{F}[X]$). *A polynomial $f \in \mathbb{F}[X]$ is a unit (has a multiplicative inverse in $\mathbb{F}[X]$) if and only if it is a nonzero constant. Thus the units of $\mathbb{F}[X]$ are exactly the nonzero elements of $\mathbb{F}$.*

Proof. A nonzero constant $c \in \mathbb{F}^\times$ has inverse $c^{-1} \in \mathbb{F} \subseteq \mathbb{F}[X]$. Conversely, if $fg = 1$ then by (2), $\deg f + \deg g = \deg 1 = 0$; since degrees of nonzero polynomials are nonnegative, $\deg f = \deg g = 0$, so f is a nonzero constant. $\square$

Definition 5.11 (Associates). Two polynomials $f, g \in \mathbb{F}[X]$ are *associates* if $f = ug$ for some unit $u \in \mathbb{F}^\times$, i.e. if they differ only by a nonzero scalar factor.

Every nonzero polynomial f has exactly one monic associate, obtained by dividing through by its leading coefficient: $\text{lc}(f)^{-1}f$. Choosing the monic representative is the standard way to pin down “the” gcd or “the” irreducible factor uniquely, and we adopt it below.

5.3 The division algorithm

The arithmetic of $\mathbb{F}[X]$ parallels that of the integers $\mathbb{Z}$ with remarkable fidelity, and the source of the parallel is a division-with-remainder statement. Here the degree plays the role that absolute value plays in $\mathbb{Z}$.

Theorem 5.12 (Division algorithm). *Let $f, g \in \mathbb{F}[X]$ with $g \neq 0$. Then there exist unique polynomials $q, r \in \mathbb{F}[X]$ such that*

$$f = qg + r, \quad \text{with} \quad \deg r < \deg g.$$

We call q the quotient and r the remainder; we write $r = f \bmod g$ and $q = f \operatorname{div} g$.

Proof. Existence. Fix $g \neq 0$ and induct on $\deg f$. If $\deg f < \deg g$ (which includes the case $f = 0$, where $\deg f = -\infty$), take $q = 0$ and $r = f$; then $f = 0 \cdot g + f$ with $\deg r = \deg f < \deg g$.

Otherwise $m := \deg f \geq \deg g =: n$. Write the leading terms as $a_m X^m$ and $b_n X^n$, with $b_n \neq 0$. Consider

$$f_1 = f - \frac{a_m}{b_n} X^{m-n} g.$$

The subtracted term has degree m and leading coefficient $(a_m/b_n)b_n = a_m$, so it cancels the leading term of f ; hence $\deg f_1 < m$ (or $f_1 = 0$). By the induction hypothesis applied to f_1 , there are q_1, r with $f_1 = q_1 g + r$ and $\deg r < \deg g$. Then

$$f = f_1 + \frac{a_m}{b_n} X^{m-n} g = \left(q_1 + \frac{a_m}{b_n} X^{m-n} \right) g + r,$$

so $q = q_1 + (a_m/b_n)X^{m-n}$ and r work. The field hypothesis enters visibly here: division by the leading coefficient b_n requires b_n^{-1} to exist.

Uniqueness. Suppose $f = qg + r = q'g + r'$ with $\deg r, \deg r' < \deg g$. Subtracting, $(q - q')g = r' - r$. If $q \neq q'$, then $\deg((q - q')g) = \deg(q - q') + \deg g \geq \deg g$ by (2); but $\deg(r' - r) \leq \max\{\deg r', \deg r\} < \deg g$ by (1). These contradict each other, so $q = q'$, and then $r = r'$. $\square$

Remark 5.13 (The underlying algorithm). The existence proof is constructive: it is precisely *polynomial long division*. We repeatedly eliminate the current leading term by subtracting an appropriate monomial multiple of g ; each step lowers the degree, so the process terminates after at most $\deg f - \deg g + 1$ steps. For example, dividing $f = X^3 + 2X + 1$ by $g = X^2 + 1$ over $\mathbb{Q}$: first subtract $X \cdot g = X^3 + X$ to obtain $X + 1$, whose degree 1 is below $\deg g = 2$; hence $q = X, r = X + 1$, and indeed $X^3 + 2X + 1 = X(X^2 + 1) + (X + 1)$.

Definition 5.14 (Divisibility). We say g *divides* f , written $g \mid f$, if there is some $q \in \mathbb{F}[X]$ with $f = qg$; equivalently (for $g \neq 0$), if the remainder $f \bmod g$ is 0. In that case g is a *divisor* or *factor* of f .

5.4 Roots and the factor theorem

We now make precise the substitution of an element of $\mathbb{F}$ for the indeterminate. This is the first place where a polynomial gives rise to a function.

Definition 5.15 (Evaluation and roots). For $f = \sum_{i=0}^n a_i X^i \in \mathbb{F}[X]$ and $\alpha \in \mathbb{F}$, the *evaluation* of f at α is the field element

$$f(\alpha) = \sum_{i=0}^n a_i \alpha^i \in \mathbb{F},$$

which substitutes α for the indeterminate X . An element $\alpha \in \mathbb{F}$ is a *root* (or *zero*) of f if $f(\alpha) = 0$.

Proposition 5.16 (Evaluation is a ring homomorphism). Fix $\alpha \in \mathbb{F}$. The evaluation map $\text{ev}_\alpha : \mathbb{F}[X] \rightarrow \mathbb{F}, f \mapsto f(\alpha)$, satisfies, for all $f, g \in \mathbb{F}[X]$,

$$(f + g)(\alpha) = f(\alpha) + g(\alpha), \quad (f \cdot g)(\alpha) = f(\alpha)g(\alpha), \quad 1(\alpha) = 1.$$

That is, ev_α is a ring homomorphism.

Proof. Additivity is immediate from coordinatewise addition: $\sum_i (a_i + b_i) \alpha^i = \sum_i a_i \alpha^i + \sum_i b_i \alpha^i$. For multiplicativity, the product fg has k -th coefficient $c_k = \sum_{i+j=k} a_i b_j$, so

$$(fg)(\alpha) = \sum_k c_k \alpha^k = \sum_k \sum_{i+j=k} a_i b_j \alpha^{i+j} = \left(\sum_i a_i \alpha^i \right) \left(\sum_j b_j \alpha^j \right) = f(\alpha)g(\alpha),$$

where the middle step expands the product of the two sums and groups terms by $k = i + j$. Finally $1(\alpha) = 1$. This is the formal reason evaluation respects the algebra: the convolution defining polynomial multiplication collapses to ordinary multiplication in $\mathbb{F}$ because $\alpha^i \alpha^j = \alpha^{i+j}$. $\square$

The link between roots and divisibility is the factor theorem, an immediate but powerful consequence of the division algorithm.

Theorem 5.17 (Remainder theorem and factor theorem). *Let $f \in \mathbb{F}[X]$ and $\alpha \in \mathbb{F}$.*

1. (Remainder theorem.) *The remainder of f upon division by the linear polynomial $X - \alpha$ equals the constant $f(\alpha)$:*

$$f = q(X - \alpha) + f(\alpha) \quad \text{for some } q \in \mathbb{F}[X].$$

2. (Factor theorem.) *α is a root of f if and only if $(X - \alpha) \mid f$.*

Proof. (1) Divide f by $X - \alpha$. Since $\deg(X - \alpha) = 1$, the remainder r has degree < 1 , hence is a constant $r \in \mathbb{F}$. Write $f = q(X - \alpha) + r$ and evaluate at α using Proposition 5.16:

$$f(\alpha) = q(\alpha)(\alpha - \alpha) + r = q(\alpha) \cdot 0 + r = r.$$

Thus $r = f(\alpha)$.

- (2) By part (1), $(X - \alpha) \mid f$ means the remainder $f(\alpha)$ is 0, i.e. α is a root. □

This single theorem yields the central counting bound of the section.

Theorem 5.18 (At most d roots). *A nonzero polynomial $f \in \mathbb{F}[X]$ of degree d has at most d distinct roots in $\mathbb{F}$.*

Proof. Induct on $d = \deg f$. If $d = 0$, then f is a nonzero constant, which is never 0, so it has $0 \leq d$ roots. Suppose $d \geq 1$ and the claim holds for all smaller degrees. If f has no root in $\mathbb{F}$, then $0 \leq d$ and the claim holds. Otherwise let α be a root. By the factor theorem, $f = (X - \alpha)q$ for some q with $\deg q = d - 1$ (by (2)). Now let $\beta \neq \alpha$ be any root of f . Then

$$0 = f(\beta) = (\beta - \alpha)q(\beta),$$

and since $\beta - \alpha \neq 0$ and $\mathbb{F}$ has no zero divisors, it follows that $q(\beta) = 0$. Thus every root of f other than α is a root of q . By the induction hypothesis q has at most $d - 1$ distinct roots, so f has at most $(d - 1) + 1 = d$ distinct roots. □

Remark 5.19. Both the field hypothesis (no zero divisors) and the degree bookkeeping are essential. Over a ring with zero divisors the bound fails: in $\mathbb{Z}/8\mathbb{Z}$ the quadratic $X^2 - 1$ has the four roots 1, 3, 5, 7. This is why the polynomial methods underlying Halo 2 require a field — and indeed a finite field $\mathbb{F}_p$ or $\mathbb{F}_q$ — beneath them.

Corollary 5.20 (Few points determine a low-degree polynomial). *Let $f, g \in \mathbb{F}[X]$ both have degree at most d . If $f(\alpha) = g(\alpha)$ for $d + 1$ distinct values $\alpha \in \mathbb{F}$, then $f = g$ as polynomials.*

Proof. The difference $h := f - g$ has $\deg h \leq \max\{\deg f, \deg g\} \leq d$ and vanishes at $d + 1$ distinct points. If h were nonzero it would have degree at most d and yet $d + 1$ distinct roots, contradicting Theorem 5.18. Hence $h = 0$, i.e. $f = g$. □

Corollary 5.20 is the deterministic skeleton of the Schwartz–Zippel principle and of Lagrange interpolation; we develop both below.

5.5 Greatest common divisors and the Euclidean algorithm

Because $\mathbb{F}[X]$ supports division with remainder, the entire theory of greatest common divisors from elementary number theory transfers verbatim, with degree replacing magnitude.

Definition 5.21 (Greatest common divisor). Let $f, g \in \mathbb{F}[X]$, not both zero. A *greatest common divisor* of f and g is a polynomial $d \in \mathbb{F}[X]$ such that

1. $d \mid f$ and $d \mid g$ (it is a common divisor), and
2. every common divisor c of f and g satisfies $c \mid d$ (it is divisible by every other common divisor).

A gcd is determined only up to multiplication by a unit; we single out the unique *monic* gcd and denote it $\gcd(f, g)$. When $\gcd(f, g) = 1$ we say f and g are *coprime* or *relatively prime*.

The existence of such a d , its uniqueness up to associates, and its expression as an $\mathbb{F}[X]$ -linear combination of f and g constitute the content of the next two results. The Euclidean algorithm delivers all of this algorithmically.

Lemma 5.22 (Euclidean step). Let $f, g \in \mathbb{F}[X]$ with $g \neq 0$, and write $f = qg + r$ as in the division algorithm. Then the pair (f, g) and the pair (g, r) have exactly the same common divisors; in particular their gcds coincide.

Proof. If $c \mid f$ and $c \mid g$, then $c \mid (f - qg) = r$, so c is a common divisor of g and r . Conversely, if $c \mid g$ and $c \mid r$, then $c \mid (qg + r) = f$, so c is a common divisor of f and g . The two sets of common divisors are equal, hence so are their maximal elements under divisibility. $\square$

Theorem 5.23 (Euclidean algorithm and Bézout's identity). Let $f, g \in \mathbb{F}[X]$, not both zero. Then $\gcd(f, g)$ exists, is unique as a monic polynomial, and there exist $s, t \in \mathbb{F}[X]$ with

$$\gcd(f, g) = sf + tg. \quad (3)$$

Concretely, define a sequence by $r_0 = f$, $r_1 = g$, and, as long as $r_i \neq 0$,

$$r_{i-1} = q_i r_i + r_{i+1}, \quad \deg r_{i+1} < \deg r_i.$$

The degrees strictly decrease, so some $r_{N+1} = 0$; the last nonzero remainder r_N , made monic, is $\gcd(f, g)$.

Proof. Termination and the gcd property. If $g = 0$ the gcd is the monic associate of f ; so assume $g \neq 0$. The remainders satisfy $\deg r_1 > \deg r_2 > \dots \geq 0$ (a strictly decreasing sequence of nonnegative integers once we pass r_1), so the algorithm halts with some $r_{N+1} = 0$ and $r_N \neq 0$. Applying Lemma 5.22 repeatedly,

$$\begin{aligned} \{\text{common divisors of } (f, g)\} &= \{\text{common divisors of } (r_1, r_2)\} \\ &= \dots = \{\text{common divisors of } (r_N, 0)\}. \end{aligned}$$

The common divisors of $(r_N, 0)$ are exactly the divisors of r_N (since everything divides 0), whose greatest element under divisibility is r_N itself. Hence r_N is a gcd of f and g : it divides both, and any common divisor divides it. Making it monic gives the distinguished $\gcd(f, g)$.

Uniqueness. If d, d' are both monic gcds, then $d \mid d'$ and $d' \mid d$ by property (2), so $d' = ud$ for a unit u ; comparing leading coefficients of two monic polynomials forces $u = 1$, whence $d = d'$.

Bézout. We show by induction on the index i that each r_i is an $\mathbb{F}[X]$ -linear combination of f and g . Indeed $r_0 = f$ and $r_1 = g$ are such combinations, and the recurrence $r_{i+1} = r_{i-1} - q_i r_i$ expresses r_{i+1} as a combination once r_{i-1} and r_i are. Hence $r_N = s_0 f + t_0 g$ for some s_0, t_0 ; dividing through by $\text{lc}(r_N)$ to make it monic yields (3). $\square$

Remark 5.24 (Extended Euclidean algorithm). Carrying the linear-combination coefficients along with the remainders — the *extended Euclidean algorithm* — produces s and t explicitly alongside the gcd. This is precisely how one inverts elements in a quotient $\mathbb{F}[X]/(m)$ by an irreducible m : if $\gcd(f, m) = 1$ then $sf + tm = 1$, so s is the inverse of f modulo m . The curves of Halo 2 and Orchard, by contrast, are defined over *prime* fields $\mathbb{F}_p$, where inversion uses the integer extended Euclidean algorithm (Theorem 2.13) or Fermat’s little theorem (section 6); the polynomial version above matters because it parallels the integer one exactly and because it computes gcds of the polynomials a proof system manipulates.

Corollary 5.25 (Euclid’s lemma in $\mathbb{F}[X]$). *Let $p, f, g \in \mathbb{F}[X]$.*

1. *If $\gcd(p, f) = 1$ and $p \mid fg$, then $p \mid g$.*
2. *If p is irreducible (Definition 5.26) and $p \mid fg$, then $p \mid f$ or $p \mid g$.*

Proof. (1) By Bézout there are s, t with $sp + tf = 1$. Multiply by g : $spg + tfg = g$. Now p divides spg trivially and divides tfg because $p \mid fg$; hence p divides their sum g .

(2) If $p \nmid f$, then $\gcd(p, f)$ is a monic divisor of p that is not the monic associate of p ; as p is irreducible its only monic divisors are 1 and its own monic associate, so $\gcd(p, f) = 1$. By part (1), $p \mid g$. $\square$

5.6 Irreducible polynomials and unique factorization

Definition 5.26 (Irreducible polynomial). A polynomial $p \in \mathbb{F}[X]$ is *irreducible* (over $\mathbb{F}$) if $\deg p \geq 1$ and p admits no factorization as a product $p = gh$ with both $\deg g \geq 1$ and $\deg h \geq 1$. Equivalently, p is nonconstant and its only factorizations are the trivial ones $p = (\text{unit}) \cdot (\text{associate of } p)$. A nonconstant polynomial that is not irreducible is *reducible*.

Remark 5.27 (Irreducibility is relative to the field). Irreducibility depends crucially on $\mathbb{F}$. The polynomial $X^2 + 1$ is irreducible over $\mathbb{R}$ (it has no real root, and a real quadratic factors iff it has a root), but over $\mathbb{C}$ it splits as $(X - i)(X + i)$, and over $\mathbb{F} = \mathbb{F}_2$ it factors as $(X + 1)^2$. The degree-1 polynomials are irreducible over every field. For later use, observe that a polynomial of degree 2 or 3 is irreducible over $\mathbb{F}$ if and only if it has no root in $\mathbb{F}$: any nontrivial factorization of a quadratic or cubic must include a linear factor, which by the factor theorem supplies a root. This rootlessness test fails for degree ≥ 4 , where a factorization into two irreducible quadratics has no roots yet is reducible.

Irreducible polynomials are the “primes” of $\mathbb{F}[X]$. We now show that $\mathbb{F}[X]$ admits unique factorization into them, exactly as $\mathbb{Z}$ factors into primes.

Theorem 5.28 ($\mathbb{F}[X]$ is a unique factorization domain). *Every nonconstant polynomial $f \in \mathbb{F}[X]$ factors as*

$$f = c p_1 p_2 \cdots p_k, \tag{4}$$

where $c = \text{lc}(f) \in \mathbb{F}^\times$ is a constant and $p_1, \dots, p_k$ are monic irreducible polynomials. This factorization is unique up to the order of the factors: if also $f = c' q_1 \cdots q_\ell$ with $c' \in \mathbb{F}^\times$ and each q_j monic irreducible, then $c = c'$, $k = \ell$, and the q_j are a reordering of the p_i .

Proof (sketch). The argument is the template of the fundamental theorem of arithmetic (Theorem 2.18), with degree in place of absolute value. *Existence:* induct on $\deg f$; an irreducible f is $\text{lc}(f)$ times its monic associate, and a reducible $f = gh$ splits into factors of smaller degree, whose factorizations multiply together. *Uniqueness:* comparing leading coefficients gives $c = c' = \text{lc}(f)$; then Euclid’s lemma (Corollary 5.25(2)) lets one match p_1 with some q_j , cancel (Corollary 5.9), and recurse, exactly as in $\mathbb{Z}$. $\square$

Remark 5.29. The structural reason underlying Theorem 5.28 is that $\mathbb{F}[X]$ is a *Euclidean domain* (via the degree function and the division algorithm), hence a *principal ideal domain*: every ideal of $\mathbb{F}[X]$ is of the form $(g) = \{qg : q \in \mathbb{F}[X]\}$ for a single generator g , namely a gcd of any generating set. Bézout's identity (3) is the concrete shadow of this fact. Euclidean $\Rightarrow$ PID $\Rightarrow$ UFD is the standard chain of implications in ring theory; we have just walked it explicitly for $\mathbb{F}[X]$.

5.7 Lagrange interpolation

Corollary 5.20 established that a polynomial of degree at most d is determined by its values at $d + 1$ points. Lagrange interpolation is the constructive converse: *any* prescribed values at $d + 1$ distinct points arise from a *unique* such polynomial. This is the algebraic backbone of secret sharing, of Reed–Solomon codes, and of the polynomial encodings in Halo 2.

Theorem 5.30 (Lagrange interpolation). *Let $x_0, x_1, \dots, x_d \in \mathbb{F}$ be $d + 1$ distinct points, and let $y_0, y_1, \dots, y_d \in \mathbb{F}$ be arbitrary target values. Then there exists a unique polynomial $f \in \mathbb{F}[X]$ with $\deg f \leq d$ and $f(x_i) = y_i$ for all $i = 0, 1, \dots, d$. The explicit formula*

$$f(X) = \sum_{i=0}^d y_i \ell_i(X), \quad \text{where} \quad \ell_i(X) = \prod_{\substack{0 \leq j \leq d \\ j \neq i}} \frac{X - x_j}{x_i - x_j} \quad (5)$$

gives it. We call the polynomials ℓ_i the Lagrange basis polynomials for the nodes $x_0, \dots, x_d$.

Proof. The basis polynomials are well defined. Each denominator $\prod_{j \neq i} (x_i - x_j)$ is a product of nonzero elements of $\mathbb{F}$ (distinctness of the nodes), hence is a nonzero element of $\mathbb{F}$ and is invertible. Thus each ℓ_i is a genuine polynomial of degree exactly d (a product of d linear factors).

The interpolation property. Evaluate ℓ_i at a node x_m . If $m = i$, every factor is $(x_i - x_j)/(x_i - x_j) = 1$, so $\ell_i(x_i) = 1$. If $m \neq i$, then the factor with $j = m$ has numerator $x_m - x_m = 0$, so $\ell_i(x_m) = 0$. In short,

$$\ell_i(x_m) = \delta_{im} = \begin{cases} 1 & m = i, \\ 0 & m \neq i. \end{cases} \quad (6)$$

Therefore $f(x_m) = \sum_i y_i \ell_i(x_m) = \sum_i y_i \delta_{im} = y_m$ for each m , as required, and $\deg f \leq \max_i \deg \ell_i = d$.

Uniqueness. If f and $\tilde{f}$ both have degree at most d and agree at the $d + 1$ distinct points $x_0, \dots, x_d$, then by Corollary 5.20 they are equal as polynomials. Hence the interpolant is unique. $\square$

Example 5.31 (A small interpolation). Over $\mathbb{Q}$, find the polynomial of degree ≤ 2 with $f(0) = 1$, $f(1) = 3$, $f(2) = 7$. The basis polynomials for nodes 0, 1, 2 are

$$\begin{aligned} \ell_0 &= \frac{(X-1)(X-2)}{(0-1)(0-2)} = \frac{(X-1)(X-2)}{2}, & \ell_1 &= \frac{(X-0)(X-2)}{(1-0)(1-2)} = -X(X-2), \\ \ell_2 &= \frac{(X-0)(X-1)}{(2-0)(2-1)} = \frac{X(X-1)}{2}. \end{aligned}$$

Then $f = 1 \cdot \ell_0 + 3 \cdot \ell_1 + 7 \cdot \ell_2 = X^2 + X + 1$, and one verifies $f(0) = 1$, $f(1) = 3$, $f(2) = 7$.

Remark 5.32 (The evaluation isomorphism). Fix $d+1$ distinct nodes $x_0, \dots, x_d$ and let $V = \{f \in \mathbb{F}[X] : \deg f \leq d\}$, a $(d+1)$ -dimensional $\mathbb{F}$ -vector space (basis $1, X, \dots, X^d$). The *evaluation map*

$$\text{ev} : V \rightarrow \mathbb{F}^{d+1}, \quad f \mapsto (f(x_0), \dots, f(x_d)),$$

is $\mathbb{F}$ -linear. Surjectivity is exactly the existence half of Lagrange interpolation, and injectivity is exactly the uniqueness half; together they say ev is a *linear isomorphism* $V \cong \mathbb{F}^{d+1}$. The Lagrange basis $\{\ell_i\}$

is precisely the basis of V dual to the standard basis of $\mathbb{F}^{d+1}$, by (6). This isomorphism — switching between a polynomial’s *coefficient representation* and its *evaluation representation* — is the conceptual core of the fast (FFT-based) polynomial arithmetic and the evaluation-domain encodings used throughout Halo 2.

Remark 5.33 (The vanishing polynomial). The product of all the linear factors over the node set,

$$Z(X) = \prod_{i=0}^d (X - x_i),$$

is the unique monic polynomial of degree $d + 1$ that vanishes at every node; we call it the *vanishing polynomial* of the set $\{x_0, \dots, x_d\}$. A polynomial g vanishes on the whole node set if and only if $Z \mid g$ — one direction is the factor theorem applied repeatedly (using that the distinct $X - x_i$ are pairwise coprime), the other is immediate. The pair (Lagrange basis, vanishing polynomial) is exactly the data a proof system manipulates when it argues that a committed polynomial agrees with prescribed values on an evaluation domain.

5.8 The formal derivative and multiplicities

Roots can occur “with multiplicity,” and to detect repeated roots without any appeal to limits we introduce a purely algebraic derivative.

Definition 5.34 (Multiplicity of a root). Let $f \in \mathbb{F}[X]$ be nonzero and $\alpha \in \mathbb{F}$ a root. The *multiplicity* of α (as a root of f) is the largest integer $m \geq 1$ such that $(X - \alpha)^m \mid f$. We write $m = \text{mult}_\alpha(f)$. A root of multiplicity 1 is *simple*; a root of multiplicity ≥ 2 is *repeated* (or *multiple*).

This is well defined: the powers $(X - \alpha)^m$ have strictly increasing degree m , so only finitely many can divide the fixed polynomial f ; the largest such m exists. Equivalently, m is determined by writing $f = (X - \alpha)^m g$ with $g(\alpha) \neq 0$.

Proposition 5.35 (Counting roots with multiplicity). If $f \in \mathbb{F}[X]$ is nonzero of degree d with distinct roots $\alpha_1, \dots, \alpha_r \in \mathbb{F}$ of multiplicities $m_1, \dots, m_r$, then

$$f = (X - \alpha_1)^{m_1} \dots (X - \alpha_r)^{m_r} g$$

for some $g \in \mathbb{F}[X]$ with no roots in $\mathbb{F}$, and $\sum_{i=1}^r m_i \leq d$. In particular the number of roots counted with multiplicity is at most d , with equality iff f splits into linear factors over $\mathbb{F}$.

Proof. The distinct linear polynomials $X - \alpha_i$ are pairwise coprime (their gcd is 1, as a common divisor would be a nonzero constant by degree). Hence if $(X - \alpha_i)^{m_i} \mid f$ for each i , their product divides f : this follows by induction using that if coprime a, b both divide f then $ab \mid f$ (write $f = au = bv$; since $\gcd(a, b) = 1$, Bézout gives $sa + tb = 1$, so $f = saf + tbv = sa(bv) + tb(au) = ab(sv + tu)$). Thus $f = \prod_i (X - \alpha_i)^{m_i} g$; by maximality of each m_i , $g(\alpha_i) \neq 0$, and g has no other roots either (any root of g would be a root of f , hence some α_i). Taking degrees, $\sum_i m_i + \deg g = d$, so $\sum_i m_i \leq d$, with equality iff $\deg g = 0$, i.e. g is a nonzero constant and f is a product of linear factors. $\square$

Definition 5.36 (Formal derivative). The *formal derivative* is the map $D : \mathbb{F}[X] \rightarrow \mathbb{F}[X]$ defined on $f = \sum_{i=0}^n a_i X^i$ by

$$f'(X) = Df = \sum_{i=1}^n i a_i X^{i-1} = a_1 + 2a_2 X + 3a_3 X^2 + \dots + n a_n X^{n-1},$$

where $i a_i$ means a_i added to itself i times in $\mathbb{F}$ (the image of the integer i under $\mathbb{Z} \rightarrow \mathbb{F}$). No limits are involved; this is a formal, algebraic definition that makes sense over any field.

Proposition 5.37 (Rules of formal differentiation). *For all $f, g \in \mathbb{F}[X]$ and $c \in \mathbb{F}$:*

1. (Linearity.) $(f + g)' = f' + g'$ and $(cf)' = cf'$.
2. (Product rule / Leibniz.) $(fg)' = f'g + fg'$.
3. (Power rule.) $((X - \alpha)^m)' = m(X - \alpha)^{m-1}$ for every $\alpha \in \mathbb{F}$ and $m \geq 1$.

Proof. (1) is immediate from the coordinatewise, linear definition of D .

(2) By linearity it suffices to prove the product rule on monomials $f = X^a, g = X^b$, since both sides are bilinear in (f, g) and every polynomial is a linear combination of monomials. For monomials, $(X^a X^b)' = (X^{a+b})' = (a+b)X^{a+b-1}$, while $f'g + fg' = aX^{a-1}X^b + X^a bX^{b-1} = (a+b)X^{a+b-1}$. The two agree. Extending by bilinearity: writing $f = \sum_a u_a X^a, g = \sum_b v_b X^b$, we have $(fg)' = \sum_{a,b} u_a v_b (X^a X^b)' = \sum_{a,b} u_a v_b (X^a)' X^b + \sum_{a,b} u_a v_b X^a (X^b)' = f'g + fg'$.

(3) Induct on m using the product rule. For $m = 1$, $(X - \alpha)' = 1 = 1 \cdot (X - \alpha)^0$. Assuming the claim for m ,

$$((X - \alpha)^{m+1})' = ((X - \alpha)^m (X - \alpha))' = m(X - \alpha)^{m-1}(X - \alpha) + (X - \alpha)^m \cdot 1 = (m+1)(X - \alpha)^m. \quad \square$$

Theorem 5.38 (Derivative test for multiple roots). *Let $f \in \mathbb{F}[X]$ be nonzero and $\alpha \in \mathbb{F}$. Then α is a repeated root of f (multiplicity ≥ 2) if and only if $f(\alpha) = 0$ and $f'(\alpha) = 0$. Consequently f has a repeated root in $\mathbb{F}$ only if $\gcd(f, f') \neq 1$; and f has no repeated roots in any field containing $\mathbb{F}$ (it is separable) if and only if $\gcd(f, f') = 1$ in $\mathbb{F}[X]$.*

Proof. Write $\text{mult}_\alpha(f) = m \geq 1$, so $f = (X - \alpha)^m g$ with $g(\alpha) \neq 0$. By the product and power rules,

$$f' = m(X - \alpha)^{m-1}g + (X - \alpha)^m g' = (X - \alpha)^{m-1}[mg + (X - \alpha)g'].$$

Evaluate the bracket at α : it equals $mg(\alpha) + 0 = mg(\alpha)$.

If $m = 1$: then $f' = g + (X - \alpha)g'$, so $f'(\alpha) = g(\alpha) \neq 0$. Thus a simple root is *not* a root of f' .

If $m \geq 2$: then $(X - \alpha)^{m-1}$ with $m - 1 \geq 1$ divides f' , so $f'(\alpha) = 0$; and of course $f(\alpha) = 0$.

Hence $f(\alpha) = f'(\alpha) = 0$ exactly when $m \geq 2$, proving the first claim. For the gcd statements: a common root $\alpha \in \mathbb{F}$ of f and f' forces $(X - \alpha) \mid \gcd(f, f')$, so a repeated root makes the gcd nontrivial. The full separability statement requires care over extensions: one shows $\gcd(f, f')$, computed in $\mathbb{F}[X]$, is unchanged under field extension (the Euclidean algorithm uses only field operations in $\mathbb{F}$), and that over a *splitting field* of f (an extension field of $\mathbb{F}$ in which f factors completely into linear factors) a repeated factor $(X - \alpha)^m, m \geq 2$, contributes a common factor $(X - \alpha)$ to f and f' by the computation above, while distinct simple linear factors contribute none. Thus $\gcd(f, f') = 1$ in $\mathbb{F}[X]$ if and only if f has no repeated root in any extension. $\square$

Remark 5.39 (The pitfall of positive characteristic). The integer coefficient i in $f' = \sum_i i a_i X^{i-1}$ is interpreted in $\mathbb{F}$, and may vanish there. Over a field of characteristic p (meaning $p = 0$ in $\mathbb{F}$, with p prime — the setting of $\mathbb{F}_p$ and $\mathbb{F}_q$), the derivative of X^p is $pX^{p-1} = 0$. Thus $f = X^p - 1$ over $\mathbb{F}_p$ has $f' = 0$, and $\gcd(f, f') = f \neq 1$; indeed $X^p - 1 = (X - 1)^p$ in $\mathbb{F}_p[X]$, a single root of multiplicity p . This characteristic- p behavior is not a pathology to avoid but a structural feature one must respect when working over the finite fields of Orchard.

5.9 Polynomials as formal objects versus functions

This section sharpens the distinction noted at the outset, and extracts from it the probabilistic principle on which the remainder of this book rests.

Definition 5.40 (Polynomial function). Every $f \in \mathbb{F}[X]$ induces a *polynomial function* $\tilde{f} : \mathbb{F} \rightarrow \mathbb{F}$, $\alpha \mapsto f(\alpha)$. The assignment $f \mapsto \tilde{f}$ is a ring homomorphism from $\mathbb{F}[X]$ to the ring $\mathbb{F}^{\mathbb{F}}$ of all functions $\mathbb{F} \rightarrow \mathbb{F}$ (with pointwise operations), by Proposition 5.16 applied at every point.

The central question is whether two *distinct* polynomials can induce the *same* function. Over an infinite field they cannot; over a finite field they can.

Proposition 5.41 (When formal equals functional). *Let $f, g \in \mathbb{F}[X]$.*

1. *If $\mathbb{F}$ is infinite, then $\tilde{f} = \tilde{g}$ (equal as functions) implies $f = g$ (equal as polynomials). Equivalently, the map $f \mapsto \tilde{f}$ is injective, and one may identify polynomials with polynomial functions.*
2. *If $\mathbb{F}$ is finite, the map $f \mapsto \tilde{f}$ is not injective. For instance, over $\mathbb{F}_p$ the nonzero polynomial $X^p - X$ induces the zero function, since by Fermat's little theorem (Corollary 3.30) $\alpha^p = \alpha$ for every $\alpha \in \mathbb{F}_p$.*

Proof. (1) Suppose $\tilde{f} = \tilde{g}$. Then $h := f - g$ satisfies $h(\alpha) = 0$ for every $\alpha \in \mathbb{F}$. If h were nonzero it would have some finite degree d and thus at most d roots by Theorem 5.18; but $\mathbb{F}$ is infinite, so h has infinitely many roots — a contradiction. Hence $h = 0$ and $f = g$.

(2) Let $\mathbb{F} = \mathbb{F}_q$ be finite. The monic polynomial $w(X) := \prod_{\alpha \in \mathbb{F}_q} (X - \alpha)$, of degree $q = |\mathbb{F}_q|$, is nonzero yet vanishes at every $\alpha \in \mathbb{F}_q$ by construction. Thus w and the zero polynomial induce the same (zero) function while being distinct formal objects, so the map is not injective. Over the prime field $\mathbb{F}_p$ this witness is $X^p - X$: Fermat's little theorem (Corollary 3.30) makes $\mathbb{F}_p$ the root set of the monic degree- p polynomial $X^p - X$, recovering the instance in the statement. $\square$

Remark 5.42 (Why the formal viewpoint is the correct one). Proposition 5.41(2) is precisely the reason we define polynomials as coefficient sequences rather than as functions. In cryptography one works over finite fields $\mathbb{F}_p, \mathbb{F}_q$, where the functional viewpoint loses information; the *degree* of a polynomial — a property of the formal object, invisible to the induced function once the field is finite — is what the security of polynomial commitment schemes rests on. A prover commits to a low-degree formal polynomial; the verifier tests it by evaluation. The gap between agreement as a function at the tested points and identity to the claimed formal polynomial is precisely the gap that the next principle quantifies.

Theorem 5.43 (Univariate Schwartz–Zippel). *Let $f \in \mathbb{F}[X]$ be a nonzero polynomial of degree at most d , and let $S \subseteq \mathbb{F}$ be a finite, nonempty set. If α is chosen uniformly at random from S , then*

$$\Pr_{\alpha \sim S} [f(\alpha) = 0] \leq \frac{d}{|S|}. \quad (7)$$

More generally, if $f, g \in \mathbb{F}[X]$ are distinct polynomials each of degree at most d , then $\Pr_{\alpha \sim S} [f(\alpha) = g(\alpha)] \leq d/|S|$.

Proof. By Theorem 5.18, the nonzero polynomial f of degree at most d has at most d roots in $\mathbb{F}$, hence at most d roots inside S . The number of $\alpha \in S$ with $f(\alpha) = 0$ is therefore at most d , and since α is uniform on S ,

$$\Pr_{\alpha \sim S} [f(\alpha) = 0] = \frac{|\{\alpha \in S : f(\alpha) = 0\}|}{|S|} \leq \frac{d}{|S|}.$$

For the general statement, apply this to the nonzero polynomial $h = f - g$, which has degree at most d ; $f(\alpha) = g(\alpha)$ iff $h(\alpha) = 0$. $\square$

Remark 5.44 (The principle stated plainly, and its application). Inequality (7) formalizes the statement that *two low-degree polynomials that agree at many points must be equal*. The deterministic version is Corollary 5.20: agreement at $d + 1$ points forces equality. Schwartz–Zippel is its probabilistic refinement: even a *single* random evaluation point detects unequal degree- $\leq d$ polynomials except with probability at most $d/|S|$. If $|S|$ is exponentially large (as for $S = \mathbb{F}_q$ with $q \approx 2^{254}$, the 255-bit fields of Orchard) and d is polynomially bounded, this failure probability is negligible. This constitutes the foundational soundness argument for polynomial identity testing, and through it for the polynomial commitment schemes and arithmetization checks at the heart of Halo 2: to verify a purported polynomial identity, one evaluates both sides at a random challenge and compares. We invoke this principle repeatedly in the analysis of those protocols.

Remark 5.45 (Looking ahead: multivariate generalization). The full Schwartz–Zippel lemma extends (7) to multivariate polynomials $f \in \mathbb{F}[X_1, \dots, X_n]$: a nonzero polynomial of total degree d vanishes at a uniformly random point of S^n with probability at most $d/|S|$. The univariate case proved here is both the base case of its inductive proof and the version most directly used in the analysis of Halo 2; the later volumes invoke only the univariate case directly; where an identity in several challenges arises (e.g. the permutation argument’s pair (β, γ) in the *Halo 2 Guide*), the stated multivariate bound applies, and follows from the univariate lemma by applying it once per variable.

6 Finite fields

A *field* is a set in which one can add, subtract, multiply, and divide by anything nonzero, with all the usual algebraic laws holding. The familiar examples $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$ are all infinite. This section develops the theory of *finite* fields: fields with only finitely many elements. These objects are the arithmetic substrate of essentially all modern public-key cryptography, and in particular of the elliptic-curve constructions underlying Halo 2 and Zcash Orchard. There the coordinates of curve points live in a prime field $\mathbb{F}_p$, while the scalars that multiply those points live in a second prime field $\mathbb{F}_q$; understanding both, and the relationship between them, begins with understanding finite fields in general.

The ring theory of section 4 and the polynomial theory of section 5 are assumed; the emphasis here is on the structure theory specific to the finite case.

6.1 The prime field $\mathbb{F}_p$ and modular arithmetic

The basic example is already in hand: by Corollary 2.28, for a prime p the ring $\mathbb{Z}/p\mathbb{Z}$ is a field, the *prime field* $\mathbb{F}_p$, because every $a \in \{1, \dots, p - 1\}$ satisfies $\gcd(a, p) = 1$ and is therefore invertible (Theorem 2.27). Its arithmetic is integer arithmetic followed by reduction modulo p . Division is the one operation whose implementation deserves comment, and two methods coexist in practice.

Inversion via the extended Euclidean algorithm. The extended Euclidean algorithm (Theorem 2.13) on input (a, p) with $0 < a < p$ returns u with $ua \equiv 1 \pmod{p}$, so $\bar{a}^{-1} = \bar{u}$, in $O(\log p)$ division steps; Example 2.29 carried this out. This is the standard general-purpose method.

Inversion via Fermat’s little theorem. By Fermat’s little theorem (Corollary 3.30), every nonzero $\bar{a} \in \mathbb{F}_p$ satisfies $\bar{a}^{p-1} = \bar{1}$, so $\bar{a} \cdot \bar{a}^{p-2} = \bar{1}$ and

$$\bar{a}^{-1} = \bar{a}^{p-2}.$$

The exponentiation is computed by *square-and-multiply*: to raise to an exponent e with binary expansion $e = \sum_i e_i 2^i$, square repeatedly to obtain a^{2^i} and multiply together those factors with $e_i = 1$, for

a total of $O(\log e)$ multiplications rather than $e - 1$. (Double-and-add, theorem 10.13, is the same algorithm written additively for elliptic-curve scalar multiplication.) Inversion thus costs $O(\log p)$ modular multiplications. The method is constant-time-friendly — the sequence of operations does not branch on the secret a — which is why implementations that must resist side-channel attacks often prefer this formula to the extended Euclidean algorithm.

6.2 Characteristic, prime subfield, and order

Let F be any field and recall from section 4.5 the ring homomorphism $\iota: \mathbb{Z} \rightarrow F$, $\iota(n) = n \cdot 1_F$, and the *characteristic* $\text{char } F$, the nonnegative generator of $\ker \iota$ (Definition 4.24). If F is finite, ι cannot be injective (its domain is infinite), so $\text{char } F = p > 0$, and p is prime by Theorem 4.26. For $F = \mathbb{F}_p$ itself, $\text{char } \mathbb{F}_p = p$.

Definition 6.1 (Prime subfield). The *prime subfield* of F is the smallest subfield of F , namely the intersection of all subfields, equivalently the subfield generated by 1_F .

Proposition 6.2. If $\text{char } F = p > 0$, the prime subfield of F is isomorphic to $\mathbb{F}_p$. If $\text{char } F = 0$, it is isomorphic to $\mathbb{Q}$.

Proof. Consider $\iota: \mathbb{Z} \rightarrow F$ as above. When $\text{char } F = p$, the kernel is $p\mathbb{Z}$, so ι factors through an injective ring homomorphism $\bar{\iota}: \mathbb{Z}/p\mathbb{Z} \rightarrow F$, whose image is a subfield isomorphic to $\mathbb{F}_p$. Any subfield contains 1_F and hence this image, so it is the prime subfield. When $\text{char } F = 0$, ι is injective and extends to an embedding of $\mathbb{Q}$ by sending $a/b \mapsto (a \cdot 1_F)(b \cdot 1_F)^{-1}$; its image is the prime subfield. $\square$

Thus every finite field F is a vector space over its prime subfield $\mathbb{F}_p$. Since F is finite it is finite-dimensional; if $\dim_{\mathbb{F}_p} F = k$, then choosing a basis $e_1, \dots, e_k$ shows that every element of F is uniquely $c_1 e_1 + \dots + c_k e_k$ with $c_i \in \mathbb{F}_p$, so F has exactly p^k elements (basis and dimension are developed formally in section 8; the argument needs only basis existence and uniqueness of coordinates).

Theorem 6.3 (Order of a finite field is a prime power). Every finite field has cardinality p^k for some prime p and integer $k \geq 1$, where $p = \text{char } F$.

Proof. As shown above, $p = \text{char } F$ is prime, and by Proposition 6.2 the prime subfield is $\mathbb{F}_p$. As just observed, F is a finite-dimensional $\mathbb{F}_p$ -vector space, say of dimension $k \geq 1$, and counting coordinate vectors gives $|F| = p^k$. $\square$

Henceforth $q = p^k$ denotes a prime power and $\mathbb{F}_q$ a field with q elements. A field of every prime-power order exists, and any two fields of the same order are isomorphic (one constructs $\mathbb{F}_q$ as a quotient $\mathbb{F}_p[x]/(f)$ by an irreducible polynomial of degree k , cf. Example 4.19); we do not prove these facts, since the fields of Halo 2 and Orchard are the prime fields $q = p$, where theorem 2.28 already settles everything. We state results for general $\mathbb{F}_q$ when the proofs are identical.

6.3 The Frobenius map and the freshman's dream

A recurring phenomenon of prime characteristic is that the p -th power map respects addition.

Lemma 6.4 (Freshman's dream). Let R be a commutative ring of prime characteristic p . Then for

all $a, b \in R$,

$$(a + b)^p = a^p + b^p, \quad \text{and more generally} \quad (a + b)^{p^n} = a^{p^n} + b^{p^n}.$$

Proof. The binomial theorem gives $(a + b)^p = \sum_{i=0}^p \binom{p}{i} a^i b^{p-i}$. For $0 < i < p$, the integer $\binom{p}{i} = \frac{p!}{i!(p-i)!}$ is divisible by p : the numerator contains the factor p , while the denominator, being a product of integers all smaller than p , is coprime to p . Since $p \cdot 1_R = 0$, every middle term vanishes in R , leaving $(a + b)^p = a^p + b^p$. The general case follows by induction on n , applying the case $n = 1$ to $a^{p^{n-1}}$ and $b^{p^{n-1}}$. $\square$

Definition 6.5 (Frobenius endomorphism). Let F be a field of characteristic p . The *Frobenius map* is

$$\text{Frob} : F \rightarrow F, \quad \text{Frob}(x) = x^p.$$

Proposition 6.6. *The Frobenius map of a field F of characteristic p is a ring homomorphism fixing the prime subfield $\mathbb{F}_p$ pointwise. If F is finite it is an automorphism of F .*

Proof. It preserves products since $(xy)^p = x^p y^p$ by commutativity, and sums by Lemma 6.4; it sends $1 \mapsto 1$. Thus it is a ring homomorphism, and any ring homomorphism of fields is injective: if $x \neq 0$ lay in the kernel, then $1 = x^{-1}x$ would map to 0, contradicting $1 \mapsto 1$. On $\mathbb{F}_p$, Fermat's little theorem (Corollary 3.30) gives $x^p = x$ for all x , so Frob fixes $\mathbb{F}_p$. When F is finite, an injective self-map of a finite set is bijective, so Frob is an automorphism. $\square$

6.4 The multiplicative group of a finite field is cyclic

For a field F write $F^\times = F \setminus \{0\}$ for its multiplicative group. When F is finite, $F^\times$ has a strikingly simple structure: it is cyclic, that is, generated by a single element. This fact underlies discrete-logarithm cryptography and the construction of finite fields with prescribed roots of unity used in Halo 2. We prove it in the natural generality of an arbitrary finite subgroup of $F^\times$; the proof rests on the root bound for polynomials (Theorem 5.18), the order theory of section 3, and one counting identity for Euler's totient φ (the order of $(\mathbb{Z}/n\mathbb{Z})^\times$, Proposition 2.30).

Lemma 6.7 (Gauss's divisor-sum identity). *For every $n \geq 1$, $\sum_{d|n} \varphi(d) = n$.*

Proof. Partition $\{1, 2, \dots, n\}$ according to the value of $\gcd(a, n)$. For a fixed divisor $d \mid n$, the integers a with $\gcd(a, n) = d$ are exactly $a = db$ with $1 \leq b \leq n/d$ and $\gcd(b, n/d) = 1$; there are $\varphi(n/d)$ of these. Hence $n = \sum_{d|n} \varphi(n/d)$, and as d ranges over divisors of n so does n/d , giving $\sum_{d|n} \varphi(d) = n$. $\square$

Theorem 6.8 (Finite subgroups of $F^\times$ are cyclic). *Let F be any field and let $G \leq F^\times$ be a finite subgroup of order N . Then G is cyclic. In particular, for a finite field F with q elements, $F^\times$ is cyclic of order $q - 1$.*

Proof. For each divisor $d \mid N$, let $\psi(d)$ be the number of elements of G of order exactly d . Every element's order divides N (Corollary 3.28), so

$$\sum_{d|N} \psi(d) = N.$$

We claim that for each $d \mid N$, either $\psi(d) = 0$ or $\psi(d) = \varphi(d)$. Suppose $\psi(d) > 0$, so some $a \in G$ has order d . The d distinct powers $a^0, a^1, \dots, a^{d-1}$ are all roots of $X^d - 1$ in F . But $X^d - 1$ has at most d roots (Theorem 5.18), so these powers are *all* the roots, and every element of G of order d is among them. Among the powers a^j , those of order exactly d are the a^j with $\gcd(j, d) = 1$ by Corollary 3.21, and there are $\varphi(d)$ of them. Hence $\psi(d) = \varphi(d)$ whenever $\psi(d) > 0$.

Therefore $\psi(d) \leq \varphi(d)$ for all $d \mid N$. Summing and using Lemma 6.7,

$$N = \sum_{d \mid N} \psi(d) \leq \sum_{d \mid N} \varphi(d) = N.$$

Equality throughout forces $\psi(d) = \varphi(d)$ for every $d \mid N$. In particular $\psi(N) = \varphi(N) \geq 1$, so G contains an element of order $N = |G|$, which generates it. For a finite field, take $G = F^\times$, of order $q - 1$. $\square$

The count $\psi(d) = \varphi(d)$ established along the way is itself useful: a cyclic group of order N contains exactly $\varphi(d)$ elements of order d for each $d \mid N$, and in particular $\varphi(N)$ generators.

Definition 6.9 (Primitive root). We call a generator of $\mathbb{F}_q^\times$ a *primitive root* (or *primitive element*) of $\mathbb{F}_q$. Equivalently, g is primitive iff $\text{ord}(g) = q - 1$.

Example 6.10. In $\mathbb{F}_7^\times$, of order 6, test $g = 3$: the powers are $3^1 = 3$, $3^2 = 2$, $3^3 = 6$, $3^4 = 4$, $3^5 = 5$, $3^6 = 1 \pmod{7}$, running through all of $\{1, \dots, 6\}$. So 3 is a primitive root. By contrast 2 has order 3 (2, 4, 1) and is not primitive. By Corollary 3.21 the remaining primitive roots are the powers 3^j with $\gcd(j, 6) = 1$, namely $3^1 = 3$ and $3^5 = 5$; indeed there are $\varphi(6) = 2$ of them.

Corollary 6.11 (Roots of unity). *Let F be a finite field with q elements. For every $n \geq 1$, the solutions of $x^n = 1$ in $F^\times$ number exactly $\gcd(n, q - 1)$, and they form the cyclic subgroup of that order. In particular, when $n \mid q - 1$ there are exactly n such solutions.*

Proof. Let g be a primitive root, so $F^\times = \langle g \rangle$ is cyclic of order $q - 1$. Writing $x = g^j$, the equation $x^n = 1$ becomes $g^{jn} = 1$, i.e. $(q - 1) \mid jn$, i.e. $\frac{q-1}{d} \mid j$ where $d = \gcd(n, q - 1)$. The values $j \in \{0, 1, \dots, q - 2\}$ satisfying this are $j = \frac{q-1}{d} \cdot t$ for $t = 0, \dots, d - 1$, exactly d of them, and they form the cyclic subgroup generated by $g^{(q-1)/d}$. When $n \mid q - 1$ we have $d = n$. $\square$

This corollary is precisely what renders a curve such as Pallas or Vesta suitable for Halo 2: one chooses q so that $q - 1$ is divisible by a large power of 2, guaranteeing a multiplicative subgroup of 2-power order on which the number-theoretic transform, and hence efficient polynomial arithmetic, operates.

6.5 Prime fields in elliptic-curve cryptography

This section closes by indicating how the prime field $\mathbb{F}_p$ — the simplest finite field — enters elliptic-curve cryptography, the setting of Halo 2 and Zcash Orchard. We develop the geometry of elliptic curves in full later; here we only locate the relevant fields.

An elliptic curve over $\mathbb{F}_p$ (with $p > 3$) typically takes the short Weierstrass form, the set of solutions

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p \times \mathbb{F}_p : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

together with a distinguished point $\mathcal{O}$ at infinity, where $a, b \in \mathbb{F}_p$ and $4a^3 + 27b^2 \neq 0$. All the arithmetic defining the group law — the computation of slopes $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$, squaring, and the inversions required to add and double points — proceeds in $\mathbb{F}_p$ using exactly the operations of Subsection 6.1. We call this $\mathbb{F}_p$ the *base field* (or *coordinate field*) of the curve.

The set $E(\mathbb{F}_p)$ forms a finite abelian group under the chord-and-tangent addition law, of order $N = |E(\mathbb{F}_p)|$ close to p (the Hasse bound, Theorem 10.15). For cryptography one selects a curve whose group has a large prime factor r ; the subgroup of order r is where keys and signatures reside. The scalars acting on this subgroup live modulo r , and because r is prime, these scalars themselves form a field.

Definition 6.12 (Base field and scalar field). For an elliptic curve used in cryptography with a prime-order subgroup of order r :

- the *base field* $\mathbb{F}_p$ is the field over which the curve is defined and in which point coordinates lie;
- the *scalar field* $\mathbb{F}_r = \mathbb{Z}/r\mathbb{Z}$ is the field of scalars by which one multiplies points, i.e. the ring of integers modulo the subgroup order r .

Both are prime fields, but in general $p \neq r$.

Remark 6.13 (Two fields, and the Pasta cycle). Scalar multiplication $P \mapsto sP$ on a point P of order r depends only on $s \bmod r$, because $rP = \mathcal{O}$; the exponents thus lie in $\mathbb{F}_r$, while the coordinates of sP lie in $\mathbb{F}_p$. A typical elliptic-curve operation therefore manipulates two distinct prime fields simultaneously. Halo 2’s Pallas and Vesta curves (the “Pasta” cycle) exploit this deliberately: they form a *cycle of curves*, meaning the base field of one is the scalar field of the other and vice versa,

$$|E_{\text{Pallas}}| = p_{\text{Vesta}}, \quad |E_{\text{Vesta}}| = p_{\text{Pallas}},$$

so that the scalar field of Pallas equals the base field of Vesta. This permits a proof system to express the verifier of one curve’s arithmetic efficiently inside the native field of the other, enabling the recursive proof composition at the heart of Halo 2. One chooses both p_{Pallas} and p_{Vesta} to be $\equiv 1 \pmod{2^{32}}$, so that by Corollary 6.11 each field has a multiplicative subgroup of order 2^{32} , furnishing the high-order roots of unity needed for fast (number-theoretic-transform-based) polynomial multiplication.

Remark 6.14 (The role of cyclicity and Frobenius in cryptography). Two structural facts from this section recur throughout the cryptographic development. First, Theorem 6.8 (cyclicity of $\mathbb{F}_q^\times$) underlies the discrete-logarithm problem and guarantees the existence of the roots of unity used in polynomial commitment schemes. Second, the Frobenius endomorphism (Definition 6.5) is not merely an abstraction: on elliptic curves over $\mathbb{F}_p$ it induces an endomorphism whose characteristic polynomial $T^2 - tT + p$ encodes the curve order $N = p + 1 - t$ via the Hasse trace t , tying the count $|E(\mathbb{F}_p)|$ directly to the field-theoretic Frobenius studied here. We take up these connections in section 10.

7 Number theory for cryptography

Modern public-key cryptography resides almost entirely inside finite cyclic groups and finite fields, and the security of the schemes used in Halo 2 and Zcash Orchard rests on number-theoretic facts about such structures. This section assembles the classical number theory of the integers modulo n that the later volumes use: the Chinese Remainder Theorem and the computation of Euler's totient, Fermat's little theorem and Euler's generalisation in their role as workhorses of modular exponentiation, primitive roots, the theory of quadratic residues with the Legendre symbol and the Euler criterion, algorithms for extracting square roots modulo a prime, and finally the discrete logarithm problem and the reason the *prime order* of a group is so important for cryptographic hardness.

Throughout, n and m denote positive integers and p, q denote primes. The section builds directly on the arithmetic of section 2, the group theory of section 3, and the cyclicity theorem of section 6; several statements proved there reappear here, restated in arithmetic language, with proofs reduced to citations.

7.1 The integers modulo n and units

Recall from section 2 that $a \in \mathbb{Z}/n\mathbb{Z}$ is a unit if and only if $\gcd(a, n) = 1$ (Theorem 2.27), and that the units form a finite abelian group $(\mathbb{Z}/n\mathbb{Z})^\times$ under multiplication (Proposition 2.30). In particular, when p is prime every nonzero residue is coprime to p , so $\mathbb{F}_p^\times = \{1, 2, \dots, p-1\}$ is a group of order $p-1$ and $\mathbb{F}_p$ is a field (Corollary 2.28).

7.2 Euler's totient function

Definition 7.1 (Euler totient). *Euler's totient function $\varphi(n)$ is the number of integers a with $1 \leq a \leq n$ and $\gcd(a, n) = 1$. Equivalently, $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|$.*

Example 7.2. We have $\varphi(1) = 1$, $\varphi(p) = p - 1$ for prime p , and $\varphi(12) = |\{1, 5, 7, 11\}| = 4$.

Computing φ requires two facts: its value on prime powers, and its behaviour on coprime factors.

Proposition 7.3 (Totient of a prime power). *For a prime p and integer $k \geq 1$, $\varphi(p^k) = p^k - p^{k-1} = p^k \left(1 - \frac{1}{p}\right)$.*

Proof. Among $1, \dots, p^k$ the integers *not* coprime to p^k are exactly the multiples of p , namely $p, 2p, \dots, p^{k-1} \cdot p$, of which there are p^{k-1} . Subtracting gives $p^k - p^{k-1}$. $\square$

Theorem 7.4 (Chinese Remainder Theorem). *If $\gcd(m, n) = 1$, then the map*

$$\mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \quad a \bmod mn \longmapsto (a \bmod m, a \bmod n)$$

is an isomorphism of rings. It restricts to a group isomorphism $(\mathbb{Z}/mn\mathbb{Z})^\times \cong (\mathbb{Z}/m\mathbb{Z})^\times \times (\mathbb{Z}/n\mathbb{Z})^\times$.

Proof. The map is a well-defined ring homomorphism since reduction modulo m and modulo n are. It is injective: if $a \equiv 0 \pmod{m}$ and $a \equiv 0 \pmod{n}$ with $\gcd(m, n) = 1$, then $mn \mid a$, so $a \equiv 0 \pmod{mn}$; thus the kernel is trivial. Both sides have mn elements, so injectivity forces bijectivity. A ring isomorphism carries units to units, and (u, v) is a unit of the product ring iff u, v are units in their factors; this gives the multiplicative statement. $\square$

Corollary 7.5 (Multiplicativity and the product formula). *If $\gcd(m, n) = 1$ then $\varphi(mn) = \varphi(m)\varphi(n)$. Hence if $n = \prod_i p_i^{k_i}$ is the prime factorisation of n , then*

$$\varphi(n) = \prod_i (p_i^{k_i} - p_i^{k_i-1}) = n \prod_{p|n} \left(1 - \frac{1}{p}\right).$$

Proof. The cardinality of a product of groups is the product of cardinalities, so Theorem 7.4 gives $\varphi(mn) = \varphi(m)\varphi(n)$. Applying this across the distinct prime powers in the factorisation and using Proposition 7.3 yields the product formula. $\square$

Gauss's divisor-sum identity $\sum_{d|n} \varphi(d) = n$, proved as Lemma 6.7, completes the totient's basic theory.

7.3 Fermat's little theorem and Euler's theorem

The statements of this subsection were proved as Corollary 3.30, as instances of Lagrange's theorem; we restate them here in the arithmetic form in which cryptography uses them.

Theorem 7.6 (Euler's theorem). *If $\gcd(a, n) = 1$, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Proof. The hypothesis says $a \bmod n \in (\mathbb{Z}/n\mathbb{Z})^\times$, a finite group of order $\varphi(n)$; now apply Corollary 3.29 (every element of a finite group of order N satisfies $g^N = e$), a consequence of Lagrange's theorem. $\square$

Corollary 7.7 (Fermat's little theorem). *Let p be prime. If $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$. For every integer a , $a^p \equiv a \pmod{p}$.*

Proof. Take $n = p$ in Theorem 7.6, using $\varphi(p) = p-1$, to get the first statement. Multiplying $a^{p-1} \equiv 1$ by a yields $a^p \equiv a$ when $p \nmid a$; and when $p \mid a$ both sides are $\equiv 0$, so $a^p \equiv a \pmod{p}$ in all cases. $\square$

Remark 7.8. Euler's theorem underlies RSA: if $ed \equiv 1 \pmod{\varphi(n)}$ then $(a^e)^d = a^{1+k\varphi(n)} \equiv a \pmod{n}$ whenever $\gcd(a, n) = 1$. For the elliptic-curve cryptography of Orchard one instead exploits Fermat's little theorem in the field $\mathbb{F}_p$: it shows every nonzero element satisfies $x^{p-1} = 1$, so one can compute inversion as $x^{-1} = x^{p-2}$, and it underlies the Euler criterion for square roots developed below.

7.4 Multiplicative order and primitive roots

Definition 7.9 (Multiplicative order). Let $\gcd(a, n) = 1$. The *multiplicative order* of a modulo n , written $\text{ord}_n(a)$, is the least positive integer k with $a^k \equiv 1 \pmod{n}$. (Such a k exists because $a^{\varphi(n)} \equiv 1$ by Theorem 7.6.)

This is the order of a as an element of the group $(\mathbb{Z}/n\mathbb{Z})^\times$, so the order theory of section 3 applies verbatim: $a^m \equiv 1 \pmod{n}$ if and only if $\text{ord}_n(a) \mid m$ (Proposition 3.10) — in particular $\text{ord}_n(a) \mid \varphi(n)$ by Euler's theorem — and $\text{ord}_n(a^j) = \text{ord}_n(a) / \gcd(j, \text{ord}_n(a))$ (Corollary 3.21).

Definition 7.10 (Primitive root). A *primitive root* modulo n is an element g with $\text{ord}_n(g) = \varphi(n)$; equivalently, g generates $(\mathbb{Z}/n\mathbb{Z})^\times$ as a cyclic group, so that $\{g^0, g^1, \dots, g^{\varphi(n)-1}\}$ lists all the units.

Primitive roots need not exist for every n (for instance $(\mathbb{Z}/8\mathbb{Z})^\times \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ has no element of order 4). The decisive case for cryptography is $n = p$ prime, where the multiplicative group is always cyclic.

Corollary 7.11 ($\mathbb{F}_p^\times$ is cyclic). *For every prime p , the group $\mathbb{F}_p^\times = (\mathbb{Z}/p\mathbb{Z})^\times$ is cyclic of order $p - 1$. In particular primitive roots modulo p exist, and there are $\varphi(p - 1)$ of them; Example 6.10 exhibits them for $p = 7$.*

Proof. $\mathbb{F}_p^\times$ is a finite subgroup (of order $p - 1$) of the multiplicative group of the field $\mathbb{F}_p$, hence cyclic by Theorem 6.8; the generator count is $\varphi(p - 1)$ by Corollary 3.21. $\square$

Remark 7.12. Cyclicity is why the prime-order groups used in elliptic-curve cryptography behave like $\mathbb{Z}/q\mathbb{Z}$ under addition once one fixes a generator: a cyclic group of order q is isomorphic to $(\mathbb{Z}/q\mathbb{Z}, +)$ via $g^k \leftrightarrow k$ (Theorem 3.40). The full classification, which we do not need, states that primitive roots modulo n exist exactly when $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p .

7.5 Quadratic residues and the Euler criterion

We now turn to squares modulo a prime, which control whether a curve point with a given x -coordinate exists, and which we need to compute square roots in $\mathbb{F}_p$.

Definition 7.13 (Quadratic residue). Let p be an odd prime and a an integer with $p \nmid a$. We say a is a *quadratic residue* modulo p (a QR) if the congruence $x^2 \equiv a \pmod{p}$ has a solution, and a *quadratic non-residue* (a QNR) otherwise.

Proposition 7.14 (Exactly half are residues). *For an odd prime p , exactly $\frac{p-1}{2}$ of the nonzero residues modulo p are quadratic residues and $\frac{p-1}{2}$ are non-residues. The squaring map $x \mapsto x^2$ is exactly two-to-one on $\mathbb{F}_p^\times$.*

Proof. Consider $\sigma : \mathbb{F}_p^\times \rightarrow \mathbb{F}_p^\times$, $\sigma(x) = x^2$. Its image is exactly the set of QRs. For b in the image, $\sigma(x) = \sigma(y)$ iff $x^2 = y^2$ iff $(x - y)(x + y) = 0$ iff $y = \pm x$; since p is odd, $x \neq -x$, so each fibre has exactly two elements. Hence $|\text{im } \sigma| = \frac{p-1}{2}$, and the complement has the same size. $\square$

Theorem 7.15 (Euler's criterion). *Let p be an odd prime and $p \nmid a$. Then*

$$a^{\frac{p-1}{2}} \equiv \begin{cases} 1 \pmod{p} & \text{if } a \text{ is a quadratic residue,} \\ -1 \pmod{p} & \text{if } a \text{ is a quadratic non-residue.} \end{cases}$$

Proof. By Fermat, $a^{p-1} \equiv 1$, so $(a^{(p-1)/2})^2 \equiv 1$, and since the only square roots of 1 modulo a prime are ± 1 , we have $a^{(p-1)/2} \equiv \pm 1 \pmod{p}$.

If a is a QR, say $a \equiv x^2$, then $a^{(p-1)/2} \equiv x^{p-1} \equiv 1$ by Fermat. Thus all $\frac{p-1}{2}$ quadratic residues are roots of the polynomial $t^{(p-1)/2} - 1$ over $\mathbb{F}_p$. By Theorem 5.18 this polynomial has at most $\frac{p-1}{2}$ roots, so the QRs are *exactly* its roots. Any QNR a is therefore not a root, yet still satisfies $a^{(p-1)/2} = \pm 1$; hence $a^{(p-1)/2} \equiv -1$. $\square$

The standard notation for quadratic-residue status is the Legendre symbol.

Definition 7.16 (Legendre symbol). For an odd prime p and an integer a , the *Legendre symbol* is

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{if } p \mid a, \\ 1 & \text{if } a \text{ is a quadratic residue modulo } p, \\ -1 & \text{if } a \text{ is a quadratic non-residue modulo } p. \end{cases}$$

Euler's criterion (Theorem 7.15) then reads, uniformly,

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p},$$

so the symbol is computable by one modular exponentiation. The criterion also shows the symbol is *completely multiplicative* in a : $\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$, since $(ab)^{(p-1)/2} = a^{(p-1)/2}b^{(p-1)/2}$.

Example 7.17 (13 is a non-square in both Pasta fields). The Pasta hash-to-curve construction used by Orchard fixes $Z = -13$ as a verifiably non-square constant in both base fields. Since $p \equiv q \equiv 1 \pmod{4}$, the element -1 is a square in both fields, so -13 is a non-square precisely when 13 is. We verify the latter here. Let p, q be the Pasta primes of section 10. Both satisfy $p \equiv q \equiv 1 \pmod{4}$, and $13 \equiv 1 \pmod{4}$, so the law of quadratic reciprocity — which for two odd primes, at least one $\equiv 1 \pmod{4}$, states that each is a square modulo the other or neither is — gives $\left(\frac{13}{p}\right) = \left(\frac{p}{13}\right)$ and $\left(\frac{13}{q}\right) = \left(\frac{q}{13}\right)$. Reducing the primes modulo 13 and comparing against the squares modulo 13, namely $\{1, 3, 4, 9, 10, 12\}$:

$$p \equiv 5 \pmod{13}, \quad q \equiv 8 \pmod{13}, \quad 5, 8 \notin \{1, 3, 4, 9, 10, 12\},$$

whence

$$\left(\frac{13}{p}\right) = \left(\frac{5}{13}\right) = -1, \quad \left(\frac{13}{q}\right) = \left(\frac{8}{13}\right) = -1.$$

Thus 13 is a quadratic non-residue modulo both p and q . (One can avoid reciprocity entirely by evaluating $13^{(p-1)/2} \pmod{p}$ and $13^{(q-1)/2} \pmod{q}$ directly via Euler's criterion; both equal -1 .)

7.6 Square roots modulo a prime

Deciding that a is a residue is one matter; *producing* an x with $x^2 \equiv a$ is another, and we need it, for instance, to recover a curve point's y -coordinate from its x -coordinate (point decompression). The difficulty depends on $p \pmod{4}$.

Proposition 7.18 (The $p \equiv 3 \pmod{4}$ shortcut). *Let $p \equiv 3 \pmod{4}$ be prime and let a be a quadratic residue modulo p with $p \nmid a$. Then*

$$x \equiv a^{\frac{p+1}{4}} \pmod{p}$$

is a square root of a ; the two square roots are $\pm x$.

Proof. Since $p \equiv 3 \pmod{4}$, $\frac{p+1}{4}$ is a positive integer. Compute

$$x^2 = a^{\frac{p+1}{2}} = a^{\frac{p-1}{2}} \cdot a \equiv \left(\frac{a}{p}\right) \cdot a \equiv 1 \cdot a = a \pmod{p},$$

using Euler's criterion and the assumption that a is a residue, so $\left(\frac{a}{p}\right) = 1$. The two roots are $\pm x$ by Proposition 7.14. $\square$

Remark 7.19. The single-exponentiation shortcut requires $p \equiv 3 \pmod{4}$, which is one reason cryptographers often favour such primes. For $p \equiv 5 \pmod{8}$ a closed-form variant (Atkin’s formula) still exists. In general the cost of square-root extraction grows with the 2-adic valuation s of $p - 1$: writing $p - 1 = 2^s d$ with d odd, the Tonelli–Shanks algorithm below performs up to s corrective iterations. The Pasta primes have $s = 32$ (chosen deliberately for FFT-friendliness, theorem 9.10), so their implementations use Tonelli–Shanks.

When $p \equiv 1 \pmod{4}$ one writes $p - 1 = 2^s d$ with d odd and $s \geq 2$. The Tonelli–Shanks algorithm operates inside the cyclic 2-power-torsion subgroup of $\mathbb{F}_p^\times$.

Theorem 7.20 (Tonelli–Shanks). *Let p be an odd prime, $p - 1 = 2^s d$ with d odd, and let a be a quadratic residue with $p \nmid a$. One can compute a square root of a modulo p using $O(s^2 + \log p)$ multiplications plus a few modular exponentiations, given any quadratic non-residue z modulo p .*

Sketch. Fix a non-residue z (found by trying random elements; by Proposition 7.14 a random element is a non-residue with probability $\frac{1}{2}$, and one checks $\left(\frac{z}{p}\right)$ by one modular exponentiation (Euler’s criterion, Definition 7.16), i.e. $O(\log p)$ modular multiplications). Set $c = z^d$; this generates the unique cyclic subgroup H of order 2^s in $\mathbb{F}_p^\times$. Indeed, by Euler’s criterion $z^{(p-1)/2} \equiv -1$, so $(z^d)^{2^{s-1}} = z^{(p-1)/2} = -1 \neq 1$ while $(z^d)^{2^s} = z^{p-1} = 1$; hence $\text{ord}_p(z^d) = 2^s$. Initialise

$$x = a^{\frac{d+1}{2}}, \quad t = a^d, \quad m = s.$$

Then $x^2 = a \cdot t$, so x is a square root of a up to the factor $t \in H$; moreover $t^{2^{s-1}} = a^{(p-1)/2} = 1$, so t lies in H and has 2-power order. The algorithm repeatedly drives t to 1: at each step find the least i with $t^{2^i} = 1$; then $b = c^{2^{m-i-1}}$ satisfies $b^2 \in H$ and replacing

$$x \leftarrow xb, \quad t \leftarrow tb^2, \quad c \leftarrow b^2, \quad m \leftarrow i$$

strictly decreases the order of t while preserving the invariant $x^2 = a t$. Since the order of t is a strictly decreasing power of 2, after at most s iterations $t = 1$ and $x^2 = a$. Correctness rests on the cyclic structure of H (Corollary 7.11); the running-time bound counts the at most s iterations, each costing $O(s)$ squarings to locate i . $\square$

Remark 7.21. In every case the square roots come in a pair $\pm x$ (Proposition 7.14), so to pin down a canonical root one fixes a convention, e.g. “choose the root whose least nonnegative representative is even”, as elliptic-curve point encodings do. If a is *not* a residue, all these algorithms detect failure (e.g. the candidate fails the final check $x^2 \equiv a$).

7.7 The discrete logarithm problem

The hardness underpinning elliptic-curve cryptography is the difficulty of *inverting* exponentiation in a finite cyclic group. We phrase it here for a generic cyclic group, since in Orchard the group is the group of points of an elliptic curve, written additively, of large prime order.

Definition 7.22 (Discrete logarithm problem). Let $G = \langle g \rangle$ be a finite cyclic group of order N , written multiplicatively, with generator g . Given $h \in G$, the *discrete logarithm* of h to base g , written $\log_g h$, is the unique residue $x \in \mathbb{Z}/N\mathbb{Z}$ with $g^x = h$. The *discrete logarithm problem* (DLP) is to compute x from (G, g, h) .

The map $x \mapsto g^x$ is a group isomorphism $(\mathbb{Z}/N\mathbb{Z}, +) \xrightarrow{\sim} (G, \cdot)$ (it is a homomorphism with trivial kernel because g has order N), so $\log_g$ is genuinely a well-defined inverse. Computing g^x from x is fast (repeated squaring, $O(\log N)$ operations); the DLP asks for the reverse, for which no efficient algorithm is known in well-chosen groups.

Remark 7.23 (Generic complexity and group choice). Two “generic” algorithms that operate in any group of order N , using only the group operation, solve the DLP in $O(\sqrt{N})$ time: *baby-step giant-step* (deterministic, $O(\sqrt{N})$ time and space) and *Pollard’s rho* (randomised, $O(\sqrt{N})$ time and $O(1)$ space). Shoup’s theorem shows $\Omega(\sqrt{N})$ is optimal for generic algorithms when N is prime. Thus to reach a λ -bit security level one requires $N \approx 2^{2\lambda}$; this is why the Pallas/Vesta scalar fields have ≈ 255 -bit prime order, targeting ≈ 128 -bit security. For the integers modulo p under multiplication, sub-exponential index-calculus attacks exist, which is why elliptic-curve groups (where no such attack is known) permit much smaller parameters.

The next result makes the phrase “well-chosen” precise: if the group order N is *composite* with small prime factors, the DLP reduces to several DLPs in small subgroups, and is therefore easy.

Theorem 7.24 (Pohlig–Hellman reduction). *Let $G = \langle g \rangle$ be cyclic of order $N = \prod_{i=1}^r q_i^{e_i}$. One can compute the discrete logarithm in G by solving the discrete logarithm in each subgroup of prime order q_i (a bounded number of times) and combining the answers with the Chinese Remainder Theorem. If the largest prime factor of N is $q_{\max}$, the total cost is*

$$O\left(\sum_i e_i (\log N + \sqrt{q_i})\right)$$

group operations; when N has a large prime factor the $O(\sqrt{q_{\max}})$ term dominates, whereas for smooth N — N a product of small primes—every term is polynomial in $\log N$ and the DLP is easy—the failure mode that Corollary 7.25 explains how to avoid.

Sketch. The reduction proceeds in two stages. (a) *CRT across coprime factors.* By the cyclic structure of G and the Chinese Remainder Theorem, G decomposes as the internal direct product of its subgroups G_i of prime-power order $q_i^{e_i}$ (the unique subgroups of those orders, Theorem 3.22). Knowing $x = \log_g h$ modulo each $q_i^{e_i}$ determines x modulo $N = \prod q_i^{e_i}$ uniquely, by the Chinese Remainder Theorem (Theorem 7.4, applied to the integers $\mathbb{Z}/N\mathbb{Z}$). To find $x \bmod q_i^{e_i}$ one projects: set $g_i = g^{N/q_i^{e_i}}$ and $h_i = h^{N/q_i^{e_i}}$; then g_i has order $q_i^{e_i}$ and $\log_{g_i} h_i \equiv x \pmod{q_i^{e_i}}$.

(b) *Lifting a prime power digit by digit.* Within a group of order q^e , write the unknown exponent in base q as $x \equiv x_0 + x_1 q + \dots + x_{e-1} q^{e-1} \pmod{q^e}$ with $0 \leq x_j < q$. Raising the relevant elements to the power q^{e-1} lands inside the order- q subgroup and isolates x_0 as a single discrete log in a group of order q ; subtracting off its contribution and repeating isolates x_1 , and so on. Each digit costs one DLP in the order- q subgroup, solvable in $O(\sqrt{q})$ by baby-step giant-step or Pollard rho, plus $O(\log N)$ for the exponentiations. Summing over the e_i digits of each of the r primes yields the stated bound. $\square$

Corollary 7.25 (Why prime order matters). *If N has a small prime factor q , then by Theorem 7.24 an attacker can recover the discrete logarithm modulo that factor in $O(\sqrt{q})$ time, leaking part of the secret x . Security therefore requires N to have a large prime factor; the safest and simplest choice is to make N itself a large prime, so that G has no proper nontrivial subgroups and Pohlig–Hellman offers no reduction at all. This is precisely why the elliptic-curve groups used in Halo 2 and Orchard have large prime order.*

Remark 7.26 (Cofactors and subgroup attacks). In practice an elliptic curve over $\mathbb{F}_p$ has order $h \cdot q$ where q is a large prime and h is a small *cofactor*; cryptographic operations stay confined to the prime-order- q subgroup. If protocols neglect to check that incoming points lie in this subgroup (or fail to clear the cofactor), an adversary can supply a low-order point and use Pohlig–Hellman in the small factor to extract information modulo h — a real-world “small-subgroup attack”. The Pasta curves of Halo 2 have cofactor $h = 1$, i.e. *prime* order, which eliminates this class of attacks at the group level.

This is the cryptographic payoff of all the number theory developed above: the multiplicative and additive structure of $\mathbb{Z}/q\mathbb{Z}$ for a large prime q is what gives the scheme both its functionality (a field of scalars) and its security (no Pohlig–Hellman reduction).

8 Linear algebra over a field

Linear algebra is the study of *vector spaces* — collections of objects that can be added together and scaled by elements of a field — together with the structure-preserving maps between them. It is the computational backbone of the cryptography this monograph develops: commitments to vectors, the linear constraints that encode arithmetic circuits, the polynomial-evaluation arguments at the heart of the inner-product argument, and the Pedersen-style multiscalar multiplications that Halo 2 and Orchard use are all, at bottom, statements about linear maps and (bi)linear forms over a finite field. This section develops the theory from the ground up, assuming only that the reader knows what a field is (a set $\mathbb{F}$ with addition and multiplication in which every nonzero element is invertible; section 4). The development then specialises to the two settings that matter here: the coordinate space $\mathbb{F}^n$ over a finite field $\mathbb{F} = \mathbb{F}_q$, and the “mixed” pairing between scalar vectors and vectors of group elements.

Throughout, $\mathbb{F}$ denotes an arbitrary field. We write its additive identity 0, its multiplicative identity 1, and call elements of $\mathbb{F}$ *scalars*. We use the field axioms freely: addition and multiplication are commutative and associative, multiplication distributes over addition, and every $a \neq 0$ has an inverse a^{-1} .

8.1 Vector spaces and subspaces

Definition 8.1 (Vector space). Let $\mathbb{F}$ be a field. A *vector space over $\mathbb{F}$* (or an $\mathbb{F}$ -*vector space*) is a set V together with two operations,

$$+ : V \times V \rightarrow V \quad (\text{vector addition}), \quad \cdot : \mathbb{F} \times V \rightarrow V \quad (\text{scalar multiplication}),$$

satisfying the following axioms for all $u, v, w \in V$ and all $a, b \in \mathbb{F}$. We call the elements of V *vectors*.

1. (*Additive abelian group.*) V is an abelian group under $+$: addition is associative and commutative, there is a zero vector $0 \in V$ with $v + 0 = v$, and every v has an additive inverse $-v$ with $v + (-v) = 0$.
2. (*Compatibility.*) $a \cdot (b \cdot v) = (ab) \cdot v$.
3. (*Identity.*) $1 \cdot v = v$.
4. (*Distributivity over vector addition.*) $a \cdot (u + v) = a \cdot u + a \cdot v$.
5. (*Distributivity over scalar addition.*) $(a + b) \cdot v = a \cdot v + b \cdot v$.

We usually write the scalar product $a \cdot v$ as av , and we drop the distinction between the scalar $0 \in \mathbb{F}$ and the vector $0 \in V$ when context makes it clear.

A few immediate consequences of the axioms recur so frequently that we record them once and for all.

Proposition 8.2. Let V be an $\mathbb{F}$ -vector space, $v \in V$, and $a \in \mathbb{F}$. Then:

1. $0 \cdot v = 0$;
2. $a \cdot 0 = 0$;
3. $(-1) \cdot v = -v$;
4. if $av = 0$ then $a = 0$ or $v = 0$.

Proof. (1) By distributivity over scalar addition, $0 \cdot v = (0 + 0) \cdot v = 0 \cdot v + 0 \cdot v$. Adding $-(0 \cdot v)$ to both sides gives $0 = 0 \cdot v$.

(2) Similarly $a \cdot 0 = a \cdot (0 + 0) = a \cdot 0 + a \cdot 0$, and cancellation gives $a \cdot 0 = 0$.

(3) Using (1) and distributivity, $v + (-1) \cdot v = 1 \cdot v + (-1) \cdot v = (1 + (-1)) \cdot v = 0 \cdot v = 0$, so $(-1) \cdot v$ is the additive inverse of v , i.e. $(-1) \cdot v = -v$.

(4) Suppose $av = 0$ and $a \neq 0$. Since $\mathbb{F}$ is a field, a^{-1} exists, and by the compatibility and identity axioms $v = 1 \cdot v = (a^{-1}a) \cdot v = a^{-1} \cdot (av) = a^{-1} \cdot 0 = 0$ using (2). $\square$

Example 8.3 (The coordinate space $\mathbb{F}^n$). The single most important example is the *coordinate space*

$$\mathbb{F}^n = \{(x_1, \dots, x_n) : x_i \in \mathbb{F}\},$$

the set of n -tuples of scalars, with componentwise operations

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n), \quad a \cdot (x_1, \dots, x_n) = (ax_1, \dots, ax_n).$$

The zero vector is $(0, \dots, 0)$. All the axioms follow from the field axioms applied coordinatewise. When $n = 0$ the result is the *trivial* (or *zero*) vector space $\{0\}$, consisting of the empty tuple alone. We typically write vectors of $\mathbb{F}^n$ as columns and in boldface, $\mathbf{x} = (x_1, \dots, x_n)^T$.

Example 8.4 (Further examples). 1. *Polynomials*. The set $\mathbb{F}[X]$ of all polynomials in one variable with coefficients in $\mathbb{F}$ is an $\mathbb{F}$ -vector space under ordinary addition and scalar multiplication. So is the subset $\mathbb{F}[X]_{<n}$ of polynomials of degree less than n ; this is exactly the setting in which polynomial commitment schemes operate.

2. *Functions*. For any set S , the set $\mathbb{F}^S$ of all functions $S \rightarrow \mathbb{F}$ is an $\mathbb{F}$ -vector space under pointwise operations. Example 8.3 is the special case $S = \{1, \dots, n\}$.

3. *Matrices*. The set $\mathbb{F}^{m \times n}$ of $m \times n$ matrices with entries in $\mathbb{F}$ is an $\mathbb{F}$ -vector space under entrywise addition and scaling.

4. *Field extensions*. If $\mathbb{F} \subseteq \mathbb{K}$ is a subfield of a larger field $\mathbb{K}$, then $\mathbb{K}$ is an $\mathbb{F}$ -vector space. For instance $\mathbb{C}$ is a 2-dimensional $\mathbb{R}$ -vector space, and the finite field $\mathbb{F}_q$ with $q = p^k$ is a k -dimensional $\mathbb{F}_p$ -vector space (Theorem 6.3 rested on exactly this).

A vector space often contains smaller vector spaces sitting inside it. These are the subspaces.

Definition 8.5 (Subspace). A subset $W \subseteq V$ of an $\mathbb{F}$ -vector space V is a (*linear*) *subspace* if it is itself a vector space under the operations inherited from V . Equivalently, W is a subspace iff it is nonempty and closed under the vector-space operations.

Proposition 8.6 (Subspace criterion). A subset $W \subseteq V$ is a subspace if and only if all three of the following hold:

1. $0 \in W$;
2. W is closed under addition: $u, v \in W \implies u + v \in W$;
3. W is closed under scalar multiplication: $a \in \mathbb{F}, v \in W \implies av \in W$.

Equivalently, $W \neq \emptyset$ and for all $a, b \in \mathbb{F}$ and $u, v \in W$ we have $au + bv \in W$.

Proof. If W is a subspace, the three conditions hold because they are part of (or follow from) the vector-space axioms restricted to W . Conversely, suppose (1)–(3) hold. Closure under $+$ and $\cdot$ makes

the operations well defined on W . The additive inverse of $v \in W$ is $(-1)v \in W$ by (3) and Proposition 8.2(3). The space V supplies the remaining axioms (associativity, commutativity, distributivity, etc.) to W , because they are universally quantified identities that hold for all elements of V , in particular for elements of W . The “equivalently” clause follows by taking $(a, b) = (1, 1)$ and $(a, b) = (a, 0)$ to recover (2) and (3), and noting $0 = 0 \cdot v$ for any v in the nonempty W . $\square$

Example 8.7. In $\mathbb{F}^3$, the set $\{(x_1, x_2, x_3) : x_1 + x_2 + x_3 = 0\}$ is a subspace (the solution set of a homogeneous linear equation), whereas $\{(x_1, x_2, x_3) : x_1 + x_2 + x_3 = 1\}$ is not, since it does not contain 0. More generally, the solution set of any homogeneous linear system $Ax = 0$ is a subspace of $\mathbb{F}^n$; we revisit this in the study of kernels.

Proposition 8.8 (Intersections and sums). *Let $\{W_i\}_{i \in I}$ be a family of subspaces of V .*

1. *The intersection $\bigcap_{i \in I} W_i$ is a subspace.*
2. *The sum $W_1 + \cdots + W_k = \{w_1 + \cdots + w_k : w_j \in W_j\}$ of finitely many subspaces is a subspace, and it is the smallest subspace containing all the W_j .*

In general the union of two subspaces is not a subspace.

Proof. (1) Each W_i contains 0, so the intersection does too. If u, v lie in every W_i and $a, b \in \mathbb{F}$, then $au + bv \in W_i$ for every i by Proposition 8.6, hence in the intersection.

(2) Closure: given $\sum_j w_j$ and $\sum_j w'_j$ in the sum and scalars a, b , we have $a \sum_j w_j + b \sum_j w'_j = \sum_j (aw_j + bw'_j)$ with $aw_j + bw'_j \in W_j$. It contains $0 = \sum_j 0$ and each W_j (take all other summands zero). Any subspace containing every W_j must, by closure under addition, contain all such sums; hence the sum is the smallest such subspace. An example shows the union claim: in $\mathbb{F}^2$ with $|\mathbb{F}| > 1$, the two axes $\{(x, 0)\}$ and $\{(0, y)\}$ are subspaces whose union omits $(1, 1) = (1, 0) + (0, 1)$. $\square$

8.2 Linear combinations, span, and linear independence

Definition 8.9 (Linear combination and span). Let $S \subseteq V$. A *linear combination* of vectors in S is any finite sum

$$a_1 v_1 + a_2 v_2 + \cdots + a_k v_k, \quad a_i \in \mathbb{F}, v_i \in S.$$

The *span* of S , written $\text{span}(S)$ or $\langle S \rangle$, is the set of all linear combinations of finitely many vectors in S . By convention, $\text{span}(\emptyset) = \{0\}$. S *spans* (or *generates*) V if $\text{span}(S) = V$, and V is *finitely generated* (or *finite-dimensional*) if some finite set spans it.

Proposition 8.10. *For any $S \subseteq V$, $\text{span}(S)$ is the smallest subspace of V containing S ; that is, it is a subspace, it contains S , and it is contained in every subspace that contains S .*

Proof. A sum of two linear combinations of vectors in S is again such a linear combination, as is a scalar multiple of one, and 0 is the empty combination; hence $\text{span}(S)$ is a subspace by Proposition 8.6, and $S \subseteq \text{span}(S)$. If W is any subspace containing S , then W contains every linear combination of elements of S (by closure under addition and scaling), so $\text{span}(S) \subseteq W$. $\square$

Definition 8.11 (Linear independence). A finite list of vectors $v_1, \dots, v_k \in V$ is *linearly independent* if the only expression of the zero vector as a linear combination of them is the trivial one:

$$a_1 v_1 + \cdots + a_k v_k = 0 \implies a_1 = \cdots = a_k = 0.$$

Otherwise the list is *linearly dependent*; we call a nontrivial relation $\sum_i a_i v_i = 0$ with some $a_i \neq 0$ a *dependence relation*. An arbitrary (possibly infinite) set S is linearly independent if every finite sublist of distinct vectors from S is independent.

Remark 8.12. Linear independence captures non-redundancy. A single vector v is independent iff $v \neq 0$. Two vectors are dependent iff one is a scalar multiple of the other. In general, a list is dependent iff some vector in it lies in the span of the others, as the next lemma makes precise.

Lemma 8.13 (Dependence and redundancy). *A list $v_1, \dots, v_k$ with $k \geq 1$ is linearly dependent if and only if some v_j is a linear combination of the others. Moreover, if $v_1, \dots, v_k$ are independent but $v_1, \dots, v_k, w$ are dependent, then $w \in \text{span}(v_1, \dots, v_k)$, and the coefficients expressing w are unique.*

Proof. If $v_j = \sum_{i \neq j} a_i v_i$, then $\sum_{i \neq j} a_i v_i + (-1)v_j = 0$ is a nontrivial relation. Conversely, given a dependence relation $\sum_i a_i v_i = 0$ with $a_j \neq 0$, one solves $v_j = -a_j^{-1} \sum_{i \neq j} a_i v_i$.

For the second statement, a dependence relation $\sum_i a_i v_i + bw = 0$ must have $b \neq 0$ (otherwise it would be a nontrivial relation among the independent v_i), so $w = -b^{-1} \sum_i a_i v_i \in \text{span}(v_1, \dots, v_k)$. Uniqueness: if $w = \sum_i c_i v_i = \sum_i c'_i v_i$, then $\sum_i (c_i - c'_i) v_i = 0$, forcing $c_i = c'_i$ by independence. $\square$

8.3 Bases, dimension, and linear maps

Definition 8.14 (Basis). A *basis* of V is a linearly independent list $b_1, \dots, b_n$ that spans V . By Lemma 8.13, every $v \in V$ is then a *unique* linear combination $v = \sum_i c_i b_i$; the scalars c_i are the *coordinates* of v in the basis, written $[v]_B = (c_1, \dots, c_n)$.

The standard basis $e_1, \dots, e_n$ of $\mathbb{F}^n$ (where e_i has a 1 in position i and 0 elsewhere) and the monomial basis $1, X, \dots, X^{n-1}$ of $\mathbb{F}[X]_{<n}$ are the two examples in constant use. That all bases of a given space have the same size rests on the following exchange principle.

Lemma 8.15 (Steinitz exchange). *If $v_1, \dots, v_m$ are linearly independent in V and $w_1, \dots, w_n$ span V , then $m \leq n$.*

Proof. We show by induction on $k = 0, 1, \dots, m$ that, after renumbering the w_j , the list $v_1, \dots, v_k, w_{k+1}, \dots, w_n$ spans V ; taking $k = m$ forces $m \leq n$, since the process consumes one w for each v . The case $k = 0$ is the hypothesis. For the step, the spanning property at stage k writes $v_{k+1} = \sum_{i \leq k} a_i v_i + \sum_{j > k} c_j w_j$. Some $c_j \neq 0$ — otherwise v_{k+1} would be a combination of $v_1, \dots, v_k$, contradicting independence — say $c_{k+1} \neq 0$ after renumbering. Solving for w_{k+1} expresses it through $v_1, \dots, v_{k+1}, w_{k+2}, \dots, w_n$, so replacing w_{k+1} by v_{k+1} preserves the span. $\square$

Theorem 8.16 (Dimension). *Let V be a finitely generated $\mathbb{F}$ -vector space. Then V has a basis, and any two bases have the same number of elements. We call this number the dimension $\dim_{\mathbb{F}} V$. In particular $\dim_{\mathbb{F}} \mathbb{F}^n = n$.*

Proof. A minimal spanning list (obtained from a finite spanning set by discarding redundant vectors, Lemma 8.13) is independent, hence a basis. If $b_1, \dots, b_m$ and $b'_1, \dots, b'_n$ are two bases, Lemma 8.15 applied in both directions gives $m \leq n$ and $n \leq m$. For $\mathbb{F}^n$, the standard basis has n elements. $\square$

Definition 8.17 (Linear map, kernel, image). A function $T : V \rightarrow W$ between $\mathbb{F}$ -vector spaces is

linear if $T(au + bv) = aT(u) + bT(v)$ for all $a, b \in \mathbb{F}$ and $u, v \in V$. Its *kernel* and *image* are

$$\ker T = \{v \in V : T(v) = 0\} \subseteq V, \quad \text{im } T = \{T(v) : v \in V\} \subseteq W;$$

both are subspaces (immediate from the subspace criterion and linearity). T is injective iff $\ker T = \{0\}$: if $T(u) = T(v)$ then $u - v \in \ker T$, and conversely. A bijective linear map is an *isomorphism*.

Theorem 8.18 (Rank–nullity). *Let $T : V \rightarrow W$ be linear with V finite-dimensional. Then*

$$\dim V = \dim \ker T + \dim \text{im } T.$$

Proof. Choose a basis $u_1, \dots, u_k$ of $\ker T$ and extend it to a basis $u_1, \dots, u_k, v_1, \dots, v_r$ of V (adjoin vectors outside the current span until the list spans; each adjunction preserves independence by Lemma 8.13, and the process stops by Lemma 8.15). We claim $T(v_1), \dots, T(v_r)$ is a basis of $\text{im } T$, which yields $\dim V = k + r = \dim \ker T + \dim \text{im } T$. *Spanning:* any $T(v)$ with $v = \sum_i a_i u_i + \sum_j c_j v_j$ equals $\sum_j c_j T(v_j)$, since $T(u_i) = 0$. *Independence:* if $\sum_j c_j T(v_j) = 0$, then $\sum_j c_j v_j \in \ker T$, so $\sum_j c_j v_j = \sum_i a_i u_i$ for some scalars a_i ; independence of the combined basis of V forces all $c_j = 0$. $\square$

Rank–nullity is the dimension-counting engine of the section: it shows, for instance, that a linear map from a higher-dimensional space into a lower-dimensional one must have a nontrivial kernel — the fact that makes vector commitments compressing and only *computationally* binding (Remark 8.26).

8.4 Bilinear forms and inner products on $\mathbb{F}^n$

The maps considered so far are linear in a single argument. Commitment and proof systems are pervaded by expressions that are linear in *each* of two arguments — most importantly the inner product. We treat these next. Because the fields of interest are finite, there is no notion of “positive length” or “angle”; the appropriate level of generality is the *bilinear form*.

Definition 8.19 (Bilinear form). A *bilinear form* on an $\mathbb{F}$ -vector space V is a function $\beta : V \times V \rightarrow \mathbb{F}$ that is linear in each argument separately:

$$\beta(au + a'u', v) = a\beta(u, v) + a'\beta(u', v), \quad \beta(u, bv + b'v') = b\beta(u, v) + b'\beta(u, v').$$

It is *symmetric* if $\beta(u, v) = \beta(v, u)$ for all u, v , and *nondegenerate* if $\beta(u, v) = 0$ for all v implies $u = 0$.

Relative to a fixed basis $b_1, \dots, b_n$ of V , every bilinear form β is represented by the unique matrix $M \in \mathbb{F}^{n \times n}$ with $M_{ij} = \beta(b_i, b_j)$, via $\beta(u, v) = [u]_B^T M [v]_B$; we do not use this representation below and record only the special case that matters.

Definition 8.20 (Standard inner product on $\mathbb{F}^n$). The *standard inner product* (or *dot product*) on $\mathbb{F}^n$ is the symmetric bilinear form

$$\langle a, b \rangle = \sum_{i=1}^n a_i b_i = a^T b, \quad a = (a_1, \dots, a_n), \quad b = (b_1, \dots, b_n).$$

Its matrix in the standard basis is the identity I_n ; it is symmetric and nondegenerate.

Remark 8.21 (No positivity over finite fields). Over $\mathbb{R}$ the standard inner product is *positive definite*: $\langle a, a \rangle = \sum a_i^2 \geq 0$, vanishing only at 0, which yields the Euclidean norm $\|a\| = \sqrt{\langle a, a \rangle}$ and the

Cauchy–Schwarz inequality. Over a finite field $\mathbb{F}_q$ there is no ordering, so “positivity” is meaningless: one may have $a \neq 0$ with $\langle a, a \rangle = 0$ (an *isotropic* vector). For example over $\mathbb{F}_5$, $a = (1, 2)$ gives $1 + 4 = 5 \equiv 0$. The algebraically robust properties that survive are bilinearity, symmetry, and nondegeneracy; these are all that the arguments in proof systems rely on.

Remark 8.22 (Functionals via the inner product). Nondegeneracy yields a clean dictionary between vectors and linear functionals. Each $a \in \mathbb{F}^n$ gives a functional $b \mapsto \langle a, b \rangle$, and the map $a \mapsto \langle a, \cdot \rangle$ is an isomorphism from $\mathbb{F}^n$ onto its *dual space* $\text{Hom}(\mathbb{F}^n, \mathbb{F})$ (both have dimension n , and the map is injective by nondegeneracy). Thus “a linear functional on $\mathbb{F}^n$ ” and “a vector in $\mathbb{F}^n$ ” are interchangeable. Evaluating a committed polynomial p of coefficient vector c at a point z is the inner product $\langle c, (1, z, z^2, \dots, z^{n-1}) \rangle$; this is precisely the shape an inner-product argument proves.

8.5 The mixed inner product with group elements

The commitments used in Halo 2 and Orchard pair a vector of *scalars* with a vector of *group elements*. We isolate the algebraic structure here; the section on elliptic curves introduces the relevant group fully, but only its abelian-group structure matters at present.

Let $(\mathbb{G}, +)$ be a finite abelian group, written additively, of prime order r with identity $\mathcal{O}$. For an integer a and $G \in \mathbb{G}$ write $[a]G$ for the a -fold sum $G + \dots + G$ (and $[0]G = \mathcal{O}$, $[-a]G = -([a]G)$; this is the *scalar multiplication* of the group). Because $\mathbb{G}$ has order r with r prime, $[a]G$ depends only on $a \bmod r$, so the operation descends to scalars from the field $\mathbb{F}_r = \mathbb{Z}/r\mathbb{Z}$: for $a \in \mathbb{F}_r$ and $G \in \mathbb{G}$, $[a]G$ is well defined. With this action $\mathbb{G}$ becomes a one-dimensional vector space over $\mathbb{F}_r$, whenever $\mathbb{G}$ is nontrivial — a viewpoint we exploit repeatedly.

Proposition 8.23 ($\mathbb{G}$ as an $\mathbb{F}_r$ -vector space). *Let $\mathbb{G}$ be a group of prime order r . Then the scalar multiplication $(a, G) \mapsto [a]G$ makes $\mathbb{G}$ into a vector space over $\mathbb{F}_r$ of dimension 1; any nonidentity element G is a basis, and every element is uniquely $[a]G$ for some $a \in \mathbb{F}_r$.*

Proof. The vector-space axioms are the standard laws of group scalar multiplication: $[a]([b]G) = [ab]G$, $[1]G = G$, $[a](G + H) = [a]G + [a]H$, and $[a + b]G = [a]G + [b]G$, all of which hold in any abelian group and respect reduction mod r . For a nonidentity G , Lagrange’s theorem in a group of prime order r forces G to have order r , so the r multiples $[0]G, [1]G, \dots, [r-1]G$ are distinct and exhaust $\mathbb{G}$ (which has r elements). Hence $\{G\}$ spans and is independent ($[a]G = \mathcal{O}$ iff $a \equiv 0$), a basis; uniqueness of a is unique representation in this basis. $\square$

Definition 8.24 (Mixed inner product / multiscalar multiplication). Let $G = (G_1, \dots, G_n) \in \mathbb{G}^n$ be a fixed tuple of group elements and $a = (a_1, \dots, a_n) \in \mathbb{F}_r^n$ a vector of scalars. Their *mixed inner product* (also called a *multiscalar multiplication*, MSM, or *Pedersen-style inner product*) is the group element

$$\langle a, G \rangle = \sum_{i=1}^n [a_i] G_i \in \mathbb{G}.$$

We use the same angle-bracket notation as for the scalar inner product; whether the second argument is a scalar vector or a vector of group elements determines which one we mean.

Proposition 8.25 (Bilinearity of the mixed product). *Fix $G \in \mathbb{G}^n$. The map $a \mapsto \langle a, G \rangle$ is $\mathbb{F}_r$ -linear from $\mathbb{F}_r^n$ to $\mathbb{G}$:*

$$\langle c a + c' a', G \rangle = [c] \langle a, G \rangle + [c'] \langle a', G \rangle.$$

Symmetrically, for fixed a the map $G \mapsto \langle a, G \rangle$ is a group homomorphism $\mathbb{G}^n \rightarrow \mathbb{G}$ that is $\mathbb{F}_r$ -linear

when one views $\mathbb{G}^n$ as an $\mathbb{F}_r$ -vector space. Thus $\langle \cdot, \cdot \rangle : \mathbb{F}_r^n \times \mathbb{G}^n \rightarrow \mathbb{G}$ is bilinear over $\mathbb{F}_r$.

Proof. Using the vector-space laws on $\mathbb{G}$ (Proposition 8.23) componentwise,

$$\langle ca + c'a', G \rangle = \sum_i [ca_i + c'a'_i]G_i = \sum_i ([c][a_i]G_i + [c'] [a'_i]G_i) = [c] \sum_i [a_i]G_i + [c'] \sum_i [a'_i]G_i,$$

which is the first claim. Linearity in G is identical by symmetry of the defining sum and the law $[a](G + H) = [a]G + [a]H$. $\square$

Remark 8.26 (Why this is the right abstraction). The mixed inner product is the computational heart of Pedersen commitments and of the Halo 2 inner-product argument. Reading $\langle a, G \rangle$ as a *commitment* to the scalar vector a under the public “basis” G , bilinearity is exactly what endows the commitment with its *homomorphic* property: a commitment to $ca + c'a'$ is the same linear combination $[c]$ - and $[c']$ -scaled of the individual commitments. Provided one chooses the G_i so that the prover knows no nontrivial relation $\sum_i [a_i]G_i = \mathcal{O}$ (the discrete-logarithm hardness assumption, treated later), the commitment is binding: the map $a \mapsto \langle a, G \rangle$ is computationally injective. This map *cannot* be injective as a set map once $n \geq 2$, since by rank-nullity its kernel (a subspace of $\mathbb{F}_r^n$ mapping into the one-dimensional $\mathbb{G}$) has dimension at least $n - 1$; binding is therefore a *computational*, not information-theoretic, statement. A typical Pedersen commitment to a single value a with randomness ρ is the special case $[a]G + [\rho]H = \langle (a, \rho), (G, H) \rangle$, which is moreover *perfectly hiding* because the $[\rho]H$ term is uniform in $\mathbb{G}$.

8.6 How this machinery appears in commitment and proof systems

We now draw the threads together; each point below is developed rigorously in the later volumes of this series, but the linear-algebraic content is already in hand.

Remark 8.27 (Witnesses, constraints, and assignments). An arithmetic circuit over $\mathbb{F}_q$ is, after *arithmetisation*, a system of polynomial constraints in the wire values. Matrices over $\mathbb{F}_q$ acting on the assignment vector $z \in \mathbb{F}_q^N$ capture the *linear* part of such a system — the wiring/copy constraints of PLONK. Satisfying the linear constraints means z lies in an affine subspace; the remaining (multiplicative) constraints are handled separately. Rank-nullity (Theorem 8.18) quantifies the residual degrees of freedom, i.e. the dimension of the witness space.

Remark 8.28 (Commitments as linear maps). A Pedersen vector commitment $a \mapsto \langle a, G \rangle = \sum_i [a_i]G_i$ is a linear map $\mathbb{F}_r^n \rightarrow \mathbb{G}$ (Proposition 8.25). Its linearity is the source of every algebraic manipulation a verifier performs: it can take known linear combinations of commitments, “fold” two commitment keys G, G' into $G + [x]G'$ under a challenge x , and check claimed openings by re-evaluating the same linear map. A polynomial commitment is the same map applied to a coefficient vector, and an *evaluation* of that polynomial at z is the scalar inner product $\langle c, (1, z, \dots, z^{n-1}) \rangle$ (Remark 8.22).

Remark 8.29 (The inner-product relation). The core statement an inner-product argument proves has the shape: “I know vectors $a, b \in \mathbb{F}_r^n$ such that $P = \langle a, G \rangle + \langle b, H \rangle + [\langle a, b \rangle]U$ for public $G, H \in \mathbb{G}^n$ and $U \in \mathbb{G}$.” Here three of the constructions meet at once: two mixed inner products $\langle a, G \rangle, \langle b, H \rangle$ (commitments to the witness vectors), one scalar inner product $\langle a, b \rangle$ (the value being argued), and the bilinearity of all three (which permits the prover and verifier to fold length- n instances into length- $n/2$ ones). The argument’s soundness ultimately reduces to the nondegeneracy of these forms together with the hardness of finding kernel elements of the commitment map — that is, to exactly the linear-algebraic and computational facts assembled in this section.

Remark 8.30 (Summary). The objects constructed above — vector spaces, bases and dimension, linear maps with their kernels and images governed by rank-nullity, bilinear forms, and the two flavours of inner product (scalar-with-scalar and scalar-with-group) — constitute the linear-algebraic vocabulary needed for the cryptography ahead. Every commitment is a linear map; every opening is an inner product; every soundness argument rests on (non)degeneracy and dimension counting. The next section applies this machinery to the polynomial spaces $\mathbb{F}[X]_{<n}$ over evaluation domains, and the elliptic-curve section realises the group $\mathbb{G}$.

9 Roots of unity and evaluation domains

The polynomial-based proof systems underlying Halo 2 and Zcash Orchard operate over a single, highly particular kind of object: a finite multiplicative subgroup of a field, generated by a *root of unity*. Nearly every operation such a system performs—committing to a polynomial, checking that two polynomials agree, enforcing that a constraint vanishes everywhere it should, multiplying or interpolating efficiently—is ultimately a computation indexed by such a subgroup. This section develops the theory of these subgroups from first principles. We assume only the basic theory of fields, polynomials over a field, and finite cyclic groups, and we recall these as needed.

Throughout, $\mathbb{F}$ denotes a field. When $\mathbb{F}$ is to be finite the notation $\mathbb{F}_q$ denotes the field with q elements (so q is a prime power); when we intend a prime field specifically we use the notation $\mathbb{F}_p$. We write the multiplicative group of nonzero elements as $\mathbb{F}^\times = \mathbb{F} \setminus \{0\}$. We reserve the letter n for the order of the domain we shall construct; throughout, $n \geq 1$.

9.1 Roots of unity

The starting point is the equation $X^n = 1$. Its solutions in a field are the roots of unity of order dividing n .

Definition 9.1. Let $\mathbb{F}$ be a field and $n \in \mathbb{N}_{>0}$. An element $\omega \in \mathbb{F}$ is an *n -th root of unity* if $\omega^n = 1$. We write

$$\mu_n(\mathbb{F}) = \{ \omega \in \mathbb{F} : \omega^n = 1 \}$$

for the set of all n -th roots of unity in $\mathbb{F}$.

Note that $0 \notin \mu_n(\mathbb{F})$, since $0^n = 0 \neq 1$. Thus every n -th root of unity is a unit, i.e. lies in $\mathbb{F}^\times$. The set $\mu_n(\mathbb{F})$ is not merely a set; it is a group.

Proposition 9.2. $\mu_n(\mathbb{F})$ is a finite subgroup of $\mathbb{F}^\times$ of order at most n .

Proof. The constant 1 satisfies $1^n = 1$, so $1 \in \mu_n(\mathbb{F})$. If $\omega, \zeta \in \mu_n(\mathbb{F})$ then $(\omega\zeta)^n = \omega^n\zeta^n = 1 \cdot 1 = 1$ because $\mathbb{F}^\times$ is abelian, so $\omega\zeta \in \mu_n(\mathbb{F})$. If $\omega \in \mu_n(\mathbb{F})$ then $(\omega^{-1})^n = (\omega^n)^{-1} = 1^{-1} = 1$, so $\omega^{-1} \in \mu_n(\mathbb{F})$. Hence $\mu_n(\mathbb{F})$ is a subgroup of $\mathbb{F}^\times$. Every element of $\mu_n(\mathbb{F})$ is a root of the polynomial $X^n - 1 \in \mathbb{F}[X]$, which has degree n ; a nonzero polynomial of degree n over a field has at most n roots, so $|\mu_n(\mathbb{F})| \leq n$. $\square$

The bound $|\mu_n(\mathbb{F})| \leq n$ need not be attained. Whether it is attained is precisely the question of the existence of a *primitive n -th root of unity*.

Definition 9.3. An element $\omega \in \mathbb{F}$ is a *primitive n -th root of unity* if $\omega^n = 1$ and $\omega^k \neq 1$ for every integer k with $1 \leq k < n$. Equivalently, ω is primitive of order n if its multiplicative order $\text{ord}(\omega)$ equals n .

Here $\text{ord}(\omega)$, the *order* of ω , is the least positive integer d with $\omega^d = 1$; it exists for any $\omega \in \mu_n(\mathbb{F})$ because n itself is such an exponent and the positive integers are well-ordered. The two formulations in theorem 9.3 agree: $\omega^k \neq 1$ for $1 \leq k < n$ together with $\omega^n = 1$ states exactly that the least positive exponent killing ω is n .

Lemma 9.4. Let $\omega \in \mathbb{F}^\times$ have order $d = \text{ord}(\omega)$. Then for any integer m we have $\omega^m = 1$ if and only

if $d \mid m$. In particular ω is an n -th root of unity if and only if $d \mid n$, and ω is a primitive n -th root of unity if and only if $d = n$.

Proof. If $d \mid m$, write $m = dk$; then $\omega^m = (\omega^d)^k = 1^k = 1$. Conversely suppose $\omega^m = 1$. Divide with remainder: $m = qd + r$ with $0 \leq r < d$. Then $1 = \omega^m = (\omega^d)^q \omega^r = \omega^r$. Minimality of d forces $r = 0$, so $d \mid m$. The remaining claims are immediate: $\omega^n = 1 \iff d \mid n$, and primitivity is the case $d = n$. $\square$

A primitive n -th root of unity, when one exists, generates all the others.

Proposition 9.5. *If $\omega \in \mathbb{F}$ is a primitive n -th root of unity, then*

$$\mu_n(\mathbb{F}) = \langle \omega \rangle = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

is a cyclic group of order exactly n , and these n powers are pairwise distinct.

Proof. The powers $1, \omega, \dots, \omega^{n-1}$ are pairwise distinct: if $\omega^i = \omega^j$ with $0 \leq i \leq j \leq n-1$, then $\omega^{j-i} = 1$ with $0 \leq j-i \leq n-1 < n$, so by primitivity (theorem 9.4, with $d = n$) we must have $j-i = 0$. Thus $\langle \omega \rangle$ has exactly n elements, all of which lie in $\mu_n(\mathbb{F})$ since $(\omega^k)^n = (\omega^n)^k = 1$. But $|\mu_n(\mathbb{F})| \leq n$ by theorem 9.2, so the inclusion $\langle \omega \rangle \subseteq \mu_n(\mathbb{F})$ is an equality of n -element sets. $\square$

Over the complex numbers a primitive n -th root of unity always exists, namely $\omega = e^{2\pi i/n}$; this is the classical picture of n equally spaced points on the unit circle. Over a finite field the situation is more delicate, and it is exactly the finite case that matters for cryptography.

9.2 Existence over finite fields: the condition $n \mid q-1$

The decisive structural fact about finite fields is that their multiplicative group is cyclic; the existence theorem for roots of unity rests on it.

Theorem 9.6 (Multiplicative group of a finite field is cyclic). *Let $\mathbb{F}_q$ be a finite field with q elements. Then $\mathbb{F}_q^\times$ is a cyclic group of order $q-1$.*

Proof. This is Theorem 6.8 applied to the finite subgroup $\mathbb{F}_q^\times$ of itself. $\square$

A generator of $\mathbb{F}_q^\times$ is classically called a *primitive element* or *primitive root* of $\mathbb{F}_q$; it is a primitive $(q-1)$ -th root of unity in the sense of theorem 9.3.

Theorem 9.7 (Existence and count of primitive n -th roots of unity). *Let $\mathbb{F}_q$ be a finite field and $n \in \mathbb{N}_{>0}$ with $\gcd(n, q) = 1$ (automatic when $n < p$, where $p = \text{char } \mathbb{F}_q$; equivalently, in characteristic p , exactly the condition $p \nmid n$). Then:*

1. $\mathbb{F}_q$ contains a primitive n -th root of unity if and only if $n \mid q-1$.
2. When $n \mid q-1$, the group $\mu_n(\mathbb{F}_q)$ is cyclic of order exactly n , and the number of primitive n -th roots of unity in $\mathbb{F}_q$ is $\varphi(n)$, where φ is Euler's totient function.

Proof. Let g be a generator of $\mathbb{F}_q^\times$, which is cyclic of order $q-1$ by theorem 9.6.

($\Rightarrow$) Suppose ω is a primitive n -th root of unity in $\mathbb{F}_q$. Then $\omega \in \mathbb{F}_q^\times$ has order n by theorem 9.3, and by Lagrange's theorem the order of any element divides the group order $q-1$. Hence $n \mid q-1$.

($\Leftarrow$) Suppose $n \mid q-1$, say $q-1 = nm$. Set $\omega = g^m$. Then $\omega^n = g^{mn} = g^{q-1} = 1$, so $\omega \in \mu_n(\mathbb{F}_q)$. Its order is

$$\text{ord}(\omega) = \text{ord}(g^m) = \frac{\text{ord}(g)}{\gcd(m, \text{ord}(g))} = \frac{q-1}{\gcd(m, q-1)} = \frac{q-1}{m} = n,$$

using the standard formula for the order of a power in a cyclic group together with $m \mid q - 1$ (so $\gcd(m, q - 1) = m$). Thus ω is a primitive n -th root of unity. By theorem 9.5, $\mu_n(\mathbb{F}_q) = \langle \omega \rangle$ is cyclic of order exactly n .

Count. In the cyclic group $\mu_n(\mathbb{F}_q) = \langle \omega \rangle$ of order n , the element ω^k has order $n / \gcd(k, n)$. This equals n precisely when $\gcd(k, n) = 1$. The number of $k \in \{0, 1, \dots, n-1\}$ with $\gcd(k, n) = 1$ is by definition $\varphi(n)$. Hence there are exactly $\varphi(n)$ primitive n -th roots of unity. (The coprimality hypothesis $\gcd(n, q) = 1$ is what guarantees $X^n - 1$ is separable, hence has n distinct roots in a splitting field over $\mathbb{F}_q$ (equivalently, in $\overline{\mathbb{F}_q}$) (theorem 5.38); that all n of them lie in $\mathbb{F}_q$ itself is exactly the condition $n \mid q - 1$; if $p = \text{char } \mathbb{F}_q$ divided n , then $X^n - 1$ would have repeated roots and $\mu_n(\mathbb{F}_q)$ would be strictly smaller. But $n \mid q - 1$ already forces $\gcd(n, q) = 1$, since $\gcd(q - 1, q) = 1$, so the hypothesis is in fact subsumed.) $\square$

Remark 9.8. The proof shows that for a finite field the clean statement is simply: $\mathbb{F}_q$ has a primitive n -th root of unity $\iff n \mid q - 1$, with no separate coprimality side-condition, because $n \mid q - 1$ automatically makes n coprime to q . We stated the condition $\gcd(n, q) = 1$ explicitly only to flag the role of the characteristic.

Example 9.9. Take $\mathbb{F}_q = \mathbb{F}_p$ with $p = 7$. Here $q - 1 = 6$, and 3 is a generator of $\mathbb{F}_p^\times$ (its powers are 3, 2, 6, 4, 5, 1, all six nonzero residues). For $n = 3 \mid 6$, set $m = 6/3 = 2$ and $\omega = 3^2 = 2$. Indeed $2^1 = 2$, $2^2 = 4$, $2^3 = 8 \equiv 1$, so $\omega = 2$ has order 3 and $\mu_3(\mathbb{F}_7) = \{1, 2, 4\}$. There are $\varphi(3) = 2$ primitive cube roots, namely 2 and 4. By contrast $n = 4 \nmid 6$, so $\mathbb{F}_7$ has *no* primitive 4-th root of unity: $X^4 - 1 = (X^2 - 1)(X^2 + 1)$ and $X^2 + 1$ is irreducible over $\mathbb{F}_7$ (it has no root since -1 is not a square mod 7).

Remark 9.10 (Two-adicity and FFT-friendly primes). For the fast algorithms below it is desirable that n be a power of two (or have many factors of two and other small primes). Theorem 9.7 shows this is possible exactly when $2^k \mid q - 1$. The largest k with $2^k \mid q - 1$ is called the *two-adicity* of the field. Designers deliberately choose primes p with high two-adicity—so that $p - 1 = 2^k \cdot c$ for large k —as scalar fields in proof systems precisely so that large smooth evaluation domains exist. Both Pasta primes satisfy $2^{32} \mid p - 1$ (two-adicity exactly 32), so each Pasta field contains primitive 2^k -th roots of unity for every $k \leq 32$, furnishing power-of-two evaluation domains of every size up to 2^{32} ; this is what makes the entire machinery of this section available to Halo 2 and Orchard.

9.3 The evaluation domain H and its vanishing polynomial

We now fix the central object of this section.

Definition 9.11. Let $\mathbb{F}$ be a field containing a primitive n -th root of unity ω . The *evaluation domain* of order n is the multiplicative subgroup

$$H = \langle \omega \rangle = \{1, \omega, \omega^2, \dots, \omega^{n-1}\} = \mu_n(\mathbb{F}) \subseteq \mathbb{F}^\times,$$

a cyclic group of order n by theorem 9.5. We frequently index its elements as $\omega^0, \omega^1, \dots, \omega^{n-1}$, and abbreviate ω^i by h_i when convenient.

Because H is a multiplicative *group*, it is closed under products and inverses, and $1 \in H$. These elementary facts are exactly what make H substantially more useful than an arbitrary set of n interpolation points: cosets, products, and shifts of H interact algebraically. The single most important polynomial attached to H is the one that vanishes precisely on it.

Definition 9.12. The *vanishing polynomial* (or *zerofier*) of H is

$$Z_H(X) = \prod_{h \in H} (X - h) \in \mathbb{F}[X].$$

Theorem 9.13 (Closed form and factorization of the vanishing polynomial). *With $H = \mu_n(\mathbb{F})$ of order n generated by a primitive n -th root of unity ω ,*

$$Z_H(X) = X^n - 1 = \prod_{i=0}^{n-1} (X - \omega^i).$$

Moreover Z_H is separable: it has n distinct roots and $\gcd(Z_H, Z'_H) = 1$.

Proof. Every $h \in H$ satisfies $h^n = 1$, hence is a root of $X^n - 1$. The n elements of H are distinct (theorem 9.5), so $X^n - 1$ has n distinct roots, namely all of H . A degree- n polynomial with n distinct roots h is $c \prod_{h \in H} (X - h)$ for some constant c ; comparing leading coefficients (both $X^n - 1$ and the product are monic) gives $c = 1$. Hence $X^n - 1 = \prod_{h \in H} (X - h) = Z_H(X)$. Separability: the derivative is $Z'_H(X) = nX^{n-1}$. The existence of the n distinct roots already forces $\text{char } \mathbb{F} \nmid n$: if $\text{char } \mathbb{F} = \ell$ divided n , say $n = \ell m$, the freshman's dream (theorem 6.4) would give $X^n - 1 = (X^m - 1)^\ell$, a polynomial with at most $m < n$ distinct roots. Hence $n \neq 0$ in $\mathbb{F}$, so nX^{n-1} is nonzero and its only root is 0, which is not a root of $X^n - 1$. Thus Z_H and Z'_H share no root, so $\gcd(Z_H, Z'_H) = 1$ and Z_H is separable. $\square$

The closed form $Z_H(X) = X^n - 1$ is the workhorse of polynomial proof systems: testing whether a polynomial f vanishes on all of H reduces to checking divisibility by $X^n - 1$, which one can verify by exhibiting a quotient $q(X)$ with $f(X) = q(X)(X^n - 1)$. Moreover, evaluating Z_H at any point costs a single exponentiation X^n , not n multiplications. We record several structural consequences below.

Proposition 9.14 (Cosets and shifts). *Let $\gamma \in \mathbb{F}^\times$ and let $\gamma H = \{\gamma h : h \in H\}$ be the coset. Then the polynomial vanishing on γH is*

$$Z_{\gamma H}(X) = \prod_{h \in H} (X - \gamma h) = X^n - \gamma^n.$$

In particular, distinct cosets of H in $\mathbb{F}^\times$ have vanishing polynomials $X^n - c$ for distinct constants $c = \gamma^n$, and the cosets of H partition $\mathbb{F}^\times$ into translates of H , on each of which X^n is constant.

Proof. Substitute $Y = X/\gamma$: $\prod_h (X - \gamma h) = \gamma^n \prod_h (Y - h) = \gamma^n (Y^n - 1) = \gamma^n ((X/\gamma)^n - 1) = X^n - \gamma^n$, using theorem 9.13. The constant is $c = \gamma^n$, and $\gamma_1 H = \gamma_2 H \iff \gamma_1/\gamma_2 \in H \iff (\gamma_1/\gamma_2)^n = 1 \iff \gamma_1^n = \gamma_2^n$, so distinct cosets give distinct c . $\square$

Proposition 9.15 (Sum of roots of unity). *If $n > 1$ then $\sum_{i=0}^{n-1} \omega^i = 0$, and more generally for any integer k ,*

$$\sum_{i=0}^{n-1} \omega^{ki} = \begin{cases} n & \text{if } n \mid k, \\ 0 & \text{if } n \nmid k. \end{cases}$$

Proof. If $n \mid k$ then $\omega^{ki} = (\omega^n)^{ki/n} = 1$ for each i , so the sum is n . If $n \nmid k$ then $\zeta := \omega^k \neq 1$ (since ω has order n , so $\omega^k = 1 \iff n \mid k$), and the finite geometric series gives

$$\sum_{i=0}^{n-1} \zeta^i = \frac{\zeta^n - 1}{\zeta - 1} = \frac{(\omega^n)^k - 1}{\zeta - 1} = \frac{1 - 1}{\zeta - 1} = 0,$$

the denominator being nonzero because $\zeta \neq 1$. The first claim is the case $k = 1$, valid for $n > 1$ since then $n \nmid 1$. $\square$

9.4 The Lagrange basis on H

Attention now passes from the group H to the polynomials we study on it. The space of polynomials of degree less than n has dimension n over $\mathbb{F}$; since H has exactly n points, their values on H determine such polynomials. The basis adapted to this situation is the Lagrange basis.

Definition 9.16. Let $\mathbb{F}[X]_{<n}$ denote the $\mathbb{F}$ -vector space of polynomials of degree strictly less than n , together with the zero polynomial. It has the monomial basis $\{1, X, \dots, X^{n-1}\}$ and so $\dim_{\mathbb{F}} \mathbb{F}[X]_{<n} = n$.

Definition 9.17. For each $i \in \{0, 1, \dots, n-1\}$, the i -th *Lagrange basis polynomial* for $H = \{\omega^0, \dots, \omega^{n-1}\}$ is

$$L_i(X) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{X - \omega^j}{\omega^i - \omega^j} \in \mathbb{F}[X]_{<n}.$$

Proposition 9.18 (Interpolation property). *The polynomials $L_0, \dots, L_{n-1}$ satisfy the Kronecker-delta conditions*

$$L_i(\omega^j) = \delta_{ij} = \begin{cases} 1 & i = j, \\ 0 & i \neq j, \end{cases}$$

they form a basis of $\mathbb{F}[X]_{<n}$, and they give the unique interpolant: for any data $(v_0, \dots, v_{n-1}) \in \mathbb{F}^n$ the polynomial

$$f(X) = \sum_{i=0}^{n-1} v_i L_i(X)$$

is the unique element of $\mathbb{F}[X]_{<n}$ with $f(\omega^i) = v_i$ for all i .

Proof. Each L_i has degree $n-1$ (a product of $n-1$ linear factors) and is well-defined because the ω^j are distinct, so the denominators $\omega^i - \omega^j$ are nonzero for $j \neq i$. Evaluation at $X = \omega^j$ with $j \neq i$ makes the factor $(X - \omega^j)$ vanish, giving $L_i(\omega^j) = 0$; at $X = \omega^i$ every factor is $(\omega^i - \omega^j)/(\omega^i - \omega^j) = 1$, giving $L_i(\omega^i) = 1$. This is $L_i(\omega^j) = \delta_{ij}$.

For the interpolation claim, $f = \sum_i v_i L_i$ lies in $\mathbb{F}[X]_{<n}$ and $f(\omega^j) = \sum_i v_i \delta_{ij} = v_j$. Uniqueness follows because, if $f, \tilde{f} \in \mathbb{F}[X]_{<n}$ agree on all n points of H , then $f - \tilde{f} \in \mathbb{F}[X]_{<n}$ has n distinct roots but degree $< n$, hence is the zero polynomial. Finally the L_i are linearly independent—if $\sum_i c_i L_i = 0$, evaluation at ω^j yields $c_j = 0$ —and there are n of them in the n -dimensional space $\mathbb{F}[X]_{<n}$, so they form a basis. $\square$

On a generic point set the Lagrange polynomials admit no closed form simpler than the defining product. On the group H they collapse to a compact closed form involving the vanishing polynomial.

Theorem 9.19 (Closed form of the Lagrange basis on H). *For $H = \mu_n(\mathbb{F})$ with primitive root ω , and for each i ,*

$$L_i(X) = \frac{\omega^i}{n} \cdot \frac{X^n - 1}{X - \omega^i} = \frac{\omega^i (X^n - 1)}{n(X - \omega^i)}.$$

In particular, dividing out one linear factor and scaling yields all n Lagrange polynomials from the single polynomial $Z_H(X) = X^n - 1$.

Proof. Fix i . From theorem 9.13, $Z_H(X) = \prod_{j=0}^{n-1} (X - \omega^j)$, so

$$\frac{Z_H(X)}{X - \omega^i} = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} (X - \omega^j),$$

which is exactly the numerator of L_i in theorem 9.17. It remains to evaluate the denominator $\prod_{j \neq i} (\omega^i - \omega^j)$. Writing $Z_H = (X - \omega^i)g$ with $g = \prod_{j \neq i} (X - \omega^j)$, the product rule (theorem 5.37) gives $Z'_H = g + (X - \omega^i)g'$, so $Z'_H(\omega^i) = g(\omega^i) = \prod_{j \neq i} (\omega^i - \omega^j)$ —the computation already made for simple roots in the proof of theorem 5.38; concretely $Z'_H(X) = nX^{n-1}$, so

$$Z'_H(\omega^i) = n(\omega^i)^{n-1} = n\omega^{in}\omega^{-i} = n \cdot 1 \cdot \omega^{-i} = \frac{n}{\omega^i},$$

using $\omega^{in} = (\omega^n)^i = 1$. Therefore

$$L_i(X) = \frac{Z_H(X)/(X - \omega^i)}{Z'_H(\omega^i)} = \frac{(X^n - 1)/(X - \omega^i)}{n/\omega^i} = \frac{\omega^i(X^n - 1)}{n(X - \omega^i)},$$

as claimed. $\square$

Remark 9.20 (Barycentric form). The closed form renders the *barycentric* interpolation formula on H especially clean. Writing $f(X) = \sum_i v_i L_i(X)$ and substituting theorem 9.19,

$$f(X) = \frac{X^n - 1}{n} \sum_{i=0}^{n-1} \frac{\omega^i v_i}{X - \omega^i}.$$

Evaluating f at a point $z \notin H$ thus costs one evaluation of $z^n - 1$ and n terms of the sum, with no per-point polynomial reconstruction. This is precisely the formula a verifier uses to evaluate a committed polynomial's Lagrange representation at a random challenge z .

Proposition 9.21 (Lagrange polynomials sum to one). $\sum_{i=0}^{n-1} L_i(X) = 1$ identically.

Proof. The polynomial $g(X) = \sum_i L_i(X) - 1$ lies in $\mathbb{F}[X]_{<n}$ (degree $< n$) and vanishes at every ω^j because $\sum_i L_i(\omega^j) = \sum_i \delta_{ij} = 1$. A polynomial of degree $< n$ with n distinct roots is zero, so $g \equiv 0$. $\square$

Example 9.22 (The all-ones value vector). The constant polynomial 1 interpolates the data $v_i = 1$ for all i ; theorem 9.21 is the statement $\sum_i 1 \cdot L_i = 1$. Likewise the values of X on H are $v_i = \omega^i$, and, for $n \geq 2$, theorem 9.18 gives $\sum_i \omega^i L_i(X) = X$ (both sides agree on H and have degree $< n$). We invoke these identities constantly when rewriting monomials in the Lagrange basis.

9.5 Evaluation and interpolation as mutually inverse linear maps

An element of $\mathbb{F}[X]_{<n}$ admits two natural descriptions: by its n coefficients, or by its n values on H . Both are coordinate systems on the same n -dimensional vector space, and passage between them is a linear isomorphism.

Definition 9.23 (Evaluation map). The *evaluation map* on H is the $\mathbb{F}$ -linear map

$$\text{ev}_H : \mathbb{F}[X]_{<n} \longrightarrow \mathbb{F}^n, \quad \text{ev}_H(f) = (f(\omega^0), f(\omega^1), \dots, f(\omega^{n-1})).$$

Linearity is immediate from $(af + bg)(\omega^i) = af(\omega^i) + bg(\omega^i)$.

Theorem 9.24 (Evaluation and interpolation are mutually inverse). *The evaluation map ev_H is a vector-space isomorphism. Its inverse is the interpolation map*

$$\text{int}_H : \mathbb{F}^n \longrightarrow \mathbb{F}[X]_{<n}, \quad \text{int}_H(v_0, \dots, v_{n-1}) = \sum_{i=0}^{n-1} v_i L_i(X).$$

That is, $\text{int}_H \circ \text{ev}_H = \text{id}_{\mathbb{F}[X]_{<n}}$ and $\text{ev}_H \circ \text{int}_H = \text{id}_{\mathbb{F}^n}$.

Proof. Both maps are linear and act between spaces of the same finite dimension n , so it suffices to verify that they compose to the identity in one order; we check both for clarity. For $v \in \mathbb{F}^n$, $\text{int}_H(v) = \sum_i v_i L_i$ has value $\sum_i v_i L_i(\omega^j) = v_j$ at ω^j by theorem 9.18, so $\text{ev}_H(\text{int}_H(v)) = v$. For $f \in \mathbb{F}[X]_{<n}$, $\text{int}_H(\text{ev}_H(f)) = \sum_i f(\omega^i) L_i$ is, by the uniqueness clause of theorem 9.18, the unique degree- $< n$ polynomial taking the values $f(\omega^i)$ on H —which is f itself. Hence the two maps are inverse, and in particular ev_H is bijective and linear, i.e. an isomorphism. $\square$

In coordinates relative to the monomial basis, ev_H is multiplication by a Vandermonde matrix, which we name here because it is the bridge to the Fourier transform.

Definition 9.25 (Vandermonde / DFT matrix). Let $f(X) = \sum_{k=0}^{n-1} c_k X^k$ have coefficient vector $c = (c_0, \dots, c_{n-1})$. Then $\text{ev}_H(f)$ has i -th entry $\sum_k c_k \omega^{ik}$. Thus, relative to the monomial basis on the domain and the standard basis on $\mathbb{F}^n$, the *Vandermonde matrix*

$$V = (\omega^{ik})_{0 \leq i, k \leq n-1} = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)^2} \end{pmatrix},$$

represents ev_H , so that $\text{ev}_H(f) = Vc$. We call V the (discrete Fourier) transform matrix on H .

Theorem 9.26 (Invertibility and the inverse transform). *The matrix V of theorem 9.25 is invertible, with*

$$(V^{-1})_{k,i} = \frac{1}{n} \omega^{-ik}.$$

Equivalently, $V^{-1} = \frac{1}{n} \bar{V}$ where $\bar{V} = (\omega^{-ik})_{i,k}$ is the transform matrix built from ω^{-1} (which is itself a primitive n -th root of unity). Consequently the inversion formula relates the coefficient and value representations by

$$c_k = \frac{1}{n} \sum_{i=0}^{n-1} f(\omega^i) \omega^{-ik}, \quad f(\omega^i) = \sum_{k=0}^{n-1} c_k \omega^{ik}.$$

Proof. It suffices to show $\bar{V}V = nI$. The (k, k') entry of $\bar{V}V$ is

$$\sum_{i=0}^{n-1} \omega^{-ik} \omega^{ik'} = \sum_{i=0}^{n-1} \omega^{i(k'-k)}.$$

By theorem 9.15 this sum is n if $n \mid (k' - k)$ and 0 otherwise. For $k, k' \in \{0, \dots, n-1\}$ one has $n \mid (k' - k) \iff k = k'$. Hence $\bar{V}V = nI$, giving $V^{-1} = \frac{1}{n} \bar{V}$ (we may divide by n since $n \neq 0$ in $\mathbb{F}$). The component formulas write out the matrix identities $\text{ev}_H(f) = Vc$ and $c = V^{-1} \text{ev}_H(f)$. $\square$

Remark 9.27. Theorem 9.24 and theorem 9.26 are two faces of one statement. The abstract face: evaluation on H is an isomorphism whose inverse is Lagrange interpolation. The concrete face: that isomorphism is the matrix V , and its inverse is, up to the scalar $1/n$, the same matrix with ω replaced by ω^{-1} . The near-symmetry between a transform and its inverse—differing only by conjugating $\omega \mapsto \omega^{-1}$ and a scale $1/n$ —is the algebraic reason the fast inverse transform costs exactly as much as the forward one.

9.6 The number-theoretic Fourier transform

The map ev_H , read as the matrix V , is the discrete Fourier transform, but over a finite field rather than over $\mathbb{C}$. Because it involves no analytic structure (no complex exponentials, no convergence), we call it the *number-theoretic transform*.

Definition 9.28 (Number-theoretic transform). Let ω be a primitive n -th root of unity in $\mathbb{F}$. The *number-theoretic transform* (NTT) of a vector $a = (a_0, \dots, a_{n-1}) \in \mathbb{F}^n$ is the vector $\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n-1})$ with

$$\hat{a}_j = \sum_{k=0}^{n-1} a_k \omega^{jk}, \quad j = 0, \dots, n-1.$$

The *inverse number-theoretic transform* is

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} \hat{a}_j \omega^{-jk}.$$

In the language above: if a is the coefficient vector of $f \in \mathbb{F}[X]_{<n}$, then $\hat{a} = \text{ev}_H(f)$ is its value vector on H ; the NTT is evaluation, and the inverse NTT is interpolation.

That the inverse formula indeed inverts the forward one is exactly theorem 9.26. The NTT enjoys the same convolution property as the classical DFT, which is the mechanism underlying fast polynomial multiplication.

Theorem 9.29 (Convolution theorem). For $f, g \in \mathbb{F}[X]$ with $\deg f + \deg g < n$, let $c = fg$ be their product (degree $< n$). Then for every $h \in H$,

$$c(h) = f(h)g(h),$$

i.e. pointwise multiplication of value vectors corresponds to polynomial multiplication. Equivalently, writing a, b for the coefficient vectors of f, g and $a * b$ for the (cyclic) convolution defining the coefficients of the product modulo $X^n - 1$,

$$\widehat{a * b}_j = \hat{a}_j \hat{b}_j \quad (j = 0, \dots, n-1),$$

so the NTT turns convolution into entrywise multiplication.

Proof. The first identity is the statement that evaluation is a ring homomorphism $\mathbb{F}[X] \rightarrow \mathbb{F}$, $f \mapsto f(h)$: $(fg)(h) = f(h)g(h)$ for any h . Since $\deg(fg) < n$, interpolation recovers the product $c = fg$ from its values on the n -point set H (theorem 9.18), so the value vectors multiply entrywise and uniquely determine c . For the convolution statement: multiplication in the quotient ring $\mathbb{F}[X]/(X^n - 1)$ corresponds to cyclic convolution of coefficient vectors, and since $H = \mu_n$ is exactly the set of roots of $X^n - 1$, the map $p \mapsto \text{ev}_H(p)$ is a ring isomorphism $\mathbb{F}[X]/(X^n - 1) \xrightarrow{\sim} \mathbb{F}^n$ (with $\mathbb{F}^n$ under entrywise operations). It sends the product (convolution) of a, b to the entrywise product of $\hat{a}, \hat{b}$. This is the displayed identity. When $\deg f + \deg g < n$ the cyclic and ordinary products coincide, so the same statement computes the genuine product fg . $\square$

Remark 9.30 (Computational significance). Theorem 9.29 reduces multiplying two degree- $< n/2$ polynomials—a quadratic-cost operation by the schoolbook method—to: transform both (NTT), multiply the value vectors entrywise (n multiplications), and transform back (inverse NTT). If the NTT runs in $O(n \log n)$ operations, the whole multiplication costs $O(n \log n)$. The fast Fourier transform supplies that conditional.

9.7 The fast Fourier transform

Computing $\hat{a} = Va$ directly from theorem 9.25 costs $\Theta(n^2)$ field multiplications: each of the n outputs is a sum of n products. The fast Fourier transform (FFT)—here a number-theoretic FFT—computes the same vector in $O(n \log n)$ operations by a divide-and-conquer recursion, provided n is a power of two (or more generally smooth). This is where the requirement that H be a *smooth* subgroup, theorem 9.10, becomes indispensable.

Theorem 9.31 (Cooley–Tukey radix-2 step). *Suppose $n = 2m$ and ω is a primitive n -th root of unity in $\mathbb{F}$. Given $f(X) = \sum_{k=0}^{n-1} a_k X^k$, split its coefficients by parity into*

$$f_{\text{even}}(Y) = \sum_{k=0}^{m-1} a_{2k} Y^k, \quad f_{\text{odd}}(Y) = \sum_{k=0}^{m-1} a_{2k+1} Y^k,$$

so that $f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$. Then $\eta := \omega^2$ is a primitive m -th root of unity, and for $j = 0, \dots, m-1$,

$$f(\omega^j) = f_{\text{even}}(\eta^j) + \omega^j f_{\text{odd}}(\eta^j), \quad (8)$$

$$f(\omega^{j+m}) = f_{\text{even}}(\eta^j) - \omega^j f_{\text{odd}}(\eta^j). \quad (9)$$

Thus the length- n NTT of a reduces to two length- m NTTs (of the even and odd subsequences, over the domain $\langle \eta \rangle$) plus $O(n)$ extra operations.

Proof. The identity $f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$ regroups the sum $\sum_k a_k X^k$ into even-index and odd-index terms; in the odd terms we pull out one factor of X , leaving X^{2k} . That $\eta = \omega^2$ is a primitive m -th root of unity follows from $\text{ord}(\omega^2) = n/\gcd(2, n) = n/2 = m$ (theorem 9.4 and the order-of-a-power formula), using $2 \mid n$. It remains to evaluate. For (8), put $X = \omega^j$; then $X^2 = \omega^{2j} = \eta^j$, so $f(\omega^j) = f_{\text{even}}(\eta^j) + \omega^j f_{\text{odd}}(\eta^j)$. For (9), put $X = \omega^{j+m}$; then $X^2 = \omega^{2j+2m} = \omega^{2j} \omega^n = \eta^j \cdot 1 = \eta^j$ (since $2m = n$ and $\omega^n = 1$), while the linear factor is $\omega^{j+m} = \omega^j \omega^m = -\omega^j$. Here $\omega^m = -1$ because ω^m has order $n/\gcd(m, n) = n/m = 2$, and -1 is the unique element of order 2 in a field (it satisfies $x^2 = 1$, $x \neq 1$, so $x = -1$; note $-1 \neq 1$ since $\text{char } \mathbb{F} \nmid n$ and n is even forces $\text{char } \mathbb{F} \neq 2$). Substituting, $f(\omega^{j+m}) = f_{\text{even}}(\eta^j) - \omega^j f_{\text{odd}}(\eta^j)$. $\square$

Remark 9.32 (The butterfly). Equations (8)–(9) constitute the *butterfly*: from the two subtransform outputs $E_j = f_{\text{even}}(\eta^j)$ and $O_j = f_{\text{odd}}(\eta^j)$ one forms the *twiddle* $t_j = \omega^j O_j$ and reads off the two outputs $E_j + t_j$ and $E_j - t_j$. Pictorially, E_j and O_j sit as two nodes on the left; each is carried to both outputs on the right, the two E_j -edges with weight 1 and the two O_j -edges with weights $+\omega^j$ and $-\omega^j$; the pair of crossing edges forms the $\times$ that names the operation. Each butterfly costs one multiplication (the twiddle) and two additions, and there are $n/2$ of them per level.

Theorem 9.33 ($O(n \log n)$ cost of the FFT). *Let $n = 2^t$ and let $T(n)$ be the number of field additions and multiplications the radix-2 recursion of theorem 9.31 uses to compute a length- n NTT. Then*

$$T(n) = 2 T(n/2) + cn \quad (n > 1), \quad T(1) = O(1),$$

for a constant c , and therefore $T(n) = O(n \log n)$. The inverse NTT has the same cost, since it is the forward NTT with ω replaced by ω^{-1} followed by scaling each entry by $1/n$ (theorem 9.26).

Proof. The recursion is exactly theorem 9.31: a length- n transform calls two length- $n/2$ transforms (on the even and odd coefficient subsequences) and then combines them with $n/2$ butterflies, each of $O(1)$ operations, for the cn overhead. Unrolling, with $n = 2^t$,

$$T(2^t) = 2T(2^{t-1}) + c 2^t.$$

Dividing by 2^t , and letting $S(t) = T(2^t)/2^t$, gives $S(t) = S(t-1) + c$, so $S(t) = S(0) + ct = O(1) + ct$. Hence $T(2^t) = 2^t S(t) = O(2^t) + ct 2^t = O(t 2^t)$. With $n = 2^t$, $t = \log_2 n$, this is $T(n) = O(n \log n)$. (Equivalently: the recursion tree has $\log_2 n$ levels, each doing $\Theta(n)$ total work across all subproblems at that level.) The inverse-transform claim is theorem 9.26: the same Cooley–Tukey recursion with ω^{-1} in place of ω , and a final pass of n divisions by n , all within $O(n \log n)$. $\square$

Remark 9.34 (Mixed radix and smoothness). The hypothesis that enables fast transforms is that n be *smooth*—a product of small primes—and, crucially, that the field actually *contain* a primitive n -th root of unity, i.e. $n \mid q-1$ (theorem 9.7). A power-of-two n dividing $q-1$ is the optimal case: pure radix-2 with the cleanest butterfly.

Example 9.35 (A length-4 transform). Take $n = 4$ with primitive 4-th root ω (so $\omega^2 = -1$, $\omega^4 = 1$, $\eta = \omega^2 = -1$ the primitive 2nd root). For $f = a_0 + a_1X + a_2X^2 + a_3X^3$, set $E = (a_0 + a_2, a_0 - a_2)$ and $O = (a_1 + a_3, a_1 - a_3)$, the two length-2 transforms over $\{1, -1\}$. Then the butterflies give

$$\hat{a}_0 = E_0 + \omega^0 O_0, \quad \hat{a}_2 = E_0 - \omega^0 O_0, \quad \hat{a}_1 = E_1 + \omega^1 O_1, \quad \hat{a}_3 = E_1 - \omega^1 O_1.$$

This uses 2 sub-transforms of 2 additions each plus 2 butterflies ($n/2 = 2$ per level, theorem 9.32; together 2 twiddle multiplications and 4 additions)—far fewer than the 16 products of the direct 4×4 matrix multiply, and the gap widens as n grows.

9.8 The role of smooth multiplicative domains in polynomial proof systems

This section closes by assembling the pieces into the reason these domains form the substrate of Halo 2 and Orchard. A proof system of the polynomial type encodes a computation as a system of polynomial identities and convinces a verifier that those identities hold. The evaluation domain $H = \mu_n$ is where the encoding lives, and every property proved above is load-bearing.

Remark 9.36 (The roles played by H , Z_H , the Lagrange basis, and the FFT).

- **Witnesses as evaluations on H .** A prover lays out the trace of a computation as the values of one or more polynomials on the n points of H . By theorem 9.24 this is the same data as a polynomial of degree $< n$; the prover may freely move between the value (Lagrange) representation, natural for stating constraints row by row, and the coefficient representation, natural for committing and for low-degree testing. The change of representation is the (inverse) NTT.
- **Constraints as divisibility by Z_H .** “Constraint polynomial C holds on every row” means C vanishes on all of H , which by theorem 9.13 means exactly that $Z_H = X^n - 1$ divides C . The prover demonstrates this by exhibiting a quotient t with $C(X) = t(X)(X^n - 1)$. The inexpensive closed form $Z_H(X) = X^n - 1$ —one exponentiation to evaluate—is what makes this check efficient at a random point. Theorem 9.14 extends the same technique to cosets γH , used to separate quotient and remainder domains.
- **Lagrange selectors.** The Lagrange polynomials L_i of theorem 9.17 are *selectors*: L_i is 1 on row i and 0 elsewhere (theorem 9.18), so boundary/instance constraints (e.g. “the first row equals the public input”) become $L_0 \cdot (\text{something})$. The closed form and barycentric formula (theorem 9.19, theorem 9.20) let the verifier evaluate these at a random challenge in $O(n)$ or even $O(\log n)$ field operations rather than reconstructing polynomials.

- **Group structure for permutations and shifts.** Because H is a cyclic group, multiplication by ω is the cyclic “next-row” shift $\omega^i \mapsto \omega^{i+1}$. Constraints relating a row to its neighbour become $f(\omega X)$ versus $f(X)$; permutation arguments (the heart of Halo 2’s copy constraints) rest on the action of the group on itself and on its cosets. An arbitrary set of n points offers none of this—it requires the multiplicative group H of theorem 9.11.
- **Smoothness for speed.** Provers manipulate polynomials of size n (often after blowing the domain up by a small constant factor for low-degree testing). Every transform between representations, every multiplication, every coset evaluation is an NTT. By theorem 9.33 these cost $O(n \log n)$ instead of $O(n^2)$ —the difference between feasible and infeasible at the sizes used in practice—*provided* n is smooth and $\mathbb{F}_q$ contains a primitive n -th root of unity. By theorem 9.7 that requires $n \mid q - 1$; by theorem 9.10 this is why we choose high-two-adicity primes for the scalar fields of the Pallas and Vesta curves.

In summary: the existence condition $n \mid q - 1$ (theorem 9.7) supplies the group H ; the group supplies the clean vanishing polynomial (theorem 9.13), the closed-form Lagrange basis (theorem 9.19), and the evaluation/interpolation isomorphism (theorem 9.24); and smoothness of n supplies the $O(n \log n)$ FFT (theorem 9.33). Together these are exactly the ingredients a polynomial proof system requires, which is why such systems rest on smooth multiplicative evaluation domains and on the fields that contain them.

10 Elliptic curves

Having developed the theory of finite fields, we now turn attention to the geometric objects that supply the cyclic groups underlying the cryptography of Halo 2 and Zcash Orchard: *elliptic curves*. An elliptic curve is, concretely, the solution set of a certain cubic equation in two variables, together with one extra “point at infinity”. The notable feature of elliptic curves is that this solution set carries the structure of an abelian group, defined by a strikingly geometric rule, and that this group is hard to compute in: solving the discrete logarithm problem in a well-chosen elliptic curve group appears to require exponential time. This combination of a clean algebraic structure with a hard computational problem is exactly what cryptography requires.

Throughout this section we work over a field K whose characteristic is neither 2 nor 3. In the applications K will be a finite prime field $\mathbb{F}_p$ with p a large prime, but it is cleaner to develop the basic theory over a general field and specialize when needed. The reader should keep in mind the two *Pasta curves*

$$E_p : y^2 = x^3 + 5 \quad \text{over } \mathbb{F}_p, \quad E_q : y^2 = x^3 + 5 \quad \text{over } \mathbb{F}_q,$$

where p and q are the two 255-bit primes

$$\begin{aligned} p &= 2^{254} + 0x224698fc094cf91b992d30ed00000001, \\ q &= 2^{254} + 0x224698fc0994a8dd8c46eb2100000001, \end{aligned}$$

already encountered in the finite-fields section (Remark 6.13). These are the curves *Pallas* (E_p , defined over $\mathbb{F}_p$) and *Vesta* (E_q , defined over $\mathbb{F}_q$); together they form a *cycle*, in that the number of $\mathbb{F}_p$ -points of Pallas equals q and the number of $\mathbb{F}_q$ -points of Vesta equals p . The equation $y^2 = x^3 + 5$ serves as the running example, with a smaller toy curve developed in parallel for hand computation.

10.1 Weierstrass equations

Definition 10.1 (short Weierstrass equation). Let K be a field of characteristic $\neq 2, 3$, and let $a, b \in K$. The *short Weierstrass equation* with coefficients a, b is the equation

$$y^2 = x^3 + ax + b. \tag{10}$$

The elements a and b are the *Weierstrass coefficients*.

Over a field of characteristic $\neq 2, 3$, an invertible affine change of variables always reduces the most general cubic curve $y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$ to the form (10): completing the square in y (possible since $2 \neq 0$) removes the xy and y terms, and then a shift $x \mapsto x - a_2/3$ (possible since $3 \neq 0$) removes the x^2 term. This is precisely why we exclude characteristics 2 and 3; those cases require the longer Weierstrass form and are not relevant to the Pasta curves, which live over large prime fields.

Not every choice of a, b gives a usable curve. We must exclude the cubics whose graph has a self-intersection or a cusp, because the group law (defined below) breaks down at such a singular point.

Definition 10.2 (discriminant and nonsingularity). The *discriminant* of the short Weierstrass equation (10) is

$$\Delta = -16(4a^3 + 27b^2) \in K. \tag{11}$$

The equation, and the cubic $f(x) = x^3 + ax + b$, is *nonsingular* (or *smooth*) if $\Delta \neq 0$, equivalently $4a^3 + 27b^2 \neq 0$.

Proposition 10.3. Let $f(x) = x^3 + ax + b \in K[x]$. The following are equivalent.

1. $4a^3 + 27b^2 \neq 0$.
2. f has no repeated root in the algebraic closure $\bar{K}$ (the smallest extension field of K in which every nonconstant polynomial over K factors into linear factors).
3. f and its derivative f' are coprime in $\bar{K}[x]$.
4. There is no point (x_0, y_0) in $\bar{K}^2$ satisfying both $y^2 = f(x)$ and the two partial-derivative equations $2y = 0$ and $f'(x) = 0$.

Proof. (2) $\Leftrightarrow$ (3): a polynomial has a repeated root iff it shares a root with its derivative, i.e. iff $\gcd(f, f') \neq 1$ in $\bar{K}[x]$. (1) $\Leftrightarrow$ (2): the discriminant of the cubic $x^3 + ax + b$ equals $-(4a^3 + 27b^2)$, i.e. agrees with $4a^3 + 27b^2$ up to sign (the curve discriminant Δ of Definition 10.2 carries the additional conventional factor 16); a univariate polynomial has a repeated root iff its discriminant vanishes. One may verify the identity directly: if $f(x) = (x - r_1)(x - r_2)(x - r_3)$ then $\prod_{i < j} (r_i - r_j)^2 = -(4a^3 + 27b^2)$, using $r_1 + r_2 + r_3 = 0$ (the coefficient of x^2 is zero). (4) $\Leftrightarrow$ (2): a singular point of the affine curve $y^2 = f(x)$ is a point where both partial derivatives of $g(x, y) = y^2 - f(x)$ vanish, namely $\partial g / \partial y = 2y = 0$ and $\partial g / \partial x = -f'(x) = 0$. Since $\text{char}(K) \neq 2$, the first forces $y_0 = 0$, and then the curve equation gives $f(x_0) = 0$; combined with $f'(x_0) = 0$ this says x_0 is a repeated root of f . Conversely a repeated root x_0 yields the singular point $(x_0, 0)$. $\square$

Definition 10.4 (elliptic curve, affine points). An *elliptic curve* over K is a short Weierstrass equation (10) with nonzero discriminant, $4a^3 + 27b^2 \neq 0$. For any field extension $L \supseteq K$ the set of *L-rational affine points* is

$$E^{\text{aff}}(L) = \{(x, y) \in L \times L : y^2 = x^3 + ax + b\},$$

and the set of *L-rational points* is

$$E(L) = E^{\text{aff}}(L) \cup \{\mathcal{O}\},$$

where $\mathcal{O}$ is a single formal symbol called the *point at infinity*. When $L = K$ one writes simply E for the curve and $E(K)$ for its rational points.

Remark 10.5. For the Pasta curves $a = 0$, $b = 5$, so $4a^3 + 27b^2 = 27 \cdot 25 = 675$. As long as the characteristic does not divide $675 = 3^3 \cdot 5^2$, i.e. the prime is not 3 or 5, the curve is nonsingular. The Pasta primes p, q are enormous, so this is automatic. More generally any curve $y^2 = x^3 + b$ with $b \neq 0$ is nonsingular away from characteristics 2, 3.

10.2 The point at infinity, geometrically

To motivate the point at infinity, view the affine plane as sitting inside the *projective plane*. A point of the projective plane $\mathbb{P}^2(K)$ is an equivalence class $[X : Y : Z]$ of nonzero triples $(X, Y, Z) \in K^3 \setminus \{0\}$, where $(X, Y, Z) \sim (\lambda X, \lambda Y, \lambda Z)$ for every $\lambda \in K^\times$. The affine points (x, y) embed as $[x : y : 1]$; the points with $Z = 0$, the *line at infinity*, are the extra “directions”. Homogenizing (10) by setting $x = X/Z$, $y = Y/Z$ and clearing denominators gives the projective Weierstrass equation

$$Y^2Z = X^3 + aXZ^2 + bZ^3. \quad (12)$$

Setting $Z = 0$ forces $X^3 = 0$, hence $X = 0$, leaving the single projective point $[0 : 1 : 0]$. This is the point at infinity $\mathcal{O}$: it lies “infinitely far up” in the vertical direction, and every vertical line $x = \text{const}$ passes through it. As we shall see, $\mathcal{O}$ is the identity element of the group law. The reader may safely picture $\mathcal{O}$ as a single point at the top (and bottom) of the y -axis where all vertical lines meet.

10.3 The chord-and-tangent group law: geometry

We now describe the procedure for “adding” two points of $E(K)$. The construction is purely geometric and rests on a single fact: a line meets a cubic curve in exactly three points, counted with multiplicity. The point at infinity is engineered precisely so that this counting is always exact.

Definition 10.6 (the group law, geometric form). Let $P, Q \in E(K)$. Define $P + Q$ as follows.

- If $P = \mathcal{O}$, set $P + Q = Q$; if $Q = \mathcal{O}$, set $P + Q = P$.
- Otherwise $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ are affine. Draw the line ℓ through P and Q ; if $P = Q$, let ℓ be the tangent line to the curve at P . The line ℓ meets the curve in a third point R' (counted with multiplicity), where if ℓ is vertical the third point is $\mathcal{O}$. Define $P + Q$ to be the reflection of R' across the x -axis: if $R' = (x_3, -y_3)$ then $P + Q = (x_3, y_3)$, and if $R' = \mathcal{O}$ then $P + Q = \mathcal{O}$.

The reflection step is what converts “the three intersection points sum to a fixed thing” into a genuine group law with $\mathcal{O}$ as identity. Concretely, the rule is: the three intersection points of any line with the curve add up to $\mathcal{O}$. The reflection across the x -axis sends an affine point (x, y) to $(x, -y)$, which (as we shall see) is its additive inverse; and it sends $\mathcal{O}$ to $\mathcal{O}$.

To convert this geometry into formulas one must (i) find the third intersection point of a line with the cubic and (ii) handle the degenerate cases. The key algebraic device is Vieta’s formula relating the roots of a cubic to its coefficients.

10.4 Explicit affine formulas

Fix an elliptic curve $E : y^2 = x^3 + ax + b$ over K . Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ be affine points.

Negation. Define

$$-\mathcal{O} = \mathcal{O}, \quad -(x_1, y_1) = (x_1, -y_1).$$

Note $(x_1, -y_1)$ is again on the curve since the equation depends on y only through y^2 . The line through P and $-P$ is the vertical line $x = x_1$, which meets the curve at P , $-P$, and $\mathcal{O}$.

Addition (the generic chord). Suppose $P, Q \neq \mathcal{O}$ and $x_1 \neq x_2$. The line through P and Q is $y = \lambda x + \mu$ with

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad \mu = y_1 - \lambda x_1.$$

Substituting into the curve equation, $(\lambda x + \mu)^2 = x^3 + ax + b$, gives

$$x^3 - \lambda^2 x^2 + (a - 2\lambda\mu)x + (b - \mu^2) = 0.$$

This cubic in x has the three roots x_1, x_2, x_3 , where x_3 is the x -coordinate of the third intersection point R' . By Vieta’s formula the sum of the roots equals the negative of the x^2 -coefficient, so $x_1 + x_2 + x_3 = \lambda^2$. Hence

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1, \quad (13)$$

where the y -formula already incorporates the reflection across the x -axis: the third intersection point is $(x_3, \lambda x_3 + \mu)$, and reflecting gives $y_3 = -(\lambda x_3 + \mu) = -(\lambda x_3 + y_1 - \lambda x_1) = \lambda(x_1 - x_3) - y_1$. Then $P + Q = (x_3, y_3)$.

Doubling (the tangent). Suppose $P = Q = (x_1, y_1)$ with $y_1 \neq 0$. Implicit differentiation of $y^2 = x^3 + ax + b$ yields the slope of the tangent line: $2y dy = (3x^2 + a) dx$, so

$$\lambda = \frac{3x_1^2 + a}{2y_1}.$$

The same Vieta computation now uses a double root at x_1 , giving $x_1 + x_1 + x_3 = \lambda^2$, hence

$$x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1, \quad [2]P = (x_3, y_3). \quad (14)$$

Edge cases. The remaining cases are exactly where the line is vertical:

- If $P = \mathcal{O}$, then $P + Q = Q$; symmetrically if $Q = \mathcal{O}$.
- If $x_1 = x_2$ but $y_1 = -y_2$ (so $Q = -P$, including the subcase $y_1 = y_2 = 0$), the line is vertical, the third point is $\mathcal{O}$, and $P + Q = \mathcal{O}$.
- If $P = Q$ and $y_1 = 0$, the tangent is vertical (the derivative slope formula has zero denominator), so $[2]P = \mathcal{O}$. These are exactly the points of order two.

Every case is covered: either one input is $\mathcal{O}$; or both are affine with $x_1 \neq x_2$ (use (13)); or both are affine with $x_1 = x_2$, in which case either $y_1 = -y_2$ (result $\mathcal{O}$) or $y_1 = y_2 \neq 0$ (use (14)).

Proposition 10.7 (closure). *For all $P, Q \in E(K)$, the point $P + Q$ computed above lies in $E(K)$. In particular $E(K)$ is closed under $+$.*

Proof. If either input is $\mathcal{O}$ the result is the other input, which lies in $E(K)$ by hypothesis; if the result is $\mathcal{O}$ it lies in $E(K)$ by definition. Otherwise (x_3, y_3) follows from (13) or (14). By construction $(x_3, -y_3)$ (the third intersection point R') satisfies both the line equation and the curve equation, hence lies on the curve; since the curve equation is invariant under $y \mapsto -y$, the reflected point (x_3, y_3) is on the curve too, so $P + Q \in E^{\text{aff}}(K) \subseteq E(K)$. All coordinates are rational expressions in x_i, y_i, a with denominators $x_2 - x_1$ or $2y_1$ that are nonzero in the relevant cases, so $x_3, y_3 \in K$. $\square$

Remark 10.8 (Complete versus incomplete addition). The chord formula (13) is *incomplete*: it fails whenever $x_1 = x_2$ (the doubling and inverse cases), and an implementation must branch into the case analysis above. *Complete* addition formulas — single rational expressions valid for *all* pairs of inputs, including $P = Q$ and $P = -Q$ — exist for the curves used here, at the cost of more field multiplications per addition. The distinction is load-bearing in this monograph’s later volumes: Halo 2’s elliptic-curve gadgets use complete formulas where an adversarially chosen input could otherwise hit an exceptional case, while the Sinsemilla hash function of Orchard deliberately uses the *incomplete* formulas nondeterministically: an exceptional case does not violate the constraints but leaves the affected output unconstrained (the prover may choose the intermediate slope freely), which the specification models by letting the hash return $\perp$ and weakening the proved statement accordingly — its Merkle path validity check is even permitted to pass on the resulting sentinel values. Safety rests not on the proof system but on the specification’s Theorem 5.4.4: any input reaching an exceptional case immediately yields a nontrivial discrete-logarithm relation among the Sinsemilla generators, so under discrete-log hardness on Pallas such inputs are infeasible to find for honest and adversarial provers alike.

10.5 Identity, inverses, commutativity, and associativity

Proposition 10.9 (identity and inverses). *The operation $+$ on $E(K)$ has the following properties.*

1. $\mathcal{O}$ is a two-sided identity: $P + \mathcal{O} = \mathcal{O} + P = P$ for all P .

2. Every P has a two-sided inverse $-P$: $P + (-P) = \mathcal{O}$, where $-\mathcal{O} = \mathcal{O}$ and $-(x, y) = (x, -y)$.
3. The operation is commutative: $P + Q = Q + P$.

Proof. (1) follows immediately from the definition. (2): for $P = (x, y)$ with $y \neq 0$, the line through P and $-P = (x, -y)$ is vertical and meets the curve at $\mathcal{O}$ as the third point, so $P + (-P) = \mathcal{O}$; for $y = 0$, $P = -P$ and $[2]P = \mathcal{O}$ directly; and $\mathcal{O} + \mathcal{O} = \mathcal{O}$. (3): the chord through P and Q does not depend on their order, and neither does the tangent in the doubling case; algebraically, formulas (13) are symmetric under the swap $(x_1, y_1) \leftrightarrow (x_2, y_2)$ (the slope λ is unchanged, and $x_3 = \lambda^2 - x_1 - x_2$ is symmetric). $\square$

Theorem 10.10 (group law). *Let E be an elliptic curve over a field K . The set $E(K)$ with the operation $+$ of Definition 10.6 is an abelian group with identity $\mathcal{O}$.*

The only group axiom not yet established is *associativity*, $(P + Q) + R = P + (Q + R)$. This is the deep and surprising part of the theory: the definition gives no reason why reflecting third intersection points should be an associative operation, and a brute-force verification through the explicit formulas requires splitting into a large number of coordinate cases and checking lengthy rational-function identities. We sketch below the conceptual proof, which exhibits the reason associativity holds.

Proof of associativity (sketch). *Sketch.* The proof uses the geometry of cubic curves. The key tool is the following *Cayley–Bacharach*-type fact: if two cubic curves C_1, C_2 in $\mathbb{P}^2$ meet in exactly nine points, and a third cubic C passes through eight of those nine points, then C passes through the ninth as well. (This follows from a dimension count: cubics form a 9-dimensional projective space of coefficients, and passing through a point is one linear condition; eight general points impose independent conditions, pinning down the pencil through them.)

To prove $(P + Q) + R = P + (Q + R)$, one writes both sides via the chord-and-tangent construction and arranges eight common intersection points among three suitably chosen lines (a degenerate cubic, namely a product of three lines) and the curve E ; the Cayley–Bacharach principle then forces the two candidate sums to share their final coordinate, hence to be equal. Translating “reflect across the x -axis” into “the three points on a line sum to $\mathcal{O}$ ” renders the bookkeeping uniform. We obtain a complete, case-free treatment by identifying $E(K)$ with the degree-zero part $\text{Pic}^0(E)$ of the divisor class group of the curve, where the group law is manifestly associative because it is inherited from addition of divisors modulo principal divisors; the map $P \mapsto [P] - [\mathcal{O}]$ is a bijection $E(K) \rightarrow \text{Pic}^0(E)$ intertwining the two operations. Either route yields a genuine proof; both lie beyond the elementary scope here, so we take associativity as established. $\square$

Remark 10.11. For $n \in \mathbb{Z}$ and $P \in E(K)$ we write $[n]P$ for the n -fold sum $P + \cdots + P$ (with $[0]P = \mathcal{O}$ and $[-n]P = -([n]P)$). This makes $E(K)$ a module over $\mathbb{Z}$, i.e. its abelian-group structure written multiplicatively-by-integers. Note that $[n]P$ is the *scalar multiplication* developed below (Definition 10.13), not a coordinate-wise product.

10.6 Projective and Jacobian coordinates

The affine formulas (13) and (14) each require a field *inversion* (dividing by $x_2 - x_1$ or $2y_1$). In a large prime field $\mathbb{F}_p$, an inversion costs roughly as much as dozens of multiplications (the implementation typically computes it via the extended Euclidean algorithm or as z^{p-2} by Fermat). When a long chain of additions arises, as in scalar multiplication, it is advantageous to defer all inversions to the very end by carrying points in a redundant coordinate system where the group law uses only additions, subtractions, and multiplications.

Remark 10.12 (Inversion-free coordinate systems). Two redundant coordinate systems serve this purpose. In *(homogeneous) projective coordinates*, a triple $[X : Y : Z]$ with $Z \neq 0$ represents the

affine point $(X/Z, Y/Z)$ on the projective curve (12), with $\mathcal{O} = [0 : 1 : 0]$. In *Jacobian coordinates*, $(X : Y : Z)$ with $Z \neq 0$ represents $(X/Z^2, Y/Z^3)$, under the equivalence $(X, Y, Z) \sim (\lambda^2 X, \lambda^3 Y, \lambda Z)$, the curve equation becoming $Y^2 = X^3 + aXZ^4 + bZ^6$. In either system the addition and doubling formulas can be rewritten with no divisions at all, at the cost of more multiplications per operation; a long chain of group operations then defers all inversions to a single final recovery of affine coordinates (compute Z^{-1} once, then the needed powers by multiplication). Jacobian coordinates usually yield the faster doubling and are the standard choice in cryptographic libraries, including the implementations used by Halo 2. For curves $y^2 = x^3 + b$ with $a = 0$, the term aZ^4 in the Jacobian doubling formulas vanishes, making doubling especially cheap — one reason the Pasta curves take $a = 0$.

10.7 Scalar multiplication and double-and-add

The fundamental operation in elliptic-curve cryptography is *scalar multiplication*: given a point P and an integer $n \geq 0$, compute

$$[n]P = \underbrace{P + P + \cdots + P}_{n \text{ times}}.$$

Computing this by $n - 1$ successive additions is infeasible when n has hundreds of bits. Instead one uses the binary expansion of n , exactly as in fast modular exponentiation.

Definition 10.13 (double-and-add). Write $n = \sum_{i=0}^{k-1} n_i 2^i$ with $n_i \in \{0, 1\}$ and $n_{k-1} = 1$. The *left-to-right double-and-add* algorithm computes $[n]P$ as follows: set $R \leftarrow \mathcal{O}$; for $i = k - 1$ down to 0, set $R \leftarrow [2]R$, and if $n_i = 1$ set $R \leftarrow R + P$; return R .

Proposition 10.14. *Double-and-add returns $[n]P$ using at most $k - 1$ doublings and at most $k - 1$ additions, where $k = \lfloor \log_2 n \rfloor + 1$ is the bit length of n .*

Proof. Let R_j be the value of R after processing bits $n_{k-1}, \dots, n_j$. We claim $R_j = \lfloor n/2^j \rfloor P$, and prove this by downward induction on j . For the top bit, after $i = k - 1$ one has $R = [2]\mathcal{O} + n_{k-1}P = [n_{k-1}]P = [1]P = \lfloor n/2^{k-1} \rfloor P$. Assuming $R_{j+1} = \lfloor n/2^{j+1} \rfloor P$, the next iteration computes $[2]R_{j+1} + [n_j]P = \lfloor 2\lfloor n/2^{j+1} \rfloor + n_j \rfloor P = \lfloor n/2^j \rfloor P$, since $2\lfloor n/2^{j+1} \rfloor + n_j = \lfloor n/2^j \rfloor$. At $j = 0$ one obtains $R_0 = [n]P$. The loop runs $k - 1$ times after the initial top bit is consumed, giving the stated counts (the doubling of $\mathcal{O}$ and addition for the top bit are free/trivial). $\square$

Thus scalar multiplication by an n -bit scalar costs $O(n)$ group operations — hence $O(n)$ field operations up to constant factors, or $O(n \log^2 p)$ bit operations with schoolbook field arithmetic. This efficiency in the “forward” direction, contrasted with the apparent hardness of the inverse problem (§10.10), is the foundation of the cryptography.

10.8 The group structure of $E(\mathbb{F}_p)$

Now specialize $K = \mathbb{F}_p$, the finite field with p elements (p prime, $p > 3$). Since $\mathbb{F}_p$ is finite, so is $E(\mathbb{F}_p)$: it has at most $2p + 1$ points (for each of the p values of x , at most two values of y , plus $\mathcal{O}$). Theorem 10.10 makes $E(\mathbb{F}_p)$ a finite abelian group. One of the central theorems of the subject governs its order.

Theorem 10.15 (Hasse). *Let E be an elliptic curve over $\mathbb{F}_p$. Write*

$$\#E(\mathbb{F}_p) = p + 1 - t,$$

which defines the integer t , the trace of Frobenius. Then

$$|t| \leq 2\sqrt{p}.$$

Equivalently, $\#E(\mathbb{F}_p)$ lies in the Hasse interval $[p + 1 - 2\sqrt{p}, p + 1 + 2\sqrt{p}]$.

Proof (sketch). *Sketch.* Heuristically, as x ranges over $\mathbb{F}_p$, the value $x^3 + ax + b$ is a nonzero square about half the time (giving two points), a nonsquare about half the time (giving none), and zero occasionally (giving one). Using the Legendre symbol $\left(\frac{\cdot}{p}\right)$, the number of affine points is

$$\#E^{\text{aff}}(\mathbb{F}_p) = \sum_{x \in \mathbb{F}_p} \left(1 + \left(\frac{x^3 + ax + b}{p}\right)\right) = p + \sum_{x \in \mathbb{F}_p} \left(\frac{x^3 + ax + b}{p}\right),$$

so $t = -\sum_x \left(\frac{x^3 + ax + b}{p}\right)$. The sum is a character sum that exhibits square-root cancellation; the sharp bound $|t| \leq 2\sqrt{p}$ is Hasse's theorem. The conceptual proof realizes t as the trace of the p -power Frobenius endomorphism $\pi : (x, y) \mapsto (x^p, y^p)$ acting on the curve. One shows π satisfies the characteristic equation $\pi^2 - [t]\pi + [p] = 0$ in the endomorphism ring, and that the associated quadratic form (the degree) is positive definite; the discriminant condition $t^2 - 4p \leq 0$ for the (complex) roots then yields $|t| \leq 2\sqrt{p}$. The full argument requires the theory of isogenies and the Weil pairing, beyond the present scope. $\square$

Hasse's theorem states that the order of $E(\mathbb{F}_p)$ is *very close* to p : it cannot deviate from $p + 1$ by more than $2\sqrt{p}$, a relative error of order $p^{-1/2}$. For cryptography this is crucial: it establishes that the group is large (about p elements, so about 255 bits for the Pasta curves) without any brute-force point count, and point-counting algorithms (Schoof–Elkies–Atkin) compute the *exact* order in polynomial time.

For the cryptographic application one wants $E(\mathbb{F}_p)$ to contain a large *prime-order* subgroup. The general structure theory of $E(\mathbb{F}_p)$ as an abelian group is not needed here, because the Pasta curves are designed so that $\#E(\mathbb{F}_p)$ is itself prime: a group of prime order is cyclic (Corollary 3.31), so $E(\mathbb{F}_p)$ is cyclic of prime order, the optimal case.

10.9 Torsion, the cofactor, and prime-order subgroups

Definition 10.16 (order of a point, torsion). The *order* of a point $P \in E(\mathbb{F}_p)$ is the least integer $m \geq 1$ with $[m]P = \mathcal{O}$ (it exists and divides $\#E(\mathbb{F}_p)$ by Lagrange's theorem). For an integer $m \geq 1$, the *m -torsion subgroup* is

$$E[m] = \{P \in E(\overline{\mathbb{F}_p}) : [m]P = \mathcal{O}\},$$

and $E(\mathbb{F}_p)[m] = E[m] \cap E(\mathbb{F}_p)$ is its rational part.

Definition 10.17 (cofactor). Suppose $\#E(\mathbb{F}_p) = h \cdot r$ where r is the (large) prime to be used and h is the remaining factor. The integer h is the *cofactor*, and r the *order* of the prime-order subgroup. A point P of order r generates a cyclic subgroup $\langle P \rangle \cong \mathbb{Z}/r\mathbb{Z}$, the *prime-order subgroup* used in cryptography.

A nontrivial cofactor $h > 1$ is undesirable: it admits “small-subgroup” points (of order dividing h) that can leak information or break protocol assumptions, so implementations either clear the cofactor by multiplying inputs by h , or use prime-order curves where $h = 1$. The Pasta curves have *cofactor* 1: $\#E_p(\mathbb{F}_p) = q$ and $\#E_q(\mathbb{F}_q) = p$ are themselves prime, so $E_p(\mathbb{F}_p) \cong \mathbb{Z}/q\mathbb{Z}$ and $E_q(\mathbb{F}_q) \cong \mathbb{Z}/p\mathbb{Z}$ are cyclic of prime order and no cofactor clearing is needed. This is one of the principal design advantages of the Pasta cycle for proof systems.

Proposition 10.18 (two-torsion). *The affine points of order 2 in $E(\mathbb{F}_p)$ are exactly the points $(x_0, 0)$ with $f(x_0) = x_0^3 + ax_0 + b = 0$. Hence $E(\mathbb{F}_p)$ has 0, 1, or 3 points of order 2 according as f has 0, 1, or 3 roots in $\mathbb{F}_p$, and $\#E(\mathbb{F}_p)$ is even iff f has a root in $\mathbb{F}_p$.*

Proof. $P = (x, y)$ has order 2 iff $P = -P = (x, -y)$ and $P \neq \mathcal{O}$, i.e. $y = -y$, i.e. $y = 0$ (as $\text{char} \neq 2$); the curve equation then forces $f(x) = 0$. Together with $\mathcal{O}$ these form the 2-torsion subgroup $E(\mathbb{F}_p)[2]$, which is trivial, $\mathbb{Z}/2\mathbb{Z}$, or $(\mathbb{Z}/2\mathbb{Z})^2$. For the parity claim, negation $P \mapsto -P$ is an involution of $E(\mathbb{F}_p)$ whose fixed points are exactly $\mathcal{O}$ and the points $(x_0, 0)$; all other points pair off into 2-element orbits, so $\#E(\mathbb{F}_p) \equiv 1 + N \pmod{2}$, where $N \in \{0, 1, 3\}$ is the number of roots of f in $\mathbb{F}_p$. Hence $\#E(\mathbb{F}_p)$ is even iff N is odd, i.e. iff $N \geq 1$, i.e. iff f has a root in $\mathbb{F}_p$. $\square$

Remark 10.19. For $y^2 = x^3 + 5$ over $\mathbb{F}_p$: the curve has a point of order 2 iff $x^3 + 5 = 0$ has a solution, i.e. iff -5 is a cube in $\mathbb{F}_p$. For the Pasta primes $\#E_p(\mathbb{F}_p) = q$ is odd (in fact prime), so there are no rational 2-torsion points; consistently, -5 is not a cube in $\mathbb{F}_p$.

10.10 The elliptic-curve discrete logarithm problem

Definition 10.20 (ECDLP). Let $P \in E(\mathbb{F}_p)$ be a point of large prime order r , generating $G = \langle P \rangle$. The *elliptic-curve discrete logarithm problem* (ECDLP) is: given P and a point $Q \in G$, find the unique integer $n \in \{0, 1, \dots, r-1\}$ with $Q = [n]P$. One writes $n = \log_P Q$.

The map $n \mapsto [n]P$ is a group isomorphism $\mathbb{Z}/r\mathbb{Z} \rightarrow G$, so a discrete logarithm always exists and is unique; the content lies in its *computational* difficulty. Scalar multiplication (the forward map) is fast by double-and-add, but no efficient algorithm is known to invert it on a well-chosen elliptic curve.

Proposition 10.21 (square-root attacks). *The baby-step giant-step method solves the ECDLP in a group of prime order r in $O(\sqrt{r})$ group operations and $O(\sqrt{r})$ space; Pollard’s rho method achieves the same expected time with $O(1)$ space.*

Proof (sketch). Baby-step giant-step writes $n = i[\sqrt{r}] + j$ with $0 \leq i, j < [\sqrt{r}]$, precomputes the “baby steps” $[j]P$ in a table, and matches against the “giant steps” $Q - [i[\sqrt{r}]]P$; a collision reveals n . Pollard’s rho achieves the same time with negligible space by a pseudo-random walk on G and the birthday paradox (a set of size r sees a repeat among $O(\sqrt{r})$ uniform samples with constant probability; Proposition 11.18 makes this quantitative): a self-collision in the walk yields a relation $[a]P + [b]Q = [a']P + [b']Q$, from which $n \equiv (a - a')(b' - b)^{-1} \pmod{r}$. Both are exponential in the bit length $\log_2 r$. $\square$

Remark 10.22 (Absence of better attacks). That nothing beats the square-root bound on a well-chosen curve is an empirical claim, not a theorem: after four decades of study, no algorithm faster than $O(\sqrt{r})$ is known for general elliptic curves. For r of cryptographic size ($r \approx 2^{254}$, the 255-bit Pasta orders) this is $\approx 2^{127}$ operations, infeasible. Curves with special structure can be weaker: if $\#E(\mathbb{F}_p) = p$ (anomalous curves) there is a polynomial-time attack via the formal group, and the MOV/Frey–Rück attack transfers the ECDLP to a finite-field DLP when the *embedding degree*—the least $k \geq 1$ with $r \mid p^k - 1$, the degree of the smallest extension of $\mathbb{F}_p$ whose multiplicative group contains a subgroup of order r —is small. Cryptographically secure curves, including the Pasta curves, are chosen to have a large prime subgroup order, large embedding degree, and $\#E(\mathbb{F}_p) \neq p$, defeating all known subexponential attacks.

Remark 10.23. The contrast with multiplicative groups $\mathbb{F}_p^\times$ is essential: in $\mathbb{F}_p^\times$ the index-calculus method solves the discrete logarithm in subexponential time $L_p[1/3]$, forcing those groups to be very large. On a well-chosen elliptic curve no index calculus is known, so the generic $O(\sqrt{r})$ bound stands

and much smaller groups suffice (about 256 bits for 128-bit security, versus thousands of bits for $\mathbb{F}_p^\times$). Smaller groups mean faster arithmetic and shorter keys, which is why elliptic curves dominate modern protocols, including Halo 2.

10.11 The j -invariant and the GLV endomorphism

Definition 10.24 (j -invariant). The j -invariant of the curve $y^2 = x^3 + ax + b$ (with $4a^3 + 27b^2 \neq 0$) is

$$j = 1728 \frac{4a^3}{4a^3 + 27b^2} \in K.$$

Proposition 10.25. Two elliptic curves over $\bar{K}$ are isomorphic (over $\bar{K}$) if and only if they have the same j -invariant. The admissible coordinate changes preserving short Weierstrass form are $(x, y) \mapsto (u^2x, u^3y)$ for $u \in \bar{K}^\times$, under which $(a, b) \mapsto (u^{-4}a, u^{-6}b)$; the combination j is invariant.

Proof (sketch). *Sketch.* A direct computation shows the only isomorphisms between short Weierstrass curves fixing $\mathcal{O}$ are the scalings $(x, y) \mapsto (u^2x, u^3y)$; substituting into the equation gives $a \mapsto u^{-4}a$, $b \mapsto u^{-6}b$. Then $4a^3/(4a^3 + 27b^2)$ is unchanged because numerator and denominator both scale by u^{-12} . Conversely, given equal j -invariants one solves for a suitable u to construct an isomorphism (with separate easy cases $j = 0$ and $j = 1728$). $\square$

Remark 10.26. For $y^2 = x^3 + b$ one has $a = 0$, so $j = 0$. The Pasta curves both have $j = 0$. Curves with $j = 0$ are special: they admit an extra automorphism of order 3, which underlies the efficient endomorphism described next. (At the other extreme, $b = 0$ gives $j = 1728$ and an order-4 automorphism.)

The vanishing $j = 0$ is not accidental in the Pasta design; it is what makes the GLV (Gallant–Lambert–Vanstone) speedup available, halving the cost of scalar multiplication.

Proposition 10.27 (the GLV endomorphism for $y^2 = x^3 + b$). Suppose $\mathbb{F}_p$ contains a primitive cube root of unity ω , i.e. $p \equiv 1 \pmod{3}$, so $\omega \in \mathbb{F}_p$ with $\omega^2 + \omega + 1 = 0$. On $E : y^2 = x^3 + b$ the map

$$\phi(x, y) = (\omega x, y), \quad \phi(\mathcal{O}) = \mathcal{O},$$

is a group endomorphism of $E(\mathbb{F}_p)$. Let $G = \langle P \rangle$ be a subgroup of prime order r with $r \equiv 1 \pmod{3}$ and $\phi(G) \subseteq G$. Then ϕ acts on G as multiplication by a scalar: there is $\lambda \in \mathbb{Z}/r\mathbb{Z}$ with $\lambda^2 + \lambda + 1 \equiv 0 \pmod{r}$ and

$$\phi(Q) = [\lambda]Q \quad \text{for all } Q \in G.$$

For the Pasta curves the hypothesis $\phi(G) \subseteq G$ is automatic, since $E(\mathbb{F}_p)$ itself is the prime-order group G , and $r \equiv 1 \pmod{3}$ holds.

Proof. First, ϕ maps the curve to itself: if $y^2 = x^3 + b$ then $(y)^2 = (\omega x)^3 + b$ because $\omega^3 = 1$, so $(\omega x, y) \in E$. One can check that ϕ is a homomorphism from the addition formulas (13): scaling all x -coordinates by ω and leaving y -coordinates fixed is compatible with the slope $\lambda_{\text{line}} = (y_2 - y_1)/(\omega x_2 - \omega x_1) = \omega^{-1} \lambda_{\text{line}}^{\text{old}}$ and with $x_3 = \lambda^2 - x_1 - x_2 \mapsto \omega^{-2} \lambda^2 - \omega x_1 - \omega x_2 = \omega \lambda^2 - \omega x_1 - \omega x_2 = \omega(\dots)$, using $\omega^{-2} = \omega$; a short computation confirms $\phi(P_1 + P_2) = \phi(P_1) + \phi(P_2)$. Moreover $\phi^2 + \phi + 1 = 0$ as an endomorphism: for affine $P = (x, y)$ the cubic $X^3 + b - y^2$ has no X^2 term, so by Vieta its three roots sum to 0; since $\omega^2 + \omega + 1 = 0$ these roots are $x, \omega x, \omega^2 x$, whence $P, \phi(P), \phi^2(P)$ are precisely the intersections (with multiplicity) of the horizontal line $Y = y$ with E , and by the rule following Definition 10.6 they sum to $\mathcal{O}$. Also ϕ has order 3: $\phi^3 = \text{id}$ since $\omega^3 = 1$, and $\phi \neq \text{id}$ since $E(\mathbb{F}_p)$

contains a point with $x \neq 0$ (at most three points have $x = 0$). Restricted to the cyclic group $G \cong \mathbb{Z}/r\mathbb{Z}$ (which ϕ preserves by hypothesis), every endomorphism is multiplication by some $\lambda \in \mathbb{Z}/r\mathbb{Z}$ (as $\text{Hom}(\mathbb{Z}/r\mathbb{Z}, \mathbb{Z}/r\mathbb{Z}) \cong \mathbb{Z}/r\mathbb{Z}$); the relation $\phi^2 + \phi + 1 = 0$ becomes $\lambda^2 + \lambda + 1 \equiv 0 \pmod{r}$. Consistently, this congruence is solvable whenever $r \equiv 1 \pmod{3}$, the hypothesis of the statement. $\square$

Remark 10.28 (how GLV speeds up scalar multiplication). Because computing ϕ takes a single field multiplication $x \mapsto \omega x$ (negligible compared to a group operation), one can decompose a scalar $n \in \mathbb{Z}/r\mathbb{Z}$ as

$$[n]P = [n_1]P + [n_2]\phi(P), \quad n \equiv n_1 + n_2\lambda \pmod{r},$$

where n_1, n_2 are each only about $\frac{1}{2} \log_2 r$ bits long (a lattice reduction in the lattice of (a, b) with $a + b\lambda \equiv 0 \pmod{r}$ finds them). The two half-length scalar multiplications, run simultaneously with shared doublings (Straus–Shamir’s trick), cost about half the doublings of a single full-length double-and-add. This is a substantial speedup, and its availability is one reason the Pasta curves take the form $y^2 = x^3 + b$ with $p \equiv 1 \pmod{3}$.

10.12 A worked toy example

We illustrate the entire machinery below on a curve small enough to compute by hand, followed by the corresponding facts for the Pasta curves.

Example 10.29 (the curve $y^2 = x^3 + 5$ over $\mathbb{F}_{11}$). Take $p = 11$ (note $11 > 3$, and $4 \cdot 0 + 27 \cdot 5^2 = 675 \equiv 675 - 61 \cdot 11 = 4 \not\equiv 0 \pmod{11}$, so the curve is nonsingular). The squares in $\mathbb{F}_{11}$ are

$$\{0^2, \dots, 10^2\} = \{0, 1, 4, 9, 5, 3\},$$

i.e. the nonzero quadratic residues are $\{1, 3, 4, 5, 9\}$. Tabulating $f(x) = x^3 + 5 \pmod{11}$ and reading off the y -values:

x	$x^3 \pmod{11}$	$f(x) = x^3 + 5$	y with $y^2 = f(x)$
0	0	5	± 4 ($4^2 = 16 \equiv 5$)
1	1	6	none ($6 \notin \text{QR}$)
2	8	2	none
3	5	10	none
4	9	3	± 5 ($5^2 = 25 \equiv 3$)
5	4	9	± 3
6	7	1	± 1
7	2	7	none
8	6	0	0
9	3	8	none
10	10	4	± 2

Counting: $x \in \{0, 4, 5, 6, 10\}$ each give 2 points (10 points), $x = 8$ gives 1 point $(8, 0)$, and adding $\mathcal{O}$ yields

$$\#E(\mathbb{F}_{11}) = 10 + 1 + 1 = 12.$$

Hasse’s bound: $p + 1 = 12$, so $t = 0$ and indeed $|t| = 0 \leq 2\sqrt{11} \approx 6.63$. The single point $(8, 0)$ has order 2 (here $-5 \equiv 6$ is a cube, namely $8^3 = 512 \equiv 6$), consistent with $\#E$ being even.

An addition. Let $P = (0, 4)$ and $Q = (5, 3)$. Since $x_1 = 0 \neq 5 = x_2$, use the chord:

$$\lambda = \frac{3 - 4}{5 - 0} = \frac{-1}{5} = (-1) \cdot 5^{-1} = (-1) \cdot 9 = -9 \equiv 2 \pmod{11},$$

since $5 \cdot 9 = 45 \equiv 1$. Then

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2 = 4 - 0 - 5 = -1 \equiv 10, \\ y_3 &= \lambda(x_1 - x_3) - y_1 = 2(0 - 10) - 4 = -24 \equiv 9 \pmod{11}. \end{aligned}$$

So $P + Q = (10, 9)$, which is on the curve: $9^2 = 81 \equiv 4$ and $10^3 + 5 = 1005 \equiv 4 \pmod{11}$, as required.

A doubling. Take $P = (0, 4)$. With $a = 0$,

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{0}{8} = 0, \quad x_3 = \lambda^2 - 2x_1 = 0, \quad y_3 = \lambda(x_1 - x_3) - y_1 = -4 \equiv 7.$$

So $[2](0, 4) = (0, 7) = -(0, 4)$; hence $[3](0, 4) = \mathcal{O}$, so $P = (0, 4)$ has order 3.

The GLV endomorphism. Since $11 \equiv 2 \pmod{3}$, there is *no* primitive cube root of unity in $\mathbb{F}_{11}$, so the GLV map is not available over $\mathbb{F}_{11}$; it would require $p \equiv 1 \pmod{3}$. (The Pasta primes do satisfy $p, q \equiv 1 \pmod{3}$, so GLV applies there.) For a small GLV-friendly illustration one could instead take $p = 13 \equiv 1 \pmod{3}$, where $\omega = 3$ satisfies $3^2 + 3 + 1 = 13 \equiv 0$, and then $\phi(x, y) = (3x, y)$ is a nontrivial endomorphism of $y^2 = x^3 + b$ over $\mathbb{F}_{13}$.

Example 10.30 (the Pasta curves). For the running example $y^2 = x^3 + 5$:

- Over $\mathbb{F}_p$ (Pallas, with p the 255-bit Pasta prime): $\#E_p(\mathbb{F}_p) = q$, a prime; the group is cyclic, $E_p(\mathbb{F}_p) \cong \mathbb{Z}/q\mathbb{Z}$; cofactor $h = 1$; the trace is $t = p + 1 - q$, with $|t| \leq 2\sqrt{p}$.
- Over $\mathbb{F}_q$ (Vesta): $\#E_q(\mathbb{F}_q) = p$, a prime; $E_q(\mathbb{F}_q) \cong \mathbb{Z}/p\mathbb{Z}$; cofactor 1.
- Both have $j = 0$ (since $a = 0$), admit the order-3 endomorphism $\phi(x, y) = (\omega x, y)$ with ω a primitive cube root of unity (which exists since $p, q \equiv 1 \pmod{3}$), and so support the GLV speedup.
- The “cycle” property $\#E_p(\mathbb{F}_p) = q$ and $\#E_q(\mathbb{F}_q) = p$ means the scalar field of one curve is the base field of the other and vice versa; this is exactly what lets recursive proof systems chain proofs across the two curves without expensive non-native field arithmetic. The base field of Pallas is $\mathbb{F}_p$ and its scalar field (the order of the group) is $\mathbb{F}_q$, matching the base field of Vesta, and symmetrically.

The discrete logarithm problem in $E_p(\mathbb{F}_p)$ and $E_q(\mathbb{F}_q)$ is believed to require about 2^{127} operations ($\sqrt{r}$ for the 255-bit $r \approx 2^{254}$), just below the 128-bit level targeted by Halo 2 and Zcash Orchard.

10.13 Summary

We have constructed the elliptic curve group from the ground up: a nonsingular short Weierstrass cubic over a field of characteristic $\neq 2, 3$, its affine points together with a point at infinity, and the chord-and-tangent addition with explicit formulas (13)–(14) and all edge cases. The point at infinity is the identity, (x, y) and $(x, -y)$ are inverses, commutativity is immediate, and associativity, though geometrically subtle, makes $E(K)$ an abelian group. Projective and Jacobian coordinates remove inversions from the inner loop, and double-and-add computes $[n]P$ in $O(\log n)$ group operations. Over $\mathbb{F}_p$ the group has order $p + 1 - t$ with $|t| \leq 2\sqrt{p}$ (Hasse); choosing the order to be prime (cofactor 1, as for the Pasta curves) yields a clean cyclic group in which the discrete logarithm problem is hard. Finally, the $j = 0$ curves $y^2 = x^3 + b$ admit the efficiently computable GLV endomorphism. The Pasta curves $y^2 = x^3 + 5$ over $\mathbb{F}_p$ and $\mathbb{F}_q$ realize all of these features simultaneously and form the 2-cycle that carries the proofs of Halo 2 and Zcash Orchard and was designed to support recursive proof composition.

11 Probability, asymptotics, and computation

Cryptography is the discipline of rendering certain computations easy for legitimate parties and infeasible for adversaries. Stating and proving such guarantees requires three intertwined languages: the language of *probability*, which permits reasoning about random keys, random challenges, and the probability that an attack succeeds; the language of *asymptotics*, which permits precise description of how resources and success probabilities scale; and the language of *computation*, which fixes the notion of an efficient algorithm and of what it means for a problem to be hard. This section develops all three from elementary foundations. The only prerequisites are familiarity with finite sets, sums, and elementary calculus — limits, the exponential function, and (for the closing subsection’s density-defined laws) the improper Riemann integral; we construct everything else here.

Until the closing probability subsection, all probability spaces are finite. This is no serious restriction for the cryptographic material: keys, messages, group elements, and the random coins of any terminating algorithm all reside in finite sets, and finite spaces need no measure theory. The closing subsection (§11.7) then extends the theory by the rungs later volumes need—countable additivity, continuity along monotone events, the waiting-time toolkit, and laws defined by densities—declaring its one measure-theoretic debt in place (Remark 11.29).

11.1 Finite probability spaces and events

The basic object is a finite set of *outcomes* together with an assignment of weights summing to one.

Definition 11.1 (Finite probability space). A *finite probability space* is a pair (Ω, Pr) where Ω is a nonempty finite set, called the *sample space*, and $\text{Pr} : \Omega \rightarrow [0, 1]$ is a function, called the *probability mass function*, satisfying the normalisation condition

$$\sum_{\omega \in \Omega} \text{Pr}[\omega] = 1.$$

We call an element $\omega \in \Omega$ an *outcome* or an *elementary event*.

Definition 11.2 (Event). An *event* is a subset $A \subseteq \Omega$. Its *probability* is

$$\text{Pr}[A] = \sum_{\omega \in A} \text{Pr}[\omega],$$

with the convention $\text{Pr}[\emptyset] = 0$ (an empty sum). The event A *occurs* on outcome ω if $\omega \in A$.

Because Ω is finite, every subset is an event. This is a convenience worth flagging: on an uncountable sample space not every subset can consistently be assigned a probability, so the general theory must fix in advance a family of admissible events; finiteness spares us that machinery, and Remark 11.29 marks the one place this series touches it. The following elementary facts hold, and we use them freely.

Proposition 11.3 (Basic axioms). For any finite probability space (Ω, Pr) and events $A, B \subseteq \Omega$:

1. $0 \leq \text{Pr}[A] \leq 1$, $\text{Pr}[\emptyset] = 0$, and $\text{Pr}[\Omega] = 1$.
2. (Complement.) $\text{Pr}[\Omega \setminus A] = 1 - \text{Pr}[A]$.
3. (Monotonicity.) If $A \subseteq B$ then $\text{Pr}[A] \leq \text{Pr}[B]$.

4. (Inclusion–exclusion for two events.) $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$.
5. (Finite additivity.) If $A_1, \dots, A_n$ are pairwise disjoint then $\Pr[\bigcup_{i=1}^n A_i] = \sum_{i=1}^n \Pr[A_i]$.

Proof. Every claim follows by manipulating the defining sums. For (1), each summand $\Pr[\omega]$ lies in $[0, 1]$ and the total is 1 by normalisation; an event’s probability is a partial sum of nonnegative terms, hence in $[0, 1]$. For (2), $\Pr[A] + \Pr[\Omega \setminus A] = \sum_{\omega \in \Omega} \Pr[\omega] = 1$ since A and $\Omega \setminus A$ partition Ω . For (3), $\Pr[B] = \sum_{\omega \in B} \Pr[\omega] = \sum_{\omega \in A} \Pr[\omega] + \sum_{\omega \in B \setminus A} \Pr[\omega] \geq \Pr[A]$, the last term being a sum of nonnegative numbers. For (5), disjointness implies that each ω in the union lies in exactly one A_i , so the double sum $\sum_i \sum_{\omega \in A_i} \Pr[\omega]$ counts each $\omega \in \bigcup_i A_i$ exactly once. For (4), write $A \cup B = A \sqcup (B \setminus A)$ (disjoint), so by (5) $\Pr[A \cup B] = \Pr[A] + \Pr[B \setminus A]$; also $B = (A \cap B) \sqcup (B \setminus A)$, giving $\Pr[B \setminus A] = \Pr[B] - \Pr[A \cap B]$. Substituting yields the claim. $\square$

The single most important example, ubiquitous in cryptography, is the uniform distribution.

Definition 11.4 (Uniform distribution). Let S be a nonempty finite set. The *uniform distribution* on S is the probability space $(S, \Pr)$ with $\Pr[\omega] = 1/|S|$ for every $\omega \in S$. The notation $x \leftarrow S$ (or $x \overset{\$}{\leftarrow} S$) denotes that we sample x according to the uniform distribution on S . Under the uniform distribution, for any event $A \subseteq S$,

$$\Pr[A] = \frac{|A|}{|S|}.$$

Thus, for uniform sampling, probability is exactly the ratio of favourable outcomes to total outcomes, recovering the classical “counting” notion of probability.

11.2 Conditional probability and independence

Frequently we learn that one event has occurred, and we must update the probability of another accordingly. Conditioning captures this.

Definition 11.5 (Conditional probability). Let A, B be events with $\Pr[B] > 0$. The *conditional probability of A given B* is

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}.$$

When $\Pr[B] = 0$ we leave the quantity undefined.

Conditioning on B restricts the sample space to B and renormalises so that B has probability 1: the map $\omega \mapsto \Pr[\{\omega\} \mid B]$ is itself a probability mass function on B , and $\Pr[A \mid B]$ is the probability it assigns to $A \cap B$. Two immediate and indispensable consequences follow.

Proposition 11.6 (Chain rule and total probability). Let $A_1, \dots, A_n$ be events with $\Pr[A_1 \cap \dots \cap A_{n-1}] > 0$. Then

$$\Pr[A_1 \cap \dots \cap A_n] = \Pr[A_1] \Pr[A_2 \mid A_1] \Pr[A_3 \mid A_1 \cap A_2] \cdots \Pr[A_n \mid A_1 \cap \dots \cap A_{n-1}].$$

Moreover, if $B_1, \dots, B_k$ partition Ω (pairwise disjoint with union Ω) and each $\Pr[B_i] > 0$, then for any event A ,

$$\Pr[A] = \sum_{i=1}^k \Pr[A \mid B_i] \Pr[B_i].$$

Proof. The chain rule follows by telescoping: each factor $\Pr[A_j \mid A_1 \cap \dots \cap A_{j-1}]$ equals $\Pr[A_1 \cap \dots \cap A_j] / \Pr[A_1 \cap \dots \cap A_{j-1}]$ by Definition 11.5, and the product collapses to $\Pr[A_1 \cap \dots \cap A_n] / 1$. (Monotonicity ensures all intermediate intersections have positive probability, so every factor is defined.) For total probability, the events $A \cap B_i$ are pairwise disjoint with union A (since the B_i partition Ω), so by finite additivity $\Pr[A] = \sum_i \Pr[A \cap B_i] = \sum_i \Pr[A \mid B_i] \Pr[B_i]$, using $\Pr[A \cap B_i] = \Pr[A \mid B_i] \Pr[B_i]$. $\square$

Independence is the formal statement that one event carries no information about another.

Definition 11.7 (Independence of events). Two events A, B are *independent* if

$$\Pr[A \cap B] = \Pr[A] \Pr[B].$$

A family $\{A_i\}_{i \in I}$ of events is *mutually independent* if for every finite subset $J \subseteq I$,

$$\Pr\left[\bigcap_{i \in J} A_i\right] = \prod_{i \in J} \Pr[A_i].$$

Remark 11.8. When $\Pr[B] > 0$, independence of A and B is equivalent to $\Pr[A \mid B] = \Pr[A]$: conditioning on B does not change the probability of A . Mutual independence is strictly stronger than pairwise independence. The standard counterexample is the following: toss two fair coins, and let A = “first is heads”, B = “second is heads”, C = “the two coins differ”. Each pair is independent, yet $\Pr[A \cap B \cap C] = 0 \neq \frac{1}{8} = \Pr[A] \Pr[B] \Pr[C]$, so the three are not mutually independent.

11.3 Random variables and expectation

A random variable assigns a value to each outcome; it summarises a random experiment by a number (or a vector, etc.).

Definition 11.9 (Random variable). Let $(\Omega, \Pr)$ be a finite probability space and T any set. A (T -valued) *random variable* is a function $X : \Omega \rightarrow T$. When $T \subseteq \mathbb{R}$, we call X *real-valued*. For $t \in T$,

$$\Pr[X = t] = \Pr[\{\omega \in \Omega : X(\omega) = t\}],$$

and more generally $\Pr[X \in B]$ denotes the probability that X takes a value in $B \subseteq T$. The function $t \mapsto \Pr[X = t]$ on the finite image of X is the *distribution* (or *law*) of X .

Definition 11.10 (Independent random variables). Random variables $X_1, \dots, X_n$ (with values in sets $T_1, \dots, T_n$) are *mutually independent* if for all $t_1 \in T_1, \dots, t_n \in T_n$,

$$\Pr[X_1 = t_1, \dots, X_n = t_n] = \prod_{i=1}^n \Pr[X_i = t_i].$$

Definition 11.11 (Expectation). Let $X : \Omega \rightarrow \mathbb{R}$ be a real-valued random variable on a finite space. Its *expectation* (or *mean*) is

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) \Pr[\omega] = \sum_{t \in \text{im}(X)} t \cdot \Pr[X = t],$$

where $\text{im}(X)$ denotes the (finite) image of X . The two expressions agree by grouping outcomes

according to their X -value.

A convenient and frequently used special case is the following: the expectation of the *indicator* of an event is exactly its probability.

Definition 11.12 (Indicator). For an event A , its *indicator random variable* $1_A : \Omega \rightarrow \{0, 1\}$ takes the value $1_A(\omega) = 1$ if $\omega \in A$ and 0 otherwise.

Proposition 11.13 (Indicator expectation). For any event A , $\mathbb{E}[1_A] = \Pr[A]$.

Proof. By Definition 11.11, $\mathbb{E}[1_A] = \sum_{\omega} 1_A(\omega) \Pr[\omega] = \sum_{\omega \in A} \Pr[\omega] = \Pr[A]$, since the indicator vanishes off A . $\square$

The principal property of expectation is its linearity, which holds with no independence assumption whatsoever. This renders expectation considerably more tractable than probability.

Theorem 11.14 (Linearity of expectation). Let $X, Y : \Omega \rightarrow \mathbb{R}$ be random variables on a finite space and $a, b \in \mathbb{R}$. Then

$$\mathbb{E}[aX + bY] = a \mathbb{E}[X] + b \mathbb{E}[Y].$$

More generally, for random variables $X_1, \dots, X_n$ and scalars $a_1, \dots, a_n$,

$$\mathbb{E}\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i \mathbb{E}[X_i].$$

This holds whether or not the X_i are independent.

Proof. Directly from the definition,

$$\mathbb{E}[aX + bY] = \sum_{\omega \in \Omega} (aX(\omega) + bY(\omega)) \Pr[\omega] = a \sum_{\omega} X(\omega) \Pr[\omega] + b \sum_{\omega} Y(\omega) \Pr[\omega] = a \mathbb{E}[X] + b \mathbb{E}[Y],$$

splitting the finite sum and extracting the constants. The n -term statement follows by induction on n . $\square$

A one-line consequence converts an expectation bound into a probability bound; it underlies every averaging (“heavy-row”) argument in the series.

Proposition 11.15 (Markov’s inequality). Let $X \geq 0$ be a real-valued random variable and $a > 0$. Then $\Pr[X \geq a] \leq \mathbb{E}[X]/a$.

Proof. Every outcome contributes nonnegatively to $\mathbb{E}[X]$, and each outcome with $X(\omega) \geq a$ contributes at least $a \Pr[\omega]$; hence $\mathbb{E}[X] \geq a \Pr[X \geq a]$. $\square$

11.4 The union bound and a birthday calculation

The most frequently used inequality in cryptographic proofs is the union bound: the probability that at least one of several events occurs is at most the sum of their probabilities. It is crude but requires no independence and almost always suffices.

Theorem 11.16 (Union bound / Boole’s inequality). *For any events $A_1, \dots, A_n$ in a finite probability space,*

$$\Pr\left[\bigcup_{i=1}^n A_i\right] \leq \sum_{i=1}^n \Pr[A_i].$$

Proof. Define the disjoint “first occurrence” events $B_1 = A_1$ and $B_i = A_i \setminus (A_1 \cup \dots \cup A_{i-1})$ for $i \geq 2$. The B_i are pairwise disjoint, $B_i \subseteq A_i$, and $\bigcup_i B_i = \bigcup_i A_i$. By finite additivity and monotonicity (Proposition 11.3),

$$\Pr\left[\bigcup_i A_i\right] = \Pr\left[\bigcup_i B_i\right] = \sum_i \Pr[B_i] \leq \sum_i \Pr[A_i]. \quad \square$$

The complementary inequality for *conjunctions* is equally useful and follows by taking complements.

Corollary 11.17 (Union bound for the good event). *If each event A_i holds except with probability at most ε_i (that is, $\Pr[\Omega \setminus A_i] \leq \varepsilon_i$), then all of them hold simultaneously except with probability at most $\sum_i \varepsilon_i$:*

$$\Pr\left[\bigcap_{i=1}^n A_i\right] \geq 1 - \sum_{i=1}^n \varepsilon_i.$$

Proof. By De Morgan, $\Omega \setminus \bigcap_i A_i = \bigcup_i (\Omega \setminus A_i)$. Apply the union bound to the complements and take the complement of both sides. $\square$

We now derive the *birthday bound*. It governs how soon collisions appear when sampling uniformly and underlies the generic attacks against n -bit hash functions and the security loss in many reductions.

Proposition 11.18 (Birthday bound). *Sample $x_1, \dots, x_k$ independently and uniformly from a set of size N . Let C be the event that some two of them collide, i.e. $x_i = x_j$ for some $i < j$. Then*

$$\Pr[C] \leq \binom{k}{2} \frac{1}{N} = \frac{k(k-1)}{2N},$$

and, on the other side,

$$\Pr[C] = 1 - \prod_{i=0}^{k-1} \left(1 - \frac{i}{N}\right) \geq 1 - e^{-k(k-1)/(2N)}.$$

In particular a collision becomes likely once $k = \Theta(\sqrt{N})$.

Proof. Upper bound. For $i < j$ let A_{ij} be the event $x_i = x_j$. Since x_i, x_j are independent and uniform, $\Pr[A_{ij}] = \sum_v \Pr[x_i = v] \Pr[x_j = v] = N \cdot (1/N)^2 = 1/N$. Now $C = \bigcup_{i < j} A_{ij}$, a union of $\binom{k}{2}$ events, so the union bound (Theorem 11.16) gives $\Pr[C] \leq \binom{k}{2}/N$.

Lower bound. The complement $\bar{C}$ is the event that all k samples are distinct. Counting ordered tuples of distinct values, $\Pr[\bar{C}] = \prod_{i=0}^{k-1} (1 - i/N)$: having fixed i distinct values, the next sample avoids them with probability $1 - i/N$; this establishes the stated equality. For $k > N$ the product contains the factor 0 and the exponential bound is trivial; for $k \leq N$ every factor is nonnegative, so the inequality

$1 - x \leq e^{-x}$ applies factor by factor. (For $0 \leq x < 1$ it follows from Bernoulli: $(1 - x/n)^n \geq 1 - x$ for $n > x$, and letting $n \rightarrow \infty$ in the limit definition of e^{-x} ; for $x \geq 1$ the left side is nonpositive.) Thus

$$\Pr[\bar{C}] = \prod_{i=0}^{k-1} \left(1 - \frac{i}{N}\right) \leq \prod_{i=0}^{k-1} e^{-i/N} = \exp\left(-\frac{1}{N} \sum_{i=0}^{k-1} i\right) = e^{-k(k-1)/(2N)}.$$

Taking complements gives $\Pr[C] = 1 - \Pr[\bar{C}] \geq 1 - e^{-k(k-1)/(2N)}$. $\square$

Remark 11.19. The threshold $k \approx \sqrt{N}$ is the reason an n -bit hash function (so $N = 2^n$) offers only about $2^{n/2}$ collision resistance: an adversary expects a collision after roughly $2^{n/2}$ random evaluations. For this reason one chooses output lengths twice the desired security level. The same square-root phenomenon recurs in random-walk discrete-log attacks (Pollard's rho), so designers size the curves used in Halo 2 and Orchard so that $\sqrt{N}$ is itself astronomically large.

11.5 Statistical distance

Comparing distributions—for example a real key-generation procedure against an idealised uniform one—requires a notion of how far apart two distributions are. The appropriate notion for “no statistical test can distinguish them well” is statistical (total variation) distance.

Definition 11.20 (Statistical distance). Let P and Q be probability distributions on the same finite set S (i.e. probability mass functions $P, Q : S \rightarrow [0, 1]$). Their *statistical distance* (or *total variation distance*) is

$$\Delta(P, Q) = \frac{1}{2} \sum_{s \in S} |P(s) - Q(s)|.$$

For random variables X, Y on S , $\Delta(X, Y)$ denotes the statistical distance of their distributions.

The factor $\frac{1}{2}$ makes Δ range over $[0, 1]$ and gives it a clean interpretation as the maximum advantage of a distinguisher, which we establish next.

Theorem 11.21 (Properties of statistical distance). Let P, Q, R be distributions on a finite set S . Then:

1. $0 \leq \Delta(P, Q) \leq 1$; and $\Delta(P, Q) = 0$ iff $P = Q$.
2. (Metric.) Δ is symmetric and satisfies the triangle inequality $\Delta(P, R) \leq \Delta(P, Q) + \Delta(Q, R)$.
3. (Variational characterisation.) $\Delta(P, Q) = \max_{A \subseteq S} (P(A) - Q(A)) = \max_{A \subseteq S} |P(A) - Q(A)|$, where $P(A) = \sum_{s \in A} P(s)$.
4. (Data-processing / post-processing.) For any (possibly randomised) function f applied to a sample, $\Delta(f(P), f(Q)) \leq \Delta(P, Q)$. In particular no algorithm can increase statistical distance.

Proof. (1) Nonnegativity is clear. For the upper bound, let $S^+ = \{s : P(s) \geq Q(s)\}$ and $S^- = S \setminus S^+$. Since $\sum_s P(s) = \sum_s Q(s) = 1$, we have $\sum_s (P(s) - Q(s)) = 0$, so $\sum_{s \in S^+} (P(s) - Q(s)) = \sum_{s \in S^-} (Q(s) - P(s)) =: D \geq 0$. Hence $\Delta(P, Q) = \frac{1}{2} (\sum_{S^+} (P - Q) + \sum_{S^-} (Q - P)) = D$. As $D = \sum_{S^+} (P(s) - Q(s)) \leq \sum_{S^+} P(s) \leq 1$, we get $\Delta \leq 1$. If $\Delta = 0$ every $|P(s) - Q(s)| = 0$, so $P = Q$; the converse is immediate.

(2) Symmetry is evident. For the triangle inequality, $|P(s) - R(s)| \leq |P(s) - Q(s)| + |Q(s) - R(s)|$ termwise by the triangle inequality on $\mathbb{R}$; summing and halving gives the claim.

(3) With D as above, take $A = S^+$: then $P(A) - Q(A) = \sum_{S^+} (P - Q) = D = \Delta(P, Q)$, so the maximum is at least Δ . Conversely, for any A , $P(A) - Q(A) = \sum_{s \in A} (P(s) - Q(s)) \leq \sum_{s \in A, P(s) \geq Q(s)} (P(s) -$

$Q(s)) \leq D = \Delta$, since dropping the negative terms only increases the sum and extending from $A \cap S^+$ to all of S^+ only increases it further. Thus the maximum over A of $P(A) - Q(A)$ is exactly Δ ; replacing A by its complement swaps the sign, giving the same value for $\max |P(A) - Q(A)|$.

(4) First take f deterministic, $f : S \rightarrow U$. For $u \in U$, $f(P)(u) = P(f^{-1}(u))$. Then

$$\Delta(f(P), f(Q)) = \frac{1}{2} \sum_u |P(f^{-1}(u)) - Q(f^{-1}(u))| \leq \frac{1}{2} \sum_u \sum_{s \in f^{-1}(u)} |P(s) - Q(s)| = \Delta(P, Q),$$

using $|\sum_s a_s| \leq \sum_s |a_s|$ on each fibre, and that the fibres $f^{-1}(u)$ partition S . A randomised f is a deterministic function of s together with independent fresh coins r ; applying the deterministic case to the joint variable (s, r) (whose two distributions $P \times U_r$ and $Q \times U_r$ have the same statistical distance $\Delta(P, Q)$, the shared coin distribution contributing nothing) yields the bound. $\square$

Remark 11.22 (Distinguishing interpretation). Part (3) states that $\Delta(P, Q)$ is exactly the best advantage of any (even computationally unbounded) test that, given a single sample, must guess whether it came from P or Q : the optimal test outputs “ P ” on the set $A = S^+$, and its advantage $\Pr_P[A] - \Pr_Q[A]$ equals $\Delta(P, Q)$. Part (4) is what makes statistical distance compose in proofs: applying the same procedure to two close inputs keeps the outputs close. If $\Delta(P, Q) \leq \varepsilon$, then P and Q are interchangeable in any analysis at the cost of an additive ε in every probability. When ε is negligible (defined below), we call P and Q *statistically indistinguishable*.

11.6 Uniform sampling and the bias of modular reduction

Cryptography requires uniform elements of a finite set throughout: a uniform scalar in $\mathbb{Z}/q\mathbb{Z}$, a uniform field element of $\mathbb{F}_p$, a uniform point in a group. The most tractable source of randomness, however, is a stream of independent uniform bits. We must therefore convert uniform bitstrings into uniform elements of S , and we must understand the bias this conversion introduces.

If $|S| = 2^\ell$ is a power of two, the conversion is exact: read ℓ uniform bits as a number in $\{0, \dots, 2^\ell - 1\}$ and index into S . The nontrivial case arises when $|S|$ is not a power of two—most importantly when $S = \mathbb{Z}/q\mathbb{Z}$ for a prime q , which is the scalar field of an elliptic curve.

A common and elementary method is *reduce-modulo*: sample a long uniform integer and reduce it mod q . The result is not perfectly uniform—some residues receive one more preimage than others—but the bias is negligible provided the integer is long enough. We make this precise below.

Proposition 11.23 (Bias of modular reduction). *Let $q \geq 2$ be an integer and let $K = 2^L$ for some nonnegative integer L . Sample U uniformly from $\{0, 1, \dots, K - 1\}$ (equivalently, read L uniform bits as an integer), and set $R = U \bmod q \in \{0, 1, \dots, q - 1\}$. Let $\mathcal{U}_q$ denote the uniform distribution on $\{0, \dots, q - 1\}$. Then the statistical distance of R from uniform satisfies*

$$\Delta(R, \mathcal{U}_q) \leq \frac{q}{K} = \frac{q}{2^L}.$$

Consequently, if $q < 2^n$ and one takes $L = n + s$ bits, then $\Delta(R, \mathcal{U}_q) < 2^{-s}$.

Proof. Write $K = mq + t$ with $m = \lfloor K/q \rfloor$ and remainder $t = K \bmod q$, so $0 \leq t < q$. For a residue $r \in \{0, \dots, q - 1\}$, the number of integers in $\{0, \dots, K - 1\}$ congruent to r modulo q is $m + 1$ if $r < t$ and m if $r \geq t$. Hence

$$\Pr[R = r] = \begin{cases} (m + 1)/K, & r < t, \\ m/K, & r \geq t. \end{cases}$$

Comparing with the target probability $1/q$ and using $m/K \leq 1/q \leq (m + 1)/K$ (since $mq \leq K \leq (m + 1)q$),

$$\left| \Pr[R = r] - \frac{1}{q} \right| \leq \frac{m + 1}{K} - \frac{m}{K} = \frac{1}{K} \quad \text{for every } r.$$

Therefore

$$\Delta(R, \mathcal{U}_q) = \frac{1}{2} \sum_{r=0}^{q-1} \left| \Pr[R = r] - \frac{1}{q} \right| \leq \frac{1}{2} \cdot q \cdot \frac{1}{K} = \frac{q}{2K} \leq \frac{q}{K}.$$

(The computation yields the sharper bound $q/(2K)$; q/K is the usual stated form.) For the final claim, $q < 2^n$ and $K = 2^{n+s}$ give $q/K < 2^n/2^{n+s} = 2^{-s}$. $\square$

Remark 11.24. The consequence is as follows: to sample a scalar mod a ~ 256 -bit prime q with statistical distance below 2^{-128} from uniform, draw $256 + 128 = 384$ uniform bits and reduce. The bias 2^{-s} , with the slack s set to the security parameter, is negligible (Definition 11.40), so the reduce-modulo sample is statistically indistinguishable from a perfectly uniform scalar, and by the data-processing inequality (Theorem 11.21(4)) substituting one for the other anywhere costs at most 2^{-128} per use. An alternative, *rejection sampling*—draw $\lceil \log_2 q \rceil$ bits, accept if the result is $< q$, else retry—gives *exactly* uniform output at the cost of a variable but, with overwhelming probability, small number of retries; for the present purposes the negligible bias of reduce-modulo is acceptable and simpler to analyse. The Orchard specification employs exactly this reduce-modulo sampling when deriving scalars from hash outputs: its ToScalar reduces the full 512-bit PRFexpand output modulo the 255-bit group order — a slack of 257 bits, comfortably beyond the 128 of the example.

11.7 Beyond finite spaces: countable additivity, limits, and densities

Everything so far lives on a finite sample space, and for the cryptographic material of the upper volumes that is enough: keys, challenges, and transcripts are finite strings. Two uses need more. A process watched for as long as it takes—a random walk pursued until it first hits a barrier—has no finite horizon, and a waiting time measured in continuous units has no finite sample space at all. This subsection extends the finite theory by the rungs those uses require: countable additivity, continuity along monotone events, a toolkit for waiting times under stopping rules, and laws defined by densities.

Definition 11.25 (Countable probability space). A *countable probability space* is a pair $(\Omega, \Pr)$ in which Ω is a countable set and $\Pr : \Omega \rightarrow [0, 1]$ satisfies $\sum_{\omega \in \Omega} \Pr[\omega] = 1$; the sum is independent of the enumeration order because its terms are nonnegative. Events are arbitrary subsets $A \subseteq \Omega$, with $\Pr[A] = \sum_{\omega \in A} \Pr[\omega]$, and Definitions 11.2–11.11 carry over with “finite” read as “countable” (a random variable’s image may now be countably infinite), with one exception: Definition 11.4 does not — equal weights cannot sum to 1 on a countably infinite set, so the uniform distribution remains a strictly finite notion. Expectation additionally requires its defining sum to converge absolutely.

Proposition 11.26 (Countable additivity). *Let $(\Omega, \Pr)$ be a countable probability space and $A_1, A_2, \dots$ pairwise disjoint events. Then*

$$\Pr\left[\bigcup_{n \geq 1} A_n\right] = \sum_{n \geq 1} \Pr[A_n],$$

and in particular the basic axioms of Proposition 11.3 hold unchanged.

Proof. Every outcome of the union lies in exactly one A_n , so both sides sum the same nonnegative terms $\Pr[\omega]$, grouped differently; a series of nonnegative terms may be summed in any order, or in groups, without changing its value. $\square$

Theorem 11.27 (Continuity along monotone events). *Let $\Pr$ be a probability assignment, on any sample space, defined for a family of events closed under complement and countable union, and countably additive on it. If $A_1 \subseteq A_2 \subseteq \dots$ is an increasing chain of events, then*

$$\Pr\left[\bigcup_{n \geq 1} A_n\right] = \lim_{n \rightarrow \infty} \Pr[A_n],$$

and dually $\Pr[\bigcap_n B_n] = \lim_n \Pr[B_n]$ for a decreasing chain $B_1 \supseteq B_2 \supseteq \dots$. On a countable space the hypothesis holds automatically, by Proposition 11.26.

Proof. Set $D_1 = A_1$ and $D_n = A_n \setminus A_{n-1}$ for $n \geq 2$. The D_n are pairwise disjoint, $\bigcup_{k \leq n} D_k = A_n$, and $\bigcup_n D_n = \bigcup_n A_n$. Countable additivity gives

$$\Pr\left[\bigcup_n A_n\right] = \sum_{n \geq 1} \Pr[D_n] = \lim_{n \rightarrow \infty} \sum_{k=1}^n \Pr[D_k] = \lim_{n \rightarrow \infty} \Pr[A_n],$$

the final step by finite additivity. The decreasing case follows by complementation. $\square$

Definition 11.28 (Infinite sequence of independent trials). Fix a finite outcome set T and a distribution p on T . An *infinite sequence of independent trials* with law p is a sequence of random variables $X_1, X_2, \dots$ such that every prefix $(X_1, \dots, X_n)$ consists of n mutually independent p -distributed trials in the sense of Definition 11.10—equivalently, the prefix’s law is that of the finite probability space T^n carrying the product weights $\Pr[(t_1, \dots, t_n)] = p(t_1) \cdots p(t_n)$. An event is *finitely determined* if membership in it is decided by some fixed finite prefix; its probability is the finite-space value. An event of the form “some prefix satisfies P ” is the union of an increasing chain of finitely determined events, and Theorem 11.27 assigns it the limit of the prefix probabilities.

Remark 11.29 (What is taken as given). Definition 11.28 presumes that a single, countably additive probability assignment on infinite sequences exists that restricts correctly to every finite prefix at once. That existence statement is the Kolmogorov extension theorem, and its proof belongs to measure theory, which lies off this series’ critical path: the space of infinite sequences is uncountable, so it escapes Definition 11.25, and not every subset of it can consistently be an event. We therefore take the extension—with its countable additivity, hence Theorem 11.27—as a foundation stone rather than a theorem. On infinite-sequence spaces, only finitely determined events and their monotone limits appear in this series, and for those the values are forced by the finite theory together with continuity; the density-defined laws of Definition 11.33 carry the analogous debt, their interval and joint-event probabilities being taken to cohere as the stated integrals.

Waiting times are the extension’s first dividend. Three short results — used by the upper volumes’ extractor and race analyses — follow from countable expectation and finitely determined events alone.

Lemma 11.30 (Tail-sum formula). *Let T be a random variable with values in $\mathbb{N} \cup \{\infty\}$ and $\Pr[T = \infty] = 0$. Then $\mathbb{E}[T] = \sum_{k \geq 1} \Pr[T \geq k]$, either side being infinite exactly when the other is.*

Proof. Expanding $\Pr[T \geq k] = \sum_{m \geq k} \Pr[T = m]$ and regrouping the doubly indexed nonnegative series, each term $\Pr[T = m]$ is counted once for every $k \leq m$, that is, m times; nonnegative series may be regrouped freely. $\square$

Proposition 11.31 (Geometric waiting time). *In an infinite sequence of independent trials, each succeeding with probability $\varepsilon > 0$, let T be the index of the first success. Then $\Pr[T \geq k] = (1 - \varepsilon)^{k-1}$ and $\mathbb{E}[T] = 1/\varepsilon$.*

Proof. The event $T \geq k$ says the first $k - 1$ trials all fail — a finitely determined event of probability $(1 - \varepsilon)^{k-1}$; in particular $\Pr[T = \infty] = \lim_k (1 - \varepsilon)^{k-1} = 0$ by continuity (Theorem 11.27). The tail-sum formula then gives $\mathbb{E}[T] = \sum_{k \geq 1} (1 - \varepsilon)^{k-1} = 1/\varepsilon$, a geometric series. $\square$

Proposition 11.32 (Wald’s identity). *Let $X_1, X_2, \dots$ be an infinite sequence of independent trials with common law, each $X_i \geq 0$ with finite mean, and let T be a stopping rule: a random index such that, for every k , whether $T = k$ is decided by the first k trials alone. If $\mathbb{E}[T] < \infty$, then*

$$\mathbb{E}\left[\sum_{i=1}^T X_i\right] = \mathbb{E}[T] \cdot \mathbb{E}[X_1].$$

Proof. Writing the sum as $\sum_{i \geq 1} X_i 1_{T \geq i}$, the event $T \geq i$ — the negation of $T \leq i - 1$ — is decided by the first $i - 1$ trials, hence independent of X_i , so $\mathbb{E}[X_i 1_{T \geq i}] = \mathbb{E}[X_1] \Pr[T \geq i]$. Summing over i and applying the tail-sum formula gives $\mathbb{E}[X_1] \mathbb{E}[T]$. The expectation on the left is that of the increasing limit of the finitely determined partial sums $\sum_{i \leq \min(T, n)} X_i$, and equals the limit of their expectations by the same free regrouping of nonnegative terms. $\square$

Definition 11.33 (Law defined by a density). A real-valued random quantity X has *density* $f : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ if $\int_{-\infty}^{\infty} f(t) dt = 1$ and

$$\Pr[a \leq X \leq b] = \int_a^b f(t) dt \quad \text{for all } a \leq b;$$

improper Riemann integrals suffice throughout this series. The *distribution function* is $F(t) = \Pr[X \leq t] = \int_{-\infty}^t f$, so $\Pr[X > t] = 1 - F(t)$, and single points carry probability zero. The expectation is $\mathbb{E}[X] = \int_{-\infty}^{\infty} t f(t) dt$ when this integral converges absolutely. Two such quantities X, Y are *independent* if $\Pr[X \in I, Y \in J] = \Pr[X \in I] \Pr[Y \in J]$ for all intervals I, J ; probabilities of joint events are then taken to factor through iterated integrals of the product $f_X(s) f_Y(t)$.

Later volumes build block-discovery and extraction analyses directly on this base: exponential clocks are density-defined laws, block-attribution sequences are infinite sequences of independent trials, extractor running times are the waiting times above, and the limit arguments instantiate Theorem 11.27.

11.8 Asymptotic notation

The discussion now turns from probability to growth rates. To address efficiency and hardness, we must compare the manner in which functions of an integer parameter grow as that parameter tends to infinity. The Bachmann–Landau notation makes such comparisons precise. Throughout, functions map $\mathbb{N}$ (or a cofinite subset thereof) to the nonnegative reals $\mathbb{R}_{\geq 0}$, and “for sufficiently large n ” means “for all n beyond some threshold n_0 ”.

Definition 11.34 (Big-O, Omega, Theta, little-o, little-omega). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ with g eventually positive. We define the following:

- $f = O(g)$ (read “ f is big- O of g ”, an asymptotic upper bound) if there exist constants $c > 0$ and n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$.
- $f = \Omega(g)$ (asymptotic lower bound) if there exist $c > 0$ and n_0 with $f(n) \geq c g(n)$ for all $n \geq n_0$. Equivalently, $f = \Omega(g)$ iff $g = O(f)$.
- $f = \Theta(g)$ (tight bound) if $f = O(g)$ and $f = \Omega(g)$; i.e. there are $c_1, c_2 > 0$ and n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$.
- $f = o(g)$ (read “little- o ”, strictly smaller order) if for every $c > 0$ there is n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$; equivalently $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- $f = \omega(g)$ (strictly larger order) if $g = o(f)$; equivalently $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Remark 11.35. The “=” in “ $f = O(g)$ ” is traditional but denotes set membership: $O(g)$ is the class of all functions bounded above by a constant multiple of g , and $f = O(g)$ means $f \in O(g)$. One never reads such equations right to left. We follow the customary abuse of notation here.

Example 11.36. The polynomial $3n^2 + 10n + 7$ is $\Theta(n^2)$; indeed for $n \geq 1$ one has $3n^2 \leq 3n^2 + 10n + 7 \leq 20n^2$. Also $n^2 = o(n^3)$ since $n^2/n^3 = 1/n \rightarrow 0$; $\log_2 n = o(n)$; $n^{100} = o(2^n)$ (any polynomial is little- o of any exponential with base > 1); and $n! = \omega(2^n)$. Note that $2^n = o(2^{n+1})$ is false, but $2^n = \Theta(2^{n+1})$ is true, since $2^{n+1} = 2 \cdot 2^n$ is within a constant factor.

Proposition 11.37 (Useful closure facts). *Let $f_1 = O(g_1)$ and $f_2 = O(g_2)$. Then $f_1 + f_2 = O(\max(g_1, g_2)) = O(g_1 + g_2)$ and $f_1 f_2 = O(g_1 g_2)$. The same closure statements hold with O replaced uniformly by Ω or Θ . Moreover $O(\cdot)$ is transitive: $f = O(g)$ and $g = O(h)$ imply $f = O(h)$.*

Proof. Choose c_1, n_1 and c_2, n_2 witnessing the two hypotheses, and let $n_0 = \max(n_1, n_2)$. For $n \geq n_0$: $f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq (c_1 + c_2) \max(g_1(n), g_2(n))$, and $\max(g_1, g_2) \leq g_1 + g_2 \leq 2 \max(g_1, g_2)$, so the two right-hand sides agree up to a constant; this proves the sum rule. For products, $f_1(n) f_2(n) \leq c_1 c_2 g_1(n) g_2(n)$. Transitivity: $f \leq c g$ and $g \leq c' h$ eventually give $f \leq cc' h$ eventually. The Ω statements follow by the equivalence $f = \Omega(g) \Leftrightarrow g = O(f)$, and Θ by combining the two. $\square$

11.9 Polynomial, exponential, and negligible functions

We now classify the growth rates relevant to cryptography. Fix a distinguished variable $\lambda \in \mathbb{N}$, the *security parameter* (discussed in detail in Subsection 11.11); informally larger λ means more security and larger keys. Resources (running time, key length) and success probabilities are functions of λ .

Definition 11.38 (Polynomial and super-polynomial). A function $f: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *polynomially bounded*, written $f = \text{poly}(\lambda)$, if there is a constant c (and a threshold) with $f(\lambda) \leq \lambda^c + c$ for all large λ ; equivalently $f(\lambda) = O(\lambda^c)$ for some constant c . It is *super-polynomial* if it grows faster than every polynomial, i.e. $f(\lambda) = \omega(\lambda^c)$ for every constant c (equivalently $\lambda^c = o(f(\lambda))$ for all c).

Definition 11.39 (Exponential). A function f is *exponential* if $f(\lambda) = 2^{\Omega(\lambda)}$; the prototypical case is $f(\lambda) = 2^{c\lambda}$ for a constant $c > 0$. A super-polynomial function with $f(\lambda) = 2^{o(\lambda)}$ is *sub-exponential*; examples are $\lambda^{\log_2 \lambda} = 2^{(\log_2 \lambda)^2}$ and the index-calculus running time $L_p[1/3] = 2^{\Theta(\lambda^{1/3}(\log \lambda)^{2/3})}$ in the bit length $\lambda = \log_2 p$. Every exponential function is super-polynomial; the converse fails by the examples above.

The central cryptographic notion is that of a *negligible* function: one that shrinks faster than the reciprocal of every polynomial. A negligible probability is one we may ignore, because no efficient (polynomially many) repetition of an experiment can amplify it to a noticeable chance.

Definition 11.40 (Negligible function). A function $\nu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $\nu = \text{negl}(\lambda)$, if for every positive constant c there exists λ_0 such that

$$\nu(\lambda) < \frac{1}{\lambda^c} \quad \text{for all } \lambda \geq \lambda_0.$$

Equivalently, $\nu(\lambda) = o(\lambda^{-c})$ for every constant $c > 0$; equivalently, $\lambda^c \nu(\lambda) \rightarrow 0$ as $\lambda \rightarrow \infty$ for every constant c . A function is *non-negligible* if it is not negligible, i.e. $f(\lambda) \geq \lambda^{-c}$ for infinitely many λ , for some constant c ; it is *noticeable* if there exist c and λ_0 with $f(\lambda) \geq \lambda^{-c}$ for all $\lambda \geq \lambda_0$ (noticeable implies non-negligible, but not conversely); it is *overwhelming* if $1 - f$ is negligible.

Example 11.41. The functions $2^{-\lambda}$, $2^{-\sqrt{\lambda}}$, and $\lambda^{-\log \lambda}$ are negligible. Indeed for $2^{-\lambda}$: given any c , since $2^{-\lambda} \lambda^c = \lambda^c / 2^\lambda \rightarrow 0$ (a polynomial over an exponential), the function meets the definition. By contrast, $1/\lambda^{100}$ is *not* negligible—taking $c = 101$ shows $1/\lambda^{100} > 1/\lambda^{101}$ for all $\lambda \geq 2$ —it is noticeable, even though it tends to 0. This is the key subtlety: tending to zero does not suffice; a negligible function must beat every inverse polynomial.

The reason “negligible” is the correct closure of “ignorable” is the following stability under the operations that appear in security proofs.

Proposition 11.42 (Closure properties of negligible functions). *Let ν, μ be negligible and let p be a polynomially bounded function. Then:*

1. $\nu + \mu$ is negligible (the class is closed under addition, hence under any fixed finite sum).
2. $p \cdot \nu$ is negligible (a polynomial times a negligible function is negligible).
3. If $f(\lambda) \leq \nu(\lambda)$ for all large λ , then f is negligible (domination by a negligible function).
4. Summing $p(\lambda)$ functions, each bounded by a single negligible function ν , yields a negligible function; in particular a union bound over polynomially many negligible events is negligible.

Proof. (1) Fix c . Applying the definition of negligibility to the exponent $c + 1$, there is a threshold beyond which $\nu(\lambda), \mu(\lambda) < \lambda^{-(c+1)} \leq \frac{1}{2} \lambda^{-c}$ (the latter once $\lambda \geq 2$); adding the two bounds gives $\nu(\lambda) + \mu(\lambda) < \lambda^{-c}$ for all large λ .

(2) Suppose $p(\lambda) \leq \lambda^d$ for large λ (and some constant d). Fix c . By negligibility applied to the exponent $c + d$, $\nu(\lambda) < \lambda^{-(c+d)}$ for large λ . Then $p(\lambda)\nu(\lambda) < \lambda^d \lambda^{-(c+d)} = \lambda^{-c}$ for large λ . Hence $p\nu$ is negligible.

(3) Immediate: if $f \leq \nu$ eventually and $\nu < \lambda^{-c}$ eventually, then $f < \lambda^{-c}$ eventually, for every c .

(4) Write the sum $\Sigma(\lambda) = \sum_{i=1}^{p(\lambda)} \nu_i(\lambda)$ where each $\nu_i \leq \nu$ by hypothesis (in security proofs all the ν_i arise from one bound). Then $\Sigma(\lambda) \leq p(\lambda) \nu(\lambda)$, which is negligible by (2); apply (3). $\square$

Remark 11.43 (Why negligible is the threshold of security). Suppose an adversary wins a game with negligible probability $\nu(\lambda)$. If it repeats the attack $p(\lambda)$ times (polynomially many, the most an efficient adversary can afford), the probability that some repetition succeeds is, by the union bound, at most $p(\lambda)\nu(\lambda)$ —still negligible by Proposition 11.42(2). Thus negligible success probability remains negligible under any efficient amplification: it is genuinely ignorable. Conversely a noticeable success probability $\geq \lambda^{-c}$ can be boosted to a constant by λ^c repetitions. The negligible/non-negligible split is the formal backbone of the phrase “the scheme is secure except with negligible probability”:

security asserts a negligible advantage, and an attack means a non-negligible one. (A merely non-negligible success probability is boosted to a constant only on the infinitely many λ where the inverse-polynomial bound holds, which already suffices to contradict a security definition.)

11.10 Algorithms, running time, and PPT

It remains to make “efficient algorithm” and “infeasible” precise. We adopt the standard model: algorithms are Turing machines (or, equivalently up to polynomial factors, programs in any reasonable model), inputs and outputs are finite binary strings, and the number of elementary steps as a function of input length is the resource we measure. Write $\{0, 1\}^*$ for the set of all finite binary strings and $|x|$ for the length of $x \in \{0, 1\}^*$.

Definition 11.44 (Deterministic polynomial-time algorithm). An algorithm M runs in (*deterministic*) *polynomial time* if there is a polynomial p such that, on every input $x \in \{0, 1\}^*$, M halts within at most $p(|x|)$ steps and produces an output. The class of decision problems solvable by such machines is P .

Determinism is insufficient for cryptography: key generation, sampling, and many algorithms require randomness. We model randomness by granting the machine access to an auxiliary tape of independent uniform random bits (the “coins”).

Definition 11.45 (Probabilistic algorithm and PPT). A *probabilistic* (or *randomised*) algorithm M is a Turing machine with an additional read-only *random tape* initialised with independent uniformly random bits $r \in \{0, 1\}^*$. For a fixed input x , the output $M(x)$ is a random variable over the choice of r ; write $M(x; r)$ when one wishes to display the coins explicitly, and then $M(x; \cdot)$ is a deterministic function. The algorithm runs in *probabilistic polynomial time* (it is *PPT*) if there is a polynomial p such that, for every input x and every choice of coins r , M halts within $p(|x|)$ steps. (Bounding the time for every r , not merely in expectation, is the standard “strict” notion and ensures M ever reads only $p(|x|)$ coins.)

Remark 11.46 (Non-uniformity and circuits). There are two standard ways to formalise an efficient adversary. The *uniform* model is a single PPT machine that works for all input lengths, as above. The *non-uniform* model allows a separate algorithm (equivalently, a Boolean circuit) of size $\text{poly}(\lambda)$ for each λ , which may be hard-wired with an arbitrary polynomial-length “advice” string depending on λ . Non-uniform adversaries are at least as powerful as uniform ones and are often more convenient in reduction proofs because the advice can encode a “best” attack. The asymptotic theory below is identical for both; security definitions in the Halo 2/Orchard setting typically state the security against non-uniform PPT adversaries, which is the more conservative choice.

With efficiency pinned down, we can state what it means for a computational task to be hard. The guiding principle is the Cobham–Edmonds thesis: “efficiently computable” = “computable in polynomial time”. Its negation gives infeasibility.

Definition 11.47 (Computational infeasibility). A computational task parameterised by the security parameter λ is (*computationally*) *feasible* if there is a PPT algorithm solving it (with overwhelming success probability). It is *infeasible* if for every PPT algorithm $\mathcal{A}$ the probability that $\mathcal{A}$ solves it is negligible in λ :

$$\Pr[\mathcal{A} \text{ solves the task on parameter } \lambda] = \text{negl}(\lambda),$$

where the probability is over the task’s randomness and $\mathcal{A}$ ’s coins. Equivalently, no efficient algorithm succeeds with non-negligible probability.

Remark 11.48 (Reading a hardness assumption). Cryptographic assumptions are statements of this form. For instance, the discrete-logarithm assumption in a family of groups $\{\mathbb{G}_\lambda\}$ of order $q(\lambda)$ asserts: for every PPT $\mathcal{A}$,

$$\Pr[\mathcal{A}(g, g^x) = x] = \text{negl}(\lambda),$$

the probability taken over a uniformly chosen generator g of $\mathbb{G}_\lambda$, a uniform $x \leftarrow \mathbb{Z}/q\mathbb{Z}$, and $\mathcal{A}$ ’s coins. The entire edifice of Halo 2 and Orchard rests on such infeasibility statements, which one makes true in practice by choosing λ (and hence the group size $\approx 2^{2\lambda}$) large enough that the best known attack—running in time $\Theta(\sqrt{q}) = \Theta(2^\lambda)$ by the birthday/rho phenomenon of Remark 11.19—is astronomically slow. The asymptotic language of this section is what permits the terms “negligible” and “polynomial” without committing to a particular λ ; the concrete-security viewpoint then instantiates λ (e.g. $\lambda = 128$) to read off actual bit-counts.

11.11 The security parameter

This section closes by making explicit the parameter that threads through every preceding definition.

Definition 11.49 (Security parameter). The *security parameter* is a positive integer $\lambda \in \mathbb{N}$ that one supplies to all algorithms of a cryptographic scheme, conventionally in unary as the string 1^λ (so that “polynomial in the input length” coincides with “polynomial in λ ”). It governs simultaneously:

- the *sizes* of objects—key lengths and the encodings of group and field elements ($\approx 2\lambda$ bits, the group order being $\approx 2^{2\lambda}$)—each $\text{poly}(\lambda)$;
- the *running time* of honest algorithms, which must be $\text{poly}(\lambda)$; and
- the *security level*: every PPT adversary’s advantage must be $\text{negl}(\lambda)$.

Remark 11.50 (Why unary, and what λ buys). Passing 1^λ (a string of λ ones, of length λ) rather than the integer λ in binary (length $\log_2 \lambda$) is a deliberate convention: it forces “polynomial-time in the input length” to mean polynomial in λ itself, so an honest key generator may take time $\text{poly}(\lambda)$ to produce λ -bit keys. As λ grows, two things happen in lockstep: honest parties pay a polynomial cost, while every adversary’s success probability falls negligibly and (concretely) the best attack’s running time grows exponentially. Asymptotic security is the statement that this gap holds for all sufficiently large λ ; the practitioner then fixes a single value (commonly $\lambda = 128$, targeting 2^{128} -operation attacks) and verifies that the concrete sizes—a ~ 255 -bit Pallas/Vesta scalar field, hash outputs of ≥ 256 bits per the birthday bound of Proposition 11.18, and sampling slack of at least 128 extra bits per Proposition 11.23 (the deployed ToScalar reduces 512 bits, a slack of 257)—deliver it. Every “negligible” in the remainder of this monograph is implicitly a function of this λ .

Part II

Crypto Guide: Cryptographic primitives from scratch

Abstract

Building on the mathematics of the *Math Guide*, this volume of *The Zcash Arboretum* develops from first principles the cryptographic primitives used in Zcash’s Orchard protocol: the provable-security model and the hardness assumptions; hash functions and the random oracle model; pseudorandom functions, block ciphers, and format-preserving encryption; symmetric and authenticated encryption and key derivation; key agreement; commitment schemes; digital signatures; interactive proofs, zero knowledge, and SNARKs; and Merkle trees with their vector-commitment abstraction.

1 The provable-security model

The preceding volume supplied the algebraic objects: groups in which discrete logarithms appear hard, finite fields, and elliptic curves. This volume turns them into cryptography. An *object* is not yet a *guarantee*. To assert that a scheme “is secure” is to make a precise mathematical claim, and to prove it demands a precise mathematical framework in which the claim lives. That framework—the apparatus of modern provable security—is the subject of this section.

The central methodological idea is the following. One almost never knows how to prove unconditionally that a cryptographic scheme is secure; such a proof would typically imply $P \neq NP$ and a great deal more. Instead we adopt a *relativised* standard of proof. One isolates a small number of computational problems—factoring, discrete logarithm, decisional Diffie–Hellman, and the like—that have resisted decades of attack, and *reduces* the security of the schemes to the conjectured hardness of those problems. A proof of security is then a proof of an implication: *if* the underlying problem is hard, *then* the scheme is secure. Equivalently, in its contrapositive and more operationally useful form: *any* adversary that breaks the scheme can be mechanically transformed into an algorithm that solves the underlying hard problem. Since no such algorithm is believed to exist, one concludes that no such adversary exists.

To render any of this rigorous, we must pin down four things: what counts as a “scheme” parametrised by a notion of input size; what counts as an “adversary”; what it means for an adversary to “break” a scheme, measured by a quantity called its *advantage*; and what it means for one problem to reduce to another. We develop these in turn, followed by the two idealisations under which proofs proceed (the standard model and the random oracle model), and finally the translation between the clean asymptotic language and the concrete “ λ -bit security” numbers that practitioners actually quote.

1.1 The security parameter

Every cryptographic scheme in this monograph is not a single object but an infinite *family* of objects, indexed by a positive integer called the security parameter.

Definition 1.1 (Security parameter). The *security parameter*, denoted $\lambda \in \mathbb{N}$, is a positive integer that controls the sizes of all objects in a cryptographic scheme: key lengths, group orders, output lengths, and so on. The system conventionally provides it to every algorithm in unary, written 1^λ (the string of λ ones), so that an algorithm running in time polynomial in the length of its input automatically gets time polynomial in λ .

Two questions immediately arise: why a family, and why unary.

The reason for a *family* is that security is inherently asymptotic. There is no such thing as an absolutely unbreakable scheme with fixed finite key length: an adversary with enough time can always enumerate a finite key space. What we can hope for is that as λ grows, the resources required to break the scheme grow far faster than the resources required to use it. The honest parties' costs (key generation, encryption, signing) should grow *slowly* in λ —polynomially—while the best attack's cost should grow *quickly*—super-polynomially, ideally exponentially. The security parameter is the dial that separates these two growth rates, and security is a statement about the limit $\lambda \rightarrow \infty$.

The reason for *unary* encoding, 1^λ rather than the $\lceil \log_2 \lambda \rceil$ -bit binary representation of λ , is a book-keeping convenience that aligns two notions of “polynomial time.” Complexity theory measures running time as a function of input *length*. If λ came in binary it would have length $\Theta(\log \lambda)$, and an algorithm “polynomial in its input length” would get only $\text{poly}(\log \lambda)$ steps—not even enough to write down a λ -bit key. Padding the parameter to length λ ensures that “polynomial in the input length” coincides with the cryptographically meaningful “polynomial in λ .” This is purely a normalisation; nothing of substance depends on it.

Remark 1.2. Because everything is indexed by λ , every quantity we discuss is really a *function* of λ : the adversary's success probability, its running time, the key length. A single number—“the advantage is at most 2^{-128} ”—either fixes a concrete λ or describes the function's behaviour for the λ of interest. Keeping the dependence on λ explicit, at least mentally, prevents nearly every confusion in this subject.

1.2 Probabilistic polynomial-time adversaries

Having fixed how scheme sizes scale, we fix what an attacker may do. We model the honest algorithms and the adversary alike as probabilistic polynomial-time machines.

Definition 1.3 (Probabilistic polynomial-time). An algorithm $\mathcal{A}$ is *probabilistic polynomial-time* (PPT) if it is a probabilistic Turing machine—one equipped with a read-only tape of independent uniformly random bits, its *random tape*—and there is a polynomial p such that on every input x , and for every setting of the random tape, $\mathcal{A}$ halts within $p(|x|)$ steps. Write $y \leftarrow \mathcal{A}(x)$ for the random variable obtained by running $\mathcal{A}$ on x with freshly sampled randomness, and $y := \mathcal{A}(x; r)$ for the deterministic output when the random tape is fixed to the string r .

Several modelling choices warrant comment.

Why polynomial time? Polynomial time is the standard mathematical abstraction of “feasible computation.” It is robust—closed under composition, and invariant across all reasonable deterministic computational models (the strong Church–Turing thesis)—which makes it a clean class to quantify over. Crucially, we require the *honest* parties to run in polynomial time too; a scheme is useful only if it can be used efficiently. The security claim is then an asymmetry between two polynomial-time worlds (honest use) and a quantified statement that *no* polynomial-time world (attack) succeeds.

Why probabilistic? Randomness is not a luxury but a necessity in cryptography. Deterministic encryption leaks whether two ciphertexts encrypt the same message; key generation must sample secrets unpredictably. We therefore grant the adversary randomness too, both for fairness and because a deterministic adversary is the special case where the random tape is ignored.

Definition 1.4 (Non-uniform PPT). A *non-uniform* PPT adversary is a family $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$ of probabilistic circuits (or, equivalently, Turing machines each supplied with an *advice string* z_λ of length $\text{poly}(\lambda)$) such that a single polynomial in λ bounds the size of $\mathcal{A}_\lambda$. The advice may depend arbitrarily on λ but not on the particular inputs drawn at run time.

Remark 1.5. The distinction between uniform adversaries (a single machine that reads 1^λ and works for all λ) and non-uniform ones (a separate circuit per λ , or a uniform machine with per- λ advice) is a genuine one. Non-uniformity grants the adversary free precomputed advice tailored to each parameter size—for example, a short hint about the group of the day. Security against non-uniform adversaries is the stronger and now-standard requirement, partly because non-uniform models compose more cleanly under reduction (one may “hard-wire” a fixed good choice of randomness or auxiliary input). All definitions below can be read in either flavour; we state hardness assumptions against non-uniform adversaries when it matters and otherwise leave the choice implicit.

Remark 1.6 (Expected versus strict polynomial time, and quantum adversaries). Two refinements recur. First, some definitions allow *expected* polynomial-time adversaries, bounding the average rather than worst-case running time; this matters for certain extraction arguments in zero-knowledge, and we flag it where it arises. Second, for post-quantum security (taken up in §2.8) one replaces PPT by *quantum* polynomial-time (QPT), a uniform family of polynomial-size quantum circuits. The structural definitions of this section (advantage, indistinguishability, reduction) are agnostic to which class of adversaries one quantifies over; only the underlying hardness assumptions change.

1.3 Distribution ensembles and indistinguishability

The most basic security goal is that two situations *look the same* to every efficient observer—a real protocol transcript versus a simulated one, an encryption of m_0 versus an encryption of m_1 , a pseudorandom string versus a truly random one. To formalise “look the same” we first formalise the two situations as *ensembles* of probability distributions, after which we define three grades of sameness.

Definition 1.7 (Probability ensemble). A *probability ensemble* is a sequence $X = \{X_\lambda\}_{\lambda \in \mathbb{N}}$ of probability distributions (equivalently, random variables), one for each value of the security parameter, where X_λ is supported on binary strings whose length is bounded by a polynomial in λ . There is often an additional index—an input a —written $\{X_\lambda(a)\}$; for clarity we suppress it unless needed.

1.3.1 The statistical distance and its variational characterisation

The natural metric on a single pair of distributions is the statistical (total-variation) distance.

Definition 1.8 (Statistical distance). Let X, Y be discrete random variables over a common finite or countable set S . Their *statistical distance* (or total variation distance) is

$$\Delta(X, Y) := \frac{1}{2} \sum_{s \in S} |\Pr[X = s] - \Pr[Y = s]|.$$

Proposition 1.9 (Variational characterisation). For X, Y over S ,

$$\Delta(X, Y) = \max_{T \subseteq S} (\Pr[X \in T] - \Pr[Y \in T]) = \max_D (\Pr[D(X) = 1] - \Pr[D(Y) = 1]),$$

where the last maximum ranges over all (even computationally unbounded, deterministic) decision functions $D : S \rightarrow \{0, 1\}$. In particular, no procedure whatsoever—efficient or not—can distinguish X from Y with advantage exceeding $\Delta(X, Y)$.

Proof. The set form is the variational characterisation in the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), whose argument is unchanged over a countable S . Only the reformulation over decision functions is new: a deterministic D is the indicator of the set $T = D^{-1}(1)$,

so the maximum over D equals the maximum over T ; a randomised D is a convex combination of deterministic ones and cannot do better. $\square$

Proposition 1.10 (Properties of statistical distance). *The statistical distance is a metric on probability distributions: $0 \leq \Delta(X, Y) \leq 1$, it is symmetric, $\Delta(X, Y) = 0$ iff X and Y are identically distributed, and it satisfies the triangle inequality $\Delta(X, Z) \leq \Delta(X, Y) + \Delta(Y, Z)$. Moreover it never increases under (possibly randomised) post-processing: for any function—indeed any randomised algorithm— f ,*

$$\Delta(f(X), f(Y)) \leq \Delta(X, Y).$$

Proof. These are the metric and data-processing parts of the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), again with the argument unchanged over a countable S . $\square$

1.3.2 The three grades of indistinguishability

We now lift the comparison of single distributions to ensembles, in three strengths, from strongest to weakest. The weakest grade requires the asymptotic notion of a vanishing quantity, which we fix once and for all.

Definition 1.11 (Negligible function). A function $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $f = \text{negl}(\lambda)$, if for every $c \in \mathbb{N}$ there exists λ_0 such that $f(\lambda) < \lambda^{-c}$ for all $\lambda \geq \lambda_0$; equivalently, f decays faster than the reciprocal of every polynomial. A function is *non-negligible* if it is not negligible; it is *noticeable* if there exists c with $f(\lambda) \geq \lambda^{-c}$ for all sufficiently large λ (noticeable functions are non-negligible, but not conversely). A function is *overwhelming* if $1 - f$ is negligible. The class is closed under addition and under multiplication by any polynomial—the closure property on which the hybrid argument below depends.

Definition 1.12 (Perfect, statistical, and computational indistinguishability). Let $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ be probability ensembles.

- X and Y are *perfectly indistinguishable*, written $X \equiv Y$, if X_λ and Y_λ are *identically distributed* for every λ ; equivalently $\Delta(X_\lambda, Y_\lambda) = 0$ for all λ .
- X and Y are *statistically indistinguishable*, written $X \approx_s Y$, if their statistical distance is negligible:

$$\Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda).$$

- X and Y are *computationally indistinguishable*, written $X \approx_c Y$, if for every PPT algorithm D (the *distinguisher*), the function

$$\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) := \left| \Pr[D(1^\lambda, X_\lambda) = 1] - \Pr[D(1^\lambda, Y_\lambda) = 1] \right|$$

is negligible in λ . The probabilities are over the draw from the ensemble and over D ’s internal randomness.

These three notions sit in a strict hierarchy. The implications are immediate from the definitions and the variational characterisation; standard examples witness the strictness.

Proposition 1.13 (Hierarchy). *For all ensembles X, Y :*

$$X \equiv Y \implies X \approx_s Y \implies X \approx_c Y.$$

Both implications are strict: there exist ensembles that are statistically but not perfectly indistinguishable, and ensembles that are computationally but not statistically indistinguishable.

Proof. If $X \equiv Y$ then $\Delta(X_\lambda, Y_\lambda) = 0$, which is negligible, so $X \approx_s Y$. If $X \approx_s Y$, then for any—in particular any PPT—distinguisher D , Proposition 1.9 gives $\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) \leq \Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda)$, so $X \approx_c Y$.

For strictness of the first implication, let X_λ be uniform on $\{0, 1\}^\lambda$ and let Y_λ be uniform on $\{0, 1\}^\lambda \setminus \{0^\lambda\}$. They are not identically distributed (the point 0^λ has probabilities $2^{-\lambda}$ and 0), so not perfectly indistinguishable, yet $\Delta(X_\lambda, Y_\lambda) = 2^{-\lambda} = \text{negl}(\lambda)$, so they are statistically indistinguishable.

For strictness of the second, the existence of a pseudorandom generator (whose existence follows from that of one-way functions — the Håstad–Impagliazzo–Levin–Luby theorem, which we use without proof) furnishes an ensemble: let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ be a PRG, let $X_\lambda := G(U_\lambda)$ for U_λ uniform on $\{0, 1\}^\lambda$, and let $Y_\lambda := U_{2\lambda}$ be uniform on $\{0, 1\}^{2\lambda}$. By definition of a PRG, $X \approx_c Y$. But X_λ is supported on at most 2^λ strings out of $2^{2\lambda}$, so a set T equal to the image of G has $\Pr[X_\lambda \in T] = 1$ while $\Pr[Y_\lambda \in T] \leq 2^\lambda / 2^{2\lambda} = 2^{-\lambda}$; hence $\Delta(X_\lambda, Y_\lambda) \geq 1 - 2^{-\lambda}$, which is not negligible. Thus $X \approx_c Y$ but $X \not\approx_s Y$. $\square$

Remark 1.14 (The price and the prize of computational indistinguishability). The strictness example is instructive. Statistically the PRG output and a random string are almost as far apart as two distributions can be—distance nearly 1—because the PRG image is a vanishing fraction of the codomain. An unbounded distinguisher simply checks membership in the image and wins. The entire content of computational indistinguishability is that *finding* that image membership test is infeasible. This is the bargain at the heart of cryptography: we trade the impossible (information-theoretic security with short keys) for the merely conjectured-hard (computational security), and in exchange we obtain the ability to encrypt long messages under short keys, build public-key primitives, and so on. Statistical and perfect security, when attainable, are unconditional and need no hardness assumption; computational security buys vastly more functionality at the cost of resting on an assumption.

A property that makes $\approx_c$ usable in proofs is that efficient operations preserve it: an adversary cannot manufacture a distinguishing edge out of indistinguishable inputs by post-processing.

Lemma 1.15 (Closure under efficient post-processing). *If $X \approx_c Y$ and M is any PPT algorithm, then the ensembles $\{M(1^\lambda, X_\lambda)\}$ and $\{M(1^\lambda, Y_\lambda)\}$ satisfy $\{M(X_\lambda)\} \approx_c \{M(Y_\lambda)\}$.*

Proof. Suppose some PPT distinguisher D separates $\{M(X_\lambda)\}$ from $\{M(Y_\lambda)\}$ with advantage $\delta(\lambda)$. Define $D' := D \circ M$, i.e. $D'(1^\lambda, z) := D(1^\lambda, M(1^\lambda, z))$. Since M and D are both PPT, so is D' . Then

$$\text{Adv}_{X,Y}^{\text{dist}}(D', \lambda) = |\Pr[D(M(X_\lambda)) = 1] - \Pr[D(M(Y_\lambda)) = 1]| = \delta(\lambda).$$

By $X \approx_c Y$ the left side is negligible, so δ is negligible. $\square$

1.3.3 The hybrid argument

Many proofs require the comparison of two ensembles that are far apart but connected by a chain of pairwise-close intermediate ensembles. The hybrid argument is the workhorse that turns such a chain into a single conclusion. It is essentially the triangle inequality, but with a refinement that explains why the number of hybrids must be polynomial.

Lemma 1.16 (Hybrid lemma). *Let $t = t(\lambda)$ be polynomially bounded, and let $H_\lambda^0, H_\lambda^1, \dots, H_\lambda^t$ be a sequence of distributions (the hybrids). Suppose that for every PPT distinguisher D the neighbouring advantages*

$$\alpha_i(D, \lambda) := |\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]|$$

are uniformly negligible: a single negligible function ν (depending on D) satisfies $\alpha_i(D, \lambda) \leq \nu(\lambda)$ for all $i \in \{0, \dots, t(\lambda) - 1\}$ simultaneously. Equivalently, $\max_{0 \leq i < t(\lambda)} \alpha_i(D, \lambda) = \text{negl}(\lambda)$, i.e. $\alpha_{i(\lambda)}(D, \lambda)$ is negligible along every index function $i(\lambda)$ —arbitrary, not merely efficiently computable, since the maximising index need not be efficient. Then $\{H_\lambda^0\}$ and $\{H_\lambda^t\}$ are computationally indistinguishable, i.e. for every PPT D , $|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \text{negl}(\lambda)$.

Proof. Fix a PPT distinguisher D and let ν be the uniform bound the hypothesis supplies for D . By the triangle inequality applied to the telescoping sum,

$$|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \left| \sum_{i=0}^{t-1} (\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]) \right| \leq \sum_{i=0}^{t-1} \alpha_i(D, \lambda),$$

and the hypothesis bounds the sum by $t(\lambda) \cdot \nu(\lambda)$, which is negligible since t is a polynomial. Hence $\{H^0\} \approx_c \{H^t\}$. $\square$

Uniformity in i is essential, and no argument derives it from the weaker hypothesis that each α_i is negligible for every *fixed* i : there the threshold beyond which the i -th term is small can depend on i , and the dependence cannot be removed. A countermodel makes this concrete: take $t(\lambda) = \lambda + 1$ and let H_λ^i be the point mass at 0 if $i \leq \lambda$ and the point mass at 1 otherwise. For every fixed i the advantage $\alpha_i(D, \lambda)$ is nonzero only at $\lambda = i$, hence negligible; yet H_λ^0 and $H_\lambda^{t(\lambda)}$ are the point masses at 0 and 1, perfectly distinguishable. Applications discharge the uniform hypothesis by reduction. When the hybrids are efficiently samplable given $(1^\lambda, i)$, the *random-index* distinguisher D' samples i uniformly from $\{0, \dots, t - 1\}$, embeds its challenge at position i , and runs D ; writing $p_i := \Pr[D(H_\lambda^i) = 1]$, its distinguishing advantage equals $|\frac{1}{t} \sum_{i=0}^{t-1} (p_i - p_{i+1})| = |p_0 - p_t|/t$ (which is at most $\frac{1}{t} \sum_i \alpha_i(D, \lambda)$), so the per-step assumption applied to the single machine D' bounds D 's total advantage by $t(\lambda) \cdot \text{Adv}(D') = \text{negl}(\lambda)$.

Remark 1.17. The hybrid argument is precisely where the “polynomially many” constraint earns its keep. A negligible per-step advantage multiplied by a polynomial number of steps remains negligible; a negligible per-step advantage multiplied by, say, 2^λ steps need not. This is why constructions that would require exponentially many hybrids do not yield security proofs, and why the closure of negligibility under polynomial factors is not a technicality but the load-bearing wall of the whole edifice.

1.4 Security as a game

We can now state what it means to “break” a scheme. The modern idiom expresses every security definition as a *game* (or experiment) played between a *challenger* and an *adversary* $\mathcal{A}$. The challenger, following a fixed protocol, sets up the scheme, answers whatever queries the adversary may pose, and at the end declares the adversary a winner or loser by outputting a single bit. Security says that no PPT adversary wins by more than it could by trivial guessing.

Definition 1.18 (Security game and advantage). A *security game* (or *experiment*) G is a PPT interactive algorithm, the challenger, that takes 1^λ , interacts with an adversary $\mathcal{A}$, and finally outputs a bit $G^\mathcal{A}(\lambda) \in \{0, 1\}$, where 1 denotes “adversary wins.” The *advantage* of $\mathcal{A}$ is a measure of how much better than trivial its winning probability is; its precise form depends on the type of game:

- *Distinguishing (indistinguishability) games*, where the challenger secretly chooses a uniform

bit $b \in \{0, 1\}$ and $\mathcal{A}$ outputs a guess b' ; the adversary wins iff $b' = b$. Here

$$\text{Adv}_G(\mathcal{A}, \lambda) := \left| \Pr[G^{\mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right| = \frac{1}{2} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|.$$

Trivial guessing achieves winning probability $1/2$, hence advantage 0.

- *Search (unforgeability/inversion) games*, where the adversary must produce a specific kind of object—a forgery, a preimage, a discrete logarithm—and wins iff it succeeds. Here the baseline winning probability is (essentially) 0, and one simply sets

$$\text{Adv}_G(\mathcal{A}, \lambda) := \Pr[G^{\mathcal{A}}(\lambda) = 1].$$

Definition 1.19 (Security relative to a game). A scheme is *secure* with respect to the game G if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_G(\mathcal{A}, \lambda) = \text{negl}(\lambda).$$

Two remarks on the shape of this definition are in order. First, the quantifier order is essential: *for every* PPT adversary, the advantage is negligible. No bound constrains the advantage of one fixed adversary; the claim is universal over the entire class. Second, the use of negl on the right is what makes the definition asymptotic and clean: it is robust to the polynomial slop that reductions introduce, and it captures “the adversary does no better than the trivial strategy, up to an amount that vanishes faster than any inverse polynomial.”

Example 1.20 (Indistinguishability of encryptions, as a game). To anchor the abstraction, consider the canonical encryption-security game, *indistinguishability under chosen-plaintext attack* (IND-CPA). For a public-key encryption scheme with algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$:

1. The challenger runs $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$ and sends pk to $\mathcal{A}$.
2. $\mathcal{A}$ chooses two equal-length messages m_0, m_1 and sends them to the challenger.
3. The challenger samples $b \leftarrow \{0, 1\}$ uniformly and returns the *challenge ciphertext* $c^* \leftarrow \text{Enc}(pk, m_b)$.
4. $\mathcal{A}$ outputs a guess b' ; the game outputs 1 iff $b' = b$.

The scheme is IND-CPA secure iff $\text{Adv}(\mathcal{A}, \lambda) = |\Pr[b' = b] - \frac{1}{2}|$ is negligible for every PPT $\mathcal{A}$. Unwinding the definitions, IND-CPA security is exactly the statement that the two ensembles “encryption of m_0 ” and “encryption of m_1 ” (for adversarially chosen m_0, m_1) are computationally indistinguishable—the game packaging and the ensemble packaging of Definition 1.12 describe the same thing.

Remark 1.21 (Game-hopping). Once we phrase security as a game, proofs become *sequences of games*. One starts from the real security game and rewrites it through a series of games $G_0, G_1, \dots, G_k$, each differing from the last by a small, justifiable change, until reaching a final game in which the adversary provably has zero (or trivially negligible) advantage. Each hop is justified either because the two games are identical unless some negligible-probability “bad” event occurs, or because distinguishing the two games would break a hardness assumption (a reduction, below), or because the change is purely syntactic. Bounding the total advantage is then a telescoping sum across hops—a structured cousin of the hybrid argument (Lemma 1.16). This discipline keeps otherwise sprawling proofs honest and auditable.

1.5 Reductions

The engine of provable security is the reduction. A *reduction* is an efficient transformation that converts a successful adversary against a scheme into a successful algorithm against a problem believed to be hard. Its existence proves the implication “ Y is hard $\Rightarrow X$ is secure” by establishing the contrapositive “ X is broken $\Rightarrow Y$ is solved.”

Definition 1.22 (Computational problem and hardness). A *computational problem* Π consists of a PPT instance generator $\text{Inst}(1^\lambda)$ producing a problem instance together with any side information, and a relation deciding when a candidate output *solves* the instance. The *success probability* of an algorithm $\mathcal{B}$ is

$$\text{Succ}_\Pi(\mathcal{B}, \lambda) := \Pr[\mathcal{B}(1^\lambda, \text{Inst}(1^\lambda)) \text{ solves the instance}].$$

The problem Π is *hard* if $\text{Succ}_\Pi(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ for every PPT $\mathcal{B}$ (for a search problem), or if every PPT $\mathcal{B}$ decides the instance with at most negligible advantage, $\text{Adv}(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ in the sense of Definition 1.18 (for a decision problem).

Definition 1.23 (Black-box reduction). Let X be a scheme with security game G_X and Π a computational problem. A (*black-box*) *reduction* from breaking X to solving Π is a PPT oracle algorithm $\mathcal{R}$ with the following property. There exist functions $\varepsilon_\Pi(\lambda)$ and $\varepsilon_X(\lambda)$ and a polynomial overhead such that for *every* adversary $\mathcal{A}$ achieving advantage $\varepsilon_X(\lambda) := \text{Adv}_{G_X}(\mathcal{A}, \lambda)$, the algorithm $\mathcal{R}^\mathcal{A}$ —which runs $\mathcal{A}$ as a subroutine, simulating $\mathcal{A}$ ’s game internally—solves Π with success probability

$$\text{Succ}_\Pi(\mathcal{R}^\mathcal{A}, \lambda) \geq \varepsilon_\Pi(\lambda),$$

where the reduction’s *loss*, discussed below, relates ε_Π to ε_X . The reduction is *black-box* because $\mathcal{R}$ uses $\mathcal{A}$ only through its input/output interface, making no use of $\mathcal{A}$ ’s code.

The structure of a reduction is always the same and is worth stating as a recipe. To prove “if Π is hard then X is secure”:

1. Assume toward the contrapositive a PPT adversary $\mathcal{A}$ that breaks X with non-negligible advantage ε_X .
2. Construct a PPT algorithm $\mathcal{R}$ that, given a random instance of Π , *embeds* that instance into a simulated security game for $\mathcal{A}$. The simulation must be faithful: the view $\mathcal{R}$ presents to $\mathcal{A}$ must follow (perfectly, statistically, or computationally) the distribution of the real game, so that $\mathcal{A}$ ’s advantage carries over.
3. Run $\mathcal{A}$. From whatever $\mathcal{A}$ does to win its game, $\mathcal{R}$ *extracts* a solution to the Π -instance.
4. Conclude that $\mathcal{R}$ solves Π with non-negligible probability, contradicting the hardness of Π . Therefore no such $\mathcal{A}$ exists, and X is secure.

A complete, genuine reduction illustrates the point.

Theorem 1.24 (Hard-core security of the Diffie–Hellman-style key, schematically). *Let $\Pi = \text{CDH}$ be the computational Diffie–Hellman problem in a cyclic group $\mathbb{G} = \langle g \rangle$ of prime order q : given (g, g^a, g^b) for uniform $a, b \in \mathbb{Z}/q\mathbb{Z}$, compute g^{ab} . Consider the “key-recovery” game G_X for a key-exchange scheme in which the adversary, given the public transcript (g^a, g^b) , wins iff it outputs the shared key g^{ab} . If CDH is hard, then no PPT adversary wins G_X with noticeable probability.*

Proof. Suppose $\mathcal{A}$ is a PPT adversary that, on input a transcript (g^a, g^b) , outputs g^{ab} with probability $\varepsilon_X(\lambda) = \text{Adv}_{G_X}(\mathcal{A}, \lambda)$. Construct $\mathcal{R}$ solving CDH. On input a CDH instance $(g, A, B) = (g, g^a, g^b)$:

1. $\mathcal{R}$ forms the simulated transcript (A, B) and feeds it to $\mathcal{A}$ as the public transcript of G_X .
2. Because (a, b) are uniform in the CDH instance *and* in the real game, the view of $\mathcal{A}$ follows *identically* the distribution of its view in G_X (the simulation is perfect; no loss in faithfulness).
3. When $\mathcal{A}$ outputs a candidate key K , $\mathcal{R}$ outputs K as its CDH answer.

Since the simulated and real views are identically distributed, $\mathcal{A}$ outputs $K = g^{ab}$ with exactly probability $\varepsilon_X(\lambda)$; whenever it does, $\mathcal{R}$ returns the correct CDH solution. Hence

$$\text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \varepsilon_X(\lambda).$$

The running time of $\mathcal{R}$ is that of $\mathcal{A}$ plus a constant amount of group bookkeeping, hence polynomial. If ε_X were noticeable, $\mathcal{R}$ would solve CDH with noticeable probability, contradicting its hardness. Therefore $\varepsilon_X(\lambda)$ is not noticeable, as claimed. In fact, since $\mathcal{R}$ is PPT and CDH is hard (Definition 1.22, search branch), $\varepsilon_X(\lambda) = \text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \text{negl}(\lambda)$. $\square$

This example is deliberately *tight*: the reduction loses nothing, calls the adversary once, and converts advantage ε_X into success probability exactly ε_X . Most reductions are not so fortunate, which leads to the quantitative anatomy of a reduction.

1.5.1 Tightness and security loss

A reduction is not merely a yes/no certificate; it carries quantitative information about *how much* security one scheme inherits from another problem. Two parameters matter: the running-time overhead and the advantage relationship.

Definition 1.25 (Reduction loss and tightness). Consider a reduction $\mathcal{R}$ that transforms an adversary running in time t with advantage ε into a Π -solver running in time $t' = t + t_{\mathcal{R}}$ with success probability ε' . The *security loss* is the factor

$$L(\lambda) := \frac{\varepsilon/t}{\varepsilon'/t'} = \frac{\varepsilon}{\varepsilon'} \cdot \frac{t'}{t},$$

the ratio of the solver's work-per-success to the adversary's work-per-success. A reduction is *tight* if $L(\lambda) = O(1)$ (a small constant, ideally 1), and *loose* if $L(\lambda)$ grows with λ (e.g. $L = q_H$, the number of random-oracle queries, or $L = q_s$, the number of signature queries, or $L = \lambda$). Two common sources of loss are: an *advantage gap* $\varepsilon' \ll \varepsilon$ (the solver succeeds far less often than the adversary, e.g. because the reduction must *guess* where to embed its instance among q queries, succeeding only with probability $1/q$); and a *time blow-up* $t' \gg t$ (the reduction runs the adversary many times, as in rewinding/forking arguments).

Remark 1.26 (Why loss matters concretely). Asymptotically, loss is invisible: a polynomial loss factor turns a negligible advantage into a negligible advantage and a hard problem into security, without qualification. Concretely, however, loss carries a cost. Suppose a scheme reduces to a problem believed to offer 128 bits of security, but the reduction loses a factor $L = 2^{40}$ (not unusual: $q_H \approx 2^{60}$ random-oracle queries times a forking-lemma square-root loss can be much worse). Then the *proof* guarantees only $128 - 40 = 88$ bits of security for the scheme. To restore a 128-bit guarantee one must enlarge the group so that the *underlying* problem offers 168 bits—larger keys, slower operations. A tight reduction lets the scheme inherit the full strength of the assumption at the smallest parameters; a loose one imposes a permanent cost on every user. This is why tightness is a first-class design goal in modern cryptography, not a theoretical nicety.

Remark 1.27 (Rewinding and the forking lemma, in brief). A recurrent source of large loss is *rewinding*: running the adversary, recording its behaviour, then restarting it from a saved state with fresh randomness on some branch to obtain a second, correlated transcript. From two transcripts that share a prefix but diverge afterward one can often extract a secret (e.g. two valid signatures on the same commitment yield the signing key, or two accepting proof transcripts yield a witness). The *forking lemma* quantifies the success probability of obtaining such a pair: if the adversary succeeds with probability ε using one of q possible random-oracle responses, the forked run succeeds with probability roughly $\varepsilon(\varepsilon/q - 1/|\text{range}|)$, i.e. on the order of ε^2/q . The square in ε is a quadratic security loss: to keep ε^2/q noticeable one needs ε noticeably large, roughly halving the bit-security extracted. Such arguments—central to the analysis of Schnorr-type and Fiat–Shamir signatures that recur in the proof systems this monograph builds toward—are the archetype of loose but still valid reductions.

1.6 The standard model and the random oracle model

Reductions reduce security to assumptions. *Which* assumptions one is willing to make defines the *model* in which a proof resides. Two models predominate.

Definition 1.28 (Standard model). A proof resides in the *standard model* if it assumes only the computational hardness of explicitly stated problems (factoring, discrete logarithm, decisional Diffie–Hellman, learning-with-errors, and so on) and grants the adversary no idealised access to any component of the scheme. Every primitive, including every hash function, is a concrete algorithm whose code the adversary may inspect and exploit.

Definition 1.29 (Random oracle model). A proof resides in the *random oracle model* (ROM) if it idealises one or more hash functions as a *random oracle*: a publicly accessible function $\mathcal{O}$ that, on each distinct input, returns an *independent, uniformly random* output of the appropriate length, consistently answering repeated queries with the same value. The oracle is available to the honest parties, to the adversary, and—crucially—to the reduction. No party holds the “code” of $\mathcal{O}$; it can only be queried.

The random oracle model is an idealisation: real hash functions are fixed, public algorithms, not truly random functions. The motivation for adopting it is that the reduction gains two capabilities that the standard model withholds and that render many otherwise-intractable proofs tractable.

Remark 1.30 (The powers of the random oracle: observability and programmability). In the ROM the reduction *simulates* the oracle for the adversary, and this yields two additional capabilities. *Observability*: the reduction sees every query the adversary makes to $\mathcal{O}$. Since the adversary, to use a hash value meaningfully, must query it, the reduction learns the adversary’s intermediate computations—for instance the preimage the adversary is about to hash. *Programmability*: the reduction may choose each oracle answer on the fly, subject only to the answers appearing uniform and consistent. It can therefore *plant* its hard-problem instance inside an oracle response—e.g. set $\mathcal{O}(x) := B$ where B is part of a CDH challenge—so that the adversary’s success necessarily reveals the solution. Neither capability exists for a concrete standard-model hash function, which fixes its outputs in advance and hides its internal queries.

Example 1.31 (The benefit of the ROM: full-domain-hash, sketched). In the “full domain hash” signature, the signer signs a message m as $\sigma = H(m)^d \bmod N$ where (N, d) is an RSA private key—a modulus $N = pq$ with secret prime factors and a secret exponent d paired with a public e so that $x^{ed} \equiv x \pmod{N}$ for x coprime to N , by Euler’s theorem (*Math Guide*, §“Fermat’s little theorem and Euler’s theorem”); thus $x \mapsto x^e$ is a permutation that only the holder of d can invert—and H maps into $\mathbb{Z}/N\mathbb{Z}$. To prove unforgeability one reduces to RSA inversion: given a challenge y (find y^d), the

reduction programs the oracle so that for a random subset of messages $H(m) := y \cdot r_m^e$ (embedding the challenge) and for the rest $H(m) := r_m^e$ (where it knows the “signature” r_m). When the forger outputs a forgery on a message where the reduction embedded the challenge, the reduction extracts y^d . This proof has *no known standard-model analogue* for the plain scheme: it relies essentially on programming H . The cost is a security loss (the reduction must guess which message the forger attacks, incurring a factor q_H unless cleverly optimised)—connecting back to Definition 1.25.

Remark 1.32 (The status of the ROM: heuristic, not theorem). A proof in the ROM is a proof about an *idealised* world: deploying the scheme instantiates $\mathcal{O}$ with a concrete hash function, and the proof does *not* formally transfer—a ROM proof rules out all attacks that treat the hash as a black box, while offering no guarantee against attacks on the specific structure of the chosen hash. Section 3.5 treats the uninstantiability results that make this gap a theorem, and the working stance they justify.

Remark 1.33 (Relatives: CRS, generic-group, and algebraic-group models). Between and beyond these poles lie further idealisations relevant to the proof systems this volume targets. The *common reference string* (CRS) model posits a trusted public string sampled from a prescribed distribution, used by many succinct proof systems. The *generic group model* (GGM) idealises the group dual to the manner in which the ROM idealises a hash: the adversary may only perform group operations through an oracle on opaque “handles,” never exploiting the bit representation of group elements; it is the natural setting for lower bounds on discrete-log-type problems. The *algebraic group model* (AGM) is an intermediate, milder idealisation requiring the adversary to “explain” every group element it outputs as a known combination of its inputs. These models trade realism for provability along the same axis as the ROM, and the constructions ahead invoke them by name where they require them.

1.7 Concrete versus asymptotic security and λ -bit security

The asymptotic language—“negligible,” “polynomial,” “for all sufficiently large λ ”—is mathematically clean but operationally silent. It establishes that a scheme is eventually secure; it does not establish whether *this* deployment with *these* parameters resists an adversary with *that* budget. Concrete (or exact) security supplies the missing numbers.

Definition 1.34 (Concrete security: (t, ε) -security). Fix the security parameter to a concrete value (so all sizes are fixed). A scheme is (t, ε) -secure for a game G if every adversary running in time at most t has advantage $\text{Adv}_G(\mathcal{A}) \leq \varepsilon$. More refined statements bound the advantage as an explicit function of the adversary’s resources, e.g. time t , number of queries q , and memory s : $\text{Adv}_G(\mathcal{A}) \leq f(t, q, s)$.

Definition 1.35 (λ -bit security). A scheme (or problem) has n bits of security, or is n -bit secure, if every adversary running in time t achieves advantage at most about $t/2^n$; equivalently, the best known attack achieving constant (say $\geq 1/2$) success probability requires on the order of 2^n elementary operations. Thus n -bit security means that breaking the scheme costs about 2^n work. One commonly summarises a scheme by the single number n —“128-bit security”—meaning no feasible adversary exists, where the horizon of feasibility lies conventionally around 2^{128} operations.

The relationship to the asymptotic picture is the following: one chooses the security parameter λ *precisely so that* the resulting concrete scheme delivers the desired number of bits of security. In the cleanest case the two coincide, $n = \lambda$, but in general one must account for the actual cost of the best attack on the underlying problem.

Example 1.36 (Why $\lambda \neq n$ for discrete logarithm). For a generic-group or hash-based primitive, doubling the output length doubles the bit security: a hash with $2n$ -bit output offers n -bit collision

resistance (birthday bound) and $2n$ -bit preimage resistance, so the parameter and the security level track simply. For discrete-logarithm-based primitives the accounting differs and instructs. In a group of prime order $q \approx 2^\ell$, generic attacks (baby-step/giant-step, Pollard’s rho) solve discrete logarithm in time $\approx \sqrt{q} = 2^{\ell/2}$. Hence a 256-bit elliptic-curve group offers only about 128 bits of security—one *halves* the field size to obtain the security level. Moreover, for discrete logarithm in the multiplicative group of a finite field, sub-exponential index-calculus attacks force ℓ into the thousands of bits for 128-bit security. This is precisely why elliptic-curve groups (no known sub-exponential attack) are preferred: they permit $\ell \approx 2n$ rather than $\ell \approx$ thousands. The single number n conceals a problem-specific translation from parameter size to security level.

Remark 1.37 (Reading a bit-security budget end to end). Combining the preceding components yields the practitioner’s parameter-selection recipe. Decide the target security level n (commonly 128). Identify the underlying hard problem and the cost of its best attack as a function of the parameter (e.g. $2^{\ell/2}$ for elliptic-curve discrete log), and set the parameter so the problem itself offers n bits (here $\ell = 2n = 256$). Then subtract the reduction’s security loss (Definition 1.25): if the proof loses $L = 2^k$ bits, the problem must offer $n + k$ bits for the *scheme* to offer n , so one enlarges the parameter accordingly—or, with a tight reduction, one need not. Finally, sanity-check against the ROM caveat (Remark 1.32): a ROM proof bounds only black-box attacks on the hash, so the concrete hash must additionally exhibit no exploitable structure. The clean asymptotic theorem “the scheme is secure if the problem is hard” thus unfolds, through the quantitative machinery of this section, into a concrete choice of bytes.

Remark 1.38 (A note on what security does and does not promise). A caution that frames everything above is in order. A proof of security is a proof *relative to a model*: a definition of the goal (the game), a class of adversaries (PPT, possibly non-uniform or quantum), and a set of assumptions (the hard problems, and any idealisation such as the ROM). It guarantees nothing outside that model. Side-channel leakage (timing, power), faulty randomness, implementation bugs, broken assumptions, and goals the game did not capture all lie beyond its reach. The value of the framework is not that it renders schemes unconditionally safe—nothing can—but that it renders the *conditions* of safety explicit, falsifiable, and modular, so that progress on assumptions and attacks translates directly into recalibrated, auditable guarantees. Everything this monograph builds rests on this discipline.

2 Hardness assumptions

Every public-key primitive in this monograph ultimately rests on the belief that some computational problem is intractable: that no realistic adversary, given a randomly drawn instance, can solve it except with negligible probability. This section assembles the family of hardness assumptions on which the discrete-logarithm world rests. It begins with the discrete logarithm problem itself, ascends the ladder of Diffie–Hellman variants, measures exactly how hard these problems can be against *generic* attacks, examines how composite group orders undermine that hardness, instantiates everything on the elliptic curves that the remainder of the book uses, and finally confronts the quantum caveat. Throughout, the lodestar is a single number: the *security parameter* λ , treated as the bit-length of security demanded. The recurring conclusion is that to obtain λ bits of security against the best generic attacks the group must have prime order q with $\log_2 q \geq 2\lambda$.

2.1 The security parameter and negligibility

Section 1 supplies the vocabulary we use throughout: a quantity is *negligible* when it decays faster than the reciprocal of every polynomial (Definition 1.11), adversaries are probabilistic polynomial-time (PPT) machines (Definition 1.3), and for *search* problems the figure of merit is the probability of outputting the exact answer, while for *decision* problems it is the distinguishing advantage of Definition 1.18. A problem is *hard* when every PPT algorithm solves a random instance with at most negligible probability (or distinguishes with at most negligible advantage); alongside this asymptotic

register we use the concrete register of §1.7, in which the fixed Pasta curves of §2.7 offer “about 126 bits” of security against the attacks catalogued below.

2.2 The discrete logarithm problem

The setting is a finite cyclic group. Recall that a group G is *cyclic* if there is an element $g \in G$, called a *generator*, with $G = \langle g \rangle = \{g^k : k \in \mathbb{Z}\}$; writing the group additively, as we shall do for elliptic curves, this reads $G = \{kg : k \in \mathbb{Z}\}$. If $|G| = q$ then $\mathbb{Z}/q\mathbb{Z} \cong G$ via $k \mapsto g^k$, an isomorphism that is trivial to evaluate in the forward direction (by repeated squaring, in $O(\log k)$ group operations) but conjecturally hard to invert. That asymmetry is the discrete logarithm problem.

Definition 2.1 (Discrete logarithm). Let $G = \langle g \rangle$ be a cyclic group of order q , written multiplicatively. For $h \in G$ the *discrete logarithm* of h to the base g , denoted $\log_g h$, is the unique residue $x \in \mathbb{Z}/q\mathbb{Z}$ with $g^x = h$. Existence and uniqueness modulo q follow from $G = \langle g \rangle$ and $\text{ord}(g) = q$: if $g^x = g^{x'}$ then $g^{x-x'} = 1$, so $q \mid x - x'$.

Definition 2.2 (Discrete logarithm problem, DLP). Fix a family $\{G_\lambda\}$ of cyclic groups, where $G_\lambda = \langle g_\lambda \rangle$ has prime order q_λ with $\log_2 q_\lambda = \Theta(\lambda)$, and a sampler that on input 1^λ outputs a description of $(G_\lambda, g_\lambda, q_\lambda)$. The *discrete logarithm problem* is: given such a description and $h = g_\lambda^x$ for x drawn uniformly from $\mathbb{Z}/q_\lambda\mathbb{Z}$, output x . The *DLP assumption* states that for every PPT algorithm $\mathcal{A}$,

$$\Pr_{x \leftarrow \mathbb{Z}/q_\lambda\mathbb{Z}} [\mathcal{A}(G_\lambda, g_\lambda, q_\lambda, g_\lambda^x) = x] \in \text{negl}(\lambda).$$

We restrict the order q to be *prime* for reasons that will become emphatic in §2.6: in prime order every non-identity element is a generator, there are no proper nontrivial subgroups through which to leak information, and one cannot split the problem across factors. For now we record the convenience that random self-reducibility affords.

Proposition 2.3 (Random self-reducibility of DLP). *In a cyclic group $G = \langle g \rangle$ of prime order q , discrete logarithm is random self-reducible: an algorithm that solves a fraction ε of instances (over uniformly random h) converts into one that solves any fixed instance with probability ε per trial. Consequently the hardness of the worst case equals the hardness of the average case.*

Proof. Let $\mathcal{A}$ succeed on a uniformly random target with probability ε . Given a fixed challenge $h = g^x$ whose logarithm we seek, draw r uniformly from $\mathbb{Z}/q\mathbb{Z}$ and form $h' = h \cdot g^r = g^{x+r}$. Because r is uniform and addition modulo q is a bijection, h' is uniformly distributed in G , independently of h ; hence $\mathcal{A}(h')$ returns $x + r \bmod q$ with probability ε . Subtract r to recover x . Each trial uses fresh r , so the trials are independent and t of them succeed with probability $1 - (1 - \varepsilon)^t$, which is overwhelming once $t = \omega(\varepsilon^{-1} \log \lambda)$. One checks correctness of each candidate by testing $g^x \stackrel{?}{=} h$. $\square$

Random self-reducibility is the formal reason cryptographers may speak of “the” hardness of DLP in a group without concern for weak instances: there are none, every instance is as hard as a random one.

2.3 Diffie–Hellman: computational and decisional

DLP asks to invert exponentiation. Many protocols (the Diffie–Hellman key exchange and ElGamal encryption being the historical motivations) need only a weaker-looking guarantee about the *product of exponents*. Suppose Alice publishes g^a and Bob publishes g^b ; their shared secret is g^{ab} , which each

computes from the other's public value and their own exponent. An eavesdropper sees g, g^a, g^b and wants g^{ab} .

Definition 2.4 (Computational Diffie–Hellman, CDH). With notation as in Definition 2.2, the *computational Diffie–Hellman problem* is: given (g, g^a, g^b) with a, b uniform in $\mathbb{Z}/q\mathbb{Z}$, output g^{ab} . The *CDH assumption* states that for every PPT $\mathcal{A}$,

$$\Pr_{a,b}[\mathcal{A}(g, g^a, g^b) = g^{ab}] \in \text{negl}(\lambda).$$

CDH delivers the shared key but not its secrecy: an adversary may be unable to *produce* g^{ab} yet still able to distinguish it from a random group element, which would already break, say, the indistinguishability of an ElGamal ciphertext. The decisional variant excludes this.

Definition 2.5 (Decisional Diffie–Hellman, DDH). The *decisional Diffie–Hellman problem* is to distinguish the two distributions over G^3

$$\mathcal{D}_{\text{dh}} = (g^a, g^b, g^{ab}), \quad \mathcal{D}_{\text{rand}} = (g^a, g^b, g^c),$$

with a, b, c independent and uniform in $\mathbb{Z}/q\mathbb{Z}$. The *DDH assumption* states that every PPT distinguisher D has negligible advantage (Definition 1.18):

$$\text{Adv}_{\text{ddh}}(D) = |\Pr_{a,b}[D(g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c}[D(g^a, g^b, g^c) = 1]| \in \text{negl}(\lambda).$$

A triple (g^a, g^b, g^c) with $c \equiv ab \pmod{q}$ is a *Diffie–Hellman triple*; DDH states that no distinguisher can recognise such triples.

Remark 2.6. DDH, like DLP, is random self-reducible: given a target triple $(u, v, w) = (g^a, g^b, g^c)$, choose r, s, t uniformly and form

$$(u', v', w') = (u^r g^t, v^s, w^{rs} v^{ts}) = (g^{(ar+t)}, g^{bs}, g^{(cr+bt)s}).$$

If $c = ab$ then $cr + bt = b(ar + t)$, so the new exponents satisfy the DH relation and (u', v', w') is a fresh uniform DH triple; if $c \neq ab$ the third exponent is uniform and independent. Thus the rerandomisation maps DH triples to DH triples and non-DH triples to uniform triples, so any noticeable advantage on random instances transfers to every instance.

2.4 The hierarchy $\text{DDH} \leq \text{CDH} \leq \text{DLP}$

The three problems are not independent; they form a chain. Write $A \leq B$ to mean “ A reduces to B ”, i.e. an oracle solving B yields an efficient solver for A ; equivalently, B is *at least as hard* as A , so if B is easy then A is easy; equivalently, the assumption “ A is hard” is the *stronger* one (it implies “ B is hard”). The reductions go

$$\text{DDH} \leq \text{CDH} \leq \text{DLP},$$

so the chain orders the problems by increasing hardness and the corresponding assumptions by decreasing strength: assuming DDH hard is the strongest commitment, assuming DLP hard the weakest.

Theorem 2.7 (DLP is at least as hard as CDH). *If DLP is solvable in PPT, then CDH is solvable in PPT. Equivalently, the CDH assumption implies the DLP assumption.*

Proof. Suppose $\mathcal{A}$ solves DLP. Given a CDH instance (g, g^a, g^b) , run $\mathcal{A}$ on (g, g^a) to recover $a = \log_g(g^a)$. Then compute $(g^b)^a = g^{ab}$ by fast exponentiation. The total cost is one DLP call and $O(\log q)$ group operations, all PPT, and the answer is exactly the CDH solution. (One need invert only one of the two exponents.) $\square$

Theorem 2.8 (CDH is at least as hard as DDH). *If CDH is solvable in PPT, then DDH is solvable in PPT (indeed with advantage overwhelmingly close to 1). Equivalently, the DDH assumption implies the CDH assumption.*

Proof. Suppose $\mathcal{A}$ solves CDH. Construct a distinguisher D for DDH as follows. On input a triple (g^a, g^b, z) , call $\mathcal{A}(g, g^a, g^b)$ to obtain a candidate w for g^{ab} , and output 1 iff $w = z$. If the triple is a DH triple then $z = g^{ab}$, and whenever $\mathcal{A}$ succeeds (probability ε , noticeable by hypothesis) one has $w = z$ and D outputs 1. If the triple is random then $z = g^c$ with c uniform and independent of a, b ; since w depends only on (g^a, g^b) , the event $w = z$ has probability exactly $1/q$. Hence

$$\text{Adv}_{\text{ddh}}(D) \geq \varepsilon - \frac{1}{q},$$

which is noticeable. Amplifying ε to $1 - \text{negl}$ by the random self-reducibility of CDH (rerandomise (g^a, g^b) as in Remark 2.6 and take a majority/consistency vote) drives the advantage to $1 - \text{negl}$. $\square$

The chain is strict in the sense that, in many natural groups, the harder assumptions fail while one believes the easier ones to hold. The principal example is the role of pairings.

Example 2.9 (DDH can be easy while CDH and DLP are hard). Some elliptic-curve groups carry an efficiently computable, non-degenerate *bilinear pairing* $e : G \times G \rightarrow G_T$ with $e(g^a, g^b) = e(g, g)^{ab}$. On such a group DDH is *easy*: to test whether (g^a, g^b, z) is a DH triple, check

$$e(g^a, g^b) \stackrel{?}{=} e(g, z),$$

which holds iff $\log_g z \equiv ab$. Yet CDH (and a fortiori DLP) may remain hard, because the pairing reveals g^{ab} only inside the *target* group G_T , not as an element of G . Such groups, called *gap groups*, separate DDH from CDH and form the foundation of pairing-based cryptography. The implication is that DDH is a genuinely stronger assumption: choosing a group where DDH is believed hard (a “DDH group”) rules out pairings of this convenient kind on G itself.

Remark 2.10. The converse reductions are not known in general. Whether CDH implies DLP (i.e. whether one can extract a from the ability to compute g^{ab}) is the longstanding *Diffie–Hellman vs. discrete log* question; partial results of den Boer and Maurer show equivalence when a suitable auxiliary group of smooth order is available, but no unconditional equivalence is known. Likewise DDH may be strictly easier than CDH, as Example 2.9 shows. In practice one assumes exactly the strength a protocol requires: signatures and key exchange often need only CDH, whereas ElGamal-style encryption needs DDH for its ciphertexts to hide the plaintext.

2.5 Generic algorithms and the square-root barrier

We now quantify the difficulty of DLP. In a group with no exploitable structure beyond the group law, the best known algorithms are *generic*: they manipulate group elements only through the operations multiply, invert, equality-test, never inspecting the bit-representation of an element. Shoup’s idealisation makes the generic model precise: a random injective *labelling* $\sigma : \mathbb{Z}/q\mathbb{Z} \rightarrow S$ encodes the element g^k as the opaque string $\sigma(k)$, and the algorithm may only ask an oracle for $\sigma(i + j)$ given $\sigma(i), \sigma(j)$, and compare labels for equality. We exhibit first two generic algorithms achieving $O(\sqrt{q})$ operations, then give a matching lower bound.

Baby-step giant-step

Proposition 2.11 (Baby-step giant-step, BSGS). *In a cyclic group of order q one can compute any discrete logarithm using $O(\sqrt{q})$ group operations and $O(\sqrt{q})$ stored group elements.*

Proof. Let $m = \lceil \sqrt{q} \rceil$. We can write any $x \in \{0, 1, \dots, q-1\}$ as

$$x = i \cdot m + j, \quad 0 \leq i < m, \quad 0 \leq j < m,$$

since $m^2 \geq q$. The target relation $g^x = h$ becomes $g^{im+j} = h$, i.e.

$$g^j = h \cdot (g^{-m})^i.$$

Baby steps: compute and store the table $\{(j, g^j) : 0 \leq j < m\}$, indexed by the group element g^j in a hash map. This costs $m-1$ multiplications. *Giant steps:* set $u = g^{-m}$ (one inversion and an exponentiation, $O(\log q)$ operations) and for $i = 0, 1, 2, \dots$ compute $h \cdot u^i$ by multiplying the previous value by u ; after each, look it up in the table. By the decomposition some pair (i, j) matches, giving $x = im + j$. The search needs at most m giant steps. Total: $O(\sqrt{q})$ operations and $O(\sqrt{q})$ memory. $\square$

Remark 2.12. BSGS is deterministic and unconditionally $O(\sqrt{q})$ -time, but its $O(\sqrt{q})$ memory is the practical bottleneck: at the 128-bit security level one cannot feasibly store $\sqrt{q} \approx 2^{128}$ table entries. Pollard's rho method matches the time bound with negligible memory, which is why it, rather than BSGS, is the realistic generic attack.

Pollard's rho

Proposition 2.13 (Pollard's rho, sketch). *In a cyclic group of order q (prime) there is a randomised algorithm computing discrete logarithms in expected $O(\sqrt{q})$ group operations and $O(1)$ storage.*

Proof. Sketch. Define a pseudo-random walk on G whose state is a group element together with its known representation $g^\alpha h^\beta$. Partition G into three sets S_1, S_2, S_3 of roughly equal size by some easily computed function of the element's label, and from a point $P = g^\alpha h^\beta$ step to

$$f(P) = \begin{cases} h \cdot P = g^\alpha h^{\beta+1}, & P \in S_1, \\ P^2 = g^{2\alpha} h^{2\beta}, & P \in S_2, \\ g \cdot P = g^{\alpha+1} h^\beta, & P \in S_3. \end{cases}$$

Each step updates the exponent pair $(\alpha, \beta) \in (\mathbb{Z}/q\mathbb{Z})^2$ by an explicit rule and costs $O(1)$ operations. The sequence $P_0, P_1, \dots$ eventually cycles (the state space is finite), tracing the shape of the Greek letter ρ : a tail leading into a loop. Modelling f as a random function over q elements, the birthday bound makes a collision $P_i = P_j$ (with $i \neq j$) appear after $O(\sqrt{q})$ steps in expectation. A collision yields

$$g^{\alpha_i} h^{\beta_i} = g^{\alpha_j} h^{\beta_j} \implies g^{\alpha_i - \alpha_j} = h^{\beta_j - \beta_i}.$$

Writing $h = g^x$ and taking logarithms modulo the prime q :

$$\alpha_i - \alpha_j \equiv x(\beta_j - \beta_i) \pmod{q}.$$

If $\beta_j - \beta_i \not\equiv 0$ it is invertible (primality of q), and $x \equiv (\alpha_i - \alpha_j)(\beta_j - \beta_i)^{-1} \pmod{q}$. Floyd's two-pointer ("tortoise and hare") method, or Brent's improvement, detects the collision with $O(1)$ memory, without storing the trajectory. The exceptional case $\beta_j \equiv \beta_i$ has probability $O(1/q)$, and we re-randomise the start to handle it. $\square$

Remark 2.14. Pollard rho parallelises almost perfectly via van Oorschot–Wiener *distinguished points*: P processors share collisions and achieve a $\sqrt{q}/P$ wall-clock with modest communication, so the relevant security measure is the *cost* of the attack, not the time on a single machine. For this reason concrete security estimates use the operation count $\sqrt{q}$ as the attacker’s budget. A further constant-factor saving (a factor $\sqrt{2}$ in groups admitting the $\pm$ automorphism $P \mapsto -P$, as on elliptic curves) lowers the count to $\sqrt{\pi q}/4$, but leaves the $\Theta(\sqrt{q})$ scaling untouched.

The generic lower bound

These attacks are not merely the best known; up to constants they are the best *possible* for any generic algorithm. This is Shoup’s theorem, the formal statement of the “square-root barrier”.

Theorem 2.15 (Shoup’s generic lower bound). *Let q be prime. Any generic algorithm that, after making at most n queries to the group oracle, outputs the discrete logarithm of a uniformly random challenge in a group of order q succeeds with probability at most*

$$\frac{\binom{n+2}{2} + 1}{q} = O\left(\frac{n^2}{q}\right).$$

In particular, to succeed with constant probability one needs $n = \Omega(\sqrt{q})$ queries.

Proof. Sketch. Use Shoup’s idealisation: a random injective label $\sigma : \mathbb{Z}/q\mathbb{Z} \rightarrow S$ encodes elements, and the algorithm learns labels only through the oracle. The challenge logarithm is an indeterminate X drawn uniformly from $\mathbb{Z}/q\mathbb{Z}$. Every group element the algorithm can form is, by induction on the queries, the image under σ of a known *linear* polynomial $a_i + b_i X \bmod q$ with coefficients the algorithm chose. Treat X as a formal variable: as long as all the polynomials $a_i + b_i X$ the algorithm has produced are pairwise distinct *as functions of X* , the labels it sees are consistent with X being any value, so the algorithm has gained no information and can do no better than guess, succeeding with probability $1/q$. It can gain information only through an “accidental” collision: two distinct polynomials $a_i + b_i X$ and $a_j + b_j X$ that happen to coincide at the actual secret value x , i.e. $(a_i - a_j) + (b_i - b_j)x \equiv 0 \pmod{q}$. For a fixed pair with $(a_i, b_i) \neq (a_j, b_j)$, this linear equation in x has at most one root modulo the prime q , so over a uniform x it occurs with probability $\leq 1/q$. With n queries (plus the inputs g and h) there are at most $\binom{n+2}{2}$ such pairs; a union bound gives total collision probability $\leq \binom{n+2}{2}/q$. Absent any collision the algorithm wins with probability $1/q$; thus its overall success is at most $\binom{n+2}{2}/q + 1/q = O(n^2/q)$. Solving $n^2/q = \Omega(1)$ gives $n = \Omega(\sqrt{q})$. $\square$

Corollary 2.16 (Why $\log_2 q \geq 2\lambda$). *To force every generic attacker to spend at least 2^λ group operations, hence to obtain λ bits of security against generic attacks, the prime group order q must satisfy*

$$\sqrt{q} \gtrsim 2^\lambda \iff \log_2 q \geq 2\lambda.$$

Proof. By Propositions 2.11 and 2.13 a generic attacker needs only $\Theta(\sqrt{q})$ operations, and by Theorem 2.15 no generic attacker can do asymptotically better. Equating the attacker’s cost $\sqrt{q}$ with the target work factor 2^λ gives $q \approx 2^{2\lambda}$, i.e. $\log_2 q \approx 2\lambda$; demanding at least this much yields $\log_2 q \geq 2\lambda$. $\square$

Example 2.17 (The 128-bit target). For $\lambda = 128$ one needs a prime subgroup of order q with $\log_2 q \geq 256$. This accounts for the standardised 256-bit elliptic curves (such as the NIST P-256 / secp256r1 family) targeting 128-bit security with 256-bit group orders: the field and the group order are both about 2^{256} , so $\sqrt{q} \approx 2^{128}$. The Pasta curves of §2.7, with 255-bit orders just above 2^{254} and ≈ 126 -bit

security, sit just below this class. Contrast finite-field DLP in $\mathbb{F}_p^\times$, where the sub-exponential index calculus (Remark 2.18) forces $\log_2 p$ up into the thousands of bits for the same λ ; elliptic curves are valued precisely because no such sub-exponential generic-beating algorithm is known.

Remark 2.18 (Why this bound is special to elliptic curves). The square-root barrier governs *generic* groups. In some concrete groups the representation of elements leaks enough structure to beat it. The multiplicative group $\mathbb{F}_p^\times$ is the cautionary case: *index calculus* exploits the fact that integers factor into small primes (“smooth” numbers) to solve DLP in sub-exponential time $\exp(O((\log p)^{1/3}(\log \log p)^{2/3}))$, far below $\sqrt{p}$. No analogue is known for the group of points on a well-chosen elliptic curve over $\mathbb{F}_p$, because there is no useful notion of a “small” point to factor over. This non-generic gap is precisely why elliptic-curve groups can be small (256 bits) where $\mathbb{F}_p^\times$ must be large (3072 bits) for the same security; it also means the “ $\log_2 q \geq 2\lambda$ ” rule guarantees protection against *generic* attacks only, and one must additionally screen a curve against the special-purpose attacks of §2.7.

2.6 Pohlig–Hellman and the necessity of large prime order

The square-root barrier assumed q prime. If instead the group order factors, discrete logarithm splits into independent pieces of the size of the factors, and the difficulty reduces to that of the *largest prime* factor. This is the Pohlig–Hellman reduction; it is the reason Definition 2.2 insisted on prime order.

Theorem 2.19 (Pohlig–Hellman). *Let $G = \langle g \rangle$ have order $N = \prod_{i=1}^r p_i^{e_i}$ with the p_i distinct primes. Then one can compute a discrete logarithm in G using*

$$O\left(\sum_{i=1}^r e_i (\log N + \sqrt{p_i})\right)$$

group operations. In particular $\sqrt{p_{\max}}$ dominates the cost, where $p_{\max}$ is the largest prime dividing N .

Proof. We compute the logarithm $x = \log_g h \bmod N$ modulo each prime power $p_i^{e_i}$ and recombine it with the Chinese Remainder Theorem, valid because the $p_i^{e_i}$ are pairwise coprime and $\prod p_i^{e_i} = N$.

Reduction to prime-power order. Fix a prime power $p = p_i$, $e = e_i$. Put $g_i = g^{N/p^e}$ and $h_i = h^{N/p^e}$; then g_i has order exactly p^e and $\log_{g_i} h_i \equiv x \pmod{p^e}$, since raising to N/p^e kills the part of x coprime to p ’s power and leaves $x \bmod p^e$. Thus it suffices to solve DLP in the cyclic group $\langle g_i \rangle$ of order p^e .

Reduction to prime order, digit by digit. Write the unknown residue in base p :

$$x \equiv x_0 + x_1 p + \cdots + x_{e-1} p^{e-1} \pmod{p^e}, \quad 0 \leq x_k < p.$$

Let $\gamma = g_i^{p^{e-1}}$, an element of order exactly p . To find x_0 , raise h_i to the power p^{e-1} :

$$h_i^{p^{e-1}} = g_i^{x p^{e-1}} = \gamma^x = \gamma^{x_0},$$

the last step because all higher digits carry a factor p^e that annihilates γ . So $x_0 = \log_\gamma(h_i^{p^{e-1}})$ is one discrete logarithm in a group of prime order p , solvable in $O(\sqrt{p})$ by BSGS or rho (Propositions 2.11, 2.13). Having x_0 , set $h_i^{(1)} = h_i \cdot g_i^{-x_0}$; then $\log_{g_i} h_i^{(1)} \equiv x - x_0 \equiv x_1 p + \cdots \pmod{p^e}$ is divisible by p , and the same procedure at exponent p^{e-2} yields x_1 . Iterating recovers $x_0, \dots, x_{e-1}$ with e prime-order DLPs of cost $O(\sqrt{p})$ each, plus $O(e \log N)$ operations for the exponentiations.

Summing over i and applying CRT gives the stated bound. $\square$

Example 2.20 (A composite-order trap). Suppose, contrary to good practice, a group has order

$$N = 2^{256} - 1 = 3 \cdot 5 \cdot 17 \cdot 257 \cdot 65537 \cdot 641 \cdot 6700417 \cdot \dots,$$

a product of small primes (it is *smooth*). Despite N having 256 bits, the largest prime factor of $2^{256} - 1$ is far below 2^{128} ; Pohlig–Hellman solves DLP in roughly $\sqrt{p_{\max}}$ operations, perhaps only about 2^{36} (the largest prime factor here is $\approx 2^{72.3}$, so $\sqrt{p_{\max}} \approx 2^{36}$), eliminating the apparent 128-bit security. A 256-bit order is necessary but *not sufficient*: it must be (almost) prime.

Corollary 2.21 (Design rule for the group order). *For DLP-based security at level λ the group must have order divisible by a prime q with $\log_2 q \geq 2\lambda$, and the cryptographic group must be (or be restricted to) the subgroup of that prime order q . When the natural group has order $N = h \cdot q$ with a small cofactor h and large prime q , one works in the order- q subgroup; clearing the cofactor (multiplying inputs by h or checking subgroup membership) prevents small-subgroup attacks that would otherwise leak $x \bmod h$ via Pohlig–Hellman on the order- h part.*

Remark 2.22. This is the deeper justification for “prime order” in Definition 2.2. Prime order q makes Pohlig–Hellman vacuous (the only factor is q itself), makes every non-identity element a generator (no element lands in a weak proper subgroup), and removes the cofactor bookkeeping entirely. The Pasta curves below are engineered to have *prime* order, so the entire group is its own prime-order subgroup and the cofactor is 1.

2.7 Instantiation on elliptic curves; the Pasta curves

We now fix the concrete groups used in the remainder of the monograph. Recall from the companion volume that an elliptic curve over a finite field $\mathbb{F}_p$ (with $p > 3$ prime) in short Weierstrass form is

$$E : y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad 4a^3 + 27b^2 \neq 0,$$

and that its set of $\mathbb{F}_p$ -points,

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

together with the chord-and-tangent addition and the point at infinity $\mathcal{O}$ as identity, forms a finite abelian group. The discrete logarithm in this group, “given P and $Q = xP$, find x ”, is the *elliptic-curve discrete logarithm problem (ECDLP)*, and CDH/DDH transcribe verbatim with multiplicative notation replaced by the additive scalar multiplication $x \mapsto xP$. Hasse’s theorem pins down the group size.

Theorem 2.23 (Hasse bound, recalled). *For E over $\mathbb{F}_p$, the number of points satisfies*

$$|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}, \quad \text{i.e.} \quad \#E(\mathbb{F}_p) = p + 1 - t, \quad |t| \leq 2\sqrt{p},$$

where t is the trace of Frobenius. Thus the group order is $p + 1$ up to a deviation of order $\sqrt{p}$.

For the group to give λ -bit security one chooses p of about 2λ bits (so $\#E(\mathbb{F}_p) \approx p \approx 2^{2\lambda}$) and insists, by Corollary 2.21, that $\#E(\mathbb{F}_p)$ be prime (cofactor 1). One then assumes ECDLP, ECCDH, and ECDDH hold in $E(\mathbb{F}_p)$ at the level Corollary 2.16 predicts, *provided* the curve avoids the known structural attacks that beat the generic $\sqrt{q}$ bound. The screening criteria are the following.

Definition 2.24 (Secure-curve criteria). A curve $E/\mathbb{F}_p$ with prime order $q = \#E(\mathbb{F}_p)$ is admissible for λ -bit DLP security if:

1. (*Size / generic.*) $\log_2 q \geq 2\lambda$, so Pollard rho costs $\geq 2^\lambda$ (Cor. 2.16).
2. (*Prime order.*) q is prime, so Pohlig–Hellman is vacuous and the cofactor is 1 (Cor. 2.21).
3. (*Anti-MOV / large embedding degree.*) The multiplicative order k of p modulo q (the *embed-*

ding degree) is large. The MOV/Frey–Rück attack uses a pairing to embed $E(\mathbb{F}_p)$ into $\mathbb{F}_{p^k}^\times$ and then runs sub-exponential index calculus there (Rem. 2.18); this is a threat only when k is small (e.g. supersingular curves, where $k \leq 6$). Demanding k huge defeats it.

4. (*Anti-Smart / not anomalous.*) $q \neq p$, i.e. the curve is not *anomalous* ($\#E(\mathbb{F}_p) = p$). On an anomalous curve the Smart–Satoh–Araki–Semaev attack uses a p -adic (“lift to the formal group / elliptic logarithm”) map to solve ECDLP in *linear* time. The trace $t = 1$ case is fatal, and one must exclude it.
5. (*Twist security.*) The *quadratic twist* E' of E (the curve $dy^2 = x^3 + ax + b$ for a fixed nonsquare $d \in \mathbb{F}_p$; every $x \in \mathbb{F}_p$ is then the x -coordinate of a point of E or of E') should also have a large prime order, so that an attacker cannot push computation onto a weak twist via a fault or an invalid-point injection.

Definition 2.25 (The Pasta curves). The *Pasta* curves are a pair of elliptic curves, *Pallas* and *Vesta*, in short Weierstrass form

$$y^2 = x^3 + 5,$$

(so $a = 0$, $b = 5$, a j -invariant-0 curve) defined over two primes p and q of 255 bits each, arranged into a cycle:

$$\#Pallas = q \text{ (Pallas is defined over } \mathbb{F}_p), \quad \#Vesta = p \text{ (Vesta is defined over } \mathbb{F}_q).$$

That is, the order of each curve equals the field of definition of the other. Both p and q are prime, of bit-length 255, and are congruent to 1 mod 2^{32} , giving large 2-adic valuation in their multiplicative groups for fast FFTs. Each curve has cofactor 1: the whole group is of prime order.

Remark 2.26 (Why a cycle, and what it buys). The defining feature $\#Pallas = q$, the base field of *Vesta*, and $\#Vesta = p$, the base field of *Pallas*, is what makes (Pallas, Vesta) an *amicable* / *cyclic* pair. It lets one curve’s scalar field be the other curve’s coordinate field, so that a proof system can use Pallas-arithmetic to verify statements about Vesta-arithmetic and vice versa without expensive non-native field emulation. This is the mechanism behind recursive proof composition in Halo 2: the recursion alternates between the two curves, each verifying a proof expressed over the other’s scalar field. The cycle structure does not affect the hardness assumptions employed, ECDLP/ECCDH/ECDDH on each curve; the cycle is an *efficiency* property, not a security one, and one screens the two curves independently against Definition 2.24.

Proposition 2.27 (The Pasta curves meet the criteria). *Pallas and Vesta satisfy criteria (1)–(4) of Definition 2.24 at $\lambda \approx 126$; criterion (5) fails for both curves — neither twist has prime order, and Pallas’s twist additionally falls below the SafeCurves 2^{100} twist-attack cost threshold (about 2^{88} , versus about 2^{108} for Vesta) — but the failure is rendered moot by mandatory point validation.*

Proof. Sketch, criterion by criterion. (1) Each order is a 255-bit prime just above 2^{254} (so $\log_2 q \approx 254.0$), giving $\log_2 q \geq 2\lambda$ with $\lambda = 127$ and generic $\sqrt{q} \approx 2^{127}$. Plain Pollard rho costs about $\sqrt{\pi q/2} \approx 2^{127.3}$ operations; the elliptic $\pm$ -automorphism speedup (Remark 2.14) lowers this to $\sqrt{\pi q/4} \approx 2^{126.8}$; and the order-6 automorphism group of these j -invariant-0 curves — the factor $\sqrt{m}$ for an order- m automorphism group of Remark 2.28, here an extra $\sqrt{3}$ — lowers it to $\sqrt{\pi q/12} \approx 2^{126.0}$. Hence “about 126 bits” of security (the same range as the 128-bit curves of Example 2.17). (2) Both orders are prime by construction, so cofactor 1 and Pohlig–Hellman vacuous. (3) Both curves have very large embedding degree (the embedding degree of each is essentially $q-1$ -sized, astronomically larger than any feasible index-calculus target), so they are *not* supersingular and MOV/Frey–Rück does not apply;

indeed $y^2 = x^3 + 5$ over these primes is ordinary. (4) Neither curve is anomalous: $\#Pallas = q \neq p$ and $\#Vesta = p \neq q$, so the trace $t = p + 1 - q \neq 1$ and the Smart attack does not apply. (5) is the one criterion the pair does not meet: the Pasta search deliberately ignored twist security (the curves were generated with the search flag `--ignoretwist`), and neither twist has prime order. The Pallas twist has order $3^2 \cdot 7 \cdot 8191 \cdot 85021 \cdot 11540602760389 \cdot P_{176}$ for a 176-bit prime P_{176} , so the best attack on the twist costs about 2^{88} , below the SafeCurves 100-bit twist-security threshold; the Vesta twist, of order $3^3 \cdot 7 \cdot 13 \cdot 2851 \cdot 30097 \cdot P_{217}$ for a 217-bit prime P_{217} , happens to clear that threshold at about 2^{108} . This is harmless in our setting: twist attacks require an implementation that accepts attacker-supplied x -coordinates without validation (x -only ladders) or skips on-curve checks. Orchard implementations never use x -only arithmetic, and every deserialised point is checked to satisfy $y^2 = x^3 + 5$ (decompression of an x on the twist fails because $x^3 + 5$ is then a non-square); with cofactor 1 there are no small subgroups on the curve itself. Invalid-curve and twist-point injection are therefore excluded by validation, not by twist structure. Therefore the standard generic analysis governs, and the security level is ≈ 126 bits. $\square$

Remark 2.28 (j -invariant 0 and endomorphisms). The choice $y^2 = x^3 + b$ gives j -invariant 0, a curve with complex multiplication by $\mathbb{Z}[\omega]$ where ω is a primitive cube root of unity. This furnishes a cheap endomorphism $\phi(x, y) = (\beta x, y)$ acting as multiplication by a fixed scalar ζ on the group, enabling the GLV speedup for honest scalar multiplication. The same endomorphism gives an attacker a slightly larger automorphism group and hence the constant-factor Pollard-rho speedup mentioned in Remark 2.14 (a factor $\sqrt{m}$ for an order- m automorphism group), but this is a small constant and does not change the $\Theta(\sqrt{q})$ scaling. CM by a small discriminant is, by itself, not known to weaken ECDLP; the danger lies only in *small embedding degree* and *anomalousness*, both excluded above.

2.8 The quantum caveat: Shor’s algorithm

Every hardness assumption above is stated against classical PPT adversaries. A large fault-tolerant quantum computer changes the picture qualitatively, and we must state precisely what it does and does not break.

Theorem 2.29 (Shor, informally). *There is a quantum algorithm that, given a cyclic group $G = \langle g \rangle$ of order q (with efficient group operations) and a target $h = g^x$, outputs x in time polynomial in $\log q$, succeeding with overwhelming probability. Consequently DLP, and hence CDH and DDH, are not hard against quantum adversaries: all three fall in quantum polynomial time, in any group, including elliptic-curve groups.*

Proof. Sketch. Discrete logarithm is an instance of the *hidden subgroup problem* over the abelian group $\mathbb{Z}/q\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}$. Define $f(\alpha, \beta) = g^\alpha h^{-\beta} = g^{\alpha - x\beta}$. Then $f(\alpha, \beta) = f(\alpha', \beta')$ iff $\alpha - x\beta \equiv \alpha' - x\beta' \pmod{q}$, so f is constant exactly on the cosets of the subgroup

$$H = \{(\alpha, \beta) : \alpha \equiv x\beta \pmod{q}\} = \langle (x, 1) \rangle \subseteq (\mathbb{Z}/q\mathbb{Z})^2,$$

a “line of slope x ”. The quantum algorithm prepares a *uniform superposition* over $(\mathbb{Z}/q\mathbb{Z})^2$ —a single register holding every pair at once, each with equal weight—evaluates f into a second register, and applies the *quantum Fourier transform*, the reversible change of basis that re-expresses the register in the characters of $(\mathbb{Z}/q\mathbb{Z})^2$; measuring the result samples characters that vanish on H , and a handful of such samples reveal the slope x by classical linear algebra modulo q . The entire procedure uses $O(\text{poly}(\log q))$ qubits and gates. Because it queries the group only through the black-box operation $(\alpha, \beta) \mapsto g^\alpha h^{-\beta}$, it is agnostic to the group’s representation: it breaks elliptic-curve DLP exactly as it breaks $\mathbb{F}_p^\times$ DLP and integer factoring (the latter via the order-finding variant). $\square$

Remark 2.30 (Quantum does not contradict the square-root barrier). There is no contradiction with Shoup’s bound (Theorem 2.15): the proof of that lower bound assumes the *generic* model for *classical* algorithms making sequential, individual oracle queries. Shor’s algorithm is non-generic in the

relevant sense, it queries the group function in superposition and exploits the periodicity revealed by the Fourier transform, an ability the classical generic model does not grant. Hence the square-root barrier and Shor’s polynomial-time attack coexist: the former bounds classical generic attackers, the latter is a quantum non-generic attacker.

Remark 2.31 (What Shor breaks and what survives). Shor’s algorithm decisively breaks every assumption in this section, the entire discrete-log family and integer factoring, in polynomial time, so all of RSA, finite-field Diffie–Hellman, and elliptic-curve cryptography are *quantum-broken*. What it does *not* break:

- *Symmetric primitives and hash functions.* The best generic quantum attack on a search problem is Grover’s algorithm, a quadratic, not exponential, speedup. A symmetric cipher with a 2λ -bit key, or a hash with a 2λ -bit output, retains λ bits of *quantum* security. Doubling key/output lengths suffices.
- *Post-quantum assumptions.* Hardness based on lattices (LWE, SIS), codes, multivariate systems, and isogenies is not known to fall to Shor and underlies post-quantum cryptography.

The discrete-log assumptions adopted here are therefore secure only under the standing premise that no cryptographically relevant quantum computer exists. Within that premise, and against all classical attacks, the Pasta curves deliver the ≈ 126 -bit security analysed in §2.7, which is the ground on which the Halo 2 constructions to come are built.

Remark 2.32 (Standing assumptions for the sequel). We summarise here the premises on which the rest of the monograph relies. For each Pasta curve $C \in \{\text{Pallas}, \text{Vesta}\}$ with prime order q_C and generator g_C :

1. ECDLP is hard in $\langle g_C \rangle$: no classical PPT adversary finds x from xg_C with non-negligible probability (concretely, cost $\geq 2^{126}$).
2. ECCDH and, where needed, ECDDH hold in $\langle g_C \rangle$ (Definitions 2.4, 2.5).
3. The adversary is classical and probabilistic-polynomial-time (Remark 2.31).

By the hierarchy of §2.4, assuming ECDDH is the strongest and implies the rest; a given construction invokes the weakest assumption it requires.

3 Hash functions and the random oracle model

A hash function compresses arbitrary data into a short, fixed-length tag. This single, simple-sounding operation is the most ubiquitous tool in cryptography: it underlies digital signatures, commitment schemes, message authentication, key derivation, proof-of-work, Merkle trees, and the Fiat–Shamir transform that turns interactive proofs into the non-interactive arguments at the heart of the Halo 2 system. A hash function can serve so many purposes because, when it behaves “ideally,” it acts like a deterministic source of fresh randomness keyed by its input. This section develops the subject from the ground up: we fix the syntax, isolate the three classical security notions and the generic birthday attack that bounds them, formalise the idealisation that the random oracle model provides (together with its discontents), treat the operations needed downstream—domain separation, hashing into a finite field, and hashing onto an elliptic curve—and close with the arithmetisation-friendly hash Poseidon that Orchard evaluates inside its circuit.

3.1 Syntax

We start from the bare data type. Throughout, $\{0, 1\}^*$ denotes the set of all finite binary strings, $\{0, 1\}^n$ the strings of length exactly n , and $|x|$ the length of a string x . We write concatenation as $x \parallel y$.

Definition 3.1 (Hash function). A *hash function* with output length $n \in \mathbb{N}$ is a function

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n.$$

We call the elements of the codomain *digests*, *hashes*, or *tags*. H is *compressing* on a domain $D \subseteq \{0, 1\}^*$ if D contains strings longer than n ; when the domain is all of $\{0, 1\}^*$ the function is overwhelmingly many-to-one.

Two remarks on the syntax are in order before we attach security to it.

Remark 3.2 (Keyed versus unkeyed; the foundations problem). Definition 3.1 describes a single, fixed, unkeyed function such as the SHA-256 standard. This is what practice deploys, but it is awkward for asymptotic security definitions: for any fixed compressing H there *exist* two strings $x \neq x'$ with $H(x) = H(x')$ (by the pigeonhole principle), and there exists a tiny program—“print these two hardcoded strings”—that outputs such a pair. How to write that program down efficiently is not known, but the asymptotic statement “no efficient adversary finds a collision” is false for a fixed function. The standard theoretical remedy is a *keyed family* (sometimes called a hash function ensemble): a function

$$H : \mathcal{K} \times \{0, 1\}^* \longrightarrow \{0, 1\}^n,$$

where one samples a key $s \in \mathcal{K}$ publicly at random and fixes it thereafter, written $H_s(\cdot) = H(s, \cdot)$. The key is not secret; it merely indexes which member of the family is in force, defeating the hardcoding objection because the adversary must succeed for a random member. Throughout this section we state security notions for the keyed family, since that is where the definitions are clean, but every concrete construction discussed is unkeyed, and the reader should mentally identify the key with “the choice of standard.” We make the distinction explicit when it matters.

Remark 3.3 (Variable output length). Some primitives produce a digest of caller-chosen length; these are *extendable-output functions* (XOFs), with syntax $H : \{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$, where $H(x, \ell)$ has length ℓ and is a prefix-consistent stream (longer requests extend shorter ones). SHAKE128 and SHAKE256 from the SHA-3 family are the canonical examples. XOFs are the natural tool for hashing to a field or curve, where one needs a controlled number of uniform bits; we revisit them in §3.7.

3.2 Security notions: preimage, second-preimage, and collision resistance

The notion of a “good” hash function admits no single definition; distinct applications require distinct guarantees, and these guarantees form a strict hierarchy. We isolate the three classical notions

below. Each takes the form of a game between a challenger and a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, and the function is secure if every PPT $\mathcal{A}$ wins with probability negligible in a security parameter λ (with the output length $n = n(\lambda)$, the key length, and a polynomially bounded challenge input length $m = m(\lambda)$) all functions of λ ; the games below sample the challenge x uniformly from $\{0, 1\}^m$. Recall that a function $\varepsilon(\lambda)$ is *negligible*, written $\varepsilon \in \text{negl}(\lambda)$, if it decays faster than the reciprocal of every polynomial.

Definition 3.4 (Preimage resistance / one-wayness). A keyed hash family H is *preimage resistant* if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_H^{\text{pre}}(\mathcal{A}) = \Pr[s \leftarrow \mathcal{K}; x \leftarrow \{0, 1\}^m; y \leftarrow H_s(x); x' \leftarrow \mathcal{A}(s, y) : H_s(x') = y] \in \text{negl}(\lambda).$$

Informally: given a digest y of a random message, no efficient adversary can find *any* message hashing to y . Note that $\mathcal{A}$ need not recover the original x ; any preimage wins.

Definition 3.5 (Second-preimage resistance). A keyed hash family H is *second-preimage resistant* if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_H^{\text{sec}}(\mathcal{A}) = \Pr[s \leftarrow \mathcal{K}; x \leftarrow \{0, 1\}^m; x' \leftarrow \mathcal{A}(s, x) : x' \neq x \wedge H_s(x') = H_s(x)] \in \text{negl}(\lambda).$$

Here the challenger *gives* the adversary a specific message x , and the adversary must produce a different message colliding with it.

Definition 3.6 (Collision resistance). A keyed hash family H is *collision resistant* if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_H^{\text{col}}(\mathcal{A}) = \Pr[s \leftarrow \mathcal{K}; (x, x') \leftarrow \mathcal{A}(s) : x \neq x' \wedge H_s(x) = H_s(x')] \in \text{negl}(\lambda).$$

Here the adversary chooses *both* messages and need only make them collide.

The three notions differ in who chooses what, and this dictates their relative strength. The following proposition records the hierarchy; it is elementary but instructive, since the reductions reveal precisely why collision resistance is the strongest demand.

Proposition 3.7 (Hierarchy of notions). *Let H be a keyed hash family that is compressing (input length $m > n$). Then:*

1. *Collision resistance implies second-preimage resistance.*
2. *Second-preimage resistance implies preimage resistance, up to a polynomial loss, when H is sufficiently compressing.*

Neither implication reverses in general.

Proof. (1) *Collision $\Rightarrow$ second-preimage.* We give a reduction: from an adversary $\mathcal{A}$ breaking second-preimage resistance we construct $\mathcal{B}$ breaking collision resistance. On input key s , $\mathcal{B}$ samples $x \leftarrow \{0, 1\}^m$ itself, runs $x' \leftarrow \mathcal{A}(s, x)$, and outputs the pair (x, x') . Whenever $\mathcal{A}$ succeeds one has $x' \neq x$ and $H_s(x') = H_s(x)$, which is precisely a collision. Hence $\text{Adv}_H^{\text{col}}(\mathcal{B}) \geq \text{Adv}_H^{\text{sec}}(\mathcal{A})$, and if the left side is negligible so is the right.

(2) *Second-preimage $\Rightarrow$ preimage (with compression).* We argue the contrapositive: an adversary $\mathcal{A}$ inverting H yields $\mathcal{B}$ finding second preimages. On input (s, x) , $\mathcal{B}$ computes $y = H_s(x)$, runs

$x' \leftarrow \mathcal{A}(s, y)$, and outputs x' . When $\mathcal{A}$ succeeds, $H_s(x') = y = H_s(x)$, so x' is a valid digest preimage; it remains to argue $x' \neq x$ with good probability. This is where compression enters. Fix $\mathcal{A}$'s coins; for each digest y the output $x' = \mathcal{A}(s, y)$ is then a single string, so at most one input per fibre satisfies $x = \mathcal{A}(s, H_s(x))$ —at most 2^n inputs in all, one per digest—and over the uniform choice of x , $\Pr[x' = x] \leq 2^n/2^m = 2^{n-m}$. Hence $\text{Adv}_H^{\text{sec}}(\mathcal{B}) \geq \Pr[\mathcal{A} \text{ succeeds}] - \Pr[x' = x] \geq \text{Adv}_H^{\text{pre}}(\mathcal{A}) - 2^{n-m}$. The loss is additive, not multiplicative: $\mathcal{A}$'s successes may concentrate on small fibres, so nothing bounds $\Pr[x' \neq x]$ conditioned on success. The hedge “sufficiently compressing” in the statement earns its keep here: for $m - n = \omega(\log \lambda)$ the term 2^{n-m} is negligible, and second-preimage resistance forces $\text{Adv}_H^{\text{pre}}(\mathcal{A})$ to be negligible as well.

Non-reversal. Contrived counterexamples demonstrate the separations. For “preimage-resistant but not collision-resistant,” take any one-way H and define $H'(x) = H(|x|)$, where $|x|$ deletes the last bit; H' inherits one-wayness on a random input, yet $x \parallel 0$ and $x \parallel 1$ constitute an immediate collision. The other separations are similar; the implications are one-directional. $\square$

Remark 3.8 (Which application needs which). The notions are not interchangeable. A password-hashing scheme storing $H(\text{password})$ requires *preimage* resistance and nothing more—an attacker who learns the digest must not recover an input. A scheme that signs $H(m)$ and permits an adversary to later substitute a forged document requires *second-preimage* resistance for the honestly chosen m . A scheme in which the *signer themselves* might cheat—preparing two contracts with the same hash, signing the benign one, and later presenting the malicious one—requires full *collision* resistance, since here the adversary controls both messages. Commitment schemes, Merkle trees, and the Fiat–Shamir transform all fall in this last, most demanding category, which is why collision resistance is the headline property of a cryptographic hash.

3.3 The birthday bound on generic collision finding

A *generic* attack that applies to *any* hash function, treating it as a black box, bounds collision resistance from above. This attack determines how large n must be: it sets the security ceiling that no construction can exceed. The attack is the birthday attack, named after the birthday paradox.

Lemma 3.9 (Birthday bound). *Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ and write $N = 2^n$. Suppose we evaluate H on q distinct inputs whose digests behave as independent uniform samples from $\{0, 1\}^n$ (the idealised model). Let C be the event that two of the q digests coincide. Then*

$$\Pr[C] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right),$$

and this satisfies the two-sided estimate

$$\Pr[C] \geq 1 - e^{-q(q-1)/(2N)}, \quad \text{more usefully} \quad \Pr[C] \leq \frac{q(q-1)}{2N} \leq \frac{q^2}{2N},$$

and for the lower direction $\Pr[C] \geq 1 - e^{-q(q-1)/(2N)} \geq \frac{1}{2}$ once $q \geq 1.18\sqrt{N}$. In particular a collision appears with constant probability after about $\sqrt{N} = 2^{n/2}$ evaluations.

Proof. This is the birthday-bound proposition of the *Math Guide* (§“The union bound and a birthday calculation”), with $N = 2^n$ and $k = q$; its proof establishes the exact product expression. Only the $1.18\sqrt{N}$ threshold is new: setting the exponent of the lower bound to $-\ln 2$ gives $q(q-1) = 2N \ln 2$, i.e. $q \approx \sqrt{2 \ln 2} \sqrt{N} \approx 1.18\sqrt{N}$, at which point $\Pr[C] \geq \frac{1}{2}$. $\square$

Remark 3.10 (Reading the bound and its memory cost). The consequence is the *square-root law*: an n -bit digest yields only $n/2$ bits of collision security. To attain the conventional 128-bit collision security

one must take $n = 256$; this is precisely why SHA-256 (and not SHA-128, which does not exist) is the workhorse, and why Fiat–Shamir transcript hashes must have at least 256-bit output. The naive attack as described stores all $q \approx 2^{n/2}$ digests, costing $2^{n/2}$ memory, but cycle-finding techniques (Floyd’s or Brent’s algorithm, and Pollard’s rho / van Oorschot–Wiener parallel collision search) achieve the same $2^{n/2}$ time in essentially constant memory by iterating $x \mapsto H(x)$ and detecting the cycle, which renders the attack genuinely practical. Note the contrast with preimage and second-preimage resistance, whose best generic attack is brute-force search costing 2^n evaluations: these notions enjoy full n -bit security, twice the collision security, since there the adversary cannot exploit the birthday coincidence among self-chosen pairs.

3.4 The random oracle model

The security notions of §3.2 capture specific, isolated properties. In practice, however, we use hash functions as if they were much stronger: as if $H(x)$ were a uniformly random value, freshly and independently sampled for each new input x , yet consistently returned on repeats. This idealisation is the *random oracle*.

Definition 3.11 (Random oracle). A *random oracle* with output length n is a function $\mathcal{O} : \{0, 1\}^* \rightarrow \{0, 1\}^n$ drawn uniformly at random from the set of all such functions. Equivalently, $\mathcal{O}$ runs *lazily*: it maintains a table; on a query x it returns the stored value if it has already seen x , and otherwise samples $\mathcal{O}(x) \leftarrow \{0, 1\}^n$ uniformly, records it, and returns it. The defining features are: outputs on *distinct* inputs are *independent* and *uniform*; outputs on *repeated* inputs are *consistent*; and one can access the table *only* by querying—there is no shorter description of $\mathcal{O}$ than its (exponential) truth table.

Definition 3.12 (Random oracle model). A scheme lives in the *random oracle model* (ROM) if every party, honest or adversarial, has oracle access to a single shared random oracle $\mathcal{O}$ in place of a concrete hash function, and one proves security relative to the random choice of $\mathcal{O}$. On deployment, a concrete hash such as SHA-256 or BLAKE2b *instantiates* $\mathcal{O}$.

The benefit of this idealisation to a proof is that it grants the analyst two properties that no concrete hash can offer but that a random oracle does, almost by definition.

Proposition 3.13 (Powers granted by the ROM). *In a security proof in the ROM, the reduction (which simulates $\mathcal{O}$ to the adversary) gains two capabilities unavailable for a concrete hash:*

1. *Extractability (observability): the reduction sees every input on which the adversary queries $\mathcal{O}$. Since the only way to learn anything about $\mathcal{O}(x)$ is to query x , if the adversary’s output depends on $\mathcal{O}(x)$ then the reduction holds x .*
2. *Programmability: the reduction may choose the response $\mathcal{O}(x)$ on a fresh query x to be any value of its liking, provided that value is distributed uniformly (so the simulation is perfect). It can thus plant a challenge—e.g. set $\mathcal{O}(x)$ equal to a value it must invert—inside an oracle answer.*

Proof. Both follow immediately from the lazy-sampling realisation (Definition 3.11). For observability, the reduction is the table-keeper: it receives each query x before answering, so it observes the adversary’s query set in full. For programmability, on a not-yet-seen input the reduction may fill the table entry with any value, and as long as that value is uniform and independent of the adversary’s view so far, the adversary cannot distinguish it from a genuine fresh sample; the table maintains consistency on repeats. $\square$

Example 3.14 (Why Fiat–Shamir needs the ROM). The Fiat–Shamir transform—which renders Halo 2’s interactive protocol non-interactive—replaces the verifier’s random challenge e by $e = \mathcal{O}(\text{transcript so far})$. We prove soundness of the resulting non-interactive argument by *rewinding*: the reduction observes (capability 1) the transcript the prover commits to, then *reprograms* (capability 2) the oracle to return a fresh, independent challenge on that same transcript, extracting two accepting transcripts that share a prefix and thereby a witness. Neither step is available with a concrete hash, whose outputs the reduction can neither observe selectively nor reprogram. This is the paradigmatic example of a proof that lives only in the ROM, and it is the reason the ROM is indispensable to the cryptography this monograph builds toward.

3.5 What the ROM idealises, and its known limits

Remark 3.15 (The heuristic and its standard justification). A ROM proof is a *heuristic*, not a theorem about the deployed system. The argument for trusting it proceeds: (i) one proves the scheme secure assuming $\mathcal{O}$ is a perfect random oracle; (ii) a hash function believed to have “no exploitable structure” instantiates $\mathcal{O}$; (iii) one *hopes* that any real attack would have to exploit some structure of the concrete hash that a random oracle lacks, and therefore would also break the (presumed structure-free) hash directly. The empirical track record is strong: schemes with ROM proofs (RSA-OAEP, RSA-FDH, Schnorr/EdDSA signatures via Fiat–Shamir, the Boneh–Franklin IBE) have resisted attack for decades, and a ROM proof reliably rules out the entire class of attacks that treat the hash as a black box, which is most of them. This is why the ROM is the dominant model for practical protocol design.

The heuristic is nonetheless known to be *unsound in the worst case*: there exist schemes provably secure in the ROM that become insecure for *every* concrete instantiation. This is not a minor caveat but a theorem, and a graduate reader should understand its shape.

Theorem 3.16 (Uninstantiability; Canetti–Goldreich–Halevi). *There exists a (contrived) signature scheme, and likewise an encryption scheme, that is provably secure in the random oracle model but is insecure when any efficiently computable function replaces the random oracle. Hence no concrete hash function can soundly instantiate the random oracle for all schemes: a ROM proof does not, in general, imply security of the instantiated scheme.*

Proof. Sketch. The separation exploits the one structural difference between a random oracle and any concrete function: a concrete function H has a *short description* (its code), whereas a random oracle does not. The construction modifies a secure scheme by adding a “backdoor” clause to, say, the signing algorithm: “if the message m , parsed as a program, satisfies $m(\langle \text{description of the hash} \rangle) = H(0)$ ”—i.e. if the adversary submits a program that correctly predicts the hash’s own output on a fixed point given the hash’s code—“then output the secret signing key.” In the ROM, $\mathcal{O}$ has no finite code for the adversary to feed in, and predicting $\mathcal{O}(0)$ without querying it is infeasible, so the backdoor clause never fires and the scheme inherits the security of the base scheme. But once a concrete H with known code instantiates $\mathcal{O}$, the adversary simply submits the program “compute $H(0)$,” the clause fires, and the secret key leaks. The base scheme is secure, the modification is invisible in the ROM, and yet *every* concrete instantiation breaks. (Sharper constructions remove the artificiality, but this version conveys the mechanism: the ROM proof relies on $\mathcal{O}$ being un-codeable, a property no real hash has.) $\square$

Remark 3.17 (How to read the limits). The uninstantiability results are *contrived*: no natural, deployed scheme is known to suffer from them. The community’s working stance is therefore: a ROM proof is strong positive evidence and rules out all black-box attacks, but it is not a guarantee, and one should prefer schemes with standard-model proofs when they are available at comparable cost, and treat the concrete hash’s structure with suspicion (e.g. avoid length-extension-prone constructions). A finer caveat relevant to post-quantum security is the *quantum* random oracle model (QROM), in which the adversary may query $\mathcal{O}$ in superposition; observability and programmability are subtler there (one

cannot freely “read off” a superposition query), and several classical ROM proofs require genuinely new techniques to port to the QROM. We flag this because Halo 2’s long-term security goals include plausible post-quantum arguments, where the QROM is the right model.

3.6 Domain separation and personalisation

A single hash function routinely serves many logically distinct purposes within one protocol—to derive keys, to commit to notes, to tag notes as spent, to seed Fiat–Shamir challenges. If all these uses draw on the literally same function H , an adversary can *replay* an output produced for one purpose as if it were produced for another, since the digests are interchangeable bit strings. *Domain separation* is the discipline of making the same physical hash behave like a family of *independent* oracles, one per purpose.

Definition 3.18 (Domain separation). Let H be a hash (modelled as a random oracle) and let $\{\tau_i\}_{i \in I}$ be a set of distinct, prefix-free *domain tags* (also called *labels* or *personalisation strings*), one per intended use. The use- i hash is

$$H^{(i)}(x) = H(\tau_i \parallel x).$$

The tags are *prefix-free* (no τ_i is a prefix of another, e.g. all of equal length, or each length-prefixed) so that the input spaces $\tau_i \parallel \{0, 1\}^*$ are pairwise disjoint.

Proposition 3.19 (Domain separation yields independent oracles). *If H is a random oracle and the tags $\{\tau_i\}$ are prefix-free, then the family $\{H^{(i)}\}_{i \in I}$ is distributed exactly as a collection of independent random oracles. Consequently a digest obtained from $H^{(i)}$ is, except with the negligible probability of a generic collision, useless to an adversary against a use $j \neq i$.*

Proof. By prefix-freeness the domains $\tau_i \parallel \{0, 1\}^*$ are pairwise disjoint, so the inputs queried through different $H^{(i)}$ are never equal. A random oracle assigns independent uniform values to distinct inputs (Definition 3.11); restricting the single oracle H to the disjoint slices indexed by the tags therefore yields independent uniform functions on each slice, which is precisely a family of independent random oracles. The replay statement follows: a value $H^{(i)}(x)$ reveals nothing about any $H^{(j)}(\cdot)$ for $j \neq i$, as the latter are independent of it. $\square$

Example 3.20 (Personalisation in practice). Three idioms implement Definition 3.18. (a) *Prefix tagging*: prepend an ASCII context string, e.g. “Zcash_nf” for nullifiers versus “Zcash_cm” for note commitments, ensuring distinct slices. (b) *Native personalisation*: BLAKE2b’s parameter block carries a 16-byte personalisation field that mixes into the IV, so `BLAKE2b(person="ZcashKDF")` and `BLAKE2b(person="ZcashPRF")` are, by construction, independent functions without touching the message at all—this is why Zcash favours BLAKE2 for its oracles. (c) *Suffix/framing in XOFs*: SHA-3 reserves domain-separation *suffix bits* (01 for SHA-3 hashing, 1111 for SHAKE) appended before padding, so that SHA3-256 and SHAKE256, sharing the same permutation, can never collide across uses. The unifying requirement, in all three, is that distinct uses induce disjoint effective inputs to the underlying primitive—realised in idiom (a) by the prefix-freeness of Proposition 3.19: an ad hoc tag that is a prefix of another silently merges two domains and reopens the replay attack.

3.7 Hashing to a field element

The cryptography ahead lives over a finite field $\mathbb{F}_p$ (and an elliptic curve over it), not over bit strings. We must turn an arbitrary message into a field element—and later a curve point—in a manner that inherits the uniformity of a random oracle. The obstacle is subtle: a hash gives uniform *bits*, but

reducing n uniform bits modulo a prime p does *not* give a uniform element of $\mathbb{F}_p$ without care, since 2^n is not a multiple of p .

Definition 3.21 (Hashing to a field, via expand-then-reduce). Let p be a prime with $\lceil \log_2 p \rceil = L$ bits. Fix a XOF or domain-separated hash expand producing arbitrary-length uniform output. To map a message m to a field element $\mathbb{F}_p$:

1. Compute $b \leftarrow \text{expand}(\tau \parallel m, L+k)$, a string of $L+k$ uniform bits, where k is a security slack (commonly $k = 128$) and τ is a domain tag.
2. Interpret b as an integer $B \in \{0, \dots, 2^{L+k} - 1\}$ and output $B \bmod p \in \mathbb{F}_p$.

Proposition 3.22 (Near-uniformity of expand-then-reduce). *If expand is a random oracle, the output of Definition 3.21 lies within statistical distance at most $p \cdot 2^{-(L+k)} \leq 2^{-k}$ of the uniform distribution on $\mathbb{F}_p$. Hence with $k = 128$ no efficient algorithm can distinguish the map from a uniform field element.*

Proof. Let B be uniform on $\{0, \dots, M-1\}$ with $M = 2^{L+k}$. For a target residue $r \in \{0, \dots, p-1\}$, the number of $B \in \{0, \dots, M-1\}$ with $B \equiv r \pmod{p}$ is either $\lfloor M/p \rfloor$ or $\lceil M/p \rceil$, differing by one. Thus $|\Pr[B \bmod p = r] - 1/p| \leq 1/M$ for each of the p residues, and the statistical distance to uniform is

$$\frac{1}{2} \sum_{r=0}^{p-1} \left| \Pr[B \bmod p = r] - \frac{1}{p} \right| \leq \frac{1}{2} p \cdot \frac{1}{M} = \frac{p}{2^{L+k+1}} \leq 2^{-(k+1)} \cdot \frac{p}{2^L} \leq 2^{-k},$$

using $p < 2^L$ so $p/2^L < 1$. The bias from naive single-block reduction (taking $k = 0$) can reach as much as 2^{-1} of a residue’s mass when p is just above a power of two; the slack k is exactly what reduces it to negligibility. $\square$

Remark 3.23 (The standardised hash-to-field). The IETF “hash-to-curve” specification packages Definition 3.21 as a routine `hash_to_field(m, count)` that produces count field elements, internally calling an `expand_message` primitive (either `expand_message_xmd` built on a fixed-output hash like SHA-256, or `expand_message_xof` built on SHAKE) with a mandatory domain-separation tag (DST). The specification sets the L parameter there to $\lceil (\lceil \log_2 p \rceil + k)/8 \rceil$ bytes with k the suite’s target security level in bits (RFC 9380’s own suites for 255-bit fields, such as the curve25519 suites, set $k = 128$, i.e. $L = 48$ bytes; the Zcash suite for Pallas and Vesta is more generous, taking $k = 256$ so that each element is reduced from a full 64-byte BLAKE2b-512 digest), exactly realising the slack of Proposition 3.22. This is the concrete recipe we use to map identities to field elements and to produce the field elements inside hash-to-curve (§3.8); the deployed Fiat–Shamir transcript realises the same wide-output-then-reduce principle of Proposition 3.22 by squeezing a personalised BLAKE2b state (*Halo 2 Guide*).

3.8 Hashing to a curve point

Finally, we must map a message to a point of an elliptic curve $E/\mathbb{F}_p$ —to derive the “nothing-up-my-sleeve” generators of a commitment scheme, or to hash to a point in a verifiable random function. Let E be given in short Weierstrass form by $y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$. Two requirements pull against each other: the output point should be *uniform* (or close to it) in the relevant subgroup, modelling a random oracle into the group; and the map should be *constant-time*, leaking nothing through timing.

Remark 3.24 (The naive try-and-increment map and its flaw). The obvious approach: hash m to a candidate $x \in \mathbb{F}_p$ (via §3.7), test whether $g(x)$ is a quadratic residue (so that a y exists), and if not increment x and retry. This is correct and roughly uniform, but the *number of trials* depends on the message—about half of all x fail the residue test (by the theory of the Legendre symbol $\left(\frac{g(x)}{p}\right)$)—so the

running time leaks information about m , an avenue for side-channel attacks. Constant-time hashing to a curve requires a map that succeeds on the *first* try for every input.

The modern solution is an explicit, everywhere-defined algebraic map from $\mathbb{F}_p$ to $E(\mathbb{F}_p)$. The basic tool is the *simplified Shallue–van de Woestijne–Ulas (SWU) map*, which applies directly to curves with $ab \neq 0$. For curves with j -invariant 0 ($a = 0$, such as the Pallas/Vesta curves of Halo 2) or 1728 ($b = 0$) the simplified SWU map does not apply directly; instead one evaluates it on an *isogenous* auxiliary curve with $a'b' \neq 0$ —one related to the target curve by an *isogeny*, a nonconstant map between curves, given by polynomial formulas in the coordinates, that respects the group law—and transports the result back through the isogeny map (see Remark 3.27).

Definition 3.25 (Simplified SWU map, high level). Let $E: y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$ with $ab \neq 0$, and fix a nonsquare $Z \in \mathbb{F}_p$ (a “nothing-up-my-sleeve” constant, e.g. the smallest nonsquare). The simplified SWU map $\text{SWU}: \mathbb{F}_p \rightarrow E(\mathbb{F}_p)$ sends a field element u to a point as follows. It computes two candidate x -coordinates,

$$x_1 = \frac{-b}{a} \left(1 + \frac{1}{Z^2 u^4 + Z u^2} \right), \quad x_2 = Z u^2 x_1,$$

(with a defined fallback when the denominator vanishes), and uses the key algebraic identity

$$g(x_2) = Z^3 u^6 g(x_1),$$

together with the fact that Z is a nonsquare, to prove that *exactly one* of $g(x_1), g(x_2)$ is a quadratic residue. It then sets $x = x_1$ if $g(x_1)$ is square and $x = x_2$ otherwise, takes $y = \sqrt{g(x)}$ with a fixed sign convention, and outputs (x, y) .

Proposition 3.26 (Correctness of the simplified SWU branch selection). *With notation as in Definition 3.25 and u such that the denominator $D = Z^2 u^4 + Z u^2$ is nonzero, the map is well-defined: exactly one of $g(x_1)$ and $g(x_2)$ is a square in $\mathbb{F}_p$, so $\text{SWU}(u)$ is a genuine point of $E(\mathbb{F}_p)$ computed without any trial loop.*

Proof. Sketch. The defining relation $g(x_2) = Z^3 u^6 g(x_1)$ is a polynomial identity in u ; substituting $x_2 = Z u^2 x_1$ into g and simplifying using the formula for x_1 verifies it, and we take it as given. Now apply the multiplicativity of the Legendre symbol:

$$\left(\frac{g(x_2)}{p} \right) = \left(\frac{Z^3 u^6}{p} \right) \left(\frac{g(x_1)}{p} \right) = \left(\frac{Z}{p} \right) \left(\frac{Z^2 u^6}{p} \right) \left(\frac{g(x_1)}{p} \right) = \left(\frac{Z}{p} \right) \left(\frac{g(x_1)}{p} \right),$$

since $Z^2 u^6 = (Z u^3)^2$ is a nonzero square and contributes a Legendre symbol of $+1$ (assuming $u \neq 0$; the case $u = 0$ maps to a fixed point). Because Z is a *nonsquare*, $\left(\frac{Z}{p} \right) = -1$, hence $\left(\frac{g(x_2)}{p} \right) = -\left(\frac{g(x_1)}{p} \right)$. The two symbols are opposite, so precisely one of $g(x_1), g(x_2)$ is a nonzero square and admits a square root y ; the branch selection picks that one. Thus the output is always a valid affine point, on the first try, for every admissible u . $\square$

Remark 3.27 (From a field point to a uniform group point; full hash-to-curve). A single SWU evaluation is *not* surjective onto $E(\mathbb{F}_p)$ and its image is not uniform; one SWU output covers only part of the curve and with a non-flat distribution. The standardised `hash_to_curve( $m$ )` therefore composes several stages: (i) `hash_to_field( $m, 2$ )` yields two independent near-uniform field elements u_0, u_1 (§3.7); (ii) apply the map to each, $Q_0 = \text{SWU}(u_0)$, $Q_1 = \text{SWU}(u_1)$; (iii) add them on the curve, $R = Q_0 + Q_1$ —summing two independent images provably smooths the distribution to within negligible distance of uniform on $E(\mathbb{F}_p)$; and (iv) *clear the cofactor* by multiplying R by $h = \#E(\mathbb{F}_p)/r$ to land in the

prime-order subgroup of order r where the cryptography occurs. For curves with $a = 0$ (j -invariant 0), such as Pallas/Vesta, the simplified SWU map of Definition 3.25 does not apply directly—its x_1 formula needs $ab \neq 0$ —so one instead evaluates simplified SWU on an isogenous auxiliary curve $E' : y^2 = x^3 + a'x + b'$ with $a'b' \neq 0$ and then transports the resulting point back to E through the *isogeny map* $E' \rightarrow E$; the obstruction is intrinsic, for the j -invariant is an isomorphism invariant: a Weierstrass change of variables rescales (a, b) to (u^4a, u^6b) , so no model of these curves attains $ab \neq 0$, and the simplified-SWU route always passes through the isogeny. (Curves in Montgomery or twisted Edwards form are hashed by a different standardised map, Elligator 2, with no isogeny; prime-order curves such as Pallas and Vesta admit no such form, their order being odd while Montgomery and Edwards curves have order divisible by four.) The result is a hash $\{0, 1\}^* \rightarrow E(\mathbb{F}_p)$ that is constant-time, able to stand in for a random oracle into the group in security arguments (no efficient distinguisher can exploit its internal structure to tell the two apart), and the standard way to produce independent generators for Pedersen-style commitments.

Remark 3.28 (Nothing-up-my-sleeve generators). A recurring use of hash-to-curve is generating the public generators $G_1, \dots, G_k$ of a vector commitment so that *no one knows any discrete-log relation* $\sum_i a_i G_i = 0$ among them—knowledge of such a relation would let a committer open a commitment two ways, breaking binding. The defence is a *nothing-up-my-sleeve* (NUMS) construction: derive each $G_i = \text{hash_to_curve}(\tau \parallel i)$ from a fixed public domain tag τ (often a human-readable string such as a protocol name) and the index i . Because the generators are outputs of a function modelled as a random oracle, finding a discrete-log relation among them reduces to the discrete-log problem on E , which is hard; and because the seed is a transparent, auditable string with no hidden parameters, no party could have engineered a backdoor relation in advance. This is exactly how Halo 2 produces its Pedersen/inner-product commitment generators, and it is the culminating application that ties this section’s three threads—collision-resistant hashing, the random-oracle idealisation, and hashing to a curve—into the commitment machinery the next sections build.

3.9 Arithmetisation-friendly hashing: Poseidon

Every hash described above (SHA-2, BLAKE2) is *bit-oriented*: its round function shuffles and XORs 32- or 64-bit words. Inside an arithmetic circuit over a large prime field $\mathbb{F}_p$ — the setting of the proof systems of the *Halo 2 Guide* — such a function is prohibitively expensive, since field arithmetic and range checks must re-express every Boolean operation. *Arithmetisation-friendly* hashes follow the opposite design: the round function consists of a few low-degree *field* operations, incurring few constraints. *Poseidon* (Grassi, Khovratovich, Rechberger, Roy, and Schafneger, 2019) is the instance Orchard employs.

The sponge construction. Poseidon is a fixed permutation $f : \mathbb{F}_p^t \rightarrow \mathbb{F}_p^t$ on a state of t field elements, operated in *sponge* mode: the state splits into a *rate* of r elements (into which input is added and from which output is read) and a *capacity* of $c = t - r$ elements (never accessed directly — the security reserve). The sponge *absorbs* the message into the rate, r elements at a time with an application of f between blocks, after which it *squeezes* the output from the rate. Collision and preimage resistance hold up to approximately $\sqrt{p^c}$ work, as for a random sponge.

The permutation. f is a sequence of rounds, each comprising three steps: *add round constants* (a fixed vector in $\mathbb{F}_p^t$); a non-linear *S-box* $x \mapsto x^\alpha$ with $\alpha \geq 3$ the smallest integer satisfying $\gcd(\alpha, p-1) = 1$, so that the map is a bijection ($\alpha = 5$ over the Pasta fields, since $3 \mid p-1$ there); and a linear *mix* by a fixed maximum-distance-separable matrix $M \in \mathbb{F}_p^{t \times t}$. The cost-saving principle (the “HADES” strategy) is that only a few *full rounds* at the start and end need the full S-box — applied to *all* t elements; in the many *partial rounds* between them the S-box touches a *single* element, while the MDS mixing nonetheless diffuses it across the whole state. The round counts stand against the known algebraic (Gröbner-basis) attacks.

Circuit cost. The only non-linear operation is $x \mapsto x^5$, which an arithmetic circuit verifies with a couple of multiplication gates per S-box; the round-constant additions and the MDS multiplication are linear and almost free. A 2-to-1 Poseidon hash costs a few hundred constraints, against tens of thousands for a bit-oriented hash — the difference between a practical circuit and an impractical one.

The Orchard instance. Orchard fixes $t = 3$ (rate 2, capacity 1), $\alpha = 5$, and 8 full plus 56 partial rounds, over the Pallas base field, at the 128-bit security level. Orchard uses it in “constant-input-length” mode as a two-input hash

$$\text{PoseidonHash}(x, y) = f([x, y, 2^{65}])_1,$$

the first state element after one permutation of the input padded by the fixed capacity value 2^{65} (so no message padding is then required). Keyed by its first argument it is the pseudorandom function $F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$ from which Orchard constructs the nullifier (*Ironwood Guide*); unlike the Pedersen and Sinsemilla hashes constructed later (§7.5, §7.7), its security rests on cryptanalytic confidence in the round function rather than on a clean assumption such as discrete log.

4 Pseudorandom functions, block ciphers, and format-preserving encryption

The previous sections developed the machinery of one-way functions, collision-resistant hashing, and the abstract notion of computational indistinguishability. We now assemble these ideas into one of the most versatile tools in symmetric cryptography: the *pseudorandom function* (PRF). A PRF is a keyed family of functions that, to any efficient observer who does not know the key, behaves exactly like a function chosen uniformly at random. From this single primitive one obtains symmetric encryption, message authentication, key derivation, and, as we show at the end, encryption schemes that preserve the format of their input — the keyed bijection on a finite set that Orchard uses to derive the diversifiers behind its shielded addresses.

The development proceeds from first principles. We recall the model of computation and adversaries (§4.1); define pseudorandom functions and permutations and state the PRP/PRF switching lemma that licenses treating a permutation as a function (§4.2); explain the realisation of pseudorandom permutations by block ciphers, with AES as the running example (§4.3); construct PRFs directly from hash functions via keyed BLAKE2 (§4.4); and finally develop format-preserving encryption and the NIST FF1 mode as a Feistel network over a PRF (§4.5).

4.1 The computational model, recalled

$\{0, 1\}^\ell$ denotes the set of bit strings of length ℓ , $\{0, 1\}^*$ the set of all finite bit strings, and $|x|$ the length of a string x . For a finite set S , the notation $x \xleftarrow{\$} S$ means that x is drawn uniformly at random from S , independently of everything else. The adversarial model is that of Section 1: adversaries are probabilistic polynomial-time (PPT) algorithms in the security parameter λ (Definition 1.3), $\text{negl}(\lambda)$ denotes a negligible function (Definition 1.11), and every security notion below is a distinguishing game between two experiments, scored by the distinguishing advantage of Definition 1.18, written $\text{Adv}(\mathcal{A}) = |\Pr[\mathcal{A} \text{ outputs } 1 \text{ in } \text{Exp}_1] - \Pr[\mathcal{A} \text{ outputs } 1 \text{ in } \text{Exp}_0]|$.

The one device not yet formalised is *oracle access*. We write $\mathcal{A}^f$ for an adversary granted a black box for the function f : it may submit inputs x of its choice and receive $f(x)$ in a single step, but learns nothing about f beyond the answers to the queries it makes. The running time always bounds the number of oracle queries, which is hence polynomial.

4.2 Pseudorandom functions and permutations

We first make a “truly random function” precise, so that we can measure pseudorandomness against it.

Definition 4.1 (Random function). Let $\text{Func}(D, R)$ denote the set of all functions from a finite set D to a finite set R . A *random function* from D to R is an element $F \xleftarrow{\$} \text{Func}(D, R)$, chosen uniformly at random. Equivalently, the values $\{F(x)\}_{x \in D}$ are independent and uniformly distributed over R .

There are $|R|^{|D|}$ functions in $\text{Func}(D, R)$, an astronomically large set for the domains of interest (e.g. $D = R = \{0, 1\}^{128}$ gives $2^{128 \cdot 2^{128}}$ functions). One can neither store nor transmit the description of such a function. The essential property of a PRF is that it replaces this object by a short key while preserving its observable behaviour. A clean mental model is *lazy sampling*: an oracle for a random function maintains a table, initially empty; on a fresh query x it samples $F(x) \xleftarrow{\$} R$, records the pair, and returns it; on a repeated query it returns the recorded value. This produces exactly the distribution of Definition 4.1 while only ever materialising the queried points.

Definition 4.2 (Keyed function). A *keyed function* is a deterministic, efficiently computable map

$$F : \{0, 1\}^\lambda \times \{0, 1\}^{\ell(\lambda)} \longrightarrow \{0, 1\}^{m(\lambda)}, \quad (k, x) \longmapsto F_k(x),$$

where ℓ, m are polynomially bounded functions of the key length λ . Here λ is the key length, $\ell(\lambda)$ the input length, and $m(\lambda)$ the output length. Fixing the first argument to a key k yields a function $F_k : \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$.

Definition 4.3 (Pseudorandom function). A keyed function F (Definition 4.2) is a *pseudorandom function* (PRF) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{prf}}(\mathcal{A}) = \left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{R \xleftarrow{\$} \text{Func}(\ell(\lambda), m(\lambda))} [\mathcal{A}^R(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Func}(\ell, m) = \text{Func}(\{0, 1\}^\ell, \{0, 1\}^m)$, and the input 1^λ (the security parameter in unary) merely conveys to $\mathcal{A}$ the parameter size.

Stated informally: an adversary handed a black box that is either F_k for a secret random key, or a freshly sampled truly random function, and allowed to query it adaptively, cannot determine which it has. Two features merit emphasis. First, the adversary obtains *oracle* access only — it never sees the key, and it cannot read the code of the function, only its input–output behaviour. Second, the comparison object is the gigantic, unstorable random function; the PRF compresses this into λ bits while remaining computationally indistinguishable from it. Information theory cannot afford this trade: a keyed function takes at most 2^λ values as k ranges over the keys, a vanishing fraction of all $|R|^{|D|}$ functions, so an *unbounded* adversary could in principle distinguish. Pseudorandomness is an inherently computational notion.

Remark 4.4 (Necessity of oracle access and of adaptivity). The adversary chooses its queries; it need not commit to them in advance and may let later queries depend on earlier answers. This *adaptive* access is the strongest reasonable model and the one that downstream constructions rely on. A weaker “non-adaptive” definition, in which the adversary fixes all queries before seeing any answer, is genuinely weaker and insufficient for many applications.

A block cipher is not merely a function but a *permutation* for each key — it must be invertible so that decryption is possible. The relevant ideal object is therefore a random *permutation*.

Definition 4.5 (Keyed permutation). A keyed function $F : \{0, 1\}^\lambda \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ (with equal input and output length) is a *keyed permutation* if for every key k the map $F_k : \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ is a bijection, and one can efficiently compute both F_k and its inverse F_k^{-1} given k . $\text{Perm}(\ell)$ denotes the set of all permutations of $\{0, 1\}^\ell$, of which there are $2^\ell!$.

Definition 4.6 (Pseudorandom permutation). A keyed permutation F is a *pseudorandom permutation* (PRP) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{prp}}(\mathcal{A}) = \left| \Pr_k [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^P(1^\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

If in addition no PPT adversary can distinguish even when given oracle access to *both* the permu-

tation and its inverse,

$$\text{Adv}_F^{\text{sprp}}(\mathcal{A}) = \left| \Pr_k[\mathcal{A}^{F_k, F_k^{-1}}(1^\lambda) = 1] - \Pr_{P \leftarrow \text{Perm}(\ell)}[\mathcal{A}^{P, P^{-1}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

then F is a *strong* pseudorandom permutation (SPRP).

The distinction between a PRP and an SPRP is the availability of a decryption oracle. A block cipher used for encryption where the attacker can also obtain decryptions of chosen ciphertexts must be an SPRP; this is the natural security target for a block cipher.

Example 4.7 (A random permutation is almost a random function). Consider an adversary that queries a length-preserving oracle on q distinct inputs. If the oracle is a random function $R \in \text{Func}(\ell, \ell)$, the answers are q independent uniform strings, and collisions (two queries with equal answers) occur. If the oracle is a random permutation $P \in \text{Perm}(\ell)$, the answers are q *distinct* uniform strings, and no collision can occur. This is the *only* statistical difference between the two ideal objects from the outside, and it is the entire content of the PRP/PRF switching lemma.

Lemma 4.8 (PRP/PRF switching lemma). *Let $\ell \geq 1$ and let $\mathcal{A}$ be any adversary — even a computationally unbounded one — making at most q queries to its oracle. Then*

$$\left| \Pr_{P \leftarrow \text{Perm}(\ell)}[\mathcal{A}^P = 1] - \Pr_{R \leftarrow \text{Func}(\ell, \ell)}[\mathcal{A}^R = 1] \right| \leq \frac{q(q-1)}{2^{\ell+1}}.$$

We use the lemma as a statement only; its proof is a standard game-hopping argument bounding the probability of the single distinguishing event of Example 4.7, an output collision. The consequence is that a secure PRP on a large domain is also a secure PRF whenever the number of queries stays well below the birthday bound $2^{\ell/2}$: for AES, $q^2/2^{129}$ is negligible for any realistic query budget. This is the licence by which the FF1 construction of §4.5 treats AES — a permutation — as the PRF its round function requires.

4.3 Block ciphers and AES

Definition 4.9 (Block cipher). A *block cipher* with block length b and key length κ is a keyed permutation

$$E : \{0, 1\}^\kappa \times \{0, 1\}^b \longrightarrow \{0, 1\}^b,$$

together with its inverse $D = E^{-1}$ satisfying $D_k(E_k(x)) = x$ for all k, x . Here E_k is *encryption* and D_k is *decryption* under key k . The security goal is that E be a strong pseudorandom permutation (Definition 4.6).

A block cipher is thus a concrete, fast, fixed-parameter instantiation of the SPRP abstraction. Whereas the PRP definition is asymptotic (a family indexed by λ), a real block cipher fixes b and κ once and for all (e.g. $b = 128$, $\kappa \in \{128, 192, 256\}$ for AES) and offers *concrete security*: an explicit function of the adversary's resources bounds the advantage of the best known attacks. We state pseudorandomness for block ciphers in concrete terms.

Definition 4.10 (Concrete (t, q, ε) -security). A block cipher E is a (t, q, ε) -*secure* PRP if every adversary running in time at most t and making at most q oracle queries has $\text{Adv}_E^{\text{prp}}(\mathcal{A}) \leq \varepsilon$; analogously for SPRP and PRF security.

The Advanced Encryption Standard. AES (the Rijndael cipher of Daemen and Rijmen, standardised in FIPS 197) is the canonical block cipher. It has block length $b = 128$ bits and three key lengths $\kappa \in \{128, 192, 256\}$, with $N_r \in \{10, 12, 14\}$ rounds respectively. Structurally it is a *substitution-permutation network* (SPN): the 128-bit state, viewed as a 4×4 array of bytes (elements of the field $\mathbb{F}_{2^8}$), passes through N_r rounds, each composing a nonlinear byte substitution (**SubBytes**, built on field inversion $a \mapsto a^{-1}$, chosen for its resistance to differential and linear cryptanalysis), a byte transposition (**ShiftRows**), an invertible column-wise linear mix (**MixColumns**), and the XOR of a 128-bit round key (**AddRoundKey**) that a key schedule expands from k . The two permutation layers together guarantee that after two rounds every output byte depends on every input byte. Each layer is individually invertible, so E_k is a permutation as Definition 4.9 requires, and decryption applies the inverses in reverse order.

Remark 4.11 (Confusion and diffusion). The SPN design is the modern embodiment of Shannon’s two principles. The nonlinear S-box supplies *confusion* — making the relationship between key and ciphertext as complex as possible. ShiftRows and MixColumns supply *diffusion* — spreading the influence of each input bit over the whole output. Iterating these for ten or more rounds is what gives AES its empirical resistance to all known attacks. No proof that AES is a PRP exists; like every practical block cipher, its security is a *conjecture* corroborated by decades of failed cryptanalysis. The theorems below therefore take “ E is a (P)PRP/SPRP” as an assumption and reason from it.

4.4 PRFs from hash functions: length extension and keyed BLAKE2

Block ciphers are one source of PRFs; cryptographic hash functions are another, often more convenient one (hash functions are unkeyed and ubiquitous). The naive approach — keying a hash function H by prepending the key, $F_k(m) = H(k \parallel m)$ — is unsafe for the iterated “Merkle–Damgård” hashes (MD5, SHA-1, SHA-256) because of the *length-extension* attack: from $H(k \parallel m)$ and $|m|$ one can compute $H(k \parallel m \parallel \text{pad} \parallel m')$ for attacker-chosen m' *without knowing* k , since the hash output is exactly the internal chaining state after absorbing $k \parallel m$.

The classical repair is *HMAC*, the nested double call $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$ for two fixed pad constants: the inner hash absorbs the message under one key-derived value, and the outer hash re-keys the fixed-length result, so the emitted digest is no longer a resumable chaining state. Bellare’s analysis shows that HMAC is a PRF whenever the underlying compression function is, up to a birthday term $q^2/2^\ell$; as a PRF it is in particular a secure message authentication code, and it is the building block of the standardised key-derivation function HKDF (§5.6). We need no more than this summary, because the shielded protocol uses HMAC nowhere: its hash of choice admits direct keying.

Keyed BLAKE2. Modern hash functions of the *sponge* or *HAIFA* families do not suffer length extension and so admit a far simpler keying. BLAKE2 (and its successor BLAKE3) is built so that it can be keyed natively: the construction loads the key as a parameter and prepends it as a padded first block, and the function’s compression mixes a counter and finalisation flags that make the output depend on the whole input in a way that does not expose the internal state. One simply writes

$$F_k(m) = \text{BLAKE2}(m; \text{key} = k),$$

and obtains a PRF directly — no nested double call is needed. No length-extension attack remains to repair, because the finalisation step (the flag XORed in for the last block) means the emitted digest is *not* the raw chaining value and cannot be used to resume the computation.

Remark 4.12 (Zcash’s use of keyed/personalised BLAKE2). The Zcash protocol uses BLAKE2b and BLAKE2s pervasively, exploiting two parameters: the *key* (for PRF/MAC use) and the *personalisation* string, a domain-separation tag baked into the initial state (16 bytes for BLAKE2b, 8 bytes for BLAKE2s). Personalisation ensures that the same input hashed for two different purposes (say, deriving a nullifier versus deriving a note commitment) yields independent-looking outputs, so each use

site is effectively an independent PRF. This is the practical realisation of the “independent random oracle per context” idiom and is cheaper and cleaner than HMAC because BLAKE2’s design already provides keying and domain separation as first-class features.

4.5 Format-preserving encryption and FF1

All constructions so far operate on bit strings of a length the cipher’s block size dictates. Many applications, however, must encrypt data drawn from a peculiar finite domain — a 16-digit credit-card number, a 9-digit national identifier, a license plate — and require the ciphertext to lie in the *same* domain, so that legacy databases, field-length constraints, and checksum formats continue to accept it. This is the problem that *format-preserving encryption* (FPE) solves.

Definition 4.13 (Format-preserving encryption). Let $\mathcal{X}$ be a finite set (the *format*, e.g. $\mathcal{X} = \{0, 1, \dots, 9\}^{16}$, the set of 16-digit strings). A *format-preserving encryption scheme* on $\mathcal{X}$ is a keyed permutation $\mathcal{E} : \{0, 1\}^k \times \mathcal{X} \rightarrow \mathcal{X}$ such that for every key k , $\mathcal{E}_k : \mathcal{X} \rightarrow \mathcal{X}$ is a bijection with efficiently computable inverse $\mathcal{D}_k$. Security requires $\mathcal{E}$ to be a (strong) pseudorandom permutation on $\mathcal{X}$ (Definition 4.6, with $\text{Perm}(\mathcal{X})$ in place of $\text{Perm}(\{0, 1\}^\ell)$): no efficient adversary distinguishes $\mathcal{E}_k$ from a uniformly random permutation of $\mathcal{X}$, even with access to the inverse.

The defining feature is that the domain $\mathcal{X}$ is *arbitrary* — in particular $|\mathcal{X}|$ need not be a power of two, so a plain block cipher does not apply. What we require is a keyed pseudorandom *bijection* on a set whose size is, say, 10^{16} . The deterministic nature is essential and distinguishes FPE from randomised block-cipher modes: $\mathcal{X}$ has no room to store a random IV, so FPE is the deterministic, length-preserving (indeed format-preserving) analogue of a block cipher on the exotic alphabet of $\mathcal{X}$. It necessarily leaks equality of plaintexts, as any deterministic cipher must, but this is the unavoidable cost of preserving format with no expansion, and it is acceptable for the tokenisation use cases FPE targets.

Reduction to integers. Any format $\mathcal{X}$ of the form “strings of length ν over an alphabet of radix ρ ” is in bijection with the integer interval $\{0, 1, \dots, \rho^\nu - 1\}$ by reading the string as a base- ρ numeral. It therefore suffices to build a keyed pseudorandom permutation of $\mathbb{Z}/M\mathbb{Z} = \{0, \dots, M - 1\}$ for an arbitrary modulus M (here $M = \rho^\nu$). FF1 splits $M = a \cdot b$ into two factors of nearly equal “size” and runs a Feistel network whose two halves live in $\mathbb{Z}/a\mathbb{Z}$ and $\mathbb{Z}/b\mathbb{Z}$.

The Feistel network. A Feistel network turns a (not necessarily invertible) round function into a permutation. We present the classical bitwise case first, then the modular generalisation that FF1 uses.

Definition 4.14 (Balanced Feistel network). Let $F_1, \dots, F_r : \{0, 1\}^w \rightarrow \{0, 1\}^w$ be arbitrary *round functions*. The r -round Feistel network $\Psi[F_1, \dots, F_r]$ is the permutation of $\{0, 1\}^{2w}$ defined as follows. Write the input as (L_0, R_0) with $L_0, R_0 \in \{0, 1\}^w$. For $i = 1, \dots, r$ set

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}),$$

and output (L_r, R_r) .

Proposition 4.15 (Feistel is a bijection regardless of the round functions). *For any choice of round functions $F_1, \dots, F_r$ (which need not be injective), $\Psi[F_1, \dots, F_r]$ is a bijection of $\{0, 1\}^{2w}$, and one com-*

puts its inverse by running the rounds backwards:

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_i(L_i), \quad i = r, \dots, 1.$$

Proof. It suffices to show each round is invertible; the composition of bijections is a bijection. The i -th round map sends $(L_{i-1}, R_{i-1}) \mapsto (R_{i-1}, L_{i-1} \oplus F_i(R_{i-1}))$. Given the output (L_i, R_i) , one recovers $R_{i-1} = L_i$ immediately, and then $L_{i-1} = R_i \oplus F_i(R_{i-1}) = R_i \oplus F_i(L_i)$, using that XOR is its own inverse. The recovery uses only forward evaluations of F_i , so it *never requires an inverse of the round function* — this is the crucial point that permits building a permutation out of a one-way-ish PRF. The displayed backward recursion is exactly this inversion applied round by round. $\square$

This proposition is the structural heart of FPE: it manufactures a *keyed bijection* from a keyed function that is not itself a bijection. The round functions derive from a PRF (in FF1, AES iterated in cipher-block-chaining fashion; the FF1 description below gives the construction), so the network is invertible without ever inverting the PRF — the construction only ever evaluates AES in the forward direction.

The Luby–Rackoff theorem explains why enough Feistel rounds over a PRF yield a pseudorandom permutation, justifying the construction’s security.

Theorem 4.16 (Luby–Rackoff). *If the round functions $F_1, \dots, F_r$ are independent PRFs on $\{0, 1\}^w$, then the Feistel network Ψ is, against an adversary making q queries:*

- with $r = 3$ rounds, a pseudorandom permutation (CPA-secure: forward queries only), and
- with $r = 4$ rounds, a strong pseudorandom permutation (CCA-secure: forward and inverse queries),

with distinguishing advantage $O(q^2/2^w)$ above the PRF advantage of the round functions.

Proof. Sketch. Replace each PRF round function by a truly random function, at the cost of r PRF terms. For the three-round case one shows that, unless two of the at most q queries induce a collision in the intermediate Feistel values (a birthday event of probability $O(q^2/2^w)$), the outputs of the network are distributed exactly as those of a random permutation: the middle round’s random function, evaluated at inputs that are distinct with high probability, produces independent uniform values that perfectly mask the relationship between input and output halves. The fourth round adds the symmetry needed to also resist inverse queries, upgrading PRP to SPRP. The full argument is a careful coupling/transcript analysis bounding the probability of any internal collision; the leftover term is the stated birthday bound. $\square$

Remark 4.17 (FF1’s use of ten rounds rather than four). Luby–Rackoff gives security only up to $q \approx 2^{w/2}$ queries, and for small domains (the FPE setting, where w is tiny — consider one half of a 16-digit number) the birthday bound $q^2/2^w$ is weak. FF1 therefore uses *ten* rounds rather than the minimal four, providing a comfortable security margin against the more powerful attacks known for small-domain Feistel (which can sometimes beat the generic birthday bound). The number of rounds buys security; it does not affect invertibility, which Proposition 4.15 guarantees for any round count.

The FF1 mode (NIST SP 800-38G). FF1 instantiates an *unbalanced, modular* Feistel network of ten rounds to build a PRP on $\mathbb{Z}/M\mathbb{Z}$ with $M = \rho^\nu$, the radix- ρ numeral interpretation of a length- ν string. It supports an associated *tweak* T — a non-secret string that selects, in effect, one permutation from a family indexed by T , so that the same key encrypts the same plaintext to different ciphertexts under different tweaks (useful for binding a token to a context). The following describes its operation.

1. **Split.** Set $u = \lfloor \nu/2 \rfloor$, $v = \nu - u$. Split the plaintext numeral string X into a left part A of u symbols and a right part B of v symbols, with integer values in $\{0, \dots, \rho^u - 1\}$ and $\{0, \dots, \rho^v - 1\}$ respectively. (Unbalanced when ν is odd; the two halves live in moduli $a = \rho^u$ and $b = \rho^v$.)

2. **Round function from a PRF.** For round $i = 0, 1, \dots, 9$, assemble a byte string Q encoding the tweak T , the round index i , and the current right half B , prepend a fixed parameter block P (encoding the radix, lengths, and tweak length), and apply $\text{PRF}(P \parallel Q)$, where PRF is AES in CBC-MAC mode: set $Y_0 = 0^{128}$, iterate $Y_j = \text{AES}_k(Y_{j-1} \oplus X_j)$ over the 16-byte blocks X_j of $P \parallel Q$, and output the final block Y_m (SP 800-38G, Algorithm 6). The iteration chains AES through an evolving state exactly as the Merkle–Damgård iteration of §4.4 chains a compression function; emitting the final state is safe here because all inputs share one fixed length, so no length-extension query arises. Further AES calls then expand the result to a long enough digest, which we read as an integer y .

3. **Modular Feistel step.** Update the halves by the modular analogue of Definition 4.14:

$$C = (A + y) \bmod m, \quad A \leftarrow B, \quad B \leftarrow C,$$

where the modulus m alternates between $a = \rho^u$ and $b = \rho^v$ according to the parity of the round (so that the addition lands in the correct half’s domain). Here modular addition replaces XOR.

4. **Output.** After ten rounds, concatenate the final A and B and render back to a length- ν radix- ρ string.

Decryption runs the rounds in reverse, replacing modular addition by modular subtraction $A \leftarrow (C - y) \bmod m$, exactly mirroring the backward recursion of Proposition 4.15.

Proposition 4.18 (FF1 is a keyed bijection on $\mathcal{X}$). *For every key k and tweak T , the FF1 map $\mathcal{E}_{k,T} : \mathcal{X} \rightarrow \mathcal{X}$ is a bijection of the format set $\mathcal{X}$, and $\mathcal{D}_{k,T}$ is its two-sided inverse.*

Proof. Each round is the map $(A, B) \mapsto (B, (A + y) \bmod m)$, where $y = y(i, T, B)$ depends only on the round index, the tweak, and the right half B — not on A . Fix the inputs to the round function (i.e. fix B, i, T); then y is a fixed constant and the map $A \mapsto (A + y) \bmod m$ is a bijection of $\mathbb{Z}/m\mathbb{Z}$ (translation by a constant in a cyclic group is a bijection, since gcd-free addition is invertible by subtracting the same constant). The full round is thus invertible: from (B, C) recover B directly as the new left value’s source, recompute $y = y(i, T, B)$ by a *forward* PRF evaluation, and set $A = (C - y) \bmod m$. Each even-indexed round is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ onto $\mathbb{Z}/b\mathbb{Z} \times \mathbb{Z}/a\mathbb{Z}$ and each odd-indexed round a bijection back onto $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ (the two products coincide when ν is even); the composition of the ten rounds, an even number, is therefore a bijection of $\mathcal{X}' = \mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$. The numeral map $(A, B) \mapsto Ab + B$ identifies $\mathcal{X}'$ with $\mathbb{Z}/M\mathbb{Z}$ as a bijection of sets (not of groups: $\gcd(a, b) = \rho^{\min(u,v)} > 1$, so the Chinese remainder theorem does not apply; only bijectivity is used), and $\mathcal{E}_{k,T}$ is a bijection of $\mathbb{Z}/M\mathbb{Z}$. Transporting through the numeral bijection $\mathcal{X} \cong \mathbb{Z}/M\mathbb{Z}$ (which is itself a bijection), $\mathcal{E}_{k,T}$ is a bijection of $\mathcal{X}$. Running the rounds backward with subtraction inverts it on both sides, so $\mathcal{D}_{k,T} = \mathcal{E}_{k,T}^{-1}$. $\square$

Remark 4.19 (“Exactly a keyed bijection”, and the locus of security). Two points complete the picture. First, FF1 is a keyed bijection by *structure*: invertibility comes for free from the Feistel construction (Proposition 4.15 and 4.18) and holds for *any* round function whatsoever, including a broken or constant one. The PRF (AES) is responsible not for invertibility but for *pseudorandomness* — making $\mathcal{E}_{k,T}$ look like a uniformly random permutation of $\mathcal{X}$ (Definition 4.13), per the Luby–Rackoff heuristic of Theorem 4.16 adapted to the modular, multi-round, tweaked setting. This clean separation — bijectivity from algebra, security from the PRF — is precisely why Feistel is the standard route to FPE.

Second, the domain-size caveat is real: the published SP 800-38G requires $\rho^\nu \geq 100$ and recommends $\rho^\nu \geq 10^6$ (its draft Revision 1 would make 10^6 mandatory), because for very small domains an attacker can recover or distinguish the keyed permutation by exhaustively cataloguing input/output pairs, and known message-recovery attacks on small-domain FPE (Bellare–Hoang–Tessaro and successors) erode the security margin. For the moderately large domains FPE is meant for (card numbers,

identifiers), and with ten rounds, FF1 is the NIST-standardised, AES-backed realisation of a keyed pseudorandom bijection on an arbitrary finite format. Its single use in Orchard is the derivation of *diversifiers*: FF1-AES256 maps an address index to the 11-byte diversifier embedded in a shielded address, so that one spending key yields many unlinkable addresses (*Ironwood Guide*).

5 Symmetric encryption, AEAD, and key derivation

This section develops symmetric-key cryptography from its information-theoretic baseline to the concrete authenticated-encryption scheme used throughout the Zcash protocol. We first recall the one-time pad and the eavesdropper (EAV) security notion that relaxes its perfect secrecy to computational adversaries. We then instantiate the resulting pseudo-one-time pad by the nonce-based stream cipher ChaCha20 and formalise chosen-plaintext security. Encryption alone guarantees confidentiality but not integrity; we therefore develop message authentication codes, the universal-hash MAC Poly1305, and the notion of authenticated encryption with associated data (AEAD), culminating in the ChaCha20-Poly1305 construction. The section concludes with key-derivation functions, which manufacture uniform keys from non-uniform secrets such as Diffie–Hellman shared values: we state the general notion, note the standardised HKDF in passing, and present the single personalised BLAKE2b call by which Zcash actually derives its note-encryption keys.

Throughout, an *adversary* is a (possibly probabilistic) algorithm $\mathcal{A}$, and $y \leftarrow \mathcal{A}(x)$ denotes running $\mathcal{A}$ on input x and assigning its output to y . For a finite set S , $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from S . Every adversary in our computational definitions is *probabilistic polynomial time* (PPT) in a security parameter λ , and $\text{negl}(\lambda)$ denotes a negligible function (Definition 1.11).

The one-time pad and EAV security. The information-theoretic baseline is the *one-time pad*: to encrypt $m \in \{0, 1\}^L$ under a uniformly random key $k \in \{0, 1\}^L$ used exactly once, output $c = m \oplus k$. Since $m \oplus k$ is uniform whatever m is, the ciphertext distribution is independent of the message, and even an unbounded eavesdropper learns nothing—Shannon’s *perfect secrecy*. The price is severe: the key must be as long as the message and must never be reused, for two ciphertexts under one pad reveal $c \oplus c' = m \oplus m'$, the *two-time-pad* failure. The computational relaxation is *EAV security* (security against an eavesdropper): an adversary who chooses two equal-length messages m_0, m_1 and sees a single ciphertext $\text{Enc}_k(m_b)$ for a hidden uniform bit b guesses b with advantage at most $\text{negl}(\lambda)$. Replacing the truly random pad by the output of a pseudorandom generator stretched from a short seed—the *pseudo-one-time pad*—achieves EAV security with a fixed-length key, at the cost of resting on a pseudorandomness assumption.

5.1 Stream ciphers and ChaCha20

A *stream cipher* is the practical instantiation of the pseudo-one-time pad: it is a keyed, seekable PRG producing a *keystream* that XORs into the plaintext. To support encrypting many messages under one key without violating the one-time-pad no-reuse rule, modern stream ciphers take an additional public input called a *nonce* (number used once) and behave like a pseudorandom function of the nonce.

Definition 5.1 (Nonce-based stream cipher). A *nonce-based stream cipher* is a deterministic function $\text{KS} : \mathcal{K} \times \mathcal{N} \times \mathbb{N} \rightarrow \{0, 1\}^*$ producing, for key k , nonce ν , and length L , a keystream $\text{KS}(k, \nu, L) \in \{0, 1\}^L$. Encryption of $m \in \{0, 1\}^L$ is $\text{Enc}_k(\nu, m) = m \oplus \text{KS}(k, \nu, |m|)$ with ciphertext (ν, c) ; decryption recomputes the keystream and XORs. The security goal is that for distinct nonces the keystreams are jointly indistinguishable from independent uniform strings, provided no nonce repeats under a fixed key.

ChaCha20, designed by Bernstein, is the stream cipher that Zcash uses (in the note-encryption layer) and that RFC 8439 standardises. Its core is the *ChaCha quarter-round*, a reversible mixing of four 32-bit words built only from modular addition, XOR, and rotation (an “ARX” design), which avoids data-dependent table lookups and hence timing side channels.

Definition 5.2 (ChaCha state and quarter-round). The ChaCha20 state is a 4×4 matrix of 32-bit words, viewed as a vector $(x_0, \dots, x_{15}) \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}$. Arithmetic $+$ is addition modulo 2^{32} , $\oplus$ is XOR, and $x \lll r$ is left rotation by r bits. The *quarter-round* $QR(a, b, c, d)$ updates four words by

$$\begin{aligned} a &+= b; & d &\oplus= a; & d &\lll= 16; \\ c &+= d; & b &\oplus= c; & b &\lll= 12; \\ a &+= b; & d &\oplus= a; & d &\lll= 8; \\ c &+= d; & b &\oplus= c; & b &\lll= 7. \end{aligned}$$

The cipher initialises the state from the 128-bit constant “expand 32-byte k” (words $x_0 - x_3$), a 256-bit key ($x_4 - x_{11}$), a 32-bit block counter (x_{12}), and a 96-bit nonce ($x_{13} - x_{15}$). One *double round* applies QR to the four columns and then to the four diagonals; the 20 rounds of ChaCha20 are ten double rounds. Writing X for the initial state and $R(X)$ for the state after the rounds, the 64-byte keystream block is the *feed-forward* sum $R(X) + X$ (word-wise, modulo 2^{32}), serialised in little-endian order.

Remark 5.3 (Feed-forward and counter mode). The rounds R form a bijection (each quarter-round is invertible), so without the final addition $R(X) + X$ the block function would be a permutation and thus let the adversary invert it trivially; the feed-forward sum destroys invertibility and is the standard “Davies–Meyer”-style construction for turning a permutation into a one-way mixing function. Incrementing the block counter x_{12} produces successive 64-byte keystream blocks, so $KS(k, \nu, L)$ is the concatenation of blocks for counters $0, 1, 2, \dots$; this *counter mode* is what makes the cipher seekable and parallelisable. We conjecture ChaCha20 to be a secure PRF of $(\nu, \text{counter})$: no attack better than brute force over the 256-bit key is known. Reusing a (k, ν) pair reproduces the same keystream and re-creates the two-time-pad failure; nonce uniqueness is a hard requirement.

5.2 Chosen-plaintext security

EAV-security models only an adversary who sees a single ciphertext. A realistic adversary may influence which messages get encrypted and observe many ciphertexts. The corresponding notion is *chosen-plaintext attack* (CPA) security, which we model by granting the adversary oracle access to the encryption algorithm.

Definition 5.4 (CPA security). For scheme Π , adversary $\mathcal{A}$, and $b \in \{0, 1\}$, the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda, b)$ is: sample $k \leftarrow \text{Gen}(1^\lambda)$; give $\mathcal{A}$ oracle access to $\text{Enc}_k(\cdot)$; at some point $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal length and receives $c^* \leftarrow \text{Enc}_k(m_b)$; $\mathcal{A}$ may continue querying the oracle and finally outputs a bit b' , which is the output. Π is *CPA-secure* if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda) := |\Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda, 0) = 1] - \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda).$$

Proposition 5.5 (Determinism precludes CPA security). *No scheme with a deterministic, stateless Enc is CPA-secure.*

Proof. The adversary queries the oracle on two distinct messages $m_0 \neq m_1$, recording $c_0 = \text{Enc}_k(m_0)$ and $c_1 = \text{Enc}_k(m_1)$. It submits the challenge (m_0, m_1) , receives $c^* = \text{Enc}_k(m_b)$, and outputs 0 if $c^* = c_0$ and 1 otherwise. Since Enc is deterministic, $c^* = c_b$ always, so $\mathcal{A}$ is always correct and its advantage is 1. $\square$

Consequently, CPA-secure schemes must be randomised or nonce-based. A nonce-based stream cipher that uses a fresh (e.g. random or counter) nonce per encryption attains CPA security under the

PRF conjecture for the keystream generator, because each encryption is effectively a fresh one-time pad. We record here the security notion toward which the development proceeds; we defer the routine PRF-based proof to the AEAD analysis below, where we treat confidentiality and integrity together.

Remark 5.6 (Semantic security). Goldwasser and Micali’s original notion, *semantic security*, demands that whatever an efficient adversary can compute about the plaintext from the ciphertext, it could compute without the ciphertext (from the message length alone). Semantic security is provably equivalent to the indistinguishability notions above; we use indistinguishability throughout because it is far easier to manipulate in reductions. The content is the same: the ciphertext is computationally useless for learning anything about the message beyond its length.

5.3 Message authentication codes

CPA security protects *confidentiality* but says nothing about *integrity*: an adversary who tampers with a ciphertext may produce a different valid plaintext without detection. Moreover, for the malleable XOR-based stream ciphers above, flipping a bit of the ciphertext flips the corresponding plaintext bit. To detect tampering, we attach a *message authentication code*.

Definition 5.7 (Message authentication code). A *message authentication code* (MAC) is a pair $(\text{Mac}, \text{Vrfy})$ with key space $\mathcal{K}$: $\text{Mac}_k(m)$ outputs a tag t , and $\text{Vrfy}_k(m, t) \in \{0, 1\}$ accepts (1) or rejects (0). Correctness requires $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$ for all k, m . When Mac is deterministic, canonical verification recomputes the tag and checks equality.

Definition 5.8 (Existential unforgeability, EUF-CMA). Consider the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{forge}}(\lambda)$: sample $k \leftarrow \text{Gen}(1^\lambda)$; give $\mathcal{A}$ oracle access to $\text{Mac}_k(\cdot)$; let Q be the set of messages $\mathcal{A}$ queries; $\mathcal{A}$ outputs a pair (m^*, t^*) and wins iff $\text{Vrfy}_k(m^*, t^*) = 1$ and $m^* \notin Q$. The MAC is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{forge}}(\lambda) := \Pr[\mathcal{A} \text{ wins}] \leq \text{negl}(\lambda).$$

A *strongly unforgeable* (sUF-CMA) MAC additionally forbids producing a new valid tag for an already-queried message: $\mathcal{A}$ wins on (m^*, t^*) as long as the oracle never returned that exact pair; write $\text{Adv}_{\Pi, \mathcal{A}}^{\text{s-forge}}$ for the advantage in this variant.

The MAC that Zcash uses, Poly1305, is not a pseudorandom-function MAC but a *one-time, universal-hash* MAC, which provides information-theoretic security per nonce and is extremely fast. The key abstraction is a universal hash family.

Definition 5.9 (ε -almost- Δ -universal hash family). Let $(\mathcal{G}, +)$ be a finite abelian group. A family of functions $\{h_r : \mathcal{X} \rightarrow \mathcal{G}\}_{r \in \mathcal{R}}$ is ε -almost- Δ -universal (ε -A Δ U) if for all distinct $x, x' \in \mathcal{X}$ and every $\delta \in \mathcal{G}$,

$$\Pr_{r \xleftarrow{\$} \mathcal{R}} [h_r(x) - h_r(x') = \delta] \leq \varepsilon.$$

Taking $\delta = 0$ recovers ordinary ε -universality (low collision probability); the Δ -universality condition controls additive differences, which is what renders the one-time MAC below secure. With $\mathcal{G} = \{0, 1\}^n$ under XOR this is the classical almost-XOR-universality; Poly1305 below takes $\mathcal{G} = \mathbb{Z}/2^{128}\mathbb{Z}$, addition modulo 2^{128} .

The Carter–Wegman paradigm turns any A Δ U family into a one-time MAC by masking the hash with a fresh pad.

Theorem 5.10 (Carter–Wegman one-time MAC). *Let $\{h_r\}$ be ε -A Δ U into $(\mathcal{G}, +)$. Define a MAC with key (r, s) , where $r \xleftarrow{\$} \mathcal{R}$ and the pad $s \xleftarrow{\$} \mathcal{G}$, by $\text{Mac}_{r,s}(m) = h_r(m) + s$. If (r, s) authenticates at most one message, then any (even computationally unbounded) adversary who sees one message–tag pair (m, t) forges a valid pair (m^*, t^*) with $m^* \neq m$ with probability at most ε .*

Proof. The adversary observes $t = h_r(m) + s$ and must output (m^*, t^*) with $m^* \neq m$ and $t^* = h_r(m^*) + s$. Subtracting the two tag equations eliminates the pad:

$$t^* - t = h_r(m^*) - h_r(m).$$

Thus the forgery succeeds iff $h_r(m^*) - h_r(m) = t^* - t$. Condition on the adversary’s view. The pad s is uniform and independent of r ; given the single observed tag t , every value of $h_r(m)$ is equally consistent (each determines a unique s), so the observation t leaks nothing about r , and the posterior on r remains uniform on $\mathcal{R}$. The adversary fixes its chosen $\delta := t^* - t$ and the distinct pair (m, m^*) independently of r ; hence by the A Δ U property

$$\Pr[h_r(m^*) - h_r(m) = \delta] \leq \varepsilon. \quad \square$$

Poly1305 instantiates this with a polynomial-evaluation hash over the prime field $\mathbb{F}_p$ for $p = 2^{130} - 5$.

Definition 5.11 (Poly1305). Fix the prime $p = 2^{130} - 5$ and work in $\mathbb{F}_p$. The key is a pair (r, s) : r is a 128-bit value subjected to a fixed bit-clamping (certain bits are forced to zero) and s is a 128-bit pad. To authenticate a message, split it into ℓ blocks $c_1, \dots, c_\ell$ of at most 16 bytes; encode each block as an integer in $[0, 2^{128})$ and add 2^{8b} where b is the block’s byte length (this appends a leading 1 bit, making the encoding prefix-free). Interpreting r and each c_i as elements of $\mathbb{F}_p$, the hash is the polynomial

$$h_r(m) = \sum_{i=1}^{\ell} c_i r^{\ell-i+1} \in \mathbb{F}_p,$$

i.e. Horner evaluation $(\dots((c_1 r + c_2)r + c_3)r \dots)r$. The tag is

$$\text{Mac}_{r,s}(m) = ((h_r(m) \bmod p) + s) \bmod 2^{128},$$

the field hash reduced and added to the pad modulo 2^{128} .

Proposition 5.12 (Poly1305 is almost- Δ -universal). *Restricted to messages of at most L bytes (hence at most $\ell_{\max} = \lfloor L/16 \rfloor$ blocks), the reduced Poly1305 hash family $\{m \mapsto h_r(m) \bmod p\}$, read into $\mathbb{Z}/2^{128}\mathbb{Z}$, is ε -almost- Δ -universal with respect to addition modulo 2^{128} , with $\varepsilon \leq 8\ell_{\max}/2^{106}$; the loss from the ideal $\ell_{\max} \cdot p^{-1}$ accounts for the 128-bit clamping of r (which restricts r to a subset of size 2^{106}) and the final reduction modulo 2^{128} .*

Proof. Sketch. Fix two distinct messages m, m' with block encodings (c_i) and (c'_i) , padded to a common length ℓ by leading zero-coefficients without changing their values (the prefix-free encoding guarantees that $m \neq m'$ yields distinct coefficient vectors). The difference of hashes is

$$h_r(m) - h_r(m') = \sum_{i=1}^{\ell} (c_i - c'_i) r^{\ell-i+1} =: P(r) \in \mathbb{F}_p,$$

a nonzero polynomial in r of degree at most ℓ (nonzero because the coefficient vectors differ and the encoding is injective). For any target difference $\delta \in \mathbb{F}_p$, the equation $P(r) = \delta$ is a polynomial

equation of degree $\leq \ell$ over the field $\mathbb{F}_p$, so it has at most ℓ roots r . Since r is (clamped) uniform over a set of 2^{106} values, a uniformly chosen r is a root with probability at most $\ell/2^{106}$. For a target difference δ taken modulo 2^{128} , each residue class modulo 2^{128} corresponds to at most eight field differences (since $p = 2^{130} - 5$), multiplying the root-counting bound by a small constant; this is the AADU property with $\varepsilon \leq 8\ell_{\max}/2^{106}$. $\square$

Combining Proposition 5.12 with the Carter–Wegman Theorem 5.10 shows that Poly1305, with a *fresh, unpredictable pad s per message*, is a one-time MAC with forgery probability at most $8\ell_{\max}/2^{106}$ per verification attempt. In the AEAD scheme below, ChaCha20 itself generates the one-time key (r, s) pseudorandomly from the nonce, so the per-nonce one-time guarantee suffices.

5.4 Authenticated encryption with associated data

We now combine confidentiality and integrity. The appropriate composite goal is *authenticated encryption*: the adversary should be unable both to learn anything about plaintexts (CPA security) and to produce any new ciphertext that decrypts to a non- $\perp$ value (ciphertext integrity). Real protocols also require ciphertexts to bind to public context (packet headers, version numbers) that is sent in the clear but must be authenticated; this is the *associated data*.

Definition 5.13 (AEAD scheme). An *authenticated encryption with associated data* (AEAD) scheme is a pair (Enc, Dec) with key space $\mathcal{K}$, nonce space $\mathcal{N}$, and associated-data space $\mathcal{D}$: $\text{Enc}_k(\nu, a, m)$ returns a ciphertext c , and $\text{Dec}_k(\nu, a, c)$ returns a message in $\mathcal{M}$ or $\perp$. Correctness: $\text{Dec}_k(\nu, a, \text{Enc}_k(\nu, a, m)) = m$ for all k, ν, a, m . The scheme authenticates but does not encrypt the associated data a .

Definition 5.14 (Ciphertext integrity, INT-CTXT). In experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{ctxt}}(\lambda)$: sample $k \leftarrow \text{Gen}(1^\lambda)$; give $\mathcal{A}$ an encryption oracle $\text{Enc}_k(\cdot, \cdot, \cdot)$, recording the set Q of returned triples (ν, a, c) ; $\mathcal{A}$ wins if it outputs $(\nu^*, a^*, c^*) \notin Q$ with $\text{Dec}_k(\nu^*, a^*, c^*) \neq \perp$. The adversary must never repeat a nonce to its encryption oracle (the *nonce-respecting* model). Π has *ciphertext integrity* if $\Pr[\mathcal{A} \text{ wins}] \leq \text{negl}(\lambda)$ for all PPT $\mathcal{A}$.

Definition 5.15 (Authenticated encryption). A nonce-respecting AEAD scheme is *secure* (an AEAD) if it is both CPA-secure (Definition 5.4, with nonces and associated data included in the challenge) and has ciphertext integrity (Definition 5.14).

The standard way to build an AEAD from a CPA-secure cipher and a strongly unforgeable MAC is *encrypt-then-MAC*: encrypt the message, then compute a MAC over the ciphertext (together with the associated data and nonce), appending the tag. Decryption verifies the tag *before* decrypting, and any tag failure returns $\perp$. The contrasting orders (MAC-then-encrypt and encrypt-and-MAC) do not generically achieve authenticated encryption; encrypt-then-MAC does.

Theorem 5.16 (Encrypt-then-MAC yields authenticated encryption). Let $\Pi_E = (\text{Enc}, \text{Dec})$ be a CPA-secure encryption scheme and let $\Pi_M = (\text{Mac}, \text{Vrfy})$ be a strongly unforgeable (sUF-CMA) MAC, with independent keys k_E, k_M . Define the composite AEAD

$$\text{Enc}'_{k_E, k_M}(\nu, a, m) = (c, t), \quad c \leftarrow \text{Enc}_{k_E}(\nu, m), \quad t \leftarrow \text{Mac}_{k_M}(\nu \| a \| c),$$

with Dec' returning $\perp$ unless $\text{Vrfy}_{k_M}(\nu \| a \| c, t) = 1$, in which case it returns $\text{Dec}_{k_E}(\nu, c)$. Then Π' is

| a secure AEAD: it is CPA-secure and has ciphertext integrity.

Proof. Ciphertext integrity. Suppose a PPT $\mathcal{A}$ outputs a fresh $(v^*, a^*, (c^*, t^*)) \notin Q$ that decrypts successfully, i.e. $\text{Vrfy}_{k_M}(v^* || a^* || c^*, t^*) = 1$. Each encryption-oracle call on (v, a, m) returns (c, t) and corresponds to a MAC oracle query on the string $w = v || a || c$ returning tag t . Because the framing $v || a || c$ parses uniquely (lengths are fixed or prepended), the pair (v^*, a^*, c^*) being fresh and yet verifying means the MAC oracle never produced the string-tag pair $(v^* || a^* || c^*, t^*)$: either $w^* := v^* || a^* || c^*$ was never queried, or it was queried but returned a tag $\neq t^*$. In both cases (w^*, t^*) is a fresh valid string-tag pair, a strong forgery against Π_M . Hence we build $\mathcal{B}$ that runs $\mathcal{A}$, answering encryption queries using its own MAC oracle and a self-chosen k_E , and outputs (w^*, t^*) ; $\mathcal{B}$ wins whenever $\mathcal{A}$ does, so $\Pr[\mathcal{A} \text{ wins}] \leq \text{Adv}_{\Pi_M, \mathcal{B}}^{\text{s-forge}}(\lambda) = \text{negl}(\lambda)$.

CPA security. We argue indistinguishability of the challenge by a hybrid that replaces the genuine tag's coupling with the message. Consider the challenge (m_0, m_1) ; the ciphertext is (c, t) with $c \leftarrow \text{Enc}_{k_E}(v, m_b)$ and $t = \text{Mac}_{k_M}(v || a || c)$. The tag t is a deterministic function of c (and k_M, v, a) and so carries no information about b beyond what c already carries. Formally, we reduce to the CPA security of Π_E : a distinguisher $\mathcal{B}'$ samples k_M itself, forwards $\mathcal{A}$'s encryption and challenge queries to its Π_E -oracle to obtain the c -components, and appends MACs it computes with k_M . Then $\mathcal{B}'$ perfectly simulates Π' to $\mathcal{A}$, and $\text{Adv}_{\Pi', \mathcal{A}}^{\text{cpa}}(\lambda) = \text{Adv}_{\Pi_E, \mathcal{B}'}^{\text{cpa}}(\lambda) = \text{negl}(\lambda)$. $\square$

Remark 5.17 (Integrity of ciphertexts and the choice of ordering). Encrypt-then-MAC permits the receiver to reject forged ciphertexts *without decrypting*, which is what blocks chosen-ciphertext and padding-oracle attacks: a rejected ciphertext never reaches the decryption logic, so it cannot leak. By contrast, MAC-then-encrypt verifies only after decrypting, exposing the decryption routine to attacker-chosen ciphertexts. Ciphertext integrity is also exactly the property that upgrades CPA security to CCA (chosen-ciphertext) security: an authenticated-encryption scheme is automatically CCA-secure, because the decryption oracle is useless to the adversary (every ciphertext it did not legitimately obtain decrypts to $\perp$ except with negligible probability).

5.5 The ChaCha20-Poly1305 AEAD

We now assemble the concrete scheme of RFC 8439, which Zcash uses. It is an encrypt-then-MAC composition of the ChaCha20 stream cipher with the Poly1305 one-time MAC, with the feature that the cipher derives *both* the key stream and the one-time MAC key from the single (k, v) input.

Definition 5.18 (ChaCha20-Poly1305 AEAD). Fix a 256-bit key k and a 96-bit nonce v . Encryption of message m with associated data a proceeds as follows.

1. *One-time MAC key.* Run ChaCha20 with key k , nonce v , and block counter 0; take the first 32 bytes of the keystream block as the Poly1305 one-time key (r, s) (clamp the first 16 bytes to form r ; the next 16 are s). Discard the rest of this block.
2. *Encrypt.* Run ChaCha20 with key k , nonce v , starting at block counter 1, to produce the keystream KS_1 (the keystream of Remark 5.3 taken from block counter 1 onward), and set $c = m \oplus \text{KS}_1(k, v, |m|)$.
3. *Authenticate.* Form the Poly1305 input by concatenating: a padded with zeros to a 16-byte boundary, c padded with zeros to a 16-byte boundary, the 8-byte little-endian length of a , and the 8-byte little-endian length of c . Compute the tag $t = \text{Poly1305}_{r,s}(\text{that input})$.

The ciphertext is (c, t) . Decryption recomputes (r, s) and the tag from (k, v, a, c) , compares it to t in constant time, returns $\perp$ on mismatch, and otherwise outputs $m = c \oplus \text{KS}_1(k, v, |c|)$.

Remark 5.19 (Length encoding prevents splicing). The trailing 8-byte lengths of a and c are essential. Without them, the zero-padding that aligns a and c to 16-byte boundaries would permit an adversary

to shift bytes between the associated data and the ciphertext (or pad-extend one of them) while leaving the Poly1305 input unchanged, breaking the binding between a and c . Appending the explicit lengths makes the MAC input an injective encoding of (a, c) , which is what the prefix-free requirement in the $\Delta\Delta U$ analysis (Proposition 5.12) demands.

Theorem 5.20 (Security of ChaCha20-Poly1305). *Model ChaCha20 as a pseudorandom function $F_k(\nu, j)$ from (ν, j) (nonce and block counter) to 64-byte blocks. In the nonce-respecting model, ChaCha20-Poly1305 is a secure AEAD: any PPT adversary making q encryption queries and q_v verification (decryption) queries whose Poly1305 input (padded associated data, padded ciphertext, and the length block) comprises at most $\ell_{\max}$ sixteen-byte blocks, $\ell_{\max} = \lceil |a|/16 \rceil + \lceil |c|/16 \rceil + 1$, has CPA advantage at most $\text{Adv}_F^{\text{prf}}(\lambda) + (\text{negligible})$ and forges (breaks ciphertext integrity) with probability at most*

$$\text{Adv}_F^{\text{prf}}(\lambda) + \frac{8q_v \ell_{\max}}{2^{106}}.$$

Proof. Sketch. Replace F_k by a truly random function Φ from (ν, j) to 64-byte blocks; this costs $\text{Adv}_F^{\text{prf}}(\lambda)$ by definition of the PRF, and we analyse the idealised scheme.

Confidentiality. Since the adversary is nonce-respecting, every encryption query uses a fresh ν . For a fresh nonce, the blocks $\Phi(\nu, 1), \Phi(\nu, 2), \dots$ used as keystream are uniform and independent of everything the adversary has seen, so $c = m \oplus \text{KS}$ is a genuine one-time pad and reveals nothing about m beyond its length; the tag is a deterministic function of c, a, ν and adds no information. Thus the idealised scheme has zero CPA advantage, and the real scheme has CPA advantage at most $\text{Adv}_F^{\text{prf}}(\lambda)$.

Integrity. For each nonce ν , the one-time key $(r, s) = \Phi(\nu, 0)[0:32]$ is uniform and independent across distinct nonces, and—because counter 0 is reserved for the MAC key while encryption uses counters ≥ 1 —independent of the keystream that produced the ciphertexts. Hence per nonce the setting is exactly the Carter–Wegman one-time-MAC setting of Theorem 5.10 with the ε - $\Delta\Delta U$ Poly1305 hash of Proposition 5.12, $\varepsilon \leq 8\ell_{\max}/2^{106}$. Consider any single verification query (ν^*, a^*, c^*, t^*) that is fresh. If ν^* was never used for encryption, the pad s for ν^* is uniform and entirely unknown to the adversary, so the tag t^* is correct with probability 2^{-128} . If ν^* equals the nonce of some encryption query (the adversary may reuse a nonce in a *verification* query, though not in encryption), then the adversary has seen exactly one tag under (r, s) , and the freshness of (a^*, c^*) means it is attempting a forgery on a new message under that one-time key; by Theorem 5.10 this succeeds with probability at most $\varepsilon \leq 8\ell_{\max}/2^{106}$. A union bound over the q_v verification queries gives forgery probability at most $8q_v \ell_{\max}/2^{106}$ in the idealised scheme, and adding back the PRF-switching cost yields the stated bound. $\square$

Remark 5.21 (Single key, two roles). ChaCha20-Poly1305 reuses one 256-bit key for both encryption and authentication, seemingly violating the independent-keys hypothesis of the encrypt-then-MAC Theorem 5.16. Reserving *block counter* 0 for the Poly1305 key and counters ≥ 1 for the keystream is exactly what restores independence: modelling ChaCha20 as a PRF, the outputs at distinct counters are independent, so (r, s) is independent of the keystream, and the two-key analysis applies. This is a recurring design pattern—domain separation by a counter or label permits one master key to safely play several roles.

5.6 Key-derivation functions

The AEAD above assumes a uniformly random 256-bit key. In practice keys come from sources that are *high-entropy but not uniform*: a Diffie–Hellman shared secret g^{xy} is a group element, not a uniform bit string; a passphrase or a hardware noise source has min-entropy spread unevenly across its bits. A *key-derivation function* (KDF) bridges this gap, turning a non-uniform secret into one or more uniformly random keys. The relevant entropy measure is min-entropy.

Definition 5.22 (Min-entropy). The *min-entropy* of a random variable X is $H_\infty(X) = -\log_2 \max_x \Pr[X = x]$. Equivalently, the best chance of guessing X in one try is $2^{-H_\infty(X)}$. The *statistical distance* between distributions X, Y on a set S is $\Delta(X, Y) = \frac{1}{2} \sum_{u \in S} |\Pr[X = u] - \Pr[Y = u]|$; we say X is ϵ -close to uniform if $\Delta(X, U_S) \leq \epsilon$.

Definition 5.23 (Key-derivation function and its security). A *key-derivation function* is a function $\text{KDF} : \mathcal{S} \times \mathcal{I} \times \{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$ taking a source secret (sample of a random variable Σ), an optional non-secret *salt* slt , an *info/context* string ctx , and a desired output length L . Its security goal: whenever Σ has min-entropy at least γ (relative to the adversary’s side information), the output $\text{KDF}(\Sigma, \text{slt}, \text{ctx}, L)$ should be computationally indistinguishable from U_L , even given slt , ctx , and that side information.

The notion a KDF targets is that of a (computational) randomness extractor: a seeded function whose output looks uniform whenever its input has enough min-entropy.

Definition 5.24 (Computational extractor). A keyed function $\text{Ext} : \mathcal{S} \times \mathcal{X} \rightarrow \{0, 1\}^n$ is a *strong* (γ, ϵ) -*computational extractor* if for every source Σ (with auxiliary information) of min-entropy at least γ , and for a uniformly random salt $\text{slt} \xleftarrow{\$} \mathcal{S}$,

$$(\text{slt}, \text{Ext}_{\text{slt}}(\Sigma)) \approx_c (\text{slt}, U_n),$$

with distinguishing advantage at most ϵ ; “strong” means the construction reveals the salt to the distinguisher.

Information theory can realise this notion unconditionally. The *leftover hash lemma* states that any 2^{-n} -universal family $\{g_r : \mathcal{X} \rightarrow \{0, 1\}^n\}$ with a uniform public seed R satisfies $\Delta((R, g_R(X)), (R, U_n)) \leq \frac{1}{2} \sqrt{2^{n-\gamma}}$ whenever $H_\infty(X) \geq \gamma$; thus investing $\gamma \geq n + 2 \log_2(1/\epsilon)$ bits of min-entropy buys n bits that are ϵ -close to uniform. We state this without proof, since the deployed mechanism below rests on a random-oracle modelling instead.

HKDF, in passing. The standardised general-purpose KDF is Krawczyk’s HKDF (RFC 5869), built from HMAC (§4.4) in an *extract-then-expand* architecture: a salted HMAC call first distils the non-uniform secret into a short uniform *pseudorandom key* (the extractor stage, justified along leftover-hash lines), and a counter-chained sequence of HMAC calls then stretches that key, under a context string, into as many output bytes as required (the expander stage, justified by HMAC’s PRF security). HKDF is the right tool when one KDF must serve arbitrary sources and output lengths. Zcash does not use it—indeed the shielded protocol uses HMAC nowhere—because its requirements are far narrower and a single hash call meets them.

The Zcash KDF. Orchard derives the note-encryption key by one personalised BLAKE2b call:

$$K_{\text{enc}} = \text{KDF}(\text{sharedSecret}, \text{epk}) = \text{BLAKE2b-256}(\text{"Zcash_OrchardKDF"}; \text{sharedSecret} \parallel \text{epk}),$$

where sharedSecret is the Diffie–Hellman shared point of the key agreement in Section 6, epk is the sender’s ephemeral public key, and the quoted string occupies BLAKE2b’s personalisation field. The analysis models this personalised hash as a random oracle: by the domain-separation principle of §3.6 (Proposition 3.19 and Example 3.20), the personalisation makes it an oracle independent of every other hash use in the protocol, and on an input containing the high-min-entropy, hard-to-guess sharedSecret its output is uniform and independent of the adversary’s view—which is exactly the

security goal of Definition 5.23. Folding epk into the input binds the derived key to this particular ephemeral, the role the context string plays in HKDF; no salt is needed because the random-oracle modelling already rules out source distributions tuned against the function.

Remark 5.25 (The role of the KDF in the larger protocol). In the Zcash note-encryption layer, a Diffie–Hellman exchange on the Pallas curve produces a shared group element, and the single BLAKE2b call above condenses that element into the symmetric key fed to ChaCha20-Poly1305. The two halves of this section thus meet exactly here: key derivation manufactures the uniform key that the AEAD’s security proof (Theorem 5.20) takes as a hypothesis, and the chain from a non-uniform shared secret to an authenticated ciphertext is complete. Section 6 assembles this chain end to end as hybrid public-key encryption.

6 Key agreement

Every primitive constructed so far solves a problem *after* the parties already share a secret. A block cipher, a message-authentication code, and an authenticated encryption scheme each presuppose a key both endpoints already hold. The most basic question remains open: how do two parties who have never met, communicating over a channel that an adversary fully observes, agree on a secret the adversary does not learn? This is the problem of *key agreement*, whose first solution—the Diffie–Hellman protocol of 1976—inaugurated public-key cryptography. This section develops key agreement from the ground up: the bare protocol, the abstraction that captures it, the precise sense in which it is secure, its elliptic-curve incarnation, and the composition with symmetric encryption that yields the in-band public-key encryption used to deliver shielded notes.

Throughout, $\mathbb{G}$ denotes a finite cyclic group, written multiplicatively, with a fixed generator g and order $n = \text{ord}(g) = |\mathbb{G}|$. We assume all parties know the order n . Two concrete instances serve as references: the multiplicative group $\mathbb{F}_p^\times$ of a prime field (or a large prime-order subgroup of it), and the group of $\mathbb{F}$ -points of an elliptic curve. The protocol depends only on the group structure together with the hardness of certain computational problems, and not on the presentation of the group.

6.1 The Diffie–Hellman protocol

The protocol rests on a single algebraic asymmetry. In a cyclic group $\mathbb{G} = \langle g \rangle$, the map

$$\exp_g : \mathbb{Z}/n\mathbb{Z} \rightarrow \mathbb{G}, \quad x \mapsto g^x$$

is a group isomorphism: it is cheap to evaluate (by repeated squaring, in $O(\log n)$ group operations) but, in the groups used here, believed infeasible to invert. Inverting it is the *discrete-logarithm problem* (DLOG) of Definition 2.2; as there, $\log_g h$ denotes the unique $x \in \mathbb{Z}/n\mathbb{Z}$ with $g^x = h$.

The Diffie–Hellman insight is that two parties can each contribute one exponent, exchange the corresponding group elements in the clear, and—by the commutativity of exponentiation—arrive at a common value an eavesdropper, seeing only the two transmitted elements, cannot compute. We describe the protocol between two parties, *Alice* and *Bob*.

Definition 6.1 (Diffie–Hellman key exchange). Fix public parameters $(\mathbb{G}, g, n)$ with $\mathbb{G} = \langle g \rangle$ of order n . The protocol proceeds as follows.

1. Alice samples $a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ uniformly, computes $A = g^a$, and sends A to Bob.
2. Bob samples $b \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ uniformly, computes $B = g^b$, and sends B to Alice.
3. Alice computes $k_A = B^a$; Bob computes $k_B = A^b$.

The value $k_A = k_B$ is the *shared secret*; the elements A and B are the parties’ *public shares* (or *public keys*), and a, b their *private keys*.

Proposition 6.2 (Correctness). *In the protocol of Definition 6.1, both parties compute the same group element, namely*

$$k_A = k_B = g^{ab}.$$

Proof. By the laws of exponents in an abelian group,

$$k_A = B^a = (g^b)^a = g^{ba} = g^{ab} = (g^a)^b = A^b = k_B,$$

where $ba = ab$ because integer multiplication is commutative and a group of order n takes its exponents modulo n . $\square$

The shared secret g^{ab} is the foundational object of the section: a value determined by A and B , computable by anyone who knows *one* of the two private keys, but conjecturally not by anyone who knows only the two public shares. Subsection 6.4 names this conjecture.

Remark 6.3 (Why a generator, and why prime order). If $\mathbb{G}$ has small subgroups, an adversary can confine a victim's contribution to one of them and learn the shared secret modulo a small factor of n (a *small-subgroup* attack). For this reason we prefer $\mathbb{G}$ of *prime* order n , in which the only subgroups are $\{1\}$ and $\mathbb{G}$ itself, so every non-identity element is a generator and no nontrivial confinement is possible. When the natural group (such as $\mathbb{F}_p^\times$, of composite order $p-1$) lacks prime order, one works inside a prime-order subgroup, and a recipient validates incoming elements by checking they lie in that subgroup—for instance by verifying $B^n = 1$ and $B \neq 1$. The analogue of this check for elliptic curves appears in Subsection 6.6.

6.2 Static and ephemeral keys

Definition 6.1 is symmetric and interactive: both parties sample fresh randomness and both speak. In practice key pairs differ by their lifetime.

Definition 6.4 (Static and ephemeral keys). We call a Diffie–Hellman key pair (x, g^x)

- *static* (or *long-term*) if its owner generates it once and reuses it across many key-agreement sessions, typically publishing it and binding it to an identity; and
- *ephemeral* if its owner generates it freshly for a single session and discards it immediately afterwards.

The two notions combine into a small taxonomy of protocols, each with its own security character.

- **Ephemeral–ephemeral (DHE).** Both parties use fresh ephemeral keys, as in Definition 6.1 read literally. Once the session ends and a, b are erased, no one—not even the parties themselves—can recover the shared secret g^{ab} . This is the defining feature of *forward secrecy*: an adversary who later compromises both parties and steals all stored long-term state still cannot decrypt recorded past sessions, because the only secrets that ever existed were the now-deleted ephemerals.
- **Static–ephemeral.** One party (say the recipient, Bob) holds a static key pair $(b, B = g^b)$ known to Alice in advance, while Alice contributes a fresh ephemeral $(a, A = g^a)$. The shared secret is again g^{ab} . Crucially this requires *no interaction from Bob at agreement time*: Alice computes g^{ab} from Bob's published B , attaches A to her message, and sends everything in a single non-interactive flow. Bob recovers $g^{ab} = A^b$ when he later reads the message. This is precisely the shape of the hybrid encryption of Subsection 6.7 and of shielded-note delivery: the sender is online once, the recipient need not be online at all, and there is no round trip.
- **Static–static.** Both keys are long-term. Then g^{ab} is a fixed function of the two identities and stays the *same in every session* between them. This is occasionally useful but offers no forward secrecy and, because the secret never varies, one must combine it with fresh per-session randomness (a nonce) before using it to key any symmetric scheme.

Remark 6.5 (Static keys make the protocol non-interactive). The static–ephemeral case is the conceptual bridge from *key exchange* (an interactive protocol) to *public-key encryption* (a one-shot algorithm). A static public key $B = g^b$ functions exactly as an encryption public key: anyone can derive a shared secret with its owner by sending a single ephemeral A . The remainder of this section formalises this view through the key-agreement abstraction and then builds encryption on top of it.

6.3 The key-agreement scheme abstraction

We now abstract away from the particular group and isolate the two operations a key-agreement scheme must provide: deriving a public key from a private one, and combining one's own private key with a counterparty's public key to produce a shared secret. Higher-level constructions program against this abstraction.

Definition 6.6 (Key-agreement scheme). A (*non-interactive*) *key-agreement scheme* over a parameter set Π consists of three algorithms together with a private-key space $\mathcal{S}$, a public-key space $\mathcal{P}$, and a shared-secret space $\mathcal{K}$:

- $\text{KeyGen}(\Pi) \rightarrow s$: a randomised algorithm outputting a private key $s \in \mathcal{S}$;
- $\text{DerivePublic}(\Pi, s) \rightarrow p$: a deterministic algorithm mapping a private key $s \in \mathcal{S}$ to a public key $p \in \mathcal{P}$;
- $\text{Agree}(\Pi, s, p') \rightarrow k$: a deterministic algorithm mapping a private key $s \in \mathcal{S}$ and a public key $p' \in \mathcal{P}$ to a shared secret $k \in \mathcal{K}$.

The scheme is *correct* if for all parameters Π and all private keys s, t in the support of $\text{KeyGen}(\Pi)$,

$$\text{Agree}(\Pi, s, \text{DerivePublic}(\Pi, t)) = \text{Agree}(\Pi, t, \text{DerivePublic}(\Pi, s)). \quad (1)$$

That is, the two parties holding s and t compute the same shared secret from the other's public key.

Example 6.7 (Diffie–Hellman as a key-agreement scheme). Fix $\Pi = (\mathbb{G}, g, n)$ with $\mathbb{G} = \langle g \rangle$ of prime order n . Set $\mathcal{S} = \mathbb{Z}/n\mathbb{Z}$ (or, to avoid the degenerate identity-only case, $\mathcal{S} = (\mathbb{Z}/n\mathbb{Z}) \setminus \{0\}$), $\mathcal{P} = \mathcal{K} = \mathbb{G}$, and define

$$\text{KeyGen}(\Pi) = a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}, \quad \text{DerivePublic}(\Pi, a) = g^a, \quad \text{Agree}(\Pi, a, B) = B^a.$$

Correctness is exactly Proposition 6.2: with $p' = \text{DerivePublic}(\Pi, t) = g^t$ and $p = \text{DerivePublic}(\Pi, s) = g^s$,

$$\text{Agree}(\Pi, s, g^t) = (g^t)^s = g^{ts} = g^{st} = (g^s)^t = \text{Agree}(\Pi, t, g^s),$$

which is (1). This is the *Diffie–Hellman key-agreement scheme* $\text{DH}_{\mathbb{G}, g}$.

Remark 6.8 (Why this is the right abstraction). The interface deliberately hides the group. A protocol designer who programs against DerivePublic and Agree can swap in any concrete scheme satisfying (1)—a prime-field group, an elliptic curve, or even a post-quantum non-interactive key agreement such as the CSIDH group action—without changing a line of the surrounding logic. The Zcash note-encryption machinery follows this pattern with one generalisation: the deployed DerivePublic takes the base point as an explicit argument, so that Sapling and Orchard derive keys over per-address *diversified bases* rather than a fixed generator (*Ironwood Guide*). It still invokes an abstract key agreement and never inspects the underlying group, which is why the construction transports from Sprout's Curve25519 to Sapling's Jubjub and Orchard's Pallas. A key-encapsulation mechanism, by contrast, cannot satisfy (1): encapsulation requires the recipient's public key before producing its ciphertext, so no counterparty-independent DerivePublic exists. The static-ephemeral one-flow pattern of Subsection 6.7 does generalise to any KEM, with the encapsulation ciphertext playing the role of the ephemeral public key, at the cost of changing the interface the surrounding logic programs against.

The output of Agree is a group element, but downstream symmetric primitives require a uniform bit string of fixed length. Bridging this gap—turning a structured, non-uniform group element into key material—is the job of a key-derivation function, introduced in §5.6 and put to work in Subsection 6.7. First we must state the precise sense in which g^{ab} is secret.

6.4 Passive security and the Diffie–Hellman assumptions

The relevant notion for the bare protocol is security against a *passive* adversary—an eavesdropper who reads the channel but does not alter it. The adversary’s entire view is the pair of public shares (g^a, g^b) , and security demands that this view reveal nothing useful about g^{ab} . The two formalisations are exactly the Diffie–Hellman problems of §2.3: the *computational* problem CDH (Definition 2.4)—compute g^{ab} from (g^a, g^b) —and the *decisional* problem DDH (Definition 2.5)—distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) . We write $\text{Adv}_{\mathbb{G},g}^{\text{cdh}}(\mathcal{A}) = \Pr[\mathcal{A}(g^a, g^b) = g^{ab}]$ and $\text{Adv}_{\mathbb{G},g}^{\text{ddh}}(\mathcal{A})$ for the corresponding advantages. Section 2.4 establishes the hierarchy $\text{DDH} \leq \text{CDH} \leq \text{DLOG}$ (Theorems 2.7 and 2.8) together with the pairing-based separation of DDH from CDH (Example 2.9).

CDH asserts only that the adversary cannot compute the *whole* secret. Keying a symmetric cipher requires more: g^{ab} must be *indistinguishable* from a uniformly random group element, so that no individual bit, nor any predicate, of it leaks. The following concrete break shows the gap is real even without pairings.

Remark 6.9 (A concrete DDH break: the Legendre symbol). In the full multiplicative group $\mathbb{F}_p^\times$ the Legendre symbol $\left(\frac{\cdot}{p}\right)$ is efficiently computable and multiplicative, so (g^a, g^b) reveals the quadratic-residue characters of g^a and g^b , and these determine the character of g^{ab} ; comparing the predicted character with that of the third component distinguishes $\mathcal{D}_{\text{dh}}$ from $\mathcal{D}_{\text{rand}}$ with constant advantage, even though CDH in $\mathbb{F}_p^\times$ is believed hard. This motivates the restriction to a prime-order subgroup (Remark 6.3), in which every element is a square and the character leak vanishes.

We can now state the passive-security theorem precisely. The natural target is that the shared secret is, to a passive eavesdropper, indistinguishable from a fresh random group element.

Theorem 6.10 (Passive security of Diffie–Hellman). *Let $\text{DH}_{\mathbb{G},g}$ be the scheme of Example 6.7. Consider a passive adversary $\mathcal{A}$ who, given the transcript $(A, B) = (g^a, g^b)$ of an honest ephemeral–ephemeral session, must distinguish the true shared secret g^{ab} from a uniformly random element of $\mathbb{G}$. Any such distinguisher $\mathcal{A}$ has distinguishing advantage exactly $\text{Adv}_{\mathbb{G},g}^{\text{ddh}}(\mathcal{A})$, and the reduction preserves running time exactly. In particular, under the DDH assumption the shared secret is pseudorandom, and a passive adversary learns nothing about it that it could not have guessed about a random group element.*

Proof. The adversary’s challenge is, by definition, to distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) with a, b, c uniform and independent—verbatim the DDH distinguishing game of Definition 2.5. Hence any distinguisher’s advantage in the passive-security game equals its DDH advantage, which is negligible under the assumption. Pseudorandomness of g^{ab} follows immediately: no efficient test separates it from uniform. $\square$

Remark 6.11 (Why pseudorandomness, not just unpredictability). Theorem 6.10 is the property a key-agreement scheme requires. Feeding g^{ab} into a key-derivation function to produce a symmetric key, we model that function (in Subsection 6.7) as extracting near-uniform bits from an input of high min-entropy that is hard to guess. CDH alone guarantees that g^{ab} is hard to guess in full but not that it lacks efficiently-predictable structure; DDH guarantees it is as good as random. In practice one often relies only on CDH together with a *hash* key-derivation function modelled as a random oracle, which manufactures the missing pseudorandomness; we make this precise below. In either case the design goal is that the derived symmetric key be indistinguishable from uniform.

6.5 Active attacks and the role of authentication

Passive security is the most the bare protocol can offer, and it is fundamentally limited. The protocol of Definition 6.1 provides *no* authentication: nothing in g^a or g^b testifies to who sent it. An *active* adversary who controls the channel can mount the *man-in-the-middle* attack.

Definition 6.12 (Man-in-the-middle attack). An active adversary M sitting on the channel between Alice and Bob runs two independent Diffie–Hellman sessions:

1. M samples $m \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ and computes g^m . When Alice sends $A = g^a$ (intended for Bob), M intercepts it and instead forwards g^m to Bob. When Bob sends $B = g^b$ (intended for Alice), M intercepts it and forwards g^m to Alice.
2. Alice, believing she shares with Bob, computes $k_A = (g^m)^a = g^{am}$. M computes the same value as $A^m = g^{am}$. Symmetrically, Bob computes $k_B = (g^m)^b = g^{bm}$, which M also computes as B^m .

Proposition 6.13 (Man-in-the-middle defeats unauthenticated DH). *After the attack of Definition 6.12, Alice and Bob each hold a shared secret that they believe is shared with the other but is in fact shared with M : M knows both Alice’s g^{am} and Bob’s g^{bm} . M can therefore decrypt, read, and re-encrypt every subsequent message in each direction, remaining completely transparent to both parties. No assumption on $\mathbb{G}$ —not even ideal hardness of DLOG—prevents this.*

Proof. By Proposition 6.2 applied to each of the two sessions, Alice and M agree on g^{am} and Bob and M agree on g^{bm} . M knows m and both A and B , so it computes both secrets directly. Since neither Alice nor Bob verifies that the group element received originated from the intended peer, neither can detect the attack from their transcripts: each sees a syntactically valid run. The hardness of DLOG is irrelevant because M never inverts anything—it knows its own exponent m . $\square$

The distinction is structural: **passive security concerns secrecy of the secret; active security additionally requires authenticity of the public shares.** Confidentiality of a channel is worthless if one established it with the wrong party. The remedy is to *authenticate the Diffie–Hellman messages*—to bind each public share to the identity of its sender—so that the recipient rejects a substituted share.

Remark 6.14 (How authentication is supplied). Several mechanisms achieve this binding, all external to the bare key exchange:

- *Signed Diffie–Hellman.* Each party signs its ephemeral share with a long-term signing key whose public half a certificate authority or a trust-on-first-use record certifies. M cannot forge Alice’s signature on g^m , so Bob rejects the substituted share. This is the structure of authenticated TLS.
- *Static keys with authenticated public keys.* If Bob’s public key $B = g^b$ is itself authentic—fetched from a trusted directory or pinned in advance—then a static–ephemeral exchange to B already authenticates Bob to Alice: only the holder of b can complete it. This is the model relevant to public-key encryption and to shielded-note delivery, where the recipient’s address is an authenticated public key. This authenticates the recipient, not the sender; sender authentication, when needed, layers on separately.
- *Out-of-band verification.* The parties compare a short fingerprint of the exchanged keys over an authentic side channel (reading digits aloud, scanning a QR code), detecting any substitution because M ’s injected g^m yields a different fingerprint.

In every case the security goal of the key agreement proper remains the passive guarantee of Theorem 6.10; authentication is a separable layer with which the key agreement composes, not a property of the group operation itself.

6.6 Elliptic-curve Diffie–Hellman

The development above is stated for an abstract cyclic group, by design: the protocol, the abstraction, and the security reductions are group-agnostic. We now instantiate the group $\mathbb{G}$ with the group of points of an elliptic curve, the instantiation used in modern systems and in Zcash specifically.

Recall from the companion volume that an elliptic curve E over a field $\mathbb{F}$ (of characteristic neither 2 nor 3, say given in short Weierstrass form $y^2 = x^3 + ax + b$ with nonzero discriminant) has a set of $\mathbb{F}$ -rational points $E(\mathbb{F})$ which, together with the point at infinity $\mathcal{O}$ as identity and the chord-and-tangent addition law, forms a finite abelian group. We write this group *additively*, so the analogue of exponentiation $g \mapsto g^x$ is *scalar multiplication* $P \mapsto [x]P = \underbrace{P + \dots + P}_x$, computable in $O(\log x)$ point additions by double-and-add.

Definition 6.15 (Elliptic-curve Diffie–Hellman). Fix an elliptic curve $E/\mathbb{F}$, a base point $P \in E(\mathbb{F})$ of large prime order n generating a subgroup $\mathbb{G} = \langle P \rangle$, and cofactor $h = |E(\mathbb{F})|/n$. The scheme ECDH, in the abstraction of Definition 6.6, is

$$\text{KeyGen} = a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}, \quad \text{DerivePublic}(a) = [a]P, \quad \text{Agree}(a, Q) = [a]Q.$$

The shared secret of two parties with private keys a, b is

$$[a]([b]P) = [ab]P = [b]([a]P),$$

the point $[ab]P$. The associated hard problems—ECDLOG, ECDH, EDDH—are Definitions 2.2, 2.4, 2.5 read with $[x]P$ for g^x .

Proposition 6.16 (Correctness of ECDH). *ECDH satisfies the correctness condition (1).*

Proof. Scalar multiplication is the additive form of the cyclic-group exponentiation map: $[x]([y]P) = [xy]P$ because the group is abelian and $\mathbb{Z}$ acts on it through $\mathbb{Z}/n\mathbb{Z}$ (as $[n]P = \mathcal{O}$). Hence $\text{Agree}(a, [b]P) = [a]([b]P) = [ab]P = [ba]P = [b]([a]P) = \text{Agree}(b, [a]P)$, which is (1). $\square$

Remark 6.17 (Why elliptic curves). Section 2.5 quantifies the choice: elliptic-curve groups admit only the generic $O(\sqrt{n})$ attacks, whereas index calculus solves DLOG in $\mathbb{F}_p^\times$ in subexponential time (Remark 2.18), so a 256-bit curve delivers the security that a multi-thousand-bit prime field requires (Example 2.17). This efficiency, together with the availability of curves engineered for cheap in-circuit arithmetic, is why the Pallas curve underlies Orchard.

Remark 6.18 (Subgroup and curve validation). The elliptic-curve setting reintroduces the small-subgroup concern of Remark 6.3 in a sharpened form. A recipient computing $\text{Agree}(a, Q) = [a]Q$ on an attacker-supplied Q must guard against two pitfalls. First, Q must actually lie on E : an *invalid-curve attack* feeds a point on a different, weaker curve sharing the same addition formulas, so the implementation must check the curve equation. Second, if the full group $E(\mathbb{F})$ has cofactor $h > 1$, an adversary can submit a Q of small order h to confine $[a]Q$ to a tiny subgroup and learn $a \bmod$ (small factor); the defence is either to multiply incoming points by the cofactor (*cofactor clearing*) or to use a prime-order curve where $h = 1$. Prime-order curves such as Pallas sidestep the cofactor issue entirely, leaving only the on-curve check.

6.7 Hybrid public-key encryption

We can now assemble the final construction of the section. Diffie–Hellman yields the two parties a shared group element, but three obstacles stand between that element and a usable encryption

scheme: (i) the shared secret is a structured group element, not uniform key bits; (ii) Diffie–Hellman by itself transports no message; and (iii) we want the static–ephemeral, non-interactive flow of Subsection 6.2, so that the recipient need not be online. The resolution is to compose key agreement with a key-derivation function and an authenticated cipher. The construction requires two ingredients from the symmetric world.

Definition 6.19 (Key-derivation function, recalled). We use the specialisation of Definition 5.23 (§5.6) obtained by omitting the optional salt and fixing the output length to ℓ : a *key-derivation function* (KDF) is a deterministic function $\text{KDF} : \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ mapping a (possibly non-uniform, high-entropy) *input keying material* $k \in \mathcal{K}$ together with an optional context string to an ℓ -bit output. Security is as there: whenever k is drawn from a distribution of sufficiently high min-entropy that is hard to predict, the output $\text{KDF}(k, c)$ is computationally indistinguishable from a uniform ℓ -bit string, even given the context c . When KDF is built from a cryptographic hash, the cleanest model is the *random oracle*: $\text{KDF}(\cdot)$ behaves like a function whose every output is an independent uniform string.

Definition 6.20 (Authenticated encryption with associated data, recalled). We recall the *authenticated encryption with associated data* (AEAD) schemes of Definition 5.13, with key space specialised to $\{0, 1\}^\ell$ to match the KDF output: a pair (Enc, Dec) , where

$$\text{Enc}_K(N, \text{ad}, m) = c, \quad \text{Dec}_K(N, \text{ad}, c) \in \{m\} \cup \{\perp\},$$

N is a nonce, ad is associated data authenticated but not encrypted, and $\perp$ denotes rejection. Correctness: $\text{Dec}_K(N, \text{ad}, \text{Enc}_K(N, \text{ad}, m)) = m$. Security is *authenticated-encryption security* (Definitions 5.14 and 5.15): encryptions of any two equal-length messages are indistinguishable under chosen-plaintext attack *and* no adversary can produce a fresh ciphertext that decrypts to anything but $\perp$ (ciphertext integrity), so confidentiality and integrity hold simultaneously.

With these in hand we define the composed scheme. It is a *public-key* encryption scheme: anyone holding the recipient’s static public key can encrypt to them, and only the holder of the matching private key can decrypt.

Definition 6.21 (DH-based hybrid public-key encryption). Fix a key-agreement scheme (Definition 6.6)—concretely ECDH over $\mathbb{G} = \langle P \rangle$ of prime order n —a secure KDF, and a secure AEAD. The recipient holds a static key pair $(b, B = [b]P)$, with B published. To encrypt a message m to B :

1. **Ephemeral key.** Sample $e \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ and set the *ephemeral public key* $E_{\text{pk}} = [e]P = \text{DerivePublic}(e)$.
2. **Agree.** Compute the shared secret $S = \text{Agree}(e, B) = [e]B = [eb]P$.
3. **Derive.** Set the symmetric key $K = \text{KDF}(S, \text{context})$, where context includes E_{pk} (and protocol labels).
4. **Encrypt.** Output the ciphertext (E_{pk}, c) , where $c = \text{Enc}_K(N, \text{ad}, m)$ for a fixed or derived nonce N and any associated data ad .

To decrypt (E_{pk}, c) with private key b :

1. **Agree.** Recompute $S' = \text{Agree}(b, E_{\text{pk}}) = [b]E_{\text{pk}} = [be]P$.

2. **Derive.** Set $K' = \text{KDF}(S', \text{context})$ with the same context (the recipient reads E_{pk} from the ciphertext).
3. **Decrypt.** Output $\text{Dec}_{K'}(N, \text{ad}, c)$, which is m on success and $\perp$ on any tampering.

Theorem 6.22 (Correctness of hybrid encryption). *For every message m and every honestly generated ciphertext (E_{pk}, c) produced as in Definition 6.21, decryption returns m .*

Proof. By correctness of the key agreement (Proposition 6.16), $S' = [b]E_{\text{pk}} = [b]([e]P) = [be]P = [eb]P = [e]([b]P) = [e]B = S$. Since KDF is deterministic and both sides supply the same context (the recipient recovers E_{pk} , and hence the context, from the ciphertext), $K' = \text{KDF}(S', \text{context}) = \text{KDF}(S, \text{context}) = K$. By AEAD correctness, $\text{Dec}_K(N, \text{ad}, \text{Enc}_K(N, \text{ad}, m)) = m$. $\square$

Theorem 6.23 (Passive security of hybrid encryption). *Model KDF as a random oracle and let AEAD be AEAD-secure. Then the scheme of Definition 6.21 is secure against a passive adversary who sees the recipient's public key B and a challenge ciphertext: under the CDH assumption in $\mathbb{G}$, no probabilistic polynomial-time adversary can distinguish encryptions of two equal-length messages of its choosing. Concretely, an adversary making at most q_H random-oracle queries has distinguishing advantage bounded by*

$$\text{Adv}^{\text{cpa}}(\mathcal{A}) \leq q_H \cdot \text{Adv}_{\mathbb{G}, P}^{\text{cdh}}(\mathcal{B}) + \text{Adv}^{\text{ae}}(\mathcal{C}),$$

for explicit reductions $\mathcal{B}$ (against CDH) and $\mathcal{C}$ (against AEAD) running in essentially the same time as $\mathcal{A}$.

Proof. Sketch. The argument proceeds by a sequence of games. *Game 0* is the real chosen-plaintext game: the adversary submits m_0, m_1 , receives the encryption of m_β for a hidden bit β , and attempts to guess β . The challenge ciphertext is (E_{pk}, c) with $E_{\text{pk}} = [e]P$, $S = [eb]P$, $K = \text{KDF}(S, \text{context})$, and $c = \text{Enc}_K(\dots, m_\beta)$.

Game 1 replaces K by an independent uniform ℓ -bit string $\tilde{K}$. In the random-oracle model, Games 0 and 1 are identical unless the adversary queries the oracle at the exact point $(S, \text{context}) = ([eb]P, \text{context})$; only then can it observe that K is the oracle's value rather than fresh randomness. But querying that point requires the adversary to produce $S = [eb]P$ from its view $(B, E_{\text{pk}}) = ([b]P, [e]P)$ —which is exactly solving a CDH instance. Formally, the reduction $\mathcal{B}$ embeds its CDH challenge $([b]P, [e]P)$ as (B, E_{pk}) and runs $\mathcal{A}$, answering all random-oracle queries with fresh uniform values. In a pairing-free group $\mathcal{B}$ cannot recognise which query, if any, carries the first component $[eb]P$, so it cannot pick the solution out on sight; instead it selects one of $\mathcal{A}$'s at most q_H oracle queries uniformly at random and outputs that query's first component as its CDH answer. Whenever $\mathcal{A}$ queries the critical point—the only event distinguishing the two games— $\mathcal{B}$'s guess is correct with probability at least $1/q_H$. Hence $|\Pr[\mathcal{A} \text{ wins Game 0}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \leq \Pr[\text{critical query}] \leq q_H \cdot \text{Adv}_{\mathbb{G}, P}^{\text{cdh}}(\mathcal{B})$. The factor q_H is a genuine security loss of the guessing kind catalogued in Definition 1.25; assuming the stronger *gap*-CDH (CDH given a DDH-decision oracle, with which $\mathcal{B}$ could test each query) would restore tightness, but we state the loose bound honestly.

In *Game 1* the AEAD key $\tilde{K}$ is uniform and independent of everything in the adversary's view, so the only route to advantage is to break the AEAD scheme directly. A standard reduction $\mathcal{C}$ turns any remaining advantage into an advantage against AEAD confidentiality: $\mathcal{C}$ forwards m_0, m_1 to its own AEAD challenger, embeds the returned ciphertext as c , and relays $\mathcal{A}$'s guess. Thus $\Pr[\mathcal{A} \text{ wins Game 1}] - \frac{1}{2} \leq \text{Adv}^{\text{ae}}(\mathcal{C})$. Summing the two bounds gives the claim. (If we instead assume KDF only to be a secure key-derivation function in the sense of Definition 6.19, the same

argument goes through under DDH in place of CDH, using Theorem 6.10 to replace S by a uniform group element before deriving K .) $\square$

Remark 6.24 (Why each ingredient is necessary). The composition is tight; dropping any piece breaks it.

- *Without the KDF*, one would key the cipher with the raw group element S , whose bit-representation is non-uniform and, in non-DDH groups, partially predictable (Remark 6.9); the KDF both standardises the format and extracts uniform randomness, and by binding E_{pk} into its context it ties the key to this particular ephemeral.
- *Without the AEAD's integrity*, a passively confidential but malleable cipher would let an active adversary tamper with c undetectably; authenticated encryption ensures any modification yields $\perp$, giving chosen-ciphertext robustness for the symmetric payload.
- *Without the ephemeral key* (a static-static secret), every message to B would reuse the same K , violating the nonce-uniqueness that AEAD security demands and destroying forward secrecy. The fresh ephemeral per message both supplies forward secrecy with respect to the ephemeral and guarantees a fresh key for each ciphertext.

Remark 6.25 (Delivering shielded notes). Definition 6.21 is precisely the pattern by which a shielded transaction delivers a note to its recipient *in band*—on the public ledger, readable by no one but the recipient. The recipient's shielded address embeds a static public key $B = [b]G_d$ on the protocol's curve (Pallas, in Orchard); the base G_d is not a protocol-wide generator but a per-address *diversified base*, the hash-to-curve image of the address's diversifier (*Ironwood Guide*). The sender, constructing the note, samples an ephemeral e , publishes $E_{pk} = [e]G_d$ over that same base as part of the transaction, derives $K = \text{KDF}([eb]G_d, \dots)$, and AEAD-encrypts the note plaintext (value, recipient, randomness, memo) under K , attaching the ciphertext to the transaction. Anyone scanning the chain sees (E_{pk}, c) but, lacking b , cannot derive K and so learns nothing; the recipient, trial-deriving $K' = \text{KDF}([b]E_{pk}, \dots)$ for incoming transactions, decrypts exactly those notes addressed to them. The non-interactivity of Subsection 6.2 is essential: the recipient need never have been online when the sender created the note. Authenticity of B (Remark 6.14) comes from the address being part of the recipient's verified payment credential, which is what defeats the man-in-the-middle concern of Subsection 6.5 in this setting; the zero-knowledge proof accompanying the transaction supplies the complementary integrity guarantees on the note's contents.

7 Commitment schemes

A *commitment scheme* is the cryptographic analogue of placing a message inside a sealed, tamper-evident envelope and handing it to another party. Once the sender delivers the envelope, its sender can no longer alter the message it contains (the scheme is *binding*), yet the recipient cannot read the message until the sender chooses to unseal it (the scheme is *hiding*). This deceptively simple primitive is among the most reusable building blocks in cryptography: it underlies coin-flipping over a telephone, zero-knowledge proofs, verifiable secret sharing, and—of central importance here—the polynomial commitments that form the backbone of modern succinct argument systems such as those used in Zcash’s Orchard protocol.

This section develops commitments from first principles. It begins with the abstract syntax and the two security properties, hiding and binding, in each of their three quantitative flavours (perfect, statistical, computational), and establishes the fundamental impossibility result that no scheme can be simultaneously perfectly hiding and perfectly binding. It then studies concrete constructions of increasing algebraic richness: hash-based commitments, the Pedersen commitment and its vector generalisation, the Sinsemilla-style algebraic hash whose collision resistance reduces to discrete logarithm, and finally polynomial commitment schemes, contrasting the pairing-based and inner-product-argument families and making precise what it means to “commit to a polynomial and later prove an evaluation.”

Throughout, $\lambda \in \mathbb{N}$ denotes the security parameter, and we use the vocabulary of Section 1: a function $\epsilon: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $\epsilon(\lambda) = \text{negl}(\lambda)$, if it shrinks faster than the reciprocal of every polynomial (Definition 1.11); $\text{poly}(\lambda)$ denotes the class of functions bounded above by some fixed polynomial in λ ; and a randomised algorithm is *probabilistic polynomial time* (PPT) if its running time is bounded by a polynomial in λ on every input (Definition 1.3). The notation $x \leftarrow D$ denotes sampling x according to a distribution D , and $x \xrightarrow{\$} S$ denotes sampling x uniformly at random from a finite set S .

7.1 Syntax

We first fix the interface of a commitment scheme as a pair of algorithms, deliberately separating the public parameters (generated once and shared by all parties) from the per-commitment data.

Definition 7.1 (Commitment scheme). A *commitment scheme* for a message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a triple of algorithms (Setup, Commit, Open):

- Setup(1^λ) is a PPT algorithm that on input the security parameter (in unary) outputs public parameters pp . These parameters implicitly determine $\mathcal{M} = \mathcal{M}_{\text{pp}}$ and $\mathcal{R} = \mathcal{R}_{\text{pp}}$, and serve as an implicit input to the remaining algorithms.
- Commit($\text{pp}, m; r$) is a deterministic algorithm that takes a message $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$ and outputs a *commitment* c . The notation Commit(pp, m) without an explicit r signifies that the algorithm samples $r \xrightarrow{\$} \mathcal{R}$ uniformly and returns it alongside c .
- Open(pp, c, m, r) is a deterministic algorithm that outputs 1 (“accept”) or 0 (“reject”).

The scheme must satisfy *correctness*: for all $\text{pp} \in \text{Setup}(1^\lambda)$, all $m \in \mathcal{M}$ and all $r \in \mathcal{R}$,

$$\text{Open}(\text{pp}, \text{Commit}(\text{pp}, m; r), m, r) = 1.$$

We call the pair (m, r) used to produce a commitment the *opening* (or *decommitment*); revealing it is the means by which the committer later unseals the envelope. In the simplest and most common setting, Open(pp, c, m, r) merely recomputes Commit($\text{pp}, m; r$) and checks whether the result equals c ; we term such a scheme *canonically opening*. We adopt this convention unless stated otherwise. For

a canonically opening scheme, correctness holds automatically, and a valid opening of c is precisely a pair (m, r) with $\text{Commit}(\text{pp}, m; r) = c$.

Remark 7.2 (The two phases). A commitment scheme operates in two temporally separated phases. In the *commit phase*, the sender computes $c \leftarrow \text{Commit}(\text{pp}, m; r)$ and transmits c (only). In the later *reveal phase*, the sender transmits (m, r) , and the receiver runs Open to verify. Hiding constrains what the receiver learns after the commit phase; binding constrains what the sender can do in the reveal phase. The setup phase, which produces pp , is a third concern: who runs it, and whether one can trust it, proves decisive for some constructions, as Remark 7.3 discusses.

Remark 7.3 (Setup models). Three setup models recur. In the *common reference string* (CRS) model, a trusted party runs Setup and we assume it erases the trapdoor randomness it used; of the schemes below, only KZG genuinely resides here. In the *uniform random string* (URS) or transparent model, pp consists only of public coins (e.g. uniformly random group elements with no known relations), so there is no trapdoor to erase and no party to trust; this is the regime of hash-based, Pedersen, and inner-product commitments—a Pedersen generator derived by hashing to the curve (Remark 3.28) is transparent, even though the textbook presentation below phrases Setup as sampling and discarding a secret. In the *plain* model there are no parameters at all, or only a fixed function such as a standardised hash. Transparent setup is highly desirable in practice because a flawed or subverted trusted setup can silently destroy binding, as we demonstrate below for KZG.

7.2 Hiding and binding

We now formalise the two security properties. Each is naturally cast as a game between a challenger and an adversary $\mathcal{A}$, and each comes in three flavours according to how strong a quantitative guarantee we demand against how powerful an adversary.

Hiding. Intuitively, a commitment should leak no information about the committed message. We capture this by demanding that the distributions of commitments to any two messages be indistinguishable.

Definition 7.4 (Hiding). For a commitment scheme $\Pi = (\text{Setup}, \text{Commit}, \text{Open})$ and an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, consider the following experiment $\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, b)$ parameterised by a bit $b \in \{0, 1\}$:

1. $\text{pp} \leftarrow \text{Setup}(1^\lambda)$;
2. $\mathcal{A}_1(\text{pp})$ outputs two messages $m_0, m_1 \in \mathcal{M}$ and a state st ;
3. the challenger samples $r \xleftarrow{\$} \mathcal{R}$ and computes $c \leftarrow \text{Commit}(\text{pp}, m_b; r)$;
4. $\mathcal{A}_2(\text{st}, c)$ outputs a bit b' , which is the output of the experiment.

The *hiding advantage* of $\mathcal{A}$ is

$$\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) = \left| \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 1) = 1] \right|.$$

The scheme is:

- *computationally hiding* if for every PPT $\mathcal{A}$, $\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) = \text{negl}(\lambda)$;
- *statistically hiding* if the bound holds even for computationally unbounded $\mathcal{A}$, i.e. $\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) = \text{negl}(\lambda)$ for all (not necessarily efficient) $\mathcal{A}$;
- *perfectly hiding* if $\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) = 0$ for all $\mathcal{A}$ and all λ .

Equivalently, the scheme is perfectly hiding iff for every fixed pp and every pair $m_0, m_1 \in \mathcal{M}$, the random variables $\text{Commit}(\text{pp}, m_0; r)$ and $\text{Commit}(\text{pp}, m_1; r)$, induced by $r \xleftarrow{\$} \mathcal{R}$, are identically distributed; it is statistically hiding iff their statistical distance is negligible. Recall that the *statistical distance* between two distributions P, Q over a finite set Ω is $\Delta(P, Q) = \frac{1}{2} \sum_{\omega \in \Omega} |P(\omega) - Q(\omega)|$, and that $\Delta(P, Q)$ equals exactly the maximum, over all (even unbounded) distinguishers, of the advantage in telling P from Q ; this accounts for the coincidence of statistical hiding and the unbounded-adversary definition.

Binding. Intuitively, a committer must not be able to open one commitment in two different ways. The binding requirement forbids producing a single commitment together with two valid openings to distinct messages.

Definition 7.5 (Binding). For Π and an adversary $\mathcal{A}$, consider the experiment $\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda)$:

1. $\text{pp} \leftarrow \text{Setup}(1^\lambda)$;
2. $\mathcal{A}(\text{pp})$ outputs a commitment c and two openings $(m_0, r_0), (m_1, r_1)$;
3. the experiment outputs 1 iff $m_0 \neq m_1$ and $\text{Open}(\text{pp}, c, m_0, r_0) = \text{Open}(\text{pp}, c, m_1, r_1) = 1$.

The *binding advantage* is $\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) = \Pr[\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda) = 1]$. The scheme is:

- *computationally binding* if for every PPT $\mathcal{A}$, $\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) = \text{negl}(\lambda)$;
- *statistically binding* if the same holds for all unbounded $\mathcal{A}$;
- *perfectly binding* if $\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) = 0$ for all $\mathcal{A}$ and all λ .

A clean reformulation is useful. For fixed pp , two openings *collide* if $\text{Commit}(\text{pp}, m_0; r_0) = \text{Commit}(\text{pp}, m_1; r_1)$ with $m_0 \neq m_1$. For a canonically opening scheme, perfect binding means $\text{Commit}(\text{pp}, \cdot; \cdot)$ has the property that distinct messages never yield a common commitment value, i.e. the message m is a deterministic function of the commitment c . Thus perfect binding is equivalent to the map $c \mapsto m$ being well defined on the set of all attainable commitments.

Remark 7.6 (Note the asymmetry of the quantifiers). Hiding quantifies over an adversary's ability to extract information from a single honestly produced commitment; binding quantifies over an adversary's ability to manufacture an ambiguous commitment of its own choosing. A scheme can be strong in one and weak in the other, and the two properties pull in opposite directions, as the impossibility result of the next subsection makes precise.

7.3 Incompatibility of perfect hiding and perfect binding

The single most important structural fact about commitments is that perfect hiding and perfect binding cannot hold simultaneously. The intuition is short: perfect hiding implies that a commitment to m_0 could equally well be a commitment to m_1 ; but then *some* opening to m_1 must exist for that very commitment, which an unbounded committer can find, breaking binding.

Theorem 7.7 (No perfectly hiding and perfectly binding scheme). *Let $\Pi = (\text{Setup}, \text{Commit}, \text{Open})$ be canonically opening with message space $\mathcal{M}$ of size at least 2. If Π is perfectly hiding, then it is not perfectly binding (and indeed an unbounded adversary breaks binding with probability 1). Consequently no nontrivial commitment scheme is simultaneously perfectly hiding and perfectly binding.*

Proof. Fix any pp in the support of $\text{Setup}(1^\lambda)$ and pick two distinct messages $m_0, m_1 \in \mathcal{M}$. For a message m , let

$$C_m = \{ \text{Commit}(\text{pp}, m; r) : r \in \mathcal{R} \}$$

be the set of commitment values attainable from m , and let D_m be the distribution on C_m that $r \xleftarrow{\$} \mathcal{R}$ induces.

Perfect hiding implies that D_{m_0} and D_{m_1} are identical distributions; in particular they have the same support, so $C_{m_0} = C_{m_1}$. Pick any c in this common support. By definition of C_{m_0} there is $r_0 \in \mathcal{R}$ with $\text{Commit}(\text{pp}, m_0; r_0) = c$, and by definition of C_{m_1} there is $r_1 \in \mathcal{R}$ with $\text{Commit}(\text{pp}, m_1; r_1) = c$. Then $(c, (m_0, r_0), (m_1, r_1))$ is a binding break: both openings are valid (by correctness of canonical opening) and $m_0 \neq m_1$.

An unbounded adversary can carry out exactly this search—enumerating $\mathcal{R}$ to find r_0 and r_1 —for any pp , and therefore wins $\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda)$ with probability 1. Hence $\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) = 1 \neq 0$, so Π is not perfectly binding. $\square$

Remark 7.8 (The information-theoretic content). The proof exposes the underlying tension as a counting fact. Perfect binding forces the map $c \mapsto m$ to be well defined, so the commitment c determines m and hence carries $\geq \log_2 |\mathcal{M}|$ bits of information about m . Perfect hiding forces c to carry *zero* bits of information about m . These are compatible only when $|\mathcal{M}| = 1$, i.e. when there is nothing to commit to. One therefore always chooses which property to make perfect (or statistical) and settles for the other being merely computational. Pedersen commitments (Subsection 7.5) are perfectly hiding and computationally binding; hash-based commitments in the random-oracle reading (Subsection 7.4) are computationally binding (from collision resistance) and computationally hiding.

Remark 7.9 (Two regimes, two failure modes). The choice has operational meaning. If binding is only computational, an adversary with sufficient computing power (e.g. one able to solve discrete logs) could retroactively change a committed value, so the soundness of any protocol built on top degrades to a computational assumption. If hiding is only computational, an adversary could retroactively learn a committed value, so any privacy guarantee degrades similarly, and *everlasting* secrecy is lost once the assumption is broken in the future. Which regime to prefer is a judgement about whether integrity or long-term confidentiality is the greater concern.

7.4 Hash-based commitments

The most elementary construction uses a cryptographic hash function. Recall that a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is *collision resistant* if no PPT adversary can find $x \neq x'$ with $H(x) = H(x')$ except with negligible probability, and *preimage/one-way* if, given $H(x)$ for a random x of appropriate length, no PPT adversary can find any x' with $H(x') = H(x)$ except with negligible probability.

Definition 7.10 (Hash commitment). Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a hash function, $\mathcal{M} = \{0, 1\}^*$ and $\mathcal{R} = \{0, 1\}^\ell$ with $\ell = \ell(\lambda)$ (e.g. $\ell = 2n$). Define

$$\text{Commit}(m; r) = H(m \parallel r),$$

where $\parallel$ denotes concatenation, and open canonically. (There are no nontrivial public parameters beyond the choice of H .)

Proposition 7.11 (Binding of the hash commitment). *If H is collision resistant, then the hash commitment is computationally binding.*

Proof. Suppose a PPT adversary $\mathcal{A}$ outputs a commitment c and two valid openings $(m_0, r_0), (m_1, r_1)$ with $m_0 \neq m_1$ such that $H(m_0 \parallel r_0) = c = H(m_1 \parallel r_1)$. Since $m_0 \neq m_1$, the strings $m_0 \parallel r_0$ and $m_1 \parallel r_1$ are

distinct (a difference in the message part survives concatenation when the message length is encoded, e.g. by prefixing $|m|$; we assume such an unambiguous encoding). Thus $\mathcal{A}$ has produced a collision for H . The reduction $\mathcal{B}$ that runs $\mathcal{A}$ and outputs $(m_0||r_0, m_1||r_1)$ therefore wins the collision game with exactly the binding advantage of $\mathcal{A}$, which must then be negligible. $\square$

Proposition 7.12 (Hiding of the hash commitment). *Model H as a random oracle. Then for messages of length bounded by $\text{poly}(\lambda)$ and randomness length $\ell = \omega(\log \lambda)$, the hash commitment is computationally hiding; concretely, an adversary making q oracle queries has hiding advantage at most $q \cdot 2^{-\ell}$.*

Proof. Sketch. In the random-oracle model, $c = H(m_b||r)$ is a uniformly random n -bit string *unless* the adversary ever queries H at the point $m_b||r$. Since r is uniform in $\{0, 1\}^\ell$ and information-theoretically hidden from $\mathcal{A}$ before it sees c , each of the adversary's q queries hits the point $m_b||r$ with probability at most $2^{-\ell}$; a union bound bounds the probability that any query does by $q \cdot 2^{-\ell}$. Conditioned on no such query, the value c is independent of b , so the two worlds $b = 0$ and $b = 1$ are identically distributed. Hence the advantage is at most $q \cdot 2^{-\ell} = \text{negl}(\lambda)$. $\square$

Remark 7.13 (Why the randomness r is essential). Without r , the deterministic commitment $c = H(m)$ fails to be hiding whenever the message has low entropy: an adversary simply hashes its two candidate messages and compares. The randomness r (a “salt” or “nonce”) injects min-entropy so that the input to H is unpredictable. This is the same phenomenon that renders *deterministic* encryption insecure for low-entropy plaintexts.

Remark 7.14 (Strength profile). Hash commitments are transparent (no trusted setup, no algebraic structure to subvert) and post-quantum plausible (collision resistance of good hash functions is not known to fall to quantum algorithms beyond a quadratic speedup, which a modest increase in n absorbs). Their weakness is that the commitment is an opaque bit string with *no algebraic structure*: one cannot, for instance, add two commitments to obtain a commitment to the sum of the messages. The next construction repairs exactly this.

7.5 The Pedersen commitment

We now turn to a commitment with rich algebraic structure. Let $\mathbb{G}$ be a cyclic group of prime order p , written additively, in which the *discrete logarithm problem* is hard. Additive notation $[a]P$ denotes the scalar multiple of $P \in \mathbb{G}$ by $a \in \mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$; thus $[a]P = P + P + \dots + P$ (a summands), $[0]P = \mathcal{O}$ is the identity, and the map $a \mapsto [a]P$ is a group homomorphism $\mathbb{F}_p \rightarrow \mathbb{G}$. For present purposes $\mathbb{G}$ is the group of $\mathbb{F}_q$ -points of an elliptic curve of prime order p ; the only structural facts we use are that $\mathbb{G} \cong \mathbb{F}_p$ as a group and that discrete logs are hard.

Recall the DLP assumption of Definition 2.2. Throughout this section we fix a group generator $\mathcal{G}$ that on input 1^λ outputs $(\mathbb{G}, p, G)$ with $\mathbb{G}$ cyclic of prime order p (a $\Theta(\lambda)$ -bit prime) and generator G ; we say the *discrete logarithm* (DLog) *assumption* holds for $\mathcal{G}$ if no PPT adversary, given $(\mathbb{G}, p, G, [a]G)$ for uniform $a \xleftarrow{\$} \mathbb{F}_p$, outputs a except with negligible probability.

Definition 7.15 (Pedersen commitment). On input 1^λ , let Setup run $(\mathbb{G}, p, G) \leftarrow \mathcal{G}(1^\lambda)$, sample $s \xleftarrow{\$} \mathbb{F}_p^\times$, and set $H = [s]G$; it outputs $\text{pp} = (\mathbb{G}, p, G, H)$ and *discards* s . The message space is $\mathcal{M} = \mathbb{F}_p$ and the randomness space is $\mathcal{R} = \mathbb{F}_p$. Define

$$\text{Commit}(\text{pp}, m; r) = [m]G + [r]H,$$

and open canonically.

The group elements G and H are two generators whose relative discrete logarithm $s = \log_G H$ is unknown to all parties (the committer included). Everything below hinges on this: knowledge of s would break binding, while perfect hiding does not even require ignorance of s .

Theorem 7.16 (Perfect hiding of Pedersen). *The Pedersen commitment is perfectly hiding.*

Proof. Fix $\text{pp} = (\mathbb{G}, p, G, H)$ with $H \neq \mathcal{O}$ (which holds since $s \neq 0$), and fix any message $m \in \mathbb{F}_p$. Because H generates $\mathbb{G}$ (it is nonzero and $\mathbb{G}$ has prime order, so every nonzero element is a generator), the map $r \mapsto [r]H$ is a bijection $\mathbb{F}_p \rightarrow \mathbb{G}$. Hence, as r ranges uniformly over $\mathbb{F}_p$, the element $[r]H$ is uniformly distributed over $\mathbb{G}$, and so is its translate

$$\text{Commit}(\text{pp}, m; r) = [m]G + [r]H.$$

The distribution of the commitment is therefore the uniform distribution on $\mathbb{G}$, *independent of m* . Consequently for any two messages m_0, m_1 the commitment distributions coincide exactly, so the hiding advantage of *every* adversary—however powerful—is 0. $\square$

Theorem 7.17 (Computational binding of Pedersen under DLog). *If the discrete logarithm assumption holds for $\mathcal{G}$, then the Pedersen commitment is computationally binding. Concretely, from any adversary $\mathcal{A}$ that breaks binding with probability ε one constructs an adversary $\mathcal{B}$ that computes discrete logarithms with probability $(1 - 1/p)\varepsilon \geq \varepsilon - 1/p$ and essentially the same running time.*

Proof. Suppose $\mathcal{A}$, given $\text{pp} = (\mathbb{G}, p, G, H)$, outputs a commitment c and two valid openings (m_0, r_0) , (m_1, r_1) with $m_0 \neq m_1$ and

$$[m_0]G + [r_0]H = c = [m_1]G + [r_1]H.$$

Rearranging,

$$[m_0 - m_1]G = [r_1 - r_0]H.$$

We claim that $r_1 \neq r_0$. Indeed, if $r_0 = r_1$ then the right-hand side is $\mathcal{O}$, forcing $[m_0 - m_1]G = \mathcal{O}$; since G has order p this gives $m_0 \equiv m_1 \pmod{p}$, contradicting $m_0 \neq m_1$. With $r_1 - r_0 \neq 0$ invertible in the field $\mathbb{F}_p$, one may write

$$H = [(m_0 - m_1)(r_1 - r_0)^{-1}]G,$$

so the discrete logarithm of H to base G is $s = (m_0 - m_1)(r_1 - r_0)^{-1} \pmod{p}$.

Now consider the reduction $\mathcal{B}$: on a DLog challenge $(\mathbb{G}, p, G, Q)$ where the goal is to find $\log_G Q$, set $H := Q$, form $\text{pp} = (\mathbb{G}, p, G, H)$, and run $\mathcal{A}(\text{pp})$. The parameters pp are distributed almost exactly as in the real scheme: there $H = [s]G$ for uniform $s \in \mathbb{F}_p^\times$, so the real H is uniform over $\mathbb{G} \setminus \{\mathcal{O}\}$, whereas the challenge $Q = [a]G$ with uniform $a \in \mathbb{F}_p$ is uniform over all of $\mathbb{G}$; the two distributions lie within statistical distance $1/p = \text{negl}(\lambda)$ (they differ only on the event $Q = \mathcal{O}$, which occurs with probability $1/p$). Moreover, when $Q = \mathcal{O}$ the relation $[m_0 - m_1]G = [r_1 - r_0]Q = \mathcal{O}$ forces $m_0 = m_1$, so $\mathcal{A}$ cannot win in that case anyway. When $\mathcal{A}$ succeeds, $\mathcal{B}$ outputs $(m_0 - m_1)(r_1 - r_0)^{-1} \pmod{p}$, which equals $\log_G Q$. Thus $\mathcal{B}$ succeeds exactly when $\mathcal{A}$ succeeds in the simulated game, giving $\text{Adv}^{\text{dlog}}(\mathcal{B}) = (1 - 1/p) \text{Adv}^{\text{bind}}(\mathcal{A}) \geq \varepsilon - 1/p$, the factor $1 - 1/p$ accounting for the event $Q = \mathcal{O}$, on which $\mathcal{A}$ wins with probability 0; this matches the convention of Theorem 7.22. Under DLog, $\varepsilon = \text{negl}(\lambda)$. $\square$

Remark 7.18 (Equivocation with the trapdoor). The proof shows more: any party that *knows* $s = \log_G H$ can open a Pedersen commitment to any message of its choosing. Given $c = [m]G + [r]H$ and a target m' , set $r' = r + (m - m')s^{-1} \pmod{p}$; then $[m']G + [r']H = [m']G + [r]H + [(m - m')]G = [m]G + [r]H = c$. This *equivocation* (or *trapdoor*) property is not a defect: it is precisely why erasing s in Setup is mandatory, and it is the engine behind the zero-knowledge simulators of many protocols, which use the trapdoor to open commitments consistently with a simulated transcript.

Theorem 7.19 (Additive homomorphism). *The Pedersen commitment is additively homomorphic: for all $m_0, m_1, r_0, r_1 \in \mathbb{F}_p$,*

$$\text{Commit}(\text{pp}, m_0; r_0) + \text{Commit}(\text{pp}, m_1; r_1) = \text{Commit}(\text{pp}, m_0 + m_1; r_0 + r_1),$$

and more generally, for scalars $\alpha, \beta \in \mathbb{F}_p$,

$$[\alpha] \text{Commit}(\text{pp}, m_0; r_0) + [\beta] \text{Commit}(\text{pp}, m_1; r_1) = \text{Commit}(\text{pp}, \alpha m_0 + \beta m_1; \alpha r_0 + \beta r_1).$$

Proof. Both sides expand by the bilinearity of scalar multiplication over the additive group $\mathbb{G}$. For the first identity,

$$([m_0]G + [r_0]H) + ([m_1]G + [r_1]H) = [m_0 + m_1]G + [r_0 + r_1]H,$$

using commutativity of $+$ in $\mathbb{G}$ and $[a]P + [b]P = [a+b]P$. The general identity follows from $[\alpha]([m]G + [r]H) = [\alpha m]G + [\alpha r]H$ and the same regrouping. $\square$

Remark 7.20 (Consequences of homomorphism). Additive homomorphism is what renders Pedersen commitments indispensable in privacy-preserving value systems. In a confidential-transaction setting (as in Zcash’s value-balancing), each input and output value v_i is committed as $[v_i]G + [r_i]H$; a transaction balances iff $\sum_i v_i^{\text{in}} = \sum_j v_j^{\text{out}}$, which a verifier checks *without learning any v_i* by testing whether $\sum_i C_i^{\text{in}} - \sum_j C_j^{\text{out}}$ is a commitment to 0, i.e. equals $[r]H$ for the *net* randomness r . The prover does not reveal r ; it proves knowledge of r by *signing* the transaction under $[r]H$ as a verification key—the binding signature of §8.8. Homomorphism turns an arithmetic relation on hidden values into a group equation on public commitments.

7.6 Pedersen vector commitments

The scalar Pedersen scheme commits to a single field element. To commit to a whole vector $\mathbf{m} = (m_1, \dots, m_n) \in \mathbb{F}_p^n$ in a single group element, the scheme uses n independent generators.

Definition 7.21 (Pedersen vector commitment). On input 1^λ and a length $n = \text{poly}(\lambda)$, let Setup output $(\mathbb{G}, p)$ together with generators $G_1, \dots, G_n, H \xleftarrow{\$} \mathbb{G}$ sampled so that no nontrivial discrete-log relation among them is known (in practice one derives them from a public seed by hashing into the curve—a “nothing-up-my-sleeve” procedure that yields a transparent setup). The message space is $\mathbb{F}_p^n$ and the randomness space is $\mathbb{F}_p$. Define

$$\text{Commit}(\text{pp}, \mathbf{m}; r) = \sum_{i=1}^n [m_i] G_i + [r] H = \langle \mathbf{m}, \mathbf{G} \rangle + [r] H,$$

where $\langle \mathbf{m}, \mathbf{G} \rangle = \sum_{i=1}^n [m_i] G_i$ denotes the “multiscalar” inner product of the scalar vector $\mathbf{m}$ with the generator vector $\mathbf{G} = (G_1, \dots, G_n)$. Open canonically.

Theorem 7.22 (Properties of the vector commitment). *The Pedersen vector commitment is perfectly hiding, and it is computationally binding under the discrete logarithm assumption in $\mathbb{G}$.*

Proof. Perfect hiding. Exactly as in Theorem 7.16: since $H \neq \mathcal{O}$ generates the prime-order group $\mathbb{G}$, the term $[r]H$ is uniform over $\mathbb{G}$ for uniform r , so $\langle \mathbf{m}, \mathbf{G} \rangle + [r]H$ is uniform over $\mathbb{G}$ regardless of $\mathbf{m}$. Hence the commitment distribution is independent of the message.

Computational binding. The reduction is to the discrete logarithm in the stronger guise that finding *any* nontrivial relation among uniformly random generators is hard (this follows from DLog by

embedding the challenge in every generator, as we now show). Concretely, suppose $\mathcal{A}$ outputs a commitment c with two valid openings $(m, r) \neq (m', r')$ and $m \neq m'$. Then

$$\sum_{i=1}^n [m_i]G_i + [r]H = \sum_{i=1}^n [m'_i]G_i + [r']H,$$

hence

$$\sum_{i=1}^n [m_i - m'_i]G_i + [r - r']H = \mathcal{O},$$

a nontrivial linear relation among $G_1, \dots, G_n, H$ (nontrivial because some $m_i \neq m'_i$).

To convert this into a discrete log, the reduction $\mathcal{B}$ takes a DLog challenge $(\mathbb{G}, p, G, Q)$ with $Q = [s]G$ for unknown s , and embeds the challenge in *every* generator: it samples $u_i, w_i \xleftarrow{\$} \mathbb{F}_p$ and sets $G_i = [u_i]G + [w_i]Q$ for $i = 1, \dots, n$, and likewise $H = [u_H]G + [w_H]Q$. Each generator is uniform in $\mathbb{G}$ (the term $[u_i]G$ alone is uniform), matching the real distribution, and the generators reveal nothing about the w -coordinates: for every candidate value of w_i there is exactly one u_i consistent with the observed G_i , so conditioned on the adversary's entire view the w_i, w_H remain independent and uniform. Writing the recovered relation as $\sum_i [a_i]G_i + [a_H]H = \mathcal{O}$ with the a_i, a_H not all zero, substitution yields the scalar equation $c_0 + c_1 s \equiv 0 \pmod{p}$ with $c_0 = \sum_i a_i u_i + a_H u_H$ and $c_1 = \sum_i a_i w_i + a_H w_H$. Since the coefficient vector is nonzero and the w -coordinates are uniform and independent of it, c_1 is uniform on $\mathbb{F}_p$, hence nonzero with probability $1 - 1/p$; then $s = -c_0 c_1^{-1} \pmod{p}$ solves the challenge. Thus $\text{Adv}^{\text{dlog}}(\mathcal{B}) \geq (1 - 1/p) \text{Adv}^{\text{bind}}(\mathcal{A})$, which forces the binding advantage to be negligible. $\square$

Remark 7.23 (A compressing commitment). The vector commitment maps n field elements (plus randomness) to a *single* group element. It is therefore *compressing*: the commitment is exponentially shorter than the message for large n . Compression is incompatible with statistical binding (there are far more messages than commitments, so collisions exist in abundance), which is precisely why binding here is only *computational*—an unbounded adversary could find a colliding opening by solving the discrete-log relation. This is the same trade-off that Theorem 7.7 dictates, now visible at the level of cardinalities.

Remark 7.24 (Homomorphism in the vector setting). The vector commitment remains additively homomorphic in both message and randomness, componentwise: $\text{Commit}(m; r) + \text{Commit}(m'; r') = \text{Commit}(m + m'; r + r')$. This linearity is the foundation of the inner-product arguments discussed in Subsection 7.8, which prove statements about $\langle m, b \rangle$ for public vectors b while revealing only a single committed group element.

7.7 Sinsemilla: an algebraic hash-based commitment

The hash commitment of Subsection 7.4 is opaque; the Pedersen commitment is algebraic, but its binding rests on Commit being a simple linear map. A family of *algebraic hash functions* occupies an intermediate position: they are collision-resistant hashes whose internal operations are group operations on an elliptic curve, so that an arithmetic circuit verifies the entire computation cheaply, yet whose collision resistance still reduces to discrete logarithm. The Sinsemilla hash used in Zcash's Orchard is the archetype. We present its core idea below, namely a Pedersen-style hash over a sequence of message chunks.

The starting point is the *Pedersen hash*. We split a message M into k chunks $m_1, \dots, m_k$, each interpreted as a scalar in some bounded range, and we fix k independent generators $P_1, \dots, P_k \in \mathbb{G}$ with unknown mutual discrete logs. Define

$$\text{PedersenHash}(M) = \sum_{j=1}^k [m_j]P_j \in \mathbb{G}.$$

This is precisely the (unblinded) Pedersen vector commitment map, now employed as a hash—its output is a group element rather than a fixed-length bit string, but composing with a fixed encoding of group elements yields an ordinary hash.

Proposition 7.25 (Collision resistance of the Pedersen hash reduces to DLog). *If no efficient algorithm can find a nontrivial relation $\sum_j [a_j]P_j = \mathcal{O}$ (with not all $a_j = 0$ and each $|a_j|$ within the allowed chunk range) among the generators $P_1, \dots, P_k$ —a hardness that follows from the discrete logarithm assumption in $\mathbb{G}$ when the P_j are sampled as independent uniform generators—then PedersenHash is collision resistant.*

Proof. A collision is a pair $M \neq M'$ with the same hash. Writing the chunk decompositions (m_j) and (m'_j) , equality of hashes gives

$$\sum_{j=1}^k [m_j]P_j = \sum_{j=1}^k [m'_j]P_j \implies \sum_{j=1}^k [m_j - m'_j]P_j = \mathcal{O}.$$

Because $M \neq M'$, the coefficient vector $(m_j - m'_j)$ is nonzero, so this is a nontrivial relation among the P_j . The reduction to discrete log is the every-generator embedding of Theorem 7.22: express each generator as $[u_i]G + [w_i]Q$ for the challenge Q and read off the unknown logarithm from the recovered relation. Hence an efficient collision-finder yields an efficient discrete-log solver, contradicting the assumption. $\square$

The plain Pedersen hash requires one independent generator per chunk, which is costly when hashing long inputs. *Sinsemilla* reorganises the computation into an iterated, two-operand form that reuses a small, fixed table of generators while preserving the discrete-log reduction. It splits the message into pieces $m_1, m_2, \dots, m_k$ of a fixed width of w bits and maintains a running accumulator $\text{Acc} \in \mathbb{G}$. It uses a fixed base point Q and a table $S : \{0, 1\}^w \rightarrow \mathbb{G}$ that assigns to each possible *value* of a w -bit piece its own generator, derived by hashing-to-curve a domain string together with that value—so the table has 2^w entries, shared by all positions. The iteration is, schematically,

$$\text{Acc}_0 = Q, \quad \text{Acc}_j = \text{Acc}_{j-1} + (\text{Acc}_{j-1} + S(m_j)) = [2]\text{Acc}_{j-1} + S(m_j),$$

with output $\text{SinsemillaHash}(M) = \text{Acc}_k$, where the double-and-incomplete-add step is the elliptic-curve operation that the arithmetic circuit verifies in very few constraints. Unrolling the recurrence exhibits the hash as a fixed positional combination

$$\text{SinsemillaHash}(M) = \sum_{j=1}^k [2^{k-j}]S(m_j) + [2^k]Q,$$

whose integer coefficients are powers of two determined by position alone; the message enters only through *which* generator $S(m_j)$ each piece selects.

Proposition 7.26 (Collision resistance of *Sinsemilla* reduces to DLog). *Modelling the generator-derivation map $m \mapsto S(m)$ and the base point Q as independent uniform generators (a “nothing-up-my-sleeve” hash-to-curve, which derives Q as well as the table), any collision in *SinsemillaHash* yields a nontrivial discrete-log relation among the generators $\{S(m)\} \cup \{Q\}$, and hence—via the same every-generator embedding as Proposition 7.25, extended to include Q —a discrete logarithm. Thus *Sinsemilla* is collision resistant under the discrete logarithm assumption in $\mathbb{G}$.*

Proof. Sketch. Consider first colliding messages $M \neq M'$ of equal piece-count k : the collision equates two combinations of the unrolled form; subtracting, the constant $[2^k]Q$ cancels and one obtains

$$\sum_{j=1}^k [2^{k-j}](S(m_j) - S(m'_j)) = \mathcal{O},$$

a relation whose coefficient on each table generator $S(v)$ aggregates the positions where M , respectively M' , carries chunk value v : writing $c_v = \sum_{j: m_j=v} 2^{k-j} - \sum_{j: m'_j=v} 2^{k-j}$, the relation reads $\sum_v [c_v] S(v) = \mathcal{O}$ over the *distinct* generators $S(v)$. Each c_v is a difference of two subset-sums of the distinct powers $2^{k-1}, \dots, 2^0$, hence an integer of absolute value below 2^k . For admissible message lengths (2^k below the group order, a finite design-time check) c_v vanishes modulo the group order only if it vanishes as an integer, and by uniqueness of binary expansions $c_v = 0$ forces the two position sets to coincide; coincidence for every v forces $M = M'$ (the messages have equal piece-count k in this case). Since $M \neq M'$, some $S(v)$ carries a nonzero net coefficient and the relation is nontrivial. For piece-counts $k > k'$ the constants do not cancel: the relation acquires the coefficient $2^k - 2^{k'}$ on Q , a nonzero integer below the group order for admissible lengths, so the collision yields a nontrivial relation among $\{S(v)\} \cup \{Q\}$. Embedding a DLog challenge in every table generator and in Q , exactly as in Theorem 7.22, applied to the relation over the distinct generators with their net coefficients, recovers the challenge logarithm. $\square$

In deployment the final piece is zero-padded to the piece width, so the deployed SinsemillaHash is collision resistant only between inputs of a fixed length for a given personalisation — exactly the property the protocol specification requires; the variable-length argument above applies to the piece-aligned presentation given here.

To turn the hash into a *hiding* commitment one appends a Pedersen blinding term: fix one further independent generator H' (hashed to the curve, with no known discrete-log relation to Q or the table) and define

$$\text{SinsemillaCommit}(m; r) = \text{SinsemillaHash}(m) + [r] H', \quad r \xleftarrow{\$} \mathbb{F}_p.$$

Hiding is inherited verbatim from Pedersen: $[r]H'$ is uniform on $\mathbb{G}$ for uniform r , so the commitment's distribution is independent of m (Theorem 7.16). Binding rests on discrete log: two valid openings to distinct messages yield either a collision in SinsemillaHash (Proposition 7.26) or a nontrivial discrete-log relation between H' and the hash generators. Orchard's note commitment NoteCommit instantiates exactly this map; its key commitment Commitlvk composes it with the x -coordinate extraction (the specification's SinsemillaShortCommit), returning a base-field element rather than a point.

Remark 7.27 (Significance for circuits). Sinsemilla's advantage is *verification cost inside a proof*. In a SNARK, the prover must re-express every step it takes as polynomial constraints; a bit-oriented hash such as SHA-256 costs tens of thousands of constraints, whereas Sinsemilla's per-piece cost is a handful of curve operations chosen to lie in the same field the proof system already works over. One obtains a collision-resistant hash and commitment (used for note commitments and Merkle trees in Orchard) that is simultaneously cheap to prove and grounded in the same discrete-log hardness as the surrounding protocol. The presentation above is the conceptual core; the *Ironwood Guide* instantiates it concretely (piece width, the incomplete-addition caveats, and the exact domain strings).

7.8 Polynomial commitment schemes

This section concludes with the family of commitments that underlies modern succinct arguments. A *polynomial commitment scheme* (PCS) lets a prover commit to a polynomial f with a short string, and later convince a verifier that $f(z) = y$ for a queried point z —again with a short proof—without revealing f . Concisely: commit to a polynomial, and later prove an evaluation.

Definition 7.28 (Polynomial commitment scheme). Fix a field $\mathbb{F}$ and a degree bound d . A *polynomial commitment scheme* is a tuple (Setup, Commit, Open, Eval, Verify):

- $\text{Setup}(1^\lambda, d) \rightarrow \text{pp}$, public parameters supporting degree $\leq d$.
- $\text{Commit}(\text{pp}, f; r) \rightarrow C$, committing to $f \in \mathbb{F}[X]_{\leq d}$ (optionally with hiding randomness r).

- $\text{Open}(\text{pp}, C, f, r) \rightarrow \{0, 1\}$, verifying that C is a commitment to the whole polynomial f .
- $\text{Eval}(\text{pp}, f, r, z) \rightarrow (y, \pi)$, producing the claimed value $y = f(z)$ and an *evaluation proof* π for a point $z \in \mathbb{F}$.
- $\text{Verify}(\text{pp}, C, z, y, \pi) \rightarrow \{0, 1\}$, checking the evaluation proof against the commitment.

Beyond the hiding and binding of the underlying commitment, a PCS must satisfy *evaluation binding*: no PPT prover can produce a commitment C , a point z , and two accepting proofs for distinct claimed values $y \neq y'$ at z , except with negligible probability. Strong PCSs additionally satisfy an *extractability* (proof-of-knowledge) property: from a successful prover one can extract an actual polynomial f of degree $\leq d$ consistent with all accepted evaluations.

A single committed value C admits “probing” at arbitrary points z because of the rigidity of polynomials: a nonzero polynomial of degree $\leq d$ has at most d roots, so two distinct degree- $\leq d$ polynomials agree on at most d points. Evaluation binding leverages this rigidity: answering a different value at even one point is the behaviour of a prover holding a different polynomial. Making this intuition stand is a separate cryptographic requirement—an accepting evaluation proof is not an opening of C , so evaluation binding does not follow from the binding of Commit; each construction proves it from its own assumption (d -strong Diffie–Hellman for KZG, discrete log via the folding extractor for the IPA). The central design problem is to make the *evaluation proof* π short and fast to verify. Two families dominate.

The pairing-based family: KZG

The Kate–Zaverucha–Goldberg (KZG) scheme commits to $f = \sum_{i=0}^d a_i X^i$ by evaluating it “in the exponent” at a secret point τ .

Definition 7.29 (KZG commitment). Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order p with a nondegenerate bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, and generators $G \in \mathbb{G}_1, \hat{G} \in \mathbb{G}_2$. Setup samples a secret $\tau \xleftarrow{\$} \mathbb{F}_p$ and publishes the *structured reference string*

$$\text{pp} = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \hat{G}, [\tau]\hat{G}),$$

discarding τ . The commitment to f is the evaluation in the exponent,

$$\text{Commit}(\text{pp}, f) = \sum_{i=0}^d [a_i][\tau^i]G = [f(\tau)]G,$$

computable from pp without knowing τ .

To prove $f(z) = y$, the prover uses the polynomial identity that z is a root of $f(X) - y$: there is a quotient $q(X) = \frac{f(X) - y}{X - z} \in \mathbb{F}[X]_{\leq d-1}$ (a genuine polynomial precisely because $f(z) - y = 0$). The proof is the commitment to the quotient, $\pi = [q(\tau)]G$, and the verifier checks the pairing equation

$$e(C - [y]G, \hat{G}) \stackrel{?}{=} e(\pi, [\tau]\hat{G} - [z]\hat{G}),$$

which, by bilinearity, is exactly the “in the exponent” form of the identity $f(\tau) - y = q(\tau)(\tau - z)$.

Evaluation binding rests on the d -strong Diffie–Hellman assumption in the bilinear group (given the powers $[\tau^i]G$, it is infeasible to produce $[(\tau - z)^{-1}]G$ for any z of one’s choosing). The reduction is one line: two accepting proofs π, π' for distinct values $y \neq y'$ at the same point z satisfy, after subtracting the two pairing equations, $\pi - \pi' = [(y' - y)(\tau - z)^{-1}]G$, so the forger’s output yields $[(\tau - z)^{-1}]G = [(y' - y)^{-1}](\pi - \pi')$, exactly the forbidden element.

Remark 7.30 (Trade-offs of KZG). KZG is notable for *constant-size* commitments and proofs (one and one group element, respectively) and *constant-time* verification (two pairings), *independent of the degree d* . Its cost is twofold. First, it requires a *trusted setup*: the secret τ is a global trapdoor; anyone who learns it can forge evaluation proofs for false statements, so one must run Setup so that τ is provably destroyed (in practice by a multi-party “powers-of-tau” ceremony in which the trapdoor stays secret as long as one participant is honest). Second, it requires pairing-friendly curves, a more delicate and less efficient setting than ordinary elliptic curves. In the spirit of Theorem 7.7, binding here is computational (it rests on the bilinear assumption).

The inner-product-argument family

The second family dispenses with pairings and with trusted setup, at the cost of larger proofs and slower verification. Its commitment is exactly the Pedersen vector commitment of Subsection 7.6, applied to the coefficient vector.

Definition 7.31 (IPA-style polynomial commitment). With transparent generators $G = (G_0, \dots, G_d)$ and blinding generator H (all with unknown mutual discrete logs, derived from a public seed), commit to $f = \sum_{i=0}^d a_i X^i$ by the Pedersen vector commitment of its coefficients,

$$\text{Commit}(\text{pp}, f; r) = \sum_{i=0}^d [a_i] G_i + [r] H = \langle a, G \rangle + [r] H.$$

The evaluation $f(z) = \sum_i a_i z^i = \langle a, z \rangle$ is an *inner product* of the coefficient vector a with the public vector $z = (1, z, z^2, \dots, z^d)$ of powers of the query point. Proving an evaluation thus reduces to proving an inner-product relation about a committed vector. The *inner-product argument* accomplishes this with a recursive “folding” protocol: at each round it halves the dimension of the vectors and of the generator set by taking a random linear combination, sending two group elements per round, until a single scalar remains. After $\log_2(d+1)$ rounds the verifier checks one final equation.

Proposition 7.32 (Properties of the IPA-based PCS). *The IPA-based polynomial commitment has transparent setup (public-coin parameters, no trapdoor), logarithmic-size evaluation proofs ($O(\log d)$ group elements), and linear-time verification ($O(d)$ group operations, though this can be batched/amortised). Its binding and evaluation binding reduce to the discrete logarithm assumption, inherited from the Pedersen vector commitment.*

Proof. Sketch. Binding of the commitment is Theorem 7.22. For evaluation binding and extractability, one shows the folding protocol is *tree special-sound* (Definition 9.15): from a suitable *tree of accepting transcripts*, branching at each round’s challenge, an extractor computes a coefficient vector a consistent with the commitment and the claimed inner product; two distinct extracted openings of the same commitment would yield a nontrivial discrete-log relation among G, H , contradicting DLog. The $O(\log d)$ proof size follows immediately from the halving recursion; the round messages are the two “cross-term” commitments L_j, R_j produced at each fold. $\square$

Remark 7.33 (Contrast and the Halo lineage). The two families embody a clean trade-off. KZG: constant proof and verification, but a structured trusted setup and pairings. IPA: transparent setup over a plain curve and only discrete-log hardness, but logarithmic proofs and linear verification. The Halo 2 system used in Zcash Orchard builds on the inner-product family precisely to avoid trusted setup, and recovers practical verification cost through *batch verification*: the deployed verifier folds the final multiscalar multiplications of many proofs into one shared multiexponentiation, amortising the per-proof group-operation cost, though the per-proof verifier work stays linear. The Halo lineage’s

“accumulation”/recursion technique extends the same deferral across recursively composed proofs—one proof attesting to another’s deferred check—but deployed Zcash does not use recursion. For this reason the inner-product commitment, despite its asymptotically heavier verification, is the appropriate foundation there.

Remark 7.34 (Hiding variants and the role of blinding). In both families one obtains a hiding variant by adding a Pedersen-style blinding term: one can augment KZG with a $[r][\gamma]G$ summand using an extra setup element $[\gamma]G$, and the IPA commitment already carries its $[r]H$. Without blinding, the schemes are binding but not hiding—the commitment is a deterministic function of f —which is acceptable when the polynomial encodes only public data, but must be repaired with randomness whenever the polynomial encodes secrets, as in the witness polynomials of a zero-knowledge proof. The interplay is once more the perfect-hiding/computational-binding regime of Pedersen, now lifted to polynomials.

Remark 7.35 (From commitments to arguments). Polynomial commitments are the linchpin that turns an *information-theoretic* proof object (a polynomial identity that holds over a large field) into a *succinct cryptographic argument*: the prover commits to its witness polynomials, the verifier challenges at random points, and the polynomial commitment’s evaluation proofs convince the verifier that the committed polynomials satisfy the required identities—without revealing them. The Schwartz–Zippel lemma supplies the soundness (a nonzero low-degree polynomial is almost surely nonzero at a random point), and the commitment supplies the binding and the succinctness. This is the bridge from the elementary commitments of this section to the full argument systems developed in the sequel.

8 Digital signatures

A digital signature is the public-key analogue of a handwritten signature: it lets the holder of a secret key produce, for any message, a short string that anyone holding the corresponding public key can check, and that nobody lacking the secret key can produce. Unlike a message authentication code, which requires the verifier to share the signer's secret, a signature is *publicly verifiable* and *non-repudiable*. Signatures are the workhorse of authenticated communication and, in the setting that motivates this monograph, the mechanism by which the spender of a note authorises a transaction. The Zcash Orchard protocol uses two closely related Schnorr-type schemes built over an elliptic curve: a *spend authorisation* signature whose key can be re-randomised so that successive uses are unlinkable, and a *binding* signature that simultaneously certifies the consistency of a transaction's value commitments and proves knowledge of a discrete logarithm. Both constructions derive from one classical object, the Schnorr identification protocol, so we develop the entire account from there.

Throughout this section $\mathbb{G}$ denotes a cyclic group of prime order q , written additively, with a fixed generator G ; the reader should keep in mind the example where $\mathbb{G}$ is the group of points of an elliptic curve over a finite field. We write scalar multiplication as $[x]P$ for $x \in \mathbb{Z}$ (equivalently $x \in \mathbb{F}_q$, since $\mathbb{G}$ has order q) and $P \in \mathbb{G}$. The *discrete logarithm* of P to base G is the unique $x \in \mathbb{F}_q$ with $P = [x]G$, when it exists; in a group of prime order generated by G it always exists and is unique. We assume the reader is comfortable with the asymptotic and probabilistic language (poly, negl, security parameter λ) developed for the other primitives.

8.1 Syntax and security goal

The exposition begins abstractly, fixing what a signature scheme *is* before asking what it means for one to be secure.

Definition 8.1 (Signature scheme). A *digital signature scheme* is a triple of probabilistic polynomial-time algorithms $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ together with, for each security parameter $\lambda \in \mathbb{N}$, a message space $\mathcal{M}_\lambda$, such that:

- $\text{KeyGen}(1^\lambda)$ outputs a pair (pk, sk) consisting of a *public* (or *verification*) key and a *secret* (or *signing*) key;
- $\text{Sign}(sk, m)$, for $m \in \mathcal{M}_\lambda$, outputs a *signature* σ ;
- $\text{Verify}(pk, m, \sigma)$ is deterministic and outputs a bit, 1 for *accept* and 0 for *reject*.

We require *correctness*: for every λ , every (pk, sk) in the support of $\text{KeyGen}(1^\lambda)$, and every $m \in \mathcal{M}_\lambda$,

$$\Pr[\text{Verify}(pk, m, \text{Sign}(sk, m)) = 1] = 1,$$

the probability taken over the coins of Sign (and, if applicable, Verify , though we take the latter to be deterministic). When the equality holds with probability 1 one speaks of *perfect correctness*; some schemes only achieve correctness with overwhelming probability, in which case we tolerate a $\text{negl}(\lambda)$ failure rate.

Correctness states that honestly generated signatures verify. It says nothing about an adversary, and the entire interest of the subject lies in pinning down what an adversary should be unable to do. The strongest standard notion, and the one to which we hold every construction, is *existential unforgeability under an adaptive chosen-message attack*. We hand the adversary the public key and a *signing oracle*: a black box that will sign any message of the adversary's choosing, as often as required, adaptively. The adversary wins if it can output a valid signature on *any* message it never submitted to the oracle.

Definition 8.2 (EUF-CMA). Let $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme and $\mathcal{A}$ an algorithm. Define the experiment $\text{Forge}_{\mathcal{A}, \Pi}(\lambda)$:

1. Run $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$.
2. Run $\mathcal{A}$ on input pk with oracle access to $\text{Sign}(sk, \cdot)$. Let Q be the set of messages $\mathcal{A}$ submits to the oracle. Let (m^*, σ^*) be $\mathcal{A}$'s output.
3. The experiment outputs 1 iff $\text{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin Q$.

We say Π is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every probabilistic polynomial-time $\mathcal{A}$ there is a negligible function negl with

$$\Pr[\text{Forge}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Remark 8.3 (Strong unforgeability). A subtle but important strengthening is *strong* unforgeability (sUF-CMA), in which “ (m^*, σ^*) was not among the pairs returned by the oracle” replaces the winning condition $m^* \notin Q$. Strong unforgeability forbids the adversary even from producing a *new* signature on an *already-signed* message. This matters whenever a signature itself serves as a unique token; for instance, certain anti-replay mechanisms break if signatures are malleable. Plain EUF-CMA permits a scheme in which, given one valid signature, one can cheaply compute a second, different valid signature on the same message. We note below which of these schemes are strongly unforgeable.

The remainder of the section constructs, from scratch, a concrete scheme meeting Definition 8.2 (and, with care, Remark 8.3) in an idealised model, and then enriches it with the re-randomisation and binding features the application requires.

8.2 The hash-and-sign paradigm

Before specialising to Schnorr, we record the single most important structuring idea in practical signature design, since it explains why every scheme below begins by hashing the message. Most “text-book” signature mechanisms (RSA, Schnorr’s identification-derived signature, ECDSA) are naturally defined only on inputs of a fixed, bounded size: an element of $\mathbb{Z}_N^*$ for RSA, a scalar in $\mathbb{F}_q$ for Schnorr and ECDSA alike. Real messages are arbitrarily long. The *hash-and-sign* paradigm bridges the gap with a cryptographic hash function.

Let $\Pi' = (\text{KeyGen}', \text{Sign}', \text{Verify}')$ be a scheme with message space $\mathcal{Y}_\lambda$ and let $H : \{0, 1\}^* \rightarrow \mathcal{Y}_\lambda$ be drawn from a collision-resistant family (Definition 3.6). *Hash-and-sign* defines $\text{Sign}(sk, m) = \text{Sign}'(sk, H(m))$ and $\text{Verify}(pk, m, \sigma) = \text{Verify}'(pk, H(m), \sigma)$, extending the message space to all of $\{0, 1\}^*$ at the cost of one hash. The composition preserves EUF-CMA security, by a two-case argument: a forgery on a fresh message m^* either reuses the digest of some queried message—exhibiting a collision in H —or presents a fresh digest, which is itself a forgery against Π' ; hence $\Pr[\mathcal{A} \text{ wins}] \leq \text{Adv}_H^{\text{cr}} + \text{Adv}_{\Pi'}^{\text{euf}}$.

Remark 8.4. In all the Schnorr-type schemes below, the hash that absorbs the message also absorbs the public key and the per-signature “commitment” value. Hashing the public key prevents *key-substitution* (duplicate-signature) attacks, and hashing the commitment is exactly what the Fiat-Shamir transform requires; Section 8.4 develops this carefully. Consequently, “hash-and-sign” in practice means hashing more than the message, but the composition above captures the message-compression part of the story.

8.3 The Schnorr identification protocol

An *identification protocol* lets a prover P convince a verifier V that it knows a secret associated with a public key, *without revealing the secret*. Schnorr’s protocol is the canonical example of a Σ -protocol: a

three-move, public-coin, honest-verifier-zero-knowledge proof of knowledge. We extract a signature from it in the next subsection, but we first study it in its own right, because the protocol confers every security property of the signature, almost mechanically, on the signature in turn.

The setup fixes $\mathbb{G} = \langle G \rangle$ of prime order q . The prover's secret key is a scalar $x \in \mathbb{F}_q$ and the public key is $X = [x]G \in \mathbb{G}$. The relation being proved is

$$\mathcal{R}_{\text{dl}} = \{ (X, x) \in \mathbb{G} \times \mathbb{F}_q : X = [x]G \},$$

the discrete-logarithm relation. The protocol runs as follows.

Definition 8.5 (Schnorr identification). On common input $X \in \mathbb{G}$, with the prover additionally holding x such that $X = [x]G$:

1. **Commitment.** The prover samples $r \leftarrow \mathbb{F}_q$ uniformly, computes $R = [r]G$, and sends R to the verifier. (R is the *commitment* or *nonce point*; r is the *nonce*.)
2. **Challenge.** The verifier samples $c \leftarrow \mathbb{F}_q$ uniformly and sends c .
3. **Response.** The prover computes $s = r + cx \in \mathbb{F}_q$ and sends s .

The verifier *accepts* iff

$$[s]G = R + [c]X. \quad (*)$$

We call a triple (R, c, s) satisfying $(*)$ an *accepting transcript* for X .

The three messages (R, c, s) follow the *commit–challenge–response* pattern; the first and last come from the prover, the middle is the verifier's "coin." Because the verifier's only message is a uniformly random value independent of R , the protocol is *public-coin*. We now verify the three defining properties of a Σ -protocol.

8.3.1 Perfect completeness

Proposition 8.6 (Perfect completeness). *If the prover and verifier follow the protocol of Definition 8.5 honestly, the verifier accepts with probability 1.*

Proof. With $R = [r]G$, $X = [x]G$, and $s = r + cx$, linearity of scalar multiplication gives

$$[s]G = [r + cx]G = [r]G + [cx]G = R + [c][x]G = R + [c]X,$$

which is exactly the verification equation $(*)$. The argument uses no randomness beyond what the honest parties already chose and holds identically for every choice of r and c ; hence acceptance is certain. $\square$

8.3.2 Special soundness

Completeness protects the honest prover. *Soundness* protects the verifier: a cheating prover who does *not* know x should fail. For Σ -protocols the appropriate notion is the very strong *special soundness*, which states that any two accepting transcripts that share the first message but differ in the challenge *reveal the witness*. This is the central mechanism: it makes the protocol a genuine proof of *knowledge*, and after Fiat–Shamir it becomes the source of unforgeability.

Proposition 8.7 (2-special-soundness). *There is an efficient extractor that, given two accepting tran-*

scripts (R, c, s) and (R, c', s') for the same statement X and the same commitment R but with $c \neq c'$, outputs the discrete logarithm x of X .

Proof. Both transcripts accept, so by $(*)$

$$[s]G = R + [c]X \quad \text{and} \quad [s']G = R + [c']X.$$

Subtracting (in $\mathbb{G}$) cancels R :

$$[s - s']G = [c - c']X = [(c - c')x]G.$$

Since G generates the group of prime order q , equality of $[s - s']G$ and $[(c - c')x]G$ forces $s - s' \equiv (c - c')x \pmod{q}$. As $c \neq c'$ in the field $\mathbb{F}_q$, the element $c - c'$ is invertible, and the extractor outputs

$$x = \frac{s - s'}{c - c'} \in \mathbb{F}_q.$$

One verifies $X = [x]G$ directly: $[s - s']G = [c - c']X$, and with $x = (s - s')(c - c')^{-1}$,

$$[x]G = [(c - c')^{-1}] [s - s']G = [(c - c')^{-1}] [c - c']X = X.$$

The computation is a single field inversion and is plainly efficient. $\square$

Remark 8.8 (From special soundness to a proof of knowledge). Special soundness implies the standard *knowledge soundness* guarantee. Suppose a (possibly cheating) prover convinces the verifier with probability ε noticeably larger than $1/q$ (the chance of guessing a single challenge). Rewind it: run it to obtain R , feed challenge c and record whether it answers; rewind to just after it output R and feed a fresh independent c' . A standard averaging argument (the “heavy-row” lemma, an instance of Markov’s inequality — *Math Guide*, §“Random variables and expectation”) shows that with probability roughly ε^2 one obtains two accepting transcripts sharing R with $c \neq c'$, whereupon the extractor of Proposition 8.7 recovers x . Thus knowing how to make the verifier accept with non-trivial probability is, up to rewinding, the same as knowing x : the protocol is a *proof of knowledge* of the discrete logarithm. This rewinding argument recurs, sharpened into the forking lemma, in the analysis of the signature.

8.3.3 Honest-verifier zero knowledge

Finally, the prover must leak nothing about x . The clean formalisation for public-coin protocols is *honest-verifier zero knowledge*: there is a simulator that, *without the witness*, produces transcripts identically distributed to real ones, provided the challenge is the honest (uniform) one.

Proposition 8.9 (Special honest-verifier zero knowledge). *There is an efficient simulator Sim that, on input a statement $X \in \mathbb{G}$ and a challenge $c \in \mathbb{F}_q$, outputs a transcript (R, c, s) whose distribution is identical to that of an honest execution conditioned on the challenge being c . In particular, for a uniformly random c the simulated and real transcript distributions coincide exactly.*

Proof. The simulator samples $s \leftarrow \mathbb{F}_q$ uniformly, sets

$$R := [s]G - [c]X,$$

and outputs (R, c, s) . By construction $[s]G = R + [c]X$, so $(*)$ holds and the transcript accepts. It remains to match distributions.

In a *real* honest transcript with challenge fixed to c : the prover picks $r \leftarrow \mathbb{F}_q$ uniformly, sets $R = [r]G$, and $s = r + cx$. The map $r \mapsto s = r + cx$ is a bijection of $\mathbb{F}_q$ (a translation), so s is uniform on $\mathbb{F}_q$, and $R = [s - cx]G = [s]G - [c]X$ is then a deterministic function of s . Therefore the real transcript is exactly: sample s uniform, set $R = [s]G - [c]X$. That is precisely the simulator’s output distribution. Hence the two distributions are identical, not merely statistically close. Averaging over a uniform c preserves identity of distributions, giving the final claim. $\square$

Remark 8.10 (On the restriction to *honest* verifiers). The simulator only matches the real distribution when the challenge is sampled *independently* of R (as an honest verifier does). A cheating verifier could try to choose c as a function of R ; against such a verifier the simulator, which must commit to R only after seeing c , no longer obviously works. For the present purposes honest-verifier zero knowledge is exactly sufficient, because after Fiat–Shamir a hash function applied to R produces the challenge, and in the random oracle model that hash behaves like an honest, R -independent coin. The deeper point is that the simulator’s existence already proves the transcript carries *no information* about x beyond what X itself reveals: anyone could have manufactured it.

In summary, the Schnorr identification protocol is a Σ -protocol for $\mathcal{R}_{\text{dl}}$: perfectly complete (Proposition 8.6), 2-special-sound (Proposition 8.7), and special honest-verifier zero knowledge (Proposition 8.9). We now convert these three properties into a signature.

8.4 From identification to signature via the Fiat–Shamir transform

The identification protocol is *interactive*: the verifier must contribute a fresh random challenge. A signature must be *non-interactive* and must bind to a message. The *Fiat–Shamir transform* achieves both simultaneously by replacing the verifier’s random challenge with the output of a public hash function applied to the commitment (together with the message and the public key). Heuristically, a hash function “looks random” and resists influence in advance, so it plays the role of the verifier’s coin; binding the message into the hash ties the proof to that message.

Definition 8.11 (Schnorr signature). Fix $\mathbb{G} = \langle G \rangle$ of prime order q and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{F}_q$. The Schnorr signature scheme is:

- $\text{KeyGen}(1^\lambda)$: sample $x \leftarrow \mathbb{F}_q$, set $X = [x]G$; output $(pk, sk) = (X, x)$.
- $\text{Sign}(x, m)$: sample $r \leftarrow \mathbb{F}_q$, compute $R = [r]G$, set $c = H(R \parallel X \parallel m)$, set $s = r + cx \in \mathbb{F}_q$; output $\sigma = (R, s)$.
- $\text{Verify}(X, m, (R, s))$: recompute $c = H(R \parallel X \parallel m)$; accept iff $[s]G = R + [c]X$.

Two conventions deserve comment. First, the signature transmits (R, s) ; it omits the challenge c because the verifier recomputes it. A common space-saving variant transmits (c, s) instead, and the verifier recovers $R = [s]G - [c]X$ before checking that $H(R \parallel X \parallel m) = c$; the two are interchangeable. Second, the hash takes the public key X alongside R and m . This is the key-prefixing of Remark 8.4; the bare unforgeability proof of a single fixed key does not require it, but it forecloses attacks that relate signatures under different keys and is standard in modern instantiations such as EdDSA.

Proposition 8.12 (Correctness). *The Schnorr signature scheme is perfectly correct.*

Proof. For an honestly produced $\sigma = (R, s)$ with $R = [r]G$, $c = H(R \parallel X \parallel m)$, and $s = r + cx$, verification recomputes the same c (it is a deterministic function of R, X, m) and checks $[s]G = R + [c]X$, which holds by the computation in Proposition 8.6. $\square$

The transform is generic: it converts *any* Σ -protocol into a non-interactive proof, and, when a message is folded into the hash, into a signature. We analyse the security of the result in the *random oracle model*, which we set up next.

8.5 Security in the random oracle model and the forking lemma

8.5.1 The random oracle model

The Fiat–Shamir heuristic replaces an interactive challenge with a hash. To *prove* anything we idealise the hash as a *random oracle*: a function H that every party (including the adversary and the reduction)

accesses only as a black box, and which returns, for each distinct input, an independent uniformly random output in its range, consistent across repeated queries.

Definition 8.13 (Random oracle model, recalled). We work in the random oracle model of Definition 3.12, with one generalisation: the oracle is sampled uniformly from the functions $\{0, 1\}^* \rightarrow \mathcal{Y}$ for an arbitrary finite range $\mathcal{Y}$ (the challenge space), rather than $\{0, 1\}^n$ as in Definition 3.11; the lazy-sampling realisation carries over verbatim.

The ROM remains a heuristic, with the uninstantiability caveat of Theorem 3.16 and the reading of Remark 3.17: a ROM proof rules out all attacks that treat the hash as a black box, which constitute the overwhelming majority of attacks, and it is the standard yardstick for Fiat–Shamir signatures. A reduction exploits the two powers of Proposition 3.13 (cf. Remark 1.30): *programmability*, whereby the reduction may choose the oracle’s answers on the fly, provided they appear uniform; and *observability*, whereby the reduction sees every query the adversary makes.

8.5.2 Simulation of signatures by programming the oracle

The first benefit of the ROM is that the reduction can simulate the EUF-CMA *signing oracle* without the secret key, using precisely the HVZK simulator of Proposition 8.9. This reduces forgery to a pure discrete-logarithm problem.

Lemma 8.14 (Signature simulation). *In the ROM, there is an efficient simulator that, given only the public key X and the ability to program H , answers signing queries with signatures distributed identically to genuine ones, except that it aborts on a programming collision that occurs with probability at most $(q_s)(q_s + q_h)/q$ over an attack making q_s signing and q_h hash queries.*

Proof. To sign m without x : sample $c \leftarrow \mathbb{F}_q$ and $s \leftarrow \mathbb{F}_q$ uniformly, set $R := [s]G - [c]X$ (the HVZK simulator), and program $H(R||X||m) := c$. Output (R, s) . The triple satisfies $[s]G = R + [c]X$, so it verifies, and by Proposition 8.9 its distribution matches that of a real signature. The simulation fails only if a previous hash or signing query *already defined* the point $H(R||X||m)$ to a different value, so that programming would be inconsistent. Since R is uniform in $\mathbb{G}$ of size q (it is $[s]G$ shifted by a fixed point, with s uniform), the adversary queried this particular input previously with probability at most $(q_h + q_s)/q$; union-bounding over the q_s signing queries gives the stated bound. Conditioned on no abort, the view is perfect. $\square$

8.5.3 The forking lemma

The second benefit is *extraction*. A successful forger, by definition, produces a fresh accepting transcript (R, c, s) where the random oracle tied c to $R||X||m^*$. If the reduction rewinds the forger and *re-programs* the oracle to answer that one critical query with a fresh challenge c' , then by special soundness two successful runs yield x . Making this precise, and quantifying the probability that the second run also succeeds and is “compatible” with the first, is the content of the *forking lemma*. We state the general (game-based) version due to Bellare and Neven and then apply it.

Lemma 8.15 (General forking lemma). *Fix an integer $Q \geq 1$, a set $\mathcal{C}$ of size $|\mathcal{C}| = h \geq 2$, and a randomised input generator Gen . Let $\mathcal{B}$ be a randomised algorithm that, on input y and elements $c_1, \dots, c_Q \in \mathcal{C}$, returns a pair (I, out) where $I \in \{0, 1, \dots, Q\}$ is an index (with $I = 0$ meaning “failure”). Let*

$$\text{acc} = \Pr[I \geq 1 : y \leftarrow \text{Gen}, c_1, \dots, c_Q \leftarrow \mathcal{C}, (I, \text{out}) \leftarrow \mathcal{B}(y, c_1, \dots, c_Q)]$$

be $\mathcal{B}$ ’s success probability. Consider the forking algorithm $F_{\mathcal{B}}(y)$: pick random coins ρ for $\mathcal{B}$ and

$c_1, \dots, c_Q \leftarrow \mathcal{C}$; $\text{run}(I, \text{out}) \leftarrow \mathcal{B}(y, c_1, \dots, c_Q; \rho)$; if $I = 0$ return $\perp$; otherwise resample $c'_I, \dots, c'_Q \leftarrow \mathcal{C}$ afresh and $\text{run}(I', \text{out}') \leftarrow \mathcal{B}(y, c_1, \dots, c_{I-1}, c'_I, \dots, c'_Q; \rho)$ with the same coins ρ ; if $I' = I$ and $c_I \neq c'_I$, return $(I, \text{out}, \text{out}')$, else return $\perp$. Define $\text{frk} = \Pr[F_{\mathcal{B}}(y) \neq \perp : y \leftarrow \text{Gen}]$, the probability taken also over $F_{\mathcal{B}}$'s coins. Then

$$\text{frk} \geq \text{acc} \left(\frac{\text{acc}}{Q} - \frac{1}{h} \right).$$

Proof. Sketch. Condition on the random coins ρ and on the prefix $c_1, \dots, c_{I-1}$ up to the forking point. For each fixed value of these, the single “pivotal” challenge c_I (the value returned at the queried index) together with the suffix determines $\mathcal{B}$'s success; abstract this as a function of c_I only by also averaging over the suffix, giving for each prefix a set S of “good” values of $c_I \in \mathcal{C}$ on which $\mathcal{B}$ succeeds with index I . The forking algorithm succeeds at this prefix iff both the first draw c_I and the independent re-draw c'_I land in S and differ. Two independent uniform draws from $\mathcal{C}$ both land in S with probability $(|S|/h)^2$, and subtracting the diagonal (equal draws) costs at most $|S|/h \cdot 1/h$; so the per-prefix fork probability is at least $(|S|/h)^2 - |S|/h^2 = (|S|/h)(|S|/h - 1/h)$. Writing $p = |S|/h$ for the prefix's success probability and taking expectations over prefixes, Jensen's inequality applied to $p \mapsto p^2$ (convexity) gives $\mathbb{E}[p^2] \geq (\mathbb{E}[p])^2 = \text{acc}^2$; accounting for the index I ranging over Q possibilities introduces the factor $1/Q$, turning this into acc^2/Q . Collecting terms yields $\text{frk} \geq \text{acc}^2/Q - \text{acc}/h$, which is the stated bound. The full argument carefully books these conditional expectations; the convexity step is where the squared probability—and hence the tightness loss—appears. $\square$

Theorem 8.16 (EUF-CMA security of Schnorr signatures). *Model H as a random oracle. If there is an adversary $\mathcal{A}$ running in time t , making at most q_h random-oracle queries and q_s signing queries, that forges a Schnorr signature with probability ε , then there is an algorithm $\mathcal{D}$ running in time $\approx 2t$ that solves the discrete-logarithm problem in $\mathbb{G}$ with probability at least*

$$\varepsilon' \geq \frac{\varepsilon^2}{q_h + q_s + 1} - \frac{(q_h + q_s + 1)}{q} - \frac{q_s(q_h + q_s)}{q}.$$

Consequently, if the discrete-logarithm problem in $\mathbb{G}$ is hard, the Schnorr signature scheme is EUF-CMA in the random oracle model.

Proof. Transform $\mathcal{A}$ into an algorithm $\mathcal{B}$ of the shape required by Lemma 8.15, with input $y = X$ (the discrete-log challenge $X = [x]G$, x unknown; here Gen is the discrete-log instance generator outputting $X = [x]G$ for uniform $x \leftarrow \mathbb{F}_q$) and challenge list $c_1, \dots, c_Q$ where $Q = q_h + q_s + 1$ bounds the number of distinct hash inputs that can become the “relevant” challenge.

Running $\mathcal{B}$. On input X and the list (c_i) , $\mathcal{B}$ runs $\mathcal{A}(X)$. It maintains the random-oracle table and answers $\mathcal{A}$'s j -th distinct hash query by handing out c_j from the list (so the oracle's answers are the externally supplied, uniformly random c_j). The simulator of Lemma 8.14 answers signing queries, programming the oracle as needed; if a programming collision occurs $\mathcal{B}$ aborts (this is the $q_s(q_h + q_s)/q$ loss). When $\mathcal{A}$ halts with a valid forgery $(m^*, (R^*, s^*))$, let $c^* = H(R^* || X || m^*)$. Because the forgery is fresh and valid, with overwhelming probability $\mathcal{A}$ actually queried the value c^* to the oracle during the run rather than guessing blindly: if $\mathcal{A}$ never queried $R^* || X || m^*$, the value c^* is uniform and unseen by $\mathcal{A}$, so $[s^*]G = R^* + [c^*]X$ holds for the fresh forgery only with probability $1/q$ (the $(q_h + q_s + 1)/q$ term collects this and similar guessing events). Let I be the index in the list at which the oracle answered this critical query; $\mathcal{B}$ outputs (I, out) with $\text{out} = (R^*, c^*, s^*, m^*)$. Thus $\text{acc} \geq \varepsilon - (\text{negligible loss terms})$.

Forking. Apply $F_{\mathcal{B}}$. The two runs use the same coins ρ and identical challenges $c_1, \dots, c_{I-1}$ up to the I -th query; because ρ and that shared prefix determine the prover's behaviour up to and including producing the commitment R^* , both runs query the same critical input $R^* || X || m^*$ at index I , but receive different answers $c_I = c^* \neq c'^* = c'_I$. When forking succeeds, it yields two accepting transcripts (R^*, c^*, s^*) and (R^*, c'^*, s'^*) , with the same commitment and different challenges.

Extraction. Feed these to the special-soundness extractor of Proposition 8.7:

$$x = \frac{s^* - s'^*}{c^* - c'^*} \in \mathbb{F}_q,$$

which is the discrete logarithm of X . Hence $\mathcal{D}$ solves the instance whenever forking succeeds, so $\varepsilon' \geq \text{frk}$. Substituting $\text{acc} \geq \varepsilon - O((q_h + q_s)/q) - q_s(q_h + q_s)/q$ and $h = q$, $Q = q_h + q_s + 1$ into Lemma 8.15's bound $\text{frk} \geq \text{acc}^2/Q - \text{acc}/h$ and simplifying yields the displayed inequality. $\square$

Remark 8.17 (The tightness loss). The dominant term of the bound is $\varepsilon' \gtrsim \varepsilon^2/q_h$. The reduction is therefore *not tight*: extracting the discrete logarithm requires running the forger *twice* and incurs a *squaring* of the success probability, divided by the number of hash queries. Concretely, if a forger succeeds with probability $\varepsilon = 2^{-40}$ after $q_h = 2^{60}$ hash queries, the reduction only guarantees a discrete-log solver of advantage about $2^{-80}/2^{60} = 2^{-140}$, far weaker than ε . To compensate, one must choose group parameters offering more bits of discrete-log security than the target signature-security level, roughly doubling the security margin demanded of $\mathbb{G}$. This quadratic loss is intrinsic to rewinding-based Fiat–Shamir proofs; tight reductions are known only under stronger assumptions (e.g. the one-more discrete log or DDH with key-prefixing) or in idealised algebraic models. The loss is the cost of converting an *interactive* proof of knowledge into a *non-interactive* signature by forking.

Remark 8.18 (Nonce reuse is fatal). Special soundness has a dark mirror. If a signer ever produces two signatures (R, s) and (R, s') on *different* messages $m \neq m'$ with the *same* commitment R — for instance by reusing the nonce r , or by drawing r from a biased source — then the challenges $c = H(R \| X \| m)$ and $c' = H(R \| X \| m')$ differ, and *anyone* can apply the extractor of Proposition 8.7 to recover the secret key $x = (s - s')/(c - c')$. Security therefore rests entirely on the nonce being fresh and unpredictable for every signature. This motivates *deterministic* nonce generation, the defining idea of EdDSA below.

8.6 RedDSA: re-randomisable Schnorr for Orchard

We reach the variant used in Zcash by way of its immediate ancestor. *EdDSA* (the Edwards-curve Digital Signature Algorithm) is the engineering refinement of Schnorr that modern systems standardise. It retains the algebra of Definition 8.11 and hardens its two implementation surfaces: the *nonce*, which EdDSA derives deterministically as a hash of a secret prefix and the message—so that no faulty random-number generator can ever repeat a nonce and leak the key (Remark 8.18)—and the *challenge hash*, which it key-prefixes by absorbing the public key alongside R and m (Remark 8.4). In the random oracle model the deterministic nonce is, to anyone ignorant of the secret prefix, a fresh uniform value per message, so the unforgeability proof of Theorem 8.16 carries over essentially unchanged.

RedDSA is EdDSA generalised in two directions: the base point is a *parameter* of the scheme (so that different uses can sign with respect to different generators), and the key may be *re-randomised*. The first generalisation is cosmetic but essential for the binding-signature application; the second is the crux of Orchard's privacy.

Definition 8.19 (RedDSA, schematic). RedDSA is parameterised by a generator $P \in \mathbb{G}$ of the prime-order group. KeyGen samples $x \leftarrow \mathbb{F}_q$ and sets the public key $X = [x]P$. To sign m under x :

1. sample a nonce r (in Zcash, $r := H(T \| X \| m)$ for a fresh random byte string T , i.e. *hedged-deterministic*);
2. $R := [r]P$;
3. $c := H(R \| X \| m) \in \mathbb{F}_q$;
4. $s := r + c x$; output $\sigma = (R, s)$.

Verification accepts iff $[s]P = R + [c]X$. Orchard instantiates RedDSA twice, both times as RedPallas—RedDSA over the Pallas curve—and the two instantiations differ *only in the base point*: the *spend-authorisation* base point for the re-randomisable signature, and the *value-commitment randomness* base point for the binding signature below. (The curve varies only across protocol generations: different curves is what distinguishes Sapling’s RedJubjub, over Jubjub, from Orchard’s RedPallas.)

Functionally RedDSA is, for a fixed base point, the Schnorr/EdDSA scheme of the previous subsections, and its EUF-CMA security in the ROM is again Theorem 8.16 with G replaced by P . The novelty is the re-randomisation operation, which we treat next.

8.7 Key re-randomisation and unlinkability

In a shielded transaction the spender must prove authority to spend a note. If the same long-term public key appeared on chain each time, an observer could link all spends by a given key, destroying privacy. Orchard’s solution is to publish, for each spend, a *fresh re-randomised* public key rk derived from the long-term pk by a random additive shift, to sign with the correspondingly shifted secret key, and to prove in zero knowledge that rk is a valid re-randomisation of an authorised pk . The signature scheme must therefore support re-randomisation *algebraically*, and the re-randomised keys must be *unlinkable*.

Definition 8.20 (Key re-randomisation). Fix the base point P . Given a secret key x with public key $X = [x]P$ and a *randomiser* $\alpha \in \mathbb{F}_q$, define

$$rsk = x + \alpha \in \mathbb{F}_q, \quad rk = X + [\alpha]P \in \mathbb{G}.$$

These are the re-randomised signing and verification keys.

Proposition 8.21 (Re-randomisation is consistent). *The pair (rsk, rk) of Definition 8.20 is a valid RedDSA key pair, i.e. $rk = [rsk]P$. Consequently a RedDSA signature produced under rsk verifies under rk .*

Proof. Directly,

$$[rsk]P = [x + \alpha]P = [x]P + [\alpha]P = X + [\alpha]P = rk,$$

using linearity of scalar multiplication. Thus (rsk, rk) is exactly a key pair of the form KeyGen produces, and signing/verification correctness (Proposition 8.12) applies verbatim. $\square$

The additive structure is what makes this work: because the public key is a *homomorphic* image $[\cdot]P$ of the scalar, shifting the secret by α shifts the public key by $[\alpha]P$, a quantity computable from α and P alone, *without* the secret key. This is precisely why a verifier (or a circuit) can take rk and α and check the relationship, while only the spender, knowing x , can sign.

We now formalise the privacy guarantee. The randomiser α is sampled *uniformly and freshly* for each use and kept secret (in Orchard it is derived inside the spend and revealed only to the circuit). The claim is that rk then carries *no information whatsoever* about which long-term key it came from.

Proposition 8.22 (Perfect unlinkability). *Fix the base point P . For any public key $X \in \mathbb{G}$, if $\alpha \leftarrow \mathbb{F}_q$ is uniform, then $rk = X + [\alpha]P$ is uniformly distributed over $\mathbb{G}$. Consequently, for any two public keys $X_0, X_1 \in \mathbb{G}$, the distributions of rk obtained by re-randomising X_0 and by re-randomising X_1 with a fresh uniform randomiser are identical. An adversary, even one of unbounded computational power and even one who chose X_0, X_1 , cannot distinguish which key a given rk re-randomises with*

probability better than $1/2$.

Proof. Since P generates $\mathbb{G}$ of order q , the map $\alpha \mapsto [\alpha]P$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$; hence $[\alpha]P$ is uniform over $\mathbb{G}$ when α is uniform over $\mathbb{F}_q$. Translation by the fixed element X is a bijection of the group $\mathbb{G}$, and a bijection carries the uniform distribution to the uniform distribution; therefore $rk = X + [\alpha]P$ is uniform over $\mathbb{G}$, *independently of* X . The distribution of rk is thus the uniform distribution on $\mathbb{G}$ regardless of which X one used, so re-randomisations of X_0 and of X_1 are identically distributed. No test can tell two identical distributions apart; the best any distinguisher can do on the resulting indistinguishability game is to guess, succeeding with probability exactly $1/2$. $\square$

Remark 8.23 (What unlinkability does and does not give). Proposition 8.22 is an *information-theoretic* (perfect) statement: it does not rest on any computational assumption. It states, however, only that the re-randomised *key* leaks nothing; the full unlinkability of a spend further requires that the accompanying signature and zero-knowledge proof leak nothing beyond their statements, that the randomiser α stay secret, and that no other transaction field correlate two spends. In Orchard, the protocol publishes rk while proving inside the Halo 2 circuit the witness that it equals $X + [\alpha]P$ for an *authorised* X ; the verifier checks the spend authorisation signature against the public rk . The signer must of course know $rsk = x + \alpha$, which requires knowing both x and α ; thus only the legitimate key-holder, in possession of α for this spend, can sign under rk . One must also verify that signing under re-randomised keys does not itself harm unforgeability. A forgery (R, s) under rk satisfies $[s]P = R + [c]rk$ with $c = H(R||rk||m)$, and subtracting $[\alpha]P$ from the verification equation gives $[s - c\alpha]P = R + [c]X$ — but this identity holds only with $c = H(R||rk||m)$, whereas the base verifier recomputes $H(R||X||m)$, so the plain subtraction converts forgeries only for the non-key-prefixed variant of Schnorr. For the key-prefixed scheme of Definition 8.19 one instead proves the security of signing under re-randomised keys in the ROM by either (i) observing that $(rsk, rk) = (x + \alpha, X + [\alpha]P)$ is itself a base-scheme key pair whose public key is uniform (Proposition 8.22), so a forger against rk is literally a forger against the base scheme at the key rk , and the reduction, which knows α , recovers x from the extracted rsk as $x = rsk - \alpha$; or (ii) when the adversary sees signatures under several randomisers α_i , by re-running the forking argument of Theorem 8.16 on the critical query $H(R^*||rk||m^*)$ — simulating signing under the remaining keys by the oracle-programming of Lemma 8.14 — to extract rsk and hence x . (A naive ROM translation that programs $H(R||rk||m) := H_{\text{base}}(R||X||m)$ must be a careful injective domain swap and fails outright with multiple randomisers, since distinct rk_i -prefixed inputs would collide on one X -prefixed point, a detectable deviation from a random oracle.)

8.8 Binding signatures as a proof of knowledge of a discrete logarithm

The final piece is the *binding signature*, an unusually elegant use of a Schnorr signature in which the “message-authenticating” role is incidental and the substantive task is to prove an algebraic balance: that the values flowing into a transaction equal the values flowing out. The construction arranges so that the binding-signature *verification key is itself a commitment* that the verifier can recompute, and that key equals $[\cdot]P$ of a secret the signer can know only if the transaction balances. A valid signature is then exactly a proof of knowledge of that discrete logarithm.

We sketch the value-commitment setup that motivates the construction. The shielded protocols commit to value with a homomorphic Pedersen commitment

$$\text{cm}(v, \rho) = [v]V + [\rho]P,$$

where V and P are two independent generators of $\mathbb{G}$ (independent meaning that no party knows the discrete log of one to base the other) and ρ is a random blinding scalar. Homomorphism is the essential property: the product (here, group sum) of commitments commits to the sum of values with the sum of blindings. Sapling commits to each note’s value separately: a transaction carries input value commitments $\text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}})$ and output value commitments $\text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}})$, and the *net commitment* is

$$B = \sum_i \text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}}) - \sum_j \text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}}).$$

Orchard has *no* per-note value commitments: each Action commits once to its *net* value change, publishing

$$cv^{\text{net}} = cm(v^{\text{old}} - v^{\text{new}}, \rho) = [v^{\text{old}} - v^{\text{new}}]V + [\rho]P$$

for the spent value v^{old} and created value v^{new} of that Action (*Ironwood Guide*), and the net commitment is the plain sum $B = \sum_i cv^{\text{net},i}$ over the transaction's Actions. In either case, by homomorphism,

$$B = [\Delta v]V + [\bar{\rho}]P,$$

where Δv is the net value ($\sum_i v_i^{\text{in}} - \sum_j v_j^{\text{out}}$ in Sapling, $\sum_i (v_i^{\text{old}} - v_i^{\text{new}})$ in Orchard) and $\bar{\rho}$ is the corresponding net blinding. The transaction *balances* (no value created or destroyed, modulo a public fee) precisely when the net value Δv equals the public balance v^{bal} . Suppose, for clarity of the core idea, that the transaction is internally balanced so that $\Delta v = 0$. Then the V -component vanishes and

$$B = [\bar{\rho}]P.$$

That is: the net commitment B is a known multiple of the single generator P if and only if the values balance, and the multiplier $\bar{\rho}$ is the net blinding factor — a quantity the signer can compute (it knows all the ρ 's) but that resists production otherwise, since producing any P -multiple representation of an *unbalanced* B would require knowing the discrete log of V to base P , which we assume hard.

Definition 8.24 (Binding signature). With B and $\bar{\rho}$ as above (assuming balance, so $B = [\bar{\rho}]P$), set the *binding verification key* $bvk := B$ and the *binding signing key* $bsk := \bar{\rho}$. The binding signature is a RedDSA signature (Definition 8.19) over base point P , signing key $\bar{\rho}$, verification key B , on a message that is the hash of the transaction (the *sighash*). The verifier recomputes B from the publicly listed value commitments and the public balance, then checks the RedDSA equation $[s]P = R + [c]B$ with $c = H(R||B||\text{sighash})$.

Proposition 8.25 (The binding signature is a proof of knowledge of $\log_p B$). *A valid binding signature on a given transaction is, in the random oracle model, a proof of knowledge of the discrete logarithm of the net commitment B to base P . Hence a valid binding signature certifies that the producer knew a scalar $\bar{\rho}$ with $B = [\bar{\rho}]P$; combined with the soundness of the accompanying zero-knowledge proofs (per Action in Orchard, per note in Sapling), this forces the transaction to balance.*

Proof. The binding signature is exactly a Fiat–Shamir-transformed Schnorr proof for the statement “ $B \in \langle P \rangle$ with known discrete log,” i.e. for the relation $\mathcal{R}_{\text{dl}}$ with base P and statement B . By the special soundness of Proposition 8.7 together with the forking argument of Theorem 8.16 (specialised to a *single* statement and no signing oracle), the forking lemma can rewind any algorithm that produces a valid signature on B with non-negligible probability to yield two accepting transcripts $(R, c, s), (R, c', s')$ with $c \neq c'$, from which the extractor recovers $\bar{\rho} = (s - s')/(c - c')$ with $B = [\bar{\rho}]P$. Thus producing a valid binding signature is equivalent, up to the standard rewinding loss, to knowing $\log_p B$.

It remains to argue that knowing $\log_p B$ forces balance. Write $B = [\Delta v]V + [\bar{\rho}']P$ for the *true* net value Δv and net blinding $\bar{\rho}'$ the commitments imply (well-defined once the accompanying proofs fix the committed openings: in Orchard the per-Action circuit fixes $v^{\text{old}}, v^{\text{new}}$ and the opening of cv^{net} ; in Sapling the per-note proofs fix each v_i, ρ_i). If the signer knows some $\bar{\rho}$ with $B = [\bar{\rho}]P$, then subtracting,

$$[\Delta v]V = [\bar{\rho} - \bar{\rho}']P,$$

so $V = [(\bar{\rho} - \bar{\rho}')(\Delta v)^{-1}]P$ whenever $\Delta v \neq 0$ (using that Δv is invertible in $\mathbb{F}_q$ for a nonzero field element). This exhibits the discrete logarithm of V to base P , contradicting the assumed independence of the generators V, P . Therefore $\Delta v = 0$: the transaction balances. (In the deployed scheme, signing against $B - [v^{\text{bal}}]V$ folds in the public balance v^{bal} ; the same argument then forces $\Delta v = v^{\text{bal}}$.) $\square$

Remark 8.26 (Why a signature and not “just” a proof). A *single* group element pair (R, s) accomplishes three things. First, it proves knowledge of $\bar{\rho}$, which (Proposition 8.25) certifies balance. Second, because the Fiat–Shamir challenge hashes the transaction *sighash*, the very same signature *authenticates the transaction*: altering any signed field changes c and invalidates (R, s) , so the binding signature doubles as an integrity check binding all the shielded components together (this is the origin of the name “binding”). Third, the construction requires *no trusted setup and no separate circuit* for the balance check — it is pure elliptic-curve arithmetic that anyone can verify. The economy is striking: the homomorphism of the commitment turns the arithmetic statement “values balance” into the algebraic statement “ B is a known multiple of P ,” and the Schnorr signature is the off-the-shelf proof of knowledge for that statement. The binding signature is, in this sense, the purest illustration in the entire protocol of the principle that a digital signature *is* a proof of knowledge of a discrete logarithm.

Remark 8.27 (Relation to the spend authorisation signature). The two RedDSA uses merit contrast. The *spend authorisation* signature (Section 8.7) uses a *re-randomised* key $rk = X + [\alpha]P$ to authorise a spend while hiding which long-term key authorised it; here the signing key $rsk = x + \alpha$ encodes the spender’s authority and α provides unlinkability. The *binding* signature uses a key $B = [\bar{\rho}]P$ that the spender does *not* choose but the transaction’s commitments *force*; here the “secret key” $\bar{\rho}$ is the net blinding factor and signing it proves balance. Both consist of the same three lines of Schnorr arithmetic; what differs is the *meaning assigned to the discrete logarithm* — authority in one case, value-conservation in the other. This is the unifying lesson of the section: once a public key is the image of a secret under $x \mapsto [x]P$, a Schnorr signature is a compact, non-interactive proof that one knows that secret, and the designer is free to choose what knowing the secret *means*.

9 Interactive proofs, zero knowledge, and SNARKs

The primitives of the preceding sections—commitments, hash functions, signatures—let one party convince another of the integrity or authenticity of a *fixed* message. We now pose a more ambitious question. Suppose a party, the *prover*, knows that a certain statement is true: that a number is composite, that a graph is three-colourable, that she holds a discrete logarithm, that a long computation halts in an accepting state. Can she convince a sceptical *verifier* of this fact? Can she do so without revealing *why* it is true—without leaking the factorisation, the colouring, the discrete logarithm, the computation transcript? And can she do so with a proof so short and so cheaply checked that it is dwarfed by the statement it certifies?

These three questions—convincing, hiding, and succinctness—are the subject of this section. The theory of interactive proofs and arguments of knowledge, zero knowledge, and succinct non-interactive arguments of knowledge (SNARKs) answer them respectively. The development culminates in the architecture that underlies the Halo 2 proving system: a polynomial interactive oracle proof compiled by a polynomial commitment scheme and made non-interactive by the Fiat–Shamir transform. We build the entire edifice from the ground up.

Throughout, n denotes a security parameter; “probabilistic polynomial time” (PPT) means a randomised algorithm whose running time is bounded by a fixed polynomial in the length of its input (and in n when n is supplied in unary as 1^n); and $\text{negl}(n)$ denotes a *negligible* function, one that eventually drops below $1/n^c$ for every constant c . We use freely the asymptotic and probabilistic language of the companion volume.

9.1 Languages, relations, and witnesses

The objects a prover proves things about are membership claims for a language, or—more usefully for cryptography—claims that one knows a witness for an instance.

Definition 9.1 (Language and relation). Fix a finite alphabet, say $\{0, 1\}$. A *language* is a set $L \subseteq \{0, 1\}^*$ of bit strings. A *binary relation* is a set $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ of pairs (x, w) ; we call x the *instance* (or *statement*) and w a *witness* for x . The relation *induces* the language

$$L_R = \{x \in \{0, 1\}^* : \exists w \text{ with } (x, w) \in R\}.$$

We write $R(x) = \{w : (x, w) \in R\}$ for the (possibly empty) *witness set* of x . The relation R is an *NP-relation* if it is *polynomially balanced*—there is a polynomial p with $|w| \leq p(|x|)$ for all $(x, w) \in R$ —and *polynomial-time decidable*: a deterministic algorithm, given (x, w) , decides whether $(x, w) \in R$ in time polynomial in $|x|$. By a foundational theorem of complexity theory, $L \in \text{NP}$ if and only if $L = L_R$ for some NP-relation R .

Example 9.2 (Three running examples). The following relations recur throughout.

1. *Discrete logarithm*. Fix a cyclic group $\mathbb{G}$ of prime order q with generator g . Set $R_{\text{dl}} = \{(h, w) : h = g^w, w \in \mathbb{Z}_q\}$. The instance $h \in \mathbb{G}$ always has a witness (the exponent exists), so the *language* is trivial; what is hard is to *produce* w . This is the prototype for which the distinction between proving membership and proving knowledge becomes essential (Subsection 9.3).
2. *Graph three-colouring*. $R_{\text{3col}} = \{(G, c) : c \text{ is a proper 3-colouring of the graph } G\}$. The induced language L_{3col} is NP-complete, so a (zero-knowledge) protocol for it yields one for every NP language by reduction.
3. *Circuit satisfiability*. $R_{\text{sat}} = \{(C, w) : C \text{ is a Boolean (or arithmetic) circuit and } C(w) = 1\}$. This is the universal target of practical proof systems: one first “arithmetises” arbitrary computations into satisfiability of an arithmetic circuit over a finite field $\mathbb{F}$, and the prover proves she knows a satisfying assignment.

The discrete-logarithm example already shows why “ $x \in L$ ” is too weak a target. Every group element is *some* power of g , so the membership language is all of $\mathbb{G}$ and proving membership is vacuous. The content lies in proving that the prover *knows* the exponent. Formalising “knows” is the central conceptual move of the section.

9.2 Interactive protocols, proofs, and arguments

Definition 9.3 (Interactive protocol). An *interactive protocol* is a pair (P, V) of algorithms, the *prover* P and the *verifier* V , that exchange messages. Both receive a common input x ; P additionally receives a private input (its witness or auxiliary string) w , and each holds its own private random tape. They alternate sending messages $a_1, a_2, \dots$ over a number of *rounds* that is polynomial in $|x|$. After the last message, V outputs a single bit: 1 (*accept*) or 0 (*reject*). We write

$$\langle P(w), V \rangle(x) \in \{0, 1\}$$

for the verifier’s output, a random variable over the parties’ coins, and call the full sequence of exchanged messages (together with x and V ’s coins, when relevant) the *transcript*. The protocol is *public-coin* if every message V sends is a fresh uniformly random string that V also reveals—equivalently, V ’s messages are its random coins—and V ’s final decision is a deterministic function of the transcript. We always require V to run in probabilistic polynomial time.

The verifier is the party we must protect; its resources are therefore fixed at PPT once and for all. The prover is the party whose power we vary, and that variation is exactly the difference between a *proof* and an *argument*.

Definition 9.4 (Interactive proof; interactive argument). Let R be a relation with language $L = L_R$, and let (P, V) be an interactive protocol. We say (P, V) is an *interactive proof system* for L with completeness error $c(\cdot)$ and soundness error $s(\cdot)$ if:

Completeness. For every $x \in L$ (with some witness w),

$$\Pr[\langle P(w), V \rangle(x) = 1] \geq 1 - c(|x|).$$

Soundness. For every $x \notin L$ and every prover strategy P^* —of *unbounded* computational power, with arbitrary auxiliary input z —

$$\Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|).$$

We require $c(|x|) + s(|x|) \leq 1 - 1/\text{poly}(|x|)$ (the two thresholds must be bounded apart); usually c and s are negligible. The collection of languages with such proofs is the complexity class IP .

(P, V) is an *interactive argument system* (also called a *computationally sound proof*) for L if completeness holds as above, and soundness holds only against cheating provers P^* that are PPT:

$$x \notin L, \quad P^* \text{ PPT} \implies \Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|).$$

Here soundness typically rests on a computational hardness assumption, and s must be negligible for all PPT P^* .

Remark 9.5 (Proofs versus arguments: which is stronger?). The two notions trade soundness strength against other resources, and neither dominates the other across all axes.

- A *proof* resists a computationally unbounded adversary: no amount of computation lets a cheating prover certify a false statement except with probability s . This is an information-theoretic, unconditional guarantee.

- An *argument* resists only *efficient* adversaries. A cheating prover with unlimited time can typically break it—e.g. by breaking the binding of a commitment or computing a discrete logarithm by brute force. In return, arguments can achieve properties *provably impossible* for proofs, most importantly *succinctness* (Subsection 9.8): a sound interactive proof for an NP-complete language with communication much smaller than the witness would collapse complexity classes that are believed distinct, whereas succinct *arguments* exist under standard assumptions.

For cryptographic applications, including everything in the SNARK literature, the prover is a real machine and the argument notion is exactly the right one. “Soundness against a bounded prover” is not a weakness to apologise for; it is the price of, and the gateway to, succinctness.

Remark 9.6 (Why interaction and randomness are essential). If the verifier were deterministic and the protocol non-interactive (a single prover message read and checked), the system would prove exactly the languages decidable in polynomial time with a certificate, i.e. NP, and would offer no hope of zero knowledge: the certificate is the witness in spirit, and V could simply replay P ’s analysis. Randomness in V ’s challenges is what forces a cheating prover to commit before it knows what it will be asked, and interaction is what lets V adaptively probe. A celebrated line of results shows that interaction buys real power: $IP = PSPACE$, the class of languages decidable with polynomial memory and unrestricted time, believed to reach far beyond NP. For *this* section the relevant point is humbler but sharper—interaction and randomness are what make zero knowledge and knowledge extraction possible at all.

Soundness amplification by repetition. Repetition upgrades a protocol with a constant soundness error such as $1/2$ to negligible error. Running k *independent* instances and accepting only if all accept drives soundness error from s to s^k for proofs (sequential repetition always works; parallel repetition works for proofs and, with care, for many arguments), while completeness error grows only to $\leq k \cdot c$ by a union bound. Taking $k = \omega(\log |x|)$, e.g. $k = |x|$, makes s^k negligible. We use this freely, and design base protocols aiming merely for a noticeable soundness gap.

9.3 Knowledge soundness and extractors

Return to R_{dl} . Soundness in the sense of Definition 9.4 asserts nothing: every $h \in \mathbb{G}$ lies in the language, so the verifier may always accept and the soundness condition (quantified over $x \notin L$) holds vacuously. We seek instead the assurance that a convincing prover *possesses* the discrete logarithm. “Possesses” admits an operational reading: if the prover holds the witness, then an efficient procedure granted full control over the prover can *compute* the witness from it. That procedure is the *extractor*, and the access it requires is the ability to *rewind*.

Definition 9.7 (Knowledge soundness; proof/argument of knowledge). Let R be a relation and (P, V) a protocol with the completeness property for R . The protocol (P, V) is a *proof of knowledge* for R with *knowledge error* $\kappa(\cdot)$ if there exists a probabilistic algorithm E , the (*knowledge*) *extractor*, and a polynomial q , such that for every (possibly cheating, possibly unbounded) prover P^* and every instance x , the following holds. Let

$$\varepsilon(x) = \Pr[\langle P^*, V \rangle(x) = 1]$$

be the probability that P^* makes V accept. Then E , given x and *oracle access to P^* with the ability to rewind it*—that is, to run P^* from any point with the same coins and feed it verifier messages of E ’s choosing—outputs a witness $w \in R(x)$ within an expected number of steps bounded by

$$\frac{q(|x|)}{\varepsilon(x) - \kappa(x)} \quad \text{whenever } \varepsilon(x) > \kappa(x).$$

The protocol (P, V) is then a *knowledge-sound* protocol; if soundness need hold only against PPT P^* , it is an *argument of knowledge*.

The definition warrants careful reading. The extractor’s running time may grow as the prover’s success probability ε shrinks toward the knowledge error κ : a prover that barely succeeds is correspondingly hard to extract from, and a prover that succeeds with probability at most κ need not yield anything at all. Yet *any* prover that succeeds noticeably above κ surrenders the witness to an efficient extractor. This is precisely the formal content of “a convincing prover knows the witness”: we define knowledge as efficient computability *relative to the prover’s own code and coins*.

Remark 9.8 (Rewinding, and the justification for assuming it). The extractor is a thought experiment, not a runtime procedure: it never executes during an honest protocol run. It exists only to give meaning to the security claim. Its power—to clone the prover’s state and replay it under different challenges—is legitimate precisely because in the security *reduction* the extractor *is* the reduction, and it holds the cheating prover’s code in hand as a subroutine. The same rewinding ability that the simulator employs to fake transcripts (Subsection 9.4) is what the extractor uses to mine real witnesses; rewinding is the single most important technical device in this entire theory.

Proposition 9.9 (Knowledge soundness implies soundness). *If (P, V) is a proof of knowledge for R with knowledge error κ , then it is an interactive proof for L_R with soundness error $s(x) \leq \kappa(x)$ for x of each length.*

Proof. Let $x \notin L_R$, so $R(x) = \emptyset$, and suppose for contradiction that some prover P^* achieves $\varepsilon(x) > \kappa(x)$. By Definition 9.7 the extractor $E^{P^*}(x)$ halts in finite expected time with output $w \in R(x)$. But $R(x)$ is empty, so E cannot output any such w —a contradiction. Hence $\varepsilon(x) \leq \kappa(x)$ for every P^* , which is precisely soundness with error κ . $\square$

The converse fails: a protocol can be sound (it accepts no false statement) yet not a proof of knowledge (one may convince the verifier of a *true* statement without knowing why). Knowledge soundness is strictly stronger, and it is the notion on which every cryptographic application actually relies. The Σ -protocols introduced below make this concrete, for there extraction takes its cleanest form.

9.4 The simulation paradigm and zero knowledge

We now require the protocol to leak nothing. The difficulty lies in specifying what “nothing” means: the verifier already learns that $x \in L$, and it observes a transcript of messages, so it certainly learns *something*. The resolution—the *simulation paradigm*, the most influential idea in modern cryptography—declares that the verifier learned nothing beyond the truth of the statement if it could have produced, entirely on its own, a transcript indistinguishable from the real one. Anything the verifier can manufacture without the prover, it did not learn from the prover.

We recall the three grades of “indistinguishable” first. Recall the statistical distance and the three grades of indistinguishability — perfect ($\equiv$), statistical ($\approx_s$), and computational ($\approx_c$) — of Definitions 1.8 and 1.12, together with Propositions 1.9 and 1.13. One adaptation is needed here: the ensembles are indexed not by the security parameter but by instances x (and auxiliary inputs), with negligibility measured in $n = |x|$; the definitions and the hierarchy carry over verbatim under this re-indexing.

Definition 9.10 (Zero knowledge). Let (P, V) be an interactive proof or argument for $L = L_R$. For a verifier strategy V^* , instance x , and auxiliary input z to V^* , let

$$\text{View}_{V^*}(P(w), V^*(z))(x)$$

denote the *view* of V^* : the random variable consisting of x , V^* 's random tape, and every message V^* receives during the interaction (with P running on witness w). The protocol is *zero knowledge* if there exists a PPT algorithm Sim , the *simulator*, such that for every $x \in L$ (with witness w) and every auxiliary z ,

$$\{\text{Sim}(x, z)\} \text{ is indistinguishable from } \{\text{View}_{V^*}(P(w), V^*(z))(x)\}.$$

According to the grade of indistinguishability (Definition 1.12) one speaks of *perfect*, *statistical*, or *computational* zero knowledge (PZK, SZK, CZK). We distinguish two regimes of V^* :

Honest-verifier zero knowledge (HVZK). The guarantee need hold only for the *honest* verifier, $V^* = V$ following the protocol (with z empty). The simulator must reproduce the distribution of an honest transcript.

(Malicious-verifier / auxiliary-input) zero knowledge. The guarantee holds for *every* PPT V^* , running an arbitrary strategy and holding an arbitrary auxiliary input z . The simulator Sim may typically use V^* as a subroutine *and rewind it*, while still running in expected polynomial time.

Remark 9.11 (The basis for the simulator's adequacy, and the role of z). The simulator receives x but *not* the witness w . If it can nonetheless output transcripts indistinguishable from real ones, then the real transcript carries no efficiently extractable information about w beyond $x \in L$ —for were it otherwise, the distinguisher could detect the difference. The crucial subtlety is that Sim wields a power P lacks: it can *rewind* V^* , retrying until V^* 's challenges align with a transcript the simulator could prepare without w . This is legitimate because Sim is, again, the reduction's tool, not a protocol participant. The auxiliary input z models prior knowledge a malicious verifier may bring (e.g. from earlier protocol runs); requiring zero knowledge to hold for all z is what makes the property *compose*—a zero-knowledge protocol remains zero knowledge when run as a subroutine inside a larger system. Practice adopts this auxiliary-input formulation.

9.5 Σ -protocols

Almost every practical zero-knowledge building block is a Σ -*protocol*: a three-move, public-coin protocol whose shape—a Greek capital sigma traced by the prover's commitment going out, the verifier's challenge coming back, and the prover's response going out again—gives the family its name. The structure is rigid enough to admit clean, checkable security definitions, and rich enough to capture an enormous range of statements through composition.

Definition 9.12 (Σ -protocol). Let R be a relation. A Σ -*protocol* for R is a three-move public-coin protocol (P, V) on common input x and prover input w with $(x, w) \in R$, of the following form, where $\mathcal{C}$ is the *challenge space*:

1. *Commitment.* P sends a first message a (the *commitment*, also called the *announcement*), computed from (x, w) and fresh randomness.
2. *Challenge.* V sends a uniformly random $e \in \mathcal{C}$.
3. *Response.* P sends z , computed from (x, w, a, e) and its randomness.

V accepts or rejects by a deterministic predicate $\text{Verify}(x, a, e, z) \in \{0, 1\}$. The protocol must satisfy:

Completeness. If $(x, w) \in R$ and P is honest, V accepts with probability 1.

Special soundness. There exists a PPT algorithm that, given x and any two accepting transcripts (a, e, z) and (a, e', z') with the *same commitment* a but *distinct challenges* $e \neq e'$, outputs a witness w with $(x, w) \in R$.

Special honest-verifier zero knowledge (sHVZK). There exists a PPT simulator that, on input x and *any* challenge $e \in \mathcal{C}$, outputs a pair (a, z) such that the simulated transcript (a, e, z) is distributed exactly (or statistically/computationally close) to a real honest transcript conditioned on the challenge being e . In particular the simulator chooses a *after* fixing e , and $\text{Verify}(x, a, e, z) = 1$.

Two accepting transcripts sharing the commitment but differing in the challenge constitute a *collision*; special soundness states that a collision is a witness. This is the local, two-transcript reformulation of knowledge extraction, and it is what renders Σ -protocols so tractable.

Theorem 9.13 (Special soundness yields knowledge soundness). *Let (P, V) be a Σ -protocol for R with challenge space $\mathcal{C}$, $|\mathcal{C}| = N$. Then (P, V) is a proof of knowledge for R with knowledge error $\kappa = 1/N$.*

Proof. Fix a cheating prover P^* on instance x , with success probability $\varepsilon = \varepsilon(x) > 1/N$. Because the protocol is public-coin with a single challenge, we may model P^* by its random tape ρ : once ρ is fixed, P^* deterministically produces the commitment $a = a(\rho)$ and, for each challenge e , a response $z = z(\rho, e)$. Consider the boolean matrix H indexed by rows ρ and columns $e \in \mathcal{C}$, with $H[\rho, e] = 1$ iff $\text{Verify}(x, a(\rho), e, z(\rho, e)) = 1$. The overall success probability equals the fraction of 1-entries (weighting rows by the probability of ρ),

$$\varepsilon = \Pr_{\rho, e} [H[\rho, e] = 1].$$

The extractor E runs the following *rewinding* procedure.

1. Run P^* with a fresh random ρ and a uniformly random challenge e ; obtain (a, e, z) . If V rejects, restart this step.
2. Once an accepting (a, e, z) appears, alternate two kinds of trial: (i) *rewind* P^* to just after it sent a (i.e. keep ρ fixed) and feed it one fresh uniform challenge $e' \neq e$; (ii) run one entirely fresh probe as in Step 1, with a new ρ and a new uniform e . If a trial of kind (i) accepts, the two transcripts collide; proceed to Step 3. If a trial of kind (ii) accepts, abandon the current row and continue Step 2 from the new accepting transcript.
3. Apply the special-soundness algorithm to (a, e, z) and (a, e', z') and output the witness w .

Correctness follows immediately from special soundness once we have two such transcripts; it remains to bound the expected running time. The expected cost of Step 1 is $1/\varepsilon$ protocol runs (the geometric waiting time of the *Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”). For Step 2, partition the trials into *phases*, one per row visited; for the current row ρ , let $\delta_\rho = \Pr_e[H[\rho, e] = 1]$ be its acceptance fraction, and call the row *heavy* if $\delta_\rho \geq \varepsilon/2$. By a Markov-type averaging argument (the standard “heavy-row” lemma), an accepting transcript lies in a heavy row with probability at least $1/2$; this applies to the transcript opening each phase, since every phase opens with a fresh accepting transcript distributed as in Step 1. Each alternation ends the phase with probability at least ε (the kind-(ii) trial alone accepts that often), so a phase consumes $O(1/\varepsilon)$ invocations of P^* in expectation—this is what the interleaving buys: even a row whose only accepting challenge is the already-used e cannot trap the extractor. Within a heavy row, a kind-(i) trial accepts with probability at least $\delta_\rho - 1/N \geq \varepsilon/2 - 1/N$ (a fresh challenge accepts with probability δ_ρ and equals the used e with probability at most $1/N$), so the phase ends in a collision rather than a row switch with probability $\Omega((\varepsilon/2 - 1/N)/\varepsilon)$. Combining, each phase yields a collision with probability

$\Omega((\varepsilon/2 - 1/N)/\varepsilon)$, so the expected number of phases is $O(\varepsilon/(\varepsilon/2 - 1/N))$, and by Wald's identity (*Math Guide*, same section) the total expected number of invocations of P^* is bounded by

$$O\left(\frac{\varepsilon}{\varepsilon/2 - 1/N}\right) \cdot O\left(\frac{1}{\varepsilon}\right) = O\left(\frac{1}{\varepsilon - 2/N}\right),$$

which is the form Definition 9.7 requires, with $\kappa = 2/N$ and a polynomial q absorbing the per-run cost. (Rows with $\delta_\rho \leq 1/N$ contribute no collisions; without the interleaved fresh probes they would trap the extractor for an unbounded expected time, which is why Step 2 alternates rather than insisting on the first row it finds.) Hence E extracts a witness in expected time $q(|x|)/(\varepsilon - 2/N)$; the crude factor of two in the threshold $\delta_\rho \geq \varepsilon/2$ is an artefact of this exposition, costing the extra $1/N$. The optimal knowledge error $\kappa = 1/N$ of the statement, matching the soundness error, is delivered by the tight extractors in the literature: Damgård's refined heavy-row analysis in his Σ -protocol lecture notes, and the k -special-soundness extractor of Attema, Cramer, and Kohl, which achieves knowledge error exactly $(k - 1)/N$ — here $k = 2$, i.e. $1/N$. $\square$

Remark 9.14 (Knowledge error and the challenge space). The knowledge error $1/N$ is also the soundness error: a prover who guesses the challenge in advance can answer that one challenge without a witness, succeeding with probability $1/N$. To render this negligible one takes $\mathcal{C}$ exponentially large—e.g. $\mathcal{C} = \mathbb{Z}_q$ for a large prime q , so $N = q$ —or repeats a small-challenge protocol $\omega(\log n)$ times. Large challenge spaces are the reason a single round of Schnorr (below) already has negligible soundness error, whereas the three-colouring protocol requires many repetitions.

The two-transcript notion generalises to protocols with several challenge rounds, and the generalisation is the exact bridge to the multi-round folding arguments of the Halo 2 system.

Definition 9.15 (Tree special soundness). Let (P, V) be a $(2\mu + 1)$ -move public-coin protocol for R , in which the prover speaks first and the verifier issues μ uniformly random challenges from a space $\mathcal{C}$. For positive integers $k_1, \dots, k_\mu$, a $(k_1, \dots, k_\mu)$ -tree of accepting transcripts for an instance x is a tree of depth μ in which every node at level i has k_i children labelled by pairwise distinct challenges for round i , every root-to-leaf path carries the prover messages of one accepting transcript, and transcripts through a common node share their prefix. The protocol is $(k_1, \dots, k_\mu)$ -special-sound if a PPT algorithm computes a witness $w \in R(x)$ from any such tree.

A Σ -protocol is the case $\mu = 1, k_1 = 2$. The extraction argument of Theorem 9.13 generalises: rewinding the prover round by round, an extractor assembles the required tree of $\prod_i k_i$ accepting transcripts, so a $(k_1, \dots, k_\mu)$ -special-sound protocol is a proof of knowledge whose knowledge error grows from $1/|\mathcal{C}|$ to the order of $(\sum_i (k_i - 1))/|\mathcal{C}|$, and whose extraction cost is polynomial provided the tree size $\prod_i k_i$ is. Both degradations are benign when $\mathcal{C}$ is exponentially large, μ is logarithmically bounded, and the k_i are constant, for then the tree size $\prod_i k_i$ is polynomial. This is the notion under which the inner-product argument's extractor operates (Proposition 7.32): each of the $\log_2 d$ folding rounds contributes one level of branching. The *Halo 2 Guide* instantiates this notion for its IPA extractor with one refinement: because each round's verification equation involves only the monomials $u^{-2}, 1, u^2$, the three challenges at every node must have pairwise distinct squares ($u \neq \pm u'$), a negligible-fraction exclusion absorbed into the knowledge error.

Proposition 9.16 (sHVZK is HVZK). A Σ -protocol satisfying special HVZK is honest-verifier zero knowledge in the sense of Definition 9.10.

Proof. The honest verifier's view is (a, e, z) together with its coins, where e is uniform in $\mathcal{C}$. The HVZK simulator draws $e \leftarrow \mathcal{C}$ uniformly, then invokes the special-HVZK simulator on (x, e) to obtain (a, z) , and outputs (a, e, z) . By special HVZK, for each fixed e the pair (a, z) is distributed as in a real run conditioned on that e ; averaging over the uniform e reproduces the honest view exactly (or up to the relevant negligible distance). Thus the simulated and real views coincide as distributions. $\square$

Example 9.17 (Schnorr’s protocol for discrete logarithm). The canonical Σ -protocol is Schnorr’s identification protocol for R_{dl} , developed in full in §8.3: the commitment is $R = [r]G$, the challenge $c \in \mathbb{F}_q$ is uniform, the response is $s = r + cx$, and the verifier checks $[s]G = R + [c]X$. Its three Σ -properties are exactly Propositions 8.6 (perfect completeness), 8.7 (2-special soundness: a collision yields the witness $x = (s - s')(c - c')^{-1}$), and 8.9 (special HVZK: sample s , solve for $R = [s]G - [c]X$). With challenge space $\mathbb{F}_q$ the knowledge error is $1/q$, already negligible in a single round (Remark 9.14); by Theorem 9.13 and Proposition 9.16, Schnorr is a perfect-HVZK proof of knowledge of a discrete logarithm.

Remark 9.18 (From HVZK to full zero knowledge). Σ -protocols are only *honest-verifier* zero knowledge: a malicious V^* might choose e as a function of a to leak information, and the sHVZK simulator (which picks e itself) does not obviously cope. Three standard remedies upgrade HVZK to malicious-verifier ZK:

1. *Sequential repetition* of a protocol with a small challenge, where V commits to nothing across rounds—this gives full ZK by a rewinding simulator but costs rounds.
2. Have V *commit* to its challenge e before seeing a (a coin-tossing or commitment step), so a malicious V^* cannot make e depend on a ; the simulator then rewinds past the opening.
3. Apply Fiat–Shamir (Subsection 9.6), which removes the verifier’s choice of e altogether by deriving e from a hash—in the random-oracle model the resulting non-interactive protocol is zero knowledge against *all* verifiers.

For the SNARK pipeline it is route 3 that matters, and HVZK is exactly the property Fiat–Shamir requires.

9.6 The Fiat–Shamir transform: from interactive to non-interactive

The only verifier messages in a public-coin protocol are random challenges. The Fiat–Shamir heuristic removes the verifier from the loop by *computing* each challenge as a hash of everything sent so far. The interaction collapses into a single non-interactive proof string, and—because the prover no longer talks to a live, possibly-malicious verifier but to a fixed public function—honest-verifier zero knowledge upgrades to full non-interactive zero knowledge. The cost is that the hash function must be idealised: the security proof rests on the *random-oracle model* (ROM) of Definition 8.13, with the oracle $\mathcal{H} : \{0, 1\}^* \rightarrow \mathcal{C}$ now mapping into the challenge space $\mathcal{C}$. We again exploit the two powers the model grants a reduction: *observability* (the reduction sees every query the adversary makes) and *programmability* (the reduction may fix $\mathcal{H}$ ’s output at a freshly queried point to any value it likes, provided the answers remain consistent and uniformly distributed).

Definition 9.19 (Fiat–Shamir transform). Let (P, V) be a public-coin interactive protocol (we describe the Σ case; the multi-round case applies the rule per round). Let $\mathcal{H}$ be a random oracle with range the challenge space $\mathcal{C}$. The *Fiat–Shamir transform* $\text{FS}[(P, V), \mathcal{H}]$ is the non-interactive protocol:

- *Prover*. Compute the commitment a as in (P, V) ; set $e = \mathcal{H}(x, a)$; compute the response z . Output the proof $\pi = (a, z)$ (equivalently (a, e, z)).
- *Verifier*. Given x and $\pi = (a, z)$, recompute $e = \mathcal{H}(x, a)$ and accept iff $\text{Verify}(x, a, e, z) = 1$.

In multi-round public-coin protocols, the i -th challenge is $e_i = \mathcal{H}(x, a_1, e_1, \dots, a_i)$ —each challenge hashes the *entire* transcript prefix. To bind a message m (turning the proof into a signature), include m in the hash: $e = \mathcal{H}(x, a, m)$.

Remark 9.20 (Hashing the whole transcript: the Fiat–Shamir pitfall). The requirement that each challenge hash *all* prior messages—the instance included—is not pedantry. The notorious “weak Fiat–Shamir” bug omits the statement x (or part of the transcript) from the hash, letting an attacker choose x *after* the challenge is fixed and forge proofs of false statements. For multi-round protocols, hashing only the latest message rather than the running prefix similarly breaks soundness. The maxim is: the challenge must be a hash of everything the verifier would have seen up to the moment it speaks.

Theorem 9.21 (Fiat–Shamir: security in the ROM). *Let (P, V) be a Σ -protocol for R with challenge space $\mathcal{C}$, $|\mathcal{C}| = N$, special soundness, and special HVZK. In the random-oracle model, $\Pi = \text{FS}[(P, V), \mathcal{H}]$ is a non-interactive argument that is:*

1. complete (perfectly, inheriting completeness of (P, V));
2. knowledge sound: *from any PPT prover P^* making at most Q random-oracle queries and producing an accepting proof for x with probability ε , an extractor obtains a witness $w \in R(x)$ with probability at least $\varepsilon \cdot (\varepsilon/Q - 1/N)$ (up to constants), in particular non-negligibly whenever ε is non-negligible and N is super-polynomial; and*
3. (non-interactive) zero knowledge: *there is a PPT simulator that, controlling the random oracle by programming, produces proofs indistinguishable from real ones without the witness, for all (even malicious) verifiers.*

Proof. Completeness is immediate: the honest prover computes $e = \mathcal{H}(x, a)$ and an honest response, which verifies, and the verifier recomputes the same e .

Zero knowledge by programming. The simulator, given x , runs the special-HVZK simulator: it draws $e \leftarrow \mathcal{C}$, obtains an accepting (a, z) , and then *programs* the random oracle by setting $\mathcal{H}(x, a) := e$. Provided (x, a) was not previously queried—which holds except with probability at most (number of prior queries) $\cdot 2^{-H_\infty(a)}$, negligible because the honest first message a has high min-entropy—the programmed oracle is consistent and uniformly distributed, and the output (a, z) is distributed exactly as a real proof. Programming remains invisible to the verifier, so this is zero knowledge against all verifiers (the verifier’s malice is moot: it has no live challenge to skew).

Knowledge soundness by the forking lemma. Let P^* be a prover that outputs an accepting $\pi = (a, z)$ for x with probability ε , making Q oracle queries. With overwhelming probability P^* actually queried the value $e = \mathcal{H}(x, a)$ used in a successful proof (else P^* guessed an unqueried random output, probability $\leq 1/N$ per proof); say it was the j -th query for some index $j \leq Q$. The extractor runs P^* once to obtain a successful transcript with its query index j ; it then *rewinds* P^* to just before the j -th query and re-runs it with *fresh, reprogrammed* oracle answers from that point on—so the first $j - 1$ answers, and hence the commitment a , remain unchanged, but the j -th answer (the challenge on (x, a)) is a new uniform $e' \in \mathcal{C}$. The *general forking lemma* guarantees that, if P^* succeeds with probability ε and there are Q query slots, then with probability at least $\varepsilon(\varepsilon/Q - 1/N)$ both runs succeed and $e \neq e'$. Conditioned on that event we hold two accepting transcripts (a, e, z) and (a, e', z') with the same a and $e \neq e'$; the special-soundness extractor (Definition 9.12) outputs $w \in R(x)$. This is exactly the knowledge-soundness conclusion, with the stated success probability. $\square$

Remark 9.22 (What the random oracle is, and is not). The ROM is an idealisation: real hash functions are fixed, public, and not random, and there exist (contrived) protocols secure in the ROM yet insecure under *every* concrete instantiation. The transform is nonetheless trusted in practice and underlies essentially every deployed non-interactive proof and signature, including the Fiat–Shamir-compiled SNARKs below. Two refinements matter for those systems: (i) the multi-round forking analysis is more delicate—one must fork each of the protocol’s rounds, and the success probability degrades with the round count and the soundness structure (special soundness must generalise to the tree special soundness of Definition 9.15); (ii) when the underlying interactive argument is computationally (not statistically) sound, the ROM extraction layers on top of the computational soundness reduction. A

subtle but practically important issue is the gap between *programmable* ROM proofs and the non-programmable setting; deployed systems adopt the programmable model.

Example 9.23 (Schnorr signatures). Fiat–Shamir applied to Schnorr’s Σ -protocol (Example 9.17), with a message m hashed into the challenge, is precisely the Schnorr signature scheme of Definition 8.11, whose EUF-CMA security Theorem 8.16 establishes by exactly the forking argument above. This is the canonical illustration that “proof of knowledge + Fiat–Shamir = signature.”

9.7 From Σ -protocols toward general computation

Σ -protocols handle algebraic relations (discrete logs, openings of commitments, linear statements) with a single move of each kind. To prove arbitrary NP statements—“I know an input making this circuit output 1”—we need protocols for general computation.

Remark 9.24 (The feasibility theorem and its cost). The classical route passes through an NP-complete problem. For graph three-colouring ($R_{3\text{col}}$ of Example 9.2) a simple commit–challenge–reveal protocol—the prover commits to a randomly recoloured copy of her colouring, the verifier challenges with one uniformly random edge, the prover opens the two endpoint commitments, and the verifier checks that the revealed colours differ—is a computational-HVZK proof of knowledge with soundness error $1 - 1/|E|$, which repetition drives to negligible. This is the constructive core of the celebrated feasibility theorem: under the existence of one-way functions (which yield the commitments), every NP language—indeed every language in $\text{IP} = \text{PSPACE}$ —has a computational zero-knowledge proof. The theorem settles *feasibility* but not *efficiency*: the protocol commits to a fresh copy of the entire colouring in each of polynomially many repetitions, so both communication and the prover’s work scale with (circuit size) $\times$ (repetitions). Practical proof systems abandon the generic reduction and instead arithmetise computation and exploit polynomial structure—the subject of the final subsection.

9.8 SNARKs: succinct non-interactive arguments of knowledge

This is the destination. A SNARK certifies a possibly enormous computation with a proof that is tiny and that one checks almost instantly, hiding the witness if desired. We assemble its definition from the notions already built, then give the modern recipe—a polynomial interactive oracle proof compiled by a polynomial commitment scheme and made non-interactive by Fiat–Shamir—that the Halo 2 system instantiates.

Definition 9.25 (Non-interactive argument; preprocessing). A *non-interactive argument* for an NP-relation R consists of three PPT algorithms:

- $\text{Setup}(1^n, R) \rightarrow \text{srs}$, producing a *structured reference string* (also “public parameters”; in the ROM this may be a single random oracle, giving a *transparent* setup with no secret to discard);
- $\text{Prove}(\text{srs}, x, w) \rightarrow \pi$, with $(x, w) \in R$, producing a proof π ;
- $\text{Verify}(\text{srs}, x, \pi) \rightarrow \{0, 1\}$.

It is *complete* if honest proofs always verify. A scheme is a *preprocessing* argument if Setup additionally depends on the circuit/relation and outputs a short verification key, amortising per-statement verifier work.

Definition 9.26 (SNARK). A *SNARK* (succinct non-interactive argument of knowledge) for an

NP-relation R is a non-interactive argument that is, in addition:

Knowledge sound. For every PPT prover P^* producing an accepting proof for some x , there is a PPT extractor (with the same setup, and—in the ROM—observing P^* ’s oracle queries and rewinding/forking it) that outputs w with $(x, w) \in R$, except with negligible probability. Formally, $\Pr[\text{Verify}(\text{srs}, x, \pi) = 1 \wedge (x, E^{P^*}) \notin R] \leq \text{negl}(n)$.

Succinct. The proof size $|\pi|$ and the verifier’s running time are *sublinear* in the size of the computation being proved—typically $|\pi| = O(\text{poly}(n))$ or $O(\text{poly}(n) \log |C|)$ and verification $O(\text{poly}(n) \log |C|)$ for a circuit C , *independent of, or only polylogarithmic in, the witness and circuit size*. (Some literature reserves “succinct” for polylogarithmic proofs; “ $O(1)$ -size” pairing-based schemes are the strongest form.)

If it also satisfies zero knowledge (Definition 9.10, in its non-interactive ROM form), it is a *zk-SNARK*.

Remark 9.27 (Why succinctness forces *arguments*, not proofs). A succinct *proof*—information-theoretic soundness with sublinear communication—for an NP-complete language is essentially ruled out: an unbounded prover could otherwise compress NP-certificates below the bounds that complexity theory forbids, and indeed a transcript shorter than the witness cannot information-theoretically pin down the witness. Succinct *arguments* escape this because a computationally bounded prover *cannot find* a convincing-but-false short proof, even though one may exist. This is the deepest reason the field works with *arguments* (Remark 9.5): succinctness and computational soundness arise together, by necessity.

Remark 9.28 (What knowledge soundness provides an application). For a deployed system, plain soundness does not suffice; knowledge soundness is the property that renders the proof *useful*. Consider a private transaction asserting “there exists a note I am authorised to spend, and the input and output values balance.” Soundness alone guarantees only that *some* satisfying witness exists—it does not prevent a prover who has no such note from proving the existential statement were she somehow to find a proof. Knowledge soundness guarantees that the proving party *actually holds* a concrete witness—a real spendable note, a real authorisation—that the extractor could recover. The application reasons: *whoever produced this proof possesses the secret it attests to*. This is what permits a verifier to safely act on the proof (release funds, grant access) as though the prover had presented the witness, while it remains hidden. In short: **soundness protects against false statements; knowledge soundness protects against provers who do not actually know what they claim**—and almost every cryptographic use case requires the latter.

The polynomial-IOP-and-commitment compilation recipe. Modern SNARKs, Halo 2 included, are not monolithic. They factor cleanly into an *information-theoretic* object and a *cryptographic* compiler, glued and de-interactivised by Fiat–Shamir. We describe the three layers in turn.

Definition 9.29 (Polynomial interactive oracle proof (Poly-IOP)). A *polynomial IOP* for a relation R over a field $\mathbb{F}$ is a public-coin interactive protocol between P and V in which, instead of sending field elements, the prover in each round sends an *oracle* for a polynomial $p_i \in \mathbb{F}[X]$ of bounded degree. The verifier sends uniformly random field-element challenges, and may *query* each polynomial oracle at points of its choice, receiving $p_i(\zeta)$. After the interaction V accepts or rejects based on the (few) evaluations it queried and its challenges. Completeness and (knowledge) soundness are as for interactive proofs, with the soundness analysis purely algebraic—typically resting on the *Schwartz–Zippel lemma*: a nonzero polynomial of degree d over $\mathbb{F}$ vanishes at a uniformly random point with probability at most $d/|\mathbb{F}|$. The verifier in a good Poly-IOP makes only $O(1)$ or $O(\log)$ queries, each a single evaluation.

Remark 9.30 (Arithmetisation: reaching a Poly-IOP). The first step in building any of these systems is *arithmetisation*: encoding the claim “I know w with $C(w) = 1$ ” as a small set of polynomial identities over $\mathbb{F}$ that hold iff the computation is correct. One expresses computations as constraint systems—rank-1 constraint systems (R1CS), or the customisable *PLONKish* gate-and-permutation form used by Halo 2—whose satisfaction is equivalent to a handful of polynomials (encoding the wire values, the gate constraints, and a permutation argument enforcing wiring/copy constraints) satisfying identities like $q_L a + q_R b + q_M ab + q_O c + q_C = 0$ on a chosen evaluation domain. Checking such identities at a single random point—legitimised by Schwartz–Zippel—is the Poly-IOP. Crucially, the Poly-IOP is unconditionally sound: it rests on no cryptographic assumptions *yet*, only the algebra of low-degree polynomials.

The cryptographic half of the recipe is the *polynomial commitment scheme* of Definition 7.28: a short, binding commitment $\text{Commit}(p)$ to a polynomial $p \in \mathbb{F}[X]$ of bounded degree, together with short proofs, verifiable against the commitment alone, that the committed polynomial satisfies $p(\zeta) = v$ —with evaluation binding and, for SNARK use, the extractability property that a prover who opens evaluations must “know” an underlying polynomial. Instantiations include KZG (pairing-based, $O(1)$ -size, trusted setup), the inner-product-argument (IPA) commitment over a single elliptic curve used in Halo 2 (transparent, no trusted setup, logarithmic-size openings), and FRI (hash-based, transparent, used in STARKs).

Theorem 9.31 (The compilation recipe, informally). *Let PIOP be a public-coin polynomial IOP for R that is complete and knowledge-sound, and let PC be a polynomial commitment scheme that is correct, evaluation-binding, and extractable. Compile them as follows: wherever the prover would send a polynomial oracle p_i , it instead sends $\text{Commit}(p_i)$; wherever the verifier would query $p_i(\zeta)$, the prover sends the claimed value with a PC evaluation proof, which the verifier checks. The result is a public-coin interactive argument of knowledge for R . Applying the Fiat–Shamir transform (Definition 9.19) in the random-oracle model—deriving every verifier challenge as a hash of the transcript so far—yields a non-interactive argument of knowledge: a SNARK. If, in addition, the Poly-IOP is made zero knowledge (e.g. the prover blinds its polynomials with random masking terms and the commitment is hiding), the SNARK is a zk-SNARK.*

Sketch. *Completeness* transfers verbatim: honest commitments and honest evaluation proofs let the verifier reconstruct every value it would have queried in the Poly-IOP, and the Poly-IOP verifier accepts. *Knowledge soundness* is layered. The SNARK extractor first runs the PC extractability guarantee on each commitment the prover produced, recovering actual polynomials $\hat{p}_i$ consistent with all opened evaluations (here the binding/extractable property of PC is the cryptographic assumption that turns the proof into an *argument*). It then feeds these $\hat{p}_i$ to the Poly-IOP’s (unconditional) knowledge extractor, which, because the Poly-IOP is sound against *any* polynomials—in particular the extracted ones—outputs a witness $w \in R(x)$; evaluation-binding ensures the prover could not have answered queries inconsistently with the committed $\hat{p}_i$, so the algebraic soundness applies. *Fiat–Shamir* converts the resulting public-coin interactive argument of knowledge into a non-interactive one in the ROM by the (multi-round generalisation of the) analysis of Theorem 9.21: challenges become transcript hashes, the forking/observability of the random oracle supplies the rewinding the extractor needs, and HVZK of the blinded protocol becomes full NIZK by programming. *Succinctness* of the proof comes entirely from PC: it consists of a constant or logarithmic number of short commitments and short evaluation proofs, so the proof *size* is sublinear in the circuit, independent of the witness. The verifier checks $O(1)$ or $O(\log)$ evaluations and a handful of field identities; its running time is sublinear only when the PC’s evaluation checks are themselves succinct—as for KZG. With the inner-product commitment each such check costs $O(d)$ group operations (Proposition 7.32), so the compiled verifier is linear-time in the circuit while the proof stays succinct. Combining, the compiled object satisfies every clause of Definition 9.26 when PC openings verify succinctly, and otherwise yields succinct proofs with linear-time verification. $\square$

Remark 9.32 (Where Halo 2 sits, and what comes next). The system this monograph builds toward, Halo 2, is exactly an instance of Theorem 9.31: a *PLONKish* arithmetisation (custom gates, lookup arguments, and a permutation argument for copy constraints) furnishes the polynomial IOP; the *inner-product-argument* polynomial commitment over a single elliptic curve furnishes a *transparent* PC with no trusted setup and logarithmic openings; and Fiat–Shamir over a concrete hash makes it non-interactive. Its signature innovation, *accumulation* (the “Halo” technique), defers and batches the expensive final IPA check across many proofs, enabling recursive composition without a trusted setup—one proof attesting to the verification of another. We have now built each ingredient named here—elliptic-curve groups, polynomial commitments, the Schwartz–Zippel lemma, the Fiat–Shamir transform, knowledge soundness via extraction—from first principles, and the chapters that follow assemble them into that system. The reader should carry forward three load-bearing ideas: *rewinding* (Remark 9.8) as the universal device behind both extraction and simulation; the clean split between an *unconditionally sound* information-theoretic core and a *cryptographically* enforced commitment; and the principle that *knowledge* soundness—not mere soundness—is what an application stakes its security on.

10 Merkle trees and commitments to sets

A recurring problem in cryptography is the following. A party — the *prover* — holds a large collection of data: a list of transactions, a set of public keys, a database of records, or, in the application of principal concern here, a growing set of *note commitments* in a shielded payment system. A second party — the *verifier* — wishes to be convinced of statements about this collection without storing it in its entirety. The two canonical statements are:

- *membership*: “the value v occurs at position i in the list”, or “ v belongs to the set”;
- *integrity*: “the collection is exactly the one committed to earlier, unaltered.”

We require a single short string — a *commitment* — that determines the entire collection, together with short *proofs* that individual elements are present. “Short” here means: of size independent of, or at worst logarithmic in, the number of elements. The *Merkle tree*, introduced by Ralph Merkle, achieves this using nothing more than a collision-resistant hash function, and it is the structural backbone of essentially every modern blockchain and of the membership argument inside Zcash’s Orchard protocol.

This section develops Merkle trees from first principles. We first recall the precise security property of the underlying compression function; we then construct the tree, define and analyse authentication paths, prove the reduction of path forgery to collision finding, and treat the engineering subtleties (domain and layer separation) that render the reduction honest. The dynamic setting follows — append-only and incrementally updatable trees and the notion of a frontier — after which we abstract the construction into the language of *vector commitments*. Finally, we show that composing a Merkle root over note commitments with a zero-knowledge proof of an authentication path yields a privacy-preserving membership primitive, which is precisely the role Merkle trees play in Orchard.

Throughout, $\{0, 1\}^*$ denotes the set of finite binary strings, $\{0, 1\}^n$ the strings of length n , and $\parallel$ denotes concatenation. The symbol λ denotes the security parameter; all algorithms are probabilistic polynomial-time (PPT) in λ unless stated otherwise, and $\text{negl}(\lambda)$ denotes a negligible function (one that decays faster than the inverse of every polynomial).

10.1 The compression function and collision resistance

The single cryptographic ingredient of a Merkle tree is a function that takes a fixed-size input and returns a (typically shorter) fixed-size output, in such a way that no one can exhibit two distinct inputs colliding on the same output. We isolate the required property as follows.

Definition 10.1 (Hash family). A *hash family* is a pair of PPT algorithms (Gen, H) . On input 1^λ , Gen outputs a key k (the *public parameters*), which determines the length $\ell = \ell(\lambda)$ of the digest. The (deterministic) evaluation algorithm computes a function

$$H_k : \{0, 1\}^* \longrightarrow \{0, 1\}^\ell.$$

When the input length is restricted to a fixed multiple of the output length — the relevant case here — we term the function a *compression function*; a two-to-one compression function has the signature

$$h_k : \{0, 1\}^{2\ell} \longrightarrow \{0, 1\}^\ell.$$

The key k models the choice of a concrete function from a family, which is what renders collision resistance a meaningful asymptotic notion (Remark 3.2); in practice we use a fixed standardised function, or inside arithmetic circuits an algebraic hash, and we suppress k from the notation when it plays no role, writing H and h . The security property we need is *collision resistance* exactly as in Definition 3.6: no PPT adversary outputs $x \neq x'$ with $H_k(x) = H_k(x')$, except with probability $\text{negl}(\lambda)$.

Recall from §3.2 that this is the strongest of the three classical hash notions (Proposition 3.7), and from Lemma 3.9 that the generic birthday attack caps an ℓ -bit digest at $\ell/2$ bits of collision security, whence the 256-bit outputs used here. Merkle trees require full collision resistance, for the reason Theorem 10.13 establishes.

10.2 Binary Merkle trees

Fix a two-to-one compression function $h : \{0, 1\}^{2\ell} \rightarrow \{0, 1\}^\ell$. We treat first the clean case in which the number of leaves is a power of two.

Definition 10.2 (Binary Merkle tree, full case). Let $d \in \mathbb{N}$ and let $L = (L_0, L_1, \dots, L_{2^d-1})$ be an ordered list of 2^d leaf values, each in $\{0, 1\}^\ell$. The *Merkle tree of depth d* over L is the complete binary tree with 2^d leaves whose nodes carry labels in $\{0, 1\}^\ell$, defined level by level from the leaves up. Index the levels 0 (leaves) through d (root). The labels of level 0 are

$$N_{0,j} = L_j, \quad 0 \leq j < 2^d,$$

and the label of each internal node is the compression of its two children's labels:

$$N_{t+1,j} = h(N_{t,2j} \parallel N_{t,2j+1}), \quad 0 \leq t < d, \quad 0 \leq j < 2^{d-t-1}. \quad (*)$$

The unique node at level d , namely $N_{d,0}$, is the *root* of the tree; write $\text{root}(L) = N_{d,0}$.

Here $N_{t,j}$ denotes the j -th node (counting from the left, starting at 0) on level t . Each level t has 2^{d-t} nodes, so the levels shrink by a factor of two as one ascends, the root being a single ℓ -bit string. The construction iterates the application of h arranged in a balanced binary fan-in.

Example 10.3 (A tree of depth 2). With $d = 2$ and leaves L_0, L_1, L_2, L_3 , the tree has seven nodes. The level-1 labels are

$$N_{1,0} = h(L_0 \parallel L_1), \quad N_{1,1} = h(L_2 \parallel L_3),$$

and the root is

$$\text{root}(L) = N_{2,0} = h(h(L_0 \parallel L_1) \parallel h(L_2 \parallel L_3)).$$

Pictorially: four leaves along the bottom, two internal nodes above them each hashing a pair, and the root above those two hashing the pair of internal nodes. The single ℓ -bit root summarises the whole list of four ℓ -bit strings.

Remark 10.4 (Padding to a power of two). We handle a list whose length m is not a power of two by fixing a depth d with $2^d \geq m$ and *padding* the remaining $2^d - m$ leaf slots with a canonical value, e.g. the all-zero string or the hash of the empty string. Padding must not create ambiguity: two different lists must never yield the same padded leaf sequence. Including the true length m in the domain-separated computation of the root (Subsection 10.5) removes any such ambiguity. An alternative is the unbalanced tree, in which a node with a single child passes that child's label upward unchanged — a convenient but subtle choice we discuss together with second-preimage attacks in Remark 10.17.

The root is intended to serve as a commitment: a short string that determines the list. The next definition and proposition make this precise.

Definition 10.5 (Merkle commitment scheme). The *Merkle commitment* to a list L of 2^d leaves is the value $\text{root}(L) \in \{0, 1\}^\ell$. The scheme is *binding* if no PPT adversary can produce two distinct lists $L \neq L'$ of equal length with $\text{root}(L) = \text{root}(L')$, except with negligible probability.

Proposition 10.6 (Binding of the root). *If h is collision resistant, then the Merkle commitment is binding. Concretely, from any two distinct equal-length lists $L \neq L'$ with $\text{root}(L) = \text{root}(L')$ one can extract, in time linear in the tree size, a collision for h .*

Proof. Let L and L' be distinct lists of 2^d leaves with equal roots, and write $N_{t,j}$ and $N'_{t,j}$ for the node labels of the two trees. Because the lists differ, there is at least one leaf index j_0 with $N_{0,j_0} \neq N'_{0,j_0}$. Consider the leaf-to-root path

$$N_{0,j_0}, N_{1,\lfloor j_0/2 \rfloor}, N_{2,\lfloor j_0/4 \rfloor}, \dots, N_{d,0}$$

in the first tree and the correspondingly indexed path in the second. At level 0 the two paths carry different labels, by choice of j_0 ; at level d they carry the *same* label, namely the common root. Hence as one ascends there is a smallest level t at which the two labels first agree:

$$N_{t,j} = N'_{t,j} \quad \text{but} \quad N_{t-1,2j} \neq N'_{t-1,2j} \quad \text{or} \quad N_{t-1,2j+1} \neq N'_{t-1,2j+1},$$

where $j = \lfloor j_0/2^t \rfloor$. By the recurrence (*),

$$h(N_{t-1,2j} \| N_{t-1,2j+1}) = N_{t,j} = N'_{t,j} = h(N'_{t-1,2j} \| N'_{t-1,2j+1}),$$

while the two 2ℓ -bit inputs $N_{t-1,2j} \| N_{t-1,2j+1}$ and $N'_{t-1,2j} \| N'_{t-1,2j+1}$ differ, since they disagree in at least one of the two halves. This is a collision for h . The extraction walks one root-to-leaf path and so costs $O(d)$ hash comparisons.

Consequently, a PPT adversary breaking binding with non-negligible probability ε yields a PPT adversary finding a collision with the same probability ε , contradicting collision resistance. Hence $\varepsilon = \text{negl}(\lambda)$. $\square$

The argument in Proposition 10.6 is the prototype of every Merkle security proof: *a discrepancy at the leaves and agreement at the root must, by the pigeonhole-like “first level of agreement” argument, manifest a collision somewhere on the path.* We reuse it almost verbatim for authentication paths.

10.3 Authentication paths and membership proofs

The root binds the whole list, but a verifier who knows only the root should be able to check a claim about a single leaf without re-reading the list. The data that enables this is the sequence of *siblings* along the leaf-to-root path.

Definition 10.7 (Authentication path). Let T be a Merkle tree of depth d over L , and fix a leaf index i with $0 \leq i < 2^d$. Write the binary expansion of i as $b_{d-1}b_{d-2} \dots b_0$, so that $b_t \in \{0,1\}$ records, at level t , whether the relevant node is a left child ($b_t = 0$) or a right child ($b_t = 1$). The *authentication path* (or *Merkle proof*, or *membership witness*) for index i is the sequence

$$\pi_i = ((s_0, b_0), (s_1, b_1), \dots, (s_{d-1}, b_{d-1})),$$

where s_t is the label of the *sibling* of the level- t node lying on the path from leaf i to the root. Explicitly, with $j_t = \lfloor i/2^t \rfloor$ the index of the path node at level t ,

$$s_t = \begin{cases} N_{t,j_t+1}, & b_t = 0 \quad (\text{path node is a left child; sibling on the right}), \\ N_{t,j_t-1}, & b_t = 1 \quad (\text{path node is a right child; sibling on the left}). \end{cases}$$

The path has exactly d entries.

The verifier, given the claimed leaf value, the index i , the authentication path, and the trusted root, recomputes the chain of labels up the tree and checks that it arrives at the root.

Definition 10.8 (Path verification). Define $\text{Verify}(\text{rt}, i, v, \pi_i)$, where rt is a root, i an index, $v \in \{0, 1\}^\ell$ a claimed leaf value, and $\pi_i = ((s_0, b_0), \dots, (s_{d-1}, b_{d-1}))$ a candidate path, as follows. First check the path against the index: output reject unless, for every t , the bit b_t carried in π_i equals bit t of the binary expansion of i . Then set $a_0 := v$, and for $t = 0, 1, \dots, d-1$ compute

$$a_{t+1} := \begin{cases} h(a_t \parallel s_t), & b_t = 0, \\ h(s_t \parallel a_t), & b_t = 1. \end{cases} \quad (\dagger)$$

Output accept if $a_d = \text{rt}$ and reject otherwise. We term the value a_d the *root recomputed from* (i, v, π_i) .

The bit b_t indicates on which side the verifier places the running value a_t before compressing, so that the order of arguments to h matches the order used when building the tree. The recomputation $(\dagger)$ folds the leaf upward through the supplied siblings, one compression per level. The following completeness statement is immediate.

Proposition 10.9 (Completeness). *For every list L , every index i , and π_i the genuine authentication path for i in the tree over L , $\text{Verify}(\text{root}(L), i, L_i, \pi_i) = \text{accept}$.*

Proof. By induction on t we show $a_t = N_{t, \lfloor i/2^t \rfloor}$, i.e. the running value equals the genuine label of the path node at level t . The base case $a_0 = L_i = N_{0,i}$ holds by definition. For the step, the path node at level t and its sibling s_t are the two children of the path node at level $t+1$; placing a_t on the side dictated by b_t and s_t on the other reproduces exactly the two arguments of h in the recurrence $(*)$, so $a_{t+1} = N_{t+1, \lfloor i/2^{t+1} \rfloor}$. At $t = d$ we obtain $a_d = N_{d,0} = \text{root}(L)$. $\square$

Proposition 10.10 (Logarithmic proof size). *An authentication path in a tree of depth d consists of d sibling labels and d direction bits, for a total of $d \cdot \ell + d$ bits. Verification performs exactly d evaluations of h . Choosing the minimal sufficient depth $d = \lceil \log_2 m \rceil$ for a list of $m \geq 2$ leaves yields proofs of $\lceil \log_2 m \rceil(\ell + 1)$ bits, logarithmic in the list length.*

Proof. Immediate from Definitions 10.7 and 10.8: there is one sibling and one compression per level, and there are d levels above the leaves. A depth- d tree accommodates m leaves precisely when $2^d \geq m$ (Remark 10.4), so $\lceil \log_2 m \rceil$ is the minimal sufficient depth. $\square$

Thus a membership proof for a set of a billion elements (2^{30}) consists of 30 hash values and 30 bits — under a kilobyte with a 256-bit hash — regardless of how the billion elements are arranged. This logarithmic succinctness is the central objective.

Example 10.11 (A depth-2 membership proof). Continue Example 10.3 and prove membership of L_2 at index $i = 2 = (10)_2$, so $b_0 = 0$, $b_1 = 1$. At level 0 the path node is $N_{0,2} = L_2$, a left child, so the sibling is $s_0 = N_{0,3} = L_3$. At level 1 the path node is $N_{1,1} = h(L_2 \parallel L_3)$, a right child, so the sibling is $s_1 = N_{1,0} = h(L_0 \parallel L_1)$. The path is $\pi_2 = ((L_3, 0), (h(L_0 \parallel L_1), 1))$. Verification computes

$$a_1 = h(L_2 \parallel L_3), \quad a_2 = h(h(L_0 \parallel L_1) \parallel a_1),$$

and indeed $a_2 = \text{root}(L)$. The verifier never saw L_0 or L_1 themselves — only the digest $h(L_0 \parallel L_1)$ — yet is convinced L_2 sits at index 2 of the committed list.

10.4 Soundness: forging a path implies a collision

We now establish the security guarantee that renders the membership proof meaningful: against a fixed, honestly computed root, no efficient adversary can produce an accepting path for a leaf value that is not the one actually committed at that position. “Cannot” is, as always, conditional on collision resistance.

Definition 10.12 (Path forgery). Fix a list L of 2^d leaves with root $rt = \text{root}(L)$. A *path forgery* at index i is a pair (v^*, π^*) with

$$\text{Verify}(rt, i, v^*, \pi^*) = \text{accept} \quad \text{and} \quad v^* \neq L_i.$$

That is, the adversary makes the verifier accept a leaf value different from the genuine one at position i , against the genuine root.

Theorem 10.13 (Soundness of Merkle membership proofs). *Let h be collision resistant. Then no PPT adversary, given the list L (equivalently, given the whole tree), produces a path forgery except with negligible probability. More precisely, there is a deterministic linear-time extractor that, from any list L , index i , and accepting forgery (v^*, π^*) with $v^* \neq L_i$, outputs a collision for h .*

Proof. Let $rt = \text{root}(L)$ and let $\pi^* = ((s_0^*, b_0^*), \dots, (s_{d-1}^*, b_{d-1}^*))$ be the forged path. Since the forgery is accepting, the index-consistency check of Definition 10.8 guarantees $b_t^* = b_t$, bit t of i , for every t .

Run the verifier, obtaining the recomputed chain $a_0^* = v^*, a_1^*, \dots, a_d^*$ defined by $(\dagger)$; by hypothesis $a_d^* = rt$. In parallel, let $a_0, a_1, \dots, a_d$ be the genuine chain for the true leaf L_i , i.e. $a_t = N_{t, \lfloor i/2^t \rfloor}$ with genuine siblings; by Proposition 10.9, $a_d = rt$ as well.

There are now two chains that *disagree at the bottom* — $a_0^* = v^* \neq L_i = a_0$ — yet *agree at the top*, $a_d^* = a_d = rt$. Let t^* be the smallest level at which they agree:

$$t^* = \min\{t : a_t^* = a_t\} \in \{1, \dots, d\}, \quad \text{so} \quad a_{t^*}^* = a_{t^*} \text{ but } a_{t^*-1}^* \neq a_{t^*-1}.$$

Such a t^* exists because the chains disagree at level 0 and agree at level d . Write $t = t^*$ and $b = b_{t-1}$ ($= b_{t^*-1}^*$). The recurrence $(\dagger)$ computes the two equal values at level t as

$$a_t^* = \begin{cases} h(a_{t-1}^* || s_{t-1}^*), & b = 0 \\ h(s_{t-1}^* || a_{t-1}^*), & b = 1 \end{cases} \quad a_t = \begin{cases} h(a_{t-1} || s_{t-1}), & b = 0 \\ h(s_{t-1} || a_{t-1}), & b = 1, \end{cases}$$

where s_{t-1} is the genuine sibling and s_{t-1}^* the supplied one. Let

$$X^* = \begin{cases} a_{t-1}^* || s_{t-1}^*, & b = 0 \\ s_{t-1}^* || a_{t-1}^*, & b = 1 \end{cases} \quad X = \begin{cases} a_{t-1} || s_{t-1}, & b = 0 \\ s_{t-1} || a_{t-1}, & b = 1. \end{cases}$$

Then $h(X^*) = a_t^* = a_t = h(X)$. Moreover $X^* \neq X$: in the half occupied by the running value, $a_{t-1}^* \neq a_{t-1}$ by minimality of t^* , so the two 2ℓ -bit strings differ in that half. Hence (X^*, X) is a collision for h . The extractor runs the verifier once and compares the two chains level by level, costing $O(d)$ work.

Finally, suppose a PPT adversary $\mathcal{A}$ produced forgeries with non-negligible probability $\varepsilon(\lambda)$. The collision finder $\mathcal{B}$ that samples L (or receives it), runs $\mathcal{A}$, and applies the extractor outputs a collision whenever $\mathcal{A}$ succeeds, hence with probability $\varepsilon(\lambda)$. Collision resistance forces $\varepsilon(\lambda) = \text{negl}(\lambda)$. $\square$

Remark 10.14 (What soundness does and does not promise). Theorem 10.13 is a statement about a *fixed honest root*. It asserts: *relative to a root that genuinely is $\text{root}(L)$, the only leaf value that admits an accepting path at position i is L_i (up to a collision in h)*. It does *not* by itself prevent an adversary who

is free to choose the root from committing to a maliciously crafted tree; that threat is exactly what binding (Proposition 10.6) rules out, and the two properties together establish that the root is a sound, binding commitment with succinct openings. An untrusted party supplying the root makes both necessary: binding ensures the root determines a unique list, and path soundness ensures openings are faithful to it.

10.5 Layer and domain separation

The proofs above tacitly assume that the only way to produce a label is through the intended recurrence. Real attacks exploit the gaps in that assumption. Two distinct hazards demand attention: confusing a leaf with an internal node, and confusing nodes at different depths or trees.

Second-preimage attacks from leaf/node confusion. Suppose, naively, that leaves are arbitrary-length data hashed by the *same* function used internally, so that an internal label $h(A||B)$ is itself a valid leaf value (it has the right length ℓ). Then an adversary can present the internal node $N_{1,0} = h(L_0||L_1)$ as if it were a leaf at the top of a *shallower* tree, producing a shorter authentication path that the verifier — who does not learn the depth — accepts. More generally, absent a way to distinguish “I am a leaf” from “I am an internal node,” a single string can play both roles, and the clean leaf-to-root structure on which Theorem 10.13 relies collapses. This is a genuine second-preimage attack: the tree’s root has a second list explaining it.

Definition 10.15 (Domain separation). A Merkle construction has *domain (and layer) separation* if it forces the function applied at distinct structural positions to differ, by prepending a position-dependent tag to its input. Concretely, fix distinct tags and define

$$\text{LeafHash}(v) = H(0x00 || v), \quad \text{NodeHash}_t(A, B) = H(0x01 || \langle t \rangle || A || B),$$

where $0x00, 0x01$ distinguish the *layer* (leaf vs. internal) and $\langle t \rangle$ is the encoded *level index*, distinguishing internal nodes by depth. Level-0 labels are $N_{0,j} = \text{LeafHash}(L_j)$ and the recurrence becomes $N_{t+1,j} = \text{NodeHash}_{t+1}(N_{t,2j}, N_{t,2j+1})$.

Proposition 10.16 (Domain separation forces honest collisions). *With the tagging of Definition 10.15, any collision extracted by the proofs of Proposition 10.6 or Theorem 10.13 is a collision for H between two inputs sharing the same leading tag, hence a collision in which a leaf is matched against a leaf and a level- t node against a level- t node. In particular no accepting forgery can substitute an internal node for a leaf, or a node at one level for a node at another, without colliding H within a single domain.*

Proof. Sketch. The extractor of Theorem 10.13 finds the smallest level t at which the genuine and forged chains agree while disagreeing just below. The *same* structural rule at that position forms both colliding inputs — both via NodeHash_t if $t \geq 1$, or both via LeafHash if the disagreement is at the leaves — because the verifier’s recomputation applies the position’s prescribed tagged function, as does the honest tree. Hence the two preimages share the leading tag $0x01||\langle t \rangle$ (or $0x00$). The cross-domain confusions are therefore impossible unless H collides *within* a fixed domain, which collision resistance of H forbids. $\square$

Remark 10.17 (Unbalanced trees and padding). Deployed failures (notably Bitcoin’s CVE-2012-2459, where a duplicate-last-node rule let two different transaction lists share a root) teach a single lesson: layer-separate leaves from internal nodes, make every padding or duplication rule injective on admissible lists (e.g. by binding the true leaf count into the root), and feed NodeHash only fixed-length inputs so that no length-extension structure of the underlying hash can intervene.

Remark 10.18 (Algebraic hashes and the arithmetic setting). When the Merkle path must be verified *inside a zero-knowledge circuit* (Subsection 10.8), arithmetic gates over a finite field $\mathbb{F}_p$ evaluate the compression function, and bit-oriented hashes like SHA-256 become enormously expensive (tens of thousands of constraints per call). One therefore uses an *algebraic* hash — Poseidon, Rescue, or in Orchard the Sinsemilla function — with compression $h : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$: Poseidon and Rescue are low-degree permutation-based maps computable in few constraints, whereas Sinsemilla is cheap in constraints via lookups and incomplete elliptic-curve additions, not via low degree. For Poseidon and Rescue, domain and layer separation arise not from byte tags but from distinct field constants (initial capacity values or round constants) fed into the permutation; Sinsemilla instead retains the tag mechanism of Definition 10.15 in encoded form: Orchard’s MerkleCRH prepends the 10-bit encoded layer index to the two 255-bit child encodings, all layers sharing one instance, while separation between distinct Sinsemilla instances comes from the personalisation string fed to hash-to-curve to derive the instance-specific base point Q (the generator table S is shared across instances). The security argument is identical, with “collision” read as a collision of the field-valued compression function. Sinsemilla in particular is collision resistant under a discrete-log assumption, fusing the hashing and the commitment into one Pedersen-like map.

10.6 Append-only and incrementally updatable trees

In the applications of concern the leaf set is not fixed once and for all: note commitments are *appended* as new transactions arrive, and the tree grows monotonically. Recomputing the root from scratch at each insertion would cost $\Theta(m)$ hashes; $O(\log m)$ per append is the objective. The key observation is that appending a leaf changes only the labels on the single path from the new leaf to the root, and that these labels depend only on a small amount of retained state called the *frontier*.

Definition 10.19 (Append-only Merkle tree). Fix a maximum depth d , giving capacity 2^d . An *append-only* (or *incremental*) Merkle tree maintains a position counter m (the number of leaves inserted so far, $0 \leq m \leq 2^d$) and supports a single mutating operation $\text{Append}(v)$, which places v at leaf index m and increments m . Empty leaf slots $m, m+1, \dots, 2^d-1$ carry a fixed *empty-leaf* value $\perp_0$; the empty-subtree labels $\perp_t$ at level t follow by precomputation from $\perp_{t+1} = \text{NodeHash}_{t+1}(\perp_t, \perp_t)$. The root after m appends is root_m , the root of the depth- d tree whose first m leaves are the inserted values and whose remaining leaves are $\perp_0$.

Definition 10.20 (Frontier). The *frontier* of an append-only tree holding m leaves is the minimal state needed to (i) compute the current root and (ii) update incrementally on the next Append . It consists, for each level $t \in \{0, 1, \dots, d-1\}$, of at most one “carry” node: the label of the *left* child of the next node to be completed at level $t+1$, retained precisely when leaf index m has a 1 bit at position t , the completed left sibling awaiting its right neighbour. Equivalently, writing m in binary as $b_{d-1} \dots b_0$, the frontier stores, for each t with $b_t = 1$, the already-finalised left-sibling label at level t along the rightmost filled path. The frontier thus comprises at most d labels — one per set bit of m — i.e. $O(\log m)$ state.

The frontier is exactly the set of subtree roots that are “complete and waiting for a right neighbour.” Equivalently, the binary representation of m is a list of completed perfect subtrees of distinct heights, and the frontier holds their roots; this is the Merkle analogue of a binary counter, and Append is the analogue of incrementing it, where a “carry” at level t corresponds to combining two height- t subtrees into one of height $t+1$.

Proposition 10.21 (Incremental append in logarithmic time). *From the frontier of an append-only tree holding $m < 2^d$ leaves, the operation $\text{Append}(v)$ computes the new frontier and the new root root_{m+1} using at most d evaluations of NodeHash and $O(d)$ additional work, while storing only $O(d)$ labels.*

Proof. Place v at level 0 as the running label $c \leftarrow v$ and let $t \leftarrow 0$. The new leaf sits at index m ; inspect the bits of m from least significant upward. While bit t of m equals 1, the node at level t on the new leaf's path is a *right* child whose left sibling is the frontier carry f_t stored at level t ; set $c \leftarrow \text{NodeHash}_{t+1}(f_t, c)$, clear that carry, and increment t (this is the carry-propagation of the binary counter). When bit t of m equals 0, the path node at level t is a *left* child with no completed right sibling yet; store c as the new carry f_t at level t and stop the propagation. Let z be the number of trailing 1-bits of m . Carry propagation costs exactly z hashes and deposits f_z at level z : the root of the just-completed height- z subtree containing the new leaf, hence the root-path node at that level. To obtain root_{m+1} , fold this value upward from level z to level d , combining at each level with the stored carry f_t (as left sibling) or the precomputed empty-subtree label $\perp_t$ (as right sibling) — one hash per level, i.e. $d - z$ hashes (zero if $z = d$). The total is exactly d evaluations of NodeHash , and the only retained state is the at-most- d carries, i.e. the frontier of $m + 1$ leaves. $\square$

Example 10.22 (Frontier as a binary counter). Take $d = 3$ (capacity 8) and insert leaves v_0, v_1, v_2 in turn. After v_0 ($m = 1 = (001)_2$): one completed height-0 subtree; frontier = $\{f_0 = v_0\}$. After v_1 ($m = 2 = (010)_2$): inserting v_1 at index 1 (a right child) triggers a carry: $c = \text{NodeHash}_1(v_0, v_1)$, which is a completed height-1 subtree; frontier = $\{f_1 = c\}$, and f_0 is cleared. After v_2 ($m = 3 = (011)_2$): v_2 at index 2 is a left child, so it becomes a new height-0 carry; frontier = $\{f_1 = \text{NodeHash}_1(v_0, v_1), f_0 = v_2\}$ — exactly the two set bits of 3. Each append touched only the path to the new leaf, never the whole tree.

Definition 10.23 (History binding). An append-only tree is *history-binding* when the protocol treats every historical root as an acceptable commitment in its own right: each membership proof verifies against the root it was formed for, and appending further leaves never invalidates a previously valid (root, leaf, path) triple for that root. In Orchard and similar systems, the chain records each block's note-commitment-tree root, a spend may prove membership against *any* recorded historical root, and the soundness of each individual proof is exactly Theorem 10.13 applied to the corresponding fixed root.

Remark 10.24 (Why append-only, not arbitrary update). Deletion or in-place update of leaves could also be supported; each still touches only one root-to-leaf path and costs $O(\log m)$ given the relevant siblings. But append-only trees enjoy a crucial monotonicity: once a leaf is inserted it is never removed, so a membership proof, once valid against the root at insertion time, remains a valid proof against *that* historical root forever. This is exactly the property a shielded pool requires — a note, once added, can always be spent by proving against some past root — and it obviates the need to track a moving target. The cost is that the verifier must accept a window of historical roots rather than a single current one.

10.7 Vector commitments

Having constructed and analysed the Merkle tree, we now name the abstraction it instantiates: the *vector commitment*, a short commitment to an ordered list with short, position-indexed openings. (The unordered sibling notion, the *cryptographic accumulator*, packages the same construction by set membership instead of position; Orchard never needs that abstraction, so we do not develop it.)

Definition 10.25 (Vector commitment). A *vector commitment* (VC) is a tuple of PPT algorithms (Setup, Commit, Open, Vf):

- $\text{Setup}(1^\lambda, n) \rightarrow \text{pp}$ produces public parameters for vectors of length n ;
- $\text{Commit}(\text{pp}, (v_0, \dots, v_{n-1})) \rightarrow (C, \text{aux})$ outputs a short commitment C and auxiliary state;
- $\text{Open}(\text{pp}, i, \text{aux}) \rightarrow \pi_i$ produces a proof that position i holds v_i ;
- $\text{Vf}(\text{pp}, C, i, v, \pi) \rightarrow \{\text{accept}, \text{reject}\}$ verifies an opening.

It is *correct* if honest openings always verify, and *position-binding* if no PPT adversary can output C, i, v, v', π, π' with $v \neq v'$ and both $\text{Vf}(\text{pp}, C, i, v, \pi)$ and $\text{Vf}(\text{pp}, C, i, v', \pi')$ accepting, except with negligible probability. The VC is *succinct* if $|C|$ and $|\pi_i|$ are bounded by a fixed polynomial in λ and $\log n$ (i.e. independent of n up to logarithmic factors).

Theorem 10.26 (Merkle trees are succinct position-binding vector commitments). *The Merkle construction, with Setup sampling h , Commit = root (and aux the full tree), Open returning the authentication path, and Vf = Verify, is a correct, position-binding vector commitment of commitment size ℓ and opening size $O(\ell \log n)$, assuming h is collision resistant.*

Proof. Correctness is Proposition 10.9. For position-binding, suppose an adversary outputs a root C , an index i , and two accepting openings (v, π) and (v', π') with $v \neq v'$. Running the verifier on each yields two recomputed chains from the leaf up to the common root C ; they agree at the top (both equal C) and disagree at the bottom ($v \neq v'$). Both openings are accepted at the same index i , so by the index-consistency check of Definition 10.8 both carry i 's direction bits, and at each level the two chains apply h with the running value on the same side. The “first level of agreement” extraction of Theorem 10.13, applied to these two chains, therefore yields two distinct equal-length inputs to h with the same image, that is, a collision. (This argument requires *no honest reference tree*: it pits the two forged openings against each other, so it binds even a maliciously chosen root, which is the strong form of position-binding.) Collision resistance bounds the success probability by $\text{negl}(\lambda)$. The size bounds are Proposition 10.10. $\square$

Remark 10.27 (Position-binding vs. path soundness). Theorem 10.13 fixed an honest root and forbade opening a position to anything but its true value; Theorem 10.26 forbids opening a single position two *different* ways, even under an adversarial root. The latter is precisely the property a verifier requires who receives a root from an untrusted source, and it is what “commitment” should mean: C determines at most one value per position. Both reduce to the same collision extraction; the difference lies only in which pair of chains we collide.

10.8 Privacy-preserving membership via zero-knowledge paths

We can now assemble the primitive that Orchard actually uses. The membership proof of Subsection 10.3 is succinct but *not private*: revealing the authentication path discloses the leaf value (e.g. the note commitment) and the index i , and the siblings can leak structural information. For a shielded payment system this is fatal: spending a note must not reveal *which* note the spender spends. The remedy is to prove the existence of a valid path *in zero knowledge*, treating the path as a private witness.

Definition 10.28 (Membership relation for ZK). Fix the domain-separated Merkle construction

of depth d . Define the *membership relation*

$$\mathcal{R}_{\text{mem}} = \left\{ \left(\underbrace{\text{rt}}_{\text{statement}} ; \underbrace{(\underbrace{v, i, \pi}_{\text{witness}})} \right) : \text{Verify}(\text{rt}, i, v, \pi) = \text{accept} \right\},$$

in which the *statement* is the public root rt and the *witness* comprises the leaf v , its index i , and the authentication path π . A zero-knowledge argument for $\mathcal{R}_{\text{mem}}$ enables a prover to convince a verifier that “some leaf is in the tree with root rt ” while revealing none of v , i , or π .

We assume a *zk-SNARK* as Section 9 develops it (Definition 9.26): a non-interactive argument with three properties — *completeness* (honest provers convince), *knowledge soundness* (an accepting prover “knows” a witness, formalised by an extractor), and *zero knowledge* (the proof reveals nothing beyond the truth of the statement, formalised by a simulator). The membership circuit encodes exactly the verifier’s folding computation ($\dagger$).

Definition 10.29 (In-circuit path verification). The *Merkle-path circuit* C_{mem} of depth d takes as public input the root rt and as private (witness) inputs the leaf v , the direction bits $b_0, \dots, b_{d-1}$, and the siblings $s_0, \dots, s_{d-1}$. It implements d copies of the compression h and the conditional swap of ($\dagger$) — gate-for-gate, the swap selects (a_t, s_t) or (s_t, a_t) according to b_t — and enforces the single output constraint $a_d = \text{rt}$. Its size is $d \cdot (\text{cost of one } h) + O(d)$ constraints. The circuit witnesses the position only through the bits $b_0, \dots, b_{d-1}$, the binary decomposition of i , so the index-consistency check of Definition 10.8 is enforced structurally — exactly as Orchard’s circuit decomposes the witnessed position into the swap bits.

Theorem 10.30 (Privacy-preserving membership). Let h be collision resistant and let (P, V) be a *zk-SNARK* for the relation $\mathcal{R}_{\text{mem}}$ (equivalently, for the satisfiability of C_{mem}). Then the protocol “prover sends a SNARK proof for rt , verifier checks it” is a membership argument that is

1. complete: a prover holding a genuine leaf and its path produces an accepting proof;
2. sound: any prover producing an accepting proof for an honest root $\text{rt} = \text{root}(L)$ knows (via the SNARK extractor) a witness (v, i, π) with $\text{Verify}(\text{rt}, i, v, \pi) = \text{accept}$; by Theorem 10.13, that v equals the genuine leaf L_i unless it thereby finds a collision in h — so the proven leaf is a real member;
3. private (zero-knowledge): the proof reveals nothing about v , i , or π beyond the bare fact that some valid leaf exists under rt .

Proof. Completeness is the completeness of the SNARK composed with Proposition 10.9: a genuine (v, i, π) satisfies C_{mem} , so the verifier accepts the honest prover’s proof. For soundness, knowledge soundness of the SNARK yields an extractor that, from any accepting prover, outputs a witness (v, i, π) satisfying C_{mem} , i.e. with $\text{Verify}(\text{rt}, i, v, \pi) = \text{accept}$. If rt is an honest root $\text{root}(L)$ and the extracted $v \neq L_i$, then (v, π) is a path forgery and Theorem 10.13 converts it into a collision for h ; under collision resistance this occurs only with negligible probability, so the extracted v is genuinely $L_i \in L$. Zero knowledge carries over verbatim from the SNARK: its simulator produces, from the statement rt alone, a proof indistinguishable from the real one, so the real proof leaks nothing about the witness (v, i, π) — in particular not the spent note’s value or position. $\square$

Remark 10.31 (Why the path is hidden but the root is public). The asymmetry is essential. The root must be *public* so that the verifier can check the proof against a state on which everyone agrees (a previously published note-commitment-tree root); soundness is relative to that public root, exactly as in Remark 10.14. The path must be *private* so that observers cannot link the spend to a particular

note. Because the SNARK is succinct, the verifier checks a proof whose *size* is essentially independent of d —polylogarithmic in it—even though the underlying witness has d siblings (pairing-based schemes such as KZG achieve constant-size proofs with fast verification; Halo 2’s IPA-based proofs grow logarithmically in size, but their verification is linear in the circuit size, amortised in deployment by batch-verifying many proofs in a single multiexponentiation—the accumulation technique that would instead make per-proof work sublinear is not deployed): the construction compresses the membership proof while simultaneously hiding it. This is the precise sense in which “a Merkle root over note commitments gives a privacy-preserving membership primitive when the path is proved in zero knowledge.”

Example 10.32 (A shielded spend, end to end). In Orchard, each shielded output appends a *note commitment* cm — itself a hiding, binding commitment to the note’s value, recipient, and randomness — as a leaf of an append-only Merkle tree built with the Sinsemilla compression function (strictly, the leaf is the extracted x -coordinate $\text{cm}_x = \text{ExtractP}(\text{cm})$, the 255-bit base-field element matching the child encodings of Remark 10.18). Periodically the protocol publishes the current root (the “anchor”) on chain. To spend a note, the owner:

1. locates the leaf cm and reads off its authentication path π and position i from the public tree;
2. chooses a recent anchor $\text{rt} = \text{root}_m$ that contains cm ;
3. proves, in zero knowledge, the statement rt with witness (cm, i, π) : that cm verifies against rt (membership), that the note opening is well-formed, and that the proof computes a derived *nullifier* correctly to prevent double-spending;
4. publishes the proof and the nullifier (but neither cm , nor i , nor π).

By Theorem 10.30, the spend convinces the network that the spender owns *some* note genuinely added to the tree — soundness rests on collision resistance of Sinsemilla and knowledge soundness of the proof system — yet the network learns nothing about *which* note, since zero knowledge hides the entire path. The Merkle root thus serves as a succinct, binding, privacy-preserving commitment to the set of all notes ever created, and the authentication path, proved in zero knowledge, is the membership primitive on which shielded spends rest.

10.9 Summary

We constructed the binary Merkle tree from a single collision-resistant compression function (Definition 10.2); showed that the root is a binding commitment to the leaf list (Proposition 10.6); defined authentication paths and proved them complete, logarithmic in size, and sound — forging one yields a collision (Propositions 10.9, 10.10, Theorem 10.13); explained why layer and domain separation must hold to make those reductions honest (Subsection 10.5); developed append-only trees and the frontier that updates the root in $O(\log m)$ per insertion (Subsection 10.6); abstracted the construction as a succinct position-binding vector commitment (Subsection 10.7); and finally combined a Merkle root over note commitments with a zero-knowledge proof of an authentication path to obtain the privacy-preserving membership primitive at the heart of Orchard (Theorem 10.30, Example 10.32). The single recurring proof technique — “disagreement at the leaves plus agreement at the root forces a collision on the path” — underlies binding, path soundness, and position-binding alike, and is the essential idea of the entire construction.

Part III

Halo 2 Guide: The Halo 2 proof system, from arithmetisation to recursion

Abstract

Assuming the mathematics of the *Math Guide* and the cryptographic primitives of the *Crypto Guide*, this volume of *The Zcash Arboretum* develops the Halo 2 proof system: the polynomial-IOP design pattern, a rigorous account of knowledge soundness and the algebraic group model, recursive proof composition and accumulation schemes, and finally Halo 2 itself — PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment and the Halo accumulation trick, and the complete protocol.

1 Introduction

This is the *Halo 2 Guide*, the third volume of *The Zcash Arboretum*, developing the cryptography behind Zcash’s Orchard protocol. The first two volumes — the *Math Guide* and the *Crypto Guide* — constructed, from the ground up, the mathematics (finite fields, elliptic curves, polynomials and evaluation domains, probability) and the cryptographic primitives (hash functions, commitments, Σ -protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes) that this volume assumes. The present volume assembles those ingredients into a *proof system*.

The development proceeds in four steps. First, we make the polynomial-IOP design pattern precise: the recipe “encode a computation as polynomial identities over an evaluation domain, then compile to a succinct argument with a polynomial commitment.” Next, we treat knowledge soundness rigorously, including the algebraic group model and the tree-of-transcripts extractor that the inner-product argument requires. We then motivate recursive proof composition and develop the accumulation schemes that make it practical. Finally, we assemble Halo 2 itself: PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment with a worked example and its accumulation (the “Halo trick”), and the complete seven-phase protocol. The *Ironwood Guide* subsequently uses this proof system to construct a shielded payment protocol.

2 Polynomial IOPs and the SNARK design pattern

The modern SNARK design pattern factors the protocol into two clean layers: a *polynomial interactive oracle proof* (PIOP) for the relation, and a polynomial commitment scheme (the *Crypto Guide*) that realises the PIOP’s oracles. This factoring, due in its current form to Bünz, Fisch, Szepieniec, and others, has substantially clarified the nature of SNARKs.

2.1 The polynomial IOP, formally

Definition 2.1 (Polynomial IOP). A *polynomial interactive oracle proof* for a relation $\mathcal{R}$ is a public-coin interactive protocol where in each round:

- The prover sends one or more *polynomial oracles* $p_i \in \mathbb{F}[X]$ (each of bounded degree).
- The verifier sends a random challenge $c \in \mathbb{F}$.

At the end of the protocol, the verifier may query each oracle at a small number of points, learning the polynomial’s evaluations at those points. The verifier’s final accept/reject is a deterministic function of the challenges, evaluations, and the public input x .

We define completeness, soundness, and knowledge soundness as usual.

A PIOP differs from a vanilla interactive argument in one essential respect: the verifier does not read the prover’s messages in full — it queries only a few evaluations of each polynomial oracle. This is what renders the verifier “succinct” even when the polynomials themselves have large degree.

2.2 Compiling a PIOP into a SNARK

Construction 2.2 (PIOP-to-SNARK compiler). Given a PIOP Π for $\mathcal{R}$, a PCS scheme (Setup, Commit, Open, Verify) for $\mathbb{F}[X]$, and a random oracle H , the compiler produces the SNARK:

1. Setup: run the PCS setup to obtain pp .
2. Prover: simulate Π , replacing each polynomial oracle p_i by its PCS commitment C_i , and each verifier-challenge / oracle-query round by the corresponding PCS opening. Use Fiat–Shamir (the *Crypto Guide*) to derive challenges from the transcript.
3. Verifier: parse the transcript, recompute Fiat–Shamir challenges, verify each PCS opening, and run the PIOP’s final accept/reject check.

A PIOP is *round-by-round* knowledge sound with error κ_Π if partial transcripts admit a “doomed” labelling under which a claim with no extractable witness starts doomed, no prover message lifts the label except with probability κ_Π over the verifier’s next challenge, and a complete doomed transcript is rejected; equivalently, the PIOP withstands a *state-restoration* prover, one permitted to rewind the verifier to any earlier round and redraw its challenge, the two notions implying each other with comparable error.

Theorem 2.3 (Soundness preservation). *If Π has round-by-round (equivalently, state-restoration) knowledge soundness κ_Π and the PCS is extractable with extraction error κ_{PCS} (the PCS definition of the *Crypto Guide*), the compiled SNARK has knowledge soundness $O(Q \cdot (\kappa_\Pi + \kappa_{\text{PCS}}))$ in the random oracle model, where Q is the adversary’s RO-query budget.*

The query budget multiplies κ_Π as well as κ_{PCS} : step 2 derives the PIOP challenges by Fiat–Shamir, so a cheating prover that re-randomises a commitment redraws every subsequent challenge, obtaining up to Q independent draws of each — the grinding attack. Round-by-round soundness confines this loss to the single factor Q ; for an r -round PIOP with plain knowledge soundness only, the naive Fiat–Shamir analysis loses a factor $Q^{O(r)}$ (§3).

This factoring is of considerable utility: research on PIOPs (PLONK, Marlin, Spartan, HyperPlonk, ...) proceeds independently of research on polynomial commitment schemes, and any compatible pair combines into a new SNARK that inherits the PIOP’s relation and the commitment scheme’s cryptographic profile. This document develops the one PCS that Halo 2 uses — the inner-product argument (§5.5) — and treats the PIOP/PCS split only insofar as it clarifies the Halo 2 construction.

2.3 A worked example: a PIOP for univariate evaluation

To render the abstraction concrete, consider a minimal PIOP for the simplest non-trivial relation: “the prover knows a polynomial p of degree $\leq d$ such that $p(z) = y$ for a public (z, y) .”

Construction 2.4 (PIOP for univariate evaluation). On input (z, y) and prover witness p :

1. Prover sends the polynomial oracle p and the quotient oracle $q := (p - y)/(X - z)$.

2. Verifier samples a random challenge $r \in \mathbb{F}$.
3. Verifier queries p and q at r .
4. Verifier accepts iff $p(r) - y = q(r) \cdot (r - z)$.

Completeness is immediate (q is a polynomial since $p(z) - y = 0$). Soundness follows from Schwartz–Zippel: if $(X - z) \nmid (p(X) - y)$, then $p(X) - y - q(X)(X - z)$ is a non-zero polynomial of degree $\leq d$, and its random evaluation is zero with probability $\leq d/|\mathbb{F}|$.

Compiling this PIOP with the IPA-PCS, applying Fiat–Shamir, yields a non-interactive SNARK for the same relation. The PCS handles polynomial hiding and binding; the PIOP handles the relation logic. This separation is precisely the modular SNARK design pattern.

2.4 PLONK as a PIOP

PLONK (§5.1) is a more elaborate PIOP. Its oracles are:

- One advice polynomial per advice column.
- A permutation running-product polynomial Z_{perm} .
- Two permuted lookup columns A', S' per lookup.
- One lookup running-product polynomial per lookup.
- A quotient polynomial h (split into chunks).

Its challenges are, in order: θ (lookup column compression), β, γ (the permutation and lookup products), y (the gate-equation combination), and x (the final evaluation point). Its final check is the PLONK gate equation $C(x) = h(x) \cdot Z_H(x)$, which the verifier checks from the evaluations of the oracles at x (and possibly ωx).

This is precisely the PIOP that phases 1–5 of the Halo 2 protocol of §5.7 compile, phase 5’s claimed evaluations answering its oracle queries; phases 6–7 bind those answers to the commitments — the multipoint reduction (the challenges $x_1, \dots, x_4$ and the combined polynomial f^*) followed by a single IPA-PCS opening of f^* .

3 A rigorous treatment of knowledge soundness

We now develop knowledge soundness with the care needed for arguments about modern SNARKs. The treatment combines the rewinding-extractor framework (the *Crypto Guide*) with the algebraic group model.

3.1 Definitional refinements

The bare definition of the *Crypto Guide* suffices for Σ -protocols but is awkward for compound systems. We record the following standard refinement.

Remark 3.1 (Witness-extended emulation). An interactive argument $(\mathcal{P}, \mathcal{V})$ has *witness-extended emulation* if there is a PPT emulator $\mathcal{E}$ such that for every PPT prover $\mathcal{P}^*$ and every input x :

- Run $\mathcal{P}^*$ honestly with $\mathcal{V}$; let τ be the transcript and b the verifier’s accept/reject bit.
- Run $\mathcal{E}^{\mathcal{P}^*}(x)$; it outputs (τ', b', w) .

The distributions of (τ, b) and (τ', b') are computationally indistinguishable, and moreover whenever $b' = 1$, $(x, w) \in \mathcal{R}$ except with negligible probability.

WEE captures simultaneously two properties: “the extractor produces a witness whenever the protocol accepts” and “the extractor’s external behaviour resembles a normal protocol execution.” It is the appropriate notion for arguments embedded in larger protocols, such as the batched verification of many proofs at once (§5.6); plain knowledge soundness, however, suffices for this volume’s statements.

3.2 The algebraic group model

The IPA’s knowledge-soundness proof proceeds through the tree extractor of §5.5, with knowledge error $O(d/|\mathbb{F}|)$. The interactive tree extractor costs $3^{\log_2 d} = d^{\log_2 3}$ transcripts and runs in expected polynomial time (§3.3); the looseness arises for the non-interactive argument, where the naive Fiat–Shamir analysis of the $\log d$ -round protocol loses a factor $q^{O(\log d)}$ in the adversary’s random-oracle query count q — quasi-polynomial for polynomial q . The *algebraic group model* (AGM; Fuchsbauer, Kiltz, Loss; CRYPTO 2018) removes this loss by restricting the adversary’s group operations.

Definition 3.2 (Algebraic adversary). An *algebraic adversary* against a group $\mathbb{G}$ is a PPT algorithm that, whenever it outputs a group element $P \in \mathbb{G}$, additionally outputs an explicit linear combination $P = \sum_i [a_i]Q_i$ expressing P as a combination of previously-seen group elements Q_i .

The AGM sits between the standard model (no restriction on the adversary) and the generic group model (group operations available only through an oracle). Most natural adversaries are algebraic — they compute group elements by known scalar multiplications and additions — so the heuristic is regarded as conservative. In the AGM the forking loss disappears: an algebraic prover outputs every group element together with an explicit linear combination of the elements it has seen, so the extractor reads candidate witnesses directly off these representations instead of reassembling them from forked transcripts. Ghoshal and Tessaro (CRYPTO 2021) carry out this analysis for Bulletproofs-style inner-product arguments, proving tight state-restoration soundness — and hence tight Fiat–Shamir security — in the AGM. The accumulation analysis of BCMS20 (Bünz, Chiesa, Mishra, Spooner; TCC 2020), by contrast, does not invoke the AGM: it proves security in the random oracle model under the DL assumption (Theorem 5.7).

3.3 Tree-extractor analysis

To render the tree extractor concrete, consider the key step at each round of the IPA halving (see §5.5):

Lemma 3.3 (Single-round extraction). *In a single round of the IPA at depth j , given three accepting sub-transcripts that share the round’s first message (L_j, R_j) but have three challenges u_j with pairwise distinct squares ($u_j \neq \pm u'_j$), the extractor recovers the round- j witness halves $\vec{a}_{\text{lo}}^{(j)}, \vec{a}_{\text{hi}}^{(j)}$ together with the consistency of the L_j, R_j cross-terms via 3×3 Vandermonde inversion (rows $(u_j^{-2}, 1, u_j^2)$), since the folded relation is quadratic in u_j .*

Sketch. The three accepting transcripts give three accepting verifier equations. Because the folded relation is quadratic in u_j (the three monomials $u_j^{-2}, 1, u_j^2$), treating these as three linear equations in three unknowns (the cross-terms and the folded commitment), the system has a unique solution because the 3×3 Vandermonde determinant with rows $(u_j^{-2}, 1, u_j^2)$ is non-zero exactly when the squares u_j^2 are pairwise distinct ($u_j \neq \pm u'_j$). Excluding the bad challenge collisions contributes an additive $O(1/|\mathbb{F}|)$ term per round to the knowledge error. $\square$

The full extraction requires three accepting transcripts at each of the $k = \log_2 d$ rounds, giving a tree of $3^k = d^{\log_2 3} \approx d^{1.585}$ leaves — polynomial in d , though more than the $O(d)$ a binary tree would cost. In the AGM, each prover invocation is direct (the extractor reads the algebraic representation);

in the standard model with forking, each invocation costs $1/\varepsilon$ expected queries to the prover, where ε is the cheating prover’s acceptance probability, yielding total cost $O(d^{\log_2 3} \cdot \text{poly}(\lambda)/\varepsilon)$.

3.4 Concrete versus asymptotic security

The literature on knowledge soundness contains numerous tightness gaps. The more important ones include:

- Bulletproofs’ naive Fiat–Shamir knowledge-soundness analysis loses a factor $q^{O(\log d)}$, exponential in the round count $\log d$ with base the query count q ; the interactive standard-model extraction itself is polynomial ($d^{\log_2 3}$ transcripts, expected polynomial time).
- Halo 2’s deployed parameters assume the AGM and/or the random oracle model heuristically.

This reflects the practical reality of deployed SNARKs: tight reductions in idealised models, loose reductions in standard models, and deployment selecting the former.

4 Recursive proof composition and accumulation schemes

We now turn to recursive proof composition: proving, inside one proof, that another proof verifies. This is what makes Halo 2’s combination of “no trusted setup” with recursion possible. Orchard’s mainnet deployment does not recurse, but the accumulation structure shapes the deployed verifier nonetheless — the deferred MSM and the batched verification of §5.6 — so we develop it with care.

4.1 Recursion and its two classical obstacles

Consider a computation iterated many times, $z_n = f^{(n)}(z_0) := f(f(\dots f(z_0)\dots))$ — Valiant’s *incrementally-verifiable computation* (TCC 2008) — or, more generally, computations that branch and merge, each consuming the proofs of its predecessors — *proof-carrying data* (Chiesa, Tromer; ICS 2010). The natural construction is *recursive SNARK composition*: at step $i + 1$ the prover proves “ $z_{i+1} = f(z_i)$ ”, and there exists a proof π_i that the verifier circuit for step i accepts.” One proof then attests to the entire history, and the per-step proving cost is independent of the history’s length.

For this to work, the SNARK’s verifier must itself be efficiently expressible as a circuit, and here the classical instantiations meet two obstacles:

1. Pairing-based SNARKs have constant-size, constant-time verifiers, but the verifier evaluates a pairing — costly to express algebraically inside a circuit, since it requires extension-field arithmetic — and these SNARKs need a trusted setup besides.
2. Transparent SNARKs built on the IPA need no pairing and no trusted setup, but their verifiers run in linear time — the $O(d)$ MSM of §5.5.2. A linear-time check dominates the verifier circuit and negates the per-step efficiency.

Halo (Bowe–Grigg–Hopwood 2019) defeats obstacle 2 directly — deferring the linear-time part of verification instead of recomputing it at every step — and sidesteps obstacle 1 by using the IPA, which involves no pairing. BCMS20 formalised the deferral as an *accumulation scheme*.

4.2 Accumulation schemes

Definition 4.1 (Accumulation scheme, informal). For a predicate Φ (for example, “this IPA opening is correct”), an *accumulation scheme* consists of:

- An *accumulator*, a compact data structure representing a batch of claims about Φ .
- A *folding* algorithm: given an old accumulator and a new claim about Φ , it produces a new

accumulator that represents both — the new accumulator is valid only if the old one was valid and the new claim holds. Folding is fast: $O(\log d)$ in the IPA instantiation below.

- A *decider*, which is expensive (linear-time) but which the scheme invokes only once at the very end to validate the accumulator. If the decider accepts, every claim folded in so far is true.

For the IPA-PCS such a scheme exists, and it resolves the recursion problem in the IPA setting: the in-circuit verifier checks only the $O(\log d)$ “succinct” part of an IPA proof and folds the $O(d)$ “deferred” MSM check into a running accumulator. The decider pays the deferred work once, at the end of the entire chain. We state the scheme precisely as Theorem 5.7, once the IPA construction of §5.5 is in hand.

4.3 The necessity of a cycle of curves

The accumulation scheme’s in-circuit folding involves arithmetic on group elements of the underlying curve. For this to be efficient inside the circuit, the group’s base field must equal the circuit’s native field. Since the circuit’s native field is the scalar field of the curve on which the prover commits the SNARK — which in turn is generally different from the curve’s own base field — a difficulty arises: an E_p -IPA proof’s group elements have coordinates in $\mathbb{F}_p$, so verifying it natively would require a circuit whose native field is $\mathbb{F}_p$. But a SNARK’s verifier circuit naturally operates over the curve’s *scalar* field $\mathbb{F}_q$ (where $q = |E_p|$): the Fiat–Shamir challenges and folding scalars are $\mathbb{F}_q$ elements. Curves with $p = q$ do exist — the *anomalous* curves, with trace of Frobenius 1 — but on them the discrete logarithm is computable in polynomial time (Smart; Semaev; Satoh–Araki, 1998–99), so no *secure* curve has $\mathbb{F}_p = \mathbb{F}_q$, and no one curve E_p can host both the proof’s coordinates and its own verifier circuit natively.

The resolution is a 2-cycle of elliptic curves, where $|E_p| = q$ and $|E_q| = p$. Then:

- A circuit committed on E_q verifies a proof generated over E_p ; that circuit’s native field is the scalar field of E_q , namely $\mathbb{F}_p$ (since $|E_q| = p$) — which coincides with the coordinate field of E_p , enabling native manipulation of E_p ’s group elements.
- A circuit over $\mathbb{F}_q$ verifies the next proof, generated over E_q .
- Proofs alternate between the two curves.

This is precisely the Pasta cycle of the *Math Guide*, where the two curves are Pallas (over $\mathbb{F}_{p_{\text{Pallas}}}$, order p_{Vesta}) and Vesta (over $\mathbb{F}_{p_{\text{Vesta}}}$, order p_{Pallas}).

5 The Halo 2 proof system

We now assemble the abstractions of the *Crypto Guide*–§4 into the concrete proof system Halo 2: a transparent PLONKish SNARK whose polynomial commitment is the inner-product argument over the Pasta cycle. This section develops Halo 2 to the level of detail required to state *what an Orchard Action proof actually is*, the subject of the *Ironwood Guide*.

5.1 PLONKish arithmetisation

Definition 5.1 (PLONKish constraint system). Fix a prime field $\mathbb{F}$ and an integer k defining a multiplicative subgroup $H := \{\omega^0, \dots, \omega^{n-1}\} \subset \mathbb{F}^*$ of size $n := 2^k$, where ω is a primitive n -th root of unity. A *PLONKish circuit* consists of:

- Columns partitioned into *fixed* (preprocessed), *advice* (prover-witness), and *instance* (public-input).

- A maximum constraint degree d .
- A finite sequence of *custom gates* $g_t(\vec{x}_0, \vec{x}_1, \dots) = 0$ — multivariate polynomials of degree $\leq d$ in column values at the current row and at offsets $\omega^r X$ for fixed small r .
- A subset $S \subseteq H \times [\text{\#cols}]$ of *equality-constraint cells* controlled by a permutation σ .
- A set of *lookup arguments*: input tuples $(A_0, \dots, A_{m-1})$ and table tuples $(S_0, \dots, S_{m-1})$, with the assertion that for every row $i \in [0, n)$, $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ matches some row of the table.

Vanilla PLONK (Gabizon–Williamson–Ciobotaru, 2019) employs a fixed five-selector universal gate

$$q_M(X)a(X)b(X) + q_L(X)a(X) + q_R(X)b(X) + q_O(X)c(X) + q_C(X) \equiv 0 \pmod{Z_H(X)},$$

where $Z_H(X) := X^n - 1$ is the *vanishing polynomial* of the domain H (it is zero exactly on H , so “ $\equiv 0 \pmod{Z_H}$ ” means “zero at every row”). “PLONKish” generalises this by permitting designer-chosen gates of higher arity and degree, each gated by a selector $q_i(X)$ vanishing on rows where the gate is inactive. The Orchard Action circuit uses ten advice columns plus a small fixed-column lookup table for Sinsemilla; its custom gates encode the algebraic constraints of the Action statement (the *Ironwood Guide*) together with the complete- and incomplete-addition arithmetic for in-circuit elliptic-curve operations (the *Math Guide* names the distinction).

Polynomial representation. The Lagrange basis $\ell_j(X)$ for H , where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$, interpolates each column into a polynomial of degree $< n$ over H . The prover commits to each polynomial via the IPA-PCS (§5.5). All subsequent arguments are *polynomial identities* on these committed polynomials, verifiable by querying them at random points.

5.2 From statement to circuit: arithmetisation in practice

Definition 5.1 states what a PLONKish circuit *is*; it does not say how a statement *becomes* one. The deployed code is organised into *chips*, reusable circuit components each packaging a set of columns, the gates and lookups declared on them, and the synthesis code that fills their cells. This subsection traces that passage: a real custom gate (the ECC chip’s incomplete point addition), two real lookup declarations (the ten-bit range check and the Sinsemilla generator table), and the synthesis machinery that turns witness data into the column vectors on which every argument of this section operates. Our sources are `halo2_proofs` 0.3.2 and `halo2_gadgets` 0.4.0, as resolved by `orchard` 0.13.1, the version we cite, with file paths relative to each crate’s `src/`. The deployed `orchard` (0.15.0) keeps `halo2_proofs` 0.3.2 but resolves `halo2_gadgets` 0.5.0, the release carrying the NU6.2 circuit correction (ZIP 257; the *Ironwood Guide*); every gate and lookup declaration cited below is identical in the two releases.

Shape first, data second. A circuit, to `halo2_proofs`, is a type implementing the `Circuit` trait, whose two principal methods split the construction into phases (`halo2_proofs::plonk/circuit.rs`). The static method `configure` receives a mutable `ConstraintSystem` and declares the *shape*: it mints fixed, advice, and instance columns, allocates selectors, registers gates and lookups, and marks columns as equality-enabled. No witness exists at this point; `configure` is not even handed an instance of the circuit type. The method `synthesize` runs later, with the data, and fills cells. The split is the proof system’s division of labour: everything `configure` declares is public and determines the verifying key — the verifier’s entire view of the circuit — while everything `synthesize` writes into advice cells is the prover’s witness. (Key generation also runs `synthesize` once, against a witness-free backend that records fixed-cell values, selector activations, and copy constraints while ignoring advice; the prover runs the same code against a backend that records only advice. One synthesis procedure, two observers: `Assembly` in `plonk/keygen.rs` and `WitnessCollection` in `plonk/prover.rs`.)

Gates are queried identities. A call `meta.create_gate(name, closure)` defines one custom gate. Inside the closure, each `query_advice(column, Rotation(r))` — and its fixed and instance analogues — yields a symbolic handle to “the value of this column r rows below the gate’s own row”: the offsets $\omega^r X$ of Definition 5.1, realised at evaluation time by querying the column polynomial at $\omega^r x$ (`halo2_proofs, poly/domain.rs, rotate_omega`). The closure combines the handles by field arithmetic into an expression tree (Expression: constants, selector nodes, queried cells, negations, sums, products, scalings), one tree per constraint, and the helper `Constraints::with_selector` multiplies every constraint c by the gate’s selector expression, storing $q \cdot c$ (`plonk/circuit.rs`). This helper is the mechanical origin of the “selector times polynomial” shape of every deployed gate but one — the generalisation, gate by gate, of the fixed vanilla-PLONK equation of §5.1. (The exception, the ECC chip’s witness-point gate, multiplies by hand: its $(q \cdot x) \cdot (y^2 - x^3 - 5)$ and the helper’s $q \cdot (x \cdot (y^2 - x^3 - 5))$ are equal polynomials but different expression trees, and the pinned verifying key freezes the tree; `halo2_gadgets, ecc/chip/witness_point.rs`.) A selector is virtual at this stage: a named boolean column, off everywhere by default, enabled row by row during synthesis. At key generation, `compress_selectors` packs the selectors into genuine fixed columns — several to a column wherever the circuit’s degree bound leaves headroom — and rewrites every selector node into a fixed-column expression (`plonk/circuit.rs, plonk/keygen.rs`). The deployed Action circuit declares 56 selectors, whose compressed columns are among the 29 fixed columns pinned in its verifying key; the pinned count is the post-compression one (`orchard 0.13.1, src/circuit_description`; since 0.14 the file lives at `src/circuit_data/circuit_description_insecure`, and the corrected post-NU6.2 key beside it, `circuit_description_fixed`, pins the identical constraint system, differing only in its permutation commitments).

Regions, the layouter, and wiring. Data enters at synthesis through a Layouter. A chip requests a *region* — a small rectangular window of cells addressed by relative offsets — fills it cell by cell, and leaves the choice of the region’s absolute starting row to the *floor planner* (`halo2_proofs, circuit.rs`). Values flow between regions not by position but by *copy constraints*: the call `enable_equality(column)` enrolls a column in the single global permutation of §5.3, and `constrain_equal` — or the assign-and-constrain combination `copy_advice` — merges the two cells’ cycles in that permutation (`plonk/permutation/keygen.rs`; the halo2 book’s *Permutation argument* chapter documents the cycle-merging). Public inputs bind the same way, an advice cell copy-constrained to an instance cell (§5.8). The deployed circuit uses the measuring V1 floor planner, which records every region’s shape in a witness-free pass, sorts the regions by advice area, and first-fits the largest ones first (`circuit/floor_planner/v1.rs`; the sort and first-fit in `v1/strategy.rs`); the orchard crate additionally pins the planner’s sort to a vendored Rust 1.56.1 `pdqsort` (feature `floor-planner-v1-legacy-pdqsort`) so that the layout — and with it the consensus verifying key — is identical across platforms and toolchains.

A deployed gate: incomplete point addition. The ECC chip’s incomplete-addition gate (`halo2_gadgets, ecc/chip/add_incomplete.rs`) computes $P + Q = R$ on Pallas under the promises $P, Q \neq \mathcal{O}$ and $x_p \neq x_q$. Its configuration is one selector $q_{\text{add-incomplete}}$ and four equality-enabled advice columns x_p, y_p, x_q, y_q ; the gate queries x_p, y_p, x_q, y_q at the current row and x_r, y_r at the next row of the *same* two columns x_q, y_q (hence their names: Q on the selector row, R below it). The two constraints, named x_r and y_r in the source, are

$$q_{\text{add-incomplete}} \cdot ((x_r + x_q + x_p)(x_p - x_q)^2 - (y_p - y_q)^2) = 0, \quad (1)$$

$$q_{\text{add-incomplete}} \cdot ((y_r + y_q)(x_p - x_q) - (y_p - y_q)(x_q - x_r)) = 0. \quad (2)$$

These are the affine chord formulas (the *Math Guide*) with the slope eliminated: for $x_p \neq x_q$ they are equivalent to $x_r + x_q + x_p = \lambda^2$ and $y_r + y_q = \lambda(x_q - x_r)$ with $\lambda := (y_p - y_q)/(x_p - x_q)$, multiplied through by $(x_p - x_q)^2$ and $(x_p - x_q)$ respectively to clear the denominators (the halo2 book’s *Addition* chapter records the same clearing). The raw degrees are three and two; with the selector factor, four

and three — well inside the deployed circuit’s constraint-degree bound of nine (below). The *complete*-addition companion gate, which branches over the identity, doubling, and inverse cases that this gate excludes, needs twelve constraint polynomials of degree up to six and five auxiliary witness cells per invocation (`ecc/chip/add.rs`); incompleteness is a bargain wherever its promises can be kept.

Synthesis for one invocation (`assign_region`, same file) makes the witness path concrete. The chip enables the selector at the region’s row i ; copies the four input coordinates into that row with `copy_advice`, which both assigns each cell and copy-constrains it back to its source; computes, *out of circuit*,

$$\lambda = \frac{y_q - y_p}{x_q - x_p}, \quad x_r = \lambda^2 - x_p - x_q, \quad y_r = \lambda(x_p - x_r) - y_p;$$

and assigns x_r, y_r into row $i + 1$ (Table 1). The slope λ is never written to any cell: the prover uses it to compute the answer, and the committed identities (1)–(2) verify the answer without it. The division is not even performed here — coordinates are carried as deferred fractions (`halo2_proofs`, `plonk/assigned.rs`), and every denominator falls to a single batch inversion when synthesis ends (`plonk/prover.rs`, `batch_invert_assigned`). The output cells are returned as a new non-identity point with no further on-curve check — sound because, once the call site guarantees distinct x -coordinates on non-identity inputs, the two identities force $R = P + Q$, a non-identity curve point.

row	x_p	y_p	x_qr	y_qr	q _{add-incomplete}
i	x_p	y_p	x_q	y_q	1
$i + 1$	–	–	x_r	y_r	0

Table 1: The incomplete-addition region: two rows by four advice columns (`halo2_gadgets`, `ecc/chip/add_incomplete.rs`). The four input cells x_p, y_p, x_q, y_q on the selector row i are copy-constrained to the cells they came from; the result cells x_r, y_r occupy the `x_qr`, `y_qr` columns of row $i + 1$, where the gate’s `Rotation::next()` queries find them. Dashes mark cells the gate does not use.

Incompleteness, and where it is discharged. Reading the two polynomials at the excluded inputs shows exactly what “incomplete” costs. If $Q = -P \neq \mathcal{O}$ (so $x_p = x_q$ and $y_p = -y_q \neq 0$), constraint (1) collapses to $-(2y_p)^2 \neq 0$: no assignment satisfies the gate — a completeness failure, never a soundness one. If $Q = P$, both polynomials vanish identically and (x_r, y_r) is *unconstrained*: a reachable doubling would let the prover claim any result. If an input were the identity in the gadget’s $(0, 0)$ affine encoding, the constraints would accept a garbage output. The deployed circuit discharges the three exclusions in three different places. Non-identity travels with the type: the gate’s inputs are `NonIdentityEccPoint` values. A fresh point entering the chip is witnessed under the gate

$$q_{\text{point-non-id}} \cdot (y^2 - x^3 - 5) = 0 \quad (\text{halo2_gadgets}, \text{ecc/chip/witness_point.rs}),$$

which $(0, 0)$ cannot satisfy because 5 is not a square and -5 is not a cube in $\mathbb{F}_{p_{\text{Pallas}}}$ (the halo2 book’s *Addition* chapter) — but no fresh witness ever reaches incomplete addition. In the deployed circuit the incomplete-addition gate fires only inside fixed-base scalar multiplication (`ecc/chip/mul_fixed.rs`), which sums precomputed windowed multiples $[(k + 2) \cdot 8^w]B$ of a fixed base B . There the addends are assembled by the type’s unchecked constructor (`from_coordinates_unchecked`) and pinned instead to the precomputed window table by `mul_fixed`’s own coordinate gates (`coords_check`, same file): the x -coordinate must equal a degree-seven polynomial in the window value whose fixed coefficients interpolate the window’s eight table x -coordinates, and the y -coordinate must satisfy $y + z_w = u^2$ against a per-window fixed z_w and a witnessed u , alongside an on-curve check — together pinning the point to the table entry selected by the window digit, and no entry of the table is the identity. The running accumulator is a sum of such entries under this very gate; the type carries the non-identity guarantee from either construction

site to the gate, which re-checks nothing. The distinctness $x_p \neq x_q$ is enforced by no constraint at all; it too is structural: the window tables store $[(k+2) \cdot 8^w]B$ rather than the naive $[(k+1) \cdot 8^w]B$ precisely because the naive offset admits a window collision that produces the fatal doubling (the halo2 book’s *Fixed-base scalar multiplication* chapter). Prover-side, `assign_region` rejects a *known* bad witness with an error — a developer diagnostic, not a constraint. (The circuit’s other incomplete additions — the variable-base ladder’s fused double-and-add rounds and Sinsemilla’s two per chunk, the ones the *Ironwood Guide*’s gadget survey counts — are separate gates built on the same discipline; `ecc/chip/mul/incomplete.rs`, `sinsemilla/chip.rs`.)

Non-algebraic relations become lookups. A gate can assert only polynomial identities. “This value has ten bits” and “this point is the j -th Sinsemilla generator” are not identities but set memberships, and the circuit declares them as lookups: a call `meta.lookup(closure)` returns pairs of an input *expression* and a table column, and pushes one lookup argument onto the constraint system (`halo2_proofs`, `plonk/circuit.rs`). Each declaration is one instance of the argument of §5.4, the input expressions playing $A_0, \dots, A_{m-1}$ and the table columns $S_0, \dots, S_{m-1}$. The deployed circuit carries exactly three (`orchard 0.13.1`, `src/circuit_description`): one shared range-check declaration, and the Sinsemilla generator declaration twice — once per Sinsemilla chip instance, each on its own half of the ten advice columns.

All three consume the same table. Loaded once at synthesis, it holds, for $0 \leq j < 2^{10}$,

$$\text{row } j : \quad (j, X(S[j]), Y(S[j])), \quad S[j] := \text{GroupHash}(\text{"z.cash:SinsemillaS"}, \text{LE}_{32}(j)),$$

in three table columns $(t_{\text{idx}}, t_x, t_y)$; the generators ship as precomputed constants that the crate’s own tests re-derive from the hash (`sinsemilla crate`, `src/sinsemilla_s.rs`; `halo2_gadgets`, `utilities/lookup_range_check.rs` for the loading). After the assignment, the layouter fills every remaining usable row of each table column with that column’s row-0 value (`halo2_proofs`, `circuit/table_layouter.rs`); a defaulted input row therefore still matches a genuine table row, and the type `TableColumn` hides its underlying fixed column exactly so that no chip can bypass this default-filling.

The range check is one declaration serving two modes (`halo2_gadgets`, `utilities/lookup_range_check.rs`):

$$q_{\text{lookup}} \cdot \left(q_{\text{running}} \cdot (z_i - 2^{10} z_{i+1}) + (1 - q_{\text{running}}) \cdot z_i \right) \mapsto t_{\text{idx}}, \quad (3)$$

with z_i and z_{i+1} the running-sum advice column at the current and next rotations. With $q_{\text{running}} = 1$ the input is a running-sum step: the prover witnesses $z_{i+1} = (z_i - a_i) \cdot 2^{-10}$ on successive rows, the expression recovers the limb $a_i = z_i - 2^{10} z_{i+1}$, and membership in $t_{\text{idx}} = \{0, \dots, 2^{10} - 1\}$ asserts that each limb of the decomposition has ten bits; a *strict* decomposition additionally copy-constrains the final z to the constant 0, range-constraining the decomposed element to $10w$ bits for w limbs. With $q_{\text{running}} = 0$ the cell itself is looked up (checks of fewer than ten bits add a small bit-shift gate in the same file). Every range check in the Action circuit funnels through this one declaration, on a single advice column.

The Sinsemilla declaration is the three-column instance (`halo2_gadgets`, `sinsemilla/chip/generator_table.rs`):

$$\left(q_{S1} \cdot m, \quad q_{S1} \cdot x_p + (1 - q_{S1}) \cdot X(S[0]), \quad q_{S1} \cdot y_p + (1 - q_{S1}) \cdot Y(S[0]) \right) \mapsto (t_{\text{idx}}, t_x, t_y), \quad (4)$$

where $m = z_i - 2^{10} \cdot q_{\text{run}} \cdot z_{i+1}$ recovers the ten-bit message word from the hash’s own running-sum decomposition (with $q_{\text{run}} := q_{S2} - q_{S2}(q_{S2} - 1)$, an expression in a fixed column $q_{S2} \in \{0, 1, 2\}$ that evaluates to 1 on the interior rows of a message piece and to 0 on final rows, where the trailing running sum is implicitly zero), and where y_p is not a witnessed cell at all but the derived expression $Y_A \cdot 2^{-1} - \lambda_1(x_A - x_p)$ over the chip’s double-and-add columns (`ecc/chip/mul/incomplete.rs` supplies $Y_A := (\lambda_1 + \lambda_2)(x_A - x_R)$ and $x_R := \lambda_1^2 - x_A - x_p$). Lookup inputs, in other words, may

be arbitrary expressions over queried cells, not merely cells; the chip proves that the generator it is adding is $S[m]$ without ever spending an advice cell on $y_{S[m]}$. The $(1 - q_{S1})$ branches default every row on which the chip is inactive to table row 0, so the membership assertion of §5.4 holds on all rows. One API restriction is visible here: the selectors gating lookup inputs (q_{lookup} , q_{running} , q_{S1}) must be declared as `complex_selectors` — simple selectors are reserved for gates, where the key-generation compression is licensed to rewrite them (`halo2_proofs, plonk/circuit.rs`).

The handoff. When `synthesize` returns, the construction has produced nothing exotic: every column — advice, fixed, instance — is a vector of $n = 2^k$ field elements; in the advice columns the deferred fractions are resolved by one batch inversion and the reserved trailing rows filled with blinding randomness (§5.9, Remark 5.2). The prover commits to each advice column directly in the Lagrange basis (`halo2_proofs, poly/commitment.rs, commit_lagrange` — the same group element as committing the interpolated coefficients, so the commitment even precedes interpolation) and then interpolates each column over H by an inverse FFT (`poly/domain.rs, lagrange_to_coeff`): these are precisely the column polynomials of §5.1. Each gate’s stored expression tree is then evaluated with the committed column polynomials substituted for the queried cells — a query at rotation r reading the polynomial at $\omega^r X$ — and the results are folded with powers of the challenge y and divided by $Z_H(X) = X^n - 1$ (`plonk/prover.rs; plonk/vanishing/prover.rs; divide_by_vanishing_poly` in `poly/domain.rs`). The gate identities declared in `configure` have become the quotient $h(X)$ of Phase 4 (§5.7). This is the point where the toy of *How Halo 2 Proves* resumes — its § *From the grid to polynomials* interpolates a four-row trace and assembles exactly this $G(X)$ whose divisibility by Z_H the rest of the protocol certifies — and where the *Ironwood Guide*’s gadget stack bottoms out: each gadget of its survey is a bundle of `configure`-time gates and lookups plus `synthesize`-time regions, and after synthesis nothing remains of the stack but rows in these same columns. The scale-up from the toy is numerical, not conceptual. The deployed Action circuit has $k = 11$ ($n = 2048$ rows), the ten advice columns of §5.1, one instance column, and twenty-nine fixed columns, with a constraint-degree bound of nine across its gates and permutation and lookup expressions — so the quotient splits into $9 - 1 = 8$ chunks, and the prover’s FFTs run over an extended domain of size $2^{14} = 8 \cdot 2^{11}$ (`orchard 0.13.1, src/circuit_description; halo2_proofs, poly/domain.rs`). The next two subsections supply the arguments this construction leans on: the permutation argument behind every copy constraint, and the lookup argument behind declarations (3) and (4).

5.3 The permutation (equality) argument

A PLONK-style permutation argument, which Halo 2 generalises to multiple columns, enforces equality constraints between distinct cells. Each cell (i, j) (column i , row j) receives a unique label $\delta_i \cdot \omega^j$ with δ_i chosen so all products are distinct as (i, j) ranges over the trace. The prover writes σ -permuted polynomials $s_i(X)$ with $s_i(\omega^j) = \delta_{i'} \cdot \omega^{j'}$ when $\sigma(i, j) = (i', j')$.

Given Fiat–Shamir challenges $\beta, \gamma \in \mathbb{F}$, the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ asserting, where $v_i(X)$ denotes the Lagrange interpolation (§5.1) of the i -th column participating in the permutation,

$$\prod_{i,j} \frac{v_i(\omega^j) + \beta \delta_i \omega^j + \gamma}{v_i(\omega^j) + \beta s_i(\omega^j) + \gamma} = 1.$$

The assignment satisfies every equality constraint (cells related by σ carry equal values) precisely when the denominator factors are a permutation of the numerator factors, in which case the product equals 1 for every β, γ ; conversely, if some constraint is violated, the product at the sampled β, γ equals 1 with probability at most $O(N/|\mathbb{F}|)$ by Schwartz–Zippel, where N is the number of cells participating in the permutation. The verifier checks this by opening Z_{perm} at x and ωx , verifying the boundary $Z_{\text{perm}}(\omega^0) = 1$ and the row-by-row update. The per-proof verifier work is $O(m + \log n)$, where m denotes the number of columns participating in the permutation.

5.4 The lookup argument

Halo 2’s lookup argument is structurally simpler than plookup (Gabizon–Williamson 2020) and supports arbitrary (possibly non-fixed) tables.

Objective. Establish that every entry of column $A(X)$ on the active domain appears as some entry of column $S(X)$.

Construction. The prover supplies polynomials $A'(X), S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' has the matching value in S' . The prover commits to A' and S' ; after receiving challenges β, γ , it commits to a running-product column $Z_{\text{lookup}}(X)$ and proves:

$$\begin{aligned} \ell_0(X)(1 - Z_{\text{lookup}}(X)) &= 0, \\ Z_{\text{lookup}}(\omega X)(A'(X) + \beta)(S'(X) + \gamma) - Z_{\text{lookup}}(X)(A(X) + \beta)(S(X) + \gamma) &= 0, \\ \ell_0(X)(A'(X) - S'(X)) &= 0, \\ (A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &= 0. \end{aligned}$$

The first two constraints, with the boundary conditions of Remark 5.2, force A' and S' to be permutations of A and S respectively; the third aligns row 0 ($A'_0 = S'_0$), where the wrap-around reference $A'(\omega^{-1}X)$ deprives the fourth constraint of force. The fourth constrains every subsequent row either to start a new run aligned with S' ($A'_i = S'_i$) or to continue the previous one ($A'_i = A'_{i-1}$). Inductively, every value of A equals some value of S' , hence of S .

A random-linear combination with a challenge θ reduces multi-column lookups to single-column: $A_c(X) := \sum_j \theta^{m-1-j} A_j(X)$. Variable tables (where S is itself an advice column) and range checks (where S enumerates the allowed range) are special cases.

Remark 5.2 (Active rows and blinding). The lookup constraints above, like the permutation constraints of §5.3, hold on the *active* rows only: the deployed system reserves the trailing rows of every column for random blinding values (§5.9). Each product-update constraint is therefore multiplied by the factor $(1 - (\ell_{\text{last}}(X) + \ell_{\text{blind}}(X)))$, where ℓ_{blind} and ℓ_{last} are Lagrange-basis selectors of the blinding rows and of the boundary row immediately before them — computed directly from the domain by both parties, not committed circuit columns — and boundary constraints such as $\ell_{\text{last}}(X)(Z(X)^2 - Z(X)) = 0$ pin the running product to $\{0, 1\}$ at the boundary row. The random rows consequently neither violate the arguments nor weaken them.

5.5 The inner-product polynomial commitment

The Halo 2 polynomial commitment, instantiating the abstract PCS of the *Crypto Guide*, is the inner-product argument of Bootle–Cerulli–Chaidos–Groth–Petit (EUROCRYPT 2016) as refined in Bulletproofs (Bünz et al., 2017). We give the construction in detail. Write $d := 2^k$ for the degree bound (equivalently, the number of generators); the round count $k = \log_2 d$ reuses the letter k of Definition 5.1, harmlessly, because in the assembled protocol of §5.7 the committed polynomials have degree $< n$, so $d = n = 2^k$ there. The letter H likewise does double duty, inherited from two literatures: $H \subset \mathbb{F}$ is the PLONK evaluation domain (as in Z_H), while $H \in \mathbb{G}$ is the Pedersen blinding generator of the *Crypto Guide*; the ambient set ($\mathbb{F}$ versus $\mathbb{G}$) disambiguates every occurrence. Two further letters carry double duty: p denotes the group order (as in $\mathbb{F}_p$) and, with an argument or in $\deg p$, the committed polynomial $p(X)$; and the permutation polynomials $s_i(X) \in \mathbb{F}[X]$ of §5.3 are distinct from the IPA structured scalars $s_i \in \mathbb{F}_p$ of §5.5.2. Type and argument disambiguate every occurrence.

Construction 5.3 (IPA-based polynomial commitment). Let $\mathbb{G}$ be a cyclic group of prime order p in which DL is hard. Choose $d = 2^k$ and sample, via hash-to-curve, $\vec{G} = (G_1, \dots, G_d)$ and $H \in \mathbb{G}$

with no known non-trivial linear relation among them.

For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in \mathbb{F}_p[X]$ and blinder $r \in \mathbb{F}_p$,

$$\text{Commit}(p; r) := \langle \vec{a}, \vec{G} \rangle + [r]H \in \mathbb{G}.$$

To open p at $x \in \mathbb{F}_p$ with claimed value v , the relation is

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\vec{a}, r)) : P = \langle \vec{a}, \vec{G} \rangle + [r]H, v = \langle \vec{a}, \vec{b}(x) \rangle, \deg p \leq d-1 \right\},$$

with $\vec{b}(x) := (1, x, x^2, \dots, x^{d-1})$.

5.5.1 The recursive halving protocol

The argument runs in $k = \log_2 d$ rounds. Initialise $\vec{a}^{(k)} := \vec{a}$, $\vec{G}^{(k)} := \vec{G}$, $\vec{b}^{(k)} := \vec{b}(x)$, and let $U \in \mathbb{G}$ be a challenge group element (in Fiat–Shamir, derived from the transcript). The prover absorbs $[v]U$ so that the relation becomes

$$P + [v]U = \langle \vec{a}, \vec{G} \rangle + [r]H + [\langle \vec{a}, \vec{b} \rangle]U.$$

In round j (descending from k to 1): split each vector into halves and let

$$\begin{aligned} L_j &:= \langle \vec{a}_{\text{lo}}^{(j)}, \vec{G}_{\text{hi}}^{(j)} \rangle + [\ell_j]H + [\langle \vec{a}_{\text{lo}}^{(j)}, \vec{b}_{\text{hi}}^{(j)} \rangle]U, \\ R_j &:= \langle \vec{a}_{\text{hi}}^{(j)}, \vec{G}_{\text{lo}}^{(j)} \rangle + [r_j]H + [\langle \vec{a}_{\text{hi}}^{(j)}, \vec{b}_{\text{lo}}^{(j)} \rangle]U. \end{aligned}$$

The verifier returns a uniform $u_j \in \mathbb{F}_p^*$. Both parties fold:

$$\begin{aligned} \vec{a}^{(j-1)} &:= u_j \vec{a}_{\text{lo}}^{(j)} + u_j^{-1} \vec{a}_{\text{hi}}^{(j)}, \\ \vec{G}^{(j-1)} &:= u_j^{-1} \vec{G}_{\text{lo}}^{(j)} + u_j \vec{G}_{\text{hi}}^{(j)}, \\ \vec{b}^{(j-1)} &:= u_j^{-1} \vec{b}_{\text{lo}}^{(j)} + u_j \vec{b}_{\text{hi}}^{(j)}. \end{aligned}$$

After k rounds each vector has length 1. The prover sends the final scalar $a := \vec{a}^{(0)}$ and an aggregated blinder r^* . The verifier accepts iff

$$P + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [a]G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U, \quad (5)$$

where $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ and the challenges determine $\vec{s}$ as below.

5.5.2 The structured scalars and the polynomial g

Write the binary expansion $i = \sum_{\ell=0}^{k-1} b_\ell 2^\ell$ for $i \in [0, d)$. By induction on the folding,

$$G^{(0)} = \langle \vec{s}, \vec{G} \rangle, \quad b^{(0)} = \langle \vec{s}, \vec{b}(x) \rangle = g(x; u_1, \dots, u_k),$$

where

$$s_i := \prod_{\ell=0}^{k-1} u_{\ell+1}^{(-1)^{1-b_\ell}}, \quad g(X; u_1, \dots, u_k) := \prod_{\ell=1}^k (u_\ell^{-1} + u_\ell X^{2^{\ell-1}}).$$

The essential asymmetry is the following: $g(X)$ has degree $d-1$ and the verifier can evaluate it at x in $O(\log d)$ field operations; but computing $G^{(0)} = \text{Commit}(g; 0)$ requires an $O(d)$ -size multi-scalar multiplication (MSM). This asymmetry is the seed of the accumulation scheme (§5.6).

Remark 5.4 (The deployed IPA normalisation). The deployed opening (halo2_proofs 0.3.2, poly/commitment/prover.rs and verifier.rs) uses the rescaled Halo convention, not the Bulletproofs one above: the folding weights are $(1, u_j^{-1})$ on the coefficient vector and $(1, u_j)$ on $\vec{b}$ and $\vec{G}$, the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$, and the deferred polynomial is $g(X) = \prod_{i=0}^{k-1} (1 + u_{k-1-i}X^{2^i})$. The opening also begins with an extra blinding commitment S and two challenges ξ, z , and ends with two scalars: the collapsed coefficient c and a synthetic blinder f . The two conventions are equivalent up to per-round rescaling of the folded vectors and relabelling (u_j^2 here corresponds to the code's u_j , with the roles of L_j and R_j exchanged); every statement of this section transfers.

5.5.3 Worked example: an IPA opening at $d = 4$

Take $d = 4$, so $k = 2$ rounds. Let $p(X) = a_0 + a_1X + a_2X^2 + a_3X^3$ with $\vec{a} = (a_0, a_1, a_2, a_3)$. The verifier holds $P = \langle \vec{a}, \vec{G} \rangle + [r]H$ (we suppress r to declutter; the H -part propagates identically) and requires $v = p(x)$ at some $x \in \mathbb{F}_p$.

Set $\vec{b}^{(2)} := (1, x, x^2, x^3)$ and $\vec{G}^{(2)} := \vec{G}$. After the prover absorbs $[v]U$, the relation is $P + [v]U = \langle \vec{a}, \vec{G} \rangle + \langle \vec{a}, \vec{b}^{(2)} \rangle U$.

Round 2. Split into halves of size 2. The prover sends

$$\begin{aligned} L_2 &= a_0G_3 + a_1G_4 + (a_0x^2 + a_1x^3)U, \\ R_2 &= a_2G_1 + a_3G_2 + (a_2 + a_3x)U. \end{aligned}$$

The verifier picks $u_2 \in \mathbb{F}_p^*$ and both parties fold to length-2 vectors as above.

Round 1. Split the length-2 vectors. The prover sends L_1, R_1 analogously; the verifier picks u_1 . After folding, all three vectors have length 1.

Final check. With $i = b_0 + 2b_1$ ranging over $\{0, 1, 2, 3\}$:

$$s_0 = u_1^{-1}u_2^{-1}, \quad s_1 = u_1u_2^{-1}, \quad s_2 = u_1^{-1}u_2, \quad s_3 = u_1u_2.$$

The verifier evaluates $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$ in four field multiplications (u_1x, x^2, u_2x^2 , and the product of the two factors), and computes $G^{(0)} = \sum_{i=0}^3 [s_i]G_{i+1}$ as a length-4 MSM. The check (5) becomes

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 \stackrel{?}{=} [a]G^{(0)} + [a \cdot g(x; u_1, u_2)] U.$$

The folding rule for $\vec{G}$ ensures that $G^{(0)}$ is a specific linear combination of the original $\vec{G}$ with coefficients $\vec{s}$; that the s_i match $\prod_{\ell} (u_{\ell}^{-1} \text{ or } u_{\ell})$ according to the binary expansion of i is exactly the closed-form identity. The verifier can evaluate g in $O(\log d)$ field operations (about $3k-2$: k coefficient products, $k-1$ squarings of x , $k-1$ factor multiplications) but recomputing $G^{(0)}$ takes $O(d)$ MSM — the asymmetry the accumulator defers.

5.5.4 Knowledge soundness via rewinding

Theorem 5.5 (Knowledge soundness of IPA). *Under the DL assumption on $\mathbb{G}$, the interactive opening protocol of Construction 5.3 is knowledge-sound for $\mathcal{R}_{\text{IPA}}$ with knowledge error $O(d/|\mathbb{F}_p|)$, via an expected-polynomial-time tree extractor. The Fiat–Shamir-compiled argument inherits the bound only up to the naive $q^{O(\log d)}$ random-oracle query loss; the tight non-interactive bound needs the AGM (§3).*

Sketch of the tree extractor. The extractor rewinds the prover $\mathcal{P}^*$ to obtain, at each round, *three* accepting transcripts that agree on (L_j, R_j) but carry pairwise distinct challenges u_j . The round- j verification equation involves the three monomials $u_j^{-2}, 1, u_j^2$, whose coefficients — the unknown representations of L_j , the folded commitment, and R_j over the generators — the extractor must determine; two transcripts would leave this system underdetermined. Three challenges with pairwise distinct squares ($u \neq \pm u'$; the colliding pairs are excluded below) yield three linear equations whose 3×3 Vandermonde-type determinant in u^2 is non-zero, so the extractor solves for the round- j witness halves $\vec{a}_{\text{lo}}^{(j)}, \vec{a}_{\text{hi}}^{(j)}$ (Lemma 3.3).

Extracting the full witness $\vec{a} \in \mathbb{F}_p^d$ requires three sibling transcripts at the deepest level, which yield one round-1 witness; three such triples, hung off pairwise distinct challenges at the next-shallower round, yield one round-2 witness; and so on, each shallower round tripling the requirement. The tree thus has $3^k = d^{\log_2 3}$ leaves — polynomial in d (§3) — yielding polynomial expected extraction time when $d = \text{poly}(\lambda)$. The theorem’s knowledge error of $O(d/|\mathbb{F}_p|)$ does not come from a tree-wide union bound. Each of the $k = \log_2 d$ rounds excludes $O(1)$ bad challenges ($u_j = \pm u'_j$), contributing $O(\log d/|\mathbb{F}_p|) \leq O(d/|\mathbb{F}_p|)$; Schwartz–Zippel terms of degree d account for the rest. Collisions among the resampled challenges inside the extractor’s 3^k -leaf tree affect only its expected running time — the extractor rewinds and resamples — not the knowledge error; a naive union bound over the roughly 3^k sibling challenge draws would bound the tree-wide collision probability by $O(d^{\log_2 3}/|\mathbb{F}_p|)$, still negligible, but that is not the source of the theorem’s figure.

This is the recursive generalisation of special soundness for Schnorr (the *Crypto Guide*): a single transcript reveals nothing, but a few transcripts on the same first message permit inversion of a small linear system; in the IPA the system is 3×3 per round and the recursion stacks k rounds. $\square$

5.6 Accumulation and the Halo trick

The verifier’s check (5) runs in $O(\log d)$ scalar operations and $\log d$ point additions, *except* for the single evaluation $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ — an MSM of length d . The accumulation idea (Bowe–Grigg–Hopwood 2019; formalised by BCMS20) is to *defer* this MSM and amortise it across many proofs.

Definition 5.6 (Atomic accumulator for IPA). An IPA *accumulator instance* is a tuple $\text{Acc} := (G^{(0)}, u_1, \dots, u_k) \in \mathbb{G} \times \mathbb{F}_p^k$. It is *valid* iff $G^{(0)} = \langle \vec{s}(\vec{u}), \vec{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0)$ — the deferred claim concerns the unblinded commitment, the blinder having been discharged by the $[r^*]H$ term of (5). Subsequent unary $\text{Commit}(\cdot)$ abbreviates $\text{Commit}(\cdot; 0)$.

Theorem 5.7 (Accumulation of IPA, BCMS20). *There is an accumulation scheme (AccVerifier, AccDecider) for IPA with:*

- AccVerifier, on input a new IPA instance and an old accumulator, runs in $O(\log d)$ and outputs a new accumulator Acc' .
- AccDecider, run once at the end of a chain, performs an MSM of length d to check $G^{(0)} = \text{Commit}(g)$.
- Soundness reduces to DL on $\mathbb{G}$ in the random-oracle model.

Operational interpretation. A Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the *other* curve, can:

1. Run the $O(\log d)$ “succinct” checks of the IPA verifier natively.
2. Witness the next-step claimed $G^{(0)'}$ and the next challenges $u'_1, \dots, u'_k$ as public inputs to the next proof, deferring the MSM check.

3. Fold the deferred MSM checks: with a random challenge ρ , the two claims $G^{(0)} = \text{Commit}(g)$ and $G^{(0)'} = \text{Commit}(g')$ reduce to the single check $G^{(0)} + [\rho] G^{(0)'} = \text{Commit}(g + \rho g')$, with both challenge tuples retained so that the combined length- d coefficient vector $\vec{s}(\vec{u}) + \rho \vec{s}(\vec{u}')$ is recomputable; a false claim survives the combination with probability at most $1/|\mathbb{F}_p|$. In the recursive instantiation the combined commitment is opened at a fresh random point inside the next proof's multipoint argument — each g_i there is evaluable in $O(\log d)$ — so the deferred claim emerging from the next IPA opening is again a single tuple of the form of Definition 5.6; the tuple type is closed under accumulation only through this prover-assisted step, not under verifier-side linear combination.

A single final MSM at the end of the chain validates the entire history. The Pasta cycle (§4.3) renders each step native: a Vesta-IPA proof's circuit, whose native field is the Vesta scalar field $\mathbb{F}_{p_{\text{Pallas}}}$, verifies the previous Pallas-IPA proof, whose group elements (the column and quotient commitments, the (L_j, R_j) pairs, the accumulator's $G^{(0)}$) have coordinates in exactly that field; the next Pallas proof verifies the Vesta proof by the symmetric argument. This cross-curve alternation is what enables proof-carrying data without expensive non-native field simulation. The same coincidence is what lets Orchard's $\mathbb{F}_{p_{\text{Pallas}}}$ -native Action circuit manipulate the Pallas points cm , rk , and cv^{net} directly (§5.8).

Orchard's deployment does not recurse: the verifier of an Action proof pays the deferred MSM itself, as part of the standard verification cost. Even there the accumulation idea is load-bearing: a Zcash node verifying many Action proofs takes a random linear combination of their deferred MSMs and evaluates the combination as one MSM, amortising the linear-time work across the whole batch — the deployed, non-recursive use of the same algebraic structure.

The mathematical core is the recognition that $G^{(0)} = \text{Commit}(g(X; u_1, \dots, u_k))$ — the linear-time check is *itself a polynomial commitment opening of a structured polynomial* and hence amenable to recursion within an IPA.

5.7 The complete Halo 2 protocol

We now assemble the pieces into the seven-phase protocol implemented in the halo2 codebase. Throughout, $\mathbb{F}$ denotes the constraint field (Orchard: $\mathbb{F}_{p_{\text{Pallas}}}$), $\mathbb{G}$ the IPA group (Orchard: $\text{Vesta}(\mathbb{F}_{p_{\text{Vesta}}})$), and H the multiplicative subgroup of $\mathbb{F}$ of order n .

Setup (preprocessed, one-time). The circuit designer specifies columns, custom gates, lookup tables, and the equality permutation σ . Both parties derive commitments to the fixed-column polynomials $f_i(X)$, selector polynomials $q_i(X)$, permutation polynomials $s_i(X)$, and lookup-table polynomials $S_j(X)$, together with the Lagrange basis on H , the vanishing polynomial $Z_H(X) := X^n - 1$, and the IPA generators $\vec{G} \in \mathbb{G}^n$ and $H \in \mathbb{G}$. The commitments to the fixed, selector, permutation, and lookup-table polynomials, together with the domain parameters, constitute the *verifying key* vk ; every part of it is recomputable from the public circuit description, which is why no trusted setup exists. To verify a proof one needs exactly three inputs: vk , the instance (public-input) values, and the proof itself.

Phase 1 — advice commitments. Both parties first absorb a digest of vk and the (unblinded) instance commitments into the transcript (§5.8), so every subsequent challenge depends on the circuit and the public input. The prover then constructs the advice column values $\vec{w}_i \in \mathbb{F}^n$ satisfying all gates and equality constraints; fills the reserved trailing rows with uniformly random field elements (the blinding rows of §5.9); interpolates each $\vec{w}_i$ to a polynomial $w_i(X)$ of degree $< n$ on H ; commits via IPA-PCS with a fresh blinder; sends the commitments.

Phase 2 — lookup compression: challenge θ . The verifier (via Fiat–Shamir, here and throughout) returns $\theta \in \mathbb{F}$. For each lookup the prover compresses multi-column inputs and tables with powers of θ , computes the permuted columns A', S' (§5.4), and commits to them.

Phase 3 — running products: challenges β, γ . The verifier returns $\beta, \gamma \in \mathbb{F}$. The prover commits to the permutation running product Z_{perm} (§5.3) and to one lookup running product Z_{lookup} per lookup (§5.4). The order matters for soundness: the permuted columns A', S' must be fixed *before* the challenges β, γ that randomise the products are drawn.

Phase 4 — quotient challenge y . The verifier returns a fresh $y \in \mathbb{F}$. The prover assembles the *gate equation*: a single polynomial $C(X)$ that is a y -weighted linear combination of (i) each custom gate, (ii) permutation constraints, (iii) lookup constraints. By construction $C(X)$ vanishes on H when all gates pass, so the quotient $h(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$ is a polynomial. Its degree, however, is bounded by $d(n - 1) - n = (d - 1)(n - 1) - 1$ for maximum gate degree d — a gate of degree d multiplies up to d column polynomials of degree $< n$, and division by Z_H removes n — which exceeds the IPA’s degree bound n . The prover therefore splits h into $\lceil (\deg h + 1)/n \rceil$ chunks $h_0, h_1, \dots$ of degree $< n$, with $h(X) = \sum_i X^{ni} h_i(X)$, and commits to each chunk separately; the verifier recombines the chunk *commitments*, weighted by powers of x^n once the evaluation point x is drawn. The prover’s FFTs correspondingly run over an *extended* domain whose size accommodates $\deg C$, a small multiple of n .

Phase 5 — evaluation challenge x . The verifier returns $x \in \mathbb{F}$. The prover evaluates every committed polynomial except the quotient chunks at x , and at the rotated points the constraints reference — ωx for the running products (and for any gate reading an adjacent row), $\omega^{-1}x$ for the permuted lookup input A' — and sends these claimed evaluations as field elements. No evaluation of h is sent: the verifier computes the expected quotient evaluation $C(x)/(x^n - 1)$ from the sent evaluations and enforces it against the collapsed chunk commitment inside the multipoint opening of phases 6–7, where the claimed evaluations themselves also bind to their commitments.

Phase 6 — multipoint opening. The claimed evaluations must now be bound to the commitments, and Halo 2 reduces all the required openings to a single IPA opening. The committed polynomials are grouped by their set of query points: most are queried at $\{x\}$ alone, while the running products are also queried at ωx (and A' at $\omega^{-1}x$), so only a handful of distinct point sets arise. Within each group, a challenge x_1 folds the polynomials into a single combination $q_i(X)$ by powers of x_1 . (The letters q and f here follow the halo2 book’s multipoint-opening notation and are unrelated to the selector and fixed-column polynomials of the Setup paragraph, which are themselves among the polynomials being folded; the index i now ranges over query-point sets, not columns.) For each group, both parties interpolate the low-degree polynomial $r_i(X)$ that agrees with the claimed evaluations on the group’s point set; the quotient $f_i(X) := (q_i(X) - r_i(X))/\prod_z (X - z)$, with z ranging over the group’s points, is a polynomial precisely when the claimed evaluations are correct. The prover combines the f_i by powers of a second challenge x_2 into $f(X)$ and commits to it. The verifier returns a fresh evaluation point x_3 , and the prover sends the evaluation of each q_i at x_3 ; the verifier reconstructs the expected value of $f(x_3)$ from the $q_i(x_3)$, the $r_i(x_3)$, and the vanishing factors — if each f_i is a genuine polynomial this value matches the committed f , and by Schwartz–Zippel a match at random x_3 convinces the verifier that every claimed evaluation was correct. A final challenge x_4 folds f and the q_i into one polynomial $f^*(X)$, whose expected evaluation at x_3 the verifier assembles from the same data; a single opening of f^* at x_3 then discharges every original opening at once.

Phase 7 — IPA opening. Prover and verifier execute the recursive halving IPA (§5.5) on f^* , yielding $\log_2 n$ pairs (L_j, R_j) , a final scalar a , and a final blinder r^* . The verifier accepts iff (5) holds.

Accumulation (recursion). The next proof’s verifier circuit *defers* the MSM check $G^{(0)} = \text{Commit}(g(X; u_1, \dots, u_k))$ at the end of phase 7 into an accumulator and folds it with prior accumulators (§5.6). A single decider at the very end validates the entire chain.

The concrete transcript. The deployed implementation instantiates Fiat–Shamir with a transcript built on the BLAKE2b hash function. Every prover message — commitments as curve points, claimed evaluations as field elements — is absorbed into the hash state in the order the protocol fixes, and each verifier challenge is squeezed from the state at its prescribed point, after everything it must depend on has been absorbed. Both parties also absorb the data they share in advance — the vk digest and the unblinded instance commitments — which therefore never appear in the proof. The proof is the ordered sequence of the prover’s messages; the verifier replays the absorptions and recomputes every challenge.

Costs. The prover’s work is dominated by FFTs over the extended domain and by one $O(n)$ -size MSM *per committed polynomial* — several tens in all: each advice column, the permuted lookup columns, the running products, the quotient chunks, and the multipoint polynomial f (typical Orchard Action circuit: $n = 2^{11}$, a few seconds on commodity hardware). The verifier performs $O(\log n)$ field and group operations per proof — recomputing the challenges, the gate-equation field evaluation, the $\log n$ rounds of phase 7 — plus the single deferred $O(n)$ MSM, amortised over a batch as in §5.6.

Proof size. Two kinds of data make up a proof. The IPA of phase 7 contributes $2 \log_2 n + 1$ group elements (a blinding commitment and the pairs (L_j, R_j) ; Remark 5.4) plus two final scalars — for Orchard’s $n = 2^{11}$, twenty-two points in the pairs; this is the only part that scales with n , and only logarithmically. Everything else scales with the *circuit structure*, independently of n : one commitment per advice column, two permuted-column commitments and one running product per lookup, the permutation-product and quotient-chunk commitments, the multipoint commitment, and one field element per claimed evaluation. Each Pasta point or field element serialises to 32 bytes. For the Orchard Action circuit the total is fixed by consensus (ZIP 225): a bundle’s proof occupies $2720 + 2272 \cdot a$ bytes for a Actions — just under 5 kB for a single-Action bundle — the per-Action term covering the per-circuit commitments and evaluations, the constant term the evaluations of the verifying key’s fixed-column and permutation polynomials together with the shared quotient, multipoint, and IPA data.

5.8 Instance columns and binding the public statement

Definition 5.1 lists *instance* columns alongside fixed and advice columns, but the arguments of §5.3–§5.7 never single them out. We single them out now, because what an Orchard Action proof *asserts* is precisely a statement about its instance values — the anchor, nullifier nf, net value commitment cv^{net} , randomised key rk, extracted new-note commitment cmx, and the enableSpends/enableOutputs flags (the *Ironwood Guide*).

The instance polynomial. An instance column is a public-input column: its n entries $\vec{\pi} = (\pi_0, \dots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness but both parties agree on them before verification. Like every other column it is interpolated over H in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \ell_j(X) \in \mathbb{F}[X], \quad \deg \pi < n,$$

where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ otherwise (§5.1). The prover writes the public values into a handful of designated rows; all remaining π_j are 0.

Binding by copy constraints. Orchard binds the public values through the permutation argument of §5.3: the instance column participates in the permutation σ , and the circuit equality-constrains each instance cell to the advice cell carrying the corresponding in-circuit value (halo2’s

`constrain_instance`). A satisfying assignment therefore exists *only* when the advice cells holding the anchor, `nf`, `cvnet`, `rk`, `cmx`, and the flags equal the public values the verifier supplied. (An alternative design binds public inputs with a dedicated gate $q(X)(a(X) - \pi(X)) = 0$, where the selector q is active exactly on the designated rows; Orchard does not use it — the copy constraint reuses the existing permutation machinery at no extra gate degree.)

Where it enters the protocol. The verifier never takes $\pi(X)$ on trust from the prover. In the deployed IPA backend, *both* parties compute the commitment to each instance polynomial — unblinded, since the values are public, so the two computations agree — and absorb it into the transcript before the first challenge (Phase 1, §5.7); every Fiat–Shamir challenge thereby depends on the public input. Thereafter the instance column is treated like any other: its claimed evaluation $\pi(x)$ appears among the Phase 5 evaluations, enters the gate equation and the permutation expressions, and the multipoint opening of Phase 6 binds it to the instance commitment. Because the verifier computed that commitment itself, from the instance values it intends to verify against, the prover has no freedom over π : an evaluation inconsistent with the verifier’s own values fails the opening argument.

Consequence. A prover who targets different public values $\vec{\pi}'$ must open against the instance commitment the *verifier* computes from the genuine $\vec{\pi}$, and must satisfy the copy constraints against those values. Knowledge soundness (§3) then extracts a witness for the relation instantiated at exactly those public values; a prover that does not know such a witness — whatever witnesses it holds for other instances — convinces the verifier with at most negligible probability. This binds the proof to its public statement: an Action proof that a node verifies against an anchor it accepts, a nullifier it will mark spent, and a value commitment it will sum into the net balance check (the *Ironwood Guide*) can only verify if the prover knew a witness consistent with *those* values.

5.9 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness (§3) establishes that a convincing prover *knows* a witness; it remains to explain why the proof *reveals* no more than that. Halo 2 is zero-knowledge through two devices — hiding commitments and blinded polynomials — assembled into a simulator.

Hiding commitments. The polynomial commitment of §5.5 is *perfectly hiding*. The prover commits a polynomial p as

$$\text{Commit}(p; r) = \langle \vec{p}, \vec{G} \rangle + [r]H, \quad r \xleftarrow{\$} \mathbb{F}_p,$$

with $\vec{p}$ the coefficient vector of p and H an independent generator of no known discrete-log relation to $\vec{G}$. The $[r]H$ term makes the commitment a *uniform* group element for every p , so a commitment by itself leaks nothing about what the prover committed (the perfect hiding of a Pedersen commitment, *Crypto Guide*). Every commitment the prover sends in phases 1–6 — the advice columns, the permuted lookup columns A' , S' , the permutation and lookup running products, the quotient chunks, and the multipoint polynomial f — has this form. The verifying-key and instance commitments are deliberately unblinded: they commit public data both parties recompute (§5.7, §5.8), so there is nothing to hide.

Blinding the witness polynomials. The prover eventually *opens* a commitment, and an opening reveals evaluations. The prover interpolates the advice columns over the domain H to polynomials of degree $< n$; to prevent those evaluations from leaking the witness, the prover does not use all n rows for circuit data. It reserves a small fixed number of rows at the bottom of the table — the *blinding rows*, never covered by an active gate — and fills them with uniform random field elements, at least one more than the number of times the verifier will open any single column. Its n evaluations pin down a degree- $(< n)$ polynomial, so with enough independent random values planted on the blinding rows

the handful the verifier actually queries are jointly uniform and independent of the circuit data. The prover blinds the permutation and lookup running products the same way; Remark 5.2 describes how the constraint system exempts the blinding rows, so the random values never violate an argument. The quotient pieces admit no such treatment — C fully determines them — so their commitment blinders, together with the blinding-row randomness they inherit through C , protect them instead; their single collapsed opening at x is the publicly computable value $C(x)/(x^n - 1)$. (The deployed implementation also commits a uniformly random polynomial alongside the quotient and opens it at the same point, keeping the multipoint argument zero-knowledge.)

Why the openings leak nothing. In each phase the verifier sees only (i) hiding commitments — uniform group elements — and (ii) a bounded number of evaluations of the committed polynomials at the Fiat–Shamir challenge points. The blinding rows make each such evaluation uniformly random subject only to the *public* relations the verifier itself checks; no information about the private witness passes through. The final inner-product opening (§5.5) likewise runs with a blinder, so it too hides its input.

The simulator. Formally, zero knowledge is the existence of a *simulator* that, given only the public input and *not* the witness, outputs a transcript indistinguishable from a real one. Halo 2’s simulator runs the protocol “backwards,” exactly as Schnorr’s does (*Crypto Guide*): it chooses the evaluations and the final opening so as to satisfy the verifier’s equations, then chooses hiding commitments consistent with them — possible precisely because the commitments are perfectly hiding and it may choose the blinders freely — and, non-interactively, *programs* the Fiat–Shamir oracle so the challenges come out as required. Since the commitments are perfectly hiding and a blinder protects every opening, the simulated transcript is distributed as a real one. Interactively this is honest-verifier zero knowledge; compiled through Fiat–Shamir in the random-oracle model it is a NIZK that reveals nothing beyond the truth of the statement.

The three properties together. For the relation fixed by a PLONKish circuit, Halo 2 thus delivers all three guarantees a SNARK should: *completeness* (an honest prover holding a satisfying witness always convinces the verifier), *knowledge soundness* (§3: from any convincing prover an extractor pulls a satisfying witness, so an accepted proof certifies one is *known*), and *zero knowledge* (a witness-free simulator reproduces the proof, so it leaks nothing else). It is exactly this combination — “the prover knows a satisfying witness, and the proof discloses nothing more” — on which the Orchard Action proof rests: knowledge soundness yields balance and nullifier integrity, zero knowledge yields privacy (*Ironwood Guide*).

Part IV

How Halo 2 Proves: One computation, carried from claim to conviction

Abstract

A companion to the *Halo 2 Guide*. That volume develops the proof system top-down — the polynomial-IOP design pattern, knowledge soundness, the inner-product commitment, accumulation. This one is bottom-up and has a single aim: to make it *obvious* why a Halo 2 proof convinces anyone of anything. We fix one tiny statement — “I know x with $x^3 + x + 5 = 35$ ” — and carry it the entire distance: into a grid of constraints, into polynomials, into a single polynomial identity, into a commitment, and finally into the verifier’s check of a handful of field elements. Every number is real (we work in $\mathbb{F}_{97}$ so the arithmetic fits on the page), and at each turn we let a cheating prover try to lie and watch precisely where the lie is caught. The heavy machinery (the inner-product argument, the formal extractor, accumulation) is sketched and deferred to the *Halo 2 Guide*; the *idea of proving* is developed in full here.

1 What we are going to prove, and what “prove” means

Pick a claim small enough to hold in one hand:

“I know a secret x such that $x^3 + x + 5 = 35$.”

The secret is $x = 3$ (indeed $27 + 3 + 5 = 35$). The whole of this guide follows that one sentence until a sceptic is convinced of it — without ever being told that $x = 3$.

Three demands make this nontrivial, and a Halo 2 proof meets all three. The proof is *complete*: an honest prover who knows x always convinces the verifier. It is *sound* (more precisely, *knowledge sound*): a prover who does *not* know a valid x is rejected, except with negligible probability — and “knowledge” is meant literally, in the sense that a satisfying x could in principle be extracted from any prover who succeeds. And it is *zero-knowledge*: the verifier finishes convinced yet learns nothing about x beyond the bare truth of the claim. Completeness is easy; the content of the subject is *soundness with zero-knowledge*, and that is what we will watch being built.

Why this toy? Because it is the Orchard spend statement in miniature. When Alice spends a note she proves “I know a note, a key, and a Merkle path (*Crypto Guide*, §“Merkle trees and commitments to sets”) such that the nullifier — the unique tag whose publication marks the note spent — is computed correctly, the value commitment (a Pedersen commitment to the note’s value; *Crypto Guide*, §“The Pedersen commitment”) opens, and the note sits in the tree” — a conjunction of a few thousand small arithmetic facts over $\mathbb{F}_{p_{\text{Pallas}}}$ (the *Ironwood Guide*, checks A1–A10). Our claim is a conjunction of *four* small arithmetic facts. The proof system does not care which; it is the same machine at two scales. Master $x^3 + x + 5 = 35$ and the Action circuit is only larger, not different.

The journey. The route has five legs, and each is a translation that loses nothing:

1. **Computation** → **grid** (§2). Rewrite the claim as a table of rows, each row one elementary gate ($\times$ or $+$), with wires between rows forcing shared values.
2. **Grid** → **polynomials** (§3). Interpolate each column of the table into a polynomial. “Every row obeys its gate” becomes “one polynomial $G(X)$ is zero at every row.”
3. **Vanishing** → **identity** (§4). “ G is zero on all rows” becomes the clean algebraic identity $G(X) = t(X)Z_H(X)$ for some quotient t . A correct computation *is* a polynomial identity; an incorrect one cannot be.

4. **Identity → one random check** (§5). The verifier tests the identity at a single random point z . Here lives the magic: a false identity survives a random point with vanishing probability (Schwartz–Zippel).
5. **Trust the openings** (§6–§9). A binding *commitment* stops the prover from changing her polynomials after seeing z ; the permutation argument enforces the wiring; an instance column pins the public “35”; and blinding makes the whole transcript reveal nothing.

Section 12 threads the legs back together into one sentence: why the verifier, having checked a few numbers, is right to be convinced.

We work throughout in the prime field $\mathbb{F}_{97}$. This is a toy size chosen so the reader can check every product by hand; the security of the real system comes from running the identical argument over a 255-bit field, where the “vanishing probability” below is genuinely 2^{-240} rather than our visible $3/97$.

2 From computation to a grid of constraints

A verifier cannot re-run Alice’s computation: that would need her secret. So we first dismantle the computation into *local* facts, each so simple it involves only a handful of values, and arrange them so that “all the local facts hold, and the values are consistently shared” is equivalent to “the whole computation ran correctly.” This rearrangement is *arithmetisation*; the Halo 2 dialect of it is *PLONKish* (the *Halo 2 Guide*, § PLONKish arithmetisation).

Break the computation into gates. Evaluate $x^3 + x + 5$ the way a machine would, naming each intermediate:

$$x^2 = x \cdot x, \quad x^3 = x^2 \cdot x, \quad s = x^3 + x, \quad s + 5 = 35.$$

Four elementary operations, two multiplications and two additions. Each becomes one row of a table. A row has three *wires* — call them a, b, c , the two inputs and the output — and a bank of *selectors* q_M, q_L, q_R, q_O, q_C that switch on whichever gate this row performs. The universal gate equation, evaluated at every row, is

$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0, \quad (1)$$

where PI carries any public input on that row (here, the target 35). A multiplication gate $a \cdot b = c$ is the choice $q_M = 1$, $q_O = -1$, the rest zero; an addition gate $a + b = c$ is $q_L = q_R = 1$, $q_O = -1$. Filling in our four operations gives the *execution trace*, Table 1. (The deployed system generalises this fixed selector bank to designer-chosen gates and lookups, declared once and filled row by row at proving time; the *Halo 2 Guide*’s section “From statement to circuit” shows that machinery on the real Orchard gates.)

row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate performed
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Table 1: The execution trace for $x = 3$. Each row satisfies the gate equation (1): e.g. row 0 gives $1 \cdot 3 \cdot 3 + (-1) \cdot 9 = 0$, and row 3 gives $1 \cdot 30 + 5 + (-35) = 0$. The selector columns q_i and the public input PI are *fixed* (known to the verifier); the wires a, b, c are the prover’s *advice* (her secret trace).

Wire the rows together: copy constraints. The table as printed has a fatal loophole. Nothing yet forces the x in row 0 to be the same x in rows 1 and 2, nor the output $x^2 = 9$ of row 0 to be the input of row 1. A cheating prover could put $x = 3$ in row 0 but $x = 8$ in row 2 and satisfy each gate *in isolation* while computing nonsense overall. We must assert that certain cells are *equal*. These are the *copy constraints* (or *equality constraints*):

$$\underbrace{a_0 = b_0 = b_1 = b_2}_{\text{all are } x=3}, \quad \underbrace{c_0 = a_1}_{x^2=9}, \quad \underbrace{c_1 = a_2}_{x^3=27}, \quad \underbrace{c_2 = a_3}_{s=30}.$$

They are what turn a list of unrelated gates into a single dataflow. Enforcing them is a small argument in its own right (§7); for now simply note what they say.

What the grid achieves. The claim is now equivalent to a purely combinatorial assertion: *there exist values for the advice cells a, b, c such that (i) every row satisfies the gate equation (1) with the fixed selectors, and (ii) every copy constraint holds.* Knowing x is exactly knowing how to fill the advice columns. The verifier knows the fixed columns and the public input; the prover must convince him that satisfying advice exists, while revealing none of it. Everything from here is machinery for doing precisely that.

3 From the grid to polynomials

The grid has a finite number of rows, and we want to make statements about “all rows at once” that a verifier can check by looking at *one* place. Polynomials are the tool: a low-degree polynomial is rigid — pinned down everywhere by its behaviour at a few points — and that rigidity is what we will weaponise.

An index set for the rows. Our table has $n = 4$ rows. Choose four distinct field elements to name them. The inspired choice is the *n -th roots of unity*: a value ω with $\omega^4 = 1$ and no smaller power equal to 1. In $\mathbb{F}_{97}$ we may take $\omega = 22$, for which $\omega^2 = 96 = -1$ and $\omega^4 = 1$, so the row labels are

$$H = \{\omega^0, \omega^1, \omega^2, \omega^3\} = \{1, 22, 96, 75\} = \{1, 22, -1, -22\}.$$

Row i now lives at the point ω^i . (The roots of unity are chosen not for elegance but for speed: they make the polynomial arithmetic below an FFT (*Math Guide*, §“The fast Fourier transform”). Any 4 distinct points would do for correctness.)

Columns become polynomials. Each column of the table is four numbers, one per row. *Interpolate*: for the advice column a there is a unique polynomial $a(X)$ of degree < 4 with

$$a(\omega^0) = 3, \quad a(\omega^1) = 9, \quad a(\omega^2) = 27, \quad a(\omega^3) = 30,$$

and likewise $b(X), c(X)$, and the (publicly known) selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI , each a polynomial in X of degree < 4 . The column *is* the polynomial, sampled at the rows. A polynomial of degree < 4 is exactly four coefficients, so no information is gained or lost — only re-encoded into a form that bends to algebra.

All gates at once, as one polynomial. Now assemble the left-hand side of the gate equation (1), but with the *polynomials* in place of the per-row numbers:

$$G(X) := q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + PI(X). \quad (2)$$

This G has degree at most 9 (the term $q_M ab$ multiplies three degree-3 polynomials). Here is the point of the whole construction. Evaluate G at the label of row i : because each polynomial returns that row's value at ω^i , $G(\omega^i)$ is *exactly* the gate equation (1) for row i . Therefore

$$G(\omega^i) = 0 \text{ for every row } i \iff \text{every gate is satisfied.}$$

For our honest trace this is true by construction: G vanishes at all four points of H . “The computation is correct” has become “ G vanishes on H .” We have replaced four separate checks with a single statement about one polynomial — and it is a statement we can now make checkable in one shot.

4 From vanishing to a single identity

“ G vanishes on the set H ” is still a statement about four points. One more step compresses it into a statement with *no* reference to the individual rows at all — a bare algebraic identity — and this is the technical heart of arithmetic proving.

The vanishing polynomial. The polynomial that is zero exactly on H , and nowhere else, and as simply as possible, is

$$Z_H(X) := \prod_{i=0}^3 (X - \omega^i) = X^4 - 1.$$

(The product of $(X - \zeta)$ over all n -th roots of unity ζ telescopes to $X^n - 1$; that is the only reason roots of unity are convenient here.) The elementary fact we lean on is the factor theorem, applied n times:

$$G \text{ vanishes on all of } H \iff (X - \omega^i) \mid G \text{ for each } i \iff Z_H(X) \mid G(X). \quad (3)$$

Divisibility is the same as the existence of a quotient. So:

$$\text{every gate holds} \iff G(X) = t(X) Z_H(X) \text{ for some polynomial } t(X).$$

A correct computation is *precisely* the existence of this quotient t . For our honest trace the division comes out even: G has degree 9, Z_H has degree 4, and

$$t(X) = \frac{G(X)}{X^4 - 1}$$

is an honest polynomial of degree 5 — the remainder is zero, as it must be.

Why a cheat cannot produce t . Suppose the prover's advice is *wrong* — some gate fails. Then G does *not* vanish on all of H , so by (3) Z_H does *not* divide G , so *no* polynomial t satisfies $G = t Z_H$. The dividing line is exact: the quotient exists if and only if the trace is valid. This is the lever the verifier will pull. Concretely, take a prover who claims $x^2 = 10$ and writes $c_0 = 10$ in row 0 (Table 1) instead of 9. Now row 0's gate reads $3 \cdot 3 - 10 = -1 \neq 0$, so the tampered G^* has $G^*(\omega^0) = -1 = 96$ while still vanishing at the other three rows. Dividing G^* by Z_H leaves a nonzero remainder; the best the cheater can do is publish the quotient $t^* = \lfloor G^*/Z_H \rfloor$, and then

$$D(X) := G^*(X) - t^*(X) Z_H(X) = 24(1 + X + X^2 + X^3)$$

is a *nonzero* polynomial of degree 3. The identity she needs is false, and D measures exactly by how much. Keep D in view: the next section is about catching it cheaply.

5 Why checking one random point is a proof

The prover wishes to convince the verifier that $G(X) = t(X) Z_H(X)$ as polynomials, with G built from her secret advice. The verifier will not read the polynomials — they encode the witness, and there are too many coefficients to be cheap anyway. Instead:

1. The prover *commits* to her polynomials a, b, c and the quotient t (Section 6 says what a commitment is and why it binds her); the selector and public-input polynomials the verifier already knows.
2. The verifier picks a point $z \in \mathbb{F}$ *uniformly at random* and sends it.
3. The prover *opens* her commitments at z : she returns the field elements $a(z), b(z), c(z), t(z)$ together with short proofs that these are the true evaluations of the committed polynomials.
4. The verifier forms $G(z)$ from the openings and the (self-evaluated) public polynomials via (2), computes $Z_H(z) = z^4 - 1$, and checks the *single field equation*

$$G(z) \stackrel{?}{=} t(z) Z_H(z). \quad (4)$$

He accepts if and only if (4) holds. That is the entire test: one equation between numbers.

The honest case (completeness). If the prover is honest then $G = t Z_H$ holds as polynomials, hence at every point, hence at z . With our trace and $z = 20$:

$$Z_H(20) = 20^4 - 1 = 46, \quad t(20) = 67, \quad G(20) = 75 = 67 \cdot 46 = t(20) Z_H(20). \quad \checkmark$$

The verifier accepts, every time, for every z .

The dishonest case (soundness), and the one idea that makes it work. Suppose the trace is invalid — some gate fails, say at row i , so $G^*(\omega^i) \neq 0$ and G^* does *not* vanish on all of H . Take *any* polynomial t^* the cheating prover might commit to (the honest quotient, or anything else) and set

$$D(X) := G^*(X) - t^*(X) Z_H(X).$$

Were D the zero polynomial, then $G^* = t^* Z_H$ would be divisible by Z_H and so vanish on all of H — which it does not. Hence D is a *nonzero* polynomial, however cleverly t^* was chosen. And D is fixed *before* z is drawn: committing to a, b, c pins down the polynomial G^* (the verifier never receives G^* itself — from the openings at z he reconstructs only the single *number* $G^*(z)$), and committing to t^* pins down the rest, so the entire polynomial D is frozen by the commitments before the challenge is revealed. The check (4) passes at z exactly when $D(z) = 0$ — that is, exactly when the random z happens to be a *root* of D . And here is the lever: *a nonzero polynomial of degree d has at most d roots* (Schwartz–Zippel, in one variable just the factor theorem). A degree- d polynomial is pinned down by any $d + 1$ of its values, so it has no freedom to vanish at more than d points without collapsing to zero everywhere; a nonzero low-degree polynomial is therefore *mostly* nonzero, its roots a sparse set that a random probe almost surely misses. So

$$\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|}.$$

A liar is caught with overwhelming probability over the verifier’s single coin toss. *This* is what “proving” means here: not that the verifier re-checks the work, but that a false statement is an algebraic object so rigid it cannot hide — it disagrees with the truth almost everywhere, and one random probe almost surely lands where it disagrees.

Watch it happen. For our $x^2 = 10$ cheat, $D(X) = 24(1 + X + X^2 + X^3)$ has degree 3, so at most 3 roots — and in $\mathbb{F}_{97}$ its roots are exactly $\{22, 96, 75\}$, the three rows where the tampered gate still held. The cheat therefore slips past only if the verifier’s random z lands in that three-element set:

$$\Pr[\text{cheat accepted}] = \frac{3}{97}.$$

At the verifier’s actual draw $z = 20 \notin \{22, 96, 75\}$, we get $G^*(20) = 20$ but $t^*(20)Z_H(20) = 64$, and $20 \neq 64$: caught. The fraction $3/97$ is large only because our field is a toy. Over Orchard’s 255-bit Pallas field the gap polynomial still has at most $\deg D$ roots — now a vanishingly small fraction of the $\approx 2^{254}$ candidate points, a soundness error on the order of 2^{-240} from a *single* draw of z — so one random challenge already suffices, and the deployed proof even replaces that coin toss with a hash of the transcript (Fiat–Shamir, §10).

Bounding the degree. The argument needs D to have *low* degree relative to $|\mathbb{F}|$ — otherwise “ $d/|\mathbb{F}|$ roots” is no constraint. This is why a circuit’s gates are kept to low degree and why t , of degree ≤ 5 here (and up to a few times n in general), is split into bounded pieces in the deployed system. Low degree is not an efficiency footnote; it is the source of soundness.

6 Why the prover cannot wriggle: binding commitments

The random-point argument has an obvious gap: it assumed the prover’s polynomials were *fixed before* she saw z . If she could choose a, b, c, t *after* learning z , she would simply pick numbers making (4) hold and reveal nothing real. A *polynomial commitment* closes exactly this gap.

What a commitment must do. A commitment to a polynomial f is a short string $C = \text{Commit}(f)$ with two properties (the *Crypto Guide* develops both):

- **Binding.** Having published C , the prover cannot later open it as two different polynomials; C pins down f . So committing first, then receiving z , then opening, leaves her no freedom: she is answering for the *one* polynomial she fixed in advance — the situation Section 5 assumed.
- **Hiding.** C reveals nothing about f itself. (This is what will let the proof be zero-knowledge, §9.)

Add an *evaluation proof*: given C and a point z , the prover can prove “ $f(z) = v$ ” for the committed f without exposing f . With binding plus trustworthy evaluation proofs, the four-step protocol of Section 5 is exactly as sound as its idealised version: the opened numbers $a(z), \dots, t(z)$ are forced to be the true evaluations of fixed polynomials, so the check (4) tests a genuine, pre-committed identity at a genuinely random point.

How Halo 2 commits, in one breath. Halo 2 uses the *inner-product argument* (IPA). Encode f by its coefficient vector $\vec{f} = (f_0, \dots, f_m)$ and fix a public list of independent curve points $\vec{G} = (G_0, \dots, G_m)$ with no known relationship. The commitment is the single curve point

$$C = \langle \vec{f}, \vec{G} \rangle = \sum_i f_i G_i,$$

a *Pedersen* vector commitment: binding because finding two openings would solve a discrete-log relation among the G_i , and hiding once a random blinding term is added. To prove $f(z) = v$ is to prove a statement about the inner product of $\vec{f}$ with the vector $(1, z, z^2, \dots)$, and the IPA does this in a logarithmic number of rounds, each *halving* the vectors, until the claim is trivial — so the evaluation proof is $O(\log m)$ group elements and, decisively, needs *no trusted setup* (§11 says why, and what the fold’s one expensive step costs). The full fold and its soundness are the business of the *Halo 2 Guide* (§“The inner-product polynomial commitment”). For our purposes one property is enough: *the commitment binds*, so the prover is honestly answering for polynomials she could not change once z was on the table.

7 Enforcing the wiring: the permutation argument

One promise from Section 2 is still unpaid: the copy constraints. The gate identity of Sections 3–5 checks each row *in isolation*; it never says that a_0, b_0, b_1, b_2 are the same value x , or that row 1’s input is row 0’s output. Without that, a prover could satisfy every gate with an incoherent dataflow. The *permutation argument* adds the missing constraint, and it does so with the same philosophy: turn “these cells are equal” into “a certain product is 1,” then check the product at a random point.

Equalities as a permutation. Group the cells into classes that must share a value — here $\{a_0, b_0, b_1, b_2\}, \{c_0, a_1\}, \{c_1, a_2\}, \{c_2, a_3\}$ — and let σ be the permutation that cycles each displayed class in the order written. Give every cell a distinct *label*: cell in row j of the a -, b -, c -column gets $\omega^j, k_1\omega^j, k_2\omega^j$ respectively, where k_1, k_2 place the three columns in disjoint cosets so no two labels collide (in $\mathbb{F}_{97}$ we may take $k_1 = 2, k_2 = 3$). Asserting “cells in a class are equal” is then equivalent to asserting that the map sending each cell’s *value-with-label* to its *value-with- σ -permuted-label* is consistent — a balance between two multisets.

The grand product. With verifier challenges β, γ , the prover builds a running product that telescopes across all $3n$ cells:

$$\prod_{\text{cells } (i,j)} \frac{v_i(\omega^j) + \beta(\text{label}) + \gamma}{v_i(\omega^j) + \beta(\sigma\text{-label}) + \gamma} = 1 \iff \text{every copy constraint holds.}$$

If the wiring is honest, numerator and denominator run over the *same* multiset of factors (just re-ordered by σ), so the ratio is 1 identically; if any wired pair disagrees, the multisets differ and the product misses 1 except with probability $O(n/|\mathbb{F}|)$ over β, γ — Schwartz–Zippel again. The prover commits to this running-product polynomial and the verifier checks its boundary ($= 1$ at the first row) and its row-to-row update at the random z , just another handful of openings.

A two-cell glimpse. Strip the product down to a single wired pair, $c_0 = a_1$ (both should be 9), carrying labels L_{c_0}, L_{a_1} that σ swaps. Their combined contribution to numerator-over-denominator is

$$\frac{(9 + \beta L_{c_0} + \gamma)(9 + \beta L_{a_1} + \gamma)}{(9 + \beta L_{a_1} + \gamma)(9 + \beta L_{c_0} + \gamma)} = 1,$$

because the shared value 9 makes the two numerator factors a mere reordering of the two denominator factors — the labels may permute under σ precisely because the values agree. Now let the prover cheat the copy, $a_1 = 8 \neq 9$: the numerator factor $8 + \beta L_{a_1} + \gamma$ matches no denominator factor, the cancellation breaks, and the pair’s contribution drifts off 1. Equal values let labels permute; one unequal value strands a factor, and the whole product feels it.

Watch it happen. For our honest trace, with $\beta = \gamma = 2$, the grand product comes out to exactly 1. Tamper one wire — set $b_1 = 4$, breaking the copy $b_1 = x = 3$ — and the same product becomes $61 \neq 1$: the inconsistent dataflow is exposed without any value being revealed.

8 Proving a value sits in a table: lookups

Gates handle algebra; some constraints are not algebra. “This chunk indexes an entry of the Sin-semilla generator table” or “this cell is a ten-bit number” has no low-degree formula. Written as one polynomial gate it would need degree about the size of the table — which in our toy field would eat the soundness budget outright (§5), and at any scale balloons the quotient polynomial the prover must split, commit, and open (§4). Yet Orchard’s circuit needs such range and table facts constantly.

The statement has a familiar shape. “Every entry of my column A appears somewhere in the table S ” is, like the copy constraints, a statement about *multisets*. One wrinkle: membership is not a permutation — A may repeat one table entry and ignore the rest — so the prover first tidies both sides. She commits to A' , her column reshuffled so equal values sit in contiguous runs, and S' , a reshuffle of the table aligned so that each run in A' begins beside its matching table value. Two *row-local* checks then pin membership: each row of A' either *starts a run* ($A'_i = S'_i$: the value is vouched for by the table) or *continues one* ($A'_i = A'_{i-1}$: equal to the row above, already vouched for). What certifies the two reshuffles is the §7 machinery with one deliberate change: *no labels*. Labelled factors pinned one particular wiring σ ; here the reshuffles are the prover’s own choice, so a single running product with unlabelled factors — row by row, $(A' + \beta)(S' + \gamma)$ against $(A + \beta)(S + \gamma)$ — certifies exactly that each primed column is *some* reshuffle of its original, which is all membership needs. Induction up the rows does the rest.

On the toy’s own four-row grid: constrain cells to two-bit values with the table $S = (0, 1, 2, 3)$ and the column $A = (2, 3, 2, 2)$. Grouped, $A' = (2, 2, 2, 3)$; align $S' = (2, 0, 1, 3)$, a reshuffle of S placing 2 beside the first run and 3 beside the second. Row 0 starts a run ($2 = 2$), rows 1 and 2 continue it (2 above), row 3 starts a run ($3 = 3$) — every check row-local, every polynomial low-degree, however large the table. A forged $A = (2, 7, 2, 2)$ dies: its 7 lands at the start of a run in A' , and no reshuffle of a table without a 7 can put a 7 beside it.

This is how the deployed circuit affords its ten-bit range checks and its 2^{10} -entry Sinsemilla generator table: a table fact costs a few committed columns and one more running product, and the gates stay low-degree. The deployed declarations of both are exhibited in the *Halo 2 Guide*, §“From statement to circuit: arithmetisation in practice”; the exact constraints, the boundary row, the padding of short columns, and multi-column tables are its §“The lookup argument”.

9 Binding the public statement, and hiding the witness

Two finishing pieces complete the picture, one for soundness and one for secrecy.

Pinning the public “35”: the instance column. A proof must attest to *this* statement, not some other the prover finds convenient. The target 35 is public, so it is not advice but *instance* data: it sits in the public-input polynomial $PI(X)$ of (2), which the verifier builds himself (it is -35 on row 3 and 0 elsewhere). Because PI enters the very identity being checked, a prover who tried to prove “ $x^3 + x + 5 = 36$ ” would be checking a *different* G and her honest trace would no longer vanish on H . The public input is thus welded into the same polynomial identity that soundness already guards (the *Halo 2 Guide*, § binding the public statement). The verifier proves something about the claim he chose, not one the prover swapped in.

Revealing nothing: zero-knowledge. So far the prover has handed over commitments and a few evaluations $a(z), b(z), \dots$ at a random z . Do these leak the witness? They could: an evaluation is a linear equation in the secret coefficients, and enough of them would pin x down. Halo 2 closes this the standard way — *blinding*. Each committed polynomial is padded with a few extra random coefficients (equivalently, the trace reserves a few random rows), and the commitment carries an independent random blinding term. The gate and permutation arguments are arranged to ignore those random rows (the *Halo 2 Guide*’s active-rows remark), so they do not disturb soundness; but the openings at z are now masked by fresh randomness and are distributed independently of x . Formally one exhibits a *simulator* that produces an identically distributed transcript knowing only that the statement is true — so whatever the verifier sees, he could have generated alone, hence learned nothing. The simulator construction is deferred to the *Halo 2 Guide* (§ zero knowledge in Halo 2); the intuition is simply: *commitments hide, and blinded openings are noise pinned to one honest constraint*.

10 Removing the verifier: the Fiat–Shamir transform

One awkwardness remains. Everything above needs a *live* verifier for exactly one service: once the commitments a challenge tests are pinned, somebody the prover cannot predict must choose that challenge — z here, β and γ for the permutation argument, the coins inside each opening round (§6). Nothing else about him matters: his messages are coin tosses, and his final act is a computation anyone could run. A payment protocol cannot work this way. Alice would have to sit in a live session with every full node that will ever validate her transaction.

The fix is almost impudent. Wherever the verifier would send a fresh challenge, the prover instead computes it herself as

$$\text{challenge} := H(\text{everything sent so far}),$$

the digest of the entire conversation to that point, public statement and every commitment included, under a fixed hash function H . The ordering that §6 fought for is now enforced by construction: a commitment is an *input* of the hash, so it is fixed before the challenge that tests it exists at all.

Why is a hash as good as a coin? Because the prover’s only route to a favourable challenge is to vary some input of H — and every input she controls is something the finished proof must stand behind: the statement she is claiming, or a commitment she must open. Each variation re-rolls the challenge blindly: one hash evaluation buys one ticket in a lottery she wins only with the soundness error, about 2^{-240} per draw of z (§5). She may grind through as many tickets as she can hash; against odds like these it changes nothing. Two fine points matter. The hash must digest the *whole* transcript — leave the statement or any commitment out, and she aims precisely at what the hash cannot see. And the guarantee is proved with H modelled as a random oracle, a strong but standard heuristic; the *Crypto Guide* (§“The Fiat–Shamir transform: from interactive to non-interactive”) gives the theorem and its caveats.

What this buys is the “N” in SNARK: *non-interactive*. The proof becomes a single static object — commitments and openings, nothing else — and verification becomes recomputing the challenges by the same hash and running the same checks. No session, no coordination: a full node that has never met Alice verifies her Action years later, exactly as the deployed system does (the *Halo 2 Guide*, §“The complete Halo 2 protocol”, describes the transcript it uses).

11 The “Halo” in Halo 2: deferring the expensive check

One cost has stayed quietly non-trivial. The opening proofs of §6 shrink by halving, so the verifier’s rounds are logarithmic — except at the very end, where checking the final fold means recomputing one combination of *all* the generators $G_0, \dots, G_m$: a wall of group arithmetic proportional to the whole circuit’s size. Even without any recursion the deployed verifier declines to pay this per proof: checking a batch of transactions, it folds all their deferred walls into one and pays a single wall for the batch (the *Halo 2 Guide*, §“Accumulation and the Halo trick”). Where the cost would be truly ruinous is *recursion*. For a circuit to verify a *proof* — the step that would let one proof attest to a whole chain of others — everything the verifier does must be re-expressed as gates, and a circuit-sized wall of group arithmetic inside a circuit defeats the point.

Halo’s namesake trick is to not pay. The expensive claim has a special shape — “this point equals that known combination of the generators” — and two claims of this shape can be *folded* into one by the move this guide keeps meeting: commit, draw a challenge, take a random combination, a few group operations in place of a wall. So the recursive verifier checks everything cheap, and instead of paying the expensive step it folds the claim into a running *accumulator* carried along the chain. One final check, run once at the chain’s end and outside any circuit, pays a single wall of group arithmetic for the entire history. The bookkeeping’s soundness reduces to the discrete logarithm, exactly like the commitments it manages (the *Halo 2 Guide*, §“Accumulation and the Halo trick”; the cycle of curves that keeps the recursive arithmetic in the right fields is its §“Recursive proof composition and accumulation schemes”).

A last remark completes the picture begun in §6: none of this ever needed a ceremony. The generators $G_0, \dots, G_m$ are nothing-up-my-sleeve points, hashed into existence, with no secret behind them that anyone ever knew — nothing to leak, nothing to destroy. The pairing-based KZG family buys constant-size opening proofs and constant-time verification at the price of a *trusted setup*: secret randomness contributed through a ceremony, whose compromise permits silent forgeries (the *Crypto Guide*, §“Polynomial commitment schemes”, constructs that scheme and the forgery its trapdoor permits). Zcash’s earlier shielded pools likewise needed parameter ceremonies for their Groth16 proof systems, though that is a separate construction. Halo 2’s inner-product commitment trades proof size for exactly this freedom; being rid of ceremonies was a stated reason Orchard chose it.

12 Why the verifier is convinced

Stand back and read the chain in the direction the verifier experiences it. He accepts a proof. What has he thereby learned?

1. The check $G(z) = t(z)Z_H(z)$ passed at his random z . Because the commitment *binds* (§6), the polynomials behind the openings were fixed before z ; because z was random and a false identity disagrees with the true one at all but $\leq \deg D$ points (§5), passing means $G(X) = t(X)Z_H(X)$ holds *as polynomials*, save for a 2^{-240} fluke.
2. $G = tZ_H$ means $Z_H \mid G$, which means G vanishes on all of H (§4), which means *every gate equation holds at every row* (§3).
3. The permutation grand product passed (§7), so the *copy constraints* hold too: the per-row gates are wired into one coherent computation, not four disconnected coincidences.
4. The public-input polynomial fixed the target at 35 (§9), so the computation he has verified is the one he asked about.

Putting these together — and recalling from §2 that filling the advice columns *is* knowing x , since those cells are the wires carrying x and its powers — there exists an assignment to the advice columns, a value of x , making the entire arithmetisation of “ $x^3 + x + 5 = 35$ ” hold. The prover demonstrably *knew* such an x ; indeed the soundness proof is constructive — an *extractor* can rewind a successful prover and read x out of her openings (the *Halo 2 Guide*, § knowledge soundness). And by zero-knowledge (§9) the verifier extracted no more than this single bit of truth. He has checked perhaps a dozen field elements and a logarithmic-size opening proof, yet he is as certain as 2^{-240} that the prover holds a secret cube-root-ish witness — without the faintest idea what it is.

The same machine, at Orchard scale. Nothing above used that our statement was small. Replace the four gates by the few thousand constraints of the Orchard Action — nullifier integrity, the value-commitment opening, the Merkle path recomputation, key re-randomisation (*Crypto Guide*, §“Key re-randomisation and unlinkability”; the *Ironwood Guide*, checks A1–A10) — and the columns grow from four rows to 2^k , the gate polynomial gains custom high-degree terms, and a Sinsemilla lookup table (§8; the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”) joins the permutation. But the spine is identical: arithmetise into a grid, interpolate to polynomials, compress “all rows correct” into one identity $G = tZ_H$, commit, and test at a random point. When a Zcash full node verifies a shielded spend, it is running exactly the check of Section 5 — one random evaluation of a committed polynomial identity — and concluding, with cryptographic certainty and learning nothing, that somewhere a valid note was spent by its rightful owner.

Where to go next. Every deferral above is discharged in the *Halo 2 Guide*: the inner-product argument and why its openings are trustworthy; the algebraic-group-model extractor behind “the prover knows x ”; the soundness of the accumulation fold sketched in §11; and the cycle of curves that makes

that recursion efficient. This guide has supplied the thing those formalisms are formalising: the picture of *why a single random number, checked against a committed polynomial, is enough to know that an unseen computation was done right.*

Part V

Consensus Guide: Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule

Abstract

Assuming only the *Math Guide*'s probability foundations, this volume of *The Zcash Arboretum* constructs Nakamoto consensus from first principles and quantifies the security it buys. It states the block tree and the most-work rule as the protocol specification defines them and as the deployed zebra implements them; builds, on the *Math Guide*'s foundations, the race-specific probability the analysis needs (memoryless clocks, the biased random walk, the negative binomial law); develops the exact double-spending analysis of Rosenfeld's *Analysis of hashrate-based double-spending* — the catch-up theorem, the confirmation formula, and the economics of the attack — with every number recomputed by script and the results instantiated for Zcash's 75-second blocks; and closes with the deployed safety rails (per-block difficulty adjustment, reorg windows, coinbase maturity, wallet confirmation policy) that frame the model's assumptions. Where the specification, the paper, and the deployed nodes disagree, the divergence is stated.

1 Scope, sources, and the two double-spends

Every volume above this one takes a working ledger for granted: the *Ironwood Guide* proves what a shielded transaction — one whose amounts and addresses are cryptographically hidden — commits to, the *Wallet Guide* builds transactions, the *Sync Guide* fetches the chain, the *FlyClient Guide* verifies it succinctly. This volume documents the mechanism that makes there be *one* chain to fetch: Nakamoto consensus — proof of work, the most-work rule, and the probabilistic finality they purchase. The treatment is quantitative throughout. Its spine is Meni Rosenfeld's *Analysis of hashrate-based double-spending* (11 December 2012; latest version 12 February 2014; arXiv:1402.2009), the paper that replaced the approximate analysis in Satoshi Nakamoto's whitepaper with an exact one; we reprove its results in full, recompute its tables by script, and instantiate its conclusions for the deployed Zcash parameters.

Two distinct attacks are both called “double-spending”, and this volume treats exactly one of them. Spending the same note twice *within* one chain is impossible in Zcash by construction: each spend reveals a nullifier, and a block that repeats a nullifier is invalid (*Ironwood Guide*, §“Prevention of double-spending: nullifiers”). What consensus must prevent is the chain-level attack: paying a merchant on the chain everyone sees, then replacing that chain wholesale with another in which the payment never happened. No zero-knowledge proof (*Crypto Guide*, §“Interactive proofs, zero knowledge, and SNARKs”) rules this out; only the economics of proof of work bounds its probability, and this volume computes that bound.

The volume assumes exactly one thing from the series below it: the *Math Guide*'s probability section (§“Probability, asymptotics, and computation”), which constructs probability from its definition — finite spaces, conditioning, independence, random variables, expectation — and extends it to the three rungs used here: countable additivity, continuity along monotone events, and density-defined laws (§“Beyond finite spaces: countable additivity, limits, and densities”). On that base, Section 3 constructs the race-specific instruments no lower volume owns — memoryless clocks, random walks, the negative binomial law; the algebra used is elementary. Readers wanting only the results can read Sections 2, 6, and 8 and take the theorems on faith.

1.1 Sources and their standing

Table 1 lists the sources. The analysis (the paper and the whitepaper) is not normative for anything; the protocol specification and the deployed nodes are the ground truth for what Zcash actually does, and where the model’s assumptions and the deployment diverge, Section 8 says so explicitly. Worked numbers throughout are computed by script (`.wip/consensus-drafts/compute.py`, whose output is the source of every numeric claim in this volume); the script reproduces both of the paper’s printed tables cell-for-cell — Table 1 to its three-decimal rounding, Table 2 to its floored thousands-and-millions convention — before departing from them.

Source	Role	Pin
Rosenfeld, arXiv:1402.2009	primary analysis	v1, 2014-02-09
Nakamoto whitepaper, §11	historical analysis	2008
Zcash protocol specification	normative	zips 69610984
ZIP-208 (Blossom)	normative	same pin
zebra	the deployed validator	fa7657f1c ¹
zcashd v6.10.0	retired validator	16ac74376
librustzcash	deployed wallet layer	e30517e43

Table 1: Sources. The paper is analysis, not specification: nothing in it binds the protocol. Deployed-behaviour claims cite crate, file, and line at the pinned commits. The retired zcashd is cited as history: its choices explain constants that survive it (Section 8.4).

Remark 1.1 (The paper’s own condition). The arXiv v1 PDF of the paper is lightly broken: it contains exactly two citations, both rendered as “[?]” (in the abstract and in §1), and no References section at all. Both plainly intend the Nakamoto whitepaper, and we read them so. The paper numbers only two of its displayed equations — its equation (1) is our Theorem 5.2, its equation (2) our equation (3) — and sets its two model assumptions as a bare numbered list; the *Assumption* environments of Section 4 are ours.

2 The block tree and the most-work rule

Zcash transactions are grouped into blocks, and every block names its parent by including the parent header’s hash (*Crypto Guide*, §“Hash functions and the random oracle model”); the genesis block alone has no parent. Blocks therefore form a tree rooted at genesis, and each root-to-leaf path — each *branch* — is one internally consistent version of history. Branches need not be consistent with one another: one branch can contain a transaction that conflicts with a transaction in a sibling branch, and that freedom is exactly the attack surface this volume quantifies.

2.1 Work, and the best chain

A single coherent history requires a rule every node can apply locally and still agree on. The rule is not “longest”: it weighs each block by the computational effort its header proves.

Definition 2.1 (Work of a block). Let `ToTarget` be the conversion from the header’s `nBits` field to its 256-bit target threshold (specification §7.7.4). The *work* of a block is

$$\text{work} = \left\lfloor \frac{2^{256}}{\text{ToTarget}(\text{nBits}) + 1} \right\rfloor$$

¹A working-branch checkout; every file cited in this volume was verified byte-identical to `ZcashFoundation/zebra` main at `74cef5e6d`, so the citations hold for both.

(specification §7.7.5, “Definition of Work”). A lower target admits fewer valid headers, so a block meeting it certifies proportionally more expected hashing; work is that certificate made additive.

The deployed validator computes this quantity with a 256-bit in-word trick: zebra evaluates $(! \text{expanded}.0 / (\text{expanded}.0 + 1)) + 1$ (zebra-chain/src/work/difficulty.rs:401, stored in a u128); the retired zcashd computed the identical expression on `arith_uint256`, with ~ spelling the complement (src/pow.cpp:144--157). The form agrees with Definition 2.1 exactly: writing t for the target and $A = 2^{256}$, the complement is $\neg t = A - 1 - t$, and

$$\left\lfloor \frac{A - 1 - t}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A - (t + 1)}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A}{t + 1} \right\rfloor,$$

so the in-word computation equals the out-of-word quotient (checked by script over random targets).

Definition 2.2 (Best valid block chain). Specification §3.3 (“The Block Chain”): a node “sums the work, as defined in §7.7.5, of all blocks in each valid block chain, and considers the valid block chain with greatest total work to be best. To break ties between leaf blocks, a node will prefer the block that it received first.”

Rosenfeld’s paper states the same rule for Bitcoin — “the branch representing the most proof of work”, with first-seen tie-breaking — so the model transfers without adjustment. The popular shorthand “longest chain” is a corruption: length and work coincide only while difficulty is constant, which is precisely the paper’s Assumption 4.2 and not a property of the deployed chain (Section 8.2). Note also what the rule does *not* say: nothing marks a chain as permanently chosen. A node that learns of a heavier branch switches to it, however deep the fork point — subject only to the node-local rails of Section 8.4. Consensus is a standing competition, not an election with a winner.

Definition 2.3 (Confirmations). A transaction has n confirmations if it is included in a block of the best chain and there are n blocks on the path from that block to the chain’s leaf, inclusive. The block containing the transaction is its first confirmation.

Different nodes may briefly disagree on the best chain when two blocks of equal cumulative work arrive in different orders; the disagreement is resolved by the next block, which makes one branch strictly heavier (Figure 1). Such ties are the benign case. The attack of Section 5 is the malicious one: a fork manufactured in private and released only when it wins.

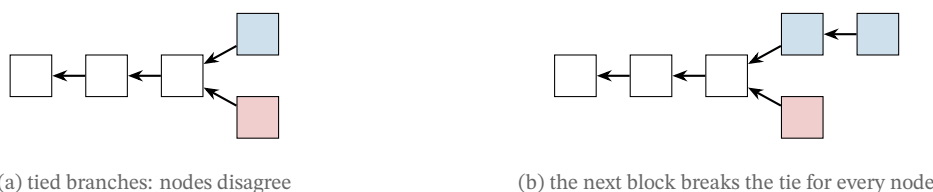


Figure 1: Branch selection. Arrows point from child to parent. In panel (a) two blocks extend the same parent and the branches carry equal work; each node keeps the one it saw first. In panel (b) a new block lands on the upper branch, which now has strictly more cumulative work; all nodes converge on it and the lower block is orphaned. At constant difficulty “more work” and “longer” coincide, which is how the paper, and this volume’s figures, may draw block counts.

2.2 The attack

The double-spend attack, as the paper lays it out (§3):

1. broadcast a transaction paying the merchant;
2. secretly mine a branch from the last pre-transaction block, containing instead a conflicting transaction paying the attacker himself;
3. wait until the merchant's transaction has n confirmations and the merchant, satisfied, hands over the goods;
4. keep extending the secret branch until it carries more work than the public one, then broadcast it. Every node switches to it, the payment to the merchant is no longer part of history, and the attacker has both the goods and the coins.

Nothing in step 4 violates any validity rule — the released branch is a perfectly well-formed chain, merely a different one. The only question is probabilistic: with what probability does the secret branch ever get ahead? Answering it exactly is the business of Sections 4 and 5; the toolkit comes first.

3 A probability toolkit

The analysis ahead needs four tools: memoryless clocks (to model block discovery), a reduction from continuous time to a coin sequence, the negative binomial distribution (to count the attacker's progress), and one normalisation lemma about races. Their foundations are the *Math Guide's*: probability spaces, conditioning, independence, random variables, and expectation are constructed there from the definition of probability onward (§“Probability, asymptotics, and computation”), and the extension beyond finite spaces — countable additivity, continuity along monotone events, density-defined laws, and infinite sequences of independent trials — is its §“Beyond finite spaces: countable additivity, limits, and densities”. What no lower volume owns are the four instruments themselves; each is constructed here on that base, only as far as the critical path requires.

3.1 Memoryless clocks and the attribution sequence

Definition 3.1 (Exponential clock). A random variable X is *exponential with rate* $\lambda > 0$ if $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ — a law defined by the density $\lambda e^{-\lambda t}$ on $t \geq 0$ in the sense of the *Math Guide's* density definition, with mean $1/\lambda$.

Lemma 3.2 (Memorylessness). An exponential X satisfies $\Pr[X > s + t \mid X > s] = \Pr[X > t]$ for all $s, t \geq 0$: having waited without success teaches nothing about the remaining wait.

Proof. $\Pr[X > s + t \mid X > s] = e^{-\lambda(s+t)}/e^{-\lambda s} = e^{-\lambda t}$. □

Lemma 3.3 (Race of two clocks). Let X and Y be independent exponentials with rates λ and μ . Then

$$\Pr[X < Y] = \frac{\lambda}{\lambda + \mu},$$

and the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$.

Proof. Integrating over the time at which X fires,

$$\Pr[X < Y] = \int_0^\infty \lambda e^{-\lambda t} \Pr[Y > t] dt = \int_0^\infty \lambda e^{-(\lambda+\mu)t} dt = \frac{\lambda}{\lambda + \mu}.$$

For the minimum, $\Pr[\min(X, Y) > t] = \Pr[X > t]\Pr[Y > t] = e^{-(\lambda+\mu)t}$. □

Proposition 3.4 (The attribution sequence). *Model the honest network and the attacker as independent exponential clocks with rates p/T_0 and q/T_0 , each restarting when either fires (the model of Section 4). Then the sequence that records, for each successive block found by anyone, who found it, is an infinite sequence of independent trials in the Math Guide’s sense — an independent, identically distributed sequence of Bernoulli trials: each block is the attacker’s with probability q and the honest network’s with probability p , independently of all earlier attributions and of all elapsed times.*

Proof. By Lemma 3.3 the first block is the attacker’s with probability $(q/T_0)/(p/T_0 + q/T_0) = q$. When a block is found, the finder’s clock restarts by construction, and the loser’s remaining wait is distributed as a fresh clock by Lemma 3.2; the state after each block is therefore probabilistically identical to the start, independent of the past. The claim follows by induction. $\square$

Proposition 3.4 is the paper’s bridge (its §3, with footnote 1 supplying the clock rates) from physical time to combinatorics: since the question “does the attacker ever get ahead?” concerns only the *order* of block discoveries, not their times, every result below is a statement about a p/q coin sequence, and the time constant T_0 disappears from the answers. Where the exponential model itself comes from — and how well Zcash’s Equihash fits it — is a deployment question, taken up in Section 8.3.

3.2 The negative binomial law

Proposition 3.5 (Progress of the loser). *In an attribution sequence with attacker probability $q = 1 - p$, let m be the number of attacker blocks found strictly before the honest network’s n -th block. Then*

$$P(m) = \binom{m+n-1}{m} p^n q^m, \quad m = 0, 1, 2, \dots$$

— the negative binomial distribution.

Proof. The event fixes the first $m+n$ trials exactly this far: the $(m+n)$ -th trial is honest (the n -th honest success), and among the first $m+n-1$ trials exactly m are the attacker’s, in any of $\binom{m+n-1}{m}$ orders. Each such order has probability $p^{n-1} q^m \cdot p$. $\square$

Lemma 3.6 (First-to- n race). *In the same sequence, let H_n be the event that the honest network reaches n blocks before the attacker does, and A_n the reverse. Then H_n and A_n partition the sample space, and*

$$\Pr[H_n] = \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^n q^m, \quad \Pr[A_n] = \sum_{h=0}^{n-1} \binom{h+n-1}{h} q^n p^h, \quad \Pr[H_n] + \Pr[A_n] = 1.$$

Proof. Among the first $2n-1$ trials one side already has n successes, so the race terminates; a tie is impossible because block counts advance one at a time. The event H_n says the n -th honest block arrives while the attacker holds $m \leq n-1$; summing Proposition 3.5 over those m gives the first sum, and A_n is the same computation with the roles of p and q swapped. $\square$

Remark 3.7 (From hash trials to exponential clocks). The exponential model is itself a limit, not an axiom. A miner’s work is a stream of candidate evaluations, each an independent Bernoulli trial with a tiny success probability; the number of trials to the first success is geometric, and a geometric law with small success probability, viewed at the scale of its mean, converges to the exponential of Definition 3.1. The step that matters is *progress-freeness*: a candidate that fails must carry no information usable by the next one. Whether Zcash’s Equihash satisfies this is examined with the deployed parameters in Section 8.3.

4 Playing catch-up

The attacker of Section 2.2 finds himself, at the moment the merchant ships, some number of blocks behind the public chain, and wins if his branch ever overtakes it. This section computes the probability of that event as a function of the deficit; the next section supplies the distribution of the deficit itself.

Both computations run under the paper's two standing assumptions, stated here nearly verbatim (the environment form is ours, Remark 1.1):

Assumption 4.1 (Constant hashrates). The total hashrate of the honest network and the attacker is constant. Combined they have a hashrate of H , of which pH belongs to the honest network and qH to the attacker, where $p + q = 1$.

Assumption 4.2 (Constant difficulty). The mining difficulty is constant, and such that with hashrate H the average time to find a block is T_0 .

Under Assumption 4.2 every block carries equal work, so “most work” and “longest” coincide and the race can be scored in block counts. How far the deployed chain sits from these assumptions — difficulty adjusts after every block, hashrate drifts — is quantified in Section 8.2; neither gap changes the shape of the conclusions.

Definition 4.3 (The deficit walk). Let z denote the honest chain's lead in blocks over the attacker's branch. By Proposition 3.4, each newly found block moves z up by 1 with probability p (honest block) or down by 1 with probability q (attacker block), independently of the past:

$$z_{i+1} = \begin{cases} z_i + 1 & \text{with probability } p, \\ z_i - 1 & \text{with probability } q. \end{cases}$$

The attack succeeds the moment z reaches -1 : the secret branch is then strictly ahead, and releasing it rewrites history.

Theorem 4.4 (Catch-up probability; Rosenfeld §3). Let a_z be the probability that the walk of Definition 4.3, started at deficit z , ever reaches -1 . Then

$$a_z = \min(q/p, 1)^{\max(z+1, 0)} = \begin{cases} 1 & \text{if } z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & \text{if } z \geq 0 \text{ and } q < p. \end{cases} \quad (1)$$

Proof. For $z < 0$ the walk has already succeeded, so $a_z = 1$; assume $z \geq 0$. Fix an integer $N > z$ and absorb the walk at both -1 and N ; let $h_N(z)$ be the probability of reaching -1 before N . Conditioning on the first step,

$$h_N(z) = p h_N(z+1) + q h_N(z-1) \quad (0 \leq z \leq N-1), \quad h_N(-1) = 1, \quad h_N(N) = 0,$$

a second-order linear recurrence. The trial solution $h_N(z) = t^z$ satisfies it exactly when t is a root of the characteristic polynomial $pt^2 - t + q = p(t-1)(t-q/p)$.

If $q \neq p$ the roots 1 and q/p are distinct, so $h_N(z) = A + B(q/p)^z$; the boundary conditions give

$$h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N}.$$

If $q = p$ the root is double and $h_N(z) = A + Bz$, giving $h_N(z) = (N - z)/(N + 1)$.

Now let $N \rightarrow \infty$. The events $E_N = \{\text{reach } -1 \text{ before } N\}$ are nondecreasing in N , and their union is the event that the walk ever reaches -1 : a walk that does so does it at a finite time, having visited finitely many states, so E_N holds for every N above the largest state visited. By continuity along monotone events (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”), $a_z = \lim_{N \rightarrow \infty} h_N(z)$. For $q < p$ the term $(q/p)^N$ vanishes and the quotient tends to $(q/p)^{z+1}$; for $q = p$, $(N - z)/(N + 1) \rightarrow 1$; for $q > p$, dividing numerator and denominator by $(q/p)^N$ sends both to -1 , so the quotient tends to 1. $\square$

Corollary 4.5 (Majority always catches up). *When $q \geq p$, the attacker overtakes the public chain with probability 1 from any finite deficit. No confirmation count defends against a majority of the hashrate.*

Corollary 4.6 (Geometric decay in the deficit). *When $q < p$, each additional block of deficit multiplies the catch-up probability by q/p . Concretely (script-computed): at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.*

Corollary 4.7 (Blocks, not time). *Equation (1) contains neither T_0 nor any other time scale: the security of a deficit is a function of block counts alone. Two chains with different block spacings but the same q offer identical security per confirmation — and therefore different security per minute (Section 6.1).*

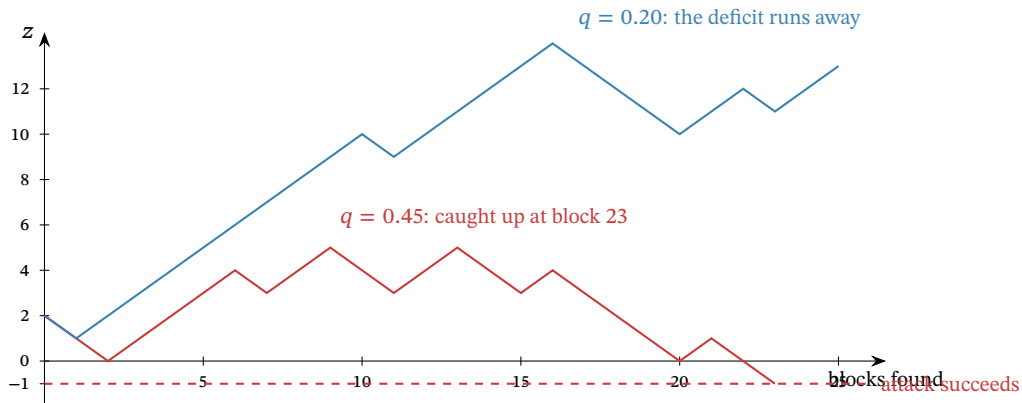


Figure 2: Two runs of the deficit walk from $z = 2$, simulated by script with fixed seeds. At $q = 0.45$ the drift is weak and this run reaches -1 after 23 blocks — the attack succeeds; at $q = 0.20$ the deficit grows roughly linearly, and by Theorem 4.4 the chance of ever returning from deficit 13 is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

5 Waiting for confirmations

The merchant’s defence is patience: ship only after the paying transaction has n confirmations (Definition 2.3). This section computes exactly how much that patience buys. The paper’s model of the attack timeline has one convention worth surfacing before the theorem.

Remark 5.1 (The pre-mined block). The paper assumes the attacker banks one block before the race is scored: “we assume one block was pre-mined by the attacker before commencing the attack” (§4). The attacker forks from the public leaf at the moment the merchant’s transaction is broadcast, and a patient attacker times the broadcast to coincide with a private block he has just found but not announced. At the moment the honest chain reaches n confirmations, the attacker who has found m further blocks therefore holds $m + 1$, and the deficit walk of Definition 4.3 starts at

$$z = n - m - 1.$$

The convention is a modelling choice, not a law: an attacker with no banked block starts at $z = n - m$, and every probability below shrinks accordingly. It is also, as Theorem 5.2 shows, precisely the choice that makes the final formula collapse into a symmetric race.

By Proposition 3.5, the number m of attacker blocks found while the honest network finds its n is negative binomial. Averaging the catch-up probability of Theorem 4.4 over m gives the central result.

Theorem 5.2 (Double-spend success probability; Rosenfeld eq. (1)). *Let the merchant wait for $n \geq 1$ confirmations, under Assumptions 4.1 and 4.2 and the convention of Remark 5.1. The probability that the attacker’s branch eventually overtakes the public chain is $r(q, n) = 1$ if $q \geq p$, and otherwise*

$$r(q, n) = 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) = 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n. \quad (2)$$

Proof. Conditioning on m and applying Theorem 4.4 at $z = n - m - 1$,

$$r = \sum_{m=0}^{\infty} P(m) a_{n-m-1} = \underbrace{\sum_{m=0}^{n-1} P(m) \left(\frac{q}{p}\right)^{n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m=n}^{\infty} P(m)}_{\text{already ahead}},$$

valid for $q < p$; when $q \geq p$ every $a_{n-m-1} = 1$ and $r = \sum_m P(m) = 1$ immediately. For the first sum, each term transforms exactly:

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} = \binom{m+n-1}{m} p^m q^n,$$

which by Lemma 3.6 is the probability that the attacker’s n -th block arrives while the honest network holds m ; summed over $m \leq n - 1$, the first sum is $\Pr[A_n]$, the probability that the attacker wins a fair first-to- n race. The second sum is $\Pr[m \geq n] = 1 - \Pr[H_n] = \Pr[A_n]$ by the same lemma. Hence $r = 2\Pr[A_n]$, the right-hand form of (2); the middle form follows from $\Pr[A_n] = 1 - \Pr[H_n]$ applied to one of the two copies. The printed upper limit in the paper’s equation (1) is n rather than $n - 1$; the $m = n$ term is identically zero, so the forms agree. $\square$

Remark 5.3 (The race doubling). The proof exposes a structure the paper’s algebra passes through silently: under the one-pre-mined-block convention, *the probability of falling behind and later catching up equals, exactly, the probability of never falling behind at all*, so the double-spend probability is twice the chance of winning a first-to- n race. The identity also absorbs the boundary case: at $q = p$ each race half is exactly $\frac{1}{2}$, giving $r = 1$ with no case split, continuously with the $q > p$ regime. The algebra here is the paper’s; reading its two sums as the two halves of one race, and the observation that the pre-mine convention is what makes them equal, are this volume’s presentational additions (verified by script over random (q, n) ; the two sums agree to floating-point precision).

Example 5.4 (A full race, exactly). Take $q = 1/5$ (a twenty-percent attacker) and $n = 6$. The deficit starts at $z = 5 - m$, the catch-up factor is $(q/p)^{6-m} = 4^{-(6-m)}$, and every quantity below is an exact rational, printed to six decimals (script-computed):

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half (sum of products)			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(0.2, 6)$			0.023308

The two halves agree to every printed digit — they are equal exactly, by Remark 5.3 — and the closed form (2) evaluates to the same 0.023308. A merchant facing a possible twenty-percent adversary who ships after six confirmations is running a 2.33% risk.

Remark 5.5 (Satoshi’s approximation). The whitepaper’s §11 computes the same quantity with one approximation and two offsetting conventions: the attacker’s progress is approximated as Poisson with mean $\lambda = nq/p$ (the law $\Pr[k] = e^{-\lambda} \lambda^k / k!$ on the nonnegative integers); no block is pre-mined, but success is scored at *breakeven* rather than strictly ahead, and those two conventions cancel exactly — the whitepaper’s catch-up factor for m attacker blocks is $(q/p)^{n-m}$, term for term the factor of Remark 5.1. The Poisson approximation is the entire gap, and it pushes the estimate down. Script-computed comparisons: at $q = 10\%$, $n = 6$, the whitepaper’s formula gives 0.0243% against the exact 0.0591% — an understatement by a factor of about 2.4; at $q = 30\%$, $n = 6$: 13.211% against 15.645%; at $q = 10\%$, $n = 10$: 0.0001% against 0.0008%. The direction is systematic, which is why the negative binomial model — exact under the stated assumptions — replaced it.

q	$n=1$	2	3	4	5	6	7	8	9	10
2%	4.000	0.237	0.016	0.0011	$8 \cdot 10^{-5}$			$< 10^{-5}$		
5%	10.000	1.450	0.232	0.039	0.0066	0.0012	$2.1 \cdot 10^{-4}$	$4 \cdot 10^{-5}$	$< 10^{-5}$	
10%	20.000	5.600	1.712	0.546	0.178	0.059	0.020	0.0067	0.0023	0.0008
20%	40.000	20.800	11.584	6.669	3.916	2.331	1.401	0.848	0.516	0.316
30%	60.000	43.200	32.616	25.207	19.762	15.645	12.475	10.003	8.055	6.511
40%	80.000	70.400	63.488	57.958	53.314	49.300	45.769	42.621	39.787	37.218
45%	90.000	85.050	81.375	78.342	75.716	73.375	71.252	69.299	67.487	65.793
48%	96.000	94.003	92.508	91.264	90.177	89.201	88.307	87.478	86.703	85.972
50%					100 for every n					

Table 2: The double-spend success probability $r(q, n)$, in percent, computed by script from Theorem 5.2. On the rows the paper’s Table 1 also prints ($q \in \{2, 10, 20, 30, 40, 48, 50\}\%$), the script reproduces its values cell-for-cell; the 5% and 45% rows are this volume’s additions.

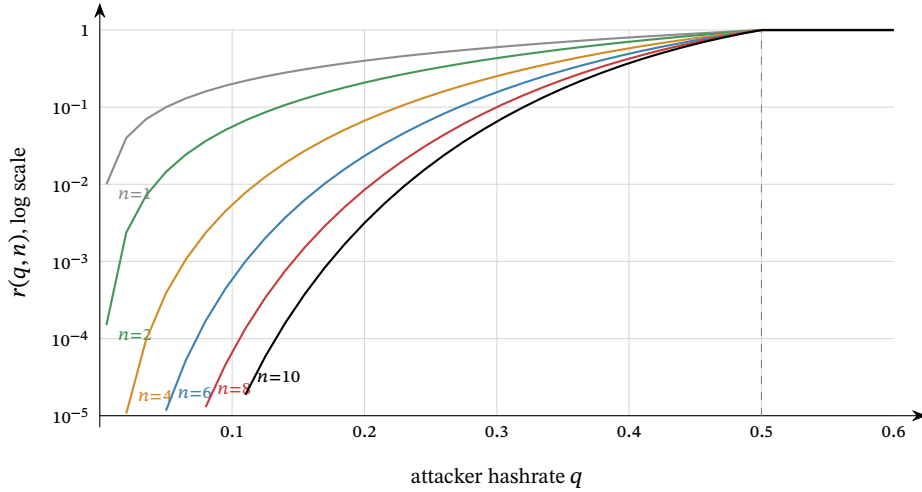


Figure 3: The success probability $r(q, n)$ against the attacker's hashrate share q , for $n \in \{1, 2, 4, 6, 8, 10\}$ confirmations, logarithmic vertical scale clipped at 10^{-5} ; coordinates computed by script from Theorem 5.2. Every curve reaches 1 at $q = \frac{1}{2}$ (dashed) and stays there: beyond the majority line, confirmations are worthless. Below it, each added confirmation drops r by a roughly constant factor — the curves are near-equally spaced in log scale — and the factor improves as q falls.

6 What the numbers say

Theorem 5.2 compresses the security of Nakamoto consensus into one two-parameter function, and its qualitative lessons are all visible in Table 2 and Figure 3. This section states them precisely, then instantiates the one lesson that is specific to Zcash.

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
2%	1	2	3
5%	2	3	4
8%	2	3	5
10%	2	4	6
15%	3	6	9
20%	4	8	13
25%	5	12	20
30%	9	20	32
35%	15	36	58
40%	34	82	133
45%	135	331	539

Table 3: The least number of confirmations pushing $r(q, n)$ below three targets, computed by script. The growth is logarithmic in the target but explosive in q : a 10% attacker is answered by 6 confirmations, a 45% attacker by 539 — more than eleven hours of Zcash blocks — and at 50% by no number at all.

Proposition 6.1 (No finite count is final). *For every $0 < q < p$ and every n , $r(q, n) \geq 2q^n > 0$.*

Proof. By Theorem 5.2, $r = 2\Pr[A_n]$, and one way for the attacker to win the first-to- n race is to find the first n blocks outright, an event of probability q^n . $\square$

Proposition 6.2 (Divergence at the majority line). *Fix a target $\varepsilon < 1$ and let $n_\varepsilon(q)$ be the least n with $r(q, n) < \varepsilon$. Then $n_\varepsilon(q) \rightarrow \infty$ as $q \uparrow \frac{1}{2}$.*

Proof. For fixed n the function $q \mapsto r(q, n)$ is a polynomial on $[0, \frac{1}{2}]$, hence continuous, and $r(\frac{1}{2}, n) = 1$: at $q = p$ each half of the race of Remark 5.3 has probability exactly $\frac{1}{2}$ by Lemma 3.6 and symmetry. So for every n there is a $\delta_n > 0$ with $r(q, n) > \varepsilon$ whenever $q > \frac{1}{2} - \delta_n$; no fixed n survives arbitrarily close to the line. $\square$

Remark 6.3 (Nothing is special about six). The paper (§5): “There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary” — six confirmations are “overkill for casual attackers, and at the same time powerless against more dedicated attackers”. Table 3 is the honest replacement: pick the adversary you defend against and the risk you accept, and read off n ; the pair (q, ε) is a policy, not a law of nature.

6.1 Zcash time: 75-second blocks

Corollary 4.7 says security is purchased in blocks, so the wall-clock price of a confirmation count is set by the target spacing — for Zcash, 75 seconds since the Blossom upgrade (ZIP-208; PostBlossomPoWTargetSpacing, specification §5.3), against Bitcoin’s 600. Thirty minutes of waiting therefore buys 3 Bitcoin confirmations but 24 Zcash confirmations, and against a 10% adversary the difference is $r = 1.712\%$ against $r \approx 3.2 \times 10^{-12}$ (script-computed): nine orders of magnitude, for the same patience. ZIP-208 makes precisely this argument for the spacing change — “the number of confirmations needed increases more slowly than the decrease in block time”.

The comparison holds q fixed, and that is its honest limit: the model contains no network, so nothing in it prices the disadvantage faster blocks impose on the *honest* side, whose blocks are found closer together and are therefore likelier to race one another and self-orphan. At 75 seconds against propagation times of fractions of a second the effect is small, but it is a modelling gap, recorded next.

Remark 6.4 (Where the model bends). Three assumptions do not survive contact with the deployed chain, and none of them changes the shape of the results. (i) *Difficulty is not constant*: Zcash retunes the target after every block (Section 8.2); over an attack lasting tens of blocks the drift is small, and the race is scored in work, not block counts, so the analysis carries over with q read as the attacker’s share of *work* production — up to a granularity caveat quantified in Section 8.2. (ii) *Hashrate is not constant*: q in the theorems is whatever share the attacker sustains for the attack’s duration; the paper’s own caveat is that T_0 re-enters only if the attacker cannot sustain his hashrate long enough, which it judges unlikely for a serious attacker. (iii) *Propagation is not instantaneous*: honest self-orphaning wastes a sliver of honest work, slightly raising the attacker’s effective share; the deployed spacing keeps the sliver thin. A fourth gap is not the model’s: the attacker of this volume follows the protocol’s validity rules perfectly and attacks only the *choice* between valid histories. Attacks on the rules themselves are other volumes’ subjects.

7 The economics of the attack

The probabilities of Section 5 answer the adversarial merchant, who assumes the customer is an attacker. The paper’s §6 answers the realistic one: when is the attack worth mounting at all? Throughout this section $q < p$ — “[o]therwise, all bets are off” (§6) — and the model is deliberately stylised; its own caveats close the section.

The attacker targets k merchants simultaneously with payments of v each, all invalidated by the same secret branch; the goods are worth αv to him, $0 < \alpha \leq 1$; he gives up after mining o blocks in

vain; and each of his blocks would earn the block income B if it ended up in the accepted chain. If the attack succeeds his blocks are all accepted (he keeps goods *and* coins *and* rewards); if it fails he has paid kv , keeps goods worth $k\alpha v$, and his o orphaned blocks forfeit oB .

Proposition 7.1 (Profitability; Rosenfeld §6). *The attacker’s expected profit over not attacking is*

$$k\alpha v - (1 - r)(kv + oB) = kv(\alpha + r - 1) - (1 - r)oB,$$

positive if and only if

$$v > \frac{(1 - r)oB}{k(\alpha + r - 1)}$$

(when $\alpha + r > 1$; when $\alpha + r \leq 1$ the attack never profits). A merchant is therefore safe whenever

$$v \leq \frac{o(1 - r)B}{k(\alpha + r - 1)} \stackrel{\alpha=1}{=} \frac{o}{k} \cdot \frac{B(1 - r)}{r}, \quad (3)$$

which with the paper’s choices $o = 20$, $k = 5$, $\alpha = 1$, $B = 25$ BTC is its equation (2), $v \leq 100(1/r - 1)$ BTC.

Proof. The gain $k\alpha v$ is certain (the goods are obtained either way); the loss $kv + oB$ is paid exactly when the attack fails, with probability $1 - r$. Rearranging the positivity condition gives the threshold; substituting the paper’s constants gives $20 \cdot 25 \cdot (1 - r)/(5r) = 100(1 - r)/r$. $\square$

7.1 Instantiation in ZEC

The block income that an orphaned block forfeits is what the miner would have collected: the miner subsidy plus fees. At the current mainnet height the block subsidy is 1.5625 ZEC (156,250,000 zatoshi, specification §7.8; zebra-chain/src/parameters/network/subsidy.rs:447), of which the two active funding streams take 20% — 8% to the Zcash Community Grants stream and 12% to the coinholder-controlled fund, both running to the third halving at height 4,406,400 (specification §7.10.1; ZIP-1016; subsidy.rs:338) — leaving the miner

$$B = 1.25 \text{ ZEC} + \text{fees}$$

per block (miner_subsidy, subsidy.rs:480). The funding-stream portion is not the attacker’s under either outcome, so $B = 1.25$ ZEC is the right forfeiture figure (fees, small on today’s chain, only raise it). Table 4 evaluates bound (3) with the paper’s $o = 20$, $k = 5$, $\alpha = 1$.

q	$n=1$	2	4	6	8	10
5%	45	339	12,909	430,929	13,664,382	420,922,251
10%	20	84	911	8,449	74,344	636,146
20%	7	19	69	209	584	1,578
30%	3	6	14	26	44	71
40%	1	2	3	5	6	8

Table 4: The maximal safe transaction value in ZEC, $\lfloor 4B(1 - r)/r \rfloor$ with $B = 1.25$ ZEC, computed by script; the analogue of the paper’s Table 2, which used $B = 25$ BTC. Values are floored and inherit every assumption of the model; the caveats of Remark 7.3 apply in full.

The table’s message survives its assumptions. Against small attackers a handful of confirmations protects any plausible payment; against a 30% attacker, six confirmations protect only a payment of about 26 ZEC, and a merchant accepting more should wait longer — the required count grows only logarithmically in the value at stake, by the geometric decay of Theorem 5.2.

Remark 7.2 (Majority is a different problem). When $q \geq p$ the economics change character entirely: the paper notes a majority attacker can double-spend at no cost beyond normal mining, reject every other miner’s blocks to take the whole issuance, and exclude transactions at will, driving honest miners out and entrenching himself — so a majority attack “is best seen as an attempt to destroy” the currency, motivated from outside it (a short position, a competing system), not an attempt to profit inside it. Confirmation policy is not the defence at that point; the defence is the cost of assembling the majority, which lies outside this model.

Remark 7.3 (The model’s own caveats). The paper flags each stylisation itself. The give-up point $o = 20$ “is a significant simplification” (an optimal attacker balances completion against compounding losses); the safe values “should be taken with a grain of salt, because of the many modeling assumptions”; and a model resting on mining *costs* rather than forfeited *rewards* would make the required confirmations linear, not logarithmic, in the transaction value — “very poor security, hence in a situation where this is relevant, we have already lost anyway” (§6). The bound’s role is the shape it reveals — logarithmic patience buys exponential safety — not its third significant digit.

8 The deployed rule and its safety rails

The model of Sections 4–7 is clean; the deployment is not. This section walks the distance between them: how the best chain is actually chosen, how difficulty actually moves, how well Equihash fits the memoryless model, and the node-local rails — reorg windows, maturity, wallet policy — that bound the theory’s tail risks in practice.

8.1 Chain selection in code

In zebra, candidate chains live in the non-finalized state as a set ordered by `impl Ord` for `Chain` (zebra-state/src/service/non_finalized_state/chain.rs:2623--2696); the best chain is the set’s maximum. The ordering’s doc comment is exact about its two criteria: “Chains with higher cumulative Proof of Work are [`Ordering::Greater`], breaking ties using the tip block hash.” Cumulative work is the sum of Definition 2.1 over the chain’s blocks (chain.rs:1803--1809).

Remark 8.1 (The tie-break nobody implements). The specification’s tie-break is first-seen (Definition 2.2), and it is also Rosenfeld’s (“the one of which the node learnt first”). The retired `zcashd` implemented it: its comparator sorted by `nChainWork`, then by `nSequenceId` — “[s]equential id assigned to distinguish order in which blocks are received” (src/main.cpp:194--213, src/chain.h:363--364) — so at equal work the incumbent tip stayed. Zebra does not: blocks are downloaded in parallel and arrival times are not tracked, so it breaks work ties by the larger tip hash, and its own doc comment calls this a “departure from the consensus rules” that “may delay network convergence, for as long as the greater hash belongs to the later mined block” (chain.rs:2629--2636). With `zcashd` retired, the specification’s first-seen rule is implemented by no running validator. For this volume’s analysis the divergence is immaterial: ties are transient states broken by the next block (Figure 1), and no theorem above depends on how a node sits out a tie.

8.2 Difficulty in motion

Assumption 4.2 freezes difficulty; Zcash does not. The specification (§7.7.3) adopts “a difficulty adjustment algorithm based on DigiShield v3/v4”, and — “[u]nlike Bitcoin, the difficulty adjustment occurs after every block”. Each block’s target is recomputed from the mean target of the last `PoWAveragingWindow = 17` blocks and the actual timespan of that window measured between medians of `PoWMedianBlockSpan = 11` timestamps; the timespan is dampened towards nominal by a factor of `PoWDampingFactor = 4` and clamped to `[|AWT(1 - 16%)|, |AWT(1 + 32%)|]` (`PoWMaxAdjustUp = 16%`, `PoWMaxAdjustDown = 32%`, `AWT = 17 × 75 s`), with the target capped

at $\text{PoWLimit} = 2^{243} - 1$ (all constants: specification §5.3; `zebra-state/src/service/check/difficulty.rs`).

Remark 8.2 (Why the race survives per-block retuning). Three observations keep Theorem 5.2 meaningful on the deployed chain. First, the adjustment is slow relative to an attack: its window is 17 blocks (≈ 21 minutes) and per-step movement is dampened fourfold and clamped, so over the tens of blocks a typical race lasts, both branches mine at nearly the difficulty prevailing at the fork unless a side works to move its own. Secondly, the secret branch computes its own schedule: the adjustment “use[s] only information from blocks preceding the given block height” (§7.7.3), so the attacker’s chain retunes from the attacker’s own timestamps, subject to the timestamp rules — $n\text{Time}$ strictly above the median of the preceding 11, at most 90×60 seconds above it (§7.6), and at most two hours ahead of a validator’s clock on receipt (a non-consensus check). Thirdly — and here the model genuinely bends — the comparison is by summed *work*, and work per block moves inversely with the target (Definition 2.1). An attacker who stretches his timestamps lowers his own difficulty and mints more, lighter blocks from the same expected hashing; compressing them does the opposite. The exchange preserves his expected work *production*, not his odds: a first-passage race depends on the granularity of its steps as well as their drift, and the same expected work packed into fewer, heavier blocks raises an underdog’s chance of ever drawing strictly ahead — lumpy progress favours the trailing side. Script-simulated at $q = 0.3$ from a three-work-unit deficit: with equal blocks the overtake probability is 0.034 (matching the analytic $(q/p)^4$), with double-work blocks 0.10, with quadruple-work blocks 0.22. Stretched timestamps therefore *hurt* the attacker; the profitable direction is compression — raising his own difficulty within the timestamp rules — and that advantage is real, is not priced by Assumption 4.2, and is bounded by the damped, clamped per-block adjustment over a race’s length. The tables of this volume are the equal-weight baseline, and the reading of q as the attacker’s share of work production holds while both branches’ per-block work stays close — which the first observation grants at the start of a race, and the clamp enforces over its ordinary length.

8.3 Equihash and the memoryless model

Zcash’s proof of work is Equihash with parameters $n = 200$, $k = 9$ (specification §7.7, §7.7.1; the algorithm is Biryukov and Khovratovich’s, ePrint 2015/946), but the quantity the race counts is gated one step later: “[t]he difficulty filter is unchanged from Bitcoin, and is calculated using SHA-256d on the whole block header (including `solutionSize` and `solution`)”, required to be at most $\text{ToTarget}(n\text{Bits})$ (§7.7.2). Each candidate Equihash solution therefore faces an independent, fresh SHA-256d lottery, exactly the Bernoulli trial of Remark 3.7.

The specification nowhere asserts progress-freeness — the argument is this volume’s, not a citation. What Equihash changes is the granularity: trials arrive in batches, one batch per memory-hard solver run, each run yielding a Poisson-distributed handful of candidate solutions. Progress within a run is discarded when a competing block arrives, so the memoryless approximation is exact between runs and coarse within one; the approximation is good exactly because a solver run (seconds) is short against the 75-second spacing. A proof-of-work whose solve time were comparable to the spacing would make block discovery visibly non-exponential and pull the attribution sequence away from Proposition 3.4; Zcash’s parameters keep the gap wide.

8.4 Reorg rails, maturity, and the finality floor

Nothing in consensus caps how deep a reorganisation may reach — by Proposition 6.1 no depth is truly final — but both validator implementations bolted rails onto the theory, and the specification acknowledges them: “A full validator MAY impose a limit on the number of blocks it will ‘roll back’ when switching from one best valid block chain to another that is not a descendent. For `zcashd` and `zebra` this limit is 100 blocks” (§3.3). Neither half of that sentence matches deployment.

The retired `zcashd` enforced `MAX_REORG_LENGTH = COINBASE_MATURITY - 1 = 99` (`src/main.h:66`): a would-be reorganisation deeper than 99 blocks did not fail over to the

heavier chain — the node refused and halted, logging “[a] block chain reorganization has been detected that would roll back %d blocks! This is larger than the maximum of %d blocks, and so the node is shutting down for your safety” (src/main.cpp:4331--4350).

The deployed zebra keeps a rolling window of `MAX_BLOCK_REORG_HEIGHT = 1000` blocks (zebra-chain/src/parameters/constants.rs:30): while the best chain exceeds that length its root blocks are committed to the finalized state (zebra-state/src/service/write.rs:455--467), and any later block at or below the finalized tip is rejected as an `OrphanedBlock` (zebra-state/src/service/check.rs:224--238) — a fork rooted below the window cannot be followed at all. The constant’s own documentation is careful about its standing: “[t]his is a local-only node policy; it is not part of consensus. The window is sized as a defence-in-depth measure against sustained consensus splits.”

Remark 8.3 (A three-way divergence, and what the rails buy). The specification says 100 for both validators; zcashd enforced 99; zebra enforces 1000 — and flags the gap itself, quoting the specification’s 100 next to its own constant (zebra-state/src/request.rs:819--827). ZIP-307’s security model inherits the stale figure (“no full Zcash node will roll back the chain by more than 100 blocks”), which no longer describes the sole running validator; the *Sync Guide* tracks that thread (§“Reorganisation detection and recovery”). For this volume’s subject the rails change nothing a merchant sees: every confirmation count in Tables 2 and 3 sits far inside a 1000-block window (≈ 21 hours), so $r(q, n)$ governs commerce exactly as derived. What the rails cap is the catastrophe tail — a secret branch deeper than the window meets a wall of nodes that will not follow it automatically, converting a consensus rewrite into a manual-intervention event. The specification adds one more floor, socially anchored: a full validator that risks Mainnet funds or displays transaction information “MUST do so only for a block chain that includes the activation block of the most recent settled network upgrade” (§3.3) — history behind a settled upgrade’s activation is not up for probabilistic debate at all.

The protocol itself hard-codes one confirmation count. A *transparent* output is the Bitcoin-inherited kind, its value and destination in cleartext on chain. “A transaction MUST NOT spend a transparent output of a coinbase transaction from a block less than 100 blocks prior to the spend” (§7.1.2; `MIN_TRANSPARENT_COINBASE_MATURITY = 100`, zebra-chain/src/transparent.rs:54) — miners wait $n = 100$ on their own income. Read through Theorem 5.2 (script-computed): $r(0.3, 100) \approx 3.7 \times 10^{-9}$ and $r(0.4, 100) \approx 0.43\%$, but $r(0.45, 100) \approx 15.7\%$ — even the deepest confirmation rule in the protocol is strong at moderate q and porous near the majority line, as Proposition 6.2 says every fixed count must be. Since the Heartwood network upgrade (ZIP-213) a coinbase may shield its outputs, and the maturity rule binds only the transparent ones.

8.5 What wallets actually wait for

Deployed confirmation policy lives in `librustzcash`. `Structure ConfirmationsPolicy` defaults to 3 confirmations for *trusted* outputs (change, and notes from senders the wallet trusts) and 10 for *untrusted* ones, with zero-confirmation shielding of transparent funds allowed (`zcash_client_backend/src/data_api/wallet.rs:444--455`, citing the draft ZIP-315); the spend anchor — the note-commitment-tree root against which a spend proves its note exists (*Crypto Guide*, §“Privacy-preserving membership via zero-knowledge paths”) — sits 3 blocks below the target height (`wallet.rs:574--576`). The Android SDK passes exactly these defaults for transfers, and for shielding the minimum policy, under which the transparent funds being shielded need no confirmations at all (`backend-lib/src/main/rust/lib.rs:2096, :2158, :2192`); its UI reports a transaction *confirmed* at 10 confirmations (`TransactionOverview.kt:89`). The policy layer above these constants is the *Wallet Guide*’s subject (§“Confirmations, trust, and spendability”).

Table 2 prices the defaults. Three confirmations hold a 10% adversary to $r = 1.712\%$ — acceptable for outputs the wallet already trusts — while ten hold the same adversary to 0.0008% and a 20% adversary to 0.316%. In the model’s terms, the trusted/untrusted split is a bet that senders one transacts with repeatedly are not mounting hashrate attacks, and the 10-row is the posture towards everyone else. Both rows, like every fixed count, are a (q, ϵ) policy in the sense of Remark 6.3, not a guarantee.

9 Relation to the rest of the series

This volume opens the deployed-protocol group of the series — everything above it takes the ledger constructed here for granted — and three threads run outward from it.

Downward-facing trust. A light client never evaluates Definition 2.2 itself: it outsources the chain view to a server and trusts “that proxy to provide a correct view of the best chain” (*Sync Guide*, §“Architecture and authority”). Every confirmation count such a client displays is counted *inside* the served view; the probabilities of this volume apply to it only insofar as the view is honest. Succinctly verifying that a served view really is the most-work chain — without downloading it — is exactly the problem the *FlyClient Guide* treats (§“The problem: verifying a chain without downloading it”), and its starting assumption is this volume’s conclusion: that the most-work chain is the one the network converges on.

The gap finality closes. Proposition 6.2 is the honest limit of Nakamoto consensus: as the adversary approaches half the work, no confirmation count survives. Crosslink’s answer is to stop paying for the tail probabilistically — a vote among a fixed validator committee, safe while fewer than a third of it misbehaves, checkpoints the proof-of-work chain, and “final” becomes an assured status rather than a shrinking r (*Crosslink Guide*, §“The ebb-and-flow model”, §“Finality as a status”). The present volume supplies the curve that design is buying its way off of; the 1000-block window of Remark 8.3 is the node-policy stopgap standing where assured finality would go.

What this volume owes downward. Exactly one debt: every theorem above stands on the *Math Guide*’s probability section — the definition of probability, its extension past finite spaces, and the continuity theorem the catch-up proof invokes. Upward it owes nothing: the citations to Ironwood, Wallet, Sync, FlyClient, and Crosslink are delimitations and pointers, not dependencies. The ledger the rest of the series builds on is, from here up, a probabilistic object with known constants — and the constants are Tables 2, 3, and 4.

Part VI

Ironwood Guide: The Zcash Orchard protocol and its Ironwood pool

Abstract

Assuming the *Math Guide*, the *Crypto Guide*, and the *Halo 2 Guide*, this volume of *The Zcash Arboretum* develops Zcash’s Orchard shielded-payment protocol: the key hierarchy, notes and note commitments, the commitment tree and the procedure by which a wallet obtains a membership path, nullifiers, note encryption and trial decryption, value commitments and the RedPallas signature, the formal Action statement the SNARK proves, and the end-to-end security analysis that derives balance and nullifier uniqueness from knowledge soundness, and privacy from zero knowledge. NU6.3 splits the protocol’s value across two pools — the legacy Orchard pool, sealed to spend-only, and the current Ironwood pool — and the volume develops the split, the quantum-recoverable Ironwood note format, and the cross-address restriction, closing with the Zcash proof-system lineage.

1 Introduction

This is the *Ironwood Guide*, the volume of *The Zcash Arboretum* that develops the cryptography behind Zcash’s Orchard protocol as deployed. It assumes three earlier volumes: the *Math Guide* (the underlying mathematics), the *Crypto Guide* (hash functions, commitments, signatures, key agreement, zero knowledge, and the remainder of the primitive toolbox), and the *Halo 2 Guide* (the general-purpose proof system). This volume assembles those tools into the Orchard protocol as carried by its current pool, *Ironwood*. ZIP-229 fixes the vocabulary up front: the *Orchard protocol* is the shared cryptographic construction — curves, Sinsemilla, the Action circuit, notes, commitments, nullifiers, keys, and note encryption — and it survives; the *Orchard pool* is the legacy value pool it ran from NU5, sealed to spend-only at NU6.3; the *Ironwood pool* is the current pool.

We develop the protocol from the problems it solves. After we fix the curve-level preliminaries and the Sinsemilla hash, we develop each of the following as the answer to one sub-problem: how a unit of value is represented (notes and note commitments); how an owner proves ownership of a real, unspent note without revealing which one (the commitment tree and a membership path); how the protocol prevents double-spending (nullifiers); how a sender privately delivers and a recipient detects a note (note encryption and trial decryption); how value is conserved without disclosure (value commitments and the binding signature); and how an owner authorises a spend (RedPallas). We then show the procedure by which a node verifies a payment without ever seeing the note, state the Action relation formally, trace the end-to-end security reductions, and close the volume with the Zcash proof-system lineage from Sprout through Sapling to Orchard, and within Orchard the three circuit versions ending at Ironwood.

2 The Orchard application: the relation proved by the SNARK

Halo 2 is a general-purpose proving system; it has no notion of money. The Orchard application converts it into a shielded-payment protocol by fixing one specific relation — the *Action statement* — that every transaction must prove. This section develops that relation from the problem it solves rather than from the symbols it uses. It begins from the question of what a private payment must accomplish, derives each cryptographic object as the answer to one sub-problem, then traces a complete payment from Alice to Bob end to end, and only then states the Action relation formally.

2.1 The problem and the four requirements on a shielded payment

A transparent ledger (Bitcoin, Zcash’s transparent pool) records, for every unspent output, its owner and its denomination (the number of units it holds), and a payment publicly transfers an output from one owner to another. A *shielded* payment must achieve the same effects — the transfer of value, the prevention of double-spending, and the conservation of total supply — while disclosing none of them. A private payment system must accordingly satisfy four requirements simultaneously:

1. *Represent a unit of value* so that its owner, value, and very existence stay hidden, yet its owner can later prove possession of it. (Notes and *note commitments* solve this, §2.5.)
2. *Prove possession of a real, unspent note* without identifying which note. (The *commitment tree* together with a membership proof solves this, §2.6.)
3. *Prevent double-spending* without a public list of which notes an owner spent. (*Nullifiers* solve this, §2.7.)
4. *Prove that no one created value* without revealing any amount. (*Value commitments* and the *binding signature* solve this, §2.9.)

These four requirements constitute the core of the protocol, and they rest on shared machinery. To render the section self-contained, we fix the build order by dependency — nothing appears before we define it — and lay it out here as a map:

1. **Preliminaries** (§2.2): the curve-level primitives every later object invokes — the hash-to-curve GroupHash, the coordinate map Extract, the point encoding $\star$, and domain separators.
2. **Sinsemilla** (§2.3): the commitment/hash built on those primitives, which both the note commitment and the Merkle tree use.
3. **Keys** (§2.4): the key hierarchy, from which the protocol derives addresses, commitments, and nullifiers.
4. **The first three requirements in turn**: notes (§2.5), the commitment tree (§2.6), and nullifiers (§2.7).
5. **Transmission** (§2.8): how the sender delivers the new note in-band and the recipient detects it.
6. **The fourth requirement**: value commitments (§2.9); together with *RedPallas* (§2.10), the signature that authorises a spend.
7. **Assembly** (§2.11): a complete Alice-to-Bob payment, end to end.
8. **The Action statement** (§2.14): the formal relation, the conjunction of in-circuit checks that makes all four hold at once.

Each later object depends only on earlier ones; the dependency root is the preliminaries below.

2.2 Preliminaries: hashing and encoding on the curve

The protocol builds every Orchard object below from a small set of curve-level primitives. We define them once, here, before anything uses them: a hash-to-curve GroupHash, the coordinate map Extract, the point encoding $\star$, and the domain separators that keep distinct uses independent. All operate on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.

The coordinate map `Extract`. $\text{Extract} : \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ maps a curve point to its x -coordinate, $\text{Extract}(x, y) := x$ (with $\text{Extract}(\mathcal{O}) := 0$ by convention). It appears wherever a later object must be a *field element* rather than a curve point — because the Merkle tree, the nullifier set, and the proof’s public inputs are all field elements. It is not injective ($\pm(x, y)$ share an x), which is harmless here: the protocol only ever requires the x -coordinate as a binding digest, never to recover the point.

Point encoding: the $\star$ superscript. $P^\star \in \{0, 1\}^{256}$ denotes the canonical bit-encoding of a point $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$: its x -coordinate (an element of $\mathbb{F}_{p_{\text{Pallas}}}$, which fits in 255 bits, since $p_{\text{Pallas}} < 2^{255}$) packed into a 256-bit string together with one sign bit, placed in the otherwise-unused top bit, recording which of the two square roots $\pm y$ is meant. Unlike `Extract`, this encoding is injective — it determines P completely — and it is what we feed, as a bit-string, into a hash or commitment.

Integer encodings LE_n . $\text{LE}_n(m) \in \{0, 1\}^n$ denotes the n -bit little-endian encoding of an integer $m \in \{0, \dots, 2^n - 1\}$: the bit string $b_0 b_1 \dots b_{n-1}$ with $m = \sum_{i=0}^{n-1} b_i 2^i$ (a field element encodes via its integer representative). It renders integers and field elements as bit strings wherever they enter a hash or commitment: LE_{32} indexes the Sinsemilla generator table (§2.3), LE_{10} tags the Merkle-tree layers (§2.6), and LE_{64} , LE_{255} encode note values and field elements (§2.5).

The hash-to-curve `GroupHash`. $\text{GroupHash}^{\mathbb{G}} : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{G}$ takes a pair (domain separator D , message M) and returns a point of $\mathbb{G}$. It is *deterministic* (the same input always yields the same point), *public* (anyone can evaluate it), and *efficiently computable*. It instantiates the IETF hash-to-curve standard: `hash_to_field` expands (D, M) — via `expand_message_xmd` over BLAKE2b-512, with D embedded in the domain-separation tag — into *two* base-field elements; each passes through the simplified SWU map onto a curve isogenous to $\mathbb{G}$; and the sum of the two resulting points is carried through the isogeny to $\mathbb{G}$ (the *Crypto Guide* develops this construction in full). For everything that follows, we use only the properties that `GroupHash` is deterministic, public, and behaves like a random choice of point.

That last property is the purpose of the construction: because every stage is public and deterministic, the public strings D, M pin down the output $\text{GroupHash}^{\mathbb{G}}(D, M)$, so no party can have chosen it to satisfy a hidden relation such as $P_2 = [s]P_1$ for a secret s . We call such points *nothing-up-my-sleeve* generators. For the security arguments we *model* `GroupHash` as a random oracle into $\mathbb{G}$ (the *Crypto Guide*): we treat its outputs as independent uniform points, so the family it produces has no efficiently-findable non-trivial discrete-log relation $\sum_i [a_i]P_i = \mathcal{O}$ — precisely the assumption the Sinsemilla binding and collision-resistance reductions invoke (§3.2). As with all random-oracle arguments this is a strong, standard, but unprovable modelling heuristic about the concrete hash, not a theorem.

Domain separators. A *domain separator* $D \in \{0, 1\}^*$ is a fixed ASCII byte string naming the use site, fed to `GroupHash` so that logically distinct uses draw independent, unrelated generators (a collision found in one context cannot be transported to another). The strings are part of the protocol specification; the ones relevant here are

<code>z.cash : Orchard</code>	fixed generators $G^{\text{SpendAuth}}, G^{\text{nf}}$ (§2.4, §2.7),
<code>z.cash : Orchard-gd</code>	the diversified base g_d (§2.4),
<code>z.cash : Orchard-NoteCommit</code>	note commitments (§2.5),
<code>z.cash : Orchard-CommitIvk</code>	the ivk commitment (§2.4),
<code>z.cash : Orchard-MerkleCRH</code>	the Merkle-tree node hash (§2.6),
<code>z.cash : Orchard-cv</code>	the value-commitment bases (§2.9),
<code>z.cash : SinsemillaQ, z.cash : SinsemillaS</code>	the Sinsemilla Q point and S table (§2.3).

2.3 The commitment primitive: Sinsemilla

Two of the objects we build — the note commitment (§2.5) and the Merkle-tree node hash (§2.6) — require the same primitive: a commitment/hash that the spender must recompute *inside the zero-knowledge circuit*. In-circuit cost is therefore the binding constraint, and it is what Sinsemilla (Bowe and Hopwood) is designed to minimise. Its collision resistance reduces to DL on a prime-order curve, so it introduces no assumption beyond the DL hardness that Orchard’s algebraic core already requires. It builds directly on the preliminaries of §2.2: its generators are GroupHash outputs, and its short form returns an Extracted coordinate.

Construction 2.1 (Sinsemilla). Fix a prime-order group $\mathbb{G}$ (Orchard: $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$) and a chunk width k ($k = 10$ in Orchard). From the domain separator D , derive the $2^k + 1$ Sinsemilla generators:

$$\begin{aligned} Q(D) &:= \text{GroupHash}^{\mathbb{G}}(\text{z.cash} : \text{SinsemillaQ}, D) \in \mathbb{G}, \\ P[m] &:= \text{GroupHash}^{\mathbb{G}}(\text{z.cash} : \text{SinsemillaS}, \text{LE}_{32}(m)) \in \mathbb{G}, \quad 0 \leq m < 2^k. \end{aligned}$$

The point $Q(D)$ is the domain-dependent starting accumulator; the 2^k points $P[0], \dots, P[2^k - 1]$ form a fixed table indexed by a k -bit chunk value, shared across all Sinsemilla domains. For a message M split into k -bit chunks $m_1, \dots, m_\ell$ (so each $m_i \in \{0, \dots, 2^k - 1\}$ indexes the table),

$$\text{Sinsemilla}_D(M) := \text{Acc}_\ell, \quad \text{Acc}_0 := Q(D), \quad \text{Acc}_{i+1} := (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i,$$

with $\dot{+}$ *incomplete* short-Weierstrass addition. The output $\text{Acc}_\ell \in \mathbb{G}$ is a curve point.

Theorem 2.2 (Sinsemilla collision resistance). *For fixed-length inputs, Sinsemilla_D is collision-resistant assuming DL on $\mathbb{G}$ and modelling the generator derivation as a random oracle. (We prove this in §3.2.)*

Definition 2.3 (Sinsemilla commitment and short commitment). The *Sinsemilla commitment* adds a hiding term to the hash:

$$\text{SinsemillaCommit}_r(D, M) := \text{Sinsemilla}_{D \parallel -M}(M) \dot{+} [r] H_D \in \mathbb{G},$$

where $r \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment randomness (trapdoor), the hash runs in the suffixed domain $D \parallel -M$, and $H_D \in \mathbb{G}$ is a fixed blinding generator for the domain D , derived as the GroupHash output

$$H_D := \text{GroupHash}^{\mathbb{G}}(D \parallel -r, \varepsilon) \in \mathbb{G}$$

— the suffix $-r$ appended to the domain string, with the empty message ε (a nothing-up-my-sleeve point, §2.2). Because H_D is a GroupHash output in its own sub-domain, no party knows a discrete-log relation between it and the generators $Q(D \parallel -M), P[m]$ that the hash uses; this independence is exactly what keeps the commitment binding — a party knowing such a relation could trade the blinding term against the message and open one commitment to two distinct messages — while hiding needs no assumption at all: for uniform r the term $[r]H_D$ is uniform on $\mathbb{G}$ and masks the hash completely. This is the same division of roles the independent base H supports in a Pedersen commitment (the *Crypto Guide*). The *short commitment* returns only its x -coordinate, a base-field

element rather than a curve point:

$$\text{ShortCommit}_r(D, M) := \text{Extract}(\text{SinsemillaCommit}_r(D, M)) \in \mathbb{F}_{p_{\text{Pallas}}},$$

with Extract the coordinate map of §2.2. The unkeyed *short hash* omits the blinding term altogether:

$$\text{ShortHash}_D(M) := \text{Extract}(\text{Sinsemilla}_D(M)) \in \mathbb{F}_{p_{\text{Pallas}}},$$

the unblinded analogue of ShortCommit — no randomness argument, collision-resistant (Theorem 2.2) but not hiding.

The commitment is perfectly hiding and computationally binding — the two properties §2.5 requires of NoteCommit . Indeed the note commitment $\text{NoteCommit}_{\text{rcm}}(\cdot)$ is exactly $\text{SinsemillaCommit}_{\text{rcm}}$ under the domain $\text{z.cash} : \text{Orchard-NoteCommit}$. The short forms appear wherever the consumer requires a field element instead of a point: ShortHash in the Merkle-tree node hash (§2.6), ShortCommit in the incoming viewing key ivk (§2.4). The latter use enters through a named instance that fixes the domain and the message encoding:

$$\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(x, y) := \text{ShortCommit}_{\text{r}_{\text{ivk}}}(\text{z.cash} : \text{Orchard-CommitIvk}, \text{LE}_{255}(x) \parallel \text{LE}_{255}(y)),$$

taking two field elements and concatenating their 255-bit little-endian encodings into the 510-bit message. Its output lies in $\{1, \dots, p_{\text{Pallas}} - 1\}$ (the zero output cannot occur for a valid key, so ivk is a usable scalar).

In-circuit cost. Each round consumes a whole k -bit chunk via one table lookup of $P[m]$ and two incomplete additions, so a 510-bit message costs 51 rounds rather than 510 bit-operations. In a PLONK-ish circuit with lookups (the *Halo 2 Guide*) this is dramatically cheaper than Sapling’s bit-by-bit Pedersen hash — which is the reason the protocol changed hash between generations.

2.4 Keys: capabilities for spending and for viewing

We build the Orchard machinery bottom-up, beginning with the keys, because everything later — addresses, note commitments, nullifiers — derives from them. A practical requirement shapes the key hierarchy: different parties should be able to perform different operations. The owner can spend. An auditor should be able to *see* incoming and outgoing payments without being able to spend. A point-of-sale terminal should be able to *detect* incoming payments without being able to see outgoing ones. Each of these is a distinct capability, and the key tree makes each capability a separate derived key — the owner can hand out a lower-level key without surrendering the powers above it.

Definition 2.4 (Orchard key hierarchy). The root secret is a 32-byte *spending key* sk , chosen uniformly at random when the owner creates the account (in a deployed wallet it typically derives from a seed phrase via ZIP-32 hierarchical-deterministic derivation, but for present purposes it is simply a uniform 256-bit string, $\text{sk} \in \{0, 1\}^{256}$). Every other key is a deterministic function of sk .

Expansion function. The expansion uses BLAKE2b-512, a conventional (non-arithmetisation-friendly) cryptographic hash with a 512-bit (64-byte) digest; we write PRF-style expressions with two arguments, the first a key and the second a message. A *pseudorandom function* (PRF) is a keyed family $\{f_K\}_K$ whose outputs are, for a secret uniform K , computationally indistinguishable from those of a truly random function; here the protocol obtains one not from BLAKE2b’s native keyed mode but as *personalised, unkeyed* BLAKE2b over the concatenation $\text{sk} \parallel t$, a PRF of t whenever sk is secret.

Accordingly, write $\text{PRFexpand} : \{0, 1\}^{256} \times \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ for the PRF instantiated as $\text{PRFexpand}(\text{sk}, t) := \text{BLAKE2b-512}(\text{``Zcash_ExpandSeed''}, \text{sk} \parallel t)$ — unkeyed BLAKE2b-512 with the fixed 16-byte personalisation string ```Zcash_ExpandSeed''` and input $\text{sk} \parallel t$, where the secret sk acts as the PRF key and the leading bytes of t are a domain-separation tag identifying

which sub-secret it produces. (We write the key as a subscript, $\text{PRFexpand}_{\text{sk}}(t)$, when convenient.) We reduce its 512-bit output into the required field or scalar set. From sk derive the three spend-side secrets

$$\begin{aligned} \text{ask} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0\text{x}06)) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{nk} &:= \text{ToBase}(\text{PRFexpand}_{\text{sk}}(0\text{x}07)) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ r_{\text{ivk}} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0\text{x}08)) \in \mathbb{F}_{p_{\text{Vesta}}}, \end{aligned}$$

so that the *spend authorising key* ask and the *CommitIvk randomness* r_{ivk} are scalars in $\mathbb{F}_{p_{\text{Vesta}}}$ (the scalar field of Pallas, i.e. the integers mod the Pallas group order p_{Vesta}), while the *nullifier key* nk is a base-field element in $\mathbb{F}_{p_{\text{Pallas}}}$. The two reduction maps read the 512-bit PRF output as an integer and reduce it modulo the relevant prime:

$$\begin{aligned} \text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Vesta}} \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Pallas}} \in \mathbb{F}_{p_{\text{Pallas}}}, \end{aligned}$$

where LEOS2IP_{512} is the little-endian octet-string-to-integer map sending a 64-byte string to the integer $\sum_{i=0}^{63} x_i 256^i \in \{0, \dots, 2^{512} - 1\}$. ToScalar lands in $\mathbb{F}_{p_{\text{Vesta}}}$, ToBase in $\mathbb{F}_{p_{\text{Pallas}}}$; the two targets differ only because the protocol uses nk as a field element (it feeds it to the nullifier PRF, §2.7) whereas it uses $\text{ask}, r_{\text{ivk}}$ as scalars multiplying group elements. Because the input is 512 bits wide while each modulus is just over 2^{254} (of bit length 255), the reduction is statistically indistinguishable from uniform on the target set (the modular bias is $O(2^{254}/2^{512}) = O(2^{-258})$, negligible). The distinct tag bytes $0\text{x}06, 0\text{x}07, 0\text{x}08$ make the three outputs independent.

Spend validating key. Let $G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ be a fixed, publicly-known generator of the Pallas group, obtained by $G^{\text{SpendAuth}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, G)$ (a nothing-up-my-sleeve point, §2.2). The *spend validating key* is then the public point

$$\text{ak} := [\text{ask}] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}).$$

Key generation canonicalises its sign: if $[\text{ask}] G^{\text{SpendAuth}}$ has odd y -coordinate, ask is replaced by $-\text{ask}$, so that ak always has even y and the x -coordinate $\text{Extract}(\text{ak})$ alone determines the point. The *full viewing key* bundles the three quantities needed to recognise and decrypt one's own notes, $\text{fvk} := (\text{ak}, \text{nk}, r_{\text{ivk}}) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Vesta}}}$. From it derive

$$\begin{aligned} \text{ivk} &:= \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk}) \in \{1, \dots, p_{\text{Pallas}} - 1\} \subset \mathbb{F}_{p_{\text{Pallas}}}, \\ K &:= \text{LE}_{256}(r_{\text{ivk}}) \in \{0, 1\}^{256}, \\ R &:= \text{PRFexpand}(K, 0\text{x}82 \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})) \in \{0, 1\}^{512}, \\ (\text{dk}, \text{ovk}) &:= (R_{[0..256)}, R_{[256..512)}) \in \{0, 1\}^{256} \times \{0, 1\}^{256}, \end{aligned}$$

where $\text{Commit}^{\text{ivk}}$ is the named ShortCommit instance of §2.3 and LE_{256} the 256-bit little-endian bit encoding of §2.2, giving K the type $\{0, 1\}^{256}$ that PRFexpand requires of its key. This yields the *incoming viewing key* ivk (a base-field scalar), the *diversifier key* dk , and the *outgoing viewing key* ovk . The dk/ovk derivation is precisely the following: we split a *single* 512-bit PRFexpand output R , keyed by r_{ivk} over the input $0\text{x}82 \parallel \text{ak} \parallel \text{nk}$ (not over an empty message), into its two 256-bit halves — dk the first, ovk the second. The diversifier d below is *not* part of this split; it derives afterwards from dk . Here $\text{Extract}(\text{ak})$ is the x -coordinate of ak (§2.2) — which, after the sign canonicalisation above, determines ak — while I2LEOSP_{256} denotes the 32-byte little-endian encoding of a field

element or point. A *diversified address* is, for any index $d_{\text{plain}} \in \{0, 1\}^{88}$,

$$\begin{aligned} d &:= \text{FF1-AES256}_{\text{dk}}(d_{\text{plain}}) \in \{0, 1\}^{88}, \\ g_d &:= \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \\ \text{pk}_d &:= [\text{ivk}] g_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \end{aligned}$$

where $\text{FF1-AES256}_{\text{dk}}$ produces the *diversifier* $d \in \{0, 1\}^{88}$, the NIST FF1 format-preserving encryption mode (built on AES-256) keyed by dk : a keyed *permutation* of the 88-bit index space $\{0, 1\}^{88}$, so that each d_{plain} maps to a distinct 88-bit diversifier d and different d_{plain} give unlinkable-looking addresses, all derived from the one key. The *address* is the pair $\text{addr} := (d, \text{pk}_d) \in \{0, 1\}^{88} \times \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.

Choosing the index d_{plain} . The receiving wallet chooses the index freely: *any* of the 2^{88} values yields a valid, spendable address for the same ivk . No derivation fixes it and no index is forbidden — any 88-bit string is a valid diversifier index. The wallet allocates indices on demand, conventionally sequentially from $d_{\text{plain}} = 0$ (so index 0 is the *default diversifier*), assigning a fresh index per payee or per invoice to keep their addresses mutually unlinkable; a randomly drawn index would yield an equally valid address, but the specification states that diversifier indices SHOULD NOT be chosen at random: ZIP-32 specifies their usage in hierarchical-deterministic wallets, whose seed-only recovery of issued addresses depends on deterministic allocation. Every index works for a reason specific to Orchard: the hash-to-curve $\text{GroupHash}^{\text{Pallas}}$ of §2.2 is total — it returns a point for every input — so every index yields a well-defined g_d ; in the negligible-probability event that the two summed SWU outputs cancel to $\mathcal{O}$, the specification substitutes the fallback $\text{GroupHash}(D, "")$. This is a deliberate improvement over Sapling, where the diversifier hash failed on roughly half of all indices and the wallet had to increment d_{plain} until it found a valid one; in Orchard no such rejection loop exists.

Rationale for routing the index through AES. The step from d_{plain} to d is a keyed *permutation* (format-preserving encryption) for three reasons that no simpler map satisfies together. (i) *Unlinkability*: wallets allocate indices in order $0, 1, 2, \dots$, and consecutive diversifiers would reveal a shared owner and an issue count; encryption under dk scatters them pseudorandomly over $\{0, 1\}^{88}$. (ii) *Bijection*: a permutation never collides two indices onto one d and is invertible — with dk the wallet recovers the index behind any of its addresses, storing no per-address secret; a plain hash is neither collision-free nor invertible. (iii) *Format preservation*: the address format fixes the diversifier at exactly 88 bits, and FF1 maps $\{0, 1\}^{88} \rightarrow \{0, 1\}^{88}$ bijectively, which a truncated hash cannot. This derivation runs *wallet-side, never in the circuit*, so a conventional fast cipher (AES-256 under NIST FF1) is appropriate; arithmetisation-friendliness is irrelevant here.

The structure is a deliberate capability ladder (Figure 1), each rung conferring strictly less power than the one above:

- sk / ask — *spend*. Whoever holds ask can authorise spends; it sits at the top and nothing below it can recover it.
- fvk (hence ivk , ovk) — *see everything*. Detects incoming notes (via ivk , which recovers pk_d) and views outgoing notes (via ovk), but cannot spend, because deriving fvk from sk is one-way.
- ivk alone — *detect incoming*. Sufficient to recognise notes addressed to the holder and scan the chain, nothing more — the key suitable for an always-online wallet server.
- A diversified address (d, pk_d) — *receive*. A public handle anyone can pay; one ivk generates 2^{88} mutually unlinkable addresses — one per diversifier index (Definition 2.4), practically inexhaustible — all spendable by the same key.

Of these, only the diversified address is *public*. The viewing keys fvk , ivk , ovk are *secret* key material — “viewing” names what they let the holder *do* (see transactions), not that one may publish them.

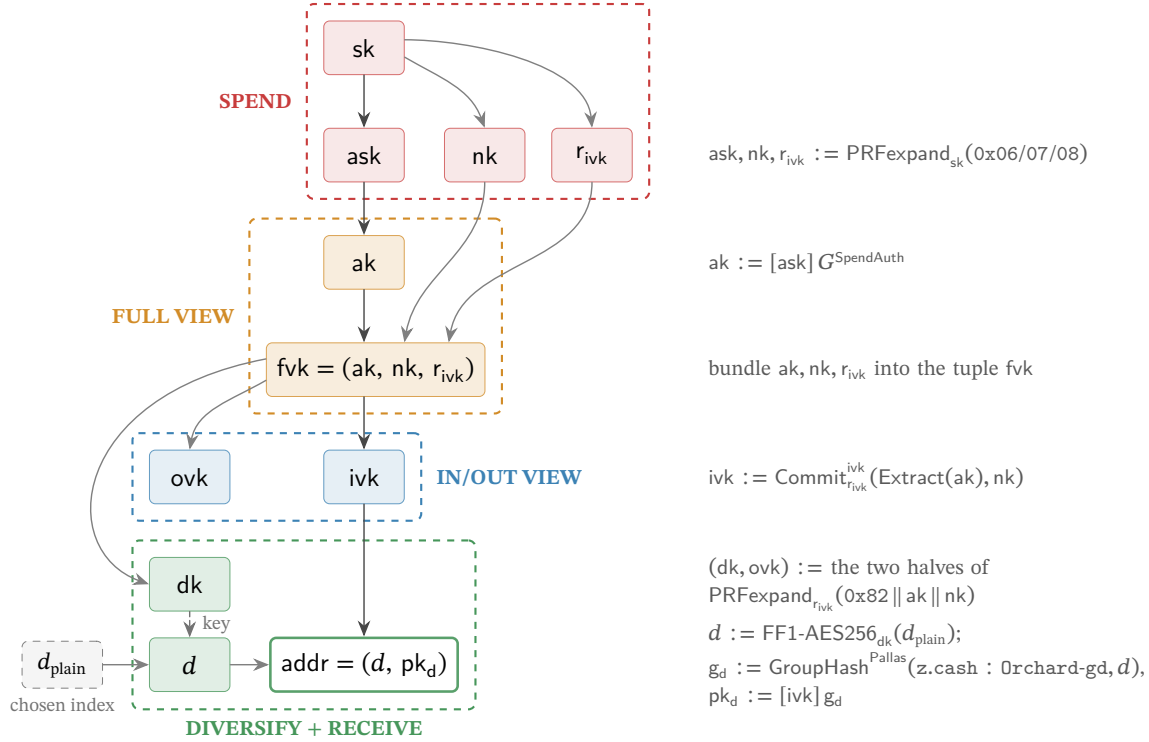


Figure 1: The Orchard key hierarchy as a capability ladder, read top to bottom. Each arrow is a derivation; the right-hand gutter shows the operation that produces the child from its parent(s). Not every edge is one-way: the bundling edges (ak, nk, r_{ivk} into the tuple fvk ; d into the address) invert by projection, and the FF1 edge is a keyed permutation the dk holder inverts — the tiers are separated by the one-way steps $sk \rightarrow (ask, nk, r_{ivk})$, $ask \rightarrow ak$, $fvk \rightarrow (ivk, dk, ovk)$, and $ivk \rightarrow pk_d$. The dk and ovk keys are the two halves of one $\text{PRFexpand}_{r_{ivk}}$ output (tag $0x82$, input $ak \parallel nk$); they play different roles and land in different tiers — ovk is the *outgoing* viewing key, grouped with the *incoming* viewing key ivk as the IN/OUT viewing tier, whereas dk is the *diversifier* key, which sits in the address layer because it generates diversified addresses. Concretely d is the format-preserving encryption FF1-AES256 of a chosen index d_{plain} under the key dk (the dashed edge marks dk entering as the cipher key), and $pk_d = [ivk] g_d$, so the address binds both keys. *Every node is secret key material except the chosen index d_{plain} , the diversifier d , and the diversified address (unshaded) — the address (d, pk_d) , and with it d , is the only value the protocol ever publishes* (shading marks tier membership, not secrecy) — in particular the viewing keys fvk, ivk, ovk are *secret*: “viewing” names what they let the holder do, not that the protocol publishes them. Each dashed tier holds a strictly weaker capability than the one above, so a holder may receive a lower-tier key without gaining the powers above it. Drawn to match Definition 2.4.

Disclosing a viewing key confers the corresponding visibility on the recipient — every payment to and from the wallet for fvk , incoming payments only for ivk , outgoing only for ovk — so the owner shares one only deliberately (with an auditor, say) and otherwise guards it like any other secret. The ladder is useful precisely because these capabilities are *separable*: one can delegate “see incoming” (ivk) without delegating “see everything” (fvk), and neither confers the ability to spend.

Two questions of motivation that the bare formulas do not answer. *Why is ivk a commitment*, $\text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk)$, *rather than a plain hash*? Because ivk must both bind *and* hide. It binds ak (spend authority) and nk (nullifier derivation) to one identity, and check A7 below proves the linkage in-circuit by recomputing the commitment from the witnessed ak , nk , r_{ivk} — but binding alone a collision-resistant hash would supply equally, and in-circuit recomputation works for any hash (check A3 recomputes the unblinded Sinsemilla MerkleCRH; Sapling even proved its BLAKE2s-based CRH^{ivk} in-circuit). What the trapdoor r_{ivk} adds is *hiding*: the blinded ShortCommit is perfectly hiding, so ivk — and hence every public $pk_d = [ivk]g_d$ — reveals nothing about ak and nk , whereas the unblinded Sinsemilla hash is only collision-resistant (Theorem 2.2) with no one-wayness claim. Hiding is what keeps the IN/OUT tier strictly below FULL VIEW: an ivk holder, such as an always-online wallet server, cannot extract nk or ak . *Why is nk separate from ask* ? So that the owner can delegate the capability to compute nullifiers (which a viewing key needs to determine which of the holder’s notes are already spent) *without* conferring the ability to spend — the viewing key holds nk but never ask .

2.5 Representation of a note: notes and note commitments

The first requirement is to represent a note such that it stays invisible on-chain yet provable by its owner. The protocol therefore never places the note on the chain at all; it publishes only a hiding commitment to it.

Definition 2.5 (Orchard note). An Orchard *note* — the private object representing one unit of value — is a tuple

$$n := (\text{addr}, v, \rho, \psi, \text{rcm}),$$

where $\text{addr} := (d, pk_d)$ is the recipient’s diversified address; $v \in \{0, \dots, 2^{64} - 1\}$ is the value in zatoshi; $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$ is a per-note uniqueness seed; $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$ is randomness the sender fixes through the note seed r_{seed} , from which it is derived deterministically (§2.8); and $\text{rcm} \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment trapdoor.

The protocol never publishes the note itself. What it records on-chain is its *note commitment*

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(g_d^\star \parallel pk_d^\star \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

a commitment that is *perfectly hiding* (the chain learns nothing about $\text{addr}, v, \rho, \psi$ from cm) and *computationally binding* (the sender cannot later claim the commitment was to a different note). These are precisely the two properties of a commitment scheme from the *Crypto Guide*; here the scheme is Sinsemilla (§2.3), chosen because it is inexpensive to recompute *inside the circuit*, which the spender must do. The chain stores $\text{cm}_x := \text{Extract}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}}$, the x -coordinate of the commitment point, because the Merkle tree consumes a field element.

The three address-related quantities appearing in NoteCommit are the key material that §2.4 defines: the diversifier $d \in \{0, 1\}^{88}$, the diversified base $g_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (the per-address generator $\text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d)$), and the transmission key $pk_d = [ivk]g_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ that identifies the recipient; the address is $\text{addr} = (d, pk_d)$. Recall the $\star$ -encoding from §2.2: $g_d^\star, pk_d^\star \in \{0, 1\}^{256}$. The NoteCommit input is the bit-string concatenation ($\parallel$) of these point encodings with the little-endian integer encodings $\text{LE}_{64}(v) \in \{0, 1\}^{64}$ and $\text{LE}_{255}(\rho), \text{LE}_{255}(\psi) \in \{0, 1\}^{255}$.

The choice of fields is as follows. addr and v are the note’s content. ρ and ψ exist solely to render the eventual nullifier unique and unpredictable; their role becomes apparent only in §2.7: a note carries randomness whose purpose is downstream.

2.6 Proof of ownership of *some* valid note: the commitment tree

The second requirement is to prove the spent note is valid without revealing which note. When Alice spends, she must convince the network that the note she is spending is a valid note that someone actually created; otherwise she could spend a note she invented. On a transparent chain this is trivial: the output is present in the set of unspent transaction outputs (the UTXO set) that every node maintains. But Alice must prove it *without revealing which note*, because revealing the note would deanonymise the payment.

The network maintains one append-only Merkle tree whose leaves are *every note commitment ever published*, in creation order. Alice proves *membership*: a Merkle authentication path from her leaf cmx to a root the network trusts. Inside the circuit the path is a private witness (check A3), so the verifier learns that her note is one of the leaves but not which one; the anonymity set is the entire tree.

Definition 2.6 (Orchard commitment tree). The Orchard *note commitment tree* is an append-only Merkle tree of fixed depth 32 (so up to 2^{32} notes). Leaves are cmx values in creation order. The parent of a left child ℓ and right child r at layer ℓ' is

$$\text{MerkleCRH}_{\ell'}^{\text{Orchard}}(\ell, r) := \text{ShortHash}_{\text{z.cash:Orchard-MerkleCRH}}(\text{LE}_{10}(\ell') \parallel \text{LE}_{255}(\ell) \parallel \text{LE}_{255}(r)) \in \mathbb{F}_{\text{Pallas}},$$

a layer-tagged Sinsemilla short hash (Definition 2.3) of the $10 + 255 + 255 = 520$ -bit message — 52 ten-bit chunks, the same LE-encoded input convention as $\text{Commit}^{\text{ivk}}$ and NoteCommit (leaves cmx , internal nodes, and the root all live in $\mathbb{F}_{\text{Pallas}}$). Appending a block’s note commitments yields a new root: the root of the tree after block h is the *anchor* of block h (a single field element), so every main-chain block defines exactly one anchor. A spend proves membership against the anchor of any main-chain block, and all Actions of one bundle prove against a single shared anchor (§2.13).

Every node is a single base-field element by necessity, not convention. The spender recomputes this hash layer by layer *inside* the Action circuit to prove membership (check A3 of §2.14), and that circuit is arithmetic over $\mathbb{F}_{\text{Pallas}}$: every wire carries one such field element. A curve-point node would carry two coordinates and an on-curve constraint at each of the 32 layers; reducing each node to its x -coordinate halves the path data the circuit threads and removes that bookkeeping. This is precisely why MerkleCRH ends in *Extract*: Sinsemilla yields a point, and *Extract* returns it to the field, so that each layer’s output is directly the next layer’s input and the one hash composes all the way to the root. Dropping the y -coordinate is harmless here (§2.2): the tree uses each node only as a binding digest, never to recover a child point, and a collision in the x -coordinate still reduces to one in Sinsemilla (Theorem 2.2).

Two design details are significant. The tag carries the layer index ℓ' so that a path at one height cannot replay at another. And empty leaf positions take an “uncommitted” sentinel value 2: the choice is not arbitrary — at $x = 2$ the curve equation gives $y^2 = 2^3 + 5 = 13$, and 13 is a non-square modulo both Pasta primes (the *Math Guide* verifies this), so 2 is not the x -coordinate of any Pallas point, hence can never equal a valid cmx and can never be silently confused with a genuine commitment.

Derivation of the authentication path. Alice’s commitment occupies a fixed *position* pos — its index in creation order — which her wallet records when it detects the note (§2.8). Fix an anchor of any block at or after the one that mined her note, and let n be the number of leaves in that block’s

tree. The authentication path of leaf pos against that anchor consists of one node per level: writing $\text{pos} = b_{31} \cdots b_1 b_0$ in binary, the level- i ancestor of her leaf is a left child when $b_i = 0$ and a right child when $b_i = 1$, and the path's level- i entry is that ancestor's *sibling* in the tree of size n (Figure 2). Check A3 reverses the construction: starting from cmx it combines the running node with one path entry per level — placing the node on the side b_i dictates (left when $b_i = 0$, right when $b_i = 1$) and the sibling opposite — and asserts that the final value equals the public anchor; the bits b_i and the siblings stay inside the witness.

Each entry is one of two kinds. When $b_i = 1$ the sibling subtree lies to the *left* of her leaf: it spans commitments older than hers, so every one of its positions is filled — a *complete* subtree — and, because the tree appends only on the right, its root is final and identical at every anchor. When $b_i = 0$ the sibling subtree lies to her *right*, and its value depends on the anchor: its root hashes the commitments it holds at size n , with positions not yet filled taking the uncommitted sentinel 2; a sibling subtree still entirely empty at size n contributes a per-level constant, the sentinel folded up through the layer-tagged hash. The path is therefore a deterministic function of pos and the first n public leaves, and of nothing else; the only secret is which leaf is hers.

In practice the wallet neither rebuilds the path from the leaves on each spend nor retains the whole tree. As it scans, it appends every published cmx — its own and everyone else's — but keeps only what a future path needs: the nodes from each of its own notes up to the root, the complete-subtree roots flanking those paths, and one *checkpoint* per block; the rest it prunes (this is the design of `librustzcash's shardtree`). It marks its note's position the moment trial decryption succeeds (§2.8). A checkpoint stores merely the position of that block's last leaf, equivalently the leaf count n — the size that fixes the block's anchor; the anchor itself is recomputed from the retained nodes on demand, never stored.

To produce the path for a spend against a chosen block, the wallet descends its retained tree to the marked leaf and, on the way back to the root, reads off each sibling subtree's root: a complete subtree contributes its stored root, a subtree lying wholly beyond the leaf count n contributes the per-level empty root, and the single subtree straddling position n is rehashed from its retained nodes. The leaf count enters only as this cutoff — every position $\geq n$ counts as empty — so one retained tree reproduces the path against any block it has checkpointed.

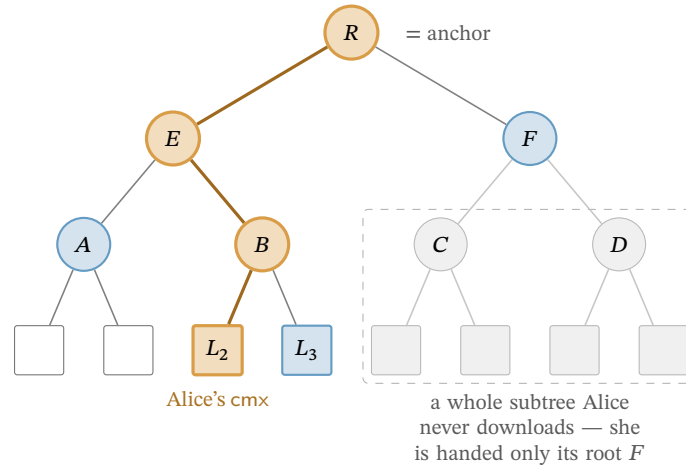


Figure 2: Computation of a Merkle authentication path (drawn at depth 3; Orchard's tree has depth 32). The leaves are the public note commitments cmx , in order. Alice's note sits at leaf L_2 (orange); to prove it is in the tree she must compute the authentication path of L_2 , then recompute the bold route from L_2 up to the root R — the *anchor* — folding in, at each level, the sibling hash beside it: that path is L_3, A, F (blue). Efficiency is the only reason not to hash the whole tree: a far subtree enters her path solely through its single root (here F), so she never downloads its leaves (faded) — the `GetSubtreeRoots` shortcut that lets even a freshly restored wallet witness a years-old note.

Nothing above requires Alice to have been online: the inputs are public, so a freshly restored wallet rebuilds the same state by replaying the chain’s note commitments. Nor need it touch all 2^{32} positions, and here the tree’s structure earns its keep. Cut the tree at half its depth. Below the cut lie the *shards* — the height-16 subtrees, 2^{16} leaves apiece, up to 2^{16} of them — and above it a height-16 *cap* whose own leaves are the shard roots (librustzcash sets the shard height to half the tree depth, $32/2 = 16$). A leaf’s 32 siblings divide along the same cut: the low 16 (levels 0 through 15) lie inside its own shard, and the high 16 (levels 16 through 31) are cap nodes, each the root of a subtree spanning whole shards. The two halves reach Alice by entirely different routes, and that asymmetry is the economy of the scheme.

Her own shard she must hash from leaves. Its 16 low siblings are the within-shard subtrees of the descent above — complete to her left, straddling or empty to her right — so she scans that shard’s commitments block by block, her own and everyone else’s, and folds them; this is the one subtree whose 2^{16} leaves she handles individually. Every *other* shard she takes as a single 32-byte root. The cap’s leaves are exactly those roots, which a light server streams one per completed shard (GetSubtreeRoots); Alice folds them upward with the same layer-tagged MerkleCRH, one application per cap level — the upper half of the tree’s 32 layers — and reads her high siblings off the result (Figure 2 is the toy-depth analogue: the faded subtree enters only through its root F). A shard that does not yet exist — the empty right of the cap — contributes the per-level empty constant rather than a download, and her own shard she holds from leaves whether or not it has completed, so she never needs its root from the server. Her level-16 sibling is thus a neighbouring shard’s root, and each sibling above it a fold of 2, 4, 8, ... roots she already holds.

A concrete case fixes the shape. Let Alice’s note be the *last* leaf of the newest shard k . Below the cut, all 16 low siblings are complete left subtrees tiling the shard’s earlier $2^{16} - 1$ positions, so she ingests essentially the whole shard. Above the cut, the cap’s right side is empty — no shard succeeds k , so those siblings are the empty constant — while its left side folds the roots of shards $0, \dots, k - 1$, each supplied by GetSubtreeRoots. Stitching the halves — her leaf folds up to the shard- k root, which folds up the cap to the anchor — gives the full 32-level path.

The 2^{16} throughout bounds the cap’s *capacity* — 2^{32} leaves over 2^{16} per shard — not Alice’s traffic. She fetches one root per shard that actually exists, $\lceil n/2^{16} \rceil$ of them: a few hundred at present tree sizes, and growing by one every 2^{16} outputs. The whole history above her shard thus arrives as a few kilobytes of roots in place of millions of leaves. She performs the fold *locally*, never asking a server for one particular note’s path, which would reveal which note is hers; the finished path is the private witness of check A3, and only the anchor is public.

Anchor choice as the tree grows. By the time Alice’s transaction reaches a block, the live tree has advanced past her chosen anchor. This costs nothing: appending leaves creates new roots and alters no old one, so the anchor still names one fixed historical tree, her path still folds to exactly that root, and check A3 asserts exactly that equality — collision resistance of the layer-tagged hash (§2.3) makes a path to that root for a non-member infeasible. Consensus accepts the anchor of *any* main-chain block from Orchard activation onward (NU5) — for the legacy Orchard-pool tree: the two pools keep separate, independent commitment trees and nullifier sets (ZIP-229), and an Ironwood Action instead proves against anchorIronwood over the Ironwood tree, empty at NU6.3 activation, whose anchors are valid from activation onward. In neither pool is there a sliding window, so proving is fully decoupled from broadcasting — Alice need not race the chain tip. The one recency caveat runs in the opposite direction: a reorganisation can strike the anchoring block from the main chain, invalidating the anchor and forcing her to rebuild the transaction; deeper anchors make the event rarer, but no depth rules it out — the often-cited 100-block figure is the specification’s description (protocol § 3.3; the retired zcashd actually enforced 99), and the deployed zebra rolls back automatically to 1000 blocks, node policy in both validators rather than consensus (*Consensus Guide*, §“Reorg rails, maturity, and the finality floor”). Deployed wallets nonetheless anchor close to the tip, by deliberate choice rather than necessity. The anchor’s tree is the anonymity set in which check A3 conceals the spent note — the proof reveals only that the note is *some* leaf committed by that anchor — so the most recent

admissible anchor, naming the largest such tree, maximises that set, while an idiosyncratically old or deep anchor would instead fingerprint the spender. `librustzcash` accordingly selects the most recent block it has checkpointed that lies at least a fixed number of confirmations behind the tip (by default, per ZIP-315, 3 confirmations for notes the wallet produced itself and 10 for notes received from third parties), trading a sliver of reorg robustness for set size and uniformity. Consensus mandates none of this; it is wallet policy. Nor is an old anchor a soundness hole: membership in an older, smaller tree implies the note was committed even earlier, which is precisely what Alice must establish — the nullifier (§2.7), not the anchor, prevents a second spend.

2.7 Prevention of double-spending: nullifiers

The third requirement: to prevent Alice from spending the same note twice, without a public registry of spent notes (which would leak exactly what is being hidden). Each note yields a single deterministic tag — its *nullifier* — that appears only when its owner spends the note. The network maintains a set of all revealed nullifiers and rejects any repeat. The nullifier must satisfy two properties that pull in opposite directions: it must be *deterministic* (so the same note always yields the same tag, making a second spend detectable) yet *unlinkable* (so that publishing it reveals nothing about which note or address it came from).

Definition 2.7 (Orchard nullifier). For a note n with commitment cm and the owner’s nullifier key nk ,

$$nf := \text{Extract}([F_{nk}(\rho) + \psi]G^{nf} + cm) \in \mathbb{F}_{p_{\text{Pallas}}},$$

where $F_{nk} : \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ is a circuit-efficient PRF (Poseidon over the Pallas base field, the *Crypto Guide*), $G^{nf} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, K) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is a fixed nothing-up-my-sleeve Pallas generator (§2.2), the scalar $F_{nk}(\rho) + \psi$ lives in $\mathbb{F}_{p_{\text{Pallas}}}$ (a valid Pallas scalar because $p_{\text{Pallas}} < p_{\text{Vesta}}$), and Extract takes the x -coordinate. (We write G^{nf} for this base to distinguish it from the spend-auth generator $G^{\text{SpendAuth}}$ of §2.4.)

We read the construction against the two requirements. **Determinism:** every ingredient (nk, ρ, ψ, cm) is fixed once the note exists, so nf is a fixed function of the note — spending it twice yields the same nf twice, and the network rejects the second. **Unlinkability:** the term $[F_{nk}(\rho) + \psi]G^{nf}$ masks the commitment with a value that appears random to anyone without nk , so no one can tie the published nf back to cm or to the address (this is where the note’s ρ and ψ perform their function). The security properties:

- *No double-spend* (the Orchard Book calls this *balance*, since double-spending is how one would create value): each note has exactly one nullifier, and forging a colliding nullifier for a different note reduces to DL on Pallas.
- *Spend unlinkability*: without nk , the published nf is unlinkable to the note or address, under the Decisional Diffie–Hellman (DDH) assumption on Pallas — that $([a]G, [b]G, [ab]G)$ is computationally indistinguishable from $([a]G, [b]G, [c]G)$ for independent uniform a, b, c — together with the PRF assumption on F .
- *Forward consistency (Faerie resistance)*: a freshly created note takes as its uniqueness seed the nullifier of the note spent alongside it, $\rho_{\text{new}} := nf_{\text{old}}$; since the network already guarantees nullifiers are unique, this forces every note’s ρ to be globally unique, closing an attack from earlier designs in which forged notes could be made to share a nullifier.

2.8 Transmission and detection of a note: encryption, trial decryption, and synchronisation

The on-chain cmx conceals the note entirely, which leaves a question the commitment alone does not resolve: how does the recipient learn of the note with which they were paid? The sender must deliver the note to them, privately, over the same public chain. Orchard achieves this with *in-band secret distribution*: every Action carries, beside its commitment, a ciphertext only the recipient can open.

Encryption (sender side). Alice holds Bob’s address (d, pk_d) , with $\text{pk}_d = [\text{ivk}]g_d$ and $g_d = \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d)$ (§2.4). She forms the *note plaintext* $\text{np} = (0x03, d, v, \text{rseed}, \text{memo})$, where the 32-byte rseed — drawn uniformly at random, fresh for each output note — is the master randomness from which she derives the note’s rcm , ψ , and the ephemeral secret esk below, and memo is a fixed 512-byte field of sender-chosen data. Each derivation is one PRFexpand call keyed by rseed and distinguished by a leading domain-separator byte; esk and ψ take $\rho := \text{nf}_{\text{old}}$ (as 32 little-endian bytes) as input — which binds the note’s secret randomness to its Action — and rcm , derived last, takes every note field:

$$\begin{aligned} \text{esk} &:= \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0x04 \parallel \rho)) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}(0x09 \parallel \rho)) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ \text{rcm} &:= \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(\text{pre_rcm})) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{pre_rcm} &:= 0x0B \parallel g_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{256}(\rho) \parallel \text{LE}_{256}(\psi), \end{aligned}$$

where esk is redrawn under a fresh rseed in the rare event it vanishes. This is the Ironwood derivation, the current note format (ZIP-2005; §5.2 gives the exact byte layout and the order’s rationale — rcm comes last because its input includes ψ); legacy lead-byte-0x02 notes — all Orchard-pool notes — instead derive $\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0x05 \parallel \rho))$ (Remark 5.3). She then runs a one-shot Diffie–Hellman to a *fresh* key:

$$\text{esk (above)}, \quad \text{epk} := [\text{esk}]g_d, \quad s := [\text{esk}]\text{pk}_d, \quad K_{\text{enc}} := \text{KDF}(s, \text{epk}),$$

and seals the plaintext under an AEAD (ChaCha20-Poly1305), $C_{\text{enc}} := \text{Enc}_{K_{\text{enc}}}(\text{np})$. The KDF is BLAKE2b-256 with personalisation Zcash_OrchardKDF over $s^* \parallel \text{epk}^*$. Because $\text{pk}_d = [\text{ivk}]g_d$, Bob can reach the same secret from his side as $[\text{ivk}]\text{epk} = [\text{ivk}][\text{esk}]g_d = [\text{esk}]\text{pk}_d$, so, beside the sender (who knows esk), exactly the holder of ivk can recover K_{enc} from the published $(\text{epk}, C_{\text{enc}})$. A second ciphertext C_{out} , sealed under the *outgoing cipher key* $\text{ock} := \text{BLAKE2b-256}(\text{Zcash_Orchardock}, \text{ovk} \parallel \text{cv}^{\text{net}} \parallel \text{cmx} \parallel \text{epk})$ (points in $\star$ -encoding; cv^{net} is the Action’s net value commitment, §2.9), carries $(\text{pk}_d, \text{esk})$ so that the *sender* — or an auditor holding ovk — can re-derive s and read the note as well; because ock binds the Action’s own cv^{net} , cmx , and epk , a C_{out} cannot be transplanted onto another Action. The Action publishes $(\text{epk}, C_{\text{enc}}, C_{\text{out}})$ alongside cv^{net} , nf , rk , cmx , with rk the randomised validating key (§2.10).

Trial decryption (recipient side). Nothing on chain marks which Action pays Bob, so his wallet *trial-decrypts every* Orchard output: for each it recomputes $s := [\text{ivk}]\text{epk}$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$, and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$. The AEAD tag makes an incorrect guess fail cleanly (it returns $\perp$), so a successful open is the detection that the note was his. He then re-derives the full note from (d, v, rseed) — taking $\rho := \text{nf}_{\text{old}}$ from the same Action — and performs the two checks that close the obvious forgeries: (i) that the sender formed the ephemeral key honestly, $[\text{esk}]g_d = \text{epk}$ for the esk re-derived from rseed (the ZIP-212 check, defeating a mauled epk); and (ii) that the note is indeed the one the Action committed on chain, $\text{Extract}(\text{NoteCommit}_{\text{rcm}}(\dots)) = \text{cmx}$. Only when both hold does he accept. A sender can therefore never make Bob accept a note inconsistent with the commitment.

Wallet synchronisation, including from cold storage. This is the operation a wallet performs when it restores from a seed. From the account’s “birthday” height it walks the chain forward and trial-decrypts every Action; a light wallet does so against *compact* blocks, which carry only the first 52 bytes of C_{enc} — those covering the compact note plaintext: the lead byte (1 byte; 0x03 for Ironwood outputs, the only allowed value in that pool — 0x02 appears only when scanning legacy Orchard-pool outputs), d (11 bytes), v (8 bytes), and $rseed$ (32 bytes) — together with epk , cmx , and the Action’s nullifier nf , which the light wallet needs twice: it supplies $\rho := nf_{\text{old}}$ for the cmx recomputation that detects the note, and it is the source of the chain’s nullifier set against which spentness is checked below. The wallet records each note that opens *with its leaf position* — the index at which it appended its cmx , read directly off the block — and that position is precisely what the commitment tree of §2.6 consumes: the wallet *marks* the position in its shard tree and can then build the note’s authentication path on demand, by the root-only assembly described there, without replaying every commitment. To determine which recovered notes remain spendable it additionally requires nk : it computes each note’s nullifier (§2.7) and discards those whose nullifier already appears in the chain’s nullifier set. The end state is a set of unspent notes, each with a value, a position, an authentication path, and a nullifier — exactly what the Action prover of §2.11 consumes.

2.9 Conservation of value: value commitments and the binding signature

The fourth requirement: to guarantee that a transaction creates no value, while concealing every amount. The instrument is the additive homomorphism of Pedersen commitments (the *Crypto Guide*): commit to each amount, and verify that the commitments *sum* to the publicly declared net flow — the individual values remain hidden, while their balance is publicly verifiable.

Definition 2.8 (Net value commitment). Each Action commits to its own value change with a Pedersen commitment

$$cv^{\text{net}} := [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [rcv] R^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

where v_{old} is the spent value, v_{new} the created value, $rcv \in \mathbb{F}_{p_{\text{Vesta}}}$ the commitment randomness (a Pallas scalar), and $V^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "v")$, $R^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "r") \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ are nothing-up-my-sleeve points (§2.2), hence independent Pallas generators.

The spender draws each rcv uniformly at random and afresh for every Action, at the moment of bundle assembly. In contrast to the note’s rcm , ψ , and esk — all derived deterministically from the note seed $rseed$ (§2.8) — rcv belongs to no note and is never transmitted; its uniformity and secrecy are precisely what make cv^{net} hiding, and what keep the aggregate bsk introduced below known only to the transaction author.

We now apply the homomorphism. Summing over the m Actions in a transaction,

$$\sum_{i=1}^m cv^{\text{net},i} = \left[\sum_i (v_{\text{old},i} - v_{\text{new},i}) \right] V^{\text{Orchard}} + [bsk] R^{\text{Orchard}}, \quad bsk := \sum_i rcv_i \in \mathbb{F}_{p_{\text{Vesta}}}.$$

If the transaction is balanced, the bracketed value-component equals the publicly declared net flow $v_{\text{balance}}^{\text{Orchard}}$ (a signed field of the transaction; by convention a positive value is net value flowing *out* of the pool — paying fees or transparent outputs, say). The network subtracts the declared part,

$$bvk := \sum_i cv^{\text{net},i} - [v_{\text{balance}}^{\text{Orchard}}] V^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

and *if and only if* the transaction balances, the V^{Orchard} -component cancels and $bvk = [bsk] R^{\text{Orchard}}$ is a public key whose secret bsk the transaction author knows. Proving knowledge of that secret

— via a *binding signature* under key bvk — is precisely proving that the accounts balance, with no amount revealed. The binding signature is RedPallas (§2.10) instantiated on the base R^{Orchard} , the value-commitment randomness base, distinct from the spend-authorisation base $G^{\text{SpendAuth}}$. A single signature over the whole transaction (concretely, over the ZIP-244 *sighash* below) additionally binds all its Actions together, so that no one can remove or transplant any of them. From NU6.3 the construction runs once per pool: a v6 transaction carries independent $v_{\text{balance}}^{\text{Orchard}}$ and $v_{\text{balance}}^{\text{Ironwood}}$ fields, and each pool’s Actions sum to their own bvk , checked by their own binding signature over the same bases $V^{\text{Orchard}}, R^{\text{Orchard}}$ (ZIP-229; §5.1).

The transaction digest (ZIP-244). The message both Orchard signatures sign — the *sighash* — is not the raw transaction but a non-malleable digest of it, which ZIP-244 assembles as a tree of personalised BLAKE2b-256 hashes, one node per transaction component. Its root is

$$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{trans}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}}),$$

where branchid is the 4-byte identifier of the active network upgrade — so a signature is valid only on the fork it was made for — and each $d_{\cdot}$ is a personalised digest of one pool’s data, an absent pool hashing the empty string. A purely shielded Orchard payment leaves the transparent and Sapling branches empty, and then the sighash coincides with the transaction identifier itself.

The Orchard branch commits to the entire bundle,

$$d_{\text{Orch}} := \text{BLAKE2b-256}(\text{ZTxIdOrchardHash}, d_C \parallel d_M \parallel d_N \parallel \text{flagsOrchard} \parallel \text{LE}_{64}(v_{\text{balance}}^{\text{Orchard}}) \parallel \text{anchor}),$$

through three digests that partition every Action’s fields: the *compact* d_C over nf , cmx , epk and the first 52 bytes $C_{\text{enc}}[0:52]$; the *memo* d_M over the 512-byte $C_{\text{enc}}[52:564]$; and the *non-compact* d_N over cv^{net} , rk , the tail $C_{\text{enc}}[564:]$ and C_{out} . Folding in every Action’s nf , cmx , cv^{net} , rk and ciphertexts beside the shared flagsOrchard , balance and anchor , the sighash pins the exact bundle: no Action can be transplanted into another transaction nor any field altered without breaking it. The SpendAuthSig under each rk and the binding signature under bvk both sign this single value. The v6 form of the digest appends the Ironwood component digest as a fifth child, d_{Irw} , under revised personalisations, and moves the Sapling, Orchard, and Ironwood anchors from effecting data to authorizing data (§5.1; ZIP-229; `librustzcash zcash_primitives, transaction/sighash_v6.rs`).

2.10 Authorisation of the spend: RedPallas

One capability remains: the spender must prove that they are *authorised* to spend the note — that they hold ask — and must do so without linking this spend to any other spend by the same key. RedPallas, a re-randomisable Schnorr signature, provides exactly this.

It is the Schnorr template of the *Crypto Guide* on Pallas, with hash H^* (BLAKE2b-512 reduced mod p_{Vesta}). The novel ingredient is *key re-randomisation*: the spender draws a randomiser $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ uniformly at random, fresh for every Action (RedPallas’s GenRandom: hash 80 uniform bytes with H^* , which makes α statistically uniform on $\mathbb{F}_{p_{\text{Vesta}}}$), and sets

$$\text{rsk} := \text{ask} + \alpha \in \mathbb{F}_{p_{\text{Vesta}}}, \quad \text{rk} := \text{ak} + [\alpha] G^{\text{SpendAuth}} = [\text{rsk}] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}).$$

The spender publishes the *randomised* validating key rk , not the real ak , and signs with rsk . Because α is fresh per spend, rk is uniformly random and unlinkable across spends, yet remains a valid key for the underlying authority. The circuit enforces that rk is a re-randomisation of the witnessed ak (check A6) and that this ak is the key bound into the note’s address (check A7), so that a valid Action proves the spender knows ask — without ever revealing ak .

2.11 A single shielded payment, end to end

The components now assemble into a transaction. Consider Alice paying Bob 5 ZEC ($v = 5 \cdot 10^8$ satoshi) from a note worth 5 ZEC. The unit of shielded transfer is an *Action*, which simultaneously spends one old note and creates one new note (a deliberately fixed shape that conceals whether a given Action is “really” a spend, an output, or both). This payment is one Action.

1. Bob hands Alice an address. Bob derives a diversified address (d, pk_d) from his ivk (§2.4) and gives it to Alice. It reveals nothing and is unlinkable to his other addresses.

2. Alice builds the Action. Alice owns an old note n_{old} worth 5 ZEC, whose commitment cmx_{old} is a leaf of the tree. She:

- computes its nullifier nf_{old} (§2.7) — she will publish this to retire the note;
- constructs Bob’s new note $n_{new} := (addr_{Bob}, 5 \cdot 10^8, \rho_{new}, \psi_{new}, rcm_{new})$ with $\rho_{new} := nf_{old}$, and its commitment cmx_{new} (§2.5);
- forms the value commitment cv^{net} with $v_{old} - v_{new} = 5 \cdot 10^8 - 5 \cdot 10^8 = 0$ (§2.9);
- selects a randomiser α and forms rk (§2.10);
- reads off a recent anchor and her note’s Merkle path (§2.6).

3. Alice proves the Action statement. She runs the Halo 2 prover on the relation of §2.14, with public inputs (anchor, cv^{net} , nf_{old} , rk , cmx_{new}) and her note, keys, path, and randomness as private witness. The proof attests, in one shot and revealing nothing: the old note is in the tree (A3), its nullifier follows correctly (A5), she is authorised to spend it (A6, A7), the spent and created notes are well-formed (A1, A2), and the value commitment is consistent (A4).

4. Alice assembles and signs the transaction. She encrypts n_{new} to Bob (so that only Bob, via his ivk , can detect and open it; §2.8), attaches a `SpendAuthSig` under rk , and signs the whole bundle with the binding signature under bvk (§2.9), which proves balance; both signatures sign the ZIP-244 transaction sighash, welding the Action into this transaction.

5. The network verifies. Consensus checks: it recognises the anchor; nf_{old} is not already in the nullifier set (no double-spend); the Halo 2 proof verifies; the `SpendAuthSig` verifies under rk ; the binding signature verifies under the recomputed bvk (balance). If all pass, it adds nf_{old} to the nullifier set and appends cmx_{new} as a new tree leaf.

6. Bob receives. Scanning with his ivk , Bob trial-decrypts the ciphertext (§2.8), recovers n_{new} , and now owns a 5-ZEC note he can later spend by repeating the entire procedure. The chain has gained one nullifier and one commitment; an observer learns that *an* Orchard payment occurred and nothing else — not sender, recipient, or amount.

That sequence constitutes the entire protocol. The formal Action statement below is precisely the list of the in-circuit checks step 3 invokes.

2.12 How value enters the pool

The walkthrough moved value already inside the pool. Value *enters* through three doors, each public in amount: the two below, and migration from another shielded pool (§5.4).

Shielding transactions. The instrument is the sign of the balancing value: by the convention of §2.9, a positive $v_{\text{balance}}^{\text{Orchard}}$ is net value leaving the pool, so a *shielding* transaction is one whose transparent inputs fund a negative $v_{\text{balance}}^{\text{Orchard}}$: its Actions spend dummy notes ($v_{\text{old}} = 0$, the anchor check A3 gated off, §2.15) and create real ones, and the binding signature of §2.9 proves the books balance. An observer learns the entering amount — the balancing field is a cleartext transaction field — but not the receiving note or address. The deployed construction path is function `propose_shielding` (`zcash_client_backend`, `data_api/wallet.rs` and `input_selection.rs`): it sweeps the given source addresses’ UTXOs, at zero confirmations under the default policy, and wallets auto-shield transparent receipts to internal keys.

Shielded coinbase (ZIP-213). Since Heartwood, new issuance may enter directly: “[c]oinbase transactions MAY contain Sapling outputs”, extended at NU5 to the Orchard pool and re-scoped at NU6.3 — from then coinbase may carry “Sapling-pool and/or Ironwood-pool outputs”, and “it is only possible to mine to an Orchard address by sending the funds to the *Ironwood pool*” (ZIP-213). Two riders keep this auditable and consistent. Coinbase maturity is amended to bind transparent outputs only (the rule the *Consensus Guide* reads through its theorem, §“Reorg rails, maturity, and the finality floor”), and every shielded coinbase output “MUST have valid note commitments” when recovered under the *all-zero* outgoing viewing key: coinbase shielding is publicly decryptable by construction, so anyone can verify what issuance entered which pool, while the notes spend exactly like any others afterwards.

The ledger of crossings. Every entry and exit is metered by ZIP-209’s pool balances: for each shielded pool, the *chain value pool balance* is the negation of the sum of that pool’s balancing fields over the whole chain (for Sprout, the sum of $v_{\text{pub_old}} - v_{\text{pub_new}}$), and a block is invalid if it would drive any pool balance — shielded, transparent, or the deferred fund (ZIP-2001’s lockbox, a chain value pool accruing a portion of each block subsidy for later disbursement) — negative, or total supply past MAX_MONEY. Entry, transfer, and exit therefore reconcile publicly at pool granularity while individual notes stay hidden; the Ironwood instantiation, and the one-way turnstile that closes the Orchard pool’s entry door entirely, are §5.4’s.

2.13 What a node verifies, without ever observing the note

Note that none of the previous subsection involved the *network*. A validating node holds no viewing key, cannot trial-decrypt, and so never learns the sender, recipient, or amount of any Action. What it *can* do is confirm that the hidden data is internally consistent — and for that it relies entirely on the proof, not on the note.

What the node receives is an Orchard-protocol *bundle*, the unit into which the transaction format groups one pool’s data: the per-Action public fields; one anchor shared by every Action in the bundle; a *single* aggregate Halo 2 proof covering all of the bundle’s Actions; and one `SpendAuthSig` per Action. The declared net flow $v_{\text{balance}}^{\text{Orchard}}$ and the binding signature appear once per bundle, and the flags `enableSpends`, `enableOutputs`, and — from NU6.3 — `enableCrossAddress` (bit 2 of the flags byte, reserved as 0 in v5 transactions before NU6.3) are likewise bundle-level fields. The current carrier is the v6 format (ZIP-229): the v5 format’s Orchard component (ZIP-225) plus a separate Ironwood component holding that pool’s bundle (`flagsIronwood`, `valueBalanceIronwood`, `anchorIronwood`, `proofsIronwood`, `vSpendAuthSigIronwood`, `bindingSigIronwood`; §5.1). Transaction versions v4, v5, and v6 all remain valid from NU6.3 onward, and the Orchard-pool sealing rules of §5.1 bind regardless of version (ZIP-229). Using only the public fields (`anchor`, `cvnet`, `nfold`, `rk`, `cmx`) of each Action, a node checks:

- the bundle’s *Halo 2 proof* of the Action statement (§2.14) — this forces each spent note of nonzero witnessed value to be a genuine tree leaf (a zero-value dummy spend skips the membership check

by A3’s gate, by design), the nullifiers and new commitments to follow correctly, and the keys to align;

- the *anchor* is a Merkle root the node has seen on its accepted main chain (§2.6);
- nf_{old} is *not already* in the nullifier set — the one check that cannot sit inside a single proof, because it concerns the global history (this prevents a double-spend);
- the *spend-authorisation signature* verifies under rk and the *binding signature* under the recomputed bvk (§§2.9–2.10), settling authority and balance.

If every Action passes, the node performs the two state changes of step 5 — recording each nf_{old} , appending each cm_{new} — which permit the next spender to prove membership and make a second spend of the same note fail. Thus the SNARK together with the two signatures enforces a payment’s validity, never the inspection of the note: the node confirms that the accounts balance and that no one forged anything *while learning nothing about the note itself*. This is the precise sense in which the knowledge soundness of the Action SNARK (§3) performs all the work — the network trusts the proof exactly because a real witness extractably backs a valid one.

2.14 The Action statement, formally

Definition 2.9 (Orchard Action statement, Ironwood circuit (NU6.3)). The Action SNARK proves, with public inputs

$$(\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutputs}, \text{disableCrossAddress})$$

and the note, keys, Merkle path, randomiser α , and the value-commitment trapdoor rcv as private witness, the conjunction:

[A1] Old-note commitment integrity — the spent note is well-formed:

$$\text{cm}_{\text{old}} = \text{NoteCommit}_{\text{rcm}_{\text{old}}}(\text{g}_{\text{d}_{\text{old}}}^* \parallel \text{pk}_{\text{d}_{\text{old}}}^* \parallel \text{LE}_{64}(\text{v}_{\text{old}}) \parallel \text{LE}_{255}(\rho_{\text{old}}) \parallel \text{LE}_{255}(\psi_{\text{old}}))$$

— or NoteCommit returns $\perp$, the specification’s escape hatch for Sinsemilla’s incomplete-addition exceptions (the remark following this definition).

[A2] New-note commitment integrity, with $\rho_{\text{new}} := \text{nf}_{\text{old}}$: $\text{Extract}(\text{NoteCommit}_{\text{rcm}_{\text{new}}}(\dots)) \in \{\text{cmx}, \perp\}$ (with $\text{Extract}(\perp) := \perp$) — the created note is well-formed and its uniqueness seed chains to the spent note’s nullifier (Faerie resistance).

[A3] Membership, gated by v_{old} — the spent note is a real leaf: recomputing the Merkle root from $\text{Extract}(\text{cm}_{\text{old}})$ up the witness path of length 32 gives root, and $\text{v}_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$. (The gate by v_{old} permits a zero-value “dummy” spend to skip the membership check, which is how an Action pads with no real input.)

[A4] Net-value commitment integrity — the amounts are consistent and in range: with witnessed (magnitude, sign), $\text{v}_{\text{old}} - \text{v}_{\text{new}} = \text{magnitude} \cdot \text{sign}$, $\text{cv}^{\text{net}} = [\text{v}_{\text{old}} - \text{v}_{\text{new}}]V^{\text{Orchard}} + [\text{rcv}]R^{\text{Orchard}}$, $\text{v}_{\text{old}}, \text{v}_{\text{new}} \in [0, 2^{64})$, $\text{sign} \in \{-1, +1\}$.

[A5] Nullifier integrity — the published nullifier is indeed this note’s:

$$\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}] G^{\text{nf}} + \text{cm}_{\text{old}}).$$

[A6] Spend authority — the published key is a re-randomisation of the note owner’s: $\text{rk} = \text{ak} + [\alpha]G^{\text{SpendAuth}}$.

[A7] Address integrity — the keys, address, and nullifier key all belong to one identity: $\text{ivk} = \perp$, or $\text{ivk} = \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$ and $\text{pk}_{\text{d}_{\text{old}}} = [\text{ivk}]\text{g}_{\text{d}_{\text{old}}}$.

[A8] / [A9] Spend / output gating: $v_{\text{old}} \cdot (1 - \text{enableSpend}) = 0$ and $v_{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$. The transaction builder sets these bundle-level flags; consensus rules constrain them — coinbase transactions, for instance, must set $\text{enableSpend} = 0$ (ZIP-229). Spending Ironwood notes is otherwise permitted; the $\text{enableSpend} = 0$ requirement is coinbase-scoped, not a pool-wide prohibition (protocol specification § 7.1, transaction consensus rules).

[A10] Cross-address gating — when $\text{disableCrossAddress} \neq 0$, the created note pays the spent note’s own receiver: $g_{d_{\text{new}}} = g_{d_{\text{old}}}$ and $pk_{d_{\text{new}}} = pk_{d_{\text{old}}}$ as full curve points, enforced by four product constraints of the A3 shape, one per affine coordinate (Definition 5.4, §5.3, gives the rows, the instance index — the flag is the tenth public input, at index 9 — and the wire encoding). Consensus requires every post-NU6.3 Orchard-pool Action to set the flag (§5.1).

Remark 2.10 (Three circuit versions, one owning ZIP pending). The relation above is the third Action-circuit version: the deployed implementation distinguishes the insecure NU5 original, the NU6.2 correction (ZIP-257, Final), and the NU6.3 Ironwood circuit, and only the last constrains the flag (`OrchardCircuitVersion::supports_cross_address_restriction`, released crate `orchard 0.15.0 src/circuit.rs`). The legacy NU5/NU6 statement is the relation A1–A9 alone, over the nine-element public-input tuple ending at `enableOutputs`. Consensus selects the verifying key by branch (ZIP-258), so from NU6.3 every Action — in either pool, whatever the transaction version — verifies against the Ironwood key. The circuit change’s owning ZIP does not yet exist as text: ZIP-2006 is a Reserved stub, and the flag’s encoding, its consensus rule, and the verifying-key selection are specified in ZIP-229/258 (§5.3 states what remains reconstructable only from the implementation).

A1, A2, and A7 take the specification’s $\perp$ -weakened forms because the circuit cannot exclude Sinsemilla’s incomplete-addition exceptional cases; the relation proved is the disjunction, not the plain conjunction. The weakening costs only a negligible soundness term: producing a $\perp$ case is itself a non-trivial DL relation among the GroupHash generators (the closing sentence of the proof of Proposition 3.1), so no efficient prover exhibits one.

Each of A1–A7 is one entry from the four-requirement map of §2.1: A1–A2 represent notes, A3 proves membership, A5 (with the chain’s nullifier set) prevents double-spending, A4 conserves value, and A6–A7 authorise the spender; A8–A10 stand outside the map as consensus-level gating constraints. This is the relation $\mathcal{R}_{\text{Orchard}}$. Every Action is an instance of it; Halo 2 produces one aggregate proof *per bundle*, covering all of the bundle’s Actions, while each Action carries its own `SpendAuthSig`. §2.15 sketches how the Action circuit arithmetises this relation, and §3 shows how soundness of the relation yields the protocol’s security properties.

2.15 The Action circuit: arithmetisation of the relation

The Action statement of §2.14 is a *relation*; what Alice actually executes is a *circuit* that proves membership in it in zero knowledge. Halo 2 (the *Halo 2 Guide*) proves satisfiability of a fixed PLONKish circuit, so the task of the application layer is to encode the conjunction A1–A10 as one such circuit — the *Action circuit*, identical for every Action — supplied the note, keys, Merkle path, randomiser α , and the value-commitment trapdoor rcv as private witness. We sketch this encoding here; the gate-level details appear in the *halo2 Book* and the *Orchard Book*.

We assemble the circuit from a small set of reusable *gadgets* (“chips”), each a pre-wired block of custom gates and lookups realising one operation. The Orchard designers *chose* the primitives so that these gadgets are inexpensive under PLONKish arithmetisation; this motivates the use of Sinsemilla and Poseidon rather than a bit-oriented hash. How any such gadget is *built* — how its gates, lookups, and cell assignments are declared, and how synthesis turns the filled columns into the polynomials the proof commits to — is the subject of the *Halo 2 Guide*’s section “From statement to circuit: arithmetisation in practice”; here we describe what each gadget enforces.

The ECC gadget. Implements the Pallas group law with the *complete* addition formulas (no edge-case failures; the *Math Guide*), together with variable- and fixed-base scalar multiplication (the for-

mer a double-and-add ladder of incomplete-addition rounds finished by a complete tail, the latter using precomputed fixed-base tables; the j -invariant-0 endomorphism serves only the recursion layer, where it cheapens multiplication by verifier challenges, not this circuit) and on-curve witnessing. It realises every group operation in the statement: $ak = [ask] G^{\text{SpendAuth}}$, the re-randomisation $rk = ak + [\alpha] G^{\text{SpendAuth}}$ (A6), the transmission key $pk_d = [ivk] g_d$ (A7), the net value commitment $cv^{\text{net}} = [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [rcv] R^{\text{Orchard}}$ (A4), and the point $[F_{nk}(\rho) + \psi] G^{\text{nf}} + cm$ inside the nullifier (A5).

The Sinsemilla gadget. Implements Sinsemilla (§2.3) by realising its generator table $P[m]$ as a Halo 2 *lookup* (the lookup argument of the *Halo 2 Guide*): each 10-bit message chunk costs one table lookup and two incomplete additions. On it rest the two composed commitment gadgets the Orchard Book names explicitly — NoteCommit, the in-circuit NoteCommit proving A1 and A2, and Commitlvk, the in-circuit short commitment proving $ivk = \text{Commit}_{r_{ivk}}^{\text{ivk}}(\text{Extract}(ak), nk)$ in A7 — each bundling the Sinsemilla rounds with the bit-decomposition and canonicity checks of their field inputs.

The Merkle-path gadget. Stacks 32 layer-tagged MerkleCRH hashes (themselves Sinsemilla short hashes, §2.6); at each layer a conditional swap orders the running node and its witnessed sibling by one bit of the leaf position, and the gadget asserts the recomputed root equal to the public anchor — check A3, gated by v_{old} so that a dummy spend may skip it.

The Poseidon gadget. Implements the Poseidon permutation (*Crypto Guide*), whose only non-linearity is $x \mapsto x^5$, to evaluate the nullifier PRF $F_{nk}(\rho) = \text{PoseidonHash}(nk, \rho)$ cheaply in-circuit; its output feeds the ECC gadget for A5.

Decomposition, range, and boolean gadgets. Lookup-based gadgets split field elements into the bit-chunks the other gadgets consume and range-check values: that $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$ and $\text{sign} \in \{-1, +1\}$ (A4), and that point and field encodings are canonical. (The flags `enableSpends`, `enableOutputs`, and `enableCrossAddress` (bit 2) are verifier-supplied public inputs — ZIP-229's flags bytes, `flagsOrchard` and `flagsIronwood`, fix them to bits outside the circuit — and enter the circuit only through the plain-gate products of A8–A10, the third negated as the Ironwood circuit's tenth public input `disableCrossAddress`, §5.3.)

Structure of the proof. These gadgets compose into a single fixed circuit whose native field is the Pallas base field $\mathbb{F}_{p_{\text{Pallas}}}$ — the field in which Pallas coordinates reside, so that the ECC gadget runs natively — proved with the Halo 2 inner-product argument over the Pasta cycle. The circuit uses $2^{11} = 2048$ rows (the *Halo 2 Guide*'s circuit-size exponent $k = 11$ — not to be confused with Sinsemilla's chunk width $k = 10$, Construction 2.1), with ten advice columns beside the fixed columns and the Sinsemilla lookup table (the *Halo 2 Guide*). Every Action, whatever it spends or creates, is one instance of this same circuit; a bundle's Actions are proved together in one aggregate proof that a node checks (§2.13) from the public inputs alone, and batch verification — the deferred linear MSMs merged across proofs (the *Halo 2 Guide*) — lets a node verify many such proofs at amortised cost approaching that of a single verification.

3 End-to-end security: derivation of Orchard's properties from the SNARK

The Halo 2 SNARK on the Action relation, together with the algebraic structure of the Orchard primitives, produces the security properties that render shielded payments functional. We establish those connections below.

3.1 Guarantees provided by a valid Action proof

A SNARK π accepted on public inputs (anchor, cv^{net} , nf_{old} , rk , cmx , ...) testifies, with overwhelming probability, that the prover knows a witness satisfying A1–A10. By knowledge soundness of Halo 2 (the *Halo 2 Guide*), an extractor can recover this witness from any successful prover. The soundness-side properties below (nullifier uniqueness, spend authority, balance) each follow from one or more conjuncts of A1–A10 plus the algebraic properties of the primitives; §3.2 first supplies the primitive reduction those arguments share, and privacy rests instead on the zero knowledge of the SNARK (§3.6) together with the DDH, Poseidon-PRF and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.2 Reduction of Sinsemilla collision-resistance to DL on Pallas

Proposition 3.1 (Sinsemilla CR $\Leftarrow$ DL). *A collision $M \neq M'$ between messages of the same chunk count ℓ with $\text{Sinsemilla}_D(M) = \text{Sinsemilla}_D(M')$ yields a non-trivial DL relation among the generators $\{P[0], \dots, P[2^k - 1]\}$.*

Sketch. Unfolding the Sinsemilla update (in the case no incomplete-addition exception occurs), with ℓ the number of k -bit chunks:

$$\text{Sinsemilla}_D(M) = 2^\ell \cdot Q + \sum_{i=1}^{\ell} 2^{\ell-i} \cdot P[m_i].$$

A collision gives $\sum_i 2^{\ell-i}(P[m_i] - P[m'_i]) = 0$, i.e. a non-trivial linear relation over $\mathbb{F}_{p_{\text{Vesta}}}$ on the generators. Since hash-to-curve modelled as a random oracle samples these independently, finding such a relation is the multi-generator discrete-log relation problem, and no efficient adversary produces one without solving DL (the *Crypto Guide* gives this reduction in its Pedersen-hash collision analysis). For differing chunk counts $\ell \neq \ell'$, the unfolded equality instead gives $(2^\ell - 2^{\ell'})Q + \sum_i \dots = 0$ with $2^\ell \not\equiv 2^{\ell'} \pmod{p_{\text{Vesta}}}$ for all admissible lengths, i.e. a non-trivial DL relation among $\{Q\} \cup \{P[0], \dots, P[2^k - 1]\}$; since Q is itself a GroupHash output modelled as an independent random generator, this case reduces to DL identically. The incomplete-addition exceptions are themselves non-trivial DL relations among the same generators and so also negligible. $\square$

Sinsemilla CR is the primitive ingredient for note-commitment binding: two distinct notes cannot share a commitment, so A1 and A2 are genuine integrity statements about cm_{old} and cm_{new} . Combined with knowledge soundness, this entails that the prover knows the specific opening of each commitment.

3.3 Nullifier uniqueness

Proposition 3.2 (Nullifier uniqueness). *Two distinct notes $n \neq n'$ cannot share a nullifier except with probability negligible in the DL hardness of Pallas (treating Poseidon outputs as random scalars under the PRF assumption).*

Sketch. A collision $\text{nf}(n) = \text{nf}(n')$ in x -coordinate gives $[F_{\text{nk}}(\rho) + \psi]G^{\text{nf}} + \text{cm} = \pm([F_{\text{nk}}(\rho') + \psi']G^{\text{nf}} + \text{cm}')$, a non-trivial linear relation among G^{nf} , cm , cm' . Expanding cm , cm' via their known openings (extracted by knowledge soundness, or supplied by the adversary who forged the notes) over the Sinsemilla generators and the blinding base H_D turns this into a known non-trivial DL relation among the independent GroupHash outputs $\{G^{\text{nf}}, Q, P[0], \dots, P[2^k - 1], H_D\}$ — distinctness of the notes forces some coefficient mismatch — and producing such a relation reduces to DL on Pallas as in Proposition 3.1. $\square$

A5 (nullifier integrity) plus knowledge soundness ensures that nf_{old} in the public inputs is genuinely the nullifier of some specific committed note, not an arbitrary value. The chain's duplicate-nullifier rejection then prevents double-spending.

3.4 RedPallas unforgeability and re-randomisation

Proposition 3.3 (RedPallas EUF-CMA $\Leftarrow$ DL). *RedPallas is EUF-CMA-secure assuming DL on Pallas in the random-oracle model. The re-randomisation extension preserves security: a forger that, given ak and signatures under re-randomised keys, produces a valid signature under $rk = ak + [\alpha] G^{\text{SpendAuth}}$ for an α of its own (possibly ak -dependent) choice yields a DL solver for ak .*

Sketch. The standard forking argument (the *Crypto Guide*, via the Bellare–Neven general forking lemma) reduces a Schnorr forger to a DL adversary, and it covers re-randomisation because RedPallas is key-prefixed: the challenge $H^*(R \parallel rk \parallel m)$ hashes the verification key, so response-shifting cannot transport signatures between keys. The reduction embeds the DL challenge as $ak := X$. It answers signing queries under any $rk_i = ak + [\alpha_i] G^{\text{SpendAuth}}$ without knowing any secret, via the honest-verifier zero-knowledge simulator with random-oracle programming: sample c, s uniformly, set $R := [s] G^{\text{SpendAuth}} - [c] rk_i$, and program $H^*(R \parallel rk_i \parallel m) := c$, aborting on programming collisions exactly as in the *Crypto Guide*'s Schnorr EUF-CMA theorem. It then forks the adversary at the forgery's critical query $H^*(R^* \parallel rk^* \parallel m^*)$, obtaining two accepting transcripts that share R^* — and, since both forks share the critical query input, the same rk^* , hence the same α^* — with distinct challenges; 2-special soundness extracts rsk^* , the discrete logarithm of rk^* , and the output $ask = rsk^* - \alpha^*$, with α^* the adversary-supplied randomiser, solves DL for ak . Separately, and only for honest signer-chosen uniform α : sampling α at random and back-deriving $ak := rk - [\alpha] G^{\text{SpendAuth}}$ reproduces the view of (ak, α, rk) exactly, so honest re-randomisations are distributed as plain Schnorr keys — the unlinkability statement of the *Crypto Guide*, which does not by itself cover adversary-chosen α . $\square$

A6 (spend authority) plus knowledge soundness ensures the prover knows α relating rk to ak ; combined with the SpendAuthSig under rk at the transaction layer, this enforces that the spender knows ask corresponding to ak , hence to the address pk_{old} via A7.

3.5 Balance preservation via the Pedersen homomorphism

The combination of A4 (per-Action value commitment integrity), the Pedersen homomorphism on cv^{net} , and the binding signature on bvk gives end-to-end balance:

Proposition 3.4 (Balance). *Any accepted Orchard bundle has $\sum_i (v_{\text{old},i} - v_{\text{new},i}) = v_{\text{balance}}^{\text{Orchard}}$ except with probability negligible in the DL hardness of the Pasta curves in the random-oracle model (Vesta for the knowledge soundness of the Action SNARK supplying the A4 openings; Pallas for the binding-signature extraction and the independence of the value-commitment bases).*

Sketch. By A4 and knowledge soundness, each $cv^{\text{net},i}$ commits to the true per-Action delta $v_{\text{old},i} - v_{\text{new},i}$. The bundle sum $\sum_i cv^{\text{net},i}$ equals $[\sum_i (v_{\text{old},i} - v_{\text{new},i})] V^{\text{Orchard}} + [\sum_i rcv_i] R^{\text{Orchard}}$ by homomorphism. Consensus computes bvk and checks the binding signature; a valid binding signature is a Fiat–Shamir proof of knowledge of $\log_{R^{\text{Orchard}}} bv_k$, so the forking extraction in the proof of Proposition 3.3 recovers bsk with $bvk = [bsk] R^{\text{Orchard}}$; combined with the openings extracted via A4, a non-zero V^{Orchard} -component of bvk would yield a discrete-log relation between the independent bases V^{Orchard} and R^{Orchard} , contradicting DL — hence the value-component equals $[v_{\text{balance}}] V^{\text{Orchard}}$ and the total value flow matches the declared transparent balance. $\square$

This is the fundamental conservation property: shielded transactions cannot mint zatoshi out of nothing.

3.6 Zero knowledge of the SNARK

Halo 2 is statistical zero knowledge in the random-oracle model (the *Halo 2 Guide*'s zero-knowledge analysis). The simulator on input the instance $(\text{anchor}, cv^{\text{net}}, nf_{\text{old}}, rk, cmx, \dots)$ operates as follows:

1. Sample the Fiat–Shamir challenges $\theta, \beta, \gamma, y, x, x_1, x_2, x_3, x_4$ and the IPA challenges $u_1, \dots, u_k$ ($k = 11$ rounds, the circuit-size exponent) uniformly, programming the random oracle to return these on the relevant transcript prefixes.
2. Sample the final IPA scalar a uniformly.
3. Use the freedom in the blinding terms ℓ_j, r_j (added at each IPA round) and the polynomial blinders (added to each committed prover polynomial: the advice columns in phase 1, the running products in phase 3, and the quotient chunks in phase 4) to satisfy the verifier's equations algebraically without knowing the witness.

The resulting transcript is statistically indistinguishable from a real proof; the only gap from perfect ZK is the random-oracle programming, which a verifier that observes only the transcript cannot detect.

ZK supplies the proof's share of Orchard's privacy: the SNARK transcript reveals nothing beyond the public inputs of each Action. That the published data leak nothing further is a separate, computational matter: the nullifier — itself a public input (§2.7) — hides its note under the PRF security of Poseidon, and the epk and ciphertexts published alongside the public inputs (§2.8) hide recipient and value under the DDH and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.7 Composition of end-to-end soundness

The full SNARK soundness composes three ingredients:

- *PIOP soundness.* Schwartz–Zippel error per polynomial-identity check, summed across constraints. For $\mathbb{F} = \mathbb{F}_{p_{\text{Pallas}}}$ with $|\mathbb{F}| \approx 2^{254}$ and constraint degree $\leq d_{\text{max}}$, the error per check is $\leq d_{\text{max}} n/|\mathbb{F}| \approx d_{\text{max}} \cdot 2^{-243}$ (the gate polynomial composes degree- d_{max} gates with column polynomials of degree $n-1$, $n = 2^{11}$; a cheating prover's recombined quotient $h \cdot Z_H$ has degree $< d_{\text{max}} n$), overwhelmingly small.
- *PCS extractability.* Reduces to DL on the IPA curve (the *Halo 2 Guide*), with the accumulation analysis (the *Halo 2 Guide*) covering the deferred-MSM batching by which a node amortises verification across proofs (§2.15); the same analysis would carry soundness across recursion, which Orchard's deployment does not use.
- *Fiat–Shamir soundness in the ROM.* The *Crypto Guide*'s FS theorem covers Σ -protocols, losing roughly a factor Q in the prover's RO-query budget; for the $\log d$ -round Halo 2 transcript the naive multi-round analysis loses $q^{O(\log d)}$ (the *Halo 2 Guide*), a bound vacuous at $Q = 2^{80}$. The negligible-error conclusion rests on the tighter treatments the *Halo 2 Guide* presents: BCMS20's direct ROM+DL analysis, or Ghoshal–Tessaro's tight Fiat–Shamir bound in the AGM.

Each step's soundness error is negligible at Orchard's parameters ($|\mathbb{F}| \approx 2^{254}$, $\lambda = 127$ under the group-order- $\approx 2^{2\lambda}$ convention; effectively ≈ 126 bits after the automorphism speedups, the *Crypto Guide*); the union bound across steps is itself negligible. Combined with the application-level reductions (Propositions 3.1–3.4), the full chain gives:

Theorem 3.5 (Orchard end-to-end security, informal). *Under DL on both Pasta curves (Pallas for the application-level primitives, Vesta for the IPA commitment) in the random-oracle model — and, for property (iv), additionally DDH on Pallas, the PRF security of Poseidon (§2.7), and the KDF/AEAD securing the note ciphertext (§2.8) — no efficient adversary can (i) produce an accepted Action without knowing a witness satisfying A1–A10 — in particular, any Action whose witnessed v_{old} is nonzero must spend a genuine leaf of the commitment tree (zero-value dummy spends, A3's gate, are accepted by design), (ii) cause two Actions to share a nullifier without spending the same note twice, (iii) cause a bundle's declared transparent balance $v_{\text{balance}}^{\text{Orchard}}$ to differ from its true net value flow $\sum_i (v_{\text{old},i} - v_{\text{new},i})$*

— in particular, to exceed it, which would create value (Proposition 3.4), or (iv) link a published Action to any specific recipient or value beyond what is explicit in the public inputs.

The four properties are, respectively, soundness of the Action SNARK, nullifier uniqueness, balance preservation, and zero knowledge. The first three reduce (informally) to DL on the Pasta curves in the ROM (Pallas for the application-level reductions, Vesta for the IPA extraction). For the fourth (privacy), the zero knowledge of the SNARK transcript is statistical — given the perfectly-hiding Pedersen and Sinsemilla blinders and the random-oracle programming, it does not rest on DL — but full unlinkability of an Action additionally rests on the DDH assumption on Pallas and the PRF security of Poseidon (§2.7) and on the KDF/AEAD securing the note ciphertext (§2.8), and is therefore computational. The asymmetry within the proof itself is typical of the discrete-log SNARK family: soundness is computational (a cheating prover is only *bounded*, not impossible), whereas the zero knowledge of the transcript is information-theoretic.

4 The Zcash proof-system landscape

This section closes with a survey of the proof systems Zcash has used across its three generations of shielded payments. Concrete trade-offs drove each transition: trusted setup risk, recursion compatibility, post-quantum posture, and on-chain efficiency.

4.1 Sprout (2016–present, deprecated)

The original shielded protocol used a *BCTV14* SNARK (Ben-Sasson, Chiesa, Tromer, Virza) over the pairing-friendly curve BN254, with constant-size proofs (296 bytes) and constant verification time. Its parameters came from a per-circuit trusted setup: the 2016 ceremony, a six-party multi-party computation that generated the BCTV14 proving and verification keys for the fixed Sprout circuit, sound provided at least one participant destroyed their secret share. Sprout demonstrated that zk-SNARK shielded payments were feasible at scale, but it remained a transitional design: the prover was slow, and the circuit hashed with SHA-256 — a bit-oriented hash whose boolean operations are non-native to arithmetic constraints, hence costly in-circuit. Decisively, a flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; the pool was deprecated and shielded Sprout value was migrated to later pools.

4.2 Sapling (2018–present)

Sapling (the second network upgrade) replaced this stack with Groth16 — 192-byte proofs, three-pairing verification — over BLS12-381, with the embedded curve Jubjub serving the in-circuit group operations and a Pedersen hash serving the Merkle tree. The result was a far faster prover, a smaller proof, and cheaper in-circuit hashing. Two structural limitations remained. Groth16 still requires a per-circuit trusted setup — the 2018 Sapling MPC, whose universal first phase was the Powers of Tau ceremony with many participants — and no known elliptic-curve cycle extends BLS12-381, so the proving system admits no efficient recursion.

4.3 Orchard (2022–present, NU5; Ironwood pool from NU6.3)

Orchard (NU5) replaced the Sapling stack with the Halo 2 / Pasta stack developed in the *Halo 2 Guide*. The principal choices are:

- Halo 2: PLONKish PIOP, IPA-PCS, accumulation-friendly. No trusted setup.
- Pasta cycle: Pallas as the application curve, with Vesta as the proving curve in the IPA layer. Designed for recursion.

- Sinsemilla hash: an algebraic hash whose collision resistance reduces to DL on Pallas — an assumption the protocol already requires elsewhere — and whose chunk-table structure maps directly onto Halo 2’s lookup argument (§2.3).
- Poseidon: serves the nullifier PRF (§2.7), where a keyed in-circuit PRF rather than a CRH is required. Unlike Sinsemilla, its security rests on cryptanalytic confidence in algebraic-attack resistance rather than on a hardness assumption such as DL.

The shift followed a consistent design principle: “transparent SNARK, recursion-ready” constituted the design target, and the designers engineered the Pasta cycle to fit.

The stack has since carried one correction and one split, neither touching its cryptography. The Action circuit exists in three versions — NU5’s original, the NU6.2 emergency correction (ZIP-257, Final), and the NU6.3 Ironwood circuit with the cross-address input (§5.3) — named `InsecurePreNu6_2`, `FixedPostNu6_2`, and `PostNu6_3` in the released crate `orchard 0.15.0` (`src/circuit.rs`; the `bef8a27` dev tree names the third variant Ironwood). And from NU6.3 the one protocol carries two value pools, the sealed Orchard pool and the current Ironwood pool (ZIP-229, Terminology; §5.1). The Halo 2 / Pasta stack is unchanged throughout.

4.4 Comparative summary

	Sprout	Sapling	Orchard
SNARK	BCTV14	Groth16	Halo 2
Curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
Trusted setup	Per-circuit (six-party MPC)	Per-circuit (large MPC)	None
Recursion-ready	No	No	Yes (via Pasta cycle)
Proof size	296 B	192 B	~5 kB (one Action)
Verification time	Constant (pairing)	Constant (3 pairings)	$O(\log n)$ amortised

Table 1: Three generations of Zcash shielded-payment proof systems. Orchard verification: $O(\log n)$ amortised ($n = 2^{11}$ rows; per-proof succinct checks, the linear MSM batched across proofs — the *Halo 2 Guide*). The Orchard column covers three circuit versions (the NU5 original; the NU6.2 correction, ZIP-257, Final; the NU6.3 Ironwood circuit with the cross-address input) and, from NU6.3, two value pools (§5.1).

The proof size grew between Sapling and Orchard; this is the cost of transparency (a one-Action bundle’s proof occupies $2720 + 2272 = 4992$ bytes in the v5/v6 encodings, including the $2 \log_2 2^{11} = 22$ group elements of the IPA rounds). The benefit, beyond the removal of trusted setup, is the foundation for recursion: future generations (Tachyon, eventual designs that aggregate many transactions under a single succinct proof) can build on Orchard’s foundation without revisiting the cryptographic stack.

5 The Ironwood pool

The NU6.3 network upgrade (ZIP-258, Draft) closes the story this volume has told so far and opens its successor: it seals the value pool that has carried Orchard payments since NU5 and starts a second pool — *Ironwood* — running the same cryptography. ZIP-229 (Draft) fixes the vocabulary this section uses throughout. The *Orchard protocol* is the shared design: the Pasta curves, Sinsemilla, the Action circuit (as modified by ZIP-2006), the note, commitment, nullifier, and key constructions, and the note encryption (as modified by ZIP-2005) — everything the preceding sections constructed. The *Orchard pool* and the *Ironwood pool* are two value pools of that one protocol, each with its own note commitment tree, anchor, and chain value pool balance. Nothing in the preceding sections is discarded; what changes is which pool new value may enter, and how an Ironwood note derives its trapdoor.

As in §6, claims about deployed behaviour cite the implementation and the specification — here the released crate orchard 0.15.0 (the version `librustzcash` pins; the Ironwood note-format, note-encryption, and v6-digest work ships in it), the orchard git tree at `bef8a27` (a 0.14.0 dev tree pre-dating the release, cited for the circuit and cross-address work it carries), `librustzcash` at commit `e30517e4`, and `protocol.tex` and the ZIPs at `zips` commit `69610984` — and forward-looking claims carry one of the three epistemic bins *specified*, *designed but unspecified*, or *open problem*.

5.1 Two pools, one protocol: activation and framing

Activation. NU6.3 carries consensus branch identifier `0x37A5165B`. ZIP-258 now fixes activation at height 4,134,000 on Testnet and 3,428,143 on Mainnet, agreeing with `zebra-chain/src/parameters/constants.rs` and `zcash_protocol/src/consensus.rs`. It also fixes the minimum network protocol version at 170160 on both networks (ZIP-258 at `zips` `ad30e59`; ZIP-204 records that NU6.3 is the last upgrade for which the two networks may share a protocol version). From activation, three consensus rules seal the Orchard pool to *spend-only*, and they bind every transaction regardless of version:

- a coinbase transaction must carry an empty Orchard component;
- every Orchard-pool Action must have cross-address transfers disabled (`enableCrossAddress = 0`, §5.3) — enforced by the Action-circuit *verifying key*, which consensus selects by branch, so a v5 transaction cannot bypass the rule;
- no new value may enter the pool: $v_{\text{balance}}^{\text{Orchard}} \geq 0$ per transaction (§5.4).

All three are ZIP-258 rules; ZIP-229 states the second’s version-independence. Notably, `zcashd` is not expected to implement NU6.3 at all; `zebra` is expected to be the only full-validator implementation from activation onward (ZIP-258).

Keys are protocol-scoped. Orchard spending-key and viewing-key material grants authority to spend or view notes in *both* pools; a receiver, and the corresponding *ivk*, is scoped to the Orchard protocol, not to a pool (ZIP-229; ZIP-326). The entire key ladder of §2.4 therefore survives unchanged: an existing wallet’s keys and addresses work in Ironwood as they stand. What distinguishes the pools is state, not identity: separate, independent note commitment trees and nullifier sets (ZIP-229), separate anchors, and separate chain value pool balances. The Ironwood tree starts empty at activation and uses the identical depth-32 MerkleCRH^{Orchard} construction of §2.6 (capacity 2^{32} , ZIP-258); the Orchard tree keeps growing after activation — every Orchard-pool Action still appends a *cmx*, since spends under the cross-address restriction are paired with fabricated or dummy outputs (§5.3) — and its anchors remain live for spend proofs. Nullifier derivation itself (§2.7) is unchanged in both pools; a note’s nullifier is checked only against its own pool’s set.

The v6 transaction format. A v6 transaction (`V6_TX_VERSION = 6`, version group `0xD884B698`; `librustzcash/zcash_protocol/src/constants.rs`) is the v5 format — with `enableSpendsOrchard/enableOutputsOrchard` renamed to `enableSpends/enableOutputs` and bit 2 of `flagsOrchard` now carrying `enableCrossAddress` — plus an Ironwood component that reuses the `OrchardAction` encoding byte-for-byte (820 bytes per Action; ZIP-229):

Value conservation runs per pool: the Ironwood component has its own balancing value $v_{\text{balance}}^{\text{Ironwood}}$ and its own binding signature, constructed exactly as in §2.9 over the Ironwood Actions’ cv^{net} values. ZIP-229 states the note-level situation exactly: every Ironwood-pool output note uses the quantum-recoverable note plaintext format (lead byte `0x03`, §5.2), no Orchard-pool output note does, and this is “the only note-level distinction between the two pools”.

Field	Size (bytes)	Content
nActionsIronwood	compactSize	Action count n
vActionsIronwood	$820 \cdot n$	OrchardAction encoding
flagsIronwood	1	same bit layout as flagsOrchard
valueBalanceIronwood	8	net flow $v_{\text{balance}}^{\text{Ironwood}}$
anchorIronwood	32	a root of the <i>Ironwood</i> tree
sizeProofsIronwood	compactSize	length of proofsIronwood
proofsIronwood	$2720 + 2272 \cdot n$	aggregate Halo 2 proof
vSpendAuthSigsIronwood	$64 \cdot n$	one SpendAuthSig per Action
bindingSigIronwood	64	the pool's binding signature

Table 2: The Ironwood component of a v6 transaction (ZIP-229). The fields after the count are present iff $n > 0$. The proof size formula is the same $2720 + 2272n$ as the Orchard component (Table 1).

The v6 digest. The ZIP-244 tree of §2.9 gains an Ironwood branch,

$$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{trans}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}} \parallel d_{\text{Irw}}),$$

with new personalisations ZTxIdIronwd_H_v6, ZTxIdIrnActCH_v6, ZTxIdIrnActMH_v6, ZTxIdIrnActNH_v6, ZTxAuthIrnwdH_v6 and revised v6 personalisations for the Sapling and Orchard branches (ZTxIdSSpendNH_v6, ZTxAuthSapliH_v6, ZTxIdOrchardH_v6, and ZTxAuthOrchaH_v6); an absent component hashes the empty string under its personalisation (ZIP-229). This digest tree is deployed: the released crate orchard 0.15.0 (the `librustzcash` pin) defines the seven Orchard and Ironwood_v6 personalisations and selects them per pool and transaction version (`bundle/commitments.rs`, `enum BundleCommitmentFormat`), and `librustzcash` at `e30517e4` assembles the v6 digests in production: `digest_orchard` and `digest_ironwood` in `zcash_primitives/transaction/txid.rs` call `Bundle::commitment`, the file itself defines only the two Sapling personalisations `ZTxIdSSpendNH_v6` and `ZTxAuthSapliH_v6` (lines 44 and 54), and `transaction/tests.rs` re-derives the empty-component digests as test vectors. The `bef8a27` dev tree predates this work: there `bundle/commitments.rs` (:7--11) defines only the five non-versioned personalisations (`ZTxIdOrchardHash`, `ZTxIdOrcActCHash`, `ZTxIdOrcActMHash`, `ZTxIdOrcActNHash`, `ZTxAuthOrchaHash`), with no per-pool or per-version dispatch. One structural change rides along: v6 moves the Sapling, Orchard, and Ironwood *anchors* from effecting data (under the `txid`) to authorizing data (under the auth digest, committed only when the component spends). A transaction can then be re-anchored — its membership proofs rebased onto a later root — without changing its `txid`, a design ZIP-229 credits to Penumbra.

One circuit, one verifying key. Both pools share the post-NU6.3 Action circuit and its keys. In the released orchard 0.15.0 a `BundleVersion` pairs a `ValuePool` (Orchard or Ironwood) with a `ProtocolVersion` (`InsecureV1`, `V2`, or `V3`): `circuit_version()` maps `V3` to `OrchardCircuitVersion::PostNu6_3` for both pools, `note_version()` maps Orchard to `V2` and Ironwood to `V3`, and `permits_cross_address_transfers()` is false exactly for (`V3`, Orchard) (`src/lib.rs`, `src/bundle.rs`). The in-circuit gate is `supports_cross_address_restriction()` on `OrchardCircuitVersion`, true only for `PostNu6_3` (`src/circuit.rs`); the `bef8a27` dev tree's `BundleFormat` enum and Ironwood circuit-version name were reworked into these before release. The pools are distinguished by their trees, nullifier sets, balances, and component position — not by separate circuits.

5.2 The Ironwood note: lead byte 0x03 and the hashed trapdoor

An Ironwood note is the same tuple $n = (\text{addr}, v, \rho, \psi, \text{rcm})$ as Definition 2.5, committed by the same `NoteCommit` of §2.5; ZIP-2005 carries the pool distinction on the note *plaintext*, through its lead byte. What changes is that lead byte and, through it, the derivation of `rcm` (ZIP-2005, Proposed;

activation bound to NU6.3). The format is specified — ZIP-2005 § 4.7.3, landed in `protocol.tex` at `zips commit 69610984` — and deployed in the released orchard 0.15.0 (the `librustzcash` pin): `enum NoteVersion (V2/V3)` carries the lead byte, `Note::from_parts` takes it as a fifth argument, and `Note::rcm()` dispatches on it (`src/note.rs`). The `bef8a27` dev tree predates the release: its `Note::from_parts (src/note.rs:182)` still takes the four components (recipient, v, ρ , $rseed$) with no version tag, and its `Note::rcm()` is the legacy single-tag trapdoor of Remark 5.3.

Definition 5.1 (Allowed lead bytes, pool-parameterised). Let `pool` be the chain value pool of a note received at block height h — one of `SaplingPool`, `OrchardPool`, or `IronwoodPool` — and let h_{Canopy} be the activation height of the Canopy network upgrade, which deployed ZIP-212’s $rseed$ -derived note plaintext format and its `0x02` lead byte. Then

$$\text{allowedLeadBytes}^{\text{pool}}(h) := \begin{cases} \{0x01\}, & h < h_{\text{Canopy}}, \\ \{0x01, 0x02\}, & h_{\text{Canopy}} \leq h < h_{\text{Canopy}} + \text{ZIP212GracePeriod}, \\ \{0x02\}, & \text{afterwards, if pool} \neq \text{IronwoodPool}, \\ \{0x03\}, & \text{otherwise.} \end{cases}$$

For Ironwood-pool note plaintexts the only allowed lead byte is `0x03` (ZIP-2005 § 3.2.1 change; landed in `protocol.tex` at `zips commit 69610984`). Ironwood coinbase outputs must carry it as well, the ZIP-213 analogue of the Sapling/Orchard `0x02` rule (ZIP-258).

Definition 5.2 (Ironwood note derivations). For a lead-byte-`0x03` note with seed $rseed$ and $\rho := nf_{\text{old}}$, the sender derives, *in this order*,

$$\begin{aligned} \text{esk} &:= \text{ToScalar}(\text{PRFexpand}_{rseed}(0x04 \parallel \rho)) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \psi &:= \text{ToBase}(\text{PRFexpand}_{rseed}(0x09 \parallel \rho)) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ \text{rcm} &:= \text{ToScalar}(\text{PRFexpand}_{rseed}(\text{pre_rcm})) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{pre_rcm} &:= 0x0B \parallel g_d^* \parallel pk_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{256}(\rho) \parallel \text{LE}_{256}(\psi), \end{aligned}$$

each field in its canonical byte encoding: the points star-encoded (32 bytes each), v as 8 and ρ, ψ as 32 little-endian bytes, so `pre_rcm` is a 137-byte string (ZIP-2005 § 4.7.3 change; `protocol.tex` at `zips commit 69610984`, where the hash is named $H_{rseed}^{\text{rcm}, \text{Orchard}}$).

The `esk` and ψ derivations are byte-identical to §2.8; only `rcm` changes, and with it the order — `rcm` now comes last because its input includes ψ (ZIP-2005). The same g_d^*, pk_d^* encodings feed both `pre_rcm` and the `NoteCommit` input of §2.5. The purpose is post-quantum *binding*: every note field now traverses BLAKE2b on the way to the trapdoor, so an adversary constrained to treat `PRFexpand` as a random oracle cannot vary any field without changing `rcm` (§6.4, which owns the security analysis, the recovery statement, and ZIP-2005’s specified-but-undeployed `qsk/qk` key branch). Every Ironwood-pool output note is a recoverable note; no Sapling or Orchard-pool output note is (ZIP-2005).

Remark 5.3 (The legacy derivation, historically). Lead-byte-`0x02` notes — all Orchard-pool notes, before and after NU6.3 — derive `rcm := ToScalar(PRfExpandrseed(0x05  $\parallel$   $\rho$ ))` as in §2.8. The specification packages both under one dispatcher, `Derive_rcmrseedOrchard(leadByte, ...)`, selecting `0x05  $\parallel$   $\rho$` for `0x02` and the hashed `pre_rcm` for `0x03` (ZIP-2005 § 4.7.3 change). The dev-tree `Note::rcm()` at `bef8a27` implements only the legacy branch above (`src/note.rs:132--136`), with `PrfExpand::ORCHARD_RCM = 0x05` (`zcash_spec 0.2.1`, `src/prf_expand.rs:88`), no version dispatch, and no `0x03`/hashed-`pre_rcm` branch. The released orchard 0.15.0 that `librustzcash` pins adds the `NoteVersion` dispatch — `V2` to `rcm_v2`, `V3` to `rcm_v3` over the `0x0B`-separated `pre_rcm (src/note.rs)` — deploying

the derivation of Definition 5.2.

Decryption is reordered. Trial decryption (§2.8) rederives the note’s quantities to check the commitment, and the lead-byte-0x03 rcm depends on g_d , pk_d , and ψ ; the specification therefore reorders both the ivk and fvk decryption procedures to recover g_d and pk_d first, then ψ , then rcm (ZIP-2005 §§4.20.2–4.20.3 changes). The routing is deployed: the released orchard 0.15.0 routes the two lead bytes to two domains, OrchardDomain (0x02) and IronwoodDomain (0x03) (src/note_encryption.rs), which zcash_client_backend scanning consumes (src/scanning.rs); the bef8a27 dev tree still has a single 0x02-only OrchardDomain. Because keys are protocol-scoped (§5.1), the *same* ivk trial-decrypts both pools — one key, two domains (ZIP-326; zcash_client_backend src/scanning.rs). The light-client surface follows suit: compact blocks gain an ironwoodActions list (the same CompactOrchardAction message) and an ironwoodCommitmentTreeSize, and the synchronisation of §2.8 runs per pool against its own tree (zcash_client_backend proto/compact_formats.proto).

5.3 The cross-address restriction in the Orchard pool

The rule that seals the Orchard pool most deeply is in-circuit: from NU6.3, every Orchard-pool Action must have cross-address transfers disabled, so that value inside the pool can no longer move between addresses — an Orchard-pool spend can pay only its own receiver. The motivation is the recoverability guarantee of §6.4: legacy 0x02 notes are unrecoverable, so consensus stops their circulation and funnels value out of the pool, while still letting owners spend (§5.4).

Definition 5.4 (Action statement, the A10 constraint in full). The tenth public input of Definition 2.9, disableCrossAddress, sits at instance index 9 (after anchor, the two cv^{net} coordinates, nf_{old} , the two rk coordinates, cmx, enableSpends, enableOutputs), and constraint A10 of the relation is, writing $\delta := \text{disableCrossAddress}$:

[A10] Cross-address gating — when $\delta \neq 0$, the new note’s receiver equals the spent note’s:

$$\begin{aligned} \delta \cdot (x(g_{d_{old}}) - x(g_{d_{new}})) &= 0, & \delta \cdot (y(g_{d_{old}}) - y(g_{d_{new}})) &= 0, \\ \delta \cdot (x(pk_{d_{old}}) - x(pk_{d_{new}})) &= 0, & \delta \cdot (y(pk_{d_{old}}) - y(pk_{d_{new}})) &= 0, \end{aligned}$$

four coordinate equalities forcing $g_{d_{old}} = g_{d_{new}}$ and $pk_{d_{old}} = pk_{d_{new}}$ as full curve points — equality of receivers, not merely of x -coordinates. The constraint shape is the product gate A3 already uses ($v_{old} \cdot (\text{root} - \text{anchor}) = 0$), reused on four extra rows of the same $q_{orchard}$ gate (crate orchard at bef8a27 src/circuit.rs: instance offsets ANCHOR through DISABLE_CROSS_ADDRESS, rows assigned in synthesize_cross_address_checks; the historical nine-element instance encoding is commitment-identical when $\delta = 0$, so pre-NU6.3 proofs and keys are untouched).

Wire encoding and polarity. The flag travels as bit 2 of the component’s flags byte, in the *enabled* sense (enableCrossAddress; bits are spends 0b001, outputs 0b010, cross-address 0b100 — librustzcash pczt/src/orchard.rs carries both canonical values, 0b0000_0011 for Orchard and 0b0000_0111 for Ironwood). The polarity is backward-compatible by design: bit 2 = 0 — the restricted state consensus requires for post-NU6.3 Orchard-pool spends — is exactly what v5 signers treating the bit as reserved-zero already produce, so existing hardware signers (ZIP-229 names the Keystone) keep signing v5 Orchard unshielding transactions without a firmware update. The Ironwood component carries the same flag and sets it: cross-address transfers stay enabled there.

Spending under the restriction. An Action spends one note and creates one (§2.11); if the created note must pay the spent note’s own receiver, how does an owner pay anyone? By exiting the pool: the

real value leaves through $v_{\text{balance}}^{\text{Orchard}} > 0$ (§5.4), and the Action’s output slot is filled with a *fabricated* zero-valued note to the spent note’s own receiver. The wallet must fill that output’s C_{enc} with *random* bytes: a real encryption would let any ivk holder — including a quantum adversary who recovered ivk from a published address, the very adversary of §6.1 — decrypt it and link the Action’s nullifier to the address (ZIP-326). In a PCZT (a *partially created Zcash transaction*, the format that carries an in-progress transaction between its constructor, prover, and signers) the fabricated output carries its recipient, value, and rseed explicitly, so a signer can classify it as a tolerable dummy rather than an opaque payment (ZIP-326; `librustzcash pczt/src/orchard.rs`). One gap remains open by construction: paying *into* an Orchard-pool receiver post-NU6.3 requires a spend at that same receiver, hence the spending key, so consensus cannot fully prevent self-sends; ZIP-326 wallet rules close the gap by forbidding sends to external Orchard-pool receivers.

Status of ZIP-2006. The restriction is deployed and enforced by consensus, but its owning ZIP does not yet exist as text: `zip-2006` is a nine-line Reserved stub (“Restricting Transfers into the Orchard Pool”; owner Daira-Emma Hopwood; created 2025-06-20; discussion `zcash/zips#1305`). *Specified* (in ZIP-229/258): the flag encoding, its consensus rule, and the verifying-key selection by branch. *Designed but unspecified*: the normative semantics — ZIP-326 phrases the restriction as equality of “expanded receivers” and carries a [TODO define] in its own text; the `enableCrossAddress` → `disableCrossAddress` instance mapping and the dummy-spend treatment are likewise reconstructable only from ZIP-229/258/326 and `crate orchard at bef8a27`, as Definition 5.4 does.

5.4 Migration and the turnstile

Value leaves the Orchard pool through a *turnstile*: a one-way, publicly metered gate, the same instrument Zcash used for the Sprout retirement. The consensus content is two rules already stated. Per transaction, from activation,

$$v_{\text{balance}}^{\text{Orchard}} \geq 0 \quad (\text{any transaction version; ZIP-258}),$$

so value may exit but never enter; and the chain maintains an Ironwood analogue of every pool-balance rule of §2.12 (the ZIP-209 extension):

$$\text{IronwoodPoolBalance} := - \sum_{\text{tx}} v_{\text{balance}}^{\text{Ironwood}}(\text{tx}),$$

zero before NU6.3, never negative, with the sum of all pool balances bounded by `MAX_MONEY` (ZIP-258). A migrating transaction spends Orchard-pool notes with $v_{\text{balance}}^{\text{Orchard}} > 0$ — releasing value to the transaction’s transparent value pool — and absorbs it into an Ironwood component with $v_{\text{balance}}^{\text{Ironwood}} < 0$. Both fields are public: the turnstile reveals the amounts crossing between pools in each transaction, exactly as ZIP-229’s privacy analysis states, and no per-transaction or aggregate cap is specified anywhere. The remaining Ironwood consensus rules are the Orchard ones of this volume, re-instantiated: `anchorIronwood` must be an earlier block’s final Ironwood treestate (§2.6), the tree capacity is 2^{32} , and the ZIP-221 history tree (the Merkle mountain range over the chain’s history whose root every block header commits to; *FlyClient Guide*, §“NU5: the commitment becomes one of two”) gains three Ironwood fields (earliest root, latest root, transaction count) aggregated exactly like the Orchard trio (ZIP-258).

Wallet posture. Wallets SHOULD proactively move *all* funds — transparent, Sprout, and Sapling included — into the Ironwood pool, the only pool whose output notes are recoverable, and SHOULD treat the sweep as ongoing (ZIP-2005; §6.4). The deployed change-routing logic implements the turnstile’s grain: change from Ironwood value stays in Ironwood, and after activation change may return to the Orchard pool only when the transaction spends Orchard notes *and* strictly less value returns than the spends remove; otherwise change is diverted to Ironwood (function

`select_change_pool`, `librustzcash zcash_client_backend/src/fees/common.rs`). Scanning narrows correspondingly: ZIP-326’s scan ranges close the Orchard pool off — external receivers scan from the birthday only to activation, internal ones until the wallet observes a zero Orchard balance after activation, a watermark that is sound precisely because $v_{\text{balance}}^{\text{Orchard}} \geq 0$ means a zero balance can never become nonzero — while Ironwood scans from the later of birthday and activation to the tip (ZIP-326).

ZIP-318’s construction. At zips commit `ad30e59` (2026-08-25), ZIP-318 is a Draft specification rather than the former Reserved stub. Its privacy-preserving migration has two phases. First, the wallet decomposes the migratable balance into canonical crossing denominations $n \cdot 10^k$ ZEC for $n \in \{1, 2, 5\}$, from 0.01 through 10000 ZEC, and creates exact-value funding notes by shielded send-to-self preparation transactions. Each funding note contains one denomination plus its canonical fee; less than 0.01 ZEC remains as a residual. Preparation transactions should contain exactly 16 Orchard Actions, use ordinary near-tip anchors, and be separated by independently drawn delays with mean 16 blocks and cap 96 blocks. Layering supplies fan-in or fan-out when one transaction cannot prepare all notes.

Second, each scheduled crossing should spend exactly one funding note in an Orchard bundle padded to two Actions and create exactly one unpadded Ironwood Action paying one canonical denomination. It has no transparent component or other shielded pool, pays the canonical fee, and sets lock time to zero. The wallet shuffles the parts and draws independent exponential broadcast delays with mean 66 blocks and cap 576 blocks. Each proof uses an Orchard anchor at a shared boundary height divisible by 144; its age in boundaries is drawn from a geometric distribution with parameter $1/2$ and truncated at four. A part scheduled at height h uses the common expiry height

$$69120 + 34560 \left\lfloor \frac{h}{34560} \right\rfloor,$$

giving between one and two months of validity at the 75-second target spacing. Because version 6 signatures do not bind the anchor, the wallet may pre-sign the schedule and later update each anchor and proof without re-signing (ZIP-318, Specification).

The procedure mitigates, but cannot remove, the public turnstile. It requires the user’s consent to the visible crossing values, separates synchronisation sessions from broadcast sessions, and offers a network-privacy path such as Tor, Nym, or a VPN. Its local schedule is deliberately stateful: restoring only the seed need not recover an in-progress migration (ZIP-318, Requirements, Non-requirements, and Privacy Implications).

Implementation boundary. The released `zcash_pool_migration` 0.1.0 crate and `zcash_client_sqlite` 0.22.0 implement the backend engine and persistent store described above: denomination and preparation planning, PCZT construction and signing, block-height scheduling, proof refresh, and state transitions (`librustzcash 91f448b`, `zcash_pool_migration/` and `zcash_client_sqlite/src/pool_migration/`). The consuming-wallet boundary remains unfinished at the cited heads. Zallet `f904040` pins the engine but its source tree exposes no migration command or RPC, and ZIP-318’s own “Reference implementation” section still contains only a TODO. The backend is therefore released; the specified mobile scheduling, network path, user consent, and application-open fallback are not a deployed end-to-end wallet flow.

6 Post-quantum security of the Orchard protocol

Every group-structured assumption in Theorem 3.5 — DL on both Pasta curves and, for property (iv), DDH on Pallas — falls to Shor’s algorithm in polynomial time on a cryptographically relevant quantum computer (CRQC; the *Crypto Guide*, §“The quantum caveat: Shor’s algorithm”); the theorem’s

remaining assumptions, the PRF security of Poseidon and the security of the KDF/AEAD, are symmetric and face only the generic speedups of §6.3. The productive question is therefore not *whether* the assumptions fail — they all do, together — but *when the failure matters*. We organise this section by *reversibility*. A break is *retroactive* when data already recorded on chain becomes exploitable the day the adversary exists: no later consensus change can help, because the data is public and permanent. A break is *prospective* when it requires a validator to *accept* a forged object after the adversary exists: disabling the affected protocol before that day removes every future acceptance event and mitigates the break completely. The Orchard protocol — both its pools — has exactly one retroactive break (§6.1); everything else on its assumption surface is prospective (§6.2) or faces only generic quantum speedups (§6.3). This section is the Orchard core seen from inside; the same reversibility axis applied across every layer of Zcash — the transparent stack, proof of work, the frontier protocols, and the migration arithmetic — is the *PQ Guide*’s subject.

Much of this section concerns design-stage material, so we make the epistemic status of every forward-looking claim explicit, in one of three bins: *specified* (a published artifact pins the construction down, which we cite), *designed but unspecified* (an informal source describes the design; we say what a specification would still need to fix), and *open problem* (no artifact exists). Claims about the deployed protocol instead cite the implementation and the specification as elsewhere in this volume. Ground truth for this section: `protocol.tex` and the ZIPs at `zips commit 69610984` (2026-07-09); `crate orchard` at `git commit bef8a27e` (v0.14.0, 2026-06-16) for circuit claims; and the released `crate orchard 0.15.0` (the `librustzcash` pin) for the `0x03` note path; we verified each cited artifact at these pins.

6.1 The retroactive break: harvest now, decrypt later against note encryption

Each Action publishes the pair $(\text{epk}, C_{\text{enc}})$ (§2.8; `crate orchard, src/note_encryption.rs`). Confidentiality of C_{enc} rests on exactly one discrete-log-dependent link: the one-shot Diffie–Hellman on Pallas between esk and pk_d (protocol §5.4.5.5; the esk derivation is `src/note.rs`, the key agreement `src/keys.rs`). The KDF and the AEAD above it are symmetric and are not the weak layer (§6.3).

A CRQC that knows a recipient’s address (d, pk_d) computes

$$\text{ivk} = \log_{g_d} \text{pk}_d$$

by Shor — one discrete logarithm for the lifetime of the address — and then runs the recipient’s own trial-decryption loop of §2.8 over the recorded chain: for every Action, $[\text{ivk}] \text{epk}$ recovers K_{enc} , and the AEAD tag identifies which ciphertexts belong to the address. The adversary obtains, for every note ever sent to that address, the plaintext $(d, v, \text{rseed}, \text{memo})$: every amount, every memo, the complete receiving history — and equally every *future* ciphertext to the same address. Two limits bound the damage. First, ivk is a viewing capability, not a spending one (§2.4): no spend authority is conferred, and ask is not exposed. Second, the attack is keyed by the address: without (d, pk_d) the adversary has neither the base g_d nor the target pk_d , and the recorded chain reveals nothing further even to an unbounded adversary (§6.2). ZIP-2005 records the same boundary: the harvest-now-decrypt-later exposure requires “both the ciphertext ... and the receiver address”.

The break is retroactive by construction. The pair $(\text{epk}, C_{\text{enc}})$ is consensus data, replicated on every full node forever; its secrecy was fixed at recording time by one specific DH instance; any protocol upgrade alters only ciphertexts not yet created. An adversary needs no quantum computer today — only storage — and every Orchard output recorded since NU5 activation is already in that corpus, which accrues Ironwood outputs from NU6.3 activation onward (ZIP-2005, Non-requirements).

Open problem: mitigation. Nothing is deployed, and nothing is specified, that protects even future ciphertexts. ZIP-2005 (Proposed) excludes privacy by an explicit Non-requirement, states that the exposure persists for all pools including Ironwood, and says only that mitigations for future transfers are “under consideration”. The tracking issues (`zcash/zips#1133`, open since 2022; `#1134`, open

since 2016) carry discussion, not drafts; the zips repository at commit 69610984 contains no occurrence of ML-KEM, Kyber, ML-DSA, or a lattice scheme. A future hybrid key encapsulation would in any case protect future outputs only; the recorded ciphertexts are unrepairable.

6.2 Prospective breaks: the discrete-logarithm integrity stack

The remainder of the assumption surface protects *integrity*, and every piece of it fails against the same adversary. The reductions of §3 each run equally well in reverse: the assumption a CRQC falsifies is exactly the one the security proof consumes.

- *Sinsemilla binding.* Collision resistance reduces to the discrete-log relation problem among the GroupHash generators (Proposition 3.1). A CRQC computes every pairwise logarithm, after which each Sinsemilla commitment opens to arbitrary messages: a second note under an on-chain cmx (breaking balance through NoteCommit non-binding), a fabricated membership path under any anchor (§2.6), and alternative pairs (ak', nk') opening the same Commit^{ivk} — the Faerie-Gold-style Spendability attacks ZIP-2005 catalogues. A balance violation committed this way is bounded only by the ZIP-209 turnstile and need not be detected at all.
- *Value commitments and the binding signature.* Binding of cv^{net} is the unknown discrete-log relation between V^{Orchard} and R^{Orchard} (Definition 2.8, Proposition 3.4); the binding signature is Schnorr under DL in the ROM. Knowledge of $\log_{V^{\text{Orchard}}} R^{\text{Orchard}}$ reopens any recorded or fresh cv to any value and yields bsk for any target bvk: silent inflation, again turnstile-bounded (crate orchard, src/value.rs).
- *RedPallas spend authorisation.* Unforgeability is DL in the ROM (Proposition 3.3). One logarithm — of ak, or of any published rk — forges spend authorisation at will (crate orchard, src/primitives/redpallas.rs); combined with the next item it completes arbitrary theft.
- *IPA knowledge soundness.* The Action SNARK is sound under DL on Vesta with Fiat–Shamir in the ROM (§3.7). A CRQC forges accepting proofs of false Action statements: minting from nothing, spending any commitment on chain, indistinguishable from honest proofs. This is the single most concentrated failure point — every other item’s exploitation route runs through it (crate halo2_proofs, src/poly/commitment.rs).

Why forking in time is a complete mitigation. Each attack above must place a forged object — a proof, a signature, a commitment opening — before a validator and have it *accepted*, and acceptance happens at current height under current consensus rules. Disabling the DL-dependent pools before a CRQC exists removes every future acceptance event; transactions already recorded were validated when the assumptions held and are never re-adjudicated. The breaks are therefore fully prospective, with two residues. Switch-off must precede the first forgery — an attack landed in the exposure window is not repaired afterwards (§6.4) — and honest funds inside a disabled pool need some other way out, which is precisely the problem ZIP-2005 exists to solve.

What the recorded chain does not leak. The same recorded data that becomes forgeable leaks no secrets, quantumly or otherwise. The value commitment is perfectly hiding, so recorded cv values reveal nothing about amounts even to an unbounded adversary (§2.9). The randomiser α is uniform, so recorded rk values and signatures carry information-theoretically no linkage to ak or ask (§3.4). Proof transcripts are statistically simulatable without the witness (§3.6). Nullifiers are keyed by nk, which never appears on chain (§2.7). Consequently a quantum adversary who does not know a recipient’s address learns nothing from the recorded chain — ZIP-2005 reaches the same conclusion — and the sole retroactive channel is the address-keyed decryption of §6.1.

6.3 Primitives facing only generic speedups

Two primitive families in Orchard carry no discrete-log structure. BLAKE2b serves PRFexpand and with it every key and note-randomness derivation (§2.4, §2.8), the KDF and the outgoing cipher key, the RedPallas challenge hash, and the ZIP-244 sighash; the ZIP-32 Orchard key tree is hardened-only, built entirely from these derivations, with no elliptic-curve operation anywhere on the derivation path (ZIP-32, Final; crate orchard, src/zip32.rs). Poseidon serves the nullifier PRF, keyed by nk (§2.7; src/note/nullifier.rs).

Against these, a quantum adversary gets generic speedups and nothing more — with the usual square-root caveat stated exactly. Grover search halves effective preimage security (the *Crypto Guide*, §“The quantum caveat: Shor’s algorithm”): 2^{256} classical evaluations become $\sim 2^{128}$ quantum ones, a margin the parameters absorb. For collisions the picture is better still: the Brassard–Høyer–Tapp $2^{n/3}$ algorithm requires random access to a quantum memory of $\sim 2^{92}$ bits at these output sizes, and the best *known* implementable generic quantum collision attack remains the classical parallel one — the assessment ZIP-2005 adopts, flagged there as best-known rather than provably best. Poseidon adds one honest caveat of its own: its PRF security is a cryptanalytic heuristic with no standard-problem reduction, classically and quantumly alike (§4); a quantum adversary gains no identified structural advantage, and since nk stays off chain, recorded nullifiers remain unlinkable either way.

The consequence is the load-bearing fact of the next subsection: the symmetric backbone survives a discrete-log break intact. A seed holder can still prove, by re-executing derivations, that their keys descend from their seed — ownership remains *provable* after the assumption that made it *enforceable* has fallen.

6.4 ZIP-2005: quantum recoverability of Ironwood notes

ZIP-2005 (“Ironwood Quantum Recoverability”, Proposed; created 2025-03-31) is the deployed-track response to the prospective breaks of §6.2. It does not make Orchard quantum-secure and says so; it arranges that funds survive the eventual switch-off.

Deployment. Activation is bound to the NU6.3 network upgrade (ZIP-258, Draft: consensus branch 0x37A5165B); the scheduling follows the NU6.2 emergency circuit correction (ZIP-257, Final), which is why the originally planned earlier deployment slipped. The pool split itself is the subject of §5; what this subsection relies on is the seal. From activation the legacy Orchard pool is spend-only: a coinbase transaction must carry an empty Orchard component, every Orchard-pool Action must have enableCrossAddress = 0, and no new value may enter the pool ($v_{\text{balance}}^{\text{Orchard}} \geq 0$) — the second rule enforced by the branch-selected Action-circuit verifying key, the other two as ordinary consensus rules (ZIP-258; ZIP-229).

Mechanism. Ironwood note plaintexts carry lead byte 0x03, and with it the hashed trapdoor derivation of §2.8: rcm derives from the 0x0B-separated pre_rcm concatenating every note field (Definition 5.2), in place of the legacy 0x05 || ρ input. Every note field now traverses BLAKE2b: an adversary constrained to treat PRFexpand as a random oracle cannot vary any field without changing rcm, so the composite of derivation and NoteCommit is post-quantum binding *provided a verifier checks the derivation* — NoteCommit itself, and the deployed Action circuit, are unchanged, and the binding claim is conditional on that future check. The same rederivation pattern covers Commit^{ivk}: the randomness r_{ivk} descends from sk through PRFexpand, or — for FROST-generated keys, where ask arises from distributed shares rather than from sk, and for hardware-held keys, where proof authority must remain separate from the on-device spend key — from a quantum key qk derived by one BLAKE3 derive_key compression from a secret qsk, with signatures of knowledge over the transaction sighash tying the claimed keys to the spend (the ZIP types it only as a message hash and pins no sighash algorithm). The Recovery Statement carries the qk and sk alternatives as a tagged union, and the key-binding condition pins that tag as well as the key value: a recovery-authorized qk witness

cannot collide with an sk-branch witness for the same ivk. The non-normative Recovery Statement then proves, in a post-quantum knowledge-sound system over a re-hashed commitment tree: the key derivations (BindKeys and the r_{ivk} branch), $ivk = \text{Commit}^{ivk}$, $pk_d = [ivk]_{g_d}$, the ψ and rcm derivations, the commitment, a membership path, the nullifier, and the α -re-randomisation tying the public rk to ak — costed at 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, and a Merkle path, the BLAKE2b compressions dominating.

Security bounds as published. The quantum analogue of the collision resistance these arguments need is Unruh’s *collapsing* property; Fehr’s theorem for HAIFA-mode hashes reduces collapsing of the rcm derivation to modelling BLAKE2b’s compression function as a random oracle — the same modelling the classical proofs already use. Both classical reductions (key binding and Spendability) are straight-line and never reprogram the oracle, so they lift cleanly to the quantum random-oracle model; by Zhandry’s compressed-oracle technique the classical $O(q^2/N)$ collision bound becomes $O(q^3/N)$, giving

$$\varepsilon_{kb}^{\text{QROM}} \leq O(q^3/N), \quad \varepsilon_{\text{Spendability}}^{\text{QROM}} \leq O(q^3/N) + \varepsilon_{kb}^{\text{QROM}},$$

with $N \approx 2^{254}$ the oracle output-space size (the Pallas scalar order, p_{Vesta} in this volume’s notation). At $q \ll 2^{60}$ the worst-case bound is $\sim 2^{-74}$; the practically achievable bound stays near the classical $q^2/N \approx 2^{-134}$, since saturating the cubic bound needs the memory-infeasible algorithm of §6.3.

Scope: binding only, Orchard only. The ZIP repairs binding, not privacy: the harvest-now-decrypt-later exposure of §6.1 persists unchanged for every pool, Ironwood included. Sapling is deliberately excluded — “to reduce overall protocol complexity and analysis effort” — since the ZIP-32 Sapling derivations differ significantly from Orchard’s hardened-only tree and would require their own analysis. The consequence is forced migration: Sapling, Sprout, and pre-Ironwood Orchard (0x02) notes are unrecoverable, hence permanently unspendable once their protocols switch off; wallets are directed to move all funds into Ironwood notes and to treat the sweep as an ongoing process, since non-recoverable notes may arrive at any time.

Specified. The 0x03 note derivation, the qsk/qk key path, and the consensus rules sealing the Orchard pool are pinned at ZIP rigor — ZIP-2005 (Proposed), ZIP-229 and ZIP-258 (Draft), all at zips commit 69610984 — and are enforced from NU6.3 activation: every Ironwood output is a recoverable note.

Designed but unspecified. The Recovery Protocol itself. ZIP-2005 presents the Recovery Statement in outline and states that its details are “subject to change”; a specification would need to pin down the post-quantum proof system (a stated requirement is that none be assumed now), the collapsing hash for the re-hashed commitment tree, the linking commitments between circuits, and the concrete signature-of-knowledge instantiations.

Open problem. A post-quantum proof system and signature scheme for *ongoing* spends — as opposed to one-shot recovery — exist nowhere as drafts (zcash/zips#1134 carries discussion only). So does the schedule: every recoverability guarantee is conditional on legacy switch-off preceding the first quantum forgery, and attacks landed in the exposure window — Spendability roadblocks that permanently block recovery, spend-authority theft, turnstile-bounded inflation — are not repaired retroactively.

6.5 Strategy: recoverability, not resistance

The successor proving stack points the same way. Ragu, the recursive proof system under development for project Tachyon, is deliberately ECDLP-based over the same Pasta cycle, its authors citing

“reduced concern (in the medium term)” for post-quantum soundness risks (ragu-tachyon book, `src/protocol/index.md`, commit 830bbcd, 2026-07-05). Read together with ZIP-2005, this fixes Zcash’s quantum strategy precisely: *recoverability, not resistance* — keep discrete-log cryptography while it stands, because its integrity failures are prospective and fork-mitigable (§6.2), and pre-position notes so that funds provably survive the eventual transition (§6.4); the one cost this strategy cannot answer is the retroactive privacy channel of §6.1, which accrues damage now. Public claims that Zcash will be “fully post-quantum” on a twelve-to-eighteen-month horizon, with ML-KEM and ML-DSA under test (CoinDesk, 2026-05-08), correspond to no ZIP, draft, or repository artifact, and ZIP-2005 itself disclaims making the protocol secure against quantum adversaries.

Part VII

Wallet Guide: The Zcash wallet layer: keys, addresses, fees, and transaction construction

Abstract

Assuming the Orchard protocol of the *Ironwood Guide*, this volume of *The Zcash Arboretum* documents the deployed wallet layer above it: hierarchical key derivation (ZIP-32), diversified and unified addresses (ZIP-316), the conventional fee (ZIP-317), the transaction-construction pipeline of `librustzcash`, partially created transactions (PCZT), the transaction lifecycle from broadcast through expiry (ZIP-203), and payment requests (ZIP-321). Every derived quantity carries its exact derivation, and every claim about deployed behaviour cites the implementing crate and file.

1 Hierarchical key derivation (ZIP-32)

The *Ironwood Guide* constructs every Orchard key — `ask`, `nk`, `rivk`, the full viewing key, `ivk`, `dk`, `ovk`, and the diversified addresses — from a single 32-byte spending key `sk`, which it treats as “simply a uniform 256-bit string”, deferring its origin to ZIP-32 (§“Keys: capabilities for spending and for viewing” there, Definition “Orchard key hierarchy”). This section supplies that origin. A deployed wallet does not draw an independent `sk` per account: it draws one *seed* and derives every account’s `sk` from it deterministically, so that a single backup recovers every key the wallet will ever hold. We develop ZIP-32’s Orchard instantiation: the hardened-only derivation tree, the standard account path `mOrchard/32'/coin_type'/account'`, the *internal* keys that stand in for a change path level, the fingerprints that name keys and seeds, and the serialised encodings — together with where the deployed stack (`orchard`, `zip32`, `zcash_keys`, `zcash_client_backend`, `zcash_client_sqlite`) enforces each rule, and where its documentation and its code disagree.

1.1 Wallet seeds

The root secret of the whole wallet is a byte string S , the *seed*. ZIP-32 imposes two requirements (ZIP-32, “Wallet seeds”): the seed **MUST** be 32 to 252 bytes long, and it **MUST** be generated with at least 256 bits of entropy — a requirement that extends to any mnemonic phrase from which the seed is obtained (ZIP-315 gives further guidance). The rationale is the seed’s position in the hierarchy: every key of every account derives deterministically from S , so a lower-entropy seed is a weak link for all of them at once, and — unlike an attack on one key — a brute-force search over low-entropy seeds runs against many wallets simultaneously.

Remark 1.1 (Where the deployed stack enforces the seed bounds). Enforcement is patchier than the **MUST** suggests, and one doc comment is wrong. The derivation function itself does not check: neither `ExtendedSpendingKey::master(orchard, src/zip32.rs)` nor `HardenedOnlyKey::master(zip32 0.2.0, src/hardened_only.rs)` inspects the seed length — BLAKE2b hashes whatever it is given — although the former’s doc comment claims “Panics if the seed is shorter than 32 bytes or longer than 252 bytes”; the claimed panic does not exist. One layer up, `UnifiedSpendingKey::from_seed(zcash_keys, src/keys.rs)` panics on seeds shorter than 32 bytes and checks no upper bound. The trait documentation of `WalletWrite::create_account` and `import_account_hd(zcash_client_backend, src/data_api.rs)` declares a panic for seeds outside 32..252 bytes, and `zcash_client_sqlite` enforces both bounds — indirectly, because it computes a seed fingerprint (Definition 1.13) via `SeedFingerprint::from_seed`, which returns `None` outside 32..252 bytes (`zip32 0.2.0, src/fingerprint.rs; zcash_client_sqlite, src/lib.rs`). The ZIP requirement therefore holds at the top of the deployed stack, not in the derivation function beneath it.

1.2 Hardened-only derivation

ZIP-32 obtains Orchard keys by instantiating its generic *hardened-only key derivation* process with two domain parameters: the master-key personalisation `Orchard.MKGDDomain = ``ZcashIP32Orchard''` (16 bytes) and the child-derivation lead byte `Orchard.CKDDomain = 0x81` (ZIP-32, “Orchard key derivation”). Non-hardened derivation — the BIP-32 mode in which a public key derives child public keys — is not defined for Orchard at all. That choice fixes the shape of the extended keys: an *Orchard extended spending key* is the pair (sk, c) of a normal 32-byte spending key and a 32-byte *chain code* c , and ZIP-32 defines *no* Orchard extended full viewing key, because with only hardened derivation a full viewing key confers no child-derivation capability and so has no use for a chain code (ZIP-32, “Orchard extended keys”; contrast the Sapling sections of the same ZIP, which do define extended FVKs). Deployed, `ExtendedSpendingKey` carries the pair inside a `HardenedOnlyKey` together with bookkeeping fields `depth`, `parent_fvk_tag`, and `child_index` (`orchard, src/zip32.rs`).

Definition 1.2 (Orchard master key generation). Given a seed S of 32 to 252 bytes, compute

$$I := \text{BLAKE2b-512}(\text{``ZcashIP32Orchard''}, S), \quad I = I_L \parallel I_R$$

with I_L, I_R the first and last 32 bytes — unkeyed BLAKE2b-512 with the 16-byte personalisation above — and set $sk_m := I_L, c_m := I_R$. The *master node* of the Orchard tree is $m_{\text{Orchard}} := (sk_m, c_m)$ (ZIP-32, “Hardened-only master key generation” instantiated per “Orchard master key generation”). Deployed master generation additionally requires sk_m to be valid in the sense of Definition 1.9, else it fails with `Error::InvalidSpendingKey (orchard, src/zip32.rs; zip32 0.2.0, src/hardened_only.rs)`.

Definition 1.3 (Orchard child key derivation). Write $i' := i + 2^{31}$ for the *hardened* index derived from $0 \leq i < 2^{31}$ (ZIP-32). Child derivation is defined only for hardened indices: for $i \geq 2^{31}$,

$$\begin{aligned} \text{CKDsk}((sk_{\text{par}}, c_{\text{par}}), i) : \quad I &:= \text{PRFexpand}_{c_{\text{par}}} (0x81 \parallel sk_{\text{par}} \parallel \text{I2LEOSP}_{32}(i)), \\ sk_i &:= I_L, \quad c_i := I_R, \end{aligned}$$

where $\text{I2LEOSP}_{32}(i)$ is the 4-byte little-endian encoding of i and PRFexpand is the BLAKE2b-512 expansion function of the *Ironwood Guide* (personalisation ```Zcash_ExpandSeed''`); derivation for $i < 2^{31}$ fails (ZIP-32, “Hardened-only child key derivation” and “Orchard child key derivation”).

Note the division of labour in the PRF call: the parent *chain code* c_{par} is the PRF key, and the parent spending key is message data. Continuing the tree therefore requires the pair — the 32-byte sk alone is a leaf capability, which is exactly the property the account path below exploits when it discards the chain code. Deployed derivation also caps the depth: the `depth` field is a byte incremented with `checked_add`, so a child of a depth-255 key cannot exist and the attempt fails with `Error::MaxDerivationDepth (orchard, src/zip32.rs)`.

Remark 1.4 (The general form: triples, and sibling instantiations). ZIP-32’s hardened-only child derivation in full generality takes a triple $(i, \text{lead}, \text{tag})$ and appends $\parallel [\text{lead}] \parallel \text{tag}$ to the PRF message; each distinct triple yields an independent output. For `lead = 0` and `tag = []` the encoding *omits* both fields, making it byte-compatible with the earlier definition of `CKDh` that lacked them; Orchard child derivation always uses `lead = 0, tag = []`, which is why Definition 1.3 shows no such suffix (ZIP-32, “Hardened-only child key derivation” and “Orchard child key derivation”). ZIP-32 defines no path *notation* for non-zero lead or non-empty tag. The deployed encoder implements the omission literally: `PrfExpand::with` skips `[lead] || tag` when the lead is zero and the tag empty (`zcash_spec 0.2.1, src/prf_expand.rs; zip32 0.2.0, src/hardened_only.rs`). The

same hardened-only process has two sibling instantiations outside the Orchard tree: *registered* key derivation (personalisation ``ZIPRegistered\_KD'', lead byte 0xAC, subtree rooted at *ZipNumber'* from an IKM of  $[\text{len}] \parallel \text{ContextString} \parallel [\text{len}] \parallel S$ ) and the deprecated *ad-hoc* derivation (personalisation ``ZcashArbitraryKD'', lead byte 0xAB) (ZIP-32, “Specification: Registered key derivation” and “Specification: Ad-hoc key derivation (deprecated)”; zip32 0.2.0, `src/registered.rs`, `src/arbitrary.rs`). They matter here only as neighbours in the domain-separation registry of Table 1.

1.3 The standard account path

Definition 1.5 (Orchard account path). The account-level key for account number $account \in \{0, \dots, 2^{31} - 1\}$ is derived along the path $m_{\text{Orchard}}/32'/\text{coin_type}'/\text{account}'$:

$$(sk_{\text{acct}}, c_{\text{acct}}) := \text{CKDsk}\left(\text{CKDsk}\left(\text{CKDsk}(\text{MKGh}^{\text{Orchard}}(S), 32'), \text{coin_type}'\right), \text{account}'\right),$$

after which the wallet keeps sk_{acct} and discards c_{acct} . The *purpose* level is $32' = 0x80000020$, registered per BIP 43; the *coin type* follows the SLIP 44 registry — 133 on Mainnet, 1 on Testnet and Regtest (all testnets share coin type 1) — and accounts are numbered sequentially from 0. Wallets MUST support this path for any account below 2^{31} (ZIP-32, “Key path levels” and “Orchard key path”; coin-type constants deployed in `zcash_protocol`, `src/constants/mainnet.rs`, `testnet.rs`, `regtest.rs`).

The path terminates at the account. Addresses are not further path children: each account exposes 2^{88} diversified addresses per scope, images of the diversifier indices under FF1 (ZIP-32, “Orchard key path”; the construction is in the *Ironwood Guide*). Nor is there a change level between account and address, where BIP-44-style transparent paths put one: shielded change returning to the originating account is indistinguishable on-chain from any other output, so a change subtree would separate nothing an observer can see, and the separation the wallet itself needs — telling its own change apart from external receipts — is achieved one level lower by the *internal key derivation* of §1.5 (ZIP-32, “Key path levels”).

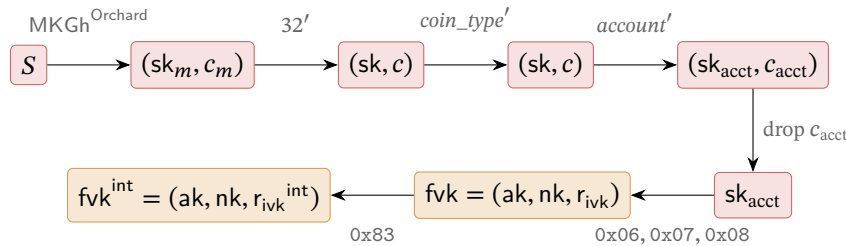


Figure 1: The deployed Orchard key path. Each horizontal arrow in the top row is one CKDsk call (Definition 1.3): a PRFexpand invocation with lead byte 0x81, keyed by the parent chain code. The chain code travels only as far as the account node $(sk_{\text{acct}}, c_{\text{acct}})$; the wallet layer keeps the bare sk_{acct} , from which the external full viewing key fvk (*Ironwood Guide*, lead bytes 0x06/0x07/0x08) and the internal fvk^{int} (Definition 1.6, lead byte 0x83) both derive. Red marks spend capability, orange full-viewing capability, per the series palette.

Deployed path derivation. Function `SpendingKey::from_zip32_seed(seed, coin_type, account)` (`orchard`, `src/keys.rs`) rejects $coin_type \geq 2^{31}$ with `Error::InvalidChildIndex`, runs `ExtendedSpendingKey::from_path` over the three hardened indices of Definition 1.5, and returns only the 32-byte account spending key — the chain code is discarded, so the wallet layer retains no Orchard child-derivation capability below the account level. Above it,

`UnifiedSpendingKey::from_seed` stores that plain `SpendingKey`, and the unified full viewing key’s Orchard component is the `FullViewingKey` computed from it (`zcash_keys`, `src/keys.rs`). The `zip32` crate types the path levels: `AccountId` is a 31-bit integer whose `TryFrom<u32>` rejects values at or above 2^{31} and which always enters paths hardened, and `ChildIndex` is hardened-only — `from_index` returns `None` unless the hardened bit is set, and `hardened(v)` asserts $v < 2^{31}$ and stores $v + 2^{31}$ (`zip32 0.2.0`, `src/lib.rs`). A distinguished index `ChildIndex::PRIVATE_USE = 0x7fffffff` is reserved for private-use subtrees; `zcashd`’s legacy Sapling account uses it, and we find no Orchard use (ZIP-32, “Sapling key path”; `zip32 0.2.0`, `src/lib.rs`).

1.4 From the account key to addresses: the layer already built

Everything below `skacct` is the hierarchy the *Ironwood Guide* constructs in “Keys: capabilities for spending and for viewing” (Definition “Orchard key hierarchy”), and we do not re-derive it: `ask`, `nk`, `rivk` from `PRFexpand` lead bytes `0x06`, `0x07`, `0x08` with the `ToScalar/ToBase` reductions and their bias analysis; the sign-canonicalised `ak = [ask] GSpendAuth`; `fvk = (ak, nk, rivk)`; `ivk = Commitrivkivk(Extract(ak), nk)`; the `(dk, ovk)` split of a single `PRFexpand` output with lead byte `0x82`; and the FF1 diversified addresses with default index 0 (protocol specification §4.2.3, “Orchard Key Components”; `orchard`, `src/keys.rs`).

What ZIP-32 adds at the wallet level is the capability boundary of the full viewing key. The holder of `fvk` obtains, for *both* the external and the internal scope of §1.5: the incoming viewing keys, the diversifier keys and outgoing viewing keys, and hence every diversified address of the account; the *index* behind any of its addresses, since the diversifier map is a keyed permutation and `FF1-AES256dk-1(d)` — FF1 decryption under the empty tweak — recovers the index (`DiversifierKey::diversifier_index`, `orchard`, `src/keys.rs`); and the scope of any of its addresses, by trying external then internal (`FullViewingKey::scope_for_address`, `orchard`, `src/keys.rs`). Two contrasts with Sapling sharpen the picture (ZIP-32, “Orchard diversifier and OVK derivation”): the diversifier key derives from the *full viewing key* rather than the extended spending key, so viewing capability suffices to locate an address’s position in the sequence; and all 2^{88} diversifiers are valid, so no rejection loop exists and the default diversifier is d_0 . What the full viewing key can *not* do: derive `ask`, or derive any child account — the former by the one-wayness of the `0x06` expansion, the latter because only hardened derivation exists and no extended FVK is defined (§1.2). Finally, no mechanism derives an outgoing viewing key per *diversified address*: payments are sent from accounts, with external or internal scope, not from individual addresses (ZIP-32, “Orchard diversifier and OVK derivation”, citing the ZIP-316 rationale “Usage of Outgoing Viewing Keys”).

1.5 Internal keys: change and auto-shielding

The path of Definition 1.5 reserved no change level, so the separation between externally visible receiving and wallet-internal operations — change outputs, and the auto-shielding of transparent funds — happens below the account instead: every account key yields a second, *internal* full viewing key. The mechanism was implemented in `zcashd` as part of ZIP-316 support and applies to keys that are children of the account level, i.e. account keys (ZIP-32, “Orchard internal key derivation”).

Definition 1.6 (Orchard internal key derivation). Given an external full viewing key `fvk = (ak, nk, rivk)`, let $K := \text{LE}_{256}(r_{ivk})$ and

$$\begin{aligned} r_{ivk}^{\text{int}} &:= \text{ToScalar}(\text{PRFexpand}_K(0x83 \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}))) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{fvk}^{\text{int}} &:= (\text{ak}, \text{nk}, r_{ivk}^{\text{int}}), \end{aligned}$$

i.e. $\text{DeriveInternalFVK}^{\text{Orchard}}$ modifies *only* r_{ivk} (ZIP-32, “Orchard internal key derivation”; protocol specification §4.2.3). The downstream keys then follow the standard FVK-level rules of the *Ironwood Guide*, applied to $\text{fvk}^{\text{int}}: \text{ivk}^{\text{int}} := \text{Commit}_{r_{\text{ivk}}^{\text{int}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$ and $(\text{dk}^{\text{int}}, \text{ovk}^{\text{int}})$ from the 0x82 split keyed by $\text{LE}_{256}(r_{\text{ivk}}^{\text{int}})$. The *expanded internal spending key* is $(\text{ask}, \text{nk}, r_{\text{ivk}}^{\text{int}})$ — the same ask — and no 32-byte internal spending key exists (ZIP-32, “Orchard internal key derivation”).

The shape is deliberately that of the (dk, ovk) derivation: the same key K , the same message body $\text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})$, a different lead byte. Its consequences: ak and nk are unchanged (the protocol specification notes $\text{ak}^{\text{int}} = \text{ak}$ and $\text{nk}^{\text{int}} = \text{nk}$, §4.2.3), so spend authorisation and nullifier derivation are common to both scopes, while ivk , dk , and ovk all separate because they derive from $r_{\text{ivk}}^{\text{int}}$. The two scopes are therefore two disjoint families of diversified addresses under one spend authority — and since $\text{ivk}^{\text{int}} \neq \text{ivk}$, a note addressed under the internal scope does not trial-decrypt under the external ivk (*Ironwood Guide*, “Transmission and detection of a note”), which is precisely the separation a wallet wants when it hands its external incoming viewing key to a payment processor. Note that the internal FVK is *computable from* the external FVK — the derivation consumes only $(\text{ak}, \text{nk}, r_{\text{ivk}})$ — so disclosing the full viewing key disclosed both scopes all along; the boundary runs at the incoming-viewing tier, not the full-viewing tier. Unlike its Sapling counterpart, the construction does not rest on non-hardened derivation, which is undefined for Orchard (ZIP-32, “Orchard internal key derivation”).

Deployed, the `zip32` crate’s `Scope` enum $\{\text{External}, \text{Internal}\}$ names the two scopes and orchard re-exports it; `FullViewingKey::rivk(Scope)`, `to_ivk`, `to_ovk`, and `derive_internal` implement Definition 1.6 (`zip32` 0.2.0, `src/lib.rs`; orchard, `src/keys.rs`). The change path is concrete: `zcash_client_backend` sends Orchard change to `ufvk.orchard().address_at(0u32, Scope::Internal)` — the internal-scope address at diversifier index 0 (`zcash_client_backend`, `src/data_api/wallet.rs`).

The transparent pool needs the same external/internal ovk split — its auto-shielding transactions carry shielded outputs whose outgoing ciphertexts want a viewing key — but P2PKH keys (pay-to-public-key-hash: the output script pays to the hash of a single public key) have no protocol-level ovk to scope, so ZIP-316 synthesises the pair.

Definition 1.7 (Transparent internal and external OVK). For a transparent P2PKH account with BIP-32 account-level extended public key (c, pk) — chain code c , keying PRFexpand as a 256-bit key, and $\text{ser}(pk)$ the 33-byte compressed public key —

$$I_{\text{ovk}} := \text{PRFexpand}_c(0xd0 \parallel \text{ser}(pk)), \quad \text{ovk}_{\text{external}} := I_{\text{ovk}}[0..32], \quad \text{ovk}_{\text{internal}} := I_{\text{ovk}}[32..64]$$

(ZIP-316, *Deriving Internal Keys*; lead byte in Table 1).

Remark 1.8 (Byte-level normalisation between ZIP and code). ZIP-32 writes the PRF key as the bit sequence $\text{I2LEBSP}_{256}(r_{\text{ivk}})$ where PRFexpand consumes bytes; the code passes `rivk.to_repr()`, the 32-byte little-endian encoding. The two are identical under the LE_{256} convention this series uses, and we write LE_{256} throughout, as the *Ironwood Guide* already does. Likewise the ak bytes fed to the 0x82 and 0x83 messages are `SpendValidatingKey::to_bytes()`, the 32-byte RedPallas point encoding — which, because key generation forces the sign bit to zero (the canonicalisation of the *Ironwood Guide*), coincides with $\text{I2LEOSP}_{256}(\text{Extract}(\text{ak}))$, matching the specification’s $\text{I2LEOSP}_{256}(\text{ak})$ for $\text{ak} \in \{0, \dots, p_{\text{Pallas}} - 1\}$ (orchard, `src/keys.rs`; protocol specification §4.2.3 and the `AuthSignPublic` type). The nk bytes are the base-field representation.

1.6 Key validity and derivation failure

Definition 1.9 (Valid Orchard spending key). A 32-byte string sk is a *valid* spending key iff all three hold: (i) $ask \neq 0$; (ii) the external $ivk = \text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk) \notin \{0, \perp\}$; (iii) the internal $ivk^{\text{int}} = \text{Commit}_{r_{ivk}^{\text{int}}}^{ivk}(\text{Extract}(ak), nk) \notin \{0, \perp\}$. Function `SpendingKey::from_bytes` returns `None` exactly when one of the three fails (orchard, `src/keys.rs`), and the specification’s key-generation algorithm mandates exactly these three discard conditions — “discard this key and repeat with a new sk ” (protocol specification §4.2.3, “Orchard Key Components”). `FullViewingKey::from_bytes` re-checks both scopes when parsing an FVK from bytes (orchard, `src/keys.rs`).

Remark 1.10 (Derivation failure has no specified recovery). ZIP-32 notes that a child spending key may yield an invalid external or internal full viewing key “with small probability” (ZIP-32, “Orchard child key derivation”) but prescribes no wallet behaviour when it does; the specification’s “repeat with a new sk ” addresses a freshly random key, not a fixed HD path, where there is nothing to redraw. The deployed stack hard-errors: `master` and `derive_child` return `Err(Error::InvalidSpendingKey)`, `from_path` propagates it, and `zcash_keys` surfaces it as `DerivationError::Orchard` (orchard, `src/zip32.rs`; `zcash_keys`, `src/keys.rs`); no layer implements a skip-to-next-index convention. This behaviour is normative in practice: an account index at which derivation fails simply has no key, and no deployed convention reassigns it. The event is negligibly rare — neither ZIP-32 nor the specification quantifies “small probability”, so Proposition 1.11 derives a bound — and the gap is theoretical.

Proposition 1.11 (Probability of an invalid key). *Model the PRFexpand outputs and the GroupHash-derived Sinsemilla generators as uniform and independent (the random-oracle model in which the Ironwood Guide analyses both primitives). A uniformly random 32-byte sk then fails the conditions of Definition 1.9 with probabilities*

$$\Pr[ask = 0] < 2^{-254}, \quad \Pr[ivk \in \{0, \perp\}] \leq 307/p_{\text{Vesta}} < 2^{-245} \quad (\text{per scope}),$$

and is invalid with probability below 2^{-244} .

Proof. The scalar ask is the `ToScalar` image of the `0x06` expansion (the conditional negation that canonicalises ak preserves zeroness), so $ask = 0$ iff a uniform 512-bit integer reduces to 0 modulo p_{Vesta} . Exactly $\lceil 2^{512}/p_{\text{Vesta}} \rceil$ of the 2^{512} inputs do, and $p_{\text{Vesta}} > 2^{254}$ gives $\lceil 2^{512}/p_{\text{Vesta}} \rceil < 2^{258}$, hence $\Pr[ask = 0] < 2^{258} \cdot 2^{-512} = 2^{-254}$.

For ivk , the two memberships separately. *Zero*: conditioned on the Sinsemilla hash not returning $\perp$, the commitment point is the hash point plus $[r_{ivk}]H_D$ — the blinding term of the *Ironwood Guide*’s “Sinsemilla commitment and short commitment” — uniform on the p_{Vesta} -element Pallas group by perfect hiding, r_{ivk} being uniform up to the $O(2^{-258})$ `ToScalar` bias. The extractor sends only the identity to 0, because $y^2 = 0^3 + 5$ has no solution in $\mathbb{F}_{p_{\text{Pallas}}}$ — 5 is a non-square there (protocol specification §5.4.9.7, note) — so $\Pr[ivk = 0] \leq 1/p_{\text{Vesta}} < 2^{-254}$. *Bottom*: the $\perp$ output arises only inside the Sinsemilla hash, since the blinding addition is complete (§5.4.8.4). On the 510-bit $\text{Commit}_{r_{ivk}}^{ivk}$ message — two 255-bit field encodings, $\lceil 510/10 \rceil = 51$ ten-bit chunks — the hash performs $2 \cdot 51 = 102$ incomplete additions (§5.4.1.9). Each fails only when one operand is the identity or shares the other’s x -coordinate — at most 3 of the p_{Vesta} group elements — so, with each accumulator modelled as uniform, the union bound gives $\Pr[\perp] \leq 306/p_{\text{Vesta}} < 2^9 \cdot 2^{-254} = 2^{-245}$, and per scope $\Pr[ivk \in \{0, \perp\}] \leq 307/p_{\text{Vesta}}$. The internal scope repeats the argument with the independent r_{ivk}^{int} (the `0x83` expansion). In total, $\Pr[sk \text{ invalid}] < (1 + 2 \cdot 307) \cdot 2^{-254} < 2^{10} \cdot 2^{-254} = 2^{-244}$. $\square$

The bound makes ZIP-32’s “small probability” concrete: below 2^{-244} per derived key, and below 2^{-242} for the union over the four nodes of the standard account path (master included, Definition 1.2).

Only the $\perp$ term is heuristic, and it is doubly guarded: any concrete input exhibiting $\perp$ would constitute a nontrivial discrete-log relation among the Sinsemilla generators (*Ironwood Guide*, the Sinsemilla collision-resistance proof), so none is expected ever to be observed.

1.7 Fingerprints and tags

Two identifiers name objects of this section without revealing them; both are BLAKE2b-256 digests with dedicated personalisations.

Definition 1.12 (Orchard FVK fingerprint and tag). For a full viewing key $\text{fvk} = (\text{ak}, \text{nk}, r_{\text{ivk}})$, the *fingerprint* is

$$\begin{aligned} \text{FVFP}(\text{fvk}) &:= \text{BLAKE2b-256}(\text{``ZcashOrchardFVFP''}, \text{fvk}_{\text{raw}}), \\ \text{fvk}_{\text{raw}} &:= \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}) \parallel \text{I2LEOSP}_{256}(r_{\text{ivk}}), \end{aligned}$$

the digest of the 96-byte raw FVK encoding fvk_{raw} (protocol specification §5.6.4.4). The *tag* is its first 4 bytes and MUST NOT be assumed unique; the master key’s parent tag is the 4 zero bytes (ZIP-32, “Orchard Full Viewing Key Fingerprints and Tags”). Deployed as `FvkFingerprint` and `FvkTag`; child derivation stamps each child with `parent_fvk_tag` = the tag of the parent’s FVK (orchard, src/zip32.rs).

Definition 1.13 (Seed fingerprint). For a seed S of 32 to 252 bytes, with $[\text{length}(S)]$ its single length byte,

$$\text{SeedFP}(S) := \text{BLAKE2b-256}(\text{``Zcash_HD_Seed_FP''}, [\text{length}(S)] \parallel S);$$

the string form is Bech32m (the checksummed base-32 string encoding of BIP-350: a human-readable part, a separator 1, and base-32 data characters ending in a six-character checksum) with human-readable part `zip32seedfp`, and no short tag is defined; the length prefix was chosen to match `zcashd`’s implementation (ZIP-32, “Seed Fingerprints”). Deployed, `SeedFingerprint::from_seed` returns `None` outside the 32..252-byte bounds (zip32 0.2.0, src/fingerprint.rs), and `zcash_client_sqlite` uses the fingerprint to bind each stored account to the seed that produced it (`zcash_client_sqlite`, src/lib.rs).

1.8 Extended-key encoding and test vectors

Definition 1.14 (Orchard extended spending key encoding). An extended spending key at depth depth , child index i (hardened offset included), with parent tag tag_{par} , serialises as the 73-byte string

$$\text{xsk} := \text{I2LEOSP}_8(\text{depth}) \parallel \text{tag}_{\text{par}} \parallel \text{I2LEOSP}_{32}(i) \parallel c \parallel \text{sk} \quad (1 + 4 + 4 + 32 + 32 \text{ bytes}),$$

with the master key at depth 0, tag `0x00000000`, index 0. The Bech32 human-readable parts are `secret-orchard-extsk-main` (Mainnet) and `secret-orchard-extsk-test` (Testnet) (ZIP-32, “Orchard extended spending keys”).

ZIP-32 defines this encoding “for completeness” and expects most wallets to expose only the plain `sk` encoding of the protocol specification (§5.6.4.5) — consistent with the deployed stack, which discards the chain code at the account level (§1.3). Indeed we find *no* deployed `librustzcash` code path that emits or parses the Bech32 form: the 73-byte layout is exercised only by the orchard crate’s embedded test-vector checks, and `zcash_keys` stores the raw spending key inside the unified spending key

encoding (orchard, src/zip32.rs, src/test_vectors/zip32.rs; zcash_keys, src/keys.rs). We classify the encoding as specified but unused in the scanned stack.

Remark 1.15 (Test vectors). The ZIP-32 Orchard vectors live in zcash-test-vectors under orchard_zip32; the orchard crate embeds four of them — the nodes m , $m/1'$, $m/1'/2'$, $m/1'/2'/3'$ from the 32-byte seed $0x00, \dots, 0x1f$ — each giving sk , c , the 73-byte xsk , and the FVK fingerprint (ZIP-32, “Test Vectors”; orchard, src/test_vectors/zip32.rs, src/zip32.rs). They do *not* cover the standard path of Definition 1.5. Separate key-component vectors ($sk \rightarrow ask, ak, nk, r_{ivk}, ivk$ and the internal-scope components) sit in src/test_vectors/keys.rs, exercised by the keys.rs tests.

Example 1.16 (The standard path, worked). No published vector covers the path of Definition 1.5, so we compute one, from the embedded vectors’ seed $S = 0x00, \dots, 0x1f$, for Mainnet ($coin_type = 133$) and account 0:

m	sk	7eee3c1017870990a3dd6891b82f80be8976c1e7dc20d60817a5e88e8b2cd4b8
	c	ab8b7a00509ef20e469b5292b61d474b7cfffcb1657924cda720250ae40526677
$m/32'$	sk	93af0a87c2e4704d8825e145557c6a5d4dcb9016170e276a2080a2dc9eb0f87
	c	6e1e808d410d6d1f84e62e67c8b60e80e5ee4de5021d575c0433a625d609c45c
$m/32'/133'$	sk	149fcee594a2a03de277ab919f22611d0ef6aa8e8e9a0bef32f01a0bcb0e8c4c
	c	07d13fdeccd21da16b467f8e785edc32f74ceeb541bede0394647f10fdb7114f
$m/32'/133'/0'$	sk	b67d8d87cab9189500afb45dbca9f92c924c1de9d9ee1451be4a78313cb223b4
	c	7ff0a6392e144d4c8f45bca21fb7827b508446ac069890aefbebbe388b7a84c0

The bottom sk is exactly what `SpendingKey::from_zip32_seed` returns for $(S, 133, 0)$; the bottom c is what it discards. The account key’s FVK fingerprint (Definition 1.12) is

da8dab741dc4f6ff8964fbbaa8840270caf47d286e9a0722e6c89e7f2c38189d5

(tag da8dab74), its parent’s fingerprint begins 7010c19b, and the account’s 73-byte encoding of Definition 1.14 is therefore

$$xsk = 03 \parallel 7010c19b \parallel 00000080 \parallel c \parallel sk, \quad 00000080 = I2LEOSP_{32}(0'),$$

with c and sk the bottom table row. The numbers are script-computed per the series conventions: a standalone reimplementation of $MKGh^{\text{Orchard}}$ and $CKDsk$ first reproduces all four embedded vectors byte for byte (sk, c, xsk); the orchard crate, driven as a library, reproduces their FVK fingerprints, returns the same account sk from `from_zip32_seed`, and validates every node above per Definition 1.9.

1.9 The PRFexpand lead-byte registry

One PRF underlies every derivation above and in the *Ironwood Guide*, but domain separation is per PRF *key*, not protocol-wide: under a fixed key, distinct lead bytes make the derived secrets mutually independent, while separate consumers may reuse a byte under different keys. Sapling derives rcm and esk from lead bytes $0x04$ and $0x05$ with no ρ suffix — the assignment swapped relative to Orchard’s, which the specification attributes to an oversight (protocol specification §4.7.3, note) — and the Sapling instantiation of ZIP-32 intentionally reuses $0x00$ – $0x02$ from the Sapling key components under a different key. Table 1 collects the accounting for the Orchard, Orchard-ZIP-32, and ZIP-316 bytes this series constructs; the Sapling-only bytes ($0x00$ – $0x03$, $0x10$ – $0x18$, of which §2.2 cites $0x10$ and $0x16$) are accounted in the specification’s list (protocol specification, the PRF^{expand} accounting under “Pseudo Random Functions” and §4.2.3; ZIP-32; zcash_spec 0.2.1, src/prf_expand.rs). The Sprout instantiation of ZIP-32 reserves the lead byte $0x80$, the personalisation ```ZcashIP32_Sprout''`, and the HRPs `zxsprout/zxtestsprout` — values the ZIP says SHOULD NOT be used in future Zcash-related specifications (ZIP-32, “Values reserved due to previous specification for Sprout”). All Orchard and ZIP-32 constants in this section match between the specification text and zcash_spec 0.2.1 — the version locked by both the orchard and librustzcash workspaces (Cargo.lock in each; likewise zip32 0.2.0).

Lead byte	PRF key	Message after the lead byte	Output
0x04	rseed	ρ	esk
0x05	rseed	ρ	rcm
0x06	sk	—	ask
0x07	sk	—	nk
0x08	sk	—	r_{ivk}
0x09	rseed	ρ	ψ
0x80	Sprout ZIP-32 child (reserved; SHOULD NOT reuse)		
0x81	c_{par}	$sk_{\text{par}} \parallel I2LEOSP_{32}(i)$	$sk_i \parallel c_i$
0x82	$LE_{256}(r_{ivk})$ (per scope)	$ak \parallel nk$	$dk \parallel ovk$
0x83	$LE_{256}(r_{ivk})$	$ak \parallel nk$	r_{ivk}^{int}
0xAB	ad-hoc ZIP-32 child (deprecated)		
0xAC	registered ZIP-32 child		
0xD0	c (chain code)	$ser(pk)$	$ovk_{\text{ext}} \parallel ovk_{\text{int}}$

Table 1: Lead-byte registry of PRFexpand messages. Point and field arguments are encoded with $I2LEOSP_{256}$; ρ is the 32-byte nullifier of the Action’s spent note (*Ironwood Guide*, “Transmission and detection of a note”). Rows 0x04–0x09 are constructed in the *Ironwood Guide* and carry the Orchard meanings of 0x04/0x05; Sapling reuses both bytes, without the ρ suffix and with the outputs swapped (0x04 $\rightarrow$ rcm, 0x05 $\rightarrow$ esk), under its own per-note rseed keys (protocol specification §4.7.3, note). Rows 0x81 and 0x83 are constructed in Definitions 1.3 and 1.6; 0xD0 in Definition 1.7; 0x80, 0xAB, 0xAC belong to other subsystems (ZIP-32) and appear for the completeness of the accounting.

2 Diversified and unified addresses (ZIP-316)

An address layer must reconcile two demands that pull apart. Each payee should receive their own address, mutually unlinkable with every other — yet all of them spendable by one key and recoverable from one seed; each shielded protocol solves this with *diversified addresses*. And Zcash is several protocols at once: transparent, Sapling, and Orchard receivers coexist, each with its own format, so a recipient wants to hand a sender *one* string from which the sender’s wallet picks the best receiver both sides support; ZIP-316 solves this above the protocols with the *unified* container encodings. The *Ironwood Guide* (“Keys: capabilities for spending and for viewing”) already constructs the Orchard-side machinery — the key tiers, the FF1-AES256 diversifier map $d := \text{FF1-AES256}_{\text{dk}}(d_{\text{plain}})$, the diversified base $g_d = \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d)$, and the rationale for a keyed permutation over the index space — and we cite it throughout. This section adds what the wallet layer builds on top: the index discipline shared across Sapling, Orchard, and transparent derivation, the raw and unified encodings, viewing-key containers, and the deployed address-generation code.

At zips ad30e59, ZIP-316 is organised into revisions: Revision 0 is Active and is what the deployed stack implements; Revision 1 was withdrawn; Revision 2 is a Draft (ZIP-316, *Revisions*). This volume documents Revision 0 and states Revision 2 in §2.8, classified *specified, not deployed*; its reference implementation is librustzcash PR #2199, absent from both the baseline and current 91f448b5 crates. Deployed-behaviour claims below are verified against librustzcash commit e30517e4 — the volume’s baseline (§7.7) — and the crates-io releases zcash_spec 0.2.1, zip32 0.2.0, sapling-crypto 0.7.0, and orchard 0.15.0 (exact versions pinned by the librustzcash workspace Cargo.lock); ZIP text is quoted from the zips repository at commit c6b7035.

2.1 Accounts: ZIP-32 key paths and wallet seeds

Every address below hangs off an *account*, a node of the ZIP-32 hierarchical-deterministic key tree. Sapling and Orchard share the path shape

$$m / \text{purpose}' / \text{coin_type}' / \text{account}',$$

with $\text{purpose} = 32'$ (0x80000020), the SLIP-44 coin type (133 on mainnet, 1 on all testnets), the account index counting from 0, and every level hardened (ZIP-32, *Key path levels*; *Sapling key path*; *Orchard key path*; deployed constants in `zcash_protocol`, `constants/`). Two absences are deliberate. There is no change level: shielded change pays the wallet’s own address and is indistinguishable from any other output, so the transparent external/internal split buys nothing (ZIP-32, *Key path levels*). And there is no non-hardened tail: Orchard ZIP-32 defines hardened-only derivation, instantiated with `MKGDDomain = "ZcashIP32Orchard"` and `CKDDomain = 0x81` over the extended spending key (sk, c) ; a child may, with small probability, yield a spending key whose external or internal full viewing key is invalid, in which case derivation at that index simply fails — ZIP-32 prescribes no recovery, and the deployed stack surfaces an error (§1.6; ZIP-32, *Specification: Orchard key derivation*). Wallets MUST support this path for accounts 0 to $2^{31} - 1$ and MUST support generating each account’s default payment address; a stream of diversified addresses MAY be supported (ZIP-32, *Key path levels*; *Sapling key path*; *Orchard key path*). The seed feeding the tree MUST be 32–252 bytes with at least 256 bits of entropy: an attack on a lower-entropy mnemonic can be run against many wallets at once, by matching derived addresses and viewing keys against observed ones (ZIP-32, *Specification: Wallet seeds*).

2.2 Diversified addresses

A diversified address multiplies addresses without multiplying keys: one incoming viewing key ivk , one diversifier key dk , and one 88-bit *diversifier* d per address. The *Ironwood Guide* (“Keys: capabilities for spending and for viewing”) constructs the map and its design rationale — a keyed permutation of the index space, rather than a hash or a random draw; the wallet layer fixes the index discipline that ZIP-32 shares between Sapling and Orchard.

Definition 2.1 (Diversifier derivation, Sapling and Orchard). For an account’s diversifier key dk , the diversifier at *diversifier index* j is

$$d_j := \text{FF1-AES256}.\text{Encrypt}(dk, "", \text{LE}_{88}(j)), \quad j \in \{0, \dots, 2^{88} - 1\},$$

where FF1-AES256 is NIST SP 800-38G FF1 with a 256-bit AES key, radix 2, $\text{minlen} = \text{maxlen} = 88$, the tweak always the empty string, and $\text{LE}_{88}(j)$ (the ZIP’s I2LEBSP_{88}) the index’s 88-bit little-endian bit encoding; input and output are 88-bit sequences. The protocol specification abbreviates the keyed permutation as $\text{PRP}_{dk}^d(d) := \text{FF1-AES256}_{dk}("", d)$ (ZIP-32, *Sapling diversifier derivation*; *Orchard diversifier and OVK derivation*; protocol specification §5.4.4, *Pseudo Random Permutations*).

The Orchard diversified payment address at index j is then

$$d := \text{PRP}_{dk}^d(\text{LE}_{88}(j)), \quad g_d := \text{DiversifyHash}^{\text{Orchard}}(d), \quad \text{pk}_d := [ivk] g_d,$$

the pair (d, pk_d) , with the address at index 0 the *default* diversified payment address; the last step is the key-agreement $\text{KA}^{\text{Orchard}}.\text{DerivePublic}$ of the protocol specification, and the *Ironwood Guide* constructs all three maps. Diversifier indices SHOULD NOT be chosen at random, and an address generator MAY encode recoverable information in the index — the dk holder recovers it by FF1 decryption (protocol specification §4.2.3, *Orchard Key Components*).

The deployed cipher is the fpe crate’s `FF1<Aes256>` at radix 2; the orchard crate pins fpe 0.6 in its `Cargo.toml`. Both protocols encrypt the index bytes `j.as_bytes()` wrapped as a `BinaryNumeralString` via `from_bytes_le` — the 11-byte little-endian index with bits least-significant-first within each byte, which is exactly $\text{I2LEBSP}_{88}(j)$ — under the empty tweak, and convert the 88-bit output back with `to_bytes_le` into the 11-byte diversifier, exactly LEBS2OSP_{88} (orchard, `src/keys.rs`; sapling-crypto 0.7.0, `src/zip32.rs`).

The permutation earns its keep quantitatively. An 88-bit diversifier is too short to be cryptographically unguessable at the 128-bit security level, and diversifiers chosen at random begin to collide by the birthday bound as the number of draws approaches 2^{44} ; a keyed permutation reaches the full 2^{88} range with no repetition, provided the input index never repeats for a given node (protocol specification §5.4.1.6, notes). The remaining rationale — per-account sequences pseudorandom and mutually uncorrelated, and the issue-count leak a bare counter would cause — is the *Ironwood Guide*’s diversifier construction, cited above.

Remark 2.2 (FF1 parameter security). The property the design requires is that FF1-AES256, with the tweak fixed to the empty string, is a secure pseudo-random permutation. As parameterized here — radix 2, an 88-bit domain, 10 rounds per NIST SP 800-38G — it is unaffected by the DKLS2020 attacks on small-domain FF1: at their parameters $r = 5$ (half the number of rounds) and $n = 44$ (half the domain size in bits), the distinguishing attack has data complexity $2^{2n((r-1)-1/2)-n} = 2^{264}$ and time complexity $2^{2n((r-1)-1/2)} = 2^{308}$, no better than brute-forcing the 256-bit key (protocol specification §5.4.4).

Definition 2.3 (Diversifier validity and default diversifiers). For Sapling,

$$\text{DiversifyHash}^{\text{Sapling}}(d) := \text{GroupJHash}^{\text{URS}}(\text{"Zcash_gd"}, \text{LEBS2OSP}_{88}(d)),$$

valued in the prime-order subgroup of Jubjub — Sapling’s elliptic curve, carrying its keys and commitments as Pallas carries Orchard’s — excluding zero, or $\perp$; the diversifier d_j is *valid* iff the result is not $\perp$, which for a given dk holds for approximately half of all indices j . The Sapling *default diversifier* is d_j for the least nonnegative j yielding a valid diversifier, and a Sapling account has at most approximately 2^{87} payment addresses (ZIP-32, *Sapling diversifier derivation*; *Sapling key path*). For Orchard, with $P := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$,

$$\text{DiversifyHash}^{\text{Orchard}}(d) := \begin{cases} \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, \text{"}) & \text{if } P = \mathcal{O}, \\ P & \text{otherwise,} \end{cases}$$

which never returns $\perp$: every one of the 2^{88} Orchard diversifiers is valid, the default diversifier is d_0 , and an account has exactly 2^{88} payment addresses (protocol specification §5.4.1.6; ZIP-32, *Orchard diversifier and OVK derivation*; *Orchard key path*).

The totality of the Orchard hash — and why it removes Sapling’s try-the-next-index rejection loop — is discussed in the *Ironwood Guide* (“Keys: capabilities for spending and for viewing”). The residual Sapling failure rate of $\frac{1}{2}$ per index is not a curiosity: it resurfaces when a single index must serve several protocols at once (§2.7).

The two protocols also place dk at different depths of the key tree. In Sapling the diversifier key is a component of the *extended spending key*, derived down the ZIP-32 tree beside the spending authority:

$$dk_m := \text{truncate}_{32}(\text{PRFexpand}_{s_{k_m}}([0x10])), \quad dk_i := \text{truncate}_{32}(\text{PRFexpand}_{I_L}([0x16] \parallel dk_{\text{par}})),$$

where I_L is the ZIP-32 child-derivation entropy (ZIP-32, *Sapling master key generation*; *Sapling child key derivation*; domain bytes also in `zcash_spec 0.2.1, src/prf_expand.rs`). Orchard instead derives dk directly from the full viewing key (ak, nk, r_{ivk}), as one half of a single PRFexpand output whose other half is ovk :

$$R := \text{PRFexpand}_{\text{LE}_{256}(r_{\text{ivk}})}(0x82 \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk)), \\ dk := R[0..32), \quad ovk := R[32..64),$$

the derivation the *Ironwood Guide* types in full ($\text{PRFexpand}_K(t)$ is BLAKE2b-512 with personalisation `Zcash_ExpandSeed` over $K \parallel t$; ZIP-32, *Conventions*; protocol specification §4.2.3). The consequence

is capability parity with a simpler key structure: any holder of the Orchard full viewing key can determine the position of any diversifier in the account's sequence, as in Sapling only a holder of the *extended* full viewing key can (ZIP-32, *Orchard diversifier and OVK derivation*).

Remark 2.4 (Reuse of r_{ivk}). The randomizer r_{ivk} serves twice: as the Commit^{ivk} trapdoor and as the PRFexpand key deriving dk and ovk . If dk or ovk is known to an adversary, this reuse prevents proving *perfect* hiding for Commit^{ivk} in this context, and modelling PRFexpand merely as a PRF is insufficient; the specification instead makes a joint assumption on Pallas scalar multiplication and PRFexpand (protocol specification §4.2.3, security note).

Inverting the permutation recovers the index:

$$j = \text{FF1-AES256.Decrypt}(dk, "", d).$$

Decryption alone proves nothing about ownership: every valid diversifier decrypts to *some* index under *every* diversifier key (sapling-crypto 0.7.0, `src/zip32.rs`, doc comment: “all valid diversifiers can be produced by all diversifier keys”), so the ownership test re-derives the address at the decrypted index under the candidate ivk and compares it with the given address. Deployed: the method `DiversifierKey::diversifier_index` performs the raw FF1 decryption in both protocols, and the checked variants are the Orchard `IncomingViewingKey::diversifier_index` and the Sapling `IncomingViewingKey::decrypt_diversifier` (orchard, `src/keys.rs`; sapling-crypto 0.7.0, `src/zip32.rs`).

Remark 2.5 (Unlinkability, and an oracle to avoid). The security requirement: given two randomly selected shielded payment addresses of *different* spend authorities and a third address derivable from either, all three using different diversifiers, it is not possible to tell from which of the two authorities the third address derives. For Sapling this holds under DDH on the Jubjub prime-order subgroup when `GroupJHash` is modelled as a random oracle, via IK-CPA key privacy of ElGamal; the specification applies the same property and notes to Orchard (protocol specification §5.4.1.6). One pitfall is normative: an implementation SHOULD avoid providing a *chosen-diversifier* oracle — deriving a new address for a given ivk at an adversary-supplied diversifier — since such an oracle trivially breaks unlinkability for the other diversified addresses of that ivk (same section, final note).

2.3 Raw encodings

An Orchard shielded payment address encodes raw in 43 bytes,

$$\text{LEBS2OSP}_{88}(d) \parallel \text{pk}_d^*,$$

the 11-byte diversifier followed by the 32-byte star-encoding of pk_d ; a decoder rejects the encoding if the point fails to parse or is the zero point (protocol specification §5.6.4.2, *Orchard Raw Payment Addresses*). A Sapling address is likewise $11 + 32 = 43$ bytes under Jubjub ctEdwards compression, rejected if parsing fails, the encoding is non-canonical, or pk_d lies outside the prime-order subgroup minus zero — the exclusion of zero dates from specification version 2025.6.0 (protocol specification §5.6.3.1, *Sapling Payment Addresses*).

The Orchard raw *full viewing key* is 96 bytes,

$$\text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk) \parallel \text{I2LEOSP}_{256}(r_{ivk}),$$

invalid if any component is a non-canonical field element, if ak is not a valid affine x -coordinate of a Pallas point, or if either the external or the internal ivk derived from it is 0 or $\perp$; the deployed `FullViewingKey::from_bytes` enforces exactly these checks, including both scopes' ivk validity (protocol specification §5.6.4.4, *Orchard Raw Full Viewing Keys*; orchard, `src/keys.rs`). The Orchard raw *incoming viewing key* is 64 bytes, $dk \parallel \text{I2LEOSP}_{256}(ivk)$, where ivk MUST be a nonzero canonical Pallas base field element; deployed as `IncomingViewingKey::to_bytes`, `from_bytes` over a

NonZeroPallasBase (protocol specification §5.6.4.3, *Orchard Raw Incoming Viewing Keys*; orchard, src/keys.rs).

None of these raw Orchard encodings has a string form of its own: no Bech32 or Bech32m encoding is defined for an individual Orchard address, incoming viewing key, or full viewing key — the unified encodings below are their only string carrier (protocol specification §5.6.4.2). Sapling, by contrast, retains standalone address strings (mainnet HRP `zs`, testnet `ztestsapling`; protocol specification §5.6.3.1), and the deployed renderer honours the asymmetry: the method `Receiver::to_zcash_address` emits a Sapling receiver as a bare `zs` address and a transparent receiver as a `t`-address, but an Orchard receiver only as a single-receiver unified address (`zcash_keys, address.rs`).

2.4 Unified addresses

A unified address (UA) bundles at most one *receiver* per pool into one string, so the recipient distributes a single address and the sender’s wallet chooses the pool. The choice is normative, by the *priority list* of Table 2: the Sender of a payment to a UA MUST use the receiver of the most preferred type it supports — Orchard over Sapling over P2SH (pay-to-script-hash: the output pays the hash of a script, revealed and satisfied when spending) over P2PKH. A wallet MAY let its user configure which receiver types it can send to, but MUST NOT allow the priority order itself to be changed, except via experiments (ZIP-316, *Encoding of Unified Addresses*; mirrored in protocol specification §5.6.4.1, *Unified Payment Addresses and Viewing Keys*).

Typecode	Item type	Receiver	UFVK item	UIVK item
0x00	transparent P2PKH	20	65	65
0x01	transparent P2SH	20	—	—
0x02	Sapling	43	128	64
0x03	Orchard	43	96	64

Table 2: Revision 0 item typecodes with byte lengths of the receiver, UFKV-item, and UIVK-item encodings. *Priority* is descending typecode among these four — 0x03 is most preferred — while encoding order is *ascending* typecode, deliberately not priority order. P2SH viewing-key items do not exist in Revision 0 (ZIP-316, *Encoding of Unified Addresses*; *Encoding of Unified Full/Incoming Viewing Keys*).

Definition 2.6 (Unified container, raw encoding). The raw encoding of a UA, UFKV, or UIVK is the concatenation, over its items in *ascending* typecode order, of

$$\text{compactSize}(\text{typecode}) \parallel \text{compactSize}(\ell_{\text{item}}) \parallel \text{item},$$

where `compactSize` is Bitcoin’s variable-length integer encoding — one byte for values below 253, otherwise a marker byte followed by the value in 2, 4, or 8 little-endian bytes — and typecode and length values MUST be at most 0x02000000 (the deployed bound is `MAX_COMPACT_SIZE` in `zcash_encoding, src/lib.rs`). A transparent receiver is the bare 20-byte hash — the P2PKH validating-key hash or the P2SH script hash — excluding the two version bytes of the base58check raw encoding (ZIP-316, *Encoding of Unified Addresses*).

Encoding in a fixed, non-priority order makes the byte encoding canonical; the deployed container preserves the parse order in `items_as_parsed` for round-trip serialization and separately offers `items()` sorted by preference (ZIP-316, *Rationale for Item ordering*; `zcash_address, kind/unified.rs`).

Revision 0 imposes containment rules. A UA or UVK MUST contain at least one non-metadata item; a Revision 0 UA MUST contain at least one *shielded* item (typecode 0x02 or 0x03), and a Revision 0 UVK likewise (Revision 2 drops the requirement for UVKs). A consumer MUST reject: duplicate typecodes; both P2SH and P2PKH present; only metadata items; items out of ascending typecode order; trailing bytes; or any constituent item failing its own validation. A consumer MUST ignore items with unrecognized typecodes (except the MUST-understand metadata of §2.8), and every wallet must still parse a container holding unrecognized items (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys; Receivers*). There is intentionally no typecode for Sprout addresses or viewing keys: ZIP-211 disabled sending funds into the Sprout pool at Canopy (ZIP-316; protocol specification §5.6.4.1).

The deployed parser implements these rules once, identically for all three container types (Address, Ufvk, Uivk): the function `try_from_items_internal` rejects out-of-order typecodes (InvalidTypecodeOrder), duplicates (DuplicateTypecode), the P2PKH/P2SH combination (BothP2phkAndP2sh), and only-transparent containers (OnlyTransparent); typecodes 0x04 through 0x02000000 parse as `Typecode::Unknown`, and larger values fail as `InvalidTypecodeValue` (`zcash_address, kind/unified.rs`).

Remark 2.7 (Unknown items versus the shielded requirement). The deployed code renders “at least one shielded item” as “not only transparent items”, and deliberately counts unknown typecodes as *not* transparent (comment in `zcash_address, kind/unified.rs`): a UA containing only a P2PKH receiver plus one unknown-typecode item parses successfully, although it contains no item of typecode 0x02 or 0x03. The ZIP’s parenthetical “currently Typecodes 0x02 and 0x03” suggests this forward-compatible reading is intended, but the ZIP nowhere states that an unknown item may satisfy the shielded requirement. We flag the divergence and do not resolve it.

Within a single container, all items obtained by hierarchical deterministic derivation SHOULD represent the same account — the ZIP-32 and BIP-44 account level (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys*). Above the parser, the shielded requirement is enforced at construction as well: `UnifiedAddress::from_receivers` returns `None` without a Sapling or Orchard receiver, and address generation fails with `ShieldedReceiverRequired` (`zcash_keys, address.rs, keys.rs`).

The deployed sendable-address surface is the enum `zcash_keys::address::Address`: a bare Sapling `PaymentAddress`, a `TransparentAddress` (P2PKH or P2SH), a Unified address, or a `Tex` address — the ZIP-320 transparent-source-only form wrapping a 20-byte P2PKH hash. A unified address can receive a memo iff it has a Sapling or Orchard receiver (`unified::Address::can_receive_memo; zcash_address, kind/unified/address.rs`).

2.5 String encoding: F4Jumble and Bech32m

The raw container still needs a string form, and the obvious choice — Bech32m directly over the raw bytes — has a human-factors flaw at UA lengths. The Bech32m checksum does not *cascade*: a user who visually verifies only the beginning and end of a long string can miss an adversarial substitution of an internal receiver. ZIP-316 therefore passes the whole payload through *F4Jumble*, an unkeyed four-round Feistel construction approximating a random permutation of the entire byte string, before the checksummed encoding: any local change diffuses over the whole string, giving near-second-preimage resistance and nonmalleability — for example, an adversary cannot delete a shielded receiver so that only a transparent one remains. Three rounds would not suffice: a controlled xor-difference attack fixes the input to H_0 . The HRP, zero-padded to 16 bytes, is appended *inside* the jumbled payload, extending the same anti-malleation protection to the HRP itself and defending against cross-network confusion and address-versus-viewing-key confusion (ZIP-316, *Jumbling; f4jumble, src/lib.rs, crate documentation*).

Definition 2.8 (Unified string encoding). Let raw be the encoding of Definition 2.6 and $padding$ the container’s HRP in US-ASCII, padded to exactly 16 bytes with zero bytes; the total length must satisfy $\ell(raw) \leq \ell_M^{\text{MAX}} - 16$. The string encoding is

$$\text{Bech32m}(\text{HRP}, \text{F4Jumble}(raw \parallel padding)),$$

ignoring the BIP-350 length restrictions (Bech32m was chosen over Bech32 for better checksum behaviour at variable lengths). Decoding MUST: Bech32m-decode, rejecting bad checksums; reject out-of-range lengths; apply F4Jumble^{-1} ; verify and strip the trailing 16-byte padding; then parse the items, rejecting the whole encoding on any failure (ZIP-316, *Encoding of Unified Addresses*; deployed as `to_jumbled_bytes` and `parse_items` in `zcash_address, kind/unified.rs`).

Container	Mainnet	Testnet	Regtest
Unified address	u	utest	uregtest
Unified FVK	uview	uviewtest	uviewregtest
Unified IVK	uivk	uivktest	uivkregtest

Table 3: Revision 0 human-readable parts, as deployed. ZIP-316 defines the mainnet and testnet rows (ZIP-316, *Revisions*); the regtest column is a `librustzcash` extension, present only in `zcash_protocol, constants/regtest.rs`.

Definition 2.9 (F4Jumble). Let M be a message of ℓ_M bytes with $38 \leq \ell_M \leq \ell_M^{\text{MAX}} := (2^{16} + 1) \cdot 64 = 4194368$, and let $\ell_H := 64$. Set $\ell_L := \min(\ell_H, \lfloor \ell_M/2 \rfloor)$ and $\ell_R := \ell_M - \ell_L$, and split $M = a \parallel b$ with $\ell(a) = \ell_L$. Then, with all letters local to this definition,

$$x := b \oplus G_0(a), \quad y := a \oplus H_0(x), \quad d := x \oplus G_1(y), \quad c := y \oplus H_1(d),$$

and $\text{F4Jumble}(M) := c \parallel d$. The round functions are

$$H_i(u) := \text{BLAKE2b-}(8\ell_L)(\text{"UA_F4Jumble_H"} \parallel [i, 0, 0], u),$$

$$G_i(u) := \text{first } \ell_R \text{ bytes of } \big\|_{j=0}^{\lceil \ell_R/\ell_H \rceil - 1} \text{BLAKE2b-512}(\text{"UA_F4Jumble_G"} \parallel [i] \parallel \text{I2LEOSP}_{16}(j), u),$$

each personalisation filling the 16-byte BLAKE2b personal field. The inverse applies the four XOR rounds in reverse order (ZIP-316, *Jumbling*).

The deployed implementation keeps the state as $left \parallel right$ with $left$ the first $\min(64, \lfloor \ell_M/2 \rfloor)$ bytes, and runs $\text{g_round}(0)$ ($right \oplus= G_0(left)$), emulating the XOF (extendable-output function; *Crypto Guide*, § “Hash functions and the random oracle model”) chunkwise in 64-byte blocks), $\text{h_round}(0)$, $\text{g_round}(1)$, $\text{h_round}(1)$, with the inverse running the same rounds in reverse; this matches the formulas above under the identification of (a, b, x, y, c, d) with the successive half-states (`f4jumble, src/lib.rs`). One bound differs: the crate accepts message lengths $48 \leq \ell_M \leq 4194368$ (`VALID_LENGTH`) — the Revision 0 minimum of 48 rather than Revision 2’s 38. ZIP-316 states the difference is unobservable for Revision 0-valid item sets, since no such set parses to 22–31 content bytes, and Revision 2 consumers MAY apply either minimum to Revision 0 containers (ZIP-316, *Rationale for length restrictions*; `f4jumble, src/lib.rs`). On the checksum side, the deployed encoder uses a custom checksum type `Bech32mZip316`, identical to `Bech32m` except that `CODE_LENGTH = 4194368 = \ell^{\text{MAX}}` lifts the BIP-350 length cap; decoding rejects any string whose HRP does not match a known network (main, test, regtest) for the container type (`zcash_address, kind/unified.rs`).

The jumble is cheap at address lengths: 4 BLAKE2b compressions for $\ell_M \leq 128$ bytes — which covers both common UA shapes, since a Sapling-plus-Orchard UA has $\ell_M = 45 + 45 + 16 = 106$ bytes and a transparent-plus-Sapling-plus-Orchard UA $\ell_M = 22 + 45 + 45 + 16 = 128$ — rising to 6 for $\ell_M \leq 192$, 10 for $\ell_M \leq 256$, and 196608 at $\ell_M = \ell_M^{\text{MAX}}$. A consumer unable to handle maximum-length inputs MUST still support $\ell_M \geq 256$ bytes: two conformance levels (ZIP-316, *Efficiency*).

Display is normative too: if a UA or UVK is abridged, at least the first 20 characters MUST be shown. The worst case — the longest mainnet HRP, `uview`, plus the separator — leaves 14 characters, about 70 bits of jumbled data, below the 2^{125} security target, so 20 is an absolute minimum; and only *initial* characters count, since different displays may otherwise show disjoint substrings of the same string (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys*, and its rationale).

Example 2.10 (A unified address, end to end). We trace the pipeline on the smallest deployed shape — an Orchard-only mainnet UA — using the test vector derived from the 32-byte seed `000102...1f` at account 9, diversifier index $j = 1$ (`zcash_address, kind/unified/address/test_vectors.rs`). Every value below is script-computed; the key half runs the deployed `SpendingKey::from_zip32_seed` and `FullViewingKey::address_at` (`orchard, src/keys.rs`).

Receiver. The account key at $m/32'/133'/9'$ yields the full viewing key, whose PRFexpand image under domain byte `0x82` gives `dk`; FF1 encryption of the index bytes (Definition 2.1) gives d_1 , and the key agreement of §2.2 gives pk_d :

```

dk          82cc9d79742fe5ae9a142b9336a98677b154fe20401eb18998dbed915b0453ce
I2LEOSP88(1) 01000000000000000000000000000000
d1         3fadb8edb20a3301e8260a
pkd*       a311f4cbd54d7d6a76baac88c244b0b121c6dc22a8bcce15898e267829fc1e01

```

Decryption inverts the permutation: `FF1-AES256.Decrypt(dk, "", d1)` returns the index bytes above, recovering $j = 1$ — subject to the ownership caveat of §2.2. The neighbouring diversifiers $d_0 = \text{e340636542ece1c81285ed}$ and $d_2 = \text{987fd74a2256c596a66f83}$ share no visible structure with d_1 : the sequence is pseudorandom in the index.

Container. The UA has one item, so the raw encoding of Definition 2.6 is the two header bytes `compactSize(0x03) || compactSize(43) = 032b` followed by the 43-byte receiver $d_1 || pk_d^*$, 45 bytes in all. Appending the padded HRP, 75 ("u") then fifteen zero bytes, gives the 61-byte F4Jumble input M with $\ell_L = 30$, $\ell_R = 31$. The four rounds of Definition 2.9, letters as there:

```

a = M[0..30)  032b3fadb8edb20a3301e8260aa311f4cbd54d7d6a76baac88c244b0b121
b = M[30..61) c6dc22a8bcce15898e267829fc1e017500000000000000000000000000000000
x = b ⊕ G0(a) e6f1529b03a0ad0a4209da8e4365ca14a5988bf1d03084a1c326f929c574ca
y = a ⊕ H0(x) 37b358eb784bb2b315fd35c595628cfa9aa8045ef271728dd0cbf84dc81a
d = x ⊕ G1(y) ad26d7fb043e09cbf18e8f4d80d2b423ecb01719170463d644be4c76daab6b
c = y ⊕ H1(d) 98979f9a7d2dfce7b1616a8d2b9975f87878171b6fcf33b303f770b4aa64

```

The plaintext structure is legible in a and b — the TLV header `032b`, the diversifier, the padding — and none of it in $F4Jumble(M) = c || d$.

String. Regrouping the 61 jumbled bytes into 98 five-bit symbols (the last padded with two zero bits) and appending the 6-symbol Bech32m checksum yields, with the HRP and separator, $1 + 1 + 98 + 6 = 106$ characters:

```
u1nztelxna9h7w0vtpd2xjht4lpu8s9cmdl8n8vcr7actf2ny45nd07cy8cyuhuvw3axcp545y0ktq9cezuzx84jyhex8dk4tdvwhu4dl
```

Decoding reverses each stage per Definition 2.8, the inverse jumble running the same rounds in reverse order. The generating scripts reproduce this string through the deployed encoder (`zcash_address, unified::Address::encode`), re-encode all 60 vectors of the file above, and match all 8 vectors of `f4jumble, src/test_vectors.rs`, in both directions.

2.6 Unified viewing keys

A unified full viewing key (UFVK) and unified incoming viewing key (UIVK) reuse the container wholesale: the same item layout (Definition 2.6), the same containment rules, and the same jumbled

Bech32m pipeline under their own HRPs (Table 3), with viewing-key items in place of receivers at the same typecodes (Table 2; ZIP-316, *Encoding of Unified Full/Incoming Viewing Keys*). The items:

- *Orchard* (0x03): the UFVK item is the 96-byte raw full viewing key of §2.3; the UIVK item is the 64-byte raw incoming viewing key $dk \parallel I2LEOSP_{256}(ivk)$.
- *Sapling* (0x02): the UFVK item is 128 bytes, $EncodeExtFVKParts(ak, nk, ovk, dk) = ak^* \parallel nk^* \parallel ovk \parallel dk$ under Jubjub compression, and SHOULD be derived from the account-level ZIP-32 extended full viewing key (ZIP-316; the encoding function is defined in ZIP-32). The UIVK item is 64 bytes, $dk \parallel I2LEOSP_{256}(ivk)$ with a zero *ivk* invalid (per protocol specification version 2025.6.0) — deliberately *more* than the protocol specification’s Sapling incoming viewing key, which is *ivk* alone: carrying *dk* is what makes UAs derivable from a UIVK (sapling-crypto 0.7.0, *src/zip32.rs*).
- *Transparent P2PKH* (0x00): the UFVK item is 65 bytes, the chain code *c* followed by the 33-byte compressed public key — the last 65 bytes of the BIP-32 extended public key — at the account level *m/44'/coin_type'/account'*; the UIVK item has the same shape one level down, at the external change level *m/44'/coin_type'/account'/0* (the deployed doc comment pins *m/44'/133'/account'/0* for mainnet). Items of type P2SH (0x01) MUST NOT appear in Revision 0 UFVKs or UIVKs (ZIP-316, *Encoding of Unified Full/Incoming Viewing Keys*).

The deployed item types mirror the sizes exactly: `unified::Fvk::{Orchard([u8; 96]), Sapling([u8; 128]), P2pkh([u8; 65])}` and the `unified::Ivk` analogues (64/64/65) (`zcash_address, kind/unified/fvk.rs, kind/unified/ivk.rs`).

Remark 2.11 (Viewing-key items are pruned). The transparent items deliberately omit the BIP-32 extended-key metadata — depth, parent fingerprint, child number — because the child number would reveal the wallet account index; deserialization reconstitutes dummy attributes (depth 3 for the account key, depth 4 for the IVK, parent fingerprint 0xffffffff) (`zcash_address, kind/unified/fvk.rs; zcash_transparent, keys.rs`). The Sapling item is pruned as well: the deployed `DiversifiableFullViewingKey` is exactly (ak, nk, ovk, dk) with no chain code, so round-tripping a UFVK loses the ability to perform non-hardened Sapling child derivation below the account (sapling-crypto 0.7.0, *src/zip32.rs*) — consistent with the ZIP, which only says the item SHOULD be *derived from* the account-level extended full viewing key.

The priority list of Table 2 does not govern viewing keys the way it governs receivers: a consumer of a UVK normally takes the *union* of the information in all items rather than selecting one, and although the current FVK and IVK types reuse the receiver typecodes, future viewing-key types need not correspond to receiver types (ZIP-316, *Encoding of Unified Full/Incoming Viewing Keys*). The deployed container nevertheless returns `items()` sorted in preference order for UVKs just as for UAs, keeping `items_as_parsed` for round-trips (`zcash_address, kind/unified.rs`); the one place preference genuinely orders viewing-key material is the selection of outgoing viewing keys, which ZIP-316 specifies separately.

Namely, ZIP-316 refines which *ovk* a sender uses for its outgoing ciphertexts (the C_{out} construction of the *Ironwood Guide*, “Transmission and detection of a note”): the Sender determines a sending account — preferably from the wallet API, else when all inputs belong to a single account — and a *preferred sending protocol*, the most preferred receiver type among the funds being sent, and SHOULD use the external or internal *ovk* (according to the transfer type) of that protocol from the account-level full viewing key; if no account can be determined, it SHOULD pick one source address of the preferred protocol and use its FVK’s *ovk* (ZIP-316, *Usage of Outgoing Viewing Keys*). Deployed: `UnifiedFullViewingKey::select_ovk(scope, input_sources)` picks the OVK of the highest-preference pool represented among the inputs (`zcash_keys, keys.rs`).

The external/internal split just used comes with a strict capability separation: a full viewing key holder can derive both the external and the internal incoming viewing keys and *ovks*; the external IVK holder can derive neither the internal IVK nor any *ovk*; the external *ovk* holder can derive neither the

internal ovk nor any IVK. For the shielded protocols this follows from one-wayness of PRFexpand and of $\text{CRH}^{\text{ivk}}/\text{Commit}^{\text{ivk}}$, with the internal-key derivations specified in ZIP-32 (ZIP-316, *Deriving Internal Keys*). The Orchard internal full viewing key replaces only the commitment randomizer — the 0x83 derivation of Definition 1.6, from which the internal dk, ivk, and ovk follow by the standard component derivations. Transparent P2PKH accounts, having no protocol-level ovk, get the synthetic 0xd0 pair of Definition 1.7 (ZIP-316, *Deriving Internal Keys*; domain bytes in `zcash_spec 0.2.1, src/prf_expand.rs`).

A UIVK derives from a UFVK item by item, preserving typecodes: the Sapling FVK maps to its IVK per protocol specification §4.2.2, the Orchard FVK to its external-scope IVK per §4.2.3, and the transparent P2PKH account key to the external-change key by non-hardened child derivation at index 0; items with unrecognized typecodes MUST be dropped (as they must be again when deriving a UA from a UIVK). The deployed method `UnifiedFullViewingKey::to_unified_incoming_viewing_key` maps Sapling via `to_external_ivk`, Orchard via `to_ivk(Scope::External)`, transparent via `derive_external_ivk`, and sets the unknown-item list to empty, matching the ZIP (ZIP-316, *Deriving a UIVK from a UFVK*; `zcash_keys, keys.rs`).

2.7 Deriving a unified address from a UIVK

A UA derives from a UIVK at a *single* diversifier index j , and j must be valid simultaneously for every item: the Sapling component requires j to yield a valid Sapling diversifier (Definition 2.3); the transparent component requires $0 \leq j \leq 2^{31}-1$, since j becomes the BIP-44 non-hardened index-level child, giving the receiver path `m/44'/coin_type'/account'/0/j`; the Orchard component imposes no constraint (ZIP-316, *Deriving a Unified Address from a UIVK*). The deployed index map makes the transparent constraint concrete: the function `to_transparent_child_index` splits the 11 little-endian index bytes into the low 4 and the high 7, requires the high 7 to be zero, and parses the low 4 as a `NonHardenedChildIndex`, whose maximum is $2^{31}-1$ — exactly the ZIP range (`zcash_keys, keys.rs`; `zcash_transparent, keys.rs`).

Which receivers a generated UA carries is decided per call, by the request type `UnifiedAddressRequest`: either `AllAvailableKeys` or `Custom` over `ReceiverRequirements`, one setting of `Require`, `Allow`, or `Omit` for each of Orchard, Sapling, and P2PKH; the constructor rejects any combination in which both Orchard and Sapling are `Omit`, since no shielded receiver could result. The preset constants are `ALLOW_ALL = (Allow, Allow, Allow)`, `SHIELDED = (Allow, Allow, Omit)`, and `ORCHARD = (Require, Omit, Omit)`. Intersecting two requirements prefers `Require` and `Omit` over `Allow` and errors on `Require` meeting `Omit` (`zcash_keys, keys.rs`).

Generation at an index, `UnifiedIncomingViewingKey::address(j, request)`, derives each requested receiver at the same j : the Orchard receiver via `address_at(j)`, which is total and never fails; the Sapling receiver via the diversifiable IVK's `address_at(j)`, which yields no receiver at an invalid diversifier — an error only when Sapling is `Required`, a silent omission under `Allow`; the transparent receiver via `to_transparent_child_index` and public derivation, where an out-of-range index errors under `Require` and omits under `Allow`. The result must retain at least one shielded receiver. The search `find_address(j, request)` increments j *only* on `InvalidSaplingDiversifierIndex`, returns `DiversifierSpaceExhausted` on overflow past $2^{88}-1$, and aborts on any other error; `default_address` is `find_address(0)`. The deployed *default unified address* therefore sits at the smallest index valid for all required receivers (`zcash_keys, keys.rs`).

Remark 2.12 (Whose convention the default address is). ZIP-316 does not specify address-search semantics at all; the smallest-valid-index default above is a `zcash_keys` convention, not a ZIP mandate — only ZIP-32's per-protocol default diversifiers (Definition 2.3) are normative. The search's tolerance is also narrow: it skips an index only for an invalid Sapling diversifier, so with a `Required` transparent receiver a search that starts or arrives at an index of 2^{31} or beyond — or a failed BIP-32 child derivation, probability about 2^{-127} — aborts generation rather than advancing (`zcash_keys, keys.rs`).

Remark 2.13 (Index 0 and cross-wallet recovery). Diversifier index 0 fails to yield a valid Sapling diversifier with probability $\frac{1}{2}$, so a wallet requiring a Sapling receiver often issues its first UA at a higher index — yet some wallets always generate a transparent P2PKH address at index 0. All Zcash wallets **MUST** therefore assume there may be transparent funds at diversifier index 0 of each ZIP-32 account, even if the wallet itself would never generate a UA at that index; reliable cross-wallet fund recovery depends on this (ZIP-316, note in *Deriving a Unified Address from a UIVK*).

2.8 Revision 2 (specified, not deployed)

Revision 2 of ZIP-316 is a Draft as of the zips snapshot c6b7035; everything in this subsection is classified *specified*, and none of it exists in the deployed crates — the reference implementation is librustzcash PR #2199, and a search of the local zcash_address sources finds none of the new HRPs, typecodes, or expiry logic. Revision 2 adds (ZIP-316, *Revisions*; Revision 2 changelog):

- *New HRPs*. Unified addresses split into zu (shielded-only; transparent items prohibited) and tu (transparent-enabled); UIVKs use uvf and UIVKs uvi; testnet forms append test to each HRP.
- *Metadata items*. Typecodes 0xC0–0xFC are reserved for metadata, which **MUST NOT** be selected as receivers and **MUST NOT** confer viewing authority. The subrange 0xE0–0xFC is *MUST-understand*: a consumer that does not recognize such an item **MUST** treat the whole container as unsupported. Typecodes 0xFFFFA–0xFFFF are experimental, and registry additions come via 2xxx-series ZIPs (ZIP-316, *Metadata Items*; *Typecode Registry*; *Adding new types*).
- *Address expiration*. Typecode 0xE0 carries a little-endian u32 Address Expiry Height — the address is expired once the chain height exceeds it — and 0xE1 a little-endian u64 Unix-time Address Expiry Time. A Revision 2 Sender **MUST** set a nonzero nExpiryHeight in transactions sent to an address that defines an Address Expiry Height; **MUST NOT** send if its normal nExpiryHeight would exceed the Address Expiry Height, which would leak the address’s expiry into the transaction; and **SHOULD** arrange expiry within 24 hours when only an expiry time is given. Expiry metadata is retained when deriving a UIVK and must not increase when deriving a UA; because shared expiry metadata can link diversified addresses, per-address variation is **RECOMMENDED** (ZIP-316, *Address Expiration Metadata*).
- *Viewing keys*. The at-least-one-shielded-item requirement is dropped for UVKs, and P2SH viewing-key items (BIP-388 wallet policies) are added (ZIP-316, Revision 2 changelog).

Remark 2.14 (A retroactive rule the deployed parser does not enforce). Revision 2 also legislates for Revision 0 strings: a Revision 0 container — identified by its u/uview/uivk HRP — **MUST NOT** contain *MUST-understand* typecodes, and consumers **MUST** reject violations (ZIP-316, *Metadata Items*). The deployed parser predates this amendment and does not enforce it: every typecode from 0x04 to 0x02000000 parses as ignorable Typecode: :Unknown, so a u-prefixed address carrying an 0xE0 expiry item parses successfully today (zcash_address, kind/unified.rs). This is correct Revision 0 behaviour under the pre-amendment text and a violation of the Revision 2-amended text; we flag the divergence rather than resolve it.

3 Fees (ZIP-317)

Every transaction pays a fee, and the fee is public: it is the transparent value the transaction leaves unclaimed. For a shielded protocol this publicity imposes two requirements that an ordinary fee market does not. The fee must be *computable from public transaction fields alone* — a fee that depended on private data (which notes were spent, what change was returned) would leak that data through the amount paid (ZIP-317, *Requirements*) — and it should be *uniform across wallets*, because a wallet paying an idiosyncratic fee fingerprints every transaction it builds. ZIP-317 resolves both with a

single *conventional fee*: a deterministic function of the transaction’s public shape that every wallet is expected to pay.

Remark 3.1 (A convention, not a consensus rule). The conventional fee is a wallet convention, not a consensus requirement. ZIP-317 (*Fee calculation*) states: “It is not a consensus requirement that fees follow this formula; however, wallets SHOULD create transactions that pay this fee, in order to reduce information leakage, unless overridden by the user.” Enforcement is economic, not validity-level: nodes are expected to relay, and the recommended block-template algorithm to preferentially mine, transactions paying at least the conventional fee (§3.6).

ZIP-317 is organised into *revisions* (Last-Updated 2026-06-26): Revision 0 is Active and is the deployed fee mechanism; Revision 1 (a fee contribution for Ironwood-pool Actions, NU6.3) and Revision 2 (memo bundles, depending on the undeployed ZIP 248) are both Draft. A contribution defined by a revision is included in all later revisions. This volume documents Revision 0 as the base rule and states Revisions 1–2 below. Although its document status remains Draft, Revision 1 is implemented and is the fee contribution used for active NU6.3 Ironwood bundles (§3.3); Revision 2 remains *specified, not deployed*.

3.1 The conventional fee

The unit of account is the *logical action*: a pool-neutral measure of how much verification work and chain space a transaction demands. Its inputs are public transaction fields defined in the protocol specification §7.1 (*Transaction Encoding and Consensus*): the total byte sizes $tx_in_total_size$ and $tx_out_total_size$ of the transparent tx_in and tx_out fields, and the counts $nJoinSplit$, $nSpendsSapling$, $nOutputsSapling$, $nActionsOrchard$, and — from v6 — $nActionsIronwood$.

Definition 3.2 (Logical actions, ZIP-317 Revision 0). The transaction’s pools contribute

$$\begin{aligned} contribution_{Transparent} &= \max\left(\left\lceil \frac{tx_in_total_size}{150} \right\rceil, \left\lceil \frac{tx_out_total_size}{34} \right\rceil\right), \\ contribution_{Sprout} &= 2 \cdot nJoinSplit, \\ contribution_{Sapling} &= \max(nSpendsSapling, nOutputsSapling), \\ contribution_{Orchard} &= nActionsOrchard, \end{aligned}$$

and $logical_actions$ is the sum of the contributions of the applicable revision (ZIP-317, *Fee calculation*).

Definition 3.3 (Conventional fee). The conventional fee, in zatoshis, is

$$conventional_fee = marginal_fee \cdot \max(grace_actions, logical_actions),$$

with $marginal_fee = 5000$ zatoshis per logical action and $grace_actions = 2$ (ZIP-317, *Fee calculation*).

The two byte-size divisors are the standard P2PKH sizes, chosen so that one typical transparent input or output weighs one logical action. The standard input size $150 = 36 + 110 + 4$ decomposes as the outpoint (36: the 32-byte transaction id and 4-byte index naming the output being spent), the script — 1 overall-length byte, 1 signature-length byte, a 72-byte signature, 1 sighash-type byte (the flag selecting which parts of the transaction the signature commits to), 1 pubkey-length byte, a 33-byte pubkey, and 1 byte of margin, totalling 110 — and the sequence field (4). The standard output size $34 = 8 + 26$ decomposes as the value (8) and the script — 1 script-length byte plus the 25-byte P2PKH script (ZIP-317, *Rationale for the chosen parameters*).

Why logical actions, and why these constants. A naïve schedule would charge per input plus per output. But an Orchard Action spends one note *and* creates one (the *Ironwood Guide*, “A single shielded payment, end to end”), and Orchard bundles are padded to a minimum shape (§3.2); charging inputs and outputs separately would penalise exactly that forced padding. Logical actions count an Action once and, via the max in each contribution, do not discriminate between pools (ZIP-317, *Rationale for logical actions*). The transparent contribution is size-based rather than count-based so that large scripts cannot be undercounted: a 2-of-3 P2SH multisig input is roughly 70% larger than a P2PKH input and is charged proportionally (ZIP-317, *Rationale for the chosen parameters*). The grace window $grace_actions = 2$ exists so that padding a one-action transaction to two actions — the standard behaviour of `zcashd` and the mobile SDKs — costs nothing; it also makes a low-value input worth sweeping: within the window, an input worth less than the marginal fee is still economic to include when a second input covers the payment plus fee. Two is also $min_actions$, the smallest economically relevant change-producing transaction (ZIP-317, *Rationale for the chosen parameters*). Finally, $marginal_fee = 5000$ makes the minimal two-action fee 10,000 satoshis, deliberately restoring the conventional fee that predated ZIP-313 (which had lowered it to 1,000).

Later revisions. Revision 1 (Draft, NU6.3) adds

$$contribution_{Ironwood} = nActionsIronwood;$$

Ironwood-pool Actions use the same Action structure as Orchard, so they are counted identically (ZIP-317, Revision 1). Revision 2 (Draft; ZIP-231 memo bundles, dependent on the unmerged ZIP 248 transaction format) adds

$$contribution_{Memos} = \max(0, nMemoChunks - free_memo_chunks),$$

$$free_memo_chunks = \begin{cases} 2 & \text{if } nOutputsSapling + nActionsOrchard + nActionsIronwood > 0, \\ 0 & \text{otherwise,} \end{cases}$$

bounding the additional fee for a ZIP-231 memo bundle by 0.0019 ZEC (ZIP-317, Revision 2). Revision 1 is implemented in the deployed crates and applies to NU6.3 Ironwood bundles (§3.3); Revision 2 is not implemented.

3.2 The fee is computed on padded counts

The counts in Definition 3.2 are those of the final, serialised transaction — *after* the builder pads each shielded bundle to its minimum shape. The deployed padding rules are those of the crates `librustzcash` builds against (commit `e30517e4`: `sapling-crypto` 0.7.0 and `orchard` 0.15.0, per its `Cargo.lock`):

- *Sapling* (`sapling-crypto` 0.7.0, `builder.rs`): a Transactional bundle with any spend or output requested (or with `bundle_required` set) uses $num_outputs = \max(outputs, 2)$, the constant 2 being `MIN_SHIELDED_OUTPUTS`. Spends are never padded under `BundleType::DEFAULT`: num_spends equals $spends$ when $spends > 0$ and 0 otherwise, so an output-only bundle has zero spend descriptions; the floor $\max(spends, 1)$ applies only when `bundle_required` is set. An empty, non-required bundle is omitted entirely.
- *Orchard* (`orchard` 0.15.0, `builder.rs`): a Transactional bundle with $requested = \max(num_spends, num_outputs) > 0$ (or `bundle_required`) uses $num_actions = \max(requested, min)$, the floor min being the bundle type’s `pad_to_minimum` count, default `DEFAULT_MIN_ACTIONS = 2`. The max form of $requested$ applies when the bundle’s flags enable cross-address transfers, as the baseline’s pre-NU6.3 bundles do (`Flags::ENABLED`); Remark 3.4 states the sum rule that replaces it when the flags disable them.

Consequently any nonempty Sapling or Orchard bundle contributes at least two logical actions, and a one-input, one-output payment within a single shielded pool pays exactly the 10,000-zatoshi minimum.

The builder wires the padded counts into the fee call: the method `get_fee` of the transaction builder passes the raw count of Sapling spends added, the Sapling output count via `BundleType::num_outputs` (padding to 2 when the bundle is nonempty), and the Orchard action count via `BundleType::num_actions` (padding to `DEFAULT_MIN_ACTIONS = 2` when nonempty) (`zcash_primitives`, `transaction/builder.rs`). The proposal-stage balance computation in `zcash_client_backend` does the same — additionally counting the prospective change outputs — using `sapling::builder::BundleType::DEFAULT` and `orchard::builder::BundleType::DEFAULT`, except for the current ZIP-318 canonical-crossing Ironwood bundle’s explicit unpadded choice (`zcash_client_backend`, `fees/common.rs` and `data_api/wallet/input_selection.rs`). Fee estimation and final construction therefore agree on the shielded counts by construction.

Remark 3.4 (The flag-dependent Orchard action count). Release orchard 0.15.0 — the version the baseline Cargo.lock pins — makes the requested-action count flag-dependent (`src/builder.rs`, `num_actions`): when a bundle’s flags disable cross-address transfers, a spend and an output can never share an Action, so `requested = num_spends + num_outputs` replaces the max. Its documentation warns explicitly that ZIP-317 fee estimation must account for the larger count. The deployed builders disable cross-address transfers only for legacy-Orchard-pool bundles under NU6.3, whose bundle version mandates the restriction; every bundle the deployed wallet builds before NU6.3 keeps the flag enabled (`Flags::ENABLED`) and the max, so the fee arithmetic above is unchanged.

3.3 The deployed fee rule

Module `transaction::fees::zip317` of `zcash_primitives` implements Definition 3.3 as the type `FeeRule`; the constructor `FeeRule::standard()` instantiates the four ZIP constants,

$$\begin{aligned} \text{MARGINAL_FEE} &= 5000, & \text{GRACE_ACTIONS} &= 2, \\ \text{P2PKH_STANDARD_INPUT_SIZE} &= 150, & \text{P2PKH_STANDARD_OUTPUT_SIZE} &= 34, \end{aligned}$$

and the derived `MINIMUM_FEE = 10,000` zatoshis is `MARGINAL_FEE · GRACE_ACTIONS`. Non-standard parameters are gated behind the `non-standard-fees` feature and reject zero input or output sizes (`zcash_primitives`, `transaction/fees/zip317.rs`). The generic trait method is

```
fee_required(params, target_height,
transparent_input_sizes, transparent_output_sizes,
sapling_input_count, sapling_output_count,
orchard_action_count, ironwood_action_count) -> Zatoshis
```

(`zcash_primitives`, `transaction/fees.rs`); the ZIP-317 implementation ignores `params` and `target_height` and computes

$$\text{fee} = 5000 \cdot \max\left(2, \max\left(\left\lceil \frac{t_{\text{in}}}{150} \right\rceil, \left\lceil \frac{t_{\text{out}}}{34} \right\rceil\right)\right) + \max(s_{\text{in}}, s_{\text{out}}) + a + i,$$

where t_{in} is the sum of the *known* transparent input sizes, t_{out} the sum of the transparent output sizes, $s_{\text{in}}, s_{\text{out}}$ the Sapling spend and output counts, a the Orchard action count, and i the Ironwood action count (zero in the V5 transactions the wallet builds before NU6.3); overflow yields a `BalanceError` (`zcash_primitives`, `transaction/fees/zip317.rs`).

One term of Definition 3.2 is absent, a scope restriction rather than a divergence. There is no Sprout term: the `FeeRule` trait has no `JoinSplit` parameter at all, because the deployed wallet stack cannot construct Sprout spends; the $2 \cdot n_{\text{JoinSplit}}$ contribution is spec-only. The Ironwood term of Revision 1 is implemented: the parameter $i = \text{ironwood_action_count}$ above enters *logical_actions*, and is zero in the V5 transactions the wallet builds before NU6.3. The deployed rule is exactly ZIP-317 Revision 1 minus Sprout.

Transparent input sizing. Transparent inputs are sized along two paths. At *proposal* stage, a UTXO (unspent transaction output, the transparent pool’s spendable coin) whose `script_pubkey` is P2PKH is counted as `InputSize::STANDARD_P2PKH = Known(150)`; any unrecognised script yields `InputSize::Unknown` (`zcash_client_backend`, `wallet.rs`, `WalletTransparentOutput::serialized_size`). An `Unknown` input makes the fee incomputable: `fee_required` returns `FeeError::UnknownP2shInputs`. The recognised forms are P2PKH, and multisig-in-P2SH where the redeem script is known (`zcash_primitives`, `transaction/fees/zip317.rs`; `zcash_transparent`, `builder.rs`). At *build* stage, the method `TransparentInputInfo::serialized_len` computes the P2PKH size exactly: $36 + (1 + 74 + 34) + 4 = 149$ bytes — the outpoint, a `scriptSig` of one `CompactSize` byte plus a 74-byte push of a 73-byte placeholder signature (`MAX_SIG_SIZE = 73`: a 72-byte DER signature plus the sighash-type byte) plus a 34-byte push of a 33-byte pubkey, and the sequence field — one byte under the ZIP’s 150, which includes a one-byte margin; P2SH multisig inputs are computed from the redeem script (`zcash_transparent`, `builder.rs`).

Remark 3.5 (149 versus 150). The proposal path therefore prices a P2PKH input one byte above its serialised size. The discrepancy is deliberate: the `FeeRule` documentation states that the rule “may slightly overestimate fees in case where the user is attempting to spend a large number of transparent inputs. This is intentional and is relied on for the correctness of transaction construction algorithms in the `zcash_client_backend` crate” (`zcash_primitives`, `transaction/fees/zip317.rs`). An overestimate can only leave a gratuity, never underfund the transaction.

Transparent output sizing. The trait method `OutputView::serialized_size` returns 8 (value) plus the script bytes plus the `CompactSize` length prefix; a P2PKH output is $8 + 1 + 25 = 34$ bytes, matching the ZIP constant (`zcash_primitives`, `transaction/fees/transparent.rs`).

The standard rule at the wallet layer. The crate `zcash_client_backend` exposes `StandardFeeRule`, an enum whose sole variant `Zip317` delegates to `FeeRule::standard()`, and the extension trait `Zip317FeeRule`, which exposes `marginal_fee()` and `grace_actions()` to the change strategies (`zcash_client_backend`, `fees.rs`, `fees/standard.rs`, `fees/zip317.rs`). There is no deployed alternative: paying the conventional fee is the only strategy the stack offers.

3.4 Change and dust under the fee rule

The fee rule does not act alone; it is consulted by a *change strategy* that turns a tentative selection of inputs and payments into a balanced transaction. Two ZIP-317 strategies exist: `SingleOutputChangeStrategy`, producing one change output, and `MultiOutputChangeStrategy`, splitting change across several notes according to a `SplitPolicy` so the wallet maintains a target number of spendable notes (`zcash_client_backend`, `fees/zip317.rs`).

Change-pool selection. Change goes to Orchard if the transaction has any Orchard input or output; otherwise to Ironwood if it has any Ironwood input or output, so change from an Ironwood spend stays in that pool; otherwise to Sapling if it has any Sapling input or output; otherwise — fully transparent flows — to a caller-specified fallback pool. Once NU6.3 is active, the turnstile overrides an Orchard-preferred result to Ironwood unless the transaction spends Orchard notes and strictly less value returns to the pool than those notes remove from it. The source marks the strategy “naive” (TODO) (`zcash_client_backend`, `fees/common.rs`, `select_change_pool`).

The balance computation. The function `single_pool_output_balance` first computes *min_fee*, the fee required with *zero* change outputs (`zcash_client_backend`, `fees/common.rs`). Then:

1. if $total_in < subtotal_out + min_fee$, it fails with `ChangeError::InsufficientFunds`, reporting $available = total_in$ and $required = subtotal_out + min_fee$ — the figure the input-selection loop of §3.5 feeds back;

2. at exact equality, when all flows are transparent and no change memo is requested, it emits no change output and the fee is *min_fee*;
3. otherwise it computes *max_fee* with the target change-output count and derives *change* = *total_in* − (*subtotal_out* + *total_fee*).

In the shielded case a *zero-valued* change output is deliberately retained even at exact balance: omitting it would let an adversary who knows the recipients' outputs conclude, from the absence of change, that the inputs sum exactly to payments plus fee; the zero-valued output also carries the change memo (zcash_client_backend, fees/common.rs).

Dust policy. A change value at or below a dust threshold is handled by a DustOutputPolicy. The default is (DustAction::Reject, None), with the unset threshold defaulting to the marginal fee, 5000 zatoshis (default_dust_threshold = fee_rule.marginal_fee()); Reject still permits exactly-zero change per the previous paragraph. The AllowDustChange action emits the dust change output anyway. The AddDustToFee action folds the dust into the fee — with a guard: if the resulting fee exceeds the computed fee by more than $10 \cdot \text{MINIMUM_FEE} = 100,000$ zatoshis the fold is not performed and the ordinary change output is kept, protecting the user from a mis-set threshold; the documentation notes that AddDustToFee inherently leaks more information (zcash_client_backend, fees.rs, fees/zip317.rs, fees/common.rs).

Uneconomic inputs. An input is classified as dust iff its value is *at most* the marginal fee (zcash_client_backend, fees/common.rs, check_for_uneconomic_inputs). Dust is admitted only where it rides along free. Per pool p ,

$$\text{allowed}_p = \min(\#dust_p, (\#outputs_p + \#change_p) - \#nondust_p) \quad (\text{saturating at } 0),$$

i.e. dust inputs that do not increase the padded action count. Then, if the baseline logical-action count is below *grace_actions*, at most *one* extra dust input may be added free — pools tried in the order transparent, Sapling, Orchard, Ironwood — admitted iff the hypothetical action count with it does not exceed the baseline, where the hypothetical count is $\max(t_{\text{in}}, t_{\text{out}}) + \max(\text{padded Sapling spends}, \text{padded Sapling outputs}) + \text{padded Orchard actions} + \text{padded Ironwood actions}$ with transparent inputs assumed P2PKH. When the presence of a change output is uncertain, the minimum allowance over both manifests is used (possible_change construction, same file). Dust beyond the allowance is returned as ChangeError::DustInputs, which the input selector converts into an exclusion list (§3.5).

Remark 3.6 (Strictness divergence from the ZIP wording). The ZIP speaks of inputs “with value *below* the marginal fee” (ZIP-317, *Requirements and Rationale for the chosen parameters*), while the deployed check classifies value $\leq \text{marginal_fee}$ — a note of exactly 5000 zatoshis included — as dust, and the SQL layer (§3.5) likewise never offers notes with value ≤ 5000 . The ChangeError::DustInputs documentation itself acknowledges the determination is “potentially conservative”. We flag this as conservative code behaviour against the ZIP wording, not an error in either (zcash_client_backend, fees/common.rs; zcash_client_sqlite, wallet/common.rs).

Splitting change. The strategy MultiOutputChangeStrategy fetches account metadata counting the wallet's existing notes above min_split_output_value (default fallback SplitPolicy::MIN_NOTE_VALUE = 500,000 zatoshis, documented as MARGINAL_FEE · 100); the split count shrinks until each split note is at least the minimum split value (or a quarter of the post-transaction average note value when no minimum is set); if fewer splits than targeted result, the fee is recomputed for the actual count. The division remainder is added to the first change output (zcash_client_backend, fees.rs, fees/zip317.rs, fees/common.rs).

Example 3.7 (Fee of a split-change transaction). Fix a five-way split policy with minimum split value 1,000,000 zatoshis and a wallet holding no other notes. Spending a 7,500,000-zatoshi Sapling note to pay 1,000,000 incurs a fee of 30,000 zatoshis — one spend against six outputs, the payment plus five change notes, is six logical actions — and leaves five change notes of 1,294,000 zatoshis each. Spending a 6,000,000-zatoshi note under the *same* policy shrinks the split: at the five-change fee of 30,000 the change totals 4,970,000 — 994,000 per note, below the minimum — so the strategy emits four change notes; the fee, recomputed for the payment plus four change notes (five logical actions), is 25,000, and each change note carries 1,243,750. The figures are recomputed by script from the ZIP-317 formula and the split rule above, and match the strategy’s own tests (`zcash_client_backend, fees/zip317.rs, test module`). The same tests verify that the strategy refuses an exact-balance two-output transaction without change, requiring the change output’s marginal fee, so that a recipient cannot reason about the sender’s input values from the absence of change.

3.5 Input selection: the fee-escalation loop

Fee and input selection are mutually dependent: adding an input can raise the logical-action count, which raises the fee, which can require another input. The deployed selector, `GreedyInputSelector`, resolves the circularity by iteration rather than by solving for a fixed point directly (`zcash_client_backend, data_api/wallet/input_selection.rs, propose_transaction`). Starting from an empty selection, each round:

1. chooses source pools by the preference order of §4.2 — the first pool in order whose selected notes cover the requirement is spent alone, and pools otherwise accumulate in preference order until the total covers it;
2. calls `ChangeStrategy::compute_balance` on the current selection (the fee evaluated on padded counts, prospective change included);
3. on `Ok`, emits the `Proposal` with the balanced fee; on `Err(DustInputs)`, adds the offending notes to an exclusion list and retries; on `Err(InsufficientFunds{required})`, sets the new target to *required* — which already embeds the fee recomputed for the enlarged, padded action count — and reselects notes from scratch with `TargetValue::AtLeast(required)`.

The loop terminates when a reselection fails to *strictly* increase the total selected value, returning `InsufficientFunds` (invariant comment at `input_selection.rs`). Escalation is thus implicit: each added note can raise the logical-action count by one, raising the required total by one marginal fee, which the next round’s selection must cover.

The note store enforces the dust rule below the selector: both note-selection queries in `zcash_client_sqlite` filter `rn.value > min_value` with `min_value = zip317::MARGINAL_FEE = 5000`, so notes worth at most 5000 zatoshis are never even offered to selection. Value-targeted selection scans notes oldest-first under a running-sum window, taking every note while the running sum is below the target plus the single note that crosses it (`zcash_client_sqlite, wallet/common.rs`). The candidate notes themselves are the product of the synchronisation described in the *Ironwood Guide* (“Transmission and detection of a note: encryption, trial decryption, and synchronisation”); this volume takes that spendable set as given.

ZIP-320 (TEX) transactions. A payment to a transparent-source (TEX) address builds two transactions: the first funds an ephemeral transparent output, which the second spends towards the TEX address. The selector prices the ephemeral input as a standard 150-byte P2PKH input and the ephemeral output as a 34-byte P2PKH output. The value the ephemeral output must carry is obtained by balancing the *second* transaction with a zero-valued ephemeral input (`EphemeralBalance::Input(0)`) and reading the required field of the resulting `InsufficientFunds` error (`zcash_client_backend,`

`input_selection.rs`, `fees/common.rs`). In `propose_send_max`, the second transaction's one-transparent-in, one-transparent-out fee is `fee_required([Known(150)], [34], 0, 0, 0, 0) = 10,000` zatoshis — the grace minimum.

Shielding. The function `propose_shielding` computes a trial balance over all spendable UTXOs of the source addresses; on `DustInputs` it removes exactly the offending outpoints and recomputes once; the proposal is emitted only if the total reaches the caller's shielding threshold, else `InsufficientFunds` (`zcash_client_backend`, `input_selection.rs`).

3.6 Relay, mempool, and block template

The conventional fee binds economically through three node-side mechanisms, all specified in ZIP-317 and ZIP-401. None is a consensus rule. The zebra claims below are verified against its sources (`zebra-chain`, `transaction/unmined/zip317.rs` and `transaction/unmined.rs`; `zebra-rpc`, `types/get_block_template/zip317.rs`; `zebrad`, `mempool/storage/verified_set.rs`); the `zcashd` claims rest on the ZIP text alone.

Definition 3.8 (Unpaid actions). For a non-coinbase transaction,

$$\text{unpaid_actions}(tx) = \max(0, \max(\text{grace_actions}, tx.\text{logical_actions}) - \lfloor tx.\text{fee}/\text{marginal_fee} \rfloor),$$

and 0 for coinbase; a block's unpaid actions are the sum over its transactions, bounded by `block_unpaid_action_limit = 50` in template construction (ZIP-317, *Recommended algorithm for block template construction*).

Relay. Nodes are expected to relay transactions paying at least the conventional fee. A transaction with more than `block_unpaid_action_limit = 50` unpaid actions will never be mined by the recommended template algorithm, and nodes MAY drop such transactions, or those above a configurable limit (ZIP-317, *Transaction relaying*). Concretely: `zcashd` v5.6.0 exposes `-txunpaidactionlimit` (default 50) and `-blockunpaidactionlimit` overriding the block limit, and its relay threshold — a separate, older mechanism — was simplified in v5.5.0 (`zcash/zcash#6542`) (ZIP-317, *Deployment*). The zebra node is stricter than the ZIP suggests: since v1.8.0 (`ZcashFoundation/zebra#8638`) it hard-codes `BLOCK_UNPAID_ACTION_LIMIT = 0`, and `mempool_checks` — run on every candidate for the mempool, the node's pool of validated transactions awaiting mining, through `VerifiedUnminedTx::new` (`zebra-consensus`, `transaction.rs`) — rejects any transaction whose unpaid-action count exceeds that limit (`zebra-chain`, `transaction/unmined/zip317.rs`). By Definition 3.8, a transaction has zero unpaid actions iff it pays at least the conventional fee, so zebra admits to its mempool — and therefore relays — only transactions paying the full conventional fee. The same check retains a legacy fee-rate floor of 100 zatoshis per 1000 bytes, clamped to `[100, 1000]` zatoshis; the source notes it cannot fail once the zero-unpaid-action check passes, the conventional fee being at least 10,000 zatoshis (same file).

Mempool limiting (ZIP 401, as amended). Each mempool transaction has

$$\text{cost} = \max(\text{memory size in bytes}, 10,000), \quad \text{eviction_weight} = \text{cost} + \text{low_fee_penalty},$$

with `low_fee_penalty = 40,000` if the transaction pays less than the ZIP-317 conventional fee and 0 otherwise, so low-fee transactions are evicted preferentially under memory pressure (ZIP-401, *Specification*). The threshold was switched from the legacy fee to the ZIP-317 conventional fee in `zcashd` v5.5.0 and `zebrad` v1.0.0-rc.7 (ZIP-317, *Mempool size limiting*; ZIP-401, *Deployment*). In zebra, method `cost` returns `max(serialized size, 10,000)` and `eviction_weight` adds the 40,000-zatoshi penalty when the

fee falls below the conventional fee (`zebra-chain, transaction/unmined.rs`); eviction samples at random by that weight (`zebrad, mempool/storage/verified_set.rs, evict_one`). Because admission already requires the full conventional fee, the penalty never fires in zebra’s own mempool.

Block template construction. The recommended algorithm assigns each candidate transaction a weight

$$tx.weight_ratio = \min\left(\frac{\max(1, tx.fee)}{conventional_fee(tx)}, weight_ratio_cap\right), \quad weight_ratio_cap = 4,$$

the $\max(1, \cdot)$ avoiding an all-zero-weight candidate set. Phase 1 selects randomly, with probability proportional to weight ratio, among candidates paying at least the conventional fee, subject to the limits on block size and on the number of signature operations a block’s scripts may perform; phase 2 selects among the remaining candidates subject additionally to the block’s unpaid actions staying at most *block_unpaid_action_limit* (Definition 3.8). Whatever an adversary submits, a block built this way carries at most 50 unpaid logical actions (ZIP-317, *Recommended algorithm for block template construction*). In zebra, the two-phase selection (`select_mempool_transactions`; `zebra-rpc, types/get_block_template/zip317.rs`) initialises the phase-2 budget to the hard-coded limit of 0, so a zebra template carries no unpaid actions at all. Overpaying beyond 4× the conventional fee therefore buys no further priority — a deliberate flattening of the fee market that keeps the conventional fee focal.

Deployment. ZIP-317 fees became the wallet default in `zcashd v5.5.0`; the block-template algorithm shipped in `zcashd v5.5.0` and `zebra v1.0.0-rc.3` (zebra has no wallet, so it computes no fees for construction) (ZIP-317, *Deployment*).

4 Transaction construction

The synchronisation procedure of the *Ironwood Guide* (§ “Transmission and detection of a note: encryption, trial decryption, and synchronisation”) leaves a wallet holding a set of unspent notes, each with a value, a position, and an authentication path available on demand. This section documents how the deployed wallet layer turns that state, together with a payment request, into signed transactions. The pipeline, implemented in `zcash_client_backend (src/data_api/wallet.rs, module documentation and the two entry points)`, has two phases. Function `propose_transfer` takes a ZIP-321 payment request, runs input selection and fee-and-change computation, and returns a *proposal* — a complete plan, down to the zatoshi, of every transaction to be built. Function `create_proposed_transactions` executes the proposal: one transaction per step, each proved and signed, all stored together via `store_transactions_to_be_sent`; it returns the transaction ids, and submission to the network remains the caller’s responsibility. The split places every decision that requires judgement — which notes to spend, what fee to pay, where change goes — into an inspectable value computed before any key is touched; execution is then deterministic given the plan.

Current-head extension. The detailed source trace below was established at the volume baseline `e30517e4`; current `librustzcash 91f448b5` preserves that pipeline and extends every pool-indexed step to Ironwood. A proposal now carries separate Ironwood inputs, anchor, witnesses, and padding. After NU6.3, an ordinary payment to an Orchard-protocol receiver targets Ironwood; Orchard change from an Orchard spend stays in Orchard through `add_orchard_change_output`, paired with the required fabricated same-receiver spend. Conventional Ironwood bundles retain default padding, while a canonical ZIP-318 crossing uses an unpadding one-Action Ironwood bundle, the shared rolling expiry, and the separately selected anchor bucket. The builder then constructs, proves, signs, and

records the Ironwood bundle beside the Orchard bundle using the same Orchard spend authority. Pool lists below that name only Sapling and Orchard describe the pinned pre-Ironwood snapshot; the invariants and role ordering apply unchanged to the added pool.

4.1 The proposal

The payment request. The input is a ZIP-321 `TransactionRequest`: a `BTreeMap` from payment index to payment, implemented in `librustzcash`’s `components/zip321` crate (`src/lib.rs`). A memo attached to a payment addressed to a transparent recipient is rejected — as `Zip321Error::TransparentMemo` at parse time and again as `Error::Payment(PaymentError::TransparentMemo)` when outputs are added to the builder (`zcash_client_backend/src/data_api/wallet.rs`) — because a transparent output has no ciphertext to carry one.

Definition 4.1 (Proposal). A *proposal* carries the fee rule under which it was computed, the minimum target height (the next block height), and a non-empty sequence of *steps* (type `Proposal<FeeRuleT, NoteRef>`; `zcash_client_backend/src/proposal.rs`), each step describing one transaction: the originating transaction request; a map from payment index to the pool that will pay it; the selected transparent inputs; the selected shielded inputs together with the anchor height at which their witnesses will be computed; references to outputs of prior steps to be spent; the *balance* — the proposed change values and the fee required — and a flag marking shielding steps. Validation of a multi-step proposal (`proposal.rs`, `Proposal::multi_step`) rejects forward references, spends of non-ephemeral change produced by a prior step, and intra-proposal double spends.

Target and anchor heights. In `zcash_client_sqlite` (`src/wallet.rs`, `get_target_and_anchor_heights`), the heights are

$$\text{target} := \text{tip} + 1, \quad \text{anchor} := \min_{\text{pools}} \max\{h \leq \text{target} - c : h \text{ checkpointed}\},$$

the “mempool height” and, with c the confirmation depth and “checkpointed” meaning present in the pool’s shard tree, the highest checkpoint at least c blocks below the target, minimised across Sapling and Orchard so a single height anchors both bundles. Anchor semantics and witness derivation are those of the *Ironwood Guide* (§ “Proof of ownership of *some* valid note: the commitment tree”, in particular “Anchor choice as the tree grows”, which already develops the ZIP-315 3/10 defaults); here we record only what the pipeline feeds in.

Definition 4.2 (Confirmations policy). The `ConfirmationsPolicy` of ZIP 315 carries a *trusted* depth (default 3), an *untrusted* depth (default 10), and an `allow_zero_conf_shielding` flag (default `true`) (`zcash_client_backend/src/data_api/wallet.rs`, `ConfirmationsPolicy`; ZIP 315 § “Trusted and untrusted TXOs”). The constructor enforces `trusted ≤ untrusted`. An output is spendable at the trusted depth if its transaction is user-marked trusted or it was received on an internal-scope key, and at the untrusted depth otherwise; outputs of a shielding transaction inherit the mined height and trust of their transparent source inputs; and transparent UTXOs are spendable at zero confirmations for shielding when the flag is set (`wallet.rs`, `ConfirmationsPolicy`).

Remark 4.3 (Anchor depth versus untrusted notes). Function `propose_transfer` obtains its height pair with `minconf = trusted` (`wallet.rs`, `propose_transfer`) — even when the notes ultimately selected are untrusted and must themselves be 10 confirmations deep. The code comment explains the choice: the shallower anchor admits the maximum number of notes, and trusted-versus-untrusted filtering is deferred to note selection, which tests each note’s own depth. The bundle-wide anchor

therefore sits at depth ≥ 3 , not ≥ 10 — which is what ZIP 315 itself recommends. A wallet SHOULD choose the anchor at the trusted confirmation depth: “if the current block is at height H , the anchor SHOULD reflect the final treestate of the block at height $H - 3$ ” (ZIP 315, § “Anchor selection”). The ZIP’s stated rationale: a rollback past the anchor block invalidates the transaction while its revealed nullifiers link it to any future transaction spending the same notes; an anchor too many blocks back prevents recently received notes from being spent; and a *fixed* depth — rather than one conditioned on whether the spent notes are trusted — avoids leaking their trust status (ZIP 315, § “Rationale for anchor selection”). Neither the ZIP nor the code comment claims an anonymity-set benefit for the shallower anchor.

Remark 4.4 (The ZIP-315 floor is advisory in the API). ZIP 315 states that wallets SHOULD NOT permit spending outputs with fewer than 3 confirmations, but the deployed API exposes `ConfirmationsPolicy::MIN` (1 confirmation) and constructors accepting lower values, gated only by a rustdoc warning about transaction distinguishability (`wallet.rs`, `ConfirmationsPolicy`). We present the ZIP recommendation as the norm and the constructors as a deliberate escape hatch.

4.2 Input selection

The deployed selector is `zcash_client_backend`’s `GreedyInputSelector` (`src/data_api/wallet/input_selection.rs`); it buckets each payment by receiver type: a transparent address becomes a transparent output; a TEX address is deferred to a second ZIP-320 step (below); a Sapling address becomes a Sapling output; a unified address is paid to its Orchard receiver if present — as an Ironwood-pool output once NU6.3 is active at the target height, the turnstile forcing such payments through the Ironwood bundle, and as an Orchard-pool output before then — else its Sapling receiver, else its transparent receiver, and a unified address with no receiver the wallet supports is an error.

The greedy loop. Selection then iterates (`input_selection.rs`): starting with no inputs, call the change strategy’s `compute_balance` (§4.3); on `InsufficientFunds{required}`, ask the backend for spendable notes covering at least `required` (`select_spendable_notes` with `TargetValue::AtLeast`, the target height, the confirmations policy, and the running exclusion list); on `DustInputs`, add the identified notes to the exclusion list and retry; on success, emit the proposal. The loop terminates because the selected value must strictly increase on every round, else the selector fails with `InsufficientFunds`. Pool preference inside the loop (`selectable_pool_preference` and the `use_pools` logic, `input_selection.rs`): the shielded pools are ordered with the family matching the payment’s outputs first — for a payment to an Orchard receiver, Ironwood (once NU6.3 is active) and then Orchard, ahead of Sapling; otherwise Sapling, then Ironwood, then Orchard — and pools the caller’s spend policy forbids are dropped. The first pool in order whose selected notes cover the requirement alone is spent alone; otherwise pools accumulate in preference order until the running total covers it. Spending stays in the pool family matching the outputs where possible, crossing pools only when needed, which keeps single-pool payments free of cross-pool linkage. Within a pool, the `zcash_client_sqlite` backend selects the *oldest* unspent notes first: a window-function running sum over notes ordered oldest-to-newest takes every note below the target plus the one note that crosses it (`zcash_client_sqlite/src/wallet/common.rs`, `select_spendable_notes`).

ZIP-320 (TEX) payments. A TEX address demands transparent funds whose provenance is a single ephemeral transparent address, so the selector builds *two* steps (`input_selection.rs`: 580--629, 974--1051; ZIP 320 § “Design considerations for Senders”). It sizes the second step first: probing `compute_balance` over the TEX outputs with a zero-valued ephemeral input and catching `InsufficientFunds{required}` yields

$$v_{\text{eph}} = \sum_{\text{TEX payments}} v + \text{fee}(\text{tr}_1),$$

where $\text{fee}(\text{tr}_1)$ is computed for exactly one standard P2PKH input (150 bytes) and the TEX payments as P2PKH outputs (34 bytes each) (`fees/common.rs`, `single_pool_output_balance`). The change-strategy contract requires the second step to balance exactly — $\text{total} = \text{fee_required}$, no change — which the selector enforces with a runtime assertion (`input_selection.rs`). Step tr_0 then replaces the TEX payments with a single ephemeral transparent “change” output of exactly v_{eph} , and step tr_1 spends it and pays the TEX addresses as P2PKH. Paying a TEX address from shielded inputs in a single step is rejected (`PaysTexFromShielded`, `wallet.rs`).

Shielding. Function `propose_shielding` (`wallet.rs` and `input_selection.rs`) targets the tip $+1$ and collects spendable UTXOs from the given source addresses — at zero confirmations under the default policy (Definition 4.2). Funds may be drawn from at most one ephemeral address at a time and never mixed with other addresses (`ProposalError::EphemeralAddressLinkability`), since co-spending would link the ephemeral address to the wallet’s others. Dust UTXOs reported by `compute_balance` are dropped and the balance recomputed once; the proposal is built only if its total reaches the caller’s shielding threshold. The resulting single step has an empty transaction request, `is_shielding` set, and all value flowing to internal change.

4.3 Fees and change

Fees and change are computed *together*, by a `ChangeStrategy` whose one obligation is `compute_balance` (`zcash_client_backend/src/fees.rs`, `ChangeStrategy`). The deployed strategies are `SingleOutputChangeStrategy` and `MultiOutputChangeStrategy` in the `fees::zip317` module; the `fees::standard` module aliases them over `StandardFeeRule`, whose only variant is `Zip317` (`fees/zip317.rs`; `fees/standard.rs`). Both delegate to a common routine, `single_pool_output_balance` (`fees/common.rs`), described below.

Definition 4.5 (ZIP-317 conventional fee, as deployed). The fee rule (`zcash_primitives`, `src/transaction/fees/zip317.rs`, `fee_required`; ZIP 317 § “Fee calculation”) is

$$\begin{aligned} \text{fee} &= \text{marginal_fee} \cdot \max(\text{grace_actions}, \text{logical_actions}), \\ \text{logical_actions} &= \max\left(\left\lceil \frac{\text{in_size}}{150} \right\rceil, \left\lceil \frac{\text{out_size}}{34} \right\rceil\right) + \max(n_{\text{spend}}^{\text{Sap}}, n_{\text{out}}^{\text{Sap}}) + n_{\text{act}}^{\text{Orch}} + n_{\text{act}}^{\text{Irw}}, \end{aligned}$$

with `marginal_fee` = 5 000 zatoshi, `grace_actions` = 2, the standard P2PKH input and output sizes 150 and 34 bytes, and hence a minimum fee of 10 000 zatoshi (`fees/zip317.rs`; the ZIP 317 parameter table). The Ironwood count $n_{\text{act}}^{\text{Irw}}$ is ZIP 317 Revision 1’s contribution (§3.3), zero in any transaction without an Ironwood bundle. The shielded counts fed in are the *padded* counts of §4.5 — `BundleType::num_outputs` and `num_actions` — both here and in the final builder check (`fees/common.rs`; `zcash_primitives/src/transaction/builder.rs`, `get_fee`).

Two deployment caveats. ZIP 317 Revision 0 additionally defines a Sprout contribution $2 \cdot n_{\text{JoinSplit}}$; the wallet never constructs `JoinSplits`, so the deployed `FeeRule` omits the term (`fees/zip317.rs`) — code and ZIP agree on every transaction the wallet can produce, and we flag the omission rather than restate the ZIP formula as implemented. Second, the rule supports P2PKH transparent inputs plus P2SH inputs of known size only; a coin with an unknown P2SH redeem script fails with `FeeError::UnknownP2shInputs`, and the rule intentionally may slightly *overestimate* the fee for many transparent inputs — an overestimate the construction algorithms in `zcash_client_backend` rely on (`fees/zip317.rs`, `FeeRule` rustdoc and `fee_required`).

Change pool. Change goes to Orchard if the transaction has any Orchard input or output flows; else to Ironwood if it has any Ironwood flows; else to Sapling if it has any Sapling flows; else — fully

transparent flows, as in shielding — to the caller-supplied fallback pool, subject after NU6.3 activation to the turnstile override of §3.4. The code marks this as deliberately naive (`fees/common.rs`, `select_change_pool`).

Balance computation. Routine `single_pool_output_balance` (`fees/common.rs`) proceeds as follows. Let in be the sum of transparent, Sapling, and Orchard inputs (including any ephemeral input) and out the corresponding sum of outputs; let fee_0 be the ZIP-317 fee with zero change outputs. If $in < out + fee_0$, fail with `required = out + fee_0` (this is the value the greedy loop feeds back into note selection). If equality holds *and* the flows are fully transparent *and* no change memo is configured, emit no change with $fee = fee_0$. Otherwise recompute the fee with the target number of change outputs (re-derived if the split policy below yields fewer), set $change = in - (out + fee)$, and divide it across the split count with the remainder added to the first output, each share a shielded change value in the selected pool carrying the configured change memo.

Remark 4.6 (Zero-valued change is mandatory). Every transaction with shielded flows produces a change output, even when inputs exactly balance outputs plus fee: a zero-valued shielded change output is created so that an observer cannot distinguish exact-balance transactions, and so that a shielded output always exists to carry the change memo (`fees/common.rs`, `single_pool_output_balance`). Zero-valued change is allowed even under the `Reject` dust action, and the code notes that zero-valued notes need no witness tracking. Only a fully-transparent-flow transaction without a change memo may omit change — the second ZIP-320 transaction is the deployed example. A configured change memo is attached to every proposed shielded change output, is deliberately discarded for the ZIP-320 step that spends the ephemeral input, and itself forces a change output to exist (`fees/common.rs`, `single_pool_output_balance`).

Dust. The default `DustOutputPolicy` is `DustAction::Reject` with no threshold override, delegating the threshold to the strategy, whose default is the marginal fee, 5 000 zatoshi (`fees.rs`, `DustOutputPolicy`; `fees/zip317.rs`; `fees/common.rs`). If the total change falls below the threshold: under `Reject`, fail with `InsufficientFunds` and the requirement raised by the shortfall (unless the change is exactly zero, which Remark 4.6 permits); under `AllowDustChange`, keep the dust output; under `AddDustToFee`, burn the change into the fee and omit the output (keeping a zero-valued one if a change memo exists) — *except* when the resulting fee would exceed the computed fee by more than $10 \cdot \text{MINIMUM_FEE} = 100\,000$ zatoshi (that is, when the folded dust exceeds 100 000), in which case the change output is kept as a defence against value loss (`fees/common.rs`).

Uneconomic inputs. An input is *dust* iff its value is at most the marginal fee; the check runs only when the marginal fee is positive (`fees/common.rs`, `check_for_uneconomic_inputs`). Some dust rides for free: per pool P , up to

$$\min(|\text{dust}_P|, (\text{outputs}_P + \text{change}_P) - \text{nondust}_P) \quad (\text{saturating})$$

dust inputs do not raise the padded action count, plus at most one extra *grace* input — tried transparent first, then Sapling, then Orchard, then Ironwood — if the baseline logical-action count is below `grace_actions` and adding the candidate does not increase the padded count (the padding rules of §4.5 are invoked directly, with and without it). The allowance is the pointwise minimum over all possible change configurations; dust beyond it is returned as `ChangeError::DustInputs`, which the greedy loop converts into exclusions.

Note management: splitting change. The `MultiOutputChangeStrategy` splits change into up to `target_output_count` equal notes (remainder to the first) whenever the account holds fewer than that many notes of at least `min_split_output_value`; the note-count metadata comes from `InputSource::get_account_metadata` filtered by that minimum, or by the

fallback `MIN_NOTE_VALUE = 100 · marginal_fee = 500 000` zatoshi (`fees.rs`, `SplitPolicy`; `fees/common.rs`; `fees/zip317.rs`, `MultiOutputChangeStrategy`). The split count is

$$\text{split} = \max(1, \text{target} - \#\{\text{qualifying notes}\}),$$

decremented (floor 1) while `change/split < min_split_output_value`; when the minimum is unset it defaults to $(\text{existing notes total} + \text{change}) / (4 \cdot \text{target})$, a quarter of the projected average note value (`fees.rs`, `SplitPolicy::split_count`). If fewer than the target number of outputs survive, the fee is recomputed for the actual count. The `SingleOutputChangeStrategy` is the degenerate `SplitPolicy::single_output()`.

4.4 Executing the proposal

Function `create_proposed_transactions` executes the steps in order (`zcash_client_backend`, `src/data_api/wallet.rs`, `create_proposed_transactions`). Only *transparent* outputs of prior steps may be spent by later steps: shielded chaining is impossible, because a note’s position in the global commitment tree — and hence its witness (the *Ironwood Guide*, § “Proof of ownership of *some* valid note: the commitment tree”) — is unknown before the note is mined. Each ephemeral output must be consumed by a later step exactly once, else `EphemeralOutputLeftUnspent`. All transactions are created first and then stored together, so a failure cannot leave a partially transmitted multi-step plan.

Anchors and witnesses per step. For each pool a step involves, the anchor is the shard-tree root at the proposal’s anchor-height checkpoint (`root_at_checkpoint_id`), or the empty-tree anchor if the pool has outputs but no inputs; each selected note’s Merkle path is computed with `witness_at_checkpoint_id_caching` at that same height (`wallet.rs`, `build_proposed_transaction`). The mechanics are those of the *Ironwood Guide* (§ “Proof of ownership of *some* valid note: the commitment tree”) and are not repeated here.

Builder configuration. The `zcash_primitives` builder is constructed with `BuildConfig::Standard` carrying the per-pool anchors and Orchard/Ironwood padding choices, which normally select `BundleType::DEFAULT`; the Orchard builder exists only if NU5 is active at the target height, and the Ironwood builder only for a format and branch supporting Ironwood (`zcash_primitives/src/transaction/builder.rs`, `BuildConfig` and `Builder::new`). The expiry height is the target plus `DEFAULT_TX_EXPIRY_DELTA = 40` blocks, the post-Blossom default of ZIP 203 (`builder.rs`, `DEFAULT_TX_EXPIRY_DELTA`; ZIP 203 § “Changes for Blossom”). The rustdoc on `Builder::new` still says “20 blocks”; code and ZIP agree, the rustdoc is stale — cite the constant, not the comment. The transaction version is `TxVersion::suggested_for_branch` of the target height’s branch — V5 for the NU5 through NU6.2 branches, V6 for the NU6.3 (Ironwood) branch (`zcash_primitives/src/transaction/mod.rs`, `suggested_for_branch`) — and the lock time of built transactions is 0 (`builder.rs`, `build_internal`).

Spends, outputs, and keys. Sapling spends are added under the external or internal (change) full viewing key according to each note’s recorded ZIP-32 scope; Orchard spends are added under the account’s Orchard full viewing key, the scope being derived internally (`FullViewingKey::scope_for_address` inside `SpendInfo::new`) (`wallet.rs`; orchard 0.15.0 `src/builder.rs`). Payment outputs are added in payment-index order, then the change outputs from the proposal balance: Sapling change to `ufvk.sapling().change_address()` (internal scope), Orchard change to `ufvk.orchard().address_at(0, Scope::Internal)`, and ephemeral ZIP-320 change values to freshly reserved ephemeral transparent addresses (`reserve_next_n_ephemeral_addresses`; the reservation persists even if construction later fails, so an address is never silently reused) (`wallet.rs`). The signing keys marshalled for the build

(wallet.rs): transparent secret keys derived per input address; two Sapling expanded spending keys — `usk.sapling()` and `usk.sapling().derive_internal()` — since external and internal notes sign under different keys; but a *single* Orchard spend-authorising key (`SpendAuthorizingKey`), because Orchard internal and external addresses share `ask` (the *Ironwood Guide*, § “Keys: capabilities for spending and for viewing”). Ironwood spends use that same Orchard full-viewing and spend-authorizing key, but enter the separate Ironwood builder and commitment tree; current output and change routing is summarized in the current-head extension above. All randomness for the build is drawn from `OsRng`.

Remark 4.7 (OVK policy: none for change). Under the default `OvkPolicy::Sender`, external outputs are encrypted with the `ovk` selected by `UnifiedFullViewingKey::select_ovk` over the input sources — the Orchard `ovk` if any Orchard input is spent, else the Sapling `ovk`, else the external-scope `ovk` derived from the transparent account key per ZIP-316 — while wallet-internal change outputs get *no* `ovk` at all (`internal_ovk = None`) (wallet.rs; `zcash_client_backend/src/wallet.rs`, `OvkPolicy`; `zcash_keys/src/keys.rs`, `select_ovk`). A change output’s C_{out} is therefore unrecoverable by *any* outgoing viewing key; the wallet finds its change through the internal-scope `ivk` during scanning (trial decryption per the *Ironwood Guide*, § “Transmission and detection of a note...”). This matches the `OvkPolicy::Sender` rustdoc but contradicts descriptions elsewhere of change “encrypted to the internal OVK”; the `None` is deliberate. Policy `OvkPolicy::Custom` supplies both keys explicitly; policy `OvkPolicy::Discard` uses none, so the sender cannot decrypt even its own external outputs.

4.5 Padding, dummies, and shuffling

The deployed bundle builders are orchard 0.15.0 and sapling-crypto 0.7.0, per `librustzcash’s` `Cargo.lock` (workspace `Cargo.toml`:66,69); the line references below are to the orchard 0.12.0 / sapling-crypto 0.6.0 sources the section was written against, whose padding logic 0.15.0 preserves under the deployed cross-address-enabled flags: the requested-action count stays $\max(s, o)$ (floored at 2 below), rising to $s + o$ only once NU6.3 forces cross-address transfers off (§5.13).

Orchard. The wallet’s bundles use `BundleType::DEFAULT`: flags `ENABLED` (spends and outputs both enabled) and `bundle_required = false` (orchard 0.12.0 `src/builder.rs`:58--61, `src/bundle.rs`:85--88). With s requested spends and o requested outputs, the action count is

$$n_{\text{act}}(s, o) = \begin{cases} 0 & \text{if } \max(s, o) = 0, \\ \max(\max(s, o), 2) & \text{otherwise,} \end{cases}$$

the floor being `MIN_ACTIONS = 2` (`src/builder.rs`:36, 75--107). The spend list is extended to n_{act} with dummy spends and the output list with dummy outputs; the two indexed lists are then shuffled *independently* and zipped pairwise into `Actions`, so the spend–output pairing within an action is random, and `BundleMetadata` records each requested spend’s and output’s post-shuffle action index (`src/builder.rs`, `build_bundle`).

Orchard dummies. A dummy spend (protocol specification § 4.8.3, “Dummy Notes (Orchard)”; orchard 0.12.0 `src/note.rs`:208--224, `src/tree.rs`:116--124, `src/note/nullifier.rs`:33--35, `src/builder.rs`:262--286) is built from a freshly random `SpendingKey`, its full viewing key’s address at index 0 (external scope), and a zero-valued note whose ρ is `Extract` of a random Pallas point (`Rho::from_nf_old(Nullifier::dummy)`), with a dummy Merkle path of a random `u32` position and 32 random $\mathbb{F}_{p_{\text{Pallas}}}$ siblings. Anchor consistency is skipped for zero-valued notes (`has_matching_anchor` returns true), which is exactly what lets a dummy coexist with any real anchor. A dummy output is a zero-valued note to a freshly random address, with no `ovk` and an all-zero memo.

Per-action randomness. At `ActionInfo` construction, the value-commitment trapdoor `rcv` is sampled uniformly from the Pallas scalar field — one per action, padding actions included (`src/builder.rs:426--432`; `src/value.rs:300--302`). When the action is built, the spend-auth randomiser α is sampled per spend, giving $rk = ak + [\alpha] G^{\text{SpendAuth}}$ (`src/builder.rs:293--310`); the output note's ρ is the action's own nf_{old} , and its `rseed` is freshly sampled by `Note::new`. The cryptographic roles of `rcv`, α , and `rseed` (with its derived ψ , `rcm`, `esk`) are those of the *Ironwood Guide* (§ “Conservation of value...” for `cv/rcv`; § “Authorisation of the spend: RedPallas” for `ak/ $\alpha$ /rk`; § “Transmission and detection of a note...” for the `rseed` derivations).

Sapling. Under `BundleType::DEFAULT` in `sapling-crypto 0.6.0` (`src/builder.rs:39--44`, `72--115`), a non-empty bundle pads outputs to $\max(o, 2)$ but never pads spends ($num_spends = s$ when $s > 0$, else 0; the $\max(s, 1)$ floor applies only with `bundle_required` set), so an output-only bundle has zero spend descriptions; the two-output floor (`MIN_SHIELDED_OUTPUTS`) is a rule the source traces to `zcash/zcash` issue 3615. An empty, non-required bundle has zero of both. Spends and outputs are extended with dummies, independently shuffled, and the positions recorded in `SaplingMetadata`. Each spend samples its own `rcv` and α (uniform in the Jubjub scalar field) and each output its own `rcv`; the binding-signature key is $bsk = \sum r_{cv_spend} - \sum r_{cv_output}$ (`src/builder.rs:236`, `262--268`, `420`, `752--760`).

4.6 Build order and signatures

Construction 4.8 (`Builder::build`). The build proceeds in a fixed order (`zcash_primitives`, `src/transaction/builder.rs`, `build` and `build_internal`):

1. check version compatibility;
2. compute the fee over the actual padded counts (Definition 4.5) and require the balance invariant below;
3. build the Sapling bundle and create its zk-proofs (proofs before signatures, because V4 transactions commit to proofs in the digest);
4. build the Orchard bundle and, on current v6 paths, the Ironwood bundle, both unproven;
5. assemble the unauthorised transaction data and compute the digest parts (`TxIdDigester`);
6. authorise the transparent inputs;
7. compute the shared shielded sighash;
8. apply the Sapling signatures;
9. create the Orchard proof and apply its signatures, then do the same for Ironwood when present;
10. freeze the result into the final `Transaction`.

Definition 4.9 (Balance invariant). Before building, the builder requires, exactly,

$$\text{value_balance} - \text{fee} = 0,$$

where the value balance sums the transparent, Sapling, Orchard, and — on current v6 paths — Ironwood balances; a negative difference is `Error::InsufficientFunds` and a positive one `Error::ChangeRequired` (`builder.rs`, `value_balance` and `build_internal`). The proposal's change outputs are precisely what make the invariant hold.

One sighash for everything. The message for *all* shielded spend-auth signatures and every present shielded pool’s binding signature is the single 32-byte ZIP-244 sighash,

$$\text{sighash} := \text{signature_hash}(\text{tx}_{\text{unauth}}, \text{SignableInput}::\text{Shielded}, \text{txid_parts}),$$

(builder.rs, build_internal; zcash_primitives/src/transaction/sighash.rs); the digest tree itself is constructed in the *Ironwood Guide* (§ “Conservation of value...”, “The transaction digest (ZIP-244)”) and not repeated. For each Orchard-protocol action (in either the Orchard or Ironwood pool), with $\text{rsk} = \text{ask} + \alpha$ and $\text{rk} = \text{ak} + [\alpha] G^{\text{SpendAuth}}$ in the notation of the *Ironwood Guide* (§ “Authorisation of the spend: RedPallas”),

$$\text{spendAuthSig} := \text{Sign}_{\text{rsk}}(\text{sighash}).$$

The binding key is $\text{bsk} = (\sum_{\text{actions}} \text{rcv})$ converted via `into_bsk`, and the builder asserts consistency with $\text{bvk} = \sum \text{cv}^{\text{net}} - \text{ValueCommit}(\text{rcv} = 0, \text{value_balance})$ (orchard 0.12.0 src/builder.rs, bundle finisher). In `Bundle::prepare`, the binding signature $\text{Sign}_{\text{bsk}}(\text{sighash})$ is created and every dummy spend is signed immediately with its stored random ask randomised by that action’s α ; `Bundle::sign` then signs every action whose ak matches the given authorising key, and `finalize` fails with `MissingSignatures` if any action remains unsigned (src/builder.rs:984--1026).

Remark 4.10 (Proving-key cache). At the pinned source-line baseline, the Orchard proving key was reconstructed inside every `Builder::build` call by `orchard::circuit::ProvingKey::build()` (orchard 0.12.0, src/circuit.rs:787--793). Current `librustzcash 91f448b5` derives the Orchard/Ironwood circuit version once and reuses a process-wide cached proving key under `std`; the no-std fallback builds it once for that transaction. The Sapling provers remain arguments, typically a `LocalTxProver`.

4.7 After the build

The wallet immediately decrypts its own shielded outputs to record sent-note data: Orchard outputs via `decrypt_output_with_key` under the internal-scope `ivk`, Sapling via `try_sapling_note_decryption` likewise, using `BundleMetadata` and `SaplingMetadata` to map each requested output to its post-shuffle position (wallet.rs, `create_proposed_transaction`). Transparent outputs are *not* reordered — the source notes “there is no reason to do so because it would not usefully improve privacy” — so `vout` index n is the n -th added transparent output. The fee recorded for each sent transaction is taken from the proposal’s balance, not recomputed from the built transaction (wallet.rs, `StepResult`).

The PCZT path. A partially-constructed-transaction path parallels the signing path: `build_for_pczt` produces the `PcztParts`, and the `Creator`, `IO-Finalizer`, and `Updater` roles attach ZIP-32 derivation paths (Orchard and Sapling $m/32'/\text{coin}'/\text{account}'$; transparent BIP-44 $m/44'/\text{coin}'/\text{account}'/\text{scope}/\text{index}$) and the proposal metadata under the proprietary fields `zcash_client_backend:proposal_info` and `:output_info`; only single-step proposals are supported (wallet.rs, `create_pczt_from_proposal`; zcash_primitives/src/transaction/builder.rs, `PcztParts` and `build_for_pczt`).

Orchard-to-Ironwood pool migration. The current `librustzcash` head (91f448b5) also contains the released `zcash_pool_migration 0.1.0` engine and `zcash_client_sqlite 0.22.0` store. The engine plans denominations, builds and signs the migration transactions as PCZTs, schedules them by block height, and persists its state through a wallet backend; the consuming application remains responsible for broadcast and for reporting the result. The complete privacy and scheduling rules, and the boundary between these released backend pieces and an end-user wallet flow, are in the *Ironwood Guide*, § “Scheduled Orchard-to-Ironwood migration (ZIP-318).”

Remark 4.11 (Source-line baseline versus current builders). The line-by-line derivation above was written against orchard 0.12.0, but the volume baseline actually pins orchard 0.15.0. That release includes the NU6.3 bundle versions, cross-address restriction, explicit change-output handling, and version-specific proving keys described in §5.13; those features are deployed, not design-stage. As a currentness check, the standalone orchard head 38bd2274 is release 0.15.5, while the current `librustzcash` lockfile at 91f448b5 resolves 0.15.3. Neither changes the padding construction derived above.

5 Partially created transactions (PCZT)

The *Ironwood Guide* constructs a transaction as if one machine held everything at once: the notes and their authentication paths, the full viewing key, the spend authorizing key, the proving key, and a source of randomness. Deployed wallets do not enjoy that luxury. A hardware signer holds the spending key but cannot compute a Halo 2 proof and may not fit a whole transaction in memory; a threshold of co-signers must each contribute a share; a proof may be delegated to a machine that must never see spending authority. ZIP-374 resolves this by standardising the intermediate object those parties exchange: the *Partially Created Zcash Transaction* (PCZT), a format that carries the information each participant in transaction creation needs to perform its step, and that holds the accumulated proofs and signatures while the set is still incomplete — signers can be fully offline (ZIP-374, Abstract). The ZIP is Status Draft, Revision 0, owned by Jack Grigg, created 2024-12-09 (ZIP-374, header); The ZIP fixes the v1 reference encoding to `pczt 0.7.0`. The current `librustzcash` head 91f448b5 ships `pczt 0.9.3`: release 0.8.0 added the v2 encoding and Ironwood support, and 0.9.0 made the convenience serializer emit the minimum encoding version that can represent the logical PCZT (`pczt`, `CHANGELOG.md`; §5.13). Thus the role semantics below describe the current released surface; references to 0.7.0 identify only the ZIP’s byte-exact v1 baseline.

The Motivation section of the ZIP names four problems the format solves: (i) offline and hardware signers that cannot compute Halo 2 or Groth16 proofs and may not hold a whole transaction in memory; (ii) proof delegation, where the prover needs the note’s private data and the full viewing key but no spending key material; (iii) multiparty transactions — FROST threshold signing (RFC 9591), transparent multisig (ZIP-48), and inputs contributed by several parties, merged deterministically; and (iv) pre-authorization and deferred proving for v6 transactions, where anchors become authorizing data (ZIP-229) (ZIP-374, Motivation).

The design deliberately reuses the role structure of Bitcoin’s PSBT (BIP 174 / BIP 370) with Zcash-specific additions. PSBT itself cannot be used: it has no representation for shielded inputs and outputs, no notion of proofs as distinct from signatures, and its key-value encoding tolerates unknown fields — undesirable for signers that must understand everything they authorize (ZIP-374, Motivation; `pczt`, `README.md`). The ZIP’s Requirements section fixes the resulting contract: the format must not require spending key material to be present (dummy-spend keys are the one exception, and must be removable before Signers see the PCZT); a signer must be able to determine from the PCZT alone exactly what it authorizes; merging must be deterministic and fail loudly on conflict; and the encoding must be compact enough for memory-constrained signing devices and strictly versioned (ZIP-374, Requirements).

Two scope restrictions follow from history. PCZTs do not support Sprout and cannot create v4-or-earlier transactions: the v4 format predates the ZIP-244 transaction digest, and the role separations — in particular proving independent of signing — do not hold for it (ZIP-374, Specification). Two PCZT versions are specified: v1 supports creation of v5 transactions (ZIP-225); v2 supports v5 and v6 (ZIP-229). A PCZT version does not pin a transaction version — a v2 PCZT may carry a v5 or a v6 transaction, a v1 PCZT only v5 (ZIP-374, Versioning). The released `pczt 0.9.3` parses both: `Pczt::parse` accepts version numbers 1 and 2, rejecting any other with `ParseError::UnknownVersion`. Its `serialize` emits v1 for a representable pre-v6 PCZT with a canonical-empty Ironwood bundle and no v2-only compact field state, and v2 otherwise; callers that require a fixed wire version use `pczt::v1::Pczt` or `pczt::v2::Pczt` (`pczt`, `src/lib.rs` and

CHANGELOG.md, 0.9.0).

5.1 Roles and lifecycle

Definition 5.1 (ZIP-374 roles). Transaction creation is divided among the following roles, each entitled to perform a specific step on a PCZT (ZIP-374, Roles; `pczt, src/roles.rs`):

Creator (a single entity). Initializes the global fields and the empty per-protocol bundles.

Constructor. Adds transparent inputs and outputs, shielded spends and outputs, and Orchard actions.

IO Finalizer (anyone). Declares the input/output set complete, computes the binding-signature keys, and signs dummy spends (§5.5).

Updater (anyone). Attaches metadata later roles need: derivation paths, full viewing keys, witnesses, anchors.

Prover. Attaches zero-knowledge proofs (§5.8).

Signer. Attaches signatures (§5.7).

Combiner. Merges parallel copies of a PCZT (§5.9).

Redactor. Removes fields that later entities do not need.

Spend Finalizer. Assembles the transparent `script_sigs` (§5.10).

Transaction Extractor. Produces and verifies the final transaction (§5.11).

A *Verifier* is “not a step in the transaction lifecycle, but a way of inspecting a PCZT” and checking its internal consistency on behalf of another entity that lacks the capacity to do so itself — for example a hardware signer (ZIP-374, Roles). The checks such a verifier runs are the per-action verification equations of Construction 5.8, exposed on the protocol crates’ types rather than in the `pczt` crate’s `verifier` module.

The lifecycle is not a fixed pipeline. Proofs and signatures slot in asynchronously: proof creation needs no spending key material, and for v5 transactions signatures commit to the anchor (through the sighash) but not to proofs, while proofs do not depend on signatures — so the Prover and Signer roles can run in either order or in parallel, their outputs merged by a Combiner (ZIP-374, Roles and Prover; `zcash_client_backend, src/data_api/wallet.rs, create_pczt_from_proposal` doc). From the v6 format onward the two roles become independent even of the anchor choice (§5.13).

5.2 Structure

Definition 5.2 (Field kinds). Every structure in a PCZT contains three kinds of fields (ZIP-374, Specification: Structure):

- *transaction effecting data*: required, part of the final transaction, always present;
- *transaction authorizing data*: initially empty, filled in as the PCZT is processed (proofs, signatures);
- *context data*: committed to by, or relevant to, the effecting data; present so that roles can be performed (openings, keys, paths).

The current `Pczt` struct is `{global, transparent, sapling, orchard, ironwood}` with all four bundles non-optional (`pczt` 0.9.3, `src/lib.rs`); the historical 0.7.0 v1 implementation carried the first three, and release 0.8.0 added the `ironwood` slot (§5.13). The bundles are not logically optional because a PCZT does not always contain a semantically valid transaction, and there may be phases where protocol-specific metadata must be stored before it is known whether that protocol has any inputs or outputs (ZIP-374, Structure; `pczt`, `src/lib.rs`, comment on `Pczt`).

Global fields. The `global` structure carries (ZIP-374, Global; `pczt`, `src/common.rs`): `tx_version` (u32; MUST be 5 in PCZT v1) and `version_group_id` (u32; must be consistent with `tx_version`); `consensus_branch_id` (u32; non-optional because it commits to the consensus rules that will apply to the transaction); `fallback_lock_time` (Option<u32>; assumed 0 if omitted); `expiry_height` (u32; ZIP-203); `coin_type` (u32, SLIP 44 — not part of the transaction and without consensus effect, present so later roles can check network-specific data such as derivation paths); `tx_modifiable` (a u8 bitfield, next paragraph); and `proprietary`, a map from UTF-8 strings to byte strings.

Definition 5.3 (The `tx_modifiable` bitfield). Field `tx_modifiable` tracks which parts of the transaction are still open to change (`pczt`, `src/common.rs`; ZIP-374, Global):

Bit	Mask	Meaning	Merge
0	0b0000_0001	Transparent Inputs Modifiable	AND
1	0b0000_0010	Transparent Outputs Modifiable	AND
2	0b0000_0100	Has SIGHASH_SINGLE	OR
3–6	—	reserved; must be zero (merge fails otherwise)	—
7	0b1000_0000	Shielded Modifiable	AND

The Creator sets bits 0, 1, and 7 and clears bit 2; a Constructor checks the relevant bit before adding an input or output and may clear it; the IO Finalizer clears bits 0, 1, and 7 if there are shielded spends or outputs, and otherwise leaves the bitfield unchanged (§5.5). A Signer clears bit 0 when adding any signature that does not use `SIGHASH_ANYONECANPAY` — which includes *all* shielded signatures — and clears bit 1 for any signature not using `SIGHASH_NONE`; a `SIGHASH_SINGLE` signature also clears bit 1, because removing the paired output would not remove the signature. Bit 2 is set by any Signer using `SIGHASH_SINGLE` and tells Constructors to preserve the input/output pairing. Every Signer clears bit 7 (`pczt`, `src/common.rs`, Global doc, merge, and tests; ZIP-374, Global).

The split between transparent and shielded modifiable flags is deliberate: Zcash’s sighash pins *all* non-transparent effecting data as soon as any input is signed, so separate flags keep purely-transparent PCZTs semantically close to PSBTs, where Constructor and Signer roles can interleave when inputs are not `SIGHASH_ALL` (ZIP-374, Rationale). The deep construction of the sighash itself is the ZIP-244 digest covered by the *Ironwood Guide* (the transaction digest); this section only ever cites it.

Transparent inputs and outputs. A `TransparentInput` carries the outpoint (`prevout_txid` [u8; 32], `prevout_index` u32), sequence (Option<u32>, default 0xFFFFFFFF), the lock-time constraints `required_time_lock_time` (≥ 500000000) and `required_height_lock_time` (> 0 , < 500000000), the `script_sig` (set by the Spend Finalizer), `value` (u64) and `script_pubkey` — both required, because the Transaction Extractor needs every input’s UTXO to derive the shielded sighash for the binding signatures — `redeem_script` (P2SH only), `partial_signatures` (a map from 33-byte public keys to signatures), `sighash_type` (Signers MUST use it; Spend Finalizers MUST fail on signatures that mismatch it), `bip32_derivation`, four preimage maps (RIPEMD160/SHA-256/HASH160/HASH256), and `proprietary` (ZIP-374, `TransparentInput`; `pczt`, `src/transparent.rs`). A `TransparentOutput` carries `value`, `script_pubkey`, `redeem_script`,

bip32_derivation, user_address (Option<UTF8String>, set by an Updater; Signers MUST parse it and confirm that it contains the recipient, directly or as a receiver within a Unified Address), and proprietary (ZIP-374, TransparentOutput). Transparent partial signatures are stored as DER-encoded ECDSA — the elliptic-curve digital signature algorithm over Bitcoin’s curve secp256k1, the transparent pool’s signature scheme — with the sighash-type byte appended, $sig_bytes = DER(sig) \parallel sighash_type$, matching the Bitcoin/PSBT convention (zcash_transparent, src/pczt/signer.rs).

The Sapling bundle. A SaplingBundle carries lists of spends and outputs, a value_sum (i128, net spends minus outputs — wide enough that per-spend and per-output values can be redacted while the net remains), the anchor ([u8; 32]; in PCZT v1 set by the Creator and immutable, because the v5 sighash commits to it), and bsk (Option; None until the IO Finalizer runs, used by the Transaction Extractor for the binding signature). Each SaplingSpend requires cv, nullifier, and rk (effecting), and optionally carries a per-spend 192-byte Groth16 zkproof (Prover), a 64-byte spend_auth_sig (Signer), recipient (43 bytes), value, rcm or rseed (rcm only for pre-ZIP-212 notes; the Prover needs one of them), rcv (the IO Finalizer compresses these into bsk; the Prover requires it; it opens cv, hence is redactable), proof_generation_key (ak, nsk) (Updater sets, Prover requires), witness (a u32 position and a 32-level path), alpha, zip32_derivation, dummy_ask (a Constructor chooses it; the IO Finalizer uses and clears it; Signers MUST reject PCZTs containing it), and proprietary. Each SaplingOutput requires cv, cmu, ephemeral_key, enc_ciphertext, and out_ciphertext (whose length depends on the transaction version), and optionally carries zkproof, recipient, value, rseed (the Prover requires it, in preference to disclosing the shared secret), rcv, ock (lets a Signer verify out_ciphertext; None under the OVK policy None), zip32_derivation, user_address, and proprietary (ZIP-374, SaplingBundle/SaplingSpend/SaplingOutput; pczt, src/sapling.rs).

The Orchard bundle. An OrchardBundle carries (pczt, src/orchard.rs; ZIP-374, OrchardBundle/OrchardAction):

actions. A list of OrchardAction structures.

flags (u8). Bit 0 enableSpends, bit 1 enableOutputs; the remaining bits are reserved zeros in PCZT v1.

value_sum ((u64, bool)). The net value of spends minus outputs. The ZIP never states the boolean’s meaning; only the reference implementation defines it (true = negative, via the orchard crate’s sign mapping) — a spec gap we flag in §5.14.

anchor ([u8; 32]). Set by the Creator; fixed for the life of a v1 PCZT.

zkproof (Option<Vec<u8>>). A single Halo 2 proof for the whole bundle, set by the Prover — unlike Sapling’s per-spend and per-output proofs.

bsk (Option<[u8; 32]>). The binding signing key, set by the IO Finalizer.

Each OrchardAction is { cv^{net} , spend, output, rcv}: the net value commitment, a spend half, an output half, and one optional rcv per action, since an action carries a single net value commitment (the *Ironwood Guide*, conservation of value).

An OrchardSpend carries: nullifier and rk (32 bytes each; effecting, required); spend_auth_sig (64 bytes; Signer); recipient (43 bytes, the raw Orchard payment address encoding; a Constructor sets it, the Prover requires it); value (the Prover requires it; a Signer may verify it against cv_{net} and confirm change amounts; redactable); rho and rseed (32 bytes each); fvk (96 bytes, $ak \parallel nk \parallel r_{ivk}$ — the full viewing key of the note’s recipient; an Updater sets it, the Prover requires it); witness (a u32 position plus a 32-level, 1024-byte authentication path; Updater sets,

Prover requires); `alpha` (32 bytes; a Constructor chooses it, the Signer requires it; redactable once `zkproof` and `spend_auth_sig` are set); `zip32_derivation`; `dummy_sk` (a Constructor chooses it for dummy notes; the IO Finalizer requires and clears it; Signers MUST reject PCZTs containing it); and `proprietary`. An `OrchardOutput` carries the effecting fields `cmx`, `ephemeral_key`, `enc_ciphertext`, and `out_ciphertext` (all required), plus `recipient`, `value`, `rseed` (the Prover requires it instead of disclosing the shared secret), `ock`, `zip32_derivation`, `user_address` (Signers MUST parse it and confirm that it contains the recipient), and `proprietary` (`pczt, src/orchard.rs`; ZIP-374, `OrchardSpend/OrchardOutput`). The witness, anchor, and the note-commitment machinery these fields feed are exactly those of the *Ironwood Guide* (the commitment tree and authentication paths); we do not re-derive them here.

ZIP-32 derivations. A `Zip32Derivation` pairs a *seed_fingerprint* (`[u8; 32]`, the ZIP-32 seed fingerprint) with a *derivation_path* (`List<u32>`), where each index may carry the hardened flag, bit 31 (`index | 0x80000000`). Shielded spend paths are generally fully hardened; transparent paths generally include a non-hardened section matching BIP 44 or the ZIP-320 ephemeral-address path (`pczt, src/common.rs`). The orchard crate rejects any non-hardened component at parse time — Orchard does not support non-hardened derivation — and its method `extract_account_index` recognizes exactly the pattern `[32', coin_type', account']` under a matching seed fingerprint (`orchard, src/pczt.rs` and `src/pczt/parse.rs`).

5.3 Encoding

Construction 5.4 (PCZT encoding). A PCZT is encoded as

$$\underbrace{[0x50, 0x43, 0x5A, 0x54]}_{\text{ASCII PCZT}} \parallel \text{I2LEOSP}_{32}(\text{version}) \parallel \text{version-specific body},$$

an 8-byte header followed by the body. A parser rejects inputs shorter than 8 bytes (`TooShort`), a wrong magic (`NotPczt`), and — in the workspace crate — any version other than 1 or 2 (`UnknownVersion`); a parser encountering an unknown version MUST NOT parse the remainder. Files use the `.pczt` extension (ZIP-374, Encoding; `pczt, src/lib.rs`, `MAGIC_BYTES`, `parse/serialize`).

The body is a profile of the Postcard wire format — byte-for-byte the `serde` serialization of the `pczt` Rust crate, with the ZIP text canonical (ZIP-374, Encoding). Its primitives:

- `varint(n)`: emit n seven bits at a time, least-significant group first, one byte per group; every byte except the last has bit 7 set; encoders MUST emit the shortest encoding. A k -bit integer occupies at most $\lceil k/7 \rceil$ bytes, and in a maximum-length encoding the final byte contributes only the high $(k \bmod 7)$ bits — the fifth byte of a `u32` is at most `0x0F`; decoders MUST reject encodings exceeding either limit.
- Signed `i128` (the Sapling `value_sum`): `encode(n) = varint(zigzag(n))` with `zigzag(n) = (n << 1) ⊕ (n >> 127)`, the right shift arithmetic and the result reinterpreted as unsigned; a non-negative n encodes as $2n$, a negative n as $2|n| - 1$; at most 19 bytes.
- `Option`: a tag byte `0x00` (`None`) or `0x01` followed by the value.
- `List`: `varint(count)` then the elements.
- `Map`: `varint(count)` then the entries in strictly ascending lexicographic key order, duplicates forbidden.
- `Structs`: field concatenation in the ZIP's field order, with no framing or tags.

The v1 body is the concatenation

Global || TransparentBundle || SaplingBundle || OrchardBundle,

with the transparent bundle a pair of input and output lists, the Sapling bundle {spends, outputs, value_sum, anchor, bsk}, and the Orchard bundle {actions, flags, value_sum, anchor, zkproof, bsk} (ZIP-374, Encoding and Data type encodings).

A minimal PCZT, byte by byte. The smallest PCZT the Creator can emit makes the positional encoding concrete. Instantiating the Creator for NU6.2 (branch 0x5437F330), expiry height 10,000,000, coin type 133, and building with no spends or outputs yields the 32-byte version-2 stream below — the encoding the workspace crate’s `serialize` emits (computed by script against the `pczt` crate; every byte is assertion-checked):

offset	field	bytes
0	magic "PCZT"	50 43 5a 54
4	version 2 (raw LE u32)	02 00 00 00
8	tx_version = 5 (varint)	05
9	version_group_id (varint)	8a ce 9c b5 02
14	consensus_branch_id (varint)	b0 e6 df a1 05
19	fallback_lock_time = None	00
20	expiry_height (varint)	80 ad e2 04
24	coin_type = 133 (varint)	85 01
26	tx_modifiable = 0b1000_0011	83
27	proprietary: empty map	00
28–31	four absent bundles	00 00 00 00

Three features of the Postcard discipline are visible at once. Integers are LEB128 varints, so `coin_type = 133` costs two bytes where 1 would cost one — and re-encoding with coin type 1 indeed shortens the stream to 31 bytes, shifting every later field: the encoding is positional, which is precisely why fields cannot be inserted within a version (cross-checked by script: single-field variants differ exactly at the annotated offsets). Absence is one 0x00 byte: the version-2 format (what `Pczt::serialize` always emits — the latest version; v1 output requires `v1::Pczt::serialize`; `pczt src/lib.rs`) wraps each of the transparent, Sapling, Orchard, and Ironwood bundles in an `Option` that collapses when the bundle is empty, so the entire bundle machinery of Construction 5.4 above costs four bytes until a first spend or output materialises. And the initial `tx_modifiable = 0x83` sets the two PSBT-inherited modifiable bits — transparent inputs (bit 0) and outputs (bit 1) — plus ZIP-374’s Zcash-specific shielded-modifiable bit 7, per Definition 5.3.

Extensibility is deliberately narrow. New fields cannot be added within an existing PCZT version, because the Postcard encoding is positional; unlike PSBT there are no non-proprietary arbitrary key-value maps, so a Signer’s view of a PCZT is complete. The proprietary maps are the only within-version extension point and are reserved for private use; format evolution happens only through new versions namespaced by the header’s 4-byte version (ZIP-374, Extensibility and Proprietary Use fields). Serialization is deterministic: the crate’s round-trip test checks `serialize(parse(serialize(p))) = serialize(p)` (`pczt, tests/end_to_end.rs, check_round_trip`).

5.4 Creation and construction

The deployed Creator, `Creator::new(consensus_branch_id, expiry_height, coin_type, sapling_anchor, orchard_anchor)`, defaults to the v5 transaction format — `tx_version = 5` (`V5_TX_VERSION`), `version_group_id = 0x26A7270A` (`V5_VERSION_GROUP_ID`); `zcash_protocol, src/constants.rs` — with initial `tx_modifiable = 0b1000_0011` (bits 0, 1, 7 set), Orchard `flags = 0b0000_0011` (spends and outputs enabled), and empty bundles with Sapling

value_sum = 0 and Orchard value_sum = (0, false); builder options with_fallback_lock_time and with_orchard_flags adjust the defaults (pczt, src/roles/creator/mod.rs). The Orchard initialization is non-negative zero, matching the v2 draft’s canonical empty OrchardBundle and EMPTY_ORCHARD (§5.14).

The crate ships no Constructor implementation (“The roles currently without an implementation are: Constructor”; pczt, src/roles.rs). In the deployed stack the Constructor step is performed by the zcash_primitives transaction builder: Builder::build_for_pczt(rng, fee_rule) checks that the value balance minus the fee is exactly zero (failing with InsufficientFunds or ChangeRequired otherwise), then builds the per-pool PCZT bundles *without proving or signing*, returning a PczarResult whose PcztParts carry the parameters, version, branch id, lock_time = 0, expiry height, and the four bundles (transparent, Sapling, Orchard, and Ironwood), together with the Sapling, Orchard, and Ironwood bundle metadata (zcash_primitives, src/transaction/builder.rs, build_for_pczt, PcztParts). Then Creator::build_from_parts(parts) assembles the Pczt, setting tx_modifiable = 0b0000_0000 — nothing modifiable, plus the Has-SIGHASH_SINGLE bit if any input uses it — fallback_lock_time = Some(lock_time), and coin_type from the network parameters (pczt, src/roles/creator/mod.rs, build_from_parts).

The Orchard side of that construction produces, per action (orchard, src/builder.rs, SpendInfo::into_pczt, OutputInfo::into_pczt, build_for_pczt):

$$cv^{\text{net}} := \text{ValueCommit}^{\text{Orchard}}(v^{\text{net}}, rcv), \quad nf_{\text{old}} := \text{the spent note's nullifier under fvk},$$

a fresh uniformly random α , and $rk := \text{Randomize}(ak, \alpha)$ — the RedPallas re-randomization of the *Ironwood Guide* (spend authorization) — populating the spend’s recipient, value, rho, rseed, fvk, witness, and alpha (all Some), plus dummy_sk for dummy spends. The output half builds the new note with $\rho := nf_{\text{old}}$ of the paired spend, computes cmx, and runs the note encryption of the *Ironwood Guide*, populating recipient, value, and rseed. It currently sets ock = None with an in-code TODO (“Extract ock from the encryptor ...”), so the deployed Constructor never fills ock even when an outgoing viewing key is used — a divergence from the ZIP’s field description (§5.14).

Bundles are padded with dummy actions and their positions randomized, so a consumer must map logical spends and outputs to bundle positions through the builders’ metadata: spend_action_index(n) and output_action_index(n) on the Orchard BundleMetadata, and SaplingMetadata for Sapling (orchard, src/builder.rs). The wallet backend uses exactly this to attach per-output metadata to the right PCZT actions (zcash_client_backend, src/data_api/wallet.rs, create_pczt_from_proposal).

5.5 IO finalization: the binding-signature key

Once the input/output set is complete, the IO Finalizer freezes it. The deployed role fails with NoSpends/NoOutputs if the transaction would have no spends or no outputs; if there are shielded spends or outputs it clears tx_modifiable bits 0, 1, and 7. It computes the ZIP-244 *shielded sighash*

$$sighash := v5_signature_hash(tx_data, \text{SignableInput}::\text{Shielded}, txid_parts),$$

where tx_data is the *effects-only* transaction data reconstructed from the PCZT — proofs and signatures do not affect the sighash — and $txid_parts = tx_data.digest(\text{TxIdDigester})$; only (tx_version, version_group_id) = (5, 0x26A7270A) is supported. The digest algorithm itself is the ZIP-244 construction of the *Ironwood Guide* (the transaction digest). It then calls the per-pool finalize_io(sighash, OsRng) (pczt, src/roles/io_finalizer/mod.rs).

Construction 5.5 (Orchard IO finalization). Every action’s rcv must be present. The role computes

$$bsk := \sum_{a \in \text{actions}} rcv_a \in \mathbb{F}_{p_{\text{Vesta}}},$$

converted through its 32-byte little-endian representation into a RedPallas binding signing key (`ValueCommitTrapdoor::into_bsk`), and recomputes

$$\text{bvk} := \left(\sum_a \text{cv}_a^{\text{net}} \right) - \text{ValueCommit}^{\text{Orchard}}(\text{value_sum}, 0),$$

where $\text{ValueCommit}^{\text{Orchard}}(v, r) := [v] V^{\text{Orchard}} + [r] R^{\text{Orchard}}$ with

$$\begin{aligned} V^{\text{Orchard}} &= \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, v), \\ R^{\text{Orchard}} &= \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, r) \end{aligned}$$

— the net value commitment of the *Ironwood Guide* (conservation of value). It fails with `ValueCommitMismatch` unless `VerificationKey(bsk) = bvk`: this is the ZIP’s requirement that `value_sum` be consistent with the value commitments and trapdoors being aggregated. Finally, for each action with `dummy_sk` present, it *takes* the key (`Option::take`, guaranteeing that no spend authorizing key material remains after this role), derives `ask = SpendAuthorizingKey(sk)`, and signs the action with the shielded sighash (`orchard, src/pczt/io_finalizer.rs`; ZIP-374, IO Finalizer).

The Sapling analogue is $\text{bsk} := \sum_{\text{spends}} \text{rcv} - \sum_{\text{outputs}} \text{rcv}$ as a Jubjub scalar, made into a RedJubjub binding signing key, checked against $\text{bvk} = (\sum_{\text{spends}} \text{cv} - \sum_{\text{outputs}} \text{cv})$ adjusted by `value_sum` as an i64 (`InvalidValueSum` if it does not fit); dummy spends are then signed with their `dummy_ask`, which is taken and cleared (`sapling-crypto, src/pczt/io_finalizer.rs`; the workspace pins version 0.7, same algorithm).

Remark 5.6 (Why a dedicated role). The key `bsk` cannot be computed by the Transaction Extractor without preserving every `rcv` until extraction — revealing the opening of every value commitment to all intermediate participants — and cannot be maintained incrementally by Constructors, because with parallel construction and a Combiner the incremental update would be wrong. A dedicated role that runs once, after IO completion, resolves both. The same role signs dummy spends so that no Signer ever parses spending key material (ZIP-374, Rationale, “Adding an IO Finalizer role”).

5.6 Updating

The deployed Updater wraps the protocol crates’ updater types. Method `update_global_with` sets proprietary fields; the wrapped Orchard `ActionUpdater` exposes, per action, the setters `set_spend_zip32_derivation` and `set_spend_proprietary` on the spend half, and on the output half the setters `set_output_zip32_derivation`, `set_output_user_address`, and `set_output_proprietary` (with Sapling and transparent analogues). In PCZT v1 anchors are fixed at creation, so the deployed Updater cannot set anchors or witnesses — those come from the Constructor (`pczt, src/roles/updater/mod.rs` and `orchard.rs`; `orchard, src/pczt/updater.rs`).

5.7 Signing

Construction 5.7 (The Signer role). The deployed Signer (`pczt feature signer`), `Signer::new`, parses all bundles, extracts an effects-only `TransactionData`, and computes `txid_parts` and the shielded sighash of §5.5 *once*, cached across signatures. Its methods are `shielded_sighash()` and `transparent_sighash(index)` — the latter

$$\text{v5_signature_hash}(\text{tx_data}, \text{SignableInput::Transparent}(\text{input } i), \text{txid_parts})$$

under the input’s declared `sighash_type` — the signing methods `sign_transparent`,

`sign_sapling`, and `sign_orchard`, their `append_/apply_` variants for externally produced signatures (a hardware device), and `finish()`. To sign an Orchard spend, `alpha` must be present; with the wallet's ask:

`rsk := ask +  $\alpha$` (the RedPallas SpendAuth randomization), `require VerificationKey(rsk) = rk`
 — failing with `WrongSpendAuthorizingKey` otherwise — then

`spend_auth_sig := RedPallas.Sign(rsk, sighash).`

The external path `apply_orchard_signature` accepts a signature iff it verifies under the action's `rk` over the shielded `sighash` (`InvalidExternalSignature` otherwise). After any shielded signature all three modifiable bits are cleared; after a transparent signature the bits are updated per its `sighash` type as Definition 5.3 specifies (`pczt, src/roles/signer/mod.rs; orchard, src/pczt/signer.rs`).

All shielded signatures behave as committing to the whole transaction — there is no shielded `SIGHASH_ANYONECANPAY` — which is why every Signer clears every modifiable bit (ZIP-374, Global, `tx_modifiable`). Before signing an Orchard spend, the deployed Signer calls `verify_nullifier` (Construction 5.8) and treats `Missing{Recipient, Value, Rho, RandomSeed}` as acceptable — those fields may legitimately have been redacted for this signer — but hard-fails on any actual inconsistency; likewise for Sapling (`pczt, src/roles/signer/mod.rs, generate_or_apply_orchard_signature`).

Construction 5.8 (Per-action verification equations). The orchard crate exposes, for Signers and Verifiers (`orchard, src/pczt/verify.rs`):

```

verify_cv_net :    $cv^{net} \stackrel{?}{=} \text{ValueCommit}^{\text{Orchard}}(v_{\text{spend}} - v_{\text{output}}, rcv);$ 
verify_nullifier : rebuild the note from (recipient,  $v, \rho, rseed$ ),
                  require fvk owns it (fvk.scope_for_address is Some),
                  and  $nf \stackrel{?}{=} \text{the note's nullifier under fvk};$ 
verify_rk :       ak from fvk,    $rk \stackrel{?}{=} \text{Randomize}(ak, \alpha);$ 
verify_note_commitment : rebuild the output note with  $\rho := nf_{\text{old}}$  of the paired spend,
                         $cmx \stackrel{?}{=} \text{Extract}(\text{NoteCommit}_{rcm}(...)),$ 

```

with the nullifier, randomization, and commitment constructions those of the *Ironwood Guide*. For dummy notes ($v = 0$) an `expected_fvk` argument is ignored in favour of the spend's own `fvk` field.

A signer implementation “should only need to implement”: Pedersen commitments using Jubjub/Pallas arithmetic (note and value commitments); BLAKE2b and BLAKE2s and the PRFs/CRHs built on them; the nullifier check; the KDF plus note decryption (`AEAD_CHACHA20_POLY1305`); the `SignatureHash` algorithm; RedJubjub/RedPallas signatures; and a source of randomness — no proving circuitry (`pczt, src/roles/signer/mod.rs, module doc`). For dependency-constrained environments a `low_level_signer::Signer` variant (not feature-gated) exposes `sign_orchard_with/sign_sapling_with/sign_transparent_with` closures over the parsed bundles plus `tx_modifiable` (`pczt, src/roles/low_level_signer/mod.rs`).

5.8 Proving

The Prover attaches proofs; per the ZIP it requires, for each Orchard action, the spend’s recipient, value, rho, rseed, fvk, witness, and alpha, plus the action’s rcv and the bundle anchor; delegating proving therefore reveals note private data, but not spending authority (ZIP-374, Prover). The deployed role (pczt feature prover) offers `requires_sapling_proofs` (any Sapling spend or output missing its zkproof) and `requires_orchard_proof` (actions non-empty and the bundle zkproof `None`), letting a wallet skip downloading the Sapling parameters when they are unneeded; `create_orchard_proof(pk)` parses the bundle and calls the orchard crate’s `Bundle::create_proof(pczt, src/roles/prover/mod.rs and orchard.rs)`.

Construction 5.9 (Orchard proof creation from a PCZT). With no actions the call is a no-op. Otherwise, for each action i the prover rebuilds the spend information (fvk_i , note_i , merkle path $_i$) — failing with `WrongFvkForNote` if fvk_i does not own the note — and the output note with $\rho_i := \text{nf}_{\text{old},i}$ (`RhoMismatch` if inconsistent), then forms

$$\begin{aligned} \text{circuit}_i &:= \text{Circuit}(\text{spend}_i, \text{output note}_i, \alpha_i, \text{rcv}_i), \\ \text{instance}_i &:= (\text{anchor}, \text{cv}_i^{\text{net}}, \text{nf}_i, \text{rk}_i, \text{cmx}_i, \text{flags}), \end{aligned}$$

rejecting the identity rk (`IdentityRk`, added in orchard 0.13.0 for the consensus rule introduced in zcashd v6.12.1 / Zebra 4.3.1), and creates a *single* Halo 2 proof over all actions, stored as the bundle’s zkproof. The proof commits to the anchor and the flags as public inputs; the statement proved is the Action relation of the *Ironwood Guide* (`orchard, src/pczt/prover.rs`).

Because v5 signatures commit to the anchor but not to proofs, and proofs do not depend on signatures, the Prover and Signer can run in either order or in parallel; the `zcash_client_backend` documentation says to use the Combiner “if you create proofs and apply signatures in parallel” (ZIP-374, Prover; `zcash_client_backend, src/data_api/wallet.rs`).

5.9 Combining, redaction, and privacy

Construction 5.10 (Merge algebra). The deployed Combiner — `Combiner::new(Vec<Pczt>)` followed by `combine()` — left-folds pairwise merges; per-protocol bundles are merged before `Global`, “because each is only interpretable in the context of its own global”. The rules (`pczt, src/roles/combiner/mod.rs, src/common.rs, src/orchard.rs`):

- Global effecting data (`tx_version`, `version_group_id`, `consensus_branch_id`, `fallback_lock_time`, `expiry_height`, `coin_type`) must be equal; `tx_modifiable` merges bit-by-bit per Definition 5.3.
- Optional fields: `merge( $\ell$ , None) =  $\ell$ ; merge(None,  $r$ ) =  $r$ ; merge(Some  $\ell$ , Some  $r$ ) =  $\ell$  iff  $\ell = r$ , else FAIL. Maps merge by key union, with values required equal on collision.`
- Bundle structure: flags must be equal; if either side’s `bsk` is set (the IO Finalizer has run), action counts and `value_sum` must match; otherwise a strict prefix extension appends the extra actions *only if* the shorter side’s global has `Shielded Modifiable` set — else FAIL. Anchors must be equal, and the per-action required fields (`cvnet`, `nullifier`, `rk`, `cmx`, `ephemeral_key`, `enc_ciphertext`, `out_ciphertext`) must be equal.

Combining must be functionally order-independent:

$$\text{Combine}(f_A(p), f_B(p)) = f_A(f_B(p)) = f_B(f_A(p)).$$

The equality-or-fail rule intentionally differs from BIP 174, which on conflicting values says the combiner “must choose the value it considers correct” — in practice last-wins — silently discarding data. Because every PCZT field is typed and known up front, requiring equality is always possible and turns every conflict into a loud failure. A Combiner **MUST NOT** combine PCZTs that do not represent the same transaction; once IO is finalized, a PCZT is uniquely identified by the txid its effecting data determines (ZIP-374, Combiner and Rationale; `pczt, src/roles/combiner/mod.rs, merge_optional` comment).

Redaction. The Redactor provides per-bundle, per-field clear methods. The Orchard ActionRedactor can clear `spend_auth_sig`; the spend’s recipient, value, rho, rseed, fvk, witness, alpha, zip32_derivation, and dummy_sk; the spend’s proprietary entries (by key or all); the output’s recipient, value, rseed, ock, zip32_derivation, and user_address; the output’s proprietary entries; and rcv. The bundle-level redactor can clear zkproof and bsk. The intended use is multiple independent Signers, each given a PCZT with only the information it needs — for example, without the other signers’ α values (`pczt, src/roles/redactor/mod.rs` and `orchard.rs`).

Privacy. A PCZT contains strictly more information than the transaction extracted from it: potentially counterparty addresses, values, note randomness, rcv (which opens cv), the spending account’s full viewing key, ZIP-32 paths, ock (which opens out_ciphertext), and user-facing address strings. Whoever receives a PCZT at a given stage learns everything present at that stage; participants **SHOULD** send it only to entities required for the remaining roles and **SHOULD** use the Redactor to strip fields. Delegating the Prover reveals note private data — not spending authority. The extracted transaction contains none of this extra information (ZIP-374, Privacy Implications).

5.10 Transparent finalization and lock time

Construction 5.11 (Script-sig assembly). The Spend Finalizer constructs each transparent input’s `script_sig`. Two forms are supported (`zcash_transparent, src/pczt/spend_finalizer.rs`; `pczt, src/roles/spend_finalizer/mod.rs`):

- *P2PKH*: require exactly one entry (`pubkey, sig`) in `partial_signatures` satisfying

$$\text{RIPEMD160}(\text{SHA256}(\text{pubkey})) = \text{the address hash};$$

then

$$\text{script_sig} := \text{PUSH}(\text{sig} \parallel \text{sighash_type}) \parallel \text{PUSH}(\text{pubkey}).$$

- *P2SH-P2MS* (m -of- n multisig):

$$\text{script_sig} := \text{OP_0} \parallel \text{PUSH}(\text{sig}_1) \parallel \cdots \parallel \text{PUSH}(\text{sig}_m) \parallel \text{PUSH}(\text{redeem_script}),$$

where the leading `OP_0` is the dummy for the `OP_CHECKMULTISIG` bug and the signatures are the first m available ones *in redeem-script pubkey order* (surplus signatures beyond the threshold are discarded; fewer than m is a failure; pubkeys must be compressed 33-byte keys).

After finalization the role clears each input’s required lock times, redeem script, partial signatures, BIP-32 derivations, and all preimage maps, keeping value and `script_pubkey` — the UTXO — so the Transaction Extractor can verify the final transaction.

Construction 5.12 (Lock-time determination). The final `nLockTime` follows BIP 370’s rule (ZIP-374, Determining Lock Time; `pczt, src/common.rs, determine_lock_time`). Let r hold if some input has a required time or height lock; let t_u (“time unsupported”) hold if some input requires a height lock (so a time lock cannot serve it), and h_u symmetrically. Then:

$$\text{nLockTime} = \begin{cases} \text{FAIL} & r \wedge t_u \wedge h_u, \\ \text{max required height locks} & r \wedge \neg h_u, \\ \text{max required time locks} & r \wedge h_u \wedge \neg t_u, \\ \text{fallback_lock_time or 0} & \neg r, \end{cases}$$

height winning when both types remain possible; an input specifying neither accepts either type. The crate fails with `IncompatibleLockTimes` when one input requires only time and another only height — and, diverging from ZIP-374/BIP 370, also when a single input specifies both lock types: `determine_lock_time` derives each type’s “unsupported” flag from any input carrying the other type’s field, so a both-specifying input sets $t_u \wedge h_u$ and fails, though the ZIP has it take the height lock.

5.11 Extraction and verification

The Transaction Extractor (`pczt` feature `tx-extractor`) checks transaction-version support and lock-time compatibility, then extracts the per-pool bundles, requiring every `script_sig`, every Sapling `zkproof` and `spend_auth_sig`, the Orchard bundle `zkproof` of canonical length for its action count, every Orchard `spend_auth_sig`, and each non-empty bundle’s anchor and `bsk`; it rejects the identity `rk` and an invalid ephemeral_key (`pczt, src/roles/tx_extractor/mod.rs`).

On the Orchard side, extraction converts the PCZT bundle into a regular bundle whose authorization is `Unbound = {proof, bsk}`; the canonical proof size is validated already at the `Unbound` stage, so the invariant “an Authorized bundle always has a canonical proof” holds across the binding step (this canonical-size check — `Proof::expected_proof_size` and `NonCanonicalProofSize` — entered orchard in 0.14.0 as the GHSA-2x4w-pxqw-58v9 fix), and `value_balance = i64(value_sum)` with `ValueSumOutOfRange` on overflow. The binding step is:

verify every `spend_auth_sig` under its `rk` over `sighash`, `binding_sig := Sign(bsk, sighash)`,

failing with the `pczt` crate’s `Error::SighashMismatch` (`src/roles/tx_extractor/mod.rs`) if any `spend` authorization does not verify — the crate maps the `None` that orchard’s `apply_binding_signature` returns on failure; the `bsk/bvk` consistency established by the IO Finalizer (Construction 5.5) guarantees the binding signature verifies under the `bvk` derived from the transaction’s value commitments and value balance (`orchard, src/pczt/tx_extractor.rs, apply_binding_signature`).

Finally the extractor *verifies* the completed transaction — proofs and signatures — “so that an invalid transaction is detected before it is broadcast” (ZIP-374, Transaction Extractor). Sapling proof verification requires caller-provided `SpendVerifyingKey/OutputVerifyingKey` (error `SaplingRequired` if absent but needed); the Orchard verifying key is generated on the fly if not provided via `with_orchard` (`pczt, src/roles/tx_extractor/mod.rs`).

5.12 The deployed wallet pipeline

The wallet layer drives the roles end to end. Function `create_pczt_from_proposal` — taking the wallet database, network parameters, spending account, OVK policy, and proposal (`zcash_client_backend` feature `pczt, src/data_api/wallet.rs`) — supports only single-step proposals. It runs the same internal transaction construction as its non-PCZT sibling `create_`

proposed_transactions; then chains `Builder::build_for_pczt`, `Creator::build_from_parts`, and `IoFinalizer::finalize_io`; and finally runs an Updater pass that:

1. stores the proposal metadata $\{from_account, target_height\}$, postcard-encoded, under the Global proprietary key `zcash_client_backend:proposal_info`;
2. stores a reduced recipient enum — `External` (0), `EphemeralTransparent` (1), or `InternalAccount` (2), the latter two carrying the receiving account — under the per-output proprietary key `zcash_client_backend:output_info` on each shielded and transparent output;
3. sets `user_address` on external outputs; and
4. attaches derivation paths: to Orchard spends and non-dummy Sapling spends the fully hardened $[32', coin_type', account']$ under the account's seed fingerprint, and to transparent inputs the BIP-44 path $[44|H, coin_type|H, account|H, scope, address_index]$ with $H = 0x80000000$ and the last two components non-hardened.

The PCZT is then handed to the caller for the Prover, Signer, and Combiner roles — on other devices if need be.

The return path is `extract_and_store_transaction_from_pczt(wallet_db, pczt, sapling_vk, orchard_vk)`: it runs the Spend Finalizer, reads back the proprietary proposal and output metadata — failing if `proposal_info` is missing, so the PCZT must have come from `create_pczt_from_proposal` — reconstructs each output's note from the PCZT's explicit recipient/value/rseed fields (outputs whose recipient is absent are dummies; outputs whose note fields were redacted are treated as deliberately unrecoverable and skipped), runs the Transaction Extractor, recovers memos via `try_output_recovery_with_pkd_esk` with `esk` re-derived from the note (post-ZIP-212; the derivation is the PRFexpand construction of the *Ironwood Guide*, note encryption), computes the fee from the PCZT's transparent input values, and stores the result as a `SentTransaction`, returning the txid. The `sapling_vk` argument is optional so callers can first check `Prover::requires_sapling_proofs` and avoid downloading the Sapling parameters (`zcash_client_backend`, `src/data_api/wallet.rs`).

Feature gating. The `pczt` crate is `no_std` (alloc-based). The `Creator`, `Verifier`, `Updater`, `Redactor`, `Combiner`, and low-level `Signer` are always available; `io-finalizer`, `prover`, `signer`, `spend-finalizer`, and `tx-extractor` are cargo features; per-pool support is gated by the `transparent`, `sapling`, and `orchard` features; and `zcp-builder` enables `Creator::build_from_parts` from the `zcash_primitives` builder output (`pczt`, `Cargo.toml`, `src/lib.rs`, `src/roles.rs`). The crate's end-to-end tests exercise the full pipeline — `Creator/Builder` → `IO Finalizer` → `Updater` → `Prover` → `Signer` → `Combiner` → `Spend Finalizer` → `Transaction Extractor` — for the `transparent_to_orchard`, `transparent_p2sh_multisig_to_orchard`, `sapling_to_orchard`, and `orchard_to_orchard` flows (`pczt`, `tests/end_to_end.rs`).

ZIP-48 multisig. ZIP-48 (transparent multisig) prescribes PCZT as its transaction-construction workflow: one participant constructs a PCZT spending multisig UTXOs, with change to a P2SH address at $m/48'/coin_type'/account'/133000'/1/*$ (the 133000' component reads “Zcash P2SH”); the PCZT is conveyed out-of-band over a private authenticated channel; each participant MUST verify that change outputs are wallet-controlled, verifies that the spend is expected, adds their signature(s), and returns the signed PCZT; one participant combines a threshold of signed PCZTs and extracts the final transaction (ZIP-48, Transaction construction). The crate's `transparent_p2sh_multisig_to_orchard` test exercises exactly this flow.

Tooling. The zcash-devtool CLI exposes the full role pipeline as subcommands: `pczt create / create-max / shield / create-manual / pay-manual` (Creator through IO Finalizer via `zcash_client_backend`), `inspect`; `update-with-derivation`; `redact`; `prove`; `sign`; `update-with-signature`, which applies externally created signatures; `combine`; `extract`; `send`; `send-without-storing`; and `to-qr / from-qr` — animated QR rendering and webcam scanning of PCZTs (feature `pczt-qr`), illustrating the constrained-transport hardware-wallet flow. Its `sign` command selects which spends to sign by matching the seed fingerprint and account in the PCZT’s `zip32_derivation` and `bip32_derivation` fields against the wallet’s seed (`extract_account_index`), then calls the Signer’s per-pool methods (`zcash-devtool`, `src/commands/pczt.rs` and `src/commands/pczt/sign.rs`).

5.13 PCZT v2: deferred anchors and the Ironwood bundle

The draft’s v2 (version number 2 in the header) extends the format for v6 transactions (ZIP-229): it adds an `IronwoodBundle`, adds a `note_version` field (an enum {V2,V3}) to Orchard-protocol bundles, makes shielded bundle anchors `Option<[u8;32]>`, and omits canonically-empty bundles from the encoding (ZIP-374, Versioning and v2 Encoding). Classification per our conventions: *implemented and released* — specified in the Draft ZIP and released in `pczt` 0.8.0 on 2026-07-23 (the v2 module with `Option`-wrapped bundles and empty-bundle omission, the `ironwood` slot, `note_version`, and optional anchors). The current 0.9.3 logical model retains these fields.

For `tx_version` 6, anchors and witnesses become deferrable: signatures commit to neither, an Updater may set or replace an anchor before proving — replacing an anchor MUST clear the bundle’s proofs, which commit to it as a circuit public input — and witnesses must root to the anchor whenever both are present, checked by the Updater and the Prover. This enables re-anchoring fully signed transactions with the `txid` unchanged, and spending *unmined* Orchard-protocol notes: an Orchard nullifier depends only on note data, with ρ fixed by the creating action rather than by tree position — impossible for Sapling even under v6, because Sapling nullifiers depend on the note’s position. An all-zeroes anchor sentinel was rejected in favour of `Option` because 0 is a valid Pallas base element (ZIP-374, Anchors and pre-authorization; Rationale, “Optional anchors in PCZT v2”).

From NU6.3, Orchard-pool actions must be created with cross-address transfers disabled — bundle flag bit 2, `enableCrossAddress`, MUST be 0 in PCZT v1 and in any Orchard bundle mined under NU6.3+ — and wallets pair each real spend with a *fabricated* zero-valued output to the spent note’s own receiver, whose `enc_ciphertext` is random bytes. Signers MUST NOT reject a zero-valued output whose ciphertext does not decrypt on that basis alone, and SHOULD classify it as a tolerable dummy (optionally checking `cmx` against the explicit recipient/value/rseed). Conversely, a fabricated zero-valued spend paired with a real output is a *real* spend — no `dummy_sk`; it is signed via the normal Signer flow with the receiver’s spend authorizing key; only fully-dummy actions carry `dummy_sk` (ZIP-374, Fabricated same-address outputs). The `IronwoodBundle` carries the Ironwood-pool component of a v6 transaction, structurally identical to `OrchardBundle` — ZIP-229 defines the Ironwood pool as a second value pool of the Orchard protocol reusing the Action structure — differing in that `enableCrossAddress` is normally 1 (cross-address transfers are the usual case in the Ironwood pool), the anchor refers to the Ironwood note commitment tree, and `note_version` MUST be V3 (the ZIP-2005 quantum-recoverable note plaintext, lead byte 0x03); a non-empty `IronwoodBundle` in a `tx_version`-5 PCZT MUST be rejected (ZIP-374, `IronwoodBundle` and Rationale).

The orchard 0.15 release line implements the NU6.3-era machinery — bundle-format-aware flag parsing (`Flags::from_byte(flags, format)`, `UnexpectedFlagBitsSet`); the check `verify_cross_address_restriction`, requiring every action’s output addressed to the same (`gd`, `pkd`) as its spend when flag bit 2 is 0, enforced by `finalize_io` and `create_proof` and provable only with an Ironwood-version proving key; same-receiver comparison via `Address::same_receiver`; and change outputs paired with a fabricated same-receiver spend (`orchard`, `src/pczt/verify.rs`, `io_finalizer.rs`, `prover.rs`, `parse.rs`, and `src/builder.rs`; `librustzcash Cargo.toml`). The current standalone head is 0.15.5; the `librustzcash` lockfile resolves

0.15.3.

5.14 Divergences between ZIP-374 and the deployed code

Per our conventions, we flag rather than resolve the following.

- *Dummy-key rejection is not implemented in the reference Signer.* ZIP-374 states that “Signers MUST reject PCZTs that contain `dummy_sk` values” (likewise `dummy_ask`), but no code path in the `pczt` Signer, through the current 0.9.3 source, inspects those fields — only the Redactor offers `clear_spend_dummy_sk/clear_dummy_ask`, and the IO Finalizer clears the keys after use. The MUST therefore binds external signer implementations, such as hardware wallets, not the crate’s own Signer type (ZIP-374, OrchardSpend.dummy_sk; pczt, src/roles/signer/).
- *A v4 PCZT is constructible but dead.* The ZIP forbids creating v4-or-earlier transactions, yet `Creator::build_from_parts` maps `TxVersion::V4` to a PCZT with `tx_version = 4` (Sprout/V3 return `None`). Extraction and therefore roles that parse transaction effects reject that version as `UnsupportedTxVersion`, so the state cannot progress (pczt, src/roles/creator/mod.rs; src/lib.rs, `extract_tx_version`).
- *Negative-zero initialization (resolved upstream).* The released pczt 0.5.1 had `Creator::new` set the Orchard `value_sum = (0, true)`, differing from the v2 draft’s canonical empty `OrchardBundle (0, false)`; the byte encodings differ, which would have prevented the v2 empty-bundle-omission rule from round-tripping. The workspace head now sets `(0, false)`, matching `EMPTY_ORCHARD`, so the divergence is closed (pczt, src/roles/creator/mod.rs; ZIP-374, v2 Encoding).
- *The `ock` field is never populated.* The ZIP describes Sapling/Orchard `ock` as `None` only under the OVK policy `None`, but the deployed Orchard builder always sets `ock = None` (in-code TODO), so the Signer capability of verifying `out_ciphertext` is not currently exercisable for wallet-created PCZTs (orchard, src/builder.rs, `OutputInfo::into_pczt`).
- *Unspecified sign convention.* The ZIP declares the Orchard `value_sum` as `(u64, bool)` without stating the boolean’s meaning; only the reference implementation defines it (`true = negative`) — a spec gap (ZIP-374, OrchardBundle; orchard, src/pczt/parse.rs).
- *Reference-version scope.* The ZIP names pczt 0.7.0 specifically as the release that defines the v1 encoding. Current pczt 0.9.3 still exposes that fixed encoding through `pczt::v1`, but also ships the ZIP’s v2 encoding and Ironwood bundle. This is no longer an implementation-versus-release skew; 0.7.0 remains a deliberate historical anchor for v1 bytes (ZIP-374, Reference implementation; pczt, CHANGELOG.md).

6 Broadcast, expiry, and the transaction lifecycle

The *Ironwood Guide* leaves off with a transaction fully proved, signed, and bound — an artefact whose internal validity is settled. This section follows that artefact through the rest of its life: serialisation and the identifier under which the network knows it, the deployed store-then-broadcast discipline, the expiry mechanism that bounds how long it can remain pending, the bookkeeping by which the wallet tracks it to one of its terminal states, the confirmation policy that governs when its outputs become spendable, and the truncate-and-rescan procedure that repairs the wallet’s view after a chain reorganisation. Throughout, we develop the shielded path; the transparent-input monitoring machinery (`TransactionsInvolvingAddress` and its filters) is gated behind the `transparent-inputs` feature of `zcash_client_backend` and is mentioned only in passing, so for a shielded-only wallet the monitoring surface is exactly the two status queries of §6.4.

6.1 Serialisation and the transaction identifier

A built transaction is serialised for broadcast by `Transaction::write`, which dispatches on version. Transactions v5 (version-group ID 0x26A7270A) use `write_v5`, while v6 (version-group ID 0xD884B698) use `write_v6`, which adds the separately encoded Ironwood bundle; v1–v4 use `write_v4` (ZIP-225; ZIP-229; protocol specification §7.1, *Transaction Encoding and Consensus*; `zcash_protocol`, `src/constants.rs`; `zcash_primitives`, `src/transaction/mod.rs`). The wallet places the resulting raw bytes in the data field of a `RawTransaction` gRPC message for submission (`zcash-devtool`, `src/commands/wallet/send.rs`).

The identifier under which the network, the mempool, and the wallet’s own database all track the transaction depends on the version. For v5 and v6, the *txid* is the root of the ZIP-244 digest tree,

$$\text{txid} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{trans}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}}),$$

for v5; v6 appends the separate Ironwood digest d_{Iron} to the same top-level preimage. Every absent shielded pool contributes its protocol-defined empty-bundle digest. Here the personalisation is the 12-byte literal `ZcashTxHash_` (one underscore) followed by `branchid` = $\text{LE}_{32}(\text{CONSENSUS_BRANCH_ID})$, the 4-byte little-endian consensus branch id, and an absent pool hashes the empty string under its own personalisation (ZIP-244, *Txid Digest*; `zcash_primitives`, `src/transaction/txid.rs`). The *Ironwood Guide* assembles this tree in full (“The transaction digest (ZIP-244)”, within its binding-signature section); we do not re-derive it. Two consequences matter for the lifecycle. First, the tree is computed over *effecting* data only: changing the transaction’s consensus effect changes the identifier, while allowed recomputation of excluded attestations such as proofs or signatures leaves both the effect and identifier unchanged. This is ZIP-244’s non-malleability property. For v1–v4 the *txid* is instead the double-SHA-256 of the serialised transaction, which is malleable and therefore reliable only once the transaction is mined (`zcash_protocol`, `src/txid.rs`). Second, because the personalisation bakes in the branch id, the same transaction has a *different* *txid* on parallel consensus branches — replay protection across forks (ZIP-244, *Txid Digest*). For a purely shielded transaction the sighash coincides with the *txid*, as the *Ironwood Guide* already notes.

The header node of the tree is the one this section leans on:

$$d_{\text{hdr}} := \text{BLAKE2b-256}(\text{ZTxIdHeadersHash}, \text{LE}_{32}(\text{version}) \parallel \text{LE}_{32}(\text{versionGroupId}) \parallel \text{LE}_{32}(\text{branchid}) \parallel \text{LE}_{32}(\text{lockTime}) \parallel \text{LE}_{32}(\text{nExpiryHeight})),$$

with version including the overwinter flag (ZIP-244, node T.1). The field `nExpiryHeight` of §6.3 is therefore *effecting* data, committed by the *txid*: expiry cannot be malleated. Zebra’s mempool relies on exactly this, matching and rejecting expired transactions by their non-malleable *txid* (`zebrad`, `src/components/mempool/storage.rs`).

One convention trips up every first-time implementer: *txid* bytes are stored and transmitted in protocol order but *displayed* byte-reversed and hex-encoded. The `Display` implementation of `Txid` reverses before encoding, and the `zcash_client_sqlite` view documentation states the same rule for its stored byte vectors (`zcash_protocol`, `src/txid.rs`; `zcash_client_sqlite`, `src/wallet/db.rs`).

6.2 Store, then broadcast

The deployed pipeline persists a transaction *before* the network ever sees it. The function `create_proposed_transactions` (`zcash_client_backend`, `src/data_api/wallet.rs`) constructs, proves, and signs each step of a proposal (§6.6 fixes the target height it builds against), then persists the results via `WalletWrite::store_transactions_to_be_sent`, whose contract reads “This must be called before the transactions are sent to the network” (`zcash_client_backend`, `src/data_api.rs`). The ordering is deliberate: a transaction the network might mine but the wallet has forgotten is unrecoverable spend history, whereas a stored transaction the network never saw is merely stale, and stored transactions “should be retransmitted while it is still possible that they could be mined”

(zcash_client_backend, src/data_api.rs). In multi-step proposals (e.g. the ZIP-320 pairs of §6.5), storage happens only after *all* steps have been created, so that failure partway through creation cannot leave a transmittable prefix behind (zcash_client_backend, src/data_api/wallet.rs).

Persisting a created transaction does two things (zcash_client_sqlite, src/wallet.rs, store_transaction_to_be_sent). First, it records the transaction itself: txid, expiry height read from tx.expiry_height(), raw bytes, fee, target height, and min_observed_height := target height (Table 4). Second, it marks every spent input as spent — Sapling and Orchard nullifiers into the *_received_note_spends tables, transparent outpoints into transparent_received_output_spends — which locks those inputs against reselection by later proposals until the transaction is mined, or expires (§6.5).

Submission itself is one gRPC call: the lightwalletd CompactTxStreamer method SendTransaction(RawTransaction) returns (SendResponse), where errorCode = 0 means accepted and any other value carries an errorMessage (zcash_client_backend, proto/service.proto; zcash-devtool, src/commands/wallet/send.rs). Zaino implements the same CompactTxStreamer service as a drop-in replacement for lightwalletd, so submission is identical against either (zaino, README.md).

Remark 6.1 (The rebroadcast responsibility boundary). The librustzcash documentation records the obligation to retransmit — stored transactions “should be retransmitted while it is still possible that they could be mined” (zcash_client_backend, src/data_api.rs), and no-expiry transactions “should be rebroadcast by the wallet until they are either mined or conflict with another mined transaction” (zcash_client_sqlite, src/wallet.rs) — but implements no rebroadcast loop itself. The obligation falls on the embedding wallet application; the deployed CLI reference (zcash-devtool) submits exactly once.

6.3 Expiry (ZIP-203)

An unmined transaction cannot be left dangling forever: an indefinitely pending payment is indeterminism for the user and dead weight for every mempool. ZIP-203 (Status Final, Category Consensus) resolves this with a consensus-enforced expiry height.

Definition 6.2 (Transaction expiry, ZIP-203). Every transaction from Overwinter onward carries a field nExpiryHeight of type uint32 — always a block height, never a UNIX timestamp. A transaction t is *expired* at block height H iff

$$(t \text{ is not coinbase}) \wedge (t.\text{nExpiryHeight} \neq 0) \wedge (H > t.\text{nExpiryHeight}),$$

and the consensus rule states: if a transaction is not a coinbase transaction and its nExpiryHeight is nonzero, it **MUST NOT** be mined at a block height greater than its nExpiryHeight; a block containing an expired transaction is invalid (ZIP-203, *Specification*; protocol specification §7.1). Expiry at height N thus makes block N the last block that may include the transaction. The sentinel nExpiryHeight = 0 disables expiry, and for non-coinbase transactions nExpiryHeight ≤ 499999999 **MUST** hold. From NU5 the bound is removed for coinbase only, and the nExpiryHeight of a coinbase transaction **MUST** instead equal its block height — this keeps v5 coinbase txids unique, since the ZIP-244 txid does not commit to the coinbase script height (ZIP-203, *Changes for NU5*; protocol specification §7.1).

The deployed default: exactly 40 blocks. ZIP-203 sets the default expiry delta at 20 blocks before Blossom and, from Blossom activation, 40 blocks from the current height — about 50 minutes at the 75-second target spacing — and permits nodes to override it (zcashd’s -txexpirydelta option; ZIP-203, *RPC*, *Config*). The deployed light-wallet stack defaults to a stricter expiry than the ZIP requires.

The constructor `Builder::new` sets

$$n\text{ExpiryHeight} := \begin{cases} \text{targetHeight} & \text{coinbase,} \\ \text{targetHeight} + \text{DEFAULT_TX_EXPIRY_DELTA} & \text{otherwise,} \end{cases}$$

with `DEFAULT_TX_EXPIRY_DELTA = 40`, the coinbase rule applied regardless of network upgrade. A public override, `Builder::with_expiry_height`, replaces this: its documentation permits disabling expiry with height 0 but discourages it as not yet well-tested end-to-end, and coinbase builders reject an override that differs from the target height (`zcash_primitives, src/transaction/builder.rs`). A wallet that accepts the default therefore expires its transactions exactly 40 blocks after their target height. The rigidity is a feature: ZIP-315 (Draft; see Remark 6.6) says wallets **SHOULD** use the default 40-block expiry, and a wallet emitting idiosyncratic expiry deltas would fingerprint its transactions, exactly as an idiosyncratic fee would (§3).

Remark 6.3 (Documentation divergence: the builder’s stale delta). The doc comment on `Builder::new` states that the expiry height is set to the target “plus the default transaction expiry delta (20 blocks)”, while the constant it actually adds is `DEFAULT_TX_EXPIRY_DELTA = 40` (`zcash_primitives, src/transaction/builder.rs`). The code matches ZIP-203 post-Blossom; the comment preserves the pre-Blossom value. We follow the code.

Mempool eviction is node policy, mining is consensus. The consensus rule of Definition 6.2 constrains block *contents*; what nodes do with their mempools is policy layered on top. ZIP-203 specifies that when a transaction expires it is removed from nodes’ mempools, together with *all of its dependent transactions*. Zebra’s `remove_expired_transactions` evicts every mempool transaction once

$$\text{tipHeight} \geq t.n\text{ExpiryHeight},$$

i.e. as soon as the tip reaches the expiry height — at which point no future block can include t — and adds the evicted transaction’s non-malleable txid to its rejection list with `SameEffectsChainRejectionError::Expired` (`zebrad, src/components/mempool/storage.rs`). The two formulations agree operationally with ZIP-203’s “block N or earlier; block $N + 1$ will be too late”.

The expiring-soon door policy. The `zcashd` node additionally refuses (as denial-of-service mitigation, not consensus) to accept into its mempool, or relay, a transaction that is *expiring soon*: one that would be expired within `TX_EXPIRING_SOON_THRESHOLD = 3` blocks of the next block height, where $\text{IsExpiredTx}(t, h) = (t.n\text{ExpiryHeight} \neq 0 \wedge t \text{ not coinbase} \wedge h > t.n\text{ExpiryHeight})$ (`zcashd, src/main.h, src/main.cpp`). The practical consequence for wallets: a transaction submitted with fewer than about four blocks of remaining life is refused at the mempool door. Zebra applies no such buffer: its only admission-time expiry check is the consensus rule itself, evaluated at the next block height. The mempool hands each candidate to the transaction verifier with height `tipHeight + 1` (`zebrad, src/components/mempool/downloads.rs`), and the verifier’s `non_coinbase_expiry_height` rejects only when that height exceeds $t.n\text{ExpiryHeight}$ (`zebra-consensus, src/transaction/check.rs`); no expiring-soon threshold appears anywhere in the mempool component. A transaction with a single block of remaining life is therefore accepted and relayed by Zebra but refused by `zcashd`.

Expiry never straddles a network upgrade. The `zcashd` node clamps a new transaction’s `nExpiryHeight` to one less than the next network-upgrade activation height (`zcashd, src/main.cpp`), so a transaction never straddles an upgrade boundary. The `librustzcash` parser *relies* on this invariant when parsing transactions it holds in unmined form: since “a transaction can’t be mined across a network upgrade boundary, ... the expiry height must be in the same epoch”, an unmined transaction with nonzero expiry is parsed under `BranchId::for_height(nExpiryHeight)`; an unmined v4 transaction with expiry 0 carries no epoch hint at all and cannot be parsed until mined (`zcash_client_sqlite, src/wallet.rs, parse_tx`).

6.4 Tracking a pending transaction

Between submission and its terminal state, a transaction exists only as a row in the wallet database and an entry in remote mempools. The wallet-side record is the `zcash_client_sqlite` `transactions` table (Table 4); note that it may hold data unrecoverable from the chain, e.g. wallet-created transactions that expired unmined (`zcash_client_sqlite`, `src/wallet/db.rs`).

Column	Semantics
<code>txid</code>	transaction identifier, protocol byte order (UNIQUE; reverse and hex-encode for display)
<code>created</code>	wallet-local creation time
<code>block</code>	height of the containing block, set only once that block has been <i>scanned</i> (foreign key to <code>blocks(height)</code>)
<code>mined_height</code>	mined height, settable from a status query without scanning the block; CHECK equal to <code>block</code> when <code>block</code> is non-null
<code>tx_index</code>	index of the transaction within its block
<code>expiry_height</code>	maximum height at which the transaction may be mined, if known
<code>raw</code>	serialised transaction bytes, if retrieved
<code>fee</code>	fee paid, if known
<code>target_height</code>	construction target height; set only for transactions this wallet created
<code>min_observed_height</code>	NOT NULL; minimum of the mempool-observation height and the mined height
<code>confirmed_unmined_at_height</code>	maximum height with positive proof of unminedness; CHECK NULL when <code>mined_height</code> is non-null
<code>trust_status</code>	user-assigned trust flag (§6.6)

Table 4: Columns of the `transactions` table (`zcash_client_sqlite`, `src/wallet/db.rs`).

Statuses and how they are learned. The backend distinguishes three externally observable states, the enum `TransactionStatus`: `TxidNotRecognized`, `NotInMainChain` (in the mempool, or mined only on a reorged-away fork), and `Mined(height)` (`zcash_client_backend`, `src/data_api.rs`). Statuses arrive by evaluating `TransactionDataRequest` values the wallet emits: `GetStatus(txid)`; `Enhancement(txid)`, which downloads the full raw transaction and subsumes `GetStatus`; and, behind the transparent-inputs feature, `TransactionsInvolvingAddress`. These are typically served by `lightwalletd`’s `GetTransaction` and `GetAddressTxids` RPCs and fed back via `WalletWrite::set_transaction_status` or `decrypt_and_store_transaction` (`zcash_client_backend`, `src/data_api.rs`). Active polling is not optional even for a shielded wallet’s own history: fully transparent transactions, and transactions containing neither shielded inputs nor shielded outputs belonging to the wallet, are invisible to the compact-block trial decryption of the *Ironwood Guide* (“Transmission and detection of a note”), so scanning alone can never confirm their fate (`zcash_client_backend`, `src/data_api.rs`).

The wire encoding of a status reply hides a protobuf quirk. The `RawTransaction.height` field signals: 0 = in the mempool; `0xffffffffffffffff` = mined on a fork that is not the main chain; any other value = mined height in the main chain (`zcash_client_backend`, `proto/service.proto`). The deployed client maps

```

gRPC NotFound    ↦ TxidNotRecognized,
height ∉ [1, 232 - 1] ↦ NotInMainChain,
otherwise        ↦ Mined(height)

```

(`zcash-devtool`, `src/commands/wallet/enhance.rs`).

Applying a status. On `TxidNotRecognized` or `NotInMainChain` for an unmined transaction, `set_transaction_status` records positive proof of unminedness, setting `confirmed_unmined_`

at_height to the current chain tip. On Mined(h) it sets mined_height to h , lowers min_observed_height to at most h , clears confirmed_unmined_at_height, links block if that block is already scanned, and advances the window of unused transparent address indices the wallet derives and watches past the highest index observed used; in every case the pending tx_retrieval_queue entry is deleted (zcash_client_sqlite, src/wallet.rs). Independently, when chain scanning itself encounters a wallet-relevant transaction in a block, put_tx_meta upserts block and mined_height to the scanned height along with tx_index, takes the minimum for min_observed_height, and clears confirmed_unmined_at_height (zcash_client_sqlite, src/wallet.rs).

When to stop asking. A GetStatus request is (re)generated for every transaction with mined_height NULL while any of the following holds (zcash_client_sqlite, src/wallet.rs, transaction_data_requests):

```
confirmed_unmined_at_height IS NULL
  ∨ (expiry_height > 0 ∧ confirmed_unmined_at_height < expiry_height)
  ∨ (expiry_height IS NULL
      ∧ confirmed_unmined_at_height < min_observed_height + certaintyDepth),
```

where certaintyDepth := PRUNING_DEPTH + DEFAULT_TX_EXPIRY_DELTA = 100 + 40 = 140 blocks, and the result is unioned with explicit tx_retrieval_queue entries. A transaction with known nonzero expiry is thus polled until unminedness is confirmed at or beyond its expiry height; one with unknown expiry until 140 blocks past first observation; and one with expiry 0 only until unminedness is first confirmed — only the first disjunct can hold for it, so a single TxidNotRecognized or NotInMainChain reply ends the polling. The in-code comment above the query promises to “continue to query indefinitely” for such transactions and expects them “rebroadcast by the wallet until they are either mined or conflict with another mined transaction” (Remark 6.1); the SQL does not implement that promise — a documentation divergence. For display, the v_transactions view exposes the terminal flag

```
expired_unmined := mined_height IS NULL ∧ 1 ≤ expiry_height ≤ max(blocks.height)
```

(zcash_client_sqlite, src/wallet/db.rs), i.e. nonzero expiry at or below the maximum scanned height.

Remark 6.4 (Documentation divergence: the retrieval-queue codes). The table documentation for tx_retrieval_queue says query_type is “0 for raw transaction (enhancement) data, 1 for transaction mined-ness information” (zcash_client_sqlite, src/wallet/db.rs), but the authoritative encoder TxQueryType::code() maps Status ↦ 0 and Enhancement ↦ 1 and round-trips through from_code (zcash_client_sqlite, src/wallet.rs). The documentation is inverted relative to the code; the code governs.

6.5 Expiry and fund availability

Storing a transaction locked its inputs (§6.2); expiry is what unlocks them. Note selection excludes a note appearing in *_received_note_spends only when the spending transaction is still *unexpired* in the following sense, evaluated at the target height H of the new proposal (zcash_client_sqlite, src/wallet/common.rs, tx_unexpired_condition):

mined_height < H	(mined)	
∨ expiry_height = 0	(never expires)	
∨ expiry_height ≥ H	(unexpired)	(1)
∨ (expiry_height IS NULL ∧ min_observed_height + 40 ≥ H)	(default-delta guess).	

A transaction that has expired unmined (mined_height NULL, $0 < \text{expiry_height} < H$) fails every disjunct, so its inputs re-enter the spendable set for target heights beyond the expiry height. ZIP-315’s known-spendable definition matches: a TXO must not be “committed to be spent in another unexpired transaction created by this wallet” (ZIP-315, Draft).

Remark 6.5 (The unknown-expiry guess, and an inverted comment). The final disjunct of (1) is a guess that the originating wallet used the default delta. Reading the predicate directly: if the sender in fact used a *larger* delta or disabled expiry, the transaction is treated as *expired* from height `min_observed_height + 41` onward while it is actually still mineable — its inputs unlock prematurely, and a respend can conflict with a late mining of the original. If the sender used a *smaller* delta, the inputs merely stay locked slightly longer than necessary. Either misclassification persists only until the wallet observes the transaction mined or enhances it to learn the true expiry height. The doc comment on `tx_unexpired_condition` describes the first case as “treated as unexpired when it shouldn’t be” (`zcash_client_sqlite, src/wallet/common.rs`), which is inverted relative to its own SQL; we state the direction the predicate implies and flag the comment rather than silently following either.

Nullifiers outlive the lock. Dropping a pending spend’s inputs from the *spendable* set is not the same as forgetting them. The query `get_nullifiers(NullifierQuery::Unspent)` deliberately over-selects: a note spent in an unexpired-but-unmined transaction still contributes its nullifier, so that scanning (the *Ironwood Guide*, “Transmission and detection of a note”) detects the spend if the transaction is later mined; a note’s nullifier leaves the watch set only once the spending transaction is mined or can never expire (`zcash_client_sqlite, src/wallet/common.rs`).

Aftermath of an expiry. ZIP-203 (*Wallet behavior and UI*) draws the user-facing conclusions: an expiring zero-confirmation transaction has even less inclusion guarantee, so UIs and services must never rely on zero-conf transactions in Zcash, and wallets should notify the user of expired transactions that must be re-sent. ZIP-315 (Draft) adds that sent transactions SHOULD be reported with confirmations and, while unmined with an expiry height, an estimate of time until expiry. Re-sending carries a privacy cost the wallet should recognise: ZIP-315 identifies repeated spends of the same TXOs across transactions — the typical aftermath of re-sending an expired payment — as an information leak linking the invalidated transaction to its replacement. For ZIP-320 (TEX) payments the deployed convention is to set the *same* expiry height on the first (shielded-to-ephemeral) and second (ephemeral-to-TEX) transactions, so another wallet on the same seed can infer when the second would have expired without observing it; if the second expires unmined, the ephemeral UTXO should be re-shielded before being re-spent (`zcash_client_backend, src/data_api.rs`).

6.6 Confirmations, trust, and spendability

Mining is not the end of caution: a freshly mined output can be un-mined by a reorganisation (§6.7), so the wallet imposes a confirmation policy before treating outputs as spendable.

Remark 6.6 (ZIP-315 is a Draft). ZIP-315 (“Best Practices for Wallet Implementations”) has Status Draft. The deployed stack nonetheless implements its central recommendations — the 3/10 confirmation split, zero-conf shielding, the fixed anchor depth, the 40-block expiry — so throughout this section deployed behaviour cites the crates, and ZIP-315 is cited as the (draft) rationale, not as a finalised standard.

Definition 6.7 (Confirmations policy). A `ConfirmationsPolicy` holds two `NonZeroU32` counts, c_{tr} (*trusted*) and c_{untr} (*untrusted*), with the constructor enforcing $c_{\text{tr}} \leq c_{\text{untr}}$, plus a flag `allow_zero_conf_shielding`. The default is $c_{\text{tr}} = 3$, $c_{\text{untr}} = 10$, with zero-conf shielding allowed, per ZIP-315; the constant `ConfirmationsPolicy::MIN` is `1/1/true`. Confirmations are counted since *and including* the block in which a note was produced (`zcash_client_backend, src/data_api/wallet.rs`).

The trusted/untrusted split is ZIP-315’s: a *trusted* TXO is one received from a party where the wallet trusts it will remain mined in its original transaction — e.g. created by the wallet’s own internal

TXO handling — and everything else is untrusted. The recommended counts rest on two observations: reorganisations are anecdotally shorter than 3 blocks, and value from a trusted TXO is always recoverable after a rollback (the wallet can just re-create the transaction), whereas an untrusted TXO may require asking the sender to re-send (ZIP-315, Draft). Users may also explicitly mark specific transactions trusted via `WalletWrite::set_tx_trust`, persisted in the `trust_status` column — ZIP-315’s “MAY enable users to mark specific external transactions as trusted” provision (`zcash_client_backend`, `src/data_api.rs`; `zcash_client_sqlite`, `src/wallet.rs`).

Definition 6.8 (Confirmations until spendable). Fix a target height H and write $x \ominus y$ for saturating subtraction (`BlockHeight` subtraction in `librustzcash` yields a `u32` via `saturating_sub`; `zcash_protocol`, `src/consensus.rs`). Let $h_{\text{tr}} := H \ominus c_{\text{tr}}$ and $h_{\text{untr}} := H \ominus c_{\text{untr}}$, and for an output of a transaction with mined height m (if any) let

$$\text{confs}_{\text{tr}} := \begin{cases} c_{\text{tr}} & m \text{ unknown} \\ m \ominus h_{\text{tr}} & \text{else,} \end{cases} \quad \text{confs}_{\text{untr}} := \begin{cases} c_{\text{untr}} & m \text{ unknown} \\ m \ominus h_{\text{untr}} & \text{else.} \end{cases}$$

Then `confirmations_until_spendable` returns, and the output is spendable iff the result is 0 (`zcash_client_backend`, `src/data_api/wallet.rs`):

- *transparent*: 0 if `allow_zero_conf_shielding`; else confs_{tr} if the transaction is trusted or the receiving key scope is internal; else $\text{confs}_{\text{untr}}$;
- *shielded*: confs_{tr} if the transaction is trusted; else, for internal scope, if the maximum mined height h_s of the transparent inputs to the shielding transaction is known, $h_s \ominus h_{\text{tr}}$ when those inputs are trusted and $h_s \ominus h_{\text{untr}}$ otherwise, and confs_{tr} when h_s is unknown; else $\text{confs}_{\text{untr}}$.

The saturation is what makes the count “decrease to a floor of zero” for long-mined outputs.

The internal-scope shielded branch encodes ZIP-315’s rule that shielded funds produced by zero-conf shielding are unavailable until the *source* UTXOs have sufficient confirmations: spendability of a shielding output is computed from $h_s = \text{max_shielding_input_height}$ and the inputs’ trust status, not from the shielding transaction’s own mined height (`zcash_client_backend`, `src/data_api/wallet.rs`).

Remark 6.9 (Deployed shielding rule vs. the ZIP-315 text). The ZIP-315 text requires both that the shielding transaction have at least the trusted confirmation count *and* that its transparent inputs have at least $c_{\text{untr}} - c_{\text{tr}}$ confirmations. The deployed rule of Definition 6.8 tests only the input heights against the trusted/untrusted thresholds and imposes no separate requirement on the shielding transaction’s own confirmations. The two rules do not coincide. Count confirmations as in Definition 6.8, write h_s for the maximum input height and $h_m \geq h_s$ for the height at which the shielding transaction was mined, and let $d := h_m - h_s$. The ZIP composite admits the output at target heights $H \geq h_s + \max(c_{\text{untr}} - c_{\text{tr}}, c_{\text{tr}} + d)$, the deployed rule (inputs untrusted) at $H \geq h_s + c_{\text{untr}}$; the thresholds agree only at $d = c_{\text{untr}} - c_{\text{tr}}$. Below that point — the common case of a promptly mined shielding transaction — the deployed rule is strictly stricter: at the default policy with $d = 1$, the ZIP releases the output at $H = h_s + 7$ while the code waits until $h_s + 10$. Above it, the deployed rule is strictly laxer: with $d = 9$ the code releases at $h_s + 10$, when the shielding transaction has a single confirmation, while the ZIP waits until $h_s + 12$. The deployed trusted-input branch ($H \geq h_s + c_{\text{tr}}$) and the unknown- h_s fallback have no ZIP counterpart at all. The deployed rule is thus implementation policy diverging from the (Draft) ZIP text — conservative precisely when the shielding transaction confirms within $c_{\text{untr}} - c_{\text{tr}}$ blocks of its inputs.

The full spendability gate. Deployed note selection admits a shielded note as spendable only when all of the following hold (`zcash_client_sqlite`, `src/wallet/common.rs`): the note’s transaction

is in a *scanned* block (`t.block` non-null); its nullifier and commitment-tree position are known and the containing shard is fully scanned (witness data exists — the *Ironwood Guide*, “Proof of ownership of *some* valid note: the commitment tree”); its mined height is at or below the anchor height; `confirmations_until_spendable = 0` under the policy; and it is not excluded by the spent-notes clause — spent in a mined or unexpired pending transaction, condition (1).

Target and anchor heights. The heights every predicate above is evaluated against are fixed at proposal time (`zcash_client_sqlite`, `src/wallet.rs` and `src/wallet/commitment_tree.rs`; `zcash_client_backend`, `src/data_api/wallet.rs`):

```
chainTip := max(scan_queue.block_range_end) - 1  (ranges are end-exclusive),
H := chainTip + 1  (the “mempool height”),
anchorHeightpool := max{ checkpoint : checkpoint ≤ H - minConf },
anchor := minpools with checkpoints anchorHeightpool  (Sapling, Orchard),
```

and `propose_transfer` uses the trusted count, `minConf = ctr` (default 3); the target height that `create_proposed_transactions` hands to `Builder::new` is the proposal’s `min_target_height()`. The anchor depth therefore *equals* the trusted confirmation count — a fixed depth for all transactions, so anchor choice does not leak whether the notes being spent were trusted, the property ZIP-315’s anchor-selection recommendation secures. The exact heights differ by one, however: ZIP-315 measures from the chain head, recommending the final treestate of block `chainTip - 3`, whereas the deployed anchor sits at `H - ctr = chainTip - 2`, one block shallower. The tree and anchor mechanics themselves are the *Ironwood Guide*’s (its commitment-tree section); the wallet-layer content is only this choice of depth.

6.7 Reorganisations: truncate and rescan

A reorganisation invalidates the wallet’s suffix of the chain: mined transactions become unmined, anchors vanish, and witness data beyond the fork point is wrong. The deployed stack detects this during scanning and repairs it by truncation.

Detection. Scanning reports continuity errors: the `ScanError` variants `PrevHashMismatch` (the parent hash of a new block does not match the current tip), `BlockHeightDiscontinuity`, and `TreeSizeMismatch` all satisfy `is_continuity_error() = true` (`zcash_client_backend`, `src/scanning.rs`). To surface reorganisations *promptly* rather than on the next unlucky scan, `update_chain_tip` marks the `VERIFY_LOOKAHEAD = 10` blocks above the maximum scanned height with `ScanPriority::Verify` whenever that height is at or below the stable height `tip - PRUNING_DEPTH`; ranges marked `Verify` must always be scanned before `ChainTip` ranges. The choice of 10 blocks corresponds to roughly 12.5 minutes, within which a user plausibly received a transaction without their wallet having scanned the block (`zcash_client_sqlite`, `src/wallet/scanning.rs`, `src/lib.rs`).

Recovery loop. On a continuity error *e*, the documented algorithm is (`zcash_client_backend`, `src/data_api/chain.rs`):

1. choose a rewind height at least one block before the error height (the module docs’ heuristic: `e.at_height() - 10`);
2. call `WalletWrite::truncate_to_height`, which may truncate *lower* than requested (below);
3. delete cached `CompactBlocks` above the returned truncation height, re-download, and rescan — `Verify` ranges first.

The truncation target is constrained by checkpoint availability: `truncate_to_height` truncates to the greatest height at most the requested one that has a note-commitment-tree checkpoint in *every* tree containing data, and errors with `RequestedRewindInvalid` — carrying the safe rewind height, the maximum over trees of their minimum checkpoint heights — if none exists. The bound `PRUNING_DEPTH = 100` is the maximum number of blocks the wallet is allowed to rewind, consistent with the bound in `zcashd`; the variant `truncate_to_chain_state` additionally permits precise truncation from provided `ChainState` frontiers even when the target checkpoint was pruned (`zcash_client_sqlite`, `src/wallet.rs`, `src/lib.rs`; `zcash_client_backend`, `src/data_api.rs`).

What truncation does. Truncating to height h (`zcash_client_sqlite`, `src/wallet.rs`):

- clamps the scan queue so the wallet’s chain-tip view returns to h ;
- resets or nulls transparent outputs’ `max_observed_unspent_height` — a spending transaction may be entirely invalidated by a reorganisation that alters the anchors its shielded spends were built against;
- *un-mines* every transaction with `mined_height > h`: `block`, `mined_height`, `tx_index`, and `confirmed_unmined_at_height` are all set `NULL`, so the transaction reverts to the pending state of §6.4 and will either be re-observed as mined or expire;
- truncates both shard trees to the checkpoint at h (tree and anchor mechanics: the *Ironwood Guide*, its commitment-tree section);
- deletes blocks above h and stale `tx_locator_map` entries.

Sent and received notes are deliberately *not* deleted: they may hold data, such as memos, unrecoverable from the chain. Balance computations must instead simply not count received notes whose transactions are no longer mined — which the spendability gate of §6.6 already guarantees, since an un-mined transaction has no scanned block and no qualifying mined height.

Terminal states. Every wallet-created transaction thus ends in exactly one of three states: *mined* with enough confirmations that truncation can no longer reach it (`PRUNING_DEPTH = 100` blocks); *expired unmined*, its inputs released (§6.5) and its record retained (Table 4) with the user notified per ZIP-203; or, for the expiry-0 transactions a caller must explicitly request via `with_expiry_height`, *pending*, with polling ending at the first confirmed-unmined reply (§6.4) and any rebroadcast falling on the embedding wallet (Remark 6.1).

7 Payment requests (ZIP-321)

The *Ironwood Guide* constructs everything a wallet needs once the sender knows where to pay: note encryption and detection, authentication paths, anchor selection, and the ZIP-244 transaction digest. What none of that settles is the step before any of it — how the payee communicates to the payer an address, an amount, and a memo, in a form a wallet can consume mechanically. ZIP-321 (“Payment Request URIs”) standardises that step as a URI scheme; it is Status Active, Category Standards/Wallet, created 2020-08-28, owned by Kris Nuttycombe and Daira-Emma Hopwood (ZIP-321, header). This section develops the format, its deployed parser — the `zip321` crate in `librustzcash` — and the pipeline that turns a parsed request into a transaction proposal and finally a transaction.

7.1 Requests, payments, and the single-transaction rule

Definition 7.1 (Payment; payment request). A *payment* is a transfer of funds implemented by a single shielded or transparent output of a Zcash transaction. A *payment request* is a request for a wallet to construct a single Zcash transaction containing one or more payments (ZIP-321, Terminology).

A ZIP-321 URI therefore represents a request for the construction of *one* transaction; implementations SHOULD construct a single transaction paying all the specified addresses (ZIP-321, URI Semantics). The number of payments one transaction can carry is limited only by transaction-construction restrictions: the ZIP notes that if every payment were Sapling, the value-balance limit of the protocol specification, §4.13 (Balance and Binding Signature, Sapling), implies construction may fail above 2109 distinct payments, and that the effective limit may be lower when Orchard or future address types are involved. Recomputed from the specification, the bound is a transaction-size cap used inside that section’s overflow argument, not a direct value-range consequence: each Sapling output contributes 948 bytes to the transaction — an `OutputDescriptionV4` comprises the 32-byte `cv`, `cmu`, and `ephemeralKey` fields, a 580-byte `encCiphertext`, an 80-byte `outCiphertext`, and a 192-byte proof; an `OutputDescriptionV5` is 756 bytes plus its 192-byte entry in `vOutputProofsSapling` — and a transaction must fit within the specification’s 2 MB maximum transaction size (numerically equal to the 2 000 000-byte block-size limit), so at most $\lfloor 2\,000\,000/948 \rfloor = 2109$ outputs fit (protocol specification, §4.13; field sizes from §7.1, “Transaction Encoding and Consensus”). The specification uses the cap to show that the balance sum cannot overflow the type of committed values; ZIP-321 reuses it as the payment-count ceiling.

7.2 URI syntax

Definition 7.2 (ZIP-321 URI grammar). A ZIP-321 URI is a string derived by `zcashurn` in the following ABNF (RFC 5234), reproduced from ZIP-321 (URI Syntax); the productions `ALPHA`, `unreserved`, and `pct-encoded` are those of RFC 3986, and `base64url` is described below.

```

zcashurn      = "zcash:" ( zcashaddress [ "?" zcashparams ] / "?" zcashparams )
zcashaddress  = 1*( ALPHA / DIGIT )
zcashparams   = zcashparam [ "&" zcashparams ]
zcashparam    = [ addrparam / amountparam / assetparam / memoparam
                  / messageparam / labelparam / otherreqparam / otherparam ]
NONZERO       = %x31-39
DIGIT         = %x30-39
paramindex    = "." NONZERO 0*3DIGIT
addrparam     = "address" [ paramindex ] "=" zcashaddress
amountparam   = "amount" [ paramindex ] "=" 1*DIGIT [ "." 1*8DIGIT ]
assetparam    = "req-asset" [ paramindex ] "=" *base64url
labelparam    = "label" [ paramindex ] "=" *qchar
memoparam     = "memo" [ paramindex ] "=" *base64url
messageparam  = "message" [ paramindex ] "=" *qchar
paramname     = ALPHA *( ALPHA / DIGIT / "+" / "-" )
otherreqparam = "req-" paramname [ paramindex ] [ "=" *qchar ]
otherparam    = paramname [ paramindex ] [ "=" *qchar ]
qchar         = unreserved / pct-encoded / allowed-delims / ":" / "@"
allowed-delims = "!" / "$" / "'" / "(" / ")" / "*" / "+" / "," / ";"

```

Three global properties of the grammar deserve emphasis before the per-parameter semantics.

No authority component. A ZIP-321 URI is not a hierarchical URI: the grammar cannot derive `//` after the scheme, so there is no authority component (ZIP-321, URI Syntax note). For a single payment the recipient address SHOULD be placed directly in the hier-part, and a URI `zcash:<address>?... MUST be treated as equivalent to zcash:?address=<address>&... (ZIP-321, URI Semantics); the query-parameter form with addrparam and distinct indices serves multiple recipients.`

Payment grouping by paramindex. Addresses, amounts, labels, and messages sharing the same `paramindex` — including the empty index — belong to the same payment. An index **MUST NOT** have leading zeros, parameter ordering is insignificant, and index values need not be sequential (ZIP-321, URI Semantics). The grammar bounds indices to `"." NONZERO 0*3DIGIT`, that is $1 \leq i \leq 9999$; the empty index denotes a distinguished payment which the deployed crate stores at map key 0. Rendering and parsing are exact inverses on this range: index 0 renders as the empty suffix, and index $i \geq 1$ renders as `." || dec(i)`; on parse, an absent index maps to key 0, and the leading-zero strings `.0` and `.01` match no production and invalidate the URI (`zip321, src/lib.rs, param_index` and `indexed_name`; ZIP-321, Invalid Examples).

Percent encoding is confined to qchar. Only the values of `label`, `message`, `otherreqparam`, and `otherparam` — the `qchar` productions — may contain pct-encoded sequences. Addresses, amounts, and parameter *names* must appear literally: the ZIP's invalid examples include `zcash:taddr?amount=1%30` (encoded amount digit), `zcash:taddr?%61mount=1` (encoded parameter name), and `zcash:%74addr...` (encoded address) (ZIP-321, URI Syntax note and Invalid Examples).

The base64url alphabet. Productions `memoparam` and `assetparam` carry binary content encoded as `base64url`, RFC 4648 §5, with padding omitted: values 62 and 63 encode as `-` and `_`, and implementations **MUST NOT** accept the standard-base64-only characters `+`, `/`, `=` (ZIP-321, URI Syntax). The deployed crate decodes with the `BASE64_URL_SAFE_NO_PAD` engine (`zip321, src/lib.rs`).

7.3 Validity and address semantics

The ZIP imposes structural validity rules beyond the grammar (ZIP-321, URI Syntax and URI Semantics):

- a purported ZIP-321 URI that cannot be parsed according to the grammar **MUST NOT** be accepted;
- if any non-address parameter carries a given `paramindex`, the URI **MUST** contain an address parameter with that index; and
- there **MUST NOT** be more than one occurrence of a given parameter name at a given index.

Implementations **SHOULD** check that each `zcashaddress` is a valid encoding per the protocol specification, §5.6, other than a Sprout address: a transparent address (Base58Check, §5.6.1.1), a Sapling payment address (Bech32 per ZIP-173, §5.6.3.1), or a Unified Address (Bech32m per BIP 350, §5.6.4.1 and ZIP-316). New address formats **SHOULD** be supported whether or not ZIP-321 is updated for them; if network context is available, each address **SHOULD** be checked for the correct network; and every requirement of ZIP-316 applies to payments to Unified Addresses (ZIP-321, URI Semantics).

Sprout addresses **MUST NOT** be supported in payment requests. The rationale is economic rather than syntactic: since Canopy, ZIP-211 restricts adding value to the Sprout pool, so senders cannot generally satisfy a request to pay a Sprout address (ZIP-321, URI Semantics). Section 7.9 flags where the deployed stack enforces this.

Forward compatibility. A parameter name prefixed `req-` that the parser does not recognise renders the *entire* URI invalid (**MUST**); unrecognised parameters without the prefix **SHOULD** be ignored. The prefix is potentially part of a future parameter's name, not a modifier applicable to arbitrary parameters, and the original parameters `address`, `amount`, `label`, `memo`, `message` never carry it, because every conformant parser is required to understand them (ZIP-321, Forward compatibility).

Backward compatibility. Implementations of the informal pre-ZIP-321 `zcash:` URI scheme generally lacked the `req=` rule, the `paramindex` multi-payment facility, the `address=` query form for single payments, and the `base64url` memo treatment (ZIP-321, Backward compatibility).

7.4 Amounts

The amount parameter is a decimal number of ZEC. If a fraction is present, the separator **MUST** be a period and both the whole and fractional parts **MUST** be nonempty; grouping separators are forbidden; leading zeros in the whole part and trailing zeros in the fraction are ignored; the fraction carries at most 8 digits; and the amount **MUST NOT** exceed 21000000 ZEC (ZIP-321, ZEC Transfer Amount). Thus 50, 050, and 50.00 all denote 50 ZEC; 0.5 and 00.500 denote 0.5 ZEC; and 50,000.00, 50., .5, and 0.123456789 are invalid.

Construction 7.3 (Amount parsing and rendering). Fix $\text{COIN} := 10^8$ zatoshis per ZEC and $\text{MAX_MONEY} := 21\,000\,000 \cdot \text{COIN} = 2\,100\,000\,000\,000\,000$ zatoshis (`zcash_protocol, src/value.rs`).

Parse. Given a string matching `1*DIGIT [ "." 1*8DIGIT ]`, let c be the whole part read as an unsigned 64-bit integer, and let $z := \text{int}(\text{rpad}_8(f, 0))$, the fraction f right-padded with 0 to 8 digits ($z := 0$ when the fraction is absent). The value is

$$v := c \cdot \text{COIN} + z$$

with checked (non-wrapping) multiplication and addition, and the parse succeeds iff $0 \leq v \leq \text{MAX_MONEY}$ (the Zatoshis range check) (`zip321, src/lib.rs, parse_amount; zcash_protocol, Zatoshis::from_u64`).

Render. Let $c := \lfloor v/\text{COIN} \rfloor$ and $z := v \bmod \text{COIN}$. Emit `dec(c)`; if $z \neq 0$, append `.` followed by `dec(z)` left-padded with 0 to 8 digits and with trailing 0 characters trimmed (`zip321, src/lib.rs, render::amount_str`).

7.5 Memos, labels, and messages

The memo parameter carries the content of the shielded memo field — the fixed 512-byte field of the note plaintext constructed in the *Ironwood Guide* (note encryption) — as `base64url` without padding. The decoded content **MUST NOT** exceed 512 bytes and, if shorter, is filled with trailing zeros to 512 bytes. A memo whose same-index address does not permit memos — a transparent address — makes the *entire* URI invalid (**MUST**) (ZIP-321, Query Keys).

Construction 7.4 (Memo encoding and decoding). Write `b64u(·)` for RFC 4648 §5 encoding with padding omitted.

Encode. For a 512-byte memo m , the parameter value is `b64u(strip0x00( $m$ ))`, where `strip0x00` removes trailing 0x00 bytes (`MemoBytes::as_slice`) (`zip321, src/lib.rs, memo_to_base64`).

Decode. Let $b := \text{b64u}^{-1}(\text{value})$, rejecting any input containing `+`, `/`, or `=`; require $|b| \leq 512$; then

$$m := b \parallel 0x00^{512-|b|}$$

(`MemoBytes::from_bytes zero-pads`) (`zip321, src/lib.rs, memo_from_base64; zcash_protocol, src/memo.rs`).

An absent memo parameter yields no memo; at output construction the wallet then substitutes the “no memo” sentinel `MemoBytes::empty() = 0xF6 \parallel 0x00511` (`zcash_protocol, src/memo.rs; zcash_client_backend, src/data_api/wallet.rs`).

The label parameter names an address (for example, the receiver’s name): a client **SHOULD**

display it alongside the address, and it **MUST** be identifiable as distinct from the address itself. The message parameter is descriptive text the client may display to describe the payment (ZIP-321, Query Keys). Both are `qchar` values, so arbitrary UTF-8 reaches them only through percent encoding (§7.2).

7.6 Custom assets: `req-asset` (specified, not deployed)

The `req-asset` parameter extends payment requests to ZSA Custom Assets (ZIP-227), encoding an Asset Identifier and an amount in a single value; absent `req-asset`, the payment is for ZEC. A URI containing both amount and `req-asset` at the same index **MUST** be rejected. Only Orchard receivers are valid for Custom Asset payments, so a Custom-Asset request **MUST** use a Unified Address encoding at least one Orchard receiver (ZIP-321, Custom Assets).

Definition 7.5 (`req-asset` value encoding). The parameter value is

$$\text{b64u}(0x00 \parallel \text{I2LEOSP}_{64}(\text{value}) \parallel \text{EncodeAssetId}(\text{AssetId})) :$$

a version byte `0x00`, the amount as an unsigned 64-bit little-endian integer of the asset's minimal units, and the 66-byte encoded Asset Identifier of ZIP-227 — a 75-byte payload, hence 100 `base64url` characters. Future versions may define different structures; implementations **MUST** dispatch on the version byte (ZIP-321, Custom Assets).

Classification per our conventions: *specified*. The parameter appears in the Active ZIP-321 text, but it depends on ZIP-227, which is Status Draft, and the deployed parser rejects every unrecognised `req-` parameter — including `req-asset` — so a Custom-Asset request is invalid to the deployed stack today (`zip321`, `src/lib.rs`, `to_indexed_param`).

7.7 The deployed parser: the `zip321` crate

The reference implementation is the `zip321` crate in `librustzcash` (`components/zip321`, a single file `src/lib.rs`), published at version 0.9.0; release 0.1.0 (2024-08-20) factored it out of `zcash_client_backend` 0.13.0, which still re-exports it as a deprecated module `zcash_client_backend::zip321` (`components/zip321/Cargo.toml` and `CHANGELOG.md`; `zcash_client_backend`, `src/lib.rs`). The Guide's baseline is the local checkout — `librustzcash` main at commit `e30517e4` — as for every other `librustzcash` citation in this volume, where the crate was 0.8.0. The optional-amount semantics shipped in release 0.7.0 (2026-04-23); current 0.9.0 changes dependencies and fixes the `test-dependencies` feature, not the parser semantics documented below.

Definition 7.6 (Types `Payment` and `TransactionRequest`). Type `Payment` holds a recipient address (`zcash_address::ZcashAddress`), an optional amount (`Option<Zatoshis>`), an optional memo (`MemoBytes`), an optional label and message, and a list `other_params` of retained unrecognised parameters. An absent amount means the *sender* chooses the amount — semantics introduced in release 0.7.0 (2026-04-23); releases up to 0.6.0 instead defaulted the field to zero, a behaviour ZIP-321 leaves unspecified (`zip321`, `src/lib.rs`; `CHANGELOG.md`, 0.7.0 entry).

Type `TransactionRequest` wraps a `BTreeMap<usize, Payment>`, map key 0 denoting the empty paramindex. Constructor `new` rejects more than 9999 payments (`Zip321Error::TooManyPayments`) and validates every non-empty request by round-tripping it through `to_uri/from_uri`; constructor `from_indexed` rejects any index key above 9999 (`zip321`, `src/lib.rs`).

Construction 7.7 (Parsing: `TransactionRequest::from_uri`). Parsing proceeds in four stages (`zip321, src/lib.rs: from_uri, lead_addr, zcashparam, to_indexed_param, to_payment`).

- (1) Match the literal `zcash:` (lowercase; nom tag), then take the characters up to `?` as the hier-part address. If nonempty, it must decode as a `ZcashAddress` and is recorded as an address parameter at payment index 0.
- (2) If input remains, require `?` followed by a list, separated by `&` and consuming the rest of the input, of `name[.index]=value` pairs, each mapped to a typed parameter (address, amount, label, message, memo, or other); any unrecognised req- name is a hard error (“Required parameter {name} not recognized”).
- (3) Group parameters by payment index, rejecting any duplicate — the same parameter kind, or the same other-parameter name, twice at one index (`Zip321Error::DuplicateParameter`).
- (4) For each index, build a `Payment`. An address parameter must be present, an explicit amount of 0 must not target a transparent-only address, and a memo requires `can_receive_memo()` on its address; the corresponding errors are `RecipientMissing`, `ZeroValuedTransparentOutput`, and `TransparentMemo`. Labels, messages, and other parameters are collected.

Construction 7.8 (Rendering: `TransactionRequest::to_uri`). An empty request renders as `zcash:.` A request with exactly one payment stored at key 0 renders in hier-part form, `zcash:<address>` followed by `?` and the payment’s parameters if any. Otherwise every payment renders in query-parameter form: `zcash:?` followed by, in ascending key order, an address parameter and then that payment’s amount, memo, label, message, and other parameters, the index suffix omitted for key 0 (`zip321, src/lib.rs, to_uri`).

Definition 7.9 (Payment validity predicate). Constructor `Payment::new(a, v, m, ...)` returns an error iff

$$(\text{transparentOnly}(a) \wedge v = \text{Some}(0)) \vee (m \neq \text{None} \wedge \neg \text{canReceiveMemo}(a)),$$

with `PaymentError::ZeroValuedTransparentOutput` and `PaymentError::TransparentMemo` for the respective disjuncts, and `Ok` of the payment otherwise (`zip321, src/lib.rs, Payment::new`). The two predicates live in `zcash_address (src/lib.rs)`: `canReceiveMemo` is true for Sprout, Sapling, and Unified addresses containing a Sapling or Orchard receiver, false for P2PKH, P2SH, and TEX; `transparentOnly` is true for P2PKH, P2SH, and TEX, and for Unified addresses that have a transparent receiver but neither a Sapling nor an Orchard receiver.

Character sets. On parse, a raw `qchar` value may contain alphanumerics plus `- . _ ~ ! $ ' ( ) * + , ; : @ %`, and a parameter name is ALPHA followed by ALPHA/DIGIT/+/-; the value is then percent-decoded and validated as UTF-8. On render, values are UTF-8 percent-encoded over the complement of `qchar`: the ASCII controls together with space and `" # % & / < = > ? [ \ ] ^ ` { | }` (the crate’s `QCHAR_ENCODE` set) (`zip321, src/lib.rs`). Unknown parameters *without* the req- prefix are retained: their percent-decoded values land in `Payment::other_params` and are re-rendered, percent-encoded, by `to_uri`.

Totals. Method `TransactionRequest::total()` sums the payment amounts with overflow checking against `MAX_MONEY`, returning `Ok(None)` when any payment lacks an amount (the total is

undefined) and `Err(BalanceError::Overflow)` on an out-of-range sum (`zip321, src/lib.rs`).

Test vectors. The crate’s test module encodes the ZIP’s own valid and invalid examples: `amount=1` parses to 10^8 zatoshis and `amount=123.456` to 12 345 600 000; the boundary 21000000 is valid and 21000000.00000001 invalid; i64-overflow and u64-wraparound amounts are invalid; `paramindex 10000` is invalid; `amount=123.` is invalid; and `req-unknown=x` is invalid (`zip321, src/lib.rs, test module`).

7.8 From request to proposal to transaction

A parsed `TransactionRequest` enters the wallet through the function `propose_transfer` in module `zcash_client_backend::data_api::wallet`. It takes the wallet database, network parameters, the account to spend from, an input selector, a change strategy, the request, and a confirmations policy, and returns a `Proposal<FeeRule, NoteRef>` by delegating to the input selector’s `propose_transaction`; the proposal is later executed by `create_proposed_transactions`. The convenience wrapper `propose_standard_transfer_to_address` places a single `Payment` into a one-element request and uses `GreedyInputSelector` with `SingleOutputChangeStrategy` (`zcash_client_backend, src/data_api/wallet.rs`).

The selector requires every payment to carry an amount: a missing amount yields `ProposalError::PaymentAmountMissing(idx)`, so a wallet UI must fill in user-chosen amounts before proposing (`zcash_client_backend, src/data_api/wallet/input_selection.rs` and `src/proposal.rs`).

Construction 7.10 (Payment-to-pool assignment). For each $(idx, \text{payment})$ in the request, `GreedyInputSelector` converts the recipient address to the wallet’s network and assigns an output pool (`zcash_client_backend, src/data_api/wallet/input_selection.rs`):

- a transparent address maps to the transparent pool, emitting a `TxOut`;
- a TEX address maps to the transparent pool of the *second* step of a two-step proposal (below), its payment re-wrapped with no memo;
- a Sapling address maps to the Sapling pool;
- a Unified Address maps to Orchard if it has an Orchard receiver and Orchard is supported; else to Sapling if it has a Sapling receiver and Sapling is supported; else to its transparent receiver; else the selection fails (`GreedyInputSelectorError::UnsupportedAddress`).

The selector then chooses spendable shielded notes to at least the required amount, preferring the pool matching the outputs, computes change under the change strategy, and returns a proposal whose steps embed the originating request and the resulting map `payment_pools : idx ↦ PoolType`.

TEX addresses (ZIP-320). A payment to a TEX address is consumed as a two-step proposal: step tr_0 excludes the TEX outputs and instead creates one ephemeral transparent change output; step tr_1 spends that ephemeral output to the TEX recipients, with the memo forced to `None`. Paying a TEX address from shielded inputs in one step is rejected (`ProposalError::PaysTexFromShielded`) (`zcash_client_backend, src/data_api/wallet/input_selection.rs` and `src/data_api/wallet.rs`).

Step validation. Constructor `Step::from_parts` enforces that the keys of `payment_pools` exactly match the request’s payment indices, that each selected pool satisfies `recipient_address()`

.can_receive_as(pool), that every payment has an amount, and the balance equation

$$v_{\text{transparent}}^{\text{in}} + v_{\text{shielded}}^{\text{in}} + v_{\text{prior steps}}^{\text{in}} = \text{total}(R) + \text{total}(\Delta), \quad (2)$$

where R is the embedded request, Δ the proposed balance — change, including ephemeral outputs, plus fee — and every sum is checked in Zatoshis; a violation is a `BalanceError` (`zcash_client_backend, src/proposal.rs`).

Serialization. A proposal serialises its transaction request as its ZIP-321 URI: the protobuf field `transactionRequest` of message `ProposalStep` (string, field 1, package `cash.z.wallet.sdk ffi`) is produced by `to_uri` and parsed back with `from_uri`, alongside `PaymentOutputPool`{`paymentIndex`, `valuePool`} entries for the pool map (`zcash_client_backend, proto/proposal.proto` and `src/proto.rs`). This round-trip is why the crate accepts and renders the empty request `zcash:` — a shielding step has no payments — a deliberate divergence from the ZIP flagged in §7.9.

Execution. At transaction creation, for each (idx, pool) in the step’s pool map the corresponding `Payment` is fetched from the embedded request. A shielded output uses the payment’s memo, or the sentinel `0xF6 || 0x00`⁵¹¹ when absent (§7.5); a memo on a transparent output raises `Error::Payment(PaymentError::TransparentMemo)`. For a Unified Address, the receiver actually paid is the one recorded in `payment_pools` — the Orchard, Sapling, or transparent receiver extracted from the UA (`zcash_client_backend, src/data_api/wallet.rs`). From here the construction is that of the *Ironwood Guide*: note encryption seals each output’s plaintext, and the spend and binding signatures sign the ZIP-244 sighash.

7.9 Divergences between ZIP-321 and the deployed stack

Our conventions require flagging every point where the deployed code and the ZIP disagree; ZIP-321 accumulates several, all in the parser.

- (D1) *Empty requests.* The crate accepts `zcash:` (and `zcash:?`) as a valid empty `TransactionRequest` and renders empty requests as `zcash: (zip321, src/lib.rs)`, but the grammar of Definition 7.2 requires either an address or `? zcashparams` after the scheme, and the ZIP defines a request as having one or more payments. The divergence is deliberate: proposal serialisation round-trips shielding steps — which have no payments — through the URI form (§7.8).
- (D2) *Sprout recipients.* ZIP-321 says Sprout addresses MUST NOT be supported (§7.3), yet the crate parses a Sprout recipient without error — `ZcashAddress::try_from_encoded` decodes Sprout (`zcash_address, src/encoding.rs`) and `to_payment` performs no Sprout check; indeed `canReceiveMemo` is true for Sprout, so even a memo to one is accepted. Rejection occurs only downstream, at proposal time, when address conversion fails with `ConversionError::Unsupported("Sprout")` because `zcash_keys::address::Address` has no Sprout variant (`zcash_address, src/convert.rs`; `zcash_client_backend, src/data_api/wallet/input_selection.rs`). The deployed stack satisfies the payment-level requirement but not URI-level rejection.
- (D3) *Case sensitivity.* ABNF literal strings (RFC 5234) are case-insensitive, so a strict reading makes `ZCASH:` — and even `AMOUNT=` — grammar-valid; the deployed parser matches only lowercase (`zip321, src/lib.rs, tag("zcash:")` and the literal parameter names). The ZIP text never addresses case, which matters for QR alphanumeric-mode URIs, which are uppercase. We adopt the deployed, lowercase-only behaviour as normative and treat the ABNF case-insensitivity as a defect of the ZIP text: an uppercased URI is unparseable to the deployed stack, so a producer wanting QR alphanumeric mode must not obtain it by uppercasing the URI.

- (D4) *Valueless parameters.* The grammar makes `=` optional for `otherparam` and `otherreqparam` (a bare `nonce` is derivable), but the crate requires `name=value`, rejecting such grammar-valid URIs (`zip321`, `src/lib.rs`, `zcashparam`).
- (D5) *Bare percent signs.* The grammar admits `%` in a value only as pct-encoded (`%HEXDIG HEXDIG`), but the parser accepts `%` as a raw character and its percent decoder passes invalid sequences through unchanged, so `message=100%` is accepted — laxer than the grammar (`zip321`, `src/lib.rs`, `qchars` and `percent_decode`). A direct test against the crate confirms it: the values `100%`, `100%zz`, and `100%2` all parse, the malformed sequences surviving percent-decoding verbatim, while `100%25` decodes to `100%`.
- (D6) *Empty list elements.* The production `zcashparam` can derive the empty string, making `a=1&&b=2` arguably grammar-valid, while the crate rejects it (its list elements must be nonempty) — a divergence in the opposite direction of (D4) and (D5).
- (D7) *Zero-valued transparent outputs.* The crate rejects `amount=0` to a transparent-only address (Definition 7.9); ZIP-321 has no such rule. The crate's changelog asserts such outputs are disallowed by consensus (`zip321`, `CHANGELOG.md`, 0.7.0 entry). The assertion is inaccurate: consensus admits zero-valued transparent outputs — `CheckTransaction` rejects only negative values and values above `MAX_MONEY` (`zcashd`, `src/main.cpp`), and the protocol specification constrains transparent outputs no further, requiring only a nonnegative remaining transparent value pool. What rejects them in practice is *standardness*: a zero-valued spendable output falls below the dust threshold (zero only for unspendable scripts, which no address payment produces), so `zcashd`'s `IsStandardTx` (`src/policy/policy.cpp`) and Zebra's equivalent mempool check (`zebrad`, `src/components/mempool/storage.rs`) refuse to admit or relay it. The crate's rule is sound — such an output is unreliable — but on standardness, not consensus, grounds.

Part VIII

Sync Guide: Light-client synchronisation: compact blocks, the service, and the scanning pipeline

Abstract

Assuming the Orchard protocol of the *Ironwood Guide* and the wallet layer of the *Wallet Guide*, this volume of *The Zcash Arboretum* documents how a light client synchronises: the compact-block format (ZIP-307) and its deployed divergences, the light-client service surface as `lightwalletd`, `Zaino`, and `Zebra` implement it, the scanning pipeline of `librustzcash` with its verify-first reorg discipline, commitment-tree synchronisation, and — stated without euphemism — what the server learns. The wire format is deployed but underspecified; where the draft ZIP and the deployed implementations disagree, this volume documents the deployment and flags the divergence.

1 Compact blocks (ZIP-307)

A wallet that trusts no server detects its notes by trial-decrypting every shielded output on the chain (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”). That obligation costs twice: one key agreement of computation per output, and the bandwidth of downloading every output. ZIP 307 (*Light Client Protocol for Payment Detection*) attacks the bandwidth half. A *compact block* is “a packaging of ONLY the data from a block needed to” (1) detect a payment to one’s shielded address, (2) detect a spend of one’s notes, and (3) update witnesses for new spend proofs (ZIP 307, “Compact Stream Format”). The deployed protocol widens this beyond ZIP 307’s Sapling-only scope to every shielded pool, and adds a fourth purpose: detecting spends of the coins of the wallet’s transparent addresses (`compact_formats.proto`, header comment, lines 21–26). The computational half survives compaction — every compact output must still be trial-decrypting — and that residual linear scan is among the costs whose removal motivates the successor design (*Tachyon Guide*, §“What the fold removes”).

1.1 Normative status and wire-format history

ZIP 307 carries Status: Draft, covers Sapling only, and nowhere mentions Orchard, `ChainMetadata`, or transparent compact data (`zip-0307.rst`, header and whole file). Its message definitions are stale relative to deployment: the ZIP text defines `CompactBlock{BlockID id = 1; vtx = 3}` and `CompactTx{txIndex, txHash, spends, outputs}`, with element messages named `CompactSpend` and `CompactOutput` (ZIP 307, “Compact Stream Format”), whereas the deployed messages of §1.2 use different names, field numbers, and fields. The renames to `CompactSaplingSpend/CompactSaplingOutput` date to `lightwallet-protocol v0.3.1` (2021-12-09), which also added `CompactOrchardAction` (`lightwallet-protocol`, `CHANGELOG.md`).

The canonical proto files live in the `zcash/lightwallet-protocol` repository and are vendored — git subtree or symlink — into `lightwalletd`, `librustzcash` (the files under `zcash_client_backend/proto/` are symlinks into `lightwallet-protocol/walletrpc/`), and `Zaino`. The repository’s README states that it “IS not a specification” and defers to ZIP 307 (`lightwalletd`, `lightwallet-protocol/README.md`) — which is itself stale. No current document therefore normatively specifies the deployed wire format. Per our conventions we classify the compact-block format *deployed but underspecified*: we cite `lightwallet-protocol` tagged releases as the de facto definition, and flag each point where ZIP text and deployment disagree (§1.8).

Tag	Date	Change
v0.3.1	2021-12-09	CompactOrchardAction added; CompactSpend/CompactOutput renamed CompactSaplingSpend/CompactSaplingOutput
v0.3.2	2021-12-09	CompactOrchardAction.encCiphertext renamed ciphertext
v0.3.5	2023-07-03	ChainMetadata and CompactBlock.chainMetadata added; GetSubtreeRoots/SubtreeRoot added; nul- lifier RPCs added; CompactSaplingOutput.epk re- named ephemeralKey
v0.4.0	2025-12-03	CompactTxIn/TxOut and vin/vout added; BlockRange.poolTypes added; LightdInfo.lightwalletProtocolVersion added; CompactTx.hash renamed txid (serialization- compatible)
v0.4.1	2026-02-20	nullifier RPCs designated deprecated in the proto
v0.5.0	2026-06-30	Ironwood fields added; protoVersion removed, field number 1 and name reserved

Table 1: Wire-format history of lightwallet-protocol (CHANGELOG.md); v0.5.0 is the current tagged release.

At the pinned revisions, the three codebases vendor three different revisions: lightwalletd v0.4.1 (protoVersion present, no Ironwood); librustzcash main v0.5.0 (protoVersion removed and reserved; Ironwood fields present); Zaino an intermediate snapshot carrying *both* the Ironwood fields and protoVersion (diff of the three compact_formats.proto copies). We flag the skew, and a changelog-date contradiction, in §1.8. Deployed-behaviour claims in this section are verified against librustzcash e30517e4, lightwalletd 61fee32e, Zaino 0e057e22, and zips 6961098 (checkouts of 2026-07-14).

Since that snapshot, lightwalletd has caught up: release 0.5.0 vendored tagged lightwallet-protocol v0.5.0, removed protoVersion, and added Ironwood parsing and service data; release 0.5.4 now reports lightwalletProtocolVersion = ``v0.5.0`` (lightwalletd 09593ed, CHANGELOG.md). Detailed behaviour comparisons below retain their explicit older pins so that their line citations and divergence findings remain reproducible.

1.2 The deployed messages

The messages live in the protobuf package cash.z.wallet.sdk.rpc and declare proto3 syntax (compact_formats.proto, lines 5–11). Proto3 semantics make every field optional: an absent field decodes to zero or empty, indistinguishable from an explicit zero. The scanner must defend against this — the default-zero tree sizes of §1.5 are the load-bearing case.

```

message CompactBlock {
    uint32 protoVersion = 1;
    uint64 height = 2;
    bytes hash = 3;
    bytes prevHash = 4;
    uint32 time = 5;
    bytes header = 6;
    repeated CompactTx vtx = 7;
    ChainMetadata chainMetadata = 8;
}

message ChainMetadata {
    uint32 saplingCommitmentTreeSize = 1;
    uint32 orchardCommitmentTreeSize = 2;
    uint32 ironwoodCommitmentTreeSize = 3;
}

message CompactTx {
    uint64 index = 1;
    bytes txid = 2;
    uint32 fee = 3;
    repeated CompactSaplingSpend spends = 4;
    repeated CompactSaplingOutput outputs = 5;
    repeated CompactOrchardAction actions = 6;
    repeated CompactTxIn vin = 7;
    repeated TxOut vout = 8;
    repeated CompactOrchardAction ironwoodActions = 9;
}

```

removed in v0.5.0; number and name reserved
block height
block hash, 32 bytes
parent block hash, 32 bytes
Unix epoch time
“should always be unset (empty)”
compact transactions
end-of-block tree sizes

Sapling tree size at end of block
Orchard tree size at end of block
v0.5.0 only

position within the block
32 bytes, protocol byte order
unpopulated in practice

coinbase input omitted
v0.5.0 only

The container messages, from the deployed protos (lightwalletd vendored copy, walletrpc/compact_formats.proto, lines 14–80; librustzcash copy, lines 14–86):

- Field header “should always be unset (empty)” per the proto comment (lines 27–30). ZIP 307’s block header validation — Equihash (*Consensus Guide*, §“Equihash and the memoryless model”), bits, difficulty — is therefore unimplementable against deployed servers; the ZIP itself flags the section as “only partially implemented: currently only prevHash is checked” (ZIP 307, “Block header validation”). Client-side, only prevHash and height continuity are enforced (§1.5). What a header-verifying client would gain from the ZIP-221 chain-history commitment is the *FlyClient Guide*’s subject.
- Field protoVersion never varies on the wire. lightwalletd never sets it, the assignment commented out as a TODO (parser/block.go, line 112); deployed Zaino serves 0 on every path (zaino-state, compact_block.rs line 285, legacy.rs line 1113), which for a proto3 scalar is byte-identical to lightwalletd’s absent field—its lone non-zero literal (zaino-fetch, src/chain/block.rs, line 218) sits in a parse-error Debug string and never reaches serialization; v0.5.0 removed the field, reserving number 1 and the name (librustzcash copy, lines 33–36). The librustzcash scanner never reads it. Flagged in §1.8.
- Message ChainMetadata states each pool’s note commitment tree size *as of the end of this block*, as uint32 (compact_formats.proto, lines 14–17): the tree holds at most 2^{32} leaves, and the scanner maps arithmetic overflow to ScanError::TreeSizeOverflow (zcash_client_backend, src/scanning.rs, lines 649–654). Section 1.5 shows how these sizes turn a compact block into note positions.
- Field txid carries the 32-byte transaction identifier in protocol byte order; the proto comment mandates that it “MUST NOT be reversed or hex-encoded” — the reversed hex form is display-only — citing protocol spec §7.1.1, “Transaction Identifiers” (compact_formats.proto, lines 52–57).

- The `fee` field is documented “present if server can provide”, but neither deployed server populates it: `lightwalletd` leaves it commented out with `TOD0: calculate fees` (`parser/transaction.go`, line 407) and `Zaino` hardcodes `fee: 0` (`zaino-fetch`, `src/chain/transaction.rs`, line 369). Clients cannot rely on it. For a transaction with no transparent inputs the proto comment gives the client-side reconstruction (`compact_formats.proto`, lines 59–64):

$$\text{fee} = \text{valueBalanceSapling} + \text{valueBalanceOrchard} + \sum \text{vPubNew} - \sum \text{vPubOld} - \sum \text{tOut},$$

with `valueBalanceIronwood` joining the sum in v0.5.0.²

- Field `vin` omits the coinbase transaction’s single null-outpoint input; a light client recognises a coinbase `CompactTx` by `index = 0`, coinbase being always first in a block. Server `lightwalletd` implements exactly this, skipping `vin` when `index = 0` (`compact_formats.proto`, lines 70–76; `parser/transaction.go`, lines 398–403). The transparent element messages are `CompactTxIn{prevoutTxid, prevoutIndex}` and `TxOut{value, scriptPubKey}` (`compact_formats.proto`, lines 82–104).

The shielded elements. Each shielded element message keeps only the detection-relevant bytes of its full on-chain description.

Message	Fields (bytes)	Payload	Full form	Saving
<code>CompactSaplingSpend</code>	<code>nf</code> (32)	32	384	~90%
<code>CompactSaplingOutput</code>	<code>cmu</code> (32), <code>ephemeralKey</code> (32), <code>ciphertext</code> (52)	116	580 + 32	~80%
<code>CompactOrchardAction</code>	<code>nullifier</code> (32), <code>cmx</code> (32), <code>ephemeralKey</code> (32), <code>ciphertext</code> (52)	148	—	—

Table 2: Compact shielded elements (`compact_formats.proto`, lines 106–130). The Sapling payload and saving figures are ZIP 307’s (“Spend Compression”, “Output Compression”); the 148-byte Orchard total is arithmetic, not stated in the sources. Payload figures count raw bytes; on the wire, a `CompactSaplingSpend` encodes to 34 bytes (36 as a `CompactTx.spends` element), a `CompactSaplingOutput` to 122 (124), and a `CompactOrchardAction` to 156 (159 — its 156-byte payload needs a two-byte length varint). Field headers add 2 bytes per 32-byte field and 2 per 52-byte ciphertext. Computed by script, proto-encoding every compact element of mainnet block 3,412,934 (3 spends, 8 outputs, 10 actions; `GetBlock`, `lightwalletd` v0.4.19, 2026-07-15); the sizes are block-independent because all field lengths are fixed.

A `CompactSaplingSpend` retains only the 32-byte nullifier of the 384-byte Spend description (`cv` 32, anchor 32, nullifier 32, `rk` 32, `zkproof` 192, `spendAuthSig` 64) — the one field needed to detect a spend of one’s own notes; ZIP 307 quotes this as a 90% bandwidth improvement (ZIP 307, “Spend Compression”). A `CompactSaplingOutput` retains `cmu`, the ephemeral key, and the first 52 bytes of `encCiphertext`, “Total size is 116 bytes” per the proto comment, against 580 bytes of ciphertext plus the 32-byte ephemeral key streamed in full form — ZIP 307’s 80% reduction (ZIP 307, “Output Compression”; `lightwalletd compact_formats.proto`, lines 114–122; full encodings per protocol spec §§7.3–7.4). A `CompactOrchardAction` serves both roles of an Action in one message: its nullifier detects spends of the input note, while `cmx`, `ephemeralKey`, and `ciphertext` detect and place the output note (`compact_formats.proto`, lines 124–130; full Action encoding per protocol spec §7.5).

²Type `uint32` additionally caps the representable fee at $2^{32} - 1$ zatoshi (≈ 42.9 ZEC); the proto comment does not remark on this.

1.3 Omissions and fetch-on-detect

Compact blocks omit the 512-byte memo, the 16-byte AEAD tag, the 80-byte outCiphertext, cv, rk, anchors, zkproofs, and signatures (ZIP 307, “Transaction privacy”; sizes per `zcash_note_encryption 0.4.1, src/lib.rs, lines 44–57`). Recovering a memo, or spend information via the outgoing ciphertext, thus requires downloading the full transaction — deployed as the `GetTransaction` RPC. ZIP 307 attaches three requirements to that fetch: the client SHOULD obscure its interest by also downloading numerous uninteresting transactions, SHOULD download all transactions of any block it touches, and MUST convey the privacy reduction to the user (ZIP 307, “Transaction privacy”).

The deployed pipeline expresses the fetch as data: scanning emits `TransactionDataRequest::Enhancement(txid)` — “download of complete raw transaction data” — which the caller satisfies via `GetTransaction` and feeds to `wallet::decrypt_and_store_transaction(zcash_client_backend, src/data_api.rs, lines 1185–1207, 2210)`. The crate’s own sync loop documents that it does not yet perform enhancement (`zcash_client_backend, src/sync.rs, lines 1–10`).

1.4 Compact trial decryption

The 52-byte ciphertext field is the prefix of `encCiphertext` that encrypts exactly the *compact note plaintext*: lead byte (1) || diversifier d (11) || value v (8) || rseed (32), so `COMPACT_NOTE_SIZE = 52`; the full plaintext appends the 512-byte memo, and the full ciphertext a 16-byte tag, giving 580 bytes (`zcash_note_encryption 0.4.1, src/lib.rs, lines 44–57`). ZIP 307’s rationale: these bytes “contain the contents and opening of the note commitment, which is all of the data needed to spend the note and to verify that the note is spendable” (ZIP 307, “Output Compression”). The *Ironwood Guide* constructs compact decryption for Orchard in §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”; we do not repeat the construction, and record here only the deployed mechanics and the integrity argument.

Function `try_compact_note_decryption` (doc-comment: “Implements the procedure specified in ZIP 307”) proceeds as follows (`zcash_note_encryption 0.4.1, src/lib.rs, lines 567–604`):

$$\begin{aligned} K &:= \text{KDF}(\text{KA.Agree}(\text{ivk}, \text{epk}), \text{ephemeralKey}), \\ \text{np} &:= \text{ciphertext}[0..52] \oplus \text{ChaCha20}_K(\text{nonce} = 0^{96})[64..116], \end{aligned}$$

a bare ChaCha20 keystream with zero nonce, sought to byte offset 64 — skipping keystream block 0, which the full AEAD reserves for the Poly1305 key (`src/lib.rs, lines 590–593`). The candidate `np` is parsed as a compact note plaintext (no memo), then `check_note_validity` accepts iff

- (i) the plaintext parses, and
- (ii) the recomputed commitment matches the on-chain one, $\text{Extract}(\text{NoteCommit}_{\text{rcm}}(\dots)) = \text{cmu}$ (respectively cmx), and
- (iii) per ZIP-212, the ephemeral secret `esk` re-derived from `rseed` reproduces the published key: the byte encoding of $\text{KA.DerivePublic}(\text{esk}, g_d)$ equals `ephemeralKey`, compared in constant time

(`src/lib.rs, lines 524–604`). Condition (ii) is the integrity substitute: truncation at 52 bytes discards the AEAD tag, so a compact decryption cannot be authenticated via Poly1305; ZIP 307 instead recomputes the note commitment from the decrypted plaintext and compares against the on-chain `cmu`, itself authenticated by the binding signature over the transaction (ZIP 307, “Output Compression”). Condition (iii) is the deployed strengthening beyond the tag: it defeats a mauled ephemeral key (*Ironwood Guide*, §“Transmission and detection of a note”).

Lead byte and the ZIP-212 grace period. The lead byte is `0x01` pre-Canopy and `0x02` under ZIP-212; deployed code switches on `zip212_enforcement`: during the grace period of 32,256 blocks after Canopy activation both lead bytes are accepted, and thereafter only `0x02` (`zcash_primitives,`

src/transaction/components/sapling.rs, lines 27–40; zcash_protocol, src/consensus.rs, line 681). ZIP 307’s pseudocode disagrees with itself here: its two “[Canopy onward]” lines are *both* conditioned on $\text{height} < \text{CanopyActivationHeight} + \text{ZIP212GracePeriod}$, the first rejecting $\text{leadByte} \notin \{0x01, 0x02\}$ and the second rejecting $\text{leadByte} \neq 0x02$ (ZIP 307, “Scanning for relevant transactions”, zip-0307.rst, lines 275–276). Read literally, the pair rejects 0x01 *during* the grace period and constrains nothing after it — the inverse of the intent; the second condition should read $\geq$. Deployed code implements the intended semantics. The defect is tracked in research/upstream-defects.md, item 1 (ZIP 307: second Canopy-onward lead-byte condition rejects 0x01 during the grace period; verified 2026-07-15 at zips 69610984). We flag the divergence in §1.8.

1.5 Scanning: spends, positions, and tree sizes

Spend detection. The scanner parses every compact nullifier field up front: each CompactSapling Spend.nf and each CompactOrchardAction.nullifier. A malformed length yields ScanError::EncodingInvalid rather than a panic. The parsed nullifiers are compared against the wallet’s known-unspent nullifiers in constant time (find_spent); unmatched nullifiers are retained in per-block nullifier maps (zcash_client_backend, src/scanning/compact.rs, lines 312–390; src/scanning.rs, lines 826–828).

Tree building and positions. Every cmu and cmx in the block — not only the wallet’s — is appended to the wallet’s note commitment tree with Retention marks, and each detected note is marked at its position (zcash_client_backend, src/scanning.rs, lines 786–824). The position is computed arithmetically, never by replaying the chain:

$$\text{pos}(\text{output } i \text{ of } tx) = \text{start_tree_size} + \text{offset}(tx) + i,$$

where $\text{offset}(tx)$ counts the outputs of earlier transactions in the block (PositionTracker:: {sapling, orchard}_note_position). The shardtree that consumes these marks is constructed in the *Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”; the observation that a light wallet reads positions directly off the block appears in §“Transmission and detection of a note”.

Positions are load-bearing, not merely convenient: the TreeSizeUnknown doc-comment records that without the tree size it is “impossible to construct the nullifier for a detected note” — the Sapling nullifier depends on the note’s position, via $\rho = \text{MixingPedersenHash}(\text{cm}, \text{NotePosition})$; Orchard’s does not, but both pools need positions for witnesses (zcash_client_backend, src/scanning.rs, lines 633–639; protocol spec, “Computing ρ values and Nullifiers” and “Dummy Notes”).

Where the tree size comes from. The start size for a block is resolved by a ladder (tree_sizes_around, zcash_client_backend, src/scanning/compact.rs, lines 589–696):

$$\text{start}(\text{pool}) := \begin{cases} s_{\text{prior}} & \text{prior BlockMetadata known;} \\ s_{\text{final}} - n_{\text{block}} & \text{chainMetadata present, by checked subtraction;} \\ 0 & \text{block below the pool’s activation height;} \\ \perp & \text{otherwise (ScanError::TreeSizeUnknown),} \end{cases}$$

where s_{final} is the chainMetadata end-of-block size and n_{block} the block’s own output count. The subtraction is checked_sub: underflow yields ScanError::TreeSizeInvalid, which is precisely the guard against proto3’s default-zero metadata (§1.2). After walking all CompactTxs the accumulated position must equal the chainMetadata size, else scanning fails with ScanError::TreeSizeMismatch{given, computed} (src/scanning/compact.rs, lines 257–287, 745–770).

The error taxonomy routes recovery: `TreeSizeMismatch`, `PrevHashMismatch` (`prevHash` against the prior block’s hash), and `BlockHeightDiscontinuity` (`height` $\neq$ `prev` + 1) report `is_continuity_error() = true` and trigger the truncate-and-rescan reorganisation path (*Wallet Guide*, §“Reorganisations: truncate and rescan”); `EncodingInvalid`, `TreeSizeUnknown`, `TreeSizeInvalid`, and `TreeSizeOverflow` do not (`zcash_client_backend`, `src/scanning.rs`, lines 602–685). The final tree size also drives checkpointing: the helpers `compact_tx_contains_last_sapling_outputs_in_block`, `..._orchard_actions_in_block`, and `..._ironwood_actions_in_block` compare the running position plus the current transaction’s output or action count against the final size, to decide when to write the block-boundary checkpoint into the commitment tree (`src/scanning/compact.rs`, lines 712–731; call sites 404, 444, 482).

The server side of ChainMetadata. Server `lightwalletd` fills the tree sizes from the node’s `getblock verbosity=1` response fields `trees.sapling.size` and `trees.orchard.size` (`lightwalletd`, `common/common.go`, lines 163–175, 399–400). The deployed backend is `zebrad` (§2.5), whose `getblock` omits each subfield while the pool’s tree is empty (`zebra-rpc`, `src/methods.rs`, `GetBlockTrees`, `skip_serializing_if`) — in which case Go’s JSON unmarshal leaves `Size = 0`, the default-zero case `TreeSizeInvalid` exists to catch. Historically, `zcashd` (retired; last release v6.10.0, 2025-10-03) computed the sizes by anchor lookup (`GetSaplingAnchorAt/GetOrchardAnchorAt` on `hashFinalSaplingRoot/hashFinalOrchardRoot`) and omitted the subfield when the anchor was unavailable (`zcashd`, `src/rpc/blockchain.cpp`, lines 279–297; the `trees` field dates to `zcashd` commit `cf2b6c796`, “rpc: Add trees field to getblock RPC output”).

1.6 The scan pipeline: chain state, batching, caching

Function `scan_cached_blocks` requires a `ChainState` — the final Sapling, Orchard, and Ironwood tree frontiers as of `from_height - 1` — and asserts `from_height = chain_state.height + 1`; all three trees have depth 32 (`zcash_client_backend`, `src/data_api/chain.rs`, lines 504–698). The `ChainState` comes from the server’s `GetTreeState` RPC via `TreeState::to_chain_state()`, which hex-decodes and byte-reverses the block hash and parses empty tree fields as empty trees (`src/proto.rs`, lines 367–463). Trial decryption of compact outputs is batched through `BatchRunners` with batch size 100 (`src/data_api/chain.rs`, line 642).

The proto accessors split their failure modes: block-level convenience accessors panic on malformed server data (`CompactBlock::hash` and `prev_hash` if not 32 bytes, `height` if outside `u32`), while per-output accessors (`cmu`, `cmx`, `nf`, ephemeral key) return `CompactFormatError::InvalidLength`, `InvalidValue`; the `From` impls that *build* compact messages from full bundles take `encCiphertext[0..COMPACT_NOTE_SIZE]` (`zcash_client_backend`, `src/proto.rs`, lines 63–286). The scanner bounds `CompactTx.index` to `u16` (the expect reads “Cannot fit more than 2^{16} transactions in a block”), and `CompactBlock.time` — `uint32` Unix epoch, “if non-zero” — flows into `ScannedBlock` and thence wallet metadata (`src/scanning/compact.rs`, lines 314–315, 552–555).

Compact blocks are cached as serialized protos. Crate `zcash_client_sqlite` offers `BlockDb`, a `compactblocks` SQLite table decoded on read (`CompactBlock::decode`), and `FsBlockDb`, one file per block plus a `BlockMeta` row {`height`, `blockhash`, `time`, `sapling_outputs_count`, `orchard_actions_count`} that enables range queries without decoding (`zcash_client_sqlite`, `src/chain.rs`, lines 30–230).

ZIP 307’s own client-operation sketch — append every `cmu` to a depth-32 incremental Merkle tree, keep a per-note incremental witness, and cache roughly 100 recent tree states because “no full Zcash node will roll back the chain by more than 100 blocks” (ZIP 307, “Creating and updating note witnesses”) — describes the pre-shardtree design. The deployed replacement is the position-marked shardtree fed by `GetSubtreeRoots` (RPC added in `lightwallet-protocol` v0.3.5; `zcash_client_backend`, `src/sync.rs`, lines 80–86), whose mechanics the *Ironwood Guide* constructs in §“Proof of ownership of some valid note: the commitment tree”.

1.7 Server inclusion semantics and successor surface

Which transactions a compact block contains is itself divergent. ZIP 307 states that “By default, CompactBlocks only contain CompactTx’s for transactions that contain Sapling spends or outputs” (ZIP 307, “Block header validation”). Server `lightwalletd` builds and caches compact representations of *all* transactions, transparent-only ones included, with `vin/vout` populated (`parser/block.go`, lines 120–125), and `GetBlock` serves them unfiltered per the v0.4.1 contract; `GetBlockRange` filters per request — with empty `poolTypes` it prunes each transaction to its Sapling and Orchard components and drops transactions left empty (`common/common.go`, `FilterTxPool/filterBlockPool`, lines 526–653) — so the default sync stream carries shielded-relevant transactions only, as Zaino’s does. The successor Zaino filters out CompactTx’s whose compact fields are all empty, and supports per-request pool filtering via `PoolTypeFilter` and `BlockRange.poolTypes` (`zaino-proto`, `src/proto/utls.rs`, `compact_block_with_pool_types`, lines 328–380, applied on the serving paths in `zaino-state`, `chain_index.rs`) — a filter added in v0.4.0, where an empty `poolTypes` means Sapling and Orchard for backward compatibility (`service.proto`, lines 38–41). Both servers therefore default to shielded-relevant transactions on the sync stream; we flag the deployed defaults against the ZIP in §1.8.

Two `lightwalletd` RPCs, `GetBlockNullifiers` and `GetBlockRangeNullifiers`, strip actions to nullifier-only, drop outputs, `vin`, and `vout`, and zero both `ChainMetadata` tree sizes (“these are not needed (we prefer to save bandwidth)”) (`frontend/service.go`, lines 212–302). Blocks so obtained therefore cannot seed the position ladder of §1.5 — zeroed sizes land in the `TreeSizeInvalid/TreeSizeUnknown` paths — so the RPCs serve spend-status refresh, not scanning. Both were declared deprecated in the `lightwallet-protocol` v0.4.0 changelog (2025-12-03); v0.4.1 designated them deprecated in the proto itself (`option deprecated = true`, `service.proto:247,267`), in favour of `GetBlockRange` with `poolTypes` (`service.proto`, lines 244–266; `CHANGELOG.md`, v0.4.0/v0.4.1).

Ironwood. The v0.5.0 fields (`ironwoodCommitmentTreeSize = 3`, `ironwoodActions = 9`) exist only in `lightwallet-protocol` v0.5.0; `librustzcash` gates the corresponding tree accounting on `NetworkUpgrade::Nu6_3`, and `lightwalletd` — the deployed service — neither parses nor serves Ironwood data (`zcash_client_backend`, `src/scanning/compact.rs`, lines 687–696; `proto/compact_formats.proto`, lines 17, 74). This describes the pinned `lightwalletd` revision: current `lightwalletd` 0.5.4 parses and serves Ironwood and vendors the tagged v0.5.0 protocol. The pool and its activation are *specified* (ZIP 2005, Proposed; ZIP 258 remains Draft as a document, while NU6.3 is active from Testnet height 4,134,000 and Mainnet height 3,428,143); deployed Zaino wires both fields through its serving builders (`zaino-state`, `chain_index/types/db/legacy.rs`), where they remain empty until NU6.3 data exists on chain.

1.8 Divergences between ZIP-307 and the deployed protocol

Per our conventions, we flag rather than resolve the following.

- *No normative specification of the deployed wire format.* ZIP 307 is Draft, Sapling-only, and its message definitions are stale — old names, old field numbers, no Orchard, no `ChainMetadata`, no `vin/vout` — while the `lightwallet-protocol` README disclaims being a specification and defers to ZIP 307 (`zip-0307.rst`, “Compact Stream Format”; `lightwallet-protocol/README.md`). The deployed format is specified only by tagged proto releases.
- *The grace-period pseudocode bug.* Both “[Canopy onward]” conditions in ZIP 307’s compact-decryption procedure test `height < CanopyActivationHeight + ZIP212GracePeriod`; read literally they reject `0x01` during the grace period and constrain nothing after it. Deployed `zip212_enforcement` accepts both lead bytes during the grace period and requires `0x02` after (`zip-0307.rst`, lines 275–276; `zcash_primitives`, `src/transaction/components/`

sapling.rs, lines 27–40; research/upstream-defects.md, item 1, verified 2026-07-15 at zips 69610984).

- *Inclusion semantics.* ZIP 307 says Sapling-relevant transactions only; the deployed default stream includes Orchard data, transparent data is available on request (`BlockRange.poolTypes`), and `GetBlock` returns all pools unconditionally (§1.7).
- *A `protoVersion` that never varies on the wire.* Server `lightwalletd` never sets it; deployed Zaino serves 0, byte-identical to an unset `proto3` scalar; and v0.5.0 removed it with field number and name reserved (issue `zcash/lightwallet-protocol#25`); `librustzcash` never reads it (§1.2).
- *Dead header-validation surface.* The proto declares header “should always be unset”; ZIP 307’s header and `finalSaplingRoot` checks are unimplementable against deployed servers, and only `prevHash/height` continuity is enforced client-side (§1.2, §1.5).
- *An advertised but unpopulated `fee` field.* The proto says “present if server can provide”; `lightwalletd` has a TODO, Zaino hardcodes zero (§1.2).
- *Vendored-revision skew at the pinned revisions.* `lightwalletd` vendors v0.4.1, `librustzcash` main v0.5.0, and Zaino an intermediate snapshot with both the Ironwood fields and `protoVersion`; Zaino’s vendored changelog dates v0.5.0 as 2026-07-03 without the `protoVersion` removal, while the canonical changelog dates it 2026-06-30 with the removal — so Zaino’s vendored proto still *declares* the field the canonical proto now reserves, though it serves only 0 on the wire (diff of the three `compact_formats.proto` copies; `zaino-proto/lightwallet-protocol/CHANGELOG.md`, lines 40–51).

2 The light-client service

A light wallet holds keys and no chain. The *Ironwood Guide* (§“Transmission and detection of a note: encryption, trial decryption, and synchronisation”) establishes what synchronisation consumes: every compact shielded output on the chain, for trial decryption; every nullifier, for spentness; the note commitment tree’s frontier and subtree roots, for witness assembly; and, for the few transactions that prove to be the wallet’s own, the full serialised bytes. The deployed layer that delivers these is a single gRPC service, `cash.z.wallet.sdk.rpc.CompactTxStreamer`, spoken between the wallet and an untrusted proxy in front of a full node. This section documents the service as deployed: the authority trail from ZIP 307 through the canonical protobuf definitions (§2.1), the compact data format (§1), the interaction model the ZIP prescribes (§2.3), the twenty RPCs and their semantics (§2.4), the server implementations — `lightwalletd`, the deployed proxy (§2.5); Zaino, its successor (§2.6); and Zebra’s newer in-process server (§2.7) — and a register of the points where the implementations and the normative texts disagree (§2.8). The linear costs this architecture accepts — trial decryption of every output, a maintained witness per note — are precisely the costs the Tachyon redesign removes (*Tachyon Guide*, §“What the fold removes”).

2.1 Architecture and authority

ZIP 307, “Light Client Protocol for Payment Detection” (Status: Draft), fixes a three-component architecture: a Zcash full node as the root of trust; a *proxy server* that converts chain data into a compact format; and the light client (ZIP 307, § “High-Level Design”). The security model is deliberately narrow: the client obtains *payment detection privacy* against an honest-but-curious proxy, and trusts that proxy to provide a correct view of the best chain; network-level privacy (Tor, Nym) is assumed to be provided separately; spending privacy is out of scope (ZIP 307, § “Security Model”). The chain has carried commitments designed to let a client check that view succinctly since Heartwood (ZIP 221, via `hashLightClientRoot`, renamed `hashBlockCommitments` at NU5, where the field also binds a

commitment to the block’s transaction signatures and proofs); no deployed client consumes them (*FlyClient Guide*, §“The deployment gap”).

Where the contract lives. The wire contract is *not* in the ZIP. The two protobuf files that define the service — `service.proto` and `compact_formats.proto` — have their canonical home in the `zcash/lightwallet-protocol` repository, which self-describes as the canonical Protobuf definitions but explicitly not a specification: “The Zcash Light Client Protocol is described in ZIP-307” (`lightwallet-protocol`, `README.md`). The authority trail is therefore circular — the ZIP defers detail to the deployed API, and the API repository defers normative force back to the ZIP — and the circle does not close: ZIP 307’s own protobuf sketch (`CompactSpend` and `CompactOutput` messages, a `BlockID` embedded in `CompactBlock`, a `RangeFilter`, a `FullTransaction` return type, a four-RPC `CompactTxStreamer`) differs from the deployed proto (`CompactSaplingSpend`, `CompactSaplingOutput`, `CompactOrchardAction`, flat height and hash fields, `BlockRange`, `RawTransaction`, twenty RPCs), and the ZIP warns that “the details of the API described below may differ from the implementation” (ZIP 307, § “Compact Stream Format”, § “Proxy operation”). We treat the canonical proto files, versioned and change-logged, as the deployed contract, and cite the ZIP for architecture and rationale. Both `lightwalletd` and Zaino vendor the proto files by git subtree (`lightwallet-protocol`, `README.md`).

Contract version. The canonical protocol is at v0.5.0 (2026-06-30). Version 0.4.0 (2025-12-03) added transparent data to the compact format — `PoolType`, `CompactTxIn`, `TxOut`, `CompactTx.vin/vout`, `BlockRange.poolTypes` — and the `LightdInfo` fields `upgradeName`, `upgradeHeight`, and `lightwalletProtocolVersion`, and declared `GetBlockNullifiers` and `GetBlockRangeNullifiers` deprecated; v0.4.1 designated the pair deprecated in the proto itself (`option deprecated = true`, `service.proto:247,267`) and documented `GetBlock`’s transparent-data behaviour. Version 0.5.0 added the Ironwood fields and removed and reserved `CompactBlock.protoVersion` (`lightwallet-protocol`, `CHANGELOG.md`).

The proto forks at the pinned revisions. The following is the July snapshot preserved for the behaviour audit. The canonical v0.4.1 has `PoolType = {INVALID, TRANSPARENT, SAPLING, ORCHARD}` and `ShieldedProtocol = {sapling, orchard}`. The fork in `librustzcash` (`zcash_client_backend/proto/service.proto` and `compact_formats.proto`) adds `IRONWOOD = 4`, `TreeState.ironwoodTree = 7`, `ShieldedProtocol.ironwood = 2`, `ChainMetadata.ironwoodCommitmentTreeSize = 3`, and `CompactTx.ironwoodActions = 9`. Zaino’s fork (`zaino-proto`) adds `IRONWOOD = 4`, `ironwoodTree = 7`, `ChainMetadata.ironwoodCommitmentTreeSize = 3`, and `CompactTx.ironwoodActions = 9`, but not `ShieldedProtocol.ironwood`, and omits the deprecated options on the two nullifier RPCs (diff of the three proto trees; `zaino`, `docs/internal_spec.md`, § “Zaino-Proto”). Zaino’s documentation states the local copies exist “for development purposes” with a plan to rely on `librustzcash`’s. The “as deployed” wire surface therefore differs by repository; this volume cites the canonical file and flags fork-only fields where they appear.

Remark 2.1 (Sources and pinning). Deployed-behaviour claims in this section were verified against `lightwalletd` master 61fee32e (2026-06-25), `zaino` 0e057e22 (2026-07-04), and `librustzcash` e30517e4 (2026-07-10); Zaino pins `zebra-chain` 11.0.0 from `crates.io` (`Cargo.lock`). File-and-line citations below refer to these revisions.

2.2 The compact format

The proxy’s one transformation is compression: it strips from each transaction everything a detecting wallet does not need. Proofs, signatures, and value commitments verify the chain, and the client has delegated chain verification to the proxy; what remains is exactly the trial-decryption input and the nullifier.

Definition 2.2 (Compact Sapling output, ZIP 307). A compact output retains three fields of a Sapling Output description:

$$\text{CompactSaplingOutput} = (\text{cmu}[32], \text{ephemeralKey}[32], \text{ciphertext} = \text{encCiphertext}[0..52]),$$

116 bytes per output against 580 + 32 bytes for the full description with its commitment — an ~80% reduction (ZIP 307, § “Output Compression”; `compact_formats.proto:114--122`). The 52-byte ciphertext prefix covers the compact note plaintext — the contents and opening of the note commitment. Truncation discards the AEAD tag; integrity of the decrypted prefix is instead checked by recomputing the note commitment, as the *Ironwood Guide* constructs in § “Transmission and detection of a note: encryption, trial decryption, and synchronisation”.

Definition 2.3 (Compact Orchard action, deployed proto). The deployed format extends the same compression to Orchard, one message per Action:

$$\text{CompactOrchardAction} = (\text{nullifier}[32], \text{cmx}[32], \text{ephemeralKey}[32], \text{ciphertext}[52])$$

(`compact_formats.proto:124--130`). The Orchard message carries its nullifier inline because an Action both spends and creates; on the Sapling side, spend compression is a separate message that reduces the 384-byte Spend description to its 32-byte nullifier — a ~90% reduction (ZIP 307, § “Spend Compression”).

Transparent data. Since v0.4.0 the compact format can carry transparent inputs and outputs (`CompactTxIn`, `TxOut`, `CompactTx.vin/vout`); v0.4.1 fixes the convention that the coinbase transaction’s single null-outpoint `vin` is omitted, and that clients identify the coinbase by testing `CompactTx.index = 0` (`service.proto:225--237`; `compact_formats.proto:70--76`).

Remark 2.4 (The fee field is dead on the wire). The proto documents `CompactTx.fee` as “present if server can provide”, with the stateless-server formula for the case of no transparent inputs,

$$\text{fee} = \text{valueBalanceSapling} + \text{valueBalanceOrchard} + \sum v_{\text{PubNew}} - \sum v_{\text{PubOld}} - \sum t_{\text{Out}}$$

(`compact_formats.proto:59--64`); with transparent inputs a stateless server cannot compute the fee at all. Neither deployed server ever sets the field: `lightwalletd`’s parser leaves it at 0 with a “TODO: calculate fees” comment (`lightwalletd, parser/transaction.go:397--432`), and Zaino’s conversions pass `fee = None`, encoded 0 (`zaino, zaino-state/src/chain_index/types/db/legacy.rs:1397--1470`). The deployed contract is therefore that the field carries no information, and a client must treat it as absent; we flag the proto’s “present if” language as unimplemented on both servers.

2.3 The interaction model of ZIP 307

The ZIP prescribes a four-phase client-server interaction (ZIP 307, § “Client-server interaction”):

- (A) *Startup*. The client fetches the latest `BlockID`. A wallet importing a pre-existing seed with no recorded birthday starts at Sapling activation, since no shielded address it can derive predates Sapling (ZIP 307, § “Importing a pre-existing seed”).
- (B) *First sync*. The client requests `GetBlockRange(X, Y)` and, on any subsequent request, verifies the hash of the overlap block — the last block it already holds — against its cached copy, so that a reorganisation is detected as a hash mismatch.

(C) *Transaction fetching*. Detected notes are completed by fetching full transactions via `GetTransaction`. Each such fetch names a txid to the proxy, so the ZIP mandates cover traffic: clients SHOULD download decoy transactions, or all transactions of a touched block, and MUST warn users of the privacy loss otherwise (ZIP 307, § “Transaction privacy”).

(D) *Incremental sync*. Later syncs repeat (B) from the cached tip, re-checking the overlap block.

Two hardening measures described by the ZIP are not deployed. Block-header validation by the client is described as only partially implemented — only `prevHash` linkage is checked (ZIP 307, § “Block header validation”). The *bucketing* privacy enhancement — for bucket size N , start each range request at

$$\text{floor}(X) = X - (X \bmod N)$$

instead of at the true resume height X , hash-checking and discarding blocks in $[\text{floor}(X), X]$, so that the request leaks only the bucket and absorbs reorganisations — is proposed and not implemented (ZIP 307, § “Block privacy via bucketing”). Both are ZIP-versus-deployed gaps a reader should not assume closed.

The reference client narrows the surface further: the sync loop in `zcash_client_backend` (`src/sync.rs:1--10,244--381`) exercises four shielded-sync RPCs (below) plus, when built with feature `transparent-inputs`, `GetAddressUtxosStream`, issued per account per cycle by `refresh_utxos` (`src/sync.rs:117--126,511--527`; cf. §5.3):

- `get_latest_block`, in `update_chain_tip`;
- `get_subtree_roots`, in `download_subtree_roots`;
- `get_block_range`, in `download_blocks`;
- `get_tree_state`, in `download_chain_state`.

Transaction enhancement via `GetTransaction` is listed as not yet implemented in that module (`src/sync.rs`). We return to the loop in the next section.

2.4 The RPC surface

The service defines twenty RPCs (`service.proto:221--326`); Table 3 groups them. Three are deprecated, one is testing-only, and six serve the transparent layer, outside this volume’s shielded-sync scope. The four marked ✓ are the ones the reference sync loop consumes for shielded sync (§2.3).

Remark 2.5 (Hash orders). Two byte orders cross the interface. The byte-typed fields `BlockID.hash` and `CompactBlock.hash` carry raw bytes in little-endian (protocol) order; `lightwalletd` reverses the big-endian hex it receives from the node’s RPCs (`lightwalletd, frontend/service.go:69--90`). Field `TreeState.hash`, by contrast, is big-endian display hex *text* (`service.proto:176--184,307--312`). Requests follow the same split: `TxFilter` takes protocol-order txid bytes that the server reverses into display hex for `getrawtransaction` (`frontend/service.go:403--442`), and the mempool exclude entries are protocol-order byte suffixes the server reverses into display-order prefixes (`frontend/service.go:600--747`). Every byte-typed hash on this interface is protocol order; every string-typed hash is display order.

Chain tip. The RPC `GetLatestBlock(ChainSpec)` returns a `BlockID` — height and hash of the best chain tip; `lightwalletd` proxies `getblockchaininfo` and returns the hash as little-endian raw bytes (`lightwalletd, frontend/service.go:69--90`). This is the chain tip the wallet’s confirmation accounting consumes (*Wallet Guide*, § “Confirmations, trust, and spendability”).

Group	RPC	Sync loop	Notes
Chain	GetLatestBlock	✓	tip height and hash
	GetBlock		one block, all pools
	GetBlockRange	✓	streamed range, pool filter
	GetBlockNullifiers		deprecated (v0.4.0; proto option v0.4.1)
	GetBlockRangeNullifiers		deprecated (v0.4.0; proto option v0.4.1)
Trees	GetTreeState	✓	frontiers at height/hash
	GetLatestTreeState		frontiers at the tip
	GetSubtreeRoots	✓	depth-16 subtree roots
Transactions	GetTransaction		full bytes by txid
	SendTransaction		submit raw bytes
Mempool	GetMempoolTx		compact mempool contents
	GetMempoolStream		full txs until next block
Transparent	GetAddressTxids		deprecated, misnamed
	GetAddressTransactions		
	GetAddressBalance		
	GetAddressBalanceStream		
	GetAddressUtxos		
	GetAddressUtxosStream		
Metadata	GetLightdInfo		server and chain metadata
	Ping		testing-only

Table 3: The twenty RPCs of CompactTxStreamer (`service.proto:221--326`, `lightwallet-protocol v0.4.1`). The ✓ column marks the shielded-sync RPCs exercised by the reference sync loop; with feature `transparent-inputs` the loop also issues `GetAddressUtxosStream` per account per cycle (`zcash_client_backend`, `src/sync.rs`).

Single block. The RPC `GetBlock(BlockID)` returns one `CompactBlock`, and per the v0.4.1 contract includes transaction data for *all* value pools, transparent vin/vout included — unlike `GetBlockRange`, which defaults to shielded-only (`service.proto:225--237`). The proxy `lightwalletd` serves the request from its block cache first and falls back to the node on a miss; a height above the tip returns `gRPC OutOfRange` (“block %d is newer than the latest block”), a lookup by hash returns `InvalidArgument` (“not yet implemented”, referencing PR #309), and height 0 with no hash is `InvalidArgument` (`lightwalletd, common/common.go:495--624, frontend/service.go:166--189`). Zaino accepts lookup by hash — hash takes precedence and is resolved to a height via its chain index — and builds the block with all pools included, matching the v0.4.1 contract; the proto makes hash support optional, so both behaviours are conformant, but they diverge (`zaino, zaino-state/src/backends/state.rs:1954--2058; zaino-proto/src/proto/utils.rs:310--323`).

Block ranges. The RPC `GetBlockRange(BlockRange)` streams consecutive `CompactBlocks` over the closed interval $[\min(s, e), \max(s, e)]$, in increasing height order iff $s \leq e$ and decreasing otherwise — `lightwalletd` computes $j = \text{high} - (i - \text{low})$, Zaino tests $s \leq e$ (`service.proto:250--255; lightwalletd, common/common.go:574--624; zaino, zaino-state/src/chain_index.rs:1774--1802`). An empty `poolTypes` field selects the legacy behaviour: prune each block to shielded (Sapling and Orchard) data only; clients MUST verify server capability before setting `poolTypes` non-empty (`service.proto:28--42`). Filtering prunes transaction components and then drops empty transactions, but a block whose transactions were all filtered is still returned, with empty `vtx` (`lightwalletd, common/common.go:574--624`). Zaino validates the request up front — start and end must be present and fit in `u32`; `POOL_TYPE_INVALID` or unknown pool values are `InvalidArgument` — streams through an `mpsc` channel of the configured `channel_size`, caps an ascending range at the tip and appends a trailing `OutOfRange` error after the valid blocks, and bounds production by a hard timeout of $4 \times$ the service timeout (`zaino, zaino-state/src/backends/state.rs:550--683; zaino-proto/src/proto/utils.rs:53--152`).

Nullifier variants. The RPCs `GetBlockNullifiers` and `GetBlockRangeNullifiers` — both deprecated (v0.4.0 changelog; proto option v0.4.1) in favour of `GetBlockRange` with `poolTypes` — return `CompactBlocks` reduced to shielded nullifier data: `lightwalletd` strips Sapling outputs and transparent vin/vout, reduces each Orchard action to its nullifier, and zeroes both commitment-tree sizes; `GetBlockRangeNullifiers` MUST ignore any `TRANSPARENT` member of `poolTypes`, and `lightwalletd` filters it out (`service.proto:239--268; lightwalletd, frontend/service.go:191--225, 259--305`). Zaino’s `FetchService` backend additionally resolves `GetBlockNullifiers` by hash, as for `GetBlock` (`zaino, zaino-state/src/backends/fetch.rs:929--1041`); its `StateService` backend ignores a supplied hash and reads only the height, silently serving height 0 — genesis — for a hash-identified request (`backends/state.rs:2033--2058; research/upstream-defects.md, item 40`).

Tree state. The RPC `GetTreeState(BlockID)` returns the note commitment tree state for a block specified by height or hash: network name, height, block hash as a hex string, block time, and the `saplingTree` and `orchardTree` frontiers as hex (`service.proto:176--184`). In `lightwalletd` the reply derives from the node’s `z_gettreestate` RPC, looping along `Sapling.SkipHash` when a reply carries no `FinalState`; `GetLatestTreeState` calls the same path at `getblockchaininfo`’s tip height (`lightwalletd, frontend/service.go:307--401`). The frontier is what seeds the wallet’s shard tree below its birthday, so that witness assembly never replays the chain’s prefix (*Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”). Zaino resolves hash-or-height, reports the BIP70 network name, and returns a seventh field `ironwoodTree` (hex, empty when absent) that exists only in Zaino’s and `librustzcash`’s proto forks (§2.1); `GetLatestTreeState` reuses `GetTreeState` at the current chain height (`zaino,`

zaino-state/src/backends/state.rs:2516--2576; zaino-proto/proto/service.proto:183). One stale cross-reference: the proto comment for `GetTreeState` cites “section 3.7 of the Zcash protocol specification”, while the current `protocol.tex` numbers note commitment trees as § 3.8.

Subtree roots. Shielded outputs of a pool are numbered from genesis, and each consecutive group of $2^{16} = 65536$ note commitments forms one depth-16 subtree of the pool’s commitment tree; the stream element is

$$\text{SubtreeRoot}_i = (\text{Merkle root of subtree } i, \text{ hash and height of the block containing output } (i + 1) \cdot 2^{16} - 1),$$

so that, for example, Sapling output 65535 — the last of subtree 0 — falls in block 558822 (`service.proto:191--200`; `lightwalletd, common/common.go:178--219`). The RPC `GetSubtreeRoots(GetSubtreeRootsArg)` streams these for the chosen shielded protocol from `startIndex`, with `maxEntries = 0` meaning all. The `lightwalletd` handler calls `z_getsubtreesbyindex` and then, per subtree, fetches the completing block via `GetBlock` at its end height to obtain the hash, reversed to big-endian display-order bytes, the opposite convention from `CompactBlock.hash` (Remark 4.6; `lightwalletd, frontend/service.go:832--907`). These roots are exactly the retained internal nodes from which the wallet’s shard tree assembles authentication paths without replaying every commitment (*Ironwood Guide*, §“Proof of ownership of some valid note: the commitment tree”). Zaino converts `startIndex` and `maxEntries` to `u16` and returns `InvalidArgument` when either exceeds `u16` range, where `lightwalletd` forwards the full `uint32` — bounded by design, since a depth-32 tree has at most 2^{16} depth-16 subtrees, but the wire contracts differ; it fills the completing block data via `z_get_block` per subtree, under a 4× service-timeout deadline. One Zebra-specific caveat: during Zebra’s initial subtree-index rebuild the underlying RPC returns an empty list for not-yet-rebuilt indices, where `zcashd` rebuilds before serving (`zaino, zaino-state/src/indexer.rs:429--452, 754--912`).

Full transactions. The RPC `GetTransaction(TxFilter)` returns the full serialised transaction. Only lookup by 32-byte `txid` is supported; a block-plus-index lookup returns `InvalidArgument` (“specify a `txid`, not a `blockhash+num`”). The `lightwalletd` handler converts the little-endian `txid` bytes to big-endian hex, calls `getrawtransaction` with `verbose=1`, and maps RPC failure to gRPC `NotFound` (`service.proto:44--51, 270--271`; `lightwalletd, frontend/service.go:403--442`). The privacy cost of this call — it names a `txid` of interest to the proxy — is the reason for the ZIP’s decoy mandate (§2.3).

Submission. The RPC `SendTransaction(RawTransaction)` submits via the node’s `sendrawtransaction`. Success returns `SendResponse{errorCode=0, errorMessage=txid hex}`; a node rejection is parsed from the “code: message” error string into a non-zero `errorCode` plus message — node-level rejection travels in-band, not as a gRPC error, so a transport-level success is not an acceptance (`service.proto:83--89`; `lightwalletd, frontend/service.go:454--509`). Wallet-side obligations around this call — store-then-broadcast ordering, rebroadcast, expiry — are the *Wallet Guide*’s subject (§“Store, then broadcast”).

Remark 2.6 (The `RawTransaction.height` sentinels). An original protobuf-definition error typed the field `uint64` rather than `int64`, forcing sentinel semantics (`service.proto:53--81`; `lightwalletd, common/common.go:626--661`):

$$\text{height} = \begin{cases} 0 & \text{transaction in the mempool,} \\ 2^{64} - 1 \text{ (0xffffffffffffffff)} & \text{mined on a non-main-chain fork (zcashd’s } -1\text{),} \\ h & \text{mined on the main chain at height } h. \end{cases}$$

The proto’s own comments contradict each other: the message-level comment says the field carries “the latest block height” when returned by `GetMempoolStream`, the field-level comment specifies the

sentinel mapping above, and a FIXME cites `librustzcash` issue #1484. The deployed servers realise both readings — `lightwalletd` streams 0, Zaino streams the current best-tip height (§2.8). We flag the `mempool-height` contract as unresolved (`research/upstream-defects.md`, which tracks this as the fourth known defect, already filed as `zcash/librustzcash#1484`).

Mempool contents. The RPC `GetMempoolTx(GetMempoolTxRequest)` streams `CompactTx` for the current mempool contents, possibly a few seconds stale. The request may carry `exclude_txid_suffixes`: `txid` byte-suffixes of any length up to 32 bytes, in client-side protocol order; the server reverses and hex-encodes each into a display-order prefix, and a mempool transaction is excluded iff its `txid` matches a prefix that matches *exactly one* mempool `txid` — when two or more `txids` match, none of the matching transactions are excluded, and unmatched entries are ignored (`service.proto:157--174,289--301`; `lightwalletd`, `frontend/service.go:600--747`). Field 2 of the request is reserved for a future `exclude_txid_prefixes`. An empty `poolTypes` defaults to Sapling-plus-Orchard pruning; `POOL_TYPE_INVALID` is rejected. The `lightwalletd` implementation backs the RPC with a process-global cache — `mempoolMap` from `txid` to `CompactTx`, plus `mempoolList` — refreshed at most every 2 s from `getrawmempool` plus per-transaction `getrawtransaction verbose=0`, reusing already-fetched entries across refreshes; mempool `CompactTx`s are built with height 0 and no fee (`lightwalletd`, `frontend/service.go:589--708`). Zaino enforces the same 32-byte suffix bound and the same exactly-one-match exclusion rule, parses transactions from stored serialised bytes, and prunes by the requested pool types before streaming (`zaino`, `zaino-state/src/backends/state.rs:2302--2429`, `chain_index/mempool.rs:381--417`).

Mempool stream. The RPC `GetMempoolStream(Empty)` streams full `RawTransactions` of the current mempool and keeps the stream open, closing it when a new block is mined (`service.proto:303--305`). The `lightwalletd` implementation is process-global state — `g_txList` in arrival order, a `g_txidSeen` set, `g_lastBlockChainInfo` — polled at most every 2 s via `getBlockchaininfo` and `getrawmempool`; the first stream to observe a tip-hash change resets the shared state and all open streams terminate; each per-client loop sleeps 200 ms between sends; transactions mined since listing (`height ≠ 0`) are skipped; streamed transactions carry `height = 0` (`lightwalletd`, `common/mempool.go:14--152`, `frontend/service.go:581--587`). Zaino streams `height = current best-tip height`, closes the stream when its mempool status flips to `Syncing` on a tip change, and additionally imposes a hard 6× service-timeout deadline — 180 s at defaults — where `lightwalletd` keeps the stream open until the next block regardless of duration; a long block interval can therefore end a Zaino stream with a deadline error where the proto’s documented close-on-new-block semantics would hold (`zaino`, `zaino-state/src/backends/state.rs:2433--2510`, `chain_index/mempool.rs:419--480`).

Server metadata. The RPC `GetLightdInfo(Empty)` returns server metadata plus chain state assembled from `getinfo` and `getBlockchaininfo`: `version`, `vendor` (“ECC LightWalletD”), `taddrSupport` `true`, `chain` name, the Sapling activation height — looked up via the Sapling consensus branch ID `0x76b809bb` in the upgrades map — the chain tip’s consensus branch ID, `block height`, `build` and `git` fields, `estimatedHeight` (less than the tip while the node itself syncs), `donation address`, and the next pending upgrade’s name and height (`service.proto:97--118`; `lightwalletd`, `common/common.go:261--319`). At the pinned revision the proxy does *not* populate the proto’s `lightwalletProtocolVersion` field 18; current `lightwalletd` 0.5.4 reports “v0.5.0”. Zaino reports `vendor` “ZingoLabs ZainoD”, hard-codes `lightwallet_protocol_version = “v0.5.0”` — then an untagged version but now the canonical tagged release (`research/upstream-defects.md`, item 3; `zaino-state/src/backends/fetch.rs:2097`, `state.rs:2776`) — defaults the Sapling activation height to 1 on regtest, and carries the connected Zebra node’s build information in the `zcashdBuild/zcashdSubversion` fields (`zaino`, `zaino-state/src/backends/state.rs:2724--2793`). Both sides are flagged in §2.8.

Ping. The RPC Ping is testing-only; `lightwalletd` disables it unless started with the flag `--ping-very-insecure`, because the call lets a client create arbitrarily many concurrent server threads — a resource-exhaustion risk (`service.proto:324--325`; `lightwalletd, frontend/service.go:920--936`). Zaino returns `gRPCUnimplemented(zaino, zaino-state/src/backends/state.rs:2782--2792)`.

Transparent-address RPCs. The six remaining RPCs (`GetAddressTxids` — deprecated and misnamed, it returns transactions — `GetAddressTransactions`, `GetAddressBalance`, `GetAddressBalanceStream`, `GetAddressUtxos`, `GetAddressUtxosStream`) proxy the node’s insight-explorer-derived RPCs `getaddresstxids`, `getaddressbalance`, and `getaddressutxos`; they serve the transparent layer and fall outside this volume’s shielded-sync scope. For these, `lightwalletd` validates t-addresses against the regex `\At[a-zA-Z0-9]{34}\z` and caps each `GetAddressTransactions` inner fetch loop with a 30-second context timeout (`lightwalletd, frontend/service.go:55--164, 511--579, 749--830`). Zaino caps the request’s address list at `UTXO_MAX_ADDRESSES = 1000` (`InvalidArgument` beyond) — a server-side fan-out bound `lightwalletd` lacks — and honours `start_height` and `max_entries` (0 meaning unlimited) as response-size bounds (`zaino, zaino-state/src/backends.rs:29--45, backends/state.rs:2591--2719`).

2.5 `lightwalletd`: the deployed proxy

The deployed server is a Go proxy: each gRPC handler in `frontend/service.go` translates its request into node JSON-RPC calls through the indirect function `common.RawRequest`. It supports both `zcashd` and `zebrad` backends, detecting the backend from the node’s subversion string (`/Zebra:` versus `/MagicBean:`); a `zcashd` backend must have the `lightwalletd` or `insightexplorer` experimental feature enabled or `lightwalletd` exits; the default assumed node is `zebrad` (`lightwalletd, cmd/root.go:200--263, common/common.go:27--35, 61--64`).

Ingestion. A single `BlockIngestor` goroutine polls `getbestblockhash` every 2 s; when the tip differs it fetches the next block via *two* `getblock` calls — first `verbose=1` for the txids and hash, a workaround because the built-in parser miscomputes v5 transaction IDs (issue #392), then the raw block by that hash, so a reorganisation between the calls cannot mix two blocks. A `prevHash` mismatch against the cache tip is treated as a reorganisation and exactly one block is dropped (`Reorg(height-1)`) per iteration; a `getblock` failure retries after 8 s (`lightwalletd, common/common.go:321--493`). The 120 s sleep (`common/common.go:483--489`) runs only when the cache is empty (`height = GetFirstHeight()`) and the node returned no block at that height; with the cache now starting at height 0 (`cmd/root.go:296--299`) the branch is reachable only against a node that cannot yet serve genesis, and its log line (“Waiting for ... Sapling activation height”) is stale wording, not stale behaviour.

The block cache. The only persistent state `lightwalletd` maintains is the compact-block cache: two append-only files per chain, `<data-dir>/db/<chain>/lengths` and `.../blocks`. With $d = \text{protobuf}(\text{CompactBlock})$ the entry for height h is

$$\text{FNV1a}_{64}(\text{LE}_{64}(h) \parallel d) \parallel d \quad \text{in blocks}, \quad \text{LE}_{32}(|d|) \quad \text{in lengths},$$

with the offset table reconstructed on startup and valid lengths constrained to $74 \leq |d| \leq 4,000,000$ bytes; corruption — a bad checksum, an impossible length, a partial entry — triggers automatic cache clearing and redownload (`lightwalletd, common/cache.go:21--230`). The cache now starts at height 0, previously Sapling activation, because compact blocks carry transparent data (`lightwalletd, cmd/root.go:265--300`). All other responses are proxied per-request; the mempool caches of §2.4 are in-memory only; no per-client state is kept.

Operational defaults. gRPC binds 127.0.0.1:9067 and HTTP 127.0.0.1:9068 (Prometheus /metrics); TLS is required — the server exits if the certificate or key is unloadable — unless `--no-tls-very-insecure` (plaintext) or `--gen-cert-very-insecure` (self-signed); the data directory defaults to `/var/lib/lightwalletd`; `--nocache` disables the disk cache; `--sync-from-height` and `--redownload` control cache rebuild; the darkside mock mode (`--darkside-very-insecure`) auto-terminates after 30 minutes (`lightwalletd, cmd/root.go:139--180, 375--424, 498--499`).

2.6 Zaino: the successor indexer

Zaino is the successor indexer, in Rust: `zainod` is the daemon, `zaino-serve` implements `CompactTxStreamer` over tonic, `zaino-state` holds the chain index and backends, `zaino-fetch` is the JSON-RPC client, and `zaino-proto` vendors the protos. The stated motivations are the `zcashd` deprecation, a Rust stack alongside `librustzcash` and Zebra, and a clean indexer/validator trust boundary: indexing functionality moves out of the validator into Zaino, with Zebra exposing its `ReadStateService` for direct read access (`zaino, README.md, § “Motivations”, § “Project Structure”; docs/internal_spec.md`).

Backends. Configuration `backend = 'fetch' | 'state'` selects one of two chain-fetch backends. Backend `StateService` is backed by Zebra’s `ReadStateService` plus a `TrustedChainSync` (`init_read_state_with_syncer`): it reads Zebra’s state store directly, and at startup blocks until the syncer’s tip height *and* hash match the validator’s JSON-RPC tip — hash compared because height alone is ambiguous during a reorganisation. Backend `FetchService` is backed by the node’s JSON-RPC interface (`zcashd` back-compatibility, or a remote Zebra) and is marked “[deprecated]: Will be eventually replaced by `BlockchainSource`”. Both feed the same `NodeBackedChainIndex` (`zaino, zaino-state/src/backends/state.rs:1,117--260, backends/fetch.rs:1,83--100`). The shipped example configuration nonetheless selects `backend = 'fetch'` — the deprecated backend — and the repository documentation does not say which backend is the recommended production deployment (`zaino, docs/example_configs/zainod.toml`); we flag the question as open.

The chain index. Server-side state is the `ChainIndex`, with three components (`zaino, docs/internal_spec.md, § “Zaino-State”; zaino-state/src/chain_index.rs:1--100`). A `Mempool` — a dashmap broadcast map keyed by txid, whose sync task polls `get_best_block_hash` and on a tip change clears the map and notifies subscribers (`chain_index/mempool.rs`). A `NonFinalisedState` — an in-memory `ArcSwap` snapshot of the top blocks of all chains. A `FinalisedState` — all blocks below the seam — served either by a versioned LMDB-backed persistent database (`v1`) or, with `ephemeral_finalised_state`, by a stateless passthrough to the validator; large syncs and version migrations run in the background while serving continues (`chain_index/finalised_state.rs:1--80`). The seam is

$$\text{floor}(t) = t - D_{\text{op}} \quad (\text{saturating at genesis}),$$

where t is the tip height and $D_{\text{op}} = \text{OPERATIONAL_NFS_DEPTH} = \text{MAX_BLOCK_REORG_HEIGHT} + 1 = 1001$ — Zebra’s reorganisation limit is local node policy, not consensus; the finalised database serves heights $\leq \text{floor}(t)$, the in-memory state serves heights above it, and in-memory retention is hard-capped at $\text{MAX_NFS_DEPTH} = D_{\text{op}} + 10 = 1011$ blocks below the tip (`zaino, zaino-common/src/consensus.rs:16--17, chain_index/non_finalised_state.rs:23--58; zebra, zebra-chain/src/parameters/constants.rs:30`).

Serving. Each gRPC handler delegates to a `LightWalletIndexer` trait implemented by both backend subscribers; streaming responses are tokio mpsc channels wrapped in typed streams (`zaino, zaino-serve/src/rpc/grpc/service.rs:159--320`). The gRPC server may

bind a public address only with TLS configured — rejected at startup otherwise — and the default listen address is 127.0.0.1:8137; Zaino additionally serves a zcashd-style JSON-RPC server, unencrypted and restricted to loopback or private bind addresses by default (zaino, README.md, § “Server network exposure”; zainod/src/config.rs:239). Service defaults: timeout 30 s, channel_size 32; stream deadlines are 4× the timeout (120 s), and 6× for GetMempoolStream (180 s) (zaino, zaino-common/src/config/service.rs:8--20). Until the non-finalised snapshot exists, GetLatestBlock and GetMempoolStream return an UnavailableNotSyncedEnough error where lightwalletd would proxy the node’s answer immediately; a code TODO says this “probably shouldn’t be an error”, so the startup behaviour may change (zaino, zaino-state/src/backends/state.rs:1940--1951).

Compatibility. Zaino’s integration test suite includes a `client_rpcs` partition that compares its LightWallet gRPC outputs against lightwalletd’s, and the project has committed to serving all Zcash JSON-RPC services required by non-validator clients, superseding zcashd in that role (zaino, docs/internal_spec.md, § “Zaino-Testutils and Integration-Tests”; docs/rpc_api.md).

Current-head update. The detailed account above deliberately describes the pinned July revision. At zaino afd609e (2026-08-28), validator support is Zebra-only and the old StateService/ FetchService split has been replaced by one ZebraValidator composite. JSON-RPC is always present; direct ZebraReadStateAdapter database access is an optional accelerator. Configuration now names the modes `direct` and `rpc`, while retaining `state` and `fetch` only as compatibility aliases (zaino-source-zebra/src/lib.rs; zainod/src/config.rs). Consequently, the behaviour-by-backend claims and divergence table below are a reproducible snapshot, not a claim about current Zaino internals.

2.7 Zebra’s in-process server

Zebra main 0a43e0e contains a third implementation, added at 85ce9a3 on 2026-08-18 after the zebra-d 6.3.0 release. It runs CompactTxStreamer inside zebra-d: most handlers call Zebra’s JSON-RPC methods in process, while compact blocks and mempool streams read the state and mempool services directly. The surface defines handlers for all twenty RPC names; nineteen are implemented, while the testing-only Ping deliberately returns Unimplemented (zebra-rpc, src/lightwalletd/methods.rs and server.rs).

This server is unreleased at the cited head, experimental, and disabled unless `rpc.lightwalletd_listen_addr` is configured. It serves plaintext HTTP/2 without TLS or authentication; Zebra therefore directs operators to bind it to loopback or a private address and terminate TLS in a reverse proxy (zebra-rpc, src/config/rpc.rs). Its existence invalidates the former statement that lightwalletd and Zaino were the only maintained server implementations, but it does not retroactively alter the pinned two-server comparison below.

2.8 Divergence register

Table 4 collects the contract-level points at which lightwalletd and Zaino at the pinned revisions differ from each other or from the normative texts; each row is developed, with citations, in the paragraph of §2.4–§2.6 that covers the RPC. The ZIP-versus-deployment gaps — the message sketch, partial header validation, unimplemented bucketing — are flagged in §2.1 and §2.3; the proto forks in §2.1.

Two rows deserve emphasis. The mempool-height row is a live specification bug: the proto’s two comments cannot both be satisfied, the deployed servers realise one reading each, and the FIXME in the canonical file (librustzcash issue #1484) records the conflict without resolving it; a client must therefore treat the height of a streamed mempool transaction as carrying no portable information beyond “not yet mined”. The fee row is a silent one: nothing on the wire distinguishes “fee is zero” from “fee not provided”, so the proto’s conditional language is, in deployed terms, a constant.

Contract point	lightwalletd	Zaino
GetBlock(Nullifiers) by hash (optional in proto)	InvalidArgument, “not yet implemented” (PR #309)	supported (hash takes precedence), but StateService ignores the hash in GetBlockNullifiers
Stream deadlines (proto: GetMempoolStream closes on new block)	none; open until next block	hard 4×/6× timeout (120 s/180 s at defaults)
RawTransaction.height in GetMempoolStream (proto comments contradict; issue #1484)	0	current best-tip height
GetSubtreeRoots argument range (proto: uint32)	forwards full range to node	InvalidArgument above u16::MAX
CompactTx.fee (proto: “present if server can provide”)	always 0 (parser TODO)	always 0 (fee = None)
lightwalletProtocolVersion (LightdInfo field 18, v0.4.0)	never populated at 61fee32	hard-coded “v0.5.0” before that canonical tag existed
Behaviour before server is synced	proxies the node immediately	UnavailableNotSyncedEnough on GetLatestBlock, GetMempoolStream
Ping	disabled unless --ping-very-insecure	Unimplemented
Transparent-address fan-out	unbounded address lists	capped at 1000 addresses

Table 4: Pinned-revision divergences across the CompactTxStreamer contract. Citations for each row appear in the covering paragraphs of §2.4–§2.6.

3 The scanning pipeline

A light wallet holds no chain. Everything it knows — which notes it owns, which of them are spent, and where each sits in the note commitment tree — it reconstructs by scanning a stream of *compact blocks* supplied by an indexing server. This section develops the deployed reconstruction pipeline of `librustzcash`’s `zcash_client_backend` and `zcash_client_sqlite` crates: the compact input format (§1), the account birthday that bounds the scan (§3.2), the priority queue that orders it (§3.3), chain-tip tracking (§3.4), the per-range scanning machinery (§3.5), persistence (§3.6), reorganisation recovery (§3.7), and the reference loop that assembles the parts (§3.8).

The cost shape of the pipeline deserves stating first. For every block in a suggested range, every compact output and action of every transaction is trial-decrypted against every prepared incoming viewing key of every tracked account — two ZIP-32 scopes per pool per account (§3.5) — so synchronisation work scales with global chain traffic multiplied by tracked keys, and is independent of the wallet’s own activity; restore-from-seed replays the whole computation from the birthday (`zcash_client_backend`, `src/data_api/chain.rs:598--699`). This is precisely the deployed cost centre that the *Tachyon Guide*’s proof-carrying wallet is designed to remove (§“The proof-carrying wallet”, cost-centre paragraph “Trial decryption of the chain”).

3.1 The compact stream

ZIP 307 (Status: Draft) defines the light-client protocol for payment detection. Its *compact output* retains, of a full shielded output, the 32-byte note commitment, the 32-byte ephemeral key, and the first 52 bytes of the note ciphertext:

$$\underbrace{32}_{\text{commitment}} + \underbrace{32}_{\text{epk}} + \underbrace{52}_{\text{ciphertext prefix}} = 116 \text{ bytes,}$$

an ~80% reduction against the full 580-byte ciphertext plus 32-byte ephemeral key; its *compact spend* is the 32-byte nullifier alone, an ~90% reduction against the 384-byte Sapling Spend description (ZIP 307 §§“Output Compression”, “Spend Compression”). Truncating at 52 bytes discards the AEAD tag, so a compact decryption authenticates differently from a full one: the wallet recomputes the note commitment from the decrypted compact plaintext and compares it with the transmitted commitment, trusting the transaction assembler for integrity (ZIP 307 §“Compact Stream Format”). The cryptography of trial decryption — key agreement, KDF, the 52-byte compact note plaintext, the commitment recomputation — is the *Ironwood Guide*’s material (§“Transmission and detection of a note: encryption, trial decryption, and synchronisation”); this volume treats it as a black box mapping a (compact output, incoming viewing key) pair to a note or $\perp$.

The deployed wire format. The format in production has evolved well beyond ZIP 307’s message definitions. A block is

```
CompactBlock { reserved 1; height=2; hash=3; prevHash=4; time=5;
               header=6; vtx=7; chainMetadata=8 }
```

where field 1 (`protoVersion`) has been removed and `ChainMetadata` carries the running per-pool note commitment tree sizes `saplingCommitmentTreeSize=1`, `orchardCommitmentTreeSize=2`, and `ironwoodCommitmentTreeSize=3` — the field on which position tracking (§3.5) depends. Each `CompactTx` carries its index, txid, fee, Sapling spends and outputs, Orchard actions, Ironwood actions (`ironwoodActions`, field 9, reusing the `CompactOrchardAction` message), and transparent vin/vout (`zcash_client_backend`, `lightwallet-protocol/walletrpc/compact_formats.proto`, vendored at `librustzcash e30517e4`).

Remark 3.1 (Normative anchoring). ZIP 307’s message layout — a `CompactBlock` with an embedded `BlockID`, messages `CompactSpend` and `CompactOutput` — does not match the deployed format,

and the ZIP remains Draft and Sapling-only. No ZIP specifies the deployed compact format or service: ChainMetadata tree sizes, CompactOrchardAction, ironwoodActions, GetTreeState, and GetSubtreeRoots exist only in the zcash/lightwallet-protocol proto files, which we therefore cite as the specification of record. Ironwood’s compact-format extensions are likewise absent from ZIP 2005, which covers quantum recoverability, not light-client synchronisation (compact_formats.proto; ZIP 307; ZIP 2005).

The fee field. The fee of a CompactTx is calculable by a stateless server only for transactions without transparent inputs, as

$$\text{fee} = \text{valueBalanceSapling} + \text{valueBalanceOrchard} + \text{valueBalanceIronwood} \\ + \sum v_{\text{PubNew}} - \sum v_{\text{PubOld}} - \sum t_{\text{Out}};$$

with transparent inputs the fee depends on prior-transaction outputs a stateless server cannot provide (compact_formats.proto:63--69).

Service surface. The reference synchroniser drives four RPCs of the CompactTxStreamer service: GetLatestBlock(ChainSpec) → BlockID; GetBlockRange(BlockRange) → stream CompactBlock, addressed by heights only, with an optional poolTypes pruning filter that clients MUST verify the server supports; GetTreeState(BlockID) → TreeState, returning network, height, hash, time, and the hex-serialised Sapling, Orchard, and Ironwood commitment trees; and GetSubtreeRoots(GetSubtreeRootsArg) → stream SubtreeRoot, returning complete-subtree roots with their completing block hash and height, selected by enum ShieldedProtocol { sapling=0; orchard=1; ironwood=2 } (lightwallet-protocol/walletrpc/service.proto:24--321).

Remark 3.2 (Deployment divergence at NU6.3). The deployed lightwalletd tree (commit 61fee32e, 2026-06-25) contains no Ironwood support: its walletrpc protos and Go code lack ironwoodActions, ironwoodCommitmentTreeSize, ironwoodTree, and ShieldedProtocol.ironwood. The protos vendored by librustzcash (commit e30517e4) define all four; Zaino (commit 0e057e22) defines the first three but not ShieldedProtocol.ironwood (§2.1), so its GetSubtreeRoots rejects an Ironwood request with InvalidArgument (zaino-state/src/indexer.rs:761--768). The reference synchroniser unconditionally requests Ironwood subtree roots (zcash_client_backend, src/sync.rs:279--296); an NU6.3-aware wallet therefore requires an updated server — at these pins neither lightwalletd nor Zaino serves the full surface.

3.2 The account birthday

Definition 3.3 (Account birthday). Type AccountBirthday pairs a prior_chain_state: ChainState with a recover_until: Option<BlockHeight>. The *birthday height* is

$$\text{birthday} := \text{priorChainState.height} + 1,$$

the height of the first block to be scanned (zcash_client_backend, src/data_api.rs:3013--3106). A ChainState holds a block height, a block hash, and the final note commitment tree frontier of each pool: Sapling (depth sapling::NOTE_COMMITMENT_TREE_DEPTH = 32), Orchard, and Ironwood — the Ironwood tree is Orchard-shaped (MerkleHashOrchard, the same depth) but the pool is distinct and keeps its own tree (zcash_client_backend, src/data_api/chain.rs:504--596).

Constructor AccountBirthday::from_treestate builds the birthday from a lightwalletd TreeState for the last block *prior* to the birthday height. The conversion (TreeState::

to_chain_state) hex-decodes the block hash — reversing the bytes, since zcashd renders block hashes byte-reversed in hex — parses the height, and converts the hex-serialised Sapling, Orchard, and Ironwood trees into per-pool frontiers; an empty tree field yields an empty tree (zcash_client_backend, src/data_api.rs:3055--3073; src/proto.rs:367--463).

Choosing a birthday. For a new wallet the crate documentation directs constructing the birthday from the treestate at tip — 100: placing the birthday below the pruning depth guarantees that no reorganisation can mine a transaction intended for the wallet *below* its birthday and so cause funds to be missed (zcash_client_backend, src/data_api.rs:3199--3211). For a restored wallet the birthday is historical, and the recover_until height marks the chain tip at restoration time: the wallet is *in recovery* until no unscanned ranges remain between the birthday height and recover_until (exclusive). The field exists to avoid confusing shifts of balance and spendability while restore-from-seed is in flight — spendability policy itself is the *Wallet Guide*’s ConfirmationsPolicy (§“Transaction construction”; §“Broadcast, expiry, and the transaction lifecycle”), which this volume does not restate. Progress reporting follows the same split: type Progress carries two scanned-notes/notes-on-chain ratios, one over the recovery window (birthday to recovery height) and one over the scan window (recovery height to tip); either denominator may be zero (zcash_client_backend, src/data_api.rs:3042--3047, 788--831).

The wallet birthday. The birthday of the *wallet* is the minimum birthday over its accounts — literally SELECT MIN(birthday_height) FROM accounts in the SQLite backend — and adding an account whose birthday lies below the current chain tip triggers a rescan of all blocks at and above that height (zcash_client_sqlite, src/wallet.rs:2967--2978; zcash_client_backend, src/data_api.rs:3153--3156). Function add_account persists the birthday height together with the Sapling and Orchard frontier tree sizes, calls rewind_to_chain_state targeting the birthday’s prior chain state — installing a Historic-priority rescan range above birthday — 1 up to the wallet’s known chain tip while preserving higher-priority queue entries — and finally inserts an Ignored-priority range covering [saplingActivation, birthday) (zcash_client_sqlite, src/wallet.rs:432--628, 4056--4224).

3.3 The scan queue

The wallet does not scan the chain linearly; it maintains a priority queue of height ranges and always works on the most urgent.

Definition 3.4 (Scan range). A ScanRange is a half-open range [start,end) of block heights paired with a ScanPriority, with helpers truncate_start, truncate_end, and split_at (zcash_client_backend, src/data_api/scanning.rs:31--125). The SQLite backend stores the queue in table scan_queue as rows (block_range_start, block_range_end, priority) with both bounds UNIQUE and a CHECK constraint start < end (zcash_client_sqlite, src/wallet/db.rs:1017--1028).

Remark 3.5 (Six live priorities plus one vestigial). We present the ladder as six live priorities and one vestige. Priority OpenAdjacent is declared and mapped to SQL code 30, but no code path in the current librustzcash tree ever assigns it: a search at commit e30517e4 finds only the enum declaration, the code mapping, and two doc-comment mentions (zcash_client_backend, src/data_api/scanning.rs:20--21). It still participates in the ordering, so we retain it in Table 5.

Suggesting work. Method WalletRead::suggest_scan_ranges returns queue entries in descending priority order. Its documented contract has two clauses: Verify ranges must be scanned before all others, to avoid continuity errors under reorganisation; and the compact-block source must

Priority	Code	Meaning (doc comment)
Ignored	0	Lowest priority; never scanned.
Scanned	10	Already scanned; not re-scanned.
Historic	20	Advance the fully-scanned height.
OpenAdjacent	30	Ranges adjacent to heights at which the user opened the wallet (vestigial; Remark 3.5).
FoundNote	40	Complete tree shards adjacent to found notes.
ChainTip	50	Complete the latest shard.
Verify	60	Previously scanned range that must be re-verified as still on the main chain (highest).

Table 5: The ScanPriority ladder, ascending, with its SQLite codes (zcash_client_backend, src/data_api/scanning.rs:11--29; zcash_client_sqlite, src/wallet/scanning.rs:34--45).

supply note commitment tree size metadata (zcash_client_backend, src/data_api.rs:1995--2010). The SQLite implementation orders by priority DESC, block_range_end DESC — nearest the tip first within a priority — and filters to priorities at or above Historic (zcash_client_sqlite, src/wallet/scanning.rs:61--90; zcash_client_sqlite, src/lib.rs:958--960).

Remark 3.6 (Contract, not type). The guarantee that a Verify range arrives first is a documented contract of the trait realised by the SQLite ORDER BY; nothing in the type system enforces it, and a third-party WalletRead implementation could violate it. We state it as a trait obligation and cite the one deployed implementation that discharges it (zcash_client_backend, src/data_api.rs:1995--2010; zcash_client_sqlite, src/wallet/scanning.rs:61--90).

Queue maintenance. Function `replace_queue_entries` performs every insertion as a spanning-tree merge: existing entries overlapping or adjacent to the query range are deleted and re-inserted together with the new entries, and overlaps are resolved by a dominance rule —

- an inserted Verify or Scanned range always replaces what it overlaps;
- otherwise an existing Scanned range wins (unless a forced rescan is requested);
- otherwise the higher priority wins, and equal priorities coalesce into one range.

Gaps between disjoint ranges are filled at Historic priority, so the queue always tiles a contiguous span (zcash_client_sqlite, src/wallet/scanning.rs:145--222; zcash_client_backend, src/data_api/scanning/spanning_tree.rs:65--153).

3.4 Tracking the chain tip

Function `update_chain_tip` feeds the queue whenever the wallet learns a new tip height tip. Two derived heights recur, with range upper bounds exclusive throughout:

$$\text{chainEnd} := \text{tip} + 1, \quad \text{stable} := \text{tip} - \text{PRUNING_DEPTH},$$

where `PRUNING_DEPTH` = 100 blocks and heights at or below `stable` are treated as not subject to reorganisation (zcash_client_sqlite, src/wallet/scanning.rs:522--523, 582--584; zcash_client_sqlite, src/lib.rs:169). The SQLite implementation proceeds as follows (zcash_client_sqlite, src/wallet/scanning.rs:488--660):

- it is a no-op if the tip is below Sapling activation;
- it is a no-op if the new tip is below the prior maximum scanned height — the caller has caught the chain mid-reorganisation, and the discontinuity error path of §3.7 will resolve it;

- (c) it computes a *tip shard entry* at ChainTip priority covering every pool’s last incomplete shard,

$$\left[\max(\text{birthday}, \min_{\text{pools}} \max(\text{subtreeEndHeight})), \text{chainEnd} \right),$$

taking the minimum over the Sapling, Orchard, and Ironwood shards tables and emitting the entry only if that minimum lies below chainEnd;

- (d) it computes a *tip entry*: with no scanned blocks, a Historic range from the wallet birthday to chainEnd (or Ignored from Sapling activation when no accounts exist); with no shard meta-data (linear scanning), Historic from maxScanned + 1; if maxScanned > stable (steady state), ChainTip from maxScanned + 1 to chainEnd; otherwise a Verify range

$$\left[\min \text{Unscanned}, \min(\text{stable} + 1, \min \text{Unscanned} + \text{VERIFY_LOOKAHEAD}) \right),$$

where $\min \text{Unscanned} := \max \text{Scanned} + 1$.

The Verify discipline is documented at the same site: because Verify outranks everything, the connectivity check — and any rewind it forces — runs before any ChainTip range from this or an earlier tip update is scanned. Constant VERIFY_LOOKAHEAD = 10 blocks is chosen as roughly 12.5 minutes at 75 s per block, a window within which a user may plausibly have received a transaction without leaving the wallet open; and the Verify range is clamped to the stable region so that it cannot itself be reorganised away and interfere with later Verify ranges (zcash_client_sqlite, src/wallet/scanning.rs:583--634; zcash_client_sqlite, src/lib.rs:172).

Remark 3.7 (Two deliberate asymmetries). First, when maxScanned = stable the Verify range above is empty: after a long offline period no Verify entry need exist, and connectivity is then checked only implicitly, by the previous-hash comparison of §3.7 when the adjacent range is scanned (zcash_client_sqlite, src/wallet/scanning.rs:625--634). Second, case (b) above means update_chain_tip deliberately does *nothing* when the tip moves backwards; reorganisation detection lives in the scanner, not in tip updates, and recovery in that case waits for a fetch failure or continuity error (zcash_client_sqlite, src/wallet/scanning.rs:488--660).

3.5 Scanning a range

Function scan_cached_blocks scans one contiguous range; it takes the network parameters, the block source, the wallet database, a starting height from_height with its prior chain state from_state, and a block-count limit. It panics unless fromHeight = fromState.height + 1 — the caller must supply the chain state immediately prior to the range (zcash_client_backend, src/data_api/chain.rs:598--699).

Keys. The scanner loads all tracked unified full viewing keys and builds a ScanningKeys set (ScanningKeys::from_account_ufrks). Per account it registers: the Sapling diversifiable full viewing key’s (External, Internal) pairs of incoming viewing key and nullifier-deriving key; the Orchard full viewing key’s (External, Internal) incoming viewing keys; and the *same* Orchard keys a second time under the Ironwood note-encryption domain — Ironwood notes are Orchard-shaped and decrypt under Orchard viewing keys, but carry version-3 note plaintexts, so IronwoodDomain is distinct from OrchardDomain, which accepts only version-2 plaintexts. Each incoming viewing key is tagged (AccountId, zip32::Scope) (zcash_client_backend, src/scanning.rs:213--256, 352--434; the Ironwood pool is ZIP 2005, “Ironwood Quantum Recoverability”, Status: Proposed, NU6.3). A scanning key derives a note’s nullifier only when it carries nullifier-deriving capability: trait method ScanningKeyOps::nf(note, position) returns None for bare incoming viewing keys, so notes detected with an IVK alone never enter the tracked-nullifier set and their spends cannot be detected (zcash_client_backend, src/scanning.rs:34--66, 168--186).

Batched trial decryption. The scanner makes two passes over the downloaded blocks. The first pass feeds every compact output into a set of BatchRunners — one per note-encryption domain: Sapling, Orchard, Ironwood — and flushes. A runner accumulates (domain, output) pairs across transactions and blocks into a Batch shared by all prepared incoming viewing keys of its domain; when the accumulated output count reaches the batch-size threshold (100 outputs in `scan_cached_blocks`) the batch is spawned on the global rayon thread pool (`rayon::spawn_fifo`) and a fresh batch begins. Results are delivered per (block hash, txid) over flume channels, and `collect_results` blocks until the relevant batch has run (`zcash_client_backend`, `src/scan.rs:194--220,461--626`; `src/scanning/compact.rs:101--238`; `src/data_api/chain.rs:642`). Within a batch, `zcash_note_encryption::batch::try_compact_note_decryption` batches the cryptography: `D::batch_epk` parses and prepares all ephemeral keys at once, shared secrets are computed for every (ivk, epk) pair — the scalar multiplications themselves cannot be batched — and `D::batch_kdf` derives all symmetric keys in one pass before per-output trial decryption of the 52-byte compact ciphertext (`zcash_note_encryption`, `src/batch.rs:42--85`); the underlying primitives are again the *Ironwood Guide*’s (§“Transmission and detection of a note: encryption, trial decryption, and synchronisation”).

The second pass. The scanner then reads the prior block metadata and the wallet’s unspent-nullifier set (`Nullifiers::unspent`, per pool), and calls `scan_block_with_runners` per block, accumulating summary counts, updating the in-memory nullifier set after each block (`Nullifiers::update_with`: nullifiers spent in the block are removed, nullifiers of newly received full-viewing-key-decrypted notes are added), and chaining each block’s metadata into the next — so a spend of a note received earlier in the same batch is detected before anything is persisted (`zcash_client_backend`, `src/data_api/chain.rs:598--699`; `src/scanning.rs:436--600`).

Within a block, spend detection (`find_spent`) compares each revealed nullifier against the tracked unspent set in constant time — a subtle `CtOption` fold over the whole set; an in-code TODO notes the resulting $O(|\text{nullifiers}| \times |\text{spends}|)$ cost and questions whether constant-time operation is warranted. Nullifiers matching no tracked note are recorded as *unlinked* in a per-transaction nullifier map, in case they match a note found later when scanning an earlier range (`zcash_client_backend`, `src/scanning.rs:826--870`). Note detection (`find_received`) assigns each decrypted note its absolute tree position, $\text{position}(o) = \text{treeSize}_{\text{tx start}} + \text{idx}(o)$, and computes a shardtree retention for every commitment in the block:

$$\text{retention}(c) = \begin{cases} \text{Checkpoint}\{\text{Marked}\} & \text{last in block, wallet's} \\ \text{Checkpoint}\{\text{None}\} & \text{last in block only} \\ \text{Marked} & \text{wallet's only} \\ \text{Ephemeral} & \text{otherwise,} \end{cases}$$

and a note is flagged `is_change` when the receiving account also spent notes in the same transaction (`zcash_client_backend`, `src/scanning.rs:804--824,872--994`).

Position tracking. Positions require tree sizes, which the compact stream must supply. Per pool, the tracker (`PositionTracker::for_compact_block`) reconstructs the block’s starting tree size: from the prior block’s metadata when available; else from the block’s `ChainMetadata` final size minus the block’s output count (error `TreeSizeInvalid` on underflow — the proto-default zero is caught here); else 0 below the pool’s activation height (Sapling, NU5, and NU6.3 for Ironwood respectively); else `TreeSizeUnknown`. The tracker advances by each transaction’s per-pool output count, and at end of block its computed size is checked against `ChainMetadata` — a mismatch is the continuity error `TreeSizeMismatch` (`zcash_client_backend`, `src/scanning/compact.rs:577--791`). Malformed input from the untrusted server is likewise caught up front: wrong-length nullifier bytes and malformed compact outputs or actions yield `ScanError::EncodingInvalid` — naming pool, txid, and index — rather than a panic (`zcash_client_backend`, `src/scanning/compact.rs:174--234,312--390`).

Output. Each block yields a `ScannedBlock`: height, hash, block time, the wallet-relevant `WalletTx` list — a transaction is retained only if it has at least one wallet spend or output in any pool — and per-pool `ScannedBundles` holding the final tree size, the (commitment, retention) list, and the nullifier map. Transparent data in a `CompactTx` is not yet scanned (librustzcash issue #2187) (`zcash_client_backend`, `src/scanning/compact.rs:240--575`; `src/data_api.rs:2520--2692`). The per-call `ScanSummary` reports the scanned range and counts of spent and received Sapling and Orchard notes; received counts may include notes already spent in later blocks (a scanning-order effect), and spent counts cover only previously detected notes (`zcash_client_backend`, `src/data_api/chain.rs:437--502, 677--685`).

Remark 3.8 (A gap in `ScanSummary`). No Ironwood counters exist in `ScanSummary`, although `ScannedBlock` and `WalletTx` carry Ironwood spends and outputs; Ironwood activity is silently excluded from the summary. We read this as an oversight in the deployed code rather than a design decision (`zcash_client_backend`, `src/data_api/chain.rs:437--502, 677--685`; `research/upstream-defects.md`, item 2: “`ScanSummary` lacks spent/received counters for the Ironwood pool”, observed at librustzcash e30517e433).

The Ironwood pool otherwise traverses the pipeline as a full peer of Sapling and Orchard: its actions are trial-decrypted under `IronwoodDomain` with the account’s Orchard viewing keys, its nullifiers form a separate set — method `get_ironwood_nullifiers` is a required trait method, deliberately without a panicking default because it sits on the scan path — and its commitment tree and scan-queue shard extensions are its own (`zcash_client_backend`, `src/scanning.rs:213--256`; `src/data_api.rs:2066--2078`).

3.6 Persistence

The scanned range is committed by a single call, `WalletWrite::put_blocks(from_state, blocks)`, which requires sequential blocks in increasing height order with `from_state` the chain state prior to the first block. The backend implementation validates, for the first block b_0 and every pool,

$$\text{fromState.height} + 1 = \text{height}(b_0) \quad \text{and} \quad \text{fromState.treeSize} + |\text{commitments}(b_0)| = \text{finalTreeSize}(b_0),$$

each subsequent height being the predecessor plus one; violation is `NonSequentialBlocks`. In `zcash_client_sqlite` the whole call runs in one database transaction (`zcash_client_backend`, `src/data_api.rs:3478--3490`; `src/data_api/11/wallet.rs:291--348`; `zcash_client_sqlite`, `src/lib.rs:1425--1431`).

Stage 1: rows. Per block, `put_blocks` persists block metadata (heights, hash, time, per-pool final tree sizes and commitment counts); per transaction it persists transaction metadata, queues the txid for full-transaction retrieval (*enhancement*), marks spent notes, and persists received notes — checking each received note’s nullifier against the *persisted* nullifier map (`detect_*_spend`) to catch notes spent in a later block range that was scanned earlier, the out-of-order counterpart of the in-memory check of §3.5. It then persists each block’s unlinked-nullifier map and prunes the map beyond `PRUNING_DEPTH` (`zcash_client_backend`, `src/data_api/11/wallet.rs:291--579, 55`).

Stage 2: trees. Note commitments are assembled into shard subtrees in parallel — chunks of 1024 commitments, built at the pool’s shard height, which is 16 for all three pools (subtrees of 2^{16} commitments; the Ironwood tree is Orchard-shaped) — and each shard tree is updated. The same checkpoint set is enforced across all three trees: each tree gains a checkpoint at every height checkpointed in any other. Checkpoints at or above the NU6.3 activation height that fall on the `ANCHOR_RETENTION_INTERVAL = 288`-block grid are retained as durable anchors exempt from checkpoint pruning (`zcash_client_backend`, `src/data_api/11/wallet.rs:597--720, 1537`; `src/data_api.rs:159, 166, 173`). This is the boundary of the present volume: shard-tree structure,

witness derivation, and anchor selection are the *Ironwood Guide*’s material (§“Proof of ownership of some valid note: the commitment tree”).

Completing the found notes’ shards. After the tree update, `scan_complete` marks the scanned range `Scanned` in the queue and, if wallet notes were found, extends the queue with `FoundNote`-priority ranges covering the blocks needed to complete each found note’s subtree — the blocks that make the note witnessable. Per pool, the shard of a note at position p is

$$\text{subtreeIndex}(p) = \lfloor p/2^{16} \rfloor \quad (\text{Address}::\text{above_position}(\text{SHARD_HEIGHT}, p)),$$

and the extension spans from the previous shard’s end height — bounded below by the wallet birthday and the pool’s activation height — to the containing shard’s end height plus one. The same pass marks notes `witness_stabilized` once their shard’s block extent is fully `Scanned` and the shard’s end height is at most the pruning floor,

$$\text{floor} := \text{maxScanned} - (\text{PRUNING_DEPTH} - 1)$$

(`zcash_client_sqlite`, `src/wallet/scanning.rs:224--473`).

Two notions of scanned height. Out-of-order scanning leaves gaps, so the backend distinguishes `block_max_scanned` — simply `max(blocks.height)` — from `block_fully_scanned`, the height to which this and all preceding blocks above the wallet birthday have been trial-decrypted. The SQLite implementation computes the latter from the queue: it is `end - 1` of the *first* `Scanned` range (by ascending start), defined only when that range starts at or below the birthday (`zcash_client_backend`, `src/data_api.rs:1974--1993`; `zcash_client_sqlite`, `src/wallet.rs:3221--3307`).

3.7 Reorganisation detection and recovery

ZIP 307 prescribes the primitive check: on each sync, re-fetch the block at the previous sync height and compare its hash with the cached copy; a mismatch means the block was orphaned (ZIP 307 §“Client-server interaction”, Phases B and D). The deployed scanner generalises this into a per-block continuity check. Function `scan_block_with_runners` first runs `check_hash_continuity`, returning `ScanError::BlockHeightDiscontinuity` if `height(b) ≠ height(prev) + 1` and `ScanError::PrevHashMismatch` if `prevHash(b) ≠ hash(prev)`; for the first block of a `scan_cached_blocks` call the prior metadata is read from the wallet database (`block_metadata(from_height - 1)`), and for subsequent blocks it is the just-scanned block’s own metadata (`zcash_client_backend`, `src/scanning/compact.rs:257--289`; `src/data_api/chain.rs:649--693`). Of ZIP 307’s block-header validation list only this previous-hash check is implemented, and the ZIP’s “block privacy via bucketing” enhancement — rounding the sync start down to a bucket multiple — is explicitly unimplemented (ZIP 307 §§“Block header validation”, “Block privacy via bucketing”).

Classification is explicit: method `is_continuity_error` on `ScanError` returns true for `PrevHashMismatch`, `BlockHeightDiscontinuity`, and `TreeSizeMismatch`; the encoding and tree-size-availability errors of §3.5 (`EncodingInvalid`, `TreeSizeUnknown`, `TreeSizeInvalid`, `TreeSizeOverflow`) are not continuity errors (`zcash_client_backend`, `src/scanning.rs:602--685`).

Recovery. On a continuity error the reference loop rewinds:

$$\text{rewind} := \text{errHeight} - 10 \quad (\text{saturating_sub}),$$

where any height at least one block before the error is valid and the 10-block offset is a heuristic. It then calls `truncate_to_height(rewind)` on the wallet database, `truncate(rewind)` on the block cache, and returns control so the caller restarts from `update_chain_tip` and a

fresh `suggest_scan_ranges` (`zcash_client_backend`, `src/sync.rs:423--453`). Truncation semantics — resolution to a shared shard-tree checkpoint at or below the requested height (error `RequestedRewindInvalid`, carrying a safe rewind height, otherwise), un-mining of transactions above it, and the deliberate retention of note rows — are the *Wallet Guide*’s material (§“Reorganisations: truncate and rescan”). This volume adds two consequences within its own scope: the scan queue is trimmed to the truncation height, rolling back the wallet’s view of the chain tip so that the next `update_chain_tip` re-installs `Verify` and `ChainTip` ranges above the new maximum scanned height; and at the pinned commit truncation covers all three shard trees and the nullifier-map locators (`zcash_client_sqlite`, `src/wallet.rs:3707--3897, 4252--4274`).

The depth constant. Constant `PRUNING_DEPTH = 100` blocks is `librustzcash`’s reorganisation-safety depth (`zcash_client_sqlite`, `src/lib.rs:169`), the nullifier-map pruning depth (`zcash_client_backend`, `src/data_api/ll/wallet.rs:55`), and ZIP 307’s suggested witness-cache size — the ZIP asserts 100 blocks is reasonable “as no full Zcash node will roll back the chain by more than 100 blocks” (ZIP 307 §“Client operation”). The claim is grounded in `zcashd`: `MAX_REORG_LENGTH = COINBASE_MATURITY - 1 = 99` (`src/main.h:66`), and a node asked to reorganise deeper aborts (`src/main.cpp:4332--4337`). Zebra’s rollback window is `MAX_BLOCK_REORG_HEIGHT = 1000`, documented as local-only node policy, not consensus (`zebra-chain/src/parameters/constants.rs:20--30`), so the ZIP’s “no full Zcash node” claim no longer holds for Zebra-backed deployments; `PRUNING_DEPTH = 100` matches the `zcashd` bound, as its doc comment says (`zcash_client_sqlite`, `src/lib.rs:166--169`). The full spec-versus-deployment divergence register for these windows is the *Consensus Guide*’s (§“Reorg rails, maturity, and the finality floor”).

3.8 The reference sync loop

Module `sync` of `zcash_client_backend` assembles the pieces into the reference flow, whose contract the chain module documents (`src/sync.rs:58--226, 393--467`; `src/data_api/chain.rs:1--152`):

- (1–2) `update_subtree_roots`: download and store complete subtree roots for Sapling, Orchard, and Ironwood via `GetSubtreeRoots` (cf. Remark 3.2);
- (3–4) `update_chain_tip` with the height from `GetLatestBlock`;
- (5) `suggest_scan_ranges`;
- (6) scan every `Verify` range first: download it via `GetBlockRange`, fetch the prior `ChainState` via `GetTreeState` at `start - 1`, scan, delete the cached blocks;
- (7) scan the remaining ranges, split into chunks of the caller-supplied batch size, restarting from step (3) whenever scanning materially changed the suggestions — a new first range outranking the range just scanned, or a truncation after a continuity error (§3.7).

Blocks pass through a `BlockCache` (an extension of `BlockSource`): `get_tip_height`, `read` — short reads allowed but contiguous from the range start — `insert` — non-contiguity permitted — `delete` and `truncate`. The loop deletes each range from the cache after scanning it, the module observing that “keeping the entire chain in `CompactBlock` files on disk is horrendous for the filesystem”; deletions are spawned concurrently and awaited before the loop returns (`zcash_client_backend`, `src/data_api/chain.rs:237--435`; `src/sync.rs:131--226`).

The module documents its own limitations: block batches are not downloaded in parallel with scanning; detected transactions are not enhanced (the full transaction, hence the memo, is not fetched); there are no progress notifications; and there is no interruption mechanism short

of process exit (`zcash_client_backend`, `src/sync.rs:1--10`). The chain-module documentation additionally recommends scanning `Historic` ranges in reverse height order — or from both ends inward — so that recent unspent notes are discovered sooner (`zcash_client_backend`, `src/data_api/chain.rs:128--134`).

4 Commitment-tree synchronisation

Scanning tells a wallet which notes it owns (§3); spending one additionally requires the note’s authentication path against a recent anchor. The construction of that path — the depth-32 tree cut at half depth into height-16 shards under a height-16 cap, the wallet hashing its own shard from scanned leaves and taking every other shard as a single downloaded 32-byte root — is the *Ironwood Guide*’s material (§“Proof of ownership of *some* valid note: the commitment tree”, paragraph “Derivation of the authentication path”). This section supplies what that construction presumes: the RPCs that deliver tree states and shard roots, their derivation inside the validator, and the client discipline — frontier-seeded scanning, reference-retained roots, shard-completion scan ranges — by which `zcash_client_backend` and `zcash_client_sqlite` turn a compact-block stream plus two RPCs into spendable witnesses.

4.1 The linear protocol and its limit

ZIP 307 (Status: Draft) specifies only the original, linear protocol: as compact blocks arrive in height order, the client appends every note commitment to a local depth-32 incremental Merkle tree and updates one incremental witness per unspent note; because incremental trees cannot be rewound, the ZIP advises caching roughly 100 recent tree and witness states, on the stated ground that no full node rolls back the chain by more than 100 blocks (`zips/zip-0307.rst:290--320`; the same figure resurfaces as `PRUNING_DEPTH`, §3.7). Under this design a wallet cannot spend a note until it has scanned every block from the note’s block to the anchor block — each witness update consumes every intervening commitment — so restore-from-seed implies a complete linear scan before the first spend. Removing that spend-before-sync impossibility is the purpose of the machinery below.

Remark 4.1 (Normative status). No ZIP specifies `GetTreeState`, `GetSubtreeRoots`, or the subtree-root chain index. ZIP 307’s Phase A leaves tree-state acquisition an explicit unresolved TODO — “Or, it has to set `X` to the block height at which Sapling activated, so as to be sent the entire commitment tree. [TODO: Decide which to specify.]” (`zips/zip-0307.rst:433--439`) — and the repository holding the canonical protobufs disclaims normativity: “This repository IS not a specification of the Zcash Light Client protocol” (`lightwallet-protocol/README.md`, v0.4.1). As in Remark 3.1, we therefore cite the proto files and the deployed implementations as the specification of record, and classify the subtree-root protocol *deployed but unspecified*.

4.2 The tree-state service surface

Definition 4.2 (Tree-state and subtree-root RPCs). The deployed light-wallet service defines three tree RPCs (`lightwallet-protocol/walletrpc/service.proto:307--316`):

- `GetTreeState(BlockID)` returns `(TreeState)` — the tree state at a block specified by height or by hash;
- `GetLatestTreeState(Empty)` returns `(TreeState)` — the tree state at the server’s chain tip;
- `GetSubtreeRoots(GetSubtreeRootsArg)` returns `(stream SubtreeRoot)` — a stream of information about roots of subtrees of the note commitment tree of the requested

shielded protocol — Sapling or Orchard in the deployed protocol.

a

^aThe proto comment refers the reader to “section 3.7 of the Zcash protocol specification” (`service.proto:308`). The reference is stale: at the pinned spec revision, §3.4 is “Transactions and Treestates”, §3.8 “Note Commitment Trees”, and §3.7 “Action Transfers and their Descriptions” (`protocol/protocol.tex:4048,4306,4248`), likely a pointer into an older revision’s numbering. We cite the current numbering.

Definition 4.3 (TreeState message). Message `TreeState`, documented in the proto as “derived from the Zcash `z_gettreestate rpc`”, carries

```
network : string = 1 ("main" | "test");  height : uint64 = 2;
hash : string = 3 (big-endian display-order hex);  time : uint32 = 4;
saplingTree : string = 5;  orchardTree : string = 6,
```

where the two tree fields are protobuf *strings* — hex encodings of the legacy-serialised commitment trees of Definition 4.4 — not bytes (`lightwallet-protocol/walletrpc/service.proto:176--184`). At `librustzcash HEAD` the vendored proto adds `ironwoodTree : string = 7`, absent from the deployed v0.4.1 surface (Remark 3.2).

Definition 4.4 (Legacy commitment-tree encoding). The tree state of one pool at one block is serialised in the legacy `CommitmentTree` wire format,

$$\text{ser}(T) = \text{Opt}(\text{left}) \parallel \text{Opt}(\text{right}) \parallel \text{Vec}(\text{Opt}(\text{parents}_1), \text{Opt}(\text{parents}_2), \dots),$$

where `Opt(·)` writes `0x00` for an absent node and `0x01` followed by the 32-byte node otherwise, `Vec(·)` is `CompactSize`-prefixed (Bitcoin’s variable-length integer: counts below 253 in one byte, larger ones as a marker byte then 2, 4, or 8 little-endian bytes), and each node is the 32-byte encoding of a base-field element — Jubjub base for Sapling, Pallas base for Orchard — with non-canonical encodings rejected at parse time. The parser `read_commitment_tree` bounds the parents vector at `DEPTH - 1` entries, and the parsed tree converts losslessly to a frontier via `to_frontier()` (`zcash_primitives/src/merkle_tree.rs:26--61,183--209`). The format’s originator is `zcashd` (retired): its Sapling final state streamed the legacy `SaplingMerkleTree` directly and its Orchard final state streamed through the `OrchardMerkleFrontierLegacySer` wrapper, so both pools use the same wire format, hex-encoded (`zcashd,src/rpc/blockchain.cpp:1402--1406,1432--1436`).

Definition 4.5 (Subtree-root messages). Message `GetSubtreeRootsArg` carries `startIndex : uint32 = 1`, the index of the first subtree to return; `shieldedProtocol = 2` with enum `ShieldedProtocol { sapling = 0; orchard = 1 }` as deployed; and `maxEntries : uint32 = 3`, where 0 means “return all entries”. Results stream in index — equivalently height — order. Message `SubtreeRoot` carries `rootHash : bytes = 2`, the 32-byte Merkle root of the completed subtree; `completingBlockHash : bytes = 3`; and `completingBlockHeight : uint64 = 4`. Field number 1 is absent, and no reserved statement documents the gap (`lightwallet-protocol/walletrpc/service.proto:186--200`).

Remark 4.6 (Byte-order wars). Two byte-order pitfalls attach to these messages. First, `SubtreeRoot.completingBlockHash` is sent in big-endian display order — `lightwalletd` sets it to `hash32.ReverseSlice(block.Hash)` — which is the *opposite* convention from `CompactBlock.hash`,

parsed into little-endian wire order; the proto documents neither convention (lightwalletd, frontend/service.go:896--900; parser/block.go:59--61,115). The reference client sidesteps the trap by ignoring completingBlockHash entirely, reading only rootHash and completing BlockHeight (zcash_client_backend, src/sync.rs:247--253). Zaino constructs the field as the block hash's bytes_in_display_order() (zaino-state/src/indexer.rs:854--861), agreeing with lightwalletd's big-endian display order; whether any deployed client consumes completingBlockHash remains an open question (the reference client ignores it). Second, a zcashd release note records that the serialised finalRoot field of z_gettreestate was byte-flipped relative to getrawtransaction output in releases up to and including v5.2.0, later rectified (zcashd, src/rpc/blockchain.cpp:1326--1330, help text).

4.3 Server-side derivation

Tree states. The RPC originates in zcashd's z_gettreestate: it accepts a block hash or height (negative heights allowed, -1 denoting the last known valid block), requires the block to be on the main chain, and returns the block's hash, height, and time together with one object per pool, {skipHash, commitments : {finalRoot, finalState}}. The finalState — the serialised tree of Definition 4.4 — is present only when the block's anchor is retrievable from the coins view (GetSaplingAnchorAt / GetOrchardAnchorAt); otherwise skipHash names the most recent prior block that has one, stopping at the pool's activation height (zcashd, src/rpc/blockchain.cpp:1303--1455).

The proxy wraps this in a retry loop. Handler GetTreeState of lightwalletd rejects a request specifying neither height nor hash, marshals a height as a decimal string and a hash as big-endian hex, and calls z_gettreestate; if the reply's Sapling finalState is empty and its Sapling skipHash is nonempty, it re-issues the RPC at the skipHash, iterating until a final state is found or no skip hash remains, and failing with "z_gettreestate did not return treestate" otherwise (lightwalletd, frontend/service.go:311--386). The walk terminates in practice because zcashd hard-stops it at the Sapling activation height (the saplingActive lambda; zcashd, src/rpc/blockchain.cpp:1408--1418). Handler GetLatestTreeState calls getblockchaininfo, takes its Blocks value as the latest height, and delegates to GetTreeState at that height (lightwalletd, frontend/service.go:388--401).

Remark 4.7 (The skip walk keys on Sapling only). Only the Sapling skipHash is consulted: when the walk lands on an earlier block, the returned TreeState carries that earlier block's height, hash, and Orchard tree — not the Orchard state of the requested block (lightwalletd, frontend/service.go:311--386).

Subtree roots. The originating RPC is z_getsubtreesbyindex "pool" start_index (limit), added in zcashd v5.6.0 and gated behind -experimentalfeatures + -lightwalletd: it returns roots of complete 2^{16} -leaf subtrees of the given pool's note commitment tree, each with end_height, the height of the block containing the note commitment that completed the subtree, reading SubtreeData records from the coins view (pcoinsTip->GetSubtreeData(pool, index)) until data is absent or the limit is reached (zcashd, src/rpc/blockchain.cpp:1457--1539; doc/release-notes/release-notes-5.6.0.md:28). The records are maintained by the validator's incremental tree: zcashd stores SubtreeData{leadbyte, root, nHeight} and LatestSubtree{leadbyte, index, root, nHeight} records under a uint64_t subtree index, fixes TRACKED_SUBTREE_HEIGHT = 16, and produces a completed subtree root exactly when the tree size crosses a 2^{16} boundary (zcashd, src/zcash/IncrementalMerkleTree.hpp:20--70,409--411; IncrementalMerkleTree.cpp:913--925):

```
current_subtree_index() = size() >> TRACKED_SUBTREE_HEIGHT.
```

Handler GetSubtreeRoots of lightwalletd accepts only Sapling or Orchard and forwards to z_getsubtreesbyindex(pool, startIndex[, maxEntries when > 0]); for each returned

subtree it fetches the block at `end_height`, populates the two completing-block fields from it, and streams the assembled `SubtreeRoot` (`lightwalletd, frontend/service.go:832--907`).

Example 4.8 (Mainnet Sapling subtrees 0 and 1). A worked datum recorded in `lightwalletd` source (a quoted `z_getsubtreesbyindex` transcript, `common/common.go:178--210`): Sapling subtree 0 — outputs 0 through 65535 — has root

```
754bb593ea42d231a7ddf367640f09bbf59dc00f2c1d2003cc340e0c016b5b13
```

with `end_height = 558822`, and subtree 1 has root

```
03654c3eacbb9b93e122cf6d77b606eae29610f4f38a477985368197fd68e02d
```

with `end_height = 670209`: measured from Sapling activation at height 419,200 (`zcash_protocol, src/consensus.rs:488`), the first 2^{17} Sapling outputs span roughly the chain’s first 250,000 post-activation blocks.

Deployed validator. Zebra implements both `z_gettreestate` and `z_getsubtreesbyindex` in its RPC surface, so a Zebra-backed indexer retains tree synchronisation after `zcashd`’s retirement (`zebra-rpc, src/methods.rs:361,2018; src/methods/trees.rs`). The successor indexer serves the same gRPC API: Zaino’s `get_tree_state` proxies `z_get_treestate` to the backing validator and hex-encodes the final states (at HEAD it also populates an `ironwood_tree` field), its `get_latest_tree_state` reads the chain height and delegates, and its `get_subtree_roots` fetches each completing block via `z_get_block(end_height)` (`zaino-state, src/backends/fetch.rs:1836--1896; src/indexer.rs:756--860; src/chain_index.rs:509--514,2397--2431`). One behavioural divergence: Zaino maps the proto’s `uint32 startIndex/maxEntries` down to `u16` and returns `InvalidArgument` when either exceeds 65535 — semantically safe, since a depth-32 tree has at most 2^{16} shards, but stricter than `lightwalletd/zcashd`, whose `u64` subtree index yields an empty stream instead (`src/chain_index.rs:509--514` versus `zcashd src/zcash/IncrementalMerkleTree.hpp:20--70`). Both backends delegate to the validator: `NodeBackedChainIndex::get_subtree_roots` calls the blockchain source (`chain_index.rs:2421--2431`); `ValidatorConnector::State` issues `ReadRequest::Sapling, Orchard, Ironwood` Subtrees to Zebra’s `ReadStateService`, and `::Fetch` proxies `z_getsubtreesbyindex` (`chain_index/source/validator_connector.rs:545--608`); only the mockchain test source self-computes (`chain_index/source/mockchain_source.rs:579--665`).

4.4 Client state: shards, cap, and retention

The client-side container is `shardtree`: “a sparse binary Merkle tree of the specified depth, represented as an ordered collection of subtrees (shards) of a given maximum height”, plus a *cap* — the tree above the shard roots — and a checkpoint set enabling truncation. The root sits at address (`DEPTH, 0`), shards at level `SHARD_HEIGHT`, and `max_subtree_index() = 2^{DEPTH-SHARD_HEIGHT} - 1` (`shardtree, src/lib.rs:61--129`). The wallet API instantiates one such tree per pool as `ShardTree(Store, DEPTH = 32, SHARD_HEIGHT = 16)` with block heights as checkpoint identifiers, via `with_sapling_tree_mut / with_orchard_tree_mut` (`zcash_client_backend, src/data_api.rs:3743--3826`). The shard height is documented as chosen to conform “to the structure of subtree data returned by `lightwalletd` when using the `GetSubtreeRoots` gRPC call”: `SAPLING_SHARD_HEIGHT = ORCHARD_SHARD_HEIGHT = 16`, half the tree depth of each pool (`src/data_api.rs:155--166`). The geometry — height-16 shards of 2^{16} leaves under the 16-level cap, at most 2^{16} shards — is the *Ironwood Guide*’s construction (§“Proof of ownership of *some* valid note: the commitment tree”); the deployed constants realise it exactly (`shardtree, src/lib.rs:111--129; zcash_client_sqlite, src/wallet/db.rs:1446--1447`). The SQLite backend constructs each tree as `ShardTree::new(store, PRUNING_DEPTH)` with

PRUNING_DEPTH = 100: at most 100 checkpoints — one per scanned block boundary — are retained before pruning, 100 blocks being the assumed maximum reorganisation depth and ZIP 307’s cache-size guidance (§4.1) (zcash_client_sqlite, src/lib.rs:169,2500,2526,2558).

What survives pruning is governed by per-node *retention* (incrementalmerkletree, src/lib.rs:76--107):

- Ephemeral — prunable together with its siblings;
- Checkpoint{id, marking} — the position is retained (its value prunable) while the checkpoint lives, decaying to Marked, Reference, or Ephemeral per the marking when the checkpoint is removed;
- Marked — retained until explicitly removed; the wallet’s own note positions;
- Reference — retained until overwritten by a node with Ephemeral, Checkpoint, or Marked retention; downloaded subtree roots and seed frontiers.

4.5 Ingesting subtree roots

The reference sync driver begins *every* run by refreshing the shard roots: function `update_subtree_roots` downloads all subtree roots — the default `GetSubtreeRootsArg`, i.e. `startIndex = 0`, `maxEntries = 0` — for Sapling and, with the orchard feature, Orchard and Ironwood, converting each streamed message by

```
CommitmentTreeRoot::from_parts(completingBlockHeight, H::read(rootHash))
```

and passing the batches, each with start index 0, to `put_sapling_subtree_roots`, `put_orchard_subtree_roots`, and `put_ironwood_subtree_roots` (zcash_client_backend, src/sync.rs:80--86,228--299). Type `CommitmentTreeRoot` pairs the subtree end height with the root hash (src/data_api/chain.rs:180--212), and the trait documentation (src/data_api.rs:3743--3826) fixes the meaning: each value “should be the Merkle root of a subtree ... containing $2^{\text{SHARD_HEIGHT}}$ note commitments”.

The SQLite backend implements both puts via one generic procedure, `put_shard_roots` parameterised by the hash H , the tree depth, and the shard height (zcash_client_sqlite, src/wallet/commitment_tree.rs:1107--1227):

- (1) no-op on empty input;
- (2) load the cap and treat it as a standalone tree of `DEPTH - SHARD_HEIGHT` levels whose level-0 leaves are shard roots: the `LevelShifter` wrapper offsets the hash,

```
emptyLeaf = H.emptyRoot(SHARD_HEIGHT),
combine( $\ell$ ,  $a$ ,  $b$ ) = H.combine( $\ell$  + SHARD_HEIGHT,  $a$ ,  $b$ ),
```

and the downloaded roots are batch_inserted at `Position::from(startIndex)` with `Retention::Reference` (src/wallet/commitment_tree.rs:1122--1181);

- (3) write the cap back;
- (4) check contiguity over `[startIndex, startIndex + n)`: `check_shard_discontinuity` rejects a call whose shard-index range is neither overlapping nor immediately adjacent to the stored `[min(shardIndex), max(shardIndex) + 1)` range, raising `Error::SubtreeDiscontinuity` — subtree roots must arrive without gaps (src/wallet/commitment_tree.rs:481--520);
- (5) upsert each root into `{pool}_tree_shards(shard_index, subtree_end_height, root_hash, shard_data)`; on conflict only the end height and root hash are updated, so existing scanned shard data is never clobbered, and the fresh `shard_data` written for new rows is a single root leaf with `RetentionFlags::EPHEMERAL`.

The downloaded root is cached beside any persisted subtree rather than merged eagerly — “we want to avoid deserializing the subtree just to annotate its root node, so we simply cache the downloaded root alongside of any already-persisted subtree” — and the read path resolves the pair: `get_shard` deserialises `shard_data` and applies `reannotate_root(Some(rootHash))(src/wallet/commitment_tree.rs:1190--1193,412--445)`.

Remark 4.9 (Idempotent full re-download). The driver fixes `startIndex = 0` on every run although `put_shard_roots` supports arbitrary start indices; the upsert path makes the repetition idempotent, at the cost of re-transferring one 32-byte root (plus completing-block metadata) per completed shard per pool on every sync (`zcash_client_backend, src/sync.rs:80--86,228--299`). As of 2026-07-15 (mainnet tip 3,413,101): 1,128 complete Sapling subtrees (last completing at block 3,390,009) plus 765 Orchard (3,388,160), so each sync re-downloads 1,893 roots — 60,576 bytes of root data, 136,378 bytes of `SubtreeRoot` messages, 145,843 bytes with the five-byte gRPC frame per stream element (computed by script; counts cross-check against `ChainMetadata` at block 3,413,000: $\lfloor 73,932,454/2^{16} \rfloor = 1,128$, $\lfloor 50,191,264/2^{16} \rfloor = 765$). No source documents the fixed `startIndex = 0` as intentional.

4.6 Frontiers as scan seeds

A `TreeState` becomes client state through `TreeState::to_chain_state()`: it hex-decodes the block hash and reverses its bytes (“Zcashd hex strings for block hashes are byte-reversed”), and parses each pool’s legacy tree (Definition 4.4) into a frontier; an *empty* tree-state string parses as the empty tree, the correct state before a pool’s activation (`zcash_client_backend, src/proto.rs:367--463`). The result is

```
ChainState{ blockHeight; blockHash;
            finalSaplingTree : Frontier<32>; finalOrchardTree : Frontier<32>;
            finalIronwoodTree : Frontier<32> },
```

“the final note commitment tree state for each shielded pool, as of a particular block height” — the Orchard and Ironwood frontiers sit behind the orchard feature (`src/data_api/chain.rs:504--596`).

Every scanned batch is seeded with the chain state of the block immediately preceding it, and the seed is verified against the server’s own metadata before any tree is touched:

$$\text{fromHeight} = \text{seed.blockHeight} + 1 \quad (\text{asserted}), \quad \text{seed.treeSize}_P + |\text{cm}_P(b_0)| = \text{finalTreeSize}_P(b_0)$$

per pool P , where b_0 is the batch’s first block and the final tree size is read from the block’s `chainMetadata`; violation yields error `NonSequentialBlocks` (`zcash_client_backend, src/data_api/chain.rs:616--635; src/data_api/ll/wallet.rs:291--348`). The metadata sizes are what give the scanner absolute leaf positions at all — without them it cannot even derive nullifiers (`TreeSizeUnknown`, a hard error), and a size contradicted by block contents is `TreeSizeMismatch`, a continuity error triggering the rewind of §3.7 (`lightwallet-protocol/walletrpc/compact_formats.proto:14--17,31--40; zcash_client_backend, src/scanning.rs:602--685`).

The batch’s note commitments are then assembled into located subtrees in parallel — rayon, chunks of 1024 commitments, `LocatedTree::from_iter` at the shard level — starting at `Position::from(seed.treeSize_P)`, and each pool’s tree is updated by `update_tree` (`zcash_client_backend, src/data_api/ll/wallet.rs:597--771`):

- (1) insert the seed frontier (`tree.insert_frontier`) with retention `Checkpoint{id = frontierHeight, marking = Reference}` — the in-source comment: “we insert the frontier with `Checkpoint` retention because we need to be able to truncate the tree back to this point”;
- (2) `insert_tree(subtree, checkpoints)` for each built subtree;

- (3) add any cross-pool checkpoints missing above the minimum retained checkpoint (`src/data_api/11/wallet.rs:1591--1664`; the cross-pool checkpoint-unification rule itself, §3.6).

Inside `shardtree`, `insert_frontier` splits a non-empty frontier at the shard boundary: the leaf and the ommers (the sibling hashes a frontier stores, one per filled subtree along the leaf’s path to the root) below the shard level enter the shard containing the frontier’s position, the ommers at levels $\geq$ `SHARD_HEIGHT` enter the cap, checkpoint retention adds a checkpoint at the leaf position, and checkpoints beyond `max_checkpoints` are pruned. An empty frontier with checkpoint retention records a checkpoint for the empty tree (`Retention::Marked` is invalid there) (`shardtree`, `src/lib.rs:323--405`).

Frontier seeding is also how a wallet is born and how it rewinds. An `AccountBirthday` wraps “the chain state prior to the birthday height” plus an optional `recover_until`; `AccountBirthday::from_treestate` builds it directly from a `TreeState` for the last block *prior* to the birthday, with `height() = priorChainState.blockHeight + 1`, and account creation stores the birthday tree sizes and rewinds the wallet to the birthday chain state, so no tree information before the birthday is ever needed (`zcash_client_backend`, `src/data_api.rs:3010--3106`; `zcash_client_sqlite`, `src/wallet.rs:438--618`; §3.2). Symmetrically, `truncate_to_chain_state` inserts each pool’s frontier from the target chain state with `Checkpoint{id = targetHeight, marking = None}`, creating the checkpoint on which the subsequent tree truncation lands (`zcash_client_sqlite`, `src/wallet.rs:3984--4034`; recovery flow, §3.7).

4.7 Shard completion and spendability

Two of the scan-queue priorities of §3.3 exist purely for this layer: `ChainTip` is documented as “blocks that must be scanned to complete the latest note commitment tree shard” and `FoundNote` as “blocks that must be scanned to complete note commitment tree shards adjacent to found notes” (`zcash_client_backend`, `src/data_api/scanning.rs:13--29`).

The tip shard. On each tip update the backend computes per pool the maximum `subtree_end_height` over the pool’s shards table — the end of the last complete shard known from `GetSubtreeRoots` — takes the minimum across pools, and enqueues

$$\lceil \max(\text{walletBirthday}, \text{minShardTip}), \text{tip} + 1 \rceil$$

at `ChainTip` priority, so the incomplete tip shard of every pool is scanned even before historic ranges (`zcash_client_sqlite`, `src/wallet/scanning.rs:475--657`; the full tip-update case analysis, §3.4).

Found notes. After a range is scanned, `scan_complete` maps each detected note position to its shard, $s = \lfloor p/2^{16} \rfloor$ (`Address::above_position(SHARD_HEIGHT, position)`), and extends the scanned range to that shard’s block extent — from the previous shard’s `subtree_end_height` (the pool activation height for shard 0, clamped to the wallet birthday) to the containing shard’s `subtree_end_height + 1` — enqueueing the extensions beyond the just-scanned range at `FoundNote` priority. Completing the shard’s block range is exactly what makes the note’s witness computable (`zcash_client_sqlite`, `src/wallet/scanning.rs:224--398`).

Witness derivation. The query `witness_at_checkpoint_id(position, id)` requires `position ≤ checkpoint.position()` (else `QueryError::NotContained`) and folds, from the note’s shard upward to the root, the root of each sibling address truncated at leaf count `positionas of + 1` — each sibling root drawn from the note’s own shard (scanned leaves), another shard’s cached root (`GetSubtreeRoots`, Definition 4.5), the cap, or a per-level empty-root constant beyond the tree size. A pruned node that cannot be replaced by an empty-root constant yields `QueryError::`

`TreeIncomplete(addresses)` — the precise failure mode of an unready witness (shardtree, `src/lib.rs:1236--1362`; `src/error.rs:124--136`). The mathematics of the fold is the *Ironwood Guide*’s (§“Proof of ownership of *some* valid note: the commitment tree”); what this volume adds is the deployed readiness condition under which it cannot fail.

Spendability. That condition is enforced in SQL. Writing U_P for the rows of view `v_{pool}_shard_unscanned_ranges`, note selection excludes any note at position p for which some $(s, e, h_0, h_1) \in U_P$ satisfies

$$s \leq p < e \wedge h_0 \leq \text{anchorHeight} \wedge h_1 > \text{walletBirthday},$$

i.e. the note’s shard has an unscanned block range at or below the anchor; the same view drives `unscanned_tip_exists`, which invalidates anchors whose containing shard is not fully scanned (`zcash_client_sqlite, src/wallet/common.rs:142--162, 762--822`). The views compute `startPosition = shardIndex << 16` and `endPositionExclusive = (shardIndex + 1) << 16`, set the shard’s block extent to begin at the previous shard’s end height (the pool activation height for the first shard), join the scan-queue rows whose block ranges overlap that extent, and filter to priority above `Scanned` (`zcash_client_sqlite, src/wallet/db.rs:1435--1494`). Unwinding the predicate recovers the informal statement: a note is spendable at a given anchor exactly when its own shard is scanned up to the anchor, every earlier shard is covered by a downloaded root, and the blocks from the last complete shard boundary to the anchor have been scanned (the `ChainTip` range above). Which anchor a transaction then selects, and at what confirmation depth, is the *Wallet Guide*’s material (§“The proposal”; §“Confirmations, trust, and spendability”).

At the pinned `librustzcash` commit — part of the NU6.3 work of Remark 3.2, not of any release deployed with `lightwalletd v0.4.1` — notes additionally gain a `witness_stabilized` flag once their shard’s block extent is fully `Scanned` and the shard’s end height has at least `PRUNING_DEPTH` confirmations (`floor = maxScanned - (PRUNING_DEPTH - 1)`); stabilised notes bypass the confirmations check at spend time, and the tip shard by definition can never stabilise (`zcash_client_sqlite, src/wallet/scanning.rs:416--473; src/wallet/common.rs:700--724`).

What this buys, and what remains. The linear protocol’s impossibility is gone: a freshly restored wallet downloads one frontier (its birthday tree state) plus one 32-byte root per completed shard, and a note found during scanning becomes spendable as soon as its own shard’s block extent and the tip range are scanned — not after the entire chain history. What remains is the trial decryption of every block from the birthday forward (§3) and the shard-completion scanning of this section; both are cost centres the *Tachyon Guide*’s proof-carrying wallet is designed to remove (§“What the fold removes”, paragraphs “Trial decryption of the chain” and “Per-note witness maintenance”).

5 What the server learns

A light client outsources block storage and filtering to a server it does not trust with its keys, then reconstructs its own state by trial decryption on the device (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”). The keys never travel; the *requests* do. This section inventories what the deployed server-side observer learns from those requests: the stated threat model, the baseline that genuinely holds, the leaks in the deployed request pattern, the operator’s instruments for recording them, and the mitigations that actually ship. Every claim about deployed behaviour cites the implementation at the commits pinned above; where the code diverges from ZIP text, we flag the divergence in place.

5.1 The stated security model

ZIP 307 names one primary goal for the light-client protocol.

Definition 5.1 (Payment detection privacy). The serving proxy must not learn which transactions are addressed to a given light wallet. Under the additional assumption of network privacy (“via Tor or similar”), the proxy must also be unable to link different connections or queries as deriving from the same wallet (ZIP 307 §“Security Model”).

The adversary of ZIP 307 is the node/proxy combination, modelled as *honest but curious*: it is trusted for chain-state correctness — the client accepts the block data it serves — but assumed to record and analyse everything it sees. The ZIP explicitly does not address spending notes privately (ZIP 307 §“Security Model”); transaction submission nonetheless flows through the same server, and we treat it in §5.5.

Remark 5.2 (The de facto wire specification). ZIP 307 has Status Draft — it was never finalised — even though the protocol it sketches is the deployed synchronisation layer. The deployed wire format has moved past the sketch: the ZIP’s CompactBlock carries vtx at field 3 and a BlockID, while the deployed message carries vtx at field 7 plus (in the v0.4.1 proto `lightwalletd` vendors) `protoVersion` — removed in v0.5.0 (§1.2) — `prevHash`, `time`, an (always unset) header, and `ChainMetadata`; the deployed CompactTx adds `txid`, `fee`, Orchard actions, and transparent vin/vout entries (CompactTxIn outpoints and full TxOuts) (ZIP 307 §“Compact Stream Format” vs. `lightwalletd`, `lightwallet-protocol/walletrpc/compact_formats.proto`). The `lightwallet-protocol` proto repository, not the ZIP, is the de facto wire specification. The planned in-protocol successor is emptier still: ZIP 314, “Privacy upgrades to the Zcash light client protocol”, has Status Reserved — a title with no content (`zips/zip-0314.rst`). The server implementation delegates likewise: the `lightwalletd` README directs developers to the external wallet app threat model for “important information about the security and privacy limitations of light wallets that use `lightwalletd`”; the repository itself contains no privacy analysis (`lightwalletd`, `README.md`).

5.2 What never leaves the device

The baseline holds. No RPC in the deployed service carries key material: across the full CompactTxStreamer surface (twenty methods), every request consists only of heights, hashes, txids, transparent addresses, raw transaction bytes, and pool or subtree selectors (`lightwalletd`, `lightwallet-protocol/walletrpc/service.proto`). Incoming viewing keys remain on the device: trial decryption of compact outputs is performed client-side by `scan_block` over locally cached CompactBlocks (`zcash_client_backend`, `src/scanning.rs`), by the mechanism of the *Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”; witness maintenance over the resulting positions is that of §“Proof of ownership of *some* valid note: the commitment tree”. Nullifier matching against the CompactSpend and CompactOrchardAction nullifier fields is likewise local.

The compact form is the reason a server mediates at all: the per-output and per-spend reductions, and the recomputed-commitment integrity substitute for the discarded AEAD tag, are those of §1 (Table 2; §1.4).

Remark 5.3 (The transparent boundary). The baseline above is a shielded-layer property. The same service carries transparent-address RPCs whose requests contain cleartext addresses — `GetAddressTxids` and `GetAddressTransactions` (an address plus a block range), `GetAddressBalance(Stream)` (an address list), `GetAddressUtxos(Stream)` (an address list plus a start height) — so for the transparent layer the server learns the wallet’s addresses outright, by declaration rather than inference (`lightwalletd`, `walletrpc/service.proto`). We treat that request family as out of scope except where the shielded sync driver itself uses it (§5.3) and where deployed mitigations attach to it (§5.7).

5.3 The observable request schedule

What the server sees is a request schedule. ZIP 307 describes it in four phases: at first start, $A = \text{GetLatestBlock}$ plus GetBlock at the tip; on each subsequent synchronisation, $B = \text{GetLatestBlock}$ followed by $\text{GetBlockRange}(X, Y)$ from the last-synced height X ; $C = \text{GetTransaction}(\text{txid})$ for each transaction of interest after detection; and $D =$ the next GetLatestBlock plus $\text{GetBlockRange}(Y, Z)$ (ZIP 307 §“Client-server interaction”).

The deployed driver refines this but does not change its shape. Per cycle, function `run` of `zcash_client_backend` (module `sync`, feature `sync`) issues: `GetSubtreeRoots` per shielded pool, `GetLatestBlock`, `GetAddressUtxosStream` over *all* transparent receivers of every account (cleartext, Remark 5.3), `GetTreeState` for the block before each scan range, and `GetBlockRange` for each suggested scan range in `batch_size` chunks (`zcash_client_backend`, `src/sync.rs`). At the pinned commit the subtree-root fetch covers Sapling, Orchard, and Ironwood. Ironwood appears in no light-client ZIP — ZIP 2005 covers quantum recoverability and ZIP 258 only NU6.3 deployment — the request exists solely because the `librustzcash` proto fork defines `ShieldedProtocol.ironwood = 2` and the driver requests it (`src/sync.rs:289--296`; cf. Remark 3.1). The module’s own documentation notes that this simple implementation does not itself enhance transactions; enhancement is part of the documented flow (the crate-root diagram’s “Enhance transactions” step) and is driven by wallets via `transaction_data_requests` (`zcash_client_backend`, `src/lib.rs`, `src/sync.rs`).

The *order* of range requests is itself data. Scan ranges carry a priority (Table 5), and `suggest_scan_ranges` returns them in descending priority, then descending end height (§3.3), so `Verify` and `ChainTip` ranges near the tip are fetched first and `FoundNote` ranges preempt Historic backfill (`zcash_client_backend`, `src/data_api/scanning.rs`; `zcash_client_sqlite`, `src/wallet/scanning.rs`). The privacy consequence of that pre-emption is Remark 5.4.

5.4 Leaks in the deployed pattern

ZIP 307 asserts that compact-block synchronisation itself “leaks no information about which transactions the light client is interested in” because the compact stream is a broadcast medium (ZIP 307 §“Transaction privacy”). The same document contradicts the claim in the immediately preceding section, for the request pattern actually deployed: “The above interaction reveals to the server at the start of each synchronisation phase (B and D) the block height which the light client had previously synchronised to. This is an information leak under our security model” (ZIP 307 §“Block privacy via bucketing”). The broadcast argument holds only for a client that fetches everything from everyone; the deployed client fetches specific ranges at specific times and follows detections with specific txids. We take the leaks in turn.

The first request reveals the birthday. On account creation, the scan queue marks the range from Sapling activation up to the account birthday as `Ignored` — never downloaded — and queues scanning from the birthday itself (`zcash_client_sqlite`, `src/wallet.rs`). The lowest height the client ever requests therefore approximates the account’s creation or recovery height, a lifetime-stable identifier of the wallet.

Each cycle reveals the prior sync height. This is the leak ZIP 307 concedes above. Its proposed mitigation is bucketing: for a bucket size N , phases B and D would request

$$\text{GetBlockRange}(X - (X \bmod N), Y) \quad \text{instead of} \quad \text{GetBlockRange}(X, Y),$$

hiding the exact previously-synced height within an N -block bucket (ZIP 307 §“Block privacy via bucketing”). The ZIP marks this “a proposed enhancement that has not been implemented”, and the deployed code confirms: `download_blocks` issues `GetBlockRange` with start and end exactly equal to the suggested scan-range bounds, with no rounding (`zcash_client_backend`, `src/sync.rs`).

Detection is followed by a fetch. During compact-block scanning, function `put_blocks` records, per scanned block, exactly the subset of transactions relevant to this wallet, and queues a retrieval request for each; `transaction_data_requests` later emits `Enhancement(txid)` or `GetStatus(txid)`, which the sync driver resolves via the `GetTransaction` RPC (`zcash_client_backend`, `src/data_api.rs`, `src/data_api/11/wallet.rs`; `zcash_client_sqlite`, `src/wallet.rs`). The set of txids fetched in full therefore *equals* the wallet’s own transaction set, and each fetch follows immediately after the scan of the containing range. To the server this is a detection oracle: phase C traffic names the very transactions that payment detection privacy (Definition 5.1) is supposed to conceal, correlated in time with the range that contains them.

Here the code diverges from the ZIP, in the unusual direction: the ZIP is stricter than the deployment. ZIP 307’s full-transaction mitigation is normative — the client “SHOULD obscure the exact transactions of interest by downloading numerous uninteresting transactions as well”, “SHOULD download all transactions in any block from which a single full transaction is fetched”, and “MUST convey to the user that fetching full transactions will reduce their privacy” (ZIP 307 §“Transaction privacy”). No decoy, chaff, or cover-traffic code exists anywhere in `librustzcash` or `lightwalletd` (negative grep over both repositories at the pinned commits), and the deployed RPC surface offers no bulk full-transaction fetch to implement the second SHOULD with — `GetBlock` returns compact data (`lightwalletd`, `lightwallet-protocol/walletrpc/service.proto`).

Enhancement fetches carry no timing decorrelation. The deployed request taxonomy is

$$\text{TransactionDataRequest} \in \{ \text{GetStatus}(\text{txid}), \text{Enhancement}(\text{txid}), \\ \text{TransactionsInvolvingAddress}\{\dots, \text{request_at}, \dots\}, \\ \text{GetSpendingTx}(\cdot) \}$$

(`zcash_client_backend`, `src/data_api.rs`; the last two variants are feature-gated). Only `TransactionsInvolvingAddress` carries a `request_at` field, documented as the instant chosen so as to “decorrelate this request to a light wallet server from other requests made by the same caller”, ignorable when the supplier of chain data is “private, trusted-for-privacy”. The shielded-note variants `Enhancement` and `GetStatus` have no analogous jitter: the SQLite backend emits them with no `request_at` (`zcash_client_sqlite`, `src/wallet.rs`). The one deployed timing defence (§5.7) protects transparent address checks, not the shielded fetch-on-detect path.

Remark 5.4 (FoundNote clustering — an inferred leak). When a note is found, `scan_complete` inserts scan ranges at priority `FoundNote` extending the scanned range so as to complete the note-commitment-tree shard(s) containing the found note, and `suggest_scan_ranges` orders by priority, so the server observes out-of-order, high-priority `GetBlockRange` fetches clustered around the blocks that contain the wallet’s own notes (`zcash_client_sqlite`, `src/wallet/scanning.rs`; `zcash_client_backend`, `src/data_api/scanning.rs`). No source documents this as a leak — neither ZIP 307 nor the code documentation — and the statement above is this volume’s own inference from the scan-queue code. It requires adversarial review — in particular, independent confirmation that `FoundNote` ranges surface to the server as distinct requests rather than being absorbed into contiguous suggested ranges — before it may be cited as fact.

Mempool polling fingerprints the session. Field `exclude_txid_suffixes` of the `GetMempoolTx` request carries suffixes of txids the client already holds, so a mempool-polling client additionally reveals which mempool transactions it has already seen, a session-linkable fingerprint (`lightwalletd`, `walletrpc/service.proto`). No source documents this as a leak; we record it for completeness.

5.5 The submission channel

Spending is out of ZIP 307’s stated scope (§5.1), but the deployed submission path runs through the same server. The `SendTransaction` handler hex-encodes the received raw transaction and forwards

it to the backing node's `sendrawtransaction` JSON-RPC (`lightwalletd, frontend/service.go`), so the operator holds the complete transaction *before* network broadcast and can link it to the submitting IP address, the TLS session, its timing, and — absent circuit isolation (§5.7) — to the same session's sync and enhancement traffic. The backing full node also sees the transaction and the `GetTransaction` txid pattern, but without client IPs: `lightwalletd` terminates the client connections. The lifecycle the transaction then follows — store-then-broadcast, expiry, confirmation counting under the `ConfirmationsPolicy` — is that of the *Wallet Guide*, §“Broadcast, expiry, and the transaction lifecycle”.

5.6 The operator's instruments

What the operator *can* record is a structural property, independent of any logging switch. The gRPC endpoint is operator-terminated TLS, so every connection exposes the client IP and its open and close times — a Prometheus gauge, `grpc_server_connections_current`, tracks the live connection count — and `go-grpc-prometheus` interceptors on both the unary and streaming paths export per-method request counts and latency histograms over the HTTP endpoint (default `127.0.0.1:9068`) (`lightwalletd, cmd/connstats.go, cmd/root.go`).

Logging narrows the distance between *can* and *does*. Every RPC handler logs its full request parameters at Debug level — `gRPC GetBlockRange(%+v)`, `gRPC GetTransaction(%+v)`, `gRPC SendTransaction(%+v)`, `gRPC GetAddressBalance(%+v)`, `gRPC GetAddressUtxos(%+v)` — and full responses at Trace level (`lightwalletd, frontend/service.go`). The default level is Info (`cmd/root.go`), so a single flag, `--log-level 5`, records every block range, txid, raw transaction, and transparent address queried, to the default log file `./server.log`, without client consent or notice. A separate unary interceptor, enabled by `--grpc-logging-insecure` (default `false`; the flag name itself marks the hazard), logs {peer address, method, duration, error} per request, and carries the source comment “TODO: anonymize the addresses. cryptopan?” (`lightwalletd, common/logging/logging.go, cmd/root.go`).³ The project's own architecture document is candid: “Logging may capture identifiable user data. It hasn't received any privacy analysis yet and makes no attempt at sanitization” (`lightwalletd, docs/architecture.md`).

At the pinned Zaino revision, the successor indexer changes none of this structurally: it serves the full `CompactTxStreamer` surface; every handler logs the method name at Info level — the literal message is `[TEST] received call`, apparent leftover test instrumentation in production code — and, under the `prometheus` feature, emits per-method request counts, duration histograms, and error counters. The metric names are `zaino.grpc.requests_total`, `zaino.grpc.request_duration_seconds`, and `zaino.grpc.errors_total`, labelled by method and code, with no peer labels; no peer-address logging exists in the dispatch path (`zaino-serve, src/rpc/grpc/service.rs, src/lib.rs`). Its README acknowledges the setting: “Due to current potential data leaks / security weaknesses ... there is a need to use anonymous transport protocols (such as Nym or Tor) to obfuscate clients' identities from Zcash's indexing servers (`Lightwalletd, Zcashd, Zaino`)”, and the `serve` crate's documentation states the ingestor “has been built so that other ingestors may be added that use different transport protocols (Nym, TOR)” — a capability anticipated, not implemented: no Nym, arti, or onion code exists in the workspace (`zaino, README.md; zaino-serve, src/lib.rs`; negative `grep` over `packages/`). Zaino is pre-1.0; all of the above is cited at commit `0e057e22` and may move.

5.7 Deployed mitigations

Two mitigations ship in the client stack. Neither touches the shielded fetch-on-detect leak of §5.4.

³The interceptor is wired on the unary path only, so streaming RPCs — including `GetBlockRange`, the bulk of sync traffic — never pass through it; the flag logs a skewed subset of traffic. Whether this is intentional is not determinable from the source.

Transport: Tor. Crate `zcash_client_backend` ships an arti-based Tor client behind feature `tor`. Method `Client::connect_to_lightwalletd` tunnels the tonic gRPC channel through Tor (behind the additional feature `lightwalletd-tonic-tls-webpki-roots`), and `Client::isolated_client` returns a handle whose streams never share circuits with the parent, documented for “when you want separate parts of your program to each have a `tor::Client` handle, but where you don’t want their activities to be linkable to one another over the Tor network” (`zcash_client_backend`, `src/tor.rs`, `src/tor/grpc.rs`). Circuit isolation is exactly the tool that would sever the submission channel of §5.5 from sync traffic. We classify this as *capability-in-librustzcash*, *deployment-status-unknown*: whether any deployed wallet routes `CompactTxStreamer` traffic through it by default — and whether `SendTransaction` uses a circuit isolated from sync — is not determinable from the local repositories.

Timing: exponential jitter, transparent addresses only. The SQLite backend schedules ephemeral-address checks by sampling the next check time from an exponential distribution,

$$t_{\text{next}} := t_{\text{from}} + \Delta, \quad \Delta \sim \text{Exp}(\lambda), \quad \lambda = 1/T, \quad \mathbb{E}[\Delta] = T,$$

with the expected interval T set to $(24 \cdot 60 \cdot 60)/n$ seconds for n tracked ephemeral addresses — one expected check per address per day — over a shuffled address order, “so that we don’t always check them in the same order” (`zcash_client_sqlite`, `src/wallet/transparent.rs`, `src/wallet/transparent/ephemeral.rs`). The intent is explicit in the API documentation: only one such query should be performed at a time, “to avoid linking multiple transparent addresses as belonging to the same wallet in the view of the light wallet server” (`zcash_client_backend`, `src/wallet.rs`, `TransparentAddressMetadata::next_check_time`). No analogous jitter exists for shielded-note enhancement (§5.4).

5.8 Summary, and the structural reading

Table 6 collects the observables.

Observable	Reveals	Mitigation status
Client IP, connection timing	wallet identity; activity windows	Tor capability in <code>zcash_client_backend</code> ; wallet deployment unknown
Lowest <code>GetBlockRange</code> start	account birthday	none
Range start, each cycle	prior sync height	bucketing proposed (ZIP 307), unimplemented
Out-of-order ranges	<code>FoundNote</code> blocks holding the wallet’s notes	none (inferred; Remark 5.4)
<code>GetTransaction</code> after scan	the wallet’s own txids, timed	decoys normative (SHOULD), unimplemented
<code>exclude_txid_suffixes</code>	mempool transactions already seen	none
<code>SendTransaction</code>	full transaction, pre-broadcast, linked to origin	circuit-isolation capability, deployment unknown
Transparent-address RPCs	addresses outright	exponential jitter and shuffling (ephemeral addresses only)

Table 6: What the server learns from the deployed light-client protocol, and the status of each mitigation. Sources: §5.4 through §5.7.

The structural reading is that no row of the table is an implementation accident. Every leak above — IP and timing exposure, range start heights, `FoundNote` clustering, fetch-on-detect, submission

linkage — is a property of server-mediated compact-block synchronisation itself: of detection by trial decryption over a server-supplied stream. A client that must ask for ranges reveals where it stopped; a client that must fetch what it detected reveals what it detected. Within the deployed protocol the upgrade path is a stub (ZIP 314, Reserved, Remark 5.2); the exit on offer is architectural. The *Tachyon Guide* treats exactly this layer as the deployed protocol’s removable cost centre: under the fold the wallet never scans, and services are asserted “fully oblivious to their clients’ transaction details” (§“What the fold removes”), while the unresolved analogues of these same leaks — the sync-service trust model, and the timing-correlation linkage the designers call potentially fatal — carry forward as that volume’s §“The sync-service trust model” and §“Timing-correlation linkage”. The present section is the baseline against which those designs ask to be measured.

Part IX

FlyClient Guide: Succinct chain verification: the sampling protocol, the deployed ZIP-221 commitment, and the unbuilt bridge

Abstract

Assuming the deployed chain and header rules of the *Ironwood Guide* and the light-client layer of the *Sync Guide*, this volume of *The Zcash Arboretum* documents succinct chain verification for Zcash: the FlyClient protocol as its paper constructs it, the ZIP-221 chain-history commitment that every Zcash block header has carried since Heartwood, and the distance between the two. The commitment is consensus-critical and universally maintained; no deployed light client consumes it. Claims about the deployed tree cite the ZIPs and the implementations; claims about the sampling protocol carry the paper’s own model assumptions, and its printed defects are registered rather than silently repaired; the bridge between the two is classified plainly as unbuilt.

1 Scope, sources, and the deployment boundary

FlyClient is a light-client protocol that verifies a proof-of-work chain by sampling logarithmically many block headers under a difficulty-weighted distribution, checking each sample against a commitment that every header carries over the entire chain before it (Bünz–Kiffer–Luu–Zamani, “FlyClient: Super-Light Clients for Cryptocurrencies”, IACR ePrint 2019/226; published at IEEE S&P 2020). The commitment is a Merkle mountain range — an append-friendly Merkle tree — and the protocol’s claim is quantitative: a client holding only the genesis block can accept the head of an n -block chain after downloading $O(\lambda \log_{1/c}(n) \log_2(n) + L)$ data rather than n headers, where c bounds the adversary’s mining power relative to the honest network’s and L is the fork length below which the adversary is unconstrained — so the client checks the last L headers in full.

This volume documents two objects and the distance between them. The first is the protocol of the paper: model, commitment, sampling distribution, security theorem, and the proof structure behind it (§2–§4). The second is the consensus structure Zcash actually deployed for it: ZIP-221’s chain-history tree, a difficulty-MMR variant whose root every Zcash block header has committed to since the Heartwood upgrade, extended at NU5 by ZIP-244’s authorising-data commitment (§5–§7). The distance is the volume’s third subject (§8): Zcash ships the server half of FlyClient in every block and validates it in every full node, yet no deployed wallet, SDK, or light-client server exposes or consumes it; deployed light clients instead trust their server for chain state outright (*Sync Guide*, §“Architecture and authority”).

1.1 Sources and pinning

Table 1 pins the artefacts this volume treats as ground truth. Two rigor conventions from the sibling volumes apply. Deployed behaviour cites crate, file, and line against the pinned commits; the classification of design-stage material follows the three-bin rule of the *Tachyon Guide* (§“Scope, sources, and method”): *specified*, *designed but unspecified*, or *open problem*. The FlyClient paper itself is treated as a primary source with registered defects: the revision of record contains internal inconsistencies (a sign typo in a difficulty bound, a vestigial pseudocode return, conflicting evaluation constants) that are catalogued in Remark 3.5 and flagged inline where they matter, never silently repaired.

Artefact	Pin	Rigor
FlyClient paper	ePrint 2019/226, rev. 2020-08-20	peer-reviewed; errata registered
ZIP 221, ZIP 244	zips @ 69610984 (2026-07-09)	merged, deployed consensus
zcash_history	0.5.0 (crates.io)	deployed (Zebra dependency)
zebra	@ de14bef05 (2026-07-17)	deployed validator
zcashd	@ 16ac74376 (2025-10-03)	historical validator (retired)
lightwalletd, Zaino, librustzcash	repo greps, 2026-07-19	negative results (§8)

Table 1: Primary sources, pinned. Verification date 2026-07-19. The paper’s local copy is `.wip/flyclient-drafts/eprint-2019-226.pdf`.

1.2 Relation to the sibling volumes

This volume assumes the *Ironwood Guide* for everything inside a block — notes, commitments, anchors, the NU5 transaction format — and cites its treatment of the note-commitment tree (§“Proof of ownership of *some* valid note: the commitment tree”) rather than re-deriving Merkle mechanics; the general theory of commitments and collision resistance is the *Crypto Guide*’s (§“Commitment schemes”). The *Sync Guide* owns the deployed light-client layer whose central trust assumption — the node/proxy combination “is trusted to provide a correct view of the current best chain state” (ZIP 307, §“Security Model”) — is precisely the assumption a FlyClient bridge would discharge; this volume owns that cross-reference. Toward the frontier volumes the relationship is converse: the *Tachyon Guide* records a draft amendment to ZIP-221 among its peripheral ZIPs, and the *Crosslink Guide*’s finality layer would change what “the best chain” means for any weight-sampling client; both interactions are taken up in §8, and per the sibling-ownership convention the later volume owns the analysis.

1.3 The deployment boundary, stated at the outset

Every claim in this volume sits on one side of a line that the material itself draws sharply.

- *Deployed and consensus-critical.* The ZIP-221 chain-history tree: maintained by every validator, its root committed in every block header since Heartwood (mainnet height 903,000) and folded into `hashBlockCommitments` since NU5 (mainnet height 1,687,104), validated on every block connect by zebra and, until its retirement, zcashd (§5–§7).
- *Specified but consumed by no one.* The same tree as a light-client instrument: ZIP-221’s stated motivation is FlyClient, and the metadata its nodes carry exists for difficulty-weighted sampling; no deployed RPC serves an MMR proof and no deployed client requests one (§8).
- *Designed but not specified for Zcash.* The FlyClient protocol itself — sampling distribution, non-interactive transcript, proof format — exists in the paper with a security theorem in the paper’s own backbone model; no ZIP specifies a Zcash instantiation, and the paper’s model does not automatically cover Zcash’s difficulty-adjustment rule (§4).

2 The problem: verifying a chain without downloading it

2.1 The cost of header-only verification

A full node verifies a proof-of-work chain by downloading and checking every block — for Bitcoin or Ethereum in 2019, more than 200 GB (ePrint 2019/226, §1). Nakamoto’s simplified payment verification reduces this to headers alone: 80 bytes rather than roughly a megabyte per Bitcoin block. The reduction is linear, not asymptotic, and the paper’s motivating arithmetic is Ethereum’s: a 508-byte header every ~15 seconds had produced, by July 2019, more than 8.2 million blocks and hence more than 3.9 GB of headers that an SPV client must fetch and store before verifying its first payment (§1,

p. 3). A device class that cannot bear this — phones, wearables, embedded clients — is exactly the class light clients exist for, and the practical alternative, a trusted wallet provider, “negates much of the benefits of these decentralized ledgers” (§1, p. 2).

An SPV client’s trust model is already weaker than a full node’s. It cannot check transaction validity or consensus rules; it assumes the most-work chain follows the rules — the paper’s Assumption 1: “The chain with the most PoW solutions follows the rules of the network and will eventually be accepted by the majority of miners” (§1, p. 2). The most-work rule itself, and the probabilistic guarantee it purchases, are the *Consensus Guide*’s subject (§“The block tree and the most-work rule”). FlyClient keeps exactly this assumption class (strengthened quantitatively, §4.1) and attacks only the cost: can a client accept the head of the chain after sublinear communication? The obstacle is that a client that no longer checks every proof of work “can be tricked into accepting an invalid chain by a malicious prover who can precompute a sufficiently-long chain using its limited computational power” (§1, p. 3).

2.2 Variable difficulty and the difficulty-raising attack

Any sampling client must contend with an attack that does not exist at fixed difficulty. Under variable difficulty the chain’s total *work*, not its length, decides validity, and an adversary may mine few but heavy blocks: Bahack’s difficulty-raising attack. The paper’s worked figure makes the threat concrete: an adversary holding one third of the honest mining power, permitted a single block of arbitrarily high difficulty, exceeds the expected weight of the honest chain with probability roughly 28% — nothing close to negligible (§3.1, p. 7). Full nodes are protected by the retargeting rule itself: difficulty moves only once per epoch and only within a clamp (the dampening filter τ ; Bitcoin operates with $\tau = 4$, epoch length $m = 2016$). A client that samples, however, never sees most transitions, so the protocol must force *every* transition it could ever rely on to be valid — the purpose of the difficulty MMR’s metadata (§3.5) and of Lemma 4 (§3.6).

2.3 Prior art: superblocks and their failure modes

The prior approach to sublinear chain proofs is the superblock: a block whose hash beats a harder target than required. Superblock counting was folklore by 2012 (the “high-value-hash highway”), was made rigorous as interactive PoPoW by Kiayias–Lamprou–Stouka, and non-interactive as NIPoPoW by Kiayias–Miller–Zindros, who also broke the interactive predecessor (ePrint 2019/226, §1, p. 3). The paper’s case against the superblock family is fourfold (§1, “Current Challenges”):

1. Fixed difficulty is assumed outright, and “it isn’t clear how to modify the super-block based protocols to handle the variable difficulties.”
2. Under variable difficulty the difficulty-raising attack of §2.2 applies, and adding transition checks to superblock proofs “is a non-obvious extension.”
3. Superblocks are identifiable and carry no extra reward, so bribing and selfish-mining strategies suppress them cheaply; NIPoPoW’s published defence reverts to full SPV, so its proofs “are therefore only succinct if no adversarial influence exists.” FlyClient distinguishes blocks only by position, leaving nothing cheap to suppress.
4. NIPoPoW transaction-inclusion proofs cost roughly 15 KB on Ethereum against FlyClient’s ~ 1.5 KB.

The comparison FlyClient claims for itself is summarised by the paper’s Table 1, reproduced as Table 2: three orders of magnitude against SPV at Ethereum’s 2019 length, under an adversary bounded by half the honest hash power and failure probability below 2^{-50} .

One reading habit matters for every number above and below: the paper’s parameter c is the ratio of adversarial to *honest* mining power, not the adversary’s share of the total. An adversary at $c = 0.9$

Block height	10 K	100 K	1,000 K	7,000 K
SPV (KB)	4,961	49,609	496,094	3,472,656
FlyClient (KB)	161	277	416	524
Improvement	31×	179×	1,275×	6,627×

Table 2: Proof sizes for Ethereum parameters (508-byte headers), assuming adversarial hash power at most $c = 1/2$ of the honest hash power and failure probability below 2^{-50} . Reproduced from ePrint 2019/226, Table 1 (p. 4). The 1,275× entry is the paper’s own slip: its SPV and FlyClient sizes give $496,094/416 \approx 1,193\times$ (Remark 3.5).

controls $c/(1+c) \approx 47.3\%$ of the network; the headline $c = 1/2$ rows of Table 2 describe an adversary with a third of total power (§7.1, p. 20).

3 Merkle mountain ranges

3.1 The structure

The commitment at the heart of FlyClient is a Merkle tree variant optimised for one operation: appending a leaf without rebuilding the tree. The paper defines it recursively (ePrint 2019/226, Definition 11, p. 28); the volume restates the definition in its own numbering because everything later — ZIP-221 included — builds on it.

Definition 3.1 (Merkle mountain range). A *Merkle mountain range* (MMR) over n leaves is a binary hash tree M with root r and depth $\lceil \log_2 n \rceil$ such that, if $n > 1$, writing $n = 2^i + j$ with $i = \lfloor \log_2(n-1) \rfloor$ (so $1 \leq j \leq 2^i$): the left child subtree of r is an MMR with 2^i leaves, and the right child subtree of r is an MMR with j leaves. Each internal node holds the hash $H(\text{left} \parallel \text{right})$ of its children’s values, for a collision-resistant H .

The left subtree is therefore the largest *perfect* binary tree the leaf count admits, and the recursion repeats on the remainder: an MMR over $n = 6$ leaves splits $6 = 4 + 2$ into a perfect four-leaf subtree and a perfect two-leaf subtree; an MMR over $n = 7$ splits $7 = 4 + 3$ and the right side splits again as $3 = 2 + 1$. Every MMR is a balanced binary tree in the paper’s sense (depth at most $\lceil \log_2 n \rceil$), so ordinary Merkle inclusion proofs of at most $\log_2(n) + 1$ hashes exist for every leaf (Definitions 9–10, p. 27).

Remark 3.2 (Peaks and bagging, absent by construction). The folklore MMR presentation — Peter Todd’s, which the paper credits for the structure — keeps a *forest* of perfect trees (“peaks”), one per set bit of n , and combines (“bags”) them into a single root only when one is needed. Neither word appears anywhere in the paper: its Definition 11 is single-rooted from the start. The two presentations coincide exactly — the peaks of the classical forest are the left children of the nodes along the paper tree’s right spine together with the terminal right-spine node, and the paper’s root is the result of bagging the peaks right-to-left — but the difference in framing matters when reading ZIP-221, which specifies the same object in yet another way (§5), and when reading `zcash_history`, which stores the peaks explicitly (§7). Costs and proofs are identical between the folklore forest and the paper’s single-rooted form. ZIP-221 instead bags left-fold-leftmost (§5.2), the opposite of the paper’s implicit right fold, so with three or more peaks its bagged root and the hashes at the top of a root-anchored path differ from the paper’s; the stored mountain array and the asymptotic costs are the same.

3.2 Appending in logarithmic time

Appending leaf x to an n -leaf MMR (the paper’s Algorithm 5, AppendLeaf, p. 29) has two cases. If the tree is perfect ($n = 2^i$), a new root is minted with the old tree as left child and x as right child,

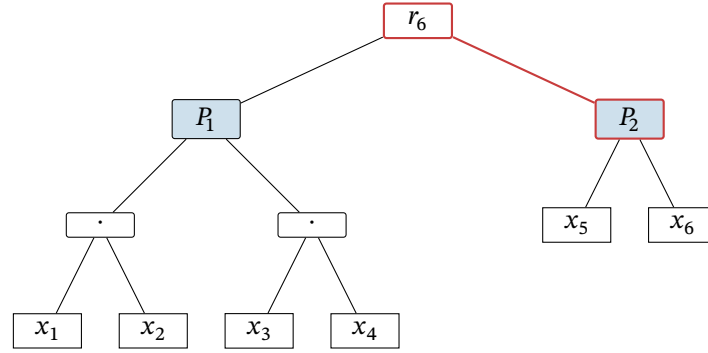


Figure 1: The MMR over six leaves $x_1, \dots, x_6$, drawn in the paper’s single-rooted form (Definition 3.1): $6 = 4 + 2$, so the root r_6 has the perfect four-leaf tree rooted at P_1 as left child and the perfect two-leaf tree rooted at P_2 as right child. The nodes P_1 and P_2 (blue fill) are exactly the *peaks* of the classical forest presentation — here the root’s left child P_1 and the terminal right-spine node P_2 (red edges) — and appending leaf x_7 rewrites only the spine: P_1 and P_2 and every node beneath them are shared unchanged with r_7 .

hashing $H(r \parallel x)$. Otherwise the append recurses into the right subtree and rehashes on the way out. Only the right spine is ever rewritten, so the cost is $O(\log n)$ and — decisive for a blockchain host — every perfect left subtree of the old MMR is shared, untouched, with the new one. Theorem 6 of the paper proves by induction that the result is the MMR over the old leaves with x appended rightmost.

3.3 The prefix-extension trick

An ordinary Merkle proof shows that a leaf lies under a root. FlyClient needs strictly more: when the verifier samples block k from a chain whose head commits to n blocks, it must check not only that block k is leaf k under the head’s root but that the MMR root stored *inside* block k ’s own header commits to exactly the first $k - 1$ of those same leaves — that the sampled block’s view of history is a prefix of the head’s. The MMR delivers this without any additional proof data.

Theorem 7 (ePrint 2019/226, p. 30). *For $k \leq n$, given $\Pi_{x_k \in M_n}$, the Merkle proof that leaf x_k is in the n -leaf MMR M_n , a verifier can regenerate r_k , the root of the k -leaf MMR M_k .*

The proof is an induction on the shape of Definition 3.1, and its content fits in one sentence: in this tree shape, the *left* siblings along the inclusion path of leaf k are exactly the roots of the perfect subtrees covering leaves $1, \dots, k - 1$, so folding them together bottom-up — the paper’s `Get_Root` routine (Algorithm 6) — reconstructs the prefix root from the very path that proved inclusion. One path, two facts: x_k is the k -th leaf under r_n , and the root over the first $k - 1$ leaves is a value the verifier can now compare against the root stored in the sampled header. The two corollaries the protocol leans on follow directly (p. 31): the MMR is position-binding for prefixes (Corollary 3: the path commits $x_1, \dots, x_k$ as the blocks of a chain that is a prefix of the head’s), and any change to any block changes every later root (Corollary 4) — so an adversary that forks the chain can reuse no honest block from after its fork point, since the honest MMR roots inside those blocks would betray the substitution.

Remark 3.3 (Printed defects in Algorithm 6). As printed, `Get_Root` assigns from an undefined proof index (“ $r = \Pi[i]$ ” where the loop variable is y), ends by comparing $y = r$ and returning a bit although its specification, its call site in Algorithm 2, and Theorem 7 all require it to return the reconstructed root, and is called with its arguments in a different order than it declares. The intended behaviour is unambiguous from Theorem 7’s proof — initialise with the first left sibling encountered, fold $r \leftarrow H(y \parallel r)$ over the remaining left siblings bottom-up, return r — and every implementation below behaves so. Registered in Remark 3.5.

3.4 Headers as chain commitments

FlyClient’s consensus change is one field. At every height i the block producer appends the hash of B_{i-1} to the running MMR and records the new root M_i in the header of B_i (§2, pp. 5–6): the header of a block commits, through one MMR root, to exactly the sequence of blocks strictly before it. A verifier holding one trusted header therefore holds a commitment to the whole chain, and the per-sample check set is (Algorithm 2 with §2): the inclusion path hashes to the head’s root; the Get_Root fold of the same path equals the root stored in the sampled header; the sampled header’s proof of work is valid.

Remark 3.4 (Indexing drift, and the invariant that survives it). The paper is not consistent about names: §2 calls the root recorded in B_i “ M_i ”, Definition 5 and Lemma 4 call the root inside the head B_n “ M_{n-1} ”, and Algorithms 1–2 speak of a chain of $n + 1$ blocks whose head commits “the first n blocks”. Every variant expresses the same invariant — *a block’s header commits to all blocks strictly before it* — and this volume uses that sentence, not any one of the indexings. ZIP-221 will shift the convention once more (the root in block i covers leaves through block $i - 1$, but the tree begins at an upgrade activation rather than at genesis; §5).

3.5 The difficulty MMR

Sampling by position is the fixed-difficulty story. Under variable difficulty the verifier must sample by *weight* — the distribution of §4.4 works over relative aggregate difficulty — and must be able to reject invalid difficulty transitions it will never individually inspect (§2.2). Both needs are served by enriching every MMR node with aggregate metadata.

Definition 7 (Difficulty MMR; ePrint 2019/226, p. 18). *A difficulty MMR is a variant of the MMR with identical properties, but such that each node contains additional data. Every node can be written as $\{h, w, t, D_{\text{start}}, D_{\text{next}}, n, \text{data}\}$, where h is a λ -bit hash output, w is an integer total difficulty, t is a timestamp, D is a difficulty target, n is the size of the subtree rooted at the node, and data is an arbitrary string. In a leaf, $n = 1$ and t is the time it took to mine the block; w, D_{start} are the block’s difficulty target and D_{next} the next block’s target under the difficulty-adjustment function. Each non-leaf node is $\{H(lc, rc), lc.w + rc.w, lc.t + rc.t, lc.D_{\text{start}}, rc.D_{\text{next}}, lc.n + rc.n, \perp\}$ for children lc, rc .*

Aggregation is thus fixed once per field: weights and times sum, targets propagate from the boundary children, counts add. The verifier recomputes every parent along a sampled path from its children and checks, per level: that $lc.D_{\text{next}} = rc.D_{\text{start}}$ (adjacent subtrees agree on the target across their boundary); that nodes below the epoch level $\log_2 m$ carry exactly their epoch’s difficulty; that nodes spanning a transition computed D_{next} by the adjustment rule; and that a node’s aggregate (w, t) is *feasible* — some legal sequence of clamped transitions between its boundary targets could have produced them. The paper bounds feasibility explicitly: maximal weight comes from raising the difficulty by the dampening factor τ as fast as allowed and then descending, minimal weight from the reverse; a maximal difficulty decrease needs an epoch span of at least $\tau m/f$ rounds and a maximal difficulty increase at most $m/(f\tau)$. For the simplified scenario $\tau^k D_{\text{start}} = D_{\text{next}}$ over n epochs (the paper’s own worked case, pp. 18–19) the printed upper bound is

$$w \leq D_{\text{start}} \cdot \frac{(\tau + 1) \tau^{(k+n)/2} - \tau^{1+k} - 1}{\tau - 1},$$

and the printed lower bound repeats the exponent $(k + n)/2$ where evaluating its own summation gives $\tau^{-(n-k)/2}$ — a sign typo, verified against the PDF at high resolution, under which the printed bound would be negative; the summation form is authoritative (Remark 3.5).

3.6 Invalid transitions never help

The transition checks would be pointless if an adversary could profit from difficulty rules the verifier fails to sample. The paper closes this with a rewriting argument.

Lemma 4 (ePrint 2019/226, p. 19). *Let $\mathcal{A}$ be an adversary in the variable-difficulty backbone model that produces, with non-negligible probability p , a chain C in which k blocks are valid. Then, assuming a collision-resistant hash function, there exists an adversary $\mathcal{A}'$ using the same number of oracle queries that respects the retargeting rules and produces, with probability at least $p - \text{negl}(\lambda)$, a chain C' containing the same valid blocks.*

The proof’s mechanism is worth keeping, because it explains why the per-node checks of §3.5 suffice. Consider an invalid adjustment between B_i and B_{i+1} in $\mathcal{A}$ ’s chain. No valid inclusion proof for B_i exists, and along any claimed path there is a highest node x' at which the verifier’s recomputation fails; every proof through x' is rejected, so *every leaf under x' ’s parent* is already unsampleable-without-rejection. The rewritten adversary $\mathcal{A}'$ replaces the difficulty transitions inside that whole subtree with valid ones consistent with the parent’s aggregate fields — possible precisely because the parent’s own $(w, t, D_{\text{start}}, D_{\text{next}}, n)$ pass the feasibility checks — and the blocks inside need no valid proof of work at all. Invalid transitions therefore buy nothing: the adversary might as well have used valid transitions and mined more invalid blocks, which is exactly the adversary the sampling analysis already counts.

Remark 3.5 (Errata register for the revision of record). The ePrint revision of 2020-08-20 — the version this volume pins — contains the following printed defects, all verified against the PDF and none affecting the results: (i) the sign typo in the difficulty-MMR lower bound (§3.5); (ii) the `Get_Root` pseudocode’s undefined index, vestigial boolean return, and call-signature mismatch (Remark 3.3); (iii) two near-duplicate evaluation paragraphs in §7 carrying conflicting MMR-node overheads (8 versus 16 bytes) — the variable-difficulty evaluation is the 16-byte one; (iv) a difficulty-bomb sentence in §7.2 asserting a proof-size increase where its own mechanism and the adjacent sentence describe a decrease; (v) `AppendLeaf`’s base case hashing $H(x \parallel r)$ in prose against $H(r \parallel x)$ in the algorithm — the algorithm is operative; (vi) Definition 7’s feasibility checks referencing $t_{\text{start}}, t_{\text{end}}$ fields the definition never declares, and drifting between D_{next} and D_{end} ; (vii) Definition 1’s target-recalculation formula uses a symbol T — in both the clamp bounds and $n(T, \Delta)$ — that its own constants list never declares; (viii) heavily overloaded symbols (δ, k, n, λ each carry two or more meanings across sections); (ix) Table 1’s 1,275× improvement entry, which its own SPV and FlyClient sizes contradict ($496,094/416 \approx 1,193\times$; the printed ratio matches a FlyClient size of ~ 389 KB) while the other three entries recompute correctly. The acknowledgements record that CCS reviewers found “Bahack style attacks and problems with the security proof in a previous version” — the variable-difficulty machinery of §3.5 is post-review repair work, which explains some of the visible seams.

4 The FlyClient protocol

4.1 The model, and what the adversary may do

The analysis lives in the variable-difficulty Bitcoin backbone model of Garay–Kiayias–Leonardos. Execution proceeds in rounds; each active player makes q sequential random-oracle queries per round; a block B with target T is valid iff $H(B) < T$; broadcast messages arrive next round, with the adversary free to reorder deliveries. Honest participation $n = \{n_r\}$ is assumed (γ, s) -respecting — in any window of fewer than s rounds, honest power varies by at most a factor γ — and targets are recalculated once per epoch of m blocks: with the last m blocks mined in Δ rounds, the raw retarget $T' = T\Delta f/m$ (aiming at m blocks per m/f rounds) is clamped so that the target moves by at most a factor τ per epoch, the *dampening filter* (ePrint 2019/226, Definition 1, §3.1, p. 7; the paper notes Bitcoin operates with $\tau = 4, m = 2016, f = 0.03$). The clamp is not a tuning detail: it is what makes the difficulty-raising attack of §2.2 survivable, and every feasibility check of §3.5 is an image of it. Backbone parameters are chosen so persistence and liveness hold, so a single chain is held by all honest full nodes.

The adversary is *rushing* (each round it sees all honest messages before choosing its own), mines with at most a μ fraction of honest power, and wants the light client to accept a chain of at least the honest chain’s cumulative difficulty. The verifier knows only the genesis block. Two structural facts already constrain the attack. A fork can carry no honest block from after its fork point — the MMR

roots inside those blocks would not match the fork (Corollary 4, §3.3) — and no adversarial work from before the fork point can be re-used after it, since the MMR forces each block to be created after everything before it. What remains is the assumption the whole analysis is priced in:

Assumption 2 ((c, L) -adversary; ePrint 2019/226, §3.2, p. 8). *There exists no adversary in the variable-difficulty model that respects the target recalculation function from Definition 1 and can, with non-negligible probability, produce a fork that contains more than L blocks such that a c fraction of the difficulty weight in these blocks is honest.*

Remark 4.1 (Reading Assumption 2, and reading c). The printed clause “a c fraction ... is honest” is loose; the meaning used throughout the paper’s own analysis is a cap on the adversary: in any fork of length at least L , at most a c fraction of the fork’s difficulty weight can be validly mined, so a fork matching the honest chain’s weight is at least a $(1 - c)$ fraction fake. Separately, c is the ratio of adversarial to *honest* mining power — an adversary at $c = 0.9$ holds $c/(1 + c) \approx 47.3\%$ of the network’s total (§7.1, p. 20). Short forks are deliberately outside the assumption: for small L the variance of mining lets any adversary win occasionally, which is why L trailing blocks are always checked in full. Connecting (c, L) formally to the backbone parameters $(\mu, \gamma, s, \dots)$ is conjectured possible and left open by the paper; the constant-difficulty case is the one bridge actually proved:

Lemma 1 (ePrint 2019/226, p. 8). *In the constant-difficulty backbone, let X be the number of blocks mined by any adversary while the honest chain adopts L blocks, the adversary’s rate at most μ times the honest adoption rate. Then for $c > \mu$,*

$$\Pr[X \geq cL] \leq e^{L(c-\mu)} \cdot (c/\mu)^{-cL}.$$

The proof is a Chernoff bound on a dominating Poisson variable with mean μL , optimised at $t = \ln(c/\mu)$; Corollary 1 then picks, for every μ and $L = \Theta(\lambda)$, a $c < 1$ making the right-hand side $2^{-\lambda}$ — so at constant difficulty the (c, L) -assumption is a theorem, not an axiom.

4.2 What security means here

The object being proved about is the paper’s SPV predicate: for a chain C ending in B_n , the predicate holds iff every block carries correct proof of work following the difficulty-adjustment function and each block’s hash is contained in its direct successor’s header (Definition 2, p. 9). Because persistence guarantees nothing about the last k blocks, proofs for chains differing only in the last k blocks count as proofs for the same chain, the highest-difficulty one representative. The verifier is a light client knowing only genesis, with oracle access to H for checking work; it receives proofs from several provers, *at least one honest* — an assumption that deliberately prices eclipse attacks out of scope — and accepts the predicate for the head of greatest cumulative difficulty. A proof protocol is *secure* if the verifier’s accepted head shares a common prefix (up to the last k blocks) with the chain of every honest full node (Definition 3), and *succinct* if every proof any efficient prover can emit has size $O(\text{polylog}(N))$ in the honest chain length N (Definition 4, inherited from the NIPoPoW paper). FlyClient is thus a NIPoPoW in the primitive sense; “NIPoPoW” as a protocol name usually means the superblock construction of §2.3, which FlyClient replaces.

4.3 The interactive game

The interactive protocol (Algorithms 1–2, p. 11) is short enough to give in full. Each prover claims a chain of $n + 1$ blocks and opens with its head, whose header carries the MMR root over the first n blocks. If all provers present the same head, the client is done. Otherwise the client samples k heights per prover from the distribution of §4.4; for each sampled height the prover returns that block’s header with its MMR inclusion path, and the client runs the per-sample check set of §3.4: the path folds to the head’s root; the `Get_Root` fold of the same path equals the root stored inside the sampled header; the sampled header’s proof of work is valid, at a target the difficulty-MMR metadata declares and

constrains (§3.5). Any failure rejects the prover’s chain; a chain surviving all samples is accepted. Provers claiming different lengths are handled by the NIPoPoW generic verifier, longest claim first.

Two consequences of the commitment structure do quiet work here. First, transaction verification decouples from synchronisation: once the head is accepted, membership of an old block is one MMR inclusion proof against the head’s root, and membership of a transaction is one ordinary SPV Merkle proof inside that block — roughly 1.5 KB on Ethereum parameters against ~15 KB for superblock NIPoPoW (§1, p. 4). Second, the unstable tip is harmless: even a maliciously chosen head cannot commit to invalid blocks or omit stable ones — its MMR must contain every stable block of the honest chain, or sampling exposes it — so the client may run the protocol against the freshest head and still learns the stable prefix correctly; it stores one recent *stable* block between executions (§4.2, pp. 11–12).

4.4 From uniform sampling to the optimal distribution

The sampling distribution is the information-theoretic heart of the protocol, and the paper builds it in three failed-then-fixed steps worth keeping, because each failure names a constraint the final distribution satisfies.

Uniform sampling fails on short forks. If the fork point a were known, every sample past it would be invalid with probability $1 - c$ and k samples would catch the fork with probability $1 - c^k$. But the verifier does not know a , and a fork near the tip hides from uniform samples: almost all land before the fork, where the adversary’s chain is honestly valid (§5.1, p. 12).

Binary search is interactive. With one honest prover guaranteed, the verifier can binary-search the first height at which two provers disagree — $\log n$ adaptive queries — and then sample k blocks past the fork point. This works but is multi-round and adaptive, so it cannot be made non-interactive by Fiat–Shamir; FlyClient needs a one-shot, public-coin protocol (§5.2, p. 13).

Dyadic suffixes get the right shape. The one-shot fix oversamples the tip: for every $j \in [0, \log n)$, sample k blocks uniformly from the last $n/2^j$ blocks, and once the interval is at most $\min(L, k)$ blocks, check all of them.

Lemma 2 (ePrint 2019/226, p. 13). *With $k \log n$ samples, the probability that the verifier fails to sample any invalid block is at most $((1 + c)/2)^k$.*

The proof needs only one of the $\log n$ intervals: whatever the fork point, some dyadic interval of size n' has it in its first half, so more than $n'/2$ of that interval is post-fork and at least $(1 - c) \cdot n'/2$ of it is fake; k uniform samples from that interval alone miss with probability at most $((1 + c)/2)^k$. The analysis discards every other interval’s samples — the paper’s own stated question, “can we achieve a better bound?”, is what the optimal distribution answers.

The optimal distribution. Reordered, the dyadic protocol is q i.i.d. draws from a staircase density; the right question is which density maximises the single-draw probability p of hitting a fake block, against an adversary who places its validly-mined blocks adversarially — then q draws catch with probability $1 - (1 - p)^q$. Model the chain as the continuum $[0, 1]$, genesis at 0, head at 1. Two structural facts pin the answer. A non-increasing density is never uniquely optimal — an adversary shifts fakes toward the lower-probability late interval, so mass should increase toward the tip (Lemma 3, pp. 14–15). Against an increasing density, the optimal adversary forking at a puts all its validly-mined weight at the end of its fork, so the segment $[a, 1 - (1 - a)c]$ is entirely fake and the single-draw catch probability at fork point a is $\int_a^{1+ac-c} f(x) dx$ (normalised). Maximising the minimum over a forces indifference — the same catch probability at every fork point — and differential analysis yields

$$f(x) = \frac{1 - c}{c(1 - x)}, \quad \int_a^{1+ac-c} f(x) dx = \frac{(c - 1) \ln c}{c} \text{ for every } a,$$

an integral independent of a , as required. The density diverges at the tip — $\int_0^1 f = \infty$ — and the repair is the protocol’s signature move: restrict sampling to $[0, 1 - \delta]$ and *always check the last δ fraction of the chain in full*. The normalised sampling density is

$$g(x) = \frac{1}{(x - 1) \ln \delta}, \quad x \in [0, 1 - \delta],$$

(both factors negative, so $g > 0$), and the single-draw catch probability becomes $p = \log_\delta(c)$: choosing $\delta = c^k$ gives $p = 1/k$ exactly, while any fork point past $(c - \delta)/c$ pushes the fake segment into the always-checked suffix and is caught with probability 1.

Theorem 2 (Optimal sampling distribution; ePrint 2019/226, p. 16). *Given that the last $\delta = c^k$ fraction of the chain contains only valid blocks and the adversary can create at most a c fraction of valid blocks after the fork point, the sampling distribution with PDF $g(x) = 1/((x - 1) \ln \delta)$ maximises the probability of catching any adversary that optimises the placement of invalid blocks.*

Optimality is an averaging argument: any candidate density must give catch probability at least p^* to each of the k fork points $a = 1 - c^i$, and the corresponding intervals tile the support, so $1 \geq kp^*$ and no density beats $1/k$. The sample count follows from $p_m = (1 - 1/k)^m \leq 2^{-\lambda}$:

$$m \geq \frac{\lambda}{\log_{1/\lambda}(1 - 1/\log_c(L/n))}, \quad m \approx \lambda \log_c(1/2) \ln(n) = O(\lambda \log_{1/c} n),$$

where the always-checked suffix of $L = \delta n$ blocks fixes $k = \log_c(L/n)$. The suffix length trades against the sample count and is bounded below by the (c, L) -assumption itself; the implementation optimises it numerically (about a hundred blocks at Ethereum parameters, Figure 4). Each sampled block costs a header plus a $\log_2(n)$ -hash Merkle path, giving:

Corollary 2 (ePrint 2019/226, p. 17). *Under the (c, L) -adversary assumption for any constant L , and using a collision-resistant hash function, the FlyClient proof size is $\Theta(\lambda \log(n) \log_{1/c}(n) + L)$.*

Unlike the superblock protocols, whose succinctness degrades to full SPV under adversarial influence, this bound holds against *all* adversaries.

4.5 Removing the verifier

The protocol is public-coin — the verifier contributes only randomness — and one-shot, so Fiat-Shamir applies: the sampling randomness is derived by hashing the head of the chain with an ordinary secure hash (the paper suggests SHA-3), and a verifier of the transcript checks the derivation along with the samples (§6.2, p. 19). Grinding is priced in proof-of-work: a cheating prover obtains fresh randomness only by re-mining the head, and the assumption bounds its total puzzle budget by $c \cdot n$, so a union bound makes the non-interactive protocol secure whenever the interactive soundness satisfies

$$p_m < \frac{2^{-\lambda}}{c \cdot n}.$$

Two deployment properties fall out (§8.1, p. 21). Proofs are *transferable*: anyone can relay a proof without recomputation. And for a given chain the proof is *unique* — the randomness is a function of the head — so one party can compute the proof for the entire network.

4.6 The theorem, and staying synchronised

Theorem 1 (FlyClient; ePrint 2019/226, pp. 10, 20). *Assuming a variable-difficulty backbone protocol such that all adversaries are limited to be (c, L) -adversaries as per Assumption 2, and assuming a collision-resistant hash function H , the FlyClient protocol is a secure NIPoPoW protocol in the random-oracle model as per Definition 3 with all but negligible probability. The protocol is succinct with proof size $O(L + \lambda \log_{1/c}(n) \log_2(n))$.*

The proof is the composition of everything above, in five steps: Assumption 2 speaks only of retargeting-respecting adversaries, and Lemma 4 (§3.6) converts every other adversary into one at no loss; Merkle soundness plus the prefix-extension corollaries make the MMR position- and weight-binding, so sampled answers are pinned to one chain; Theorem 2 with Corollary 2’s parameters makes evasion of sampling negligible; Fiat–Shamir is secure for one-round public-coin protocols in the random-oracle model; and the transcript — L suffix blocks plus $O(\lambda \log_{1/c} n)$ sampled blocks, each with a $\log_2(n)$ -hash path — has the stated size.

A synchronised client also stays synchronised cheaply. Theorem 3 (Subchain proofs, §8.2, p. 22): a client that accepted a chain of length n , later handed a proof for the extension from n to n' together with one MMR proof that its block n lies in block n' ’s commitment, accepts nothing it would not have accepted from a full n' -proof — a fork after n faces a fresh FlyClient instance with block n as genesis, and a fork before n fails the prefix proof outright. In practice no custom subproof is needed: the full proof is transferable, so a returning client verifies only the part past its checkpoint, and servers can precompute checkpoint proofs at chosen heights.

4.7 What the evaluation showed

The implementation created and verified every tested proof in under a second. At Ethereum’s 2019 parameters ($c = 1/2$, $\lambda = 50$, n up to seven million), the numerically-optimised trailing suffix stays near a hundred blocks, the sample count near 400–470, and the whole proof under 530 KB — against 3.4 GB of SPV headers, the 6,627× of Table 2 — with the paper’s “MMR proof optimization” (unexplained in the text, but plausibly deduplication of shared path nodes across samples) accounting for roughly a third of the size (§7, Figures 4, 6). One curiosity in the Ethereum data is instructive: proof size *falls* between three and four million blocks because the difficulty bomb concentrated so much weight near the tip that the always-checked suffix covered a larger weight fraction, cutting the sample count — difficulty growth helps the sampler. Two caveats frame all the numbers.

Remark 4.2 (Ethereum is outside the theorem). Ethereum’s moving-average difficulty rule does not fit the variable-difficulty backbone model, so Theorem 1 does not cover it; the paper is explicit that running FlyClient on Ethereum is possible “only with heuristic security guarantees” (§3.1, pp. 7–8; §7, p. 20). The point recurs for Zcash in §8: whether Zcash’s own averaging-window retarget (distinct from both Bitcoin’s and Ethereum’s) satisfies the model is exactly the kind of question a Zcash FlyClient ZIP would have to answer, and none exists.

The second caveat is historical and recorded in the paper’s own acknowledgements: CCS reviewers found “Bahack style attacks and problems with the security proof in a previous version” — so the variable-difficulty analysis is repair work, added after review. Finally, the paper closes the circle to proofs of sequential work: the chain construction is essentially Cohen–Pietrzak PoSW with block hashes as leaves, and their verifier is FlyClient with a uniform distribution — a FlyClient chain *is* a PoSW, with the stronger guarantee that no point of the chain has a constant fraction of corrupted leaves (§8.4, pp. 23–24).

5 ZIP-221: the deployed chain-history tree

Zcash deployed the server half of FlyClient as consensus. ZIP-221 — title “FlyClient — Consensus-Layer Changes”, Status Final, deployed with the Heartwood upgrade — replaces the block header’s Sapling treestate commitment with a commitment to the root of a Merkle mountain range over the chain’s history, each node of which carries the metadata a difficulty-sampling verifier needs. The header field’s lineage tells the story compactly: `hashReserved` before Sapling, `hashFinalSaplingRoot` from Sapling, renamed `hashLightClientRoot` by ZIP-221 at Heartwood, renamed again `hashBlockCommitments` by ZIP-244 at NU5 (§6). The stated motivation is the paper’s protocol verbatim: MMR proofs are to let light clients verify “the validity of a block chain received

from a full node”, block inclusion, and “certain metadata of any block or range of blocks in that chain” (ZIP 221, §“Motivation”), with header downloads dropping from linear to logarithmic.

5.1 Epoch scoping: one tree per network upgrade

The single largest structural departure from the paper is what the tree covers. The paper’s MMR runs from genesis; ZIP-221’s does not:

“The protocol requires that an MMR that commits to the inclusion of all blocks since the preceding network upgrade ($B_x, \dots, B_{n-1}$) is formed for each block B_n . The root M_n of the MMR MUST be included in the header of B_n .” (ZIP 221, §“Motivation”; x is the activation height of the preceding network upgrade.)

Three consequences follow from the exact definition (§“Tree nodes and hashing”). First, the commitment is delayed one block: the root inside B_n covers leaves through B_{n-1} only, so a block never commits to itself — the ZIP’s rationale notes the corollary that a block’s final Sapling root now surfaces one block later than it did pre-Heartwood, and accepts it because anchoring on the most recent treestate “is already unsafe in reorgs”. Second, the tree *resets at every network upgrade*: the preceding-upgrade height x is the last activation height strictly below n , so the block at an activation height A carries the completed previous epoch’s root, and the very next block $A + 1$ already commits to the new epoch’s single-leaf tree over B_A . Every consensus branch thus owns a self-contained tree — which composes with the personalisation scheme below into cross-branch domain separation — and any future Zcash FlyClient must chain per-epoch proofs across upgrade boundaries, a composition the ZIP does not discuss. Third, the genesis and activation edge cases are handled by sentinel: `hashChainHistoryRoot` is undefined at genesis, and in the block that activates the ZIP the header field is all zero bytes, which “MUST NOT be interpreted as a root hash”.

5.2 Numbering, peaks, and bagging

Where the paper specifies its MMR as a single-rooted recursion (Definition 3.1) and never says “peak”, ZIP-221 specifies the same object in the classical presentation the paper omitted (Remark 3.2), adapted from Grin’s: nodes are numbered in *creation order* — each leaf followed immediately by every internal node its arrival completes — and the forest of perfect subtrees (“mountains”, leftmost peak always the highest) is *bagged* into a single root by repeatedly joining the two *leftmost* peaks: a left fold, so peaks P_1, P_2, P_3 bag as $(P_1 || P_2) || P_3$. Navigation is arithmetic on positions: a node at position p with altitude h has its right sibling at $p + 2^{h+1} - 1$ and its left child at $p - 2^h$, with the ZIP’s own caveat that bagging nodes fall outside these rules. The ZIP’s eleven-leaf worked example — 19 nodes at positions 0–18, peaks at positions 14, 17, 18 with altitudes 3, 1, 0 — and both navigation identities reproduce exactly under this volume’s independent implementation (script-verified 2026-07-19). Reorgs run the construction backwards: deleting the rightmost leaf pops the right spine and re-bags, $O(\log k)$ in the right subtree’s leaf count.

5.3 Node metadata: the complete field list

Every node — leaf, internal, root, treated uniformly — carries the same field vector, serialised in the exact order of Table 3. A leaf takes both members of each earliest/latest pair from its own block; an internal node inherits *earliest*-fields from its left child, *latest*-fields from its right child, and sums the additive fields. The ZIP splits the vector by purpose: seven fields — the timestamp, target, and height pairs, plus the cumulative work — are the “FlyClient Requirements”, chosen “via discussion with an author of the FlyClient paper” to let a light client reason about the difficulty-adjustment algorithm over a block range, exactly the role Definition 7 of the paper assigns its $(w, t, D_{\text{start}}, D_{\text{next}}, n)$ payload (§3.5); the remaining fields are Zcash-specific light client aids with no counterpart in the paper. Among the latter, the shielded transaction counts are the interesting design: they let a client

reliably learn whether a malicious server is withholding shielded transactions, and let it skip header ranges containing none — the ZIP notes the dominant light-client cost is transmitting Equihash solutions in headers. An earlier draft counted Sprout and Sapling together; it was rejected because a Sapling-only client could not distinguish a withheld Sapling transaction from an unrequested Sprout one, and the ZIP fixes the pattern that every future pool gets its own count — which NU5’s Orchard trio and the draft Ironwood trio then follow (§6).

#	Field	Leaf value	Internal rule
1	hashSubtreeCommitment	the block’s consensus block hash, internal byte order, <i>unpersonalised</i>	personalised BLAKE2b-256 of the two children’s full serialisations
2	nEarliestTimestamp	header nTime	inherit left
3	nLatestTimestamp	header nTime	inherit right
4	nEarliestTargetBits	header nBits	inherit left
5	nLatestTargetBits	header nBits	inherit right
6	hashEarliestSaplingRoot	final Sapling treestate root	inherit left
7	hashLatestSaplingRoot	final Sapling treestate root	inherit right
8	nSubTreeTotalWork	$\lfloor 2^{256} / (\text{ToTarget}(\text{nBits}) + 1) \rfloor$	sum (modulo 2^{256})
9	nEarliestHeight	block height	inherit left
10	nLatestHeight	block height	inherit right
11	nSaplingTxCount	count of transactions with non-empty vSpendSapling or vOutputsSapling	sum
12	hashEarliestOrchardRoot	final Orchard treestate root	inherit left
13	hashLatestOrchardRoot	final Orchard treestate root	inherit right
14	nOrchardTxCount	count of transactions with non-empty vActionsOrchard	sum

Table 3: The ZIP-221 node field vector, in serialisation order. Fields 1–11 are the V1 vector (Heartwood onward); fields 12–14 join at NU5 and are omitted before it. Hash fields are `char[32]`, timestamps and target bits `uint32`, work `uint256`, heights and counts `CompactSize` integers. The seven FlyClient-requirement fields are 2–5 and 8–10. Sources: ZIP 221, §“Tree Node specification”; ZIP 252.

Serialised size follows from the widths: the V1 fixed portion is $32 + 4 + 4 + 4 + 4 + 32 + 32 + 32 = 144$ bytes plus three `CompactSize` integers of 1–9 bytes each, giving the ZIP’s stated 147–171 bytes; NU5 adds 64 fixed bytes and one more `CompactSize`, giving 212–244 (script-verified against both stated ranges). Two of the ZIP’s own field notes are worth carrying: `nLatestTimestamp` may be *smaller* than `nEarliestTimestamp` in subtrees narrower than `PoWMedianBlockSpan`, because header timestamps are not consensus-monotonic at that scale; and within one branch, the leaf hash of B_{n-1} equals `hashPrevBlock` of the following header exactly, so the tree adds no new hashing at the leaves.

5.4 Hashing and personalisation

All internal hashing is BLAKE2b-256 with the 16-byte personalisation `ZcashHistory || CONSENSUS_BRANCH_ID` — the ASCII prefix’s twelve bytes followed by the branch ID’s four little-endian bytes, so for Heartwood’s branch `0xF5B9230B` the trailing bytes are `0B 23 B9 F5`. Baking the branch ID into the personalisation means “valid nodes from one consensus branch cannot be used to make false statements about parallel consensus branches” (ZIP 221, §“Rationale”) — composed with epoch scoping, each upgrade’s tree is domain-separated from every other’s. Two boundaries of the scheme are exact: a *leaf*’s hash field is the block’s consensus block hash itself, unpersonalised; and an *internal* node hashes the concatenation of its children’s full canonical serialisations — metadata and all, not merely their digests — so every aggregate field is hash-bound at every level. The header commitment is one more hash of the same kind: `hashChainHistoryRoot` is the personalised BLAKE2b-256 digest of the *serialised root node*, so the 32-byte header field commits to the whole epoch’s aggregates — total work, boundary timestamps and targets, boundary treestate roots, shielded transaction counts — not

only to the leaf set.

Remark 5.1 (Which branch ID, at an activation block). The ZIP’s prose sets the personalisation branch ID to that of “the epoch of the block containing the commitment”, but its pseudocode hashes with the branch ID carried by the *nodes* (`make_parent` uses the left child’s, `make_root_commitment` the root’s). Within one epoch-scoped tree the two readings coincide everywhere except at an activation block, whose header carries the *previous* epoch’s finished root: there the pseudocode — and the ZIP’s own summary that “each node in the tree commits to the consensus branch that produced it” — resolve the tension in favour of the tree’s own branch ID, and the deployed implementations agree (§7).

5.5 The consensus rules at Heartwood

The consensus delta is three rules on one renamed field (ZIP 221, §“Block header semantics and consensus rules”): before activation, the Sapling rule persists under the new name (`hashLightClientRoot` is the final Sapling treestate root); in the activation block, the field **MUST** be all zero bytes, not interpreted as a root; in every subsequent block, the field **MUST** equal `hashChainHistoryRoot`. The header’s byte format and version are untouched — a deliberate boundary, argued in the rationale: ZIP-301 obliges miners to ignore jobs with unknown header versions, so a version bump risks mining-hardware breakage for a change that is semantically invisible to miners. The rationale also disposes of two alternatives: overloading is safe against a miner contriving a Sapling treestate whose root collides with an MMR root (a preimage over 32 layers of Pedersen hashing), and the seemingly cleaner home for the root — replacing `hashPrevBlock`, exactly the paper’s suggestion (§3.4) — was rejected because it would require “massive refactoring” and hurt performance, reorgs in particular being “fragile, performance-critical” and reliant on backwards iteration over the chain history.

5.6 Deltas from the paper’s construction

For a reader holding both objects, the differences reduce to six that matter and a summary judgement. The tree is epoch-scoped, not genesis-rooted, with a one-block commitment delay (§5.1). Node metadata is richer than Definition 7 of the paper: the seven `FlyClient` fields realise the paper’s $(w, t, D_{\text{start}}, D_{\text{next}}, n)$ payload — with both ends of each pair carried explicitly rather than one derived — and the Sapling/Orchard fields have no paper counterpart. Hashing is personalised, branch-separated BLAKE2b-256 over full child serialisations, where the paper hashes bare child values with an unspecified generic H . The root reaches the header through an existing repurposed field rather than a new one — and, from NU5, only as the first input of an outer commitment (§6). The bagging order is fixed left-fold-leftmost (§5.2) — the opposite of the right fold the paper’s recursion implies (Remark 3.2), so the two constructions’ bagged roots differ whenever the tree has three or more peaks. The ZIP’s own summary is accurate: “Other than the metadata commitments, the MMR tree’s construction is standard”, and its append algorithm is “adapted from” the paper’s.

Remark 5.2 (Defect ledger for a Final ZIP). The ZIP’s illustrative pseudocode — explicitly non-normative, but the only executable statement of intent the ZIP contains — has at least six defects in the checked-out text: the node class declares `nSaplingTxCount` twice where the second must read `nOrchardTxCount`; `serialize` and `get_peaks` reference instance fields without `self./node.` qualification; `make_parent` reads `sapling_root/orchard_root` attributes that exist on the block type, not the node type; `append` and `delete` read `latest_height/earliest_height` where the node fields are named `nLatestHeight/nEarliestHeight`; and a stray Rust-ism (`peaks.push`) survives in the Python. None affects the normative field table or consensus rules; all are trivially repaired by the reference implementations (§7). Separately, no ZIP-221 test vectors exist anywhere in the zips repository.

Remark 5.3 (The ZIP’s own security caveat). ZIP-221 is unusually candid about the limits of its security story, and the caveat is load-bearing for §8: the `FlyClient` paper “only analyses these security properties

in chain models with slowly adjusting difficulty, such as Bitcoin”, leaves rapidly-adjusting chains — “such as Zcash or Ethereum” — as an open problem with “only heuristic security guarantees”, and although the difficulty-reasoning fields were chosen with a paper author, “the use of these fields has not been analysed in the academic security literature”. The ZIP concludes that “a more in-depth security analysis of FlyClient should be performed before designing a FlyClient-based light client ecosystem for Zcash”. This is the same boundary Remark 4.2 drew from the paper’s side: Zcash’s averaging-window retarget stands outside the proven model, and nobody has closed the gap in either direction since.

6 NU5: the commitment becomes one of two

6.1 Why an authorising-data root exists

NU5’s ZIP-244 made transaction identifiers non-malleable by excluding witness data — proofs and signatures — from the txid digest. The repair creates a gap: `hashMerkleRoot`, now built over non-malleable txids, no longer commits to the whole serialised transaction. ZIP-244 fills it with a parallel commitment: each transaction’s *authorising data commitment* digests exactly the excluded material (transparent input scripts; Sapling proofs and signatures; Orchard proofs, spend-authorisation signatures, and binding signature) under fixed BLAKE2b-256 personalisations, gathered by an outer per-branch digest, with pre-v5 transactions assigned the placeholder of 32 0xFF bytes; and `hashAuthDataRoot` is the root of a padded binary Merkle tree (personalisation `ZcashAuthDataHash`, null leaves `[0u8; 32]`) over those commitments, in insertion order identical to the txid tree, so one path position names the same transaction in both trees. The pair (txid tree, auth tree) restores full commitment to every byte a block carries.

6.2 hashBlockCommitments

Rather than widen the header, ZIP-244 overloads the ZIP-221 field a second time. From NU5 the field is named `hashBlockCommitments` and holds the personalised digest

$$\text{BLAKE2b-256}_{\text{ZcashBlockCommit}}(\text{hashLightClientRoot} \parallel \text{hashAuthDataRoot} \parallel [0\text{u8}; 32]),$$

whose first input is the ZIP-221 chain-history root, second the authorising-data root, and third a 32-zero-byte terminator. The chain-history root therefore no longer appears directly in any NU5 header; a client checking it must carry the auth-data root (or its subtree) alongside any MMR proof. The terminator is deliberate architecture: the ZIP describes the value as “a linked list of hashes terminated by arbitrary data”, so a future upgrade can replace the zeros with the hash of a further structure ending in its own terminator; full validators must always use the entire structure of the latest *activated* protocol version they support, and servers for older light clients SHOULD supply the replacement value so old verification equations keep closing. Two activation details differ from Heartwood’s: there is *no* zero-byte sentinel at NU5 — the activation block already uses the new derivation, its inner chain-history input being the completed Canopy-epoch root — and the same upgrade extends the node vector with the Orchard trio (Table 3, fields 12–14), following the one-count-per-pool pattern. The draft ZIP-258 continues both patterns for NU6.3: an Ironwood trio (fields 15–17, aggregated identically to the Orchard trio, node size 277–317 bytes — the arithmetic checks against the field widths), with `hashChainHistoryRoot` changing “only through the added node fields” and `hashBlockCommitments` otherwise unaffected. That trio is the one this series’ Ironwood volume already cites at its turnstile (*Ironwood Guide*, §“Migration and the turnstile”). ZIP-258 remains Draft as a document, but these rules and fields are active consensus and are implemented by Zebra; the ZIP proposes that `zcashd` will not implement NU6.3.

Upgrade	Branch ID	Mainnet	Testnet	Node vector
Heartwood	0xF5B9230B	903,000	903,800	V1 (fields 1–11)
NU5	0xC2D6D0B4	1,687,104	1,842,420	V2 (+ Orchard trio)
NU6.3	0x37A5165B	3,428,143	4,134,000	V3 (+ Ironwood trio)

Table 4: Chain-history activation constants. Sources: ZIP 250, ZIP 252, ZIP 258 (Draft). The NU5 testnet height is the second activation: a first testnet NU5 at height 1,599,200 under branch 0x37519621 (an earlier Orchard circuit) was rolled back, the testnet forking from height 1,599,199.

6.3 Activation constants

7 Deployed implementations

Three codebases realise ZIP-221: the `zcash_history` crate (“to be used in zebra and via FFI bindings in `zcashd`”, its own words), Zebra’s wrapper and state service, and `zcashd`’s C++ cache with a Rust FFI bridge. Zebra pins version 0.5.0 of the crate; `zcashd` pins 0.4.0 — a skew with consequences registered in §7.4.

7.1 The `zcash_history` crate

The crate stores the MMR in its *array representation*: leaves and completed internal nodes in append order, addressed by `u32` index. Two link kinds separate what persists from what does not (`src/lib.rs:42--49`): `Stored(u32)` indexes the persistent array; `Generated(u32)` indexes an ephemeral per-instance vector holding the *bagging* nodes that join unequal peaks into a root — and serialisation refuses entries with `Generated` children (`src/entry.rs:98--100`), so exactly ZIP-221’s node array persists, never the bag. An entry serialises as a one-byte tag (node or leaf), two little-endian `u32` child indices for nodes, then the node data (`src/entry.rs:69--106`); the node data is ZIP-221’s field vector in ZIP order, `CompactSize` integers included, with maxima asserted in tests at exactly the ZIP’s figures — 171 bytes (V1), 244 (V2), 317 (V3, Ironwood), entry maximum $317 + 9 = 326$ (`src/node_data.rs:463--482`). One deliberate absence: the consensus branch ID is *context, not data* — it is never serialised, and every deserialiser must supply it (`src/node_data.rs:135--139`).

Hashing matches §5.4 exactly (`src/version.rs:21--26`, `43--65`, `102--106`): the personalisation is `ZcashHistory || LE32(branch ID)`; a parent’s commitment hashes the concatenation of both children’s *full serialisations*; and the hash of the root node’s serialisation is `hashChainHistoryRoot`. A dedicated test (`v3_combine_hash_commits_to_ironwood_fields`) pins the property that parent hashes commit to child metadata, and a conformance module walks all three node versions against the `zcash-test-vectors` ZIP-221 vectors, generated by an independent Python reimplementation (`src/test_vectors.rs:1--13`).

The `Tree` type is explicitly a *view*, not a container (`src/tree.rs:5--15`): it is instantiated per operation from a caller-supplied peak set — `Tree::new(length, peaks, extra)`, which inserts the peaks and left-folds them into generated bagging nodes — used for a few appends or truncations, and dropped. Appending (`src/tree.rs:137--194`) pushes the leaf, collects peaks by descending from the root and stopping at complete subtrees, merges equal-sized peaks right-to-left into *stored* nodes, and re-bags the remainder left-to-right into *generated* nodes; truncation runs the inverse, consuming the caller-supplied extra nodes down the right spine. Completeness itself is derived from metadata, not shape bookkeeping: a subtree is complete iff `end_height - (start_height - 1)` is a power of two (`src/entry.rs:38--45`) — an elegant economy with a sharp edge registered below. Notably, the crate never computes which array positions are peaks; every caller reimplements the walk — the crate’s own example 1-based, Zebra and `zcashd` in a matching 0-based form (highest altitude $\lfloor \log_2(n+1) \rfloor - 1$, first candidate $2^{\text{alt}+1} - 2$, descend left while past the end, then hop right siblings) — and the same fourteen-leaf, twenty-five-node ASCII diagram annotates the walk in both Zebra and `zcashd`, Zebra’s comment attributing the code to the crate example and the explanation to `zcashd`.

7.2 Zebra

Zebra’s wrapper (`zebra-chain`, `src/primitives/zcash_history.rs`) builds leaves from blocks: branch ID from the block’s network upgrade, subtree commitment the block hash, both ends of each pair the block’s own time, target, treestate roots and height, work from the block’s difficulty threshold, and the pool transaction counts — with a named `BlockCommitmentTreeRoots` struct whose stated purpose is that the Orchard and Ironwood roots, which share a Rust type, “cannot be silently swapped, which would corrupt the ZIP-221 chain-history commitment” (lines 23–34). Above it, `NonEmptyHistoryTree` (`src/history_tree.rs`) dispatches the node version by upgrade — Heartwood/Canopy to V1, NU5 through NU6.2 to V2, NU6.3/NU7 to V3, panic before Heartwood — and implements the epoch reset with one line: on a push whose network upgrade differs from the tree’s, “This is the activation block of a network upgrade. Create a new tree.” — the value is wholesale replaced by a fresh single-leaf tree (lines 238–247). Every push then prunes: the peak-set walk retains only peak entries, and the in-memory crate tree is rebuilt from peaks alone, neutralising the view type’s unbounded growth. The whole object is wrapped once more as `HistoryTree(Option<NonEmptyHistoryTree>)`, empty before Heartwood — the crate itself cannot represent an empty tree (its constructor panics on an empty peak set), so emptiness lives a level up. A guard worth quoting sits on push: heights must be consecutive because “`librustzcash` assumes the heights are correct and corrupts the tree if they are wrong” (lines 224–236) — the completeness-from-heights economy, fenced.

Validation splits across two layers. Structurally, `Commitment::from_bytes` (`zebra-chain`, `src/block/commitment.rs:108--151`) parses the header field by epoch into the five-armed enum — pre-Sapling reserved, final Sapling root, the all-zero `ChainHistoryActivationReserved` at the Heartwood activation height, `ChainHistoryRoot` through Canopy, `ChainHistoryBlockTxAuthCommitment` from NU5 — rejecting a non-zero activation sentinel outright. Contextually, the state service (`zebra-state`, `src/service/check.rs:134--222`) compares: for Heartwood/Canopy blocks, the parent chain’s tree hash against the header root; for NU5 onward, it recomputes

$$\text{BLAKE2b-256}_{\text{ZcashBlockCommit}}(\text{history root} \parallel \text{auth_data_root} \parallel [\text{0u8}; 32])$$

from the parent tree and the block’s own authorising-data tree and compares that. The check runs in the non-finalized state (in parallel with anchor checks and the chain push) and again for checkpointed blocks in the finalized state; `zebra-consensus` itself never touches the field, a division of labour the code documents — the checkpoint verifier “doesn’t bind the transaction authorizing data”, the state services do. Persistence is minimal by design: one RocksDB column family holding exactly one record — the current tip tree’s peaks — under the unit key `()`, atomically replaced per block write; a previous epoch’s tree exists nowhere in Zebra once its activation is finalized. Reorgs never truncate: the non-finalized state keeps an Arc-shared `HistoryTree` snapshot per height per chain fork, so rolling back is dropping snapshots, and the crate’s truncation machinery goes unused. The mining path closes the loop: `getblocktemplate` responses carry `chain_history_root`, “The FlyClient chain history root as of the end of the chain tip block” (`src/response.rs:530--536`) — producers, too, consume the tree.

7.3 zcashd

The `zcashd` node keeps the tree behind an FFI whose Rust side is thin (`src/rust/src/history.rs`): fixed-width byte arrays (244-byte nodes, 253-byte entries — version 0.4.0’s V2 maxima), three entry points (`append`, `remove`, `hash_node`), and a branch-ID dispatch that names Sprout, Overwinter, Sapling, Heartwood, and Canopy for V1 — omitting Blossom, which falls through the *everything else* arm to V2 (harmless: history trees exist only from Heartwood). The C++ side (`src/coins.cpp:384--630`) holds one `HistoryCache` *per epoch*, keyed by consensus branch ID; `PushHistoryNode` preloads the peak set with the same walk as everywhere else, appends through the

FFI (at most 32 new nodes per append — the leaf plus at most 31 ancestors, the 2^{32} tree bound made concrete), and `PopHistoryNode` handles reorgs case by case, including the pleasing impossibility proof in an exception: “a history tree cannot have two nodes” — one leaf is length 1, two leaves are length 3. Validation happens in `ConnectBlock` (`src/main.cpp:3505--3614`): the chain-history root is read from the *previous block’s* epoch’s cache — which at an activation block is the previous epoch’s final root, exactly ZIP-221’s rule — and the header field is checked against the Heartwood sentinel, the bare root (pre-NU5), or `DeriveBlockCommitmentsHash`, byte-identical to Zebra’s recomputation. Persistence is the opposite pole from Zebra’s: LevelDB stores every epoch’s *full node array* indefinitely — per-node records keyed by (epoch, index), plus a length and root per epoch — because `zcashd`’s rollback model reopens old epochs’ trees, and its RPC exposes `chainhistoryroot` on `getblock`. A width workaround mirrors Zebra’s: V1 nodes written by pre-NU5 clients occupy 171-byte records that NU5-aware clients read and zero-pad into 244-byte buffers.

7.4 Divergence register

The two validators agreed on every consensus-relevant byte for as long as both ran; the register below records where they differ in structure, and one place they would have differed in consensus had both crossed the NU6.3 boundary unchanged.

- *Version skew at NU6.3.* `zcashd`’s 0.4.0 dispatch sends every post-Canopy branch ID to V2 nodes, while Zebra selects V3 (Ironwood fields) for NU6.3 — as the code stands, the two would compute divergent `hashChainHistoryRoot` values from the activation block onward. ZIP-258 resolves the tension by fiat rather than by code: `zcashd` “will not implement” NU6.3, and Zebra is expected to be the sole full validator (§6); the skew is a documented retirement, not a live consensus bug — but the code alone does not say so.
- *Epoch retention.* Zebra destroys a superseded epoch’s tree (tip-only, single-record persistence); `zcashd` retained every epoch’s full array, because its rollback path could reopen them. Two correct answers to the same question, shaped by each node’s reorg model.
- *Truncation asymmetry.* The crate’s `truncate_leaf` and extra-node machinery are exercised only by `zcashd`; Zebra’s per-height snapshots make deletion a non-event. A bug in truncation would be a `zcashd`-only bug.
- *A skippable check.* `zcashd` verifies the NU5 recomputation only under the `fCheckAuthDataRoot` flag; the pre-NU5 root equality has no such flag. Zebra checks unconditionally on both paths.
- *Fragile economy, fenced differently.* Subtree completeness derived from height metadata means wrong heights corrupt tree shape silently; Zebra fences with a consecutive-height assertion citing exactly this failure, `zcashd` relies on `ConnectBlock` ordering.
- *Edge-case fallback.* Zebra’s NU5 check substitutes the 32-zero-byte sentinel as the “previous root” if the NU5-style commitment appears at the Heartwood activation height itself — reachable only on Regtest or configured testnets; `zcashd` has no equivalent arm.

8 The deployment gap

8.1 Who computes the root, who verifies it

Every Zcash block since Heartwood carries a correct chain-history commitment, because two classes of software maintain the tree. Consensus validators compute and check it: Zebra in the state service and, until its retirement, `zcashd` in `ConnectBlock` (§7). Miners publish it without computing it: both nodes’ `getblocktemplate` hand the miner a `defaultroots` object containing `chainhistoryroot` (and, from NU5, `authdataroot` and `blockcommitmentshash`), so template consumers hash what the

node computed — the state-service response type behind the template documents the field as “The FlyClient chain history root as of the end of the chain tip block”, and Zebra’s built-in miner rides the same RPC. Even this producer-facing surface has had consumer-visible bugs: a `zcashd` comment memorialises that v4.6.0’s legacy template fields “were accidentally set to always be the NU5 value” (`src/miner.cpp:462--466`).

Verification, meanwhile, is purely *reconstructive*: a validator maintains the tree itself and compares 32-byte roots. Nobody, in any deployed software, verifies an MMR *proof* — the operation the structure exists to serve. The RPC surface tells the same story from the read side: `zcashd`’s `getblock` exposes `chainhistoryroot` per block, while Zebra — the maintained node — never emits the root for an existing block at all; its `getblock` code carries the literal comment “`chainhistoryroot` would be here. Undocumented. TODO: decide if we want to support it” (`zebra-rpc, src/methods.rs:3971`), and no RPC in either node serves MMR nodes or assembled proofs.

8.2 The light-client stack consumes none of it

The deployed light-client protocol was specified before ZIP-221 and never updated toward it: ZIP-307 does not reference ZIP-221 once, and its security model is the one the *Sync Guide* documents at length — the node/proxy combination “is trusted to provide a correct view of the current best chain state”. The negative results are uniform across the stack (all greps 2026-07-19): `lightwalletd` contains no FlyClient, chain-history, or MMR code and serves no related endpoint; Zaino ships the same gRPC protocol and touches the field only to pass it through — serialising the header’s `hashBlockCommitments` bytes, and carrying a comment on its header type, “Optional fields may be added for: `hashLightClientRoot` (FlyClient proofs)...”, a capability anticipated, not implemented; neither `zcash_client_backend` nor `zcash_client_sqlite` nor `zcash_primitives` depends on `zcash_history`; the Android SDK has no trace. What the wallet stack does instead is exactly the *Sync Guide*’s subject: hash-chain continuity (`prevHash` and `height`) and nothing more — ZIP-307’s own text concedes its header-validation section “has been only partially implemented: currently only `prevHash` is checked”, and a grep of the wallet backend for Equihash or difficulty code returns nothing.

The consequence deserves plain statement: a Zcash FlyClient would not be “SPV, but smaller”. Deployed Zcash wallets perform no proof-of-work verification whatsoever, so a FlyClient verifier would be the *first* PoW verification the wallet stack has ever done — and the seven metadata fields ZIP-221 carries for it, plus the shielded-transaction counts added specifically so light clients could detect withholding (§5.3), have waited six years without a single consumer.

8.3 What an actual Zcash FlyClient still needs

- *Prover data availability.* A proof server needs random access to the full node array. Zebra persists peaks only (§7.2); `zcashd` persists everything a prover needs (§7.3) but exposes none of it and is being retired.
- *Serving endpoints.* No RPC or gRPC anywhere serves an MMR node, an inclusion path, or an assembled proof — the entire wire surface remains to be designed and specified.
- *The sampling client.* No implementation of the verifier loop — weight-proportional sampling under *g*, per-sample checks, difficulty-transition feasibility over ZIP-221’s metadata — exists in any wallet or SDK.
- *Specification work.* Open in full: a proof wire format; the Fiat–Shamir transcript definition for Zcash’s header; the composition rule for chaining per-epoch trees across upgrade boundaries (§5.1 — the ZIP is silent); the security analysis Zcash’s own rapidly-adjusting difficulty rule requires (Remark 5.3); and the marriage of FlyClient header verification with ZIP-307 scanning, which replaces the header-chain trust but not the compact-block detection channel.

- *Practicality data.* The one end-to-end datapoint is academic and recent (web-sourced: Perazzo-Capecci, “Catching the Fly”, arXiv 2604.26736, April 2026): the first working Zcash FlyClient prover, built on a zebra fork with four added RPCs and a ~10-hour database backfill to recover the node array Zebra discards; proofs of ~1.2 MiB at three million blocks (~320 KiB with their distillation), with Equihash’s 1,344-byte solutions dominating per-sampled-header cost. The numbers say mobile plausibility and on-chain-verifier marginality — and that the retrofit cost of Zebra’s peaks-only economy is real but bounded.

Remark 8.1 (An inversion of readiness). The deployment gap has a neat summary: the node that has the prover’s data (zcashd, full per-epoch node arrays in LevelDB) has no future, and the node that has the future (Zebra) has no data. Neither has an interface. Meanwhile consensus keeps paying the feature’s maintenance cost — the NU6.3 upgrade extends the node vector a second time (§6), Zebra already implements it, and still nothing reads the tree.

8.4 Elsewhere: velvet paths and inspired bridges

Context from beyond Zcash, all web-sourced. ECC’s Heartwood announcements framed ZIP-221 as “Flyclient support” enabling interoperability — capability language, with no client deliverable then or since; a 2021 community-forum thread asking about FlyClient implementation drew no answers. The follow-up literature validated Zcash’s deployment choice from an unexpected angle: superlight clients deployed by *velvet* fork — the uncontentious path the paper itself proposed (ePrint 2019/226, §8.3, p. 23) — admit *chain-sewing* attacks, in which portions of independent forks are stitched into a proof (Kiayias–Polydouri–Zindros, AFT 2021); Zcash’s hard-fork deployment forecloses the class, a point the 2026 prover paper makes explicitly. No published attack on FlyClient’s sampling under its stated model was found; the critical literature attacks velvet deployments and the realism of the adversary model, not the distribution. The nearest production relative is FlyClient-*inspired* rather than FlyClient: Harmony’s Horizon bridge design checkpoints MMR roots for an on-chain client — the roots are final by supermajority vote of a fixed validator committee, a Byzantine-fault-tolerant (BFT) protocol safe while fewer than a third of the committee misbehaves arbitrarily, not by probabilistic PoW sampling. Claims that Grin or Beam “use FlyClient” conflate carrying MMR roots in headers (which Mumblewimble chains do natively — the paper’s own observation) with running a sampling verifier, for which no evidence surfaced.

8.5 Relation to the frontier

Two frontier designs of this series touch the tree, and per the sibling-ownership convention this volume owns both pointers. The *Tachyon Guide* records a draft amendment to ZIP-221 among Tachyon’s peripheral ZIPs — unmerged, “still under discussion and nothing has been decided yet” in the designers’ words — so the aggregation layer of a Tachyon-era chain may yet become another node field, exactly as Orchard and Ironwood did. The *Crosslink Guide* changes the meaning of the quantity FlyClient proves: a FlyClient transcript establishes cumulative *proof-of-work weight*, while Crosslink’s trailing finality layer makes a prefix of the chain final by BFT assertion — under such a hybrid, a weight-sampling proof of the unfinalized suffix and a certificate for the finalized prefix are different objects with different trust bases, and no analysis of their composition exists in either direction. Neither interaction is more than noted here; both are open in exactly the sense of §8.3’s specification bullet.

8.6 Closing the boundary

The volume’s three-bin summary, restated with everything above behind it. *Specified and deployed:* the chain-history tree — epoch scoped, metadata-rich, branch-separated — maintained by every validator and committed in every header for six years, with conformance test vectors and two independent implementations that agree byte for byte. *Designed but unspecified for Zcash:* the FlyClient pro-

TOCOL itself, proved secure in a model that Zcash's difficulty rule is not known to satisfy — the ZIP says so, the paper says so, and nothing published since closes the question. *Open*: the bridge — data availability, wire format, epoch composition, the sampling client, and the security analysis — of which only an academic prototype exists, on a fork, six years after the consensus layer shipped its half. The chain has kept its side of an agreement whose other party has yet to arrive.

Part X

Voting Guide: The May–June 2026 Zally shielded-voting deployment snapshot

Abstract

This volume of *The Zcash Arboretum* documents the commit-pinned May–June 2026 shielded-voting deployment snapshot. No specification of the protocol has merged anywhere; the commit-pinned implementation is the de facto specification, and this volume documents that implementation: ballot structure and eligibility proving against a snapshot of the note-commitment tree, the layered nullifier defences against double voting, threshold ElGamal tallying under a validator DKG, and — stated without euphemism — the trust assumptions and the one confirmed gap between the published description and the deployed circuit.

1 The protocol and its provenance

This volume documents the *Shielded Voting Protocol* — internally “Zally”, shipped to users as *Coinholder Polling* — the first governance mechanism to run over Zcash’s shielded pool in production. The protocol lets a holder of Orchard notes vote with the weight of their unspent funds, privately: the funds never move, the notes and addresses are never revealed, and only aggregate totals become public. The internal name appears in the client library’s own description of itself (“the Zally governance protocol”, valargroup/zcash_voting@624cf198, README.md); the draft ZIP carries the public name (draft-valargroup-shielded-voting.md, zcash/zips PR #1200); the wallet changelog carries the product name (zashi-android@b4edd6f6, CHANGELOG.md). This section fixes what shipped, where the design came from, and — decisive for every citation in this volume — what the specification actually is.

1.1 What shipped

In outline: a voting round pins a snapshot of the Zcash chain — an Orchard note-commitment root and a nullifier-set root. A holder proves in zero knowledge that they control notes which existed and were unspent at the snapshot, and delegates their quantised weight — one ballot per 0.125 ZEC (12,500,000 zatoshi per ballot, voting-circuits/src/params.rs) — to a round-scoped governance hotkey. The hotkey casts each vote as ElGamal-encrypted split shares on a dedicated permissioned chain, svoted, whose ballot surface is five messages (vote-sdk/proto/svote/v1/tx.proto); relayers reveal shares with timing decorrelation; and a per-round threshold *Election Authority* — a distributed key generation among the chain’s validators — decrypts aggregates only (vote-sdk/x/vote/keeper/keeper_tally.go). The remainder of this volume constructs each of these objects in turn; here we record only their provenance.

Deployment. Coinholder Polling shipped in Zodi — the wallet formerly Zashi; the team left ECC in January 2026 and the application package identity is unchanged — version 3.5.0 on Android and iOS, released 2026-05-28, with signing support for the Keystone hardware wallet (zashi-android@b4edd6f6, CHANGELOG.md, entry [3.5.0 (1736)]); a follow-up release, 3.5.2, followed on 2026-06-04 (ibid., entry [3.5.2 (1742)]). The intended first production use — the NU7 sentiment coinholder poll, announced 2026-05-15 with a 2026-06-10 open, its close extended on 2026-05-29 from June 24 to June 29 to end with the parallel ZCAP poll of the same questions — never ran. On 2026-06-07 — three days before the open, the day after the NU7 scope collapsed to the Ironwood supply-soundness upgrade — the poll was paused “for both coin holders and the Zcash Foundation ZCAP”, and as of 2026-06-14 the next round “is delayed until after Ironwood activates”. No results exist. A production

round was provisioned in the signed round registry on 2026-06-05 (§4.1), but no public evidence shows a ballot cast (searched 2026-07-15).⁴

Authorship. Valar Group designed and implemented the protocol, led by Dev Ojha (GitHub ValarDragon); the draft ZIP is authored by GitHub user p0mvn, also of Valar Group. The upstream orchard crate gained the protocol’s one in-tree dependency — the `unstable-voting-circuits` feature, which widens circuit internals (the `NoteCommit`, `nullifier-derivation`, and `Commitivk` chips) for out-of-tree reuse — in a commit by Adam Tucker (zcash/orchard @ c0b6b917, 2026-04-23). ZODL engineers str4d (Jack Grigg), Daira-Emma Hopwood, and Kris Nuttycombe reviewed — they did not design — the architecture in Q1 2026 (ZODL retroactive grant application, ZcashCoinholder-GrantsProgram issue #33); ZODL also built the wallet integration.

1.2 Lineage

Coin-weighted shielded polling on Zcash has one prior implementation: hhanh00’s `zcash-vote`, a standalone application that ran the 2024 developer-fund poll and the April–May 2025 lockbox poll (hhanh00/`zcash-vote` @ f05f5f25, `src/election.rs`; it depends on a forked orchard at rev 75448e67 with a `vote` feature). Its ZIP — `zcash/zips` PR #853, “Coin Voting ZIP” — has been open since 2024-05-28 and remains unmerged as of 2026-07-08. Zally is independent of it: neither codebase references the other (verified by search over both trees, 2026-07-08), and Zally reuses the upstream orchard crate through a feature flag where `zcash-vote` patches in its fork. The two share the core idea — pin a snapshot, prove note eligibility against it, and prevent double voting with a per-election nullifier — but Zally adds hotkey delegation through a dedicated note type (the `Vote Authority Note`), split ElGamal shares, a threshold Election Authority, relayers, private eligibility queries by single-server PIR (private information retrieval: the server answers without learning which entry was queried), a dedicated chain, and hardware signing, and it lives inside the production wallet rather than beside it.

Two adjacent mechanisms are not ancestors. ZCAP, the Zcash Community Advisory Panel, polls an identified panel through Helios with no coin weighting; the June 2026 NU7 question was scheduled through both instruments in parallel and paused for both together (§1.1), which makes the distinction operational, not academic. Hopwood’s draft `draft-ecc-onchain-accountable-voting.md` (created 2025-02-21, in the merged zips tree at c6b70358) targets the complement — accountable on-chain voting by identified key-holders — and shares no design lineage with either.

1.3 The source situation: code as the de facto specification

No normative document exists. The would-be specification, `zcash/zips` PR #1200 (“[ZIP TBD] Shielded Voting Protocol”, opened 2026-03-05), is open and unmerged as of 2026-07-08, not updated since 2026-06-02, and carries a review comment flagging a section as “Underspecified”; the companion PIR draft, PR #1198, is likewise open. The design book the implementation itself cites — `valargroup/shielded-vote-book`, linked from the `zcash_voting` README — is private,

⁴Announcement: josh, “NU7 Sentiment Polling: Questions for Community Review & Coinholder Voting via Zodl”, forum topic 55713, 2026-05-15; extension: *ibid.* post 71, 2026-05-29; pause: *ibid.* post 73, 2026-06-07. Wayback snapshot of the topic carrying the pause notice, 2026-07-15: <https://web.archive.org/web/20260715193300/https://forum.zcashcommunity.com/t/nu7-sentiment-polling-questions-for-community-review-coinholder-voting-via-zodl/55713>; pre-pause schedule (RSS snapshot, 2026-06-05): <https://web.archive.org/web/20260605074855/https://forum.zcashcommunity.com/t/nu7-sentiment-polling-questions-for-community-review-coinholder-voting-via-zodl/55713.rss>. Scope change: “Ironwood: Verifying the Soundness of Zcash’s Circulating Supply”, topic 56044, 2026-06-06, Wayback 2026-07-15: <https://web.archive.org/web/20260715193458/https://forum.zcashcommunity.com/t/ironwood-verifying-the-soundness-of-zcash-s-circulating-supply/56044>. Deferrals: ZODL updates, topic 56072, 2026-06-07, Wayback 2026-07-15: <https://web.archive.org/web/20260715193328/https://forum.zcashcommunity.com/t/subduing-the-world-update-from-zodl/56072>; and topic 56211, 2026-06-14, Wayback 2026-07-15: <https://web.archive.org/web/20260715193352/https://forum.zcashcommunity.com/t/irrational-thinking-zodl-update/56211>.

as is its mirror `z-cash/shielded-vote-book` (both return 404 to anonymous access, verified 2026-07-08). The public documentation (`valargroup.gitbook.io/shielded-vote-docs`, fetched 2026-07-08) is prose: its circuit pages list public and private inputs and some Poseidon preimages (the arithmetisation-friendly hash constructed in the *Crypto Guide*, §“Arithmetisation-friendly hashing: Poseidon”), but omit the governance-nullifier, VAN-nullifier, and share-nullifier formulas, the nullifier-domain derivation, the proposal-bitmask width, and the indexed-Merkle-tree leaf model — and it misstates the ballot-quantisation rule, the sole open problem this volume records (the closing paragraph of this section).

The consequence governs this volume: the commit-pinned implementation of Table 1 is the de facto specification, and what we document is that implementation. Every governance-specific formula in the chapters that follow is read from these sources at these commits, never from designer prose; where the prose and the code disagree, the code is the protocol, and we flag the disagreement.

Repository	Commit	Date	Role
<code>valargroup/voting-circuits</code>	<code>4c39abd5</code>	2026-06-03	circuits: prover <i>and</i> verifier
<code>valargroup/vote-sdk</code>	<code>cb915f51</code>	2026-06-05	vote chain: validation, DKG, tally
<code>valargroup/zcash_voting</code>	<code>624cf198</code>	2026-06-07	client: proofs, scheduling, persistence
<code>valargroup/vote-nullifier-pir</code>	<code>603e4e3b</code>	2026-06-03	PIR: nullifier non-membership
<code>valargroup/spendability-pir</code>	<code>a924008d</code>	2026-06-04	PIR: spendability and witness queries
<code>zcash/orchard</code>	<code>c0b6b917</code>	2026-04-23	unstable-voting-circuits feature

Table 1: The de facto specification: the sources this volume documents, pinned by commit. Wallet-side integration is pinned separately: `zcash-android-wallet-sdk` @ `f386369e` depends on `zcash_voting` =0.11.0 and `orchard` =0.14.0 with the `unstable-voting-circuits` feature (`backend-lib/Cargo.toml`); `zashi-android` @ `b4edd6f6`, `zashi-ios` @ `8ab0e6ed`.

Current-head boundary (2026-08-30). The implementation has moved substantially since this snapshot. Current heads are `voting-circuits` `bd2531f` (release 0.11.2), `vote-sdk` `7cf9b9c`, `zcash_voting` `4535b0d`, and `vote-nullifier-pir` `20e9c4a` (release 0.11.2). The current delegation builder requires Ironwood V3 notes, the circuit verifying keys and integration APIs have changed, and the client, chain, and PIR services have gone through several coordinated release lines. The one-ballot under-claim window proved in this volume remains explicitly documented in the current circuit. Unless a paragraph names one of these later heads, every formula and deployment-behaviour claim below is about the pinned snapshot in Table 1; this is historical protocol documentation, not a compatibility guide for the current stack.

The single-crate caveat. `Crate voting-circuits` exposes both the provers and the verifiers of all three circuits (`verify_delegation_proof`, `verify_vote_proof`, and `verify_share_reveal_proof`, each in its circuit directory’s `prove.rs`), and the chain consumes the same crate for validation: `vote-sdk/circuits/Cargo.toml` pins `voting-circuits` at version 0.8.0, and `vote-sdk/x/vote/ante/validate.go` invokes it. Wallet and chain therefore cannot drift apart — but their agreement is a tautology, not evidence of correctness. No independent second implementation exists that could catch a mis-transcribed relation, and no merged specification exists against which either side could be audited. The reader should weigh every constraint we state in later sections with this in mind.

Classification. Design-stage material in this volume obeys a three-bin discipline. Every claim is exactly one of: *specified* — a merged normative artifact exists and we cite it; *designed but unspecified* — the design is concrete and open-source but no normative artifact pins it, so we cite the implementation and state what a specification would need to fix; or an *open problem* — the design itself leaves the question unresolved. The bin is always visible in the prose.

Specified. Only the inherited primitives: NoteCommit, nullifier derivation, Commit^{ivk}, the Sinsemilla Merkle instance, RedPallas, the ZIP-244 sighash, and ZIP-32 key derivation — each specified in the Zcash protocol specification and the ZIPs, constructed in the *Ironwood Guide* (§“Representation of a note: notes and note commitments”, §“Prevention of double-spending: nullifiers”, §“Keys: capabilities for spending and for viewing”, §“Proof of ownership of *some* valid note: the commitment tree”, §“Authorisation of the spend: RedPallas”), and reused unchanged through the unstable-voting-circuits feature (zcash/orchard @ c0b6b917). No governance-specific object has a specification of that rigor.

Designed but unspecified. Everything else this volume presents: the five-message ballot structure, the delegation relation with its indexed-Merkle-tree non-membership argument, the coordinator-posted snapshot roots, the Vote Authority Note and its synthetic-action authorisation, all four double-vote layers, the split-share ElGamal encoding and vote commitment, the relayer protocol and the trust it grants, the per-round DKG and chain-verified tally, and the PIR subsystem. For each we cite the pinned commit and file, and we note what PR #1200 would need to pin down for the object to change bins.

Open problems. One, and only one, was found: the ballot-quantisation relation enforced by the delegation circuit is strictly weaker than the exact floor division the public documentation states — the relation admits a one-ballot *under*-claim for roughly a third of note-value totals, while an over-claim remains impossible (voting-circuits/src/delegation/circuit.rs, src/delegation/README.md §8). We state and analyse the gap where delegation weight is constructed. It is the only spec-versus-code disagreement this volume records.

2 Ballots and eligibility

A voting round is a conversation in five messages on the vote chain, and admission to it is a zero-knowledge proof against a pinned Zcash snapshot. This section constructs both sides of that admission: the wire surface of a round (§2.1); the eligibility relation the delegation circuit proves — ownership of Orchard notes that existed at the snapshot and were unspent at it (§2.2, §2.3); the provenance of the two snapshot roots those proofs are checked against (§2.4); and the quantization that turns note value into voting weight (§2.5). Except where marked otherwise, every claim below sits in the *designed but unspecified* bin of §1.3: we read it from the commit-pinned sources of Table 1, and each subsection closes by recording its bin. The quantization carries this volume’s single open problem; we state and prove it where it arises.

2.1 The five-message round

The ballot surface of the vote chain is five protobuf messages (vote-sdk/proto/svote/v1/tx.proto). In lifecycle order: a coordinator opens the round, holders delegate weight, governance hotkeys cast votes, helper servers reveal shares, and the round closes with a tally. The vote chain is a Cosmos-SDK application, and an ordinary transaction there is a fee-paying envelope of messages signed by an on-chain account. The three voter-facing messages (2–4) are not ordinary Cosmos transactions: they bypass that envelope entirely, and the chain authenticates each by its zero-knowledge proof — messages 2 and 3 additionally by a RedPallas signature; message 4 carries none — rather than by an account signature (tx.proto:10–13; vote-sdk/x/vote/ante/validate.go).

Message 1: MsgCreateVotingSession. The round-opening message carries the snapshot pin and the agenda: snapshot_height, snapshot_blockhash, proposals_hash, vote_end_time, the two ledger anchors nullifier_imt_root and nc_root, the proposal list, and display metadata (tx.proto:36–48). It is not a public RPC: it is absent from the Msg service and executes only as

the payload of a threshold-approved coordinator action (`vote-sdk/x/vote/keeper/msg_server_coordinator_actions.go:294-298`). The handler derives the round identifier on-chain as one Poseidon hash over the creation height, the snapshot block hash and the proposals hash (each split into two field elements), the end time, and both anchors, so `vote_round_id` commits to the snapshot (`tx.proto:32-35; msg_server.go:30-48`).

Message 2: `MsgDelegateVote` (ZKP 1). The delegation message carries `rk` (the randomized spend-validating key), `spend_auth_sig` (RedPallas), `signed_note_nullifier` (the nullifier of the synthetic keystone note), `cmx_new` (the output note commitment), `van_cmx` (the Vote Authority Note commitment), `gov_nullifiers` (up to five governance nullifiers), `proof`, `vote_round_id`, and `sighash` — a *client-provided* 32-byte value over which the chain verifies the RedPallas signature without recomputing it (`tx.proto:55-65`). This section proves the eligibility half of the message; the Vote Authority Note is constructed in §3.1, and the synthetic keystone note with its signature binding — the ρ -chain through which the RedPallas signature commits to the whole delegation — in §3.2.

Message 3: `MsgCastVote` (ZKP 2). A cast vote carries `van_nullifier`, `vote_authority_note_new` (the successor Vote Authority Note), `vote_commitment`, `proposal_id`, `proof`, `vote_round_id`, the anchor height `vote_comm_tree_anchor_height`, `vote_auth_sig`, and `r_vpk`, a compressed Pallas point that the chain decompresses for the verifier; unlike message 2, the `sighash` here is recomputed on-chain (`tx.proto:72-84`, including the removed-field comments).

Message 4: `MsgRevealShare` (ZKP 3). A share reveal carries `share_nullifier`, `enc_share` — 64 bytes, one lifted-ElGamal ciphertext $C_1 \parallel C_2$ (ElGamal with the plaintext in the exponent, Definition 3.2) as compressed Pallas points — `proposal_id`, `vote_decision`, `proof`, `vote_round_id`, and `vote_comm_tree_anchor_height` (`tx.proto:89-97`). Helper servers, not wallets, submit it: the share nullifier is constructed in §3.3, and the helper protocol and the trust it grants are treated in §4.1.

Message 5: `MsgSubmitTally`. The closing message carries `vote_round_id`, `creator` — which the chain checks against the *block proposer*: the proposer’s node injects the message during tallying, and the proto comment naming the session creator is unenforced (`x/vote/keeper/msg_server_tally_decrypt.go:77-79; app/prepare_proposal.go:194-212`) — and one `TallyEntry` {`proposal_id`, `vote_decision`, `total_value`, `decryption_proof`} per (proposal, decision) pair (`tx.proto:104-116`). The `total_value` field is denominated in ballots despite its proto comment saying zatoshi; we return to this in §3.6.

Classification. Designed but unspecified: the message set, field semantics, and transport exist only in `vote-sdk/proto/svote/v1/tx.proto` at commit `cb915f51` (Table 1). A specification would need to pin the byte-level encoding and length of every field, the validation order, the envelope-bypass transport, and the round-identifier derivation — the last currently fixed only by a proto comment and the FFI implementation it describes.

2.2 Eligibility: the note-bundle relation (ZKP 1)

To delegate weight, a holder must prove three things about a set of Orchard notes without revealing any of them: the notes are theirs; each existed at the snapshot; each was unspent at the snapshot. The delegation circuit proves the conjunction — one Halo 2/IPA circuit (the proof system and inner-product commitment of the *Halo 2 Guide*, §“The Halo 2 proof system”, §“The inner-product polynomial commitment”) over Pallas at $K = 14$ (16,384 rows) with 14 public inputs (`voting-circuits/src/delegation/circuit.rs:111` and the module README, both at commit `4c39abd5`).

Bundle shape. The circuit carries exactly five note slots (`MAX_REAL_NOTES`); a wallet with fewer notes pads the remaining slots with zero-value notes, and one with more produces multiple delegation proofs. The fixed width is deliberate: every delegation publishes the same shape — five governance nullifiers at public-input offsets 8–12 — so the published proof does not reveal how many real notes it bundles (`src/delegation/README.md`, `Inputs`). The two ledger anchors enter as public inputs `nc_root` (offset 6) and `nf_imt_root` (offset 7); the nullifier domain `dom` (offset 13) is exposed for API compatibility but constrained in-circuit to `Poseidon(tag, vote_round_id)`, where `tag` is the ASCII string `governance authorization` zero-padded to a Pallas base-field element (`src/domain_tags.rs`; `src/delegation/imt.rs:26–35`). Poseidon throughout the protocol is the `P128Pow5T3` instance, width 3, rate 2 (`src/protocol_hash.rs`).

Per-slot conditions. Each slot proves six conditions, numbered 9–14 in the source⁵ (`src/delegation/circuit.rs:1497–1805`):

- *Note commitment integrity* (condition 9). The prover witnesses the note plaintext $(g_d, pk_d, v, \rho, \psi)$, the trapdoor `rcm`, and the claimed commitment `cm`; the circuit recomputes `NoteCommitrcm` over the witnessed fields, constrains it equal to `cm`, and extracts `cmx = Extract(cm)` as an internal wire. The commitment scheme is the deployed Orchard `NoteCommit` — specified bin, reused unchanged through the `unstable-voting-circuits` feature (§1.3).
- *Address ownership* (condition 11). The circuit checks $pk_d = [ivk^*] g_d$, where ivk^* is selected by a witnessed boolean `is_internal` through the custom gate `q_scope_select`, $ivk^* = ivk + is_internal \cdot (ivk_{int} - ivk)$; both candidates are derived in-circuit by `Commitivk` from the witnessed r_{ivk} and its internal-scope counterpart (condition 5, global). Change notes therefore carry weight on equal terms with externally received ones. A wrong scope flag selects the wrong key and the ownership check rejects.
- *Snapshot membership* (condition 10). The circuit computes the Sinsemilla Merkle root from the leaf `cmx` over the witnessed 32-level authentication path, and the per-note gate `q_per_note` enforces

$$v \cdot (\text{root} - \text{anchor}) = 0,$$

with anchor the public `nc_root` (`circuit.rs:1777–1798`). A real note ($v \neq 0$) must resolve to the anchor; a zero-value padding note — which has no leaf in the Zcash tree — satisfies the gate vacuously.

- *Real nullifier* (condition 12). The circuit derives the note’s true Orchard nullifier $nf = \text{DeriveNullifier}_{nk}(\rho, \psi, cm)$ — the deployed Orchard rule, constructed in the *Ironwood Guide* (§“Prevention of double-spending: nullifiers”) — in-circuit and *never publishes it*: it exists only as a private wire feeding conditions 13 and 14 (`circuit.rs:1682–1701`).
- *Snapshot non-spentness* (condition 13). The circuit proves non-membership of `nf` in the snapshot’s nullifier set under the public `nf_imt_root` — the construction of §2.3. The root equality is unconditional: padding notes, too, must carry a valid non-membership proof (`README.md` §13).
- *Governance nullifier* (condition 14). The circuit computes and publishes $nf^{gov} = \text{Poseidon}(nk, dom, nf)$ at the slot’s public-input offset. Keyed by the private `nk`, the published value stays unlinkable to `nf` even after the note is later spent on the main chain

⁵The headline counts disagree — `circuit.rs` line 3 says “all 14 conditions” but line 108 “all 15 conditions”; `README.md` lines 3 and 52 say “15” and “conditions 9–15” — yet the condition-by-condition definitions in both files stop at 14: the `README`’s numbered sections run 1 through 14, and the sole “condition 15” in the sources is a synthesis-stage comment (`circuit.rs:1089–1098`) labelling the padded slots’ zero value — a builder convention with no corresponding constraint. We state the total as fourteen — eight keystone-global conditions plus six per-slot conditions instantiated five times — and record the “15” headline counts and the `cond-15` comment as stale.

(`circuit.rs:1703–1729`). Its role in double-vote prevention is treated in §3.3. Chain-side, the delegation handler applies the same atomic check-and-set to all five published slots, padding included (`vote-sdk/x/vote/keeper/msg_server.go:148–154`); a padding slot’s nullifier enters the same round-scoped set, unique because the wallet samples fresh ρ and $rseed$ per synthetic padding note (`src/delegation/builder.rs, build_padding_slot`), and a re-used padding witness is rejected as a duplicate delegation — a completeness cost, never a soundness one.

The conjunction across a slot reads: the prover’s key material owns a note whose commitment lies in the Zcash note-commitment tree at the snapshot (membership under `nc_root`) and whose nullifier had not been revealed by the snapshot (non-membership under `nf_imt_root`) — existed, and unspent, at the pinned height.

Classification. The relation as a whole is designed but unspecified: it exists only as the circuit at `voting-circuits @ 4c39abd5`, which is simultaneously the prover and the verifier (§1.3). Its ingredients `NoteCommit`, nullifier derivation, `Commitivk`, and the Sinsemilla Merkle instance are the specified bin, inherited through `zcash/orchard @ c0b6b917`. A specification would need to pin the public-input layout, the custom-gate semantics, the fixed five-slot shape, and the padding-note rules.

2.3 Snapshot non-spentness: the Poseidon indexed Merkle tree

Membership alone would let a spent note vote: the note-commitment tree is append-only and a spent note’s commitment remains in it forever. Snapshot eligibility therefore needs a second, negative statement — the note’s nullifier is *not* in the set of nullifiers revealed up to the snapshot height. Proving non-membership in a Merkle set requires structure beyond a plain accumulator; the protocol uses an indexed Merkle tree over the sorted nullifier set.

Construction. The tree is Poseidon-based with depth 29 (`IMT_DEPTH, voting-circuits/src/delegation/imt.rs:19`). Each leaf is a *punctured range* — a triple of sorted boundaries, hashed and covering the punctured open interval

$$\text{leaf} = \text{Poseidon}(nf_{lo}, nf_{mid}, nf_{hi}), \quad (nf_{lo}, nf_{hi}) \setminus \{nf_{mid}\},$$

so the set members appear only as interval boundaries or interior punctures, and everything strictly between them is certified absent. In-circuit, non-membership of the private nf costs 31 Poseidon calls (2 for the leaf, 29 for the path) and proves (`imt.rs:51–70; src/delegation/README.md §13`):

1. the leaf authenticates: the 29-level path from the leaf hash resolves to the public `nf_imt_root`;
2. the nullifier lies strictly inside the interval: the offsets $x_{lo} = nf - nf_{lo} - 1$ and $x_{hi} = nf_{hi} - nf - 1$ are each range-checked to $[0, 2^{250})$; and
3. the nullifier misses the puncture: $(nf - nf_{mid}) \cdot \delta = 1$ for a witnessed inverse δ .

What it excludes — and what it does not. The statement excludes exactly the nullifiers revealed on the Zcash chain up to the snapshot height: the note was unspent when the snapshot was taken. It excludes nothing after that height — a note spent the block after the snapshot still delegates its weight, because snapshot voting fixes the electorate at a single height by design. Two soundness assumptions live *outside* the circuit (`imt.rs:56–70; README.md §13`). First, the tree contract: the circuit checks only the one authenticated leaf, so the committed leaves must form the canonical punctured-range tree for the set — sorted, deduplicated, non-overlapping except for shared boundaries. A malformed tree in which some set member lies strictly inside another leaf’s interval would certify a spent note as unspent, and no per-leaf gate can detect the global overlap. Second, the span bound: the 250-bit offset checks require $nf_{hi} - nf_{lo} \leq 2^{250}$ in canonical field order, which the builder guarantees

by pre-populating 33 sentinel nullifiers spaced at most 2^{249} apart, plus $p - 1$ to close the tail (the `SpacedLeafImtProvider` strategy). The tree is built by the PIR operator and its root posted by the coordinator — which is where the next subsection picks up.

Classification. Designed but unspecified (`voting-circuits @ 4c39abd5`). A specification would need to pin the leaf model, the sentinel schedule, and above all the canonical-tree contract — including which party is accountable for the tree’s well-formedness, since the circuit cannot check it.

2.4 The snapshot roots: posted, never verified

Both anchors originate off-chain, in coordinator tooling. The note-commitment root `nc_root` is recomputed by Rust FFI as the Sinsemilla hash of an Orchard frontier (the incremental-tree append state constructed in the *Crypto Guide*, §“Append-only and incrementally updatable trees”) fetched from a public `lightwalletd` server; the nullifier root `nullifier_imt_root` is fetched from the running PIR service (`vote-sdk/api/snapshot.go:44-66`). The coordinator posts both in message 1; the handler stores them verbatim in the round record and folds them into `vote_round_id` (`x/vote/keeper/msg_server.go:36-138`); and at validation the ante handler looks the stored roots up from the round and passes them to the FFI verifier as the public inputs of ZKP1 (`x/vote/ante/validate.go:147-177`).

Nothing on the vote chain verifies that the posted roots match real Zcash state: `svoted` runs no Zcash light client, and the circuit documentation is explicit that it “proves statements relative to the public inputs supplied by the verifier; it does not authenticate where those inputs came from” (`voting-circuits/src/delegation/README.md`, Public Input Provenance). The trust structure that results is asymmetric. Honest wallets are safe from a *wrong* snapshot in the sense that their proofs simply fail: their witnesses — Merkle paths and IMT leaves — come from the real chain and cannot resolve to fabricated roots. And the roots are public, so anyone with a Zcash node can audit them after the fact. But nothing prevents a coordinator quorum from pinning fabricated roots for which it alone holds matching witnesses, minting arbitrary voting weight that the chain would accept; detection is post hoc and protocol-external. The trust root for eligibility is therefore the coordinator threshold together with the PIR operator that builds the nullifier tree of §2.3.

Classification. Designed but unspecified, with an explicit trust assumption that must be flagged wherever eligibility is discussed. A specification would need either chain-side verification (a light client or a succinct proof that the roots match a Zcash block) or, failing that, a normative audit procedure with a defined challenge window.

2.5 Weight: ballot quantization

Delegated weight is denominated in ballots. The circuit sums the five slot values into $v_{\text{total}} = \sum_{i=1}^5 v_i$ (internal wires produced by condition 9, consumed by conditions 7 and 8; `src/delegation/README.md`, Inputs), and one ballot represents $D = 12,500,000$ zatoshi (0.125 ZEC): the constant `BALLOT_DIVISOR` (`voting-circuits/src/params.rs:14`). The honest ballot count is $q = \lfloor v_{\text{total}}/D \rfloor$.

Condition 8 (`src/delegation/circuit.rs:1290-1427`) witnesses the count q (`num_ballots`) and a remainder r as *free advice*, and constrains

$$q \cdot D + r = v_{\text{total}}, \quad 0 \leq r < 2^{24}, \quad 1 \leq q \leq 2^{30}. \quad (1)$$

The range checks run through a 10-bit lookup table: the circuit multiplies r by 2^6 and checks the product to 30 bits (so $r < 2^{24}$), and checks $q - 1$ to 30 bits directly — which enforces both bounds at once, since $q = 0$ wraps $q - 1$ to $p - 1 \approx 2^{254}$ and fails. The lower bound $q \geq 1$ doubles as a minimum stake: a bundle worth less than 0.125 ZEC admits no valid witness. The upper bound is safe slack: 2^{30} ballots is about 134 million ZEC, far above the 21 million supply. The remainder bound 2^{24} is the

tightest power-of-two-aligned bound that keeps completeness — every honest $r < D$ fits — but it is looser than D , and that slack admits a second witness.

Proposition 2.1 (One-ballot under-claim window). *Fix v_{total} and let $\hat{q} = \lfloor v_{\text{total}}/D \rfloor$ and $\hat{r} = v_{\text{total}} \bmod D$ form the honest witness, assumed valid ($\hat{q} \geq 1$). The pairs satisfying the constraints of Equation (1) are exactly $(\hat{q}, \hat{r})$ together with $(\hat{q} - 1, \hat{r} + D)$, the latter admissible if and only if*

$$\hat{r} < 2^{24} - D = 4,277,216 \quad \text{and} \quad \hat{q} \geq 2.$$

In particular, a prover can claim exactly one ballot fewer than floor division yields — for approximately 34.2% of totals under a uniform residue model, since $(2^{24} - D)/D \approx 0.3422$ — and can never claim more.

Proof. Suppose (q_1, r_1) and (q_2, r_2) both satisfy Equation (1) for the same v_{total} . Subtracting the two instances of the linear constraint gives $(q_1 - q_2)D \equiv r_2 - r_1 \pmod{p}$. The range checks bound the canonical representatives: $|q_1 - q_2| \cdot D \leq (2^{30} - 1)D < 2^{54}$ and $|r_2 - r_1| < 2^{24}$, both far below $p > 2^{254}$, so the congruence holds over the integers. Then $|q_1 - q_2| = |r_2 - r_1|/D < 2^{24}/D < 2$, forcing $|q_1 - q_2| \in \{0, 1\}$. The case 0 gives $r_1 = r_2$. The case $q_1 = q_2 + 1$ gives $r_2 = r_1 + D$, which is admissible precisely when $r_2 < 2^{24}$ (the remainder range check) and $q_2 \geq 1$ (the count range check) — for the honest pair, the stated window. The over-claim direction from the honest witness, $(\hat{q} + 1, \hat{r} - D)$, requires $\hat{r} - D < 0$; its canonical field representative is at least $p - D$, which fails the shifted 30-bit range check. No further pairs exist. $\square$

Impact and status. The deviation is self-harming: the only dishonest witness *under-claims*, discarding one ballot of the prover’s own weight, and aggregate totals consequently err low, never high. No downstream check repairs or re-derives the count: the Vote Authority Note commitment (condition 7) hashes whatever q the prover chose, and the vote-proof circuit later opens the same value. The gap is documented by the implementers themselves — `src/delegation/README.md` §8, “Soundness scope”, carries the analysis above together with two single-constraint tightenings (a 24-bit check on $(D - 1) - r$, or on $r + (2^{24} - D)$, either of which pins $r \in [0, D)$ at a cost of roughly 3–4 rows) — and it is unfixed at commit 4c39abd5. The public documentation states exact floor division (gitbook ZKP 1 page, fetched 2026-07-08), which the circuit does not enforce: this is the single spec-versus-code disagreement recorded in this volume (§1.3), conservative in direction — the code is the protocol, and the code is weaker.

Classification. The quantization relation as implemented — Equation (1) — is designed but unspecified. The gap between it and the stated floor-division rule is this volume’s one *open problem*: the design intends exact division, the circuit proves less, and the tightening exists on paper only. A specification must either adopt the weaker relation and its under-claim window explicitly, or mandate one of the documented tightening constraints.

3 Vote encryption, nullifiers, and tally

Delegation leaves the governance hotkey holding a Vote Authority Note (VAN): a blinded commitment to the hotkey address, a ballot count, and a per-proposal authority mask. Three problems remain. The hotkey must cast a vote whose weight never appears in public; the protocol must stop every form of double-voting — the same coins delegated twice, the same VAN spent twice, the same proposal voted twice along a chain of successor VANs, the same encrypted share counted twice; and the aggregate totals must become public in a form every chain node verifies, without any single party ever holding the decryption key. This section develops the answers in order: the VAN itself, the synthetic keystone note whose signature authorises its minting, a four-layer double-vote discipline, a sixteen-share lifted-ElGamal encoding, a per-round threshold Election Authority, and a chain-verified tally.

Classification. Every governance-specific object in this section is *designed but unspecified*: the normative artifact is the implementation, cited commit-pinned — `valargroup/voting-circuits` at 4c39abd5 (2026-06-03) and `valargroup/vote-sdk` at cb915f51 (2026-06-05). File paths beginning `src/` refer to the former; paths beginning `x/vote/`, `crypto/`, or `proto/` to the latter. The would-be specification (zcash/zips PR #1200, opened 2026-03-05) remains unmerged as of 2026-07-08, and the single `voting-circuits` crate both proves and verifies, so no independent implementation cross-checks these formulas. The *specified* bin holds only the inherited Orchard primitives — `NoteCommit`, nullifier derivation, `Commitivk`, `RedPallas`, the Sinsemilla Merkle instance — reused through the orchard crate’s `unstable-voting-circuits` feature (zcash/orchard commit c0b6b917). One *open problem* reaches into this section from the delegation circuit; Remark 3.3 flags it where it lands.

Throughout this section, Poseidon denotes the P128Pow5T3 instance over $\mathbb{F}_{p_{\text{Pallas}}}$ (width 3, rate 2), applied with the fixed-length padding of the stated arity. Domain separation uses two encodings, both pinned by unit tests (`src/domain_tags.rs`): small numeric tags (0 for VAN commitments, 1 for vote commitments), and short ASCII tags — “governance authorization”, “vote authority spend”, “share spend” — zero-padded to 32 bytes and read as little-endian integers, which are canonical in $\mathbb{F}_{p_{\text{Pallas}}}$ because the top byte stays zero. We abbreviate the ASCII tags T_{gov} , T_{spend} , T_{share} , and write `rid` for the voting round identifier and `pid` for the proposal identifier.

3.1 The Vote Authority Note

The VAN is the vote chain’s carrier of authority: one field element that commits to who may vote (the hotkey address), with how much weight (the ballot count), in which round, and on which proposals (the authority mask).

Definition 3.1 (VAN commitment). Let (g_d, pk_d) be the hotkey’s diversified address in the voting key hierarchy, n_b the ballot count, $\text{auth} \in [0, 2^{16})$ the proposal-authority mask, and r_{van} a uniformly random blind. The VAN commitment is the two-layer hash

$$\text{van} := \text{Poseidon}(\text{core}, r_{\text{van}}), \quad \text{core} := \text{Poseidon}(0, x(g_d), x(pk_d), n_b, \text{rid}, \text{auth}),$$

with $x(\cdot)$ the x -coordinate (`src/gadgets/van_integrity.rs`, lines 61–78; one gadget is shared by the delegation and vote-proof circuits).

The two layers divide labour. The inner layer is structural, and its preimage is low-entropy: the round id is public, the mask at mint is a fixed constant, and ballot counts are small integers — a bare inner hash would admit dictionary attacks on the weight and the address. The outer layer closes them with the uniform blind (module documentation, `src/gadgets/van_integrity.rs`, lines 23–27).

The address enters by x -coordinates alone, which identifies (g_d, pk_d) only up to coordinated negation. The construction is safe because delegation additionally binds the full points through `NoteCommit` into the public cmx_{new} ; the module documentation warns that any future caller dropping that side constraint must widen the preimage to full points (`src/gadgets/van_integrity.rs`, lines 14–22).

The mask `auth` is a 16-bit bitmask minted at $2^{16} - 1 = 65535$ — all bits set, a constant baked into the verification key (`src/delegation/circuit.rs`, lines 162 and 1464–1467). Bit 0 is a permanent sentinel that no vote may consume, so usable bits are 1 through 15 and a round supports at most 15 proposals (`src/vote_proof/circuit.rs`, lines 123–143).

The vote chain keeps one per-round Poseidon Merkle tree of depth 24 (`src/params.rs`, line 25; per-round state in `x/vote/keeper/keeper_voting.go`, lines 72–80): delegation appends the freshly minted VAN commitment; each cast appends the successor VAN and then the vote commitment. The reduced depth is a sizing choice — governance produces one leaf per delegation plus two per vote, well within $2^{24} \approx 1.7 \times 10^7$ leaves (`src/params.rs`, lines 16–25).

3.2 Keystone authorisation: the synthetic signed note

The delegation message is authorised by a RedPallas spend-auth signature under the randomized key rk (§2.1, message 2), and the intended signer is a Keystone-class hardware wallet, which signs only Orchard Actions. The voting client therefore wraps the delegation in an Orchard-Action shape around a *synthetic* keystone note: a 1-zatoshi note that exists on no chain — the circuit witnesses no Merkle path for it and checks no anchor; the value is 1 rather than 0 only because the wallet UI does not render zero-value spends (`src/delegation/README.md`, “Integration: the Keystone (signed) note is synthetic”).

Conditions 1–6 of the delegation circuit govern this branch (`src/delegation/circuit.rs`, lines 16–29); the load-bearing pair is conditions 3 and 2. Condition 3 forces the keystone note’s ρ to a Poseidon hash of everything the delegation publishes,

$$\rho_{\text{signed}} = \text{Poseidon}(\text{cmx}_1, \dots, \text{cmx}_5, \text{van}, \text{rid}),$$

the arity-7 fixed-length instance (`rho_binding_hash`, `circuit.rs:176–190`). Condition 2 then derives the keystone nullifier by the deployed Orchard rule — constructed in the *Ironwood Guide* (§“Prevention of double-spending: nullifiers”) —

$$\text{nf}_{\text{signed}} = \text{DeriveNullifier}_{\text{nk}}(\rho_{\text{signed}}, \psi_{\text{signed}}, \text{cm}_{\text{signed}}),$$

and exposes it as a public input. The chain verifies the RedPallas signature under rk over the *client-provided* sighash and checks that the proof’s $\text{nf}_{\text{signed}}$ equals the wrapping Action’s nullifier (`x/vote/ante/validate.go`), so the hardware wallet’s signature commits — through the chain $rk \rightarrow \text{nf}_{\text{signed}} \rightarrow \rho_{\text{signed}} \rightarrow \text{van}$ — to the five bundle commitments, the VAN, and the round identifier. Removing any link breaks the binding; this chain is why the chain-side acceptance of a client-provided sighash is safe. The remaining conditions are supporting: condition 4 proves $rk = [\alpha] \text{SpendAuthG} + ak$ for a witnessed randomizer α (the *Ironwood Guide*, §“Authorisation of the spend: RedPallas”); condition 5 ties ak , nk , and r_{ivk} to the same Commit^{ivk} chain the real-note slots use (§2.2, condition 11); conditions 1 and 6 recompute the keystone and output note commitments from witnessed openings — knowledge checks, not membership checks (`src/delegation/README.md`, the three-bucket audit note).

3.3 Double-vote prevention: four layers

A ballot can be replayed at four distinct joints: the Zcash note (delegate it twice), the VAN (spend it twice), the proposal bit (vote the same proposal again through a successor VAN), and the encrypted share (feed it to the accumulator twice). One defence guards each joint. The first, second, and fourth are nullifiers; chain-side, each kind lands in its own type- and round-scoped set under an atomic check-and-set (`x/vote/keeper/keeper_voting.go`, lines 19–48; type constants `x/vote/types/keys.go`, lines 96–101). The third is arithmetic, enforced in-circuit.

Layer 1: the governance nullifier (delegation). Each delegated note exposes

$$\text{nf}^{\text{gov}} := \text{Poseidon}(\text{nk}, \text{dom}, \text{nf}), \quad \text{dom} := \text{Poseidon}(T_{\text{gov}}, \text{rid}),$$

where nk and nf are the Zcash note’s nullifier key and real Orchard nullifier, both computed in-circuit — the real nullifier is never published (`src/delegation/circuit.rs`, lines 1703–1729). The domain dom is a public input, but the circuit re-derives it from the public round id and constrains equality (`src/delegation/circuit.rs`, lines 1041–1055), so a prover cannot substitute a foreign domain to escape the round scoping. The chain rejects duplicates in the governance-typed, round-scoped set. The attack closed: delegating the same unspent-at-snapshot note into two bundles in one round, which would double its weight. Keying by the secret nk serves privacy: when the note is later spent on mainnet and its real nullifier becomes public, no observer can link the pair — the governance trail does not deanonymise the payment trail.

Layer 2: the VAN nullifier (casting). Spending a VAN publishes

$$\text{nf}^{\text{van}} := \text{Poseidon}(\text{nk}_v, T_{\text{spend}}, \text{rid}, \text{van}_{\text{old}}),$$

where nk_v is the voting hierarchy’s nullifier-deriving key, proved to belong to the witnessed address through the $\text{Commit}^{\text{ivk}}$ chain (`src/vote_proof/circuit.rs`, lines 203–222). A cast proves membership of van_{old} in the round tree without identifying the leaf, and this nullifier is the only spend mark. The attack closed: re-spending the old VAN, whose mask still carries the bit the vote just consumed — without the nullifier, one delegation would vote the same proposal arbitrarily often from the same leaf.

Layer 3: the authority-mask decrement (casting). Layer 2 kills the parent, not the lineage: every cast mints a successor VAN, and a successor still carrying the consumed bit would re-enable the same vote. Condition 6 of the vote-proof circuit therefore forces

$$\text{auth}_{\text{new}} = \text{auth}_{\text{old}} - 2^{\text{pid}}.$$

Field arithmetic cannot express a variable exponent as a polynomial gate, so the prover witnesses 2^{pid} and a lookup over the table $\{(j, 2^j)\}_{j=0}^{15}$ proves it; a field-inverse gate excludes $\text{pid} = 0$ (the sentinel); a 16-bit decomposition of auth_{old} with a one-hot selector anchored at pid proves the consumed bit was actually set ($\sum_j \text{sel}_j b_j = 1$), and the recomposition returns auth_{new} with exactly that bit cleared, implicitly range-checked to 16 bits (`src/vote_proof/gadgets/authority_decrement.rs`, lines 34–93). The successor VAN commits auth_{new} and re-enters the tree. The invariant bought by layers 2 and 3 together: one delegation yields at most one vote per proposal, and at most 15 proposals per round.

Layer 4: the share nullifier (reveal). Revealing the i -th encrypted share of a vote publishes

$$\text{nf}_i^{\text{share}} := \text{Poseidon}(T_{\text{share}}, \text{vc}, i, \text{blind}_i),$$

where vc is the vote commitment (§3.4) and blind_i the share-commitment blind (`src/share_reveal/circuit.rs`, lines 161–187). The attack closed: the tally accumulator is a raw homomorphic sum (§3.6), so re-submitting one accepted ciphertext would double-count its weight; the share-typed set admits each (vc, i) once. The blind is load-bearing for privacy rather than soundness: the vote commitment is public in the cast message and i ranges over sixteen values, so an unblinded nullifier could be evaluated by any observer against every commitment in the tree, linking each reveal to its vote by dictionary test — collapsing exactly the anonymity that the reveal proof’s non-identifying membership provides. Blinds never appear on chain.

3.4 Vote encoding: sixteen lifted-ElGamal shares

Casting must convey weight to the tally without publishing it per vote. The instrument is additively homomorphic encryption to a key that no single party holds (§3.5).

Definition 3.2 (Vote share encryption). Fix the generator

$$G := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, G),$$

Orchard’s spend-authorisation base. A vote on proposal pid splits its ballot count n_b into sixteen prover-chosen shares $v_0, \dots, v_{15}$ subject to

$$\sum_{i=0}^{15} v_i = n_b, \quad v_i \in [0, 2^{30}),$$

and encrypts each under the round’s Election Authority key $ea_{pk} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$:

$$C_{1,i} := [r_i] G, \quad C_{2,i} := [v_i] G + [r_i] ea_{pk},$$

with fresh nonzero randomness r_i (`src/gadgets/elgamal.rs`; sum and range are conditions 8 and 9, `src/vote_proof/circuit.rs`, lines 416–457 and 1026–1036).

The scheme is *lifted*: the message enters the exponent, so componentwise addition of ciphertexts encrypts the sum of plaintexts and the accumulator needs no key. The price is paid at decryption, which yields $[v] G$ and recovers v only as a bounded discrete logarithm — the 2^{28} bound of §3.6. Reusing `SpendAuthG` avoids introducing a second fixed base; the in-circuit 22-window short-scalar tables share the same generator (`src/gadgets/elgamal.rs`, lines 57–66). The range check closes wraparound: shares are field elements, and without it a decomposition could encode negative shares as large residues while still satisfying the sum constraint. Sixteen shares serve weight privacy under staggered reveal: the prover-chosen random decomposition leaves no weight fingerprint on any single revealed ciphertext (`src/vote_proof/circuit.rs`, lines 418–423). Client-side, the randomness r_i and the blinds derive from a BLAKE2b-512 PRF (pseudorandom function; the keyed-BLAKE2 construction of the *Crypto Guide*, §“PRFs from hash functions: length extension and keyed BLAKE2”) with personalisation `ZcashVote_Expand` under distinct domain bytes (`src/domain_tags.rs`, lines 28–41).

The cast binds everything into one public field element:

$$vc := \text{Poseidon}(1, \text{rid}, h_{\text{sh}}, \text{pid}, d), \quad h_{\text{sh}} := \text{Poseidon}(sc_0, \dots, sc_{15}),$$

$$sc_i := \text{Poseidon}(\text{blind}_i, x(C_{1,i}), x(C_{2,i}), y(C_{1,i}), y(C_{2,i})),$$

where d is the vote decision (`src/gadgets/vote_commitment.rs`, lines 42–55; `src/shares_hash.rs`, lines 39–61). The y -coordinates are included deliberately: an x -coordinate identifies a point only up to sign, and hashing both coordinates blocks ciphertext sign-malleability. The leading tag 1 separates vote commitments from VANs (tag 0) in the shared tree and is baked into the verification key, so an honest verifier cannot be driven to confuse the two leaf kinds (`src/gadgets/vote_commitment.rs`, lines 11–13). A cast message reveals vc , the VAN nullifier, the successor VAN, and pid — neither the decision nor the weight. Note that the cast publicly links vc to nf^{van} , so per-vote weight privacy rests on the encryption, not on unlinkability.

Remark 3.3 (Quantization gap, propagated). The sum condition ties $\sum_i v_i$ to the ballot count n_b minted at delegation, so its soundness inherits the delegation circuit’s quantization relation: the constraint $n_b \cdot 12,500,000 + \text{rem} = v_{\text{total}}$ with $\text{rem} < 2^{24}$ is strictly weaker than floor division and admits a one-ballot *under-claim* for roughly 34% of note values; an over-claim is impossible (Proposition 2.1 in §2.5; `src/params.rs`, lines 9–13; `src/delegation/README.md`, §8). This is the protocol’s one open problem — the public design prose states exact floor division, the code does not enforce it.

3.5 The Election Authority: a per-round Joint–Feldman DKG

The tally requires a key that opens aggregates; any party holding that key alone could equally open each accumulated share the moment it lands on chain. The protocol therefore splits the round key across the bonded validator set by a Joint–Feldman distributed key generation, and re-runs the ceremony every round, so compromise of one round’s key reaches no other round and the validator set may change between rounds.

Each ceremony validator — registered through a whitelisted Pallas-key path — samples a secret polynomial of degree $t - 1$ over the Pallas scalar field, posts its t Feldman coefficient commitments (Pallas points) on chain, and delivers to every other validator its evaluation share, encrypted under ECIES: an ephemeral Pallas Diffie–Hellman, the symmetric key $\text{SHA-256}(E \parallel S_x)$ over the compressed ephemeral point and the shared-secret x -coordinate, and ChaCha20-Poly1305 with a zero nonce — safe because each envelope uses a fresh ephemeral

key (`crypto/ecies/ecies.go`, lines 21–33). The chain validates point well-formedness and the commitment count (`x/vote/keeper/msg_server_ceremony.go`, lines 137–149). When the n -th contribution arrives, the chain sums the commitment vectors componentwise; the round key ea_{pk} is the combined constant term, and each validator’s verification key VK_i is the combined commitment polynomial evaluated at i in the exponent (`x/vote/keeper/msg_server_ceremony.go`, lines 224–269). No dealer ever holds the round secret: it exists only as the sum of shares.

The parameters are fixed in code: reconstruction threshold $t = \max(2, \lfloor (2n + 2)/3 \rfloor)$ for $n \geq 2$ validators (the degenerate $n = 1, t = 1$ case exists for local testing), ceremony finalization quorum $\lceil 4n/5 \rceil$ acknowledgements — stricter than t for most n — with non-ackers stripped from the ceremony and non-participants suspended from the active validator set (`x/vote/keeper/keeper_ceremony.go`, lines 46–79; `keeper_ceremony_jailing.go`).

The known key-bias attack on plain Joint–Feldman applies: contributions arrive sequentially through the block-proposer path, so the last contributor sees every prior commitment and can steer the combined key to a chosen point. The repository’s design document concedes the bias and argues it is harmless here — the shifted key reveals nothing about the round secret under the discrete-log assumption, and the key serves only ElGamal encryption, whose IND-CPA security (indistinguishability under chosen-plaintext attack; the *Crypto Guide*, §“Security as a game”) holds for any valid key — citing Gennaro et al. (2007) for the security of Joint–Feldman threshold decryption despite a biased key (`vote-sdk/docs/tss-ceremony.md`, “Why single-phase”). The argument is itself designed but unspecified: a repository document by the designers, not a normative artifact or independent analysis.

Remark 3.4 (Threshold collusion). Any t colluding validators can reconstruct the round secret offline and decrypt every individual on-chain ciphertext; summing one vote’s sixteen shares then yields that vote’s exact per-decision weight, which the cast message already links in public to a VAN nullifier. The linkage stops at the VAN layer — the governance nullifier’s keyed PRF keeps Zcash notes and addresses out of reach. No published source claims per-vote weight privacy against a threshold coalition; a specification would have to state this limit explicitly (inference from `crypto/elgamal/bsgs.go` and `proto/svote/v1/tx.proto`, lines 88–99; recorded 2026-07-08).

3.6 Tally: chain-verified threshold decryption

Accumulation. On each accepted reveal, the chain adds the 64-byte ciphertext componentwise into the accumulator keyed by the triple (round, proposal, decision), validating well-formedness and rejecting identity components (`x/vote/keeper/keeper_tally.go`, lines 33–74). By linearity, the accumulator encrypts the summed ballots of all revealed shares for that cell. Nothing decrypts until the round enters its tallying phase.

Partial decryptions. During tallying, each ceremony validator’s node injects — through the block-proposer path; the mempool route is rejected — one submission covering every non-empty accumulator: the partial decryption $D_i := [s_i] C_1$ under its DKG share s_i , with a Chaum–Pedersen DLEQ proof that $\log_G VK_i = \log_{C_1} D_i$. The proof is the pair (e, z) ; the verifier recomputes $R_1 = [z] G - [e] VK_i$ and $R_2 = [z] C_1 - [e] D_i$ and accepts iff

$$e = H(\text{svote-pd-dleq-v1} \parallel G \parallel VK_i \parallel C_1 \parallel D_i \parallel R_1 \parallel R_2),$$

with H a BLAKE2b-256 hash-to-scalar; the tag separates this proof family from the tally-level DLEQ family (`svote-dleq-v1`), barring cross-family replay (`crypto/elgamal/dleq.go`, lines 30–145). The verification key VK_i is re-derived on the fly from the round’s combined Feldman commitments rather than stored (`x/vote/keeper/msg_server_tally_decrypt.go`, lines 257–269). The DLEQ is what makes the tally trustless against its own operators: one forged D_i would silently shift every total. Duplicate submissions per validator are rejected, and each submission must cover every non-empty accumulator, so a proposer cannot inject a partial set that later wedges the tally (`msg_server_tally_decrypt.go`, lines 199–294).

Final verification. The block proposer alone may submit the claimed plaintext totals. For each entry the chain Lagrange-combines at least t stored partials at zero (the coefficients λ_i below are the Lagrange basis polynomials of the *Math Guide*, §“Lagrange interpolation”, evaluated at zero),

$$\sum_i \lambda_i D_i = \left[\sum_i \lambda_i s_i \right] C_1 = [s] C_1,$$

recovering the round secret only in the exponent (`crypto/shamir/partial_decrypt.go`, lines 37–71), and checks $C_2 - [s] C_1 = [v] G$ against the claimed total v ; an empty accumulator forces $v = 0$; any $v \geq 2^{28}$ is rejected outright (the constant `TallyBSGSBound`, an exclusive bound; `x/vote/types/keys.go`, lines 33–35). A completeness check requires the entries to cover every non-empty accumulator — without it a malicious proposer could finalize with results omitted (`msg_server_tally_decrypt.go`, lines 102–177). Only then does the round finalize.

Recovering the logarithm. The chain verifies by re-encoding $[v] G$, so only the proposer must solve discrete logarithms, by baby-step/giant-step under $N = 2^{28}$: a table of $m = \lceil \sqrt{N} \rceil = 2^{14} = 16,384$ baby steps $j \mapsto [j] G$, giant steps subtracting $[m] G$, a hit at (i, j) giving $v = im + j$ within at most $m + 1$ iterations (`crypto/elliptic/bsgs.go`, lines 11–110). The bound is ample: the full 21-million-ZEC supply quantizes to 1.68×10^8 ballots at 12,500,000 zatoshi per ballot (`src/params.rs`, line 14), below $2^{28} \approx 2.68 \times 10^8$, so no honest aggregate is ever truncated.

Two flags. Totals are denominated in *ballots*; the protobuf comment on the tally field says zatoshi and is wrong — the wallet multiplies by 12,500,000 for display (`proto/svote/v1/tx.proto`, lines 111–116, against `zashi-android TallyResults.kt`, lines 44–56, commit `b4edd6f6`). And the tallying phase carries a timeout of 21,600 seconds (six hours), after which the round finalizes *without* results (`x/vote/types/keys.go`, lines 28–31) — an availability caveat, not an integrity one, but a specification must decide whether a resultless finalization is acceptable.

What a specification must pin down. Until zips PR #1200 merges, the commit-pinned sources above are the only citable definition of: the Poseidon preimage layouts and the domain-tag registry (currently pinned by unit tests); the three nullifier formulas, their set scoping, and their key encodings; the decrement lookup table and the sentinel-bit convention; the share count, the per-share range bound, and their interaction with partial reveals; the DKG parameter formulas, ceremony state machine, and jailing policy; the DLEQ transcript formats and tags; and the BSGS bound and the unit denomination of totals as consensus parameters.

4 Trust model and verdict

The preceding sections constructed the protocol’s mechanisms from commit-pinned source (Table 1); this section assembles what those mechanisms do *not* guarantee. We first map every party the protocol trusts and state exactly what each is trusted with, then classify the volume’s claims into the three bins of §1.3, and close with the verdict: what a formal specification would have to pin down before the protocol’s guarantees become checkable, and what the protocol delivers today against the instruments scheduled for the same June 2026 question. File citations below are at the commits of Table 1; wallet files are at `zashi-android @ b4edd6f6`.

Every trust relation in this map is itself designed but unspecified: each is concrete, open-source behaviour at the pinned commits, and none is stated in a normative artifact. The inventory of §4.2 records the full classification.

4.1 The trust map

Helper servers hold the vote decision and the linkage. The payload a wallet sends to each chosen helper — struct `SharePayload` in `zcash_voting/src/types.rs` — carries the plaintext `vote_decision`, the vote commitment’s `tree_position` in the per-round tree, the `shares_hash`, all sixteen ciphertexts, the sixteen share commitments, and the revealed share’s blinding factor (`primary_blind`); the helper builds the Merkle path and the share-reveal proof server-side and submits `MsgRevealShare` itself (`vote-sdk/internal/helper/processor.go`). A helper therefore learns, before any reveal, how the voter voted, and holds precisely the share-to-commitment linkage that the on-chain data blinds; composed with the public pairing of `vote_commitment` and `van_nullifier` in `MsgCastVote` (`proto/svote/v1/tx.proto`), that linkage ties the decision to the vote’s delegated authority instance. This is a broader grant than a timing-relay framing suggests. What a helper never receives is weight: the plaintext share values and the ElGamal randomness are marked `SECRET` in the client library and stripped from the wire type (`types.rs`, `EncryptedShare` versus `WireEncryptedShare`). The timing decorrelation itself is client policy, not a protocol guarantee: the wallet posts each share to half of the configured helpers rounded up, each with an independently random future `submit_at` scheduled no later than a last-moment buffer — two fifths of the round duration, capped at six hours — before the vote deadline (`zcash_voting/src/share_policy.rs`); a censoring helper causes partial weight loss, mitigated only by the multi-server fan-out and by a single-share fallback described below.

Any t colluding validators decrypt every posted ciphertext. Function `ThresholdForN` sets the reconstruction threshold $t = \max(2, \lfloor (2n + 2)/3 \rfloor)$ for n ceremony validators (`vote-sdk/x/vote/keeper/keeper_ceremony.go`). Any t of them can reconstruct the round’s ElGamal secret key offline from their dealt shares and decrypt every `enc_share` on the chain — not merely the aggregates the tally publishes. The decision attached to each ciphertext is public at reveal time in any case (`MsgRevealShare` carries `vote_decision`); what threshold collusion adds is the share values. In the single-share last-moment mode the exposure is immediate: the client builds a payload for share 0 only, “which carries all the voting weight”, and the remaining fifteen zero-value shares are never revealed (`zcash_voting/src/vote_commitment.rs`), so one decryption yields one vote’s exact weight. In the sixteen-share mode, recovering a per-vote weight additionally requires grouping shares by vote — a linkage the chain withholds (share nullifiers are blinded), the helpers of the previous paragraph possess outright, and the client-side timing policy merely obfuscates. The linkage stops at the Vote Authority Note layer in every case: the governance nullifier is keyed by the holder’s nullifier-deriving key `nk`, so mainnet notes and addresses stay unlinkable even after a later spend (`voting-circuits/src/delegation/circuit.rs`, condition 14). We flag the analytical gap explicitly: neither the public documentation nor the source claims — or denies — per-vote weight privacy against a colluding Election Authority threshold. The limit stated here is our adversarial inference from `crypto/elgamal/bgs.go`, the message surface, and the share-range condition of the vote-proof circuit; the property is absent from every published analysis.

The chain is closed and its lifecycle is coordinator-gated. The vote chain enforces a positive-security message whitelist at the ante handler: only enumerated message types pass, standard bank transfers are disabled — explicitly “because unrestricted transfers would let anyone accumulate enough stake to create a validator, bypassing the controlled validator set” (`vote-sdk/proto/svote/v1/tx.proto`, comment on `MsgAuthorizedSend`) — and post-genesis validator creation is disallowed outside the registered-Pallas-key path (`vote-sdk/app/ante_whitelist.go`). Round lifecycle, privileged funding, and software upgrades are threshold-gated *coordinator actions* proposed and approved by a vote-manager set (`MsgProposeCoordinatorAction`, `MsgApproveCoordinatorAction`); the default genesis ships a single vote manager, and the coordinator threshold defaults to one (`vote-sdk/x/vote/module.go`, `DefaultVoteManagerAddresses`; `tx.proto`, `MsgUpdateVoteManagers`). In production the vote-

manager custodians are named — by the configuration repository, not by the chain. At the volume’s pinned commit of `valargroup/token-holder-voting-config` (2785311d, 2026-05-27; the authentication root discussed below), the `trusted_keys` notes of `prod/static-voting-config.json` label three Keplr-derived keys as vote-manager keys for stated `sv1` addresses — key ids `valargroup`, `tachyon`, and `zodl`; a fourth, `shielded-labs`, arrived on 2026-05-29 (commit 82711181), and all four are present at head `e7d11990` (2026-06-25). These annotations are the custodians’ claims about themselves; the deployed chain’s on-chain `VoteManagerAddresses` set is published nowhere we could find. The production coordinator threshold is not publicly established: the repository carries no genesis file, no announcement states the value, and the registry’s only production round entry — `3fda6c83b054...`, added 2026-06-05 by pull request #330 — carries a single signature (key id `valargroup`), consistent with a threshold of one at the configuration layer but an inference, not a source. No validator identity is published either: the closest public artifact is the ten-operator `vote_servers` list of `prod/dynamic-voting-config.json` (at `e7d11990`), and a vote server is an RPC endpoint, not evidence that its operator runs a validator. We searched public artifacts on 2026-07-15; the deployed chain’s public REST endpoint would settle threshold and validator set directly and remains unqueried.⁶ The same coordinators post the snapshot: the eligibility anchors `nc_root` and `nullifier_imt_root` arrive in `MsgCreateVotingSession` as client-fetched values — the note-commitment root recomputed from a `lightwalletd` frontier, the nullifier root taken from the operator’s PIR service (`vote-sdk/api/snapshot.go`) — and the chain stores them without verifying that they describe any real Zcash state; it runs no light client (`x/vote/keeper/msg_server.go`). Honest wallets’ proofs fail against fabricated roots, and anyone can audit the posted roots after the fact, but nothing in the protocol prevents a coordinator quorum from pinning a snapshot of its own invention.

The wallet vendor’s configuration is the authentication root. A voter’s assurance that they are voting in a genuine round reduces to a static configuration file bundled with the wallet: fetched from a commit-pinned URL (`valargroup/token-holder-voting-config @ 2785311d`) and verified against a hard-coded SHA-256 digest, it names the `ed25519` keys that must sign the dynamic per-round configuration — helper endpoints, PIR endpoints, and each round’s Election Authority key (`StaticVotingConfig.kt`, `BUNDLED_PINNED_SOURCE`). The round authenticator rejects a round whose on-chain `ea_pk` disagrees with the signed entry (`RoundAuthenticator.kt`, status `EA_PK_MISMATCH`) — the wallet’s only defence against an attacker-substituted encryption key, under which every cast share would be attacker-decryptable. An on-chain endorsement registry (`MsgEndorseRound`) is secondary. The trust root of the whole client-side protocol is therefore the custodians of the config-signing keys together with the application distribution channel; the custodians are named only by the configuration’s own `trusted_keys` annotations (above), the distribution channel in no published artifact.

Tally liveness admits finalization without results. The tally verification itself is real: validators’ nodes auto-inject partial decryptions whose Chaum–Pedersen DLEQ proofs are checked on-chain against Feldman-derived verification keys, and the proposer-only `MsgSubmitTally` is accepted only if every claimed total matches the Lagrange combination of stored partials, covers every non-empty accumulator, and lies below the discrete-log recovery bound 2^{28}

⁶Static configuration at the pinned commit, Wayback snapshot 2026-07-15: <https://web.archive.org/web/20260715193658/https://raw.githubusercontent.com/valargroup/token-holder-voting-config/2785311d/prod/static-voting-config.json>; at head, via the served URL: <https://web.archive.org/web/20260715193616/https://voting.valargroup.org/prod/static-voting-config.json>; dynamic configuration at `e7d11990`: <https://web.archive.org/web/20260715193707/https://raw.githubusercontent.com/valargroup/token-holder-voting-config/e7d11990/prod/dynamic-voting-config.json>. Operators named in `vote_servers`: `valargroup`, `ruzcash`, `zend`, `ShieldedLabs-JM`, `Legends`, `stardust-staking`, `zodl`, `ZecHub`, `katz`, `Keplr` — a list not maintained as a roster: the 2026-06-14 ZODL update reports the “decommissioned Stardust servers” removed from the app while `stardust-staking` remains in the configuration of 2026-06-25. Both files follow the schema of draft ZIP 1244, “Shielded Voting Wallet API” (zcash/zips PR #1244).

(`vote-sdk/x/vote/keeper/msg_server_tally_decrypt.go`; `crypto/shamir/feldman.go`; the `TallyEntry` field `decryption_proof` is dead — “reserved for future use” — the binding check is the recomputation). But the threshold t is a liveness parameter as well as a secrecy parameter: if fewer than t validators decrypt, the round sits in TALLYING until the timeout — 21,600 seconds by default (`x/vote/types/keys.go`, `DefaultTallyTimeout`) — and end-of-block processing then finalizes it with `tally_timed_out = true` and *empty results* (`x/vote/module.go`). A finalized round in which every cast vote remains permanently undisclosed is a protocol outcome, not an excluded failure mode; the design accepts it as the price of not blocking the chain on an unreachable threshold.

The tally wire format mislabels its units. The comment on `TallyEntry.total_value` reads “Decrypted aggregate (zatoshi)” (`vote-sdk/proto/svote/v1/tx.proto:114`). It is wrong: the accumulated plaintexts are *ballot counts* — the vote-proof circuit constrains each vote’s shares to sum to the Vote Authority Note’s ballot count, and the wallet converts totals to zatoshi by multiplying by the ballot divisor 12,500,000 (`TallyResults.kt`, `toTallyResults`; constant `BALLOT_DIVISOR_ZATOSHI` in `VotingCryptoClient.kt`). The recovery bound corroborates the correct reading: 2^{28} ballots is roughly 33.6 million ZEC — comfortably above the monetary supply — whereas 2^{28} zatoshi is under 2.7 ZEC. A consumer that trusts the comment under-reports every result by a factor of 12.5 million. In a protocol whose only specification is its implementation, a wrong comment in the wire definition is not a cosmetic defect; it is an error in the closest thing the protocol has to a spec.

4.2 Claim inventory: the three bins

Specified. Only the inherited primitives: `NoteCommit`, nullifier derivation, `Commitivk`, the Sinsemilla Merkle instance, `RedPallas`, the ZIP-244 sighash, and ZIP-32 derivation, each specified in the protocol specification and the ZIPs, constructed in the *Ironwood Guide* (§“Representation of a note: notes and note commitments”, §“Prevention of double-spending: nullifiers”, §“Keys: capabilities for spending and for viewing”, §“Proof of ownership of *some* valid note: the commitment tree”, §“Authorisation of the spend: `RedPallas`”), and reused unchanged through the `unstable-voting-circuits` feature (`zcash/orchard @ c0b6b917`). No governance-specific object reaches this bin.

Designed but unspecified. Everything this volume constructed: the five-message ballot surface; the delegation relation with its note-bundle eligibility and indexed-Merkle-tree non-membership argument; the coordinator-posted, chain-unverified snapshot roots; the Vote Authority Note and its synthetic-action authorization — including the asymmetry that the chain verifies the delegation signature over a *client-provided* sighash, relying on the circuit’s ρ -chain for binding, while the cast-vote sighash is recomputed on-chain (`x/vote/ante/validate.go`); all four double-vote layers; the sixteen-share lifted-ElGamal encoding, vote commitment, and bitmask decrement; the helper protocol and the trust it grants; the per-round Joint-Feldman DKG with threshold $t = \max(2, \lfloor (2n + 2)/3 \rfloor)$ and finalization quorum $\lceil 4n/5 \rceil$; the chain-verified tally, its 2^{28} recovery bound, and the timeout path; the PIR eligibility subsystem; and every trust relation of §4.1. Each is concrete and open-source at the pinned commits; none is normative. The would-be specifications — `zcash/zips` PRs #1200 and #1198, and the prior-art PR #853 — were all open and unmerged as of 2026-07-08.

Open problem. One: the ballot-quantization relation. The delegation circuit constrains $\text{num_ballots} \cdot 12,500,000 + \text{remainder} = v_{\text{total}}$ with $\text{remainder} < 2^{24}$, which is strictly weaker than the floor division the public documentation states: the pair $(q - 1, r + 12,500,000)$ also satisfies every constraint whenever the honest remainder obeys $r < 2^{24} - 12,500,000 = 4,277,216$ — about 34.2% of residues — and the honest ballot count is at least two, while the opposite deviation fails the range check, so a prover can under-claim exactly one ballot and can never over-claim

(`voting-circuits/src/delegation/circuit.rs`; `src/delegation/README.md` §8, which states the window, proves over-claim impossible, and lists unimplemented tightening constraints). The deviation is self-harming, but it is the volume’s one confirmed disagreement between the code and the code’s own public description, and the exact relation a specification must choose — floor division or the shipped weaker relation — is unresolved.

4.3 Verdict

What a specification must pin down. For the protocol’s guarantees to become checkable — by an auditor, by a second implementation, or by this volume — a normative artifact would need to fix, at minimum:

1. the three circuit relations in full: public-input layouts, every condition, and the exact quantization rule — either closing the under-claim window or normatively accepting the shipped relation;
2. every governance-object formula: the four nullifiers and the nullifier-domain derivation, the Vote Authority Note commitment (with the argument that its x -only encoding is bound elsewhere), the vote commitment, and the shares hash;
3. the snapshot: how `nc_root` and the nullifier root are derived, and *who verifies them* — at present, on-chain, nobody;
4. the authorization model: which sighashes the chain recomputes, which it accepts from the client, and why the delegation path is safe;
5. the Election Authority: the threshold and quorum formulas, key registration and rotation, and — the largest analytical gap — an explicit confidentiality claim stating against whom per-vote weight privacy holds;
6. the helper interface: the exact payload, the trust granted by it, and the status of the timing policy as guarantee or convention;
7. tally semantics: the verification equation, the unit of `total_value` (ballots), and the meaning of a timed-out round;
8. independent verification: a second implementation of at least the verifiers, breaking the single-crate tautology of §1.3.

What the protocol delivers today. Measured against the instruments beside it (§1.2), the deliverable is real but conditional. Against ZCAP, the protocol opens participation to every Orchard coinholder in proportion to holdings, with no identified panel and no registrar, and it hides identity and weight from public observers — where ZCAP offers the converse trade: an identified panel on mature, independently analysed voting software. Against the prior coin-weighted instrument, `hhanh00’s zcash-vote`, it adds threshold decryption in place of a single tallying authority, blinded delegation through the Vote Authority Note, hardware signing, PIR-private eligibility queries, and wallet-native deployment — at the price of the trust map of §4.1: a permissioned chain, a coordinator quorum, helper servers, and a vendor-held configuration root. Integrity of the aggregate totals is the strongest property: the tally check is executed on-chain and every input to it is public, so any observer of the vote chain can replay it, and a fabricated snapshot is auditable after the fact. Individual privacy is the weakest: it holds against outside observers, degrades to helper honesty for the linkage of decisions to votes, and degrades to the honesty of all but $t - 1$ validators for individual weights — a limit no published analysis states. On that evidence the protocol is fit for the use provisioned for June 2026 — a non-binding sentiment poll with publicly recomputable totals — though that use has yet to occur (§1.1). Binding governance would additionally require the specification above, an independent verifier, and a confidentiality analysis of the Election Authority — none of which existed as of 2026-07-08.

Part XI

ZSA Guide: Zcash Shielded Assets: issuance, transfer, and the counterfeiting boundary

Abstract

Assuming the Orchard protocol of the *Ironwood Guide*, this volume of *The Zcash Arboretum* documents Zcash Shielded Assets: per-asset value bases and asset identity, transparent issuance and finalization (ZIP-227), shielded transfer and burn (ZIP-226), and the split-note construction that guards the counterfeiting boundary. The upstream ZIPs are in flux — their transaction carrier was withdrawn and their fee terms removed upstream — so every claim is classified as specified, designed-but-unspecified, or open, and where ZIP text, fork text, and the implementation disagree (as they do on the split-note nullifier derivation), all three readings are stated with the divergence flagged rather than silently resolved.

1 Scope, status, and sources

Zcash Shielded Assets (ZSA) is the proposed extension of Orchard that carries user-issued assets in the shielded pool. Two consensus ZIPs define it: ZIP 226, “Transfer and Burn of Zcash Shielded Assets”, and ZIP 227, “Issuance of Zcash Shielded Assets” — both Status Draft, Category Consensus, created 2022-05-01, owned by Pablo Kogan and Vivek Arte (QED-it), Daira-Emma Hopwood, and Jack Grigg, with credits to Daniel Benarroch, Aurelien Nicolas, Deirdre Connolly, and Teor, and discussed in the single thread `zcash/zips#618` (`zip-0226.rst` and `zip-0227.rst` lines 1–18 @ 69610984). The combined protocol, OrchardZSA, enables issuance (ZIP 227) and transfer and burn (ZIP 226) of Custom Assets on the Zcash chain; ZIP 227 must only be implemented in conjunction with ZIP 226, and issuance is deliberately transparent — issuance outputs are unencrypted — so that every asset’s circulating supply can be tracked on chain (`zip-0226.rst` lines 42–43; `zip-0227.rst` lines 47, 54).

Terminology is ZIP 227’s: an *Asset* is a type of note transferable on the Zcash blockchain; ZEC is the default — and currently the only defined — Asset on mainnet, TAZ on testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (`zip-0227.rst` lines 35–39).

The design reduces to one alteration of Orchard’s value algebra. The fixed value base V^{Orchard} of the homomorphic Pedersen value commitment (*Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”) is replaced, per Asset, by an *Asset Base* witnessed inside the circuit; the same Asset Base must appear in the old and the new note commitments and as the base of the Action’s value commitment, so balance holds separately per Asset Identifier without revealing which Assets — or how many distinct ones — a transaction transfers (`zip-0226.rst` lines 59–63). The transparent protocol is unchanged: unshielding is impossible for Custom Assets, burn is the only supply-reduction mechanism and sends to no address, and the Native Asset cannot be burnt (`zip-0226.rst` lines 206–230).

1.1 Scope and deferrals

This volume develops only the delta over Orchard: the per-asset note type and note commitment; the Asset Base and its derivation from an Asset Identifier; the per-asset value commitment, the burn mechanism, and the burn-aware balance equation; split notes, the mechanism that replaces dummy spends for Custom Assets; the changes to the Action circuit; the issuance layer of ZIP 227 — issuance keys, issue notes, reference notes, and the node-side issuance state — and the note-plaintext extension.

Everything the delta rests on is constructed in earlier volumes and is cited, never re-derived: notes and note commitments (*Ironwood Guide*, §“Representation of a note: notes and note commitments”); value commitments, the bases V^{Orchard} and R^{Orchard} , and the binding signature (*Ironwood*

Guide, §“Conservation of value: value commitments and the binding signature”); the Action statement and dummy spends (*Ironwood Guide*, §“The Action statement, formally”); nullifiers (*Ironwood Guide*, §“Prevention of double-spending: nullifiers”); Sinsemilla (*Ironwood Guide*, §“The commitment primitive: Sinsemilla”); and, at the wallet layer, transaction construction, PCZT, and fees (*Wallet Guide*, §“Transaction construction”, §“Partially created transactions (PCZT)”, §“Fees (ZIP-317)”).

Deployment status is stated once, in §1.2. Subsequent sections construct their objects without requalifying each of them with it.

1.2 Status

Both ZIPs are Status Draft. ZIP 226’s Deployment section reads: “The Zcash Shielded Assets protocol is scheduled to be deployed in Network Upgrade 7 (NU7). This ZIP currently assumes that this will be the case” (`zip-0226.rst` lines 425–426). ZIP 227’s Deployment section is “TBD”, and its Test Vectors and Reference Implementation entries read “LINK TBD” (`zip-0227.rst` lines 661–675).

The network-upgrade reality is thinner than the assumption. ZIP 254, the NU7 deployment ZIP, is Withdrawn and delegates to `draft-arya-deploy-nu7` (Status Draft), which fixes the consensus branch ID `0x77190AD8` and the minimum network protocol versions 170150 (testnet) and 170160 (mainnet), leaves the activation height TBD on both networks, and lists no feature ZIPs at all (`zip-0254.md`, `draft-arya-deploy-nu7.md` @ 69610984). The repository README’s “NU7 Candidate ZIPs” list (218, 230, 231, 233, 234, 235, 2002, 2003, plus a possible ZIP 317 update) does not include ZIP 226 or ZIP 227, still names the withdrawn ZIP 230, and is explicitly non-decisional: “no decision has been made on whether to include each of these ZIPs”; the deployment draft “will define which ZIPs are included in NU7” (README.rst lines 70–92 @ 69610984). In the current zips tree, ZIP-204 assigns NU7 protocol versions 170180 (Testnet) and 170190 (Mainnet), but `draft-arya-deploy-nu7` still says 170150 and 170160; the draft is therefore internally stale even though its activation heights remain TBD (zips `ad30e59`, ZIP-204 and `draft-arya-deploy-nu7`). In the Zcash Foundation’s February 2026 NU7 sentiment polls, the proposal that drew “universal or near-universal support” was Tachyon, not ZSA. ZSA’s own standing in those polls is split: ZCAP supported ZSAs at 70.4%, the coinholder poll ran 98.6% against, and the Foundation did not recommend ZSAs for NU7, writing that “before any decision is made, we need to understand why coinholders oppose ZSAs so strongly” ([zfnd.org](https://zfnd.org/nu7-polling-results-what-we-heard-and-where-we-go-from-here/), “NU7 Polling Results: What We Heard and Where We Go From Here”, 2026-02-23, <https://zfnd.org/nu7-polling-results-what-we-heard-and-where-we-go-from-here/>).

The specification’s baseline moved beneath it in 2026. ZIP 226 is now defined relative to the protocol with ZIP 2005 (Ironwood Quantum Recoverability; Status Proposed, deploying with NU6.3 “Ironwood”) applied, and states that ZIP 2005 “is expected to deploy before any ZSA activation” (`zip-0226.rst` line 45, which still cites the ZIP under its older name “Orchard Quantum Recoverability”; `zip-2005.md` header). The dependency points upward: if the NU6.3 content changes, the ZSA specification inherits the change.

The carrier format has no live specification. ZIP 226 defines its transaction structure and sighash by deferring to ZIP 230 (the v6 transaction format) and ZIP 246 (the v6 digests) (`zip-0226.rst` lines 373–405, 456–462); both are now Withdrawn. ZIP 230 is retitled “Withdrawn Version 6 Transaction Format” and warns: “This ZIP has been obsoleted by ZIP 229, and will not be deployed. Transaction version number 6 is now defined by ZIP 229” (`zip-0230.rst` lines 5, 14, 37–40; withdrawal commit `04db8968`, 2026-05-16). ZIP 246 is retitled “Digests for the Withdrawn Version 6 Transaction Format” (`zip-0246.rst` lines 4–11, 36–44; commit `00f37219`, merged 2026-07-09). The ZIP 229 that now owns transaction version 6 (Status Draft, created 2026-06-13, for NU6.3) excludes ZSA by declared non-requirement: “The v6 transaction format need not support ZSAs, the `zip233Amount` field, the explicit fee field, or per-transparent-input sighash information that appeared in the withdrawn ZIP 230, or those features and other extensibility affordances planned for ZIP 248”, and “The value 6 for the transaction version number need not avoid collisions with the withdrawn ZIP 230 or with the current draft of ZIP 248” (`zip-0229.md` lines 154–162). ZIP 248, the intended extensible-format home for the ZIP 230/231/246 material, exists only as the unmerged pull request `zcash/zips#1156`.

The rot reaches into the ZSA texts themselves: ZIP 227’s SigHash consensus rule cites “modifications described in ZIP 226 [#zip-0226-txiddigest]”, an anchor ZIP 226 no longer contains (`zip-0227.rst` lines 474, 693). This volume therefore bins the entire transaction-format layer — wire format, digest tree, sighash — as designed but unspecified (§1.4), even though the cryptographic core above it is specified.

The v6 note-plaintext lead byte is formally unassigned. Occurrences of ZIP 230’s constant (originally 0x03) were changed to the placeholder `{{LEADBYTE}}` “to clarify that this ZIP does not reserve that lead byte value, and to avoid confusion with the different specification of lead byte 0x03 in ZIP 2005”; ZIP 226 correspondingly requires the placeholder `{{ZSALEADBYTE}}` (`zip-0230.rst` lines 41–44; `zip-0226.rst` line 162; commit 8418662, 2026-05-16). The pinned implementation ships 0x03 (§1.3).

The ZSA fee terms lost their upstream home on 2026-06-26: ZIP 317’s change history for that date records “Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)”, so ZIP 227’s “Changes to ZIP 317” section now points at a fee calculation that contains no ZSA terms (`zip-0317.rst` lines 11–15, 668–676; `zip-0227.rst` lines 604–610). The issuance and asset-creation contributions survive only in the QED-it zips fork and in `librustzcash-zsa` (§1.3); they are binned designed but unspecified.

Two claims commonly attached to ZSA are stated here at exactly the rigor the sources support. ZIP 226 lists reference implementations — the QED-it forks of `zcashd`, `orchard`, `librustzcash`, and `halo2` — and test vectors at QED-it/`zcash-test-vectors` (`zip-0226.rst` lines 428–439). No artifact in the local corpus records an audit of ZSA (a repository-wide search over the ZIP texts and all three QED-it forks finds none), and none documents a testnet deployment; the only local traces of node integration are the temporary-zebra cargo feature and the fork ecosystem itself. Both claims therefore rest on web sources, pinned here by URL and date, and both are scoped. Audit: Least Authority TFA GmbH, *OrchardZSA Protocol Security Audit Report* (commissioned by ZCG as Zcash Ecosystem Security Lead), Updated Final Audit Report 30 January 2025. Scope: the QED-it `orchard zsa1-vs-main` diff (PR QED-it/`orchard`#7, including the ZSA circuit extensions) and the QED-it `halo2` gadget changes (PR QED-it/`halo2`#18), at revisions `a7c02d22` (initial) and `01e85a5f` (verification); findings: no issues, four suggestions (all but one resolved). <https://leastauthority.com/wp-content/uploads/2025/01/Least-Authority-ZCG-OrchardZSA-Protocol-Updated-Final-Audit-Report.pdf>. The audit does not cover `librustzcash-zsa` or the v6 transaction format layer, and the audited revisions predate the pinned `cf801a5d`. Testnet: ZecHub, *Zcash Shielded News Vol. 61* (2025-12-17): “A feature-complete version of ZSAs is now available on testnet” (<https://zechub.substack.com/p/zcash-shielded-news-vol61>); the testnet is QED-it’s public Zebra-based single-node ZSA testnet (see [forum.zcashcommunity.com](https://forum.zcashcommunity.com/thread/53293) thread #53293, “ZSA Testnet Question”). This volume accordingly asserts “audited” only of the `orchard/halo2` circuit layer at the audited revisions, and “running on QED-it’s public testnet” of the stack as a whole — and nothing stronger.

1.3 Sources

Table 1 lists the primary sources, each pinned to a commit. All facts in this volume were verified firsthand at the stated commits; the only web sources cited are pinned by URL and date — the Least Authority OrchardZSA audit report (2025-01-30), ZecHub *Shielded News* Vol. 61 (2025-12-17), and the Zcash Foundation’s NU7 Polling Results post (2026-02-23), all three in §1.2.

Current-head boundary (2026-08-30). The QED-it forks have moved since the source snapshot: `orchard b578ad9` is based on 0.15.0 and adds the NU6.3 cross-address restriction to the ZSA circuit, while `librustzcash c5c232d` synchronises the NU6.2-era upstream transaction changes. Neither change supplies a current normative carrier for ZSA: upstream ZIPs 230 and 246 remain withdrawn, ZIPs 226 and 227 remain Draft, and the NU7 deployment draft still does not select them. Detailed

Artifact	Pin / date	Role
zcash/zips (upstream)	69610984, 2026-07-09	primary ZIP ground truth
QED-it/orchard, branch zsa1	cf801a5d, 2026-06-29	crate orchard 0.14.0
QED-it/librustzcash, branch zsa1	3f9b53fc, 2026-04-17	v6 carrier, fee rule
QED-it/zips, branch zsa1	fd71419a, 2026-02-11	fork baseline for diffing

Table 1: Primary ZSA sources, pinned.

wire, circuit, and fee claims below therefore continue to describe the explicit pins in Table 1; later fork behaviour is not silently attributed to that public-testnet snapshot.

Upstream versus fork. Upstream ZIP 226 diverged from the fork text at fd71419a in three substantive ways: it added the ZIP 2005 dependency paragraph; it changed the split-note ψ^{nf} from “sampled uniformly at random on $\mathbb{F}_{p_{\text{Pallas}}}$ ” to a PRFexpand derivation whose domain-separator byte 0x0A is reserved by ZIP 2005; and it replaced the lead byte 0x03 with the `{{ZSALEADBYTE}}` placeholder. ZIP 227 upstream differs from the fork only in punctuation and one reference title.

Code versus ZIP. Where the pinned code disagrees with the pinned ZIP text, this volume presents the ZIP formula and flags the divergence inline. Two content divergences head the list. First, the split-note nullifier randomizer: ZIP 226 derives ψ^{nf} with domain byte 0x0A, while orchard-zsa @ cf801a5d computes it with PRFexpand domain 0x09 over a random per-note seed (src/circuit.rs lines 316–319) — the older fork-era derivation. Second, the note-plaintext lead byte: a placeholder upstream, but 0x03 in code (src/primitives/zcash_note_encryption_domain.rs lines 46–49) — the very value ZIP 2005 now specifies differently. A third divergence is one of status rather than content: the digest personalisations in orchard-zsa match the withdrawn ZIP 246 exactly, and librustzcash-zsa implements TxVersion::V6 with the withdrawn ZIP 230 wire format — the entire carrier layer the code implements is specified only by withdrawn ZIPs (§1.2). Further code-versus-text divergences confirmed at the same pins — the omitted explicit-fee digest field, the missing memo branch and its non-spec substitute digest, the empty-action issuance rule, and the valueBurn bound wording — are inventoried in §6.6, §3.7, and §4.4.

Pin consistency. The two implementation forks are not mutually consistent snapshots: librustzcash-zsa @ 3f9b53fc patches orchard to QED-it rev 77d3274c, older than the orchard-zsa checkout head cf801a5d; it also pins sapling-crypto @ 59535fb5 and zcash_spec @ d5e84264, while orchard-zsa pins halo2_proofs/halo2_gadgets @ ef3d0ba2 and the same zcash_spec (Cargo.toml lines 234–239 and 118–123 respectively). Each fork is cited at its own pin; where the gap between them is load-bearing, the text says so.

Gating. In librustzcash-zsa, all ZSA and v6 code sits behind `cfg(zcash_unstable = "nu7")`; in orchard-zsa, issuance sits behind the cargo feature `zsa-issuance` (which pulls in `secp256k1`) and node integration behind `temporary-zebra`.

Non-sources. The branch `1tf/zsa` of upstream zcash/orchard points at a799082e (2025-11-25), an ancestor of `main`, and contains no ZSA code; it is not usable ground truth. Node-side enforcement is a second gap: ZIP 227 specifies chained per-node issuance state (the `issued_assets` map), but no node implementation is checked out locally, so the supply and burn logic verified for this volume is library-side only (orchard-zsa src/bundle/burn_validation.rs, src/issuance.rs).

1.4 Method: the three bins for ZSA

This volume adopts the series’ classification of design-stage claims (*Voting Guide*, §“The source situation: code as the de facto specification”): every claim is *specified*, *designed but unspecified*, or an *open problem*, and the bin stays visible in the prose. The distribution differs sharply from the Tachyon volume’s. Bin (i) holds the entire cryptographic core — the per-asset note and its commitment, the Asset Base derivation, value commitments and the balance equation, split notes, the circuit statement, issuance keys, issue notes and reference notes, the issuance state machine, and burn — because the ZIP texts state it at consensus rigor and the pinned implementation cross-checks it. Bin (ii) holds the carrier layer: the v6 wire format, the digest tree and sighash, the note-plaintext lead byte, and the ZSA fee terms, each specified today only by withdrawn ZIPs, the QED-it fork texts, or code (§1.2). Bin (iii) holds NU7 inclusion and activation, the eventual lead-byte assignment, and the carrier format’s future home. Per the series conventions, every formula in this volume is transcribed verbatim from the pinned ZIP text or the pinned code, and the citation names which.

2 Asset identity

An OrchardZSA note denominates value in one of many assets sharing a single shielded pool, so the protocol must attach to every note an asset identity satisfying two demands. First, uniqueness: “The Asset identification should be unique (among all shielded pools) and different issuer public keys should not be able to generate the same Asset Identifier” (`zip-0227.rst:76`). Second, usability inside the circuit: the identity must ultimately be a Pallas point, because it serves as the asset-specific base of the value commitment (§2.4). ZIP 227 meets both demands with one derivation chain,

$$\begin{aligned} \text{asset_desc} &\longrightarrow \text{assetDescHash}; \\ (\text{issuer}, \text{assetDescHash}) &\longrightarrow \text{AssetId} \longrightarrow \text{AssetDigest} \longrightarrow \text{AssetBase}, \end{aligned}$$

which the ZIP itself presents as a relation diagram (`zip-0227.rst:210–224`): the issuer-chosen description string is hashed, the hash paired with the issuer key into an identifier, the identifier hashed into a digest, and the digest mapped onto the Pallas curve. Every Asset issued under ZIP 227 is scoped to the issuer that issued it, and its Asset Identifier is globally unique, “used across all Zcash protocols that support ZSAs” (`zip-0227.rst:227–240`).

This section constructs the chain end to end: the issuance key pair at its root (§2.1); the hash steps from description to identifier (§2.2); the group hash from identifier to base point (§2.3); the native asset’s special position outside the chain (§2.4); and the parts of the chain consensus sees on the wire versus recomputes (§2.5). Deployment status for the whole protocol is fixed once, in §1.2, and not restated per object.

2.1 The issuance key pair

The OrchardZSA protocol adds exactly two keys to the key material of Orchard: the *issuance authorizing key* `isk`, which authorizes the issuance of Asset Identifiers by a given issuer and is used only by that issuer, and the *issuance validating key* `ik`, which validates issuance transactions (`zip-0227.rst:84–91`). Neither key touches the spend path. The ZIP’s rationale: the issuance key structure is independent of the original key tree but derived analogously via ZIP 32, which keeps issuance details and Asset Identifiers consistent across multiple shielded pools, and separates issuance authority from spend authority — allowing a potential transfer of issuance authority without compromising spend authority (`zip-0227.rst:524`).

Definition 2.1 (Issuance authorization signature scheme). The scheme `IssueAuthSig` is instantiated as a BIP-340 Schnorr signature over the `secp256k1` curve, with constants set per the `secp256k1`

standard parameters as described in BIP 340. Its associated types are: messages are arbitrary byte sequences; signatures are 64-byte sequences or $\perp$; public keys are 32-byte sequences or $\perp$; private keys are 32-byte sequences (zip-0227.rst:122–135). Signing fixes the BIP-340 auxiliary data to $a = [0x00]^{32}$: the signature is $\sigma := \text{Sign}(\text{isk}, M)$ with auxiliary data a , or $\perp$ if Sign fails; on the wire it is encoded as $\text{issueAuthSig} = [0x00] \parallel \sigma$ (zip-0227.rst:190–207).

In the implementation, type `ZSASchnorr` realizes the scheme with `ALGORITHM_BYTE = 0x00`, secret keys as `secp256k1::SecretKey`, public keys as `XOnlyPublicKey`, and signing via `sign_schnorr_with_aux_rand` with the all-zero 32-byte auxiliary array (orchard-zsa src/issuance/auth.rs:163–273 @ cf801a5d).

Construction 2.2 (Issuance key derivation). The issuance authorizing key is generated by ZIP 32’s hardened-only key derivation — the maps `MKGh` and `CKDh`, constructed in the *Wallet Guide*, §“Hardened-only derivation”, and not re-derived here — instantiated in a dedicated Issuance context (zip-0227.rst:139–166). Let S be a seed of at least 32 and at most 252 bytes.

- (1) *Context.* The issuance context sets `Issuance.MKGDomain = ZcashSA_Issue_V1` and `Issuance.CKDDomain = 0x81` (zip-0227.rst:148–149).
- (2) *Master key.* Set $m_{\text{Issuance}} := \text{MKGh}^{\text{Issuance}}(S)$.
- (3) *Child derivation.* Set $\text{CKDsk}((\text{sk}_{\text{par}}, c_{\text{par}}), i) := \text{CKDh}^{\text{Issuance}}((\text{sk}_{\text{par}}, c_{\text{par}}), i)$. The two-argument call fixes the general form’s *lead* = 0 and *tag* = [], exactly the case whose suffix the ZIP-32 encoding omits (zip-0032.rst:449–462; the *Wallet Guide*’s remark “The general form: triples, and sibling instantiations”).
- (4) *Path.* Derive along $m_{\text{Issuance}} / \text{purpose}' / \text{coin_type}' / \text{account}'$, with *purpose* fixed to 227 (0xe3; hardened 0x800000e3 per BIP 43), *coin_type* as in the ZIP 32 key path levels, and *account* fixed to index 0 (zip-0227.rst:160–164).
- (5) *Key extraction.* From the generated pair (sk, c) , set $\text{isk} := \text{sk}$ (zip-0227.rst:164–166).

The implementation realizes the context as a struct `Issuance` implementing the `zip32` crate’s hardened-only `Context` trait, with `MKG_DOMAIN` set to `ZcashSA_Issue_V1` and with `CKD_DOMAIN` reusing the Orchard byte, constant `PrfExpand::ORCHARD_ZIP32_CHILD`; the seed-level constructor `from_zip32_seed` derives along $[227', \text{coin_type}', 0']$ and rejects any non-zero account with error `NonZeroAccount` (orchard-zsa src/issuance/auth.rs:38–99, 219–236 @ cf801a5d). The `zip32` crate 0.2.0 implements `MKGh` and `CKDh` exactly, and its `derive_child(index)` calls `ckdh_internal(index, 0, [])` — *lead* = 0, *tag* = [], omitted from the PRF message (src/hardened_only.rs:95–130).

Remark 2.3 (Domain separation between the Orchard and Issuance trees). The byte `Issuance.CKDDomain = 0x81` equals `Orchard.CKDDomain` (zip-0032.rst:476), and the implementation reuses `PrfExpand::ORCHARD_ZIP32_CHILD` (zcash_spec src/prf_expand.rs:119 @ d5e84264). Separation between the two ZIP-32 trees therefore rests entirely on the differing `MKGDomain`: distinct master secret keys and chain codes feed every subsequent `PrfExpand` call. Neither ZIP states whether the byte reuse is deliberate; ZIP 32 asks only that `CKDDomain` values be tracked as part of the global set of domains defined for `PrfExpand`. The derivation is *specified*; we flag the shared byte because the separation argument is easy to misattribute to it.

Definition 2.4 (Issuance validating key and issuer identifier). The issuance validating key is $\text{ik} := \text{PubKey}(\text{isk})$, where `PubKey` is the BIP-340 algorithm and the output is in big-endian order as defined in BIP 340; the derivation returns $\perp$ if `PubKey` fails, which happens with very

low probability, in which case the keys are discarded and derivation repeated with a different `isk` (`zip-0227.rst:168–183`). Its encoding prepends a signature-scheme byte, which **MUST** be `0x00`, indicating BIP 340:

$$\text{ik_encoding} = [0x00] \parallel \text{ik} \quad (33 \text{ bytes}),$$

and the issuer identifier is defined by `issuer = ik_encoding` (`zip-0227.rst:103–110, 178–179`). The encoding currently appears on chain only in the `issuer` field of an issuance bundle (`zip-0227.rst:178–180`).

Remark 2.5 (Permanence of the issuer binding). ZIP 227 notes that “the equality of `issuer` and `ik_encoding` might not hold in future when key rotation is specified for issuance key pairs” (`zip-0227.rst:103–110`). No rotation mechanism exists today: an asset’s identity is permanently bound to the one `ik` under which it was issued, and the ZIP’s remedy for key compromise is to finalize the asset and reissue under a new Asset Identifier: “If an issuer identifier is compromised, the finalize boolean for all the Assets issued with that key should be set to 1; then the issuer should change to a new issuance authorizing key (hence a new issuer identifier), and issue new Assets, each with a new `AssetId`” (`zip-0227.rst:640–645`, Security and Privacy Considerations, “Issuance Key Compromise”). This is a *stated limitation* of the specified design; the rotation mechanism itself is an open design area.

2.2 From description to identifier

An issuer names each of its assets with an *asset description* `asset_desc`: a non-empty byte sequence, which **SHOULD** be a well-formed UTF-8 code unit sequence according to Unicode 15.0.0 or later (`zip-0227.rst:237–241`). Within the scope of an issuer, Asset Identifier uniqueness is obtained by way of the asset description (`zip-0227.rst:237–240`). The current ZIP imposes no maximum length — its rationale notes that a description “can be longer than 512 bytes without incurring chain costs”, the 512-byte cap being an earlier design (`zip-0227.rst:528–539`). The description is also global: ZIP 226 states that “Every Asset is defined by an Asset description, `asset_desc`, which is a global byte string (scoped across all future versions of Zcash)”, so that in a future upgrade the same string can carry over to a different shielded pool, with nodes transforming the identifier chain between pools without balance violations (`zip-0226.rst:93–103`).

Definition 2.6 (Asset Identifier). The description is hashed,

$$\text{assetDescHash} := \text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \text{asset_desc})$$

(`zip-0227.rst:243–247`), and the Asset Identifier is the pair

$$\text{AssetId} := (\text{issuer}, \text{assetDescHash})$$

(`zip-0227.rst:249–253`), with the canonical encoding

$$\text{EncodeAssetId}(\text{AssetId}) := [0x00] \parallel \text{issuer} \parallel \text{assetDescHash} \quad (66 = 1 + 33 + 32 \text{ bytes}),$$

where the initial `0x00` is a version byte enabling future issuance protocols (`zip-0227.rst:255–261`). The version byte is not to be confused with the byte that follows it — the first byte of `issuer`, which indicates the signature scheme (Definition 2.4); every current encoding happens to begin with two `0x00` bytes of distinct meaning.

The pair structure carries the uniqueness argument. Because the issuer identifier is a component of `AssetId`, two different issuers cannot produce the same identifier; in the ZIP’s words, the Custom Asset “is described via a combination of the issuer identifier and an asset description string, to preclude the possibility of two different issuers creating colliding Custom Assets” (`zip-0227.rst:525`).

Within one issuer, distinct descriptions separate assets via `assetDescHash`; for every new Asset there MUST be a new and unique Asset Identifier (`zip-0226.rst:93–96`).

Remark 2.7 (Why a hash, not the string). ZIP 227 replaces the direct description string of an earlier design with a collision-resistant hash for five stated reasons (`zip-0227.rst:528–559`): a fixed 32 bytes per Issue Action costs less bandwidth than up to 512 bytes; descriptions can exceed 512 bytes without chain cost; carrying the byte string on chain would not make the user-visible description consensus-visible anyway (the string could itself be a hash of an off-chain description); absent key rotation, marking an issuer as trusted is insufficient — each Asset Identifier needs independent out-of-band verification that conveys the description; and an on-chain description is an “attractive nuisance” — hashing forces wallets to obtain the description from a trusted party, a registry, or direct user approval. Consistent with the last point, wallets MUST NOT display just the `asset_desc` string as the name of the Asset; the ZIP suggests a petname system mapping names to Asset Identifiers via client configuration, or the Asset Digest as a compact unique byte sequence, e.g. in QR codes (`zip-0227.rst:263–266`).

Implementation. The identifier is the enum variant `AssetId : V0`, holding the validating key and the 32-byte description hash; `encode_asset_id` emits the version byte `0x00`, the 33-byte key encoding, and the hash — 66 bytes, matching `EncodeAssetId` (`orchard-zsa src/note/asset_base.rs:20–60 @ cf801a5d`). The function `compute_asset_desc_hash` enforces non-emptiness through its argument type `NonEmpty<u8>` and *panics* if the description is not well-formed UTF-8 (`orchard-zsa src/issuance.rs:135–154 @ cf801a5d`). The specification says SHOULD; the implementation enforces a hard MUST at its API boundary. Since only the 32-byte hash is consensus-visible, neither variant is consensus-enforceable: we classify the UTF-8 constraint as an issuer-side convention — *specified* as a SHOULD, with the stricter behaviour an implementation choice.

2.3 From identifier to base point

Two further steps land the identifier on the Pallas curve. The full chain, collected:

```
assetDescHash := BLAKE2b-256(ZSA-AssetDescCRH, asset_desc),
AssetId := (issuer, assetDescHash),
AssetDigestAssetId := BLAKE2b-512(ZSA-Asset-Digest, EncodeAssetId(AssetId)),
AssetBaseAssetId := ZSAValueBase(AssetDigestAssetId),
```

where the digest is defined at `zip-0227.rst:268–278`, the base at `zip-0227.rst:280–290`, and the OrchardZSA instantiation of the final step is

```
ZSAValueBase(AssetDigest) := GroupHashPallas(z.cash : OrchardZSA, AssetDigest)
```

(`zip-0227.rst:292–301`). The subscript `AssetId` may be omitted when clear from context (`zip-0227.rst:224`). The group hash is the protocol specification’s `GroupHashP` (§5.4.9.8, “Group Hash into Pallas and Vesta”), whose construction — `expand_message_xmd` over BLAKE2b, two field elements through the simplified SWU map, summed and carried through the isogeny — the *Ironwood Guide* develops in “Preliminaries: hashing and encoding on the curve”; we reuse it as a deterministic, public, nothing-up-my-sleeve map into the Pallas group and do not re-derive it here.

ZIP 226 types the result as a *non-trivial* point: `AssetBase` is “the unique element of the Pallas group that identifies each Asset in the Orchard protocol ... a valid group element that is not the identity and is not $\perp$ ” (`zip-0226.rst:111–117`), a non-identity point of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ in our notation. Its byte form is

```
asset_base := LEBS2OSP $\ell_p$ (reprp(AssetBase)),
```

the 32-byte representation stored in note plaintexts and burn fields (`zip-0226.rst:114`; `zip-0230.rst:540–545`). ZIP 226 notes the encoding assumes canonicity, “true for the Pallas group, but may not hold for future shielded protocols” (`zip-0226.rst:111–117`). Table 2 collects the sizes along the chain.

Object	Bytes	Form	Source
ik	32	x-only BIP-340 key, big-endian	zip-0227.rst:130–133
ik_encoding = issuer	33	$[0x00] \parallel \text{ik}$	zip-0227.rst:178–179
assetDescHash	32	BLAKE2b-256 output	zip-0227.rst:243–247
EncodeAssetId(AssetId)	66	$[0x00] \parallel \text{issuer} \parallel \text{assetDescHash}$	zip-0227.rst:255–259
AssetDigest	64	BLAKE2b-512 output	zip-0227.rst:268–278
asset_base	32	LEBS2OSP _{ℓ_p} (repr _{$\mathbb{P}$} (·))	zip-0230.rst:540–545

Table 2: Byte sizes along the asset-identity derivation chain.

Why the base must be a genuine group-hash output. The chain’s final step is not a formality. In a transfer, the output note of a split Action cannot be allowed to carry an arbitrary Pallas point as its base: “We must enforce it to be an actual output of a GroupHash computation ... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds” (zip-0226.rst:299–307). The circuit therefore ensures the value base points of all output notes are actual GroupHash outputs “as they originate in the Issuance protocol which is publicly verified”, and ZIP 226 adds: “We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state” (zip-0226.rst:104–106, 299–307). The enforcement mechanism belongs to the transfer sections of this volume (§5).

Remark 2.8 (The identity-point corner case). The protocol specification’s $\text{GroupHash}^G(D, M)$ has the full group as codomain, so its output can in principle be the zero point (protocol.tex §5.4.9.8; the *Crypto Guide* constructs the hash-to-curve pipeline). ZIP 227’s ZSAValueBase never states what happens in that event; the non-identity requirement comes from ZIP 226’s typing of the note field (zip-0226.rst:111–117) and from the implementation, which panics inside `AssetBase::custom` (“this will happen with negligible probability”), rejects the identity in `AssetBase::from_bytes`, and carries an `Error::AssetBaseCannotBeIdentityPoint` check in `IssueAction::verify` that the panic renders unreachable (orchard-zsa src/note/asset_base.rs:98–138, src/issuance.rs:216–244 @ cf801a5d). Whether the intended consensus behaviour is “reject the transaction” or “the issuer must pick a different description” is unspecified in the ZIP text. The event has negligible probability; the specification gap is nevertheless real, and we classify the corner case as *open*.

2.4 The native asset

ZEC is the default — and currently the only defined — Asset for mainnet, TAZ for testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (zip-0227.rst:35–39). The native asset does not pass through the derivation chain at all. Its base is fixed by definition: “the Asset Base of the ZEC Asset will be kept as the original value base point, V^{Orchard} ” (zip-0226.rst:98); for ZEC,

$$\text{AssetBase}_{\text{AssetId}} := V^{\text{Orchard}},$$

“so that the value commitment for ZEC notes is computed identically to the Orchard protocol deployed in NU5. As such $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v)$ as defined in ZIP 224 is used as $\text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(V^{\text{Orchard}}, v)$ ” (zip-0226.rst:182–194). The point $V^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "v")$ is already constructed, together with its companion R^{Orchard} , in the *Ironwood Guide*, “Conservation of value: value commitments and the binding signature”, Definition “Net value commitment”; we cite it and do not re-derive it.

Two consequences follow. First, indistinguishability: ZIP 230 states that “An Orchard note is essentially equivalent to an OrchardZSA note with $\text{AssetBase} = V^{\text{Orchard}}$; in particular, they have the same note commitment ... and need not be distinguished in note plaintexts” (zip-0230.rst:531–538), and ZIP 226’s privacy discussion states that OrchardZSA note commitments for the native as-

set are indistinguishable from those for non-native Assets (zip-0226.rst:83–84). Second, exclusion from burning: the first consensus rule for `assetBurn` requires $\text{AssetBase} \neq V^{\text{Orchard}}$ for every $(\text{AssetBase}, v) \in \text{assetBurn}$ (zip-0226.rst:220–223) — the native asset cannot be burnt.

Implementation. The native base is `AssetBase::zatoshi()`, computed as the Pallas hash-to-curve over personalization "z.cash:Orchard-cv" of the message "v" — exactly V^{Orchard} ; `AssetBase::is_zatoshi()` is a constant-time equality with that point; and `ValueCommitment::derive` computes $[v]V + [\text{rcv}]R$ with $V = \text{asset.cv_base}()$ and R the fixed base R^{Orchard} , so a ZEC note’s value commitment is bit-identical to NU5 Orchard’s (orchard-zsa @ cf801a5d: note/asset_base.rs:140–155, constants/fixed_bases.rs:35–45, value.rs:380–400).

2.5 Identity on the wire

Of the whole chain, consensus sees exactly two links on the issuance side: the issuer and the description hash. The v6 transaction’s issuance bundle carries `issuerLength` (compactSize), `issuer` (the issuer identifier per ZIP 227), `nIssueActions` (compactSize), `vIssueActions`, and `issueAuthSig` — the signature bytes preceded by a 0x00 byte indicating BIP 340. The first four fields are always present; `issueAuthSig` is present iff `nIssueActions > 0`, and if `nIssueActions = 0` then `issuerLength` MUST be 0 (zip-0230.rst:166–179, 215–217, 430–443). Each Issue Action carries `assetDescHash` as byte[32], `nNotes` (compactSize, possibly zero, to finalize without issuing), `vNotes` (115 bytes per note description: 43-byte recipient, 8-byte value, 32-byte ρ , 32-byte rseed), and `flagsIssuance` with finalize in the least significant bit; the burn encoding carries `assetBase` as byte[32] (zip-0230.rst:367–383, 446–494). On the transfer side, the v6 Orchard note plaintext inserts the asset between rseed and the memo key (the 32-byte key that replaces the in-band 512-byte memo under ZIP-231 and from which the recipient derives the key decrypting its memo in the transaction-level memo bundle; §4.7),

$$[\text{leadByte}] \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}}$$

(zip-0230.rst:523–553).

The consensus rules bind identity to validation: the `ik_encoding` used to validate `issueAuthSig` MUST be taken from the `issuer` field and MUST start with byte 0x00, and for each Issue Action, the Issue Note’s `AssetBase` is derived from the `assetDescHash` field of the action and the `issuer` field of the bundle (zip-0227.rst:483–491). The implementation makes the recomputation structural. Function `read_bundle` decodes the issuer via `IssueValidatingKey::decode`, checking the algorithm byte; function `read_action` reads the 32-byte hash and recomputes the base as `AssetBase::custom` of `AssetId::new_v0(ik, asset_desc_hash)`. The Asset Base is never carried on the wire for issuance, always re-derived (librustzcash-zsa @ 3f9b53fc, crate `zcash_primitives`, file `transaction/components/issuance.rs`, lines 22–90). Redundantly with construction, `IssueAction::verify` checks that every note’s base equals the one derived from the validating key and the description hash, failing with error `IssueBundleIkMismatchAssetBase` otherwise (orchard-zsa src/issuance.rs:216–244 @ cf801a5d). An asset’s identity is therefore not an on-chain object at all: it is a recomputable function of two consensus-visible byte strings, anchored to the issuer key that signs the bundle.

Classification. The entire derivation chain of this section is *specified*: every formula above appears in the ZIP texts at rigor and is realized by the pinned implementation forks at the pinned commits (orchard-zsa @ cf801a5d, librustzcash-zsa @ 3f9b53fc). Both ZIPs are Status Draft, Category Consensus, owned by Pablo Kogan, Vivek Arte, Daira-Emma Hopwood, and Jack Grigg, created 2022-05-01 and discussed in zcash/zips issue #618 and PR #680 (zip-0226.rst:1–18, zip-0227.rst:1–18); deployment status is stated in the scope section. Three reference gaps qualify the bin. First, ZIP 227’s Test Vectors and Reference Implementation sections read “LINK TBD” and

its Deployment section “TBD” (zip-0227.rst:661–675); the de-facto assertion-checked vectors live in the fork: file `test_vectors/asset_base.rs`, generated from the `zcash-hackworks` test-vectors script `orchard_zsa_asset_base.py`, is replayed by a test that recomputes the full chain from (ik_encoding, asset_desc) to `asset_base(src/note/asset_base.rs:236–261 @ cf801a5d)`, with companion vectors for the signature scheme and the ZIP-32 derivation in `issuance_auth_sig.rs` and `zip32.rs`. The vectors’ fixed [u8; 512] description arrays are a relic of the dropped 512-byte cap and imply no bound. Second, the corner cases flagged above remain: the identity-point behaviour is *open* (Remark 2.8), the UTF-8 constraint is a SHOULD with a stricter implementation (§2.2), and key rotation is an open design area (Remark 2.5). Third, upstream `zcash/zips @ 69610984` is the text of record; the QEDit `zips` fork (`zips-zsa @ fd71419a`) is essentially identical for the asset-identity material, and the differences — upstream’s later additions of a ZIP-2005 dependency, a nullifier ψ^{nf} derivation, and a lead-byte rule — do not touch this chain.

3 Issuance and finalization (ZIP-227)

The preceding section moved Custom Assets between shielded notes; this one brings them into existence. ZIP-227 (Status Draft, Category Consensus; owners Pablo Kogan and Vivek Arte of QEDit, Daira-Emma Hopwood, and Jack Grigg) specifies the issuance mechanism for Custom Assets on the Zcash chain: a per-transaction *issuance bundle* in which a publicly named issuer creates new notes, a per-Asset *finalize* switch that ends issuance forever, and a node-maintained global supply map. The draft’s Test Vectors, Reference Implementation, and Deployment sections all read TBD (ZIP-227, header and closing sections @ `zips 6961098`). Deployment standing — Draft ZIPs, externally audited at circuit scope, running on QED-it’s public testnet, contested for NU7 — is fixed once in §1 and not relitigated here.

Issuance is deliberately *not* shielded. The ZIP’s Motivation states: “This ZIP only enables *transparent* issuance. As a first step, transparency will allow for proper testing of the applications that will be most used in the Zcash ecosystem, and will enable the supply of Assets to be tracked” (ZIP-227, *Motivation*). The issuer, the Asset, the issued amounts, and the finalization status are all consensus-visible; only the recipients of the issued notes are shielded, because the notes themselves enter the Orchard note commitment tree (§3.3).

Encoding ground truth. One sourcing decision governs every wire format and digest below, and we state it once. ZIP-227 normatively cites ZIP-230 for all issuance encodings and ZIP-246 for all issuance digests, yet upstream both are Withdrawn: ZIP-230 is the “Withdrawn Version 6 Transaction Format”, obsoleted by ZIP-229, its lead byte replaced by the placeholder `{{LEADBYTE}}`, and ZIP-246 carries “Digests for the Withdrawn Version 6 Transaction Format”, with v6 digests now assigned to ZIP-229 (ZIP-230 and ZIP-246, headers and warnings @ `zips 6961098`) — and ZIP-229 carries no issuance-bundle fields. The QEDit fork of the ZIPs repository retains ZIP-230 and ZIP-246 at Status Draft with lead byte 0x03, and its copy of ZIP-227 differs from upstream only cosmetically (`zips-zsa @ fd71419`). We therefore present the ZIP-230/246 issuance encoding as what it is: the fork-specified design, deployed on QED-it’s public testnet (ZecHub Vol. 61, 2025-12-17; §1.2) and implemented by the pinned fork libraries (the circuit layer audited by Least Authority, 2025-01-30), with no merged upstream v6 specification behind it. The NU7 standing of the whole format stack is part of the deployment picture of §1.

Implementation pins. Claims about deployed behavior cite the QEDit libraries at fixed commits: `orchard-zsa` branch `zsa1 @ cf801a5d` (2026-06-29), `librustzcash-zsa` branch `zsa1 @ 3f9b53fc` (2026-04-17), and the patched `zcash_spec` (QED-it fork @ `d5e8426`) that carries the new PRF domain byte; upstream `orchard`’s branch `1tf/zsa` is context only, not ground truth (§1.3, *Non-sources*). All `librustzcash-zsa` transaction-format code sits behind `cfg(zcash_unstable = "nu7")`; file paths we cite within that repository are relative to `zcash_primitives/src/`. Except where flagged inline,

everything in this section is *specified*: ZIP text at rigor, mirrored by these libraries. Three flags recur: fork-only specification (the wire format above, and fees, §3.9); library-versus-ZIP divergence, noted where it occurs; and node-side enforcement of global state, which no local repository exhibits (§3.7).

3.1 Issuance keys and the issuer identifier

Issuance authority is a new capability, deliberately disjoint from spend authority. The rationale in the ZIP: the issuance key tree is independent of the spending key tree but derived analogously (via ZIP 32), so an organization can delegate issuance without exposing funds, and a future scheme can replace the issuance leaf with a multisignature without touching spending keys (ZIP-227, *Rationale*, and *Issuance Key Derivation*).

The key material itself is constructed once, in §2.1: the BIP-340 signature scheme (Definition 2.1), the hardened-only ZIP-32 derivation of isk in the Issuance context (Construction 2.2), and the validating key with the issuer identifier, $ik := PubKey(isk)$ and $issuer = ik_encoding = [0x00] \parallel ik$ (Definition 2.4). We restate only the parameters the issuance layer consumes: seed S of 32 to 252 bytes, $Issuance.MKGDomain = ZcashSA_Issue_V1$, $Issuance.CKDDomain = 0x81$, hardened path $m_{Issuance} / 227' / coin_type' / 0'$, and $isk := sk$ at the leaf (ZIP-227, *Issuance Key Derivation*). No rotation exists today: the on-chain issuer name is the validating key itself, and the ZIP notes the equality might not hold in future if key rotation is ever specified (Remark 2.5; §3.8).

3.2 Asset identity: description, digest, and Asset Base

Every Custom Asset is named by its issuer plus a human-meaningful description, and consumed by the protocol as a Pallas point. The full derivation chain — `assetDescHash`, `AssetId`, `EncodeAssetId` with its version byte, `AssetDigest`, and `AssetBase = ZSAValueBase(AssetDigest)` — is constructed once, in §2.2 and §2.3 (Definition 2.6); the hash-not-string rationale is Remark 2.7, and the UTF-8 SHOULD-versus-panic delta between ZIP and implementation is recorded in §2.2. The native Asset bypasses the chain entirely: its base is the Orchard value-commitment base $V^{Orchard}$ (§2.4). An Asset Base equal to the Pallas identity is invalid everywhere — `AssetBase::custom` panics after hashing to the curve and `AssetBase::from_bytes` rejects the identity on parse (Remark 2.8). What the issuance layer adds to the chain is consensus-side recomputation: validators re-derive every Issue Note’s `AssetBase` from the bundle’s issuer and the action’s `assetDescHash`, never reading it off the wire (§2.5; consensus rules, §3.7).

3.3 Issue notes and the derived ρ

Definition 3.1 (Issue note). An *Issue Note* is a tuple $(d, pk_d, v, AssetBase, \rho, rseed)$,

$$Note^{Issue} := \{0, 1\}^{\ell_d} \times KA^{Orchard}.Public \times \{0 \dots 2^{\ell_{value}} - 1\} \times \mathcal{E}(\mathbb{F}_{p_{Pallas}})^* \times \mathbb{F}_{p_{Pallas}} \times \{0, 1\}^{8 \cdot 32},$$

a diversifier and diversified transmission key as in Orchard, a value v with $\ell_{value} = 64$, a non-identity Asset Base, the nullifier-input field element ρ , and a 32-byte `rseed` that MUST be sampled uniformly at random by the issuer (ZIP-227, *Issue Note*; the ZIP writes the fourth and fifth components $\mathbb{P}^*$ and $\mathbb{F}_{q_{\mathbb{P}}}$, our $\mathcal{E}(\mathbb{F}_{p_{Pallas}})^*$ and $\mathbb{F}_{p_{Pallas}}$).

Issue notes never appear on chain as commitments: the wire format carries the note fields themselves (§3.4), and each validator computes the note commitment — by the OrchardZSA `NoteCommit` of §4 — and appends it to the Orchard note commitment tree. Precisely because the nullifier key of the recipient is unknown to validators, the note’s future spend cannot be linked to the issuance transaction (ZIP-227, *Issue Note* and *Issuance Protocol*). Issuance is thus transparent at creation and shielded from the first spend onward.

Computation of ρ . An Orchard note takes $\rho := \text{nf}_{\text{old}}$ from the Action that creates it (the *Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”); an issue note has no such Action, so ZIP-227 derives ρ from the transaction’s OrchardZSA transfer bundle instead:

$$\begin{aligned} \text{DeriveIssuedRho}(\text{nf}, i_A, i_N) &:= \text{ToBase}(\text{PRFexpand}_{\text{nf}}(0x84 \parallel \text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N))), \\ \rho &:= \text{DeriveIssuedRho}(\text{nf}_{0,0}, \text{index}_{\text{Action}}, \text{index}_{\text{Note}}), \end{aligned}$$

with the PRF keyed by nf as 32 little-endian bytes (I2LEOSP_{256}), where $\text{nf}_{0,0}$ is the nullifier of the input note in the *first* Action of the *first* Action Group of the OrchardZSA bundle (the v6 format partitions a bundle’s Actions into one or more Action Groups, each carrying its own anchor and proof; §6.2), and the two indices are the zero-based positions of the Issue Action within the issuance bundle and of the note within its action (ZIP-227, *Computation of ρ*). The domain-separator byte 0x84 extends the PRFexpand lead-byte registry (the *Wallet Guide*, §“The PRFexpand lead-byte registry”); the patched `zcash_spec` carries it as `ORCHARD_DERIVED_ISSUE_RHO` (QED-it `zcash_spec`, `src/prf_expand.rs` line 107 @ d5e8426). The implementation computes the identical expression — PRFexpand keyed by the 32-byte little-endian nullifier over 0x84 followed by two 4-byte little-endian indices, reduced by `ToBase` (`orchard-zsa`, `src/note.rs` lines 479–489 @ cf801a5d). This derivation is why an issuance bundle cannot travel alone: requiring at least one Action Group both supplies $\text{nf}_{0,0}$ and blocks replay — a transaction containing only an issuance bundle would have a SIGHASH computed solely from that bundle, so a duplicated bundle would share the SIGHASH (ZIP-227, *Rationale*).

In the builder, notes are created with ρ unset, via `new_issue_note`; once the transfer bundle’s first nullifier is known, `update_rho_for_issuance_note` sets ρ and resamples `rseed` in a loop until the note commitment is not $\perp^7$ (`orchard-zsa`, `src/note.rs` lines 457–489 @ cf801a5d).

Reference notes. The first time an Asset is issued, the issuer **MUST** place a *reference note* as the first note of that Issue Action: a note of value $v = 0$ whose recipient (d, pk_d) is the default diversified payment address (diversifier index 0) derived from the *all-zero* Orchard spending key (ZIP-227, *Reference Notes*). The implementation hard-codes the resulting 43-byte raw address as `RAW_REFERENCE_RECIPIENT`, byte-identical to the array in the ZIP, with `ReferenceKeys::sk() = SpendingKey::from_bytes([0; 32])`; the predicate `is_reference_note` requires zero value and exactly this recipient, `get_reference_note` accepts only the *first* note of an action, and the builder inserts the reference note itself when told the issuance is a first issuance — adding a recipient for an Asset that already has notes fails with `CannotBeFirstIssuance` (`orchard-zsa`, `src/constants/reference_keys.rs` lines 14–33 and `src/issuance.rs` lines 251–258, 459–507 @ cf801a5d). The reference note is retained in global state as `note_ref` (§3.7).

The ZIP mandates reference notes and stores them in consensus state, but states no rationale for their existence. The evident function — that every issued Asset Base is guaranteed at least one note in the commitment tree, available as a ZIP-226 split-input source for a first spend — is our reading, stated nowhere in the ZIP text or its rationale sections; we classify the mechanism itself as *specified* and this purpose as *designed but unspecified*.

3.4 The issuance bundle and its encoding

An *Issuance Bundle* carries the issuer identifier, the list of Issue Actions, and one authorizing signature: `issuer`, which lets validators verify that each `AssetId` is properly associated with this issuer; `vIssueActions`, an array of Issue Actions; and `issueAuthSig`, the encoding of a signature over the transaction SIGHASH by the issuance authorizing key `isk` (ZIP-227, *Issuance Bundle*). An *Issuance Action*

⁷Strictly, this conditions `rseed` on a negligible-probability event, a technical deviation from the ZIP’s “sampled uniformly at random” (ZIP-227, *Issue Note*).

groups the notes issued for one Asset: the 32-byte `assetDescHash`, the note list, and a flags byte carrying `finalize` (ZIP-227, *Issuance Action*). Table 3 gives the v6 serialization; the encoding rigor caveat of the section introduction applies.

Field	Type	Notes
<code>issuerLength</code>	<code>compactSize</code>	MUST be 0 if <code>nIssueActions</code> = 0
<code>issuer</code>	<code>byte[issuerLength]</code>	= <code>ik_encoding</code> , 33 bytes when present
<code>nIssueActions</code>	<code>compactSize</code>	
<code>vIssueActions</code>	<code>IssueAction[nIssueActions]</code>	
<code>issueAuthSig</code>	<code>IssueAuthSignature</code>	present iff <code>nIssueActions</code> > 0
<i>Per IssueAction:</i>		
<code>assetDescHash</code>	<code>byte[32]</code>	
<code>nNotes</code>	<code>compactSize</code>	may be 0 (finalize-only)
<code>vNotes</code>	<code>IssueNoteDescription[nNotes]</code>	115 bytes each
<code>flagsIssuance</code>	<code>byte</code>	finalize at LSB; other bits MUST be 0
<i>Per IssueNoteDescription (43 + 8 + 32 + 32 = 115 bytes):</i>		
<code>recipient</code>	<code>byte[43]</code>	$\text{LEBS2OSP}_{88}(d) \parallel \text{pk}_d^*$
<code>value</code>	<code>uint64</code>	little-endian
<code>rho</code>	<code>byte[32]</code>	as for Orchard notes
<code>rseed</code>	<code>byte[32]</code>	as for Orchard notes

Table 3: ZSA issuance fields of the v6 transaction format (fork ZIP-230, *Transaction Format, Issuance Action Description*, and *Issue Note Description* @ zips-zsa fd71419; upstream text identical apart from the Withdrawn status).

Three encoding facts deserve emphasis. First, the `recipient` field is exactly an Orchard raw payment address (the *Wallet Guide*, §“Raw encodings”), and together with the bundle’s `issuer` and the action’s `assetDescHash`, the four fields of an `IssueNoteDescription` determine the Issue Note completely (ZIP-230, *Issue Note Description*; ZIP-227, *Issue Note*) — the note commitment is deliberately absent, since validators recompute it (§3.3). Second, an action with `nNotes` = 0 is legal: it lets an issuer finalize an Asset without issuing more of it (ZIP-230, *Issuance Action Description*; finalization semantics in §3.8). Third, the flags byte is strict: `IssuanceFlags::from_byte` returns `None` whenever any bit other than the LSB is set, and deserialization fails on such bytes, enforcing ZIP-230’s “remaining bits are set to 0” at parse time (orchard-zsa, `src/issuance.rs` lines 57–110; `librustzcash-zsa`, `transaction/components/issuance.rs` lines 89–91).

The `librustzcash-zsa` serializer pins down the absent-bundle corner cases: `read_bundle` reads `issuer` as a length-prefixed vector and returns `None` only for an empty issuer with zero actions, erroring on an empty issuer with non-empty actions and on a non-empty issuer with zero actions; `write_bundle` encodes an absent bundle as two zero `compactSizes`. The authorization serializes as a length-prefixed `sighashInfo` followed by the length-prefixed signature encoding, and the parser accepts exactly `[0x00]` as `sighashInfo` (`transaction/components/issuance.rs` lines 20–191 and `sighash_versioning.rs` @ 3f9b53fc; the meaning of `[0x00]` is §3.5).

In the orchard-zsa object model, the type `IssueBundle<T: IssueAuth>` holds the validating key `ik` (an `IssueValidatingKey<ZSASchnorr>`), a non-empty vector of actions, and an authorization typestate `T`; an `IssueAction` holds the 32-byte `asset_desc_hash`, its `Vec<Note>`, and its `IssuanceFlags` (`src/issuance.rs` lines 47–124 @ cf801a5d).

3.5 Authorization: BIP-340 over the transaction sighash

The issuance authorization signature scheme `IssueAuthSig` is instantiated as a BIP-340 Schnorr signature over `secp256k1` with the standard parameters — not `RedPallas`: issuance is a transparent capability, and the choice buys direct compatibility with existing `secp256k1` tooling. Batch verification MAY be used, and precomputation MAY be used if and only if it produces equivalent results (ZIP-227,

Issuance Authorization Signature Scheme). The associated types (same section):

$$\text{Message} = \mathbb{B}^{\mathbb{N}}, \quad \text{Signature} = \mathbb{B}^{\mathbb{V}[64]} \cup \{\perp\}, \quad \text{Public} = \mathbb{B}^{\mathbb{V}[32]} \cup \{\perp\}, \quad \text{Private} = \mathbb{B}^{\mathbb{V}[32]},$$

that is, arbitrary byte-sequence messages, 64-byte signatures, 32-byte x-only public keys, and 32-byte secret keys, with $\perp$ as the failure value. Signing fixes the BIP-340 auxiliary data to the all-zero block:

$$a := [0x00]^{32}, \quad \sigma := \text{Sign}(\text{isk}, M) \text{ with auxiliary data } a, \quad \text{issueAuthSig} := [0x00] \parallel \sigma,$$

returning $\perp$ on failure; validation returns 0 if $\sigma = \perp$, else 1 exactly when the BIP-340 *Verify*(ik, *M*, σ) succeeds⁸ (ZIP-227, *Issuance Authorization Signing and Validation*). With the scheme byte, the two wire encodings are 33 bytes (*ik_encoding*) and 65 bytes (*issueAuthSig*).

What is signed. The message is the transaction SigHash as defined by protocol specification §4.10 under the ZIP-226 modifications, with sighash type SIGHASH_ALL, and the consensus rule requires *IssueAuthSig.Validate*(ik, SigHash, *issueAuthSig*) = 1 (ZIP-227, *Specification: Consensus Rule Changes*). Under ZIP-246 the signature digest includes branch S.5, the *issuance_digest*, identical to the txid branch — so the issuance signature commits to *all* effecting data of the transaction, including the issuance bundle’s own effecting data, though never to any signature (ZIP-246, *Signature Digest*). ZIP-246 also introduces sighash algorithm versioning: each bundle carries *sighashInfo* = [*sighashVersion*] || *associatedData*, versions are numbered from 0 per transaction version, version 0 is by convention the “commit to all effecting data” algorithm, and for v6 only version 0 is defined, with empty associated data for the Transparent, Sapling, Orchard, and Issuance bundles — hence the constant [0x00] the parser demands (ZIP-246, *Sighash versioning*).

Implementation. The ZSASchnorr scheme wraps the secp256k1 crate: signing builds a keypair from isk and calls *sign_schnorr_with_aux_rand*(msg, keypair, &[0u8; 32]) — the ZIP’s $a = [0x00]^{32}$ — ik is the x-only public key, and *encode*() of key and signature each prepend ALGORITHM_BYTE = 0x00, with *decode*() rejecting any other lead byte (orchard-zsa, src/issuance/auth.rs lines 163–303 @ cf801a5d). Bundle construction is a tpestate chain — *AwaitingNullifier* → (*update_rho*) → *AwaitingSighash* → (*prepare(sighash)*) → *Prepared* → (*sign(isk)*) → *Signed* — so a bundle cannot be signed before its notes’ ρ values and the sighash exist (src/issuance.rs lines 261–304 and 404–599 @ cf801a5d). On the transaction side, *v6_signature_hash* includes the txid *issuance_digest*, and the builder signs the issue bundle with *prepare(*shielded_sig_commitment)* where that commitment is *signature_hash*(tx, *SignableInput::Shielded*, txid_parts) — the same SIGHASH_ALL shielded digest the Sapling and Orchard signatures sign (librustzcash-zsa, transaction/sighash_v6.rs lines 10–51 and transaction/builder.rs lines 1416–1417, 1453–1460 @ 3f9b53fc). The builder also realizes the $\text{nf}_{0,0}$ rule constructively: *first_nullifier* returns the first Action’s nullifier only for an OrchardZSA bundle, and building with an issuance builder but no such bundle fails with *BundleTypeNotSatisfiable* (transaction/builder.rs lines 1356–1366 and 1594–1602 @ 3f9b53fc).

A divergence to flag. ZIP-246’s txid and signature digests include a sixth branch, T.6/S.6 *memo_digest* (ZIP-246, *Txid Digest* and *Signature Digest*), but librustzcash-zsa’s *to_hash* hashes only the header, transparent, Sapling, Orchard, and issuance branches — the fork does not implement ZIP-231 memo bundles (transaction/txid.rs lines 415–460 @ 3f9b53fc). The digest the fork code signs is therefore not byte-identical to the digest ZIP-246 specifies; whichever is corrected, the issuance signature’s coverage claim above should be read against the fork’s five-branch digest today.

⁸The ZIP’s *Validate* definition writes *Verify*(key, *M*, σ) for the parameter it names ik — an editorial slip we flag when citing (ZIP-227, *Issuance Authorization Signing and Validation*).

3.6 Transaction digests

The v6 digest tree extends the ZIP-244 structure the *Ironwood Guide* presents in §“Conservation of value: value commitments and the binding signature”: the txid digest is BLAKE2b-256 with personalization `ZcashTxHash_ || CONSENSUS_BRANCH_ID` over six branches T.1–T.6 (header, transparent, Sapling, Orchard, issuance, memo), and the signature digest has the same six-branch structure with S.5 identical to T.5; a signature digest is produced for each transparent input, Sapling input, OrchardZSA Action, and additionally for each Issuance Action (ZIP-246, *Txid Digest* and *Signature Digest*). The issuance branches are three nested hashes and one authorizing hash, each over the wire encodings of Table 3 (ZIP-246, *txid_digest* T.5 and A.4: *issuance_auth_digest*):⁹

```
issuance_digest = BLAKE2b-256(ZTxIdSAIssueHash,
                               issuerLength || issuer || issue_actions_digest),
issue_actions_digest = BLAKE2b-256(ZTxIdIssuActHash,
                                   ||actions assetDescHash || issue_notes_digest || flagsIssuance),
issue_notes_digest = BLAKE2b-256(ZTxIdIacNoteHash,
                                   ||notes recipient || value || rho || rseed),
issuance_auth_digest = BLAKE2b-256(ZTxAuthZSA0rHash, issueAuthSig field encoding),
```

where the A.4 preimage is the ZIP-230 IssueAuthSignature composite

```
sizeSighashInfo || sighashInfo || sizeSignature || signature
```

(ZIP-230, *Issuance Authorization Signature*), and a transaction without issuance components hashes the empty string under the same personalizations.

The code computes exactly these. On the txid side, `hash_issue_bundle_txid_data` hashes

```
compactSize(|ik_encoding|) || ik_encoding || issue_actions_digest
```

under `ZTxIdSAIssueHash`; the actions digest hashes, per action, the 32-byte hash, the notes digest, and the flags byte; the notes digest hashes, per note, the 43-byte recipient, the 8-byte little-endian value, and the two 32-byte fields. On the authorizing side, `hash_issue_bundle_auth_data` hashes

```
compactSize(|sighashInfo|) || sighashInfo || compactSize(65) || [0x00] || σ,
```

asserting the 65-byte length and the 0x00 lead. The empty-bundle variants hash the empty string, and all four personalizations match ZIP-246; both digests live in `src/bundle/commitments/issuance.rs` lines 11–86 (orchard-zsa @ cf801a5d). For the wiring into transaction identifiers, `TxidDigester::digest_issue` maps the bundle to its `commitment()`; `to_hash` appends the digest only when the version has `orchard_zsa()`; and `BlockTxCommitmentDigester` takes the authorizing commitment (`librustzcash-zsa`, `transaction/txid.rs` lines 381–383, 445–452, 595–598 @ 3f9b53fc).

Deterministic regression digests are pinned in the tests — for a seeded two-asset bundle,

```
issuance_digest = ee70e3b61674fd0428ac0020cc4fc5819386e39c4eb3c63357c84c998195bcdb,
issuance_auth_digest = 6df77af7b5323d99376336b770e4c5b06ffc195de81ac7692d1b08b6eb19534d
```

— and cross-implementation test vectors ship in `src/test_vectors/` (`asset_base.rs`, `issuance_auth_sig.rs`), exercised by the asset-base and authorization tests (orchard-zsa, `src/bundle/commitments/issuance.rs` lines 156–190 and `src/test_vectors/` @ cf801a5d). The upstream ZIP-227 Test Vectors section, by contrast, still reads LINK TBD.

⁹Two editorial defects in ZIP-246 to note when citing: the T.5 child list labels `issuerLength/issuer/issue_actions_digest` as T.5a/T.5b/T.5c, while the following section headings call `issue_actions_digest` “T.5a” and `issue_notes_digest` “T.5ai”; and the A.4 empty case reads “In the case that the transaction has no Orchard Actions” where “no Issuance Actions” is evidently intended.

3.7 Global issuance state and the consensus rules

Supply integrity is the one property issuance cannot delegate to cryptography: no transaction field simultaneously witnesses everything issued and everything burnt for an Asset, so — along the lines of the ZIP-209 pool turnstile — every node maintains a per-Asset circulation record (ZIP-227, *Rationale for Global Issuance State*).

Definition 3.2 (Global issuance state). The map

$$\begin{aligned} \text{issued_assets} : \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^* &\rightarrow \{0 \dots \text{MAX_ISSUE}\} \times \{0, 1\} \times \text{Note}^{\text{Issue}}, \\ \text{AssetBase} &\mapsto (\text{balance}, \text{final}, \text{note}_{\text{ref}}), \end{aligned}$$

records, for every issued Asset, the amount in circulation (issued minus burnt), the finalization status — which can never change from 1 to 0 — and the Asset’s reference note. A missing entry ($\text{issued_assets}(\text{AssetBase}) = \perp$) is read as $\text{balance} = 0$, $\text{final} = 0$, $\text{note}_{\text{ref}} = \perp$; the constant $\text{MAX_ISSUE} = 2^{64} - 1$ caps the total supply of any Custom Asset (ZIP-227, *Global Issuance State*).

The map threads through the chain deterministically: it **MUST** be updated by a node during the processing of any transaction that contains burn information or an issuance bundle; the input state for the OrchardZSA activation block is the empty map; the input state of a block’s first transaction is the final state of the preceding block; each subsequent transaction’s input state is the preceding transaction’s output state; and the block’s final state is its last transaction’s output state (ZIP-227, *Management of the Global Issuance State*).

Per-transaction rules. For a v6 transaction (ZIP-227, *Specification: Consensus Rule Changes*):

- (T1) `nActionGroupsOrchard` **MUST** be 0 or 1, and `nAGExpiryHeight` **MUST** be 0; the v6 parser of `librustzcash-zsa` enforces both at deserialization, in `transaction/components/orchard.rs` lines 107–132 @ 3f9b53fc;
- (T2) the output state $\text{issued_assets}_{\text{OUT}}$ **MUST** be initialized to the input state $\text{issued_assets}_{\text{IN}}$;
- (T3) the `assetBurn` set **MUST** satisfy the ZIP-226 rules (§4: tuples $(\text{AssetBase}, v)$ with the Native Asset excluded, $0 < v \leq \text{MAX_BURN_VALUE} = 2^{63} - 1$, and no duplicate `AssetBase`; ZIP-226, burn rules); and
- (T4) for every $(\text{AssetBase}, v) \in \text{assetBurn}$, the node **MUST** check $\text{balance} \geq v$ for the entry of `AssetBase` in $\text{issued_assets}_{\text{OUT}}$ and subtract v from that balance, *prior to* processing the issuance bundle.

Issuance-bundle rules. For a transaction carrying an issuance bundle (ZIP-227, *Specification: Consensus Rule Changes*):

- (I1) the transaction **MUST** also contain at least one OrchardZSA Action Group (the p /replay rationale of §3.3);
- (I2) the encoding `ik_encoding` **MUST** be taken from the `issuer` field and **MUST** start with the byte 0x00 (BIP-340);
- (I3) the signature **MUST** satisfy `IssueAuthSig.Validate(ik, SigHash, issueAuthSig) = 1`;
- (I4) for each Issue Action: every `IssueNoteDescription` **MUST** be a valid ZIP-230 encoding; the `AssetBase` is derived from `assetDescHash` and `issuer` (§3.2); and the Asset’s final bit in $\text{issued_assets}_{\text{OUT}}$ **MUST NOT** be 1;

- (I5) if $\text{note}_{\text{ref}} = \perp$, the first Issue Note MUST be a reference note (§3.3), and the node MUST set note_{ref} to it;
- (I6) for every Issue Note note_j : its ρ MUST equal the value computed per §3.3; the supply update
- $$\text{issued_assets}_{\text{OUT}}.\text{balance} + v_j \leq \text{MAX_ISSUE}, \quad \text{issued_assets}_{\text{OUT}}.\text{balance} += v_j$$
- MUST hold and be applied; and the node MUST compute the note commitment per ZIP-226;
- (I7) if $\text{finalize} = 1$ in $\text{flags}_{\text{Issuance}}$, the node MUST set $\text{final} = 1$.

The commitments enter the tree in a fixed order: for each Action Group of the OrchardZSA bundle, the commitment of every new note per Action; then, for each Issue Action of the issuance bundle, the commitment of every Issue Note (ZIP-227, *Addition to the Note Commitment Tree*).

The rationale ties the pieces together: balances of issued Assets must never go negative, MAX_ISSUE is a practical limit that also lets an issuer create an Asset’s complete supply in a single transaction, and the same map records finalization so that further issuance of a finalized Asset is rejected (ZIP-227, *Rationale for Global Issuance State*). Beyond consensus, the ZIP RECOMMENDS that full validators keep a mutable `issuanceSupplyInfoMap` from `AssetId` to `(totalSupply, finalize)` for wallets and applications tracking total supply (ZIP-227, *Implementing Zcash Nodes*).

The library realization. Function `verify_issue_bundle` takes the bundle, the sighash, a caller-supplied view of global state (`get_global_records`), and the first nullifier. It rejects any sighash kind other than `AllEffecting`, verifies the BIP-340 signature over the 32-byte sighash with the bundle’s `ik`, folds the actions over a `BTreeMap<AssetBase, AssetRecord>`, and returns the updated records — `amount`, `is_finalized`, `reference_note` — for the caller to persist (orchard-zsa, `src/issuance.rs` lines 647–802 @ cf801a5d). The fold realizes the per-action and per-note rules as typed errors:

- `IncorrectRhoDerivation` — a note’s ρ differs from the recomputed derivation of §3.3;
- `MissingReferenceNoteOnFirstIssuance` — no record exists for the Asset, and the action’s first note is not a reference note;
- `IssueActionPreviouslyFinalizedAssetBase` — reissuance against a finalized record;
- `IssueBundleIkMismatchAssetBase` and `AssetBaseCannotBeIdentityPoint` — a note’s Asset Base differs from the one `IssueAction::verify` derives from `(ik, assetDescHash)`, or the derived point is the Pallas identity;
- `ValueOverflow` — the checked u64 value sum overflows, the code’s realization of $\text{MAX_ISSUE} = 2^{64} - 1$ (`src/issuance.rs` lines 216–244; `src/value.rs` lines 143–145).

A signature-skipping variant, `check_issue_bundle_without_sighash`, exists for verifying historical blocks from a trusted checkpoint (`src/issuance.rs` lines 647–705 @ cf801a5d).

Three boundary observations, each load-bearing for anyone re-deriving consensus from these libraries:

- *Empty-action divergence.* The implementation rejects an action with no notes unless it is finalized (`IssueActionWithoutNoteNotFinalized`, `src/issuance.rs` lines 216–219), but ZIP-227’s consensus-rule list contains no such MUST, and ZIP-230 merely notes that `nNotes` may be zero “to only finalize”. A ZIP-conformant validator would accept an empty, non-finalizing action that the reference implementation rejects.
- *First-issuance detection.* The ZIP keys the reference-note requirement on $\text{note}_{\text{ref}} = \perp$; the code keys it on the complete absence of an `AssetRecord` (`get_global_records` returning `None`). The two are equivalent under the invariant that note_{ref} is set exactly on first issuance — rule (I5) — and no rule ever clears it, so an entry exists if and only if its note_{ref} is set.

- *Node-side enforcement locus.* Neither `orchard-zsa` nor `librustzcash-zsa` contains the `issued_assets` chain-state maintenance or the consensus caller of `verify_issue_bundle`: the libraries return updated records for a node to persist, and the QEDit node integration is not among our local ground-truth repositories. The global-state rules are therefore *specified and library-supported*, with their node enforcement unverified locally.

3.8 Finalization

The `finalize` boolean signals that no further issuance of the specific Custom Asset is possible: every issuance action carries it in `flagsIssuance` (a requirement of the ZIP’s Requirements section), setting it sets the Asset’s final bit — rule (I7) — and transactions attempting to issue further amounts of a previously finalized Asset are rejected — rule (I4) (ZIP-227, *Requirements and Issuance Action*). The bit is monotone: consensus state never changes final from 1 to 0 (Definition 3.2).

Finalization is the ZIP’s whole lifecycle policy, and its Concrete Applications read as patterns over one bit (ZIP-227, *Concrete Applications*):

- *One-time issuance:* set `finalize = 1` on the first action; the entire supply exists from one transaction — `MAX_ISSUE` is sized to permit this (§3.7).
- *Stopping supply:* issue with `finalize = 1` in a final issuance transaction, or in one containing a single note of value 0 (the wire format admits note-less finalizing actions as well, Table 3; the library insists on the finalizing variant, §3.7).
- *NFTs:* issue multiple actions in one transaction, each with `value = 1` at the Asset’s fundamental unit and `finalize = 1` — each action’s Asset is thereafter unique and non-replicable.

Finalization is also the entirety of the compromise story. No key rotation exists for issuance keys; on compromise of `isk`, the ZIP’s guidance is to set `finalize = 1` for all Assets issued with that key, switch to a new `isk` — hence a new issuer identifier, since `issuer = ik_encoding` — and issue new Assets with new `AssetIds` (ZIP-227, *Issuance Key Compromise*). The old Assets keep circulating and burning; they simply never grow again.

3.9 Fees

The fee treatment of issuance is the sharpest fork/upstream split in the ZSA stack, and we bin it explicitly. The QEDit fork of ZIP-317 adds two contributions to the logical-action count of the conventional fee (the *Wallet Guide*, §“Fees (ZIP-317)”, for the surrounding machinery: *marginal_fee* = 5000 zatoshis, *grace_actions* = 2):

$$\begin{aligned} \text{contribution}_{\text{ZSAIssuance}} &= n\text{ZSAIssueNotes}, \\ \text{contribution}_{\text{ZSACreation}} &= \text{creation_cost} \cdot n\text{AssetCreations}, \end{aligned}$$

where `nZSAIssueNotes` counts `IssueNote` outputs, `nAssetCreations` counts Custom Assets newly added to the global issuance state, and `creation_cost` = 100 logical actions (fork ZIP-317, *Fee calculation* @ zips-zsa fd71419). The stated rationale: every new Asset adds a row that validators track in perpetuity (§3.7), and the two-orders-of-magnitude creation cost disincentivizes junk assets.

Upstream, these contributions no longer exist: ZIP-317’s change history records, on 2026-06-26, “Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)” (ZIP-317, change history @ zips 6961098) — while ZIP-227’s own *Changes to ZIP 317* section still claims the conventional fee accounts for both. The formulas above survive only in the QEDit fork and its libraries: *fork-specified*, implemented in `librustzcash-zsa` behind `cfg(zcash_unstable = "nu7")` (`transaction/fees/zip317.rs`, `CREATION_COST = 100` @ 3f9b53fc), and awaiting an upstream resolution that the NU7 process (§1) has yet to provide.

4 Transfer and burn (ZIP-226)

ZIP-226 (*Transfer and Burn of Zcash Shielded Assets*; Status: Draft; owners Pablo Kogan and Vivek Arte of QEDit, Daira-Emma Hopwood, and Jack Grigg) defines, jointly with the issuance protocol of ZIP-227 (§3), the OrchardZSA transfer and burn protocol. We develop it as four deltas against the Orchard baseline, each citing the construction it modifies in the *Ironwood Guide* rather than re-deriving it: the note gains one field, the *Asset Base* (§4.1); the note commitment becomes a case split that keeps ZEC commitments identical to NU5 (§4.2); the value commitment trades its fixed value base for the per-asset base, which turns the binding signature into a per-asset balance check and gives burn its consensus meaning (§4.3, §4.4); and dummy spends give way to *Split Inputs* for Custom Assets (§5). We then state the extended Action statement (§4.6) and the carrier format with its digests and fees (§4.7).

Ground truth for this section: upstream zips at commit 69610984, the QEDit fork zips-zsa at fd71419a, crate orchard-zsa at cf801a5d (branch zsa1), and librustzcash-zsa at 3f9b53fc (branch zsa1); we verified every cited artifact at these commits. ZIP-226 names the QED-it zcash-test-vectors and the QED-it forks of zcashd, orchard, librustzcash, and halo2 as its test vectors and reference implementations. Deployment standing is fixed once, in §1, and not relitigated per object; two upstream facts nonetheless shape the text. First, ZIP-226 states that ZSA “is scheduled to be deployed in Network Upgrade 7 (NU7)” and “currently assumes that this will be the case”, while the transaction format it rides on has been withdrawn upstream (§4.7). Second, upstream ZIP-226 is now defined relative to the protocol with ZIP-2005 (Ironwood Quantum Recoverability) applied, which is expected to deploy before any ZSA activation; the QEDit fork’s ZIP-226 lacks this clause, and the pinned implementation does not implement ZIP-2005 constructs. Where the difference is material — the split-note ψ^{nf} (§5) and the lead byte (§4.7) — we flag it in place.

4.1 One new field: the OrchardZSA note

Definition 4.1 (OrchardZSA note). An OrchardZSA note extends the Orchard note — the tuple $(\text{addr}, v, \rho, \psi, \text{rcm})$ of the *Ironwood Guide*, “Representation of a note: notes and note commitments” — by exactly one field:

$$n := (\text{addr}, v, \text{AssetBase}, \rho, \psi, \text{rcm}),$$

where $\text{AssetBase} \in \mathcal{E}^* := \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$ is “a valid group element that is not the identity and is not bottom” (ZIP-226; the ZIP writes $\mathbb{P}^*$ for $\mathcal{E}^*$). All other components are as in protocol specification §3.2. Formally,

$$\begin{aligned} \text{Note}^{\text{OrchardZSA}} &:= \{0, 1\}^{88} \times \text{KA}^{\text{Orchard}}.\text{Public} \times \{0, \dots, 2^{64} - 1\} \times \mathcal{E}^* \\ &\quad \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \text{NoteCommit}^{\text{Orchard}}.\text{Trapdoor}, \end{aligned}$$

where the ZIP’s $\mathbb{F}_{q_P}$ is this series’ $\mathbb{F}_{p_{\text{Pallas}}}$ and $\ell_{\text{value}} = 64$ (ZIP-226).

The byte encoding of the new field is the star-encoding of the series’ conventions: ZIP-226 sets $\text{asset_base} := \text{LEBS2OSP}_{\ell_P}(\text{repr}_{\mathbb{P}}(\text{AssetBase}))$, which is $\text{AssetBase}^* \in \{0, 1\}^{256}$ in 32 bytes.

An Asset Base is never a free choice of the transactor. Every Custom Asset’s base is derived by the ZIP-227 issuance chain — constructed in §3, reproduced here because the soundness argument of §5

depends on its shape:

```

assetDescHash := BLAKE2b-256("ZSA-AssetDescCRH", asset_desc),
AssetId := (issuer, assetDescHash),
EncodeAssetId(AssetId) := 0x00 || issuer || assetDescHash,
AssetDigest := BLAKE2b-512("ZSA-Asset-Digest", EncodeAssetId(AssetId)),
AssetBase := GroupHashPallas(z.cash : OrchardZSA, AssetDigest)

```

(ZIP-227, which names the last map ZSAValueBase). For the native Asset (ZEC/TAZ) the Asset Base is instead *defined* to be

$$V^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "v"),$$

the fixed value-commitment base of the *Ironwood Guide*, “Conservation of value: value commitments and the binding signature” (ZIP-226). This single identification is what keeps every ZEC computation below identical to NU5: ZIP-226 uses $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v)$ as $\text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(V^{\text{Orchard}}, v)$. The implementation realizes it as $\text{AssetBase}::\text{zatoshi}() = \text{hash_to_curve}(\text{"z.cash:Orchard-cv"})(b"v")$ (orchard-zsa, src/note/asset_base.rs, src/constants/fixed_bases.rs).

4.2 The note commitment: a case split

Definition 4.2 (OrchardZSA note commitment). For an OrchardZSA note,

$$\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(g_d^*, pk_d^*, v, \rho, \psi, \text{AssetBase}) := \begin{cases} \text{NoteCommit}_{\text{rcm}}^{\text{Orchard}}(g_d^*, pk_d^*, v, \rho, \psi) & \text{if } \text{AssetBase} = V^{\text{Orchard}}, \\ \text{cm}_{\text{ZSA}} & \text{otherwise,} \end{cases}$$

where

$$\text{cm}_{\text{ZSA}} := \text{Sinsemilla}_{\text{z.cash:ZSA-NoteCommit-M}}(g_d^* || pk_d^* || \text{LE}_{64}(v) || \text{LE}_{255}(\rho) || \text{LE}_{255}(\psi) || \text{AssetBase}^*) \dot{+} [\text{rcm}] \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-NoteCommit-r}, "")$$

(ZIP-226; the ZIP writes I2LEBSP for the series' LE_k). The signature is

$$\text{NoteCommit}^{\text{OrchardZSA}} : \text{NoteCommit}^{\text{Orchard}}.\text{Trapdoor} \times \{0, 1\}^{256} \times \{0, 1\}^{256} \times \{0, \dots, 2^{64} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathcal{E}^* \rightarrow \text{NoteCommit}^{\text{Orchard}}.\text{Output},$$

with trapdoor and output types those of $\text{NoteCommit}^{\text{Orchard}}$, unchanged (ZIP-226).

The two branches carry the design's two promises. In the native branch the Asset Base is not hashed at all: a ZEC note commitment under OrchardZSA is the NU5 commitment, bit for bit. In the ZSA branch the message appends AssetBase^* (256 bits) after ψ , against the baseline layout of the *Ironwood Guide*, “Representation of a note”:

$$g_d^* || pk_d^* || \text{LE}_{64}(v) || \text{LE}_{255}(\rho) || \text{LE}_{255}(\psi) \mapsto \dots || \text{LE}_{255}(\psi) || \text{AssetBase}^*,$$

under the fresh hash domain $\text{z.cash:ZSA-NoteCommit-M}$ but the *same* randomness base $\text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-NoteCommit-r}, "")$. ZIP-226's rationale: “the nested structure of the Sinsemilla-based commitment allows us to add the Asset Base as a final recursive step”, and the output “is still indistinguishable from the original Orchard ZEC note commitments, by definition of the Sinsemilla hash function”. Because the trapdoor and output types are unchanged, ZSA

commitments land in the same note commitment tree as ZEC commitments and are indistinguishable from them there (ZIP-226). One caveat bounds the claim: Orchard (v5) note commitments *are* distinguishable from OrchardZSA (v6) note commitments, because the two protocols use different transaction versions (ZIP-226).

Remark 4.3 (Implementation). Function `NoteCommitment::derive` computes *both* commitments. For ZEC it calls `CommitDomain::new` on the prefix `"z.cash:Orchard-NoteCommit"`; for ZSA it calls `new_with_separate_domains` on the *M*-prefix `"z.cash:ZSA-NoteCommit"` and the shared *r*-prefix `"z.cash:Orchard-NoteCommit"`. It then selects between them in constant time on `asset.is_zatoshi()` (`orchard-zsa, src/note/commitment.rs`). The *sinsemilla* crate appends the `-M` and `-r` suffixes to these prefixes (*sinsemilla* 0.1.0, `src/lib.rs`).

The nullifier of an ordinary (non-split) OrchardZSA note is generated exactly as in Orchard (protocol specification §4.16; *Ironwood Guide*, “Prevention of double-spending: nullifiers”); only Split Inputs use the modified, randomized formula of §5 (ZIP-226).

4.3 Per-asset value: the variable-base value commitment

Definition 4.4 (OrchardZSA net value commitment). Each Action commits to its net value change in its Asset:

$$\begin{aligned} cv^{\text{net}} &:= \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v^{\text{net}}) = [v^{\text{net}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}, \\ v^{\text{net}} &:= v_{\text{old}} - v_{\text{new}}, \quad R^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "r"), \end{aligned}$$

where `AssetBase` is the Action’s Asset Base and the randomness base R^{Orchard} is unchanged (ZIP-226).

Exactly one thing changes against the baseline $cv^{\text{net}} = [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ of the *Ironwood Guide*, “Conservation of value”: the fixed-base multiplication by V^{Orchard} becomes a variable-base multiplication by the Action’s `AssetBase`, while the randomness term keeps its fixed base (ZIP-226). For ZEC, $\text{AssetBase} = V^{\text{Orchard}}$ and the commitment reduces to the NU5 one (§4.1).

The prover still signs the SIGHASH with $\text{bsk} = \sum_{i=1}^n \text{rcv}_i$ over the n Actions of a transfer (ZIP-226); the verifier’s key acquires burn terms — one zero-trapdoor commitment subtracted per `assetBurn` entry, each a bare multiple of its base — displayed in this volume as (2) (§5.1). Why this checks balance *per asset*: “the committed values must sum to zero per Asset Base, as the Pedersen commitments add up homomorphically only with respect to the same value base point” (ZIP-226). Partitioning the Actions into a ZEC set and a Custom Asset set, the custom-asset commitments must cancel against the declared burns up to pure randomness terms — the cancellation equation is displayed in §5.2 — so $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$ holds if and only if, per Custom Asset, the sum of net Action values equals its `assetBurn` entry (or 0 if absent), and the ZEC component equals $v_{\text{balance}}^{\text{Orchard}}$ (ZIP-226). The binding-signature mechanics — bsk/bvk cancellation and *RedPallas* on the base R^{Orchard} — are those of the *Ironwood Guide*, “Conservation of value”, and we do not restate them.

Remark 4.5 (Implementation: derive_bvk). Function `derive_bvk_raw` computes the ZIP equation verbatim: $\sum cv^{\text{net}}$ minus the zero-trapdoor commitment of `value_balance` at `AssetBase::zatoshi()` minus, over the burn set, the zero-trapdoor commitment of each value at its asset (`orchard-zsa, src/bundle.rs`). Function `ValueCommitment::derive` computes $V \cdot \text{value} + R \cdot \text{rcv}$ with V the point `asset.cv_base()` and R the point `hash_to_curve("z.cash:Orchard-cv")` applied to `b"r"`, the value signed per the sign of the value sum (`orchard-zsa, src/value.rs`).

4.4 Burn

Burning is the value-balance mechanism read as an exit door. The mechanism “does NOT send Assets to a non-spendable address, it simply reduces the total number of units of a given Custom Asset in

circulation”; it is enforced at consensus level as an extension of the value-balance mechanism, and it “makes it globally provable that a given amount of a Custom Asset has been destroyed”. The Native Asset (ZEC/TAZ) cannot be burnt this way (ZIP-226).

ZIP-226 attaches three consensus rules to the burn set, whose cardinality it denotes L :

1. for every $(\text{AssetBase}, v) \in \text{assetBurn}$: $\text{AssetBase} \neq V^{\text{Orchard}}$;
2. $v > 0$ and $v \leq \text{MAX_BURN_VALUE} := 2^{63} - 1$;¹⁰
3. every AssetBase has at most one entry in assetBurn .

Burn data lives *per Action Group*: the $\text{ActionGroupDescription}$ carries $n\text{AssetBurn}$ (compactSize) and $v\text{AssetBurn}$, an array of 40-byte entries — asset_base ($\text{byte}[32]$) followed by valueBurn (uint64 , little-endian) — so that parties assembling separate Action Groups can each burn their own Assets (withdrawn ZIP-230; §4.7).

The transaction-level field $\text{valueBalanceOrchard}$ (int64) remains the net *ZEC-only* flow, “the net value of Orchard spends minus outputs” (withdrawn ZIP-230), and the transparent protocol is unchanged, so a Custom Asset cannot be unshielded by transfer: “All Custom Assets are contained within the shielded pool” (ZIP-226). Burning is the only exit. What a burn reveals is exactly its assetBurn entry: “the amount and identifier of the Asset being burnt”. What transfers hide: “the overall balance is checked without revealing which (or how many distinct) Assets are being transferred”; OrchardZSA ZEC commitments are indistinguishable from non-native OrchardZSA commitments, and the amounts and identifiers of transferred Assets stay private (ZIP-226).

A burn also updates ZIP-227’s global issuance state: $\text{issued_assets}(\text{AssetBase}).\text{balance} \in \{0, \dots, \text{MAX_ISSUE}\}$ is “computed as the amount of the Asset that has been issued less the amount of the Asset that has been burnt”, and the map **MUST** be updated during processing of any transaction containing burn information (ZIP-227). Supply sufficiency — that a burn not exceed the circulating supply — is therefore a state-level rule of the issuance protocol, not one of ZIP-226’s three per-transaction rules above; we construct the state machine in §3. The implementation enforces both layers side by side: $\text{validate_bundle_burn}$ returns the per-transaction errors ZatoshiAsset , ZeroAmount , InvalidAmount , and DuplicateAsset , and the state-level errors $\text{AssetNotFoundInState}$ and $\text{InsufficientSupply}$ (`orchard-zsa, src/bundle/burn_validation.rs`).

4.5 Split Inputs

Padding is where the asset dimension bites. An Orchard transfer with more outputs than inputs pads with *dummy* spends: zero-value notes whose Merkle membership check is gated off by v_{old} (constraint A3 of the *Ironwood Guide*, “The Action statement, formally”). For Custom Assets that padding is unsound, because the circuit forces the output note’s Asset Base to equal the input note’s (§4.6) — and a dummy input’s Asset Base is chosen freely by the prover. ZIP-226 states the attack that must be excluded: without enforcing that every Custom Asset input exists in the note commitment tree, “the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds”. The fix: dummy notes remain in use for ZEC notes only; Custom Assets always prove Merkle membership, padding instead with Split Inputs, which force the output’s Asset Base to equal a genuinely issued, GroupHash -derived base (ZIP-226).

The construction is developed once, in §5: the *Split Input* — a copy of a previously issued, tree-resident note of the target Asset, spent with $\text{split_flag} = 1$ and its value zeroed in the value-commitment computation only (Definition 5.1); its randomized nullifier with the offset point L^{Orchard} ,

¹⁰ZIP-230’s AssetBurn table instead says valueBurn is “checked by consensus to be non-zero, and less than MAX_BURN_VALUE ”. The implementation rejects only amounts strictly greater than MAX_BURN_VALUE , i.e. permits equality, agreeing with ZIP-226 (`orchard-zsa, src/bundle/burn_validation.rs`, which defines $\text{MAX_BURN_VALUE} = (1 \ll 63) - 1$). A specification-text defect, not a consensus divergence.

(3); the three-way divergence over the derivation of ψ^{nf} (§5.6); and the builder mechanics of padding per asset, dummy notes for ZEC and split spends for Custom Assets (§5.8). The three wallet strategies ZIP-226 offers for sourcing the copied note are enumerated in §5.2 (`zip-0226.rst:287–291`). Burn entry points reject duplicate Assets and amounts that do not fit in 63 bits (`add_burn`, `orchard-zsa`, `src/builder.rs`).

4.6 The Action statement, extended

A bin statement first. ZIP-226’s “Circuit Statement” section describes the constraint changes informally and defers “all modifications in the Circuit” to an external document; no statement at the rigor of the *Ironwood Guide*’s Definition “Orchard Action statement” exists in any specification. The implemented circuit is the de facto specification, and every gate-level claim below cites it (`orchard-zsa`, `src/circuit/circuit_zsa.rs`). Bin: *designed and implemented, but unspecified*.

ZIP-226 states the deltas against A1–A9 as follows:

- (a) the Asset Base is witnessed once as an auxiliary input, and both note commitment integrity constraints (A1, A2) switch to $\text{NoteCommit}^{\text{OrchardZSA}}$ with that same witness — one variable simultaneously enforces input/output Asset equality and commitment correctness;
- (b) an auxiliary boolean `is_native_asset` is constrained to equal 1 exactly when $\text{AssetBase} = V^{\text{Orchard}}$;
- (c) `enableZSA = 0` forces `is_native_asset = 1`;
- (d) the value commitment’s fixed-base multiplication (A4) becomes a variable-base multiplication by the witnessed Asset Base;
- (e) a boolean `split_flag` replaces the spent value inside the value commitment: the committed net value becomes $v' - v_{\text{new}}$, with $v' = 0$ if `split_flag = 1` and $v' = v_{\text{old}}$ otherwise; `split_flag = 1` forces `is_native_asset = 0`;
- (f) the Merkle path constraint (A3) becomes $(v_{\text{old}} = 0 \wedge \text{is_native_asset} = 1) \vee \text{ValidMerklePath}$ — the dummy skip survives only for the native Asset;
- (g) nullifier integrity is randomized for split notes (§5).

The circuit realizes these deltas in a single gate, `q_orchard`, whose twelve named constraints are stated once, in §5.5, in the code’s names (the code names the ZIP’s `is_native_asset` as `is_zatoshi_asset`); we do not restate them here.

Two gadgets carry the arithmetic weight. The in-circuit value commitment computes `[magnitude]asset_base` by variable-base long scalar multiplication on the witnessed non-identity point, applies `mul_sign` for the sign, and adds the fixed-base `[rcv]R^{\text{Orchard}}` (`src/circuit/value_commit_orchard.rs`). The in-circuit note commitment shares one Sinsemilla evaluation between the two branches of Definition 4.2: it muxes the initial point Q between Q_{ZEC} (domain `z.cash:Orchard-NoteCommit-M`) and Q_{ZSA} (domain `z.cash:ZSA-NoteCommit-M`; the hard-coded point is test-asserted in `src/constants/sinsemilla.rs`) on `is_zatoshi_asset`, hashes the common message prefix once, hashes the two suffixes (ZEC: the zero-padded piece h_2 ; ZSA: the Asset Base bits), muxes the resulting hash point, and adds the shared blinding term `[rcm]R` — the shared randomness base of Definition 4.2 is what makes this sharing possible (`src/circuit/note_commit.rs`). The Asset Base message pieces: $h_{\text{ZSA}} = (\text{bits } 249\text{--}253 \text{ of } \psi) \parallel (\text{bit } 254 \text{ of } \psi) \parallel (\text{bits } 0\text{--}3 \text{ of } x(\text{asset}))$; $i = \text{bits } 4\text{--}253 \text{ of } x(\text{asset})$; $j = (\text{bit } 254 \text{ of } x(\text{asset})) \parallel (\tilde{y}(\text{asset})) \parallel 8 \text{ zero bits (ibid.)}$.

The public instance grows from 9 to 10 field elements:

(anchor, $\text{cv}^{\text{net}}.x$, $\text{cv}^{\text{net}}.y$, nf_{old} , $\text{rk}.x$, $\text{rk}.y$, cmx ,
enable_spends, enable_outputs, enable_zsa),

with the new `enable_zsa` last. In the vanilla flavor `enable_zsa` is always false, and since halo2 pads instances with zeros, old nine-element proofs and new proofs behave identically; the crate accordingly carries two circuit flavors, `OrchardVanilla` and `OrchardZSA` (`orchard-zsa`, `src/circuit.rs`, `src/flavor.rs`). At the bundle level the `enableZSAs` flag is bit 2 of `flagsOrchard` (LSB order, after `enableSpendsOrchard` and `enableOutputsOrchard`; the implementation constant is `FLAG_ZSA_ENABLED = 0b0000_0100`), and coinbase transactions MUST set `enableSpendsOrchard` and `enableZSAs` to 0 (withdrawn ZIP-230; `orchard-zsa`, `src/bundle.rs`).

4.7 The carrier: transaction format, digests, and fees

The byte-level carrier has no merged deployment path upstream. Upstream ZIP-230 (the OrchardZSA v6 transaction format) is Status: Withdrawn, as is upstream ZIP-246 (its digests), and the new ZIP-229 (Version 6 Transaction Format; Draft, created 2026-06-13) states as a non-requirement that “the v6 transaction format need not support ZSAs”. The QEDit fork retains ZIP-230 and ZIP-246 as Draft. We therefore document the carrier from the withdrawn upstream texts, the fork, and the implementation, and bin it accordingly: the transfer/burn cryptography above is Draft-specified, while the encoding it rides on is *designed and implemented but not specified upstream* (§1).

Note plaintext. The OrchardZSA note plaintext adds `asset_base` (32 bytes) after `rseed`. Per withdrawn ZIP-230 the v6 Orchard note plaintext is

$$\text{np} := [\text{leadByte}] \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}},$$

with a 32-byte memo key K^{memo} replacing the 512-byte memo field, giving `encCiphertext` of 132 bytes and an `OrchardZSAAction` of 372 bytes (`cv` 32 + `nullifier` 32 + `rk` 32 + `cmx` 32 + `ephemeralKey` 32 + `encCiphertext` 132 + `outCiphertext` 80). ZIP-230 notes that an Orchard note “is essentially equivalent to an OrchardZSA note with `AssetBase = V^{\text{Orchard}}`” and that the two need not be distinguished in plaintexts.

Remark 4.6 (The lead byte is unassigned upstream). Upstream ZIP-226 requires `leadByte` to be the unresolved placeholder `{{ZSALEADBYTE}}` when the protocol’s note-creation sections (§4.7.3 “Sending Notes”, §4.8.3 “Dummy Notes”) are invoked in the computation of ρ and ψ for OrchardZSA notes — a sentence with no counterpart in the fork’s ZIP-226 and no implementation counterpart. Upstream ZIP-230 replaced its original `0x03` with the placeholder `{{LEADBYTE}}` because ZIP-2005 separately claims lead byte `0x03`. The QEDit fork ZIPs still say `0x03`, and the implementation hard-codes `NOTE_VERSION_BYTE_V3 = 0x03` with `MEMO_SIZE = 512` and no K^{memo} — the pre-memo-bundle layout, giving `COMPACT_NOTE_SIZE_ZSA = 52 + 32 = 84` and a 612-byte `encCiphertext`, diverging from ZIP-230’s 132 bytes (`orchard-zsa`, `src/primitives/zcash_note_encryption_domain.rs`, `src/primitives/orchard_primitives.rs`). No lead byte is currently reserved for ZSA notes upstream. Bin: *open*.

Digests. Withdrawn ZIP-246 extends the ZIP-244 digest tree — the baseline is the *Ironwood Guide*, “Conservation of value”, its ZIP-244 paragraph — with a per-Action-Group `orchard_burn_digest` (node T.4a.vi) under `orchard_action_groups_digest` (personalization `ZTxId0rcActGHash`), beside the compact and non-compact action digests, `flagsOrchard`, `anchorOrchard`, and `nAGExpiryHeight`. The compact digest covers `nullifier`, `cmx`, `ephemeralKey`, and `encCiphertext` (personalization `ZTxId60ActC_Hash`); the non-compact digest covers `cv`, `rk`, and `outCiphertext` (`ZTxId60ActN_Hash`). The burn digest is BLAKE2b-256 with personalization `ZTxId0rcBurnHash` over, per `assetBurn` tuple,

$$\text{assetBase} \parallel \text{valueBurn} \quad (64\text{-bit unsigned little-endian}),$$

the empty burn set hashing to `BLAKE2b-256(ZTxId0rcBurnHash, [ ])` (ZIP-246). One disambiguation: the header-digest field T.1g `burn_amount` is ZIP-233’s network-sustainability burn, unrelated to asset burn (ZIP-246, T.1; ZIP-230, Transaction Format).

Parsing and bundle types. Function `read_v6_bundle` (gated by `zcash_unstable = "nu7"`) reads `nActionGroups`, which must be 0 or exactly 1 (“A V6 transaction must contain at most one action group”), then the actions, flags, anchor, `nAGExpiryHeight` (must be 0), the burn vector (32-byte Asset Base plus note value), the proof, per-action versioned signatures, `valueBalance`, and a versioned binding signature; the write path is symmetric via `write_burn` (`librustzcash-zsa`, `zcash_primitives/src/transaction/components/orchard.rs`). The bundle type carries the burn set as the field `burn: Vec<(AssetBase, NoteValue)>` beside `value_balance`, and `binding_validating_key()` returns `derive_bvk(actions, value_balance, burn)`; the builder collects burns in a `BTreeMap<AssetBase, NoteValue>`, canonically ordered and duplicate-free by construction (`orchard-zsa`, `src/bundle.rs`, `src/builder.rs`). One open seam: ZIP-230 permits `nActionGroupsOrchard > 1` with per-group burn fields (the multi-party rationale of §4.4), the implementation rejects more than one group, and ZIP-226’s `bvk` equation is written over a single flat action/burn set; how multi-group aggregation would interact with the binding signature is unexercised. Bin: *open*.

Tree ordering. Per Action Group, every new OrchardZSA note commitment is appended to the note commitment tree per Action; afterwards, Issue Note commitments are appended per Issue Action (ZIP-227, “Addition to the Note Commitment Tree”; §3).

Fees. The QEDit fork of ZIP-317 adds `contribution_ZSAIssuance` and `contribution_ZSACreation` terms — issue notes and asset creations respectively — to the conventional-fee formula; it adds no burn term, so a burn contributes to fees only through the Actions that fund it. Upstream ZIP-317 contains no ZSA changes. The conventional-fee baseline is the *Wallet Guide*, “Fees (ZIP-317)”.

5 Split notes and the counterfeiting boundary

An Orchard bundle pads to a uniform shape with *dummy notes*: an Action with no real input spends a zero-valued note under a freshly sampled key, and the membership constraint waves it through because the check is gated by the spent value — constraint A3 of the Action statement, $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$, “how an Action pads with no real input” (the *Ironwood Guide*, “The Action statement, formally”; its node-verification section, “What a node verifies, without ever observing the note”, records that zero-value dummy spends are accepted by design). The construction is protocol §4.8.3, implemented in `orchard-zsa`, `src/builder.rs`, `SpendInfo::dummy`: a random spending key, a zero-valued note, and a random Merkle path that is never checked. The exemption is sound there because every Orchard value commitment rides the *fixed* base V^{Orchard} (*Ironwood Guide*, “Conservation of value: value commitments and the binding signature”): a dummy spend can move no value, whatever path it fails to prove. ZIP-226 replaces the fixed base with a witnessed one, and the same exemption becomes a mint. This section develops the attack that forces the change, the *split note* that ZIP-226 substitutes for the dummy note on custom assets, and the exact constraint set — specified and implemented — that draws the counterfeiting boundary.

Ground truth for this section: ZIP-226 and ZIP-227 at `zcash/zips` commit 69610984 (2026-07-09); the pinned implementation, crate `orchard` of the QED-it fork (`orchard-zsa`, branch `zsa1`) at commit `cf801a5d` (2026-06-29); the QED-it `zips` fork at `fd71419a` where it diverges from upstream. Split-note logic lives entirely in the `orchard` crate; the `librustzcash` fork consumes its bundle API and contains no split-note code of its own (verified by search at `3f9b53fc`).

5.1 The counterfeit construction the boundary excludes

ZIP-226 generalises the net value commitment from the fixed base V^{Orchard} to a per-asset base (ZIP-226, “Value Commitment”):

$$\begin{aligned} \text{cv}^{\text{net}} &:= \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}_{\text{AssetId}}, v_{\text{AssetId}}^{\text{net}}) \\ &= [v_{\text{AssetId}}^{\text{net}}] \text{AssetBase}_{\text{AssetId}} + [\text{rcv}] R^{\text{Orchard}}, \end{aligned} \quad (1)$$

with $v^{\text{net}} := v_{\text{old}} - v_{\text{new}}$ and R^{Orchard} the randomness base of the *Ironwood Guide*’s net value commitment (“Conservation of value: value commitments and the binding signature”), unchanged. The native asset keeps the old base: “the Asset Base of the ZEC Asset will be kept as the original value base point V^{Orchard} ” (ZIP-226, “Asset Identifiers”; in code `AssetBase::zatoshi()` is `GroupHashPallas(z.cash : Orchard-cv, "v"), orchard-zsa, src/note/asset_base.rs, src/constants/fixed_bases.rs`). Verification subtracts the declared native balance and the declared burns (ZIP-226, “Value Balance Verification”; implemented in `src/bundle.rs, derive_bvk_raw`):

$$\begin{aligned} \text{bvk} &= \left(\sum_{i=1}^n \text{cv}_i^{\text{net}} \right) - \text{ValueCommit}_0^{\text{OrchardZSA}}(V^{\text{Orchard}}, v_{\text{balanceOrchard}}) \\ &\quad - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v), \end{aligned} \quad (2)$$

and the binding signature under bvk proves knowledge of $\text{bsk} = \sum_i \text{rcv}_i$ exactly as in Orchard.

Suppose now that the dummy exemption survived unchanged: a padding Action proves no membership, and — since the base of (1) is a private witness — may witness *any* point A as its asset base. Choose $A := [-1] V^{\text{Orchard}}$, witness $v_{\text{old}} = 0$ and $v_{\text{new}} = v$:

$$\text{cv}_1^{\text{net}} = [0 - v] A + [\text{rcv}_1] R^{\text{Orchard}} = [v] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}.$$

Pair it with a second Action that outputs a genuine v -valued ZEC note,

$$\text{cv}_2^{\text{net}} = [0 - v] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}},$$

and declare $v_{\text{balanceOrchard}} = 0$ with no burns. The V^{Orchard} -components cancel in (2),

$$\text{bvk} = \text{cv}_1^{\text{net}} + \text{cv}_2^{\text{net}} = [\text{rcv}_1 + \text{rcv}_2] R^{\text{Orchard}},$$

the attacker knows $\text{bsk} = \text{rcv}_1 + \text{rcv}_2$, the binding signature verifies — and a v -valued ZEC note now exists that no one paid for. Any known multiple or linear combination of existing bases serves equally. (*Classification*: the construction is assembled here from the specified formulas (1) and (2); ZIP-226 states the attack only in words.) The ZIP’s own statement, verbatim: the output note of split Actions “cannot contain just any Asset Base. We must enforce it to be an actual output of a GroupHash computation... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds” (ZIP-226, “Split Notes”, rationale).

The boundary to draw is therefore precise: every asset base that enters a value commitment must be proven to be an output of GroupHash — a point of unknown discrete-log relation to V^{Orchard} and R^{Orchard} — and the one place an unproven base could enter is an Action with no real input.

5.2 The split input

Definition 5.1 (Split Input, Split Action). ZIP-226 defines a *Split Input* as “an Action input used to ensure that the output note of that Action is of a validly issued AssetBase when there is no corresponding real input note, in situations where the number of outputs are larger than the number

of inputs”, and a *Split Action* as an Action containing a Split Input (ZIP-226, “Terminology”).

Mechanically, a Split Input is a *copy of a previously issued note* — one whose commitment is already in the note commitment tree — with exactly two changes: a boolean witness `split_flag` is set to 1, and the note’s value is replaced by 0 *in the value-commitment computation only* (ZIP-226, “Split Notes”). The copied note’s commitment cm_{old} still opens with the true value: in the circuit the value is zeroed solely in the cv^{net} equation, while the old-note commitment check consumes v_{old} unchanged (orchard-zsa, `src/circuit/circuit_zsa.rs`: the gate zeroes v_{old} via the $(1 - split_flag)$ factor in the value equation, lines 127–131, whereas the `NoteCommit` call at lines 637–655 takes v_{old} as witnessed). The copy is therefore not a spend — it moves no value and, as §5.6 shows, reveals no usable nullifier — but it drags the original note’s asset base into the proof, where an equality constraint passes it to the output note.

The split Action still balances, per asset. Committed values must sum to zero for each asset base separately, “as the Pedersen commitments add up homomorphically only with respect to the same value base point” (ZIP-226, “Value Balance Verification”): for a correct transaction the custom-asset commitments satisfy

$$\sum_{j \in S_{CA}} cv_j^{net} - \sum_{(AssetBase, v) \in assetBurn} [v] AssetBase = \sum_{j \in S_{CA}} [rcv_j] R^{Orchard},$$

so the split Action’s $cv^{net} = [0 - v_{new}] AssetBase + [rcv] R^{Orchard}$ cancels against the real spending Actions of the same asset. The builder implements exactly this: `ActionInfo::value_sum` uses a spent value of 0 when `split_flag` is set (orchard-zsa, `src/builder.rs`, lines 495–506), and `derive_bvk_raw` computes (2) over the resulting commitments (`src/bundle.rs`, lines 482–517).

For the note to copy, ZIP-226 offers the wallet three options: (1) another note of the same Asset Base being spent in the same transaction; (2) a different unspent note of that Asset Base, which is *not* thereby spent; (3) an already spent note of that Asset Base — the zeroed value prevents double spending (ZIP-226, “Split Notes”). The ZIP adds that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before”; §5.7 examines how far that sentence actually holds against the constraint system.

5.3 Membership as well-formedness: the issuance anchor

The fix ZIP-226 specifies, verbatim: “for Custom Assets we enforce that every input note to an OrchardZSA Action must be proven to exist in the set of note commitments in the note commitment tree. We then enforce this real note to be unspendable in the sense that its value will be zeroed in split Actions and the nullifier will be randomized... the proof itself ensures that the output note is of the same Asset Base as the input note. In the circuit, the split note functionality will be activated by a boolean private input (aka the `split_flag` boolean)” (ZIP-226, “Split Notes”, rationale).

Why does tree membership prove that an asset base is a `GroupHash` output? Because for custom assets the tree is fed exclusively from two sources that both enforce it. Transfer outputs inherit their base from a proven input by the in-circuit equality constraint (§5.4). Issuance outputs are checked publicly: a consensus rule obliges validators to recompute each Issue Note’s `AssetBase` from the issuance bundle’s issuer key and the `IssueAction`’s `assetDescHash` (ZIP-227, “Specification: Consensus Rule Changes”), along the derivation chain constructed in §2.3 (ZIP-227, “Asset Bases”; recomputed in orchard-zsa, `src/note/asset_base.rs`), and then appends the issue-note commitments to the *same* Orchard note commitment tree — transfer-Action commitments first, then issuance notes (ZIP-227, “Addition to the Note Commitment Tree”). Hence a custom-asset note commitment resident in the tree commits, by induction along the two rules, to a genuine `GroupHash` output. ZIP-226 states the conclusion: “This ensures that the value base points of all output notes of a transfer are actual outputs of a `GroupHash`, as they originate in the Issuance protocol which is publicly verified” (“Split Notes”, rationale); and, on the identifier side, “We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state” (“Rationale for Asset Identifiers”).

One clarification the ZIP’s sentence invites: “every input note to an OrchardZSA Action” is scoped to *custom-asset* inputs. Native-asset (ZEC) Actions in a v6 bundle retain the dummy-note exemption unchanged — “The ZEC-based Actions will still include dummy input notes, whereas the OrchardZSA Actions will include split input notes and will not include dummy input notes” (ZIP-226, “Backward Compatibility”; the “Split Notes” rationale states the same). The constraint-level form of this scoping is the disjunction in §5.4.

5.4 The specified circuit delta

We state ZIP-226’s circuit changes (“Circuit Statement”) as a delta against the Action statement A1–A9 of the *Ironwood Guide* (“The Action statement, formally”). Two new boolean witnesses appear: `split_flag`, and `is_native_asset`, constrained to equal 1 exactly when the witnessed base is the native one, $\text{AssetBase} = V^{\text{Orchard}}$. One public input appears: the bundle-level `enableZSA` flag (a bit of the `v6 flagsOrchardZSA` field), with the constraint $\text{enableZSA} = 0 \implies \text{is_native_asset} = 1$ — switching the flag off collapses the circuit to native-asset behaviour and, transitively via the split constraint below, disables split notes (ZIP-226, “Overview” and “Circuit Statement”, The `enableZSA` Flag; in code the flag is the tenth public input, taken from `Flags::zsa_enabled()`, `orchard-zsa, src/circuit.rs`, lines 476–494, 560).

- **A1, A2 (note commitment integrity).** Both commitment checks replace $\text{NoteCommit}^{\text{Orchard}}$ with $\text{NoteCommit}^{\text{OrchardZSA}}$, which takes the witnessed `AssetBase` as an additional final argument; the *same* once-witnessed base feeds the old-note check, the new-note check, and the value commitment (ZIP-226, “Circuit Statement”, Asset Base Equality). For the native asset $\text{NoteCommit}^{\text{OrchardZSA}}$ coincides with $\text{NoteCommit}^{\text{Orchard}}$; otherwise it is the Sinsemilla commitment over the extended message ending in the asset-base encoding, in the hash domain “z.cash:ZSA-NoteCommit-M” with the blinding base shared with Orchard (“z.cash:Orchard-NoteCommit-r”) (ZIP-226, “Note Structure and Commitment”). The full $\text{NoteCommit}^{\text{OrchardZSA}}$ construction is Definition 4.2; its in-circuit mux is described in §4.6.
- **A3 (membership).** The value gate becomes a disjunction (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions — Merkle Path Validity):

$$(\nu_{\text{old}} = 0 \wedge \text{is_native_asset} = 1) \vee (\text{Valid Merkle Path}),$$

so the dummy skip survives *only* for the native asset. Every custom-asset input — split or not, zero-valued or not — must prove membership.

- **A4 (value commitment).** Two changes. First, “the fixed-base multiplication constraint between the value and the value base point of the value commitment, `cv`, is replaced with a variable-base multiplication between the two” (ZIP-226, “Circuit Statement”, Value Commitment Correctness), the base being the witnessed `AssetBase`. Second, the input value is replaced by a generic v' (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions):

$$\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v' - \nu_{\text{new}}), \quad v' = \begin{cases} 0 & \text{if } \text{split_flag} = 1, \\ \nu_{\text{old}} & \text{otherwise,} \end{cases}$$

together with $\text{split_flag} = 1 \implies \text{is_native_asset} = 0$: split notes exist only for custom assets, and a zero-valued native dummy cannot masquerade as one.

- **A5 (nullifier).** The derivation changes for split notes “to prevent the identification of notes” (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions); §5.6 develops it.
- **A6, A7 (spend authority, address integrity).** Untouched by the ZIP’s change list — and, decisively for §5.7, they remain *unconditional*: no `split_flag` factor relaxes them.

5.5 The implemented gate

A naming note, once: the ZIP’s `is_native_asset` is `is_zatoshi_asset` throughout the implementation, assigned 1 exactly when the witnessed asset equals `AssetBase::zatoshi()` (`orchard-zsa`, `src/circuit/gadget.rs`, `circuit_zsa.rs`). We keep the ZIP name in prose and the code name inside code-derived formulas.

The implemented circuit concentrates the delta in a single gate, `q_orchard`, which carries twelve named constraints where the vanilla circuit’s corresponding gate has four (`src/circuit/circuit_zsa.rs`, lines 68–185, versus `src/circuit/circuit_vanilla.rs`, lines 59–100). In the code’s names:

$$\begin{aligned} & \text{bool_check}(\text{split_flag}), \quad \text{bool_check}(\text{is_zatoshi_asset}), \\ & v_{\text{old}} \cdot (1 - \text{split_flag}) - v_{\text{new}} = \text{magnitude} \cdot \text{sign}, \\ & (v_{\text{old}} + 1 - \text{is_zatoshi_asset}) \cdot (\text{root} - \text{anchor}) = 0, \\ & v_{\text{old}} = 0 \vee \text{enable_spends} = 1, \quad v_{\text{new}} = 0 \vee \text{enable_outputs} = 1, \\ & \text{is_zatoshi_asset} \cdot (\text{asset}_x - \text{zatoshi}_x) = 0, \quad \text{is_zatoshi_asset} \cdot (\text{asset}_y - \text{zatoshi}_y) = 0, \\ & (1 - \text{is_zatoshi_asset}) \cdot (\Delta_x \cdot \Delta_x^{\text{inv}} - 1) \cdot (\Delta_y \cdot \Delta_y^{\text{inv}} - 1) = 0, \\ & (1 - \text{split_flag}) \cdot (\psi_{\text{old}} - \psi^{\text{nf}}) = 0, \quad \text{split_flag} \cdot \text{is_zatoshi_asset} = 0, \\ & (1 - \text{enable_zsa}) \cdot (1 - \text{is_zatoshi_asset}) = 0, \end{aligned}$$

where Δ_x, Δ_y abbreviate the code’s `diff_asset_x`, `diff_asset_y` — the witnessed asset’s coordinates minus the native base’s — and ψ^{nf} is the code’s `psi_nf` (§5.6). Four points merit unpacking.

The folded value equation. The single constraint $v_{\text{old}} \cdot (1 - \text{split_flag}) - v_{\text{new}} = \text{magnitude} \cdot \text{sign}$ folds the ZIP’s case-split v' into one expression; the witness generator supplies the matching net value, $-v_{\text{new}}$ for split spends and $v_{\text{old}} - v_{\text{new}}$ otherwise (`circuit_zsa.rs`, lines 462–501).

The Merkle factor. The disjunction of A3 is encoded as the single product $(v_{\text{old}} + 1 - \text{is_zatoshi_asset}) \cdot (\text{root} - \text{anchor}) = 0$. The code comment argues the encoding safe from wrap-around: `is_zatoshi_asset` is boolean-checked and v_{old} is range-checked to 64 bits in the note-commitment evaluation, so over $\mathbb{F}_{p_{\text{Pallas}}}$ the first factor vanishes exactly when $v_{\text{old}} = 0$ and `is_zatoshi_asset` = 1 (`circuit_zsa.rs`, lines 132–139). The consequence bears repeating: for every custom-asset input the factor is nonzero, so `root` = `anchor` is mandatory — a strictly stronger membership requirement than vanilla Orchard’s v_{old} -gate, applying even to zero-valued non-split custom-asset inputs.

Proving inequality. The assignment `is_zatoshi_asset` = 0 must itself be sound: the circuit must verify that the witnessed base *differs* from V^{Orchard} . The prover witnesses field inverses $\Delta_x^{\text{inv}}, \Delta_y^{\text{inv}}$ (assigned 0 when the corresponding difference vanishes), and the triple product above forces at least one difference to be exhibited invertible (`circuit_zsa.rs`, lines 160–170 for the gate, 792–841 for the assignment).

The variable-base value commitment. With the base witnessed, the vanilla circuit’s fixed-base *short* multiplication $[v^{\text{net}}] V^{\text{Orchard}}$ — whose 64-bit range check is implicit in the short-multiplication decomposition — is no longer available. The ZSA gadget makes the range check explicit: the magnitude is decomposed against the Sinsemilla lookup table as six 10-bit windows plus one 4-bit short check (64 bits), then promoted to a full-width scalar (`ScalarVar::from_base`) and fed to the variable-base long multiplication — the code notes it is “optimized for ~255-bit scalar” and leaves a TODO to specialise it for 64 bits — after which the sign is applied and the blind $[rcv] R^{\text{Orchard}}$ added (`src/circuit/value_commit_orchard.rs`, lines 38–127). Two supporting details: the

ZSA circuit uses the extended lookup configuration `PallasLookupRangeCheck4_5BConfig` (an additional lookup-table column for 4- and 5-bit short checks), and it refuses to synthesise under `CircuitVersion::InsecureUnanchoredBase`, the pre-NU6.2 `halo2_gadgets` variable-base scalar-multiplication version whose base point was not anchored — directly load-bearing here, because the entire counterfeiting boundary rests on the multiplication being by the *witnessed* `AssetBase` and nothing else (`circuit_zsa.rs`, lines 22, 51, 190–197, 334–337; `circuit.rs`, lines 151–185).

The flag interactions are tested asymmetrically: the three legal combinations of $(\text{is_zatoshi_asset}, \text{split_flag})$ — $(1, 0)$, $(0, 1)$, and $(0, 0)$ — are proven passing, while the $(1, 1)$ combination is excluded at witness-generation time by the test helper’s assertion (“We cannot create a split note with a zatoshi asset”), so the constraint $\text{split_flag} \cdot \text{is_zatoshi_asset} = 0$ is not itself exercised by a failing-proof test (`circuit_zsa.rs`, lines 1174–1312). Documented cost: the ZSA proof measures 5120 bytes for one Action and 7392 for two, against the vanilla 4992 and 7264, at the shared circuit size $K = 11$; the `enableZSA` bit is the tenth public input, at index 9 (the proof-size tests of `circuit_zsa.rs` and `circuit_vanilla.rs`; `circuit.rs`, lines 68–80, 536–563).

5.6 The split nullifier

A split input copies a note that may sit unspent in someone’s wallet; its Action must nevertheless publish *some* nullifier. Publishing the note’s true nullifier would be harmful in both directions: it would mark an unspent note spent (bricking it), and it would collide with the honest nullifier if the same note were spent for real in the same transaction. ZIP-226 therefore randomises the split note’s nullifier (“Split Notes”):

$$\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi^{\text{nf}}] G^{\text{nf}} + \text{cm}_{\text{old}} + L^{\text{Orchard}}), \quad (3)$$

in the notation of the *Ironwood Guide*’s A5 (“The Action statement, formally”; ZIP-226 writes $\mathcal{K}^{\text{Orchard}}$ for G^{nf} and computes the bracketed sum in the Pallas base field). Two deltas against A5: the note’s ψ_{old} is replaced by a fresh ψ^{nf} , and a fixed offset point

$$L^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, "L")$$

is added (ZIP-226, “Split Notes”; the constant is pinned and test-asserted against the `GroupHash` evaluation in `orchard-zsa, src/constants/nullifier_1.rs`). The gate constraint $(1 - \text{split_flag}) \cdot (\psi_{\text{old}} - \psi^{\text{nf}}) = 0$ of §5.5 makes the substitution split-only: non-split OrchardZSA nullifiers coincide with the Orchard derivation, as the ZIP states (“Note Structure and Commitment”: the non-split nullifier “is generated in the same manner as in the Orchard protocol”).

In circuit, the nullifier gadget computes the standard point $\text{cm}_{\text{old}} + [F_{\text{nk}}(\rho_{\text{old}}) + \psi^{\text{nf}}] G^{\text{nf}}$ from the witnessed ψ^{nf} , forms the offset variant by adding the constant L^{Orchard} , and muxes the two on `split_flag` before extraction (`src/circuit/derive_nullifier.rs`, lines 54–111). Out of circuit, `Nullifier::derive` mirrors the computation and selects the offset variant in constant time exactly when the note carries a `rseed_split_note` (`src/note/nullifier.rs`, lines 73–90; `src/note.rs`, lines 415–426).

Deriving ψ^{nf} : a three-way divergence. The value of ψ^{nf} is a private witness, constrained by the circuit only in the *non-split* case; how a wallet derives it for split notes is therefore wallet-level, not consensus-level. The three extant texts disagree; per the volume’s policy (§1.3), the upstream ZIP formula comes first. Upstream ZIP-226 — written against the ZIP-2005 baseline, “expected to deploy before any ZSA activation” — specifies

$$\psi^{\text{nf}} := \text{ToBase}(\text{PRFexpand}_{\text{rseed_nf}}(0x0A \parallel \rho_{\text{old}}))$$

with `rseed_nf` uniform on 32 bytes and the domain byte `0x0A` “reserved by ZIP 2005 for this use” (`zip-0226.rst:297 @ 69610984`). The QED-it zips fork specifies neither derivation: there ψ^{nf} “is

sampled uniformly at random” on the Pallas base field (fork `zip-0226.rst:293 @ fd71419a`). The implementation derives

$$\psi^{\text{nf}} := \text{ToBase}(\text{PRFexpand}_{\text{rseed_split_note}}(0x09 \parallel \rho_{\text{old}})),$$

the ordinary ψ domain byte keyed by a fresh random 32-byte `rseed_split_note` (`orchard-zsa, src/circuit.rs:316–319, src/note.rs:114–116` and `417–425`; domain byte `PSI = 0x09` per the QED-it `zcash_spec` fork, `src/prf_expand.rs:89 @ d5e84264`). All three yield a uniformly distributed witness; the implementation matches neither ZIP text exactly. (*Classification*: each derivation is specified in its own text; the divergence is consensus-invisible.)

Why the offset L^{Orchard} ? The randomised ψ^{nf} already makes the split nullifier unlinkable to the note’s true nullifier. ZIP-226 states no rationale for additionally adding L^{Orchard} . A plausible purpose is robustness against a prover who *chooses* $\psi^{\text{nf}} = \psi_{\text{old}}$ — the circuit does not forbid it in the split case — and would thereby publish the note’s true nullifier from a zero-valued split Action, bricking the original note or colliding with an honest spend of it elsewhere in the transaction; the constant offset makes even that choice yield a different group element. (*Classification*: designed-but-unspecified; the preceding sentence is our inference, stated in neither ZIP nor code.)

5.7 Ownership, in fact and in prose

Recall the ZIP’s claim that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender” (§5.2). The constraint system says otherwise. Spend authority (A6) and address integrity (A7) are unchanged *and unconditional* in the ZSA circuit: the proof establishes $\text{rk} = [\alpha] G^{\text{SpendAuth}} + \text{ak}$ and $\text{ivk} = \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$ with $\text{pk}_{\text{dold}} = [\text{ivk}] g_{\text{dold}}$ — for split inputs too (`orchard-zsa, src/circuit/circuit_zsa.rs, lines 554–623`; the old-note commitment equality at lines 625–659). Because the copied note’s cm_{old} must open to the actual tree leaf (Sinsemilla binding plus the mandatory Merkle check of §5.5), the witnessed pk_{dold} is the original recipient’s, and A7 then forces the prover to know $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ consistent with it — that is, to hold the copied note’s full viewing key. The ZIP’s ownership sentence holds for *spentness* — a nullified note is a perfectly good split input, precisely because the value is zeroed — but not for ownership in the key-material sense. The implementation never exercises the freedom the prose suggests: the builder only ever copies one of the sender’s own spends in the same bundle, option (1) of §5.2 (`src/builder.rs, pad_spend`). (*Classification*: the constraint reading is specified — the ZIP’s constraint list and the implemented circuit agree; the rationale prose overstates.)

One genuine exception exists, and neither ZIP states it. ZIP-227 requires the first note of any asset’s first issuance to be a *reference note*: value 0, recipient the default diversified address of the all-zero Orchard spending key — a key anyone can derive (ZIP-227, “Reference Notes”; `orchard-zsa, src/constants/reference_keys.rs` pins the all-zero spending key and the 43-byte raw address). Because that spending key is public, anyone can satisfy A6/A7 for the reference note, which makes it a universally available split input for its asset — the one case in which “ownership does not matter” holds literally. (*Classification*: inference; the connection between reference notes and split inputs is stated in neither ZIP nor the implementation.)

5.8 Builder mechanics: padding, per asset

The two padding regimes coexist per ZIP-226’s rule quoted in §5.3: dummy inputs for ZEC Actions, split inputs for custom-asset Actions. The builder partitions spends and outputs by asset, pads each side of each partition to equal length, shuffles within each asset, then shuffles the assembled Actions across assets (`orchard-zsa, src/builder.rs, lines 1081–1164`). The padding spend is asset-dependent (`pad_spend, lines 928–944`): for the native asset, `SpendInfo::dummy` exactly as in Orchard; for a custom asset, `create_split_spend` on the first real spend of that asset in the bundle —

and if the bundle spends none, `BuildError::NoSplitNoteAvailable` (“No split note has been provided for this asset”): unlike a dummy, a split input cannot be conjured from nothing. Output-side padding uses dummy outputs of the padding asset in both regimes.

The copy itself is minimal. Function `Note::create_split_note` duplicates the note and sets exactly one new field, `rseed_split_note` — a fresh random 32-byte seed, drawn by `RandomSeed::random` and resampled until the derived `esk` is valid — asserting the asset is not the native one; `SpendInfo::create_split_spend` keeps the original note’s full viewing key and Merkle path and raises `split_flag` (`src/note.rs`, lines 436–442, 95–103; `src/builder.rs`, lines 302–318). Recipient, value, asset, ρ , `rseed` — hence `cmold` — are identical to the tree-resident original; the only witness-level differences downstream are `split_flag = 1` and the ψ^{nf} drawn from the new seed. Witness assembly routes exactly that: `Witnesses::from_action_context_unchecked` takes ψ^{nf} from `rseed_split_note` when present and from `rseed` otherwise — so $\psi^{\text{nf}} = \psi_{\text{old}}$ for every non-split spend — and packs $(\psi^{\text{nf}}, \text{asset}, \text{split_flag})$ into the ZSA witness extension that the vanilla flavour leaves unknown (`src/circuit.rs`, lines 199–244, 300–345).

The counterfeiting boundary, restated once in full: a custom-asset output note’s base equals its Action’s input-note base (the A1/A2 equality on the shared witness); that input note is a tree leaf (the mandatory Merkle factor); every custom-asset tree leaf descends from a publicly recomputed `GroupHash` output (the issuance anchor of §5.3); and the copy that makes padding possible moves no value (the zeroed v') and emits an unlinkable, non-colliding nullifier (3). Each link is enforced by a constraint quoted above; breaking any one of them re-opens the mint of §5.1.

6 Transaction integration and outlook

The preceding sections construct the OrchardZSA objects — asset bases (§2), notes and Actions (§4, §5), issuance (§3), and burn (§4.4). This section places them in a transaction: the v6 wire format (ZIP-230), the digest tree that yields transaction identifiers, sighashes, and the authorizing-data commitment (ZIP-246), the transaction-level consensus rules of ZIP-226/227, the fee treatment (ZIP-317), the state of the QED-it implementation stack at commit-pinned sources, the obligations the format imposes on wallets, and the deployment outlook. Deployment status — Draft ZIPs, externally audited at circuit scope, running on QED-it’s public testnet, contested for NU7 — is stated once in the scope section (§1.2) and not relitigated per object; what this section adds is the classification of each format artifact, because the ZSA transaction layer is the part of the protocol where specification and implementation have diverged furthest. Table 4 pins the sources; all repository facts below were verified firsthand at the stated commits, on 2026-07-14.

Source	Commit	Date	Role
zcash/zips	6961098	2026-07-09	upstream ZIP texts
QED-it zips fork	fd71419	2026-02-11	fork ZIP texts (pre-withdrawal)
QED-it orchard fork, zsa1	cf801a5d	2026-06-29	OrchardZSA bundle, circuit, issuance
QED-it librustzcash fork, zsa1	3f9b53fc	2026-04-17	v6 assembly, digests, fees
zcash/orchard@ltf/zsa	a799082e	2025-11-25	upstream tracking branch

Table 4: Pinned sources for this section. The `ltf/zsa` branch merges ZSA-compatible Halo 2 plus PCZT external spend-authorization signatures; it is the beginning of an upstreaming path, not a deployable stack.

6.1 Format status: version 6 no longer means OrchardZSA

The carrier format for everything in this volume is ZIP-230, the OrchardZSA v6 transaction format, with its digests in ZIP-246. Both are *withdrawn* upstream. ZIP-230 is retitled “Withdrawn Version 6 Transaction Format”; its warning states that it is obsoleted by ZIP-229 and “will not be deployed.

Transaction version number 6 is now defined by ZIP 229.” (`zip-0230.rst`, header and warning). ZIP-246 is likewise withdrawn (“Digests for the Withdrawn Version 6 Transaction Format”); v6 transaction identifiers, authorizing-data commitments, and signature digests are now specified by ZIP-229 (`zip-0246.rst`, header). The QED-it fork copies of both remain Status: Draft under their original titles — the fork (@fd71419) predates the withdrawal.

ZIP-229 (Status: Draft, created 2026-06-13) redefines v6 for NU6.3, introducing the Ironwood pool in the aftermath of the Orchard soundness vulnerability whose mitigation deployed as NU6.2 (ZIP-257, Status: Final). Its Non-requirements state explicitly that “the v6 transaction format need not support ZSAs, the `zip233Amount` field, the explicit fee field, or per-transparent-input sighash information that appeared in the withdrawn ZIP 230,” and that version number 6 need not avoid collision with withdrawn ZIP-230 or draft ZIP-248 (ZIP-229, Non-requirements). The same transaction version number therefore now denotes two incompatible byte formats, and the ZSA one is the orphan.

Classification, per the three-bin discipline of this series (the *Tachyon Guide*, §“The three-bin method”): the byte-level format below is fixed at ZIP rigor only by withdrawn upstream text and pinned fork code — *designed but unspecified* (§1.4) — with no deployment path under its current ZIP numbers. We document it because it is the format the pinned implementation speaks and the reference against which any respecification will be diffed; §6.8 states what would have to happen for it to become normative again.

6.2 The v6 wire format (ZIP-230)

Common and transparent fields. The common header carries header (`fOverwintered` and the version), `nVersionGroupId`, `nConsensusBranchId`, `lock_time`, `nExpiryHeight`, an explicit 8-byte fee (`uint64`, per ZIP-2002), and an 8-byte `zip233Amount` (`uint64`, per ZIP-233) (ZIP-230, Transaction Format). Making the fee an explicit field — rather than the implicit difference between transparent inputs and outputs — is one of the format’s two departures from v5 visible already in the header; the second is that signatures become versioned, below. The transparent fields add `vSighashInfo`: one `TransparentSighashInfo` (a `compactSize` length followed by `sighashInfo` bytes) per transparent input (ZIP-230, Transaction Format).

Orchard fields: Action Groups. The Orchard component becomes a vector of *Action Groups*: `nActionGroupsOrchard`, `vActionGroupsOrchard`, then a transaction-level `valueBalanceOrchard` (`int64`) and `bindingSigOrchard`. The balance and binding signature are present iff `nActionGroupsOrchard > 0`; an absent `valueBalanceOrchard` means $v_{\text{balance}}^{\text{Orchard}} = 0$ (ZIP-230, Transaction Format). Each `ActionGroupDescription` contains, in order (ZIP-230, OrchardZSA Action Group Description):

- `nActionsOrchard` (`compactSize`, MUST be > 0) and `vActionsOrchard`, at 372 bytes per OrchardZSAAction;
- `flagsOrchard`, LSB-first: `enableSpendsOrchard`, `enableOutputsOrchard`, `enableZSAs`, remaining bits zero;
- `anchorOrchard` (32 bytes) — each group carries its own anchor;
- `nAGExpiryHeight` (`uint32`);
- `nAssetBurn` and `vAssetBurn`, at 40 bytes per `AssetBurn` entry;
- `sizeProofsOrchard` and `proofsOrchard`, one aggregated proof for the group;
- `vSpendAuthSigsOrchard`, one `OrchardSignature` per Action.

Field `nAGExpiryHeight` exists for forward compatibility with ZSA atomic swaps. ZIP-228 is now a Draft that uses this field to expire swap orders, but it still depends on the withdrawn ZIP-230/246 carrier; for the core OrchardZSA proposal the field is zero by consensus (ZIP-230, Rationale for

`nAGExpiryHeight`; §6.4). A normative rule of ZIP-230 (its consensus-rule list, not the rationale) states “In NU7, `nExpiryHeight` MUST be set to 0” — read literally, a constraint on the transaction-level ZIP-203 field that would contradict ordinary expiry semantics; context, the ZIP-227 consensus rule, and the implementation (which constrains only `nAGExpiryHeight`) all indicate `nAGExpiryHeight` is meant. We flag this as an apparent editorial defect in the withdrawn text. Burn fields sit inside each Action Group rather than at transaction level so that, in a future multi-party Action Group world, each party can burn assets within its own group (ZIP-230, Rationale for including Burn fields inside OrchardZSA Action Groups).

Action and burn encodings. An `OrchardZSAAction` occupies 372 bytes: `cv` (32), `nullifier` (32), `rk` (32), `cmx` (32), `ephemeralKey` (32), `encCiphertext` (132), and `outCiphertext` (80) (ZIP-230, OrchardZSA Action Group Description). The 132-byte ciphertext is the memo-bundle-era layout: the note plaintext gains the 32-byte asset base and replaces the in-band 512-byte memo with a 32-byte memo key K^{memo} (ZIP-231), the memo chunks themselves moving to a transaction-level memo bundle (fields `nMemoChunks`, `pruned`, `vMemoChunks`, present iff `fAllPruned` = 0) whose pruned entries are replaced by their digests (ZIP-230, Transaction Format; §6.3). The v6 note plaintext is

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}},$$

with `asset_base` = $\text{LEBS2OSP}_{\ell_p}(\text{repr}_p(\text{AssetBase}))$, and the lead byte mandatory for all v6 note plaintexts (ZIP-230, note plaintext encoding). Upstream, the lead byte is the unassigned placeholder `{LEADBYTE}`: the original assignment 0x03 was taken by ZIP-2005, and no replacement is assigned (§6.6). For the same K^{memo} reason the `Sapling OutputDescriptionV6` shrinks `encCiphertext` to 100 bytes (ZIP-230, Sapling Output Description). An `AssetBurn` entry is 40 bytes: `AssetBase` (32, the encoding of `AssetBase`^{Orchard}) and `valueBurn` (uint64), consensus-checked non-zero and less than `MAX_BURN_VALUE` (ZIP-230, OrchardZSA Action Group Description; §6.4).

Issuance bundle. The issuance component carries five fields: `issuerLength` (compactSize), `issuer`, `nIssueActions`, `vIssueActions`, and `issueAuthSig`. The signature is present iff `nIssueActions` > 0; the first four fields are always present, and if `nIssueActions` = 0 then `issuerLength` MUST be 0 — an empty issuance bundle is exactly two zero compactSize values (ZIP-230, Transaction Format). An `IssueAction` carries `assetDescHash` (32 bytes), `nNotes` and `vNotes` at 115 bytes per `IssueNoteDescription`, and `flagsIssuance` with `finalize` in the LSB; `nNotes` may be zero, so a finalize-only action is expressible. An `IssueNoteDescription` is recipient (43 bytes, the raw Orchard payment-address encoding), `value` (uint64), `rho` (32), `rseed` (32). The `IssueAuthSignature` is `sizeSighashInfo` and `sighashInfo`, then `sizeSignature` and a signature “preceded by a 0x00 byte indicating a BIP 340 signature” (ZIP-230, issuance field encodings) — the same 0x00 scheme byte that prefixes the issuer key encoding (ZIP-227, Issuance Authorization Signature Scheme).

Versioned signatures. The `SaplingSignature` and `OrchardSignature` types prepend a compactSize-prefixed `sighashInfo` to the 64-byte RedJubjub or RedPallas signature (ZIP-230, signature encodings). ZIP-246 defines the versioning:

$$\text{sighashInfo} := [\text{sighashVersion}] \parallel \text{associatedData},$$

where version 0 is the “commit to all effecting data” algorithm; for v6, only version 0 is defined, with empty `associatedData`, for all four of Transparent, Sapling, Orchard, and Issuance (ZIP-246, Sighash versioning). The machinery buys the ability to introduce — and, critically, to retire — sighash algorithms per transaction version without a new format.

Coinbase. For coinbase transactions the `enableSpendsOrchard` and `enableZSAs` bits MUST be zero (ZIP-230, consensus rules), and ZIP-235 requires all coinbase transactions to be v6 from its activation, scheduled for NU7 (ZIP-230, Implications for Mining).

6.3 Digests: txid, sighash, and authorizing data (ZIP-246)

ZIP-246 extends the ZIP-244 digest tree — constructed for vanilla Orchard in the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature” — from four top-level branches to six:

$$\text{txid} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{transp}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}} \parallel d_{\text{issue}} \parallel d_{\text{memo}}),$$

the nodes T.1–T.6 of the ZIP, each itself a personalized BLAKE2b-256 (ZIP-246, TxId Digest). New relative to ZIP-244 are the fee and burn fields in the header digest, the Sapling output digests, the entire Orchard Action-Group subtree, the issuance subtree, and the memo subtree. The header digest T.1 is

$$d_{\text{hdr}} := \text{BLAKE2b-256}(\text{ZTxIdHeadersHash}, \text{version} \parallel \text{versionGroupId} \parallel \text{branchid} \parallel \text{lockTime} \parallel \text{expiryHeight} \parallel \text{LE}_{64}(\text{fee}) \parallel \text{LE}_{64}(\text{burnAmount}))$$

(ZIP-246, T.1). The Orchard branch T.4 hashes the concatenated per-group digests and the balance, $d_{\text{Orch}} = \text{BLAKE2b-256}(\text{ZTxIdOrchardHash}, d_{\text{AGs}} \parallel \text{LE}_{64}(v_{\text{balance}}^{\text{Orchard}}))$, with a defined empty case; each group digest (T.4a) covers a compact digest (nf, cmx, ephemeralKey, encCiphertext), a non-compact digest (cv, rk, outCiphertext), flagsOrchard, anchorOrchard, nAGExpiryHeight, and a burn digest over the (AssetBase, valueBurn) tuples (ZIP-246, T.4a). The issuance branch T.5 hashes issuerLength $\parallel$ issuer and a per-action digest (per action: assetDescHash, a per-note digest over (recipient, value, ρ , rseed), flagsIssuance), each level with a defined empty case (ZIP-246, T.5). The memo branch T.6 hashes a nonce and the concatenation of per-chunk digests; the v6 pruned memo bundle stores exactly these memo_chunk_digest / memo_digest values in place of pruned chunks, so pruning preserves the txid (ZIP-246, T.6; ZIP-230, Transaction Format). Table 5 collects the personalization strings; the implementation defines the same constants (§6.6).

Node	Personalization	Content
T.1	ZTxIdHeadersHash	header, incl. fee and burn amount
T.3b.i	ZTxId6SOutC_Hash	Sapling v6 outputs, compact
T.3b.ii	ZTxId6SOutN_Hash	Sapling v6 outputs, non-compact
T.4	ZTxIdOrchardHash	Action-Group digests, balance
T.4a	ZTxIdOrcActGHash	one Action Group
T.4a.i	ZTxId60ActC_Hash	nf, cmx, ephemeralKey, encCiphertext
T.4a.ii	ZTxId60ActN_Hash	cv, rk, outCiphertext
T.4a.vi	ZTxIdOrcBurnHash	burn tuples
T.5	ZTxIdSAIssueHash	issuer, issue-actions digest
—	ZTxIdIssuActHash	per action: desc. hash, notes, flags
—	ZTxIdIacNoteHash	per note: recipient, value, ρ , rseed
T.6	ZTxIdMemo___Hash	nonce, chunks digest
—	ZTxIdMemoCksHash, ZTxIdMemoCk_Hash	memo chunk digests
A.1	ZTxAuthTransHash	scripts, now incl. per-input sighashInfo
A.3	ZTxAuthOrchaHash	group auth digests, bindingSigOrchard
A.3a	ZTxAuthOrcAGHash	proofsOrchard, spend-auth digest
A.3a.ii	ZTxAuthOrSASHash	spendAuthSig encodings
A.4	ZTxAuthZSAOrHash	issueAuthSig encoding

Table 5: ZIP-246 digest personalizations (ZIP-246, TxId Digest and Authorizing Data Commitment). Unnumbered rows are children whose labels the ZIP text assigns inconsistently; see the closing remark of this subsection.

Signature digest. The sighash reuses the tree with the transparent branch replaced by an input-specific variant:

$$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{transp}}^S \parallel d_{\text{Sap}} \parallel d_{\text{Orch}} \parallel d_{\text{issue}} \parallel d_{\text{memo}}),$$

produced per transparent input, per Sapling input, per OrchardZSA Action, and — new in v6 — per Issue Action; the Sapling, Orchard, issuance, and memo branches are identical to their txid-side counterparts, so the issuance bundle and the memo bundle enter every sighash (ZIP-246, Signature Digest). On this digest ZIP-227 hangs its authorization rule: the SigHash is the §4.10 SIGHASH transaction hash with the ZIP-226 modifications, computed with SIGHASH_ALL, and the issuance authorization signature MUST satisfy $\text{IssueAuthSig.Validate}(\text{ik}, \text{SigHash}, \text{issueAuthSig}) = 1$, where ik is decoded from the `issuer` field and its encoding is required to begin with `0x00`, the BIP-340 scheme byte (ZIP-227, Specification: Consensus Rule Changes).

Authorizing-data commitment. The commitment to authorizing data follows the same pattern: A.1 covers transparent scripts together with the new per-input `TransparentSighashInfo`; A.2 keeps the ZIP-244 structure with field encodings altered by `sighashInfo`; A.3 commits to the per-group proofs and spend-authorization signatures and to `bindingSigOrchard`; and a new A.4 commits to the `issueAuthSig` field encoding, with a defined empty case (ZIP-246, Authorizing Data Commitment).

Remark 6.1 (The reference text is itself unfinished). ZIP-246 is internally incomplete even as withdrawn text: the authorizing-data section carries “TODO: Update the Authorizing Data Commitment below for the `sighashInfo` changes to the tx format,” Rationale and Reference implementation are TBD, and the subtree labels are inconsistent — `issue_actions_digest` is listed as T.5c but headed T.5a with children T.5a.i, and the burn-digest children are labelled T.4b.i/T.4b.ii under node T.4a.vi (`zip-0246.rst`). Anyone respecifying the format inherits these defects as open editorial work; the implementation, not the ZIP, is currently the more precise artifact.

6.4 Transaction-level consensus rules

One Action Group. Although the format is a vector of Action Groups, ZIP-227 collapses it for OrchardZSA: per transaction, `nActionGroupsOrchard` MUST be 0 or 1, and `nAGExpiryHeight` MUST be 0 (ZIP-227, Specification: Consensus Rule Changes). The vector form is used by draft ZIP-228’s proposed ZSA swaps, which would relax the one-group rule for swap bundles; it is not active in the core ZIP-227 rules.

Issuance requires an Action Group. A transaction with an issuance bundle MUST also contain at least one OrchardZSA Action Group (ZIP-227, Specification: Consensus Rule Changes). The rule is load-bearing twice over. First, issue-note ρ derivation consumes the first Action’s nullifier: each issued note’s ρ is computed by the function ZIP-227 names `DeriveIssuedRho` — constructed in §3.3 — keyed by $\text{nf}_{0,0}$, the nullifier of the input note of the first Action in the first Action Group, over the Issue Action and note indices (ZIP-227, Computation of ρ), which gives every issued note a globally unique ρ because nullifiers are unique. Second, without a spend in the transaction the issuance-only SIGHASH would be replayable; the mandatory Action Group ties the issuance signature to a nullifier that consensus consumes (ZIP-227, Rationale).

Global issuance state. Nodes maintain a map `issued_assets` from asset bases to triples (`balance`, `final`, `note_ref`), chained empty map $\rightarrow$ per-block $\rightarrow$ per-transaction. Burns decrement `balance`, requiring `balance  $\geq v$` , before the transaction’s issuance bundle is processed; issuance requires `final  $\neq 1$` , increments `balance` subject to the overflow bound $\text{MAX_ISSUE} = 2^{64} - 1$, and sets `final` when `finalize = 1`. The first issuance of an asset MUST include a *reference note*: value 0, paid to the default address of the all-zero Orchard spending key (ZIP-227, Global Issuance State and Reference Notes). The 43-byte raw encoding of that address is fixed in the ZIP and reproduced as a constant in the implementation (§6.6).

Burn rules and the extended balance equation. For every $(\text{AssetBase}, v)$ in `assetBurn`: $\text{AssetBase} \neq V^{\text{Orchard}}$ (ZEC cannot be burned this way), $0 < v \leq \text{MAX_BURN_VALUE}$, and no two

entries share an `AssetBase`; the bound $\text{MAX_BURN_VALUE} := 2^{63} - 1$ is chosen for compatibility with signed 64-bit value-balance fields (ZIP-226, Burn Mechanism). Burns enter value balance through the binding key. Where vanilla Orchard computes `bvk` by subtracting the committed declared balance from the sum of net value commitments (the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”), ZIP-226 subtracts one zero-randomness commitment per burn entry as well:

$$\begin{aligned} \text{bvk} := & \left(\sum_{i=1}^n \text{cv}^{\text{net},i} \right) - \text{ValueCommit}_0^{\text{OrchardZSA}}(v^{\text{Orchard}}, v_{\text{balance}}^{\text{Orchard}}) \\ & - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v), \end{aligned}$$

and the prover signs the SIGHASH with $\text{bsk} = \sum_{i=1}^n \text{rcv}_i$ (ZIP-226, Burn Mechanism). The signature verifies only if every non-ZEC asset nets to exactly its declared burns — the same Pedersen-homomorphism argument as for ZEC, run per asset base.

Commitment-tree order. Note commitments enter the global tree in a fixed order: first, per Action Group, the commitments of the new OrchardZSA notes of each Action; then, per Issue Action, the issue-note commitments (ZIP-227, Specification: Consensus Rule Changes). Wallets and nodes must agree on this order to agree on note positions.

6.5 Fees (ZIP-317): specified in the fork, removed upstream

The fee treatment chains through three documents: ZIP-226 (Transaction Fees) defers to ZIP-317 “as described in ZIP 227”; ZIP-227 (Changes to ZIP 317) states that the conventional fee is altered for the presence of issuance actions and the creation of new Custom Assets, pointing at ZIP-317’s Fee calculation section. Fees remain payable in ZEC only (ZIP-227, rationale). All variants share the conventional-fee form documented in the *Wallet Guide*, §“Fees (ZIP-317)”:

$$\text{conventionalFee} := \text{marginalFee} \cdot \max(\text{graceActions}, \text{logicalActions}),$$

with $\text{marginalFee} = 5000$ zatoshis and $\text{graceActions} = 2$ (ZIP-317, Fee calculation).

The pointer now dangles upstream: ZIP-317 (Last-Updated 2026-06-26) removed the ZSA contributions — its Change History records “Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)” — leaving Revision 0 (active base), Revision 1 (Ironwood-pool Actions, NU6.3), and Revision 2 (memo bundles, draft) (ZIP-317, Revisions and Change History). The ZSA treatment survives in the QED-it fork copy (@ fd71419): a requirement that “users should be discouraged from issuing new ‘garbage’ Custom Assets,” the parameter $\text{creationCost} = 100$ logical actions, and two contributions summed into `logicalActions` alongside the base terms of the deployed rule:

$$\begin{aligned} \text{contribution}_{\text{ZSAIssuance}} &:= n\text{ZSAIssueNotes}, \\ \text{contribution}_{\text{ZSACreation}} &:= \text{creationCost} \cdot n\text{AssetCreations}, \end{aligned}$$

where $n\text{ZSAIssueNotes}$ counts issue-note outputs and $n\text{AssetCreations}$ counts Custom Assets newly added to the global issuance state (fork ZIP-317, Fee calculation). The rationale is state rent paid once: each new asset adds an `issued_assets` row tracked in perpetuity, while subsequent issuance, finalization, and burn only mutate the row. The implementation matches the fork text: `zcash_primitives/transaction/fees/zip317.rs` defines `CREATION_COST = 100` behind `cfg(zcash_unstable = "nu7")` and adds $n\text{ZSAIssueNotes} + 100 \cdot n\text{AssetCreations}$ to the standard logical-action count. As with the deployed rule (the *Wallet Guide*, §“The deployed fee rule”), paying the conventional fee is not a consensus requirement in any ZIP-317 variant. Classification: the ZSA fee rule is *fork-specified*; whether it returns upstream as a ZIP-317 revision, as Ironwood’s did, is undecided.

6.6 Implementation status at the pinned commits

Constants and serialization. All NU7 code in the `librustzcash` fork sits behind the configuration gate `cfg(zcash_unstable = "nu7")`. The constants are `V6_TX_VERSION = 6`, `V6_VERSION_GROUP_ID = 0x77777777`, `BranchId::Nu7 = 0x77190AD8`, with NU7 activation `None` on Mainnet, 4,000,000 on Testnet, 1 on Regtest (`zcash_protocol`, `constants.rs` and `consensus.rs`). The v6 reader parses the header fragment, one `vSighashInfo` per transparent input (required to equal the version-0 encoding), the Sapling v6 bundle, the Orchard v6 bundle, and the issuance bundle (`zcash_primitives`, `transaction/mod.rs`). The Orchard reader rejects more than one Action Group (“A V6 transaction must contain at most one action group”), requires at least one Action, and rejects a nonzero `nAGExpiryHeight`; the writer emits exactly one group with `nAGExpiryHeight = 0` (`transaction/components/orchard.rs`) — the ZIP-227 consensus rules hard-coded at the serialization layer. Issuance-bundle serialization matches ZIP-230, including the two-zero-compactSize empty bundle and rejection of the inconsistent issuer/action combinations (`transaction/components/issuance.rs`). The sighash implementation extends the v5 hash with the issuance digest for ZSA-capable versions, with sighash-info constants `ORCHARD_SIGHASH_INFO_V0 = ISSUE_SIGHASH_INFO_V0 = [0x00]` (`sighash_v6.rs`, `sighash_versioning.rs`). The orchard fork defines the ZIP-246 personalizations of Table 5 (`bundle/commitments.rs`, `bundle/commitments/issuance.rs`), asserting the 0x00 BIP-340 byte when hashing the 65-byte issuance signature encoding.

Verification code. Function `derive_bvk_raw` implements the extended balance equation of §6.4 verbatim (orchard fork, `bundle.rs`); `validate_bundle_burn` enforces the ZIP-226 burn rules — non-ZEC asset, non-zero amount, amount $\leq 2^{63} - 1$, no duplicates — plus presence in and sufficient supply under the global issuance state (`bundle/burn_validation.rs`); and `verify_issue_bundle` checks the BIP-340 signature over the 32-byte sighash, the per-note ρ against `DerivelssuedRho`, non-finalization, u64-checked supply addition, and the reference note on first issuance, whose 43-byte `RAW_REFERENCE_RECIPIENT` constant equals the array in ZIP-227 (`issuance.rs`, `constants/reference_keys.rs`). The 0x84 domain separator for issued-note ρ lives in the QED-it `zcash_spec` fork (rev d5e8426). Proof sizes record the circuit cost: the ZSA aggregated proof has base size 2848 bytes and 2272 bytes per Action, against a vanilla base of 2720 (`primitives/orchard_primitives_zsa.rs`, `orchard_primitives_vanilla.rs`).

Divergences from the withdrawn ZIP text. Four are material; each is an explicit interim measure or a pending reassignment, not a bug, but each blocks byte-compatibility with ZIP-230/246 as written.

1. *No explicit fee field.* The v6 reader serializes no fee, and `zip233Amount` only behind the `zip-233` cargo feature; the header `txid` digest correspondingly omits ZIP-246’s T.1f fee field, with the comment “TODO: Factor this out ... when implementing ZIP 246 in full” (`transaction/mod.rs`, `txid.rs`).
2. *No memo bundles.* The forks implement the pre-ZIP-231 layout: OrchardZSA notes carry a full 512-byte memo, giving a 612-byte `encCiphertext` with an 84-byte compact prefix (52 vanilla bytes plus the 32-byte asset base), and Sapling v6 outputs are read in the v5 580-byte format (`primitives/zcash_note_encryption_domain.rs`, `transaction/components/sapling.rs`). No memo fields and no T.6 are implemented; instead the action-group `txid` digest inserts a non-spec memos digest (personalization `ZTxId0rcActMHash`) between the compact and non-compact digests and hashes only the 84-byte compact slice into the compact digest, with TODOs “Remove once new Memo Bundles are implemented (ZIP-231)” (`primitives/orchard_primitives_zsa.rs`).
3. *Lead byte.* The implementation hard-codes `NOTE_VERSION_BYTE_V3 = 0x03`; upstream ZIP-230 withdrew that claim after ZIP-2005 took 0x03, leaving the ZSA lead byte an unassigned

placeholder. The fork’s choice predates upstream ZIP-2005 assigning 0x03 to Ironwood notes and directing future proposals to pick a new value; the collision is fork-only context, with no format ambiguity on Zcash mainnet. Reassignment is a hard prerequisite for any deployment: the lead byte selects the note-plaintext parser.

4. *Split-note ψ^{nf}* . Upstream ZIP-226 now derives the split-note nullifier randomizer as $\psi^{\text{nf}} = \text{ToBase}(\text{PRFexpand}_{\text{rseed}_{\text{nf}}}(\text{0x0A} \parallel \rho^{\text{old}}))$ with rseed_{nf} sampled uniformly at random, the first-byte domain 0x0A “reserved by ZIP 2005 for this use” (ZIP-226, split-note nullifier); the fork at cf801a5d realizes the older fork text (uniformly random ψ^{nf}) via a separate `rseed_split_note` under the ordinary ψ domain 0x09 (`note.rs`, `circuit.rs`).

Audit scope. Public reporting supports both standing claims, at scope: Least Authority’s *OrchardZSA Protocol Security Audit Report* (ZCG-commissioned, Updated Final 30 January 2025) reviewed the QED-it orchard zsa1-vs-main diff (PR QED-it/orchard#7, the ZSA circuit extensions) and the QED-it halo2 changes (PR QED-it/halo2#18) at revisions a7c02d22/01e85a5f and identified no issues; ZecHub *Shielded News* Vol. 61 (2025-12-17) reports a feature-complete ZSA build on QED-it’s public Zebra testnet; both sources are pinned in §1.2. Neither covers `librustzcash-zsa` or the v6 carrier layer, and the audited revisions predate cf801a5d. Two facts bound what the past audit covers: the audited revisions predate both the NU6.2 Orchard circuit remediation (ZIP-257) — and the ZSA circuit is a fork of the Orchard circuit — and the ZIP-2005-relative rebasing of upstream ZIP-226. A mainnet path implies re-audit against both.

6.7 Wallet integration

ZIP-230 and ZIP-226 impose two obligations. First, all wallets SHOULD switch to sending only v6 transactions once permitted: a v5 transaction reveals that all its Orchard notes are ZEC, and only v6 OrchardZSA outputs are quantum-recoverable (ZIP-230, Implications for Wallets; ZIP-226, Backwards Compatibility with ZEC Notes). Second, wallets MUST support parsing and trial-decrypting v6 outputs — the new lead byte and the memo-bundle changes — by activation, or rescan once support lands: funds sent to an un-upgraded wallet are temporarily inaccessible, because the new lead byte selects different rcm/ψ derivations and a v5-decryption wallet rejects the note (ZIP-230, Implications for Wallets). Payment requests have a specified extension path: ZIP-321’s `req-asset` parameter carries an Asset Identifier and amount (the *Wallet Guide*, §“Custom assets: req-asset (specified, not deployed)”).

Multi-party and hardware signing is the open end. No PCZT structure exists for OrchardZSA bundles or issuance bundles: the `librustzcash` fork’s transaction extractor panics with “PCZT support for ZSA is not implemented” when handed an OrchardZSA bundle, the orchard fork’s PCZT module is written against the vanilla flavor only, and the draft PCZT v2 targets ZIP-229’s Ironwood v6, not ZSA (the *Wallet Guide*, §“Partially created transactions (PCZT)” and §“PCZT v2: deferred anchors and the Ironwood bundle”). Classification: *open* — hardware-wallet and multi-party signing for ZSAs is unspecified in any draft.

6.8 Outlook

ZIP-226 and ZIP-227 remain Status: Draft. ZIP-226’s Deployment section reads “The Zcash Shielded Assets protocol is scheduled to be deployed in Network Upgrade 7 (NU7). This ZIP currently assumes that this will be the case”; ZIP-227’s Deployment is TBD and its test-vector and reference-implementation entries are “LINK TBD,” with ZIP-226 naming the QED-it forks and QED-it/zcash-test-vectors as reference artifacts (ZIP-226/227, Deployment). The deployment vehicle is thinner still: the NU7 deployment draft is unnumbered (`draft-arya-deploy-nu7`, Status: Draft, replacing the withdrawn ZIP-254), fixes `CONSENSUS_BRANCH_ID = 0x77190AD8` and minimum protocol versions (170150 Testnet, 170160 Mainnet) with activation heights TBD, and lists *no* ZIPs. Those version numbers now conflict with ZIP-204’s NU7 assignment of 170180 and 170190, so the

deployment draft needs an editorial update. The repository’s NU7 candidate list (ZIPs 218, 230, 231, 233, 234, 235, 2002, 2003, plus a possible ZIP-317 update) does not include ZIP-226/227, still names the withdrawn ZIP-230, and states that “no decision has been made on whether to include each of these ZIPs” (zcash/zips README, NU7 Candidate ZIPs). Governance sentiment is split: in the February 2026 NU7 sentiment polls, ZCAP supported ZSAs at 70.4% while the coinholder poll ran 98.6% against; the Zcash Foundation did not recommend ZSAs for NU7, writing that “before any decision is made, we need to understand why coinholders oppose ZSAs so strongly” (zfnd.org, “NU7 Polling Results: What We Heard and Where We Go From Here”, 2026-02-23, <https://zfnd.org/nu7-polling-results-what-we-heard-and-where-we-go-from-here/>).

Between the pinned implementation and any deployment stand three respecification problems, none of which is per-object protocol work.

1. *A carrier.* Transaction version 6 now belongs to NU6.3’s Ironwood format (ZIP-229), whose non-requirements exclude the ZSA machinery (§6.1). The designated successor for what ZIP-230/246 deferred — an extensible transaction format — is ZIP-248, which exists only as open PR [zcash/zips#1156](#). The entire ZSA and memo-bundle format stack therefore has no merged specification; if ZSAs are selected for NU7, the byte format of §6.2 must be respecified under new numbers, and §6.3 re-anchored to whatever digest ZIP results.
2. *A base protocol.* Upstream ZIP-226 is now “defined relative to the Zcash protocol with the changes specified in ZIP 2005 applied,” with ZIP-2005 (Ironwood Quantum Recoverability) “expected to deploy before any ZSA activation” (ZIP-226, header notes). NU6.3 (ZIP-258, testnet activation 4,134,000, Mainnet 3,428,143) introduces the Ironwood pool and restricts transfers into the Orchard pool (ZIP-2006, currently a Reserved stub). No ZIP specifies whether OrchardZSA then targets the restricted Orchard pool or is rebased onto Ironwood; the audited revisions pre-date the entire NU6.2/NU6.3 pivot. Classification: *open*.
3. *Reconvergence of spec and code.* The divergence list of §6.6 — fee field, memo bundles, lead byte, ψ^{nf} — plus the orphaned fee treatment of §6.5 must land on one side or the other: either the respecified ZIPs adopt what the code does, or the code moves. Every item is known and mechanical; none is done.

The summary judgment this volume can defend: the ZSA transaction layer is *designed and implemented, but currently unspecified* in the sense this series requires — there is no merged normative text a mainnet node could be held to. The byte-level facts of this section are fixed by withdrawn ZIP text and pinned fork code, and they will remain the diff base for the respecification; the open items are enumerated above and in the scope section, and every one of them is legible, which is more than can be said for most protocols at this stage.

Part XII

Tachyon Guide: The Tachyon scaling architecture, at design stage: what is specified, what is designed, what is open

Abstract

This volume of *The Zcash Arboretum* documents a protocol that does not yet exist. Tachyon is the proposed rearchitecture of Zcash payments around proof-carrying wallet state: the wallet folds each block’s effects into a recursive proof, so synchronisation ceases to scale with chain length, and the chain ceases to carry the wallet’s burdens. Because the design is in motion, this volume’s organising device is a strict classification: every claim is *specified*, *designed but unspecified*, or an *open problem*. The strongest chapter is the last kind. The reader should expect this volume to age faster than its siblings and to be revised as the design lands.

1 Scope, sources, and method

Project Tachyon is a proposed redesign of Zcash’s shielded protocol around three ideas: wallet state becomes proof-carrying data, so a wallet proves the validity and unspentness of its own notes and a verifier checks a proof rather than replaying history; nullifiers *evolve* per epoch, so consensus checks duplicates over a bounded window and permanently prunes older state; and untrusted *oblivious synchronization services* keep wallets’ proofs current without learning their transaction details (Bowe, “Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02; Bowe–Miers, eprint 2025/2031, 2025-11-03). The intended proof layer is Ragu, a recursive-proof (PCD) framework over the Pasta cycle with no trusted setup (tachyon-zcash/ragu @ 830bbcda). None of this is deployed; none of it is specified at consensus rigor.

1.1 Why a design-stage volume

The sibling volumes document a deployed protocol: every claim in the *Ironwood Guide* cites shipped code and a final specification. Tachyon has neither — no protocol specification, no merged ZIP, no code on any network. It nevertheless warrants a volume now, for two reasons. First, its standing: of all proposals in the February 2026 NU7 sentiment poll, Tachyon drew the most unambiguous support (§1.4), so it is the direction Zcash’s shielded layer is most likely to take, and the design deserves scrutiny *before* it hardens into consensus rules. Second, its volatility: the design is single-committer, actively churning, and self-admittedly stale in places (§1.3); fixing its current state against commit-pinned sources creates the baseline any later normative work can be diffed against. The cost of writing now is stated plainly: this volume can assert nothing normatively, and its strongest chapter is the one on open problems.

1.2 The three-bin method

Design-stage material invites a failure mode the deployed-protocol volumes never face: prose that reads as specification but rests on a blog post. We guard against it by classifying every claim in this volume into exactly one of three bins, and keeping the bin visible in the prose.

Definition 1.1 (Three-bin classification). A claim about Tachyon is exactly one of:

- (i) *specified* — a normative artifact pins it down (a merged ZIP, the protocol specification, or

deployed code); we cite the artifact, with file and commit;

- (ii) *designed but unspecified* — an informal source (design book, eprint note, blog post, forum post, issue) describes a concrete mechanism, but no specification fixes it; we cite the source and its date (plus commit for repositories) and state what a specification would still need to pin down;
- (iii) *an open problem* — no source, formal or informal, resolves it; where the designers themselves acknowledge the gap, we quote them.

The bins are this volume’s organizing device. Each subsequent section groups its material by bin — §4 carries the specified bin exhaustively, §5.1 and §5.2 the lower two — or closes each construction with a *Classification* paragraph naming its bin; a claim appearing outside such a block names its bin inline. Claims about deployed or extant code cite the crate and file; claims about design carry a date, because the design has already pivoted once (§1.3) and undated statements from the wrong era are actively misleading.

1.3 The source landscape

Table 1 lists the primary sources, each pinned to a commit or date. All repository facts below were verified firsthand at the stated commits; web sources were fetched 2026-07-08.

Artifact	Pin / date	Rigor
eprint 2025/2031 (Bowe–Miers)	rev. 2025-11-03	8-page note; no theorems
tachyon-zcash/tachyon (book, crate)	26fdc165, 2026-07-07	design book; crypto stubbed
tachyon-zcash/ragu	830bbcd, 2026-07-05	pre-release code; unaudited
tachyon-zcash/zips PRs #1–#4	opened 2026-06-02; open	drafts, not upstreamed
zcash/zips (upstream)	c6b70358, 2026-07-05	zero Tachyon ZIPs
Blog, forum t/50789, ZK Podcast 388	2025-04-02 to 2026-01-31	designer prose

Table 1: Primary Tachyon sources, pinned. Verification date 2026-07-08.

Current-head boundary (2026-08-30). The detailed design below remains a reproducible snapshot of the commits in Table 1, not a reconstruction of later main branches. For current status, we also checked tachyon fd022416 (2026-08-27), ragu f4f6315b (2026-08-22), and upstream zips ad30e59. The Tachyon README still lists note commitments, nullifier derivation, and proof creation, merging, verification, and stamp compression as stubbed. Ragu has added extensive formal-verification and fuzzing material and published crate documentation, but still labels itself unaudited, under heavy development, and unsuitable for production. Upstream zips still contains no core Tachyon ZIP. Thus the status classification has not changed; later implementation and book details are not silently projected backward onto the pinned construction.

The eprint note. The canonical publication is Bowe and Miers, “A Note on Notes: Towards Scalable Anonymous Payments via Evolving Nullifiers and Oblivious Synchronization” (eprint 2025/2031; received 2025-11-02, revised 2025-11-03). It is a self-described “short note” of eight pages: it fixes the two central ideas and their motivation, states the fatal risks in the authors’ own words, and contains zero theorems and no formal security definitions. It anchors bin (ii) for the core concepts and, unusually, much of bin (iii): its frankest passages are the acknowledged unsolved problems.

The project book and crate. The repository tachyon-zcash/tachyon @ 26fdc165 (2026-07-07) carries the most concrete design material: an mdBook elaborating nullifier derivation, the proof tree, the anchor chain, aggregation, and a ZIP dependency map, beside the zcash_tachyon crate. Three

caveats govern every citation of it. First, it is a single-committer artifact (`git shortlog` lists only Tal Derei) with no independent publication of its post-eprint machinery. Second, the crate does not implement the load-bearing cryptography: its README lists note commitments, nullifier derivation, and proof creation, merging, and verification as “Stubbed (pending Ragu PCD and Poseidon integration)” (README.md line 49 @ 26fdc165). Third, the book is self-admittedly stale: issue #121 (2026-05-26) records a pivot in the payment design — “Since we’re using PIR to make repeated payments flows without OOB, there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows” — so every statement from the out-of-band era (the 2025 blog posts, the book’s key chapter) must be dated pre-2026-05-26, and we date them throughout.

The ZIP landscape. The project’s ZIP program exists at two rigors, and the split is the clearest single measure of Tachyon’s maturity. The five *core* ZIPs — Shielded Protocol, Bundle, Accumulator, Aggregator, Oblivious Synchronization (tracking issues #103–#107) — are empty stubs: each book file carries only a tracking link and blank section headings (book/src/zips/ @ 26fdc165, 216–245 bytes per stub). Four *peripheral* amendments — to ZIP 317 (fees), ZIP 209 (pool balance), ZIP 221 (history tree), and ZIP 248 (bundle registration) — exist as ZIP-0-conformant drafts, but only as PRs #1–#4 on the project’s own zcash/zips fork, open since 2026-06-02 and not upstreamed; the ZIP 248 they amend is itself unmerged upstream (open PR zcash/zips#1156, idle since 2026-05-26). Upstream zcash/zips @ c6b70358 contains zero Tachyon ZIPs: a repository-wide search finds only the ZIP editor list, one author affiliation in protocol/protocol.tex, and one footnote citing the founding blog post. The NU7 deployment draft (draft-arya-deploy-nu7.md, Status Draft, branch 0x77190AD8, activation heights TBD) schedules no Tachyon ZIP — nor any other.

Ragu. The proof layer lives in tachyon-zcash/ragu @ 830bbcd (2026-07-05): a working PCD pipeline over the Pasta cycle, at workspace version 0.0.0, with no release, no crates.io publication, no audit, and in-code security placeholders (the Fiat–Shamir domain tag is literally b“FIXME”; crates/ragu_pcd/src/lib.rs lines 50–51). We treat Ragu’s code state in its own section; here it matters only as a pinned source.

Informal sources. The remaining corpus is designer prose: Bowe’s blog (founding post 2025-04-02; “Tachyaction at a Distance” 2025-05-15; post-quantum post 2025-10-16, after which the index ends and the promised deep-dive posts never appeared), the forum megathread t/50789 (28 posts, 2025-04-02 to 2026-01-31), the project site (dateless roadmap), and the ZK Podcast 388 transcript (2026-01-21). These sources carry bin (ii) intent and bin (iii) admissions, never specification. We quote them with dates. The “two orders of magnitude” headline is the project’s own (tachyon.z.cash) with no published derivation; the circulating TPS estimates are analyst characterization, not project claims; no primary document derives either (§5.3).

1.4 Standing

Two institutional facts frame the volume. First, governance: in the Zcash Foundation’s February 2026 NU7 sentiment poll, Tachyon drew “universal or near-universal support across every group” — “the most unambiguous signal from the entire poll” — with the stated next step to “assess technical readiness and scope for inclusion in NU7” (zfnd.org, 2026-02-23). Support is not scheduling: as noted above, the NU7 deployment draft lists no ZIPs at all. Second, editorship: Sean Bowe and Tal Derei serve as ZIP editors “associated with Project Tachyon” (zip-0000.rst line 196 @ c6b70358), and the protocol specification’s author block carries the same affiliation (protocol/protocol.tex line 2953). The people who would shepherd Tachyon ZIPs through the process are the protocol’s designers — an institutional fact we record without inference.

1.5 What this volume can and cannot deliver

In bin (i) we can place only the standing environment: the poll outcome and editorships above, the adjacent network-upgrade context (NU6.2 and NU6.3 active, although the ZIP-258 document remains Draft), the out-of-band substrate ZIPs the design leans on, and Ragu’s code state at 830bbcd. Bin (ii) holds the entire core protocol — evolving nullifiers, dual-nullifier spends, the anchor chain, the proof tree, delegation, aggregation, the distinct shielded pool the notes inhabit — every part of it cited to the book at 26fdc165 as non-normative, dated, and in places already superseded. No byte format, consensus rule, security theorem, or performance figure appears in this volume as a project claim, because none exists at that rigor.

The strongest chapter is therefore the one on open problems, and this is not a concession but the point: the best-documented statements in the corpus are the designers’ own acknowledgments of what remains unsolved — timing correlation by synchronization services (“fatal to the entire approach”; eprint 2025/2031 §1.3), data availability after validator pruning (“unspendable, potentially permanently”; eprint 2025/2031 §1.4), and recoverability, whose entire current written resolution is one sentence in issue #121. The events that would obsolete parts of this volume are catalogued as revisit triggers in §5.4; we revisit at any of them.

2 The proof-carrying wallet

An Orchard wallet reconstructs its state from the public chain. To detect its notes it trial-decrypts every published Action from its birthday height onward, and to keep its authentication paths current it ingests every published commitment — its own and everyone else’s — into a pruned local tree (the *Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation” and §“Proof of ownership of some valid note: the commitment tree”; the deployed mechanism is librustzcash’s shardtree). Both costs scale with global chain traffic, not with the wallet’s own activity. Tachyon inverts the relationship: the wallet holds a *proof* of its own spendability — “this note entered the pool, and no nullifier of it has been published since” — and advances that proof by folding in each block’s deltas, retaining nothing else about the chain. The concept is Bowe’s founding post (“Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02); the publication of record is the Bowe–Miers note (eprint 2025/2031, 2025-11-02), eight pages with no theorems; the concrete elaboration is the project book (tachyon-zcash/tachyon, commit 26fdc165, 2026-07-07).

Nothing in this section is consensus-specified. Upstream zcash/zips carries no Tachyon ZIP (repository state at commit c6b70358, 2026-07-05), and the core “Tachyon Shielded Protocol” ZIP is an empty stub (tachyon-zcash/tachyon issue #103). We therefore tag every claim with its bin: *specified* (a citable artifact at spec or code rigor), *designed but unspecified* (an informal source, together with what a specification would still need to pin down), or *open problem*.

2.1 Wallet state as an accumulation instance

The state object is the *spendable*, the triple

$$(cm, nf_e, \text{anchor}),$$

carried as a proof header: the note’s commitment, its nullifier for the current epoch e , and a running commitment to the pool state up to which absence has been proven (tachyon book, src/proof-tree.md and src/nullifiers.md at 26fdc165). A Tachyon note has a distinct nullifier per epoch: from the note’s trapdoor ψ and the wallet’s nullifier key nk a per-note master key $mk := \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, nk)$ roots a GGM tree whose leaf at epoch e yields nf_e , and the commitment $cm := \text{Poseidon}_{\text{Tachyon-CmDerive}}(rcm, pk, v, \psi)$ digests ψ and pk (which pins nk), so a note’s entire nullifier sequence is frozen when its commitment enters the pool. Construction 3.4 states the derivation in full — the climb and leaf domains, and the working depth and arity the crate pins (§3.4) while the book leaves them symbolic. The proof behind the header certifies a single

sentence: the note whose commitment is cm was minted — its creation *stamp*, the proof object recording its transaction’s published commitments and nullifiers (Definition 3.1), is folded into the pool state — and none of its nullifiers, from the creation epoch through e , appears in the pool up to anchor.

The fold advances this state without the wallet touching the chain. A synchronisation service — holding only an opaque window of the note’s future nullifiers, never the note, cm , ψ , or mk — assembles an *Unspent* segment: `UnspentSeed` proves one wallet-supplied nullifier absent from one stamp’s tachygram set (a *tachygram* is a 32-byte field element representing a note commitment or a nullifier, the two indistinguishable to consensus), `EmptyBlockUnspentSeed` covers empty blocks, `UnspentFuse` composes contiguous segments, and `UnspentEpochFuse` crosses an epoch boundary, splicing the completed epoch’s nullifier into the segment’s elapsed history. The wallet then binds and folds: `VerifyUnspent` proves the segment’s crossed nullifiers and its tip are genuine GGM leaves of the note identified by cm , and `SpendableLift` consumes the prior spendable together with the verified segment, checks commitment equality, nullifier continuity, and anchor adjacency, and emits the advanced spendable (book, `src/proof-tree.md` at 26fdc165). Each step consumes at most two prior proofs and emits one — proof-carrying data over the accumulation-style recursion of the Halo lineage (the *Halo 2 Guide*, §“Recursive proof composition and accumulation schemes”; the Halo-specific device in §“Accumulation and the Halo trick”).

The intended proof system is Ragu, a Rust PCD framework implementing a modified Halo recursive SNARK with no trusted setup over the Pasta cycle. A working pipeline exists in code — application registration, seed (leaf proofs), fuse (binary merge), `rerandomize`, `verify`, with passing multi-node PCD-tree integration tests (`crates/ragu_pcd/src/{lib.rs,fuse/mod.rs,verify.rs}` at commit 830bbcd, 2026-07-05). The same code is explicit that it is pre-release: the `Proof` struct carries full polynomials and verification is linear-time (no inner-product opening argument or decider exists anywhere in the workspace), the Fiat-Shamir domain tag is the literal `b"FIXME"` (`crates/ragu_pcd/src/lib.rs:50--51`), and registry binding carries an in-code `FIXME(security)`. Downstream, the `zcash_tachyon` crate registers all nineteen proof-tree steps but compiles only against Ragu’s mock feature, with note commitments, nullifier derivation, and proof creation/merge/verification “Stubbed (pending Ragu PCD and Poseidon integration)” (`tachyon README.md` at 26fdc165).

Classification. The Ragu and `zcash_tachyon` code facts are *specified* in the only sense available before a spec: verifiable source at a pinned commit. The wallet-state design — the headers, the nineteen steps, the fold — is *designed but unspecified*: it exists in the project book and nowhere at ZIP rigor, and a specification would need to pin the header encodings, the per-step relations, the Poseidon instances and domain tags, the GGM depth and arity, and the epoch length. The book’s soundness section is prose — no formal definitions, theorems, or machine-checked statements, and the eprint contains none either — so recursive soundness of the tree, and whether Ragu’s linear-time verifier can be carried to a succinct decider at the sizes Tachyon needs, are *open problems* (`crates/ragu_pcd/src/verify.rs:122` records three missing verifier checks; the performance figures in the Ragu book are self-labelled rough estimates).

2.2 What the fold removes

Two cost centres of the deployed protocol disappear; both removals are properties of the *designed-but-unspecified* core above and inherit its bin except where noted.

Trial decryption of the chain. Nothing on today’s chain marks which Action pays a wallet, so detection is exhaustive: the wallet trial-decrypts every Orchard output from its birthday height, and restore-from-seed replays that scan over the entire chain since the birthday (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”). Under Tachyon

the wallet never scans. Detection and delivery move to the payment channel (§2.4), and the absence-tracking that synchronisation actually requires is delegated to a service that works only on the opaque nullifier windows of §2.1: the eprint asserts services are “fully oblivious to their clients’ transaction details” (eprint 2025/2031), and the book’s role table assigns the service the Unspent steps beside the note-free anchor-chain segments, and none that touch the note (src/proof-tree.md at 26fdc165). Groundwork exists at ZIP rigor: ZIP 2004, “Remove the dependency of consensus on note encryption” (Status: Draft at c6b70358), makes the consensus layer independent of the ciphertexts a delivery redesign would drop, though its ephemeralKey handling is an in-file TODO. No formal definition of “oblivious” exists — no threat model and no security definition — which we treat as an *open problem* (the trust model, §5.1.3; the timing-correlation attack the eprint calls potentially fatal, §5.2.1). The aggregate cost claim is likewise unproven: the designer’s estimate that total service effort is “asymptotically similar” to today’s validators is a forum statement (t/50789 #28, 2025-11-21) with no published complexity analysis, and no project benchmark exists — circulating throughput and size figures are analyst estimates (Messari; CoinDesk Research, 2026-06-30), not project figures.

Per-note witness maintenance. Today the wallet appends every published cmx to a pruned local shard tree, marks the positions of its own notes, stores a checkpoint per block, and rebuilds each authentication path on demand from retained subtree roots (*Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”; the design of librustzcash’s shardtree). Tachyon has no note commitment tree. Pool state is a hash chain (§2.3); membership of the note’s creation is proven once, when SpendableInit checks cm against the creation stamp’s tachygram set; and the lineage thereafter carries no path — there are no siblings to refresh as others append, because there is no tree (book, src/proof-tree.md, src/anchor.md at 26fdc165). What remains is the forward obligation: the wallet must keep proving absence up to a recent anchor before it can spend, work proportional to its own note count and elapsed epochs rather than to chain traffic, and delegable as above. *Designed but unspecified* throughout: the creation-stamp membership argument and the anchor-rooting rule exist only in the book.

2.3 Validator-side consequences

A node today maintains two unbounded structures: the append-only commitment tree, from which it recognises anchors, and the ever-growing nullifier set, against which it checks each Action’s nf_{old} — the one check that cannot sit inside a proof, because it concerns global history (*Ironwood Guide*, §“What a node verifies, without ever observing the note”). The Tachyon design replaces both with one running Poseidon hash chain and bounds the duplicate check to a recent window.

Each stamp accepted into a block advances the pool state by

$$\text{anchor}' := \text{Poseidon}_{\text{Tachyon-StampFld}}(\text{anchor}, x, y),$$

where (x, y) are the coordinates of the stamp’s tachygram-set commitment; an empty block advances it by $\text{Poseidon}_{\text{Tachyon-EmptyBlk}}(\text{anchor})$; and the crossing into epoch $e + 1$ by $\text{Poseidon}_{\text{Tachyon-EpochStp}}(\text{anchor}, e + 1)$, the only domain that absorbs the epoch index (book, src/anchor.md at 26fdc165). Anchors are the end-of-block states of this chain; a spendable or stamp proves continuity from one anchor to a later one rather than membership in a tree.

The double-spend check becomes a windowed duplicate check: consensus checks every tachygram of a published stamp for duplicates over every block of the current and the immediately preceding epoch, rejects on a hit (book, src/tachygrams.md at 26fdc165), and may permanently prune tachygrams older than that window (forum t/50789 #22, 2025-11-04; the book states no pruning rule). Two epochs suffice in the design because a stamp’s anchor may have been lifted within its epoch (so the whole anchor epoch is scanned, not merely blocks after the anchor), and because a spend publishes the note’s nullifiers for both the anchor epoch e and epoch $e + 1$ — the next-epoch nullifier published in e is exactly what a second spend of the same note would publish in $e + 1$ (book, src/tachygrams.md at 26fdc165; the dual publication itself is in src/nullifiers.md).

Classification. The window rule and prunability are *designed but unspecified*: the window rule appears in the book only, prunability only in a forum statement (t/50789 #22, 2025-11-04); the eprint states just that “validators must still check for duplicates”; no consensus ZIP drafts the rule. A specification would need the epoch length — the parameter the entire pruning model hinges on, which the project itself marks as requiring empirical analysis (tachyon-zcash/tachyon issue #79) — and the normative force of end-of-block anchoring, on which the book’s own source wavers between “may”, “should”, and “must” in an inline comment (src/anchor.md). The consequence of pruning is an *open problem*: what validators discard, wallets and services still need, and the eprint concedes that the sudden shutdown of syncing services “could render the funds of all their dependent users unspendable, potentially permanently” (eprint 2025/2031, §1.4); no data-availability or incentive design exists (§5.2.2).

2.4 Payment delivery

Removing detection-by-trial-decryption reopens the question the deployed protocol answers with in-band secret distribution (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”): how does the recipient learn of the note? The project’s answer changed in May 2026. We present both stages and date every statement; all quotations are verbatim from the cited artifacts.

The out-of-band era (all statements predate 2026-05-26). The design of “Tachyaction at a Distance” (Bowe, 2025-05-15) made Tachyon “out-of-band payments for the first time in a Zcash shielded protocol”: the sender delivers the note plaintext to the recipient over a channel outside the chain, and the on-chain ciphertext disappears. The simplification cascades through the key hierarchy — key diversification, viewing keys, and payment addresses are removed; the note drops the diversifier and ρ ; the diversified transmission key pk_d gives way to a payment key deterministic per spending key (§3.2, which owns the construction; the book’s src/keys.md opens with the in-file warning “this is significantly outdated”). Unlinkability becomes the wallet layer’s problem, deferred to the URI substrate: ZIP 321 payment-request URIs and ZIP 324 URI-encapsulated payments. That substrate is *specified* only in part. ZIP 321 is Status: Active. ZIP 324 is Status: Draft, predates Tachyon and mentions it nowhere, declares that “It is outside the scope of this proposal to establish a secure communication channel”, derives its rseed under a domain separator that is the literal 0xFIXME, is Sapling-only, and ends with a “Publication Blockers” section (zips zip-0324.rst at c6b70358). ZIP 231 memo bundles (Status: Draft) decouple memo data from individual outputs at full cryptographic rigor, but their v6 carrier was orphaned when ZIP 230 was withdrawn (zips zip-0231.md, zip-0230.rst at c6b70358). The out-of-band payment protocol itself was never specified at any rigor; the project roadmap listed two unreconciled delivery approaches (tachyon.z.cash/roadmap/, fetched 2026-07-08). The founding post had already conceded the cost: without in-band distribution “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (Bowe, 2025-04-02).

The pivot (2026-05-26). Issue #121 of tachyon-zcash/tachyon, opened 2026-05-26 and still open as of 2026-07-08, replaces the out-of-band story in two sentences, which we quote whole: “Since we’re using PIR to make repeated payments flows without OOB, there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows.” Repeated-payment flows are thus to use private information retrieval (PIR), a query protocol in which the server learns nothing about which record the client fetched, rather than sender-to-recipient out-of-band delivery — the roadmap adds a post-quantum key exchange for the channel — while in-band secret distribution returns, scoped to recover-from-seed and restoring exactly the property the founding post conceded was lost. The protocol is “developed independently in collaboration with” the team and has no published specification; its public trace is a roadmap paragraph and podcast/whiteboard discussion (tachyon.z.cash/roadmap/; ZK Podcast 388, 2026-01-21). *Designed but unspecified* at its thinnest:

a specification would need the PIR scheme and its database and server model, the key-exchange construction, the recover-from-seed format (which secrets stay in-band, under which keys, at what fee treatment), and the consequences for the key-hierarchy removals above — the book is flagged stale against the pivot by issue #121 itself, so which of `src/keys.md`’s removals survive is undetermined. The post-quantum claim also needs scoping: Bowe’s “Zcash and Quantum Computers” (2025-10-16) limits Tachyon’s quantum benefit to privacy — the harvest-now-decrypt-later threat — while ECC-based proof soundness remains quantum-vulnerable; whether retained in-band recovery ciphertexts reuse the ECC key agreement, and so retain its harvest-now exposure, is undetermined. Recoverability at large remains an *open problem* by the designer’s own account: “Getting this right will be challenging, but we simply have no choice but to do it” (forum t/50789 #22, 2025-11-04); on ZK Podcast 388 (2026-01-21) the recovery infrastructure was still “a can of worms”. The single sentence of issue #121 is the entire written resolution to date.

3 The design as published

The preceding section reconstructed the architecture; this one fixes the design’s object model as the project book publishes it: the mdBook of `tachyon-zcash/tachyon @ 26fdc165` (2026-07-07, single-committer; §1.3), the design’s most complete public statement. Each subsection states what the book fixes — exact formulas, fields, and domain strings — names the Orchard analogue in one contrast, and closes with the object’s bin under Definition 1.1 and what a specification must still pin down; mechanisms the architecture section constructs and gaps the status sections classify are cross-referenced, not restated. Every claim here is *designed but unspecified* by default. Where the `zcash_tachyon` crate at the same commit realizes an object, the realization is *specified* in the only sense available before a spec — verifiable source at a pinned commit (§4.3) — with the README split as the boundary: keys, value commitments, signatures, and bundle construction “implemented and tested”; note commitments, nullifier derivation, proofs, and stamp compression “Stubbed”. One caveat cuts across every Poseidon-derived object: the crate routes all Poseidon calls through Ragu’s mock sponge (`src/digest/poseidon.rs`), so derivation *structure* and domain tags are code-anchored while the outputs come from Ragu’s provisional PoseidonFp instantiation (width $T = 5$, rate 4; the mock sponge evaluates the real permutation out-of-circuit), not from any Tachyon-pinned Poseidon instance. (The book’s $\mathbb{F}_p$, $\mathbb{F}_q$ are the series’ $\mathbb{F}_{p_{\text{Pallas}}}$, $\mathbb{F}_{p_{\text{Vesta}}}$.)

Staleness is stated once, here. The book post-dates the May-2026 payment pivot (issue #121; §1.3), yet two chapters carry the in-file marker `<!-- todo: this is significantly outdated -->` (`src/keys.md` line 3, whole chapter; `src/authorization.md` line 228, its Detailed Sequence), and the key chapter’s out-of-band claims are pre-pivot statements, dated pre-2026-05-26 (§2.4, §5.1.2); we document the book as published and do not relitigate per object. Eight `src/SUMMARY.md` entries carry no file: the four section parents (Data Model, Cryptographic Primitives, Protocol, Network Roles — the last has a filed Overview child), the three network-role pages, and the testnet chapter. One terminological drift: the bundle chapter’s *tachystamp* is the *stamp* of the proof-tree chapter and the crate; we write *stamp*.

3.1 Bundles and tachygrams

Definition 3.1 (Tachyon bundle objects). A *tachygram* is a 32-byte field element representing either a nullifier or a note commitment; consensus does not distinguish the two. A *tachyaction* indistinguishably represents the creation or destruction of a note — a commitment tachygram proves creation, a nullifier tachygram destruction — and is cryptographically bound to one tachygram that it does *not* contain. It contains *cv*, a 32-byte homomorphic commitment to the created or destroyed value; *rk*, a 32-byte public key bound to one tachygram; and *sig*, a 64-byte RedPallas signature by *rk* over the transaction sighash. A *stamp* contains an anchor, the recursive proof

(which may be aggregated), and the covered actions' tachygrams; the proof establishes that each tachygram creates a new note or destroys an existing one, that tachygrams are correctly bound to action keys, and that the action balance effect matches the pool balance effect. A *bundle* collects the tachyactions, the integer net pool effect value_balance, a binding signature over the same sighash as the action signatures, and the stamp (book, src/bundle.md @ 26fdc165).

The covering proof witnesses a commitment to the *unordered set* of the stamp's tachygrams as a root polynomial, $\text{tachygram_acc}(X) = \prod_i (X - \text{tg}_i)$, and this set is the atomic unit that folds into pool state to contribute to the anchor (book, src/tachygrams.md @ 26fdc165); the fold and the two-epoch duplicate scan the set feeds are constructed at §2.3.

Wire format. The first byte, `tachyonBundleState`, selects one of three states: 0x00 non-tachyon (no bundle), 0x01 stamped, 0x02 stripped; the body layout (value balance as i64, the (cv,rk) action vector, per-action and binding signatures) is fixed only in the crate's doc tables (crates/tachyon/src/bundle/mod.rs). The stamp trailer (state 0x01) carries `cActionsTachyon` (32 bytes, a “compressed Pedersen commitment” to the stamp's merged action-digest set; §3.6), `anchorTachyon` (32 bytes), `nTachygrams` (compactsize), `vTachygrams` (32*n* bytes), and `proofTachyon`, a `PROOF_SIZE` blob — “serialized proof of fixed size”. The stripped trailer (state 0x02) carries only `tachyonAggregateId`, the 64-byte nonzero `wtxid` (the ZIP 239 identifier `wtxid || auth_digest`; §3.8) of the covering aggregate: the all-zero `wtxid`, naming no aggregate, is rejected on read even for a stripped bundle whose action vector is empty, and the stripped trailer carries no `cActionsTachyon` — that field rides on the stamp and strips away when a bundle becomes an adjunct.

The Orchard contrast: an Orchard Action publishes typed, distinguishable fields (cv^{net} , nf^{old} , rk, cmx) plus note ciphertexts, with one per-transaction Halo 2 proof (the *Ironwood Guide*, §“What a node verifies, without ever observing the note”); a tachyaction is single-effect, ciphertext-free, does not carry its tachygram, and its proof is strippable and aggregatable across transactions — a lifecycle with no Orchard analogue.

Classification. *Designed but unspecified*, with the wire format's structure specified-as-code: the crate's doc tables mirror the book's byte-for-byte and the nonzero-`wtxid` rule is enforced on read (bundle/mod.rs, stamp/mod.rs). A specification must still pin `PROOF_SIZE`, which no number in the book fixes (the crate defers to Ragu's `PROOF_SIZE_COMPRESSED`), and the point-compression convention, fixed only informally per context (the trailer compresses; the anchor absorbs complete coordinates, §3.5).

3.2 The key hierarchy

Definition 3.2 (Tachyon key hierarchy). The spending key `sk` is raw 32-byte entropy, matching Orchard's representation and preserving the full 256-bit key space. With $\text{PRFexpand}_{\text{sk}}(t) = \text{BLAKE2b-512}(\text{Zcash_ExpandSeed}, \text{sk} || t)$, the book derives

$$\text{ask} = \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([0x21])), \quad \text{nk} = \text{ToBase}(\text{PRFexpand}_{\text{sk}}([0x22])),$$

a long-lived scalar in $\mathbb{F}_{P_{\text{Vesta}}}$ and a base-field element in $\mathbb{F}_{P_{\text{Pallas}}}$, with $\text{ak} = [\text{ask}] \mathcal{G}$; the payment key is

$$\text{pk} = \text{Poseidon}(\text{PK_DOMAIN}, \text{ak}_x, \text{nk}),$$

with ak_x the x -coordinate of ak — the chapter leaves `PK_DOMAIN` symbolic; only the domain-separator table fixes it as `Tachyon-PkDerive` (§3.8). The *proof authorizing key* $\text{pak} = (\text{ak}, \text{nk})$ enables proof construction without spend authority; the per-note delegation bundle (ak, mk) re-

stricts a prover to one note, all epochs, where mk is the note’s GGM master key (§3.4). (Book, `src/keys.md @ 26fdc165`.)

Sign normalization and tiers. RedPallas requires $ak = [ask] \mathcal{G}$ to have $\tilde{y} = 0$ (bit 255 of the compressed encoding); if $\tilde{y}(ak) = 1$ the derivation negates ask ($[-ask] \mathcal{G} = -[ask] \mathcal{G}$), once, at derivation time (`src/keys.md`; `crates/tachyon/src/keys/private.rs` lines 83–98). The key ask cannot sign directly and ak cannot verify directly: both are randomized per action into rsk and rk (§3.7), so each on-chain rk is unlinkable to ak ; the book names the tiers private scalar $\rightarrow$ circuit witness $\rightarrow$ public on-chain (`SpendAuthorizingKey`, `SpendValidatingKey`, `ActionVerificationKey`). The payment key is *deterministic per spending key* — every note from one sk shares one pk , no per-note diversification — and since pk is committed in cm , accumulator membership transitively pins ak and nk : a wrong nk gives a wrong pk , a wrong cm , and inclusion fails.

The Orchard contrast: the root sk , the PRFexpand construction, the $ask \rightarrow ak$ derivation with the same sign normalization, and RedPallas re-randomization survive (the *Ironwood Guide*, §“Keys: capabilities for spending and for viewing”; tag bytes differ, $0x06/0x07/0x08$ against $0x21/0x22$), while r_{ivk} , $fvk = (ak, nk, r_{ivk})$, ivk , dk , ovk , the FF1-AES256 diversifier d , g_d , and pk_d are removed, and pk , pak , and the per-note (ak, mk) bundle are new with no Orchard counterpart.

Classification. The derivations and normalization are *specified* as tested code in the crate’s key module, with pk computed through the mock sponge (`keys/private.rs`). The intent around pk — unlinkability deferred out-of-band — is the pre-pivot claim re-examined at §5.1.2. A specification must pin `PK_DOMAIN` normatively, the encoding of ak_x , and — after the pivot — which key-hierarchy removals survive.

3.3 Notes and commitments

Definition 3.3 (Tachyon note). A note carries exactly four fields: $pk \in \mathbb{F}_{p_{\text{Pallas}}}$, the recipient’s payment key; v , a u64 value in zatoshi with $1 \leq v \leq 2.1 \times 10^{15}$; $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$, the nullifier trapdoor; and $rcm \in \mathbb{F}_{p_{\text{Pallas}}}$, the commitment trapdoor. Zero-value notes are forbidden. The commitment binds all four,

$$cm = \text{Poseidon}_{\text{Tachyon-CmDerive}}(rcm, pk, v, \psi),$$

and is the published tachygram of an output action; a spend action publishes *two* nullifiers, at the present epoch e and the next epoch $e + 1$, as its tachygrams (`book, src/notes.md @ 26fdc165`; the commitment already appears in §2.1, the dual-nullifier rationale in §2.3).

The Orchard contrast: an Orchard note is a 5-tuple $(addr = (d, pk_d), v, \rho, \psi, rcm)$ under a perfectly hiding Sinsemilla `NoteCommit`, with $v \in [0, 2^{64})$ (the *Ironwood Guide*, §“Representation of a note: notes and note commitments”) and load-bearing zero-value dummies (*Ironwood Guide*, §“The Action statement, formally”); Tachyon drops the address and ρ entirely — “no diversifier, no rho, no unique value for faerie gold defense” since “the nullifier construction doesn’t require global uniqueness” (`crates/tachyon/src/note.rs`) — so where Orchard chains $\rho^{\text{new}} := nf^{\text{old}}$ (*Ironwood Guide*, §“Prevention of double-spending: nullifiers”), ψ reuse here harms only the owner (§5.1.1).

Classification. *Designed but unspecified*, and the commitment scheme is the book’s clearest internal churn marker: the note and domain-separator chapters fix Poseidon under `Tachyon-CmDerive`, while the crate’s module doc calls the scheme “TBD” — “(e.g. Sinsemilla, Poseidon)” pending what “is efficient inside Ragu circuits” (`note.rs`; cited at §5.1.1). A specification must pin the scheme and the field encodings; the ψ -uniqueness responsibility gap is classified at §5.1.1.

3.4 Epochs and the GGM nullifier tree

The derivation’s endpoints — the master key $\text{mk} = \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, \text{nk})$, the per-epoch leaf nullifier, the delegation window δ with its sentinel, and the spendable lineage — are constructed at §2.1 and §5.1.1; none of that machinery appears in the eprint note, whose evolving-nullifier sketch is a flat PRF still taking ρ as input (quoted at §5.1.1). What the book fixes beyond those statements is the climb itself.

Construction 3.4 (GGM climb). The tree has depth D and arity A , covering leaf epochs $e \in [0, A^D - 1]$; a node at depth d covers a contiguous range of A^{D-d} consecutive epochs, and a node may only climb toward the leaves, never back toward the root — the property that makes absence-proof delegation trustless. To reach the leaf at epoch e , decompose e into $d \in [0 \dots D]$ base- A direction chunks, most significant first, and climb one step per chunk:

$$\text{KDF}_{\psi}^{\text{climb}}(e, 0) = \text{KDF}_{\psi}^{\text{root}}(\text{nk}) = \text{mk},$$

$$\text{KDF}_{\psi}^{\text{climb}}(e, d) = \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\text{KDF}_{\psi}^{\text{climb}}(e, d-1), \lfloor e/A^{D-d} \rfloor \bmod A).$$

The value at $\text{KDF}_{\psi}^{\text{climb}}(e, D)$ is the leaf key for epoch e ; the final Tachyon-NfDerive hash from leaf key to nullifier is constructed at §5.1.1 (book, `src/nullifiers.md` @ 26fdc165).

Parameters. The book leaves D and A symbolic everywhere. The crate pins working values: $\text{GGM_TREE_ARITY} = 4$, $\text{GGM_CHUNK_SIZE} = 2$ bits, $\text{GGM_TREE_DEPTH} = 10$ under the invariant $A^D = \text{GGM_MAX_INDEX} + 1$ with $\text{GGM_MAX_INDEX} = \text{EPOCH_MAX} = 2^{20} - 1$ ($4^{10} = 2^{20}$ epochs tile the index space exactly), and $\text{EPOCH_SIZE} = 2^{12} = 4096$ blocks, via a shift constant that drops to 4 under `cfg(test)` — placeholder values, not decisions (`crates/tachyon/src/keys/ggm.rs`, `src/constants.rs`). Nothing in the book maps the epoch index to block height; the epoch length exists only in code, and its tuning is the open issue of §5.1.4.

The Orchard contrast: Orchard derives *one* nullifier per note, $\text{nf} = \text{Extract}([F_{\text{nk}}(\rho) + \psi] \mathcal{G}^{\text{nf}} + \text{cm})$ — a Poseidon PRF plus a curve point — checked against the global, ever-growing nullifier set (the *Ironwood Guide*, §“Prevention of double-spending: nullifiers”); Tachyon replaces it with a per-epoch sequence, pure Poseidon with no curve point and no Extract, checked only over the two-epoch window of §2.3.

Classification. *Designed but unspecified*; the tree walk is code-anchored in structure (`keys/ggm.rs`) but its cryptography sits on the stubbed side of the README split. A specification must pin D , A , the epoch length and activation semantics (§5.1.4), and the chunk encoding; the tip-nullifier exposure of the delegation flow is the open problem of §5.2.4.

3.5 Anchors, stamps, and the proof tree

The three anchor-chain formulas are constructed at §2.3; the unresolved normative force of end-of-block anchoring is classified at §5.1.5. The book adds two statements worth fixing here: intra-block states exist between stamp absorptions, and a proof such as `SpendableInit` “will produce a header that is likely at an intra-block state, and should be lifted to the end of a block by proving state continuity” (`src/anchor.md`; that the epoch step is the only domain absorbing the epoch index is already fixed at §2.3 — all cross-epoch binding rides on it); and absorbing the *complete* (x, y) coordinates of the stamp’s tachygram-set commitment, rather than a compressed encoding, makes the binding unambiguous — the crate sharpens this to a Vesta point whose $\mathbb{F}_{p_{\text{Vesta}}}$ coordinates are each split into two 128-bit little-endian limbs, a six-element Poseidon absorption (`digest/poseidon.rs`; the sequence, tachygram-set, and action-set commitments all live on Vesta, `primitives/seq.rs`, `primitives/sets.rs`).

The step graph. Each proof step accepts arbitrary witness inputs and up to two PCD inputs, and emits a new PCD. The book fixes exactly nineteen steps over eight header types (Table 2) and a role matrix: the wallet runs every step touching the note’s commitment or master key; the sync service runs the anchor and Unspent steps over wallet-shared nullifier values, never note material (§5.1.3); the aggregator touches only published StampHeaders and the public anchor chain — the three anchor-chain steps to build lift segments, then StampLift and MergeStamp (book, src/proof-tree.md @ 26fdc165).

Construction 3.5 (Sentinel-terminated sequence composition). An Unspent’s elapsed holds one nullifier coefficient per epoch-boundary crossing in its span, forward-chronological, terminated by a sentinel coefficient 1 at the crossing count $s = \text{epoch_end} - \text{epoch_start}$; the sentinel keeps every committed sequence nonzero (an empty window is the constant 1), so the commitment never falls on the identity point, and it pins the sequence’s exact length. Step NullifierStep builds its single-leaf commitment homomorphically, $[\text{nf}] \mathcal{G}_0 + \mathcal{G}_1$, with $\mathcal{G}_1$ carrying the sentinel. Composition is checked as polynomial identities at a Fiat–Shamir challenge:

$$C(X) = L(X) + X^s (R(X) - 1) \quad (\text{UnspentFuse}),$$

$$C(X) = L(X) + X^s (p - 1) + X^{s+1} R(X) \quad (\text{UnspentEpochFuse}),$$

$$R(X) = E(X) + X^s (\text{nf_end} - 1) + X^{s+1} \quad (\text{VerifyUnspent}),$$

where p is the left half’s completing tip nf_end and E the elapsed sequence: in each identity the appended term overwrites the left sentinel and the trailing term re-terminates. The inputs L , R , and p are bound by the recursive verification of the input PCDs *before* the challenge, and each identity is linear in them — which is what makes the splice sound. Step MergeStamp fuses two stamps at equal anchors by requiring, for each of the action-digest and tachygram multisets, that the merged set polynomial equal the *product* of the two inputs’ (book, src/proof-tree.md @ 26fdc165).

Lineage and spend binding. Step SpendableInit checks cm among the creation stamp’s tachygrams and roots the chain at $\text{pre_epoch_anchor.next_epoch}(\text{epoch})$, pinning the starting GGM leaf index to the consensus epoch — without it, a note spent in its creation epoch crosses no boundary and the index is a free witness. On present_nf the book disagrees with itself: its walkthrough binds present_nf to the proven leaf by equality at init — matching the step’s NullifierHeader input in Table 2 — while its reference section calls it “unconstrained here”, pinned to a genuine leaf only by the first lift — another unresolved internal inconsistency at 26fdc165 (src/proof-tree.md). Step SpendBind requires $\text{spendable.cm} = \text{note.commitment}()$ — rejecting a phantom note reusing ψ under a different value and cm — and its SpendHeader is consumed only by SpendStamp, so the note never propagates. Step SpendStamp composes it with a length-2 NullifierHeader range, witnesses nf_next , binds the published pair to the boundary leaves by equality, and guards both nullifiers nonzero (the guard’s rationale is quoted at §5.1.1); deferring this binding and the action digest here keeps each step within its per-step gate budget. The book’s privacy claim: the note’s age never becomes public — the lineage carries a single current nullifier, not a polynomial with a consumed offset.

Stamps. Step OutputStamp is the only stamp-producing step taking an anchor as direct witness (an output has no prior chain state); it rejects zero and over-range values, and the wallet typically anchors outputs at the spends’ height so MergeStamp proceeds without an intervening lift. Step StampLift advances a composed stamp’s anchor toward the tip before publication; validators reconstruct both set commitments from published bundles, check the proof against them, and confirm the anchor against the chain. An aggregated stamp has the same shape as any other and is eligible for further

Header	Fields			
AnchorChain	(start, end)			
Unspent	(anchor_prev, (epoch_start, nf_start), elapsed, (epoch_end, nf_end), anchor_last)			
VerifiedUnspent	(cm, anchor_prev, (epoch_start, nf_start), (epoch_end, nf_end), anchor_last)			
NfPrefixHeader	(cm, node, depth, index)			
NullifierHeader	(cm, (epoch_start, nf_start), nf_seq_commit, (epoch_end, nf_end))			
SpendableHeader	(cm, present_nf, anchor)			
SpendHeader	(cm, (cv, rk), present_nf, anchor)			
StampHeader	(action_commit, tachygram_commit, anchor)			

Step	Left	Right	Witness	Output
AnchorSeed	—	—	start, stamp_commit	AnchorChain
EmptyBlockSeed	—	—	start	AnchorChain
AnchorFuse	AnchorChain	AnchorChain	—	AnchorChain
UnspentSeed	—	—	anchor_prev, (epoch, nf), stamp_tg_set	Unspent
EmptyBlockUnspentSeed	—	—	anchor_prev, (epoch, nf)	Unspent
UnspentFuse	Unspent	Unspent	left_elapsed_seq, combined_elapsed_seq, right_elapsed_seq	Unspent
UnspentEpochFuse	Unspent	Unspent	left_elapsed_seq, combined_elapsed_seq, right_elapsed_seq	Unspent
VerifyUnspent	Unspent	NullifierHeader	elapsed_seq, nf_seq	VerifiedUnspent
NfMasterSeed	—	—	note, pak	NfPrefixHeader
NfPrefixStep	NfPrefixHeader	—	chunk	NfPrefixHeader
NullifierStep	NfPrefixHeader	—	—	NullifierHeader
NullifierFuse	NullifierHeader	NullifierHeader	left_seq, merged_seq, right_seq	NullifierHeader
SpendableInit	AnchorChain	NullifierHeader	(pre_epoch_anchor, pre_cm_anchor), creation_set, present_nf	SpendableHeader
SpendableLift	SpendableHeader	VerifiedUnspent	—	SpendableHeader
SpendBind	SpendableHeader	—	note, rcv, alpha, pak	SpendHeader
OutputStamp	—	—	rcv, alpha, note, anchor	StampHeader
SpendStamp	SpendHeader	NullifierHeader	nf_next	StampHeader
MergeStamp	StampHeader	StampHeader	(action_set, tachygram_set) × left, merged, right	StampHeader
StampLift	StampHeader	AnchorChain	—	StampHeader

Table 2: The eight proof-tree headers (top) and nineteen steps (bottom), fields verbatim (book, src/proof-tree.md @ 26fdc165). All nineteen steps are registered in the crate against Ragu’s mock (crates/tachyon/src/stamp/proof/mod.rs).

aggregation.

The Orchard contrast: Orchard proves membership on every spend via a 32-layer Sinsemilla Merkle path against a per-block tree anchor (the *Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”); the no-tree, proven-once replacement is stated at §2.2.

Classification. *Designed but unspecified* throughout: the nineteen steps exist as circuit-shaped Rust against the mock (`stamp/proof/mod.rs`), not as mathematics, and the soundness section is prose (§2.1; §5.1.5). A specification must pin the header encodings, the per-step relations, the polynomial-commitment scheme and coefficient generators $\mathcal{G}_i$ (the book’s Commit is abstract), and the Poseidon instance.

3.6 Aggregation

The lifecycle nomenclature is fixed in Table 3: wallets broadcast *autonomes*; aggregators strip and merge stamps into *aggregates*; miners strip covered transactions into *adjuncts*, and a block carrying an aggregate must include every covered transaction sans its now-redundant proof. Aggregates subdivide into *innocent* (no Tachyon actions of its own, e.g. other-pool-only activity) and *based* (proving its own actions, e.g. a coinbase aggregate paying miner rewards through a Tachyon action). The process is decompress–merge–recompress: select transactions; deserialize and validate stamps; match or update anchors (StampLift); merge stamps — tachygrams as exclusive sets, proofs recursively; serialize, compress, publish (book, `src/aggregation.md` @ 26fdc165).

Term	State	Provenance	Validation
autonome	stamped	wallet	complete proof with all inputs
aggregate	stamped	aggregator	merged proof needs input from other transactions
adjunct	stripped	miner	input to merged proof from another transaction

Table 3: Bundle lifecycle nomenclature (book, `src/aggregation.md` @ 26fdc165).

The action-set indicator. An aggregate’s stamp proves its own plus every covered adjunct’s actions but carries no proof header — verifiers reconstruct it from covered actions, which before block inclusion are not at hand. The stamp trailer therefore carries `cActionsTachyon`, an author-provided *assistive indicator* mirroring the action-set commitment the proof attests to, letting an observer cheaply tell an autonome from an aggregate and judge a collection of adjuncts complete without verifying the proof. It is authorization data, not effecting data — it rides the strippable trailer, contributes to `auth_digest`, and is absent from adjuncts — and *not* a soundness mechanism: the proof binds the action set regardless, and a wrong indicator only harms its author. Function `Stamp::verify` uses it as a fast-fail gate before expensive proof verification (book, `src/aggregation.md`; code, `stamp/mod.rs`).

The Orchard contrast: no analogue exists — an Orchard bundle’s proof is per-transaction, never stripped, never merged across transactions (the *Ironwood Guide*, §“What a node verifies, without ever observing the note”).

Classification. *Designed but unspecified*, shading into *open problem* inside the chapter’s own file: the tachygram set-union validation algorithm and the aggregation limits are in-file TODOs, and the closing comment lists undecided economics — aggregate size limited by commitment size before block size (“on the order of thousands of actions”), single-aggregate merging serial though multiple aggregates parallelize, ser/de expensive, and the optimal-aggregate, miner-integrate-vs-p2p, and selection-order questions open (`src/aggregation.md`, comments). The “Block Layout” section the transaction-identifiers chapter cites does not exist in `aggregation.md` at this commit — a dangling link. A specification must pin the union-validation algorithm, the limits, and the gossip protocol.

3.7 Authorization

A bundle carries three authorization layers: per-action signatures binding each tachyaction to its tachygram, value commitments hiding individual values while preserving their algebraic sum, and a binding signature proving the declared balance (book, src/authorization.md @ 26fdc165).

Construction 3.6 (Deterministic action randomizer). The planner selects arbitrary per-action entropy θ ; the custody device receives each note and θ to independently confirm the planning work. The randomizer is deterministic from (θ, cm) under per-effect personalizations:

$$\alpha_{\text{spend}} = \text{BLAKE2b-512}_{\text{Tachyon-Spend}}(\theta \parallel \text{cm}), \quad \alpha_{\text{output}} = \text{BLAKE2b-512}_{\text{Tachyon-Output}}(\theta \parallel \text{cm}).$$

Every action signs with a unique rsk , published as $\text{rk} = [\text{rsk}] \mathcal{G}$. For a spend,

$$\text{rsk} = \text{ask} + \alpha, \quad \text{rk} = [\text{ask} + \alpha] \mathcal{G} = \text{ak} + [\alpha] \mathcal{G},$$

so the planning device derives rk from the validating ak without ever holding ask , and custody confirms rk before signing. For an output, $\text{rsk} = \alpha$ and $\text{rk} = [\alpha] \mathcal{G}$: no authority is needed. Because α derives from $\theta \parallel \text{cm}$, the key rk is itself a commitment to cm , and the signature transitively binds each action to its tachygram without the tachygram appearing in the action.

Bundle commitment and sighash. The bundle’s effect digest accumulates per-action digests into a set polynomial:

$$d_i = \text{Poseidon}_{\text{Tachyon-ActionDg}}(\text{cv}_i \parallel \text{rk}_i), \quad \text{action_acc} = \text{Commit}\left(\prod_i (X - d_i)\right),$$

hashed as $\text{BLAKE2b-512}_{\text{ZTxIdTachyonHash}}(\text{action_acc} \parallel \text{value_balance})$. The same action-digest accumulation is bound to the stamp proof; the stamp itself is excluded from the bundle commitment because it strips during aggregation. All signatures — action and binding — sign one transaction-wide sighash, computed at the transaction layer from every pool’s bundle commitment and passed to the Tachyon layer as opaque bytes. Consensus recomputes the action digests from the visible actions and checks them against both the sighash (via the bundle commitment) and the stamp (via the polynomial commitment in the PCD header); a modified action breaks both checks.

Value balance. The value commitment and binding algebra are Orchard’s, deliberately: with v signed (positive spends, negative outputs) and rcv random in $\mathbb{F}_{p_{\text{Vesta}}}$,

$$\text{cv} = [v] \mathcal{V} + [\text{rcv}] \mathcal{R}, \quad \text{bvk} = \sum_i \text{cv}_i - [\text{value_balance}] \mathcal{V} = \left[\sum_i v_i - \text{value_balance} \right] \mathcal{V} + [\text{bsk}] \mathcal{R},$$

where $\text{bsk} = \sum_i \text{rcv}_i$: the planner signs the sighash with bsk directly, without custody assistance, and validators’ reconstructed bvk equals $[\text{bsk}] \mathcal{R}$ exactly when $\sum_i v_i = \text{value_balance}$. The generators are *shared with Orchard*, from the domain z.cash:Orchard-cv . Two footnotes flag churn: the rcv derivation “may be revised in the future to incorporate a hash of the note commitment”, and the binding signature uses `reddsa::orchard::Binding`, hardcoding $\mathcal{R}$ — “We should consider defining a unique personalization.”

The Orchard contrast: the value machinery is identical by construction — same Pedersen cv , same bvk equation, literally the same generators (the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”) — while the randomizer differs in kind: Orchard draws α uniformly at random per Action with no planner/custody split (*Ironwood Guide*, §“Authorisation of the spend: RedPallas”), and every Orchard Action is a paired spend-plus-output carrying one `SpendAuthSig` (*Ironwood Guide*, §“A single shielded payment, end to end”; §“What a node verifies, without ever observing the note”).

Classification. This is the *specified-as-code* side of the README split: the randomizer, re-randomized keys, value commitments, and both signature layers are implemented and tested (entropy.rs, keys/private.rs, value.rs, reddsa.rs). Unpinned by any artifact: the sighash construction (opaque bytes at this layer), the rcv derivation, the generator personalization, the Commit scheme behind action_acc, and — flagged in-file — whether α moves to a curve-native derivation (src/domain-separators.md, comment).

3.8 Transaction identifiers and domain separators

A bundle’s authorization form changes through aggregation — stamping, merging, and stripping produce bit-different authorizations of the same effecting data — and wtxid = txid || auth_digest (ZIP 239) must distinguish the forms. Tachyon routes every mutable part through auth_digest: txid commits to effecting data only, action_acc || value_balance, so stripping, merging, and re-stamping leave it unchanged, while the authorization side commits to the signatures plus the stamp trailer:

$$\begin{aligned} \text{auth_digest_contribution} &= \text{BLAKE2b-256}_{\text{ZTxAuthTachyHash}}(\text{vActionSigs}||\text{bindingSig}||T), \\ T &= \begin{cases} \text{cActionsTachyon}_{32}||\text{anchor}_{32}||\text{vTachygrams}||\text{proof} & \text{if stamped} \\ \text{stampWtxid}_{64} & \text{if stripped} \end{cases} \end{aligned}$$

(book, src/transaction-identifiers.md @ 26fdc165). Each physical form thus yields a distinct wtxid. The personalization ZTxAuthTachyHash is declared “a placeholder until a Tachyon-ZIP amendment to ZIP-244 fixes it.” An adjunct’s covering-aggregate reference is a wtxid, assigned by the miner at block assembly; the covering aggregate must itself be top-level in the block — never stripped, never further aggregated — so the reference is stable.

A book-internal (and book-code) discrepancy. The transaction-identifiers formula writes BLAKE2b-256 for the auth_digest contribution — matching ZIP 244’s digests — while the book’s own domain-separator table (Table 4) and the crate (hash_length(64); digest/blake2b.rs) both say BLAKE2b-512: the hash width is an unresolved encoding at 26fdc165. The code also fixes encodings the book leaves loose: the bundle commitment hashes the action commitment’s complete coordinates $x||y$ then value_balance as a little-endian i64, and a transaction with no Tachyon bundle contributes the personalized hash of the *empty* string — documented as distinct both from a stripped bundle and from a bundle with no actions and zero balance (blake2b.rs, bundle/mod.rs).

The Orchard contrast: the construction mirrors the ZIP-244 personalized-BLAKE2b pattern of ZTxIdOrchardHash (the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”, paragraph “The transaction digest (ZIP-244)”; the novelty is that the entire stamp rides auth_digest, keeping txid invariant under aggregation.

Domain separators. Table 4 is the book’s complete table — its most specification-like artifact, byte-exact strings each realized as a code constant. The crate fixes the in-circuit realization the book omits: every Poseidon domain tag is a 16-byte ASCII string absorbed as the *first* sponge input, embedded as $\mathbb{F}_{P_{\text{Pallas}}}$ via $\text{Fp}::\text{from_u128}(\text{u128}::\text{from_le_bytes}(\text{tag}))$, with arities: action digest 5 (domain, cv and rk as full coordinate pairs), payment key 3, note commitment 5, master key 3, prefix step 3, nullifier 2, anchor stamp step 6, anchor empty step 2, anchor epoch step 3 (digest/poseidon.rs) — all through the mock sponge.

Classification. *Designed but unspecified*, at the highest concreteness the corpus reaches: byte-exact strings with code constants, yet two entries are self-declared placeholders — ZTxAuthTachyHash, and the value-commitment generator borrowed from Orchard’s z.cash:Orchard-cv — and one is internally inconsistent (the 256/512 width, above). A specification must fix the hash widths, the personalization bytes, and above all the Poseidon instance the eight domains parameterize — parameters,

Purpose	Hash	Value
Spend α	BLAKE2b-512	Tachyon-Spend
Output α	BLAKE2b-512	Tachyon-Output
Bundle commitment	BLAKE2b-512	ZTxIdTachyonHash
Bundle auth digest	BLAKE2b-512	ZTxAuthTachyHash
PRF expand (0x21 ask, 0x22 nk)	BLAKE2b-512	Zcash_ExpandSeed
Nullifier prefix key	Poseidon	Tachyon-NfPrefix
Nullifier derivation	Poseidon	Tachyon-NfDerive
Note commitment	Poseidon	Tachyon-CmDerive
Action digest	Poseidon	Tachyon-ActionDg
Payment key derivation	Poseidon	Tachyon-PkDerive
Anchor stamp step	Poseidon	Tachyon-StampFld
Anchor empty step	Poseidon	Tachyon-EmptyBlk
Anchor epoch step	Poseidon	Tachyon-EpochStp
Value commitment	hash-to-curve	z.cash:Orchard-cv

Table 4: Complete domain-separator table (book, `src/domain-separators.md` @ 26fdc165). The auth-digest hash width is the discrepancy noted in the text: the book’s transaction-identifiers formula says BLAKE2b-256; this table and the crate say BLAKE2b-512.

rate, and capacity exist in no Tachyon artifact (the crate computes every domain through Ragu’s mock sponge, inheriting its provisional PoseidonFp, width $T = 5$, rate 4).

Taken together, the book at 26fdc165 fixes an object model at a concreteness no other Tachyon artifact approaches, and still leaves every load-bearing parameter symbolic: the GGM shape and epoch length (§5.1.4), the proof size, the Poseidon instance, the commitment scheme and its generators, the sighash, and the note-commitment scheme itself. The sections that follow classify these gaps severally; the open problems are the design’s own (§5.2).

4 What is specified today

Every claim in this volume is classified *specified*, *designed but unspecified*, or *open problem*. This section is the specified bin, and it is exhaustive: a statement appears here only if a verifiable artifact backs it — a ZIP with a recorded status in `zcash/zips`, or source code read at a pinned commit. Our ground truth is `zcash/zips` at commit c6b70358 (2026-07-05), `tachyon-zcash/ragu` at 830bbcd (2026-07-05), and `tachyon-zcash/tachyon` at 26fdc165 (2026-07-07); repository metadata (releases, package registries, continuous integration) was queried on 2026-07-08. The two lower bins follow in §5.1 (designed but unspecified) and §5.2 (open problems).

The headline fact is negative. No component of the Tachyon core — the shielded protocol, the bundle format, the accumulator, the aggregator, oblivious synchronization — is specified anywhere: upstream `zcash/zips` contains no Tachyon ZIP or pull request, and the five core ZIP stubs in the project’s own tracker are empty files (book/`src/zips/` @ 26fdc165; `tachyon-zcash/tachyon` issue #114). What the specified bin actually contains is periphery: one Active payment-request substrate, four groundwork drafts in varying states of health, and the code state of the proof layer and the protocol crate. We inventory each in turn.

4.1 ZIP 321 as substrate, and the four groundwork drafts

All ZIP facts below are read from `zcash/zips` @ c6b70358; line numbers refer to the files at that commit.

ZIP 321 — Payment Request URIs (Status: Active). This is the only entry at deployed rigor. It defines the `zcash:` URI scheme by which a payer is asked for payment: a grammar of address,

amount, memo (base64url), label, and message parameters, with numeric parameter indices for multi-recipient requests (zip-0321.rst). Its role here is substrate: the pre-pivot Tachyon design removes per-note key diversification, and the design book assigns per-payment unlinkability to “out-of-band payment protocols (ZIP 321 payment requests, ZIP 324 URI encapsulated payments)” (tachyon book/src/keys.md @ 26fdc165).

ZIP 324 — URI-Encapsulated Payments (Status: Draft). Created 2019-07-17, the draft predates Tachyon by six years and mentions it nowhere. It specifies sending a payment through any secure messaging channel as a URI encoding the secret spending key of an ephemeral, freshly funded address; anyone who learns the URI finalizes the payment by sweeping the encapsulated funds into their own wallet (zip-0324.rst, Abstract). Three limits bound what it pins down. First, the construction is Sapling-only: the encapsulated note is a Sapling note, keys derive under the path `m_Sapling/324' / ...`, and Orchard appears zero times. Second, core details are unpinned: the master-key derivation $\text{PRFexpand}_{sk_m}(\cdot)$ carries the literal placeholder `[0xFIXME]` as its domain separator (line 148), the note-identification section is marked “out of date ... To be revised” (line 384), and the draft closes with a “Publication Blockers” section (line 458). Third, the hard part is out of scope by declaration: “It is outside the scope of this proposal to establish a secure communication channel for transmission of URI-Encapsulated Payments” (line 107) — the unlinkability Tachyon defers to this document is thus deferred again, to the messaging layer.

ZIP 231 — Memo Bundles (Status: Draft). The cryptographic core is at full ZIP rigor. Memos leave the note plaintext and move into a per-transaction *memo bundle*: a sequence of 272-byte encrypted chunks capped at $\text{memo_chunk_limit} \times 256 = 16384$ bytes of plaintext (16 KiB) per transaction (lines 209–216); each chunk sealed with ChaCha20-Poly1305 in a variant of the STREAM construction, the final chunk distinguished by a nonce suffix (lines 264–274); per-output 32-byte memo keys K^{memo} with two reserved values — all-0x00 for publicly decryptable, all-0xFF for no memo (lines 222–231); chunks of all memos interleaved by an order-preserving shuffle that MUST reveal to a recipient only the size of their own memo and the total count of chunks they cannot decrypt (lines 282–297); a two-pass decryption algorithm, one pass under counter nonces and one under the final-chunk nonce (lines 323–351); independent prunability via `fAllPruned`, with relay of pruned transactions forbidden (lines 391–434); and a fee treatment under ZIP 317 Revision 2, two free chunks whenever the transaction has shielded outputs (line 422 ff.). The gaps are equally explicit: the sighash commitment is “TODO: finish this as a modification to ZIP 248” (line 417); the lead byte is the unassigned placeholder `{{MBLEADBYTE}}` (lines 490, 588); and the carrier is orphaned — the transaction fields that would hold a memo bundle were defined by the withdrawn ZIP 230, and their designated new home, ZIP 248, is unmerged.

ZIP 230 — Version 6 Transaction Format (Status: Withdrawn). The draft carried the byte-precise OrchardZSA v6 wire format: Action Groups, the ZSA issuance bundle, the memo-bundle carrier fields, an explicit fee field, `zip233Amount`, and per-transparent-input `vSighashInfo` (zip-0230.rst, field tables). Its withdrawal warning states that “Transaction version number 6 is now defined by ZIP 248” (lines 39–40) — already stale, since ZIP 229 (Status: Draft, created 2026-06-13) now defines v6 for the NU6.3 pool and declares as non-requirements exactly the withdrawn machinery: ZSAs, `zip233Amount`, the explicit fee field, per-input sighash information, and ZIP 248’s “other extensibility affordances” (zip-0229.md, Non-requirements). The ZIP 230 byte format survives only as reference text with no deployment path.

ZIP 246 — Digests for the Version 6 Transaction Format (Status: Withdrawn). Created 2025-02-12 and withdrawn 2026-06-24, the draft defined v6 transaction identifiers, authorizing-data commitments, and signature digests; its withdrawal note reassigns all three to ZIP 229 (lines 39–41). Its one surviving idea is sighash-algorithm versioning: a closed per-transaction-version set of sighash

algorithms, selected via `sighashInfo = [sighashVersion] || associatedData` (lines 106–123). The concept now lives only in withdrawn text and in the unmerged ZIP 248 pull request (zcash/zips#1156, idle since 2026-05-26); ZIP 229 itself hashes in the plain ZIP-244 style.

The pattern across the four drafts is uniform. Each piece of groundwork either lacks its carrier (ZIP 231), was withdrawn in favor of a narrower NU6.3 format (ZIPs 230 and 246), or defers its hardest component to another layer (ZIP 324). The single live v6 specification today is ZIP 229; the extensible-format home that ZIPs 230, 231, and 246 all defer to, ZIP 248, exists only as an open pull request.

4.2 Ragu at commit 830bbcd: the proof layer as code

Ragu is the proof-carrying-data (PCD) framework on which Tachyon’s wallet-state proofs are to run. No protocol specification exists: every protocol chapter of the Ragu book — assumptions, arithmetization, NARK, split accumulation, registry polynomial, recursion, analysis — is marked `<!-- todo -->` in the book index (`book/src/SUMMARY.md`). At this commit the code is therefore the only specification of what Ragu does, and we read the code.

What the pipeline implements. The repository is a `no_std` Rust workspace (version 0.0.0, edition 2024, MSRV 1.90) implementing a modified version of the Halo recursive-SNARK construction, with no trusted setup (`README.md`, `Cargo.toml`). The Pasta cycle is the only implemented curve cycle: `crate ragu_pasta` provides the sole `Cycle` implementation, with the Pallas base field as the circuit field (`crates/ragu_pasta/src/lib.rs`). A working PCD pipeline exists end to end: an application registers its steps through `ApplicationBuilder::register` and finalizes; seeding creates a leaf; `fuse` merges two proof-carrying nodes into one through an eleven-stage prover pipeline (files `_01_application.rs` through `_11_circuits.rs` under `crates/ragu_pcd/src/fuse/`); `rerandomize` and `verify` complete the interface. Claims are tracked in two registries, native (Pallas-field) and nested (Vesta-field, CycleFold-style). An integration test builds and verifies a multi-node PCD tree (`crates/ragu_pcd/tests/nontrivial.rs`).

Verification is linear-time; no opening argument exists. Proofs are not succinct. Type `Proof` carries the full sparse polynomials of the argument — fifteen native polynomials plus the bridge polynomials, some twenty in all (`ragu_pcd/src/proof/mod.rs`). Function `verify()` checks a proof by sampling challenges, rebuilding the `revdot` claim polynomials, and evaluating the carried polynomials directly (`ragu_pcd/src/verify.rs`); an in-code note records that the native `c`-claim is tautological in the verifier and meaningful only inside the circuit. No inner-product-argument opening, decider, or proof-compression step exists anywhere in the workspace. Verification therefore runs in time linear in the proof, and the succinct final verifier Tachyon needs is future work, not present code.

Soundness placeholders. Three placeholders sit on the main branch, each self-documented. The Fiat–Shamir domain-separation tag is the literal `RAGU_TAG: &[u8] = b"FIXME"`, beneath the comment “choose a permanent domain separation tag before release” (`ragu_pcd/src/lib.rs`, lines 50–51). Registry binding — the mechanism that replaces per-circuit verification keys — runs six hash iterations from the placeholder “nothing-up-my-sleeve” challenges $w = 2$, $x = 3$, $y = 5$, under an in-code `FIXME(security)` stating that six iterations “is insufficient to fully bind the registry polynomial” (`ragu_circuits/src/registry.rs`, lines 673–692; issues #78, #316). The verifier, finally, omits three checks recorded in an in-code `TODO`: `registry_wx0_poly`, `registry_wx1_poly`, and `registry_wy_poly` (`verify.rs`, lines 122–124). Beyond soundness, zero knowledge is opt-in rather than default: the documentation of `fuse()` warns that accepting an RNG “is not an indication that the resulting proof-carrying data is zero-knowledge; that must be ensured by performing `Application::rerandomize` at a later point” (`fuse/mod.rs`), and `rerandomize` itself seeds a trivial proof under a comment reading “this is a temporary hack” (`lib.rs`, line 262).

Assurance machinery. The engineering hygiene around this pre-release core is heavy and, at the pinned commit, green. The workspace carries 445 `#[test]` functions and twelve property-testing suites; line coverage stood at 93.8% (Codecov snapshot, 2026-03-08). Twenty libFuzzer targets — witness generation and cheating, revdot and algebraic identities, Poseidon differentials, metamorphic driver behavior, verifier rejection, among others — run on a twice-weekly cron (`qa/fuzz/fuzz_targets/`, `.github/workflows/fuzz-cron.yml`: Sundays and Wednesdays). A Lean 4 formalization covers twenty-seven gadget-circuit files, and CI checks that the Lean sources extracted from the Rust gadgets are current (`qa/lean/Ragu/Circuits/`, `.github/workflows/fv.yml`). Supply-chain review runs under `cargo-vet` (`qa/supply-chain/`) and workflow linting under `zizmor`. All continuous-integration workflows were green at 830bbcd4 (GitHub Actions, queried 2026-07-08).

What is absent. No security audit exists: the README warns, verbatim, “Ragu is under heavy development and has not undergone auditing. Do not use this software in production.” No benchmark has been published: a bench harness runs in CI (`.github/workflows/bench.yml`) but publishes no numbers, and the only performance figures anywhere in the project are the Ragu book’s self-labelled rough estimates — which belong to the designed-but-unspecified bin, not to this one. No release exists: the workspace version is 0.0.0, the repository has zero GitHub releases, and no ragu crate is on crates.io (registry queries, 2026-07-08).

4.3 The `zcash_tachyon` crate: integration against a mock

Crate `zcash_tachyon` (`tachyon-zcash/tachyon @ 26fdc165`) implements and tests the conventional cryptographic plumbing: key derivation via PRFexpand with sign normalization, Pedersen value commitments over Pallas sharing Orchard’s generators, randomized RedPallas spend-authorization signatures, binding signatures, and bundle construction (README.md). Everything proof-shaped is stubbed. The crate consumes Ragu exclusively through its mock feature, pinning the dependency to a contributor fork (`github.com/turbocrime/ragu`, rev 98a15d32) with features `mock` and `legacy-deps` (`crates/tachyon/Cargo.toml`). The mock is, in its own words, an “API-level mock of `ragu_pcd`” that “mirrors the shape of the real `ragu_pcd` API so downstream consumers (e.g. Zebra) can integrate against it ahead of the real implementation” (`ragu src/mock/mod.rs @ 830bbcd4`); it performs no proof cryptography, though its Sponge does evaluate the real ragu Poseidon permutation out-of-circuit. Against this interface the crate registers its nineteen proof steps (`crates/tachyon/src/stamp/proof/mod.rs`). The README’s own stub list is exact: note commitments, nullifier derivation (the GGM tree PRF), proof creation, merge, and verification, and stamp compression/decompression all await “Ragu PCD and Poseidon integration”. The shape of the integration is therefore specified by code; its cryptographic content is not.

Taken together, the specified bin holds one Active URI substrate; one out-of-band payment draft that is Sapling-only with its core derivation unpinned; one memo-bundle draft whose cryptography is complete but whose carrier is orphaned; two withdrawn format ZIPs surviving as reference text; a proof framework whose pipeline runs but whose verifier is linear-time, incomplete, and Fiat–Shamir-tagged b“FIXME”; and a protocol crate integrating against a no-crypto mock. Every load-bearing Tachyon design — evolving nullifiers, the proof tree, aggregation, oblivious synchronization — lives outside this bin, in the sections that follow.

5 Designed but unspecified, and open problems

This section carries the two lower bins of the classification of Definition 1.1: constructions that exist in an informal source but in no artifact at ZIP or protocol-specification rigor, and problems the designers themselves acknowledge as unsolved. Nothing here is normative, and the global fact that forces both bins is established in §4: upstream `zcash/zips` (commit c6b70358, 2026-07-05) contains

zero Tachyon ZIPs or pull requests, and the five core ZIP pages in the project’s own design book — shielded protocol, bundle, accumulator, aggregator, and oblivious syncing service — are eleven-line skeletons with empty *Dependencies*, *Design Considerations*, and *ZIP Draft* headings (book/src/zips/@ 26fdc165, 2026-07-07).

For every entry we state three things: *what exists*, with the source and commit (source pinning as in §1.3; the mechanisms themselves are constructed in §2); *what a specification must pin down* before the entry can move to the specified bin (for open problems, what a resolution must establish); and *what would falsify the design* — the observation that would not merely delay the entry but refute it.

5.1 Designed but unspecified

5.1.1 Nullifier evolution and re-randomization for oblivious synchronisation

What exists. The ePrint note introduces evolving nullifiers only as a sketch: “As a concrete example, assume the nullifier for preventing the double-spend of a note is modified from $\eta \leftarrow \text{PRF}(k_{\text{note}}, \rho)$ to $\eta_e \leftarrow \text{PRF}(k_{\text{note}}, \rho || e)$ ” with e “a monotonic counter known as an epoch identifier” (ePrint 2025/2031, §1.3). The concrete derivation exists only in the design book (book/src/nullifiers.md @ 26fdc165): a Goldreich–Goldwasser–Micali tree of depth D and arity A rooted at the per-note master key

$$\text{mk} = \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, \text{nk}),$$

descended one Poseidon call per base- A chunk of the epoch index, with the epoch- e nullifier one further call under a separate domain, $\text{nf} = \text{Poseidon}_{\text{Tachyon-NfDerive}}(\text{KDF}_{\psi}^{\text{climb}}(e, D))$. The note commitment $\text{cm} = \text{Poseidon}_{\text{Tachyon-CmDerive}}(\text{rcm}, \text{pk}, v, \psi)$ digests ψ and (through pk) nk , so one commitment freezes the note’s entire nullifier sequence. Delegation hands a service a window of future nullifiers committed on coefficient generators with a sentinel, $\delta = \sum_i [\Delta_{e+i}] \mathcal{G}_i + \mathcal{G}_d$, re-based so any window stands alone. None of this is implemented: the zcash_tachyon crate lists “Note commitments” and “Nullifier derivation (GGM tree PRF)” under “Stubbed (pending Ragu PCD and Poseidon integration)” (README.md @ 26fdc165; §4.3), and crates/tachyon/src/note.rs calls the commitment scheme “TBD”. The material is single-committer and churning: alternatives remain open in the tracker for the derivation itself (issues #86, #139) and for the absence-proof ranges it feeds (#87, #97, #98; all open, fetched 2026-07-08).

What a specification must pin down. The depth D and arity A (symbolic in the book; the crate pins working values, §3.4); the Poseidon instances, their parameters, and the byte-exact domain tags; the encoding of the epoch index into chunks; the uniqueness rule for ψ — the book states that two notes sharing ψ share mk and hence the same nullifier sequence, “so spending one publishes the other’s nullifiers” (nullifiers.md), yet no rule assigns responsibility for preventing reuse; the generators $\mathcal{G}_i$ and the window-commitment format; and the nonzero guards on published nullifiers, which the book motivates only in prose (a zero nullifier “would collide with the note’s own cm in the tachygram scan”, proof-tree.md).

What would falsify the design. A distinguisher on the per-epoch values: obliviousness rests entirely on nullifiers at distinct epochs being unlinkable, so a practical distinguisher for Poseidon-as-PRF in this mode — linking nf_e to $\text{nf}_{e'}$ without mk — collapses the construction. Likewise a derivation-chain break: a valid absence proof over values that are not the note’s genuine leaves would let a spender evade the double-spend check the whole system replaces the nullifier set with.

5.1.2 The PIR detection design

What exists. One sentence. Issue #121 (2026-05-26, verified via the GitHub API 2026-07-08) records the pivot in full: “Since we’re using PIR to make repeated payments flows without OOB,

there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows.” The same issue flags the book’s key-hierarchy chapter as stale, and the chapter carries the marker `<!-- todo: this is significantly outdated -->` (book/src/keys.md @ 26fdc165). Everything else written about payment detection predates this pivot and describes the abandoned fully-out-of-band design (blog “Tachyaction at a Distance”, 2025-05-15): removal of key diversification, viewing keys, and payment addresses, with unlinkability deferred to ZIP-321 payment requests and ZIP-324 encapsulated payments — the latter itself a 2019 draft whose rseed domain separator is the literal `0xFIXME` and which declares the transport channel out of scope (zips @ c6b70358). The project roadmap describes the PIR-plus-post-quantum-key-exchange payment protocol as developed independently in collaboration with the team; no document of any rigor describes it (tachyon.z.cash/roadmap, dateless, fetched 2026-07-08).

What a specification must pin down. The PIR family (single-server computational or multi-server information-theoretic, with the corresponding non-collusion assumption); the database the queries run against, who maintains it, and how a detection index is derived from a payment; the query and server cost model as the database grows; the post-quantum key exchange (named nowhere); and the retained in-band channel — its ciphertext format, its interaction with the removal of viewing keys, and which flows are entitled to it. The design currently consists of a sentence naming a primitive; every one of these is unwritten.

What would falsify the design. Cost: single-server PIR touches the whole database per query, so a database that grows with chain history can make detection more expensive than the trial decryption Tachyon set out to eliminate. Privacy: query-pattern or timing leakage at the PIR server re-introduces exactly the correlation channel of §5.2.1. Either failure, demonstrated at realistic scale, refutes the pivot rather than the parameters.

5.1.3 The sync-service trust model

What exists. Role prose. The ePrint note asserts services are fully oblivious to their clients’ transaction details and keep ephemeral per-client state; the book’s role table is the most precise statement: the sync service runs `UnspentSeed`, `EmptyBlockUnspentSeed`, `UnspentFuse`, and `UnspentEpochFuse` over nullifier values the wallet shared, and “never sees a note, cm, psi, or mk” (book/src/proof-tree.md @ 26fdc165). The security requirement is stated by the designers in the negative: “it is essential that the syncing service be incapable of distinguishing between transactions they assisted with and ones they did not” (ePrint 2025/2031, §1.3). The mechanism offered is epoch unlinkability plus proof re-randomization, the latter by citation: the problem “can be generally addressed by re-randomizing the proof, as many recursive proof systems permit, such as those involving accumulation schemes” (ibid., citing Bünz–Chiesa–Mishra–Spooner, ePrint 2020/499). On the implementation side, ragu does expose `Application::rerandomize` (§4.2), but its own API documentation warns that fused proof-carrying data is not zero-knowledge until re-randomization is performed, and the re-randomization path rests on an in-code “temporary hack” (crates/ragu_pcd/src/fuse/mod.rs, lib.rs @ 830bbcda). The corresponding ZIP stub is empty (book/src/zips/oss.md).

What a specification must pin down. A definition of “oblivious” as a game — what the adversarial service observes (requests, timing, shared windows), what it must not distinguish, and against which collusion (service alone, service plus network observer, several services comparing notes); what state a service may retain across sessions, “ephemeral” being currently a one-word promise; the client–service protocol itself, including handoff between services, which no document describes; and whether re-randomization is mandatory per exchange and on which side. No security definition of any of this exists in the corpus.

What would falsify the design. The designers’ own criterion, inverted: a service strategy that distinguishes assisted from non-assisted transactions. The timing-correlation channel of §5.2.1 is the known candidate, and the designers state plainly that its success would be fatal.

5.1.4 Epoch parameterization

What exists. An open issue. The epoch period — the single parameter the pruning model, the delegation window, the dual-nullifier rule, and the data-availability pressure all hinge on — is untuned: issue #79 (“Tune epoch period”, 2026-05-16, verified via the GitHub API 2026-07-08) records, beyond a pull-request reference, only that it “requires empirical analysis”. The GGM depth D and arity A , which bound the number of epochs a note can ever traverse, are symbolic throughout the book; the crate pins working values (§3.4) that no design document yet ratifies. The ePrint note supplies only the qualitative trade: with long epochs (“e.g., years”) the ecosystem “can perhaps organically support” storage of past nullifiers, but payment-scale throughput requires epochs “perhaps even as short as a single block”, which is precisely the regime that concentrates history into few providers (ePrint 2025/2031, §1.4).

What a specification must pin down. The epoch length in blocks and its activation semantics; D and A , and therefore the lifetime ceiling A^D epochs a note can survive; the default and maximum delegation window; and the reorganisation behaviour of the two-epoch consensus scan (§5.1.5), which the book does not treat at all.

What would falsify the design. The absence of a satisfying point: if every epoch length either (short) grows the historical nullifier database past what unincentivised parties store, or (long) grows the per-note absence-proof work and time-to-spend past what wallets and services sustain, then the parameter the design assumes exists does not. The designers’ own scaling narrative places the target regime at the short end, where their own data-availability warning applies (§5.2.2).

5.1.5 The rollover soundness model

What exists. A book-only consensus rule and a prose soundness argument. The published design rejects note expiry — the ePrint note considers expiring notes and nullifiers and discards it because “users must move their funds into a new note, once per window, or lose their funds” (ePrint 2025/2031, §1.1) — so any expiry-based description of Tachyon is obsolete; notes stay spendable indefinitely via delegated absence proofs. What replaces expiry is rollover: a spend publishes *two* nullifiers, for the anchor epoch and the next, and consensus checks every tachygram for duplicates over exactly the current and immediately preceding epoch, rejecting a duplicate within that two-epoch window (book/src/tachygrams.md @ 26fdc165). Everything older may be discarded: “Validators must only maintain the leading edge of revealed nullifiers (to prevent duplicates in concurrent transactions) but can otherwise permanently prune” (forum t/50789, post #22, 2025-11-04). The ePrint note carries only the looser “validators must still check for duplicates”. The book’s Soundness section argues the composition in prose: splice identities (Construction 3.5) checked at a Fiat–Shamir challenge, commit-equality binding of witnessed sequences, and cm-threading through the spendable lineage (book/src/proof-tree.md). There are no formal definitions, no theorems, and no machine-checked statements; the founding post itself concedes that recursive-soundness bounds of PCD constructions are “largely ignored in practice” (blog, 2025-04-02). The tracker carries live soundness and malleability issues against the design: “Bundle commitment is order-free over actions, so bundles are malleable to action reordering” (#137), unchecked padding in proof encodings (#161), and sub-block state landing in stamp order rather than transaction order (#135; all open, fetched 2026-07-08). Even the rule’s modality is unsettled: the anchor chapter hedges, in an in-file comment, between “may”, “should”, and “must” for which pool states consensus acknowledges (book/src/anchor.md).

What a specification must pin down. The consensus rule as normative text: the exact scan set, the epoch-transition tolerance, whether pruning is a permission or an obligation, and behaviour under reorganisation across an epoch boundary; the tachygram-set commitment format the scan runs over; and a soundness statement with a proof — at minimum, that two spends of one note accepted in any pair of epochs necessarily collide inside some single two-epoch scan window, given the dual-nullifier publication rule, and that the statement survives composition with the recursive proof system’s actual extraction bound.

What would falsify the design. A double-spend across the pruning boundary: two accepted spends of one note whose published nullifier pairs never meet inside one scan window. The two-epoch window is the entire replacement for the permanent nullifier set; a counterexample — or a demonstrated action-reordering exploit graduating from the open malleability issues — refutes the state-pruning claim that motivates the project.

5.2 Open problems

Each entry below is acknowledged as unsolved by the designers; we quote the acknowledgement. These are not gaps a specification could close by transcription — in each case the missing object is the construction itself.

5.2.1 Timing-correlation linkage

What exists. The problem statement, in the designers’ words: wallets hold multiple notes and update their unspent proofs together — “such as when first turned on after a period of inactivity, or charging on WiFi at home. Transactions spending these notes to different parties could still be linked together by their use of incremental unspent proofs that were computed at correlated times. This is not a small problem; in fact, it would be fatal to the entire approach” (ePrint 2025/2031, §1.3). The remedy on offer is one clause — re-randomizing the proof, by citation to accumulation schemes (ePrint 2020/499) — plus a gesture in the founding post at “network privacy countermeasures like mixnets” and a promised future post, which never appeared (blog index ends 2025-10-16). Ragu’s `rerandomize` exists as code (§5.1.3), but no published argument connects it to the property the ePrint needs: that a re-randomized proof is indistinguishable from one the service never touched, under the timing side information the service actually has. Collusion of a service with a network observer has no published analysis anywhere in the corpus.

What a resolution must establish. A formal indistinguishability property for re-randomized PCD in ragu’s accumulation regime, a proof that `rerandomize` achieves it, and — separately, because re-randomization does not touch timing — a traffic discipline (batching, cover queries, or a mixnet layer with a named threat model) that removes the correlation channel the designers describe. The transport side is currently one word in one blog post.

What would falsify the design. The designers have already stated it: a working correlation attack that links two spends through the service that helped maintain both “would be fatal to the entire approach”. This is the only entry in the corpus where the falsification criterion is supplied verbatim by the designers, and it remains open.

5.2.2 Data availability, liveness, and incentives

What exists. A designer-acknowledged hazard with no mechanism. Validators prune what wallets and services later need to prove absence over (§5.1.5); the history must therefore live somewhere off-consensus. The ePrint note describes the end state at scale: “one could easily imagine a point where the historical nullifier database grows so large that it can only be stored by a few highly centralized

providers ... the sudden shutdown of these services could render the funds of all their dependent users unspendable, potentially permanently” (ePrint 2025/2031, §1.4). No new layer is planned: “Tachyon will not involve a new DA layer; the blockchain will continue to be used to post nullifiers and note commitments, but not much else involving Tachyon” (forum t/50789, post #25, 2025-11-16); a DA layer “would probably exist solely to incentivize replication of the historic nullifiers/commitments, and more importantly to incentivize running oblivious syncing services” (ibid.). The designer’s stated position is deferral: “I’m looking forward to talking about this more in the future, but it’s not a concern of mine right now” (post #22, 2025-11-04). The community objection is on the record and unrebutted: building the non-inclusion proof “requires knowledge of every used nullifier” (post #19, 2025-10-29) — someone must keep everything consensus discards. No incentive design of any kind exists.

What a resolution must establish. Who stores the pruned nullifier and commitment history, under what incentive, with what replication factor; a liveness bound — how long a wallet’s funds remain spendable after its services vanish; and a recovery path for a user whose absence-proof lineage lapses over a pruned interval. The ePrint’s own remedy is a sketch of a few paragraphs, framed as a possibility (“The second possibility is to build a form of data-availability (DA) layer”, §1.4), not a design.

What would falsify the design. The designers’ scenario realised: the throughput target forces short epochs (§5.1.4), short epochs force a large historical database, storage centralises, and a provider shutdown strands funds — “unspendable, potentially permanently”. A scaling design whose failure mode at its own target scale is permanent loss of user funds, with no incentive mechanism even sketched, is refuted by the first such event; the open problem is to make the event impossible rather than unlikely.

5.2.3 Recoverability

What exists. An acknowledged regression and a one-sentence mitigation. Removing in-band secret distribution removes the property that the chain itself is the backup: “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (blog, 2025-04-02). The designers accept the cost as forced: “Getting this right will be challenging, but we simply have no choice but to do it” (forum t/50789, post #22, 2025-11-04). Asked directly, the answer remains non-committal: the replacement infrastructure “opens up a can of worms like who runs it? ... What if it’s the people building it like don’t exist anymore? How do I get my funds?” (ZK Podcast episode 388 transcript, 2026-01-21). The entire written resolution as of 2026-07-08 is the second sentence of issue #121: “We’re retaining in-band secret distribution for recover from seed flows” — an issue whose first sentence declares the book stale against it. The roadmap’s alternative delivery path warns of “backup/recoverability trade-offs”; a direct forum question about seed-phrase recovery (2025-10-15) went unanswered.

What a resolution must establish. A complete recover-from-seed procedure: what the retained in-band channel carries and where it lives in a protocol whose validators prune; how a restored wallet discovers its notes’ creation epochs and positions; and how it rebuilds an absence lineage over history that consensus discarded — which makes recoverability a dependent of the data-availability problem (§5.2.2), a dependency no document states.

What would falsify the design. A seed-only restoration that cannot terminate: a user with a correct seed phrase whose funds are unrecoverable because the services holding the required history no longer exist. For a system marketed as a payment protocol upgrade, demonstrated non-recoverability at any realistic scale refutes the wallet model, not a parameter of it.

5.2.4 Tip-nullifier exposure versus the obliviousness claim

What exists. A marketing claim and a design detail in tension. The project overview states: “the service never learns your actual nullifiers, because the protocol forces them to periodically evolve in an unlinkable way” (tachyon.z.cash/overview, fetched 2026-07-08). The design book’s delegation construction hands the service one nullifier per epoch-boundary crossing “*plus the nullifier of the epoch in progress at the span’s tip, which is carried separately because that epoch is not yet complete*” (book/src/nullifiers.md @ 26fdc165). The tip value is not a spent, superseded nullifier: the spendable’s current nullifier is “the nullifier the wallet would publish to spend now” (ibid.), so a service holding it can watch the pool and recognise the client’s spend in that epoch — linking a service session to an on-chain transaction, exactly the linkage obliviousness is sold as preventing. The role table shows the escape: UnspentSeed and the other segment steps are marked *possible* for the wallet (book/src/proof-tree.md), so a wallet can keep the leading edge private and delegate only completed epochs. No document mandates that flow, and none analyses the leak when it is not followed.

What a resolution must establish. Either a mandated flow in which the tip nullifier never leaves the wallet — with the wallet-side cost of sealing each epoch stated — or a bound on what a service holding the tip learns, under the timing side information of §5.2.1. Until one of these exists, the overview’s “never” is falsified by the design book’s own construction, and the accurate statement is: the service never learns *past* nullifiers, and learns the current one unless the wallet does extra work no document requires of it.

What would falsify the design. Service-side spend detection in the delegated flow — trivially demonstrable today from the book’s data flow. The narrow claim is already false as advertised; what remains open is whether the protocol will mandate the wallet-side variant that makes it true.

5.3 Quantitative claims without a published basis

The headline figures attached to Tachyon are not project measurements, and a volume at this series’ standard must say so plainly.

- The homepage states that Tachyon “shrinks transactions by two orders of magnitude and removes runaway state growth for validators” (tachyon.z.cash, fetched 2026-07-08). No published benchmark, size specification, or derivation supports the factor; it refers to marginal on-chain size after aggregation and miner-side stripping, and no primary document computes it.
- The circulating throughput figures — roughly 2,200 TPS, or 660 TPS at ~1.5 kB per transaction, and a 9.3 kB → 450 B size reduction — trace to Messari and CoinDesk analyst reports (the latter commissioned by GenZcash, 2026-06-30), not to the project. They are analyst estimates. Bowe’s posts contain no TPS numbers, and the founding post’s promised PCD-toolkit benchmarks (2025-04-02) have not appeared.
- The only in-project performance numbers are ragu’s: verifier time “approximately 100ms” at recursion threshold $K = 11$ and a break-even against Orchard batch verification at roughly 50 transactions — self-labelled “rough estimates that should be validated with empirical benchmarks” (ragu book, protocol/analysis.md @ 830bbcd4). A criterion bench harness runs in CI and publishes no numbers.
- The claim that syncing services do only “a similar amount of computational effort (asymptotically)” to today’s validators is prefixed “I believe” in its only source (forum t/50789, post #28, 2025-11-21); no complexity analysis or benchmark exists.
- The founding post’s timeline — “many of the major scalability improvements should be able to hit mainnet within a year” (2025-04-02) — is falsified: as of 2026-07-08 nothing runs on testnet

or mainnet, no ZIP is upstreamed, the core ZIP stubs are empty, the crate’s proof layer is stubbed, and ragu is explicitly pre-audit. The current roadmap carries no dates.

5.4 Revisit triggers

The classifications above are dated 2026-07-08 and decay. Any of the following moves material between bins and obsoletes parts of this section: merge of ZIP-248 (the transaction-format substrate every Tachyon bundle design defers to, open as zcash/zips PR #1156); upstreaming of the project’s four fork amendment drafts (PRs #1–#4, open since 2026-06-02); a filled core ZIP stub (#103–#107); a published PIR payment protocol; and ragu’s first published benchmark or audit. The update-ZIP integrations remain undecided in the designers’ own words — “still under discussion and nothing has been decided yet” for the ZIP-221 history-tree leaf and the ZIP-317 fee contribution, “blocked on ZIP-248 reaching editor consensus” for bundle registration (book/src/zips/ @ 26fdc165).

6 Security posture and outlook

This closing section states the volume’s synthesis. We compress the preceding chapters into a single conservation law (§6.1), place Tachyon at its one point of contact with the post-quantum programme (§6.2), fix its relation to the surrounding consensus roadmap (§6.3), and end with the volume’s verdict: the concrete artifacts that must exist before any Tachyon claim can be checked rather than believed (§6.4). Ground truth is pinned as throughout the volume: the design book at tachyon-zcash/tachyon @ 26fdc165 (2026-07-07), Ragu at tachyon-zcash/ragu @ 830bbcd4 (2026-07-05), the ZIP corpus at zcash/zips @ c6b70358 (2026-07-05); web sources carry individual dates. The material compressed here is constructed in §2.1 (the proof-carrying fold and its nineteen steps), §5.1.1 (nullifier evolution), and §5.1.3 (the sync-service model); Ragu’s code state is §4.2; the open problems are §5.2.

6.1 The conservation law

The designers claim the scaling gain costs nothing in the two properties Zcash exists to provide. The eprint promises “continual, permanent pruning of nullifiers by validators”, achieved “without compromising privacy”; its syncing services are *untrusted*, hold only “ephemeral state per client”, and “cannot link their clients to any transactions that ultimately appear on the public ledger” (Bowe–Miers, eprint 2025/2031, abstract; revised 2025-11-03). Nothing in the design asks a user to trust a third party with funds or with integrity — a service that lies is caught by proof verification — and the assumption family, Pallas/Vesta discrete logarithms plus random-oracle heuristics, is the one already deployed in Orchard. In this exact sense soundness and privacy are preserved *in principle*.

The qualification carries weight. Soundness of the proof tree is argued in prose (the book’s Soundness section, book/src/proof-tree.md @ 26fdc165; the eprint contains zero theorems and no formal security definitions); the term “oblivious” has no security definition anywhere in the corpus; the hiding of the new Poseidon-based commitments is asserted, not proven; and the founding post itself concedes that the recursive soundness bounds of PCD constructions are “largely ignored in practice” (Bowe, “Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02). Each of these is *designed but unspecified* at best, and none changes bins in this section.

What the design spends is availability. Pruning does not destroy the history a wallet needs; it relocates the obligation to keep it. Validators drop exactly what wallets later require to prove their notes unspent, and someone must hold it. The designers rule out a protocol-level home for that obligation: “Tachyon will not involve a new DA layer; the blockchain will continue to be used to post nullifiers and note commitments, but not much else involving Tachyon” — while observing that under Tachyon users “become more sensitive to the availability of the history in order to make their funds spendable”, and that a data-availability layer, if one arose, “would probably exist solely to incentivize replication” (forum t/50789 #25, 2025-11-16). The computation relocates the same way: the

expectation that syncing services will spend “a similar amount of computational effort (asymptotically) that validators currently do” is a designer’s “I believe” (forum t/50789 #28, 2025-11-21) with no published complexity analysis behind it.

We compress the relocation into a pick-two. Deployed Orchard provides three availability properties simultaneously — and pays for them with the unbounded validator state Tachyon exists to remove:

- (i) *full obliviousness*: no helper party learns anything about the wallet’s notes or transactions;
- (ii) *no liveness dependence*: spendability does not require any particular external party to be running;
- (iii) *chain as complete backup*: a seed phrase plus the public chain recover all funds.

Once validators prune, a wallet retains at most two. Keeping (ii) and (iii) means the chain retains everything a recovering wallet needs — that is today’s protocol, not Tachyon. Keeping (i) and (ii) means the wallet ingests and stores the pruned nullifier history itself, continuously; the chain stops being its backup, exactly as the founding post concedes: “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (2025-04-02). Keeping (iii) through services — delegated absence proofs reconstructing spendability from seed — surrenders (ii): “the sudden shutdown of these services could render the funds of all their dependent users unspendable, potentially permanently” (eprint 2025/2031, §1.4). And the retained (i) is itself the design’s most fragile member: correlated proof-update timing linking a wallet’s transactions “is not a small problem; in fact, it would be fatal to the entire approach”, with the remedy — proof re-randomization — supplied by citation rather than construction (eprint 2025/2031, §1.3); this volume classifies it an *open problem*.

The project’s own trajectory moves inside this triangle rather than out of it. The May-2026 pivot retains “in-band secret distribution for recover from seed flows” (tachyon-zcash/tachyon issue #121, 2026-05-26), buying back the note-discovery half of (iii) at the price of on-chain ciphertexts for those flows, while spendability from seed still rests on delegated absence proofs and therefore on (ii); the roadmap’s fallback — dedicated note-delivery infrastructure — concedes “UX or backup/recoverability trade-offs” in as many words (tachyon.z.cash/roadmap, fetched 2026-07-08). Classification of the trilemma itself: the framing is this volume’s; every fact inside it is designer-acknowledged and individually cited; and no Tachyon document states the trade-off in one place. A specification will have to, because the chosen corner determines what a wallet must store — the difference between a seed phrase that recovers funds and one that does not.

6.2 The post-quantum intersection

The post-quantum analysis of this series (the *Ironwood Guide*, §“Post-quantum security of the Orchard protocol”) reaches one headline conclusion about deployed Orchard: exactly one quantum threat is retroactive. Note encryption is the canonical harvest-now-decrypt-later target — every (epk, C_{enc}) pair recorded on chain today becomes decryptable by a future discrete-log-capable adversary who also holds the recipient’s address, exposing amounts, memos, and the full receiving history, permanently — while every integrity break (Sinsemilla binding, value-commitment binding, RedPallas forgery, Halo 2 soundness) is prospective and mitigable by switching pools off in time. The deployed mitigation programme declines to touch the retroactive item: “The situation with respect to Privacy is unchanged for any pool”; the note encryption of the deployed pools “is subject to ‘Harvest Now, Decrypt Later’ attacks”, with countermeasures only “under consideration” (ZIP 2005, Non-requirements and Privacy discussion, zips @ c6b70358).

Tachyon is the only artifact in the corpus that addresses this liability, and it does so structurally rather than cryptographically: it removes the ciphertext. “We plan to remove in-band secret distribution from Tachyon transactions to fully defend against all quantum attacks on privacy in on-chain transactions. This step is essential, because harvest now, decrypt later threats are perilous

for blockchain protocols” (Bowe, “Zcash and Quantum Computers”, 2025-10-16). Where no key-agreement ciphertext is published, there is nothing to harvest, and the remaining on-chain objects — hiding commitments, zero-knowledge proofs, re-randomized keys — leak nothing to a quantum adversary who lacks the address. Three qualifications bound the claim. First, the defence is prospective only: ciphertexts already recorded remain harvestable forever; no protocol change can repair them. Second, the pivot of issue #121 retains in-band distribution for recover-from-seed flows, and whether those ciphertexts use the deployed discrete-log key agreement or the “post-quantum key exchange” the roadmap names for its PIR payment protocol is pinned down nowhere — the PIR protocol has no published specification at all (tachyon.z.cash/roadmap, fetched 2026-07-08). Third, no post-quantum privacy analysis of Tachyon exists; the statistical hiding of its Poseidon-based commitments is asserted, not proven. Bin: *designed but unspecified*, and the recovery-flow cryptography is the first thing a specification must fix before the post-quantum privacy claim becomes evaluable.

On the soundness side the same project deepens the discrete-logarithm commitment. Ragu is an ECDLP-based recursive SNARK by declared design: the requirement of small recursive proofs “essentially limits us to ECDLP-based constructions with our existing payment protocol architecture”, with a footnote recording “reduced concern (in the medium term) for post-quantum *soundness* risks” that cites the same blog post (book/src/protocol/index.md, ragu @ 830bbcd). The designers’ asymmetry argument is explicit: “Quantum privacy breaks are usually retroactive, whereas soundness breaks are usually not” (Bowe, 2025-10-16); a prospective soundness failure is answerable by reaction — turnstiles bound inflation, and ZIP 2005 re-anchors note ownership in symmetric primitives so that funds survive a migration. The argument is coherent, and it prices one thing incompletely: depth. Under Tachyon the discrete-log assumption no longer underwrites a single Action circuit but the recursion substrate itself, and the wallet’s state — the proof-carrying data that replaces the chain as the user’s record — is a stack of DL-sound proofs whose recursive soundness bounds the founding post already concedes are “largely ignored in practice” (2025-04-02). A future migration off the assumption is then a migration of the state model, not the replacement of one circuit. Bin: *open problem*, jointly with the unquantified recursion-soundness bound; no document in the corpus analyses either.

6.3 Relation to Crosslink and the upgrade environment

Crosslink — the finality/proof-of-stake effort led by Shielded Labs — and Tachyon are formally independent. The founding post sees “no reason to believe that Tachyon will conflict with any of the active areas of development such as Crosslink and ZSAs” (2025-04-02); Shielded Labs states that “Crosslink and Tachyon are separate efforts that don’t necessarily interact with each other with regards to roadmapping + timelines” (forum t/50789 #11, 2025-10-28); and no interaction analysis exists in either direction — a repository-wide search of the ZIP corpus finds Crosslink only in ZIP 218 (25-second block target spacing, Status: Draft, created 2026-03-13; zips @ c6b70358, searched 2026-07-08): one body sentence declaring the proposal “complementary to, and does not compete with,” finality mechanisms such as Crosslink, plus the citation footnote. The independence is therefore asserted, not examined; both proposals alter the environment the other assumes (consensus finality and block cadence on one side, validator state and epoch-windowed double-spend checking on the other), and Tachyon’s epoch period is itself untuned (tachyon-zcash/tachyon issue #79). The near-term environment is fixed without Tachyon: NU6.2 is settled (ZIP 257, Status: Final — the emergency Action-circuit correction), NU6.3 carries the Ironwood pool and the ZIP 2005 recoverability programme (ZIPs 258, 229, 2005; Draft/Draft/Proposed), and Tachyon’s own standing is a mandate without a vehicle: “universal or near-universal support across every group ... the most unambiguous signal from the entire poll” in the February 2026 NU7 sentiment poll, with the stated next step only to “assess technical readiness and scope for inclusion in NU7” (zfnd.org, NU7 Polling Results, 2026-02-23), while the deployment draft (draft-arya-deploy-nu7.md, heights TBD, zips @ c6b70358) lists no Tachyon ZIP. Beyond NU7 there is nothing to relate Tachyon to: the ZIP corpus contains no NU8 artifact of any kind (repository-wide search of zcash/zips @ c6b70358, 2026-07-08); the relation to

NU8 becomes statable only when an NU8 deployment draft exists.

6.4 The checkability boundary

Every claim in this volume carries one of three labels: *specified*, *designed but unspecified*, or *open problem*. The distribution of those labels is the verdict. The specified bin contains standing and environment facts only — the poll result, editor affiliations, the NU6.2/NU6.3 context, the ZIP 321/324 out-of-band substrate, and Ragu’s code state — and not one byte format, consensus rule, or security theorem of Tachyon itself. Three artifact classes would move the boundary.

Benchmarks. No performance figure exists at project rigor. The founding post promised benchmarks of a proof-carrying-data toolkit “to set a floor on the performance of shielded transaction aggregation and oblivious syncing services” (2025-04-02); none has appeared. The only in-project numbers — roughly 100 ms aggregated verification at recursion threshold $K = 11$ and a break-even against Orchard batch verification near 50 transactions — are self-labelled “rough estimates that should be validated with empirical benchmarks” (book/src/protocol/analysis.md, ragu @ 830bbcdca); the CI bench harness publishes no output; the circulating throughput and size figures are analyst estimates, not project figures; and the homepage’s two-orders-of-magnitude headline has no published derivation. A meaningful benchmark moreover requires code that does not yet exist: Ragu’s Proof carries full polynomials, verification is linear-time, and no inner-product opening argument or decider is implemented (crates/ragu_pcd/src/{proof/mod.rs,verify.rs} @ 830bbcdca). Until a benchmark of non-mock Ragu PCD with succinct proofs exists, every scaling claim is unfalsifiable.

The nullifier-evolution specification. The one genuinely novel cryptographic object — the evolving nullifier — exists at eprint-sketch and design-book rigor only: the eprint offers a PRF sketch “as a concrete example” (2025/2031, §1.3); the concrete GGM/Poseidon derivation, the dual-nullifier spend, and the two-epoch duplicate window live only in the book (book/src/{nullifiers,tachygrams}.md @ 26fdc165); the crate stubs the derivation and calls the commitment scheme “TBD” (crates/tachyon/src/note.rs); the derivation is churning through open alternatives (issues #139, #86, #87/#97/#98) with live soundness and malleability issues (#161/#137/#135); and the epoch period, GGM depth, and arity on which the entire pruning model hinges are untuned (issue #79, “requires empirical analysis”). A specification must pin the derivation formulas and domain separators, the epoch parameters, the dual-nullifier consensus rule and the pruning rule, and — above all — the security definitions (per-epoch unlinkability, service obliviousness) that nothing in the corpus states formally.

Core ZIPs. The consensus path is empty. The five core ZIPs that would carry the protocol — Shielded Protocol (#103), Bundle (#104), Accumulator (#105), Aggregator (#106), OSS (#107) — are headers-only stubs of 216–245 bytes each (book/src/zips/ @ 26fdc165); the four peripheral amendment drafts sit unmerged on the project’s own fork (tachyon-zcash/zips PRs #1–#4, opened 2026-06-02, open as of 2026-07-08); upstream zcash/zips carries zero Tachyon ZIPs; and the extensible-format substrate everything defers to, ZIP 248, is itself an unmerged PR idle since 2026-05-26 (zcash/zips#1156). Beside the ZIPs stand Ragu’s own declared gates: a Fiat-Shamir domain tag that is literally b"FIXME" (crates/ragu_pcd/src/lib.rs:50--51), registry binding flagged `FIXME(security)` as insufficient (crates/ragu_circuits/src/registry.rs:673--692), three missing verifier checks (crates/ragu_pcd/src/verify.rs:122), and no audit: “Ragu is under heavy development and has not undergone auditing” (README.md @ 830bbcdca).

The events that would obsolete this volume’s classification are the revisit triggers of §5.4, read forward. Until they occur, the posture admits a one-sentence summary: soundness and privacy are conserved in intention, availability is spent by design, and none of it is yet checkable.

Part XIII

Crosslink Guide: Trailing finality for Zcash: the construction, its formal models, and the wallet-visible contract

Abstract

Assuming the Orchard protocol of the *Ironwood Guide* and the wallet layer of the *Wallet Guide*, this volume of *The Zcash Arboretum* documents Crosslink, the trailing-finality construction targeted at Zcash: the ebb-and-flow model and what its theorems actually prove, the construction as the Trailing Finality Layer book specifies it, stake and delegation as far as any source pins them down, and the wallet-visible contract two ledgers create. The design is in motion and its sources disagree — the implementation drops mechanisms the book calls central, and the machine-checked model formalises a neighbouring rule set — so every claim carries its classification, every divergence is stated rather than resolved, and the questions a future ZIP must answer are listed as such.

1 Scope, status, and sources

Crosslink is a hybrid proof-of-work/BFT (Byzantine-fault-tolerant) consensus construction for Zcash: it retains the existing best-chain (PoW) protocol and adds a trailing finality layer (TFL), a BFT protocol (one that reaches agreement by explicit votes and remains correct while at most a bounded fraction of its participants misbehave arbitrarily) whose blocks and the best-chain blocks commit to one another — the eponymous cross-links. The current version is Crosslink 2 (tfl-book @ fe6e1d6, src/design/crosslink.md lines 1–3; the-arguments-for-bounded-availability-and-finality-overrides.md line 9). The construction descends from the ebb-and-flow security model and the Snap-and-Chat construction of Neu, Taş, and Tse (“Ebb-and-Flow Protocols: A Resolution of the Availability–Finality Dilemma”, eprint 2020/1091; arXiv 2009.04987, v1 2020-09-10, v3 2021-02-04; published IEEE S&P 2021). Snap-and-Chat’s two-ledger architecture Crosslink keeps; its show-stopping defect for Zcash — users cannot in general spend outputs of finalized transactions taken from a rolled-back fork’s snapshot, because chain validity in Π_{lc} , Snap-and-Chat’s longest-chain sub-protocol, cannot depend on a node’s local finalized ledger — is what the TFL book records as forcing the redesign: “We are forced to conclude that the chain ch_i^t must include the information needed to calculate LOG_{fin} ” (tfl-book @ fe6e1d6, notes-on-snap-and-chat.md lines 73–107). No successor paper covering Crosslink itself has appeared (arXiv listing checked 2026-07-15).

Nothing about Crosslink is deployed on any Zcash network and nothing is consensus-specified: the ZIP repository at 6961098 (2026-07-09) contains no Crosslink, TFL, or NU8 ZIP or draft, and the implementers’ own ZIP draft is a Google document that “will enter the formal ZIP process as the prototype approaches maturity” (grep of zcash/zips @ 6961098; zebra-crosslink @ 6d02a1b, book/src/design.md line 7). This volume is therefore a design-stage volume in the sense the *Tachyon Guide* established (§“Scope, sources, and method”): it fixes the current state of a moving design against commit-pinned sources, classifies every claim by rigor, and asserts nothing normatively. The design corpus splits three ways — a construction book that has been static since 2024-12-13, an implementation whose design has since moved past the book in documented ways, and a machine-checked model of one narrow slice — and the first duty of this section is to say which document is authoritative for what.

1.1 The best-chain protocol as a black box

Legacy consensus internals are the *Consensus Guide*’s subject, and this volume does not re-derive them for proof of work. Equihash, difficulty adjustment, and block relay never appear here; the best-chain protocol enters only through the interface Crosslink’s model assumes of any Π_{bc} (tfl-book @

fe6e1d6, construction.md lines 245–281): a set of chains rooted at a genesis block; a bc-block-validity rule depending only on the block and its ancestors; block production hard unless selected in proportion to resources; a score function strictly increasing along a chain (accumulated work, for PoW); honest nodes adopting a highest-scoring valid chain; and headers that commit to blocks, with header validity (PoW and difficulty adjustment) checkable before block download. The book’s own assessment is that “Typically, Bitcoin-derived best chain protocols do not need much adaptation to fit into this model” (construction.md line 281).

The safety assumptions Crosslink places on this black box are Prefix Consistency at confirmation depth σ (PSS2016’s “consistency”) and Prefix Agreement at depth σ , both stated as unconditional properties of executions; the probabilistic reasoning that justifies them is deliberately separated out, with NTT2020 Theorem 2 supplying the corresponding bound $e^{-\Omega(\sqrt{\sigma})}$ on rollback failure (tfl-book @ fe6e1d6, construction.md lines 285–374, 351–357). We state these assumptions formally in the construction section and use them as the interface; we never derive them from PoW internals. Where the interface touches deployed reality we cite the deployed code: Zebra bounds its best non-finalized chain at `MAX_BLOCK_REORG_HEIGHT = 1000` blocks, and its own comment concedes that “deeper reorganisations are outside Zebra’s rollback window. This is a local-only node policy; it is not part of consensus” (zebra @ 9f3166b, zebra-chain/src/parameters/constants.rs lines 20–30) — the standing admission that today’s Zcash has no consensus-level finality at any depth.

1.2 The three bins, and a proved tag

Design-stage prose invites the failure mode the deployed-protocol volumes never face: text that reads as specification but rests on a draft. Following the series convention and the *Tachyon Guide* (§“The three-bin method”), every claim in this volume is exactly one of *specified* (a pinned artifact at specification rigor fixes it; we cite file and commit), *designed but unspecified* (an informal source describes a concrete mechanism no specification fixes; we cite source and date), or an *open problem* (no source resolves it; where the designers acknowledge the gap, we quote them). Crosslink adds a dimension Tachyon lacked: parts of its corpus carry actual proofs. We therefore additionally tag a claim *proved* when a peer-reviewed theorem or a machine-checked artifact establishes it, always together with the scope at which it was established.

The inventory, developed across the rest of the volume, maps as follows. Proved: the NTT2020 theorems for ebb-and-flow and Snap-and-Chat (peer-reviewed, IEEE S&P 2021), and the Quint invariant `prefix_inv` at its small checked scope (§1.3). Specified (pinned, verifiable artifacts only): the Quint `isValid` model itself and the quoted prototype and Zebra constants. Designed but unspecified: the Crosslink 2 construction rules and the two Assured Finality safety theorems, with book-level prose proofs — one of which is textually defective (§1.3), leaving that theorem effectively unproven — plus the zebra-crosslink staking and tokenomics design, the BFT subprotocol instantiation (tenderlink), active-set selection, slashing, and the implementation’s no-Stalled-Mode variant. Open: precise liveness bounds and the dependence on L , completion of the LOG_{ba} safety development, production values of σ and L , and NU8 inclusion and dates (classification synthesized from the sources cited throughout this section).

1.3 The source landscape

Table 1 lists the sources, pinned. All repository facts were verified firsthand at the stated commits; web sources were fetched 2026-07-15.

Current-head boundary (2026-08-30). The four Crosslink design repositories were rechecked: tfl-book remains at fe6e1d6, crosslink-spec at 8edc3e9, zebra-crosslink at 6d02a1b, and crosslink-protocol-guide at b77d845. Current upstream zips ad30e59 still contains no Crosslink or NU8 ZIP. The repository-backed design and classification below therefore remain current; web-sourced rollout statements retain their explicit 2026-07-15 access date.

Artifact	Pin / date	Role and rigor
tfl-book (ECC)	fe6e1d6, 2024-12-13	construction; static since pin
Neu-Taş-Tse	eprint 2020/1091; S&P 2021	peer-reviewed lineage
crosslink-spec (Informal Systems)	8edc3e9, 2025-07-23	Quint model; proof of concept
quint-crosslink (zmanian)	empty upstream (zero refs)	nothing to examine
zebra-crosslink (Shielded Labs/Eiger)	6d02a1b, 2026-04-21	prototype + own design book
zcash/zip	6961098, 2026-07-09	zero Crosslink/NU8 ZIPs
zcash/zebra	9f3166b	deployed Π_{bc} facts
crosslink-protocol-guide	b77d845, 2026-03-26	secondary; visual tour
Shielded Labs web/forum/Arborist	fetchd 2026-07-15	status only, dated

Table 1: Primary Crosslink sources, pinned. Verification date 2026-07-15.

The TFL book. The construction source of record is the ECC Trailing Finality Layer book, an mdBook formerly deployed at <https://electric-coin-company.github.io/tfl-book/> (404 since at least 2026-07-15, following the repository’s transfer to github.com/daira/tfl-book) and now rendered at <https://daira.github.io/tfl-book/>; the local clone HEAD is fe6e1d6, dated 2024-12-13, the merge of PR #162 “crosslink2” by Daira-Emma Hopwood, with no later commits (git log). Prehistory: a design sketch was presented at Zcon4 by Nate Wilcox (“Interactive Design of a Zcash Trailing Finality Layer”); book version 0.1.0 introduced Crosslink, and 0.2.0 updated the construction and security analysis to Crosslink 2 (src/version-history.md). Three rigor caveats govern every citation of it. First, the book self-describes as “an early and incomplete protocol design proposal”, listing as missing the PoS subprotocol selection, issuance and supply mechanics, transaction semantics integration, the transition plan, ZIPs, and the security and economic analyses (src/introduction/status-and-next-steps.md lines 1–30). Second, it is internally inconsistent about its own version: book.toml still titles the deployed book v0.1.0 while src/version-history.md records 0.2.0 (“Crosslink 2”) as current (book.toml line 9 vs src/version-history.md lines 3–5). Third, and most consequentially, its rigor is uneven across chapters: the construction chapter is specification-grade prose, but the liveness argument carries explicit TODOs (“make that more precise”; “more detailed argument needed, especially for the dependence on L ”), and the safety section is marked “\$TODO\$ Not updated for Crosslink 2 below this point” immediately after it, with the $\text{LOG}_{\text{fin}}/\text{LOG}_{\text{ba}}$ proofs below that marker still written in Snap-and-Chat/Crosslink-1 notation — including the sanitization machinery Crosslink 2 exists to remove (security-analysis.md lines 5–50, 54, 139–150). Two further defects are findings of this volume, developed in the security-analysis section: the proof text under “Safety Theorem: Final Agreement of Π_{bft} implies Assured Finality” is a verbatim duplicate of the Prefix Agreement proof and never uses Final Agreement (security-analysis.md lines 87–91 vs 73–77), and the book itself concedes that “some rules, e.g. the Linearity rule, only contribute heuristically to security in the analysis so far”, with the LOG_{ba} development trailing off mid-sentence (security-analysis.md lines 275–281, 257).

The machine-checked model. The repository crosslink-spec (Informal Systems) is pinned at 8edc3e9 (2025-07-23, the merge of PR #4; shallow local clone, depth 1). It formalizes exactly one slice of Crosslink: the isValid check of a BFT proposal against local PoW and BFT block stores, written in Quint as a literate source (validity.md, tangled to src/validity.qnt via lmt; a transpiled validity.tla is committed) (README lines 25–32; validity.md lines 1–7). Its own README fixes its standing: “a proof of concept”, whose logic was “inferred ... from an insightful Youtube talk” and “might not be up-to-date with the current protocol design ideas” (README line 27). What was actually checked is premium material at a stated scope: the invariant prefix_inv — the PoW history defined by BFT finalization is a path in the PoW block store — survived random simulation of 10,000 traces of 20 steps and TLC model checking in about three minutes at scope confirmation_depth = 1 with stores of at most 8 blocks, and a three-block finalization witness trace was produced by both sim-

ulation and TLC (README lines 61–93; `validity.md` lines 376–421, 464–481). The model diverges from the book in ways we record as model-versus-specification discrepancies, not protocol facts: its `linear()` requires a strict-ancestor snapshot where the book’s Linearity rule permits repeating the parent snapshot (and the book needs that permission for BFT liveness; the Quint source itself comments “this actually seems allowed. TODO: perhaps remove”); it models the BFT store as a single linear list with no forks and omits signatures, notarization proofs, epochs, voting, `bft-last-final`, and every Π_{bc} -side rule; and its `add_pow` assigns block hashes colliding with genesis, an artifact PR #4 fixed only on the BFT side (`validity.md` lines 173–176, 132–137, 244–247, 270–278, 433–440 vs `tfl-book construction.md` lines 573, 611–618, 113–193). The second Quint repository, `quint-crosslink` (remote `zmanian/quint-crosslink`), is empty upstream as well as locally — `git ls-remote` returns zero refs (checked 2026-07-15); there is nothing to examine.

The implementation and its fork of the book. The prototype is `zebra-crosslink`, Shielded Labs’ fork of Zebra built together with Eiger; the local clone HEAD is `6d02a1b`, dated 2026-04-21, the merge of PR #282. It ships its own `mdBook` whose `cl2-construction.md` and `cl2-security-analysis.md` are declared “almost direct paste” copies of the TFL book chapters, committed verbatim “for the purposes of precisely committing to a ... precise set of book contents for a security design review”, with the explicit note that Daira-Emma Hopwood and Jack Grigg “have not reviewed and do not necessarily endorse” the changes (`book/src/design/cl2-construction.md` lines 3–7). The fork then diverges from the book in two documented, load-bearing ways. It drops Stalled Mode entirely — “The zebra-crosslink design in this book diverges from this text by *not* including ‘Stalled Mode’ rules” — relying instead on stakers redelegating to prevent hazards such as BFT stalls (`cl2-construction.md` lines 10–11); and it removes the proposer’s outer signature on `bft-blocks`, conceding that “a direct hash or copy of the most recent BFT finality certificate is insufficient evidence to ensure that specific bitwise copy is canonical” (`cl2-construction.md` lines 14–15). The fork’s own warnings bound what this volume may claim for it: “These extensions and changes from the Crosslink 2 construction may violate some of the assumptions in the security proofs” and “We are using a custom-fit in-house BFT protocol for prototyping. Its own security properties need to be more thoroughly verified” (`book/src/design.md` lines 11–13). That in-house engine is `tenderlink` (`git.sr.ht/~azmr/stuff`, rev `0e88509`), Shielded Labs’ lightweight Tendermint implementation that replaced the earlier Malachite (Informal Systems) integration — at the pinned commit `Cargo.lock` contains no `malachitebft` crates and `src/malctx.rs` survives only as dead code — with `ed25519-zebra` finalizer keys and BLAKE3 block hashes (`zebra-crosslink/Cargo.toml` lines 74–79; `src/lib.rs`; `src/chain.rs` lines 224–226); the prototype parameters are $\sigma = 3$ and finalization gap bound 7, with the source stating “No verification has been done on the security or performance of these parameters” (`src/chain.rs` lines 219–221). The staking design (the “nutshell” chapter) self-describes as a “provisional sketch” and matches Shielded Labs’ published staking design post of 2025-10-27 on Orchard-only staking with revealed amounts, one-epoch unbonding, and power-of-ten stake quantization; on epoch length the sources diverge — the post proposes five-day epochs, the chapter a one-day staking day plus an approximately six-day locked phase (`book/src/design/nutshell.md` lines 13–133; <https://shieldedlabs.net/crosslink-updated-staking-design/>, accessed 2026-07-15).

Secondary sources. The community “A Visual Tour of Zcash Crosslink” (`crosslink-protocol-guide` @ `b77d845`, 2026-03-26, MIT) is a short mermaid-diagram tour with a TFL lexicon; we use it for orientation only, never as a claim source. Incidental ZIP references exist — draft ZIP 218 (“25-second Block Target Spacing”, created 2026-03-13) cites Crosslink as complementary, and draft-ecc-onchain-accountable-voting cites the TFL book’s Assured Finality definition — but neither specifies any part of Crosslink (`zips` @ `6961098`, `zip-0218.md` lines 77, 742).

1.4 Status: milestones, testnets, and the absence of dates

Shielded Labs’ public roadmap records five completed milestones: M1 “Placeholders and Processes” (2025-03-21), M2 “Disconnected BFT” (2025-05-16), M3 “PoW Consensus” (2025-08-28), M4 “Staking Transaction Semantics” (2026-01-28), and M5 “Pre-Productionization, Tech Debt, Final Refactoring” (2026-04-15). The Productionization Phase is stated as Q2 2026 – Q2 2027, in progress, described as deploying Crosslink on a testnet, iterating, “and, if there is consensus, integrating it into the Zcash protocol”; no NU8 or mainnet activation date is given anywhere on it (shieldedlabs.net/roadmap/, accessed 2026-07-15).

Deployment practice runs ahead of specification. FeatureNet, a series of incentivized testnets announced 2026-04-02, launched Season 1 on 2026-04-15: miners, finalizers, and stakers earn real ZEC, with 500 ZEC allocated across seasons (Season 1: 25 ZEC, split 40% miners / 40% stakers / 20% Dev Fund), and the announcement states that “any decision to activate Crosslink on mainnet will follow Zcash’s existing governance process” ([forum.zcashcommunity.com t/55210](https://forum.zcashcommunity.com/t/55210), accessed 2026-07-15). As of the 2026-06-25 Arborist call, two FeatureNet workshops had completed, “using the current stalled network to strengthen tooling, improve diagnostics, and better handle rollback and fork scenarios rather than simply resetting the network”; Workshop 3 was tentatively set for 2026-07-23, and the team was “prototyping a user-driven slashing tool, drafting ZIPs for the Proof-of-Stake subsystem, and implementing protocol improvements” (zechub.substack.com, published 2026-06-27, accessed 2026-07-15).

On dates this volume is deliberately silent, because the primary sources are. The community “NU8 Decision Framework” thread (opened 2026-02-11) records that no governance framework or dates exist for consensus-change inclusion, while crediting execution: “Shielded Labs has executed well. Milestones 1 through 4 landed on or ahead of schedule” ([forum.zcashcommunity.com t/54670](https://forum.zcashcommunity.com/t/54670), accessed 2026-07-15). A secondary analyst report claims NU8 “may be activated as early as Q4 2026”; we found no primary statement supporting it and do not repeat it as fact (messari.io, unverified secondary).

1.5 Which document is normative for what

Throughout this volume, “Crosslink 2” means the construction as published in the TFL book at `fe6e1d6`: its model, rules, and safety-theorem statements are the referent of every unqualified claim, and the book is normative for the construction — while its security analysis is only partly updated to the construction it accompanies, as recorded above. “The zebra-crosslink design” means the implementation design at `6d02a1b`: its book is normative for what Shielded Labs is building, including where that contradicts the TFL book (Stalled Mode, the outer signature); where the two conflict we present both and tag the divergence explicitly, since the book has been static since 2024-12-13 while the implementation design has moved. The Quint model is normative for nothing; it is evidence, at its stated scope, that one slice of the design is coherent, and a source of recorded discrepancies. Design goals frame both documents but bind neither: the TFL book targets expected time-to-finality below 30 minutes, cost-of-attack for a 1-hour rollback not reduced versus today, halting cost above 24 hours of PoW mining revenue, and unchanged wallet (lightwalletd, full-node API) and GetBlockTemplate-miner interfaces (tfl-book @ `fe6e1d6`, `src/design/goals.md` lines 22, 28–36, 73). The 30-minute goal is uncheckable today because no production σ or L exists in any source.

1.6 Relation to the sibling volumes

The interface-stability goal above is why the deployed-stack volumes remain accurate under Crosslink and why this volume cites rather than re-derives them. Anchor semantics and reorg behaviour of the commitment tree are those of the *Ironwood Guide* (§“Proof of ownership of *some* valid note: the commitment tree”, in particular “Anchor choice as the tree grows”); what a wallet counts as confirmed is the *Wallet Guide*’s ZIP-315 ConfirmationsPolicy and expiry treatment (§“Transaction construction” and §“Broadcast, expiry, and the transaction lifecycle”), whose trusted/untrusted depths are

exactly the policy layer a finality layer would let wallets strengthen. The header-chain commitment a finality certificate would sit beside, and its unbuilt light-client bridge, are the *FlyClient Guide*'s (§“Relation to the frontier”), which owns the composition pointer in the converse direction. The *Tachyon Guide* already records the institutional relationship between the two frontier efforts — formally independent, with no interaction analysis in either direction (§“Relation to Crosslink and the upgrade environment”); this volume owns the converse cross-reference.

2 The ebb-and-flow model

Crosslink attaches a BFT finality layer to Zcash's proof-of-work chain, and every security claim it makes is phrased against a model published before any Zcash design work began: the *ebb-and-flow* formulation of Neu, Nusret Tas, and Tse, “Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma” (arXiv:2009.04987; v1 2020-09-10, v2 2020-12-28, v3 2021-02-04; the abstract page notes “Forthcoming in IEEE Symposium on Security and Privacy 2021”; hereafter [NTT2020]). This section documents that model: the impossibility it responds to, its execution environments, its two-ledger security definition, the snap-and-chat construction and what the paper actually proves about it, and — because the Crosslink books adopt the model only after amending it — the specific points where the Trailing Finality Layer book departs. Throughout, the best-chain protocol Π_{bc} remains a black box, per this volume's scope rule: we record what the model assumes of it (chain structure, confirmation by depth, header validity), never its internals.

Sources. The primary text is arXiv:2009.04987v3 (accessed 2026-07-15). The Trailing Finality Layer book (tfl-book @ fe6e1d6f) cites the IACR ePrint mirror 2020/1091 by page number; every passage the book quotes was verified identical in arXiv v3, and we cite section, definition, and theorem numbers of v3 rather than pages. The Shielded Labs/Eiger book (zebra-crosslink @ 6d02a1b8, 2026-04-21) carries the Crosslink 2 construction and security analysis as c12-construction.md and analysis/c12-security-analysis.md under book/src/design/; a diff against tfl-book @ fe6e1d6f shows only 20 added admonition lines (14 in the construction copy, 6 in the security-analysis copy) and no other change, none touching the [NTT2020] references, the ebb-and-flow discussion, Prefix Agreement, or Assured Finality — the model presentation is textually unchanged between the two books (diff run 2026-07-15), so we cite the tfl-book chapter files throughout. The community crosslink-protocol-guide (@ b77d8456) contains no occurrence of “ebb” at all and is not a source for this section (grep, 2026-07-15). The ZIPs repository (@ 69610984) contains no Crosslink or NU8 trailing-finality draft; its only mentions are ZIP 218 citing “mechanisms such as Crosslink” in the context of faster block times, and draft-ecc-onchain-accountable-voting citing the book's Assured Finality definition (§2.8).

Remark 2.1 (Binning). Claims in this section about what [NTT2020] states and proves are verifiable against the published text and carry no bin tag; within its model the paper's theorems are settled scholarship, and we cite them by number. Claims about Crosslink itself remain design-stage and carry the series' three bins (*specified*, *designed but unspecified*, *open problem*); no normative Crosslink artifact exists (no draft ZIP in zips @ 69610984), so nothing in this section reaches the specified bin except pinned source code. Where the book disputes the adequacy of the paper's model, the sources disagree about the model, not about what the paper proves within it; we record such disputes as findings with both citations.

Remark 2.2 (Terminology). The name *ebb-and-flow* denotes the security model and goal (Definition 2.8); *snap-and-chat* denotes the construction proposed in the same paper to achieve that goal (§2.4). The distinction is the book's own admonition (tfl-book @ fe6e1d6f, notes-on-snap-and-chat.md, “Terminology”), and it matters because Crosslink adopts the model while replacing the construction. The book further replaces the paper's “longest chain protocol” by *best-chain protocol* (Π_{bc}): Bitcoin and Zcash follow the consensus-valid chain with most accumulated work, not the longest, and [NTT2020]'s own footnote 2 does not require a longest-chain protocol anyway (§2.4). Finally,

[NTT2020] uses “depth” for what Bitcoin and Zcash call “height”; and `zcashd` counts the tip at depth 1 where the book counts it at depth 0 (`tf1-book`, `construction.md` line 66). We write Π_{lc} when reporting the paper and Π_{bc} when reporting the books. Notation for chains: for a chain c and $\sigma \in \mathbb{N}$, the truncation $c^{\lfloor \sigma}$ removes the last σ blocks of c ; the relation $x \leq y$ says x is a prefix of y ; and $x \sim y$ (“ x and y agree”) says $x \leq y$ or $y \leq x$. Subscripts ($\leq_{bc}$, $\sim_{bft}$) name the chain kind.

2.1 The availability–finality dilemma

The paper opens from an impossibility. Its abstract states the position in three sentences (arXiv:2009.04987v3, Abstract): “The CAP theorem says that no blockchain can be live under dynamic participation and safe under temporary network partitions. To resolve this availability–finality dilemma, we formulate a new class of flexible consensus protocols, ebb-and-flow protocols, which support a full dynamically available ledger in conjunction with a finalized prefix ledger. The finalized ledger falls behind the full ledger when the network partitions but catches up when the network heals.”

The impossibility itself is attributed, not proved, in [NTT2020] (§ I-A): “it is impossible for any protocol to be both safe under network partition and live under dynamic participation: individual nodes in the network cannot distinguish between the two scenarios to act differently. This intuition is formalized in [3] and its connection to the CAP theorem [17] was made precise recently in [18].” Reference [3] is Pass and Shi, “The sleepy model of consensus” (ASIACRYPT 2017); reference [17] is Gilbert and Lynch’s CAP theorem exposition (SIGACT News 33(2), 2002); reference [18] is Lewis-Pye and Roughgarden, “Resource Pools and the CAP Theorem” (arXiv:2006.10698; v1 2020-06-18; no journal note on the abstract page, accessed 2026-07-15), which the book cites as [LR2020] (`tf1-book @ fe6e1d6f`, `the-arguments-for-bounded-availability-and-finality-overrides.md`).

The resolution is a two-ledger abstraction. A *conservative* client trusts only the finalized ledger, which is a prefix of the longer, dynamically available ledger believed by an *aggressive* client; “This ebb-and-flow property avoids a system-wide determination of availability versus finality and instead leaves this decision to the clients” ([NTT2020] § I-B). Zcash wallets already practice a client-side version of this choice at the confirmation-depth layer: the ZIP-315 `ConfirmationsPolicy` parameterizes, per wallet, how deep an output must be before it is treated as spendable (*Wallet Guide*, § “Confirmations, trust, and spendability”). The ebb-and-flow model lifts the same freedom to the level of which ledger a client reads.

2.2 Execution model and environments

The model of [NTT2020] § III-A has five cornerstones: n total nodes; time proceeding in slots with synchronized clocks (footnote 4: bounded clock offsets can be captured as part of the network delay); a public-key infrastructure with unique cryptographic identities; a random oracle as the source of randomness; and a probabilistic polynomial-time adversary. Corruption is static and Byzantine: “Before the protocol execution starts, the adversary gets to corrupt (up to) f nodes, then called adversarial. Adversarial nodes surrender their internal state to the adversary and can deviate from the protocol arbitrarily (Byzantine faults) under the adversary’s control. The remaining $(n - f)$ nodes are honest and follow the protocol as specified.”

Dynamic participation is modelled by sleeping: “The adversary chooses, for every time slot and honest node, whether the node is awake or asleep in that slot ... An honest node that is asleep in a slot does not execute the protocol in that slot, and messages that would have arrived in that slot are queued and delivered in the first slot in which the node is awake again. Adversarial nodes are always awake.” The paper is explicit that the permissioned setting and dynamic participation are “two orthogonal aspects of the environment” ([NTT2020] § III-A).

Partial synchrony is modelled with a single asynchrony-to-synchrony transition: “Although in reality, multiple such periods of (a-)synchrony could alternate, we follow the long-standing practice

in the BFT literature and study only a single such transition” ([NTT2020] § III-A). Two adversary-environment pairs then formalize the two regimes.

Definition 2.3 (P1 environment $(\mathcal{A}_1(\beta), \mathcal{Z}_1)$; [NTT2020] § III-A). The pair $(\mathcal{A}_1(\beta), \mathcal{Z}_1)$ formalizes a partially synchronous network under dynamic participation, with respect to the fraction β of adversarial nodes: the adversary $\mathcal{A}_1$ corrupts $f = \beta n$ nodes. Before a global stabilization time GST, the adversary can delay network messages arbitrarily; after GST, it must deliver all messages sent between honest nodes within Δ slots. Before a global awake time GAT, the adversary determines which honest nodes are awake or asleep and when; after GAT, all honest nodes are awake. Both GST and GAT are chosen by the adversary, are unknown to the honest nodes, and can be causal functions of the randomness in the protocol.

Definition 2.4 (P2 environment $(\mathcal{A}_2(\beta), \mathcal{Z}_2)$; [NTT2020] § III-A). The pair $(\mathcal{A}_2(\beta), \mathcal{Z}_2)$ formalizes a synchronous network under dynamic participation, with respect to a bound β on the fraction of awake nodes that are adversarial: at all times the adversary must deliver all messages sent between honest nodes within Δ slots; at all times it determines which honest nodes are awake or asleep and when, subject to the constraint that at most a fraction β of awake nodes are adversarial and at least one honest node is awake.

The global awake time exists because without it the model would collapse into the impossibility (footnote 5): “Without slightly restricting dynamic participation via a GAT after which all nodes are awake, this adversary would fall under the CAP theorem so that no secure protocol against it can exist. In many applications it is realistic that every now and then there is a period in which all nodes are awake.” The two classical regimes are recovered at the extremes ([NTT2020] § I-D): “If $\text{GAT} = 0$, then the environment is the classical partially synchronous network, and the ledger LOG_{fin} has the optimal resilience achievable in that environment. On the other hand, if $\text{GST} = 0$ and $\text{GAT} = \infty$, then the environment is a synchronous network with dynamic participation, and the ledger LOG_{da} has the optimal resilience achievable in that environment.”

Remark 2.5 (Finding: the single-transition idealization). The book argues at length that the DLS1988 GST formalization was intended for one-shot consensus and “cannot correctly be applied to liveness properties of non-terminating protocols”, and that [NTT2020]’s acknowledgement quoted above (“long-standing practice”) “is not adequate: ... it is not valid in general to infer that properties holding for the first transition to synchrony also apply to subsequent transitions” (tfl-book @ fe6e1d6f, construction.md lines 13–41; textually unchanged in zebra-crosslink @ 6d02a1b8, cl2-construction.md). This is the stated reason Crosslink 2 does not take synchrony or partial synchrony as an implicit assumption of its communication model. The dispute concerns the model’s adequacy for non-terminating liveness, not the correctness of the paper’s proofs within the model. Classification: the critique and the resulting communication-model choice are *designed but unspecified* (book prose, no normative artifact).

2.3 Security definitions

Definition 2.6 (Ledger security; [NTT2020] Definition 1). Let T_{confirm} be a polynomial function of the security parameter σ . A state machine replication protocol Π outputting a ledger LOG is *secure after time T* , with transaction confirmation time T_{confirm} , if LOG satisfies:

- *Safety*: for any two times $t \geq t' \geq T$ and any two honest nodes i and j awake at times t and t' respectively, either $\text{LOG}_i^t \leq \text{LOG}_j^{t'}$ or $\text{LOG}_j^{t'} \leq \text{LOG}_i^t$.

- *Liveness*: if a transaction is received by an awake honest node at some time $t \geq T$, then for any time $t' \geq t + T_{\text{confirm}}$ and honest node j awake at time t' , the transaction is included in $\text{LOG}_j^{t'}$.

A protocol safe (live) after $T = 0$ is called simply safe (live). Here LOG_i^t denotes the ledger in the view of node i at time t ; for longest-chain protocols, σ is the confirmation delay for transactions.

Definition 2.7 (Flexible protocol; [NTT2020] Definition 2). A *flexible protocol* is a pair of state machine replication protocols (Π_1, Π_2) , where Π_1 and Π_2 have the same input transactions txs and output ledgers LOG_1 and LOG_2 , respectively.

Definition 2.8 $((\beta_1, \beta_2)$ -secure ebb-and-flow protocol; [NTT2020] Definition 3). A (β_1, β_2) -secure ebb-and-flow protocol is a flexible protocol $(\Pi_{\text{da}}, \Pi_{\text{fin}})$ outputting an available ledger LOG_{da} and a finalized ledger LOG_{fin} , such that, for security parameter $\sigma := T_{\text{confirm}}$:

1. *P1 (Finality)*: under $(\mathcal{A}_1(\beta_1), \mathcal{Z}_1)$, the ledger LOG_{fin} is safe at all times, and there exists a constant C such that LOG_{fin} is live after time $C(\max\{\text{GST}, \text{GAT}\} + \sigma)$, except with probability $\text{negl}(\sigma)$.
2. *P2 (Dynamic Availability)*: under $(\mathcal{A}_2(\beta_2), \mathcal{Z}_2)$, the ledger LOG_{da} is secure, except with probability $\text{negl}(\sigma)$.
3. *Prefix*: for any honest node i and time t , $\text{LOG}_{\text{fin},i}^t$ is a prefix of $\text{LOG}_{\text{da},i}^t$.

Here $\text{negl}(\cdot)$ decays faster than all polynomials: for all $c > 0$ there exists σ_0 such that $\text{negl}(\sigma) < \sigma^{-c}$ for all $\sigma > \sigma_0$.

Optimality. The resilience constants are sharp in the paper’s model (§ III-C): “Observe that no ebb-and-flow protocol can tolerate a Byzantine adversary $\mathcal{A}_1(\beta_1)$ with $\beta_1 \geq 1/3$ in a partially synchronous network. Similarly, no ebb-and-flow protocol can tolerate a Byzantine adversary $\mathcal{A}_2(\beta_2)$ with $\beta_2 \geq 1/2$ in a synchronous network. Hence the security of Π_{sac} implies that it is optimally resilient. We denote the worst-case adversary-environments as $(\mathcal{A}_1^*, \mathcal{Z}_1) := (\mathcal{A}_1(1/3), \mathcal{Z}_1)$ and $(\mathcal{A}_2^*, \mathcal{Z}_2) := (\mathcal{A}_2(1/2), \mathcal{Z}_2)$.”

Relation to flexible BFT. Ebb-and-flow goes beyond the flexible BFT of Malkhi, Nayak, and Ren ([NTT2020] reference [20]) in two ways ([NTT2020] § I-E): dynamic participation is brought in as a client belief, and prefix consistency between the two ledgers is required “in all circumstances” rather than only when both clients’ assumptions hold. Where flexible BFT supports a tradeoff for f between $n/3$ and $n/2$, “The snap-and-chat protocol achieves $(n/2, n/3)$, simultaneously optimal. No tradeoff is necessary” (Fig. 3 caption). The remaining gap to Blum, Katz, and Loss ([NTT2020] reference [27]), a non-flexible protocol with an optimal synchrony/asynchrony tradeoff, “can be interpreted as the value of flexibility”.

2.4 The snap-and-chat construction

Construction 2.9 (Snap-and-chat; [NTT2020] §§ I-D, III-B). Nodes run two off-the-shelf protocols in parallel: a dynamically available protocol Π_{lc} (footnote 2: “Longest chain protocols are representative members of this class of protocols, hence the notation Π_{lc} , but this class includes many other protocols as well”) and a partially synchronous BFT protocol Π_{bft} . Each node i takes

snapshots of its confirmed Π_{lc} ledger ch_i^t and inputs them into Π_{bft} ; the output Ch_i^t of Π_{bft} is an ordered list — a chain of chains — of snapshots, where a BFT block carries as payload only a reference $B.ch$ to an LC block identifying the snapshot. The ledgers are extracted as

$$\text{LOG}_{fin,i}^t := \text{sanitize}(\text{flatten}(Ch_i^t)), \quad \text{LOG}_{da,i}^t := \text{sanitize}(\text{flatten}(Ch_i^t) \parallel ch_i^t),$$

where flatten concatenates the referenced chains of LC blocks as ordered, $\parallel$ is concatenation, and sanitize removes all but the first occurrence of each block ([NTT2020] § III-B3, Fig. 5 caption). At transaction level (footnote 6): “Formally, LOG_{da} and LOG_{fin} are now represented as sequences of LC blocks. Proper transactions ledgers are readily obtained by concatenating the transactions contained in the blocks and removing duplicate and invalid transactions (sanitization).”

The coupling from Π_{lc} into Π_{bft} is a boycott: “in the Π_{bft} sub-protocol, each honest node boycotts the finalization of snapshots that are not confirmed in Π_{lc} in its view” (§ I-D). Concretely, Streamlet’s voting rule is extended — “An honest node only votes for a proposed BFT block B if it views $B.ch$ as confirmed” — and this is the only change (line 19 of Algorithm 1; “Sleepy is applied unaltered and the modification required for Streamlet is minor”). “In addition, ch_i^t is used as side information in Π_{bft} to boycott the finalization of invalid snapshots proposed by the adversary” (§ III-B).

Instantiation. The analyzed instance takes Π_{lc} to be Sleepy consensus (Pass-Shi; permissioned longest chain, where “blocks of a certain depth on the longest chain are confirmed”) and Π_{bft} to be Streamlet (epochs with pseudo-random leaders; a block is notarized when at least two-thirds of nodes vote for it; “Out of three adjacent notarized blocks from consecutive epochs, the middle one gets finalized along with its prefix”) ([NTT2020] § III-B). Section III-D extends the analysis to HotStuff and PBFT as Π_{bft} with minor modifications, and Theorem 2.10 carries over.

Theorem 2.10 (Snap-and-chat security; [NTT2020] Theorem 1). *The protocol Π_{sac} is a $(1/3, 1/2)$ -secure ebb-and-flow protocol. (Proved under the static-corruption, synchronized-slot, PKI plus random-oracle model of §2.2: P1 under $(\mathcal{A}_1^*(1/3), \mathcal{Z}_1)$, P2 under $(\mathcal{A}_2^*(1/2), \mathcal{Z}_2)$, Prefix by construction; Appendix D proves the same with HotStuff in place of Streamlet.)*

Proof structure. We record what is proved and how ([NTT2020] § III-C, Appendix B, dependency diagrams Figures 7 and 8, boxes 1–8), because the book’s departures target specific joints of this argument. P1 safety is Lemma 1 — safety of Π_{bft} by quorum intersection, unaffected by the voting-rule change — plus the observation that ledger extraction preserves safety. P1 liveness is Lemma 4, which needs Theorem 2.11 and Lemmas 2–3 (Streamlet liveness, each carrying the highlighted extra condition that “the leaders’ proposals reference LC blocks that are viewed as confirmed by all honest nodes”). P2 is the security of Π_{lc} under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, taken from Pass-Shi (their Theorem 3 and Lemma 1), plus Lemma 2.12. Prefix holds by construction.

Theorem 2.11 (Longest-chain security after $\max\{\text{GST}, \text{GAT}\}$; [NTT2020] Theorem 2). *For all $p < (n - 2f)/(2\Delta n(n - f))$, there exists a constant $C > 0$ such that for any GST and GAT specified by $(\mathcal{A}_1^*, \mathcal{Z}_1)$, the protocol $\Pi_{lc}(p)$ is secure after $C(\max\{\text{GST}, \text{GAT}\} + \sigma)$, with transaction confirmation time $T_{\text{confirm}} = \sigma$, except with probability $e^{-\Omega(\sqrt{\sigma})}$. (Here p is the probability that a given node produces a block in a given slot; the constant C depends on p , n , f , and Δ (footnote 9); footnote 10 improves the error to $A_\epsilon e^{-a_\epsilon \sigma^{1-\epsilon}}$ for any $\epsilon > 0$ via the DKT+2020 recursive bootstrapping argument.)*

The proof of Theorem 2.11 extends Pass-Shi pivots to *GST-strong pivots* ([NTT2020] Definition 4 and eq. (8)): a slot $t \geq \max\{\text{GST}, \text{GAT}\}$ such that for any $t_a \leq t \leq t_b$, the convergence opportunities (slots in which exactly one honest node produces a block and no other honest block appears within Δ on either side, forcing honest views to converge unless the adversary counters with a block of its

own) counted only after $\max\{t_a, \text{GST}, \text{GAT}\}$ outnumber the adversarial slots in $[t_a, t_b]$ — only post-transition convergence opportunities count, because their useful properties fail under asynchrony or while honest nodes may sleep.

Lemma 2.12 (Consistency of LOG_{fin} with Π_{lc} under P2; [NTT2020] Lemma 5). *The ledger LOG_{fin} is a safe prefix of the output of Π_{lc} in the view of every honest node at all times under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, except with probability $e^{-\Omega(\sqrt{\sigma})}$. The key proof step: for a BFT block to become final in the view of an honest node under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, at least one vote from an honest node is required, and honest nodes only vote for a BFT block if they view the referenced LC block as confirmed; this holds even if the final BFT blocks conflict in the output of Π_{bft} .*

Why the composition is safe. Figure 9 of [NTT2020] presents the trichotomy. (a) When both sub-protocols are safe, ch and Ch do not fork. (b) When Π_{lc} is unsafe, “snapshots taken by different nodes or at different times can conflict. However, Π_{bft} is still safe and thus orders these snapshots linearly. Any transactions invalidated by conflicts are sanitized during ledger extraction. As a result, LOG_{fin} remains safe.” (c) When Π_{bft} is unsafe, in a synchronous network with $n/3 < f < n/2$, “finalization of a snapshot requires at least one honest vote, and thus only valid snapshots become finalized. Since finalized snapshots are consistent, LOG_{fin} is consistent with LOG_{lc} .”

Simulation evidence. The paper simulated Sleepy-plus-Streamlet with $n = 100$ nodes, $f = 25$ adversarial, $\Delta = 1$ s, Sleepy rate $\lambda = 0.1 \text{ s}^{-1}$ (per-node $\lambda_0 = 10^{-3} \text{ s}^{-1}$), confirmation depth $k = 20$, and Streamlet $\Delta_{\text{bft}} = 5$ s ([NTT2020] § IV). Under dynamic participation, LOG_{da} grows steadily while LOG_{fin} stalls whenever fewer than a two-thirds quorum (67 nodes) is awake, catching up afterwards; under intermittent partitions (honest nodes split $2(n-f)/3$ and $(n-f)/3$), BFT finalization stalls, the parts’ LOG_{da} drift apart, and on reunion nodes converge on the longer LOG_{da} while LOG_{fin} catches up. Simulation code: <https://github.com/tse-group/ebb-and-flow> (footnote 7). The paper offers no implementation against a deployed chain; this matters in §2.6.

2.5 The proof-of-work setting

Section V-B of [NTT2020] adapts the model to the shape Crosslink actually instantiates: nodes come in two flavors, *miners* (quantified by hash rate, powering the PoW longest chain) and *validators* (unique cryptographic identities, providing finality). The informal theorem for this setting reads: in a network environment where (1) communication is asynchronous until a global stabilization time GST after which it becomes synchronous, (2) validators are always awake, and (3) miners can sleep and wake at any time — (P1, Finality) the ledger LOG_{fin} is safe at all times and live after GST, if fewer than 33% of validators are adversarial, fewer than 50% of awake hash rate is adversarial, and awake hash rate is bounded away from zero; (P2, Dynamic Availability) if $\text{GST} = 0$, the ledger LOG_{da} is safe and live at all times, if fewer than 66% of validators are adversarial, fewer than 50% of awake hash rate is adversarial, and awake hash rate is bounded away from zero ([NTT2020] § V-B).

The paper explains why the requirements exceed the permissioned case: under P1, “fewer than 50% of awake hash rate have to be adversarial, and awake hash rate has to be bounded away from zero, since otherwise Π_{lc} might not be live and there would be no input for Π_{bft} to finalize”; under P2, “fewer than 66% of validators have to be adversarial, since otherwise adversarial validators could finalize Π_{lc} blocks that are not on the longest chain, which would later enter the prefix of LOG_{da} ” ([NTT2020] § V-B). The paper closes the section with an open question, which we record verbatim: “It remains an open question whether these additional requirements are fundamental or a limitation of the snap-and-chat construction.” Classification: *open problem*, the paper’s own.

The deployed interface today. On the black-box side of the boundary, the only datum this volume records about the deployed Π_{bc} is its rollback bound: Zebra bounds its best non-

finalized chain by `MAX_BLOCK_REORG_HEIGHT` (1000 blocks), whose documentation states “This is a local-only node policy; it is not part of consensus. The window is sized as a defence-in-depth measure against sustained consensus splits”; Zebra finalizes its oldest blocks once the chain grows past this height (zebra @ 9f3166b9, zebra-chain/src/parameters/constants.rs:20--30, zebra-state/src/constants.rs:16--19). Nothing in today’s Zcash consensus provides finality; the constant is a node policy, which is precisely the gap Crosslink addresses.

2.6 Findings: where the book amends the paper

The book accepts the ebb-and-flow model as the starting point and then records defects and gaps that shape Crosslink. We reproduce each finding with both citations; all book material below is *designed but unspecified* (prose in `tf1-book` @ fe6e1d6f, textually unchanged in `zebra-crosslink` @ 6d02a1b8; no normative artifact).

Scope of the one-honest-vote argument. The trichotomy case (c) above “is technically correct but has to be interpreted with care. It only applies when the number of malicious nodes f is such that $n/3 < f < n/2$. What we are trying to do with Crosslink is to ensure that a similar conclusion holds even if Π_{bft} is completely subverted, i.e. the adversary has 100% of validators (but only $< 50\%$ of Π_{lc} hash rate)” (`notes-on-snap-and-chat.md`, “Comment on security assumptions”). Crosslink’s safety target is therefore strictly stronger than what Theorem 2.10 case (c) delivers.

A hidden safety assumption in Lemma 5. The proof of Lemma 2.12 assumes that “since Π_{lc} is safe ..., the fact that one honest node sees that snapshot as confirmed implies that every honest node sees the same snapshot as confirmed” ([NTT2020] Lemma 5 proof). The book replies that “while this may be a reasonable assumption to make for parts of the security analysis, in practice we should always require any adversary to do the relevant amount of Proof-of-Work to construct block headers that are plausibly confirmed” (`notes-on-snap-and-chat.md`, admonition quoting ePrint p. 9).

A subtlety in sanitization. Two readings of ledger extraction — concatenate transactions then sanitize, versus concatenate blocks, deduplicate blocks, then sanitize — “are equivalent in the setting considered by [NTT2020], but the argument for their equivalence is not obvious”; the equivalence requires that the *only* reasons for contextual invalidity in Π_{lc} are double-spends and missing inputs, and “strictly speaking Snap-and-Chat should require of Π_{lc} that there is no such other reason. This does not seem to be explicitly stated anywhere in [NTT2020]” (`notes-on-snap-and-chat.md`, “Subtlety in the definition of sanitization”). Zcash violates the premise: transaction expiry can be handled (the height-bounded validity of ZIP-203; *Wallet Guide*, §“Expiry (ZIP-203)”), but missing anchors break the argument, because a transaction invalid for a missing anchor can become valid later (anchors are the tree roots developed in the *Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”).

Spending finalized outputs. In `snap-and-chat`, transactions in ch_i^t must be able to spend outputs of finalized transactions that are not in ch_i^t ’s own history — they may come from another fork’s snapshot. Since consensus validity of the tip must be a deterministic function of the chain itself, Π_{lc} “must include the information needed to calculate $\text{LOG}_{\text{fin},i}^s$ ”. The book’s assessment: “The authors of [NTT2020] probably just missed this: the paper only has evidence that they simulated their construction, rather than implementing it for Bitcoin or any other concrete block chain as Π_{lc} ” (`notes-on-snap-and-chat.md`, “Spending finalized outputs” and “Finalization Availability”; compare the simulation-only evidence in §2.4). The repair adds a property the paper does not have — *finalization availability* — via a commitment $\text{LF}(H)$ in each block header to the last final BFT block known to the block producer, governed by the following rule.

Definition 2.13 (Extension rule; tfl-book @ fe6e1d6f, notes-on-snap-and-chat.md). If H is not the genesis block header, then the final BFT block committed to by H either descends from or equals the final BFT block committed to by H ’s parent:

$$\text{LF}(H^{[1]}) \leq_{\text{bft}} \text{LF}(H).$$

The book states that the Extension rule “will be preserved into Crosslink 2”, and that it yields, by construction and regardless of Π_{bft} : if node i observes no rollback deeper than σ in Π_{lc} , then i observes no instability in $\text{LOG}_{\text{fin},i}$, even if Π_{bft} ’s assumptions are violated (notes-on-snap-and-chat.md, “Finalization Availability”).

2.7 From dynamic availability to bounded availability

The book’s one model-level departure from ebb-and-flow concerns what happens during a long partition. Since partition cannot be prevented, the CAP theorem — “that loosely speaking is made precise by [LR2020]” — implies that any finality-providing protocol may stall for as long as the partition lasts; strict dynamic availability then makes the finalization gap grow without bound (tfl-book @ fe6e1d6f, the-arguments-for-bounded-availability-and-finality-overrides.md). The book argues that spending under an unbounded gap is unacceptable, and replaces strict dynamic availability with *bounded availability* — a weakening of dynamic availability in the sense of [DKT2020, arXiv:2010.08154] — plus a *Stalled Mode* (coinbase-only blocks) and explicitly governed *finality overrides*. Block production keeps running during a stall rather than halting, because of tail-thrashing attacks: during a stall, an adversary with 10% of hash power can build a 10-block fork in 100 block times, and the k -block tail can “thrash” between chains (same chapter).

Definition 2.14 (Finality depth and the Stalled Mode rule; tfl-book @ fe6e1d6f, notes-on-snap-and-chat.md). For a block header H , let

$$\begin{aligned} \text{tailhead}(H) &:= \text{lastcommonancestor}(\text{snapshot}(\text{LF}(H)), H), \\ \text{finalitydepth}(H) &:= \text{height}(H) - \text{height}(\text{tailhead}(H)). \end{aligned}$$

Consensus rule (Stalled Mode variant): for every Π_{lc} block H , either $\text{finalitydepth}(H) \leq L$, or H follows the Stalled Mode restrictions (for Zcash: coinbase-only blocks), where L is the finality gap bound.

Definition 2.15 (Bounded hazard-freeness; tfl-book @ fe6e1d6f). For a finality gap bound of L blocks: there is never observed to be, for any node i at time t , a more-available ledger $\text{LOG}_{\text{da},i}^t$ containing a hazardous transaction that originates in a block H of ch_i^t with $\text{finalitydepth}(H) > L$ (notes-on-snap-and-chat.md, Definition “Bounded hazard-freeness”).

Classification: the bounded-availability argument, Stalled Mode, and finality overrides are *designed but unspecified* — book prose with no draft ZIP (zips @ 69610984). The governance shape of finality overrides in particular has no normative text.

2.8 Crosslink 2’s counterpart properties

The book restates the model’s goals in a different proof style before replacing the construction: security arguments take the form “in an execution where safety properties are not violated, undesirable thing cannot happen”, with probabilistic reasoning separated out (tfl-book @ fe6e1d6f, construction.md lines 342–359). On the relation to the paper: [NTT2020] Theorem 2 “essentially

says that under certain conditions Prefix Agreement holds except with probability $e^{-\Omega(\sqrt{\sigma})}$; the bound is not tight (footnote 10’s bootstrapping via DKT+2020 § 4.2 brings it to $A_\varepsilon e^{-a_\varepsilon \sigma^{1-\varepsilon}}$); and in the proofs of Lemma 5 and Theorem 1 this probability “just passes through the proof from the premisses to the conclusions, because the argument is not probabilistic”. The counterpart properties, in the book’s execution-based style (the star is the book’s protocol-name wildcard: $\Pi_{\star bc}$ ranges over $\{\Pi_{\text{orig}bc}, \Pi_{bc}\}$, likewise for bft, so each property is stated once for the original and the adapted subprotocol alike):

Definition 2.16 (Prefix Consistency; tfl-book @ fe6e1d6f, construction.md). An execution of $\Pi_{\star bc}$ has *Prefix Consistency* at confirmation depth σ iff for all times $t \leq u$ and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u ,

$$(\text{ch}_i^t)^{[\sigma]} \leq_{bc} \text{ch}_j^u.$$

(Taken from PSS2016’s “consistency”).

Definition 2.17 (Prefix Agreement; tfl-book @ fe6e1d6f, security-analysis.md). An execution of $\Pi_{\star bc}$ has *Prefix Agreement* at confirmation depth σ iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u ,

$$(\text{ch}_i^t)^{[\sigma]} \sim_{bc} (\text{ch}_j^u)^{[\sigma]}.$$

Definition 2.18 (Final Agreement; tfl-book @ fe6e1d6f, security-analysis.md). An execution of $\Pi_{\star bft}$ has *Final Agreement* iff for all $\star bft$ -valid blocks C in honest view at time t and C' in honest view at time t' , $\text{bftlastfinal}(C) \sim_{\star bft} \text{bftlastfinal}(C')$.

Definition 2.19 (Assured Finality; tfl-book @ fe6e1d6f, construction.md line 505). An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u , $\text{fin}_i^t \sim_{bc} \text{fin}_j^u$.

Theorem 2.20 (Safety from either subprotocol; tfl-book @ fe6e1d6f, security-analysis.md). *Prefix Agreement of Π_{bc} at confirmation depth σ implies Assured Finality; and, independently, Final Agreement of Π_{bft} implies Assured Finality. Safety of Crosslink 2’s main goal therefore holds under reasonable assumptions about either subprotocol.*

The book notes that its LOG_{fin} safety proof from Prefix Agreement “does not depend on any safety property of Π_{bft} , and is an elementary proof. ([NTT2020, Theorem 2] is a much more complicated proof that takes nearly 3 pages, not counting the reliance on results from [PS2017].)” This is the book’s answer to the one-honest-vote scope finding of §2.6: the target regime includes a completely subverted Π_{bft} .

Theorem 2.21 (Ledger prefix property; tfl-book @ fe6e1d6f, construction.md lines 522–536). *For any node i honest at time t , and any confirmation depth μ , $\text{fin}_i^t \leq (\text{ba}_\mu)_i^t$, where $(\text{ba}_\mu)_i^t = (\text{ch}_i^t)^{[\mu]}$*

if $\text{fin}_i^t \leq (\text{ch}_i^t)^\mu$, and fin_i^t otherwise. This is the analogue of the Prefix property of Definition 2.8, and it holds by construction of $(\text{ba}_\mu)_i^t$.

The definitions above are shapes for comparison with [NTT2020]; the constructions behind them are canonical in this volume’s construction section — fin and ba_μ in §3.6, the block structure in §3.3, and bft -last-final with the validity rules in §3.4 and §3.5.

Prefix Consistency and partitions. When Prefix Consistency is assumed of PoW protocols, it implicitly rules out partitions of the honest network longer than about σ block times; GKL2020 “proves” it only because the required no-partition assumption is implicit in the (partially) synchronous communication model. BFT protocols, by contrast, can be safe under full asynchrony — which is why Crosslink 2’s communication model makes no implicit synchrony assumption, “or else we could end up with a protocol with weaker safety properties than Π_{origbft} alone” (tfl-book @ fe6e1d6f, construction.md lines 317–333; compare Remark 2.5).

What Crosslink 2 does not inherit. The model section must not imply that Crosslink 2 inherits [NTT2020]’s P2 as proved. The book itself notes that its “ LOG_{ba} does not roll back past the agreed LOG_{fin} ” theorem “is not as strong as we would like”, and of the bound L on non-Stalled-Mode blocks after the finalization point, “we have also not proven that so far” (security-analysis.md). Classification: Crosslink 2’s dynamic availability analogue is an *open problem* in the book’s own accounting; its safety counterparts (Theorem 2.20) are *designed but unspecified*, proved in book prose only for the Prefix Agreement half (the Final Agreement proof is defective, §1.3) and fixed by no normative artifact. The Assured Finality definition already circulates beyond the book: the ZIPs repository’s draft-ecc-onchain-accountable-voting cites it (zips @ 69610984, line 169; the draft’s placement in the voting landscape is documented in the *Voting Guide*, §“Lineage”).

2.9 Context: Gasper, Casper FFG, and the successor line

The paper’s motivating negative result is a balancing attack on Gasper (Ethereum’s then-candidate consensus) in the standard synchronous model with adversarial — not stochastic — network delay, showing Gasper is not live and that its block proposal mechanism is rendered unsafe; for large n , any positive adversarial fraction f/n suffices, with the first opportune epoch after about n/f epochs and the per-epoch launch probability approximately β ([NTT2020] §§ II, V-A, Appendix A). Appendix E recapitulates the bouncing attack on Casper FFG and concludes that the “bidirectional interaction” between a finality gadget and its blockchain “is intricate to reason about and a gateway for liveness attacks”, in contrast to the “completely decoupled” confirmation and finalization of snap-and-chat.

A finding follows for this volume: the book acknowledges Appendix E’s warning while Crosslink 2 itself “involves a bidirectional dependence between Π_{bft} and Π_{bc} ” — the book takes the bouncing attack as a lesson to be addressed, not a prohibition (tfl-book @ fe6e1d6f, security-analysis.md lines 15–20). The Extension rule of Definition 2.13 and the finality-depth rule of Definition 2.14 are both instances of that dependence.

The successor paper. The same authors identified a second dilemma: no protocol can provide both accountable safety and liveness under dynamic participation. Their resolution, *accountability gadgets*, checkpoints a longest-chain protocol using any BFT protocol as a black box (“The Availability-Accountability Dilemma and its Resolution via Accountability Gadgets”, arXiv:2105.06075; latest v3 2022-05-18; accessed 2026-07-15). We record it here for attribution and for the outlook; whether Crosslink’s design draws on it is a question for the design-lineage section, not this one.

Deployment status. As verified on 2026-07-15, Crosslink is implementation-stage, not consensus-stage: Shielded Labs completed Crosslink Milestone 5 on 2026-04-15 and pivoted to long-running

“seasonal” FeatureNets; an incentivized FeatureNet paying real ZEC rewards had run for roughly six weeks as of the 2026-05-28 Arborist call; hardening is slated for 2026, followed by productionization (ZIP finalization and security audits), with mainnet inclusion pending community approval. Sources, all accessed 2026-07-15:

- <https://zechub.substack.com/p/arborist-call-zcash-protocol-updates-ad8> (2026-05-28);
- <https://zechub.substack.com/p/arborist-call-zcash-protocol-updates-330> (2026-04-30);
- <https://forum.zcashcommunity.com/t/shielded-labs-crosslink-deployment-updates/49706>;
- <https://shieldedlabs.net/crosslink-roadmap-q1-2025/>.

The sibling volume’s view of the upgrade landscape is the *Tachyon Guide*, §“Relation to Crosslink and the upgrade environment”.

2.10 Machine-checked status

One machine-checked artifact exists, and its scope must be stated precisely to avoid overclaiming. The Informal Systems Quint specification (crosslink-spec @ 8edc3e94) is a proof-of-concept model of *only* the `isValid` check for a Crosslink BFT proposal against a single node’s PoW and BFT block stores — the Tail Confirmation rule (σ -block tail) and the Linearity rule — with the PoW and BFT chains modelled abstractly (actions `add_pow` and `try_add_bft`). The README states that the logic was “inferred ... from an insightful Youtube talk” and “might not be up-to-date with the current protocol design ideas” (README.md). It does *not* formalize the ebb-and-flow ledgers $\text{LOG}_{\text{fin}}/\text{LOG}_{\text{da}}$ or the P1/P2/Prefix properties of Definition 2.8: the machine-checked coverage of the ebb-and-flow model itself is nil.

What is checked. As formalized (validity.md, “Validity Function”), the check is `isValid(bft_store, pow_store, p)` requiring: the store contains `p.block`; the proposal context equals `bft_store.last()`; `sigmaConfirmed` — a fork-free tail of at least σ PoW blocks, some element of which has the proposed block as its parent, and whose parents all lie within the tail or are the proposed block itself; and `linear`. The invariant `prefix_inv`, requiring for all consecutive indices i

```
pow_store.isDescendant(bft_store[i].pow_hash, bft_store[i+1].pow_hash)
```

and glossed “The PoW history defined by finalization is a path in the PoW blockstore”, passed random simulation (10000 traces of 20 steps, no violation) and TLC model checking after transpiling to TLA+ on the verification scope (stores of up to 8 blocks, `confirmation_depth = 1`, about three minutes). The property `finalization_witness` generates traces with 3 BFT blocks, and `runFinalizationTest` is an assertion-checked concrete execution. Files: `src/validity.qnt` (tangled from `validity.md` via `lmt`), `src/validity.tla`, `src/validity.cfg`; instances: verification with `confirmation_depth = 1` and store limit 8, simulation with `confirmation_depth = 2` and no limit (README.md §§ 2–3; `validity.md`, “Invariants”/“Tests”, instances block lines 463–482).

Finding: divergences from the book. Function `linear` requires

```
work_store.isDescendant(context.pow_hash, b.hash)
and not (context.pow_hash == b.hash),
```

the second conjunct carrying the specification’s own comment “// this actually seems allowed. TODO: perhaps remove” — that is, it is *stricter* than the book’s Linearity rule $\text{snapshot}(B_{\text{bft}}^{[1]}) \leq_{\text{bc}} \text{snapshot}(B)$, which permits equality. Additionally, `isValid` requires `p.bftContext = bft_store.last()` (proposals only on the current BFT tip), an abstraction not present as such in the book (crosslink-spec @ 8edc3e94, `validity.md` lines 131–176; `tfl-book` @ fe6e1d6f, `construction.md`, Linearity rule).

The repository `quint-crosslink` (`github.com/zmanian/quint-crosslink`) contains no commits locally, and `git ls-remote` against the upstream returns no refs — the repository is empty upstream as well (checked 2026-07-15). The Quint ground truth is `crosslink-spec` in its entirety.

Classification. The Quint artifact itself is *specified* in the only sense available before a protocol specification: verifiable, machine-checked source at a pinned commit. The claim it checks (`prefix_inv`) is far weaker than the model properties of this section; every ebb-and-flow-level property of Crosslink 2 remains *designed but unspecified* or *open*, per §2.8.

3 The Crosslink construction

Crosslink couples Zcash’s proof-of-work best-chain protocol to a Byzantine-fault-tolerant (BFT) protocol so that transactions become final some time after they first appear in PoW blocks — the property the Trailing Finality Layer book names *Trailing Finality*, and whose target is *Assured Finality*: the assurance that transactions cannot be reverted *by that protocol*. The book deliberately eschews “absolute finality”, because out-of-band forks can always revert anything (`tfl-book` @ `fe6e1d6`, `src/terminology.md`:11–13, 45). The two-ledger shape — a conservative finalized ledger beside an available one — descends from the ebb-and-flow paradigm of Neu, Tas, and Tse, “Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma” (arXiv 2009.04987, v3 of 2021-02-04; IEEE S&P 2021; accessed 2026-07-15).

The current version of the construction is *Crosslink 2*: the book’s construction chapter defines it as such, and version-history entry 0.2.0 records the update of both the construction and its security analysis from Crosslink 1 (`construction.md`:1–3; `src/design/crosslink.md`:3; `src/version-history.md`). Throughout this section, a citation of the form `construction.md:n` means `src/design/crosslink/construction.md` in the ECC Trailing Finality Layer book at commit `fe6e1d6`; other files of that repository are cited by path at the same commit.

Crosslink 2 modifies a BFT protocol Π_{origbft} and a best-chain protocol Π_{origbc} into two coupled protocols Π_{bft} and Π_{bc} , by adding structural elements, changing validity rules, and changing honest-node behaviour. A Crosslink 2 node must participate in both Π_{bft} and Π_{bc} ; acting as a bft-proposer, a bft-validator, or a bc-block-producer is optional, but all such actors are assumed to be Crosslink 2 nodes (`construction.md`:103–111).

The stakes for the rest of the Zcash stack are concrete. Today confirmation depth is policy, not property: wallets gate spendability on heuristic depths (*Wallet Guide*, §“Confirmations, trust, and spendability”) and recover from rollbacks by truncate-and-rescan (*Wallet Guide*, §“Reorganisations: truncate and rescan”); an Orchard anchor is only as settled as the chain suffix beneath it (*Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”); and the coin-weighted vote fixes its electorate at a snapshot height that nothing today prevents from reorganising (*Voting Guide*, §“The snapshot roots: posted, never verified”). Crosslink 2 is the construction intended to replace those heuristics with a protocol property.

Classification. Every rule in this section is *designed but unspecified*: the book is the only source at design rigor, and no Crosslink or trailing-finality ZIP exists — as of zips @ 6961098 (2026-07-09) the sole mention in the ZIP corpus is a citation footnote in ZIP 218 (“25-second Block Target Spacing”, Status: Draft, created 2026-03-13). Two artifact classes attach *specified* status, in the pre-spec sense of verifiable pinned sources, to fragments of the material: a machine-checked Quint model of one validity check (§3.8) and prototype code (§3.9) — and both diverge from the book text in ways we record. Per-subsection Classification paragraphs refine this default.

3.1 Model and notation

The communication model takes neither synchrony nor partial synchrony as an implicit assumption: unless otherwise specified, messages can be arbitrarily delayed or dropped; a message is received at most once, and is nonmalleably authenticated. The book argues at length against applying the Global Stabilization Time formalization of partial synchrony (Dwork, Lynch, and Stockmeyer, 1988) to liveness of non-terminating block-chain protocols (construction.md:13–41). Where a subprotocol’s own analysis needs timing assumptions — as Streamlet’s liveness analysis does below — those assumptions are imported explicitly, not ambiently.

Throughout, subscript $\star$ ranges over the two chain flavours bc and bft; a block determines the chain of its ancestors with itself as tip, and we pass between block and chain freely.

Definition 3.1 (Chain notation). Let C be a $\star$ -chain and $k \in \mathbb{N}$.

- (i) *Truncation.* The chain $C|_{\star}^k$ is C with the last k blocks pruned, except that if $\text{len}(C) \leq k$ the result is the genesis chain consisting only of $\text{Origin}_{\star}$. The block at depth k in C is the tip of $C|_{\star}^k$. For a bft-proposal P with parent block B and $k \geq 1$, define $P|_{\text{bft}}^k := B|_{\text{bft}}^{k-1}$ (construction.md:61–63, 121).
- (ii) *Prefix and agreement.* We write $B \leq_{\star} C$ iff the chain with tip B is a prefix of the chain with tip C (including $B = C$). Blocks B and C *agree*, written $B \sim_{\star} C$, iff $B \leq_{\star} C$ or $C \leq_{\star} B$; they *conflict* otherwise (construction.md:69–75).
- (iii) *Linearity.* A time series $S : I \rightarrow \star\text{-block}$ is $\star$ -linear iff $t \leq u$ implies $S(t) \leq_{\star} S(u)$ (construction.md:69–75).

Lemma 3.2 (Linear prefix). If $A \leq_{\star} C$ and $B \leq_{\star} C$, then $A \sim_{\star} B$.

Proof. The chain of ancestors of C is $\star$ -linear, and blocks A and B both lie on that chain (construction.md:77–82). $\square$

Definition 3.3 (Agreement on a view). An execution of a protocol Π has *Agreement on the view* $V : \text{Node} \times \text{Time} \rightarrow \star\text{-chain}$ iff for all times t, u and all Π nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u , we have $V_i^t \sim_{\star} V_j^u$ (construction.md:96–99).

Remark 3.4 (Depth conventions). The book’s “depth” differs from both neighbouring conventions: NTT2020 uses *depth* for what the book calls height, and the book’s count differs by one from the zcashd confirmation-depth convention — the tip sits at depth 0, not 1 (construction.md:65–67; arXiv 2009.04987). The confirmation-depth defaults quoted from the *Wallet Guide* in §3.7 use the wallet convention.

3.2 The two black boxes

Consistent with the scope of this series, both subprotocols enter the construction only through their interfaces. We record what Crosslink 2 assumes of each; we do not derive the internals of either — in particular, PoW block production remains a black box.

Requirements on Π_{origbc} . Block producers are selected by a resource-proportional random process. A function $\text{score} : \text{bc-valid-chain} \rightarrow \text{Score}$ is fixed, with $<$ a strict total order on Score and, for

any non-genesis bc-valid-chain H , $\text{score}(H|_{\text{bc}}^1) < \text{score}(H)$; for PoW instantiations the score is accumulated work. Honest nodes choose a highest-scoring valid chain as their best chain, with any tie-breaking rule. Headers commit to blocks, and header validity is necessary but not sufficient for block validity (construction.md:245–290, 255–257). The model also fixes a predicate $\text{iscoinbaseonlyblock} : \text{bc-block} \rightarrow \text{boolean}$, true iff the block has exactly one transaction and that transaction is a coinbase transaction — one that only distributes newly issued funds and has no inputs (construction.md:269–271).

The two chain-quality properties the analysis consumes are stated as unconditional properties of executions; the probabilistic reasoning that makes them plausible for a PoW protocol is deliberately separated from the construction (construction.md:245–290, 337–366).

Definition 3.5 (Prefix Consistency). An execution of Π_{bc} has *Prefix Consistency* at confirmation depth σ iff for all times $t \leq u$ and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u , we have $\text{ch}_i^t|_{\text{bc}}^\sigma \leq_{\text{bc}} \text{ch}_j^u$ (construction.md:287–290; the property is PSS2016 §3.3 “consistency”).

Definition 3.6 (Prefix Agreement). An execution of Π_{bc} has *Prefix Agreement* at confirmation depth σ iff it has Agreement on the view $(i, t) \mapsto \text{ch}_i^t|_{\text{bc}}^\sigma$ (construction.md:337–340).

Requirements on Π_{origbft} . The BFT protocol proceeds in epochs with proposals, ballots, and a *notarization rule*. The notarization rule must require at least a two-thirds absolute supermajority of the epoch’s voting units to have been cast for a proposal P , and may require other conditions; a notarization proof can be obtained only with knowledge of ballots collectively satisfying the rule (construction.md:129).

A *★bft-block* consists of (P, proof_P) re-signed by the same proposer using a strongly unforgeable signature scheme (*Crypto Guide*, §“Digital signatures”). It is bft-block-valid iff P is bft-proposal-valid, proof_P validly proves that the notarization rule was satisfied by ballots cast for P , and the proposer’s outer signature on (P, proof_P) is valid. A bft-proposal’s parent reference hashes the *entire* parent bft-block: proposal, proof, and outer signature (construction.md:156–161).

A voting unit is cast *non-honestly* iff it is cast other than by its holder (for example after key compromise), it is double-cast for distinct proposals, or its holder should not have cast that vote according to the honest-voting conditions in its view (construction.md:131–154).

Definition 3.7 (One-third bound on non-honest voting). An execution of Π_{bft} has the *one-third bound on non-honest voting* property iff for every epoch, strictly fewer than one third of the total voting units for that epoch are *ever* cast non-honestly (construction.md:138–141).

The bound quantifies over ballots cast at *any* time in the entire execution. Consequently the validator keys of honest nodes must remain secret indefinitely, and rotated-out keys must be securely deleted (construction.md:131–154; tfl-book issue #140).

Finality inside the BFT protocol is a function of context, never an objective predicate: it is correct to talk about the last final block *of a chain*, but not to call any bft-block objectively bft-final (construction.md:171–187).

Definition 3.8 (Required properties of bft-last-final). Function $\text{bft-last-final} : \text{bft-block} \rightarrow \text{bft-block-or-}\perp$ outputs, for a bft-block-valid context C , the last ancestor of C that is final in the context of C . Required: $\text{bft-last-final}(C) = \perp$ iff C is not bft-block-valid; if C is bft-block-valid, then $\text{bft-last-final}(C) \leq_{\text{bft}} C$ (hence is itself bft-block-valid), and for all bft-valid-blocks D

with $C \leq_{\text{bft}} D$, $\text{bft-last-final}(C) \leq_{\text{bft}} \text{bft-last-final}(D)$. Also $\text{bft-last-final}(\text{Origin}_{\text{bft}}) = \text{Origin}_{\text{bft}}$ (construction.md:177–183).

Definition 3.9 (Final Agreement). An execution of Π_{bft} has *Final Agreement* iff for all bft-valid blocks C in honest view at time t and C' in honest view at time t' , we have $\text{bft-last-final}(C) \sim_{\text{bft}} \text{bft-last-final}(C')$. A block is *in honest view* if a party observes it at some time at which that party is honest (construction.md:200–206).

Worked instance: adapting Streamlet. The book’s running Π_{origbft} is an adapted Streamlet. Its notion of “notarized” is identified with bft-block-validity by having the proposer aggregate a two-thirds-supermajority proof into a submitted block, making notarization objectively and feasibly verifiable, so that valid chains consist of notarized blocks; the liveness timing analysis then assumes a notarized block is received within two epochs, and progress needs five consecutive epochs with honest leaders. Streamlet’s finality rule — three adjacent blocks with consecutive epoch numbers finalize the chain up to the second — becomes origbft-last-final: for an origbft-valid-block C , $\text{origbft-last-final}(C)$ is the last origbft-valid-block $B \leq_{\text{origbft}} C$ such that either $B = \text{Origin}_{\text{origbft}}$ or B is the second block of a group of three adjacent blocks with consecutive epoch numbers (construction.md:210–243, 224, 234–241, 618).

Classification. *Designed but unspecified* throughout; the requirements are the book’s model choices, argued but not proven against any deployed BFT protocol. A specification would need to fix the concrete Π_{origbft} , its voting-unit assignment, and the notarization proof format.

3.3 Structural additions

Crosslink 2 adds three structural elements (construction.md:417–421):

- (1) each bc-header has, in addition to the origbc-header fields, a `context_bft` field that commits to a bft-block;
- (2) each bft-proposal has a `headers_bc` field containing a sequence of exactly σ bc-headers, zero-indexed and deepest first;
- (3) each non-genesis bft-block likewise has a `headers_bc` field with exactly σ bc-headers; the genesis bft-block has `headers_bc = null`.

Definition 3.10 (Snapshot). For a bft-block or bft-proposal B :

$$\text{snapshot}(B) := \begin{cases} \text{Origin}_{\text{bc}} & \text{if } B.\text{headers_bc} = \text{null}, \\ B.\text{headers_bc}[0] \upharpoonright_{\text{bc}}^1 & \text{otherwise} \end{cases}$$

(construction.md:423–430). The snapshot is the parent of the deepest supplied header, so the σ headers sit directly on top of it.

Definition 3.11 (LF and the finalization candidate). For a bc-block H :

$$\begin{aligned} \text{LF}(H) &:= \text{bft-last-final}(H.\text{context_bft}), \\ \text{candidate}(H) &:= \text{last-common-ancestor}(\text{snapshot}(\text{LF}(H)), H \upharpoonright_{\text{bc}}^{\sigma}). \end{aligned}$$

When H is the tip of a node's bc-best-chain, $\text{candidate}(H)$ gives the candidate finalization point, subject to the local no-rollback condition of updatefin (Construction 3.17; `construction.md:431–438`).

Why full headers. The `headers_bc` field demonstrates to any party seeing a proposal or block that the snapshot had been confirmed when the proposal was made. Full headers, rather than hashes, let a validator check Proofs-of-Work and difficulty adjustment locally *before* downloading blocks, limiting denial-of-service; nodes may trim headers overlapping with ancestor bft-blocks. The genesis special case — `headers_bc = null` with the snapshot patched to Origin_{bc} — avoids a hash cycle between the two genesis blocks (`construction.md:440–454`).

Why only context_bft. No separate `final_bft` field is needed in bc-headers: the context block alone determines its last final ancestor, and `bft-last-final` is efficiently computable for typical BFT protocols (`construction.md:456–460`).

Classification. *Designed but unspecified.* A specification would need to pin the commitment encoding of `context_bft`, the header wire format, and the trimming rule.

3.4 The BFT side: Π_{bft}

Definition 3.12 (Π_{bft} validity). The genesis rule is that Origin_{bft} is bft-block-valid (`construction.md:568`). A bft-proposal (respectively, non-genesis bft-block) B is bft-proposal-valid (respectively, bft-block-valid) iff *all* of:

Inherited origbft rules. The corresponding `origbft-proposal-validity` (respectively, `origbft-block-validity`) rules hold for B ; these are assumed to include the *Parent rule* that B 's parent is bft-valid.

Linearity rule. $\text{snapshot}(B|_{bft}^1) \leq_{bc} \text{snapshot}(B)$.

Tail Confirmation rule. The headers $B.\text{headers}_{bc}$ form the σ -block tail of a bc-valid-chain. (`construction.md:570–577`.)

Finality-in-context is unchanged from $\Pi_{origbft}$: `bft-last-final` is defined the same way as `origbft-last-final` (`construction.md:621–623`).

Objective validity versus honest voting. The validity rules are separated from the honest voting condition on purpose: even a proposal voted for by 100% of validators is not bft-proposal-valid unless it satisfies the rules, and a purportedly valid bft-block carrying a valid notarization proof is not recognized by *any* honest node if it fails the other bft-block-validity rules. This separation is essential to keeping the finalized chain safe against a complete break of Π_{bft} — even of the validator signature algorithm — as long as Π_{bc} remains safe (`construction.md:581–585`).

The Linearity rule. Within a bft-valid-chain, the referenced bc-snapshots cannot roll back. The rule replaces and implies Crosslink 1's Increasing Score rule; it prevents attacks that propose bc-chains forked from much earlier (lower-difficulty) blocks; it bounds the disruption to the bounded-available ledger even if Π_{bft} is subverted, because the finalized chain is a Π_{bc} chain; and it eliminates the Snap-and-Chat/Crosslink 1 “sanitization” step entirely. The rule is relative to the parent bft-block's snapshot even when that parent is not yet final in the current context (`construction.md:587–609`).

The rule deliberately permits keeping the *same* snapshot as the parent. The allowance is required to preserve liveness of Π_{bft} relative to Π_{origbft} : honest proposers may need to propose at a minimum rate — Streamlet, for example, needs three notarized blocks in consecutive epochs and assumes proposals from honest leaders each epoch. The alternative of null proposals was rejected because it would require specifying null-proposal rules in Π_{origbft} (construction.md:611–619). The machine-checked model of §3.8 diverges from the book exactly here.

Construction 3.13 (Honest proposal). An honest proposer chooses $P.\text{headers_bc}$ as the σ -block tail of its bc-best-chain if that is consistent with the Linearity rule, and otherwise sets $P.\text{headers_bc}$ to the same headers_bc as P 's parent bft-block. It makes no proposals until its bc-best-chain is at least $\sigma + 1$ blocks long, since otherwise headers_bc cannot be constructed; at exactly $\sigma + 1$ blocks the snapshot is $\text{Origin}_{\text{bc}}$, which satisfies Linearity because $\text{Origin}_{\text{bft}}$'s snapshot is $\text{Origin}_{\text{bc}}$ (construction.md:625–637).

Construction 3.14 (Honest voting). A validator first updates its view of both subprotocols with the bc-headers in $P.\text{headers_bc}$, downloading and validating the referenced bc-blocks, and recursively any unseen bft-chains referenced by their context_bft fields. The validation order — supermajority checks first, then PoW header checks, before block download — bounds denial-of-service, and the Linearity rule ensures only one bc-chain need be validated per bft-chain. The validator votes for P only if:

Valid proposal criterion. Proposal P is bft-proposal-valid.

Confirmed best-chain criterion. The chain $\text{snapshot}(P)$ is part of the validator's bc-best-chain at a bc-confirmation-depth of at least σ .

(construction.md:639–657.)

The Confirmed best-chain criterion is subtle. It ensures that $\text{snapshot}(P)$ is bc-block-valid with σ bc-block-valid blocks after it in the validator's best chain, but $P.\text{headers_bc}[\sigma - 1]$ need *not* be in the validator's best chain — the chain may fork after $\text{snapshot}(P)$. Proposal validity must remain objective. Versus Snap-and-Chat: that construction had the confirmed-snapshot voting condition, but neither shipped headers with proposals nor made objective confirmation a validity matter; shipping headers improves the finalization-latency/security trade-off (construction.md:659–678).

Classification. *Designed but unspecified.* The prototype does not yet enforce this subsection's rules on its voting path (§3.9).

3.5 The best-chain side: Π_{bc}

All Crosslink 2 consensus-rule changes to Π_{bc} are non-contextual; modulo them, contextual validity is the same as in Π_{origbc} . The bc-transaction consensus rules are entirely unchanged: no reordering, no sanitization; parent bc-chains and heights are preserved (construction.md:267, 701–703).

Definition 3.15 (Π_{bc} validity). For the genesis bc-block we must have $\text{Origin}_{\text{bc}}.\text{context_bft} = \text{Origin}_{\text{bft}}$, and therefore $\text{LF}(\text{Origin}_{\text{bc}}) = \text{Origin}_{\text{bft}}$ (construction.md:684). A bc-block H is bc-block-valid iff *all* of:

Inherited origbc rules. Block H satisfies the corresponding origbc-block-validity rules.

Valid context rule. $H.\text{context_bft}$ is bft-block-valid.

Extension rule. $\text{LF}(H|_{\text{bc}}^1) \leq_{\text{bft}} \text{LF}(H)$.

Last Final Snapshot rule. $\text{snapshot}(\text{LF}(H)) \leq_{\text{bc}} H$.

Finality depth rule. Define $\text{finality-depth}(H) := \text{height}(H) - \text{height}(\text{snapshot}(\text{LF}(H)))$; then either $\text{finality-depth}(H) \leq L$ or $\text{is-stalled-block}(H)$.

(`construction.md:686–693`; the community protocol guide restates the Last Final Snapshot rule verbatim, `crosslink-protocol-guide @ b77d845, src/tfl-lexicon.md:3–5`.)

Stalled Mode. The finality gap at time t is, roughly, the number of bc-blocks not yet finalized; the Finality depth rule makes this precise as the objective finality-depth of a block. If the gap exceeds the threshold L , that likely signals an exceptional or emergency condition in which accepting spends into new bc-blocks is undesirable. The Stalled Mode policy is modelled by the predicate $\text{is-stalled-block} : \text{bc-block} \rightarrow \text{boolean}$; a block satisfying it is a *stalled block* (`construction.md:401–413`). The node-local prose notion (“blocks not yet finalized at time t ”) and the objective per-block notion differ in corner cases just after a reorg (`construction.md:695–699`).

Two cautions. First, a bc-block producer is only constrained to produce stalled blocks while its view of the finalization point is not advancing; an adversary that subverts the BFT protocol *without* stalling finalization is never constrained by Stalled Mode (`construction.md:401–413`). Second, the book warns that an honest bc-block-producer must *not* use information from the BFT protocol beyond the specified consensus rules to decide which bc-valid-chain to follow; additional constraints could let an adversary that subverts Π_{bft} influence the bc-best-chain in ways outside the safety argument (`construction.md:715–717`).

Construction 3.16 (Honest block production). The producer must follow the Π_{bc} validity rules, including producing a stalled block when the Finality depth rule requires it. To choose $H.\text{context_bft}$ it considers the set

$$\{ T : T \text{ is bft-block-valid and } \text{LF}(H|_{\text{bc}}^1) \leq_{\text{bft}} \text{bft-last-final}(T) \},$$

chooses one of the longest such chains C — breaking ties first by maximizing the score of the snapshot of $\text{bft-last-final}(C)$, then by smallest hash — and sets $H.\text{context_bft} = C$. Rationale: choosing a longest chain follows Streamlet’s build-on-longest-notarized; the score tie-break inhibits an adversary from finalizing a lesser-scoring chain (`construction.md:705–731`).

Classification. *Designed but unspecified*; the deployment track has, at the pinned commit, dropped the Stalled Mode rules and does not enforce the Extension, Last Final Snapshot, or Finality depth rules in bc-block validity (§3.9).

3.6 The two ledgers

Each node i derives two chains from its views: a locally finalized chain fin_i^t and a locally bounded-available chain $(\text{ba}_\mu)_i^t$, for a per-node confirmation depth μ . The book’s macros render the task names `localfin` and `localba $_\mu$` as `fin` and `ba`, and `snapshotlf( $H$ )` as `snapshot(LF( $H$ ))` (`tfl-book @ fe6e1d6, macros.txt:29, 63–64`); we use the rendered forms.

The finalized chain. The locally finalized chain starts at $\text{fin}_i^0 = \text{Origin}_{\text{bc}}$ and must not be exposed to clients until the node has synced (`construction.md:462–488`).

Construction 3.17 (Finalization update (updatefin)). On a best-chain update at time t , with previous state fin_i^s , node i computes $N := \text{candidate}(\text{ch}_i^t)$ and:

- (i) if $N \succeq_{\text{bc}} \text{fin}_i^s$, sets $\text{fin}_i^t \leftarrow N$;
- (ii) otherwise sets $\text{fin}_i^t \leftarrow \text{fin}_i^s$, and if additionally $N \not\leq_{\text{bc}} \text{fin}_i^s$ — that is, N conflicts with fin_i^s — records a *finalization safety hazard*, possible only if the global safety assumptions are violated. The hazard record includes ch_i^t and the fin update history since the last update that was an ancestor of N .

(construction.md:475–486.)

Definition 3.18 (Local finalization linearity). Node i has *Local finalization linearity* up to time t iff the time series of bc-blocks $\text{fin}_i^{r \leq t}$ is bc-linear (construction.md:466–469).

The update rule guarantees Local finalization linearity *by construction* (construction.md:462–488). Local state is needed because on a reorg — even one shorter than σ — the value $\text{candidate}(\text{ch}_i^t)$ may temporarily roll back; if Final Agreement holds, the new snapshot is an ancestor of the old and will roll forward along the same path, but applications must not see the temporary rollback (construction.md:497–499). This is precisely the service today’s applications lack: wallets repair rollbacks after the fact by truncate-and-rescan (*Wallet Guide*, §“Reorganisations: truncate and rescan”), and anchor validity tracks the unfinalized chain (*Ironwood Guide*, §“Proof of ownership of some valid note: the commitment tree”).

Lemma 3.19 (Local fin-depth). *In any execution of Crosslink 2, for any node i honest at time t , there exists a time $r \leq t$ such that $\text{fin}_i^t \leq_{\text{bc}} \text{ch}_i^r \upharpoonright_{\text{bc}}^\sigma$.*

Proof. By $\text{candidate}(\text{ch}_i^u) \leq_{\text{bc}} \text{ch}_i^u \upharpoonright_{\text{bc}}^\sigma$ applied at the last time fin changed, or at genesis (construction.md:490–495). $\square$

Definition 3.20 (Assured Finality). An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u , we have $\text{fin}_i^t \sim_{\text{bc}} \text{fin}_j^u$ (construction.md:504–506).

Assured Finality is the main safety goal of Crosslink 2, intended to hold under reasonable assumptions about *either* Π_{origbft} *or* Π_{origbc} . Its restriction to $i = j$ is equivalent to Local finalization linearity, so Assured Finality implies Local finalization linearity for all honest nodes; the converse implication does not hold — the local property is necessary but not sufficient (construction.md:473, 501–508).

Why $\text{candidate}(H)$, not $\text{snapshot}(\text{LF}(H))$. The Last Final Snapshot rule guarantees $\text{snapshot}(\text{LF}(H)) \leq_{\text{bc}} H$, but *not* that its depth relative to H is at least σ . Using $\text{candidate}(H)$ ensures the finalization point is at least σ -confirmed, which the proof of Assured Finality needs: $\text{candidate}(H) \leq_{\text{bc}} H \upharpoonright_{\text{bc}}^\sigma$ combines with the Local fin-depth lemma and Prefix Agreement. An open book TODO offers the alternative of strengthening the Last Final Snapshot rule to $\text{snapshot}(\text{LF}(H)) \leq_{\text{bc}} H \upharpoonright_{\text{bc}}^\sigma$ (construction.md:510–518).

Definition 3.21 (Locally bounded-available chain). For a per-node confirmation depth μ :

$$(\text{ba}_\mu)_i^t = \begin{cases} \text{ch}_i^t|_{\text{bc}}^\mu & \text{if } \text{fin}_i^t \leq_{\text{bc}} \text{ch}_i^t|_{\text{bc}}^\mu, \\ \text{fin}_i^t & \text{otherwise} \end{cases}$$

(construction.md:522–527).

The “otherwise” arm is needed because after fin switches forks under Zcash’s damped difficulty adjustment ($\text{PoWAveragingWindow} = 17$ blocks, $\text{PoWMedianBlockSpan} = 11$ blocks, quoted by the book from the Zcash protocol specification § diffadjustment), the truncated best chain $\text{ch}_i^t|_{\text{bc}}^\mu$ might not descend from fin_i^t even with all safety assumptions satisfied; the definition also makes the Ledger prefix property trivial to prove. The security goal for ba_μ is Agreement on the view ba_μ (Definition 3.3; construction.md:545–555, 552).

Theorem 3.22 (Ledger prefix property). For any node i that is honest at time t , and any confirmation depth μ , $\text{fin}_i^t \leq_{\text{bc}} (\text{ba}_\mu)_i^t$.

Proof. By construction of $(\text{ba}_\mu)_i^t$ (construction.md:531–536). $\square$

Lemma 3.23 (Local ba-depth). In any execution of Crosslink 2, for any confirmation depth $\mu \leq \sigma$ and any node i honest at time t , there exists a time $r \leq t$ such that $(\text{ba}_\mu)_i^t \leq_{\text{bc}} \text{ch}_i^r|_{\text{bc}}^\mu$.

Proof. Either $(\text{ba}_\mu)_i^t = \text{fin}_i^t$, and the Local fin-depth lemma applies with $\mu \leq \sigma$; or $(\text{ba}_\mu)_i^t = \text{ch}_i^t|_{\text{bc}}^\mu$, trivially with $r = t$ (construction.md:538–543). $\square$

Stall, do not finalize unsafely. In normal operation fin_i^t lags only a few blocks behind $(\text{ba}_\sigma)_i^t$, unless the chain enters Stalled Mode. With $\mu = \sigma$, the choice between following fin and following ba is a choice of stall behaviour: stall immediately (fin), or keep processing user transactions until L blocks have passed (ba) (construction.md:415).

Syncing and checkpoints. Implementations should bake a checkpointed bft-block $B_{\text{checkpoint}}$ into each release; node i exposes fin_i^t and $(\text{ba}_\mu)_i^t$ only once synced, that is, once $B_{\text{checkpoint}} \leq_{\text{bft}} \text{LF}(\text{ch}_i^t)$, $\text{snapshot}(B_{\text{checkpoint}}) \leq_{\text{bc}} \text{fin}_i^t$, and the timestamp of fin_i^t is within a threshold of the current time (construction.md:557–562).

The chapter closes: “At this point we have completed the definition of Crosslink 2. In Security Analysis of Crosslink 2, we will prove it secure.” The structural rules above are the complete definition; all proofs beyond the in-chapter lemmas live in security-analysis.md (construction.md:735); this volume’s account of that analysis is §6.

Classification. *Designed but unspecified*, with the candidate-versus-strengthened-rule fork explicitly open in the book.

3.7 Parameters

The construction has three parameters and fixes production values for none of them.

- The bc-confirmation-depth $\sigma \in \mathbb{N}^+$ (construction.md:391). No production value is fixed anywhere in the sources.
- The finalization gap bound $L \in \mathbb{N}^+$, significantly greater than σ ; “In practice, L should be at least 2σ ” (construction.md:391, 405).

- The per-node depth μ with $0 < \mu \leq \sigma$; the default and recommendation is $\mu = \sigma$. A smaller μ is at the node’s own risk and does not affect fin (construction.md:395–399).

The prototype ships PROTOTYPE_PARAMETERS with $\sigma = 3$ and $L = 7$ — satisfying $L \geq 2\sigma$ — under the explicit caveat “No verification has been done on the security or performance of these parameters” (zebra-crosslink @ 6d02a1b, zebra-crosslink/src/chain.rs:216–222).

For calibration: today’s wallet policy defaults are 3 confirmations for trusted and 10 for untrusted outputs under the ZIP-315 ConfirmationsPolicy (*Wallet Guide*, §“Confirmations, trust, and spendability”) — a per-wallet risk heuristic, where σ is a consensus-visible input to finalization. On the node side, the k -deep interface constant of the deployed best-chain protocol is MAX_BLOCK_REORG_HEIGHT = 1000 in mainline Zebra, documented as “a local-only node policy; it is not part of consensus” and sized as defence-in-depth against sustained consensus splits (zebra @ 9f3166b, zebra-chain/src/parameters/constants.rs:30); the zebra-crosslink fork still uses the older value 99 = MIN_TRANSPARENT_COINBASE_MATURITY – 1 (zebra-crosslink @ 6d02a1b, zebra-state/src/constants.rs:31, zebra-chain/src/transparent.rs:52). How that constant will relate to σ is undetermined.

Classification. The parameter constraints are *designed but unspecified*; the concrete values are an *open problem*. The quoted constants are *specified* in code at the pinned commits.

3.8 The machine-checked fragment

The Informal Systems Quint specification (crosslink-spec @ 8edc3e9) is an explicit proof of concept covering *only* the isValid check for BFT proposals — Tail Confirmation plus Linearity — over abstract single-node PoW and BFT block stores. Its README states the logic was inferred mostly from a YouTube talk and “might not be up-to-date with the current protocol design ideas”. The Quint file src/validity.qnt is tangled from validity.md via the lmt tool (README.md:25–32; validity.md:1–7).

The checked predicate is:

```
pure def isValid(bft_store, pow_store, p) = all {
  pow_store.contains(p.block),
  p.bftContext == bft_store.last(),
  sigmaConfirmed(p.block, p.confirmation_tail, confirmation_depth),
  linear(p.block, p.bftContext, pow_store)
}
```

where sigmaConfirmed(b, t, sigma) requires a fork-free confirmation tail of size *at least* σ whose parents chain back to b, and linear(b, context, work_store) requires work_store.isDescendant(context.pow_hash, b.hash) *and* not(context.pow_hash == b.hash) (crosslink-spec @ 8edc3e9, validity.md:131–138, 150–158, 171–176).

Machine-checked results. The invariant prefix_inv — “the PoW history defined by BFT finalization is a path in the PoW block store”, formalized for all adjacent indices i as

```
pow_store.isDescendant(bft_store[i].pow_hash, bft_store[i+1].pow_hash)
```

— found no violation over 10000 random traces of 20 steps, and was model checked by TLC (via quint compile --target tlaplus) in about three minutes on the verification instance (confirmation_depth = 1, at most 8 blocks per store; the simulation instance uses confirmation_depth = 2, unbounded). The reachability witness finalization_witness (3 BFT blocks stored) was found by both the simulator and TLC, and the run finalizationTest passes under quint test (README.md:39–89; validity.md:376–461, 405–410, 463–482).

Divergences from the book. The model checks a rule set that neighbours, but does not match, the book’s:

- Its linear predicate *forbids* keeping the same snapshot as the parent, via the conjunct `not(context.pow_hash == b.hash)`, whereas the book’s Linearity rule allows equality and argues the allowance is necessary for Π_{bft} liveness (§3.4). The spec itself carries the comment “this actually seems allowed. TODO: perhaps remove” (`validity.md:171–176` versus `construction.md:573, 611–619`).
- Its `sigmaConfirmed` accepts a tail of size $\geq \sigma$ where the book requires exactly σ headers; its proposal carries the snapshot block itself plus an *unordered* set of tail blocks, against the book’s ordered deepest-first header sequence with `snapshot(B) = B.headers_bc[0][1]bc`; and its `isValid` requires `p.bftContext == bft_store.last()`, modelling the BFT side as a single forkless linear chain, unlike the book’s set of bft-chains (`validity.md:65–96, 131–158` versus `construction.md:417–436`).
- Not modelled at all: `context_bft` in bc-blocks, the Extension, Last Final Snapshot, and Finality depth rules, Stalled Mode, bft-last-final, voting and notarization proofs, and signatures (`crosslink-spec @ 8edc3e9 README.md, validity.md`).

Classification. The results above are *specified* in the strongest pre-spec sense available — machine-checked at a pinned commit — but of a *neighbouring* rule set: stricter than the book on snapshot equality, looser on tail size and BFT forks, and silent on the whole Π_{bc} side. We therefore cite the Quint model as a proof-of-concept formalization, never as verification of the book’s construction.

3.9 Divergences in the deployment track

The Shielded Labs/Eiger implementation (`zebra-crosslink @ 6d02a1b`) carries a byte-for-byte copy of the book’s construction chapter, prepended with three warning admonitions: the copy is “an almost direct paste” committing to a precise text for a security design review, not reviewed or endorsed by Daira-Emma Hopwood or Jack Grigg; the `zebra-crosslink` design diverges by *not* including the Stalled Mode rules, on the rationale that stakers redelegating prevents hazards such as BFT stalls; and it does not use the outer signature, trading a hazard many participants could leverage for one only the current proposer could, judged not worth it — with the recorded consequence that a direct hash or copy of the most recent BFT finality certificate is insufficient evidence that a specific bitwise copy is canonical (`book/src/design/cl2-construction.md:3–17`, diffed against `construction.md` at `fe6e1d6`).

At the pinned commit, the further construction-level divergences are:

- *Fat pointer for context_bft.* The fork adds `fat_pointer_to_bft_block` to the PoW block header: a bundle of Ed25519 signed precommit votes — a 44-byte vote template whose first 32 bytes are the BLAKE3 hash of the target BFT block, plus 96-byte entries of public key and vote signature — constructed from the deciding round’s signed precommit votes (tenderlink’s `FatPointerToBftBlock3`, converted at `zebra-crosslink/src/lib.rs:1846–1860`); the earlier Malachite commit-certificate conversion survives only in the undeclared dead-code module `src/malctx.rs`. It replaces both the book’s plain `context_bft` commitment and the notarization-proof/outer-signature structure with a self-certifying pointer (`zebra-chain/src/block/header.rs:109–133, 195–201`; `zebra-crosslink/src/lib.rs:1811–2052`).
- *Immediate finality instead of bft-last-final.* The BFT engine is Tendermint-style: the in-house tenderlink package (`git.sr.ht/~azmr/stuff` rev `0e88509`), driven directly from `zebra-crosslink/src/lib.rs`. Each tenderlink-decided BFT block is treated as immediately final: the handler (installed as `tenderlink::ClosureToPushDecidedBlock`, `lib.rs:1234`) sets the latest final block from the decided block’s headers and issues `zebra_state::Request::CrosslinkFinalizeBlock` so the PoW state finalizes that ancestor

— the implementation’s fin extraction. With no BFT-final block known, the node falls back to the PoW block at `MAX_BLOCK_REORG_HEIGHT` depth (zebra-crosslink/Cargo.toml:74–79; zebra-crosslink/src/lib.rs:235–330, 499–601).

- *Validity rules mostly unimplemented.* The Rust `BftBlock` carries the fields `version`, `height`, `previous_block_fat_ptr`, `finalization_candidate_height`, and `headers`; its sole constructor `try_from` validates only `headers.len() =  $\sigma$` and logs “not yet implemented: all the documented validations”; deserialization rejects more than 2048 headers. The vote-path validation checks only that the new block’s previous fat pointer hashes to the current tip’s, and that the node knows the PoW block referenced by `headers.first()`. The Linearity rule, Tail Confirmation validation, and the Confirmed best-chain criterion are *not* enforced on this path; the decided-block handler additionally verifies fat-pointer signatures, with a TODO to check the public keys against the roster (zebra-crosslink/src/chain.rs:61–151, 99–104; zebra-crosslink/src/lib.rs:522–541, 790–822).
- *Header-order inconsistency.* The in-memory header order is documented as “reversed from the specification” (built tip-first), and the accessors disagree at this commit: `finalization_candidate()` returns `headers.last()` while the decided-block handler computes the newly final PoW hash from `headers.first()`; the consistency assertion against `finalization_candidate_height` is commented out, and header fetching carries the comment “TODO: do we want these in chain order or walk-back order” (chain.rs:52–54, 124–126; lib.rs:543–546, 809, near :432).

Beyond the book’s construction, the deployment track adds a mechanism-design layer — finalization rewards paid only in blocks that advance finality, a staking transaction format with the Crosslink Staking Orchard Restriction, staking epochs, a roster in ledger state, and the General Ledger State Design Invariant that all ledger state, including the PoS/BFT roster and finality status, is computable entirely from PoW blocks and transactions, with finality status the sole “height-eventual” state (book/src/design/nutshell.md:23–128).

Classification. The code facts above are *specified* in the pinned-commit sense, but the track is moving: Milestone 5 (“Pre-Productionization”) completed 2026-04-15, productionization is ongoing with no confirmed mainnet activation date, and the first independent security and mechanism-design review is complete (forum.zcashcommunity.com/t/shielded-labs-crosslink-deployment-updates/49706; shieldedlabs.net/roadmap; x.com/ShieldedLabs status 2019547550009946528; all accessed 2026-07-15). Every implementation claim in this subsection is pinned to commit 6d02a1b and must be re-verified against a post-productionization commit before it is treated as the deployed construction.

4 Stake, delegation, and slashing

Crosslink’s finality vote is stake-weighted, so the construction owes answers to the questions every proof-of-stake system faces: how ZEC becomes voting weight, how a wallet delegates without surrendering custody, and what punishes a finalizer that misbehaves. This section collects those answers — and their absences. The division of labour among the sources is unusual and governs every citation below: the ECC design book abstains from the whole topic, listing “PoS subprotocol selection” and “Issuance and supply mechanics, such as how much ZEC stakers may earn” among its “Major Missing Components” (tfl-book, src/introduction/status-and-next-steps.md at fe6e1d6f), so every concrete stake, delegation, and slashing decision lives with Shielded Labs: the zebra-crosslink design book (below, “the implementation book”) and two dated posts¹¹.

¹¹Shielded Labs, “Crosslink: Updated Staking Design”, <https://shieldedlabs.net/crosslink-updated-staking-design/> (forum post 2025-10-27, blog dated 2025-11-05); “Crosslink Early Tokenomics Design”, forum thread t/52154 (2025-

Nothing in this section reaches the *specified* bin. No Crosslink or staking ZIP exists: at `zcash/zips @ 69610984` (2026-07-09) the only Crosslink references are a citation footnote in ZIP 218 and a `tfl-book` reference in `draft-ecc-onchain-accountable-voting`; the “IO Finalizer” and “Spend Finalizer” of ZIP 374 are unrelated PCZT roles. The implementation book states “We will be refining a ZIP Draft [Google Doc link] which will enter the formal ZIP process as the prototype approaches maturity” (`book/src/design.md` at `6d02a1b8`). The machine-checked corpus is equally silent: the Informal Systems Quint model covers only the proposal-validity function (Tail Confirmation and Linearity rules) and models no stake, roster, voting power, delegation, bonding, or slashing (`crosslink-spec @ 8edc3e94`, `validity.md`, `src/validity.qnt`). Everything below is therefore *designed but unspecified* at best, or implementation-defined prototype behaviour, or an *open problem*; we keep the bin visible throughout. The implementation book itself warns that its extensions “may violate some of the assumptions in the security proofs” of the Crosslink 2 construction and calls its staking chapter “a provisional sketch” that “will eventually be entirely redundant with... a canonical ZIP” (`book/src/design.md`, `book/src/design/nutshell.md` at `6d02a1b8`).

4.1 Stake in the abstract construction

The Crosslink 2 construction treats stake as an abstraction: “For each epoch, there is a fixed number of voting units distributed between the star-bft-nodes, which they use to vote for a star-bft-proposal” (`tfl-book`, `src/design/crosslink/construction.md` at `fe6e1d6f`; the implementation book mirrors the passage verbatim in `book/src/design/cl2-construction.md`). The notarization rule for a proposal P “must require at least a two-thirds absolute supermajority of voting units in P ’s epoch to have been cast for P ”, and may require more (*ibid.*). How ZEC-denominated stake maps onto voting units is delegated to the unselected PoS subprotocol — precisely the “Major Missing Component” above.

The adversary bound is stated over the same abstraction. Notation follows the book: Π_{bft} is the BFT subprotocol as adapted by Crosslink, Π_{origbft} its underlying engine; the book’s $\star_{\text{bft}}$ is a wildcard ranging over both.

Definition 4.1 (One-third bound on non-honest voting). An execution of Π_{bft} has the *one-third bound on non-honest voting* property iff for every epoch, *strictly* fewer than one third of the total voting units for that epoch are ever cast non-honestly. A voting unit is cast non-honestly iff: it is cast other than by the holder of the unit (due to key compromise or any flaw in the voting protocol, for example); or it is double-cast; or the holder, following the conditions for honest voting in $\Pi_{\star_{\text{bft}}}$ according to its view, should not have cast that vote (`tfl-book`, `src/design/crosslink/construction.md` at `fe6e1d6f`, near-verbatim).

The quantifier “ever” is load-bearing. The book’s own security caveat spells out the consequence: the bound counts ballots cast at *any* time in the execution, including votes cast with a compromised key for a long-finished epoch; “Therefore, validator keys of honest nodes must remain secret indefinitely. Whenever a key is rotated, the old key must be securely deleted”, with further discussion deferred to `tfl-book` issue #140 (*ibid.*). We return to what this demands of finalizer key management in §4.7.

The one commitment the ECC book does make about delegation is a goal, not a mechanism: “The protocol should enable users of shielded mobile wallets to delegate ZEC to PoS consensus providers and earn a return on that ZEC coming via ZEC issuance or fees. Doing this may expose users to a risk of loss of delegated ZEC (such as through ‘slashing fees’). The protocol must guarantee that PoS consensus providers have no discretionary control over such delegated funds (including that they cannot steal those funds)” (`tfl-book`, `src/design/goals.md` at `fe6e1d6f`). Note the tension recorded for §4.6: the goal contemplates “slashing fees” as a user risk; the deployed design forswears them.

09-12); deployment updates, forum thread `t/49706`; forum threads under `https://forum.zcashcommunity.com/t/`. All fetched 2026-07-15.

Classification. *Designed but unspecified:* the voting-unit model and Definition 4.1 come from a design book with security arguments, not a specification. The mapping from stake to voting units — proportional weighting as in the prototype (§4.8) versus discretized units, and how the per-epoch roster snapshot is taken — is an *open problem*: no design document fixes it.

4.2 Positions, the roster, and the active set

The canonical staking statement is one sentence: “Staking in Crosslink takes place exclusively within the Orchard pool” (Shielded Labs, Updated Staking Design, 2025-10-27/11-05). Participants lock ZEC to delegate stake to a chosen finalizer and earn rewards; Penumbral-style fully shielded delegations were considered and deferred to future versions, prioritizing time-to-market (ibid.). The 2025-10/11 design predates NU6.3’s pool terminology: read “the Orchard pool” there as the Orchard-protocol shielded pool — since NU6.3, the Ironwood pool. That pool is the one this series constructs in the *Ironwood Guide* (§“Representation of a note: notes and note commitments”); nothing below alters its note machinery.

Definition 4.2 (Staking position, roster, stake weight, active set). The *Roster* is a Ledger State table tracking staking positions created by stakers and attached to specific finalizers. Each *staking position* comprises at least: a target finalizer verification key, a staked ZEC amount, and an unstaking/redelegation verification key. The finalizer verification key “serves as the sole on-chain reference to a specific finalizer”; an entity may produce any number of finalizer keys. Unstaking/redelegation keys “are unique to the position itself (and not linked on-chain to a user’s other potential staking positions or other activity)”. The sum of outstanding staking positions designating a specific finalizer verification key at a given block height is the *stake weight* of that finalizer, and equals its BFT voting weight. At a given height the top K (“currently” $K = 100$) finalizers by stake weight are the *active set* (implementation book, `book/src/design/nutshell.md` at 6d02a1b8).

The roster obeys the implementation book’s General Ledger State Design Invariant: “All changes to Ledger State can be computed entirely from (PoW) blocks, transactions, and data transitively referred to by such. This includes all existing Ledger State in mainnet Zcash today, the PoS and BFT roster state (see below), and the finality status of blocks and transactions” (ibid.). Roster and active set are thus height-immediate — a function of the chain up to the height in question. Finality status is the sole height-eventual Ledger State: a property of height H computable only from a later attesting block; all nodes seeing the attesting block agree, and if the attesting block rolls back, the protocol guarantees an alternative block will eventually attest the same candidate (ibid.). Wallets today approximate the missing finality signal with ZIP-315 confirmation depths (*Wallet Guide*, §“Transaction construction”); Crosslink turns it into consensus state.

Classification. *Designed but unspecified* throughout. The active-set bound is explicitly provisional (“currently” 100), the book’s own TODO list flags “active set selection” as unresolved (`book/src/design/nutshell.md` at 6d02a1b8), and the prototype applies no top- K truncation at all (§4.8).

4.3 Staking actions and draft consensus rules

The implementation book extends the transaction format with an optional staking-action field carrying four abstract actions (`book/src/design/nutshell.md` at 6d02a1b8):

- *stake* — creates a staking position assigning a transparent ZEC amount, which must be a power of 10, to a finalizer, and includes a signature verification key authorizing later redelegate/unstake actions;

- *redelegate* — identifies a position, a new finalizer, and a signature valid under the position’s published verification key;
- *unbond* — moves a position into the *unbonding* state: redelegations become invalid and the amount stops contributing to staking weight for finalizers or the staker;
- *claim* — removes an unbonding-state position and contributes the ZEC to the chain value pool balance, which — by the Orchard restriction below — may only go to an Orchard destination and/or fees.

Value flows must balance per the Zcash protocol specification’s chain value pool accounting (§4.17); the Orchard side of that balance is the value-commitment machinery of the *Ironwood Guide* (§“Conservation of value: value commitments and the binding signature”). The draft consensus rules, verbatim from the implementation book (book/src/design/nutshell.md at 6d02a1b8):

1. *Orchard restriction* (context-free validity check): “A transaction which contains any *staking actions* must not contain any other fields contributing to or withdrawing from the Chain Value Pool Balances *except* Orchard actions, explicit transaction fees, and/or explicit ZEC burning fields.”
2. *Staking epochs*: “Once Crosslink activates at block height $H_{\text{crosslink_activation}}$, *staking epochs* begin, which are contiguous [sic] spans of block heights with two phases: *staking day* and the *locked phase*.” The staking-day length is a protocol constant of roughly 24 hours of blocks; the locked-phase length is a constant of roughly 6×24 hours, “so that *staking day* is approximately one day per week.”
3. *Locked staking actions restriction*: “If a transaction includes any staking actions *except for* redelegate, and that transaction is in a block height in a *locked phase* of the *staking epoch*, then that block is invalid.”
4. *Unbonding delay*: “If a transaction is [sic] block height H includes any *claim* staking actions which refer to *unbonding state* positions which have not existed in the unbonding state for at least one full staking epoch, that block is invalid.” The source notes the implication that the same staking day cannot include both an *unbond* and a *claim* for one position.
5. *Amount quantization*: “If a transaction includes any staking actions with an amount which is not a power of 10 ZEC (or ZAT), that transaction is invalid.” The book’s TODO list clarifies the intent that the restriction apply only to the *stake* (create-position) action.

The quantization is a privacy device. The blog states it plainly: “Staking transactions must be made in exponential increments of 10 (e.g., 1, 10, 100, 1,000, 10,000, etc.)”; “This approach prevents observers from easily inferring individual staking patterns, preserving privacy while maintaining accountability”; and, on the other side of the trade, “For security and transparency, staked amounts are revealed so the community can verify the total ZEC staked and its distribution across finalizers”, with “Transactions... processed in fixed 5-day epochs, grouping all commitments into a single batch” (Updated Staking Design, 2025-10-27/11-05). Quantisation of shielded weight has precedent in this series: the shielded voting protocol quantises note value into 0.125-ZEC ballots for the same linkability reason (*Voting Guide*, §“Ballots and eligibility”).

Two gaps are visible already at this level. First, the *stake* action assigns “a transparent ZEC amount” while staking is exclusively Orchard-pool: how the revealed, quantized amount balances against Orchard value fields under §4.17 is unspecified, and the prototype sidesteps it entirely — a transaction carrying a staking action bypasses the `has_inputs_and_outputs` consensus check with a “TODO: real staking transactions with inputs/outputs” (zebra-consensus/src/transaction/check.rs at 6d02a1b8). Second, the phrase “a power of 10 ZEC (or ZAT)” leaves the bucket range ambiguous: whether sub-1-ZEC buckets (10^k zatoshi) are permitted, and hence the

minimum stake, is unspecified; the blog’s examples start at 1 ZEC. How staking actions pay fees, and whether a transaction may carry several, are open TODOs in both book and implementation (book/src/design/nutshell.md at 6d02a1b8).

The delegation-safety goals for the first deployment are recorded as issues: “Delegating to a finalizer does not enable the finalizer to steal your funds” and delegation “does not leak information that links the user’s action to other information about them, such as their IP address, their other ZEC holdings that they are choosing not to stake, or their previous or future transactions” (GH #124, implementation book, book/src/design/scoping.md at 6d02a1b8). The explicit first-deployment trade-offs cut the other way: *not* “supporting a large number of finalizers so that any user who wants to run their own finalizer can do so”, no support for casual users running finalizers, and no tokens representing staked ZEC that can be traded while the stake stays locked — the last excluded in both the tfl-book goals and the blog’s V1 (ibid.; tfl-book, src/design/goals.md at fe6e1d6f).

Classification. *Designed but unspecified:* the actions and rules above are draft consensus text in a self-described provisional sketch. The transaction format (value balancing), fee payment, bucket range, and action multiplicity are *open problems* acknowledged in the sources.

4.4 Epochs and parameter drift

The staking-time parameters have changed at every publication and the sources now disagree. Table 2 pins each statement to its date.

Source, date	Epoch	Unbonding	Also fixed
Early Tokenomics Design (forum t/52154, 2025-09-12)	10-day staking / batching windows	30 days	top-100 roster; 40/40 issuance split; no protocol slashing; finalizer commissions expected
Updated Staking Design (forum 2025-10-27; blog 2025-11-05)	fixed 5-day epochs, one batch per epoch	1 epoch; funds available in 5–10 days	Orchard-pool exclusivity; power-of-10 amounts
Mechanism-design audit proposal (2025-12-11)	5-day epochs	5–10 days	40/40 split; “no protocol-level slashing in V1”
Implementation book (6d02a1b8, snapshot 2026-04-21)	staking day (~1 day) + locked phase (~6 days)	≥ 1 full staking epoch	redelegate exempt from the locked phase

Table 2: Staking-parameter statements by source and date. Sources: forum t/52154 (2025-09-12); Shielded Labs blog (2025-10-27/11-05); book/src/design/analysis/history/2025-12-11-mech-design/proposal.md and book/src/design/nutshell.md, both at 6d02a1b8. Web sources fetched 2026-07-15.

The blog shortened the September windows “citing liquidity and accessibility”; the implementation book — which post-dates the blog — lengthens the effective cycle again to approximately one staking day per week. Which parameterization is current is unresolved. The divergence is not cosmetic on one point: the book’s locked-phase rule exempts *redelegate*, making redelegation valid at any height, while the blog has participants “reallocate their stake to another finalizer during these windows”, implying batched redelegation. Redelegation is the sole protocol-level punishment mechanism (§4.6), so its latency is security-relevant.

Classification. Each row is *designed but unspecified* with a date; the current values are an *open problem*. A ZIP must pin epoch length, staking-day length, and unbonding duration in blocks, and must fix redelegation latency explicitly.

4.5 Rewards and issuance

The implementation book routes staking rewards through the coinbase, gated on finality: consensus rules require the coinbase transaction to transfer the Crosslink-reward portion of new issuance *only* in blocks that advance finality. Per-block Crosslink rewards otherwise accumulate as *pending finalization rewards*, and a finality-updating block must distribute the entire pending amount to stakers and finalizers, computed against the active set as of the most recent final block prior to the update (book/src/design/nutshell.md at 6d02a1b8). With $c := \text{COMMISSION_FEE_RATE} = 0.1$ a fixed protocol parameter, the slices of the pending amount S_{total} are

$$S_{\text{finalizer}} = c \cdot \frac{W_{\text{finalizer}}}{W_{\text{total}}} \cdot S_{\text{total}} \quad (\text{active-set finalizers only}), \quad S_{\text{staker}} = (1 - c) \cdot \frac{W_{\text{staker}}}{W_{\text{total}}} \cdot S_{\text{total}}, \quad (1)$$

where $W_{\text{participant}}$ is that participant’s total stake weight and $1 - c = 0.9$ is called the staker reward rate. The staker weight W_{staker} counts all of a staker’s positions regardless of whether they are delegated to an active-set finalizer; the sum of all slices is S_{total} or less, “with the difference coming from stake assigned to finalizers outside the active set” (ibid.).

The issuance side is thinner. The Early Tokenomics post fixes “the block reward should be split equally (40/40) between miners and finalizers” (forum t/52154, 2025-09-12), and the mechanism-design audit proposal confirms the “40/40 issuance split” as an audit assumption (book/src/design/analysis/history/2025-12-11-mech-design/proposal.md at 6d02a1b8). The destination of the remaining 20% is not stated in any source we located. How Crosslink rewards interact with the issuance schedule, halvings, and the 21M cap is exactly the “Issuance and supply mechanics” component the tfl-book lists as missing (tfl-book, src/introduction/status-and-next-steps.md at fe6e1d6f). The commission model is also unsettled: the book fixes $c = 0.1$ for all active-set finalizers, but the September post said “We also expect finalizers to charge commissions”, suggesting per-finalizer market rates (forum t/52154, 2025-09-12).

One UX goal collides with the quantization rule. Goal GH #158 asks for compounding returns without user or wallet action (implementation book, book/src/design/scoping.md at 6d02a1b8), but a reward paid to an Orchard destination cannot silently increase a power-of-10 position. No source resolves the collision; the book’s own TODO list flags “rewards distribution via coinbase” and “quantization of amounts / time” as unresolved (book/src/design/nutshell.md at 6d02a1b8).

Classification. *Designed but unspecified:* Equation (1) and the pending-rewards rule are draft consensus text. Issuance interaction, the 20% remainder, the commission model, reward eligibility across the unbond–claim lifecycle, and the compounding mechanism are *open problems*. The prototype implements none of this (§4.8).

4.6 Slashing, and its deliberate absence

The design commitment is verbatim and unambiguous: “There will be no automatic, protocol-level slashing in this design, so the power lies fully in the stakers’ hands to reward and censure node operators” (forum t/52154, 2025-09-12); the audit proposal restates the constraint as “no protocol-level slashing in V1” (book/src/design/analysis/history/2025-12-11-mech-design/proposal.md at 6d02a1b8).

The ECC book never specified slashing either; it appears only as discussion. A “Potential changes” passage observes that “In most PoS protocols, the requirement to have a minimum amount of stake in order to make a proposal acts as a gatekeeping filter on proposals, and potentially allows parties that make invalid proposals to be slashed”, that stake-holding validators “can be slashed”, and that “there is no technical reason why the ability to make a bft-proposal has to be gatekept by a stake requirement — given the situation of Zcash in which we already have a mining infrastructure” (tfl-book, src/design/crosslink/potential-changes.md at fe6e1d6f). The closest the book comes

to accountability machinery is a sketch in its questions file: a *bft-final-vee* — two final bft-blocks with the same parent — as objective rollback evidence; the observation that more than $1/3$ of stake voting for conflicting blocks in one epoch proves the adversary-stake assumption was violated; and the note that such evidence “can be posted in a transaction to the bc-chain”, optionally latching Stalled Mode until manual intervention (tfl-book, `src/design/crosslink/questions.md` at `fe6e1d6f`). None of this is adopted as a rule.

What replaces slashing is a chain of accountability between roles, stated in the implementation book’s Five Component Model (`book/src/design/five-component-model.md` at `6d02a1b8`). Stakers police finalizers: it is “the job of the stakers to hold the finalisers accountable by detecting and punishing misbehavior... by redelegating stake away from ill-behaved finalizers to well-behaved finalizers”. Users police stakers, ultimately via “an emergency user-led hard fork (which is what happened in the STEEM/HIVE hard fork)”. The same file bounds finalizer power: finalizers “cannot unilaterally compromise liveness or censorship-resistance (even if an attacker controls $> 2/3$ of the stake-weighted votes)”; “A plurality of the finalisers (controlling $> 1/3$ of the stake-weighted votes) could execute stall attacks”; and finalizers “cannot unilaterally execute rollback attacks even though they can prevent rollback attacks”.

The chain has a load-bearing dependency on censorship-resistance of block production. A collusion controlling both $> 1/2$ hashpower and $> 2/3$ stake could violate finality and execute rollback and unavailability attacks; and “Censorship-resistance is necessary for stakers to stake, unstake, and redelegate, which means censorship-resistance is necessary for stakers to be able to perform their duty of holding the finalizers accountable. Therefore, if an attacker controls $> 1/2$ of the hashpower (and is thereby able to censor), then the attacker can prevent the stakers from holding the finalizers accountable” (ibid.; the file ends mid-list on the “ $> 2/3$ stake and $< 1/2$ hashpower” scenario — the document is incomplete). The reliance on stakers is so central that the zebra-crosslink design drops the tfl-book’s Stalled Mode entirely: “This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls” (`book/src/design/cl2-construction.md` at `6d02a1b8`).

Shielded Labs frames the substitution quantitatively as Penalty-for-Failure-to-Defend (PFD): penalties are assessed per property ($\text{PFD}_{\text{liveness}}$ versus $\text{PFD}_{\text{finality}}$); Tendermint-style slashing yields a “financial value gauge” PFD, while PoW — and Crosslink’s slashing-free PoS — yields a *rate* PFD, the loss of future revenue rate; the two types “cannot be directly compared” (`book/src/design/analysis/pffd.md` at `6d02a1b8`, a paste of GH issue #264). Redelegation-based punishment is thus a rate penalty standing in for the gauge penalty of slashing. For redelegation to work, stakers must be able to observe misbehaviour: Crosslink finality “is accountable because the chain contains explicit voting records which indicate each participating finalizer’s on-chain behavior” (`book/src/design/terminology.md` at `6d02a1b8`), and goal GH #214 asks that “Skilled users can easily collect records of finalizer behavior and performance” (`book/src/design/scoping.md` at `6d02a1b8`) — accountability is informational, feeding redelegation, not punitive. The terminology echoes the accountability line of the Ebb-and-Flow literature: the construction cites Neu-Tas-Tse [NTT2020, arXiv 2009.04987] as background (tfl-book, `src/design/crosslink/construction.md` at `fe6e1d6f`), and the same authors’ accountability-gadgets paper (arXiv 2105.06075, Financial Cryptography 2022) is the natural successor for this topic; NTT2020 remains at v3 (2021-02-04) as of 2026-07-15.

The backstop of the chain — the user-led emergency hard fork — has a drafted scope. The governance proposal limits legitimate emergency intervention to layer-1 security failures, explicitly including “situations in which would-be miners, finalizers, or stakers are being systematically censored or prevented from participating in block production, finalization, or staking”; it excludes “staking or unbonding parameters” changes from emergency scope; and it states that “Control over a large share of hashpower, finalization power, or stake does not itself constitute a violation” (`book/src/design/userled-emergency-hardforks.md` at `6d02a1b8`).

Classification. The absence of slashing is *designed but unspecified* — a dated, verbatim design commitment confirmed by the audit constraints. The accountability chain and PFD framing are design rationale at the same bin. The tfl-book’s evidence sketches (bft-final-vee, conflicting-vote proofs) are an *open problem*: whether future versions add evidence-based slashing, and what a ZIP must say about accountability records, is unresolved — recall that the tfl-book’s own delegation goal budgets for “slashing fees” (§4.1).

4.7 Finalizer keys

Definition 4.2 makes the finalizer verification key the sole on-chain reference to a finalizer, and makes position keys deliberately unlinkable to a user’s other activity. Everything else about finalizer key management is prototype fact or gap. The implementation uses single ed25519 keys per finalizer (ed25519-zebra; “ed25519 public key of the finalizer”), and votes are plain ed25519 signatures: the vote wire format is 76 bytes — a 32-byte ed25519 finalizer public key, a 32-byte BLAKE3 value hash (all-zero encoding Nil), an 8-byte height, and a 4-byte round whose most significant bit flags a commit — with a signed vote appending a 64-byte ed25519 signature over those 76 bytes (zebra-crosslink/src/lib.rs, MalVote, lines 1811–2052, and zebra-chain/src/block/header.rs, lines 109–133, at 6d02a1b8). No FROST or threshold-signing design for finalizer keys is sourced anywhere in the Crosslink corpus: a search for “frost” over tfl-book, crosslink-protocol-guide, crosslink-spec, and zebra-crosslink returns only upstream Zebra RedPallas/Orchard files unrelated to finalizers (negative grep, 2026-07-15). Threshold custody is established practice elsewhere in the Zcash stack — FROST per RFC 9591 in the PCZT signing flow (*Wallet Guide*, §“Partially created transactions (PCZT)”) — which makes its absence here a visible gap rather than an exotic ask.

The gap matters because of Definition 4.1’s caveat: honest validator keys must remain secret *indefinitely*, with secure deletion on rotation (tfl-book, src/design/crosslink/construction.md at fe6e1d6f, issue #140) — yet no key-rotation or revocation mechanism for finalizer verification keys in the roster is specified in any source. Bootstrap is equally unspecified: no source examined (tfl-book, implementation book, either blog post) defines eligibility or formation of the initial finalizer set. The prototype bootstraps ad hoc: the roster starts empty; keys are deterministically derived from the config field `insecure_user_name` (“temp seed for private/public key pair”); test instrumentation can force-include roster members (`TFInstr::ROSTER_FORCE_INCLUDE`); and a node adds itself with `voting_power 0` when absent from the roster it hands to the BFT engine (zebra-crosslink/src/lib.rs, zebra-crosslink/src/test_format.rs at 6d02a1b8).

Classification. The ed25519 key and vote formats are implementation-defined prototype facts. Key rotation, revocation, threshold custody, and initial-roster formation (including any minimum self-stake, and eligibility beyond top- K by stake weight) are *open problems* in every design source.

4.8 The prototype

The zebra-crosslink prototype (@ 6d02a1b8, 2026-04-21) implements a reduced, implementation-defined variant of the above; we record it as deployed-code fact about a prototype, not as design. The placeholder activation height is `TFL_ACTIVATION_HEIGHT = BlockHeight(2000)` (zebra-crosslink/src/lib.rs).

Roster mechanics. Staking state changes take effect only when their PoW block is finalized: function `update_roster_for_block()` runs while walking each newly finalized block and applies `StakingAction` commands from `Transaction::VCrosslink` transactions. The variants of `StakingActionKind` are `Add`, `Sub`, `Clear`, `Move`, and `MoveClear` — names diverging from the book’s `stake/redelegate/unbond/claim` — with fields including `insecure_target_name` and `insecure_source_name`, defined in the Shielded Labs `librustzcash` fork (git rev 33dc74bf; not present upstream) (zebra-crosslink/src/lib.rs, zebra-chain/src/transaction.rs,

Cargo.toml at 6d02a1b8). The BFT engine is Tenderlink — Shielded Labs “moved away from Malachite and now maintain our own lightweight version of Tendermint, called Tenderlink” (Updated Staking Design, 2025-10-27/11-05; dependency tenderlink 0.1.0, rev 0e885095) — and the roster handed to it is sorted by descending (stake, public key) — “Needs to uniquely & stably tie-break members with the same stake” — with cumulative stake tracked “to make weighted round-robin easy” for proposer selection (zebra-crosslink/src/lib.rs, Cargo.toml at 6d02a1b8).

Rewards. The prototype ignores the coinbase design entirely: a constant $R = \text{pos_total_reward} = 6000$ zats (“arbitrary scale”) is apportioned per newly finalized PoW block as $\text{reward}_i = \lfloor v_i \cdot R / \sum_j v_j \rfloor$ over finalizer voting powers v_i , with the integer-division dust going to the largest-stake finalizer, and the result added directly to `voting_power` — auto-compounding into stake weight rather than paid via coinbase. The code acknowledges its artifacts: “the compounding is per PoW block” and “finalizers added in the same PoS will get rewards for PoW blocks they couldn’t have actually voted on” (zebra-crosslink/src/lib.rs at 6d02a1b8).

Gaps against the draft rules. (1) No power-of-10 quantization check, no staking-epoch/staking-day/locked-phase gating, and no unbonding-delay enforcement exist anywhere in the staking path. (2) No top- $K = 100$ active-set truncation: every roster member with non-zero power participates. (3) Staking transactions are stubs: a `VCrosslink` transaction carrying a staking action bypasses the `has_inputs_and_outputs` check, so no value balancing against the chain value pools occurs. (4) The bonding lifecycle (unbond/claim) is not implemented; `Sub/Clear` remove weight immediately (zebra-consensus/src/transaction/check.rs, zebra-crosslink/src/lib.rs at 6d02a1b8; negative grep for `quantiz/unbond/epoch`, 2026-07-15).

Status. Crosslink remains in feature-net validation: the Incentivized Crosslink Feature Net launched with Milestone 5 (completed 2026-04-15; Season 1 from 2026-04-15; 500 ZEC allocated overall, 25 ZEC for Season 1; “Only cTAZ earned via block rewards counts”), following Milestone 4’s independent security review (concluded 2026-02-10). No NU8 activation date is stated (deployment-updates thread, fetched 2026-07-15).

Classification. Implementation-defined throughout, and self-describedly so; the book warns that the in-house BFT protocol’s “own security properties need to be more thoroughly verified” (book/src/design.md at 6d02a1b8). None of this prototype behaviour constrains the eventual ZIP.

4.9 What a specification must pin down

We close with the open problems, each traceable to a gap documented above.

1. Epoch parameterization: epoch length, staking-day length, and unbonding duration, in blocks (Table 2).
2. Redelegation latency: locked-phase exemption versus batched windows — the timing of the sole protocol-level punishment (§4.4).
3. The staking transaction format: how a revealed, quantized, “transparent” amount balances against Orchard value fields under protocol specification §4.17 (§4.3).
4. Quantization bucket range and minimum stake: “power of 10 ZEC (or ZAT)” (§4.3).
5. Commission: protocol-fixed $c = 0.1$ versus per-finalizer market rates (§4.5).
6. Issuance: the 40/40 split’s remaining 20%, pending-reward interaction with halvings and the 21M cap (§4.5).
7. Reward mechanics: coinbase distribution versus compounding, and the GH #158 goal against power-of-10 positions (§4.5); reward eligibility across the unbond–claim lifecycle and for stake on non-active finalizers.

8. Fees and multiplicity of staking actions per transaction (§4.3).
9. Voting-unit mapping and the per-epoch roster snapshot (§4.1).
10. Initial-roster formation, minimum self-stake, and finalizer eligibility beyond top- K (§4.7).
11. Finalizer key rotation and revocation under the indefinite-secrecy requirement (§4.7).
12. Accountability records, and whether any future version adopts evidence-based slashing (§4.6).
13. A formal model: nothing in the machine-checked corpus covers the roster, delegation, epochs, unbonding, or reward arithmetic (crosslink-spec @ 8edc3e94 models proposal validity only).

5 The wallet-visible contract

Crosslink changes what a Zcash node presents to everything above it. Today a wallet, an exchange, or the voting stack consumes one object — the node’s best chain — and layers its own confirmation policy on top (*Wallet Guide*, §“The proposal”). Under Crosslink 2 the node maintains two chains and a per-block finality status, and every consumer must decide which chain it reads and what it does when the two stop tracking each other. This section fixes that contract as the sources state it: the two ledger views and the theorems relating them (§5.1), finality as a queryable status (§5.2), the interface the prototype implements today (§5.3), and the four wallet-facing decisions no source resolves — anchor policy (§5.4), confirmation policy (§5.5), stall behaviour (§5.6), and the service-facing protocol (§5.7).

Two texts carry the design (pinned, with the rest of the corpus, in Table 1). The ECC Trailing Finality Layer book (Electric-Coin-Company/tfl-book @ fe6e1d6f, 2024-12-13; hereafter the *tfl-book*) defines the Crosslink 2 construction and has not changed since; its own status page calls the design “an early and incomplete protocol design proposal” (src/introduction/status-and-next-steps.md). The Shielded Labs design book inside the implementation repository (zebra-crosslink @ 6d02a1b8, 2026-04-21; hereafter the *SL book*) pastes the construction verbatim, declares divergences in admonitions, and self-describes as a provisional precursor to “a canonical Zcash Improvement Proposal submission” (book/src/design/nutshell.md). No such ZIP exists: a repository-wide search of zcash/zips @ 69610984 (2026-07-09) finds no Crosslink deployment artifact and no NU8 artifact of any kind (cf. *Tachyon Guide*, §“Relation to Crosslink and the upgrade environment”). Shielded Labs’ productionization phase runs Q2 2026–Q2 2027 with no announced mainnet activation (shielded-labs.net/roadmap and forum thread t/49706, fetched 2026-07-15). Nothing in this section is therefore *specified*; claims are *designed but unspecified* at best, prototype code is cited as code, and the open problems are named as such.

5.1 Two ledger views

We write $\leq_{bc}$ for the prefix order on bc-valid chains, $B \sim_{bc} C$ when one of B, C is a prefix of the other, and $C|k$ for chain C with its last k blocks removed. The bc-best-chain of node i at time t is ch_i^t , notation the tfl-book adopts from the ebb-and-flow model of Neu, Tas, and Tse (arXiv 2009.04987v3, 2021-02-04, still the latest revision as of 2026-07-15; tfl-book construction.md:5).

Definition 5.1 (Ledger views of Crosslink 2). Fix the bc-confirmation-depth $\sigma \in \mathbb{N}^+$. At every time t , each node i derives two node-local chains from its bc-best-chain and its BFT context:

- (i) the *finalized chain* $localfin_i^t$: a bc-valid chain obtained at the fixed confirmation depth σ via the finalization update rule (Construction 3.17, applied to the candidate of Definition 3.11);

(ii) the *bounded-available chain*, for a per-node depth μ with $0 < \mu \leq \sigma$:

$$(\text{localba}_\mu)_i^t = \begin{cases} \text{ch}_i^{t|\mu} & \text{if } \text{localfin}_i^t \leq_{\text{bc}} \text{ch}_i^{t|\mu}, \\ \text{localfin}_i^t & \text{otherwise.} \end{cases}$$

(tfl-book src/design/crosslink/construction.md:391–399, 520–527.)

Both views are plain block chains. Snap-and-Chat and Crosslink 1 instead exposed *sanitized transaction logs* — flattened sequences with invalid transactions filtered out — and the tfl-book drops that machinery “because we do not need sanitization, there is no need to express it as a log of transactions rather than blocks” (construction.md:393); §5.4 returns to why this matters to wallets. The otherwise-case of the definition covers post-reorg difficulty situations in which a new best chain outscores the old one in fewer than σ blocks, and it makes the following theorem trivial (construction.md:547–555).

Theorem 5.2 (Ledger prefix property). *For any node i that is honest at time t , and any confirmation depth μ : $\text{localfin}_i^t \leq_{\text{bc}} (\text{localba}_\mu)_i^t$.*

Proof. By construction of $(\text{localba}_\mu)_i^t$: the otherwise-branch returns localfin_i^t itself (tfl-book construction.md:531–536). $\square$

The depth μ is the node’s own choice; the default “should be” $\mu = \sigma$, and choosing $\mu < \sigma$ “is at the node’s own risk and may increase the risk of rollback attacks against $(\text{localba}_\mu)_i^t$ (it does not affect localfin_i^t)” (construction.md:395–399, Security caveat admonition).

Monotonicity and hazards. The finalized view never moves backward: the update rule advances localfin_i to the new candidate only when that candidate descends from the current value, which yields *Local finalization linearity* — the time series $\text{localfin}_i^{r \leq t}$ is bc-linear. If the candidate instead conflicts, the node keeps its old value and “should record a finalization safety hazard”, which “can only happen if global safety assumptions are violated”; the record should include ch_i^t and the history of localfin updates (construction.md:462–488). Restricted to a single node, Assured Finality (Definition 5.3) is equivalent to this linearity; across nodes, linearity is necessary but not sufficient (construction.md:466–473, 508).

The syncing gate. Neither view is exposed to a client before the node has synced: “this chain state should not be exposed to clients of the node until it has synced”, where synced means a baked-in checkpoint bft-block $B_{\text{checkpoint}}$ satisfies $B_{\text{checkpoint}} \leq_{\text{bft}} \text{LF}(\text{ch}_i^t)$, $\text{snapshot}(B_{\text{checkpoint}}) \leq_{\text{bc}} \text{localfin}_i^t$, and the timestamp of localfin_i^t is within a threshold of the current time (construction.md:464, 529, 557–562).

Latency, and which view an application reads. In the steady state the finalized view trails the tip by about $\mu+1+\sigma$ bc-blocks — at most “slightly more than a factor of 2” over Snap-and-Chat’s σ_{sac} when $\mu = \sigma = \sigma_{\text{sac}}$ (tfl-book security-analysis.md:265–269; questions.md:27–31). The book states the application-facing choice explicitly: since localfin lags localba_σ by only a few blocks outside Stalled Mode, “the main factor influencing the choice of a given application to use localfin_i^t or $(\text{localba}_\mu)_i^t$ is not the average latency, but rather the desired behaviour in the case of a finalization stall: i.e. stall immediately, or keep processing user transactions until L blocks have passed” (construction.md:415). Stalls are §5.6.

Parameters. Production values of σ and of the finalization gap bound L are unchosen. The tfl-book requires L “significantly greater than σ ” and “at least 2σ ” in practice (construction.md:391, 405); the prototype ships $\{\sigma = 3, L = 7\}$ with the caveat “No verification has been done on the security or performance of these parameters” (zebra-crosslink/src/chain.rs:206–222). Wall-clock statements are additionally provisional: block target spacing is 75 s today, and ZIP 218 (Status: Draft) proposes 25 s at NU7, describing itself as “complementary to ... finality mechanisms such as Crosslink” (zips/zip-0218.md:3, 76–78).

Classification. *Designed but unspecified* throughout: Definition 5.1 and Theorem 5.2 come from a design book at pre-ZIP rigor, and the constants are prototype code, not consensus.

5.2 Finality as a status

The tfl-book’s term of art is *assured finality*: a protocol property that assures transactions cannot be reverted *by that protocol*. It eschews “absolute finality” because “it is not feasible for any protocol to prevent reversing final transactions ‘out of band’”; “final” in the book always means assured finality, in contrast to proof-of-work’s probabilistic finality (tfl-book src/terminology.md:11–13, 19), whose exact arithmetic — and its divergence as the adversary nears half the work — is the *Consensus Guide*’s subject (§“What the numbers say”).

Definition 5.3 (Assured Finality). An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u : $\text{localfin}_i^t \sim_{bc} \text{localfin}_j^u$ (tfl-book construction.md:501–506).

Theorem 5.4 (Two-sided safety). *An execution of Crosslink 2 has Assured Finality if either of the following holds:*

- (i) *the Π_{bc} subprotocol has Prefix Agreement at confirmation depth σ — honest best-chains, truncated by σ , are pairwise compatible across all nodes and times; or*
- (ii) *the Π_{bft} subprotocol has Final Agreement — the last-final blocks of any two bft-valid blocks in honest view are pairwise compatible.*

(tfl-book security-analysis.md:72–77, 86–91; hypothesis definitions at construction.md:337–340, 203–206.)

Assured Finality therefore survives the failure of either subprotocol’s safety assumption alone; it fails only under the weaker of the two, which is the keystone guarantee a wallet’s “final” badge would rest on. We defer the proof of branch (i), which runs through the Local fin-depth lemma and the Linear prefix lemma, to the security analysis (§6.1); Prefix Agreement and Final Agreement are stated in full as Definitions 3.6 and 3.9.

One caveat bounds what is actually proved. Below the marker “\$TODO\$ Not updated for Crosslink 2” the tfl-book’s safety chapter still argues in the Snap-and-Chat sanitized-log formalism (LOG_{fin} , LOG_{ba} , san-ctx), and the LOG_{ba} agreement proof trails off unfinished (“What we want to show is that, under some conditions on executions, ...”) (security-analysis.md:54, 137–160, 253–257); the SL book copies this state verbatim. The bin split for this section: Theorem 5.4 is completely stated, but only branch (i) carries a correct written proof; branch (ii)’s pasted proof is defective (Remark 6.3); transaction-log-level safety for the bounded-available view is *designed but unspecified* with an incomplete proof.

The Shielded Labs definition. The SL book defines “crosslink finality” by five properties: *accountable* (on-chain voting records), *irreversible within protocol scope* (undoing requires software modification or human intervention), *objectively verifiable* (determinable from local block history alone), *globally consistent* (all sufficiently-synced nodes agree on finality status), and an *asymmetric cost-of-attack defense* that is a literal TODO in the source (book/src/design/terminology.md, “Crosslink Finality”). Its ledger-state framing makes finality the sole “height-eventual Ledger State”: a property of height H computable only from a later attesting block, on which all observers of that attesting block agree “even though they may observe the attesting block rollback. In the event of such a rollback, the protocol guarantees that an alternative block will eventually attest to the finality status of the same candidate block”; the General Ledger State Design Invariant requires all ledger state, finality status and the proof-of-stake roster included, to be computable entirely from PoW blocks and transactions (book/src/design/nutshell.md:106–128). For light clients the SL book adds a negative constraint: its design drops the tfl-book’s “outer signature”, so “a direct hash or copy of the most recent BFT finality certificate is insufficient evidence” that a given bitwise copy is or will become canonical (book/src/design/cl2-construction.md:3–16) — any wire-format finality proof must be designed around that.

Classification. Definition 5.3 and Theorem 5.4 are *designed but unspecified* (complete statements and proofs, pre-ZIP rigor); the SL five-property definition is *designed but unspecified* with its fifth property an acknowledged gap; ledger-level localba safety is an *open problem* in the sources’ own accounting.

5.3 The interface implemented today

The prototype (zebra-crosslink @ 6d02a1b8) exposes finality through a typed internal service and a JSON-RPC surface. The internal state service defines

```
enum TFLBlockFinality { NotYetFinalized, Finalized, CantBeFinalized }
```

with request variants

```
IsTFLActivated, FinalBlockHeightHash, FinalBlockRx, SetFinalBlockHash,
BlockFinalityStatus(height, hash), TxFinalityStatus(txid), Roster,
FatPointerToBFTChainTip, StakingCmd
```

(zebra-state/src/crosslink.rs:11–49; variant FinalBlockRx is a broadcast receiver). The RPC methods, six of the ten doc-commented as “Placeholder” (zebra-rpc/src/methods.rs:246–420), are:

- `is_tfl_activated`;
- `get_tfl_roster_zec` and `get_tfl_roster_zats`;
- `get_tfl_fat_pointer_to_bft_chain_tip`;
- `get_tfl_final_block_hash` and `get_tfl_final_block_height_and_hash`;
- `get_tfl_block_finality_from_hash` and `get_tfl_tx_finality_from_hash`;
- `set_tfl_finality_by_hash`;
- `subscribe_tfl_new_final_block_hash`.

The docs concede that the final-block RPCs “for the sake of testing ... currently treat pre-reorg block as final”.

Block-level finality is computed as a tri-state: a height above the final height returns `NotYetFinalized` (“N.B. this may be invalidated by the time it is received”); otherwise the status is `Finalized` iff the best-chain hash at that height equals the queried hash, else `CantBeFinalized`

(zebra-crosslink/src/lib.rs:1327–1374). This matches the SL book’s globally-consistent claim only below the final height, as it must: above it, the answer is a moving target.

Two behaviours contradict the SL book’s own definition of crosslink finality, and they sit exactly on the surface a wallet would consume:

- (i) Transaction finality omits the fork check. Request `TxFinalityStatus` returns `Finalized` iff `tx.height ≤ final_height`, with a literal `// TODO: CantBeFinalized` — it never checks best-chain membership of the containing block, so a transaction on a losing fork below the final height is reported `Finalized` (zebra-crosslink/src/lib.rs:1412–1431). A blocking wait built on `FinalBlockRx` loops until some status other than `NotYetFinalized` is returned (zebra-rpc/src/methods.rs:2150–2195).
- (ii) Method `set_tfl_finality_by_hash` is intended to be regtest-only but currently ships `regtest_override = true` (zebra-rpc/src/methods.rs:1975–2000).

Both are prototype-status defects rather than design statements, but a finality status that is neither objectively verifiable nor globally consistent is the current observable behaviour.

Before activation: the fallback depth. Whenever no BFT-final block is known (including before TFL activation), the implementation falls back to the block-locator’s last hash as its final block: function `tfl_reorg_final_block_height_hash` yields, in effect, the block at depth `MAX_BLOCK_REORG_HEIGHT`, which the fork pins at 99, one less than the transparent coinbase maturity of 100; an internal `latest_final_block`, once set by BFT, takes precedence (zebra-crosslink/src/lib.rs:235–332; zebra-state/src/constants.rs:14–31). Upstream Zebra meanwhile moved the same constant to 1000 (commit 02f9648a5, 2026-06-15; relocated to zebra-chain by 8767a91ba), documenting it as “a local-only node policy; it is not part of consensus” sized “as a defence-in-depth measure against sustained consensus splits” (zebra @ 9f3166b9, zebra-chain/src/parameters/constants.rs:20–30). The lesson for prose about pre-Crosslink “de facto finality”: the rollback depth is node policy, not consensus, and it currently differs by a factor of ten between upstream and the very fork implementing Crosslink. The *Ironwood Guide*’s anchor-depth discussion (§“Proof of ownership of *some* valid note: the commitment tree”) states the same policy-not-limit framing, and the *Consensus Guide* owns the full three-way divergence register — specification, retired zcashd, deployed zebra — in §“Reorg rails, maturity, and the finality floor”.

Classification. *Specified as code* at prototype rigor only — every RPC is a self-described placeholder, and nothing in this subsection is consensus.

5.4 Anchor semantics: the undecided keystone

Crosslink 2 leaves transaction validity alone: “The consensus rules applying to bc-transactions are entirely unchanged, including any rules that depend on bc-block height or previous bc-blocks”, because the construction never reorders or sanitizes transactions; contextual validity in Π_{bc} is that of Π_{origbc} modulo the non-contextual new block rules (tfl-book construction.md:267, 701–703). The Orchard anchor rule is therefore inherited as-is: a spend may reference the treestate of *any* main-chain block from Orchard activation onward, with no sliding window (*Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”, paragraph “Anchor choice as the tree grows”; mainnet floor 1,687,104) — where “main chain” now means the localba history. No source restricts spend anchors to the finalized prefix: not the tfl-book, not the SL book, not the implementation (grep over zebra-crosslink @ 6d02a1b8: no anchor-rule changes), and not the ZIP corpus (@ 69610984).

The wallet-level anchor policy under two ledgers is thus the single most consequential undecided item in the contract. A future ZIP must decide two things: (a) whether consensus adds any anchor restriction — nothing is proposed anywhere — and (b) what a wallet’s default anchor and note-selection policy becomes relative to the finalized height. The branch costs are concrete:

- *Anchor only in localfin.* Every spend inherits finalization latency — about $\mu + 1 + \sigma$ blocks in the steady state (§5.1), up to L during a stall — and recently received notes cannot be spent until their treestate finalizes. In exchange, anchor invalidation becomes impossible whenever Assured Finality holds, eliminating the surviving-reorg caveat the *Wallet Guide* records (§“The proposal”: a rollback past the anchor invalidates the transaction while its revealed nullifiers link any retry).
- *Keep the tip-relative anchor.* ZIP 315’s rule — “if the current block is at height H , the anchor SHOULD reflect the final treestate of the block at height $H-3$ ” (zips/zip-0315.rst:586–587) — preserves latency and the anonymity-set size and uniformity that motivate near-tip anchoring (*Ironwood Guide*, *ibid.*), and retains the reorg caveat for the non-final suffix.

The bounded-availability argument already couples anchors to guaranteed balances: a party’s guaranteed minimum balance is “the minimum balance taken over all possible transaction histories that extend the finalized chain”, accounting for republication of its transactions without consent; “we know that shielded transactions cannot be reapplied after their anchors have been invalidated. It may be desirable to further constrain re-use in order to make guaranteed minimum balances easier to compute” (tfl-book the-arguments-for-bounded-availability-and-finality-overrides.md:50). That sentence is the closest any source comes to proposing an anchor restriction, and it proposes nothing concrete.

Why plain chains matter here: under Snap-and-Chat sanitization, expiry preserved the idempotence argument (an expired transaction stays expired on recheck), but anchors broke it — “it is technically possible for a duplicate transaction that was invalid because of a missing anchor to *become* valid in a subsequent block ... the same commitment treestate can be produced by two unrelated transactions” — and some finalized outputs could become unspendable outright, transactions in LOG_{fin} stranded on a fork no longer in any best chain (tfl-book notes-on-snap-and-chat.md:67–70, 125 and “Spending finalized outputs”). Crosslink 2 fixes both by construction, via the Linearity and Last Final Snapshot rules, and avoiding sanitization “would avoid breaking assumptions that may be made by existing Zcash node implementations and other consumers of the Zcash block chain” (security-analysis.md:263; potential-changes.md:348–363). The wallet-visible consequence: under Crosslink 2, exactly the anchor and expiry semantics documented in the *Ironwood Guide* and *Wallet Guide* carry over — on each of the two chains separately.

Classification. The inherited anchor rule is *designed but unspecified* (construction text, both books). The anchor policy under two ledgers — both the consensus question and the wallet default — is an *open problem*, and this volume flags the branch-cost analysis above as its own synthesis.

5.5 Confirmation policy under two ledgers

The deployed baseline is ZIP 315 (Status: Draft), which the *Wallet Guide* develops in §“The proposal”: a ConfirmationsPolicy with trusted depth 3, untrusted depth 10, zero-conf shielding allowed by default, an advisory floor of 1, and the $H - 3$ anchor rule quoted above. ZIP 315 predates Crosslink and never mentions it.

How that policy maps onto a finality status is unresolved everywhere. The candidate shapes: the trusted/untrusted distinction collapses to final/not-final; it becomes a three-state policy (final / trusted-unfinalized / untrusted-unfinalized); or it stays unchanged with finality surfaced only in UX. Shielded Labs commits to exposing the status — “augmentation of existing RPC APIs for blocks and transactions to include finality status” (book/src/design/deliverables.md, GH #105) — but no mapping into any confirmations policy exists in any source. A related open point is whether displayed confirmation counts should saturate at “final”: the SL book argues that when users’ rollback thresholds diverge during a stall, “those users who require finality and those users who do not ... diverge in their expectations and behavior, which is a growing economic and governance risk”

(book/src/design/security-properties.md:17–19) — finality removes that fragmentation only if wallets adopt it uniformly.

The ECC design goals pull in two directions at once: wallet developers “should not be required to make any changes through protocol transitions as long as they rely solely on the lightwalletd protocol or a full node API”; yet “the Crosslink construction may require wallets to alter their context of valid transactions differently from the current NU5/NU6 consensus protocol”; and there must be “no security or safety degradation due to wallet user behavior ... assuming users follow their current behaviors unchanged” (tfl-book src/design/goals.md:9, 15–16, 28). The first goal is untestable today: no lightwalletd or zaino wire protocol for finality status exists in any source.

Classification. *Open problem* in every direction that matters to a wallet author; the only *specified*-adjacent artifact is ZIP 315 itself, a Draft that predates the construction.

5.6 Stalls: the divergence that decides wallet behaviour

The finality gap at time t is, roughly, the number of bc-blocks not yet finalized; when roughly constant it corresponds to the time a transaction takes to finalize after inclusion. A gap exceeding L “likely signals an exceptional or emergency condition, in which it is undesirable to keep accepting user transactions that spend funds” (tfl-book construction.md:403–405). The construction enforces this as a block-validity rule: with $\text{finality-depth}(H) := \text{height}(H) - \text{height}(\text{snapshot}(\text{LF}(H)))$, every bc-block must satisfy $\text{finality-depth}(H) \leq L$ or be a stalled block (construction.md:680–693).

Stalled Mode, as ECC designed it. The proposed Zcash instantiation of is-stalled-block is coinbase-only blocks — “Since funds cannot be spent in coinbase-only blocks, the vast majority of attacks that we are worried about would not be exploitable” — possibly with shielded coinbase disabled; mining pools cannot distribute rewards during the stall (a rug-pull risk the book notes); and Stalled Mode “can only be entered by consensus of a larger validator set, or if there is an availability failure of the finalization protocol” (tfl-book the-arguments-for-bounded-availability-and-finality-overrides.md:24, 103–111, 187).

The divergence. The SL book pastes the construction verbatim but disclaims exactly this rule: “The zebra-crosslink design in this book diverges from this text by *not* including ‘Stalled Mode’ rules. This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls” (book/src/design/cl2-construction.md:9–12). No stalled-mode or coinbase-only enforcement exists anywhere in the implementation (grep over zebra-crosslink @ 6d02a1b8 sources). The paste disclaimer adds that Daira-Emma Hopwood and Jack Grigg “have not reviewed and do not necessarily endorse its content or design changes relative to Crosslink 2”; apart from the admonitions the construction text is byte-identical to the tfl-book’s.

Remark 5.5 (Expiry as stall cleanup — synthesis). Neither book discusses the interaction with transaction expiry; the following is this volume’s synthesis. Under the ECC branch a stall is a hard availability stop with automatic cleanup: coinbase-only blocks keep heights advancing, so every pending spend reaches its expiry height — target plus `DEFAULT_TX_EXPIRY_DELTA` = 40 blocks (ZIP 203) — within about 40 blocks of the stall, and the wallet’s locked notes release under the unexpired-spend condition (*Wallet Guide*, §“Expiry (ZIP-203)” and §“Expiry and fund availability”). Under the SL branch there is no such cleanup: spends keep confirming into an unbounded finalization gap — precisely the condition the ECC arguments chapter deems unacceptable — and a service’s `NotYetFinalized` exposure grows without bound. Whether a Crosslink ZIP should couple the expiry delta to L (for instance $\Delta < L$, or Δ on the order of the finality latency) is unaddressed in any source.

Overrides. Long rollbacks are argued to need a formalized, governance-specified procedure, with the Bitcoin 2010 value-overflow rollback (53 blocks, about 9 hours) and the Ethereum DAO fork as precedents; bounded availability is complementary because it bounds the period of spend transactions an intentional rollback could affect, and the envisioned procedures preserve coinbase-only mining rewards across the rollback (tfl-book the-arguments...md:29, 148–198). On the governance side, the SL draft ZIP on user-led emergency hardforks “explicitly rejects the use of an emergency user-led hard fork to mitigate harm arising from application-layer failures” — not wallet bugs, bridge exploits, custodial failures, nor DAO-style reversals (book/src/design/userled-emergency-hardforks.md:13) — bounding what the escape hatch may be used for.

Classification. The Finality depth rule is *designed but unspecified* (tfl-book normative text); its omission is likewise *designed but unspecified* (SL book admonition plus absent code). The conflict itself is a finding: a ZIP must pick one branch, and each branch changes what wallets and exchanges must handle during a stall — a hard availability stop with expiry churn, or indefinitely growing unfinalized exposure. Remark 5.5 is synthesis.

5.7 Exchanges, bridges, and downstream volumes

The stated service-facing goal is uniformity: “CEXes and decentralized services like DEXes/bridges are willing to rely on Zcash transaction finality. E.g. Coinbase reduces its required confirmations to no longer than Crosslink finality. All or most services rely on the same canonical (protocol-provided) finality instead of enforcing their own additional delays or conditions, so users get predictable transaction finality across services” (book/src/design/scoping.md:10, 30; GH #131, #128). The committed interface deliverables are finality status on block and transaction RPCs (GH #105), proof-of-stake RPCs for finalizer identities, staking weights, and delegation info (GH #104), and “contingency mode, finality traversal” RPCs (GH #172); external validation partnerships prioritize wallet developers, finalizers, and exchanges (book/src/design/deliverables.md).

The risk model a service would price is the SL five-component allocation: miners “cannot unilaterally violate finality, even if an attacker controls $> 1/2$ of the hashpower”; finalizers cannot unilaterally compromise liveness or censorship-resistance even with $> 2/3$ of stake-weighted votes, and cannot execute rollbacks though they can prevent them; more than $1/3$ of stake can stall; and a collusion of $> 1/2$ hashpower with $> 2/3$ stake can violate finality, execute rollbacks, and mount unavailability attacks (book/src/design/five-component-model.md:24–46). The SL book separates Finality Progress from Liveness precisely because the anticipated failure is a stall under continued block production, and part of the design intent is to “convert censorship attacks into stall attacks” (book/src/design/security-properties.md:17–19, 35).

What remains open on this surface: deposit semantics during a finalization stall; whether `CantBeFinalized` can ever surface for a transaction a service already credited; the contingency-mode and finality-traversal RPC semantics (GH #172 is open); and the light-client wire protocol of §5.5. Each is an *open problem*: the slogan is designed, the contract behind it is not.

Downstream volumes. For coin-voting, Crosslink finality bears on the snapshot pin: a round pins (`snapshot_height`, `snapshot_blockhash`) into its round identifier (*Voting Guide*, §“The five-message round”), and requiring the pinned height to be at or below the finalized height would make the pin irreversible under Assured Finality; the orthogonal trust gap — snapshot roots “posted, never verified” (*Voting Guide*, §“The snapshot roots: posted, never verified”) — is untouched by finality and remains open. This resolution analysis is this volume’s synthesis. Project Tachyon is formally independent of Crosslink; the *Tachyon Guide* (§“Relation to Crosslink and the upgrade environment”) records the independence as asserted rather than examined, and the NU8 vacuum it documents is unchanged at `zcash/zips @ 69610984` (2026-07-09).

Classification. The service goals and deliverables are *designed but unspecified* (scoping and deliverables documents with open issue numbers); the five-component allocations are *designed but unspecified* security claims pending the volume’s security analysis; every concrete service-facing behaviour during a stall is an *open problem*.

6 Security posture and open problems

The security posture of Crosslink 2 is best stated as an inventory of four artifacts of unequal maturity. The design book states its central safety theorem twice over, from two independent assumptions — though the second statement’s pasted proof is defective (Remark 6.3) — and then breaks off: everything below a stated line of its analysis is marked “Not updated for Crosslink 2”, and its last proof sketch ends mid-sentence (tfl-book @ fe6e1d6, 2024-12-13, src/design/crosslink/security-analysis.md, lines 54 and 253–257). A formal model machine-checks exactly one validity predicate of the BFT side and nothing else (crosslink-spec @ 8edc3e9, 2025-07-23). The implementation departs from the analyzed design in two load-bearing ways that it documents itself (zebra-crosslink @ 6d02a1b, 2026-04-21). An independent mechanism-design review finds the construction’s safety “structurally strong once finality is reached” and its finality progress “economically fragile in the baseline design” (nikete.com/crosslink-zebra-audit, completion announced 2026-02-05). We take the four in turn, keep the bin of every claim visible — *specified* here can only mean verifiable source at a pinned commit, since no Crosslink ZIP exists (zips @ 6961098, 2026-07-09) — and close with the open-problem ledger and its revisit triggers. The rules and properties named below are stated in §3.4, §3.5, and §3.6; the source pins are those of Table 1.

6.1 The safety theorems and their assumptions

The book’s central safety goal is *Assured Finality* (Definition 3.20): in a given execution, for all times t, u and all nodes i, j such that i is honest at time t and j is honest at time u , the finalized views agree, $\text{fin}_i^t \sim_{\text{bc}} \text{fin}_j^u$ — either is a prefix of the other (tfl-book @ fe6e1d6, construction.md, lines 504–506). The name is chosen against “absolute finality”: the property binds *the protocol*, and the book states explicitly that out-of-band reversal by community fork remains possible (terminology.md, lines 11–13). The implementation’s gloss, “crosslink finality”, strengthens the wording: accountable via explicit on-chain voting records, irreversible within protocol scope, objectively verifiable from local block history, globally consistent (zebra-crosslink @ 6d02a1b, book/src/design/terminology.md, lines 7–13). Two theorems each imply Assured Finality from a safety property of one subprotocol alone.

Theorem 6.1 (Prefix Agreement implies Assured Finality). *In an execution of Crosslink 2 in which subprotocol Π_{bc} has Prefix Agreement (Definition 3.6) at confirmation depth σ — all honest nodes’ bc-best-chains, truncated by σ blocks, pairwise agree across all times — that execution has Assured Finality (tfl-book @ fe6e1d6, security-analysis.md, lines 72–77; Prefix Agreement at construction.md, lines 96–99 and 337–340).*

The written proof is complete and short: the Local fin-depth lemma places each fin_i^t inside some earlier σ -truncated best chain of node i (construction.md, lines 491–494), Prefix Agreement relates any two such chains, and the Linear prefix lemma — two prefixes of a common chain agree — closes the argument (construction.md, lines 77–82). The book contrasts this with the corresponding result for Snap-and-Chat: “[NTT2020, Theorem 2] is a much more complicated proof that takes nearly 3 pages, not counting the reliance on results from [PS2017]” (security-analysis.md, line 162). The comparison is fair in kind but not in coverage: NTT2020’s Theorem 2 is a probabilistic security statement about the longest-chain protocol itself (failure probability $e^{-\Omega(\sqrt{\sigma})}$ for block-production rate $p < (n - 2f)/(2\Delta n(n - f))$; arXiv:2009.04987v3, Appendix C), whereas Theorem 6.1 *assumes* the corresponding property of Π_{bc} as a black box. That division of labor is this series’ scope rule in protocol

form: Crosslink consumes k -deep-confirmation guarantees of the PoW layer; it does not re-derive them.

Theorem 6.2 (Final Agreement implies Assured Finality). *In an execution of Crosslink 2 in which subprotocol Π_{bft} has Final Agreement (Definition 3.9) — for all bft-valid blocks C, C' in honest view at any times, $\text{bft-last-final}(C) \sim_{\text{bft}} \text{bft-last-final}(C')$ — that execution has Assured Finality (tfl-book @ fe6e1d6, security-analysis.md, lines 86–91; Final Agreement at construction.md, lines 203–206).*

Remark 6.3 (The written proof of Theorem 6.2 is defective). The proof text pasted under the theorem in the book is a verbatim duplicate of the proof of Theorem 6.1: it invokes Prefix Agreement of Π_{bc} , not Final Agreement of Π_{bft} (security-analysis.md, lines 86–91). The intended route presumably runs through the Linearity rule, as sketched at construction.md, line 592, but no correct written proof exists in the sources we have read. Bin: *open problem* — the theorem statement is the design’s load-bearing claim for the case of a subverted PoW layer, and it currently has no proof.

The pair of theorems, where both hold, gives the design’s headline property: safety of the finalized ledger requires only *one* of $\{\Pi_{\text{bc}}$ safety, Π_{bft} safety $\}$ — the stronger of the two. The book carries the same two-route structure down to the ledger level: in an execution where Π_{bc} has Prefix Agreement at depth σ or Π_{bft} has Final Agreement, any two honest nodes’ finalized ledgers LOG_{fin} agree at all times (security-analysis.md, lines 117–176). That section, however, sits below the “Not updated for Crosslink 2” line (line 54), so its statements are for Crosslink 1 and their transfer is unverified.

Two further results are proved, both assuming the *one-third bound on non-honest voting* (Definition 3.7): strictly fewer than one third of each epoch’s voting units are ever cast non-honestly, counted over the entire execution (construction.md, lines 131–141). First, uniqueness: any bft-valid block for a given epoch in honest view commits to the same proposal, by quorum intersection of two two-thirds vote sets, adapted from [CS2020, Lemma 1] (security-analysis.md, lines 208–213). Second, snapshot validity: with Prefix Consistency of Π_{bc} at depth σ added, every bc-chain snapshot contributing to LOG_{fin} eventually sits in every honest node’s bc-best-chain (security-analysis.md, lines 232–251).

Assumption transfer. The theorems assume properties of the *modified* subprotocols Π_{bc} and Π_{bft} , while the underlying literature supports the *original* protocols Π_{origbc} and Π_{origbft} . In the BFT direction the transfer is argued: Π_{bft} only adds structural components and tightens validity rules, so every Π_{bft} execution maps to a Π_{origbft} execution with the additions erased. In the PoW direction it is acknowledged as unproven: the modifications “seem unlikely to have any significant effect on this property”, followed by “TODO: that is a handwave; we should be able to do better, as we do for Π_{bft} below” (security-analysis.md, lines 119 and 170). Bin: *open problem*.

The residual attack the design accepts. If the network is partitioned and Π_{bft} is subverted by more than one third of validators, the adversary can vote with an additional one third on each side of the fork, so that each side’s last-final bft-block is consistent with its own fork. The book calls this unavoidable: “ Π_{bc} doesn’t include the mechanisms needed to maintain finality under partition; it takes a different position on the CAP trilemma.” Snap-and-Chat, by contrast, loses safety under BFT subversion even *without* a partition; Crosslink’s stated aim is to limit safety loss to attacks “like” the unavoidable one (security-analysis.md, lines 109–115). The corresponding goal exceeds NTT2020’s own argument: the paper’s Figure-9c reasoning — one honest vote is needed to finalize a snapshot, hence only valid snapshots finalize — applies only for adversarial fractions $n/3 < f < n/2$, whereas the book wants the conclusion to survive an adversary holding 100% of validators but under 50% of Π_{ic} hash rate (notes-on-snap-and-chat.md, lines 205–225).

Classification. Every statement above is *designed but unspecified*: the proofs live in a design book at a pinned commit, not in a ZIP, a peer-reviewed paper, or machine-checked form. Within that bin, Theorem 6.1 is complete on its stated assumptions; Theorem 6.2 lacks a correct written proof (Remark 6.3); the ledger-level and snapshot-validity theorems predate Crosslink 2; and the $\Pi_{\text{origbc}} \rightarrow \Pi_{\text{bc}}$ assumption transfer is an acknowledged gap.

6.2 Liveness and the unfinished analysis

The liveness side of the analysis is prose, and the book says so. Two conclusions are argued but not proven: subprotocol Π_{bc} remains live under the same conditions as Π_{origbc} , because the added consensus rules never obstruct producing a coinbase-only stalled block on the current best chain; and Π_{bft} remains live under the same conditions as Π_{origbft} , because the Linearity and Tail Confirmation rules always permit an honest proposer to re-propose its parent’s headers_{bc} (tfl-book @ fe6e1d6, security-analysis.md, lines 25–42). Two open TODOs sit inside the argument: that a new higher-scoring snapshot eventually becomes final (“make that more precise”), and that Stalled Mode triggers only when the BFT liveness assumptions are violated — “more detailed argument needed, especially for the dependence on L ”, where the dependence involves σ , network delay, honest hash rate, and honest proposers and voting units (lines 46 and 50). The quantitative relationship among these parameters was never worked out anywhere in the sources we have read; the book’s design guidance is only that L be “significantly greater than σ ” and “in practice ...at least 2σ ” (construction.md, lines 391 and 405). Bin: *open problem*.

Three further admissions bound what the analysis currently delivers. First, the rollback exposure of the available ledger: the book proves that LOG_{ba} never rolls back past the agreed LOG_{fin} , but notes the result is weaker than desired, since rollbacks of LOG_{ba} down to the finalization point “may be of unbounded length” — bounding them by L is asserted loosely and “we have also not proven that so far” (security-analysis.md, lines 180–190). Second, rule necessity: the claim that every Crosslink 2 rule is individually necessary is qualified — “some rules, e.g. the Linearity rule, only contribute heuristically to security in the analysis so far” (lines 275–281). Third, validation cost: honestly voting on a proposal may recursively require validating the referenced bft-chain, each bft-block’s headers_{bc} chain, and so on — the book’s own gloss is “Wait what, how much validation is that?” — with denial-of-service bounded only by validation ordering (check the supermajority and PoW first) and by the Linearity rule limiting live bc-chains per bft-chain to one (construction.md, lines 641–652).

Remark 6.4 (Fork-choice independence). Crosslink 2 deliberately does not modify the fork-choice rule of Π_{bc} , and the book identifies this as the reason the bidirectional-dependence liveness failure of Casper-FFG-style designs (NTT2020, Appendix E, “Bouncing Attack on Casper FFG”) does not arise: an honest bc-block-producer “must not use information from the BFT protocol, other than the specified consensus rules, to decide which bc-valid-chain to follow” (security-analysis.md, lines 15–23; construction.md, lines 715–717). The same modularity is why requiring a node’s bc-best-chain to extend $\text{snapshot}(B)$, for B the last final bft-block in that node’s view, was rejected as a fork-choice constraint: under a potentially subverted Π_{bft} , “we cannot say anything useful about $\text{snapshot}(B)$ ”, so the constraint would break the modular safety and liveness arguments. Honest validators may instead simply refuse to vote for proposals embedding long rollbacks — the voting rule is “only if”, not “iff” — at an accepted cost in BFT liveness (“and that’s okay”; questions.md, lines 44–58).

Remark 6.5 (The model is not NTT2020’s model). The book’s communication model deliberately avoids global-stabilization-time partial synchrony as a base assumption — messages may be arbitrarily delayed or dropped — with a detailed critique of applying the DLS1988 GST formalization to liveness of non-terminating blockchain protocols, including in NTT2020 and the Streamlet paper; safety properties are stated as unconditional properties of executions, with probabilistic reasoning factored out separately (construction.md, lines 13–41 and 342–366). A consequence worth recording: NTT2020 defines a (β_1, β_2) -secure ebb-and-flow protocol by exactly such GST/GAT-parameterized environments (Definition 3) and proves Snap-and-Chat optimally $(1/3, 1/2)$ -secure (Theorem 1;

arXiv:2009.04987v3), and no theorem of that shape appears for Crosslink 2 in any source we have read. The two analyses are therefore not directly comparable, by the book’s own choice of model. Bin: *open problem* — no (β_1, β_2) -style security statement for Crosslink 2 exists in any pinned source.

The book’s status chapter has not moved since 2024: “This is an early and incomplete protocol design proposal. It has not been well vetted for feasibility and safety”, with “Security and safety analyses” still on the missing-components list beside subprotocol selection, issuance, a transition plan, and ZIPs (`status-and-next-steps.md`, lines 3 and 19–27). The book has been untouched since 2024-12-13 while the implementation advanced through 2026-04; which document is the authoritative security argument for the protocol actually being built is itself an open problem (§6.7).

6.3 Bounded availability, Stalled Mode, and overrides

Crosslink’s finality is *conditional* by construction. An honest validator votes for a proposal only if it is bft-proposal-valid and its snapshot is σ -confirmed in the validator’s own bc-best-chain (`construction.md`, lines 654–657), so a coalition holding more than one third of an epoch’s voting units can stall finality indefinitely: finality arrives only while a two-thirds quorum remains live and honest enough. The design’s answer is to make the stall visible and bounded in its consequences rather than impossible: once the gap between the tip and the finalized snapshot exceeds L blocks, the Finality depth rule forces every new block to be a coinbase-only “stalled” block (`the-arguments-for-bounded-availability-and-finality-overrides.md`, lines 180–188), and Stalled Mode “can only be entered by consensus of a larger validator set, or if there is an availability failure of the finalization protocol” (tfl-book @ fe6e1d6, `the-arguments-...md`, line 187). The same machinery is a censorship defense: part of the design intent is “to ‘convert censorship attacks into stall attacks’” — an attacker cannot censor targeted transactions without stalling the whole service, and a stall is immediately and consensually visible while censorship tends to be visible only to its victims (`book/src/design/security-properties.md`, line 35).

The argument for bounding availability at all is the CAP position: finality plus dynamic availability plus the possibility of partition implies an unbounded finalization gap (made precise by [LR2020]), and allowing spends into an unbounded gap risks a crisis of confidence; the book cites Ethereum’s May 2023 double finality stall (≈ 25 and ≈ 64 minutes, a Prysm resource-exhaustion bug) as evidence that stalls happen, that short ones self-heal, and that long ones need human intervention (tfl-book @ fe6e1d6, `the-arguments-...md`, lines 31–41 and 135–139). The reason to prefer coinbase-only blocks over halting outright is the tail-thrashing attack: during a stall of a halted chain, an adversary with 10% of hash power builds a 10-block fork in 100 block times because the honest chain is not advancing, letting the k -block tail thrash between chains — and hash rate is expected to *fall* during a stall as miners anticipate rollback (lines 200–229). Two caveats attach. The constraint binds only while a producer’s view of the finalization point is not advancing: “an adversary that has subverted the BFT protocol in a way that does not keep the finalization point from advancing, can always avoid being constrained by Stalled Mode” (`construction.md`, lines 409–411). And finality can be overridden deliberately: the book argues override must remain possible for exploited flaws (Bitcoin’s 2010 value-overflow rollback of 53 blocks over roughly nine hours; Ethereum’s DAO fork), should be *formalized* under a well-defined governance process, and composes with bounded availability — validators asked to stop cause a stall, the chain enters Stalled Mode after L blocks, only a bounded window of user spends is exposed to the rollback, and miners’ coinbase-only rewards would be preserved across it (`the-arguments-...md`, lines 148–198).

Remark 6.6 (Long-range attacks and key hygiene). The one-third bound counts ballots cast at *any* time in the execution, so a compromised old validator key used to vote for a long-finished epoch violates it retroactively. The book’s consequence is operational: “validator keys of honest nodes must remain secret indefinitely. Whenever a key is rotated, the old key must be securely deleted.” The recommended mitigation is a checkpoint bft-block baked into each released node version, with local finality exposed to clients only once synced past it; improvements are tracked as tfl-book issue #140 (`construction.md`, lines 149–154 and 557–562). Bin: *designed but unspecified*, with the issue open.

Classification. The Finality depth rule and Stalled Mode are *designed but unspecified* consensus rules of the book’s Crosslink 2; the implementation has dropped them (§6.5). The governance process that a formalized finality override presupposes does not exist in any pinned source; the accountability machinery that would let a bc-transaction carry objective evidence of a BFT safety violation (two final bft-blocks with the same parent, a “bft-final-vee”) is a change marked [Recommended] but not adopted into Crosslink 2, and remains unimplemented (potential-changes.md, lines 12–24; questions.md, lines 41–42). Both are *open problems*.

6.4 Machine-checked coverage: the Quint model

The only machine-checked artifact in the corpus is the Informal Systems Quint specification, and its own README bounds its scope: it is a self-described proof of concept that encodes “a tiny part of the system, namely the validity check, of a crosslink proposal within the BFT consensus mechanism”, with most of the required information “inferred ...from an insightful Youtube talk”, a warning that “the encoded logic might not be up-to-date with the current protocol design ideas”, and a closing question to the community “whether we should move forward with applying for a grant” (crosslink-spec @ 8edc3e9, README.md, lines 27 and 93).

What the model checks is nonetheless crisp. The validity predicate is

$$\begin{aligned} \text{isValid}(bft, pow, p) = & pow.\text{contains}(p.\text{block}) \wedge p.\text{bftContext} = bft.\text{last}() \\ & \wedge \text{sigmaConfirmed}(p.\text{block}, p.\text{confirmation_tail}, \sigma) \\ & \wedge \text{linear}(p.\text{block}, p.\text{bftContext}, pow), \end{aligned}$$

over a single node’s pow_store and bft_store (validity.md, lines 131–176). Against it the spec generates a finalization witness (reachability of three BFT blocks) and checks the invariant prefix_inv — “the PoW history defined by BFT finalization is a path in the PoW block store” — by random simulation (10,000 traces of 20 steps, roughly 132 seconds, no violation found) and by TLC model checking after transpilation to TLA⁺, on stores of at most 8 blocks at confirmation depth 1, in about three minutes (README.md, lines 37–89; validity.md, lines 405–410 and 463–482; src/validity.cfg). The model contains no network, no adversary, no voting, no epochs, no signatures, none of the bc-side rules (Extension, Last Final Snapshot, Finality depth), no Stalled Mode, and no liveness property.

Remark 6.7 (A divergence the spec itself flags). The Quint operator linear(*b*, *ctx*, *store*) conjoins descent with $\neg(\text{ctx.pow_hash} = b.\text{hash})$: it *rejects* a proposal whose snapshot equals the parent’s snapshot. The book’s Linearity rule uses $\leq_{bc}$, which includes equality — and equality is required for Π_{bft} liveness, since an honest proposer repeats its parent’s headers_bc when it cannot extend. The spec’s own comment reads “this actually seems allowed. TODO: perhaps remove” (crosslink-spec @ 8edc3e9, validity.md, lines 171–176; tfl-book @ fe6e1d6, construction.md, lines 71, 573, 611–616, and 627).

Classification. The Quint model and its check results are *specified* in the only sense available — runnable, pinned source — but of deliberately narrow scope. Machine-checking has not progressed since: the spec’s last commit is 2025-07-23, the companion quint-crosslink repository contains no commits at all, and a web search on 2026-07-15 found no public evidence of a funded extension. Coverage of the bc-side rules, of any multi-node or adversarial property, and of liveness is an *open problem*.

6.5 The implementation diverges from the analyzed design

The zebra-crosslink book embeds verbatim copies of the tfl-book’s construction and security-analysis chapters, made “for the purposes of precisely committing to a (potentially incomplete, confusing, or inconsistent) precise set of book contents for a security design review”; diffing them against tfl-book @ fe6e1d6 shows the only changes are the warning admonitions (three prepended to

the construction copy, one — the paste notice — to the security-analysis copy), and the copies state that Daira-Emma Hopwood and Jack Grigg “have not reviewed and do not necessarily endorse” them (zebra-crosslink @ 6d02a1b, book/src/design/cl2-construction.md and design/analysis/cl2-security-analysis.md). The project’s own design chapter is franker still: “These extensions and changes from the Crosslink 2 construction may violate some of the assumptions in the ssecurity [sic] proofs. This still needs to be reviewed after this design is more complete”, and “We are using a custom-fit in-house BFT protocol for prototyping. It’s [sic] own security properties need to be more thoroughly verified” (book/src/design.md, lines 11–12). Two of the admonitions record load-bearing divergences.

Divergence 1: Stalled Mode is dropped. “The zebra-crosslink design in this book diverges from this text by *not* including ‘Stalled Mode’ rules. This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls” (book/src/design/cl2-construction.md, lines 10–12). The implementation as designed therefore has an unbounded finalization gap with spends continuing — precisely the condition the tfl-book’s bounded-availability chapter argues is unacceptable, and the regime whose tail-thrashing analysis motivated Stalled Mode in the first place (tfl-book @ fe6e1d6, the-arguments-....md, lines 17–27; §6.3). No pinned source reconciles the two positions.

Divergence 2: the outer signature is dropped. “The zebra-crosslink design does not use the ‘outer signature’ referred to in this chapter... This means a direct hash or copy of the most recent BFT finality certificate is insufficient evidence to ensure that specific bitwise copy is canonical or will become canonical in the chain” (book/src/design/cl2-construction.md, lines 14–16). The book’s model has the proposer re-sign (P , proof_P) with a strongly unforgeable signature (tfl-book @ fe6e1d6, construction.md, lines 156–159); the implementation’s replacement mechanism and its security argument are undocumented in the sources we have read.

Which BFT protocol, exactly. The book’s analysis instantiates Π_{origbft} as adapted Streamlet: notarization at two-thirds of voting units, finality at the second of three adjacent notarized blocks with consecutive epochs, liveness after five consecutive honest-leader epochs (tfl-book @ fe6e1d6, construction.md, lines 218–241 and 614–618). The implementation at HEAD instead uses “tenderlink” 0.1.0, an in-house Tendermint-style engine pinned from git.sr.ht/~azmr/stuff at revision 0e885095...; an earlier Malachite (Informal Systems) integration survives only as dead code — `src/malctx.rs` still imports `malachitebft_*` crates but is not declared as a module, and `Cargo.lock` contains no `malachitebft` packages (zebra-crosslink @ 6d02a1b, zebra-crosslink/Cargo.toml; Cargo.lock, lines 5445–5459; src/lib.rs). No published analysis shows tenderlink satisfies the properties the Crosslink safety arguments assume of Π_{origbft} — Final Agreement, objective block validity, sound two-thirds notarization. Bin: *open problem*, in the project’s own words quoted above.

Parameters, and the black box today. The prototype ships $\sigma = 3$ and $L = 7$ under `PROTOTYPE_PARAMETERS`, annotated “No verification has been done on the security or performance of these parameters” (zebra-crosslink/src/chain.rs, lines 219–222); production values are unspecified everywhere. For comparison, the Π_{bc} interface that Crosslink would consume today is the fork’s hard rollback limit `MAX_BLOCK_REORG_HEIGHT = 99` — the non-finalized state never reorganizes deeper, and zebra-crosslink reuses the constant as its pre-BFT fallback finality depth (zebra-crosslink @ 6d02a1b, zebra-state/src/constants.rs, line 31; zebra-crosslink/src/lib.rs, lines 260–292); upstream Zebra has since raised the same constant to 1000 (zebra @ 9f3166b, zebra-chain/src/parameters/constants.rs, line 30), and the *Ironwood Guide*’s 100-block

“reorg horizon” for anchor selection is reconciled with the policy-not-limit framing in §5.3. We cite the bound as interface, per this series’ scope rule; its derivation belongs to the PoW layer.

Implementation evidence of reorg fragility. Deriving the pre-Crosslink final block from Zebra’s block locator proved subtle: the node wrote three alternative implementations of `tfl_reorg_final_block_height_hash()` and cross-asserted them, and the retained comment reads “NOTE: possible race condition: only for testing / Sam: YES! Indeed there were race conditions.” The node also keeps a stateful `latest_final_block` that shadows the state-service view, mirroring the book’s node-local fin_i state machine, whose “record finalization safety hazard” branch arises exactly when the global assumptions fail (zebra-crosslink @ 6d02a1b, zebra-crosslink/src/lib.rs, lines 235–332; tfl-book @ fe6e1d6, construction.md, lines 471–488). The interleaving of BFT finality with PoW reorg handling is where the implementation has already met race conditions; it deserves adversarial review once the design settles.

6.6 Economic security: the mechanism-design review

The zebra-crosslink design decomposes security across five components: wallets (privacy and censorship defense), miners (liveness and censorship resistance; unable to violate finality even with more than half the hash power), finalizers (more than one third of stake-weighted votes can stall but cannot unilaterally roll back), stakers (holding finalizers accountable solely by *redelegating* — there is no slashing), and users (last-resort accountability via a user-led emergency hard fork, with STEEM/HIVE cited as precedent). Its own attack scenarios: a collusion of more than half the hash power *and* more than two thirds of stake can violate finality and execute rollback or unavailability attacks; and because staking and redelegation transactions themselves need censorship resistance, an attacker with more than half the hash power can censor redelegations and thereby disable the sole staker-accountability mechanism (zebra-crosslink @ 6d02a1b, book/src/design/five-component-model.md, lines 5–52). The first deployment concentrates the finalizer set deliberately: the active set is the top $K = 100$ finalizers by stake weight, and support for finalizers run by arbitrary or casual users is explicitly out of scope (book/src/design/scoping.md, lines 53–57; nutshell.md, line 104).

The independent review of this mechanism is the nikete (Nicolas Della Penna) audit: pre-registered 2025-12-11, completion announced by Shielded Labs on 2026-02-05, report at nikete.com/crosslink-zebra-audit. Its verdict: “safety (irreversibility) is structurally strong once finality is reached”, but “finality progress (liveness) is economically fragile in the baseline design and requires explicit incentive constraints”. Three failure modes: *hostage holdout* — pending rewards accumulate until a finality-updating block, so a blocking coalition rationally delays finality to grow its future payout; *zombie set* — the finality gadget becomes permanently inoperable through validator attrition during a freeze; and *phantom security* — if stake can exit economically while a frozen validator set retains voting weight, the effective cost to corrupt the frozen set decays over time. Its recommendations: signer-conditioned commission, first-inclusion miner bounties tied to finalized height, finality-gap telemetry, anti-jackpot constraints on pending rewards during stalls, and governance-level liveness resets (nikete.com/crosslink-zebra-audit, accessed 2026-07-15; pre-registration at book/src/design/analysis/history/2025-12-11-mech-design.md and .../2025-12-11-mech-design/proposal.md @ 6d02a1b8). The audit’s ground rules bind its scope: no protocol-level slashing in V1 is a design constraint, the assumed economics were a 40/40 issuance split and 5-day epochs with 5–10-day unbonding, and privacy impacts on stakers or finalizers were out of scope (.../2025-12-11-mech-design/proposal.md, lines 33, 58, and 64–66). Whether any recommendation has been incorporated is not visible in the repositories read.

The reward mechanics the audit targets are design-stage: pending rewards R distribute at a finality-updating block as $S_{\text{finalizer}} = c \cdot W_{\text{finalizer}}/W_{\text{total}}$ and $S_{\text{staker}} = (1 - c) \cdot W_{\text{staker}}/W_{\text{total}}$ with commission rate $c = 0.1$, totals possibly summing below S_{total} because stake assigned outside the active set earns nothing (book/src/design/nutshell.md, lines 19–37). The hostage-holdout finding is a direct consequence of that “until a finality-updating block” accumulation. One privacy cost is already fixed at

design stage: staking positions are ledger state with on-chain amount, target-finalizer key, and per-position redelegation key — “staked amounts are revealed so the community can verify the total ZEC staked and its distribution across finalizers” — mitigated only by quantizing stakes to powers of ten ZEC; the Crosslink Staking Orchard Restriction confines staking transactions to Orchard, fees, and burns, and Penumbrastyle shielded delegation is deferred to “continued exploration in future versions” (nutshell.md, lines 39–104; shieldedlabs.net/crosslink-updated-staking-design/, 2025-10-27, accessed 2026-07-15).

Classification. The five-component model, the reward split, and the staking design are *designed but unspecified*; the audit findings are *specified* as a pinned report but normative for nothing. Whether the no-slashing, redelegation-only accountability model survives the phantom-security finding is an *open problem* — the audit proposal itself lists “unbounded divergence between PoW progress and a stalled BFT subprotocol” among the failure modes to examine, which is exactly the regime the implementation entered by dropping Stalled Mode (§6.5).

6.7 Standing, siblings, and the open-problem ledger

Normative standing. No Crosslink ZIP exists: the zips repository at 6961098 (2026-07-09) contains Crosslink only as a citation footnote in draft ZIP 218 (25-second block target spacing) and as the source of the Assured Finality definition cited by draft-ecc-onchain-accountable-voting; the zebra-crosslink book says the ZIP draft exists as a Google Doc, to be refined “as the prototype approaches maturity” (book/src/design.md, line 7). NU8 scoping is an open governance discussion (“NU8 Decision Framework”, forum thread opened 2026-02-11) in which Crosslink competes with the 25-second-blocks proposal and with Tachyon; the *Tachyon Guide* records the same environment from the other side (§“Relation to Crosslink and the upgrade environment”). Shielded Labs’ roadmap shows Milestones 1–5 complete (2025-03-21 through 2026-04-15) and a productionization phase running Q2 2026 – Q2 2027 — testnet deployment, design iteration, potential integration (shieldedlabs.net/roadmap/, accessed 2026-07-15). No confirmed mainnet activation date exists.

What Crosslink would change for the siblings. Today the stack expresses transaction confirmation entirely as depth policy on the black-box PoW chain: the *Wallet Guide* documents the ZIP-315 ConfirmationsPolicy with trusted depth 3 and untrusted depth 10 (§“Transaction construction”, Definition “Confirmations policy”), and the *Ironwood Guide* records the 100-block reorg horizon governing anchor robustness (§“Proof of ownership of *some* valid note: the commitment tree”). Crosslink would add a protocol-level notion beneath both: finality status is the sole “height-eventual” ledger state it introduces — computable for height H only from a later attesting block, with the guarantee that if the attesting block rolls back, “an alternative block will eventually attest to the finality status of the same candidate block” (zebra-crosslink @ 6d02a1b, book/src/design/nutshell.md, lines 116–124). How wallet policy would consume assured finality is unspecified in every source read. Separately, to prevent a terminology collision: the *Voting Guide*’s “finalization” is an application-level tally event with its own caveat — a round can finalize with empty results (§“Trust model and verdict”, “Tally liveness admits finalization without results”) — and has no relation to consensus finality.

The open-problem ledger. We close with the ledger, each entry carrying its bin and, where one exists, its revisit trigger.

1. *The authoritative security argument.* The analyzed design (tfl-book, frozen 2024-12-13, analysis incomplete below security-analysis.md, line 54) and the built design (zebra-crosslink, advancing through 2026-04, Stalled Mode and outer signature dropped) have diverged with no document proving the latter. Trigger: completion or supersession of the tfl-book analysis.

2. *The Final Agreement proof defect* (Remark 6.3) and the unproven $\Pi_{\text{origbc}} \rightarrow \Pi_{\text{bc}}$ assumption transfer (§6.1).
3. *The parameter gap*. No quantitative relationship among L , σ , network delay, and honest hash and stake exists; the prototype's $\sigma = 3$, $L = 7$ carry an explicit no-verification warning, against a rollback bound that is itself only node policy — 1000 blocks in mainline Zebra at 9f3166b, 99 in the zebra-crosslink fork (§6.5).
4. *The Stalled Mode question*. Whether the eventual ZIP adopts the book's bounded availability or the implementation's redelegation-only stance, and whether dropping the rule reopens the unbounded-gap and tail-thrashing hazards the book used to justify it (§6.5). Trigger: a filed Crosslink ZIP.
5. *The BFT engine*. No analysis shows tenderlink — or any concrete Π_{origbft} candidate — satisfies Final Agreement, objective validity, and sound notarization (§6.5).
6. *Machine-checking*. Coverage stops at `isValid`, with one flagged divergence from the book (Remark 6.7); the bc-side rules, adversarial settings, and liveness are unchecked. Trigger: any extension of the Quint spec or content in `quint-crosslink`.
7. *Accountability machinery*. On-chain evidence of BFT safety violations remains a non-adopted [Recommended] change (§6.3); long-range key hygiene improvements remain tfl-book issue #140 (Remark 6.6).
8. *Audit follow-through*. Adoption of the mechanism-design recommendations, and survival of the no-slashing model against phantom security, are unresolved (§6.6). Shielded Labs states further reviews by established firms precede activation.
9. *Design-source discrepancies*. The staking epoch is roughly seven days in the nutshell (a one-day staking day plus a six-day locked phase) but five days in the 2025-10-27 blog post and the audit proposal; which is current is unrecorded (`nutshell.md`, lines 64–80; `proposal.md`, line 58).

Part XIV**FROST Guide: Threshold RedPallas: FROST, its re-randomised form, and where Zcash uses it****Abstract**

Assuming the RedPallas signatures of the *Ironwood Guide*, this volume of *The Zcash Arboretum* documents threshold signing for Zcash: FROST as its paper constructs it and as RFC 9591 standardises it, the re-randomised form that survives RedPallas key re-randomisation, and the surfaces that use it — spend authorisation, recoverability custody, and finalizer keys. The audit boundary is stated plainly: the deployed re-randomised path has not been audited, and every claim beyond the paper, the RFC, and the draft ZIP carries its classification.

1 Scope, sources, and deployment surfaces

This volume documents threshold custody of Zcash spend authority: how t of n parties, no one of whom ever holds the whole signing key, jointly produce a signature that consensus accepts as an ordinary RedPallas spend-authorisation signature. Three artefacts carry the construction. The base scheme is FROST, the two-round threshold Schnorr protocol of Komlo and Goldberg, standardised as RFC 9591 (§2). The Zcash layer is Re-Randomized FROST (Gouvêa–Komlo, IACR ePrint 2024/436), which shifts every share by a common randomizer so that the aggregate signature verifies under the per-Action randomised key; ZIP 312 specifies it for Zcash, and the `frost-rerandomized` and `reddsa` crates ship it (§3). The deployment picture — the running stack, the proposed uses, and the gaps — closes the volume (§4).

Three custody surfaces anchor the treatment, one per rigor bin (§1.3): spend-authorisation custody for Orchard, where the construction is specified and shipped; custody of the ZIP-2005 quantum spending key qsk, where a Proposed consensus ZIP constrains a FROST ceremony whose recovery protocol it defers; and Crosslink finalizer keys, where no threshold design exists at all. The surfaces and their bins are fixed in §1.4.

1.1 Scope

The object under study is a t -of- n signing protocol whose output a verifier cannot tell apart from single-party spend authorisation. ZIP 312 states the requirements directly: the signatures must verify as Sapling or Orchard spend-authorisation signatures, and should meet the protocol specification’s “Signature with Re-Randomizable Keys” criteria. Plain FROST fails the first clause — it signs for a fixed group key, and consensus verifies only per-Action randomised keys — and closing that gap is the volume’s central construction (§3).

The volume develops RedPallas only, per its title. ZIP 312 itself defines two ciphersuites — FROST(Jubjub, BLAKE2b-512) for Sapling RedJubjub and FROST(Pallas, BLAKE2b-512) for Orchard RedPallas — and we cite the Jubjub suite where the ZIP fixes both at once, but develop only the Pallas suite (§3.5).

ZIP 312’s stated non-requirements bound the scope from below: it does not remove the Coordinator role; it defines no key-generation procedure (out of scope there, exactly as in RFC 9591); and it provides no network privacy. Key generation nevertheless appears twice in this volume, both times forced by sources: once as published with the base scheme (§2.2), and once as the unspecified-but-shipped DKG surface of the `reddsa` crate, binned in §1.3.

One altitude note governs every deployment claim. The deployment is library-level, never consensus-level: no consensus rule knows FROST exists, and none needed to change. The verifier accepts the aggregate because it is a valid RedPallas signature; as ZIP 312’s rationale puts it, the method by which the commitment R was generated is irrelevant to the verifier.

1.2 The source ladder

Table 1 pins every primary source. All repository facts in this volume were verified firsthand at the stated commits; web sources were fetched 2026-07-15; local copies of the papers and the RFC are archived with the volume’s drafts.

Artefact	Pin / date	Rigor
RFC 9591 (IRTF CFRG)	June 2024	Informational; standards rigor
ePrint 2024/436 (Gouvêa–Komlo)	recv. 2024-03-13	preprint; no venue
ZIP 312	6961098410, 2026-07-09	Draft (Wallet category)
ZIP 2005	6961098410, 2026-07-09	Proposed (Consensus category)
ZcashFoundation/frost	2016e44ba4, 2026-04-23	shipped code; workspace 3.0.0
ZcashFoundation/reddsa	3792daa95e, 2026-05-22	shipped code; crate 0.5.2
ePrint 2020/852; 2022/833	SAC 2020; CRYPTO 2022	peer-reviewed antecedents

Table 1: Primary FROST sources, pinned. Verification date 2026-07-15.

The standard. RFC 9591, “The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures” (D. Connolly, Zcash Foundation; C. Komlo, University of Waterloo and Zcash Foundation; I. Goldberg, University of Waterloo; C. A. Wood, Cloudflare; IRTF CFRG stream, Category Informational, June 2024), is the top of the ladder and the only artefact here at standards rigor. Two of its four authors are Zcash Foundation: the standard and the Zcash deployment share provenance. It fixes the base two-round protocol and leaves key generation out of scope; its deltas against the SAC 2020 paper are catalogued in §2.8.

The paper. “Re-Randomized FROST”, by Gouvêa and Komlo (both Zcash Foundation), is IACR ePrint 2024/436 (<https://eprint.iacr.org/2024/436>): received 2024-03-13, approved 2024-03-15, publication info “Preprint”, no peer-reviewed venue as of 2026-07-15. We read it in full and work from it throughout §3. Its headline result, Theorem 1, proves unforgeability in the random-oracle model under AOMDL — the same assumptions as plain FROST (§3.3). The construction it describes is specified and shipped; the security theorem above it rests on a preprint, and we say so wherever we lean on it.

The ZIP. ZIP 312, “FROST for Spend Authorization Multisignatures” (owners Conrado Gouvêa, Chelsea Komlo, and Deirdre Connolly; Status Draft, Category Wallet; Discussions-To `zcash/zips#382`, pull request `zcash/zips#662`), is the Zcash specification of the construction.¹² It is visibly unfinished — its Created field still reads 2022-08-dd, day unfilled — so we cite it as a moving object, by repository commit (`zcash/zips @ 6961098410, 2026-07-09`) and Draft status, never as a stable specification. Its threat model trusts the Coordinator and every key-share holder with transaction privacy and unlinkability, never with security: a rogue Coordinator cannot create a signed transaction without the approval of MIN_PARTICIPANTS participants. What it specifies — the unlinkability requirement, the randomizer flow, and the two ciphersuites — is developed in §3.4 and §3.5.

The crates. Repository `ZcashFoundation/frost (@ 2016e44ba4, 2026-04-23)` is a Rust workspace at version 3.0.0 (`rust-version 1.81`): `crate frost-core` carries the ciphersuite-generic protocol, six ciphersuite crates instantiate it (`ed25519`, `ed448`, `P-256`, `ristretto255`, `secp256k1`, `secp256k1-tr`), `frost-rerandomized` carries the re-randomised layer, and `gencode` generates the per-suite code. The repository also vendors an audit report dated 2021-03-23 (`zcash-frost-audit-report-20210323.pdf`) and the source of the ZF FROST book (published at

¹²The FROST ZIP is ZIP 312 alone. ZIP 313, its numeric neighbor, is “Reduce Conventional Transaction Fee to 1000 satoshis” — Status Obsolete, obsoleted by ZIP 317 — and has nothing to do with FROST.

frost.zfnd.org), whose “FROST with Zcash” chapters (zcash-devtool demo, Ywallet demo, FROST server) and serialisation-format page — the page ZIP 312 cites for `encode_signing_package` — feed §4. Repository ZcashFoundation/reddsa (@ 3792daa95e, 2026-05-22; crate version 0.5.2) holds the Zcash ciphersuites: its `frost` feature enables `frost-rerandomized 3.0.0`, and `src/frost/redpallas.rs` defines FROST(Pallas, BLAKE2b-512) as `struct PallasBlake2b512`, ZIP 312’s reference implementation surface (§3.6). Crate reddsa also ships machinery ZIP 312 does not specify — DKG hash personalisations and even-Y normalisation hooks — binned in §1.3.

The antecedent papers. FROST itself is Komlo–Goldberg, IACR ePrint 2020/852 (SAC 2020); the security analysis the re-randomised extraction argument extends is Bellare–Crites–Komlo–Maller–Tessaro–Zhu, “Better than Advertised Security for Non-Interactive Threshold Signatures” (CRYPTO 2022; ePrint 2022/833). They are the paper’s references [10] and [1] respectively; §2 works from both.

1.3 The three-bin rule

The series rule is binary for deployed behaviour and ternary for design. A claim about deployed behaviour cites the implementation — crate and file — and the specification section it realises; a claim about a design-stage protocol is classified *specified*, *designed but unspecified*, or an *open problem*, and the bin stays visible in the prose. This volume sits astride the boundary: one surface is shipped code, one is a consensus proposal, one is an acknowledged gap. Applied once, here, for the whole volume:

1. Re-Randomized FROST for spend-authorisation custody is *specified*: RFC 9591 published June 2024, ZIP 312 in Draft, ePrint 2024/436, and shipped code — `frost-rerandomized 3.0.0` and `reddsa 0.5.2`. The deployment is library-level, not consensus (§1.1).
2. Custody of the ZIP-2005 quantum spending key is specified at the ZIP level (Status Proposed, Category Consensus), but the Recovery Protocol it exists to serve is “to be introduced, potentially, in a future upgrade” — *designed but unspecified*.
3. Crosslink finalizer threshold custody is an *open problem*: the *Crosslink Guide*’s own classification, which we adopt.

One surface refuses the clean split, and we bin it here once. No ZIP specifies FROST key generation: ZIP 312 declares it out of scope (as does RFC 9591), and ZIP 271’s May-2025 note confirms the gap — ZIP 312’s missing key-generation text is its stated reason not to couple disbursement of the *lock-box* (the in-protocol reserve that has accrued a portion of each block subsidy since NU6, per ZIP 2001) to that specification. Yet crate reddsa ships DKG support: hash personalisations `FROST_RedPallasD` (HDKG) and `FROST_RedPallasI` (HID), beyond ZIP 312’s H1–H5 and HR, plus even-Y normalisation hooks (`post_generate/post_dkg`, applying `into_even_y` to secret shares and the public key package) so that the group verifying key meets the encoding constraint of Orchard’s spend validating key. Deployed code with no specification above it: we classify the DKG surface *designed but unspecified* and cite the crate at commit 3792daa95e wherever we rely on it.

1.4 The three deployment surfaces

Spend-authorisation custody (specified). The product ZIP 312 promises is signatures “indistinguishable from RedPallas Orchard Spend Authorization Signatures”. The single-party object being imitated is settled in the *Ironwood Guide*: §“Authorisation of the spend: RedPallas” constructs $rsk := ask + \alpha$ and $rk := ak + [\alpha] G^{\text{SpendAuth}} = [rsk] G^{\text{SpendAuth}}$, with `GenRandom` hashing 80 uniform bytes through $H^\otimes$, and §“RedPallas unforgeability and re-randomisation” proves EUF-CMA from discrete log in the random-oracle model. Per the layering rule we cite both and re-prove neither; this volume’s work begins where the single signer ends, at splitting `ask` into shares no signing coalition ever reassembles (§3). The wallet-side integration path is likewise on the record: the *Wallet Guide*,

§“Partially created transactions (PCZT)”, records ZIP 374’s motivation (iii), threshold signing (for example FROST, RFC 9591) — the multiparty transaction format FROST signing rides in §4.3.

Custody of the quantum spending key (designed but unspecified). ZIP 2005, “Ironwood Quantum Recoverability” (owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; created 2025-03-31), writes FROST into its requirements twice: the design should be fully compatible with FROST threshold multisignatures (citing ZIP 312), and should lose no pre-quantum security when FROST and/or hardware wallets are used. Its “Usage with FROST” section then constrains the ceremony: the key ak is derived jointly, via the FROST DKG, from shares of ask ; the participants must privately agree on a value sk and use it with $use_qsk = \text{true}$ to derive nk , qsk , qk , and $rivk_ext = H_{qk}^{rivk_ext}(ak, nk)$; all participants must obtain the same ak , nk , and $rivk_ext$; and each participant must hold qsk at least as securely as ask . The Recovery Protocol requires a RedDSA signature under ak — preserving t -of- n for as long as discrete log on Pallas stands — together with a proof SoK^{qsk} of knowledge of a qsk with $H^{qk}(qsk) = qk$. The asymmetry is the point to retain: the key qsk is held by *every* participant, so a quantum adversary needs only qsk and the threshold property is gone; the ZIP accordingly recommends moving funds to a fully post-quantum threshold protocol once one is available. The deferred Recovery Protocol is what parks this surface in the middle bin; the full treatment is §4.7.

Crosslink finalizer keys (open problem). The *Crosslink Guide*, §“Finalizer keys”, records the prototype state: one ed25519 key per finalizer (ed25519-zebra), a 76-byte vote format signed by appending a 64-byte ed25519 signature over those 76 bytes, and — by negative search over the four Crosslink repositories (tfl-book, crosslink-protocol-guide, crosslink-spec, zebra-crosslink), dated 2026-07-15 there — no FROST or threshold-signing design for finalizer keys sourced anywhere in the corpus. That guide’s classification bins threshold custody, key rotation, revocation, and initial-roster formation as open problems, and observes that FROST’s presence in the PCZT flow makes the absence “a visible gap rather than an exotic ask”. We adopt the classification and return to the surface in §4.8.

Adjacent surfaces, cited but not developed. Three further ZIPs touch FROST and mark how wide the surface is growing. ZIP 271 (“Dev Fund Extension and One-Time Disbursement”, Proposed) anticipates spending lockbox one-time-disbursement chunks “to a FROST address”, and its May-2025 note that ZIP 312 does not specify key generation is its stated reason not to couple the two specifications. ZIP 326 (“NU6.3 Consequences for Wallets”, Draft, created 2026-06-30) cites ZIP 2005’s “Usage with FROST” section. ZIP 374 (“Partially Created Zcash Transaction Format”, Draft, created 2024-12-09) lists FROST per RFC 9591 under its multiparty-transactions motivation. Each appears in this volume exactly where it bears (§4); none is developed as a surface of its own.

2 FROST

The base object of this volume is the threshold Schnorr scheme FROST (*Flexible Round-Optimized Schnorr Threshold* signatures) of Komlo and Goldberg, published at SAC 2020 and maintained as IACR ePrint 2020/852; we work from the final revision of 22 December 2020. The scheme lets any t of n parties, each holding a Shamir share of a signing key s , produce a signature (R, z) that verifies under the ordinary single-party Schnorr equation for the group public key $Y = g^s$ — a verifier cannot distinguish a FROST signature from a single-signer one. That indistinguishability is precisely what makes the scheme deployable under Zcash’s RedDSA validators: the consensus rules never learn that a threshold ceremony occurred. The deployment surfaces this volume targets are spend-authorisation custody over RedPallas (the *Ironwood Guide*, §“Authorisation of the spend: RedPallas”), custody of the ZIP-2005 quantum spending key (whose “Usage with FROST” section constrains the key generation), and Crosslink finalizer keys (the *Crosslink Guide*, §“Finalizer keys”).

Throughout, $\mathbb{G}$ is a group of prime order q with generator g , and H, H_1, H_2 are hash functions with outputs in $\mathbb{Z}_q^*$, modelled as random oracles. The paper fixes the single-party scheme it must match — the Schnorr signature scheme of the *Crypto Guide*, §“From identification to signature via the Fiat-Shamir transform” — restated here in the paper’s multiplicative notation: a Schnorr signature over m under secret key $s \in \mathbb{Z}_q$ and public key $Y = g^s \in \mathbb{G}$ is produced by sampling a nonce $k \xleftarrow{\$} \mathbb{Z}_q$, setting $R = g^k$, $c = H(R, Y, m)$, $z = k + s \cdot c$, and outputting $\sigma = (R, z)$; verification recomputes c and accepts iff $R = g^z \cdot Y^{-c}$. FROST’s whole design problem is to compute the pair (R, z) jointly, with k and s existing only as shares, without opening the concurrency attacks described in §2.4.

2.1 Shamir sharing and share conversion

Two secret-sharing mechanisms cooperate in FROST, and keeping them distinct is the key to reading the signing equation.

Shamir sharing (the long-lived key). A dealer — or, in §2.2, every participant acting as a dealer simultaneously — selects $t-1$ random coefficients $a_1, \dots, a_{t-1}$ and forms the degree- $(t-1)$ polynomial

$$f(x) = s + \sum_{i=1}^{t-1} a_i x^i, \quad f(0) = s,$$

giving participant P_i the share $(i, f(i))$. Any t shares determine f by Lagrange interpolation and hence $s = f(0)$; fewer than t reveal nothing. For a signing set $S = \{p_1, \dots, p_\alpha\}$ of α participant identifiers ($t \leq \alpha \leq n$), the interpolation weight of participant i at zero is the Lagrange coefficient

$$\lambda_i = \prod_{j=1, j \neq i}^{\alpha} \frac{p_j}{p_j - p_i},$$

computable from the identifiers alone — the coalition can be chosen *after* shares are dealt, which is what makes FROST’s “any t of n , decided at signing time” flexibility possible.

Additive sharing and conversion (the per-signature nonce). A sum $s = \sum_i s_i$ with each summand chosen locally is an additive sharing: it needs no dealer and no interaction. The Cramer-Damgård-Ishai share conversion observation, used by FROST in its simplest case, is that additive shares convert locally to Shamir form: if $s = \sum_{i \in S} s_i$, then the values s_i/λ_i are Shamir shares of the same s (evaluations of a degree- $(\alpha-1)$ polynomial interpolating them). FROST uses exactly this to make the group nonce k : each signer contributes an additively-shared summand non-interactively, and the λ_i weights in the signing equation perform the conversion so that the response aggregates by plain summation. No DKG runs at signing time.

2.2 Key generation: Pedersen’s DKG with a knowledge proof

FROST generates its long-lived shares with a variant of Pedersen’s DKG — n parallel executions of Feldman’s verifiable secret sharing, one with each participant as dealer — hardened with a proof of knowledge of each dealer’s free coefficient. The paper’s Figure 1, reproduced:

Round 1.

1. Every participant P_i samples t random values $(a_{i0}, \dots, a_{i(t-1)}) \xleftarrow{\$} \mathbb{Z}_q$ and defines the degree- $(t-1)$ polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{ij} x^j$.
2. Every P_i computes a proof of knowledge of a_{i0} : with $k \xleftarrow{\$} \mathbb{Z}_q$, $R_i = g^k$, $c_i = H(i, \Phi, g^{a_{i0}}, R_i)$, $\mu_i = k + a_{i0} \cdot c_i$, set $\sigma_i = (R_i, \mu_i)$, where Φ is a context string preventing replay across ceremonies.

3. Every P_i computes the public commitment $\vec{C}_i = \langle \phi_{i0}, \dots, \phi_{i(t-1)} \rangle$ with $\phi_{ij} = g^{a_{ij}}$.
4. Every P_i broadcasts $\vec{C}_i, \sigma_i$.
5. On receiving $\vec{C}_\ell, \sigma_\ell$, participant P_i verifies $R_\ell \stackrel{?}{=} g^{\mu_\ell} \cdot \phi_{\ell 0}^{-c_\ell}$ with $c_\ell = H(\ell, \Phi, \phi_{\ell 0}, R_\ell)$, *aborting on failure*; on success all σ_ℓ are deleted.

Round 2.

1. Each P_i securely sends each other participant P_ℓ the share $(\ell, f_i(\ell))$, then deletes f_i and every share except its own $(i, f_i(i))$.
2. Each P_i verifies each received share against the dealer's commitment:

$$g^{f_\ell(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} \phi_{\ell k}^{i^k \bmod q},$$

aborting if the check fails.

3. Each P_i computes its long-lived signing share $s_i = \sum_{\ell=1}^n f_\ell(i)$, stores it securely, and deletes the $f_\ell(i)$.
4. Each P_i computes its public verification share $Y_i = g^{s_i}$ and the group public key $Y = \prod_{j=1}^n \phi_{j0}$.

Three design decisions deserve comment. First, the *proof of knowledge of a_{i0}* is the delta over plain Pedersen DKG: it forecloses rogue-key attacks in the $t \geq n/2$ regime, where a participant could otherwise choose its commitment as a function of others' to bias the aggregate key (the fix was suggested to the authors by Shlomovits and Turkel; the paper's changelog dates it to the 2020-07-08 revision). Second, the *complaint handling* is abort-only: where Gennaro et al. add a complaint/disqualification phase to make the DKG robust, FROST aborts on any failed share check and expects the cause to be investigated out of band; the paper notes that implementations wanting robustness can substitute the Gennaro et al. construction. Third, a *consistent view* of the broadcast commitments $\vec{C}_i$ across participants is assumed, not provided — the paper defers to whatever broadcast mechanism the deployment supplies. Each participant ends holding (i, s_i) , with $Y_i = g^{s_i}$ public for share verification during signing and Y the single key the outside world sees.

2.3 Preprocessing and the two-round signing protocol

Signing splits into a message-independent *preprocess* stage and a single online round (equivalently, run as one two-round protocol with the commitment exchange inlined). A semi-trusted *signature aggregator* SA coordinates: it is trusted only for liveness and correct misbehaviour reporting, not for unforgeability — a malicious SA can at worst deny service or falsely accuse, never learn the private key or cause improper messages to be signed.

Preprocess(π) (paper Figure 2). Each participant P_i generates π single-use nonce pairs and publishes their commitments: for $1 \leq j \leq \pi$,

$$(d_{ij}, e_{ij}) \stackrel{\$}{\leftarrow} \mathbb{Z}_q^* \times \mathbb{Z}_q^*, \quad (D_{ij}, E_{ij}) = (g^{d_{ij}}, g^{e_{ij}}),$$

appending (D_{ij}, E_{ij}) to a list L_i published to a predetermined location, and storing the secret halves for later.

Sign(m) (paper Figure 3). The aggregator SA selects a signing set S of α participants ($t \leq \alpha \leq n$), fetches each member's next unused commitment pair, and forms the ordered list $B = \langle (i, D_i, E_i) \rangle_{i \in S}$, sending (m, B) to every $P_i, i \in S$. Each participant then:

1. validates m (a signer signs only messages it is willing to sign) and checks $D_\ell, E_\ell \in \mathbb{G}^*$ for every commitment in B , aborting on failure;
2. computes the *binding values*

$$\rho_\ell = H_1(\ell, m, B), \quad \ell \in S,$$

the group commitment and challenge

$$R = \prod_{\ell \in S} D_\ell \cdot (E_\ell)^{\rho_\ell}, \quad c = H_2(R, Y, m);$$

3. computes its response, converting its Shamir share to the additive aggregation domain via λ_i (over the set S):

$$z_i = d_i + (e_i \cdot \rho_i) + \lambda_i \cdot s_i \cdot c;$$

4. securely deletes $((d_i, D_i), (e_i, E_i))$, then returns z_i to SA.

Finally SA recomputes $\rho_i, R_i = D_i \cdot (E_i)^{\rho_i}, R = \prod_{i \in S} R_i$ and c , and verifies every share against the public data:

$$g^{z_i} \stackrel{?}{=} R_i \cdot Y_i^{c \cdot \lambda_i},$$

identifying and reporting the misbehaving participant and aborting if any check fails; otherwise it publishes $\sigma = (R, z)$ with $z = \sum_{i \in S} z_i$.

Correctness. The proof is two interpolations (paper §6.1). The key polynomial $F_1(x) = \sum_{j=1}^n f_j(x)$ from key generation has $s_i = F_1(i)$ and $s = F_1(0)$. The nonce values $(i, (d_i + e_i \cdot \rho_i)/\lambda_i)$ define a degree- $(\alpha - 1)$ polynomial $F_2(x)$ with $F_2(0) = \sum_{i \in S} d_i + e_i \cdot \rho_i = k$. Setting $F_3(x) = F_2(x) + c \cdot F_1(x)$, each response is $z_i = \lambda_i F_3(i)$, so $z = \sum_{i \in S} z_i$ interpolates $F_3(0) = k + c \cdot s$; with $R = g^k$ and $c = H_2(R, Y, m)$, the pair (R, z) is a correct Schnorr signature on m .

2.4 Why the binding factor exists: Drijvers and ROS

The one non-obvious ingredient is $\rho_i = H_1(i, m, B)$. The binding factor is FROST's answer to a pair of concurrency attacks that broke or bounded every prior two-round scheme.

The Drijvers attack. Against two-round Schnorr multisignatures in which the adversary may open T parallel sessions, Drijvers et al. showed how to forge by finding a hash output that is a sum of others: $c^* = H(R^*, Y, m^*)$ with $c^* = \sum_{j=1}^T H(R_j, Y, m_j)$, using Wagner's k -tree algorithm for the generalised birthday problem in time $O(\kappa \cdot b \cdot 2^{b/(1+\lg \kappa)})$ for $\kappa = T + 1$ and b the bitlength of the group order. The attack transfers from the n -of- n multisignature setting to a t -of- n threshold scheme with an adversary controlling up to $t - 1$ signers, and it is why the Gennaro et al. scheme must cap its signing parallelism. Benhamouda et al.'s subsequent polynomial-time ROS solver (for $\ell > \lg q$ parallel sessions) turned the same structural weakness from subexponential to practical against schemes including two-round MuSig and GJKR-DKG-based threshold Schnorr; their paper concludes that “the current version of” FROST is *not* affected.

How binding forecloses it. Because each signer derives $\rho_i = H_1(i, m, B)$ before responding, its share z_i is bound to this message, this signing set, and this exact commitment list. Responses cannot be recombined across sessions, messages, or reordered signer sets: any such mix changes some ρ_ℓ , hence R , hence c , and the transplanted share fails verification. The paper’s own generalisation (its Equation 1, with linear combinations γ_j) states the residual attack surface precisely: the forger must find

$$H_2(R^*, Y, m^*) = \sum_j \gamma_j \cdot H_2(R_j, Y, m_j),$$

where $R^* = g^{r^*} \cdot \prod_j \hat{R}_j^{\gamma_j}$ is itself a function of every input on the right-hand side. Wagner’s algorithm needs the two sides of the collision to vary independently; here every candidate combination on the right moves the left-hand target, degrading the generalised birthday collision into multiple preimage attacks. The paper’s footnote 7 concedes this step is heuristic: sufficiency of the condition is immediate, necessity is not proved.

2.5 Nonce hygiene

The preprocessed pairs (d_{ij}, e_{ij}) are one-time keys in the strictest sense. The paper’s stipulations:

- each pair is used at most once, and the signer *securely deletes* it in the same signing step that returns z_i (Figure 3, step 6);
- an accidentally reused (d_{ij}, e_{ij}) can expose the long-term share s_i ;
- a SA that maliciously replays a commitment pair achieves nothing: the participant simply aborts, so replay grants no power;
- the commitment store (if a server) is trusted only for availability and freshness; possession of the public (D_{ij}, E_{ij}) lists grants no signing power.

See §2.8 for RFC 9591’s sharpening of nonce generation (hedged sampling, and an explicit warning that deterministic nonces enable complete key recovery in the multi-party setting).

2.6 The security theorem as stated

The paper proves EUF-CMA security, in the random oracle model, by reduction to the *discrete logarithm problem* in $\mathbb{G}$ — deliberately not to one-more discrete log (the adversary must return the discrete logarithms of $\ell + 1$ challenges after at most ℓ queries to a discrete-log oracle), so that the negative result of Drijvers et al., which rules out security reductions from that assumption for a family of two-round Schnorr multisignatures, does not apply. The proof is for an intermediate scheme, *FROST-Interactive*, in which the binding value is produced by a one-time verifiable random function $\rho_i = a_{ij} + b_{ij} \cdot H_\rho(m, B)$ with committed keys.

Theorem 6.1 (ePrint 2020/852). *If the discrete logarithm problem in $\mathbb{G}$ is (τ', ϵ') -hard, then the FROST-Interactive signature scheme over $\mathbb{G}$ with n signing participants, a threshold of t , and a preprocess batch size of π is $(\tau, n_h, n_p, n_s, \epsilon)$ -secure whenever*

$$\epsilon' \leq \frac{\epsilon^2}{2n_h + (\pi + 1)n_p + 1} \quad \text{and}$$

$$\tau' = 4\tau + (30\pi n_p + (4t - 2)n_s + (n + t - 1)t + 6) \cdot t_{\text{exp}} + O(\pi n_p + n_s + n_h + 1),$$

such that t_{exp} is the time of an exponentiation in $\mathbb{G}$, assuming the number of participants compromised by the adversary is less than the threshold t .

Here n_h, n_p, n_s bound the forger’s random-oracle, preprocess, and signing queries. The architecture is Bellare–Neven generalised forking: the DL challenge ω is embedded as the honest participant’s free-coefficient commitment ϕ_{t0} in a simulated KeyGen, the simulator is run twice on the same tape with oracle answers diverging from index J , and two forgeries yield $\text{dlog}(Y) = (z' - z)/(h'_J - h_J)$. The VRF is what lets the simulator win the programming race: it can compute R before the forger does and program $H_2(R, Y, m)$ with success probability 1.

The step from FROST-Interactive to plain FROST (replace the VRF by $\rho_i = H_1(i, m, B)$) is argued *heuristically* in the paper’s §6.2, not proven — the Equation-1 gap of §2.4. This gap is what the later literature closed, differently and under a different assumption (§2.7).

2.7 The corrected and extended analyses

Three later results matter for how this volume cites FROST’s security.

Crites–Komlo–Maller (ePrint 2021/1375). This work, “How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures” (IACR ePrint 2021/1375, received 2021-10-12, last revised 2022-08-03), introduced the optimisation now called *FROST2*: a single binding factor $d = h_1(vk, lr)$ shared by all signers, replacing the original per-signer $d_i = h_1(vk, lr, i)$ (that original is retroactively *FROST1*) and reducing the signers’ scalar multiplications from linear in the number of signers to constant.

Bellare–Crites–Komlo–Maller–Tessaro–Zhu (CRYPTO 2022). The paper “Better than Advertised Security for Non-interactive Threshold Signatures” — its FROST/BLS part maintained as Bellare–Tessaro–Zhu, ePrint 2022/833, June 2022 — gave the analysis now cited by RFC 9591. The authors define a hierarchy $\text{TS-UF-0} \leq \dots \leq \text{TS-UF-4}$ (and strong variants TS-SUF-2/3/4) grading what counts as a trivial forgery. Their results, proven in the ROM under the *one-more discrete logarithm* (OMDL) assumption, without the AGM, and in the trusted-dealer setting (the DKG is explicitly not modelled):

- FROST1 is TS-SUF-3-secure, and is *not* TS-UF-4-secure;
- FROST2 is TS-SUF-2-secure, and is *not* TS-UF-3-secure — the shared binding factor loses the guarantee that the signer set that committed is the signer set that signed;
- concretely (their Theorem 5.1, verbatim bound), for a TS-SUF-2 adversary A making at most q_s preprocessing and q_h random-oracle queries there is an OMDL adversary B making at most $2q_s + ns$ challenge queries (ns their number of parties) with

$$\text{Adv}_{\text{FROST2}[\mathbb{G}]}^{\text{ts-suf-2}}(A) \leq \sqrt{q \cdot (\text{Adv}_{\mathbb{G}}^{\text{omdl}}(B) + 3q^2/p)}, \quad q = q_s + q_h + 1,$$

and an analogous Theorem 5.4 for FROST1 at TS-SUF-3.

The security FROST actually deploys with is therefore: OMDL + ROM, static corruptions below t , trusted-dealer keys — a different basis than the original paper’s DL-based proof of the interactive variant plus heuristic bridge.

Gouvêa–Komlo (ePrint 2024/436). The preprint “Re-Randomized FROST” (received 2024-03-13) is the design ZIP 312 builds on: FROST unchanged, plus algorithms $\text{GenRand}(\alpha \xleftarrow{\$} \mathbb{Z}_p)$, $\text{RandShare}(\bar{x}_i \leftarrow x_i + \hat{\alpha} \text{ with } \hat{\alpha} = H_r(\alpha, \text{aux}), \text{aux an optional external-protocol transcript contribution})$, $\text{RandPKShare}(\bar{PK}_i = PK_i \cdot g^{\hat{\alpha}})$ and $\text{RandPK}(\bar{PK} = PK \cdot g^{\hat{\alpha}})$. Correctness rests on the $\sum_{i \in S} \lambda_i = 1$ identity, giving $z = r + c(x + \hat{\alpha})$. Its Theorem 1 proves unforgeability in the ROM under *algebraic*

OMDL (AOMDL) — the same assumptions as plain FROST per BCKMTZ — with the bound (its Equation 3)

$$\text{Adv}_{TS,A}^{\text{sec}} \leq \sqrt{q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2q^2/p - \text{negl}(\lambda)}, \quad q = q_h + q_s,$$

via the local forking lemma; the randomizer being public lets the reduction strip it ($z_0 \leftarrow z^* - c^*H_r(\alpha^*, \text{aux}^*)$) during extraction. The coordinator chooses the randomizer and is trusted for privacy and unlinkability only, not for unforgeability. The full treatment of this paper belongs to the re-randomisation chapter (§3).

2.8 RFC 9591 against the paper: what the standard changed

RFC 9591 (IRTF CFRG, Informational, June 2024; Connolly, Komlo, Goldberg, Wood) is FROST as implementations ship it, including the `frost-core` 3.0.0 workspace this volume cites. Reconciling it against ePrint 2020/852:

- *Two rounds only.* The RFC specifies the two-round protocol; the paper’s single-round-with-preprocessing variant (batch parameter π , commitment server) is declared out of scope.
- *No DKG.* The paper’s Figure-1 KeyGen is dropped; key generation is out of scope, with *trusted-dealer* generation given in Appendix C only. The BCKMTZ analysis the RFC cites matches this framing (it, too, models a trusted dealer).
- *Binding factor inputs widened.* The paper binds $\rho_i = H_1(i, m, B)$. The RFC computes, per participant,

$$\text{binding_factor}_i = H_1(\text{PK_enc} \parallel H_4(\text{msg}) \parallel H_5(\text{encoded commitment list}) \parallel \text{enc}(i)),$$

adding the *group public key* to the hash and pre-hashing the message and commitment list. The per-participant structure means RFC 9591 standardises FROST1: the RFC’s §7.2 rules the single-binding-factor (FROST2) optimisation NOT RECOMMENDED, citing the loss of the started-set-equals-signed-set guarantee (the TS-SUF-3 vs TS-SUF-2 separation of §2.7).

- *Hedged nonces.* The paper samples (d, e) uniformly; the RFC hedges: each nonce is $H_3(r \parallel \text{enc}(sk_i))$ with r a fresh 32-byte random string, so a weak RNG is backstopped by the key share. It warns that fully deterministic derivation enables complete key recovery in the multi-party setting.
- *Five domain-separated oracles.* H_1 (binding factor), H_2 (challenge), H_3 (nonce), H_4 (message pre-hash), H_5 (commitment-list pre-hash), each with a per-ciphersuite contextString; the challenge input order `group_comm_enc` $\parallel$ `PK_enc` $\parallel$ `msg` equals the paper’s $H_2(R, Y, m)$.
- *Roles renamed and channels stated.* The signature aggregator becomes the *Coordinator*; the channel is assumed authenticated. The Coordinator SHOULD verify the aggregate signature and MAY verify individual shares — the paper’s per-share check $g^{z_i} = R_i \cdot Y_i^{c\lambda_i}$ survives as `verify_signature_share` — with identifiable abort, as in the paper, and no robustness.
- *Ciphersuites.* The RFC ships FROST(Ed25519, SHA-512), FROST(ristretto255, SHA-512) (RECOMMENDED), FROST(Ed448, SHAKE256), FROST(P-256, SHA-256), and FROST(secp256k1, SHA-256); no Zcash suite appears in the standard itself. The two Zcash suites, FROST(Jubjub, BLAKE2b-512) and FROST(Pallas, BLAKE2b-512), are instead defined by ZIP 312, whose H_2 personalisation `Zcash_RedPallasH` makes the FROST challenge equal RedDSA’s H° — the precise sense in which the standardised protocol plugs into the spend-authorisation verifier of the *Ironwood Guide*, §“Authorisation of the spend: RedPallas”.

- *Security claim.* The RFC claims EUF-CMA per BCKMTZ, citing both the original paper and BCKMTZ — i.e., the deployed claim rests on the OMDL/ROM analysis, not the paper’s DL-based theorem for the interactive variant.

Classification: FROST as in RFC 9591 and its Zcash ciphersuites as in ZIP 312 are *specified*; ZIP 312 itself remains Draft status (zips commit 6961098410, 2026-07-09), with its key-generation procedure explicitly out of scope and PedPoP DKG for Zcash deployment *designed-but-unspecified* (implemented in `frost-core`’s `dkg` module but not standardised in any ZIP or RFC).

3 Re-Randomized FROST for RedPallas

The FROST of §2, standardised as RFC 9591 (§2.8), signs for a *fixed* group public key: the aggregate (R, z) verifies under the single-signer Schnorr equation for PK , and under nothing else. Zcash spend authorisation verifies under no fixed key at all. Every Sapling Spend and every Orchard Action carries a freshly randomised validating key, and consensus checks the spend-authorisation signature against that per-Action key — a key which, as §3.1 makes precise, no set of FROST shares interpolates to unless every share is shifted. This chapter presents the design that closes the gap: the Re-Randomized FROST of Gouvêa and Komlo (IACR ePrint 2024/436), its Zcash specification in ZIP 312, the cipher-suite FROST(Pallas, BLAKE2b-512), the deployed implementation in the `frost-rerandomized` and `reddsa crates`, and the custody surfaces the construction serves.

A rigor note before the material. RFC 9591 is the only artefact here at standards rigor. The Gouvêa–Komlo paper is a preprint — received 2024-03-13, no peer-reviewed venue — and ZIP 312 carries Draft status (zips commit 6961098410, 2026-07-09). Following the series rule, every claim beyond the RFC is binned as *specified*, *designed but unspecified*, or an *open problem*; the closing classification is in §3.7.

3.1 The mismatch: consensus verifies only randomised keys

The protocol specification (§4.15, spend authorization signatures) prescribes, for every Sapling Spend and every Orchard Action, the following signer-side flow. The signer chooses a fresh spend-auth randomizer α and:

1. samples $\alpha \leftarrow \text{SpendAuthSig.GenRandom}()$;
2. derives $\text{rsk} = \text{SpendAuthSig.RandomizePrivate}(\alpha, \text{ask})$;
3. derives $\text{rk} = \text{SpendAuthSig.DerivePublic}(\text{rsk})$;
4. proves the Spend or Action statement with α in the auxiliary (secret) input and rk in the primary input;
5. signs: $\text{spendAuthSig} = \text{SpendAuthSig.Sign}_{\text{rsk}}(\text{SigHash})$.

Consensus therefore verifies the signature under $\text{rk} = \text{ak} + [\alpha] G^{\text{SpendAuth}}$, never under ak itself.

The randomisation algorithms are RedDSA’s, from the protocol specification (§5.4.7):

$$\begin{aligned} \text{RedDSA.RandomizePrivate}(\alpha, \text{sk}) &:= \text{sk} + \alpha \pmod{r_{\mathbb{G}}}, \\ \text{RedDSA.RandomizePublic}(\alpha, \text{vk}) &:= \text{vk} + [\alpha] G, \end{aligned}$$

with identity randomizer 0; `RedDSA.GenRandom` chooses T uniformly among byte sequences of length $(\ell_H + 128)/8$ — 80 bytes at $\ell_H = 512$ — and returns $H^{\otimes}(T)$, where $H^{\otimes}(B) := \text{LEOS2IP}_{\ell_H}(H(B)) \pmod{r_{\mathbb{G}}}$. Signing draws its nonce as $r = H^{\otimes}(T \parallel \text{vk}^* \parallel M)$ for fresh uniform T , sets $R = [r]G$ and $S = (r + H^{\otimes}(R^* \parallel \text{vk}^* \parallel M) \cdot \text{sk}) \pmod{r_{\mathbb{G}}}$, and outputs $\sigma = R^* \parallel S^*$. Validation recomputes $c = H^{\otimes}(R^* \parallel \text{vk}^* \parallel M)$ and accepts iff $R \neq \perp$, $S < r_{\mathbb{G}}$, R^* is canonical (post-ZIP-216), and $[h_{\mathbb{G}}](-[S]G + R + [c]\text{vk}) = 0_{\mathbb{G}}$. RedPallas specialises the template with $\mathbb{G}$ the Pallas group,

$\ell_H = 512$, and $H(x) = \text{BLAKE2b-512}(\text{Zcash_RedPallasH}, x)$; $\text{SpendAuthSig}^{\text{Orchard}}$ uses the generator $G^{\text{SpendAuth}} = \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, "G")$ and “is instantiated as RedPallas with key re-randomization”. The security notion the protocol requires of the scheme is SURK-CMA (Strong Unforgeability with Re-randomized Keys), adapted from Fleischhacker et al. (PKC 2016, §3), whose oracle answers queries of the form (m, α) — message and adversary-chosen randomizer together.

The construction and its single-party security are settled in the *Ironwood Guide*: §“Authorisation of the spend: RedPallas” builds $\text{rsk} := \text{ask} + \alpha$ and $\text{rk} := \text{ak} + [\alpha] G^{\text{SpendAuth}}$, and §“RedPallas unforgeability and re-randomisation” proves EUF-CMA under DL in the ROM *including* adversary-chosen α , by key-prefixed forking with $\text{ask} = \text{rsk}^* - \alpha^*$. Per the layering rule we cite those results and do not re-prove them.

The threshold problem is now visible. A FROST sharing of ask gives shares sk_i with $\sum_{i \in S} \lambda_i \text{sk}_i = \text{ask}$ for every signing set S ; every signature that coalition can produce verifies under ak . But no honest transaction ever asks for a signature under ak : the verifier’s key is rk , shifted by a fresh α per Action. The shares must be shifted too — and, as the next subsection shows, the correct shift is the *same* α for every participant, not a Shamir sharing of α .

3.2 Re-Randomized FROST: the construction

The design is due to Gouvêa and Komlo (both Zcash Foundation), “Re-Randomized FROST”, IACR ePrint 2024/436, received 2024-03-13; we work from the full text, with the construction figure and both displayed equations verified against page renders of the PDF. As in §2 we quote the paper in its own multiplicative notation (g^x for $[x] G$, $PK \cdot g^{\hat{\alpha}}$ for $PK + [\hat{\alpha}] G$); the additive RedPallas form returns with ZIP 312 in §3.4.

Definition 3.1 (Perfectly rerandomizable threshold scheme; ePrint 2024/436, Definition 5). A threshold signature scheme

$$TS = (\text{Setup}, \text{KeyGen}, (\text{Sign}, \text{Sign}'), \text{Combine}, \text{Verify})$$

is *perfectly re-randomizable* if there exist additional PPT algorithms GenRand , RandShare , RandPKShare , and RandPK , and a randomness distribution X , such that the Rand^* algorithms take a randomizer $\alpha \in X$ and an auxiliary string $\text{aux} \in \{0, 1\}^*$.

The paper’s Remark 1 explains aux : it models an optional contribution from an external protocol transcript, so that a new session changes the re-randomised key even under a repeated α .

Construction 3.2 (Re-Randomized FROST; ePrint 2024/436, Figure 5). The additions to plain FROST — the paper’s grey-boxed lines — are exactly four algorithms:

$$\begin{aligned} \text{GenRand}() : \alpha &\xleftarrow{\$} \mathbb{Z}_p; \\ \text{RandShare}(x_i, \alpha, \text{aux}) : \hat{\alpha} &\leftarrow H_r(\alpha, \text{aux}); \quad \bar{x}_i \leftarrow x_i + \hat{\alpha}; \\ \text{RandPKShare}(PK_i, \alpha, \text{aux}) : \overline{PK}_i &\leftarrow PK_i \cdot g^{\hat{\alpha}}; \\ \text{RandPK}(PK, \alpha, \text{aux}) : \overline{PK} &\leftarrow PK \cdot g^{\hat{\alpha}}. \end{aligned}$$

The FROST core runs unchanged on the shifted inputs. Algorithm $\text{Sign}()$ samples $(r_k, s_k) \xleftarrow{\$} \mathbb{Z}_p^2$ and commits $(R_k, S_k) = (g^{r_k}, g^{s_k})$. Algorithm Sign' , given the randomised share $\bar{x}_k$ and randomised key $\overline{PK}$ in place of x_k and PK , derives each binding factor a_i by H_{non} from the participant

identifier, the message, $\overline{PK}$, and the full commitment list $\{(i, R_i, S_i)\}_{i \in S}$, then computes

$$R \leftarrow \prod_{i \in S} R_i \cdot S_i^{a_i}, \quad c \leftarrow H_{\text{sig}}(\overline{PK}, R, m), \quad z_k \leftarrow r_k + s_k a_k + c \lambda_k \bar{x}_k.$$

Algorithm Combine outputs $z \leftarrow \sum_{i \in S} z_i$, $\sigma \leftarrow (R, z)$; $\text{Verify}(\overline{PK}, m, \sigma)$ recomputes the challenge and accepts iff $R \cdot \overline{PK}^c = g^z$.^a

^aThe paper’s own argument order for H_{sig} is inconsistent: Sign' and Combine write $H_{\text{sig}}(PK, R, m)$, while Verify and the reduction’s table write $H_{\text{sig}}(PK, m, R)$ — and both differ from the order RFC 9591 (§4.6) and RedDSA fix, $c = H_2(R^* \parallel PK^* \parallel m)$, which ZIP 312 and the deployed reddsa crate use. We read the paper’s orderings as a presentational slip with no normative weight; the deployed order is RedDSA’s. The paper also twice refers to “Figure 4” where Figure 5 is meant.

The paper is explicit that this is the whole design: “We do not make any changes to the plain FROST protocol.” The only interface change is which share and which public key Sign' is handed.

Correctness is one identity. The paper’s Equation (2):

$$z = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c \sum_{i \in S} \lambda_i \bar{x}_i = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c \left(\sum_{i \in S} \lambda_i x_i + \hat{\alpha} \sum_{i \in S} \lambda_i \right) = r + c(x + \hat{\alpha}) = r + c \hat{x}, \quad (1)$$

“because x is Shamir secret shared” and “ $\sum_{i \in S} \lambda_i = 1$ since it is the interpolation of the constant polynomial $f(X) = 1$ ”. The output is a regular re-randomised signature for $\overline{PK} = PK \cdot g^{\hat{\alpha}}$ with $\hat{\alpha} = H_r(\alpha, \text{aux})$.

The identity deserves emphasis, because it is the entire reason the construction is this simple. Every participant adds the *same* $\hat{\alpha}$ to its share — no dealing, no per-participant shares of the randomizer — and the Lagrange weights, summing to one over any interpolating set, carry the common shift through interpolation so that the group secret moves by $\hat{\alpha}$ rather than by $|S| \cdot \hat{\alpha}$. Had the weights summed to anything else, or varied with S in a way the signers could not predict, the shifted shares would interpolate to a key nobody computed.

3.3 Unforgeability: TRUF, AOMDL, and Theorem 1

The game. The paper’s unforgeability notion (Definition 6, game TRUF, its Figure 4) extends threshold unforgeability with adversarial randomizers. Corruption is *static*: the adversary fixes its up-to- $(t - 1)$ corrupt parties before key generation (the paper titles its Figures 2 and 4 “static unforgeability games”). The signing oracle $\mathcal{O}^{\text{Sign}'}(k, \text{ssid}, m, S, \alpha, \text{aux}, \{\rho_i\})$ lets the *adversary* choose the randomizer and the auxiliary string per query; a forgery $(m^*, \sigma^*, \alpha^*, \text{aux}^*)$ wins if m^* was never queried and σ^* verifies under PK or under $\text{RandPK}(PK, \alpha^*, \text{aux}^*)$. The unrandomised notion is the special case randomizer = 0. The lineage is Fleischhacker et al. (PKC 2016) — the same single-party re-randomisable-key framework, and re-randomised-Schnorr EUF result, that the protocol specification’s SURK-CMA definition adapts (§3.1).

The assumption. AOMDL is the *algebraic one-more discrete logarithm* game (the paper’s Figure 1, taken from the MuSig2 paper): the adversary receives $\ell + 1$ challenges, may query a discrete-log oracle only on linear combinations $(\alpha, \{\beta_i\})$ of environment-generated elements, and must return all $\ell + 1$ discrete logarithms having made at most ℓ queries. The algebraic restriction on oracle queries is what §2.7’s OMDL does not impose.

The theorem. Verbatim (Theorem 1 with Equation (3); the radical verified against the rendered page):

Theorem 1 (ePrint 2024/436). *Rerandomized-FROST is unforgeable in the random oracle model, assuming the AOMDL assumption holds in $\mathbb{G}$, concretely*

$$\text{Adv}_{TS, \mathcal{A}}^{\text{sec}} \leq \sqrt{q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2q^2/p - \text{negl}(\lambda)}, \quad q = q_h + q_s, \quad (2)$$

with q_h random-oracle and q_s signing queries.

Remark 3.3 (Placement of the negligible term). The printed bound places $-\text{negl}(\lambda)$ *inside* the square root; we transcribe it as printed. The customary form of such forking-lemma bounds — including the BCKMTZ bound quoted in §2.7 — carries additive slack outside the radical, and a negative term inside is dimensionally odd for an advantage bound.

Proof shape. The reduction $\mathcal{B}$ receives $2q_s + 1$ AOMDL challenges and sets $PK = X_0$. It simulates trusted-dealer key generation by sampling the corrupt parties’ shares and producing every verification share by Lagrange interpolation in the exponent, so that PK_i is implicitly $g^{f(i)}$ without $\mathcal{B}$ knowing any honest share. Round-one answers are challenge pairs $R_i = X_i, S_i = X_{i+1}$. In round two the reduction runs RandPK and RandPKShare honestly — it cannot run RandShare , holding no share — and derives each honest signature share in the exponent, as $Z_k = R_i \cdot S_i^{a_k} \cdot \widehat{PK}_k^c$, querying the AOMDL linear-combination oracle for the scalar z_k ; the simulation is perfect. The local forking lemma of Bellare–Dai–Li (ASIACRYPT 2019) reprograms H_{sig} at the forgery’s challenge query. The randomizers, being public, are stripped before extraction: when $\alpha \neq \perp$,

$$z_0 = z^* - c^* H_r(\alpha^*, \text{aux}^*), \quad z_1 = z^{**} - c^{**} H_r(\alpha^{**}, \text{aux}^{**}), \quad y_0 = \frac{z_0 - z_1}{c^* - c^{**}},$$

and the remaining challenge logarithms are extracted as in the FROST proof of Bellare–Crites–Komlo–Maller–Tessaro–Zhu (CRYPTO 2022; §2.7). The count closes the one-more game: $2q_s$ oracle queries against $2q_s + 1$ extracted logarithms.

What the theorem does not say. Four caveats frame every citation of this result. First, the paper proves *unforgeability only*: it contains no formal unlinkability definition and no unlinkability theorem; that property is argued at the ZIP layer (§3.4) by matching RedDSA’s distributions, under the protocol specification’s SURK-CMA framing. Second, corruptions are static. Third, the threshold-scheme definition assumes centralised (trusted-dealer) key generation, which the paper says “can [be] adapted” to a DKG — adapted, not proven. Fourth, the paper’s §5.1 sketches how a DKG-style contribution round could remove the trusted-for-privacy coordinator (all parties contribute to the randomizer, none learns it) at the cost of extra communication rounds; ZIP 312 does not adopt this. The cost of re-randomisation itself is one fixed-base scalar multiplication per participant and per coordinator; the paper’s Table 1 benchmarks (Pallas suite, Ryzen 9 5900X) put the signing overhead at 83% with 2 signers, 45% at 7, and 8% at 67.

3.4 ZIP 312: the specification

ZIP 312, “FROST for Spend Authorization Multisignatures” (Status Draft, Category Wallet, created 2022-08; owners Gouvêa, Komlo, Connolly; zcash/zips PR #662), specifies Re-Randomized FROST as a delta against RFC 9591. Its target property beyond unforgeability is *unlinkability*, which the ZIP defines as statistical independence of signatures from the signers’ long-term keys: for uniform randomizers and absent metadata leakage, it is “impossible to determine whether two transactions were generated by the same party”.

The modifications to RFC 9591. Five, and only five:

- *Randomizer generation.* A new helper `randomizer_generate(msg, commitment_list)`:
 $\text{draw } \text{rng_randomizer} \leftarrow \text{random_bytes}(\text{Ns}), \text{form}$
 $\text{randomizer_input} = \text{rng_randomizer} \parallel$
 $\text{encode_signing_package}(\text{commitment_list}, \text{msg}),$
 and return $H_R(\text{randomizer_input})$. Implementations MAY use another encoding “as long as all values (the message, and the identifier, binding nonce and hiding nonce for each participant) are unambiguously encoded”.
- *Round one* is unchanged.
- *Round two.* The Coordinator generates the randomizer and sends it to each signer “over a confidential and authenticated channel” — plain FROST requires authentication only — together with the message and the commitment list. Each participant then runs `sign` with sk_i + randomizer in place of its share and $\text{PK} + [\text{randomizer}] G$ in place of the group public key.
- *Aggregation* shifts the group key the same way: $\text{PK} += [\text{randomizer}] G$.
- *Share verification* shifts both keys: $\text{PK}_i += [\text{randomizer}] G$ and $\text{PK} += [\text{randomizer}] G$ in `verify_signature_share`.

The ZIP’s “randomizer” is the paper’s *effective* randomizer $\hat{\alpha}$: per the paper’s §7 implementation notes, `GenRand` and H_r are folded into the single hash call above, and the value distributed to signers is the already-hashed scalar.

The algebra, in RedDSA notation. The ZIP’s Rationale restates the correctness identity of (1) additively, and it is worth displaying because it is the entire consensus-facing content of the design. A valid RedDSA response must satisfy $z = r + c \cdot \text{sk}$. Each FROST share contributes

$$z_i = (\text{hiding_nonce}_i + \text{binding_nonce}_i \cdot \rho_i) + \lambda_i \cdot c \cdot \text{sk}_i,$$

and under re-randomisation the key terms become $c \cdot (\text{sk} + \text{randomizer})$ on the single-signer side and, summed over the coalition,

$$\sum_{i \in S} \lambda_i c (\text{sk}_i + \text{randomizer}) = c \left(\sum_{i \in S} \lambda_i \text{sk}_i + \text{randomizer} \cdot \sum_{i \in S} \lambda_i \right) = c (\text{sk} + \text{randomizer}),$$

since $\sum_i \lambda_i \text{sk}_i = \text{sk}$ by Shamir and $\sum_i \lambda_i = 1$ (the ZIP proves the latter in its `sum-lambda-proof` footnote, citing [math.SE 1325342](https://math.SE/1325342)). Every participant adds the same randomizer to its share; the weights-sum-to-one identity is exactly why the group secret shifts by α and not by $|S| \cdot \alpha$.

Who learns the randomizer. The threat model is the ZIP’s sharpest departure from RFC 9591. The Coordinator generates the randomizer and is “trusted with the privacy of the transaction (which includes the unlinkability property)” but *not* with unforgeability: “a rogue Coordinator ... should not be able to create signed transactions without the approval of `MIN_PARTICIPANTS` participants”. All key-share holders receive the randomizer and are likewise trusted with privacy only. The role assignment is natural: the Coordinator fits the transaction builder, who already creates the zero-knowledge proof — and the proof needs α as auxiliary input (step 4 of the flow in §3.1). The confidentiality requirement is not optional hygiene: anyone learning $\hat{\alpha}$ can un-randomise the on-chain key, $\text{ak} = \text{rk} - [\hat{\alpha}] G$, so the paper advises encrypting the randomizer “so that eavesdroppers can’t break unlinkability”. Non-requirements are stated with equal care: the coordinator-less variant of RFC 9591 §7.5 is unsupported;¹³ share holders can always link the signing operation to the eventual on-chain transaction; key generation is out of scope; network privacy is out of scope.

¹³ZIP 312’s reference for the variant mislabels it as RFC 9591 “Section 7.3: Removing the Coordinator Role”; the section is 7.5 (§7.3 is “Nonce Reuse Attacks”), and the reference’s URL anchor targets the correct section.

The message is a hash. The signed message is the SigHash, which conveys nothing about the transaction to a signer reading it. Signers MUST therefore check the SigHash against transaction data sent over the same channel, or recompute it themselves; the mechanism is out of the ZIP’s scope. This is the threshold echo of a rule the single-signer flow gets for free: a signer who cannot see what it signs authorises whatever the builder chose.

Unlinkability, and where it is proven. Nowhere, formally: the paper proves unforgeability only (§3.3), and the ZIP argues unlinkability in its Rationale by matching distributions — “randomizer_generate generates randomizer uniformly at random as required by RedDSA.GenRandom”, and the signing flow is compatible with RedDSA.RandomizePrivate, RedDSA.RandomizePublic, RedDSA.Sign, and RedDSA.Validate — while its Requirements defer the formal notion to the protocol specification’s SURK-CMA definition. The distribution-matching argument does connect to a proved statement: the *Ironwood Guide*’s §“RedPallas unforgeability and re-randomisation” shows honest uniform re-randomisations are distributed as fresh Schnorr keys. What no source supplies is a threshold-specific unlinkability theorem. Bin: the mechanism is *specified* (ZIP 312, Draft); its unlinkability guarantee rests on a distribution argument, not a theorem.

3.5 The ciphersuite FROST(Pallas, BLAKE2b-512)

ZIP 312 defines the ciphersuite in RFC 9591’s template. The group is Pallas with base point $G^{\text{SpendAuth}}$ as in protocol §5.4.7.1, of order p_{Vesta} ; RandomScalar is uniform in $[0, \text{Order}-1]$; SerializeElement is the point compression $\text{repr}_{\text{Pallas}}(P^*)$ in series notation) and DeserializeElement is $\text{abst}_{\text{Pallas}}$, erroring on $\perp$ and on the identity element; SerializeScalar is little-endian over 32 bytes and DeserializeScalar rejects values $\geq$ the order. The hash is BLAKE2b-512 with 16-byte personalisation strings and $N_h = 64$; scalar-valued hashes interpret the 64 output bytes as a little-endian integer reduced mod the order. Table 2 lists the oracles.

Oracle	Role	Personalisation
H_1	binding factor	FROST_RedPallasR
H_2	challenge	Zcash_RedPallasH
H_3	nonce	FROST_RedPallasN
H_4	message pre-hash	FROST_RedPallasM
H_5	commitment-list pre-hash	FROST_RedPallasC
H_R	randomizer	FROST_RedPallasA

Table 2: FROST(Pallas, BLAKE2b-512) hash personalisations (ZIP 312). The challenge hash reuses RedPallas’s own personalisation; the FROST additions use the FROST_ prefix.

The single load-bearing row is H_2 : ZIP 312 states it is “equivalent to $H^{\otimes}(m)$ ” of RedPallas, so the FROST challenge is the on-chain challenge, and signature verification for the suite is RedPallas validation itself. The ZIP’s compatibility rationale completes the argument that a FROST aggregate is a plain RedPallas signature: the (R, S) output split matches RedDSA.Validate’s parsing; FROST and Zcash use the same challenge input order (R , public key, message); and the fact that r and R are FROST-generated rather than produced by RedDSA.Sign’s nonce derivation is irrelevant to a verifier, which sees only the distribution of R . A parallel suite FROST(Jubjub, BLAKE2b-512) — personalisations $\text{FROST_RedJubjub}\{R, N, M, C, A\}$ and Zcash_RedJubjubH — serves Sapling spend authorisation; this volume works the Pallas suite and treats the Jubjub twin as its mechanical transliteration.

3.6 Deployed implementation: frost-rerandomized and reddsa

Claims in this subsection cite ZcashFoundation/frost at commit 2016e44 (2026-04-23, workspace version 3.0.0) and ZcashFoundation/reddsa at commit 3792daa (2026-05-22, crate version 0.5.2), examined 2026-07-15. The pinned commits are the published releases — tag frost-core/v3.0.0

(crates.io 2026-04-23) and reddsa 0.5.2 — so main-at-commit and the released crates coincide. ZIP 312 names reddsa as the reference implementation of both ciphersuites. A current-head check on 2026-08-30 found frost 0966bd1 and the same reddsa head 3792daa; the seven later FROST commits update the Rust edition, minimum compiler, dependencies, and CI, not the re-randomized protocol derived below.

The generic layer. Crate `frost-rerandomized` (`frost-rerandomized/src/lib.rs`) implements Construction 3.2 over any `frost-core` ciphersuite. Trait `impl Randomize<KeyPackage>` computes the randomised signing share as `signing_share + randomizer` and the randomised verifying share by adding the `randomizer_element`; struct `RandomizedParams` carries the randomizer (“also called alpha” in its documentation), `randomizer_element = [randomizer]G`, and `randomized_verifying_key = PK + randomizer_element`. The current flow differs instructively from a literal reading of the ZIP: the Coordinator sends a randomizer *seed*, not the randomizer, and each participant recomputes the randomizer from the seed and its own round-one commitments. The derivation, its divergence from the draft text — the message is no longer a hash input — and the reconciliation in flight in zips PR #895 are treated once, in §4.2.

The RedPallas suite. Module `src/frost/redpallas.rs` in `reddsa` implements the `frost-core` Ciphersuite trait for `PallasBlake2b512` (contextual ID “FROST(Pallas, BLAKE2b-512)”). Its `generator()` is `orchard::SpendAuth::basepoint()`, whose constant bytes are reproducible as `pallas::Point::hash_to_curve("z.cash:Orchard")(b"G")` — the protocol’s $G^{\text{SpendAuth}}$. Oracles $H_1/H_3/H_4/H_5$ carry the `FROST_RedPallas{R,N,M,C}` personalisations of Table 2; H_2 goes through the crate’s `HStar` default, `Zcash_RedPallasH`; the `RandomizedCiphersuite::hash_randomizer impl` uses `FROST_RedPallasA` (the ZIP’s H_R); scalar reduction is the 64-byte-wide `from_bytes_wide` (`src/hash.rs`), whose Pallas implementation delegates to `pasta’s from_uniform_bytes` (`src/orchard.rs`). Beyond ZIP 312, the crate defines two further personalisations — `HDKG = FROST_RedPallasD` and `HID = FROST_RedPallasI`, the DKG and identifier hashes — for key generation, which the ZIP does not specify at all.

Even-Y normalisation. One deployment detail appears in neither the paper nor the ZIP 312 body: the `reddsa` hooks `post_generate/post_dkg` normalise fresh key material to the even-Y representative the protocol specification requires of `ak`; the full mechanism is §4.1. The *randomised* key `rk` need not satisfy the even-Y condition — only `ak` does.

The RedPallas re-randomised API. Module `src/frost/redpallas/rerandomized.rs` re-exports the re-randomised flow for the suite — `sign_with_randomizer_seed()`, `aggregate()`, and `aggregate_custom()` — together with the type aliases `Randomizer` and `RandomizedParams` over `PallasBlake2b512`. The module README’s worked example runs the full trusted-dealer plus re-randomised signing flow and ends by verifying the aggregate under the `randomized_verifying_key()` of its `RandomizedParams` — the executable form of this chapter’s claim that a FROST aggregate is a plain RedPallas signature under `rk`.

3.7 Custody surfaces and classification

(a) Spend-authorisation custody. The primary surface: a t -of- n sharing of `ask` signing Orchard Actions (and, via the Jubjub twin suite, Sapling Spends). FROST replaces step 5 of the signer flow in §3.1; steps 1–4 move to the Coordinator-as-builder, who draws the randomizer, proves the Action statement with α as auxiliary input, and publishes `rk` — the circuit and consensus rules are untouched, per the *Ironwood Guide*, §“Authorisation of the spend: RedPallas”. Bin: *specified* (ZIP 312, Draft), with key generation explicitly out of the ZIP’s scope; DKG-based key generation for Zcash remains *designed but unspecified* (implemented in `frost-core`, standardised nowhere), as §2.8 already concluded for

the base protocol. Related anticipations elsewhere in the ZIP corpus: ZIP 316 requires FROST unified viewing key material to follow ZIPs 312 and 2005, and ZIP 271 anticipates lockbox disbursement to “a FROST address”, noting (as of May 2025) that ZIP 312 does not specify key generation.

(b) ZIP 2005 quantum spending key custody. ZIP 2005’s “Usage with FROST” section constrains threshold deployments of its quantum-recovery hierarchy: the key ak is derived jointly, by FROST DKG, from participants’ shares of ask , and the participants privately agree on a value sk from which nk , qsk , qk , and $rivk_{ext}$ derive. The full ceremony, its derivations, and its asymmetric threat model — every participant holds qsk , so the threshold property does not survive a quantum adversary — are developed once, in §4.7. Bin: the FROST-facing requirements are *specified* (ZIP 2005’s requirements list calls for full compatibility with ZIP 312 FROST, hardware wallets, and their combination); the combined DKG-plus-shared- sk key generation is *designed but unspecified* — no source specifies the protocol by which participants “privately agree” on sk .

(c) Crosslink finalizer keys. The *Crosslink Guide*, §“Finalizer keys”, bins threshold custody of finalizer keys as an *open problem*, and we adopt that bin. One precision for this volume: the applicable protocol there is *plain* FROST — the finalizer design has no key re-randomisation, so the natural suite is RFC 9591’s FROST(Ed25519, SHA-512), not either ZIP 312 suite. The prototype state and this observation are developed once, in §4.8.

(d) PCZT signing. The multiparty transaction flow that would host the Coordinator role is the PCZT of the *Wallet Guide*, §“Partially created transactions (PCZT)”: ZIP 374 (Partially Created Zcash Transaction Format, Draft, owner Jack Grigg) names threshold signing (“for example FROST”, per RFC 9591) in its Motivation among the multiparty use cases its Signer role must accommodate.

Classification. In the series’ three bins: base FROST and its five ciphersuites are *specified* at standards rigor (RFC 9591, §2.8); Re-Randomized FROST, its threat model, and both Zcash ciphersuites are *specified* by ZIP 312 at Draft status, with the unforgeability theorem resting on a preprint (ePrint 2024/436, no venue) and unlinkability on a distribution argument rather than a theorem; key generation — FROST DKG for Zcash, and ZIP 2005’s shared- sk agreement — is *designed but unspecified*; threshold custody of Crosslink finalizer keys is an *open problem*. Deployed-behaviour claims (seed-based randomizer derivation, even- Y normalisation, the extra DKG personalisations) are implementation facts of the pinned commits, cited as such.

4 Deployment and open questions

The preceding sections dealt in designs and proofs; this one deals in what ships. We pin the exact code path a Zcash threshold-custody deployment exercises today, reconcile it against the draft ZIP text where the two have diverged, then walk the three custody surfaces this volume targets — spend authorisation through PCZT, the ZIP-2005 quantum spending key, and Crosslink finalizer keys — and close by sorting every artefact into the series’ three bins (*specified*, *designed-but-unspecified*, *open problem*), each with the concrete trigger on which its classification must be revisited.

4.1 The deployed stack

Two Zcash Foundation repositories carry the implementation. The `frost` workspace (commit 2016e44ba4, 2026-04-23, tagged `frost-core/v3.0.0`; workspace version 3.0.0, MSRV 1.81) provides the generic `frost-core`, the RFC 9591 ciphersuite crates `frost-ristretto255`, `-ed25519`, `-ed448`, `-p256`, `-secp256k1`, and `-secp256k1-tr`, and the generic re-randomisation layer `frost-rerandomized`. Beyond RFC 9591’s two-round protocol the workspace implements trusted-dealer key generation (the RFC’s appendix), the Pedersen-with-knowledge-proof DKG “as specified

in the original paper” (ePrint 2020/852, §2.2 — not in the RFC), Repairable-Threshold-Scheme share recovery (ePrint 2017/1155), share refresh, and Re-Randomized FROST (ePrint 2024/436, §3). The README declares the crates “considered stable and feature complete”, with SemVer guarantees.

The `frost-rerandomized` crate is `no_std`; its `Randomize` implementations add the randomizer group element to every verifying share of a `PublicKeyPackage` and the randomizer scalar to the signing share of a `KeyPackage`, whose `randomize` documentation warns: “You MUST NOT reuse the randomized key package for more than one signing.” Its README notes that “the main ciphersuite crates do not re-expose the rerandomization functions... The exceptions are the Zcash ciphersuites in `reddsa` which do expose the randomized functionality” — the re-randomised layer exists, in shipped form, for Zcash alone.

The `reddsa` crate (commit 3792daa95e, 2026-05-22, version 0.5.2) supplies those ciphersuites: its feature `frost = ["frost-rerandomized", "alloc"]` pulls `frost-rerandomized` 3.0.0, and its modules `reddsa::frost::redpallas` and `reddsa::frost::redjubjub` implement the two ZIP-312 ciphersuites (`PallasBlake2b512`, `JubjubBlake2b512`), each with a `keys` module (trusted-dealer, DKG, repairable) and a `rerandomized` submodule re-exporting `sign_with_randomizer_seed`, `aggregate`, `aggregate_custom`, `Randomizer`, and `RandomizedParams`. Function `hash_randomizer` instantiates ZIP 312’s H_R as $H^\circledast$ (the code’s `HStar`) under the personalisation `FROST_RedPallasA` (respectively `FROST_RedJubjubA`), matching §3.5.

The `RedPallas` module additionally enforces the even- y requirement that ZIP 2005’s amended protocol-spec §4.2.3 places on `ak` (§4.7): trait `keys::EvenY` negates the group verifying key, every verifying share, the signing shares, and the VSS commitment coefficients whenever the serialised key has $\bar{y} = 1$ (the code’s evenness test is `serialized[31] & 0x80 == 0`), and the `post_generate` and `post_dkg` ciphersuite hooks call `into_even_y` on fresh key material, so trusted-dealer and DKG outputs alike land on the even- y representative. Negation commutes with Shamir sharing because it is linear. The `RedJubjub` module carries no such machinery; the constraint is Pallas/Orchard-specific.

4.2 The randomizer flow: draft ZIP against shipped crate

ZIP 312 (“FROST for Spend Authorization Multisignatures”; owners Gouvea, Komlo, Connolly; Status Draft; created 2022-08) specifies, at zips commit 6961098410, the randomizer generation

```
signing_package_enc = encode_signing_package(commitment_list,msg),
randomizer = HR(random_bytes(Ns) || signing_package_enc),
```

with H_R for `FROST(Pallas, BLAKE2b-512)` being `BLAKE2b-512` under the personalisation `FROST_RedPallasA`, output reduced to a Pallas scalar (an element of $\mathbb{F}_{p_{\text{Vesta}}}$); H_2 is `Zcash_RedPallasH`, i.e. exactly `RedPallas`’s $H^\circledast$ (§3.5). For Jubjub the personalisations are `FROST_RedJubjub{R,N,M,C,A}` and `Zcash_RedJubjubH`, with base point G^{Sapling} . Round 2 then proceeds as plain FROST with three substitutions,

```
ski ← ski + randomizer,      PKi ← PKi + ScalarBaseMult(randomizer),
group_public_key ← group_public_key + ScalarBaseMult(randomizer),
```

the second applying in share verification. The Coordinator sends the randomizer over a “confidential and authenticated channel” — strictly stronger than plain FROST’s authenticated-only channel assumption, because the randomizer links the ceremony to the on-chain `rk`.

The deployed crate has moved past this text. Crate `frost-rerandomized` 3.0.0 deprecates `Randomizer::new(rng, signing_package)` — the exact draft-ZIP flow above, message included — in favour of `Randomizer::new_from_commitments`: the Coordinator draws an N_s -byte `randomizer_seed`, sends the `seed`, and each participant regenerates

```
randomizer = hash_randomizer(seed || encode_group_commitments(commitments))
```

locally via `regenerate_from_seed_and_commitments`. The message is no longer an input, and — the crate’s stated rationale — participants no longer need to fully trust the Coordinator’s RNG, since their own round-one commitments enter the hash. In the paper’s terms (ePrint 2024/436, Definition 5 and Remark 1), the commitment encoding plays the role of the optional contributory string *aux*, which models “the transcript of some external protocol”; the paper’s own §7 implementation notes describe the older design, with the message still included, in the same terms.

The gap between draft and crate is being closed in zips PR #895 (“[ZIP 312] Specify key generation, change randomizer handling”; opened August 2024, 21 commits, still open as of 2026-07-15, awaiting review by *daira* and *nuttycom*). It adds a key-generation specification, revises the randomizer handling (the randomizer “couldn’t possibly be generated using the message as an input”), integrates the ZIP-2005 qsk material, renames the scheme consistently to “Re-Randomized FROST”, and adds *Daira-Emma Hopwood* as a ZIP owner. Its commit history also retires the draft’s serialisation reference — the ZF FROST book’s moving-target serialisation-format page — in favour of RFC 9591’s serialisation function. Until it merges, this volume presents the crate flow — message omission included — as the observed fact of the pinned commit, draft lag rather than a conformance defect, and the ZIP text as in flight.

The trust geometry is worth stating exactly. The paper’s §5.1 holds that the central randomizer-choosing party is “trusted with preserving privacy of the scheme” but “not trusted with security”, and that removing it would require a DKG-style contributory sub-protocol at extra rounds and complexity. ZIP 312’s threat model matches: the Coordinator is trusted with transaction privacy and unlinkability but cannot forge without `MIN_PARTICIPANTS` approvals, and every key-share holder is trusted with transaction privacy. Non-requirements are equally explicit: ZIP 312 does not remove the Coordinator role, does not (at this commit) provide key generation, and places network privacy out of scope.

4.3 PCZT as the integration surface

ZIP 374’s Motivation names “multiparty transactions — threshold signing (for example FROST [RFC 9591])” among the problems the PCZT format solves; the *Wallet Guide*, §“Partially created transactions (PCZT)”, documents the format and its role graph. The insertion point for FROST is the Signer role (*Wallet Guide*, §“Signing”): to sign an Orchard spend the action’s re-randomisation scalar α must be present, the local-key path computes $\text{rsk} := \text{ask} + \alpha$ — the re-randomisation of the *Ironwood Guide*, §“Authorisation of the spend: RedPallas” — and checks $\text{VerificationKey}(\text{rsk}) = \text{rk}$, and the *external* path, `apply_orchard_signature`, accepts a signature if and only if it verifies under the action’s rk over the shielded sighash (`InvalidExternalSignature` otherwise). A FROST-aggregated RedPallas signature is exactly such an external signature; for it to verify, the Coordinator must use the PCZT action’s α as the randomizer, so that the re-randomised group key `RandPK` produces equals that action’s rk .

PCZT v2 has since shipped in `pczt` 0.8.0. Current `pczt` 0.9.3 carries separate Orchard and Ironwood bundles and tags an externally produced `SpendAuthSignature` with its value pool and action index; `apply_orchard_spend_auth_signature` dispatches to the corresponding bundle and verifies under that action’s rk . The invariant in the preceding paragraph — one shared shielded sighash and the action’s own α — is unchanged for v6 Ironwood spends (`librustzcash` 91f448b5, `pczt/src/roles/signer/mod.rs`).

The end-to-end flow has been demonstrated (ZF FROST book, `zcash/devtool-demo.md`):

1. `zcash-devtool pczt create` produces the PCZT;
2. the `zcash-sign` tool (from `ZcashFoundation/frost-zcash-demo`, now `frost-tools`) prints a SIGHASH and a Randomizer from the transaction plan;
3. `frost-client` runs the FROST protocol over `frostd` with that SIGHASH as the message and that randomizer;
4. `zcash-sign sign` writes the aggregated signature into `frost_pczt.signed`;

5. `zcash-devtool pczt combine` merges the signed and proven PCZTs;
6. `zcash-devtool pczt send` broadcasts.

Here `zcash-devtool` builds on `zcash_keys` with the features "orchard", "unstable", and "unstable-frost". The `unstable-frost` feature (`= ["orchard"]`) exposes the constructor `UnifiedFullViewingKey::from_orchard_fvk` — a UFVK from an Orchard FVK alone, since a FROST wallet has no unified spending key — and the `zcash_keys` changelog (0.3.0, 2024-08-19) frames the flag as existing “to temporarily expose API features that are needed specifically when creating FROST threshold signatures”, to be removed “once key derivation for FROST has been fully specified and implemented”. Repository `librustzcash` itself, that is, classifies FROST key derivation as not fully specified.

One check remains outside every specification: ZIP 312 requires that signers “MUST check that the given SIGHASH matches the data sent from the Coordinator, or compute the SIGHASH themselves”, and explicitly leaves the mechanism out of scope. In the demonstrated flow the SIGHASH travels from `zcash-sign` to the participants through `frost-client`; by what mechanism a share-holding participant gains independent confidence in it is unspecified.

4.4 Wallet-integration constraints

The ZF FROST book’s technical-details chapter records the constraints a FROST-capable Zcash wallet inherits. FROST cannot cover transparent inputs, which authorise by ECDSA; the book accordingly suggests supporting Orchard only (“Orchard is simpler to handle”). With a DKG-generated `ak`, the remaining Orchard key components derive from `ak` plus either an `sk` used only for `nk` and `rivk`, or independently generated `nk` and `rivk`. A seed phrase alone can never restore a FROST wallet: restoration needs the key share, all verifying shares, the participant identifiers, and the FVK components, so share backup is a JSON-file affair and share recovery falls to the Repairable Threshold Scheme (§4.1).

The Foundation’s own status report (“The State of FROST for Zcash”, 2025-05-20) names wallet integration the missing piece (“The missing piece is for wallets to integrate FROST”): no mainstream wallet ships FROST, and “technical-inclined users can use FROST with Zcash today using our `frost-client` command line tool”. The same post lists the “lack of a standardized key generation specification for FROST” as the interoperability limiter, flags metadata protection (e.g. Tor) as needing work, and offers ZF as a candidate operator of a production `frostd`.

4.5 Transport: `frostd` and `frost-client`

The reference transport is `frostd`, a JSON-HTTP session server: clients authenticate with key pairs, the Coordinator creates sessions naming the participants’ public keys, and all FROST messages are end-to-end encrypted, so the server itself is untrusted. The CLI `frost-client` drives it; both live in `ZcashFoundation/frost-tools` (renamed from `frost-zcash-demo`). Least Authority, as Zcash Ecosystem Security Lead, audited `frost-client` and `frostd` (announcement 2025-05-01) with no high-severity findings, all findings addressed on main. No ZIP specifies this transport; ZIP 312 places network privacy out of scope (§4.2).

4.6 Audit coverage, precisely

Two audits exist, and neither covers the code path Zcash custody would run. NCC Group audited the v0.6.0 release (commit 5fa17ed) of `frost-core`, `frost-ed25519`, `frost-ed448`, `frost-p256`, `frost-secp256k1`, and `frost-ristretto255` — including trusted-dealer and DKG key generation and FROST signing — with all findings addressed and reviewed by NCC (public report 2023-10-23). The repository README is explicit: “This does not include `frost-secp256k1-tr` and rerandomized FROST.” Earlier, the 2021 “Audit of FROST implementation” (JP Aumasson and Adrien Hamelink, Taurus; Omer Shlomovits, ZenGo; dated 2021-03-23) covered the pre-rewrite implementation in the

RedJubjub repository’s `src/frost.rs` (≈ 404 lines at commit 2ebc08f, 2021-02-25) — two-round FROST with a trusted dealer only, RedJubjub only — concluding “We did not find any major security issue.”

The composition, therefore: `frost-rerandomized` plus the `reddsa` RedPallas/RedJubjub ciphersuites — the exact stack of §4.1 — has no completed third-party audit, and the only audit ever to touch a Zcash FROST ciphersuite predates the `frost-core` rewrite.

4.7 Custody of the ZIP-2005 quantum spending key

ZIP 2005 (“Ironwood Quantum Recoverability”; owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; activation at the NU6.3 activation height) is the second custody surface. Its “Usage with FROST” section derives ak jointly, from the participants’ shares of ask , via FROST DKG, and then constrains the ceremony: participants “MUST privately agree on a value sk , and then use it with `use_qsk = true` to derive nk , qsk , qk , and (using the ak output by the DKG protocol) $rivk_ext$ ”; the protocol MUST ensure all participants obtain the same ak , nk , and $rivk_ext$. The derivations, verbatim from the ZIP:

$$\begin{aligned} H^{qsk}(sk) &= \text{truncate}_{32}(\text{PRFexpand}_{sk}([0x0C])), \\ H^{qk}(qsk) &= \text{BLAKE3.derive_key}(\text{"Zcash ZIP 2005 qk-derivation v1"}, qsk, 32), \\ H^{rivk_ext}_{qk}(ak, nk) &= \text{ToScalar}^{\text{Orchard}}(\text{PRFexpand}_{qk}([0x0D] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))), \end{aligned}$$

with the `PRFexpand` domain-separator bytes `0x0A–0x0D` registered by ZIP 2005. In the `use_qsk = true` branch of the amended protocol-spec §4.2.3, the wallet obtains ak^p “by any suitably secure method that ensures the last bit of $\text{repr}_p(ak^p)$ is 0” — the requirement the `reddsa` EvenY machinery of §4.1 enforces — and sets $ak = \text{Extract}_p(ak^p)$, $qsk = H^{qsk}(sk)$, $qk = H^{qk}(qsk)$, and $rivk = H^{rivk_ext}_{qk}(ak, nk)$. The ZIP’s list of admissible generation methods for ak^p explicitly includes “FROST distributed key generation [zip-0312-key-generation] in which the corresponding spend-authorization material is t -of- n shared among the participants”.

The threat model shifts accordingly. The host wallet in a multi-signer FROST setup is the ZIP-312 Coordinator; it holds nk , ak , and qk . With FROST and hardware wallets, spend authorisation breaks only by (1) possession of qsk *plus* finding discrete logarithms on Pallas — and “the adversary is an authorized FROST signer — they hold a copy of qsk by construction” is the ZIP’s variant 1.a — or (2) breaking RedDSA, FROST, the DKG, or the proving system. Each participant MUST treat qsk as a secret held at least as securely and reliably as ask : it is not needed until the Recovery Protocol, but losing it can lose funds once the current Orchard protocol is disabled. A quantum adversary holding only qsk — which every participant holds — may steal funds, so the ZIP RECOMMENDS that once a fully post-quantum threshold multisignature protocol exists, all FROST-held funds migrate to its pool. FROST’s key advantage — signing with shares without ever reconstructing ask — is retained through the Recovery Protocol, which continues to require a RedDSA signature.

One tension is unresolved. FROST DKG exists precisely to avoid a commonly-known secret, yet the ceremony above requires the participants to “privately agree on a value sk ” from which nk and qsk derive — and neither ZIP 2005 nor the crates give a protocol for that agreement. ZIP 2005’s Deployment section points at PR #895 (“There is no significant existing deployment of FROST, so we can write this into ZIP 312 from the start”), so the resolution is expected there: as of 2026-07-15 the PR’s commit history drafts a contributory sk -generation section (2026-03-05), still unmerged, so the shared- sk agreement stays *designed but unspecified* with #895’s merge as its trigger.

4.8 Crosslink finalizer keys

The third surface is prospective only. The *Crosslink Guide*, §“Finalizer keys”, records that the prototype uses single ed25519 keys per finalizer (ed25519-zebra), with 76-byte votes signed by appending a 64-byte ed25519 signature over those 76 bytes, and that no FROST or threshold-signing design for

finalizer keys is sourced anywhere in the Crosslink corpus (negative search over tfl-book, crosslink-protocol-guide, crosslink-spec, and zebra-crosslink, dated 2026-07-15 in that guide); key rotation, revocation, threshold custody, and initial-roster formation are classified open problems in every design source.

We record one observation of our own, clearly marked as such: finalizer votes are plain ed25519, with no key re-randomisation, so the fitting ciphersuite would be RFC 9591’s FROST(Ed25519, SHA-512) via the NCC-audited `frost-ed25519` crate, with no re-randomisation layer — finalizer keys are deliberately long-lived roster identities, not unlinkable one-time keys, so the entire apparatus of §3 is unnecessary there. No published design says this; it is this volume’s inference from the vote format and the audit table.

4.9 Classification and revisit triggers

Per the series’ three-way scheme:

- *Specified.* FROST as in RFC 9591 (§2.8); Re-Randomized FROST for spend-authorisation custody, per ZIP 312 at Draft status — PR #895’s in-flight randomizer and key-generation changes are its named revisit trigger and do not move the bin (§4.2); the `frost-core/frost-rerandomized/reddsa` API surface at the pinned versions (§4.1); the PCZT Signer external-signature path (§4.3).
- *Designed-but-unspecified.* FROST key generation for Zcash (out of scope at the pinned ZIP-312 commit; the crates implement the PedPoP DKG of ePrint 2020/852, but no ZIP specifies it); the ZIP-2005 `use_qsk` FROST key-generation constraints (ZIP Proposed, referencing the unmerged #895; the `sk-agreement` step has no protocol, §4.7); the transport (`frostd` exists and is audited, but no ZIP covers it, §4.5).
- *Open problem.* Crosslink finalizer threshold custody (§4.8); post-quantum threshold signing for the `qsk-era` migration (§4.7); hardware-wallet FROST share custody — ZIP 2005’s threat model assumes hardware-wallet FROST signers, but no device implements a share signer.

Two former revisit triggers have fired without changing these bins. NU6.3 is active, but `zcash_keys` still gates `from_orchard_fvk` behind `unstable-frost` pending a specified FROST derivation. PCZT v2 is released, and the current signer preserves the α and external-signature surface for Ironwood as described in §4.3. The remaining classifications expire on the following identifiable events, in rough order of expected impact:

- *PR #895 merges.* The randomizer flow, the key-generation specification, the `sk-agreement` protocol, and the ZIP-312/ZIP-2005 integration all re-derive from the merged text; the serialisation reference it pins replaces the ZF-FROST-book moving target (§4.2).
- *An audit of the re-randomised stack is announced.* The “no completed third-party audit” claim of §4.6 must be rechecked immediately before any printing in any case.
- *A finalizer key-management design is published.* The Crosslink bin moves, and the `frost-ed25519` observation is either confirmed or discarded (§4.8).
- *A post-quantum threshold multisignature protocol exists.* ZIP 2005’s RECOMMENDED migration of FROST-held funds becomes actionable (§4.7).

Part XV

PQ Guide: Post-quantum Zcash: the cross-layer assumption inventory, the reversibility divide, and the migration surface

Abstract

This volume of *The Zcash Arboretum* assembles the post-quantum picture that no single layer can see. It inventories every deployed cryptographic surface — the shielded core, the transparent layer, the Sprout and Sapling remnants, proof of work, and the frontier protocols — against the two quantum algorithms that matter, and sorts every exposure along one axis: *retroactive* (damage accruing now, in recorded data) against *prospective* (exploitable only once a quantum adversary exists, and mitigable by switching off in time). It quantifies quantum mining through the *Consensus Guide*’s own tables, situates ZIP-2005’s recoverability-not-resistance strategy and its stated gaps, computes the migration arithmetic against the published NIST standards, and registers the distance between public statements and the repositories — which contain, at the pinned commits, no deployed post-quantum cryptography at all. The Orchard-core analysis is the *Ironwood Guide*’s; this volume cites it and extends the frame to everything else.

1 Scope, method, and the two clocks

Every volume below this one proves what an assumption buys while it holds. This volume asks the cross-layer question: what happens to each purchase when the assumption falls to a quantum adversary — and, since the falls are not simultaneous in their *effects*, in what order the damage arrives. The treatment is an inventory and a synthesis, not a re-derivation: the deep analysis of the Orchard core is the *Ironwood Guide*’s (§“Post-quantum security of the Orchard protocol”), the cryptographic definitions are the *Crypto Guide*’s, and this volume constructs only what no lower volume owns — the cross-layer inventory, the quantum-mining analysis, and the migration arithmetic.

Two quantum algorithms carry the whole subject. Shor’s algorithm solves the discrete-logarithm problem in polynomial time in *any* group, elliptic curves included, taking DLP, CDH, and DDH with it (*Crypto Guide*, §“The quantum caveat: Shor’s algorithm”); every discrete-log-based object in Zcash — which is nearly every asymmetric object — fails completely against it. Grover’s algorithm gives a quadratic speedup on unstructured search and nothing more: symmetric primitives and hashes retain half their security exponent, a margin the deployed 256-bit parameters absorb. The standing premise, inherited from the same section, is that no cryptographically relevant quantum computer (CRQC) exists today; nothing in this volume predicts when one will, and every claim is conditioned explicitly on that event.

1.1 The reversibility axis

One distinction organises everything, and the series already owns its two halves. The *Crypto Guide*’s remark “Two regimes, two failure modes” (§“Incompatibility of perfect hiding and perfect binding”) observes the asymmetry: a computationally *binding* commitment leaves a future solver able to change a committed value, while a computationally *hiding* one loses everlasting secrecy the day the assumption falls, for everything ever recorded. Which direction each failure *points in time* is the *Ironwood Guide*’s operational recasting, and it turns the asymmetry into a test: a break is *prospective* when its exploitation requires a validator to *accept* a forged object, because acceptance happens at the current chain height — disabling the protocol first removes every future acceptance event, so forking in time is a complete mitigation; a break is *retroactive* when recorded data itself becomes exploitable, and no consensus change can help, because the adversary’s copy of the chain is beyond the protocol’s reach.

Along that axis, the deployed protocol has exactly one retroactive break — harvest-now-decrypt-later against note encryption — and a stack of prospective ones; Section 2 walks the full inventory. Figure 1 draws the divide.

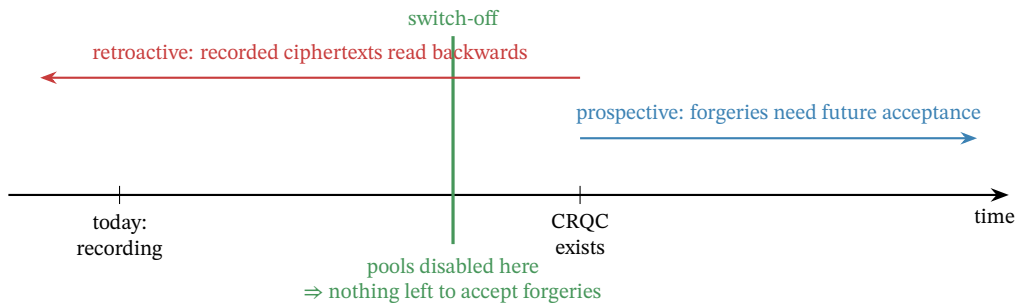


Figure 1: The reversibility divide. A prospective break (blue) needs a validator to accept a forged object after a CRQC exists; switching the vulnerable protocols off first (green) leaves it nothing to act on. The retroactive channel (red) runs the other way: ciphertexts recorded today are read by tomorrow’s adversary, and no protocol change reaches the adversary’s copy of the chain.

1.2 Sources and their standing

Table 1 lists the sources. Normative weight is ZIP-2005’s (Proposed) and the cited Final ZIPs’; the NIST standards are normative for their own schemes and serve here only as the published size and security baselines; everything else is analysis or code. Worked numbers are computed by script (`.wip/pq-drafts/compute.py`, whose output is the source of every numeric claim here); deployed-behaviour claims cite crate, file, and line at the pinned commits.

Source	Role	Pin
ZIP-2005 (Ironwood Quantum Recoverability)	Proposed	zips 69610984
ZIPs 213, 229, 258, 326; ZIP-209	normative/draft	same pin
FIPS 203/204/205 (ML-KEM, ML-DSA, SLH-DSA)	published 2024-08-13	retrieved 2026-08-19
Ben-Sasson et al., ePrint 2018/046; ethSTARK 2021/582	analysis	retrieved 2026-08-19
Aggarwal et al., arXiv:1710.10377	analysis (2017)	retrieved 2026-08-19
zebra	deployed validator	8e9ff3b2c
reddsa 0.5.1 / halo2_proofs 0.3.2 / orchard 0.15.3	deployed crates	Cargo.lock
zebra-crosslink	prototype	6d02a1b8

Table 1: Sources. The quantum-relevant repository state was swept at these pins: the zips repository and the protocol specification contain zero occurrences of ML-KEM, Kyber, ML-DSA, Dilithium, SPHINCS, Falcon, or “lattice”; the sole post-quantum bytes in any ecosystem checkout are an inert, feature-gated transitive dependency (Section 6.3).

2 The inventory: every deployed surface, sorted

Table 2 is the volume’s spine: every cryptographic surface a mainnet validator enforces today (with the one frontier prototype, Crosslink, marked as such), its assumption, what a CRQC does to it, and on which side of the divide the damage falls. The Orchard-core rows compress the *Ironwood Guide*’s five-subsection analysis and the committed inventory it rests on; the remaining rows are this volume’s. The subsections walk the table by failure mode.

Surface	Assumption	Consequence of a CRQC	Side
Note encryption (all shielded pools)	DH on the pool’s curve	decrypt every recorded ciphertext to a retro <i>known address</i> : amounts, memos, full receiving history	
Sinsemilla commitments (NoteCommit, Commit ^{ivk} , MerkleCRH)	Pallas DLP (binding)	equivocate openings: forge notes, fake Merkle paths, Spendability attacks	prosp.
Value commitments + binding signature	Pallas DLP	reopen cv to any value; forge balance — silent inflation, bounded only by the ZIP-209 turnstiles	prosp.
RedPallas spend authorisation	Pallas DLP (ROM)	forge spend-auth from public rk; with forged proofs, theft	prosp.
Halo 2 IPA proof soundness	Vesta DLP (ROM)	prove arbitrary false Action statements — the most concentrated point	prosp.
Sapling stack (Groth16, RedJubjub, Jubjub DH)	BLS12-381 / Jubjub DLP	one logarithm forges the whole pool’s integrity	prosp.
Sprout stack (Groth16, Ed25519 JoinSplitSig)	BLS12-381 / Curve25519 DLP	statements and signatures of JoinSplits (Sprout’s shielded transfers, spending and creating notes under one proof) forgeable	prosp.
Transparent scripts (ECDSA, secp256k1)	secp256k1 DLP	spend any output whose public key is exposed	prosp.
Crosslink finalizer votes (prototype; ed25519)	Curve25519 DLP	forge roster signatures — the finality premise itself (§4)	prosp.
Proof of work (Equihash + SHA-256d filter)	generic search	Grover: quadratic speedup, heavily damped (§3)	neither
BLAKE2b backbone, ZIP-32 tree, Poseidon-as-PRF	symmetric / hash	security exponent halves; parameters absorb it	survives

Table 2: The deployed surfaces under a CRQC. “retro” = retroactive (recorded data becomes exploitable; no protocol change helps); “prosp.” = prospective (requires future validator acceptance; switch-off before a CRQC is a complete mitigation, with the residues of Section 5). Orchard-core rows: *Ironwood Guide*, §“Post-quantum security of the Orchard protocol”.

2.1 The retroactive wound

One row of Table 2 is red, and it is the one accruing damage today. Every shielded output ever recorded carries an ephemeral key and a ciphertext; for the Orchard pools, one Shor logarithm per *address* yields the incoming viewing key ($\text{ivk} = \log_{g_d} \text{pk}_d$), after which the adversary runs the recipient’s own trial-decryption loop over the whole recorded chain — amounts, memos, and the complete receiving history of that address, forever. The full anatomy, and the two structural limits — the attack is keyed by the address, and it confers viewing rather than spending — are the *Ironwood Guide*’s (§“The retroactive break: harvest now, decrypt later against note encryption”). Three extensions belong to the cross-layer frame:

- *Every pool, not one.* Sapling and Sprout note encryption are equally DH-keyed and their ciphertexts equally recorded; ZIP-2005 itself states that note encryption for the “Ironwood, Orchard, Sapling, and Sprout pools” is subject to harvest-now-decrypt-later given ciphertext and receiver address. The corpus is the union of every shielded output since 2016, growing with each block — including Ironwood’s, whose recoverability machinery deliberately excludes privacy (Section 5).
- *Where the address is already public, so is the history.* Shielded coinbase outputs must decrypt under the all-zero outgoing viewing key (*Ironwood Guide*, §“How value enters the pool”), so their contents were never confidential; and ZIP-213 itself advises miners who rely on the “post-quantum privacy” of an address to use a coinbase-specific one — the ZIP’s only pre-2005 use of the phrase.
- *What stays dark even then.* Without the address, the recorded chain leaks nothing to an unbounded adversary: value commitments are perfectly hiding, the spend-auth randomiser masks

ask information-theoretically, and proof transcripts are statistically simulatable (*Ironwood Guide*, §“Prospective breaks: the discrete-logarithm integrity stack”, closing paragraph; the underlying theorems are the *Crypto Guide*’s and *Halo 2 Guide*’s). The retroactive channel is address-shaped, exactly.

2.2 The prospective integrity stack

Every other broken row fails forward, and for the Orchard core the *Ironwood Guide* already orders the dominoes — Sinsemilla binding, value commitments, RedPallas, and the IPA’s knowledge soundness, the last “the single most concentrated failure point” because every other item’s exploitation route runs through it. The cross-layer frame adds four members the core analysis does not cover.

The transparent layer. Script signatures are ECDSA over secp256k1 (specification §“Signature”), verified in deployment by the retired zcashd’s own C++ interpreter bundled as a crate (libzcash_script, depend/zcash/src/pubkey.cpp:23--42, driven from zebra-script/src/lib.rs:62--76). Exposure is per-output and public-key-shaped: an address spent from before has its key on chain — forgeable the day a CRQC exists, no acceptance of any new object required beyond an ordinary transaction; an unspent P2PKH or P2SH output exposes only a hash until its first spend, leaving a between-broadcast-and-confirmation window thereafter (ZIP-2005’s own analysis, which also notes P2PK and bare multisig were never produced by Zcash wallets). Two special cases sharpen the picture. The deferred lockbox is secured by consensus accounting alone — the pool has no output, no address, and no key, so a CRQC finds nothing to steal there; only a consensus change moves it. The 78,750 ZEC already disbursed under ZIP-271, by contrast, sit in a 2-of-3 P2SH multisig whose extended public keys are published in the ZIP itself, so hash-hiding provides those particular funds no pre-spend quantum cover.

The Sprout remnant. Version-4 transactions remain consensus-valid — ZIP-2003 (disallowing them) is Draft and absent from the NU6.3 upgrade list — so zebra still runs Groth16 JoinSplit verification and Ed25519 JoinSplitSig checks (zebra-consensus/src/transaction.rs:1230, 1265--1268; ed25519-zebra). ZIP-2005 states the consequence plainly: the broken proof system suffices to forge JoinSplit statements, and both Sprout signature layers are forgeable. One logarithm on BLS12-381 undoes Sprout and Sapling integrity alike; one on Pallas/Vesta undoes Orchard and Ironwood.

The Sapling pool. Groth16 over BLS12-381 with RedJubjub authorisation and Jubjub key agreement — the same stack shape as Orchard with every assumption transplanted, and deliberately *excluded* from ZIP-2005’s recoverability (its stated analysis-economy decision). Sapling funds therefore face the harder edge of the strategy: prospective integrity failure like Orchard’s, but no recovery door — migration before switch-off is the only remedy the design offers.

The finality prototype. Crosslink’s roster votes are plain ed25519 signatures under long-lived finalizer keys (Section 4); a CRQC forges the very signatures whose aggregation is supposed to make history *assuredly* final.

2.3 The surviving backbone

What remains standing is exactly what ZIP-2005 builds on. The BLAKE2b expansion registry, the hardened-only ZIP-32 key tree (whose ≥ 256 -bit seed requirement the ZIP text motivates by quantum analysis), Poseidon in its PRF role, and the consensus hashes — SHA-256d in the difficulty filter, BLAKE2b in the ZIP-221 history tree — face only generic speedups: Grover halves the security exponent, $2^{256} \rightarrow 2^{128}$, a margin the parameters absorb, and the sharper quantum collision algorithms

remain memory-bound curiosities (the *Ironwood Guide*’s §“Primitives facing only generic speedups” carries the careful statement, including its best-known-not-provably-best flag). The load-bearing consequence, stated there and relied on everywhere in Section 5: because key derivation is purely symmetric, *ownership remains provable after the assumption that made it enforceable has fallen* — the seed still derives the keys; what is lost is only the DL-based machinery for exercising them.

3 Quantum mining

The one broken row that is neither retroactive nor prospective is proof of work, because Grover attacks it and Grover only halves an exponent. No volume of the series analyses this; the *Consensus Guide* builds the double-spend model on a fixed hashrate share q but never asks what a quantum miner does to q . This section closes that gap, reusing that volume’s machinery.

The gated quantity is a hash lottery: a block is valid when the SHA-256d of its header falls below the target (*Consensus Guide*, §“Equihash and the memoryless model”; the difficulty filter is deployed at `zebra-consensus/src/block/check.rs:112--139` over the SHA-256d header hash). That filter is unstructured search, exactly Grover’s domain — with three deflations that the honest statement must carry, and a fourth specific to Zcash: the candidates a classical Equihash miner feeds the filter come from Wagner-structured solver runs, not brute force, so a quantum miner would have to embed that structured search in its Grover oracle to gain on the whole pipeline rather than on the filter alone.

Remark 3.1 (The three deflations). Aggarwal, Brennen, Lee, Santha, and Tomamichel (arXiv:1710.10377, 2017) state them (published in *Ledger*, 2018). *Quadratic, not exponential*: Grover “can perform the hashcash [proof of work] by performing quadratically fewer hashes”, so N classical trials become $\sqrt{N}$ — a square root, not a collapse. *Gate speed*: at 2017 difficulty “the extreme speed of current specialized ASIC hardware ... coupled with much slower projected gate speeds for current quantum architectures, essentially negates this quadratic speedup ... giving quantum computers no advantage”; only a hypothesised 100 GHz gate speed would let a quantum miner “solve the [proof of work] about 100 times faster”. *Parallelisation*: Grover parallelises badly — across d processors the search *time* falls only as $\sqrt{d}$, against the linear scaling a classical pool enjoys, so a quantum miner cannot buy back the speed by fanning out. The date matters: these are 2017 projections, cited as the structural shape of the advantage, not a forecast.

The model needs no rebuilding, only a reading. A quantum miner is one whose search is faster per unit hardware; fold the advantage into an effective speed multiplier g on the attacker’s raw hashing, and the *Consensus Guide*’s hashrate share becomes

$$q_{\text{eff}}(f, g) = \frac{gf}{gf + (1 - f)} \quad \text{for a raw share } f \text{ of the total hashing rate}$$

(the honest remainder being $1 - f$; at $g = 1$, $q_{\text{eff}} = f$, the plain hashrate share of that volume), after which every result of that volume applies to q_{eff} unchanged — its double-spend probability $r(q, n)$, its confirmation tables, its majority threshold at $q = \frac{1}{2}$. Table 3 computes the effect (by script, from that volume’s $r(q, n)$).

The reading is reassuring and precise. Grover halves the search *exponent* — a quadratic search over an H -bit target is $2^{H/2}$ rather than 2^H trials — so the ideal *rate* multiplier grows as the square root of the expected trial count, not to some fixed bound; what keeps the realised g small is not the algorithm but the gate-speed deflation of Remark 3.1 (no advantage at 2017 parameters, about 100 under its hypothesised 100 GHz gates). The table spans that uncertainty deliberately. Read at a modest near-term operating point of $g = 2$, even a 25% raw share reaches only $q_{\text{eff}} = 0.40$, short of majority, and a 10% share sits at $r = 1.44\%$ over six confirmations; read at the 100 GHz hypothesis, a $g = 100$ miner is over the majority line from a 1% raw share. Either way a quantum miner rewrites the double-spend arithmetic the way any faster miner does — by raising q — and proof of work degrades gracefully where the signature layers shatter. This asymmetry is the same one Aggarwal et al. draw for Bitcoin: the proof of work is “relatively resistant”, the signatures “much more at risk”.

raw f	speedup g	q_{eff}	$r(q_{\text{eff}}, 6)$	$r(q_{\text{eff}}, 10)$
1%	1	0.010	$<10^{-6}$	$<10^{-6}$
1%	10	0.092	0.037%	0.0004%
1%	100	0.503	100%	100%
5%	10	0.345	28.0%	16.0%
10%	2	0.182	1.44%	0.15%
10%	10	0.526	100%	100%
25%	2	0.400	49.3%	37.2%

Table 3: A per-unit mining speedup g applied to an attacker’s raw share f of total hashing, and the resulting double-spend success against 6 and 10 confirmations, computed by script from the *Consensus Guide*’s $r(q, n)$. The threshold that matters is the majority line: $q_{\text{eff}} > \frac{1}{2}$ iff $g > (1 - f)/f$, i.e. a 10% miner needs $g > 9$, a 5% miner $g > 19$, a 1% miner $g > 99$.

Remark 3.2 (Lumpy progress, quantum edition). The *Consensus Guide*’s difficulty remark (§“Difficulty in motion”) shows that the overtake probability of a work-scored race depends on the *granularity* of progress, but its mechanism is work *per block* under difficulty manipulation — which quantum mining leaves unchanged, since a quantum miner’s blocks carry the same work as anyone else’s at the network difficulty. A distinct channel does remain: the *inter-arrival* dispersion of a quantum miner’s block discoveries need not match the memoryless stream the model assumes, and its direction (Grover completions may be more regular, not less) is unanalysed here. It is flagged as the one place the reduction to q_{eff} is not exact; the effect is second-order against the deflations above.

4 The frontier surfaces

Two designs beyond the deployed core carry quantum-relevant structure their own volumes state but do not analyse; the closing volume must.

Crosslink finality. The trailing-finality prototype (*Crosslink Guide*, §“Finalizer keys”) gives each finalizer a single ed25519 key and makes a roster vote a plain ed25519 signature over a 76-byte payload (zebra-crosslink/src/lib.rs:2011 onward, at the prototype pin). The construction’s whole point is to convert proof-of-work’s *probabilistic* finality — the $r(q, n)$ of Section 3 — into *assured* finality by BFT signatures. Under a CRQC that inverts: the finalizer keys are long-lived roster identities that must stay secret indefinitely, and Shor forges their signatures, so an adversary manufactures the two-thirds agreement the BFT layer contributes. The prototype sources no threshold or post-quantum design for these keys anywhere; the *FROST Guide*’s own inference (§“Crosslink finalizer keys”) is that the natural fit would be FROST(Ed25519, SHA-512) — itself classical. Crosslink’s safety is a disjunction — assured finality from the proof-of-work prefix *or* the BFT agreement (*Crosslink Guide*, §“The safety theorems and their assumptions”) — so a CRQC does not erase finality outright; it deletes the BFT half of that disjunction, collapsing the guarantee back onto the probabilistic proof-of-work floor of Section 3. The upgrade the BFT layer was meant to buy is exactly what a quantum adversary forges.

FROST threshold spending. FROST rerandomised RedPallas is the deployed threshold path, and it is DL-based like everything it signs for (reddsa, src/frost/redpallas.rs: group Pallas, an aggregated signature verifying as an ordinary RedPallas spend-auth). ZIP-2005 extends its quantum-recovery mechanism to FROST through a shared quantum spending key qsk that every participant holds (*FROST Guide*, §“Custody of the ZIP-2005 quantum spending key”) — which defeats the t -of- n threshold against a quantum adversary, since one compromised participant’s qsk suffices. The ZIP accordingly recommends migrating FROST-held funds to a genuinely post-quantum threshold protocol “as soon as” one exists; none does, and the derivation-agreement gap (participants must privately

agree a seed sk , with no protocol for it) remains open. The frontier inherits the core’s strategy and its unfinished edges both.

5 ZIP-2005 in the cross-layer frame

The deployed answer to all of this is one Proposed ZIP, and the *Ironwood Guide* anatomises its mechanism (§“ZIP-2005: quantum recoverability of Ironwood notes”) — the 0x0B-tagged hashed derivation that makes each Ironwood note’s commitment *conditionally* post-quantum-binding, the qsk path for non-seed keys, the non-normative Recovery Statement, and the QROM security bounds. This volume adds only the cross-layer placement.

What it is. A recoverability mechanism, not a resistance one: ZIP-2005 states in terms that it “does not by itself make the protocol secure against quantum adversaries”, and arranges instead that Ironwood-pool funds survive the eventual, necessary switch-off of the DL-based pools. Its lever is the surviving backbone of Section 2.3: because the seed still derives every key symmetrically *and* the 0x0B-tagged hashed derivation makes each Ironwood note commitment conditionally post-quantum-binding, a future Recovery Protocol can let a holder prove ownership and re-mint under post-quantum assumptions — ownership outliving enforceability.

What it is not, mapped onto the inventory (Table 2):

- *The retroactive wound is untouched.* Privacy is an explicit non-requirement; harvest-now-decrypt-later persists for every pool including Ironwood, with mitigations only “under consideration”. The one break that is bleeding today gets nothing.
- *Several surfaces are out of scope entirely.* Transparent ECDSA is deferred to a Reserved ZIP-2007 stub; and the Sapling, Sprout, and *legacy-Orchard* (lead-byte-0x02) pools get no recoverability at all — their notes are *unrecoverable*, so once those protocols switch off, funds not migrated out are permanently unspendable. For those pools migration is not advice but the only exit.
- *Recovery itself is unbuilt.* The Recovery Protocol is designed-but-unspecified (“subject to change”), and it fixes no post-quantum proof system or commitment-tree hash — a stated requirement to leave them open. The recoverability claim is thus conditional on an artifact that does not yet exist.
- *Timing is load-bearing.* Every prospective mitigation assumes switch-off happens *before* a CRQC; attacks in the exposure window — Spendability roadblocks, spend-auth theft, and the turnstile-bounded inflation that “would not necessarily be detected” — are not repaired retroactively.

The honest summary is the *Ironwood Guide*’s, promoted here to the whole protocol: the strategy is recoverability, not resistance — keep discrete-log cryptography while it stands, arrange that ownership survives its fall, and race to switch the vulnerable pools off before the fall arrives. The strategy answers integrity comprehensively and privacy not at all.

6 The migration surface

Recoverability defers the problem; resistance eventually requires replacing the broken primitives with post-quantum ones, and the replacements are large. This section prices the swap against the published standards, so the cost is a number rather than an intuition.

6.1 What the replacements weigh

The NIST finals (FIPS 203/204/205, published 2024-08-13) fix the sizes. Table 4 lists the relevant tiers beside their classical counterparts as the series states them — the 64-byte RedPallas signature (ZIP-230 signature encodings; *Ironwood Guide*, the v6 transaction table), the 32-byte ephemeral key (*Sync Guide*), the 192-byte Groth16 proof (*Wallet Guide*). The ratios are the story.

Classical (deployed)	bytes	Post-quantum (NIST)	bytes	ratio
RedPallas signature	64	ML-DSA-44 signature	2420	37.8×
		ML-DSA-65 signature	3309	51.7×
		SLH-DSA-128s signature	7856	122.8×
Ephemeral key epk	32	ML-KEM-768 ciphertext	1088	34×
		ML-KEM-768 encaps. key	1184	—
Groth16 proof	192	STARK proof (hash-only soundness): ~hundreds of kB (order; see below)		

Table 4: Post-quantum replacement sizes against deployed counterparts. ML-DSA figures read from FIPS 204 Table 2, SLH-DSA from FIPS 205 Table 2, and ML-KEM from FIPS 203 Table 3; ML-DSA private keys compress to a 32-byte seed. Ratios computed by script.

Two of the three swaps are merely expensive. Replacing RedPallas with ML-DSA-44 multiplies each signature by nearly 38; replacing the Diffie–Hellman ephemeral key with an ML-KEM-768 ciphertext multiplies it by 34. A one-Action bundle today occupies $820 + 64 + 64 + (2720 + 2272) = 5940$ bytes (the OrchardAction encoding, its spend-auth and binding signatures, and the proof, all as the *Ironwood Guide* and *Halo 2 Guide* state them); substituting ML-DSA for the two signatures and an ML-KEM ciphertext for the ephemeral key, but *leaving the proof unchanged*, already brings it to about 11,700 bytes — a 1.97× inflation from the signature and KEM layers alone (by script). The compact-block cost is sharper still: a per-output ephemeral key of 32 bytes becomes 1088, taking the CompactOrchardAction payload from 148 bytes to about 1200, an 8.1× bandwidth multiplier on exactly the wire format the light-client stack was built to keep small (*Sync Guide*).

6.2 The proof problem is the hard one

The third swap is not merely expensive; it is unchosen, and it dominates the size budget. A discrete-log proof system — Halo 2’s IPA, or its declared successor Ragu, which is deliberately ECDLP-based (the *Tachyon Guide*’s reading) — has no post-quantum soundness. The established post-quantum route is hash-based: FRI/STARK systems rest, in Ben-Sasson, Bentov, Horesh, and Riabzev’s words, on “the existence of collision-resistant hash functions ... for interactive solutions, and ... the random oracle model ... for noninteractive ones” — assumptions “not known to be susceptible to attacks by large-scale quantum computers”. The cost is proof size: the same authors note that “ZK-SNARKs are roughly 1000× shorter than ZK-STARK proofs”, and the ethSTARK grant milestone compresses a 100,000-hash workload (3.2 MB) to 200 kB at 80-bit security. That 200 kB is three orders of magnitude past a 192-byte Groth16 proof and about forty times a ~5 kB Halo 2 bundle proof — and a hash-based system is what the Recovery Statement of Section 5 would have to be instantiated in. This is why ZIP-2005 leaves the proof system unchosen: the recoverability design commits to the *shape* of a post-quantum future without paying its size, deferring the largest bill to the day resistance, not recovery, is finally built. The Recovery Statement’s in-circuit cost — roughly seven BLAKE2b compressions, a BLAKE3, two Sinsemilla commitments, two Pallas scalar multiplications, and a Merkle path (*Ironwood Guide*) — is small precisely because it re-executes the *surviving* symmetric backbone; the proof of that circuit is the part that inflates.

6.3 The register

The series’ method is to classify design-stage claims and to cite firsthand; applied to the ecosystem’s public posture, the method yields a register worth stating plainly, because the gap it records is large.

In the repositories, at the pinned commits: the zips repository and the protocol specification contain *zero* occurrences of ML-KEM, Kyber, ML-DSA, Dilithium, SPHINCS, SLH-DSA, Falcon, or “lattice”. A recursive search for post-quantum scheme names across the ecosystem checkouts finds no protocol code: the only post-quantum crates anywhere are pqcrypto-mlkem and pqcrypto-mlds in

a single `Cargo.lock` (zallet’s), pulled transitively and behind a `zcashd-import` feature gate by a wallet-interchange library, and a lone doc comment in the same crate noting ML-KEM work in a dependency’s ecosystem — both inert, called by no protocol code. No deployed post-quantum cryptography exists in Zcash at these pins.

In public statements: a claim of a “fully post-quantum” Zcash “in 12–18 months” appears in conference coverage (CoinDesk, 2026-05-08, reporting Swihart at Consensus Miami), and a claim that ML-KEM and ML-DSA are “being tested” in a separate CoinDesk Research long-form piece — neither corresponding to any ZIP, draft, or repository artifact. ZIP-2005 itself is the honest counterweight: it disclaims making the protocol quantum-secure and confines its promise to recoverability of one pool. This volume records the divergence without adjudicating intent: the design record and the marketing record are simply different documents, and a reader is owed both.

What is genuinely specified, designed, or open, in the series’ three bins: the Ironwood note derivations and the seal rules are *specified* (ZIP-2005, NU6.3); the Recovery Protocol and the FROST qsk custody are *designed but unspecified*; and a deployed post-quantum signature for ongoing spends, a chosen post-quantum proof system, a privacy answer to the retroactive wound, and the switch-off schedule itself are *open problems*. The protocol’s quantum posture is a well-specified plan to *recover* from a break it has, by deliberate choice, not yet built the means to *prevent*.

7 Relation to the rest of the series

This volume closes the Frontier group and the series. It owns nothing that a lower volume constructs: the quantum caveat, the hiding/binding regimes, and the commitment and proof primitives are the *Crypto Guide*’s and *Halo 2 Guide*’s; the Orchard-core post-quantum analysis and ZIP-2005’s mechanism are the *Ironwood Guide*’s; the double-spend arithmetic that Section 3 feeds a quantum miner into is the *Consensus Guide*’s; the qsk custody is the *FROST Guide*’s; the recorded-ciphertext liability and its one structural mitigation are the *Tachyon Guide*’s; the finalizer keys are the *Crosslink Guide*’s. What this volume adds is the join: the single table that places every layer’s exposure on one axis, the quantum-mining analysis no volume owned, the migration arithmetic, and the register of what is built against what is said. Read from below, the series proves what each assumption buys; read from here, it shows what is owed when the bill comes due — and that Zcash has chosen to keep the assumptions, insure the funds against their fall, and run for the exit before it comes.